

13 General-purpose I/Os (GPIO)

13.1 GPIO introduction

Each general-purpose I/O port has four 32-bit configuration registers (GPIOx_MODER, GPIOx_OTYPER, GPIOx_OSPEEDR and GPIOx_PUPDR), two 32-bit data registers (GPIOx_IDR and GPIOx_ODR), a 16-bit reset register (GPIOx_BRR), and a 32-bit set/reset register (GPIOx_BSRR).

In addition, all GPIOs have a 32-bit locking register (GPIOx_LCKR), two 32-bit alternate function selection registers (GPIOx_AFRH and GPIOx_AFRL), a secure configuration register (GPIOx_SECCFGR), and a high-speed low-voltage register (GPIOx_HSLVR).

13.2 GPIO main features

- Output states: push-pull or open drain + pull-up/down
- Output data from output data register (GPIOx_ODR) or peripheral (alternate function output)
- Speed selection for each I/O
- Input states: floating, pull-up/down, analog
- Input data to input data register (GPIOx_IDR) or peripheral (alternate function input)
- Bit set and reset register (GPIOx_BSRR) for bit-wise write access to GPIOx_ODR
- Lock mechanism (GPIOx_LCKR) provided to freeze the I/O port configurations
- Analog function
- Alternate function selection registers
- Fast toggle capable of changing every two clock cycles
- Highly flexible pin multiplexing allows the use of I/O pins as GPIOs or as one of several peripheral functions
- TrustZone® security support
- I/Os state retention during Standby mode

13.3 GPIO functional description

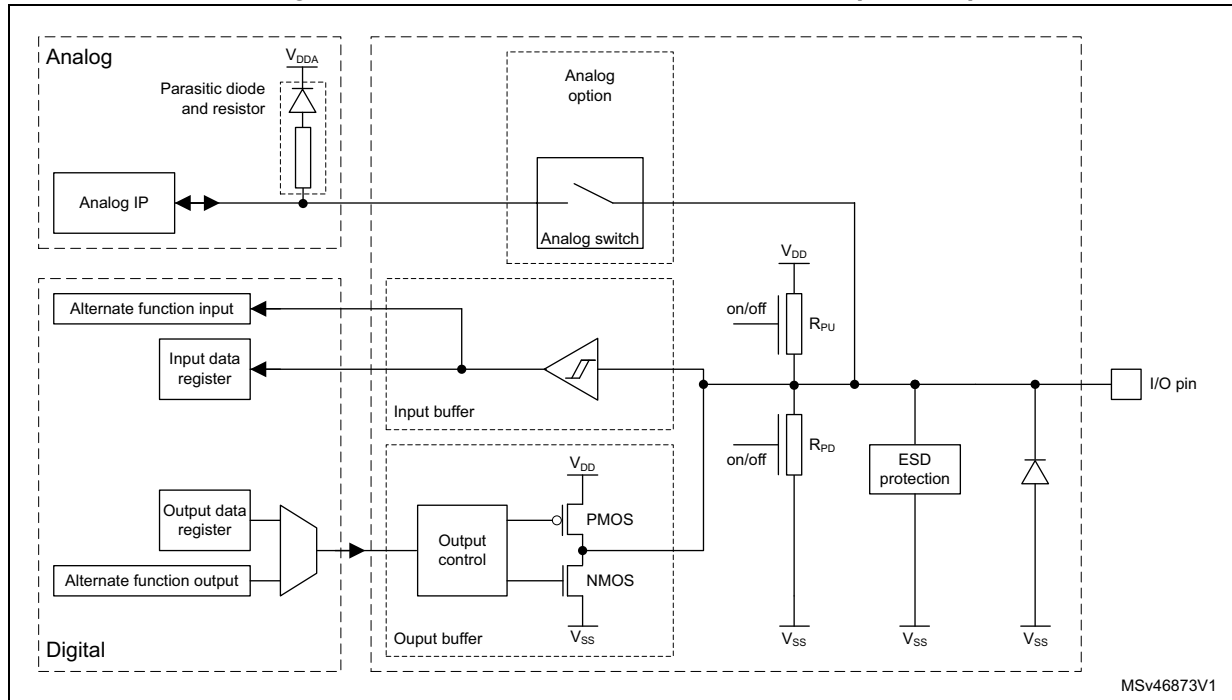
Subject to the specific hardware characteristics of each I/O port listed in the datasheet, each port bit of the general-purpose I/O (GPIO) ports can be individually configured by software in several modes:

- Input floating
- Input pull-up
- Input-pull-down
- Analog
- Output open-drain with pull-up or pull-down capability
- Output push-pull with pull-up or pull-down capability
- Alternate function push-pull with pull-up or pull-down capability
- Alternate function open-drain with pull-up or pull-down capability

Each I/O port bit is freely programmable, however the I/O port registers must be accessed as 32-bit words, half-words, or bytes. The GPIOx_BSRR and GPIOx_BRR registers allow atomic read/modify accesses to any of the GPIOx_ODR registers. In this way, there is no risk of an IRQ occurring between the read and the modify access.

Figure 64 shows the basic structure of a 3- or 5-V tolerant GPIO (TT or FT). Table 131 gives the possible port bit configurations.

Figure 64. Structure of 3- or 5-V tolerant GPIO (TT or FT)



Note: On a TT GPIO, the analog switch is not present, it is replaced by a direct connection. The analog bloc parasitic circuitry does not allow 5-V tolerance.

Table 131. Port bit configuration⁽¹⁾

MODE(i)[1:0]	OTYPE(i)	OSPEED(i)[1:0]	PUPD(i)[1:0]		I/O configuration	
01	0	SPEED[1:0]	0	0	GP output	PP
	0		0	1	GP output	PP + PU
	0		1	0	GP output	PP + PD
	0		1	1	Reserved	
	1		0	0	GP output	OD
	1		0	1	GP output	OD + PU
	1		1	0	GP output	OD + PD
	1		1	1	Reserved (GP output OD)	

Table 131. Port bit configuration⁽¹⁾ (continued)

MODE(i)[1:0]	OTYPE(i)	OSPEED(i)[1:0]		PUPD(i)[1:0]		I/O configuration	
10	0	SPEED[1:0]		0	0	AF	PP
	0			0	1	AF	PP + PU
	0			1	0	AF	PP + PD
	0			1	1	Reserved	
	1			0	0	AF	OD
	1			0	1	AF	OD + PU
	1			1	0	AF	OD + PD
	1			1	1	Reserved	
00	x	x	x	0	0	Input	Floating
	x	x	x	0	1	Input	PU
	x	x	x	1	0	Input	PD
	x	x	x	1	1	Reserved (input floating)	
11	x	x	x	0	0	Input/output	Analog
	x	x	x	0	1	Reserved	
	x	x	x	1	0	Input/output	Analog, PD
	x	x	x	1	1	Reserved	

1. GP = general-purpose, PP = push-pull, PU = pull-up, PD = pull-down, OD = open-drain, AF = alternate function.

13.3.1 General-purpose I/O (GPIO)

During and just after reset, the alternate functions are not active and most of the I/O ports are configured in analog mode.

The debug pins are in AF pull-up/pull-down after reset:

- PA15: JTDI in pull-up
- PA14: JTCK/SWCLK in pull-down
- PA13: JTMS/SWDIO in pull-up
- PB4: NJTRST in pull-up
- PB3: JTDO/TRACESWO in floating state no pull-up/pull-down

BOOT0 is in input mode during the reset until at least the end of the option byte loading phase (see [Section 13.3.14](#)).

When the pin is configured as output, the value written to the output data register (GPIOx_ODR) is output on the I/O pin. It is possible to use the output driver in push-pull mode or open-drain mode (only the low level is driven, high level is high-Z).

The input data register (GPIOx_IDR) captures the data present on the I/O pin at every AHB clock cycle.

All GPIO pins have weak internal pull-up and pull-down resistors, which can be activated or not, depending upon the value in the GPIOx_PUPDR register.

13.3.2 I/O pin alternate function multiplexer and mapping

The device I/O pins are connected to on-board peripherals/modules through a multiplexer that allows only one peripheral alternate function (AF) connected to an I/O pin at a time. In this way, there is no conflict between peripherals available on the same I/O pin.

Each I/O pin has a multiplexer with up to 16 alternate function inputs (AF0 to AF15) that can be configured through the GPIOx_AFRL (for pin 0 to 7) and GPIOx_AFRH (for pin 8 to 15) registers:

- After reset, the multiplexer selection is alternate function 0 (AF0). The I/Os are configured in alternate function mode through GPIOx_MODER register.
- The specific alternate function assignments for each pin are detailed in the device datasheet.

In addition to this flexible I/O multiplexing architecture, each peripheral has alternate functions mapped onto different I/O pins to optimize the number of peripherals available in smaller packages.

To use an I/O in a given configuration, the user must proceed as follows:

- **Debug function:** after each device reset these pins are assigned as alternate function pins immediately usable by the debugger host.
- **GPIO:** configure the desired I/O as output, input, or analog in the GPIOx_MODER register.
- **Peripheral alternate function:**
 - Connect the I/O to the desired AFx in one of the GPIOx_AFRL or GPIOx_AFRH register.
 - Select the type, pull-up/pull-down, and output speed via the GPIOx_OTYPER, GPIOx_PUPDR and GPIOx_OSPEEDR registers respectively.
 - Configure the desired I/O as an alternate function in the GPIOx_MODER register.
- **Cortex[®]-M33 alternate function (EVENTOUT)**
 - The Cortex-M33 output EVENTOUT signal can be output as alternate function on I/O pin. An event can be signaled through the configured pin after executing SEV instruction
 - EVENTOUT signal can be used internally as a trigger for some peripherals (see [Table 139: Peripherals interconnect matrix](#))
- **Additional functions:**
 - For the ADC and DAC, configure the desired I/O in analog mode in the GPIOx_MODER register and configure the required function in the ADC and DAC registers.
 - For the additional functions like RTC, WKUPx, and oscillators, configure the required function in the related RTC, PWR, and RCC registers. These functions have priority over the configuration in the standard GPIO registers.

Refer to the “Alternate function mapping” table in the device datasheet for the detailed mapping of the alternate function I/O pins.

13.3.3 I/O port control registers

Each of the GPIO ports has four 32-bit memory-mapped control registers (GPIOx_MODER, GPIOx_OTYPER, GPIOx_OSPEEDR, GPIOx_PUPDR) to configure up to 16 I/Os.

- GPIOx_MODER is used to select the I/O mode (input, output, AF, analog).
- GPIOx_OTYPER and GPIOx_OSPEEDR are used to select the output type (push-pull or open-drain) and speed.
- GPIOx_PUPDR is used to select the pull-up/pull-down whatever the I/O direction. I/O port data registers.

Each GPIO has two 16-bit memory-mapped data registers: input and output data registers (*GPIO port input data register (GPIOx_IDR) (x = A to I)* and *GPIO port output data register (GPIOx_ODR) (x = A to I)*).

GPIOx_ODR stores the data to be output, it is read/write accessible. The data entered through the I/Os are stored into the input data register (GPIOx_IDR), a read-only register.

13.3.4 I/O data bitwise handling

The bit set reset register (GPIOx_BSRR) is a 32-bit register that allows the application to set and reset each individual bit in the output data register (GPIOx_ODR). The bit set reset register has twice the size of GPIOx_ODR.

To each bit in GPIOx_ODR, correspond two control bits in GPIOx_BSRR: BS(i) and BR(i). When written to 1, BS(i) sets the corresponding ODR(i) bit. When written to 1, BR(i) resets the ODR(i) corresponding bit.

Writing any bit to 0 in GPIOx_BSRR does not have any effect on the corresponding bit in GPIOx_ODR. If there is an attempt to both set and reset a bit in GPIOx_BSRR, the set action takes priority.

Using the GPIOx_BSRR register to change the values of individual bits in GPIOx_ODR is a “one-shot” effect that does not lock the GPIOx_ODR bits. The GPIOx_ODR bits can always be accessed directly. The GPIOx_BSRR register provides a way of performing atomic bitwise handling.

There is no need for the software to disable interrupts when programming the GPIOx_ODR at bit level: one or more bits can be modified in a single atomic AHB write access.

13.3.5 GPIO locking mechanism

The GPIO control registers can be frozen by applying a specific write sequence to the GPIOx_LCKR register. The frozen registers are GPIOx_MODER, GPIOx_OTYPER, GPIOx_OSPEEDR, GPIOx_PUPDR, GPIOx_AFRL, GPIOx_AFRH and GPIOx_HSLVR.

To write the GPIOx_LCKR register, a specific write/read sequence must be applied. When the right LOCK sequence is applied to the bit 16 in this register, the value of LCKR[15:0] is used to lock the configuration of the I/Os (during the write sequence the LCKR[15:0] value must be the same). When the LOCK sequence is applied to a port bit, the value of the port bit can no longer be modified until the next MCU reset or peripheral reset. Each GPIOx_LCKR bit freezes the corresponding bit in the control registers (GPIOx_MODER, GPIOx_OTYPER, GPIOx_OSPEEDR, GPIOx_PUPDR, GPIOx_AFRL and GPIOx_AFRH).

The LOCK sequence can be performed only using a word (32-bit long) access to the GPIOx_LCKR register, because GPIOx_LCKR bit 16 must be set at the same time as the [15:0] bits.

13.3.6 I/O alternate function input/output

Two registers are provided to select one of the alternate function inputs/outputs available for each I/O. With these registers, the user can connect an alternate function to some other pin as required by the application.

This means that some peripheral functions are multiplexed on each GPIO using the GPIOx_AFRL and GPIOx_AFRH alternate function registers. The application can thus select any one of the possible functions for each I/O. The AF selection signal being common to the alternate function input and alternate function output, a single channel is selected for the alternate function input/output of a given I/O.

To know which functions are multiplexed on each GPIO pin, refer to the device datasheet.

13.3.7 External interrupt/wake-up lines

All ports have external interrupt capability. To use external interrupt lines, the port can be configured in input, output, or alternate function mode (the port must not be configured in analog mode). Refer to [Section 18: Extended interrupts and event controller \(EXTI\)](#).

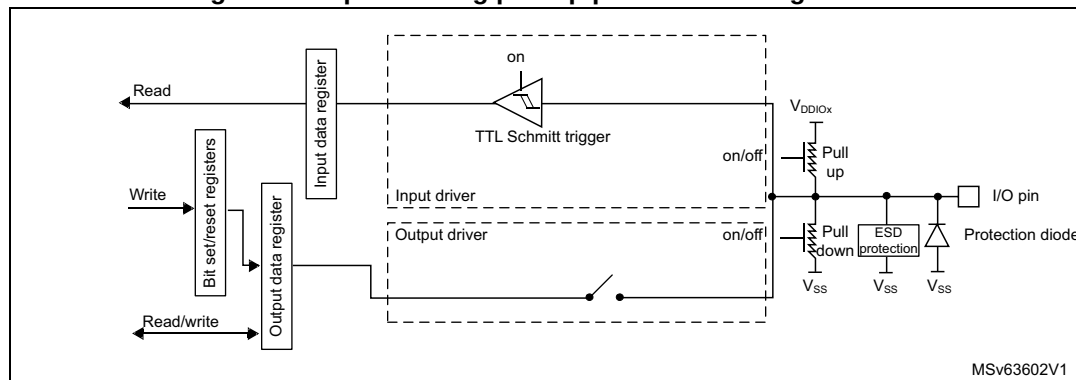
13.3.8 Input configuration

When the I/O port is programmed as input:

- The output buffer is disabled.
- The Schmitt trigger input is activated.
- The pull-up and pull-down resistors are activated depending on the value in the GPIOx_PUPDR register.
- The data present on the I/O pin are sampled into the input data register every AHB clock cycle.
- A read access to the input data register provides the I/O state.

[Figure 65](#) shows the input configuration of the I/O port bit.

Figure 65. Input floating/pull-up/pull-down configurations



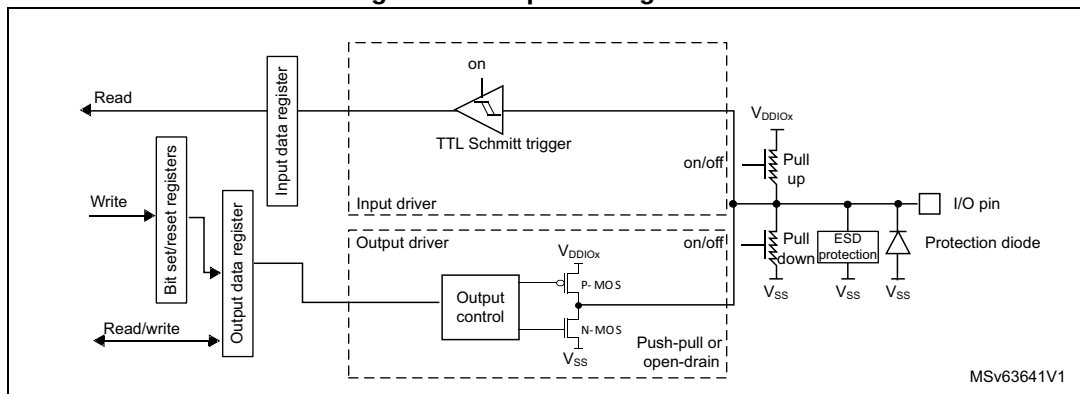
13.3.9 Output configuration

When the I/O port is programmed as output:

- The output buffer is enabled:
 - Open-drain mode: a 0 in the output register activates the N-MOS whereas a 1 in the output register leaves the port in high-Z (the P-MOS is never activated).
 - Push-pull mode: a 0 in the output register activates the N-MOS whereas a 1 in the output register activates the P-MOS.
- The Schmitt trigger input is activated.
- The pull-up and pull-down resistors are activated depending on the value in the GPIOx_PUPDR register.
- The data present on the I/O pin are sampled into the input data register every AHB clock cycle.
- A read access to the input data register gets the I/O state.
- A read access to the output data register gets the last written value.

Figure 66 shows the output configuration of the I/O port bit.

Figure 66. Output configuration



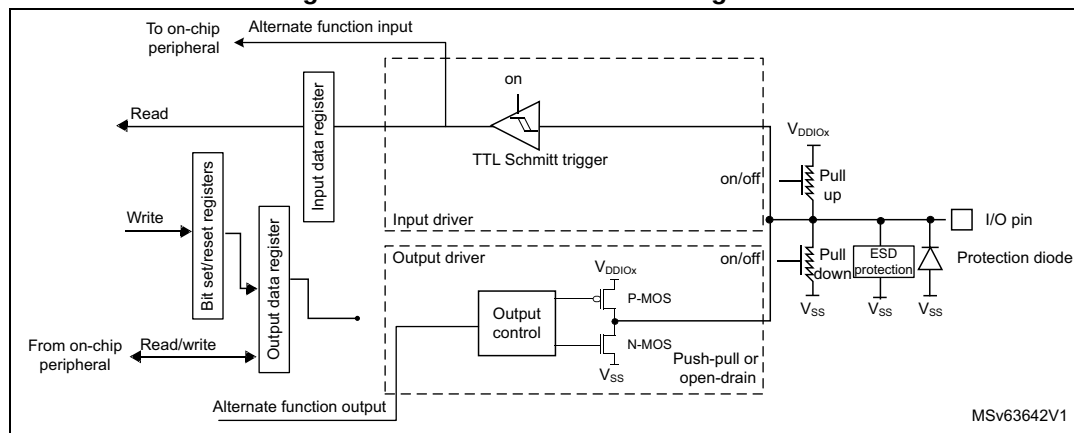
13.3.10 Alternate function configuration

When the I/O port is programmed as alternate function:

- The output buffer can be configured in open-drain or push-pull mode.
- The output buffer is driven by the signals coming from the peripheral (transmitter enable and data).
- The Schmitt trigger input is activated.
- The weak pull-up and pull-down resistors are activated or not depending on the value in the GPIOx_PUPDR register.
- The data present on the I/O pin are sampled into the input data register every AHB clock cycle.
- A read access to the input data register gets the I/O state.

Figure 67 shows the alternate function configuration of the I/O port bit.

Figure 67. Alternate function configuration



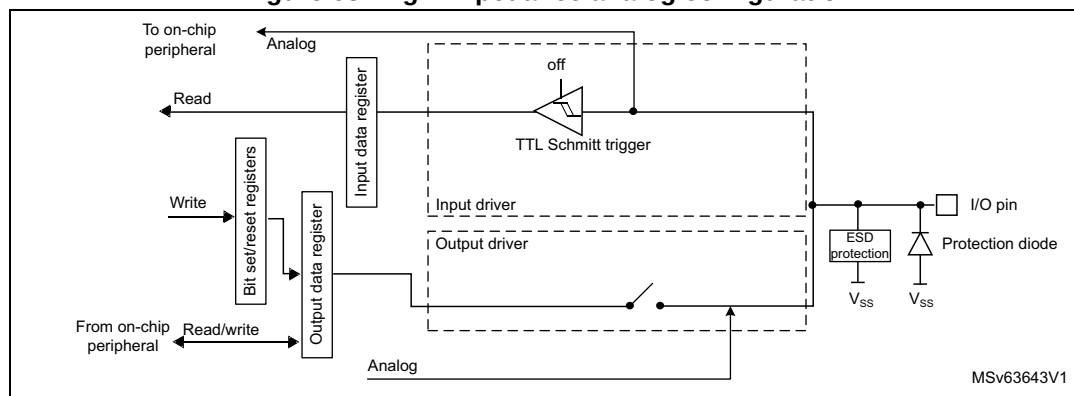
13.3.11 Analog configuration

When the I/O port is programmed as analog configuration:

- The output buffer is disabled.
- The Schmitt trigger input is deactivated, providing zero consumption for every analog value of the I/O pin. The output of the Schmitt trigger is forced to a constant value (0).
- The weak pull-up is disabled by hardware. The weak pull-down is configurable.
- Read access to the input data register gets the value 0.

Figure 68 shows the high-Z, analog-input configuration of the I/O port bits.

Figure 68. High-impedance analog configuration



13.3.12 Using the HSE or LSE oscillator pins as GPIOs

When the HSE or LSE oscillator is switched off (default state after reset), the related oscillator pins can be used as normal GPIOs.

When the HSE or LSE oscillator is switched on (by setting the HSEON or LSEON bit in the RCC_CSR register), the oscillator takes control of its associated pins and the GPIO configuration of these pins has no effect.

When the oscillator is configured in a user external clock mode, only the pin is reserved for clock input, and the OSC_OUT or OSC32_OUT pin can still be used as normal GPIO.

13.3.13 Using the GPIO pins in the RTC supply domain

The PC13/PC14/PC15/PI8 GPIO functionality is lost when the core supply domain is powered off (when the device enters Standby mode). In this case, if their GPIO configuration is not bypassed by the RTC configuration, these pins are set in an analog input mode.

For details about I/O control by the RTC, refer to [Section 51.3: RTC functional description](#)

13.3.14 I/Os state retention during standby mode

In the Standby mode, the I/Os are by default in floating state.

If the IORETEN bit in the PWR_IOPRETR register is set, the I/Os state is sampled during standby entry. The state of I/Os is applied to the pin via pull-up and pull-down resistors. The pull-up and pull-down resistors remains applied after Standby wake-up until the IORETEN bit in the PWR_IOPRETR register is cleared by software.

13.3.15 TrustZone security

The TrustZone security is activated by the TZEN option byte in the FLASH option byte register. When the TrustZone is active (TZEN = 0xB4), each I/O pin of GPIO port can be individually configured as secure through the GPIOx_SECCFGR register.

When the selected I/O pin is configured as secure, its corresponding configuration bits for alternate function, mode selection, I/O data are secure against a nonsecure access. In case of nonsecure access, these bits are RAZ/WI.

The I/Os with peripherals functions are also conditioned by the peripheral security configuration (see [Section 5: Global TrustZone® controller \(GTZC\)](#) for more details):

- For peripherals for which the I/O pin selection is done through alternate functions registers: if the peripheral is configured as secure, it cannot be connected to a nonsecure I/O pin. If this is not respected, the input data to the secure peripheral is forced to 0 (I/O input pin value is ignored) and the output pin value is forced to 0, thus avoiding any secure information leak through nonsecure I/Os.
- For I/Os with analog switches, directly controlled by peripherals (such as ADC for instance): If the I/O is secure, the I/O analog switch cannot be controlled by a nonsecure peripheral. If this is not respected, the switch remains open. This prevents the redirection of secure data to a nonsecure peripheral or I/O through the analog path. Refer to [Section 3: System security](#) for more details.
- Some of the paths between I/Os “additional functions” and peripherals are not blocked if the I/O is secure and the peripheral is nonsecure. Therefore, it is recommended to configure those peripherals as secure even when not used by the application. Refer to [Section 3: System security](#) for the list of concerned peripherals. When the path has a security control, it follows the same rule as I/O selection through alternate functions.

Refer to the device pins definition table in datasheet for more information about peripherals alternate functions and additional functions mapping.

After reset, all GPIO ports are secure.

The table below gives a summary of the I/O port secured bits following the security configuration bit in the GPIO_SECCFGR register. When the I/O bit port is configured as secure:

- Secured bits: read and write operations are only allowed by a secure access. Non secure-read or write accesses on secured bits are RAZ/WI. There is no illegal access event generated.
- Nonsecure bits: no restriction. Read and write operations are allowed by both secure and nonsecure accesses.

When the TrustZone security is disabled (TZEN = 0xC3 in FLASH_OPTSR2 register), all registers bits are nonsecure. The GPIOx_SECCFGR register is RAZ/WI.

Table 132. GPIO secured bits

Secure configuration bit	Secured bit	Register name	Nonsecure access on secure bits
SECy = 1 in GPIOx_SECCFGR ^{(1) (2)}	MODEy[1:0]	GPIOx_MODER	RAZ/WI
	OTy	GPIOx_OTYPER	
	OSPEEDy[1:0]	GPIOx_OSPEEDR	
	PUPDy[1:0]	GPIOx_PUPDR	
	IDy	GPIOx_IDR	
	ODy	GPIOx_ODR	
	BSy and BRy	GPIOx_BSRR	
	LCKy	GPIOx_LCKR	
	BRy	GPIOx_BRR	
	AFSELy[3:0]	GPIOx_AFRH	
		GPIOx_AFRL	
	HSLVy	GPIOx_HSLVR	

1. GPIOx, x = A to I. For x = A to H, y = 0 to 15. For x = I, y = 0 to 11.

2. The number of GPIOx ports depends upon the device. Refer to the product datasheet for availability of a particular port. If not present, consider the associated bits as reserved, and keep them at reset value.

13.3.16 Privileged and unprivileged modes

All GPIO registers can be read and written by privileged and unprivileged accesses, whatever the security state (secure or nonsecure).

13.3.17 High-speed low-voltage mode (HSLV)

Some I/Os have the capability to increase their maximum speed at low voltage by configuring them in HSLV mode. The I/O HSLV bit controls whether the I/O output speed is optimized to operate at 3.3 V (default setting) or at 1.8 V (HSLV = 1).

Caution: The I/O HSLV configuration bit must not be set if the I/O supply (V_{DD} or V_{DDIO2}) is above 2.7 V. Setting it while the voltage is higher than 2.7 V can damage the device. The I/O HSLV bit can be set only when the corresponding option bit is activated (IO_VDD_HSLV or IO_VDDIO2_HSLV depending on the I/O supply, refer to [Section 7.4: FLASH option bytes](#)).

There is no hardware protection associated to this feature so it is recommended to use it only as a static configuration for fixed I/O supply.

13.3.18 I/O compensation cell

The I/O commutation slew rate (tfall / trise) can be adapted by software depending on process, voltage and temperatures conditions, to reduce the I/O noise on power supply. Refer to [Section 14: System configuration, boot, and security \(SBS\)](#) for more details.

13.4 GPIO registers

This section gives a detailed description of the GPIO registers. Note that the number of GPIOx ports depends upon the device. Refer to the product datasheet for the availability of a particular port. If not present, consider the associated bits as reserved, and keep them at reset value.

The peripheral registers can be written in word, half word, or byte mode.

13.4.1 GPIO port mode register (GPIOx_MODER) (x = A to I)

Address offset: 0x00

Reset value: 0xABFF FFFF (for port A)

Reset value: 0xFFFF FEBF (for port B)

Reset value: 0xFFFF FFFF (for ports C..H)

Reset value: 0x00FF FFFF (for port I)

Note: For STM32H523/33xx devices, the reset value for port B is 0xFF3F FEBF.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
MODE15[1:0]		MODE14[1:0]		MODE13[1:0]		MODE12[1:0]		MODE11[1:0]		MODE10[1:0]		MODE9[1:0]		MODE8[1:0]	
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
MODE7[1:0]		MODE6[1:0]		MODE5[1:0]		MODE4[1:0]		MODE3[1:0]		MODE2[1:0]		MODE1[1:0]		MODE0[1:0]	
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bits 31:0 **MODEy[1:0]**: Port x configuration I/O pin y (y = 15 to 0)

These bits are written by software to configure the I/O mode.

00: Input mode

01: General purpose output mode

10: Alternate function mode

11: Analog mode (reset state)

Note: The bitfield is reserved and must be kept to reset value when the corresponding I/O is not available on the selected package.

13.4.2 GPIO port output type register (GPIOx_OTYPER) (x = A to I)

Address offset: 0x04

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
OT15	OT14	OT13	OT12	OT11	OT10	OT9	OT8	OT7	OT6	OT5	OT4	OT3	OT2	OT1	OT0
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bits 31:16 Reserved, must be kept at reset value.

Bits 15:0 **OTy**: Port x configuration I/O pin y (y = 15 to 0)

These bits are written by software to configure the I/O output type.

0: Output push-pull (reset state)

1: Output open-drain

Note: The bit is reserved and must be kept to reset value when the corresponding I/O is not available on the selected package.

13.4.3 GPIO port output speed register (GPIOx_OSPEEDR) (x = A to I)

Address offset: 0x08

Reset value: 0x0C00 0000 (for port A)

Reset value: 0x0000 00C0 (for port B)

Reset value: 0x0000 0000 (for the other ports)

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
OSPEED15[1:0]		OSPEED14[1:0]		OSPEED13[1:0]		OSPEED12[1:0]		OSPEED11[1:0]		OSPEED10[1:0]		OSPEED9[1:0]		OSPEED8[1:0]	
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
OSPEED7[1:0]		OSPEED6[1:0]		OSPEED5[1:0]		OSPEED4[1:0]		OSPEED3[1:0]		OSPEED2[1:0]		OSPEED1[1:0]		OSPEED0[1:0]	
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bits 31:0 **OSPEEDy[1:0]**: Port x configuration I/O pin y (y = 15 to 0)

These bits are written by software to configure the I/O output speed.

00: Low speed

01: Medium speed

10: High speed

11: Very-high speed

Note: Refer to the device datasheet for the frequency specifications, the power supply, and the load conditions for each speed.

The bitfield is reserved and must be kept to reset value when the corresponding I/O is not available on the selected package.

13.4.4 GPIO port pull-up/pull-down register (GPIOx_PUPDR) (x = A to I)

Address offset: 0x0C

Reset value: 0x6400 0000 (for port A)

Reset value: 0x0000 0100 (for port B)

Reset value: 0x0000 0000 (for the other ports)

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
PUPD15[1:0]		PUPD14[1:0]		PUPD13[1:0]		PUPD12[1:0]		PUPD11[1:0]		PUPD10[1:0]		PUPD9[1:0]		PUPD8[1:0]	
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
PUPD7[1:0]		PUPD6[1:0]		PUPD5[1:0]		PUPD4[1:0]		PUPD3[1:0]		PUPD2[1:0]		PUPD1[1:0]		PUPD0[1:0]	
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bits 31:0 **PUPDy[1:0]**: Port x configuration I/O pin y (y = 15 to 0)

These bits are written by software to configure the I/O pull-up or pull-down

00: No pull-up, pull-down

01: Pull-up

10: Pull-down

11: Reserved

Note: The bitfield is reserved and must be kept to reset value when the corresponding I/O is not available on the selected package.

13.4.5 GPIO port input data register (GPIOx_IDR) (x = A to I)

Address offset: 0x10

Reset value: 0x0000 XXXX

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
ID15	ID14	ID13	ID12	ID11	ID10	ID9	ID8	ID7	ID6	ID5	ID4	ID3	ID2	ID1	ID0
r	r	r	r	r	r	r	r	r	r	r	r	r	r	r	r

Bits 31:16 Reserved, must be kept at reset value.

Bits 15:0 **IDy**: Port x input data I/O pin y (y = 15 to 0)

These bits are read-only. They contain the input value of the corresponding I/O port.

Note: The bit is reserved and must be kept to reset value when the corresponding I/O is not available on the selected package.

13.4.6 GPIO port output data register (GPIOx_ODR) (x = A to I)

Address offset: 0x14

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
OD15	OD14	OD13	OD12	OD11	OD10	OD9	OD8	OD7	OD6	OD5	OD4	OD3	OD2	OD1	OD0
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bits 31:16 Reserved, must be kept at reset value.

Bits 15:0 **ODy**: Port output data I/O pin y (y = 15 to 0)

These bits can be read and written by software.

Note: For atomic bit set/reset, the OD bits can be individually set and/or reset by writing to the GPIOx_BSRR or GPIOx_BRR registers (x = A to I).

The bit is reserved and must be kept to reset value when the corresponding I/O is not available on the selected package.

13.4.7 GPIO port bit set/reset register (GPIOx_BSRR) (x = A to I)

Address offset: 0x18

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
BR15	BR14	BR13	BR12	BR11	BR10	BR9	BR8	BR7	BR6	BR5	BR4	BR3	BR2	BR1	BR0
w	w	w	w	w	w	w	w	w	w	w	w	w	w	w	w
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
BS15	BS14	BS13	BS12	BS11	BS10	BS9	BS8	BS7	BS6	BS5	BS4	BS3	BS2	BS1	BS0
w	w	w	w	w	w	w	w	w	w	w	w	w	w	w	w

Bits 31:16 **BRy**: Port x reset I/O pin y (y = 15 to 0)

These bits are write-only. A read to these bits returns the value 0x0000.

0: No action on the corresponding ODy bit

1: Resets the corresponding ODy bit

Note: If both BSy and BRy are set, BSy has priority.

The bit is reserved and must be kept to reset value when the corresponding I/O is not available on the selected package.

Bits 15:0 **BSy**: Port x set I/O pin y (y = 15 to 0)

These bits are write-only. A read to these bits returns the value 0x0000.

0: No action on the corresponding ODy bit

1: Sets the corresponding ODy bit

Note: The bit is reserved and must be kept to reset value when the corresponding I/O is not available on the selected package.

13.4.8 GPIO port configuration lock register (GPIOx_LCKR) (x = A to I)

This register is used to lock the configuration of the port bits when a correct write sequence is applied to bit 16 (LCKK). The value of bits [15:0] is used to lock the configuration of the GPIO. During the write sequence, the value of LCKR[15:0] must not change. When the LOCK sequence has been applied on a port bit, the value of this port bit can no longer be modified until the next MCU reset or peripheral reset.

Note: A specific write sequence is used to write to the GPIOx_LCKR register. Only word access (32-bit long) is allowed during this locking sequence.

Each bit freezes a specific configuration register (control and alternate function registers).

Address offset: 0x1C

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	LCKK
															rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
LCK15	LCK14	LCK13	LCK12	LCK11	LCK10	LCK9	LCK8	LCK7	LCK6	LCK5	LCK4	LCK3	LCK2	LCK1	LCK0
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bits 31:17 Reserved, must be kept at reset value.

Bit 16 LCKK: Lock key

This bit can be read any time. It can only be modified using the lock key write sequence.

0: Port configuration lock key not active

1: Port configuration lock key active. The GPIOx_LCKR register is locked until the next MCU reset or peripheral reset.

- LOCK key write sequence:

WR LCKR[16] = 1 + LCKR[15:0]

WR LCKR[16] = 0 + LCKR[15:0]

WR LCKR[16] = 1 + LCKR[15:0]

- LOCK key read

RD LCKR[16] = 1 (this read operation is optional but it confirms that the lock is active)

Note: During the LOCK key write sequence, the value of LCK[15:0] must not change.

Any error in the lock sequence aborts the LOCK.

After the first LOCK sequence on any bit of the port, any read access on the LCKK bit returns 1 until the next MCU reset or peripheral reset.

Bits 15:0 LCKy: Port x lock I/O pin y (y = 15 to 0)

These bits are read/write but can only be written when the LCKK bit is 0

0: Port configuration not locked

1: Port configuration locked

Note: The bit is reserved and must be kept to reset value when the corresponding I/O is not available on the selected package.

13.4.9 GPIO alternate function low register (GPIOx_AFRL) (x = A to I)

Address offset: 0x20

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
AFSEL7[3:0]				AFSEL6[3:0]				AFSEL5[3:0]				AFSEL4[3:0]			
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
AFSEL3[3:0]				AFSEL2[3:0]				AFSEL1[3:0]				AFSEL0[3:0]			
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bits 31:0 **AFSELy[3:0]**: Alternate function selection for port x I/O pin y (y = 7 to 0)
These bits are written by software to configure alternate function I/Os.

- 0000: AF0
- 0001: AF1
- 0010: AF2
- 0011: AF3
- 0100: AF4
- 0101: AF5
- 0110: AF6
- 0111: AF7
- 1000: AF8
- 1001: AF9
- 1010: AF10
- 1011: AF11
- 1100: AF12
- 1101: AF13
- 1110: AF14
- 1111: AF15

Note: The bitfield is reserved and must be kept to reset value when the corresponding I/O is not available on the selected package.

13.4.10 GPIO alternate function high register (GPIOx_AFRH) (x = A to H)

Address offset: 0x24
Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
AFSEL15[3:0]				AFSEL14[3:0]				AFSEL13[3:0]				AFSEL12[3:0]			
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
AFSEL11[3:0]				AFSEL10[3:0]				AFSEL9[3:0]				AFSEL8[3:0]			
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bits 31:0 **AFSELY[3:0]**: Alternate function selection for port x I/O pin y (y = 15 to 8)

These bits are written by software to configure alternate function I/Os.

0000: AF0
 0001: AF1
 0010: AF2
 0011: AF3
 0100: AF4
 0101: AF5
 0110: AF6
 0111: AF7
 1000: AF8
 1001: AF9
 1010: AF10
 1011: AF11
 1100: AF12
 1101: AF13
 1110: AF14
 1111: AF15

Note: The bitfield is reserved and must be kept to reset value when the corresponding I/O is not available on the selected package.

13.4.11 GPIO port bit reset register (GPIOx_BRR) (x = A to I)

Address offset: 0x28

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
BR15	BR14	BR13	BR12	BR11	BR10	BR9	BR8	BR7	BR6	BR5	BR4	BR3	BR2	BR1	BR0
w	w	w	w	w	w	w	w	w	w	w	w	w	w	w	w

Bits 31:16 Reserved, must be kept at reset value.

Bits 15:0 **BRy**: Port x reset IO pin y (y = 15 to 0)

These bits are write-only. A read to these bits returns the value 0x0000.

0: No action on the corresponding ODy bit

1: Reset the corresponding ODy bit

Note: The bit is reserved and must be kept to reset value when the corresponding I/O is not available on the selected package.

13.4.12 GPIO high-speed low-voltage register (GPIOx_HSLVR) (x = A to I)

Address offset: 0x2C

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
HSLV 15	HSLV 14	HSLV 13	HSLV 12	HSLV 11	HSLV 10	HSLV 9	HSLV 8	HSLV 7	HSLV 6	HSLV 5	HSLV 4	HSLV 3	HSLV 2	HSLV 1	HSLV 0
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bits 31:16 Reserved, must be kept at reset value.

Bits 15:0 **HSLVy**: Port x high-speed low-voltage configuration (y = 15 to 0)

These bits are written by software to optimize the I/O speed when the I/O supply is low.

Each bit is active only if the corresponding IO_VDD_HSLV/IO_VDDIO2_HSLV user option bit is set. It must be used only if the I/O supply voltage is below 2.7 V.

Setting these bits when the I/O supply (VDD or VDDIO2) is higher than 2.7 V may be destructive.

0: I/O speed optimization disabled

1: I/O speed optimization enabled

Note: Not all I/Os support the HSLV mode. Refer to the I/O structure in the corresponding datasheet for the list of I/Os supporting this feature. Other I/Os HSLV configuration must be kept at reset value.

The bit is reserved and must be kept to reset value when the corresponding I/O is not available on the selected package.

13.4.13 GPIO secure configuration register (GPIOx_SECCFGR) (x = A to I)

When the system is secure (TZEN = 0xB4), this register provides write access security and can be written only by a secure access. It is used to configure a selected I/O as secure. A nonsecure write access to this register is discarded.

When the system is not secure (TZEN = 0xC3), this register is RAZ/WI.

Address offset: 0x30

Reset value: 0x0000 FFFF (for ports A to H)

Reset value: 0x0000 0FFF (for port I)

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
SEC15	SEC14	SEC13	SEC12	SEC11	SEC10	SEC9	SEC8	SEC7	SEC6	SEC5	SEC4	SEC3	SEC2	SEC1	SEC0
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bits 31:16 Reserved, must be kept at reset value.

Bits 15:0 **SECy**: I/O pin of Port x secure bit enable y (y = 15 to 0)

These bits are written by software to enable or disable the I/O port pin security.

0: The I/O pin is nonsecure

1: The I/O pin is secure. Refer to [Table 132](#) for all corresponding secured bits.

Note: The bit is reserved and must be kept to reset value when the corresponding I/O is not available on the selected package.

13.4.14 GPIO register map

Table 133. GPIO register map and reset values

Offset	Register name	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0																																
0x00	GPIOx_MODER (x = A to I)	MODE15[1:0]				MODE14[1:0]				MODE13[1:0]				MODE12[1:0]				MODE11[1:0]				MODE10[1:0]				MODE9[1:0]				MODE8[1:0]				MODE7[1:0]				MODE6[1:0]				MODE5[1:0]				MODE4[1:0]				MODE3[1:0]				MODE2[1:0]				MODE1[1:0]				MODE0[1:0]			
	Reset value for port A	1	0	1	0	1	0	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1														
	Reset value for port B	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	0	1	0	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1														
	Reset value for ports C...H	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1														
	Reset value for port I	0	0	0	0	0	0	0	0	0	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1														
0x04	GPIOx_OTYPER (x = A to I)	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	OT15	OT14	OT13	OT12	OT11	OT10	OT9	OT8	OT7	OT6	OT5	OT4	OT3	OT2	OT1	OT0																																
	Reset value																	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0															
0x08	GPIOx_OSPEEDR (x = A to I)	OSPEED15[1:0]		OSPEED14[1:0]		OSPEED13[1:0]		OSPEED12[1:0]		OSPEED11[1:0]		OSPEED10[1:0]		OSPEED9[1:0]		OSPEED8[1:0]		OSPEED7[1:0]		OSPEED6[1:0]		OSPEED5[1:0]		OSPEED4[1:0]		OSPEED3[1:0]		OSPEED2[1:0]		OSPEED1[1:0]		OSPEED0[1:0]																																	
	Reset value for port A	0	0	0	0	1	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0															
	Reset value for port B	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0															
	Reset value for ports C...I	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0															
0x0C	GPIOx_PUPDR (x = A to I)	PUPD15[1:0]		PUPD14[1:0]		PUPD13[1:0]		PUPD12[1:0]		PUPD11[1:0]		PUPD10[1:0]		PUPD9[1:0]		PUPD8[1:0]		PUPD7[1:0]		PUPD6[1:0]		PUPD5[1:0]		PUPD4[1:0]		PUPD3[1:0]		PUPD2[1:0]		PUPD1[1:0]		PUPD0[1:0]																																	
	Reset value for port A	0	1	1	0	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0																
	Reset value for port B	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0															
	Reset value for ports C...I	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0															
0x10	GPIOx_IDR (x = A to I)	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	ID15	ID14	ID13	ID12	ID11	ID10	ID9	ID8	ID7	ID6	ID5	ID4	ID3	ID2	ID1	ID0																																
	Reset value																	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X																
0x14	GPIOx_ODR (x = A to I)	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	OD15	OD14	OD13	OD12	OD11	OD10	OD9	OD8	OD7	OD6	OD5	OD4	OD3	OD2	OD1	OD0																																
	Reset value																	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0															
0x18	GPIOx_BSRR (x = A to I)	BR15	BR14	BR13	BR12	BR11	BR10	BR9	BR8	BR7	BR6	BR5	BR4	BR3	BR2	BR1	BR0	BS15	BS14	BS13	BS12	BS11	BS10	BS9	BS8	BS7	BS6	BS5	BS4	BS3	BS2	BS1	BS0																																
	Reset value	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0															
0x1C	GPIOx_LCKR (x = A to I)	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	LCKK	LCK15	LCK14	LCK13	LCK12	LCK11	LCK10	LCK9	LCK8	LCK7	LCK6	LCK5	LCK4	LCK3	LCK2	LCK1	LCK0																																
	Reset value																0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0																
0x20	GPIOx_AFRL (x = A to I)	AFSEL7[3:0]			AFSEL6[3:0]			AFSEL5[3:0]			AFSEL4[3:0]			AFSEL3[3:0]			AFSEL2[3:0]			AFSEL1[3:0]			AFSEL0[3:0]																																										
	Reset value	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0																
0x24	GPIOx_AFRH (x = A to H)	AFSEL15[3:0]			AFSEL14[3:0]			AFSEL13[3:0]			AFSEL12[3:0]			AFSEL11[3:0]			AFSEL10[3:0]			AFSEL9[3:0]			AFSEL8[3:0]																																										
	Reset value	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0																

Table 133. GPIO register map and reset values (continued)

Offset	Register name	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0x28	GPIOx_BRR (x = A to I)	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	BR15	BR14	BR13	BR12	BR11	BR10	BR9	BR8	BR7	BR6	BR5	BR4	BR3	BR2	BR1	BR0
	Reset value																	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0x2C	GPIOx_HSLVR (x = A to I)	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	HSLV15	HSLV14	HSLV13	HSLV12	HSLV11	HSLV10	HSLV9	HSLV8	HSLV7	HSLV6	HSLV5	HSLV4	HSLV3	HSLV2	HSLV1	HSLV0
	Reset value																	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0x30	GPIOx_SECCFGR (x = A to I)	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	SEC15	SEC14	SEC13	SEC12	SEC11	SEC10	SEC9	SEC8	SEC7	SEC6	SEC5	SEC4	SEC3	SEC2	SEC1	SEC0
	Reset value for A to H																	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1
	Reset value for port I																	0	0	0	0	1	1	1	1	1	1	1	1	1	1	1	1

Refer to [Section 2.3: Memory organization](#) for the register boundary addresses.

14 System configuration, boot, and security (SBS)

14.1 SBS introduction

The devices feature a set of configuration registers located in the SBS. On top of various device configurations, this peripheral controls key boot and security features, including debug control and secure storage control.

14.2 SBS main features

- System configuration
 - Manage safety feature
 - Enable/disable the FMP (Fast-mode Plus) high-drive capability of some I/Os and voltage booster for I/O analog switches
 - Manage the I/O compensation cell
 - Configure register security access
 - Configuration for Cortex-M33 FPU interrupts
 - Selection and configuration of the Ethernet PHY interface
- Boot control
 - Upon system reset, configure the Cortex-M33 boot address and the temporal isolation level depending on the current configuration such as `PRODUCT_STATE` (`sbs_product_state`), `BOOT_UBE` (`sbs_irot_select`), or `TZEN` (`sbs_tzen`).
 - Manage the temporal isolation feature implemented thanks to the hide protect level (HDPL) monotonic counter
- Debug control
 - Control the opening of the device debug interface, ensuring the sequencing of events that guarantee the device security
- Hardware secure storage control
 - Control the secure storage selections: OBK-HDPL selection (selects the area of secure storage of the flash memory and selects the corresponding DHUK in SAES), EPOCH selection
- Security status
 - SRAM and caches erase status

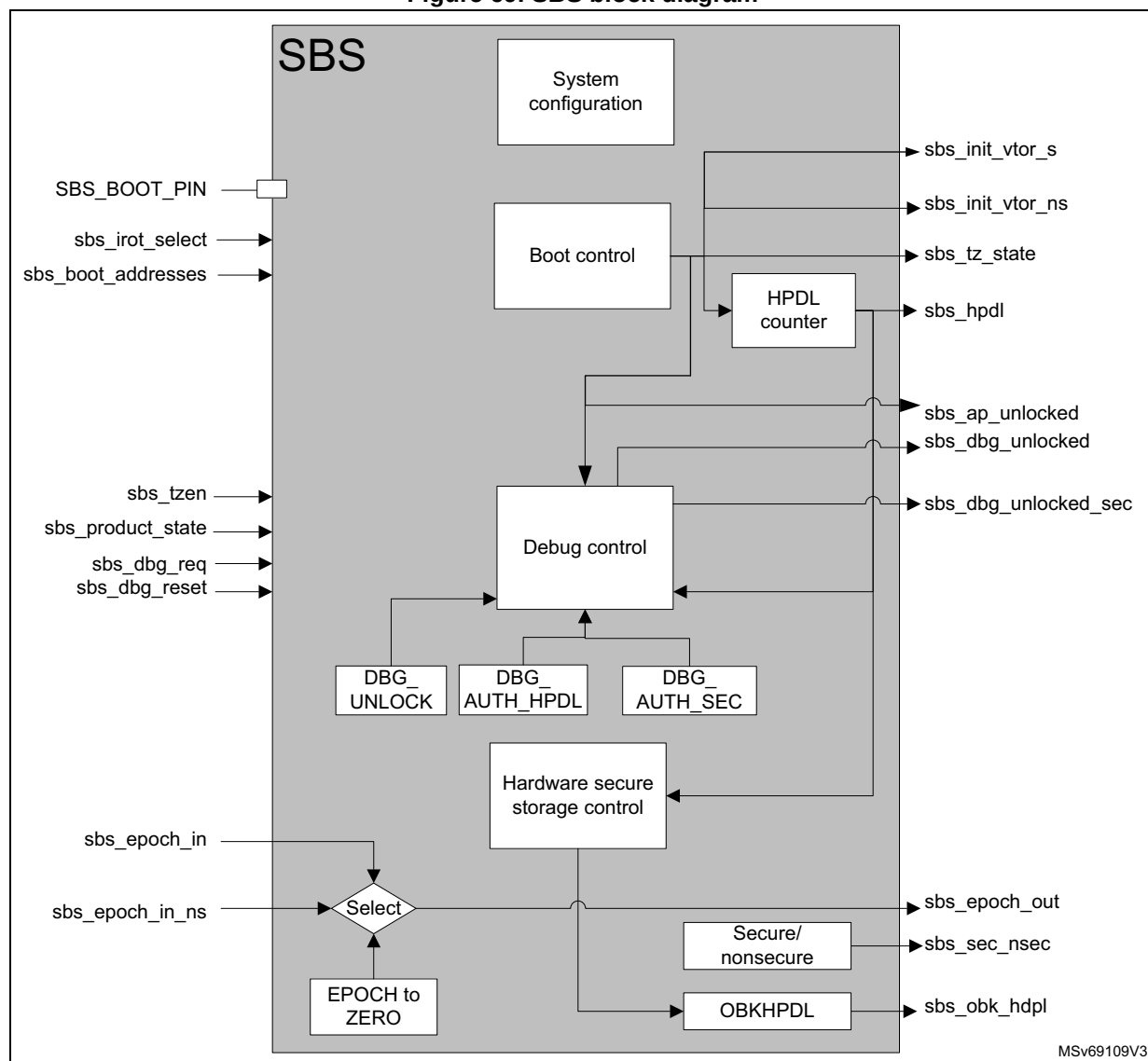
14.3 SBS functional description

14.3.1 SBS block diagram

Figure 69 shows the SBS block diagram, including the four main functions:

- System configuration
- Boot control
- Debug control
- Hardware secure storage control

Figure 69. SBS block diagram



14.3.2 SBS signals

[Table 134](#) details the user relevant internal signals that interface the SBS.

Table 134. SBS internal input/output signals

Signal name	Type	Description
BOOT0	Input	Select booting on user flash memory or Bootloader when TrustZone is disabled (TZEN = 0xC3), or on RSS when TrustZone is enabled (TZEN = 0xB4).
sbs_irot_select		Signal based on BOOT_UBE option byte to select the iRoT between STiRoT and user flash memory (OEMiRoT)
sbs_tzen		Signal based on TZEN option byte to activate/deactivate the TrustZone
sbs_boot_addresses		List of addresses defined by the flash memory: – NSBOOTADD: nonsecure boot address – SECBOOTADD: secure boot address
sbs_product_state		Signal based on PRODUCT_STATE option byte to activate the different security mechanisms depending on the product use. Expected values are described in Section 7: Embedded flash memory (FLASH) .
sbs_dbg_req		Launch the debug authentication protocol when booting.
sbs_init_vtor_s	Output	Vector address for Cortex-M33 secure entry point
sbs_init_vtor_ns		Vector address for Cortex-M33 nonsecure entry point
sbs_tz_state		Inform the Cortex-M33 on the secure state of the core.
sbs_hdpl		HDPL (temporal isolation level, ID of the boot level, used to isolate boot levels) This signal reflects a monotonic counter that can be incremented from 0 to 3, and that is reset to zero only on software reset or POR.
sbs_ap_unlocked		Control the Cortex-M33 access port.
sbs_dbg_unlocked		Unlock the debug (when set to 1) for the Cortex-M33 nonsecure part.
sbs_dbg_unlocked_sec		Unlock the debug (when set to 1) for the Cortex-M33 secure part.
sbs_dbg_reset	Input	This signal is used to control the reset of the debug authentication configuration to be done with a system reset or a power-on reset. The configuration is done through the DBGMCU using the DCRT bitfield of DBGMCU_CR register.
sbs_obk_hdpl	Output	Select secure storage domain (OBK-HDPL) for current HDPL, or greater ones (to allow provisioning).
sbs_epoch_out		EPOCH counters (NS_EPOCH and SEC_EPOCH, inputs from flash memory) are used to manage the REPLAY protection.
sbs_sec_nsec		Reflect secure/nonsecure selection for the secure storage.

14.3.3 SBS reset and clocks

The SBS configuration port is clocked by the AHB bus clock. There is a general reset and a debug configuration reset controlled in DBGMCU.

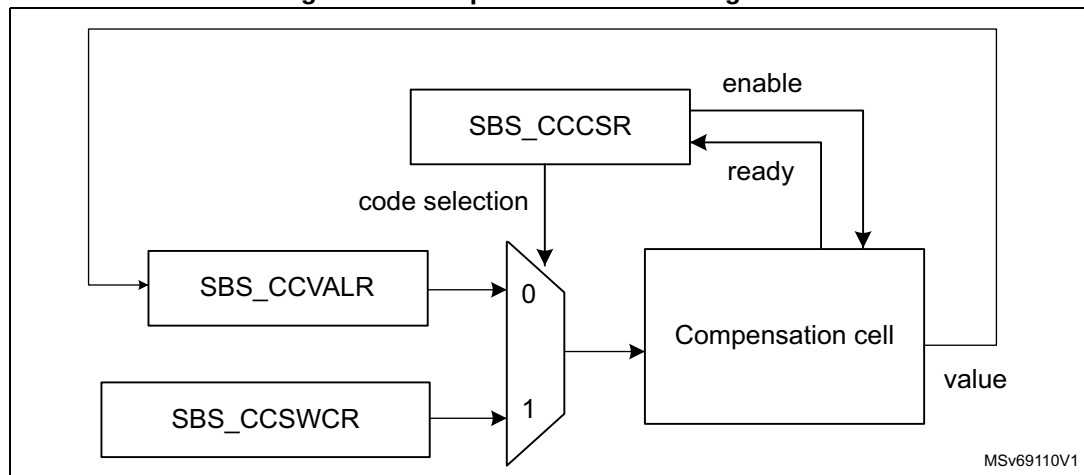
14.3.4 SBS system configuration

SBS I/O compensation cell management

The I/O compensation cell generates an 8-bit value for the I/O buffer (4 bits for N-MOS and 4 bits for P-MOS), that depends on PVT operating conditions (process, voltage, temperature). These bits are used to control the output impedance in the I/O buffer, and the slew rate of the I/O commutation (t_{fall} and t_{rise}), to reduce the I/O noise on power supply.

As shown in [Figure 70](#), the compensation cell is split in two blocks, one to provide an optimal code for the current PVT, and another to drive the block controlled by the software.

Figure 70. Compensation cell management



The compensation cell value can be read when the READY flag is set in `SBS_CCCSR`. With `CODESEL` in `SBS_CCCSR`, the application can select the value to apply between two options: the code from the cell or the code from `SBS_CCSWCR`.

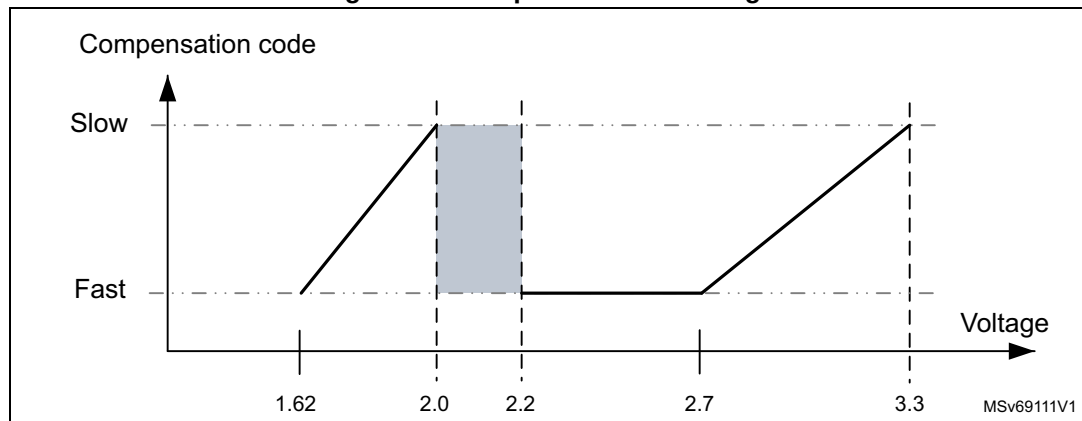
Two compensation cells are embedded in STM32H5 devices, one for the I/Os supplied by V_{DDIO} power rail, another for the I/Os supplied by V_{DDIO2} power rail.

By default, the compensation cells are disabled, and a fixed code is applied to all the I/Os.

Note: The compensation cell can be used only when $2.7\text{ V} \leq V_{\text{DDIOx}} \leq 3.6\text{ V}$ or $1.62\text{ V} \leq V_{\text{DDIOx}} \leq 2\text{ V}$ (see [Figure 71](#)).

Note: The compensation cell can be used only when the CSI oscillator is enabled, see [Section 11: Reset and clock control \(RCC\)](#) for more details on CSI oscillator.

Figure 71. Compensation cell usage



SBS TrustZone security and privilege

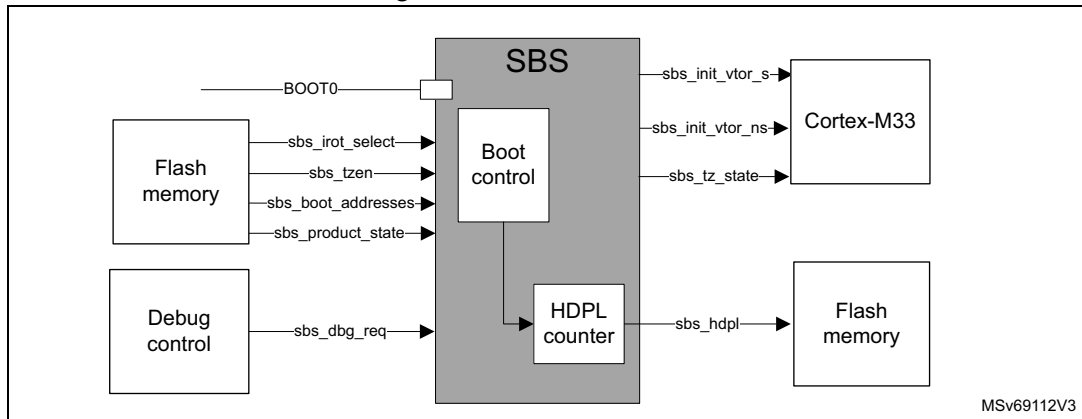
- SBS TrustZone security**
 When the TrustZone security is activated, the SBS is able to protect secure registers from being modified by nonsecure accesses.
 The TrustZone security is activated by the TZEN option byte in FLASH_OPTSR2_PRG.
 A nonsecure read/write access to a secured register is RAZ/WI and generates an illegal access event. An illegal access interrupt is generated if the SBS illegal access event is enabled in the GTZC.
- Privileged/unprivileged mode**
 The SBS registers can be read and written by privileged and unprivileged accesses except the SBS registers for CPU configuration:
 - SBS_CSLCKR, SBS_FPUIMR and SBS_CNSLCKR
 - FPUSEC in SBS_SECCFGR
 An unprivileged access to a privileged register is RAZ/WI.

14.3.5 SBS boot control

The SBS can be used to control the boot entry points considering the product settings. The main boot control actions are listed below:

- Run product with or without TrustZone enabled.
- Select between STiRoT or OEMiRoT.
- Boot when launching a debug authentication sequence.
- Select boot between the bootloader or the user flash memory boot.
- Initialize the HDPL boot value.

Figure 72. SBS boot control



The boot configurations are selected considering the product settings:

- **BOOT0**: to select booting on user flash memory or RSS (root secure services)
- **BOOT_UBE** option byte to select the iRoT between STiRoT and OEMiRoT
- **TZEN** option byte to activate/deactivate the Trust Zone
- **sbs_boot_addresses**: list of addresses defined by the flash memory:
 - **NSBOOTADD**: nonsecure boot address
 - **SECBOOTADD**: secure boot address
- **PRODUCT_STATE**: option byte to activate the different security mechanisms depending on the product use. Expected values are described in [Section 7: Embedded flash memory \(FLASH\)](#).
- **sbs_dbg_req**: used to launch the debug authentication protocol when booting

The boot control logic sets the following data:

- **sbs_init_vtor_s**: vector address for Cortex-M33 secure entry point
- **sbs_init_vtor_ns**: vector address for Cortex-M33 nonsecure entry point
- **sbs_tz_state** (secure/nonsecure): informs the Cortex-M33 on the secure state of the core.
- **HDPL** (see the description below)

SBS HDPL (temporal isolation level) management

The HDPL is a monotonic counter incremented during the boot stages. The HDPL is reset to its default value only after a power-on or a system reset. This default value (0 or 1) depends on the device life cycle, as defined in boot logic.

The STM32H5 series devices use HDPL information to automatically isolate code and its associated secrets (like keys) during the boot process. Incrementing HDPL ensures that private code and data for one boot stage cannot be directly accessible from later boot stages.

The HDPL is used by the user flash memory, see [Section 7: Embedded flash memory \(FLASH\)](#) for more details. The HDPL can take values from 0 to 3. When reaching the value 3, HDPL keeps this value until reset. The current HDPL value is readable in HDPL bitfield in **SBS_HDPLSR**.

To increment the HDPL by one, the application must write 0x6A to INCR_HDPL in SBS_HDPLCR. After such increment, and before doing any subsequent action, the user must check that the HDPL has effectively been incremented, by reading SBS_HDPLSR.

Table 135. HDPL encoded values

HDPL	Code
0	0xB4
1	0x51
2	0x8A
3	0x6F
	All other values

Table 136. SBS boot logic

Inputs						Outputs			
sbs_product_state	sbs_dbg_req	sbs_tzen	sbs_irotselect	BOOT0	sbs_boot_addresses	sbs_init_vtor_s	sbs_init_vtor_ns	sbs_hdpl	sbs_tz_state
Any except Locked	1	x	x	x	BOOT_DBG_AUTH_ADD	BOOT_DBG_AUTH_ADD	x	1	Secure
Open	0	0	x	0	NSBOOT_ADD	x	NSBOOT_ADD	1	Nonsecure
	0	0	x	1	BOOT_ST_RSS_ADD	BOOT_ST_RSS_ADD	x	0	Secure
	0	1	x	0	SECBOOT_ADD	SECBOOT_ADD	x	1	secure
	0	1	x	1	BOOT_ST_RSS_ADD	BOOT_ST_RSS_ADD	x	0	Secure
Provisioning	0	x	x	x	BOOT_ST_RSS_ADD	BOOT_ST_RSS_ADD	x	0	Secure
iRoT-Provisioned	0	0	x	x	NSBOOT_ADD	x	NSBOOT_ADD	1	Nonsecure
	0	1	0	x	BOOT_ST_IROT_ADD	BOOT_ST_IROT_ADD	x	1	Secure
	0	1	1	x	SECBOOT_ADD	x	SECBOOT_ADD	1	Secure
TZ-closed	-	1	0	x	BOOT_ST_IROT_ADD	BOOT_ST_IROT_ADD	x	1	Secure
	-	1	1	x	SECBOOT_ADD	x	SECBOOT_ADD	1	Secure

Table 136. SBS boot logic (continued)

Inputs						Outputs			
sbs_product_state	sbs_dbg_req	sbs_tzen	sbs_irotselect	BOOT0	sbs_boot_addresses	sbs_init_vtor_s	sbs_init_vtor_ns	sbs_hdpl	sbs_tz_state
Closed	0	0	x	x	NSBOOT_ADD	x	NSBOOT_ADD	1	Nonsecure
	0	1	0	x	BOOT_ST_IROT_ADD	BOOT_ST_IROT_ADD	x	1	Secure
	0	1	1	x	SECBOOT_ADD	x	SECBOOT_ADD	1	Secure
Locked	0	0	x	x	NSBOOT_ADD	x	NSBOOT_ADD	1	Nonsecure
	0	1	0	x	BOOT_ST_IROT_ADD	BOOT_ST_IROT_ADD	x	1	Secure
	0	1	1	x	SECBOOT_ADD	x	SECBOOT_ADD	1	Secure
Regression	0	0	x	x	BOOT_DBG_AUTH_ADD	BOOT_DBG_AUTH_ADD	x	0	Secure
	0	1	x	x	BOOT_DBG_AUTH_ADD	BOOT_DBG_AUTH_ADD	x	0	Secure
NS-Regression	0	1	x	x	BOOT_DBG_AUTH_ADD	BOOT_DBG_AUTH_ADD	x	0	Secure

14.3.6 SBS debug control

The SBS debug control is used to manage debug opening, taking care on the product context (PRODUCT_STATE, TZEN, HDPL) on register settings, or through a debug authentication control.

When the debug is forbidden, the mailbox access port, Cortex-M33 access port and CPU debug interface are locked. In this situation, the debugger cannot access the CPU and no effective debug can be done. Refer to the [Section 66: Debug support \(DBG\)](#) for more details.

Authenticated debug sequence

1. The external host requests to launch the debug authentication protocol, via the DBGMCU access port mailbox. The rest of the device is kept under reset.
2. SBS selects the STMicroelectronics RSS-DA (debug authentication library) boot address, and requests the CPU to be released from reset.

3. The CPU running RSS-DA library executes the debug authentication protocol in the system flash memory. If the device is closed, the access port mailbox is closed until RSS-DA acknowledges the authentication sequence start request.
4. The authentication method depends on TrustZone activation:
 - When TrustZone is activated (TZEN = 0xB4), the authentication method is based on certificates. As soon as a debug certificate chain is fully verified by the device, if the certificate concerns a debug permission, the RSS-DA programs the debug opening of the Cortex-M33. Alternatively, the certificate can authorize partial or full regression, allowing debug on a regressed part.
 - When TrustZone is disabled (TZEN = 0xC3), the authentication method is based on a password. This method only allows the full regression of the product to be controlled.
5. Above reopenings are effective only when HDPL in SBS_HDPLSR has a value equal or superior to the value programmed in DBG_AUTH_HDPL in SBS_DBGCR. In case of authentication failure, the user is informed through the host interface.

Note: The debug authentication library in system flash memory is available only when HDPL = 0 or 1 in SBS_HDPLSR. Only this library can perform the steps 3 and 4 described above.

Debug reset

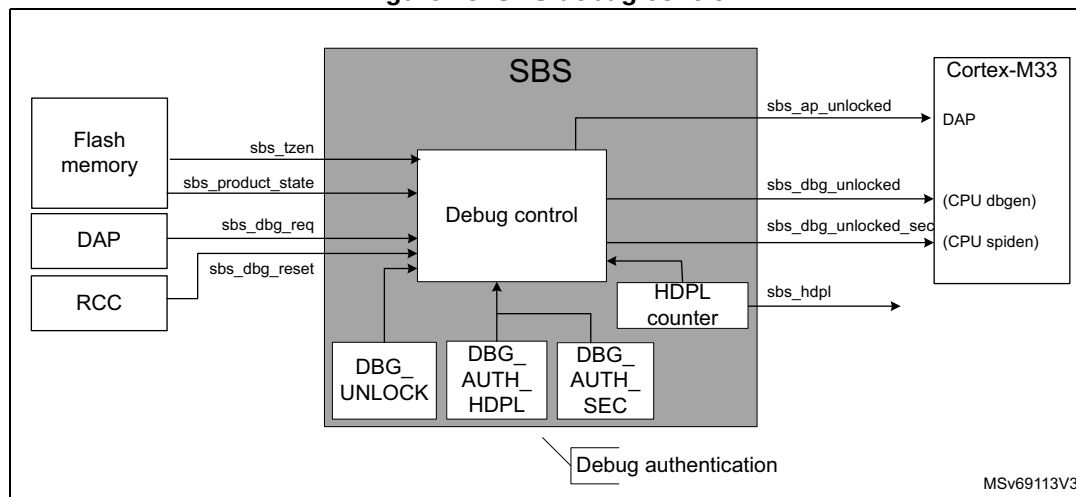
The debug opening can have the configuration to be reset by system or power-on reset, depending on a DBGMCU register field.

Debug locking

The debug configuration can be locked thanks to DBGCFG_LOCK in SBS_DBGLOCKR. SBS_DBGCR is then no longer writable.

When DBGCFG_LOCK is set to 1, it can be reset only by system or power-on reset. The configuration is done through the DBGMCU using the DCRT field of DBGMCU_CR.

Figure 73. SBS debug control



Inputs used to control the debug opening:

- `sbs_dbg_req`: input signal that a host requests to launch the debug authentication protocol. When this signal is set, the boot address is set to launch the debug authentication library.
- `sbs_tzen`: inform on the selected platform configuration related to TrustZone activation. When TZEN value is set to zero (0xC3), the platform does not support TrustZone and the simplified life cycle is proposed. See [Section 7: Embedded flash memory \(FLASH\)](#) for more details.
- `sbs_product_state`: provide the current PRODUCT_STATE (from flash memory). See [Section 7: Embedded flash memory \(FLASH\)](#) for more details.
- `sbs_dbg_reset`: reset information coming from the RCC, and reset SBS_DBGCR if this register is configured to be reset.

Configuration

- `DBG_UNLOCK` in SBS_DBGCR: debug unlock when `DBG_AUTH_HDPL` is reached
- `AP_UNLOCK` in SBS_DBGCR: access port unlock
- `DBG_AUTH_HDPL` in SBS_DBGCR: authenticated debug temporal isolation level. Define the HDPL value from which the debug can be opened. Value can only be from 1 to 3.
- `DBG_AUTH_SEC` in SBS_DBGCR: specify if the debug reopening is for nonsecure only or for both (secure and nonsecure).
- `DBG_LOCK` in SBS_DBGLOCKR: lock the current debug configuration (released only by a reset).

Outputs

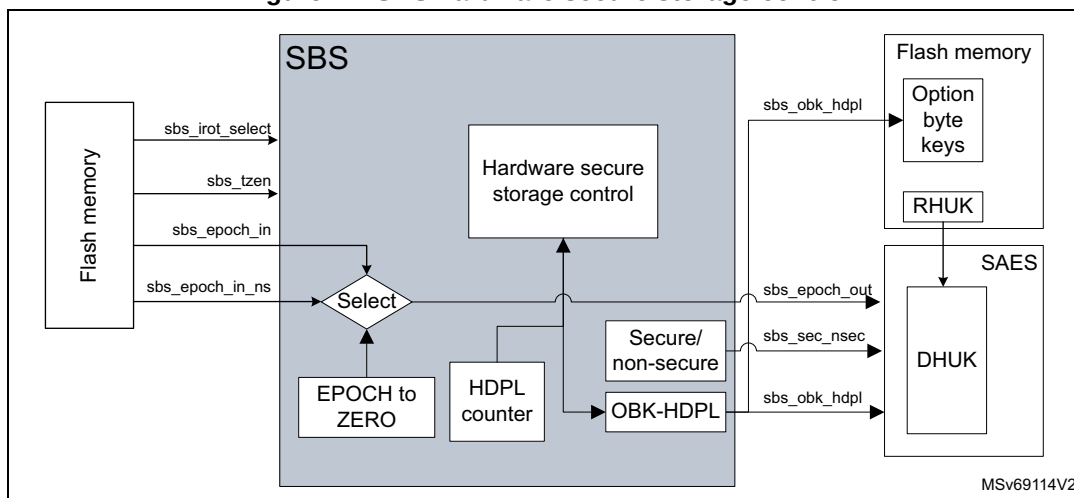
- `sbs_ap_unlocked`: signal controlling the Cortex-M33 access port
 - 1: 0xB4
 - 0: all other codes
- `sbs_dbg_unlocked`: unlock the debug (when set to 1) for the Cortex-M33 nonsecure.
 - 1: 0xB4
 - 0: all other codes
- `sbs_dbg_unlocked_sec`: unlock the debug (when set to 1) for the Cortex-M33 secure.
 - 1: 0xB4
 - 0: all other codes

14.3.7 SBS hardware secure storage control

This feature ensures the isolation of keys and data related to RoT (root-of-trust) when re-opening the debug (when product in the field).

This includes a dedicated area called OB-Keys in the flash memory (see [Section 7: Embedded flash memory \(FLASH\)](#) for more details), and the key derivation (DHUK) mechanism protecting data using a hardware key different for the identified domains.

Figure 74. SBS hardware secure storage control



- `sbs_obk_hdpl`: select secure storage context for current HDPL, or greater ones (to allow provisioning). This signal is set thanks to the `SBS_NEXTHDPLCR` register. This information is used by the flash memory to select the right area, and by the SAES when processing the derived key to generate a key derived related to HDPL context.
- `sbs_epoch_out`: EPOCH counters are used to manage the REPLAY protection. These counters are incremented when regressions are done. Injecting the EPOCH in the DHUK (in the SAES) ensures that all information encrypted with the DHUK cannot be replayed after a regression.
- `sbs_sec_nsec`: apply a different protection DHUK for secure and nonsecure assets.

All keys encrypted using the SAES/DHUK inherit of the RHUK property: unique per device.

All data encrypted thanks to the SAES using the DHUK are specific to a combination of [sbs obk hdp] + sbs epoch out + sbs sec nsec].

Inputs used to control the hardware secure storage control

sbs_epoch_in and sbs_epoch_in_ns: 24-bit values coming from the flash memory and representing regression counters respectively for secure and nonsecure.

Configuration

- NEXTHDPL in SBS_NEXTHDPLCR: access to next HDPL secure storage areas.
- EPOCH_SEL in SBS_EPOCHSELCR: select the EPOCH source related to the secure storage under control (SEC EPOCH, NS EPOCH, or FORCED To Zero).

Table 137. OBK-HDPL logic

HDPL[7:0] in SBS_HDPLSR	NEXTHDPL[1:0]			
	0x0	0x1	0x2	0x3
0 (0xB4)	0xB4	0x51	0x8A	0x6F
1 (0x51)	0x51	0x8A	0x6F	
2 (0x8A)	0x8A	0x6F		
3 (0x6F)	0x6F			
Others				

14.4 SBS interrupts

SBS does not support interrupts.

14.5 SBS registers

14.5.1 SBS temporal isolation control register (SBS_HDPLCR)

Address offset: 0x010

Reset value: 0x0000 00B4

Reset: system reset

Register security: no restriction

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	INCR_HDPL[7:0]							
								r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w

Bits 31:8 Reserved, must be kept at reset value.

Bits 7:0 **INCR_HDPL[7:0]**: increment HDPL value

0xB4: no increment

0x6A: recommended value to increment HDPL level by one

Others: all other values allow a HDPL level increment.

14.5.2 SBS temporal isolation status register (SBS_HDPLSR)

Address offset: 0x014

Reset value: 0xFFFF XXXX

The reset value depends on boot case: booting with HDPL0 for ST code, or HDPL1 for all other cases. See [Table 136](#) for more details.

Reset: system reset

Register security: no restriction

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	HDPL[7:0]							
								r	r	r	r	r	r	r	r

Bits 31:8 Reserved, must be kept at reset value.

Bits 7:0 **HDPL[7:0]**: temporal isolation level

This bitfield returns the current temporal isolation level.

0xB4: HDPL0, RSS

0x51: HDPL1, iRoT

0x8A: HDPL2, uRoT

0x6F: HDPL3, application (secure/nonsecure)

14.5.3 SBS next HDPL control register (SBS_NEXTHDPLCR)

Address offset: 0x018

Reset value: 0x0000 0000

Reset: system reset

Register security: no restriction

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	NEXTHDPL[1:0]	
														rw	rw

Bits 31:2 Reserved, must be kept at reset value.

Bits 1:0 **NEXTHDPL[1:0]**: index to point to a higher HDPL than the current one

Index to add to the current HDPL to point (through OBK-HDPL) to the next secure storage areas (OBK-HDPL = HDPL + NEXTHDPL). See [Table 137: OBK-HDPL logic](#) for more details.

14.5.4 SBS debug control register (SBS_DBGCR)

Address offset: 0x020

Reset value: 0x0000 0000

Reset: debug reset (system reset or power-on reset)

Register security: HDPL0/1, RAZ/WI otherwise

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
DBG_AUTH_SEC[7:0]								DBG_AUTH_HDPL[7:0]							
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
DBG_UNLOCK[7:0]								AP_UNLOCK[7:0]							
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bits 31:24 **DBG_AUTH_SEC[7:0]**: control debug opening secure/nonsecure

Write 0xB4 to this bitfield to open debug for secure and any other values only opens nonsecure

Bits 23:16 **DBG_AUTH_HDPL[7:0]**: authenticated debug temporal isolation level

Writing to this bitfield defines at which HDPL the authenticated debug opens.

0x51: HDPL1

0x8A: HDPL2

0x6F: HDPL3

Note: Writing any other values is ignored. Reading any other value means the debug never opens.

Bits 15:8 **DBG_UNLOCK[7:0]**: debug unlock when DBG_AUTH_HDPL is reached

Write 0xB4 to this bitfield to open the debug when HDPL in SBS_HDPLSR equals to DBG_AUTH_HDPL in this register.

Bits 7:0 **AP_UNLOCK[7:0]**: access port unlock

Write 0xB4 to this bitfield to open the device access port.

14.5.5 SBS debug lock register (SBS_DBGLOCKR)

Address offset: 0x024

Reset value: 0x0000 00B4

Reset: debug reset (system reset or power-on reset)

Register security: HDPL0/1, RAZ/WI otherwise

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	DBGCFG_LOCK[7:0]							
								rw	rw	rw	rw	rw	rw	rw	rw

Bits 31:8 Reserved, must be kept at reset value.

Bits 7:0 **DBGCFG_LOCK[7:0]**: debug configuration lock

Reading this bitfield returns 0x6A if the bitfield value is different from 0xB4.

0xC3 is the recommended value to lock the debug configuration using this bitfield.

0xB4: Writes to SBS_DBGCR allowed (default)

Others: Writes to SBS_DBGCR ignored

14.5.6 SBS RSS command register (SBS_RSSCMDR)

Address offset: 0x034

Reset value: 0x0000 0000

Reset: power-on reset

Register security: always secure (RAZ/WI if nonsecure)

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
RSSCMD[15:0]															
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bits 31:16 Reserved, must be kept at reset value.

Bits 15:0 **RSSCMD[15:0]**: RSS command

The application can use this bitfield to pass on a command to the RSS, executed at the next reset.

When RSSCMD ≠ 0 and PRODUCT_STATE is in Open, then the system always boots on RSS whatever is the boot pin value.

14.5.7 SBS EPOCH selection control register (SBS_EPOCHSELCR)

Address offset: 0x0A0

Reset value: 0x0000 0000

Reset: system reset

Register security: Secure when TZ_STATE = 1 (RAZ, WI in nonsecure access). Nonsecure protection when TZ_STATE = 0. This register is protected by privileged whatever is TZ_STATE, RAZ/WI if non privileged access.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	EPOCH_SEL[1:0]	
														rw	rw

Bits 31:2 Reserved, must be kept at reset value.

Bits 1:0 **EPOCH_SEL[1:0]**: select EPOCH value to be sent to the SAES

00: NS_EPOCH (nonsecure) counter input selected

01: SEC_EPOCH counter input selected

1x: EPOCH forced to zero (value used to retrieve PUF reference value at boot time)

14.5.8 SBS security mode configuration control register (SBS_SECCFGR)

Address offset: 0x0C0

Reset value: 0x0000 0000

Reset: system reset

Register security: always secure. RAZ/WI if nonsecure transaction and TZ_STATE = 1.
RAZ/WI if TZ_STATE = 0.

This register is programmed by secure software if the user wants functions configurable through system registers to be secure or not.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	FPUSE C	Res.	CLASS BSEC	SBSSE C
												rw		rw	rw

Bits 31:4 Reserved, must be kept at reset value.

Bit 3 **FPUSEC**: FPU security enable

0: SBS_FPUIMP register accessible through secure or nonsecure transaction

1: SBS_FPUIMP register accessible only through secure transaction

Note: This bit can be written only through privilege transaction.

Bit 2 Reserved, must be kept at reset value.

Bit 1 **CLASSBSEC**: ClassB security enable

0: SBS_CFGR2 register accessible through secure or nonsecure transaction

1: SBS_CFGR2 register accessible only through secure transaction

Bit 0 **SBSSEC**: SBS clock control, memory-erase status register and compensation cell register security enable

0: SBS_MESR, SBS_CCCSR, SBS_CCVALR, SBS_CCOWCR registers accessible through secure or nonsecure transaction

1: SBS_MESR, SBS_CCCSR, SBS_CCVALR, SBS_CCOWCR registers accessible only through secure transaction

14.5.9 SBS product mode and configuration register (SBS_PMCR)

Address offset: 0x100

Reset value: 0x0000 0000

Reset: system reset

STM32H523/533/562/563/573xx devices only

Register security:

- When TrustZone is activated (TZEN = 0xB4):
 - depending on ADC12SEC secure input for ANASWVDD and IO_ANA_BOOST_EN configuration bits
 - depending on I/O PBx_SEC_EN secure input for PBx_FMP configuration bits
 - depending on Ethernet ETHSEC secure input for ETH_SEL_PHY configuration bits
 - depending on SDCE_SEC_EN in SBS_SECCFGR for SMPS_DIV_CLOCK_EN
- When TrustZone is disabled (TZEN = 0xC3) there is no access restriction.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	ETH_SEL_PHY[2:0]			Res.	PB9_FMP	PB8_FMP	PB7_FMP	PB6_FMP
								rw	rw	rw		rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.

Bits 31:24 Reserved, must be kept at reset value.

Bits 23:21 **ETH_SEL_PHY[2:0]**: Ethernet PHY interface selection

000: GMII or MII

001: reserved (RGMII)

100: RMII

Others: reserved

Refer to the product datasheet for the availability of Ethernet. If not present, consider the associated bits as reserved, and keep them at reset value.

Bit 20 Reserved, must be kept at reset value.

Bit 19 **PB9_FMP**: Fast-mode Plus driving capability activation on PB9

This bit can be read and written only with secure access if PB9 is secure in GPIOB. This bit enables the Fm+ driving mode for PB9 when PB9 is not used by I2C peripheral. This can be used to drive a LED for instance.

0: PB9 pin operates in standard mode.

1: Fm+ mode is enabled on PB9 pin and the speed control is bypassed.

Bit 18 **PB8_FMP**: Fast-mode Plus driving capability activation on PB8

This bit can be read and written only with secure access if PB8 is secure in GPIOB. This bit enables the Fm+ driving mode for PB8 when PB8 is not used by I2C peripheral. This can be used to drive a LED for instance.

0: PB8 pin operates in standard mode.

1: Fm+ mode is enabled on PB8 pin and the speed control is bypassed.

Bit 17 **PB7_FMP**: Fast-mode Plus driving capability activation on PB7

This bit can be read and written only with secure access if PB7 is secure in GPIOB. This bit enables the Fm+ driving mode for PB7 when PB7 is not used by I2C peripheral. This can be used to drive a LED for instance.

0: PB7 pin operates in standard mode.

1: Fm+ mode is enabled on PB7 pin and the speed control is bypassed.

Bit 16 **PB6_FMP**: Fast-mode Plus driving capability activation on PB6

This bit can be read and written only with secure access if PB6 is secure in GPIOB. This bit enables the Fm+ driving mode for PB6 when PB6 is not used by I2C peripheral. This can be used to drive a LED for instance.

0: PB6 pin operates in standard mode.

1: Fm+ mode is enabled on PB6 pin and the speed control is bypassed.

Bits 15:8 Reserved, must be kept at reset value.

Bits 7:0 Reserved, must be kept at reset value.

14.5.10 SBS product mode and configuration register [alternate] (SBS_PMCR)

Address offset: 0x100

Reset value: 0x4000 0000

Reset: system reset

STM32H543/553xx devices only

Register security:

- When TrustZone is activated (TZEN = 0xB4):
 - depending on I/O PBx_SEC_EN secure input for PBx_FMP configuration bits
 - depending on Ethernet ETH_SEC_EN secure input for ETH_SEL_PHY configuration bits
- When TrustZone is disabled (TZEN = 0xC3), there is no access restriction.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	ETH_TX_LPI	ETH_PD_ACK	ETH_INT_POL	ETH_SEL_PHY[3:0]				Res.	PB9_FMP	PB8_FMP	PB7_FMP	PB6_FMP
				r	r	rw	rw	rw	rw	rw		rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.

Bits 31:28 Reserved, must be kept at reset value.

Bit 27 **ETH_TX_LPI**: ETH TxLPI status

This bit is set and reset by hardware.

When set, this bit indicates that the MAC has entered Tx LPI mode.

Bit 26 **ETH_PD_ACK**: ETH power-down acknowledge

This bit is set and reset by hardware.

When set, this bit indicates that the Ethernet controller has completed its power-down sequence and its clocks can be stopped.

0: ETH functional / power-down sequence not yet completed

1: ETH power-down sequence completed

Bit 25 **ETH_INT_POL**: ETH external PHY interrupt polarity configuration

0: active high

1: active low

Bits 24:21 **ETH_SEL_PHY[3:0]**: Ethernet PHY interface selection

0000: GMII or MII
 0001: reserved (RGMII)
 0100: RMII
 Others: reserved

Bit 20 Reserved, must be kept at reset value.

Bit 19 **PB9_FMP**: Fast-mode Plus driving capability activation on PB9

This bit can be read and written only with secure access if PB9 is secure in GPIOB. This bit enables the Fm+ driving mode for PB9 when PB9 is not used by I2C peripheral. This can be used to drive a LED for instance.

0: PB9 pin operates in standard mode.

1: Fm+ mode is enabled on PB9 pin and the speed control is bypassed.

Bit 18 **PB8_FMP**: Fast-mode Plus driving capability activation on PB8

This bit can be read and written only with secure access if PB8 is secure in GPIOB. This bit enables the Fm+ driving mode for PB8 when PB8 is not used by I2C peripheral. This can be used to drive a LED for instance.

0: PB8 pin operates in standard mode.

1: Fm+ mode is enabled on PB8 pin and the speed control is bypassed.

Bit 17 **PB7_FMP**: Fast-mode Plus driving capability activation on PB7

This bit can be read and written only with secure access if PB7 is secure in GPIOB. This bit enables the Fm+ driving mode for PB7 when PB7 is not used by I2C peripheral. This can be used to drive a LED for instance.

0: PB7 pin operates in standard mode.

1: Fm+ mode is enabled on PB7 pin and the speed control is bypassed.

Bit 16 **PB6_FMP**: Fast-mode Plus driving capability activation on PB6

This bit can be read and written only with secure access if PB6 is secure in GPIOB. This bit enables the Fm+ driving mode for PB6 when PB6 is not used by I2C peripheral. This can be used to drive a LED for instance.

0: PB6 pin operates in standard mode.

1: Fm+ mode is enabled on PB6 pin and the speed control is bypassed.

Bits 15:0 Reserved, must be kept at reset value.

14.5.11 SBS FPU interrupt mask register (SBS_FPUIMR)

Address offset: 0x104

Reset value: 0x0000 001F

Reset: system reset

Register security: depends on FPUSEC in SBS_SECCFGR

This register is accessible only through privilege transaction.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	FPU_IE[5:0]					
										rw	rw	rw	rw	rw	rw

Bits 31:6 Reserved, must be kept at reset value.

Bits 5:0 **FPU_IE[5:0]**: FPU interrupt enable

Set and cleared by software to enable the Cortex-M33 FPU interrupts

FPU_IE[5]: inexact interrupt enable (interrupt disabled at reset)

FPU_IE[4]: input abnormal interrupt enable

FPU_IE[3]: overflow interrupt enable

FPU_IE[2]: underflow interrupt enable

FPU_IE[1]: divide-by-zero interrupt enable

FPU_IE[0]: invalid operation interrupt enable

14.5.12 SBS memory erase status register (SBS_MESR)

Address offset: 0x108

Reset value: 0x0000 000X (bit 0 is not affected by system reset)

Register security: depends on SBSSEC in SBS_SECCFGR

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	IPMEE
															rc_w1
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	MCLR
															rc_w1

Bits 31:17 Reserved, must be kept at reset value.

Bit 16 **IPMEE**: ICACHE erase status

This bit is set by hardware when ICACHE and PKA RAMs erase is completed after potential tamper detection or a product state regression (refer to [Section 52: Tamper and backup registers \(TAMP\)](#) for more details).

This bit is cleared by software by writing 1 to it.

0: ICACHE and PKA RAM erase on going

1: ICACHE and PKA SRAM erase done

Bits 15:1 Reserved, must be kept at reset value.

Bit 0 **MCLR**: device memories erase status

This bit is set by hardware when SRAM2, BKPSRAM, ICACHE, DCACHE and PKA RAMs erase is completed after power-on reset or tamper detection or product state regression (refer to [Section 52: Tamper and backup registers \(TAMP\)](#)).

This bit is not reset by system reset and is cleared by software by writing 1 to it.

0: memory erase on going if not yet cleared by software

1: Memory erase done

14.5.13 SBS compensation cell for I/Os control and status register (SBS_CCCSR)

Address offset: 0x110

Reset value: 0x0000 0000

Reset: system reset

Register security: depends on SBSSEC in SBS_SECCFGR

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	Res.	Res.	Res.	Res.	RDY2	RDY1	Res.	Res.	Res.	Res.	CS2	EN2	CS1	EN1
						r	r					rw	rw	rw	rw

Bits 31:10 Reserved, must be kept at reset value.

Bit 9 **RDY2**: VDDIO2 compensation cell ready flag

This bit provides the status of the VDDIO2 compensation cell.

0: VDDIO2 compensation cell not ready

1: VDDIO2 compensation cell ready (code value provided by the cell can be used)

Bit 8 **RDY1**: VDDIO compensation cell ready flag

This bit provides the status of the compensation cell.

0: VDDIO compensation cell not ready

1: VDDIO compensation cell ready (code value provided by the cell can be used)

Bits 7:4 Reserved, must be kept at reset value.

Bit 3 **CS2**: code selection for VDDIO2 power rail (reset value set to 1)

This bit selects the code to be applied for the I/O compensation cell.

0: Code from the cell (available in SBS_CCVR)

1: Code from SBS_CCCR

Bit 2 **EN2**: enable compensation cell for VDDIO2 power rail

This bit enables the I/O compensation cell.

0: I/O compensation cell disabled

1: I/O compensation cell enabled

Bit 1 **CS1**: code selection for VDDIO power rail (reset value set to 1)

This bit selects the code to be applied for the I/O compensation cell.

0: Code from the cell (available in the SBS_CCVR)

1: Code from SBS_CCCR

Bit 0 **EN1**: enable compensation cell for VDDIO power rail

This bit enables the I/O compensation cell.

0: I/O compensation cell disabled

1: I/O compensation cell enabled

14.5.14 SBS compensation cell for I/Os value register (SBS_CCVALR)

Address offset: 0x114

Reset value: 0x0000 0088

Reset: system reset

Register security: depends on SBSSEC in SBS_SECCFGR

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
APSRC2[3:0]				ANSRC2[3:0]				APSRC1[3:0]				ANSRC1[3:0]			
r	r	r	r	r	r	r	r	r	r	r	r	r	r	r	r

Bits 31:16 Reserved, must be kept at reset value.

Bits 15:12 **APSRC2[3:0]**: compensation value for the PMOS transistor

This value is provided by the cell and must be interpreted by the processor to compensate the slew rate in the functional range.

Bits 11:8 **ANSRC2[3:0]**: Compensation value for the NMOS transistor

This value is provided by the cell and must be interpreted by the processor to compensate the slew rate in the functional range.

Bits 7:4 **APSRC1[3:0]**: compensation value for the PMOS transistor

This value is provided by the cell and must be interpreted by the processor to compensate the slew rate in the functional range.

Bits 3:0 **ANSRC1[3:0]**: compensation value for the NMOS transistor

This value is provided by the cell and must be interpreted by the processor to compensate the slew rate in the functional range.

14.5.15 SBS compensation cell for I/Os software code register (SBS_CCSWCR)

Address offset: 0x118

Reset value: 0x0000 7878

Reset: system reset

Register security: depends on SBSSEC in SBS_SECCFGR

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
SW_APSRC2[3:0]				SW_ANSRC2[3:0]				SW_APSRC1[3:0]				SW_ANSRC1[3:0]			
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bits 31:16 Reserved, must be kept at reset value.

Bits 15:12 **SW_APSRC2[3:0]**: PMOS compensation code for the V_{DDIO} power rails

This bitfield is written by software to define an I/O compensation cell code for PMOS transistors of the VDDIO power rail. This code is applied to the I/O when CS2 is set in SBS_CCSR.

Bits 11:8 **SW_ANSRC2[3:0]**: NMOS compensation code for VDDIO power rails

This bitfield is written by software to define an I/O compensation cell code for NMOS transistors of the VDD power rail. This code is applied to the I/O when CS2 is set in SBS_CCSR.

Bits 7:4 **SW_APSRC1[3:0]**: PMOS compensation code for the VDD power rails

This bitfield is written by software to define an I/O compensation cell code for PMOS transistors of the VDDIO power rail. This code is applied to the I/O when CS1 is set in SBS_CCSR.

Bits 3:0 **SW_ANSRC1[3:0]**: NMOS compensation code for VDD power rails

This bitfield is written by software to define an I/O compensation cell code for NMOS transistors of the VDD power rail. This code is applied to the I/O when CS1 is set in SBS_CCSR.

14.5.16 SBS Class B register (SBS_CFGR2)

Address offset: 0x120

Reset value: 0x0000 0000

Reset: system reset

Register security: depends on CLASSBSEC in SBS_SECCFGR

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	ECCL	PVDL	SEL	CLL
												rs	rs	rs	rs

Bits 31:4 Reserved, must be kept at reset value.

Bit 3 **ECCL**: ECC lock

This bit is set and cleared by software. It can be used to enable and lock the flash memory double ECC error with break input of TIM1/8/15/6/17.

0: double ECC error flag disconnected to timer break inputs

1: double ECC error flag connected to timer break inputs

Bit 2 **PVDL**: PVD lock

This bit is set by software and cleared only by a system reset. It can be used to enable and lock the PVD connection with TIM1/8/15/16/17 break inputs.

0: PVD interrupt disconnected from timer break inputs. PVD_EN and PVD_SEL[2:0] in the PWR registers are read/write.

1: PVD interrupt is connected to timer break inputs. PVD_EN and PVD_SEL[2:0] in the PWR registers are read only

Bit 1 **SEL**: SRAM ECC error lock

This bit is set by software and cleared only by a system reset. It can be used to enable and lock the SRAM double ECC error signal with break input of TIM1/8/15/16/17.

0: SRAM double ECC error flag disconnected from timer break inputs
1: SRAM double ECC error flag connected to timer break inputs

Bit 0 **CLL**: core lockup lock

This bit is set by software and cleared only by a system reset. It can be used to enable and lock the lockup (HardFault) output of Cortex-M33 with TIM1/8/15/16/17 break inputs.

0: lockup output disconnected from timer break inputs
1: lockup output connected to timer break inputs

14.5.17 SBS CPU nonsecure lock register (SBS_CNSLCKR)

Address offset: 0x144

Reset value: 0x0000 0000

Reset: system reset

Register security: This register can be read and written by privileged access only.
Unprivileged access is RAZ/WI.

This register is used to lock the configuration of nonsecure MPU and VTOR_NS registers of the Cortex-M33.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	LOCKNSMPU	LOCKNSVTOR
														rs	rs

Bits 31:2 Reserved, must be kept at reset value.

Bit 1 **LOCKNSMPU**: nonsecure MPU register lock

This bit is set by software and cleared only by a system reset. When set, this bit disables write access to nonsecure MPU_CTRL_NS, MPU_RNR_NS and MPU_RBAR_NS registers.

0: nonsecure MPU registers write enabled
1: nonsecure MPU registers write disabled

Bit 0 **LOCKNSVTOR**: VTOR_NS register lock

This bit is set by software and cleared only by a system reset.

0: VTOR_NS register write enabled
1: VTOR_NS register write disabled

14.5.18 SBS CPU secure lock register (SBS_CSLCKR)

Address offset: 0x148

Reset value: 0x0000 0000

Reset: system reset

Register security: This register can be written only when the access is secure/privilege.

A nonsecure read/write access is RAZ/WI and generates an illegal access event. When the system is not secure (TZ_STATE = 0), this register is RAZ/WI.

This register is used to lock the configuration of PRIS and BFHFNMINs in the AIRCR, SAU, secure MPU and VTOR_S registers of the Cortex-M33.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	LOCKSAU	LOCKSMPU	LOCKSVTAIRCR
													rs	rs	rs

Bits 31:3 Reserved, must be kept at reset value.

Bit 2 **LOCKSAU**: SAU registers lock

This bit is set by software and cleared only by a system reset. When set, this bit disables write access to SAU_CTRL, SAU_RNR, SAU_RBAR and SAU_RLAR registers.

0: SAU registers write enabled

1: SAU registers write disabled

Bit 1 **LOCKSMPU**: secure MPU registers lock

This bit is set by software and cleared only by a system reset. When set, this bit disables write access to secure MPU_CTRL, MPU_RNR and MPU_RBAR registers.

0: Secure MPU registers writes enabled

1: Secure MPU registers writes disabled

Bit 0 **LOCKSVTAIRCR**: VTOR_S and AIRCR register lock

This bit is set by software and cleared only by a system reset. When set, this bit disables write access to VTOR_S register, PRIS and BFHFNMINs bits in the AIRCR register.

0: VTOR_S register PRIS and BFHFNMINs bits in the AIRCR register write enabled

1: VTOR_S register PRIS and BFHFNMINs bits in the AIRCR register write disabled

14.5.19 SBS flitf ECC NMI mask register (SBS_ECCNMIR)

Address offset: 0x14C

Reset value: 0x0000 0000

Reset: system reset

Register security: secure access only when TZ_STATE = 1 (RAZ/WI in nonsecure access).
No security protection if TZ_STATE = 0.

This register is accessible only through privilege transaction.

This register sets up the expected behavior on NMI regarding double ECC errors from the flash memory.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	ECCNMI_MASK_EN
															rw

Bits 31:1 Reserved, must be kept at reset value.

Bit 0 **ECCNMI_MASK_EN**: NMI behavior setup when a double ECC error occurs on FLASH data part

0: NMI generated if a double ECC error in the flitf data part

1: NMI not generated if a double ECC error in the flitf data part

14.5.20 SBS register map

Table 138. SBS register map and reset values

Offset	Register name	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0					
0x010	SBS_HDPLCR	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	INCR_HDPL[7:0]												
	Reset value																									1	0	1	1	0	1	0	0					
0x014	SBS_HDPLSR	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	HDPL[7:0]												
	Reset value																									X	X	X	X	X	X	X	X					
0x018	SBS_NEXTHDPLCR	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	NEXTHDPL[1:0]					
	Reset value																															0	0					
0x020	SBS_DBGCR	DBG_AUTH_SEC[7:0]							DBG_AUTH_HDPL[7:0]							DBG_UNLOCK[7:0]							AP_UNLOCK[7:0]															
	Reset value	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0					
0x024	SBS_DBGLOCKR	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	DBGCFG_LOCK[7:0]												
	Reset value																									1	0	1	1	0	1	0	0					
0x028 to 0x030	Reserved	Reserved																																				
0x034	SBS_RSSCMDR	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	RSSCMD[15:0]																					
	Reset value																0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0						
0x038 to 0x09C	Reserved	Reserved																																				
0x0A0	SBS_EPOCHSELCR	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	EPOCH_SEL[1:0]					
	Reset value																															0	0					
0x0A4 to 0x0BC	Reserved	Reserved																																				
0x0C0	SBS_SECCFGR	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	FPUSEC					
	Reset value																														0	CLASSBSEC	SBSSEC					
0x0C4 to 0x0FC	Reserved	Reserved																																				
0x100	SBS_PMCR	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	ETH_SEL_PHY [2:0] ⁽¹⁾			Res.	Res.	PB9_FMP	PB8_FMP	PB7_FMP	PB6_FMP	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.						
	Reset value									0	0	0			0	0	0	0																				
0x100	SBS_PMCR ⁽²⁾	Res.	Res.	Res.	Res.	ETHXLPI	ETHPDACK	ETHINTPOL	ETH_SEL_PHY [3:0] ⁽¹⁾			Res.	Res.	PB9_FMP	PB8_FMP	PB7_FMP	PB6_FMP	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.					
	Reset value					0	0	0	0	0	0			0	0	0	0																					
0x104	SBS_FPUIMR	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	FPU_IE[5:0]											
	Reset value																										1	1	1	1	1	1	1					

Table 138. SBS register map and reset values (continued)

Offset	Register name	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	
0x108	SBS_MESR	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	IPMEE	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	MCLR	
	Reset value																0																X	
0x110	SBS_CCCSR	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	RDY2	RDY1	Res	Res	Res	Res	Res	CS2	EN2	CS1	EN1	
	Reset value																						0	0	0					0	0	0	0	
0x114	SBS_CCVALR	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	APSRC2 [3:0]				ANSRC2 [3:0]				APSRC1 [3:0]				ANSRC1 [3:0]				
	Reset value																	0	0	0	0	0	0	0	0	1	0	0	0	1	0	0	0	
0x118	SBS_CCSWCR	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	SW_APSRC2 [3:0]				SW_ANSRC2 [3:0]				SW_APSRC1 [3:0]				SW_ANSRC1 [3:0]				
	Reset value																	0	1	1	1	1	0	0	0	0	1	1	1	1	0	0	0	
0x120	SBS_CFGR2	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	ECCL	PVDL	SEL	CLL	
	Reset value																												0	0	0	0		
0x124 to 0x140	Reserved	Reserved																																
0x144	SBS_CNSLCKR	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	LOCKNSMPU	
	Reset value																														0	0	LOCKNSVTR	
0x148	SBS_CSLCKR	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	LOCKSAU
	Reset value																														0	0	LOCKSMPU	
0x14C	SBS_ECCNMIR	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	ECCNMI_MASK_EN
	Reset value																																	0

1. Refer to the product datasheet for the availability of Ethernet. If not present, consider the associated bits as reserved, and keep them at reset value.
2. Only for STM32H543/553xx devices.

Refer to [Section 2.3: Memory organization](#) for the register boundary addresses.

15 Peripherals interconnect matrix

15.1 Interconnect matrix introduction

Several peripherals have direct connections between them, enabling autonomous communication and/or synchronization: this approach saves CPU resources, and power supply consumption. In addition, these hardware connections remove software latency and help the design of predictable system.

Depending on peripherals, these interconnections can operate in Run, Sleep, and Stop modes.

15.2 Connection summary

Table 139. Peripherals interconnect matrix⁽¹⁾⁽²⁾⁽³⁾

Source	Destination																									
	TIM1	TIM8	TIM2	TIM3	TIM4	TIM5	TIM6	TIM7	TIM12	TIM15	TIM16	TIM17	LPTIM1	LPTIM2	LPTIM3/4/5/6	ADC1/2	ADC3	COMP1	COMP2	DAC1/2	GPDMA1/2	TAMP	RTC	DTS	PLAY	AES/SAES
TIM1	-	1	1	1	1	1	-	-	1	1	-	-	-	-	-	2	2	17	17	4	-	-	-	-	-	-
TIM8	1	-	1	1	1	1	-	-	1	1	-	-	-	-	-	2	2	17	17	4	-	-	-	-	-	-
TIM2	1	1	-	1	1	1	-	-	1	1	-	-	-	-	-	2	2	17	17	4	10	-	-	-	20	-
TIM3	1	1	1	-	1	1	-	-	1	1	-	-	-	-	-	2	2	17	17	-	-	-	-	-	20	-
TIM4	1	1	1	1	-	1	-	-	1	1	-	-	-	-	-	2	2	-	-	4	-	-	-	-	-	-
TIM5	1	1	1	1	1	-	-	-	1	1	-	-	-	-	-	-	-	-	-	4	-	-	-	-	-	-
TIM6	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	2	2	-	-	4	-	-	-	-	-	-
TIM7	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	4	-	-	-	-	-	-
TIM12	1	1	1	1	1	1	-	-	-	-	-	-	-	-	-	-	-	-	-	4	10	-	-	-	-	-
TIM13	1	1	1	1	1	1	-	-	1	1	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-
TIM14	1	1	1	1	1	1	-	-	1	1	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-
TIM15	1	1	1	1	1	-	-	-	1	-	-	-	-	-	-	2	2	-	17	4	10	-	-	-	-	-
TIM16	1	1	1	1	1	-	-	-	1	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-
TIM17	1	1	1	1	1	-	-	-	1	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-
LPTIM1/2	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	2	2	17	-	4	10	-	-	21	20	-
LPTIM3/4/5/6	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	10	-	-	-	-	-
ADC1	3	3	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	3	-
ADC2	3	3	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	13	-	-	3	-
ADC3	-	3	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-
COMP1	16	16	16	16	16	16	-	-	16	16	-	-	6	6	6	-	-	-	-	4	10	-	-	-	20	-
COMP2	16	16	16	16	16	16	-	-	16	16	-	-	6	6	6	-	-	-	-	4	10	-	-	-	20	-

Table 139. Peripherals interconnect matrix⁽¹⁾⁽²⁾⁽³⁾ (continued)

Source	Destination																										
	TIM1	TIM8	TIM2	TIM3	TIM4	TIM5	TIM6	TIM7	TIM12	TIM15	TIM16	TIM17	LPTIM1	LPTIM2	LPTIM3/4/5/6	ADC1/2	ADC3	COMP1	COMP2	DAC1/2	GPDMA1/2	TAMP	RTC	DTS	PLAY	AES/SAES	
GPDMA1/2	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	10	-	-	-	-	-	
EXTI	-	-	-	-	-	-	-	-	-	-	-	-	6	-	6	2	-	-	-	-	4	10	-	-	21	-	-
RTC wake-up	-	-	-	-	-	7	-	-	-	-	7	-	-	-	-	-	-	-	-	-	10	-	-	-	-	-	
RTC alarm	-	-	-	-	-	-	-	-	-	-	-	-	6	-	6	-	-	-	-	-	10	-	-	-	-	-	
TAMP	-	-	-	-	-	-	-	-	-	-	-	-	6	-	6	-	-	-	-	-	10	-	14	-	-	15	
HSE	-	-	-	-	-	-	-	-	-	-	5	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	
LSE	-	-	5	-	-	-	-	-	-	5	5	-	-	-	-	-	-	-	-	12	-	-	-	-	-	-	
CSS in LSE	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	13	-	-	-	-	
CSI	-	-	-	-	-	-	-	-	5	5	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	
HSI	-	-	-	-	-	-	-	-	5	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	
LSI	-	-	-	-	-	-	-	-	-	-	5	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	
MCO1	-	-	-	-	-	-	-	-	-	-	-	5	-	-	-	-	-	-	-	-	-	-	-	-	19	-	
MCO2	-	-	-	-	-	-	-	-	-	5	-	-	-	-	-	-	-	-	-	-	-	-	-	-	19	-	
V _{CORE}	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	11	11	-	-	-	-	-	-	-	-	-	
V _{REFINT}	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	11	11	-	-	-	-	-	-	-	-	-	
V _{sensor}	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	11	11	-	-	-	-	-	-	-	-	-	
V _{BAT8}	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	11	11	-	-	-	-	-	-	-	-	-	
V _{BAT} / Temp. monitor	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	13	-	-	-	-	
System errors	8	8	-	-	-	-	-	-	-	8	8	8	-	-	-	-	-	-	-	-	-	-	-	-	-	-	
System flash memory	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	15	
USART1/2	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	20	
SPI1/2	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	20	
AES/SAES	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	15	
Ethernet	-	-	9	9	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	
PLAY	18	18	18	-	-	-	-	-	-	-	-	-	6	6	-	2	-	-	-	-	4	10	21	-	-	-	
EVENTOUT	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	10	-	-	-	-	20	

1. Numbers in this table are links to corresponding subsections of [Section 15.3](#).

2. “-” means no interconnect.

3. Refer to the product datasheet for the availability of peripherals. If the peripheral is not present, consider the associated interconnection as unavailable.

15.3 Interconnection details

15.3.1 Master to slave interconnection for timers

From timer (TIM1/2/3/4/5/8/12/13/14/15/16/17) to timer (TIM1/2/3/4/5/8/12/15).

Purpose

Some timers are linked together internally for synchronization or chaining.

When one timer is configured in master mode, it can reset, start, stop, or clock the counter of another timer configured in slave mode.

A description of the feature is provided in [Section 44.4.23: Timer synchronization](#).

The synchronization modes are detailed in:

- [Section 43.3.30](#) for advanced-control timers TIM1/8
- [Section 44.4.22](#) for general-purpose timers TIM2/3/4/5
- [Section 46](#) for the general-purpose timers TIM12/13/14
- [Section 47.4.23](#) for the general-purpose timer TIM15

Triggering signals

The output (from master) is on signal TIMx_TRGO (and TIMx_TRGO2 for TIM1/8) following a configurable timer event. It can be also from signals tim16_oc1 and tim17_oc1 in case of TIM16/17. The input (to slave) is on signals TIMx_ITR0/1/2/3.

The possible master/slave connections are given in:

- [Table 456](#) for advanced-control timers TIM1/8
- [Table 480](#) for general-purpose timers TIM2/3/4/5
- [Table 502](#) for the general-purpose timers TIM12/13/14
- [Table 514](#) for the general-purpose timer TIM15

Active power mode

Timers are optionally active in Run and Sleep modes. The effects of low-power modes on TIMx are given in:

- [Table 469: Effect of low-power modes on TIM1/TIM8](#)
- [Table 488: Effect of low-power modes on TIM2/TIM3/TIM4/TIM5](#)
- [Table 504: Effect of low-power modes on TIM12/TIM13/TIM14](#)
- [Table 521: Effect of low-power modes on TIM15/TIM16/TIM17](#)

15.3.2 Triggers to ADCs

From EXTI, timers (TIM1/2/3/4/6/8/15), low-power timers (LPTIM1/2), and PLAY to ADC1/ADC2/ADC3.

Purpose

A conversion (or a sequence of conversions) can be triggered either by software or by an external event (such as timer capture or input pins). For ADC12, if the EXTEN[1:0] (for a regular conversion) or JEXTEN[1:0] bits (for an injected conversion) are different from 0b00, external events can trigger a conversion with the selected polarity.

More details in:

- [Section 28.4.18: Conversion on external trigger and trigger polarity \(EXTSEL, EXTEN, JEXTSEL, JEXTEN\)](#)
- EXTEN[1:0] defined in [ADC configuration register \(ADC_CFGR\)](#)
- JEXTEN[1:0] defined in [ADC injected sequence register \(ADC_JSQR\)](#)

General-purpose timers (TIM2/3/4), basic timer (TIM6), advanced-control timers (TIM1/8), and general-purpose timer (TIM15) can be used to generate the ADC triggering event through the timer outputs tim_oc and tim_trgo.

Low-power timers (LPTIM1/2) can be used to generate the ADC triggering event through the LPTIM channels, in addition to the EXTI on channels 11 and 15.

PLAY can be used to generate the ADC triggering event through the PLAY outputs 7 and 9.

Triggering signals

For ADC1/ADC2/ADC3, the input triggering signals and the description of the interconnection between ADC1/ADC2/ADC3 and timers are given in:

- adc_ext_trg: [Table 282: ADC interconnection](#)
- adc_jext_trg: [Table 282: ADC interconnection](#)
- [Section 28.4.18: Conversion on external trigger and trigger polarity \(EXTSEL, EXTEN, JEXTSEL, JEXTEN\)](#)
- [Section 28.4.25: Timing diagrams example \(single/continuous modes, hardware/software triggers\)](#)

Active power mode

This interconnection is active in Run and Sleep modes for all ADCs. The timers are active only in Run and Sleep modes. The effects of low-power modes are given in:

- [Table 469: Effect of low-power modes on TIM1/TIM8](#)
- [Table 488: Effect of low-power modes on TIM2/TIM3/TIM4/TIM5](#)
- [Table 521: Effect of low-power modes on TIM15/TIM16/TIM17](#)
- [Table 527: STM32H5 LPTIM features](#)
- [Table 542: Effect of low-power modes on the LPTIM](#)

15.3.3 ADC analog watchdogs as triggers to timers and PLAY

From ADC1/ADC2/ADC3 to TIM1/8 and from ADC1/ADC2 to PLAY.

Purpose

The internal analog watchdog output signals coming from ADC1/ADC2/ADC3 are connected to on-chip timers. ADC1/ADC2/ADC3 can provide trigger events through analog watchdog signals to advanced-control timers (TIM1/8) to reset, start, stop, or clock the counter.

Settings description of the ADC analog watchdog and timer trigger, are provided in:

- [Section 43.3.6: External trigger input](#) for TIM1/8
- [Table 457](#) for the internal ADC1/ADC2 sources connected to TIM1/8 (tim_etr) input multiplexer
- [Section 28.4.28](#) for the ADC1/ADC2/ADC_AWDy_OUT signal output generation

Triggering signals

The output (from ADC) is on signals ADCn_AWDx_OUT, with n being the ADC instance and x = 1, 2, 3 (three watchdogs per ADC). The input (to timer) is on signal TIMx_ETR (external trigger).

The inputs (to PLAY) are on signal PLAY1_IN[15:0].

Active power mode

ADC1/ADC2/ADC3 are active in Run and Sleep modes.

15.3.4 Triggers to DAC

From timer (TIM1/2/4/5/6/7/8/15), low-power timers (LPTIM1/2), COMP1/2, and EXTI to DAC.

Purpose

General-purpose timers (TIM2/4/5/15), basic timers (TIM6/7), advanced-control timers (TIM1/8), low-power timers (LPTIM1/2) outputs channels (lptim1_ch1 and lptim2_ch1), comparators (COMP1/2) output channels (comp1_out and comp2_out), and EXTI can be used as triggering event to start a DAC conversion.

Triggering signals

The output (from timer) on the TIMx_TRGO signal and from low-power timers are directly connected to corresponding DAC inputs.

The selection of input triggers on DAC is provided in:

- [Table 306: DAC interconnection](#)
- [Section 30.4.8: DAC trigger selection](#)

Active power mode

This interconnect is active in Run, Sleep, and Stop modes.

15.3.5 Clock sources to timers

From HSE, LSE, LSI, HSI, and MCO to timers (TIM2/12/15/16/17) and LP timers (LPTIM1/2).

Purpose

A timer input or counter can receive different clock sources, and can be used, for example, to calibrate the internal oscillator on a reference clock.

External clocks (HSE, LSE), internal clocks (LSI, CSI, HSI), and microcontroller output clock (MCO) can be used as input to timers:

- LSE, HSI, and CSI are assigned to general-purpose timers TIM2/12 as external inputs signals. LSE can be selected as counter clock provided by an external clock source in

mode1 (tim_ti1_in) and mode2 (external trigger input tim_etr_in). Inputs assignment and clock selection description are detailed in:

- [Section 44.4.5: Clock selection](#) for TIM2
- External clock mode1: [Table 500: Interconnect to the tim_ti1 input multiplexer](#) for TIM12, tim_ti1_in4 (HSI), and tim_ti1_in5 (CSI)
- External clock mode2: [Table 481: Interconnect to the tim_etr input multiplexer](#) for tim_etr3 (LSE)
- LSE, LSI, CSI, and HSE are assigned to general-purpose timers TIM15/16/17 as external inputs signals. LSE/LSI/CSI/HSE can be selected as counter clock provided by an external clock source in mode1 (tim_ti1 or tim_ti2 signals). Inputs assignment and clock selection description are detailed in:
 - [Section 47.4.6: Clock selection](#) for TIM15/16/17. External clock mode1: external input pin (tim_ti1 or tim_ti2, if available)
 - [Table 452: Interconnect to the tim_ti1 input multiplexer](#), tim_ti1_in1 (LSI-TIM16), tim_ti1_in2 (LSE-TIM16/HSE-TIM17), tim_ti1_in4 (LSE-TIM15), and tim_ti1_in5 (CSI-TIM15)
- Microcontroller output clock (MCO): MCO1 is connected as external input to general-purpose timer TIM17, MCO2 is connected as external input to general-purpose timer TIM15.
 - [Table 512: Interconnect to the tim_ti1 input multiplexer](#) for TIM15/TIM16/TIM17
- LSI and LSE can be selected as input capture 2 to LPTIM1 as described in [Table 536: LPTIM1 input capture 2 connections](#).
- HSI/1024, CSI/128, and HSI/8 can be selected as input capture 2 to LPTIM2 as described in [Table 537: LPTIM2 input capture 2 connections](#).

Triggering signals

lptim_ic2_mux1 LPTIM input capture selection can be set in the LPTIM configuration register 2 (LPTIM_CFGR2). For timers, the internal clock signal can be selected as counter clock provided by an external clock source in mode1 (tim_ti1_in) and mode2 (external trigger input tim_etr_in).

Active power mode

This feature is available under Run and Sleep modes.

15.3.6 Triggers to low-power timers

From EXTI, TAMP, PLAY, comparators (COMP1/2), GPDMA1, and RTC alarm to low-power timers (LPTIM1/2/3/4/5/6).

Purpose

LPTIM1/2/3/4/5/6 counters can be started either by software, or after the detection of an active edge on one of the eight trigger inputs (see [Section 48.4.7: Trigger multiplexer](#)).

RTC alarm, TAMP input detection, COMP1_OUT, and GPDMA transfer complete can be used as a trigger to start LPTIM counters (LPTIM1/2).

Triggering signals

This trigger feature is described in [Section 48.4.7: Trigger multiplexer](#) and the following sections. The input selection is described in [Table 532: LPTIM1/2/3/4/5/6 external trigger connections](#).

Active power mode

This interconnection is active in Run, Sleep, and Stop modes.

15.3.7 RTC wake-up as inputs to timers

From RTC to timer (TIM16).

Purpose

RTC wake-up interrupt can be used as input to general-purpose timer (TIM16) channel 1.

Triggering signals

RTC wake-up signal is connected to tim_ti1_in3 signal as described in [Table 512: Interconnect to the tim_ti1 input multiplexer](#) for TIM16.

Active power mode

This interconnection is active down to Stop mode. Timers are not active but the count is performed at wake-up.

15.3.8 System errors as break signals to timers

From system errors to timers (TIM1/8/15/16/17).

Purpose

CSS, CPU lockup, SRAM2/3 ECC double errors, SRAM1 parity errors, FLASH ECC double-error detection, and PVD can generate system errors in the form of timer break toward timers (TIM1/8/15/16/17).

The purpose of the break function is to protect power switches driven by PWM signals generated by the timers.

Triggering signals

The possible sources of break are described in:

- [Section 43.3.18: Using the break function](#) for TIM1/8
- [Section 47.4.15: Using the break function](#) for TIM15/16/17
- [Table 460: System break interconnect](#) for TIM1/8
- [Table 516: System break interconnect](#) for TIM15/16/17

Active power mode

Timers are optionally active in Run and Sleep modes. The effects of low-power modes on TIMx are given in:

- [Table 469: Effect of low-power modes on TIM1/TIM8](#)
- [Table 488: Effect of low-power modes on TIM2/TIM3/TIM4/TIM5](#)

- [Table 521: Effect of low-power modes on TIM15/TIM16/TIM17](#)

15.3.9 Triggers for communication peripherals

From Ethernet to timers (TIM2/TIM3).

Purpose

To synchronize system clock with network, internal connections are available between timers and PTP to check for clock drifts.

Triggering signals

- The outputs (from timer) are directly connected to Ethernet PTP triggers inputs. More details are given in the sections below:
- [Section 63.3: Ethernet pins and internal signals](#)
- [Table 476: Interconnect to the tim_ti1 input multiplexer](#)
- [Table 481: Interconnect to the tim_etr input multiplexer](#)

Active power mode

These interconnections remain active in Run and Sleep modes.

15.3.10 Triggers to GPDMA1/2

From EXTI, RTC (alarm/wake-up), TAMP, timers (TIM2/12/15), low-power timers (LPTIM1/2/3/4/5/6), comparators (COMP1/2), PLAY, EVENTOUT, GPDMA1 transfer complete (gpdma1_chx_tc, gpdma2_chx_tcf) to GPDMA1/2.

Purpose

A GPDMA trigger can be assigned to GPDMA channel x. A programmed GPDMA transfer can be triggered by a rising/falling edge of a selected input trigger event. The trigger mode can also be programmed to condition the LLI link transfer. More details are given in the sections below:

- [Section 16.3.7: GPDMA triggers](#)
- [Section 16.4.12: GPDMA triggered transfer](#)
- [GPDMA channel x transfer register 2 \(GPDMA_CxTR2\)](#) for more details on:
 - Trigger selection TRIGSEL[5:0] field
 - Trigger mode (LLI) defined by TRIGM[1:0]
 - Trigger polarity as defined by TRIGPOL[1:0]

Triggering signals

GPDMA trigger mapping is specified in [Table 146: Programmed GPDMA1/2 trigger](#), according to GPDMA_CxTR2.TRIGSEL[5:0].

Active power mode

This interconnection remains functional in Sleep mode.

Refer to [Section 16.6: GPDMA in low-power modes](#)

15.3.11 Internal analog signals to analog peripherals

From internal analog source to ADC (ADC1/ADC2/ADC3).

Purpose

The internal reference voltage (V_{REFINT}), the internal temperature sensor (V_{SENSE}), the internal digital core voltage (V_{DDCORE}), and the V_{BAT} monitoring channel are connected to ADC (ADC1/ADC2/ADC3) input channels.

This is according to:

- [Section 28.2: ADC main features](#)
- [Section 28.4.11: Channel selection \(ADC_SQRy, ADC_JSQR\)](#)

15.3.12 Clock source for the DAC sample and hold mode

From LSI/LSE to DAC1.

Purpose

DAC1 can run in Stop mode. The sample and hold block and its associated registers use the LSI or LSE clock source (dac_hold_ck) in Stop mode.

[Table 305: DAC internal input/output signals](#): dac_hold_ck, Input, DAC low-power clock used in sample and hold mode

Active power mode

This feature remains available in Run, Sleep and Stop modes.

15.3.13 Internal tamper sources

From internal peripherals, clocks, or monitoring, to tamper.

Purpose

To detect any abnormal activity or tentative to corrupt the device, embedded tamperers alert the system of undesired events. Different actions can be taken as consequence.

The list of tamper sources can be found in [Table 569: TAMP interconnection](#).

Active power mode

This interconnection is active in all power modes if the tamper source is activated.

15.3.14 Output from tamper to RTC

From TAMP to RTC.

Purpose

The RTC can timestamp a tamper event to retrieve history of the detection. The RTC can also control GPIOs, and set a signal based on tamp or alarm status outside the MCU.

Refer to [Section 51.3.3: GPIOs controlled by the RTC and TAMP](#) for more details.

Active power mode

This interconnection remains active in all power modes.

15.3.15 Encryption keys to AES/SAES

From TAMP backup registers, system flash memory to and in between SAES and AES.

Purpose

The encryption mechanism requires an hardware key that must be stored in a protected nonvolatile memory. Different approaches are implemented to load them in a non-readable way. Tamper backup registers or system flash memory can be used to store respectively BHK or RHUK, and to implement a dedicated bus to pass it to the SAES.

Refer to [Section 38.4.14: SAES operation with wrapped keys](#) for more details.

The AES encryption mechanism (faster than the SAES) can benefit from the sharing key of the SAES. Refer to [Section 38.4.15: SAES operation with shared keys](#) for more details.

Active power mode

AES and SAES are operating under Run and Sleep modes.

15.3.16 Comparators as inputs, trigger, or break signals to timers1

From comparators to timers (TIM1/2/3/4/5/8/12/15).

Purpose

The comparators (COMP1/2) output values can be connected to timers (TIM1/2/3/4/5/8/12/15) input captures, TIMx_ETR, or timer break signals. The connection to ETR is described in [Section 43.3.6: External trigger input](#).

Comparators (COMP1/2) output values can also generate break input signals for timers (such as TIM1 or TIM8).

The sources for the break (tim_brk) channel are one of the following:

- External: connected to one of the TIMx_BKIN pin (as per selection done in the AFIO controller) with polarity selection and optional digital filtering.
- Internal: coming from comparators, tim_brk_cmpx input (refer to [Section 43.3.2: TIM1/TIM8 pins and internal signals](#) for product specific implementation).

Triggering signals

The tim_etr and tim_brk signals connected TIM1/8 (coming from COMP1/2) are given in:

- tim_etr: external trigger internal input bus. These inputs can be used as trigger, external clock or for hardware cycle-by-cycle pulse width control.
- tim_brk ([Table 458: Timer break interconnect](#) and [Table 459: Timer break2 interconnect](#))
- [Section 43.3.6: External trigger input](#)
- [Section 43.3.18: Using the break function](#)

For TIM2/3/4/5, the sources connected to the tim_ti[1:4] input multiplexers coming from comparators and some other peripherals, are given in:

- [Table 476: Interconnect to the tim_ti1 input multiplexer](#)
- [Table 477: Interconnect to the tim_ti2 input multiplexer](#)
- [Table 478: Interconnect to the tim_ti3 input multiplexer](#)
- [Table 479: Interconnect to the tim_ti4 input multiplexer](#)
- [Table 481: Interconnect to the tim_etr input multiplexer](#)

For TIM12, the sources connected to timers coming from comparators and other peripherals, are given in:

- [Table 500: Interconnect to the tim_ti1 input multiplexer](#)
- [Table 500: Interconnect to the tim_ti1 input multiplexer](#)

For TIM15, the sources connected to timers coming from comparators and other peripherals are given in:

- [Table 512: Interconnect to the tim_ti1 input multiplexer](#)
- [Table 513: Interconnect to the tim_ti2 input multiplexer](#)
- [Table 524: TIM15 register map and reset values](#)

Active power mode

Run, Sleep, and wake-up capability in Stop 0, Stop 1, and Stop 2 modes for trigger sources.

Input and break remain active in the same low-power modes as timer activities, specifically during Run and Sleep modes.

15.3.17 Blanking sources to comparators

From timers (TIM1/2/3/8/15) and low-power timers (LPTIM1/2) to comparators (COMP1/2).

Purpose

Advanced-control timers (TIM1/8), general-purpose timers (TIM2/3/15), and low-power timers (LPTIM1/2) can be used as blanking window input to COMP1/2.

The blanking function is described in [Section 32.3.7: Comparator output blanking function](#).

The blanking sources are given in:

- COMP1 control and status register
- COMP2 control and status register

Triggering signals

Timer output signal TIMx_Ocx are the inputs to the blanking source of COMP1/2.

The low-power timer output signal lptimx_ch2 are the input to the blanking source of COMP1.

Active power mode

This feature is available under Run and Sleep modes.

15.3.18 Play as input to timers

From PLAY to TIM1/TIM2/TIM3/TIM8.

Purpose

The PLAY outputs can be connected to the advanced-control timers (TIM1/8) and general-purpose timers (TIM2/3).

Triggering signals

Timer input signals of the timers are connected to the output signals of the PLAY `play_out[3:0]`.

More details for connections are given in:

- [Table 183: PLAY logic array input *n* signal selection](#)
- [Table 452: Interconnect to the `tim\_ti1` input multiplexer](#)
- [Table 476: Interconnect to the `tim\_ti1` input multiplexer](#)
- [Table 458: Timer break interconnect](#)

Active power mode

These interconnections remain active in Run and Sleep modes.

15.3.19 Clock source to PLAY

MCO1/2 to PLAY

Purpose

Microcontroller output clock (MCO): MCO1/2 can be used as an input to the programmable logic array PLAY.

Triggering signals

MCO1/2 can be connected to the PLAY internal inputs `play_in[15:0]`.

The inputs connections are detailed in [Table 183: PLAY logic array input *n* signal selection](#).

Active power mode

This feature is available under Run and Sleep modes.

15.3.20 Inputs to PLAY

From timers(TIM2/3), low-power timers (LPTIM1/2), comparators (COMP1/2), USART1/2, SPI1/2, and CPU to PLAY.

Purpose

General-purpose timers (TIM2/3), low-power timers (LPTIM1/2), comparators (COMP1/2), USART1/2, SPI1/2, and CPU are connected to PLAY1 inputs.

Triggering signals

The PLAY inputs `play_in[15:0]` are connected to the timers (TIM2/3), low-power timers (LPTIM1/2), comparators (COMP1/2), USART1/2, SPI1/2, and CPU.

The input connections are detailed in [Table 183: PLAY logic array input n signal selection](#).

Active power mode

This feature is available under Run and Sleep modes. It is also available under Stop mode for the low-power timers (LPTIM1/2), the comparators (COMP1/2), USART1/2, SPI1/2, except for the general-purpose timers (TIM2/3), and CPU.

15.3.21 Triggers to DTS

From EXTI, low-power timers (LPTIM1/2) and PLAY to DTS.

Purpose

A temperature measurement can be triggered either by software, by low-power timers, by PLAY, or by an external event.

Triggering signals

The input triggering signals and the description of the interconnection between DTS, PLAY and low-power timers, are given in [Table 299: Trigger configuration](#).

16 General purpose direct memory access controller (GPDMA)

16.1 GPDMA introduction

The general purpose direct memory access (GPDMA) controller is a bus master and system peripheral.

The GPDMA is used to perform programmable data transfers between memory-mapped peripherals and/or memories via linked-lists, upon the control of an off-loaded CPU.

16.2 GPDMA main features

- Dual bidirectional AHB master
 - Memory-mapped data transfers from a source to a destination:
 - Peripheral-to-memory
 - Memory-to-peripheral
 - Memory-to-memory
 - Peripheral-to-peripheral
- Transfer arbitration based on a 4-grade programmed priority at channel level:
- One high-priority traffic class, for time-sensitive channels (queue 3)
 - Three low-priority traffic classes, with a weighted round-robin allocation for non time-sensitive channels (queues 0, 1, 2)
- Per channel event generation, on any of the following events: transfer complete, half-transfer complete, data transfer error, user setting error, link transfer error, completed suspension, and trigger overrun
 - Per channel interrupt generation, with separately programmed interrupt enable per event
 - 8 concurrent GPDMA channels:
 - Per channel FIFO for queuing source and destination transfers (see [Section 16.3.2](#))
 - Intra-channel GPDMA transfers chaining via programmable linked-list into memory, supporting two execution modes: run-to-completion and link step mode
 - Intra-channel and inter-channel GPDMA transfers chaining via programmable GPDMA input triggers connection to GPDMA task completion events
 - Per linked-list item within a channel:
 - Separately programmed source and destination transfers
 - Programmable data handling between source and destination: byte-based reordering, packing or unpacking, padding or truncation, sign extension and left/right realignment
 - Programmable number of data bytes to be transferred from the source, defining the block level
 - Linear source and destination addressing: either fixed or contiguously incremented addressing, programmed at a block level, between successive burst transfers

- 2D source and destination addressing: programmable signed address offsets between successive burst transfers (non-contiguous addressing within a block, combined with programmable signed address offsets between successive blocks, at a second 2D/repeated block level, for a reduced set of channels (see [Section 16.3.2](#))
- Support for scatter-gather (multi-buffer transfers), data interleaving and deinterleaving via 2D addressing
- Programmable GPDMA request and trigger selection
- Programmable GPDMA half-transfer and transfer complete events generation
- Pointer to the next linked-list item and its data structure in memory, with automatic update of the GPDMA linked-list control registers
- Channel abort and restart
- Debug:
 - Channel suspend and resume support
 - Channel status reporting, including FIFO level, and event flags
- TrustZone support:
 - Support for secure and nonsecure GPDMA transfers, independently at a first channel level, and independently at a source/destination and link sublevels
 - Secure and nonsecure interrupts reporting, resulting from any of the respectively secure and nonsecure channels
 - TrustZone-aware AHB slave port, protecting any GPDMA secure resource (register, register field) from a nonsecure access
- Privileged/unprivileged support:
 - Support for privileged and unprivileged GPDMA transfers, independently at a channel level
 - Privileged-aware AHB slave port

16.3 GPDMA implementation

16.3.1 GPDMA instances

There are two instances of the GPDMA in the devices, named as GPDMA1 and GPDMA2.

Each GPDMA instance has the same channel-based implementation and is connected to the same requests and triggers, as detailed in the following sub-sections.

16.3.2 GPDMA channels

A given GPDMA channel *x* is implemented with the following features and intended usage. To make the best use of the GPDMA performances, the table below lists some general recommendations, allowing the user to select and allocate a channel, given its implemented FIFO size and the requested GPDMA transfer.

Table 140. GPDMA1/2 channel implementation

Channel x	Hardware parameters		Features
	dma_fifo_size[x]	dma_addressing[x]	
x = 0 to 3	2	0	Channel x (x = 0 to 3) is implemented with: – a FIFO of 8 bytes, 2 words – fixed/contiguously incremented addressing These channels must be typically allocated for GPDMA transfers between an APB or AHB peripheral and SRAM.
x = 4 to 5	4	0	Channel x (x = 4 to 5) is implemented with: – a FIFO of 32 bytes, 8 words – fixed/contiguously incremented addressing These channels may be used for GPDMA transfers between a demanding AHB peripheral and SRAM, or for transfers from/to external memories.
x = 6 to 7	4	1	Channel x (x = 6 to 7) is implemented with: – a FIFO of 32 bytes, 8 words – 2D addressing These channels may be used for GPDMA transfers between a demanding AHB peripheral and SRAM, or for transfers from/to external memories.

16.3.3 GPDMA in low-power modes

The GPDMA wake-up feature is implemented in the device low-power modes as per the table below.

Table 141. GPDMA1/2 wake-up in low-power modes

Feature	Low-power modes
Wake-up	GPDMA1/2 in Sleep mode

16.3.4 GPDMA requests

A GPDMA request from a peripheral can be assigned to a GPDMA channel x, via REQSEL[7:0] in GPDMA_CxTR2, provided that SWREQ = 0.

The GPDMA requests mapping is specified in the table below.

Table 142. Programmed GPDMA1/2 request

GPDMA_CxTR2.REQSEL[7:0]	Selected GPDMA request
0	adc1_dma
1	adc2_dma
2	dac1_ch1_dma
3	dac1_ch2_dma
4	tim6_upd_dma

Table 142. Programmed GPDMA1/2 request (continued)

GPDMA_CxTR2.REQSEL[7:0]	Selected GPDMA request
5	tim7_upd_dma
6	spi1_rx_dma
7	spi1_tx_dma
8	spi2_rx_dma
9	spi2_tx_dma
10	spi3_rx_dma
11	spi3_tx_dma
12	i2c1_rx_dma
13	i2c1_tx_dma
14	Reserved
15	i2c2_rx_dma
16	i2c2_tx_dma
17	Reserved
18	i2c3_rx_dma
19	i2c3_tx_dma
20	Reserved
21	usart1_rx_dma
22	usart1_tx_dma
23	usart2_rx_dma
24	usart2_tx_dma
25	usart3_rx_dma
26	usart3_tx_dma
27	uart4_rx_dma
28	uart4_tx_dma
29	uart5_rx_dma
30	uart5_tx_dma
31	usart6_rx_dma
32	usart6_tx_dma
33	uart7_rx_dma ⁽¹⁾
34	uart7_tx_dma ⁽¹⁾
35	uart8_rx_dma ⁽¹⁾
36	uart8_tx_dma ⁽¹⁾
37	uart9_rx_dma ⁽²⁾
38	uart9_tx_dma ⁽²⁾
39	uart10_rx_dma ⁽²⁾

Table 142. Programmed GPDMA1/2 request (continued)

GPDMA_CxTR2.REQSEL[7:0]	Selected GPDMA request
40	uart10_tx_dma ⁽²⁾
41	uart11_rx_dma ⁽²⁾
42	uart11_tx_dma ⁽²⁾
43	uart12_rx_dma ⁽²⁾
44	uart12_tx_dma ⁽²⁾
45	lpuart1_rx_dma
46	lpuart1_tx_dma
47	spi4_rx_dma
48	spi4_tx_dma
49	spi5_rx_dma ⁽²⁾
50	spi5_tx_dma ⁽²⁾
51	spi6_rx_dma ⁽²⁾
52	spi6_tx_dma ⁽²⁾
53	sai1_a_dma ⁽²⁾
54	sai1_b_dma ⁽²⁾
55	sai2_a_dma ⁽²⁾
56	sai2_b_dma ⁽²⁾
57	ospi1_dma
58	tim1_cc1_dma
59	tim1_cc2_dma
60	tim1_cc3_dma
61	tim1_cc4_dma
62	tim1_upd_dma
63	tim1_trg_dma
64	tim1_com_dma
65	tim8_cc1_dma
66	tim8_cc2_dma
67	tim8_cc3_dma
68	tim8_cc4_dma
69	tim8_upd_dma
70	tim8_tig_dma
71	tim8_com_dma
72	tim2_cc1_dma
73	tim2_cc2_dma
74	tim2_cc3_dma

Table 142. Programmed GPDMA1/2 request (continued)

GPDMA_CxTR2.REQSEL[7:0]	Selected GPDMA request
75	tim2_cc4_dma
76	tim2_upd_dma
77	tim3_cc1_dma
78	tim3_cc2_dma
79	tim3_cc3_dma
80	tim3_cc4_dma
81	tim3_upd_dma
82	tim3_trg_dma
83	tim4_cc1_dma
84	tim4_cc2_dma
85	tim4_cc3_dma
86	tim4_cc4_dma
87	tim4_upd_dma
88	tim5_cc1_dma
89	tim5_cc2_dma
90	tim5_cc3_dma
91	tim5_cc4_dma
92	tim5_upd_dma
93	tim5_trg_dma
94	tim15_cc1_dma
95	tim15_upd_dma
96	tim15_trg_dma
97	tim15_com_dma
98	tim16_cc1_dma ⁽²⁾
99	tim16_upd_dma ⁽²⁾
100	tim17_cc1_dma ⁽²⁾
101	tim17_upd_dma ⁽²⁾
102	lptim1_ic1_dma
103	lptim1_ic2_dma
104	lptim1_ue_dma
105	lptim2_ic1_dma
106	lptim2_ic2_dma
107	lptim2_ue_dma
108	dcmi_dma or pssi_dma ⁽³⁾⁽⁴⁾
109	aes_out_dma

Table 142. Programmed GPDMA1/2 request (continued)

GPDMA_CxTR2.REQSEL[7:0]	Selected GPDMA request
110	aes_in_dma
111	hash_in_dma
112	ucpd1_rx_dma
113	ucpd1_tx_dma
114	cordic_read_dma ⁽¹⁾
115	cordic_write_dma ⁽¹⁾
116	fmac_read_dma ⁽²⁾
117	fmac_write_dma ⁽²⁾
118	saes_out_dma
119	saes_in_dma
120	i3c1_rx_dma
121	i3c1_tx_dma
122	i3c1_tc_dma
123	i3c1_rs_dma
124	i2c4_rx_dma ⁽²⁾
125	i2c4_tx_dma ⁽²⁾
126	Reserved
127	lptim3_ic1_dma ⁽²⁾
128	lptim3_ic2_dma ⁽²⁾
129	lptim3_ue_dma ⁽²⁾
130	lptim5_ic1_dma ⁽²⁾
131	lptim5_ic2_dma ⁽²⁾
132	lptim5_ue_dma ⁽²⁾
133	lptim6_ic1_dma ⁽²⁾
134	lptim6_ic2_dma ⁽²⁾
135	lptim6_ue_dma ⁽²⁾
136	i3c2_rx_dma ⁽⁵⁾
137	i3c2_tx_dma ⁽⁵⁾
138	i3c2_tc_dma ⁽⁵⁾
139	i3c2_rs_dma ⁽⁵⁾
140	Reserved
141	Reserved
142	adc3_dma ⁽⁶⁾

1. Not available in STM32H523/533xx devices.

2. Not available in STM32H523/533xx and STM32H543/553xx devices.

3. Depends on which exclusive function is used.
4. Not available in STM32H543/553xx devices.
5. Not available in STM32H562/563/573xx devices.
6. Available only on STM32H543/553xx devices.

16.3.5 GPDMA block requests

Some GPDMA requests must be programmed as a block request, and not as a burst request. Then BREQ in GPDMA_CxTR2 must be set for a correct GPDMA execution of the requested peripheral transfer at the hardware level.

Table 143. Programmed GPDMA1/2 request as a block request

GPDMA block requests
lptim1_ue_dma
lptim2_ue_dma
lptim3_ue_dma ⁽¹⁾
lptim5_ue_dma ⁽¹⁾
lptim6_ue_dma ⁽¹⁾

1. Not available in STM32H523/533 and STM32H543/553xx devices.

16.3.6 GPDMA channels with peripheral early termination

A GPDMA channel, if implemented with this feature, can support the early termination of the data transfer from the peripheral which does also support this feature.

Table 144. GPDMA1/2 channel with peripheral early termination

GPDMA channel x with peripheral early termination
x = 0 and x = 7

This GPDMA support is activated when the channel x is programmed with GPDMA_CxTR2.PFREQ = 1. Then, the peripheral itself can initiate and request a data transfer completion, before the GPDMA has transferred the whole block (see [Section 16.4.14](#) for more details).

Table 145. Programmed GPDMA1/2 request with peripheral early termination

Programmed GPDMA channel x request with peripheral early termination
i3c1_rx_dma
i3c2_rx_dma ⁽¹⁾

1. Not available in STM32H562/563/573 devices.

16.3.7 GPDMA triggers

A GPDMA trigger can be assigned to a GPDMA channel x, via TRIGSEL[5:0] in GPDMA_CxTR2, provided that TRIGPOL[1:0] defines a rising or a falling edge of the selected trigger (TRIGPOL[1:0] = 01 or TRIGPOL[1:0] = 10).

Table 146. Programmed GPDMA1/2 trigger

GPDMA_CxTR2.TRIGSEL[5:0]	Selected GPDMA trigger
0	exti0
1	exti1
2	exti2
3	exti3
4	exti4
5	exti5
6	exti6
7	exti7
8	tamp_trg1
9	tamp_trg2
10	tamp_trg3
11	lptim1_ch1
12	lptim1_ch2
13	lptim2_ch1
14	lptim2_ch2
15	rtc_alra_trg
16	rtc_alrb_trg
17	rtc_wut_trg
18	gpdma1_ch0_tc
19	gpdma1_ch1_tc
20	gpdma1_ch2_tc
21	gpdma1_ch3_tc
22	gpdma1_ch4_tc
23	gpdma1_ch5_tc
24	gpdma1_ch6_tc
25	gpdma1_ch7_tc
26	gpdma2_ch0_tc
27	gpdma2_ch1_tc
28	gpdma2_ch2_tc
29	gpdma2_ch3_tc
30	gpdma2_ch4_tc

Table 146. Programmed GPDMA1/2 trigger (continued)

GPDMA_CxTR2.TRIGSEL[5:0]	Selected GPDMA trigger
31	gpdma2_ch5_tc
32	gpdma2_ch6_tc
33	gpdma2_ch7_tc
34	tim2_trgo
35	tim15_trgo
36	tim12_trgo
37	lptim3_ch1 ⁽¹⁾
38	lptim3_ch2 ⁽¹⁾
39	lptim4_ait ⁽¹⁾
40	lptim5_ch1 ⁽¹⁾
41	lptim5_ch2 ⁽¹⁾
42	lptim6_ch1 ⁽¹⁾
43	lptim6_ch2 ⁽¹⁾
44	comp1_out ⁽²⁾
45	EVENTOUT ⁽³⁾
46	comp2_out ⁽²⁾
47	Reserved
48	Reserved
49	Reserved
50	Reserved
51	Reserved
52	Reserved
53	Reserved
54	Reserved
55	Reserved
56	Reserved
57	Reserved
58	Reserved
59	Reserved
60	Reserved
61	Reserved
62	Reserved
63	Reserved
64	Reserved
65	Reserved

Table 146. Programmed GPDMA1/2 trigger (continued)

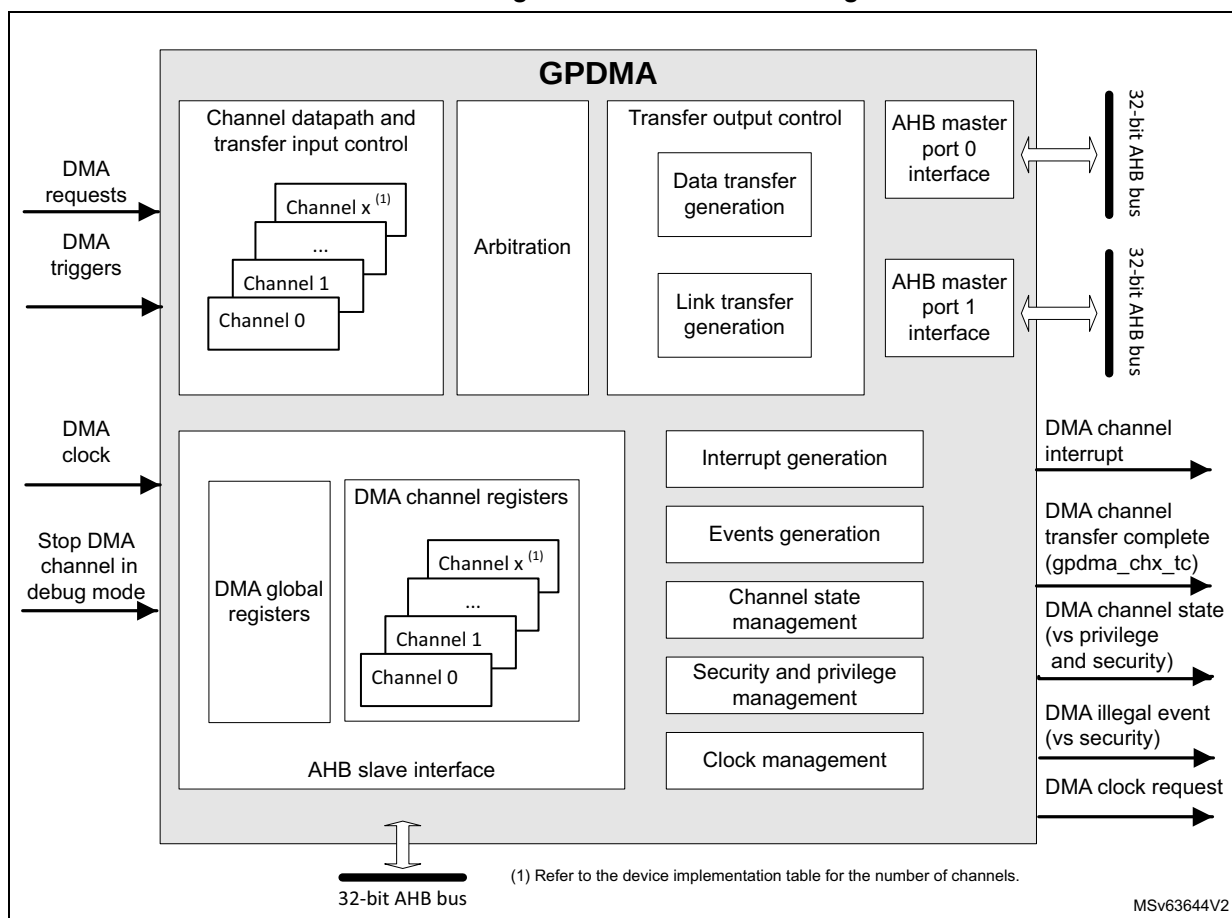
GPDMA_CxTR2.TRIGSEL[5:0]	Selected GPDMA trigger
66	Reserved
67	Reserved
68	Reserved
69	Reserved
70	Reserved
71	Reserved
72	play_out15 ⁽²⁾

1. Not available in STM32H523/533xx and STM32H543/553xx devices.
2. Available only on STM32H543/553xx devices.
3. Not available in STM32H562/563/573xx devices.

16.4 GPDMA functional description

16.4.1 GPDMA block diagram

Figure 75. GPDMA block diagram



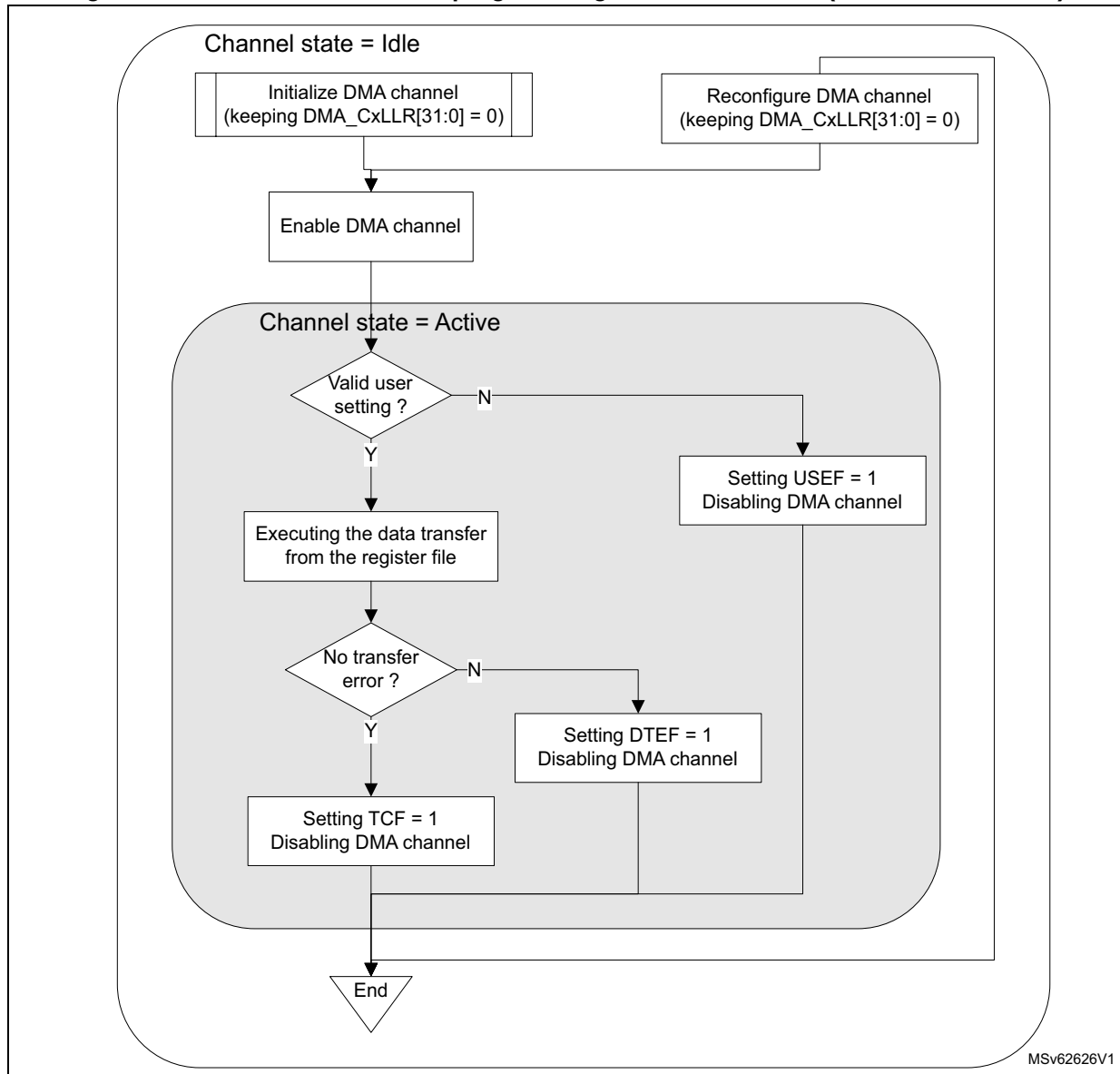
16.4.2 GPDMA channel state and direct programming without any linked-list

After a GPDMA reset, a GPDMA channel x is in idle state. When the software writes 1 in GPDMA_CxCR.EN, the channel takes into account the value of the different channel configuration registers (GPDMA_CxXXX), switches to the active/non-idle state and starts to execute the corresponding requested data transfers.

After enabling/starting a GPDMA channel transfer by writing 1 in GPDMA_CxCR.EN, a GPDMA channel interrupt on a complete transfer notifies the software that the GPDMA channel is back in idle state (EN is then deasserted by hardware) and that the channel is ready to be reconfigured then enabled again.

Figure 76 illustrates this GPDMA direct programming without any linked-list (GPDMA_CxLLR = 0).

Figure 76. GPDMA channel direct programming without linked-list (GPDMA_CxLLR = 0)



16.4.3 GPDMA channel suspend and resume

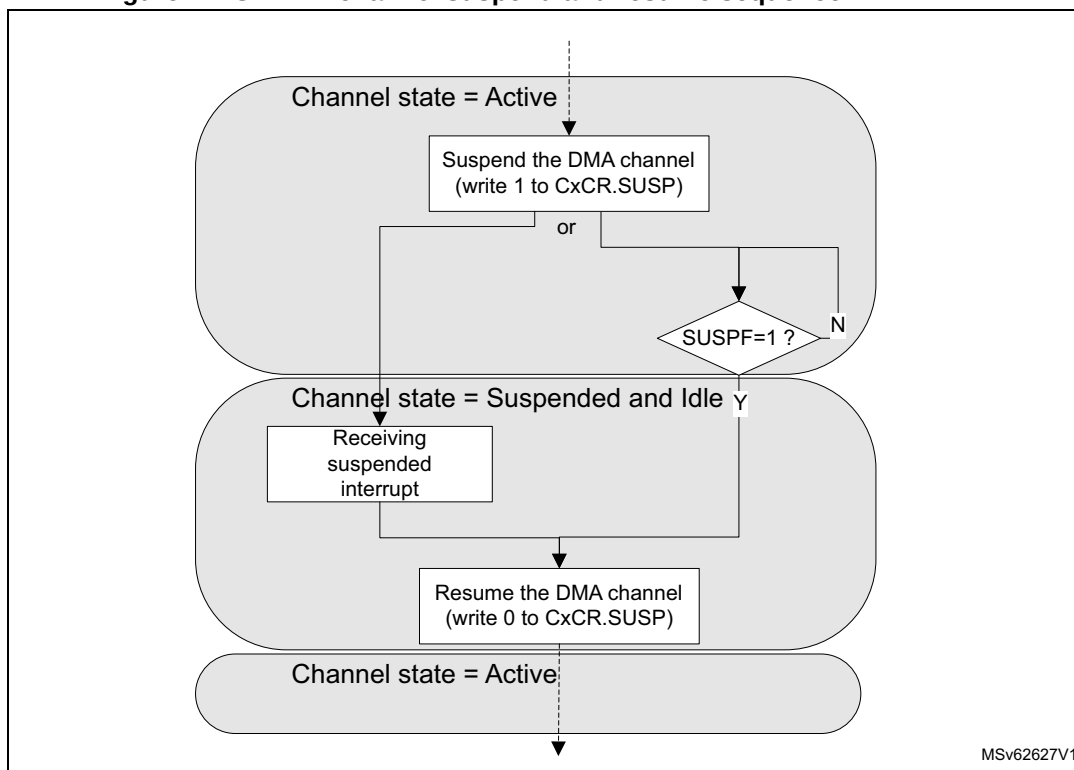
The software can suspend on its own a channel still active, with the following sequence:

1. The software writes 1 into the GPDMA_CxCR.SUSP bit.
2. The software polls the suspended flag GPDMA_CxSR.SUSPF until SUSPF = 1, or waits for an interrupt previously enabled by writing 1 to GPDMA_CxCR.SUSPIE. Wait for the channel to be effectively in suspended state means wait for the completion of any ongoing GPDMA transfer over its master ports. Then the software can observe, in a steady state, any read register or register field that is hardware modifiable.

Note: An ongoing GPDMA transfer can be a data transfer (a source/destination burst transfer) or a link transfer for the internal update of the linked-list register file from the next linked-list item.

3. The software safely resumes the suspended channel by writing 0 to GPDMA_CxCR.SUSP.

Figure 77. GPDMA channel suspend and resume sequence



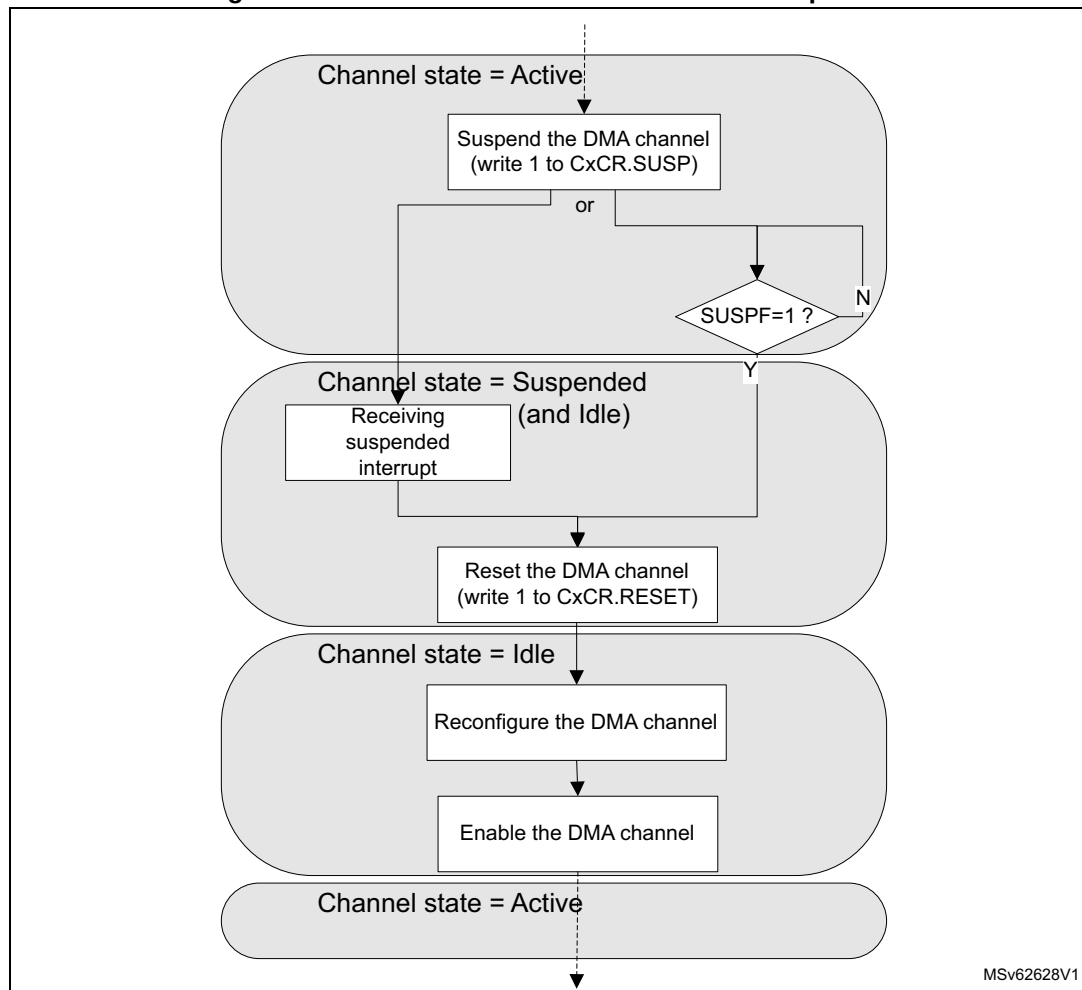
Note: A suspend and resume sequence does not impact the GPDMA_CxCR.EN bit. Suspending a channel (transfer) does not suspend a started trigger detection.

16.4.4 GPDMA channel abort and restart

Alternatively, like for aborting a continuous GPDMA transfer with a circular buffering or a double buffering, the software can abort, on its own, a still active channel with the following sequence:

1. The software writes 1 into the GPDMA_CxCR.SUSP bit.
2. The software polls suspended flag GPDMA_CxSR.SUSPF until SUSPF = 1, or waits for an interrupt previously enabled by writing 1 to GPDMA_CxCR.SUSPIE. Wait for the channel to be effectively in suspended state means wait for the completion of any ongoing GPDMA transfer over its master port.
3. The software resets the channel by writing 1 to GPDMA_CxCR.RESET. This causes the reset of the FIFO, the reset of the channel internal state, the reset of the GPDMA_CxCR.EN bit, and the reset of the GPDMA_CxCR.SUSP bit.
4. The software safely reconfigures the channel. The software must reprogram the hardware-modified GPDMA_CxBR1, GPDMA_CxSAR, and GPDMA_CxDAR registers.
5. In order to restart the aborted then reprogrammed channel, the software enables it again by writing 1 to the GPDMA_CxCR.EN bit.

Figure 78. GPDMA channel abort and restart sequence



16.4.5 GPDMA linked-list data structure

Alternatively to the direct programming mode, a channel can be programmed by a list of transfers, known as a list of linked-list items (LLI). Each LLI is defined by its data structure.

The base address in memory of the data structure of a next LLI_{n+1} of a channel x is the sum of the following:

- the link base address of the channel x (in `GPDMA_CxLBAR`)
- the link address offset (`LA[15:2]` field in `GPDMA_CxLLR`). The linked-list register `GPDMA_CxLLR` is the updated result from the data structure of the previous LLI of the channel x .

The data structure for each LLI may be specific.

A linked-list data structure is addressed following the value of the following `UT1`, `UT2`, `UB1`, `USA`, `UDA`, `UT3` and `UB2` if present, and `ULL` bits in `GPDMA_CxLLR`.

In linked-list mode, each GPDMA linked-list register (`GPDMA_CxTR1`, `GPDMA_CxTR2`, `GPDMA_CxBR1`, `GPDMA_CxSAR`, `GPDMA_CxDAR`, `GPDMA_CxTR3` and `GPDMA_CxBR2` if present, and `GPDMA_CxLLR`) is conditionally and automatically updated from the next linked-list data structure in the memory, following the current value of the

GPDMA_CxLLR register that was conditionally updated from the linked-list data structure of the previous LLI.

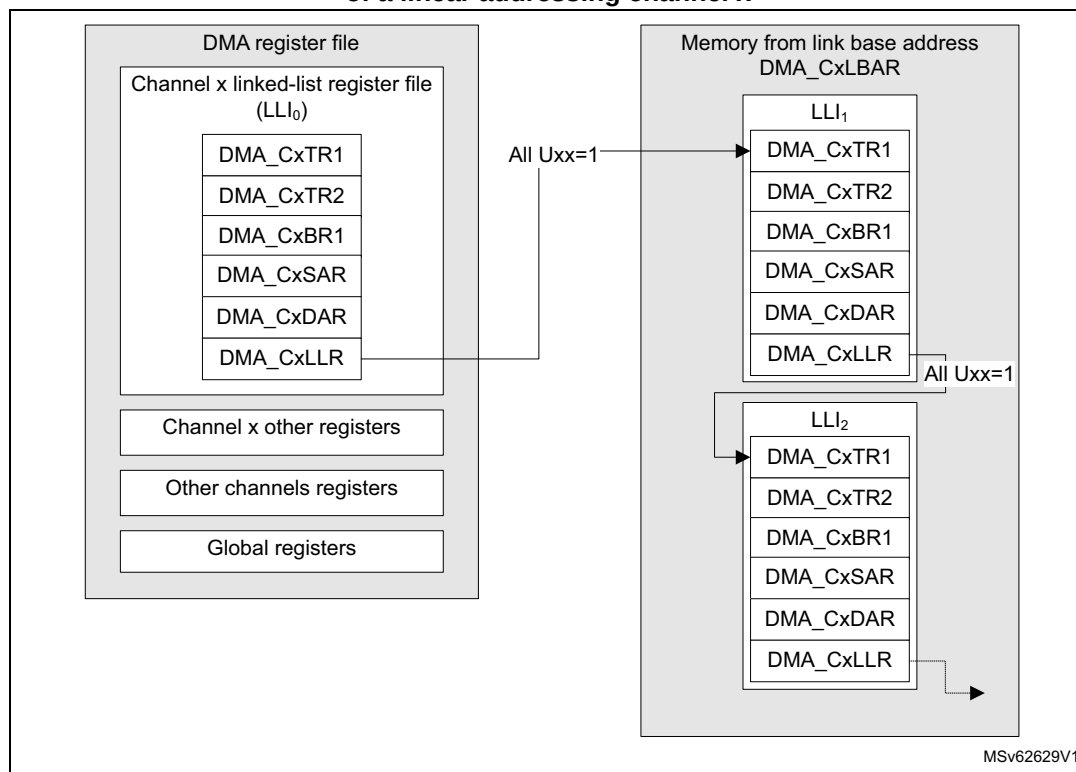
Caution: The user must program the pointer to the next linked-list data structure (GPDMA_CxLLR[15:0]) not to exceed the 64-Kbyte addressable space defined by the link base address register (GPDMA_CxLBAR). The complete linked-list data structure must be included in the 64-Kbyte addressable space.

Static linked-list data structure

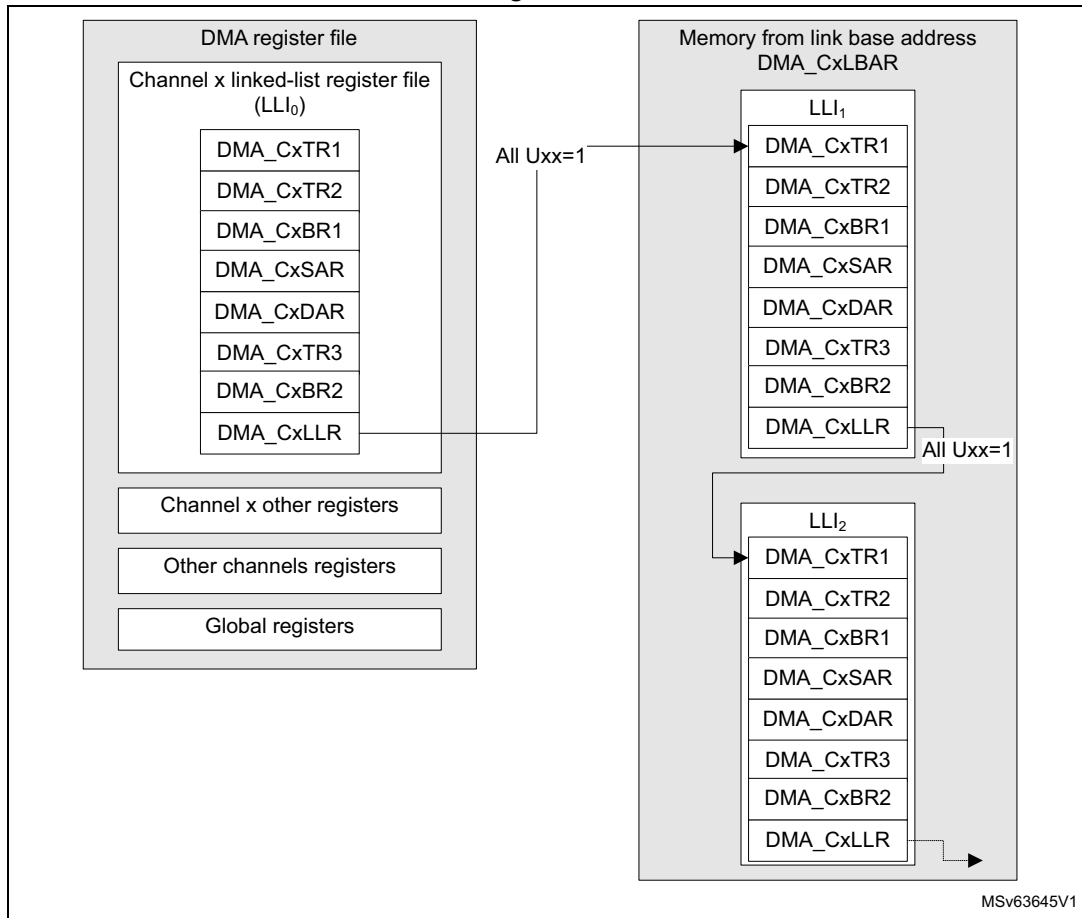
For example, when the update bits (UT1, UT2, UB1, USA, UDA, plus UB2 and UT3 if present, and ULL) in GPDMA_CxLLR are all asserted, the linked-list data structure in the memory is maximal with:

- channel x (x = 0 to 5) contiguous 32-bit locations, including GPDMA_CxTR1, GPDMA_CxTR2, GPDMA_CxBR1, GPDMA_CxSAR, GPDMA_CxDAR and GPDMA_CxLLR (see [Figure 79](#)) and including the first linked-list register file (LLI₀) and the next LLIs (such as LLI₁, LLI₂) in the memory
- channel x (x = 6 to 7), contiguous 32-bit locations, including GPDMA_CxTR1, GPDMA_CxTR2, GPDMA_CxBR1, GPDMA_CxSAR, GPDMA_CxDAR, GPDMA_CxTR3, GPDMA_CxBR2 and GPDMA_CxLLR, (see [Figure 80](#)), and including the first linked-list register file (LLI₀) and the next LLIs (such as LLI₁, LLI₂) in the memory

**Figure 79. Static linked-list data structure (all Uxx = 1)
of a linear addressing channel x**



**Figure 80. Static linked-list data structure (all Uxx = 1)
of a 2D addressing channel x**



Dynamic linked-list data structure

Alternatively, the memory organization for the full list of LLIs can be compacted with specific data structure for each LLI.

If $UT1 = 0$ and $UT2 = 1$, the link address offset of the register `GPDMA_CxLLR` is pointing to the updated value of the `GPDMA_CxTR2` instead of the `GPDMA_CxTR1` which is not to be modified (see [Figure 81](#)).

Example: if $UT1 = UB1 = USA = 0$ and if $UT3 = UB2 = 0$, when channel x is with 2D addressing, and if $UT2 = UDA = ULL = 1$, the next LLI does not contain an (updated) value for `GPDMA_CxTR1`, nor `GPDMA_CxBR1`, nor `GPDMA_CxSAR`, nor `GPDMA_CxTR3`, nor `GPDMA_CxBR2` when channel x is with 2D addressing. The next LLI contains an updated value for `GPDMA_CxTR2`, `GPDMA_CxDAR`, and `GPDMA_CxLLR`, as shown in [Figure 82](#).

Figure 81. GPDMA dynamic linked-list data structure of a linear addressing channel x

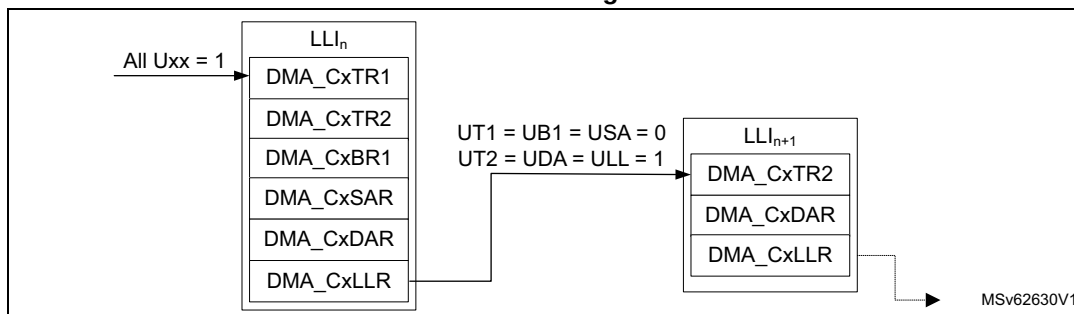
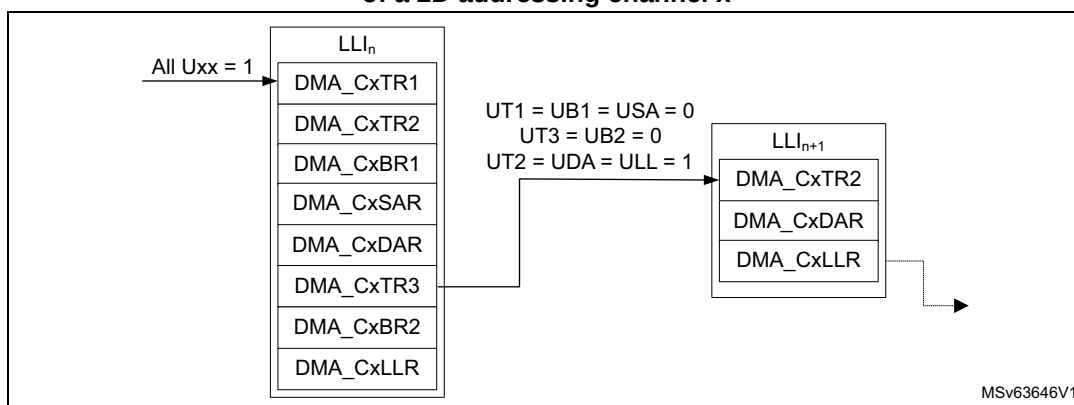


Figure 82. GPDMA dynamic linked-list data structure of a 2D addressing channel x



16.4.6 Linked-list item transfer execution

A LLI_n transfer is the sequence of:

1. a data transfer: GPDMA executes the data transfer as described by the GPDMA internal register file (this data transfer can be void/null for LLI_0)
2. a conditional link transfer: GPDMA automatically and conditionally updates its internal register file by the data structure of the next LLI_{n+1} , as defined by the GPDMA_CxLLR value of the LLI_n .

Note:

The initial data transfer as defined by the internal register file (LLI_0) can be null (GPDMA_CxBR1.BNDT[15:0] = 0 and GPDMA_CxTR2.PFREQ = 0) provided that the conditional update bit UB1 in GPDMA_CxLLR is set (meaning there is a non-null data transfer described by the next LLI_1 in the memory to be executed).

Depending on the intended GPDMA usage, a GPDMA channel x can be executed as described by the full linked-list (run-to-completion mode, GPDMA_CxCR.LSM = 0) or a GPDMA channel x can be programmed for a single execution of a LLI (link step mode, GPDMA_CxCR.LSM = 1), as described in the next sections.

16.4.7 GPDMA channel state and linked-list programming in run-to-completion mode

When GPDMA_CxCR.LSM = 0 (in full list execution mode, execution of the full sequence of LLIs, named run-to-completion mode), a GPDMA channel x is initially programmed, started

by writing 1 to GPDMA_CxCR.EN, and after completed at channel level. The channel transfer is:

- configured with at least the following:
 - the first LLI₀, internal linked-list register file: GPDMA_CxTR1, GPDMA_CxTR2, GPDMA_CxBR1, GPDMA_CxSAR, GPDMA_CxDAR, and GPDMA_CxLLR, plus GPDMA_CxTR3 and GPDMA_CxBR2
 - the last LLI_N, described by the linked-list data structure in memory, as defined by the GPDMA_CxLLR reflecting the before last LLI_{N-1}
- completed when GPDMA_CxLLR[31:0] = 0, GPDMA_CxBR1.BRC[10:0] = 0, and GPDMA_CxBR1.BNDT[15:0] = 0, at the end of the last LLI_{N-1} transfer

GPDMA_CxLLR[31:0] = 0 is the condition of a linked-list based channel completion and means the following:

- The 16 low significant bits GPDMA_CxLLR.LA[15:0] of the next link address are null.
- All the update bits GPDMA_CxLLR.Uxx are null (UT1, UT2, UB1, USA, UDA and ULL, plus UB2 and UT3).

The channel may never be completed when GPDMA_CxLLR.LSM = 0:

- If the last LLI_N is recursive, pointing to itself as a next LLI:
 - either GPDMA_CxLLR.ULL = 1 and GPDMA_CxLLR.LA[15:2] is updated with the same value
 - or GPDMA_CxLLR.ULL = 0
- If LLI_N is pointing to a previous LLI

In the regular data transfer completion at a block level, GPDMA_CxBR1.BNDT[15:0] = 0 and GPDMA_CxBR1.BRC[10:0] = 0 (if present). Alternatively, a block transfer may be early completed by a peripheral (such as an I3C in Rx mode), and then BNDT[15:0] is not null (see [Section 16.4.14](#) for more details).

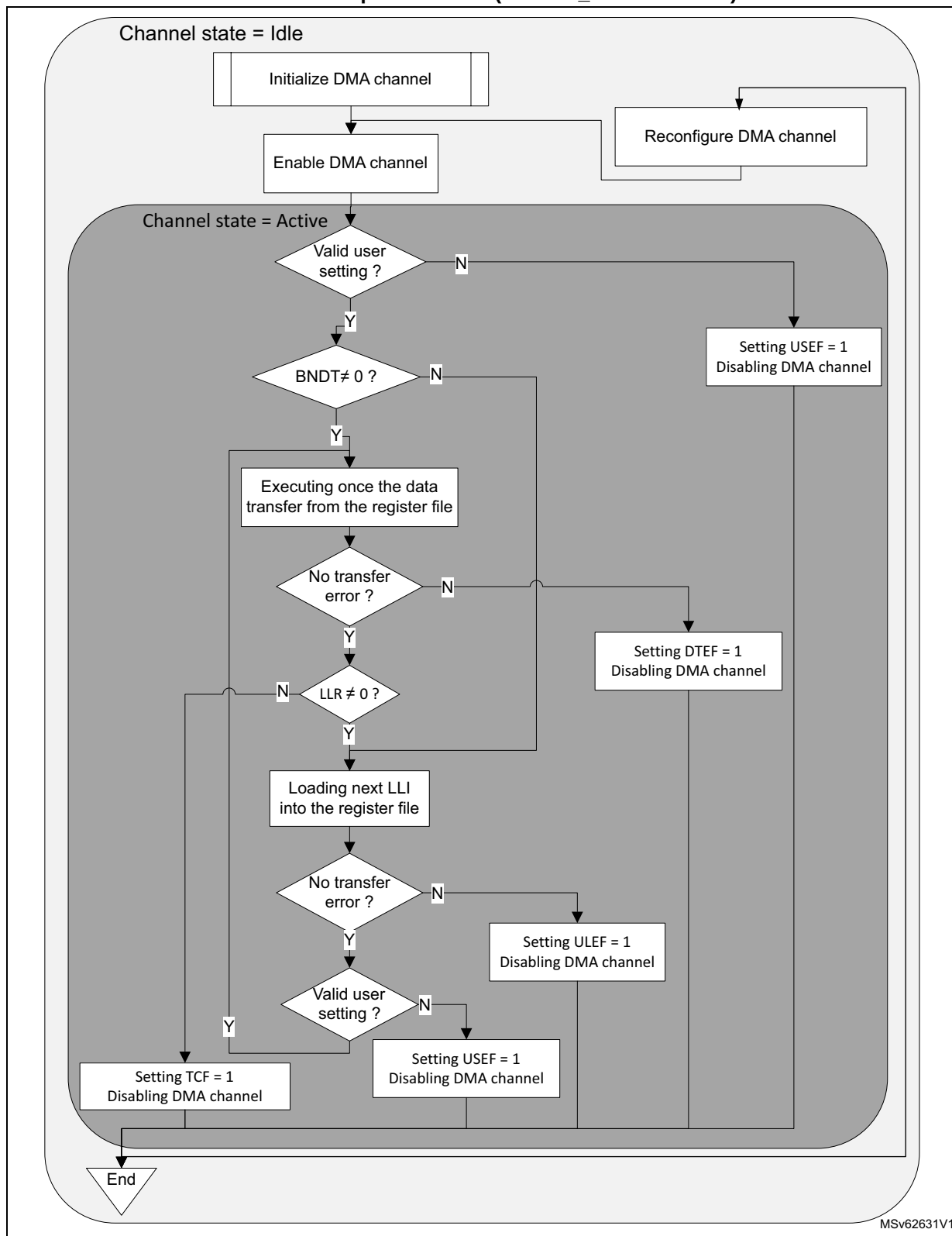
In typical run-to-completion mode, allocation of an GPDMA channel, including detailed programming, occurs once during GPDMA initialization. This process reserves a data communication link and GPDMA service during run-time for repeated transfers, such as transfers from a peripheral to memory, from memory to a peripheral, or memory-to-memory transfers. The reserved data communication link may consist of a dedicated channel, or the channel may be shared. A repeated transfer consists of a sequence of linked list items (LLIs).

[Figure 83](#) depicts the GPDMA channel execution and its registers programming in run-to-completion mode.

Note: [Figure 83](#) is not intended to illustrate how often a TCEF can be raised, depending on the programmed value of TCEM[1:0] in GPDMA_CxTR2. It can be raised at (each) block completion, at (each) 2D block completion, at (each) LLI completion, or only at channel completion. In run-to-completion mode, whatever is the value of TCEM[1:0], at the channel completion, the hardware always set TCEF = 1 and disables the channel.

In [Figure 83](#), BNDT ≠ 0 is the typical condition for starting the first data transfer in this figure. This condition becomes (BNDT ≠ 0 and PFREQ = 1) if the peripheral requests a data transfer with early termination (see [Section 16.3.6](#)).

Figure 83. GPDMA channel execution and linked-list programming in run-to-completion mode (GPDMA_CxCR.LSM = 0)



Run-time inserting a LLI_n via an auxiliary channel, in run-to-completion mode

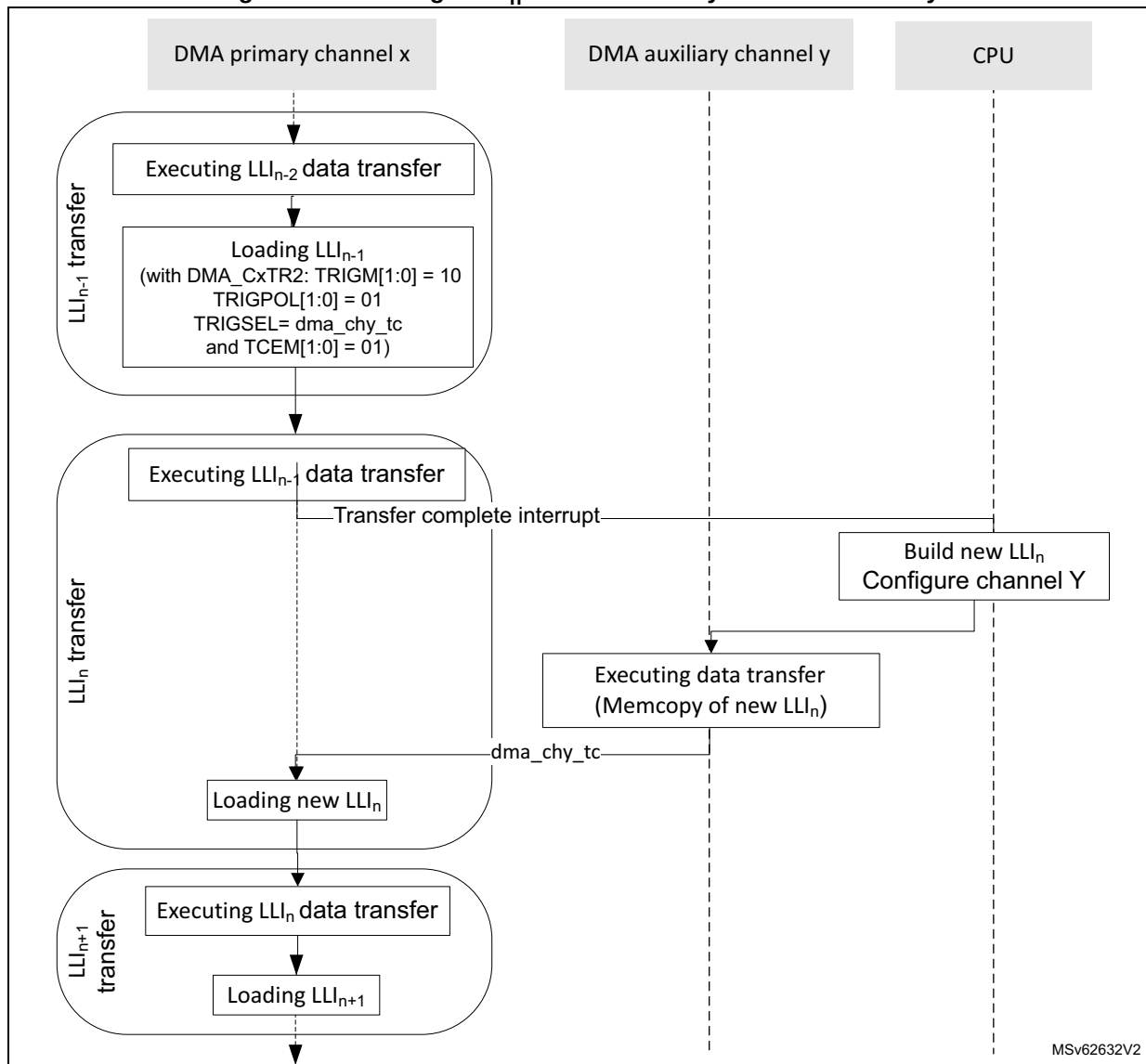
The start of the link transfer of the LLI_{n-1} (start of the LLI_n loading) can be conditioned by the occurrence of a trigger, when programming the following fields of the `GPDMA_CxTR2` in the data structure of the LLI_{n-1} :

- `TRIGM[1:0] = 10` (link transfer triggering mode)
- `TRIGPOL[1:0] = 01` or `10` (rising or falling edge)
- `TRIGSEL[5:0]` (see [Section 16.3.7](#) for the trigger selection details)

Another auxiliary channel y can be used to store the channel x LLI_n in the memory and to generate a transfer complete event `gpdma_chy_tc`. By selecting this event as the input trigger of the link transfer of the LLI_{n-1} of the channel x , the software can pause the primary channel x after its LLI_{n-1} data transfer, until it is indeed written the LLI_n .

[Figure 84](#) depicts such a dynamic elaboration of a linked-list of a primary channel x , via another auxiliary channel y .

Caution: This use case is restricted to an application with a LLI_{n-1} data transfer that does not need a trigger. The triggering mode of this LLI_{n-1} is used to load the next LLI_n .

Figure 84. Inserting a LLI_n with an auxiliary GPDMA channel y

16.4.8 GPDMA channel state and linked-list programming in link step mode

When $GPDMA_CxCR.LSM = 1$ (in link step execution mode, single execution of one LLI), a channel transfer is executed and completed after each single execution of a LLI, including its (conditional) data transfer and its (conditional) link transfer.

A GPDMA channel transfer can be programmed at LLI level, started by writing 1 into $GPDMA_CxCR.EN$, and after completed at LLI level:

- The current LLI_n transfer is described with:
 - $GPDMA_CxTR1$ defines the source/destination elementary single/burst transfers.
 - $GPDMA_CxBR1$ defines the number of bytes at a block level ($BNDT[15:0]$) and, for channel x ($x = 6$ to 7), the number of blocks at a 2D/repeated block level ($BRC[10:0]+1$) and the incrementing/decrementing mode for address offsets.

- GPDMA_CxTR2 defines the input control (request, trigger) and the output control (transfer complete event) of the transfer.
- GPDMA_CxSAR/GPDMA_CxDAR define the source/destination transfer start address.
- GPDMA_CxTR3 for channel x (x = 6 to 7) defines the source/destination additional address offset between burst transfers.
- GPDMA_CxBR2 for channel x (x = 6 to 7) defines the source/destination additional address offset between blocks at a 2D/repeated block level.
- GPDMA_CxLLR defines the data structure and the address offset of the next LLI_{n+1} in the memory.
- The current LLI_n transfer is completed after the single execution of the current LLI_n:
 - after the (conditional) data transfer completion (when GPDMA_CxBR1.BRC[10:0] = 0, and GPDMA_CxBR1.BNDT[15:0] = 0
 - after the (conditional) update of the GPDMA link register file from the data structure of the next LLI_{n+1} in memory

Note: *If a LLI is recursive (pointing to itself as a next LLI, either GPDMA_CxLLR.ULL = 1 and GPDMA_CxLLR.LA[15:2] is updated with the same value, or GPDMA_CxLLR.ULL = 0), a channel in link step mode is completed after each repeated single execution of this LLI.*

In the regular data transfer completion at a block level, GPDMA_CxBR1.BNDT[15:0] = 0 and GPDMA_CxBR1.BRC[10:0] = 0. Alternatively, a block transfer may be early completed by a peripheral (such as an I3C in Rx mode), and then BNDT[15:0] is not null (see [Section 16.4.14](#) for more details).

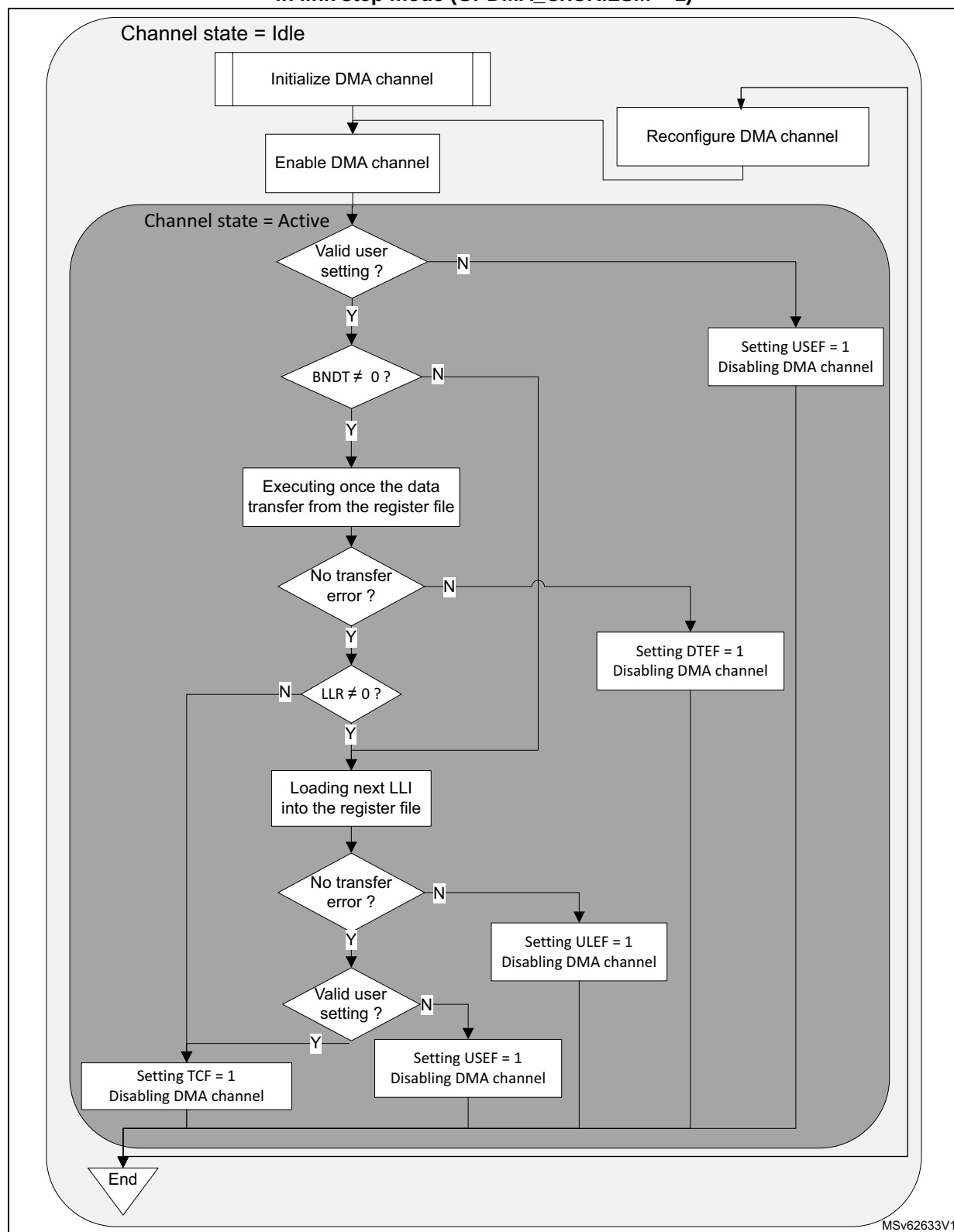
The link step mode can be used to elaborate dynamically LLIs in memory during run-time. The software can be facilitated by using a static data structure for any LLI_n (all update bits of GPDMA_CxLLR have a static value, LLI_n.LLR.LA = LLI_{n-1}.LLR.LA + constant).

[Figure 85](#) depicts the GPDMA channel execution mode, and its programming in link step mode.

Note: *[Figure 85](#) is not intended to illustrate how often a TCEF can be raised, depending on the programmed value of TCEM[1:0] in GPDMA_CxTR2. It can be raised at (each) block completion, at (each) 2D block completion, at (each) LLI completion, or only at the last LLI data transfer completion. In link step mode, the channel is disabled after each single execution of a LLI, and depending on the value of TCEM[1:0] a TCEF is raised or not.*

In [Figure 85](#), BNDT ≠ 0 is the typical condition for starting the first data transfer. This condition becomes (BNDT ≠ 0 and PFREQ = 1) if the peripheral requests a data transfer with early termination (see [Section 16.3.6](#)).

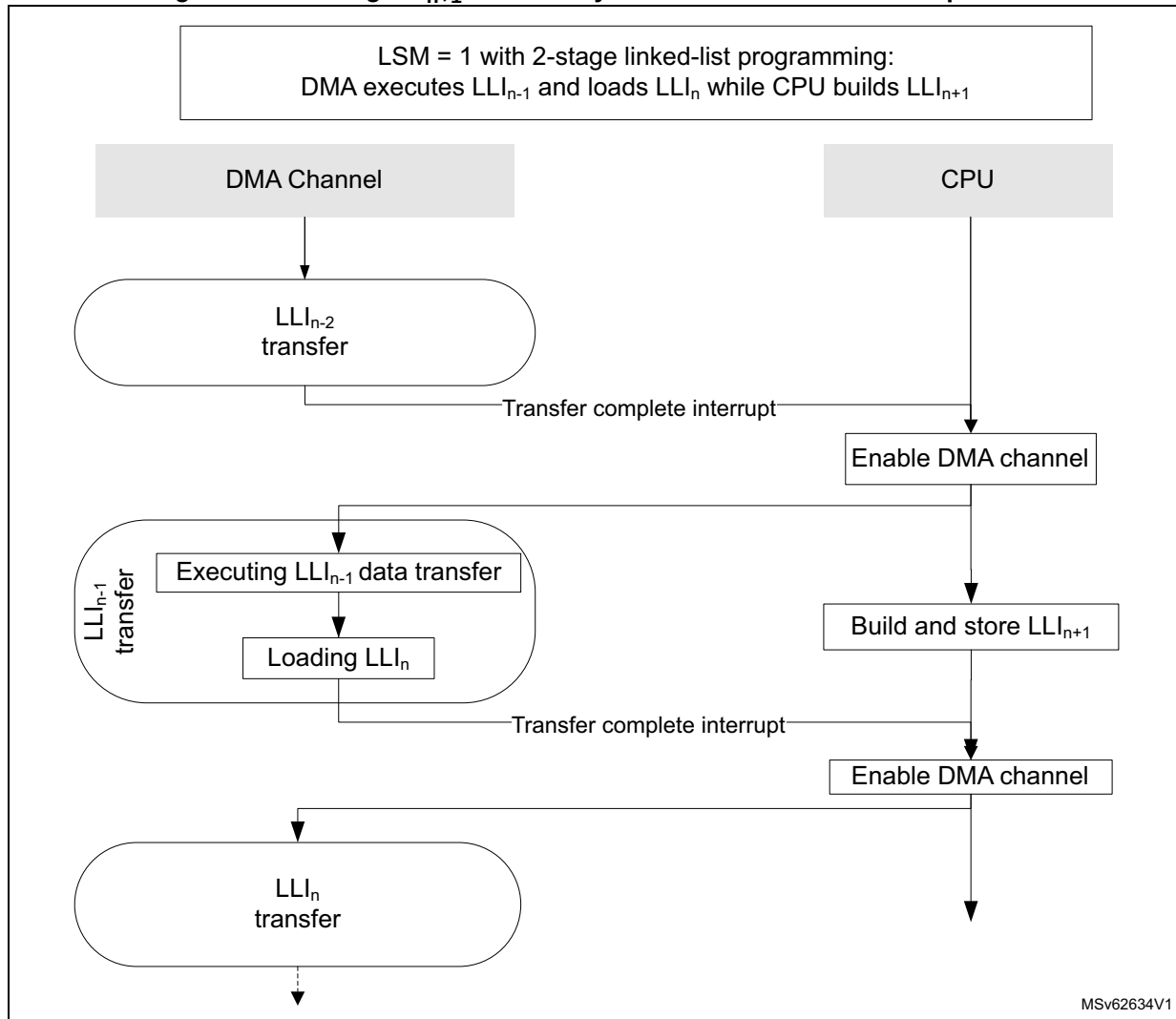
Figure 85. GPDMA channel execution and linked-list programming in link step mode (GPDMA_CxCR.LSM = 1)



Run-time adding a LLI_{n+1} in link step mode

During run-time, the software can defer the elaboration of the LLI_{n+1} (and next LLIs), until/after GPDMA executed the transfer from the LLI_{n-1} and loaded the LLI_n from the memory, as shown in [Figure 86](#).

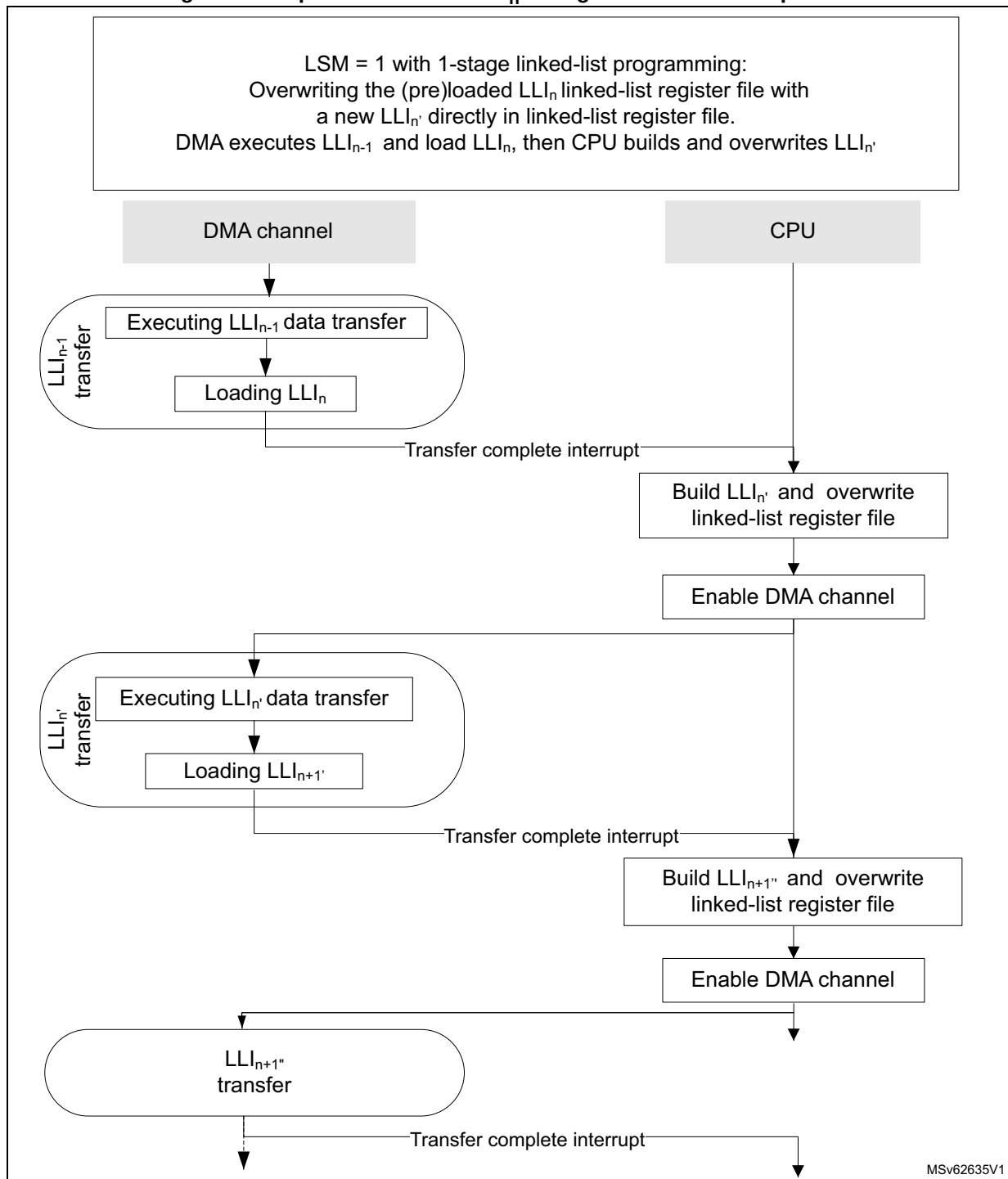
Figure 86. Building LLI_{n+1} : GPDMA dynamic linked-lists in link step mode



Run-time replacing a LLI_n with a new LLI_n' in link step mode (in linked-list register file)

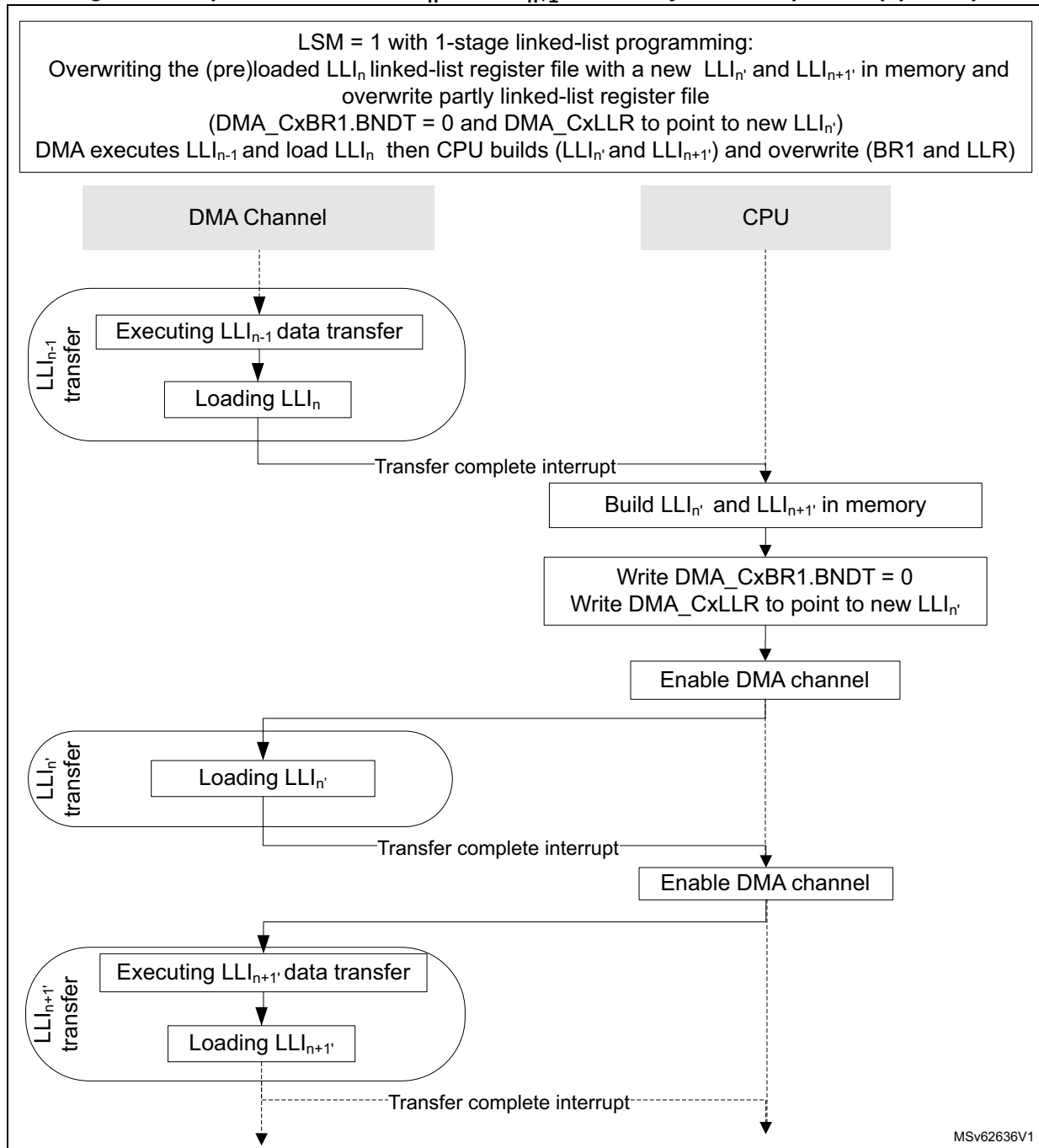
In this link step mode, during run-time, the software can build and insert a new LLI_n' , after GPDMA executed the transfer from the LLI_{n-1} and loaded a formerly elaborated LLI_n from the memory by overwriting directly the linked-list register file with the new LLI_n' , as shown in [Figure 87](#).

Figure 87. Replace with a new LLI_n in register file in link step mode



Run-time replacing a LLI_n with a new LLI_n , in link step mode (in the memory)

The software can build and insert a new LLI_n , and LLI_{n+1} , in the memory, after GPDMA executed the transfer from the LLI_{n-1} and loaded a formerly elaborated LLI_n from the memory, by overwriting partly the linked-list register file (GPDMA_CxBR1.BNDT[15:0] to be null and GPDMA_CxLLR to point to new LLI_n), as shown in [Figure 88](#).

Figure 88. Replace with a new LLI_n and LLI_{n+1} in memory in link step mode (option 1)

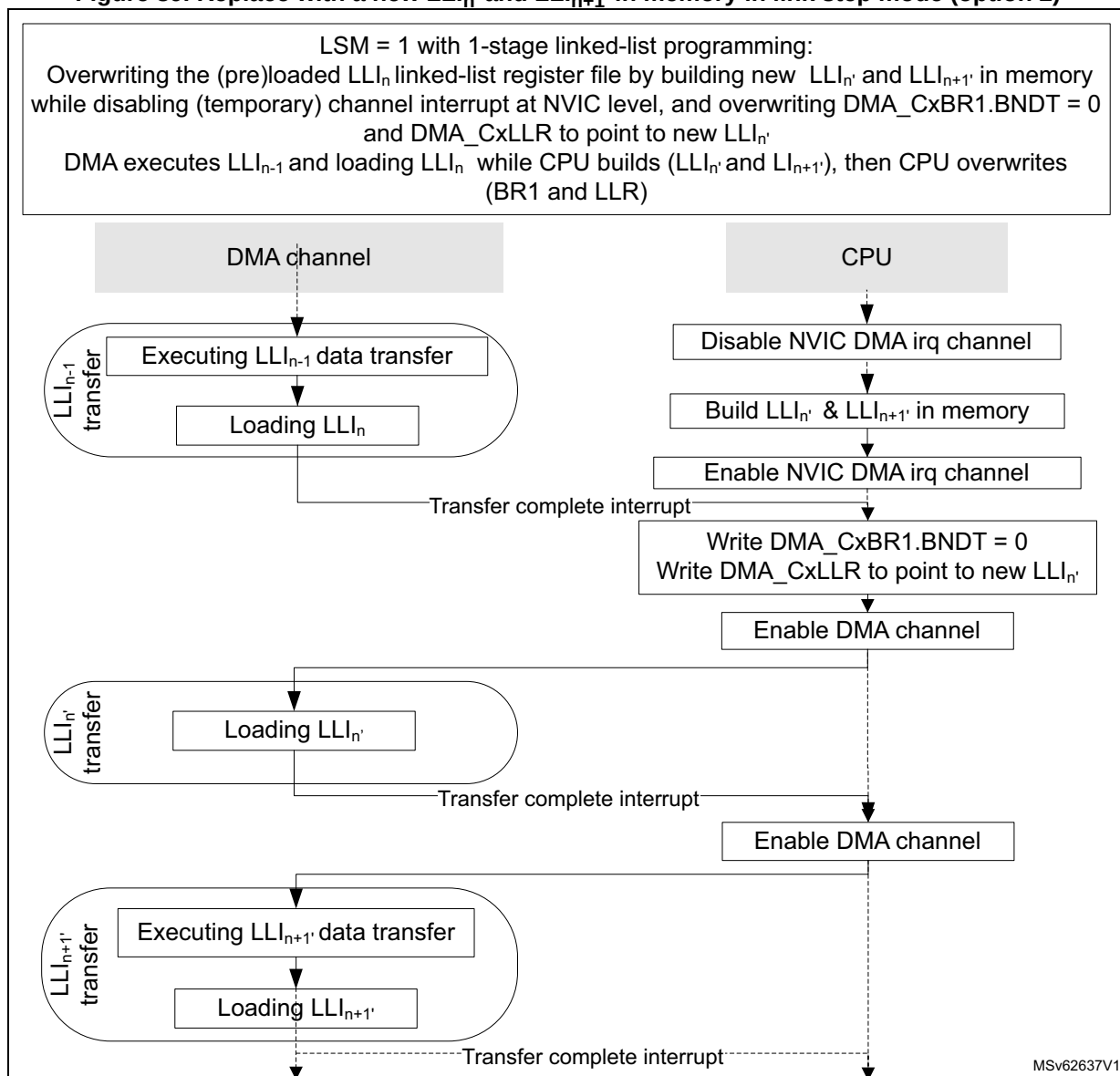
Run-time replacing a LLI_n with a new LLI_n , in link step mode

Other software implementations exist. Meanwhile GPDMA executes the transfer from the LLI_{n-1} and loads a formerly elaborated LLI_n from the memory (or even earlier), the software can do the following:

1. Disable the NVIC for not being interrupted by the interrupt handling.
2. Build a new LLI_n , and a new LLI_{n+1} .
3. Enable again the NVIC for the channel interrupt (transfer complete) notification.

The software in the interrupt handler for LLI_{n-1} is then restricted to overwrite $GPDMA_CxBR1.BNDT[15:0]$ to be null and $GPDMA_CxLLR$ to point to new LLI_n , as shown in [Figure 89](#).

Figure 89. Replace with a new LLI_n and LLI_{n+1} , in memory in link step mode (option 2)



16.4.9 GPDMA channel state and linked-list programming

The software can reconfigure a channel when the channel is disabled (GPDMA_CxCR.EN = 0) and update the execution mode (GPDMA_CxCR.LSM) to change from/to run-to-completion mode to/from link step mode.

In any execution mode, the software can:

- reprogram LLI_{n+1} in the memory to finally complete the channel by this LLI_{n+1} (clear the GPDMA_CxLLR of this LLI_{n+1}), before this LLI_{n+1} is loaded/used by the GPDMA channel
- abort and reconfigure the channel with a LSM update (see [Section 16.4.4](#).)

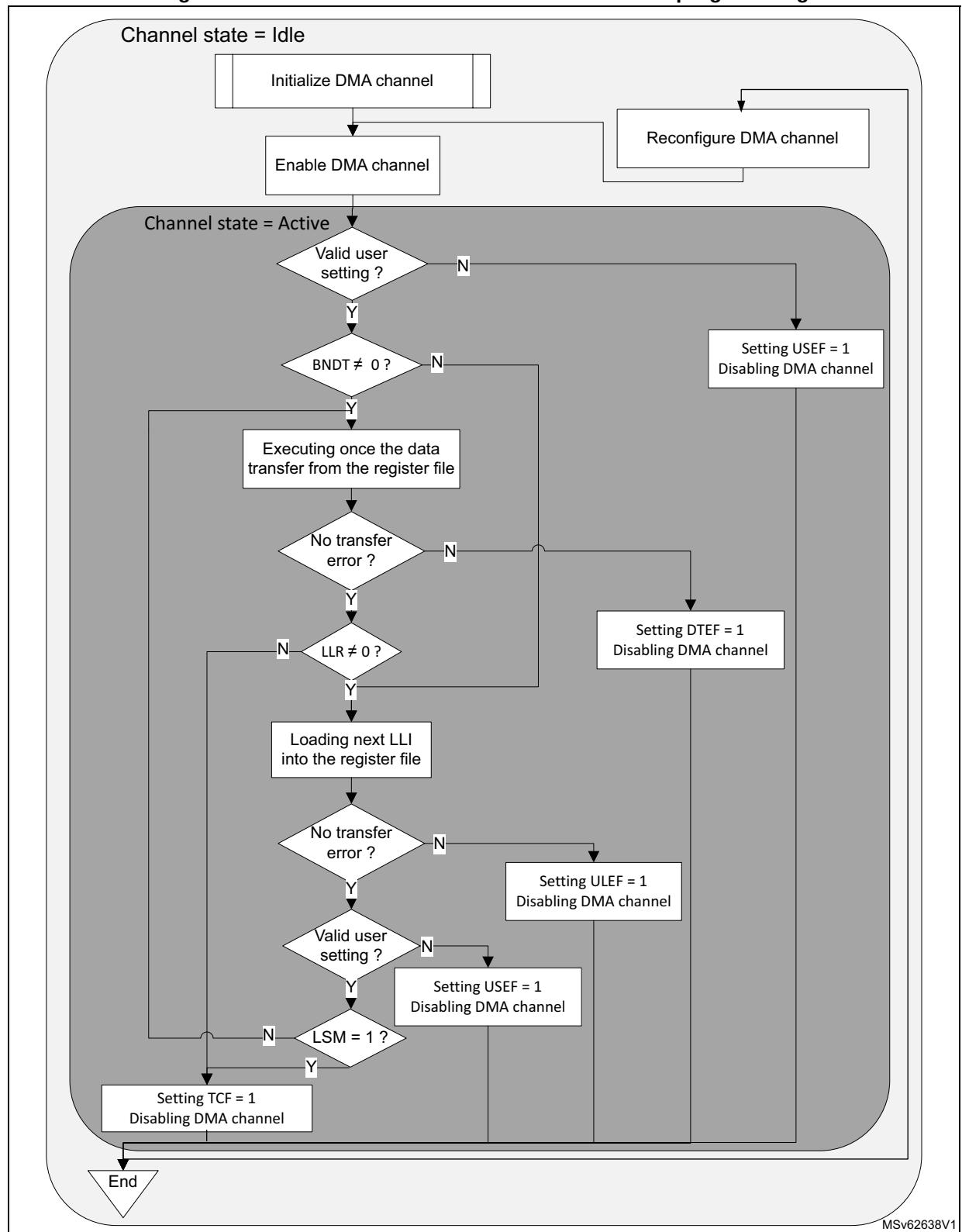
In link step mode, the software can clear LSM after each a single execution of any LLI, during LLI_{n-1}.

[Figure 90](#) shows the overall and unified GPDMA linked-list programming, whatever is the execution mode.

Note: [Figure 90](#) is not intended to illustrate how often a TCEF can be raised, depending on the programmed value of TCEM[1:0] in GPDMA_CxTR2. It can be raised at (each) block completion, at (each) 2D block completion, at (each) LLI completion, or only at the last LLI data transfer completion. In run-to-completion mode, whatever is the value of TCEM[1:0], at the channel completion the hardware always set TCEF = 1 and disables the channel. In link step mode, the channel is disabled after each single execution of a LLI, and depending on the value of TCEM[1:0] a TCEF is raised or not.

In [Figure 90](#), BNDT ≠ 0 is the typical condition for starting the first data transfer. This condition becomes (BNDT ≠ 0 and PFREQ = 1) if the peripheral requests a data transfer with early termination (see [Section 16.3.6](#)).

Figure 90. GPDMA channel execution and linked-list programming



16.4.10 GPDMA FIFO-based transfers

There is a single transfer operation mode: the FIFO mode. There are FIFO-based transfers. Any channel x is implemented with a dedicated FIFO whose size is defined by `dma_fifo_size[x]` (see [Section 16.3.2](#) for more details).

GPDMA burst

A programmed transfer at the lowest level is a GPDMA burst.

A GPDMA burst is a burst of data received from the source, or a burst of data sent to the destination. A source (and destination) burst is programmed with a burst length by the field `SBL_1[5:0]` (respectively `DBL_1[5:0]`), and with a data width defined by the field `SDW_LOG2[1:0]` (respectively `DDW_LOG2[1:0]`) in the `GPDMA_CxTR1` register.

The addressing mode after each data (named beat) of a GPDMA burst is defined by `SINC` and `DINC` in `GPDMA_CxTR1`, for source and destination respectively: either a fixed addressing or an incremented addressing with contiguous data.

The start and next addresses of a GPDMA source/destination burst (defined by `GPDMA_CxSAR` and `GPDMA_CxDAR`) must be aligned with the respective data width.

The table below lists the main characteristics of a GPDMA burst.

Table 147. Programmed GPDMA source/destination burst

SDW_LOG2[1:0] DDW_LOG2[1:0]	Data width (bytes)	SINC/DINC	SBL_1[5:0] DBL_1[5:0]	Burst length (data/beats)	Next data/ beat address	Next burst address	Burst address alignment
00	1	0 (fixed)	n = 0 to 63 ⁽¹⁾	n+1	+ 0	+ 0	1
01	2						2
10	4						4
00	1	1 (contiguously incremented)			+ 1	+ (n + 1)	1
01	2				+ 2	+ 2 * (n + 1)	2
10	4				+ 4	+ 4 * (n + 1)	4
11	forbidden user setting, causing USEF generation and none burst to be issued.						

1. When `S/DBL_1[5:0] = 0`, burst is of length 1. Then burst can be also named as single.

The next burst address in the above table is the next source/destination default address pointed by `GPDMA_CxSAR` or `GPDMA_CxDAR`, once the programmed source/destination burst is completed. This default value refers to the fixed/contiguously incremented address.

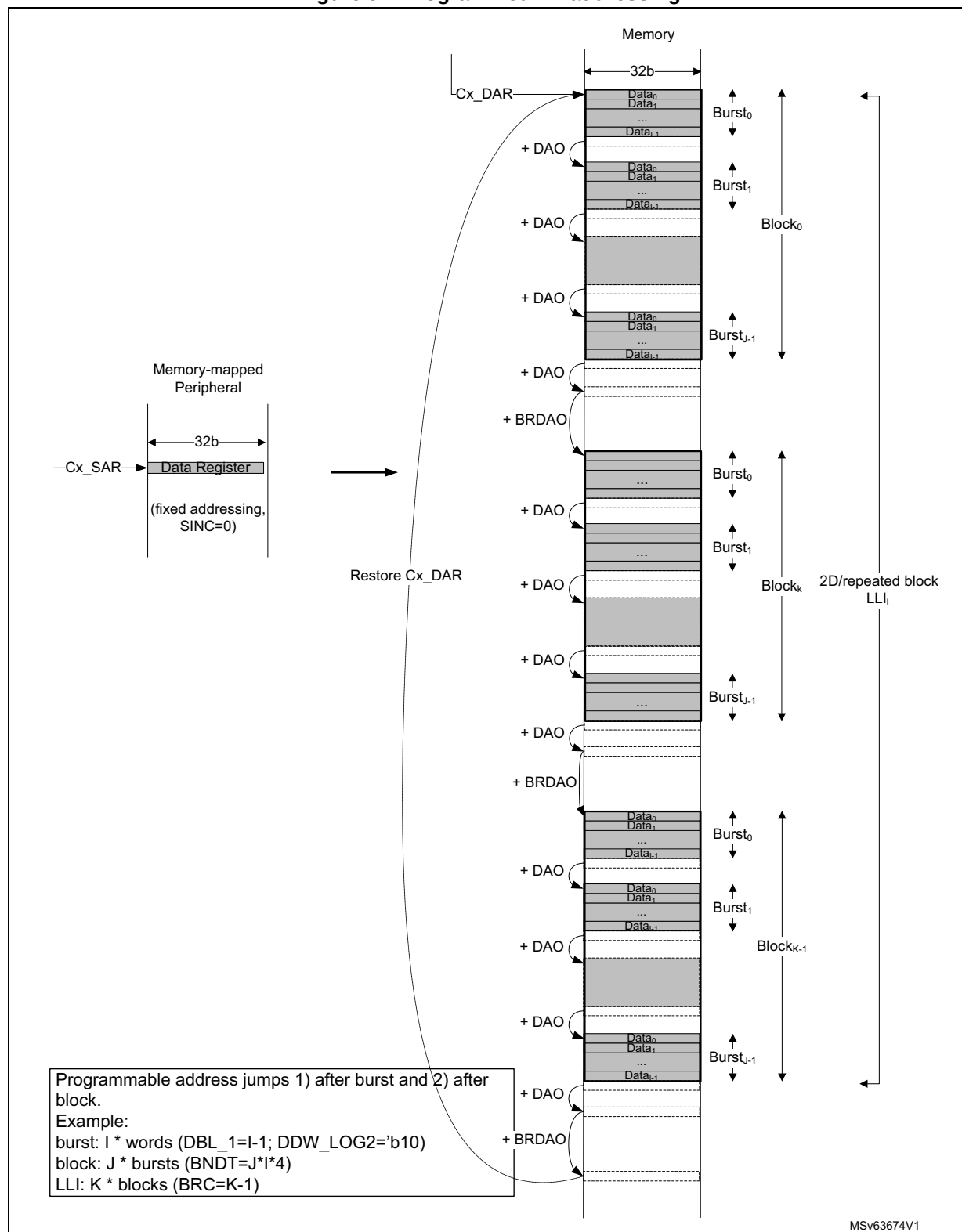
GPDMA burst with 2D addressing (channel x = 6 to 7)

When the channel has additional 2D addressing feature, this default value refers to the value without taking into account the two programmed incremented or decremented offsets. These two additional offsets (with a null default value) are applied:

- after each completed source/destination burst, as defined respectively by GPDMA_CxTR2.SAO[12:0]/DAO[12:0] and GPDMA_CxBR1.SDEC/DDEC
- after each completed block, as defined respectively by GPDMA_CxBR2.BRSAO[15:0]/BRDAO[15:0] and GPDMA_CxBR1.BRSDEC/BRDDEC)

Then, a 2D/repeated block can be addressed with a first programmed address jump after each completed burst, and with a second programmed address jump after each block, as depicted by [Figure 91](#) with a 2D destination buffer.

Figure 91. Programmed 2D addressing



GPDMA FIFO-based burst

In FIFO-mode, a transfer generally consists of two pipelined and separated burst transfers:

- one burst from the source to the FIFO over the allocated source master port, as defined by GPDMA_CxTR1.SAP
- one burst from the FIFO to the destination over the allocated destination master port, as defined by GPDMA_CxTR1.DAP

GPDMA source burst

The requested source burst transfer to the FIFO can be scheduled as early as possible over the allocated port, depending on the current FIFO level versus the programmed burst size (when the FIFO is ready to get one new burst from the source):

when $\text{FIFO level} \leq 2^{\text{dma_fifo_size}[x]} - (\text{SBL_1}[5:0]+1) * 2^{\text{SDW_LOG2}[1:0]}$

where:

- FIFO level is the current filling level of the FIFO, in bytes.
- $2^{\text{dma_fifo_size}[x]}$ is the half the FIFO size of channel x, in bytes (see [Section 16.3.2](#) for the implementation details and dma_fifo_size[x] value).
- $(\text{SBL_1}[5:0]+1) * 2^{\text{SDW_LOG2}[1:0]}$ is the size of the programmed source burst transfer, in bytes.

Based on the channel priority (GPDMA_CxCR.PRIO[1:0]), this ready FIFO-based source transfer is internally arbitrated versus the other requested and active channels.

GPDMA destination burst

The requested destination burst transfer from the FIFO can be scheduled as early as possible over the allocated port, depending on the current FIFO level versus the programmed burst size (when the FIFO is ready to push one new burst to the destination):

when $\text{FIFO level} \geq (\text{DBL_1}[5:0]+1) * 2^{\text{DDW_LOG2}[1:0]}$

where:

- FIFO level is the current filling level of the FIFO, in bytes.
- $(\text{DBL_1}[5:0]+1) * 2^{\text{DDW_LOG2}[1:0]}$ is the size of the programmed destination burst transfer, in bytes.

Based on the channel priority, this ready FIFO-based destination transfer is internally arbitrated versus the other requested and active channels.

GPDMA burst vs source block size, 1-Kbyte address boundary and FIFO size

The programmed source/destination GPDMA burst is implemented with an AHB burst as is, unless one of the following conditions is met:

- When half of the FIFO size of the channel x is lower than the programmed source/destination burst size, the programmed source/destination GPDMA burst is implemented with a series of singles or bursts of a lower size. Each transfer is of a size that is lower than or equal to half of the FIFO size, without any user constraint.
- If the source block size (GPDMA_CxBR1.BNDT[15:0]) is not a multiple of the source burst size but is a multiple of the data width of the source burst (GPDMA_CxTR1.SDW_LOG2[1:0]), the GPDMA modifies and shortens bursts into singles or bursts of lower length, in order to transfer exactly the source block size, without any user constraint.

- If the source/destination burst transfer have crossed the 1-Kbyte address boundary on a AHB transfer, the GPDMA modifies and shortens the programmed burst into singles or bursts of lower length, to be compliant with the AHB protocol, without any user constraint.
- If the source/destination burst length exceeds 16 on a AHB transfer, the GPDMA modifies and shortens the programmed burst into singles or bursts of lower length, to be compliant with the AHB protocol, without any user constraint.

In any case, the GPDMA keeps ensuring source/destination data (and address) integrity without any user constraint. The current FIFO level (software readable in GPDMA_CxSR) is compared to and updated with the effective transfer size, and the GPDMA re-arbitrates between each AHB single or burst transfer, possibly modified.

Based on the channel priority, each single or burst of a lower burst size versus the programmed burst, is internally arbitrated versus the other requested and active channels.

Note: In linked-list mode, the GPDMA read transfers related to the update of the linked-list parameters from the memory to the internal GPDMA registers, are scheduled over the link allocated port, as programmed by GPDMA_CxCR.LAP.

GPDMA data handling: byte-based reordering, packing/unpacking, padding/truncation, sign extension and left/right alignment

The data handling is controlled by GPDMA_CxTR1. The source/destination data width of the programmed burst is byte, half-word or word, as per the SDW_LOG2[1:0] and DDW_LOG2[1:0] fields (see [Table 148](#)).

The user can configure the data handling between transferred data from the source and transfer to the destination. More specifically, programmed data handling is orderly performed with:

1. Byte-based source reordering
 - If SBX = 1 and if source data width is a word, the two bytes of the unaligned half-word at the middle of each source data word are exchanged.
2. Data width conversion by packing, unpacking, padding or truncation, if destination data width is different than the source data width, depending on PAM[1:0]:
 - If destination data width > source data width, the post SBX source data is either right-aligned and padded with 0 s, or sign extended up to the destination data width, or is FIFO queued and packed up to the destination data width.
 - If destination data width < source data width, the post SBX data is either right-aligned and left-truncated down to the destination data width, or is FIFO queued and unpacked and streamed down to the destination data width.
3. Byte-based destination re-ordering:
 - If DBX = 1 and if the destination data width is not a byte, the two bytes are exchanged within the aligned post PAM[1:0] half-words.
 - If DHX = 1 and if the destination data width is neither a byte nor a half-word, the two aligned half-words are exchanged within the aligned post PAM[1:0] words.

Note: Left-alignment with 0s-padding can be achieved by programming both a right-alignment with a 0s-padding and a destination byte-based re-ordering.

The table below lists the possible data handling from the source to the destination.

Table 148. Programmed data handling

SDW_ LOG2 [1:0]	Source data	Source data stream ⁽¹⁾	SB X	DDW_ LOG2 [1:0]	Destination data	PAM[1:0] ⁽²⁾	DB X	DH X	Destination data stream ⁽¹⁾
00	Byte	B ₇ ,B ₆ ,B ₅ , B ₄ ,B ₃ ,B ₂ , B ₁ ,B ₀	x	00	Byte	xx	x		B ₇ ,B ₆ ,B ₅ ,B ₄ ,B ₃ ,B ₂ ,B ₁ ,B ₀
				01	Half-word	00 (RA, 0P)	0	x	0B ₃ ,0B ₂ ,0B ₁ ,0B ₀
							1		B ₃ 0,B ₂ 0,B ₁ 0,B ₀ 0
						01 (RA, SE)	0		SB ₃ ,SB ₂ ,SB ₁ ,SB ₀
							1		B ₃ S,B ₂ S,B ₁ S,B ₀ S
						1x (PACK)	0		B ₇ B ₆ ,B ₅ B ₄ ,B ₃ B ₂ ,B ₁ B ₀
							1		B ₆ B ₇ ,B ₄ B ₅ ,B ₂ B ₃ ,B ₀ B ₁
				10	Word	00 (RA, 0P)	0	0	000B ₁ ,000B ₀
							1		00B ₁ 0,00B ₀ 0
							0	1	0B ₁ 00,0B ₀ 00
							1		B ₁ 000,B ₀ 000
						01 (RA, SE)	0	0	SSSB ₁ ,SSSB ₀
							1		SSB ₁ S,SSB ₀ S
							0	1	SB ₁ SS,SB ₀ SS
							1		B ₁ SSS,B ₀ SSS
						1x (PACK)	0	0	B ₇ B ₆ B ₅ B ₄ ,B ₃ B ₂ B ₁ B ₀
							1		B ₆ B ₇ B ₄ B ₅ ,B ₂ B ₃ B ₀ B ₁
							0	1	B ₅ B ₄ B ₇ B ₆ ,B ₁ B ₀ B ₃ B ₂
							1		B ₄ B ₅ B ₆ B ₇ ,B ₀ B ₁ B ₂ B ₃
01	Half-word	B ₇ B ₆ ,B ₅ B ₄ , B ₃ B ₂ , B ₁ B ₀	x	00	Byte	00 (RA, LT)	x	x	B ₆ ,B ₄ ,B ₂ ,B ₀
						01 (LA, RT)			B ₇ ,B ₅ ,B ₃ ,B ₁
						1x (UNPACK)			B ₇ ,B ₆ ,B ₅ ,B ₄ ,B ₃ ,B ₂ ,B ₁ ,B ₀

Table 148. Programmed data handling (continued)

SDW_ LOG2 [1:0]	Source data	Source data stream ⁽¹⁾	SB X	DDW_ LOG2 [1:0]	Destination data	PAM[1:0] ⁽²⁾	DB X	DH X	Destination data stream ⁽¹⁾
01	Half- word	B ₇ B ₆ ,B ₅ B ₄ , B ₃ B ₂ , B ₁ B ₀	x	01	Half-word	xx	0	x	B ₇ B ₆ ,B ₅ B ₄ ,B ₃ B ₂ ,B ₁ B ₀
							1		B ₆ B ₇ ,B ₄ B ₅ ,B ₂ B ₃ ,B ₀ B ₁
				10	Word	00 (RA, 0P)	0	0	00B ₃ B ₂ ,00B ₁ B ₀
							1		00B ₂ B ₃ ,00B ₀ B ₁
							0	1	B ₃ B ₂ 00,B ₁ B ₀ 00
							1		B ₂ B ₃ 00,B ₀ B ₁ 00
						01 (RA, SE)	0	0	SSB ₃ B ₂ ,SSB ₁ B ₀
							1		SSB ₂ B ₃ ,SSB ₀ B ₁
							0	1	B ₃ B ₂ SS,B ₁ B ₀ SS
							1		B ₂ B ₃ SS,B ₀ B ₁ SS
						1x (PACK)	0	0	B ₇ B ₆ B ₅ B ₄ ,B ₃ B ₂ B ₁ B ₀
							1		B ₆ B ₇ B ₄ B ₅ ,B ₂ B ₃ B ₀ B ₁
							0	1	B ₅ B ₄ B ₇ B ₆ ,B ₁ B ₀ B ₃ B ₂
							1		B ₄ B ₅ B ₆ B ₇ ,B ₀ B ₁ B ₂ B ₃
10	Word	B ₇ B ₆ B ₅ B ₄ , B ₃ B ₂ B ₁ B ₀	0	00	Byte	00 (RA, LT)	x	x	B ₁₂ ,B ₈ ,B ₄ ,B ₀
						01 (LA, RT)			B ₁₅ ,B ₁₁ ,B ₇ ,B ₃
						10 (UNPACK)			B ₇ ,B ₆ ,B ₅ ,B ₄ ,B ₃ ,B ₂ ,B ₁ ,B ₀
				01	Half-word	00 (RA, LT)	0		B ₅ B ₄ ,B ₁ B ₀
							1		B ₄ B ₅ ,B ₀ B ₁
						01 (LA, RT)	0		B ₇ B ₆ ,B ₃ B ₂
							1		B ₆ B ₇ ,B ₂ B ₃
						1x (UNPACK)	0		B ₇ B ₆ ,B ₅ B ₄ ,B ₃ B ₂ ,B ₁ B ₀
							1		B ₆ B ₇ ,B ₄ B ₅ ,B ₂ B ₃ ,B ₀ B ₁

Table 148. Programmed data handling (continued)

SDW_ LOG2 [1:0]	Source data	Source data stream ⁽¹⁾	SB X	DDW_ LOG2 [1:0]	Destination data	PAM[1:0] ⁽²⁾	DB X	DH X	Destination data stream ⁽¹⁾
10	Word	B ₇ B ₆ B ₅ B ₄ , B ₃ B ₂ B ₁ B ₀	0	10	Word	xx	0	0	B ₇ B ₆ B ₅ B ₄ ,B ₃ B ₂ B ₁ B ₀
							1		B ₆ B ₇ B ₄ B ₅ ,B ₂ B ₃ B ₀ B ₁
							0	1	B ₅ B ₄ B ₇ B ₆ ,B ₁ B ₀ B ₃ B ₂
							1		B ₄ B ₅ B ₆ B ₇ ,B ₀ B ₁ B ₂ B ₃
			1	00	Byte	00 (RA, LT)	x	x	B ₁₂ ,B ₈ ,B ₄ ,B ₀
						01 (LA, RT)			B ₁₅ ,B ₁₁ ,B ₇ ,B ₃
						1x (UNPACK)			B ₇ ,B ₅ ,B ₆ ,B ₄ ,B ₃ ,B ₁ ,B ₂ ,B ₀
				01	Half-word	00 (RA, LT)	0		B ₆ B ₄ ,B ₂ B ₀
							1		B ₄ B ₆ ,B ₀ B ₂
						01 (LA, RT)	0		B ₇ B ₅ ,B ₃ B ₁
							1		B ₅ B ₇ ,B ₁ B ₃
						1x (UNPACK)	0		B ₇ B ₅ ,B ₆ B ₄ ,B ₃ B ₁ ,B ₂ B ₀
							1		B ₅ B ₇ ,B ₄ B ₆ ,B ₁ B ₃ ,B ₀ B ₂
				10	Word	xx	0	0	B ₇ B ₅ B ₆ B ₄ ,B ₃ B ₁ B ₂ B ₀
							1		B ₅ B ₇ B ₄ B ₆ ,B ₁ B ₃ B ₀ B ₂
							0	1	B ₆ B ₄ B ₇ B ₅ ,B ₂ B ₀ B ₃ B ₁
							1		B ₄ B ₆ B ₅ B ₇ ,B ₀ B ₂ B ₁ B ₃

1. Data stream is timely ordered starting from the byte with the lowest index (B₀).

2. RA= right aligned, LA = left aligned, RT = right truncated, LT = left truncated, OP = zero bit padding up to the destination data width, SE = sign bit extended up to the destination data width.

16.4.11 GPDMA transfer request and arbitration

GPDMA transfer request

As defined by GPDMA_CxTR2, a programmed GPDMA data transfer is requested with one of the following:

- a software request if the control bit SWREQ = 1: This is used typically by the CPU for a data transfer from a memory-mapped address to another memory mapped address (memory-to-memory, GPIO to/from memory)
- an input hardware request coming from a peripheral if SWREQ = 0: The selection of the GPDMA hardware peripheral request is driven by the REQSEL[7:0] field (see [Section 16.3.4](#)). The selected hardware request can be one of the following:
 - an hardware request from a peripheral configured in GPDMA mode (for a transfer from/to the peripheral data register respectively to/from the memory)
 - an hardware request from a peripheral for its control registers update from the memory
 - an hardware request from a peripheral for a read of its status registers transferred to the memory

Caution: The user must not assign a same input hardware peripheral GPDMA request via GPDMA_CxTR.REQSEL[7:0] to two different channels, if at a given time this request is asserted by the peripheral and each channel is ready to execute this requested data transfer. There is no user setting error reporting.

GPDMA transfer request for arbitration

A ready FIFO-based GPDMA source single/burst transfer (from the source address to the FIFO) to be scheduled over the allocated master port (GPDMA_CxTR1.SAP) is arbitrated based on the channel priority (GPDMA_CxCR.PRIO[1:0]) versus the other simultaneous requested GPDMA transfers to the same master port.

A ready FIFO-based GPDMA destination single/burst transfer (from the FIFO to the destination address) to be scheduled over the allocated master port (GPDMA_CxTR1.DAP) is arbitrated based on the channel priority (GPDMA_CxCR.PRIO[1:0]) versus the other simultaneous requested GPDMA transfers to the same master port.

An arbitrated GPDMA requested link transfer consists of one 32-bit read from the linked-list data structure in memory to one of the linked-list registers (GPDMA_CxTR1, GPDMA_CxTR2, GPDMA_CxBR1, GPDMA_CxSAR, GPDMA_CxDAR or GPDMA_CxLLR, plus GPDMA_CxTR3, GPDMA_CxBR2). Each 32-bit read from memory is arbitrated with the same channel priority as for data transfers, in order to be scheduled over the allocated master port (GPDMA_CxCR.LAP).

Whatever the requested data transfer is programmed with a software request for a memory-to-memory transfer (GPDMA_CxTR2.SWREQ = 1), or with a hardware request (GPDMA_CxTR2.SWREQ = 0) for a memory-to-peripheral transfer or a peripheral-to-memory transfer and whatever is the hardware request type, re-arbitration occurs after each granted single/burst transfer.

When an hardware request is programmed from a destination peripheral (GPDMA_CxTR2.SWREQ = 0 and GPDMA_CxTR2.DREQ = 1), the first memory read of a (possibly 2D/repeated) block (the first ready FIFO-based source burst request), is gated by the occurrence of the corresponding and selected hardware request. This first read request to memory is not taken into account earlier by the arbiter (not as soon as the block transfer is enabled and executable).

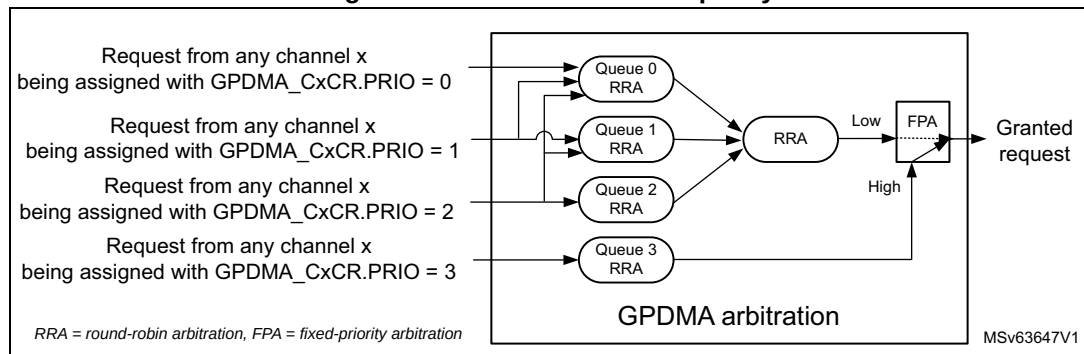
GPDMA arbitration

The GPDMA arbitration is directed from the 4-grade assigned channel priority (GPDMA_CxCR.PRIO[1:0]). The arbitration policy, as illustrated in [Figure 92](#), is defined by:

- one high-priority traffic class (queue 3), dedicated to the assigned channels with priority 3, for time-sensitive channels
This traffic class is granted via a fixed-priority arbitration against any other low-priority traffic class. Within this class, requested single/burst transfers are round-robin arbitrated.
- three low-priority traffic classes (queues 0, 1, or 2) for non time-sensitive channels with priority 0, 1, or 2
Each requested single/burst transfer within this class is round-robin arbitrated, with a weight that is monotonically driven from the programmed priority:
 - Requests with priority 0 are allocated to the queue 0.
 - Requests with priority 1 are allocated and replicated to the queue 0 and queue 1.

- Requests with priority 2 are allocated and replicated to the queue 0, queue 1, and queue 2.
- Any queue 0, 1, or 2 equally grants any of its active input requests in a round-robin manner, provided there are simultaneous requests.
- Additionally, there is a second stage for the low-traffic with a round-robin arbiter that fairly alternates between simultaneous selected requests from queue 0, queue 1, and queue 2.

Figure 92. GPDMA arbitration policy



GPDMA arbitration and bandwidth

With this arbitration policy, the following is guaranteed:

- Equal maximum bandwidth between requests with same priority
- Reserved bandwidth (noted as B_{Q3}) to the time-sensitive requests (with priority 3)
- Residual weighted bandwidth between different low-priority requests (priority 0 versus priority 1 versus priority 2).

The two following examples highlight that the weighted round-robin arbitration is driven by the programmed priorities:

- **Example 1:** basic application with two non time-sensitive GPDMA requests: req0 and req1. There are the following programming possibilities:
 - If they are assigned with same priority, the allocated bandwidth by the arbiter to req0 (B_{req0}) is **equal** to the allocated bandwidth to req1 (B_{req1}).

$$B_{req0} = B_{req1} = 1/2 * (1 - B_{Q3})$$
 - If req0 is assigned to priority 0 and req1 to priority 1, the allocated bandwidth to req0 (B_{P0}) is **3 times less** than the allocated bandwidth to req1 (B_{P1}).

$$B_{req0} = B_{P0} = 1/2 * 1/2 * (1 - B_{Q3}) = 1/4 * (1 - B_{Q3})$$

$$B_{req1} = B_{P1} = (1/2 + 1) * 1/2 * (1 - B_{Q3}) = 3/4 * (1 - B_{Q3})$$
 - If req0 is assigned to priority 0 and req1 to priority 2, the allocated bandwidth to req0 (B_{P0}) is **5 times less** than the allocated bandwidth to req1 (B_{P2}).

$$B_{req0} = B_{P0} = 1/2 * 1/3 * (1 - B_{Q3}) = 1/6 * (1 - B_{Q3})$$

$$B_{req1} = B_{P2} = (1/2 + 1 + 1) * 1/3 * (1 - B_{Q3}) = 5/6 * (1 - B_{Q3})$$

The above computed bandwidth calculation is based on a theoretical input request, always active for any GPDMA clock cycle. This computed bandwidth from the arbiter must be weighted by the frequency of the request given by the application, that cannot be always active and may be quite much variable from one GPDMA client (example I2C at 400 kHz) to another one (PWM at 1 kHz) than the above x3 and x5 ratios.

- **Example 2:** application where the user distributes a same non-null N number of GPDMA requests to every non time-sensitive priority 0, 1 and 2. The bandwidth calculation is then the following:
 - The allocated bandwidth to the set of requests of priority 0 (B_{P0}) is

$$B_{P0} = 1/3 * 1/3 * (1 - B_{Q3}) = 1/9 * (1 - B_{Q3})$$
 - The allocated bandwidth to the set of requests of priority 1 (B_{P1}) is

$$B_{P1} = (1/3 + 1/2) * 1/3 * (1 - B_{Q3}) = 5/18 * (1 - B_{Q3})$$
 - The allocated bandwidth to the set of requests of priority 2 (B_{P2}) is

$$B_{P2} = (1/3 + 1/2 + 1) * 1/3 * (1 - B_{Q3}) = 11/18 * (1 - B_{Q3})$$
 - The allocated bandwidth to any request n (B_n) among the N requests of that priority P_i ($i = 0$ to 2) is $B_n = 1/N * B_{P_i}$
 - The allocated bandwidth to any request n of priority 0_i (B_{n, P_i}) is

$$B_{n, P0} = 1/N * 1/9 * (1 - B_{Q3})$$

$$B_{n, P1} = 1/N * 5/18 * (1 - B_{Q3})$$

$$B_{n, P2} = 1/N * 11/18 * (1 - B_{Q3})$$

In this example, when the master port bus bandwidth is not totally consumed by the time-sensitive queue 3, the residual bandwidth is such that 2.5 times less bandwidth is allocated to any request of priority 0 versus priority 1, and 5.5 times less bandwidth is allocated to any request of priority 0 versus priority 2.

More generally, assume that the following requests are present:

- I requests ($I \geq 0$) assigned to priority 0
 If $I > 0$, these requests are noted from $i = 0$ to $I-1$.
- J requests ($J \geq 0$) assigned to priority 1
 If $J > 0$, these requests are noted from $j = 0$ to $J-1$.
- K requests ($K > 0$) assigned to priority 2
 These requests are noted from $k = 0$ to $K-1$
- L requests ($L \geq 0$) assigned to priority 3
 If $L > 0$, these requests are noted from $l = 0$ to $L-1$.

As B_{Q3} is the reserved bandwidth to time-sensitive requests, the bandwidth for each request L with priority 3 is:

- $B_l = B_{Q3} / L$ for $L > 0$ (else: $B_l = 0$)

The bandwidth for each non-time sensitive queue is:

- $B_{Q0} = 1/3 * (1 - B_{Q3})$
- $B_{Q1} = 1/3 * (1 - B_{Q3})$
- $B_{Q2} = 1/3 * (1 - B_{Q3})$

The bandwidth for the set of requests with priority 0 is:

- $B_{P0} = I / (I + J + K) * B_{Q0}$

The bandwidth for each request i with priority 0 is:

- $B_i = B_{P0} / I$ for $L > 0$ (else $B_{P0} = 0$)

The bandwidth for the set of requests with priority 1 and routed to queue 0 is:

- $B_{P1, Q0} = J / (I + J + K) * B_{Q0}$

The bandwidth for the set of requests with priority 1 and routed to queue 1 is:

- $B_{P1,Q1} = J / (J + K) * B_{Q1}$

The total bandwidth for the set of requests with priority 1 is:

- $B_{P1} = B_{P1,Q0} + B_{P1,Q1}$

The bandwidth for each request j with priority 1 is:

- $B_j = B_{P1} / J$ for $J > 0$ (else $B_j = 0$)

The bandwidth for the set of requests with priority 2 and routed to queue 0 is:

- $B_{P2,Q0} = K / (I + J + K) * B_{Q0}$

The bandwidth for the set of requests with priority 2 and routed to queue 1 is:

- $B_{P2,Q1} = K / (J + K) * B_{Q1}$

The bandwidth for the set of requests with priority 2 and routed to queue 2 is:

- $B_{P2,Q2} = B_{Q2}$

The total bandwidth for the set of requests with priority 2 is:

- $B_{P2} = B_{P2,Q0} + B_{P2,Q1} + B_{P2,Q2}$

The bandwidth for each request k with priority 2 is:

- $B_k = B_{P2} / K$ ($K > 0$ in the general case)

Thus finally the maximum allocated residual bandwidths for any i, j, k non-time sensitive request are:

- in the general case (when there is at least one request k with a priority 2 ($K > 0$)):
 - $B_i = 1/I * 1/3 * I/(I + J + K) * (1 - B_{Q3})$
 - $B_j = 1/J * 1/3 * [J/(I + J + K) + J/(J + K)] * (1 - B_{Q3})$
 - $B_k = 1/K * 1/3 * [K/(I + J + K) + K/(J + K) + 1] * (1 - B_{Q3})$
- in the specific case (when there is no request k with a priority 2 ($K = 0$)):
 - $B_i = 1/I * 1/2 * I/(I + J) * (1 - B_{Q3})$
 - $B_j = 1/J * 1/2 * [J/(I + J) + 1] * (1 - B_{Q3})$

Consequently, the GPDMA arbiter can be used as a programmable weighted bandwidth limiter, for each queue and more generally for each request/channel. The different weights are monotonically resulting from the programmed channel priorities.

16.4.12 GPDMA triggered transfer

A programmed GPDMA transfer can be triggered by a rising/falling edge of a selected input trigger event, as defined by GPDMA_CxTR2.TRIGPOL[1:0] and GPDMA_CxTR2.TRIGSEL[5:0] (see [Section 16.3.7](#) for the trigger selection).

The triggered transfer, as defined by the trigger mode in GPDMA_CxTR2.TRIGM[1:0], can be at LLI data transfer level, to condition the first burst read of a block, the first burst read of a 2D/repeated block for channel x ($x = 6$ to 7), or each programmed single read. The trigger mode can also be programmed to condition the LLI link transfer (see TRIGM[1:0] in GPDMA_CxTR2 for more details).

Trigger hit memorization and trigger overrun flag generation

The GPDMA monitoring of a trigger for a channel x is started when the channel is enabled/loaded with a new active trigger configuration: rising or falling edge on a selected trigger (respectively TRIGPOL[1:0] = 01 or TRIGPOL[1:0] = 10).

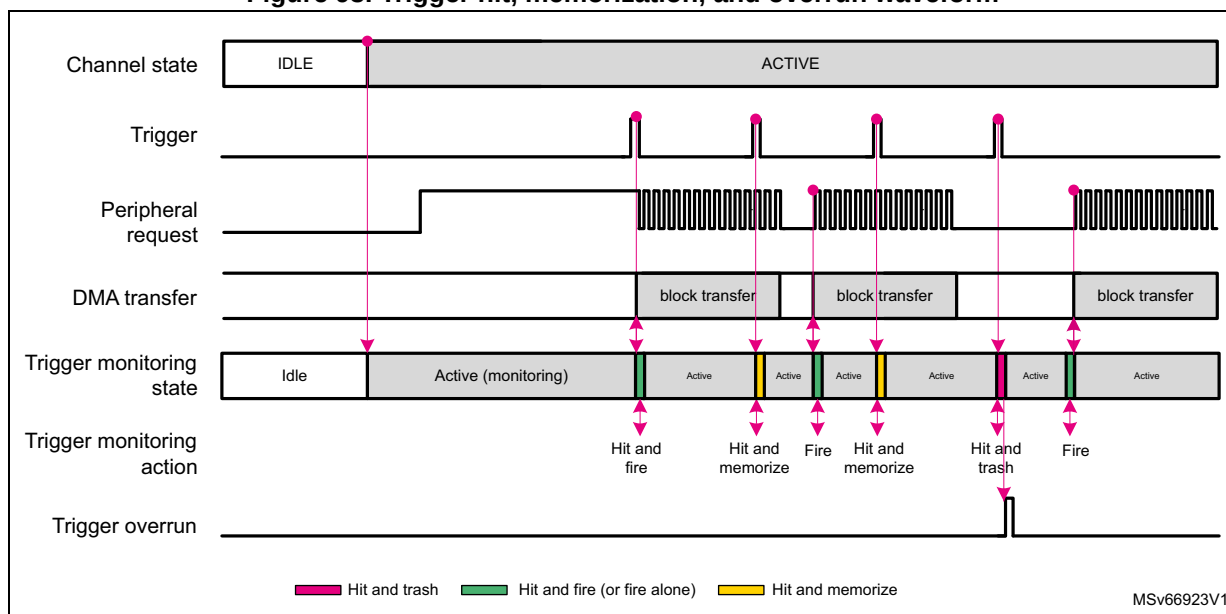
The monitoring of this trigger is kept active during the triggered and uncompleted (data or link) transfer. If a new trigger is detected, this hit is internally memorized to grant the next transfer, as long as the defined rising/falling edge and TRIGSEL[5:0] are not modified, and the channel is enabled.

Transferring a next LLI_{n+1} , that updates the GPDMA_CxTR2 with a new value for any of TRIGSEL[5:0] or TRIGPOL[1:0], resets the monitoring, trashing the possible memorized hit of the formerly defined LLI_n trigger.

Caution: After a first new trigger hit $_{n+1}$ is memorized, if another trigger hit $_{n+2}$ is detected and if the hit $_n$ triggered transfer is still not completed, hit $_{n+2}$ is lost and not memorized. A trigger overrun flag is reported (GPDMA_CxSR.TOF = 1) and an interrupt is generated if enabled (if GPDMA_CxCR.TOIE = 1). The channel is not automatically disabled by hardware due to a trigger overrun.

Figure 93 illustrates the trigger hit, memorization and overrun in the configuration example with a block-level trigger mode and a rising edge trigger polarity.

Figure 93. Trigger hit, memorization, and overrun waveform



Note: The user can assign the same input trigger event to different channels. This can be used to trigger different channels on a broadcast trigger event.

16.4.13 GPDMA circular buffering with linked-list programming

GPDMA circular buffering for memory-to-peripheral and peripheral-to-memory transfers, with a linear addressing channel

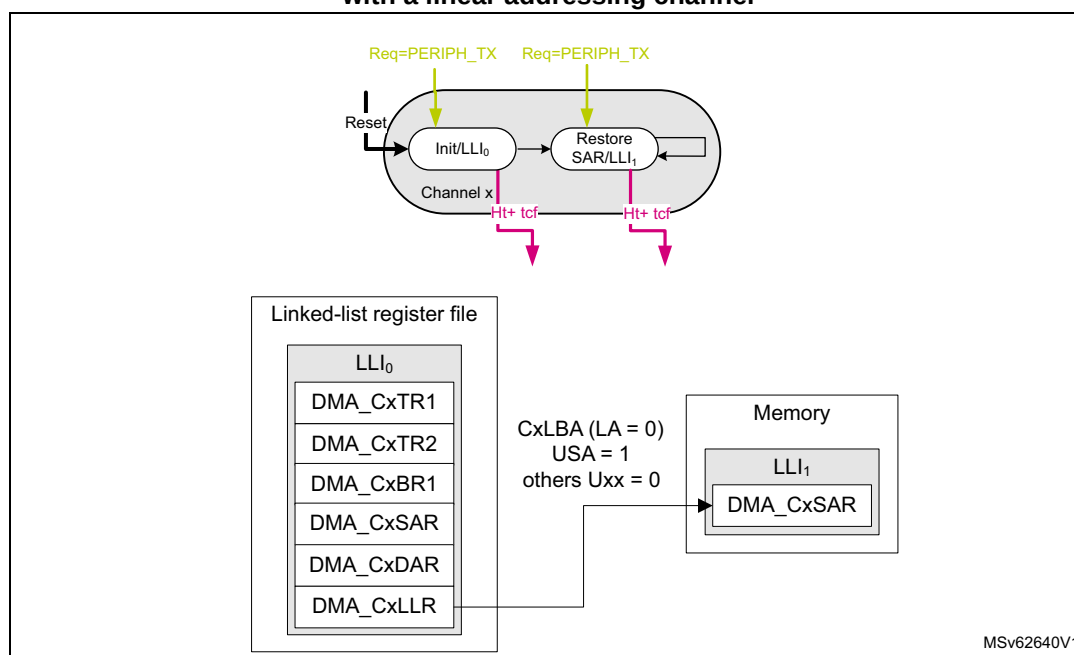
For a circular buffering, with a continuous memory-to-peripheral (or peripheral-to-memory) transfer, the software must set up a channel with half-transfer and complete transfer

events/interrupts generation (GPDMA_CxCR.HTIE = 1 and GPDMA_CxCR.TCIE = 1), in order to enable a concurrent buffer software processing.

LLI₀ is configured for the first block transfer with the linear addressing channel. A continuously-executed LLI₁ is needed to restore the memory source (or destination) start address, for the memory-to-peripheral transfer (respectively the peripheral-to-memory transfer). GPDMA automatically reloads the initially programmed GPDMA_CxBR1.BNDT[15:0] when a block transfer is completed, and there is no need to restore GPDMA_CxBR1.

Figure 94 illustrates this programming with a linear addressing GPDMA channel and a source circular buffer.

Figure 94. GPDMA circular buffer programming: update of the memory start address with a linear addressing channel



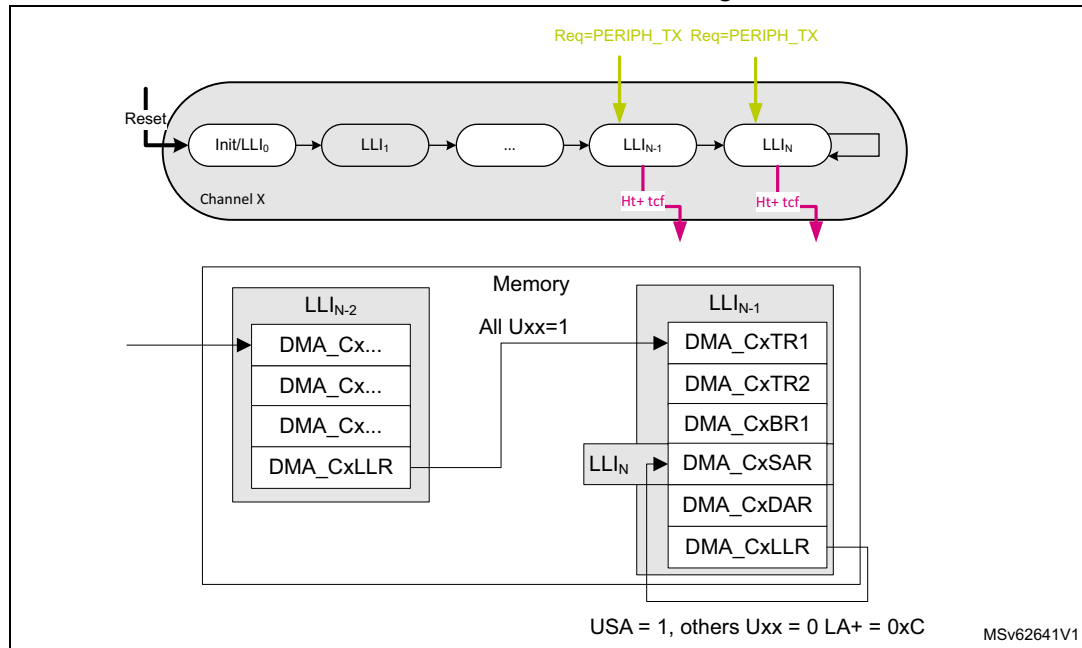
Note:

With a 2D addressing channel, the user may use a single LLI with GPDMA_CxBR1.BRC[10:0] = 1, and program a negative memory block address offset with GDMA_CxBR2 and GDMA_CxBR1, in order to jump back to the memory source or the destination start address.

If circular buffering must be executed after some other transfers over the shared GPDMA channel x, the before-last LLI_{N-1} in memory is needed to configure the first block transfer. And the last LLI_N restores the memory source (or destination) start address in memory-to-peripheral transfer (respectively in peripheral-to-memory transfer).

Figure 95 illustrates this programming with a linear addressing shared GPDMA channel, and a source circular buffer.

Figure 95. Shared GPDMA channel with circular buffering: update of the memory start address with a linear addressing channel



16.4.14 GPDMA transfer in peripheral flow-control mode

A peripheral with the peripheral flow-control mode feature can decide to early terminate a GPDMA block transfer, provided that the allocated channel is implemented with this feature (see [Section 16.3.6](#)).

If the related GPDMA channel x is also programmed in peripheral flow-control mode (GPDMA_CxTR2.PFREQ = 1):

- The GPDMA block transfer starts as follows:
 - If GPDMA_CxBR1.BNDT[15:0] $\neq 0$, the programmed value is internally taken into account by the GPDMA hardware.
 - If GPDMA_CxBR1.BNDT[15:0] = 0, the GPDMA hardware internally considers a 64-Kbyte value for the maximum source block size to be transferred.
- The GPDMA block transfer is completed as soon as the first occurrence of one of the following condition occurs:
 - when GPDMA_CxBR1.BNDT[15:0] = 0
 - when the peripheral early terminates the block. The complete transfer event is generated if programmed, depending on GPDMA_CxTR2 (see [GPDMA channel \$x\$ transfer register 2 \(GPDMA_CxTR2\)](#)). Then the software can read the current number of transferred bytes from the source (GPDMA_CxBR1.BNDT[15:0]), and/or read the current source or destination address of the buffer in memory (GPDMA_CxSAR[31:0] or GPDMA_CxDAR[31:0]).

In peripheral flow-control mode:

- a destination peripheral with a hardware requested transfer is not supported: memory-to-peripheral transfer is not supported.
- Data packing from a source peripheral is not supported.
- 2D/repeated block is not supported.
- GPDMA_CxBR1.BNDT[15:0] must be programmed as a multiple of the source (peripheral) burst size.

16.4.15 GPDMA secure/nonsecure channel

The GPDMA controller is compliant with the TrustZone hardware architecture at channel level, partitioning all its resources so that they exist in one of the secure and nonsecure worlds at any given time.

Any channel x is a secure or a nonsecure hardware resource, as configured by GPDMA_SECCFGR.SECx.

When a channel x is configured in secure state by a secure and privileged agent, the following access control rules are applied:

- A nonsecure read access to a register field of this channel is forced to return 0, except for GPDMA_SECCFGR, GPDMA_PRIVCFGR, GPDMA_RCFGLOCKR that are readable by a nonsecure agent.
- A nonsecure write access to a register field of this channel has no impact.

When a channel x is configured in secure state, a secure agent can configure separately as secure or nonsecure the GPDMA data transfer from the source (GPDMA_CxTR1.SSEC) and the GPDMA data transfer to the destination (GPDMA_CxTR1.DSEC).

When a channel x is configured in secure state and in linked-list mode, the loading of the next linked-list data structure from the GPDMA memory into its register file, is automatically performed with secure transfers via the GPDMA_CxCR.LAP allocated master port.

The GPDMA generates a secure bus that reflects GPDMA_SECCFGR, to keep the other peripherals informed of the secure/nonsecure state of each GPDMA channel x.

The GPDMA also generates a security illegal access pulse signal on an illegal nonsecure access to a secure GPDMA register. This signal is routed to the TrustZone interrupt controller.

When the secure software must switch a channel from a secure state to a nonsecure state, the secure software must abort the channel or wait until the secure channel is completed before switching. This is needed to dynamically re-allocate a channel to a next nonsecure transfer as a nonsecure software is not allowed to do so and must have GPDMA_CxCR.EN = 0 before the nonsecure software can reprogram the GPDMA_CxCR for a next transfer. The secure software may reset not only the channel x (GPDMA_CxCR.RESET = 1) but also the full channel x register file to its reset value.

16.4.16 GPDMA privileged/unprivileged channel

Any channel x is a privileged or unprivileged hardware resource, as configured by a privileged agent via GPDMA_PRIVCFGR.PRIVx.

When a channel x is configured in a privileged state by a privileged agent, the following access control rules are applied:

- An unprivileged read access to a register field of this channel is forced to return 0, except for GPDMA_PRIVCFGR, GPDMA_SECCFGR, GPDMA_RCFGLOCKR that are readable by an unprivileged agent.
- An unprivileged write access to a register field of this channel has no impact.

When a channel is configured in a privileged (or unprivileged) state, the source and destination data transfers are privileged (respectively unprivileged) transfers over the AHB master port.

When a channel is configured in a privileged (or unprivileged) state and in linked-list mode, the loading of the next linked-list data structure from the GPDMA memory into its register file, is automatically performed with privileged (respectively unprivileged) transfers, via the GPDMA_CxCR.LAP allocated master port.

The GPDMA generates a privileged bus that reflects GPDMA_PRIVCFGR, to keep the other peripherals informed of the privileged/unprivileged state of each GPDMA channel x.

When the privileged software must switch a channel from a privileged state to an unprivileged state, the privileged software must abort the channel or wait until that the privileged channel is completed before switching. This is needed to dynamically re-allocate a channel to a next unprivileged transfer as an unprivileged software is not allowed to do so, and must have GPDMA_CxCR.EN = 0 before the unprivileged software can reprogram the GPDMA_CxCR for a next transfer. The privileged software may reset not only the channel x (GPDMA_CxCR.RESET = 1) but also the full channel x register file to its reset value.

16.4.17 GPDMA error management

The GPDMA is able to manage and report to the user a transfer error, as follows, depending on the root cause.

Data transfer error

On a bus access (as a AHB single or a burst) to the source or the destination:

- The source or destination target reports an AHB error.
- The programmed channel transfer is stopped (GPDMA_CxCR.EN cleared by the GPDMA hardware). The channel status register reports an idle state (GPDMA_CxSR.IDLEF = 1) and the data error (GPDMA_CxSR.DTEF = 1).
- After a GPDMA data transfer error, the user must perform a debug session, taking care of the product-defined memory mapping of the source and destination, including the protection attributes.
- After a GPDMA data transfer error, the user must issue a channel reset (set GPDMA_CxCR.RESET) to reset the hardware GPDMA channel data path and the content of the FIFO, before the user enables again the same channel for a next transfer.

Link transfer error

On a tentative update of a GPDMA channel register from the programmed LLI in the memory:

- The linked-list memory reports an AHB error.
- The programmed channel transfer is stopped (GPDMA_CxCR.EN cleared by the GPDMA hardware), the channel status register reports an idle state (GPDMA_CxSR.IDLEF = 1) and the link error (GPDMA_CxSR.ULEF = 1).
- After a GPDMA link error, the user must perform a debug session, taking care of the product-defined memory mapping of the linked-list data structure (GPDMA_CxLBAR and GPDMA_CxLLR), including the protection attributes.
- After a GPDMA link error, the user must explicitly write the linked-list register file (GPDMA_CxTR1, GPDMA_CxTR2, GPDMA_CxBR1, GPDMA_CxSAR, GPDMA_CxDAR and GPDMA_CxLLR, plus GPDMA_CxTR3 and GPDMA_CxBR2), before the user enables again the same channel for a next transfer.

User setting error

On a tentative execution of a GPDMA transfer with an unauthorized user setting:

- The programmed channel transfer is disabled (GPDMA_CxCR.EN forced and cleared by the GPDMA hardware) preventing the next unauthorized programmed data transfer from being executed. The channel status register reports an idle state (GPDMA_CxSR.IDLEF = 1) and a user setting error (GPDMA_CxSR.USEF = 1).
- After a GPDMA user setting error, the user must perform a debug session, taking care of the GPDMA channel programming. A user setting error can be caused by one of the following:
 - a programmed null source block size without a programmed update of this value from the next LLI₁ (GPDMA_CxBR1.BNDT[15:0] = 0 and GPDMA_CxLLR.UB1 = 0)
 - a programmed non-null source block size being not a multiple of the programmed data width of a source burst transfer (GPDMA_CxBR1.BNDT[2:0] versus GPDMA_CxTR1.SDW_LOG2[1:0])
 - when in packing/unpacking mode (if PAM[1] = 1), a programmed non-null source block size being not a multiple of the programmed data width of a destination burst transfer (GPDMA_CxBR1.BNDT[2:0] versus GPDMA_CxTR1.DDW_LOG2[1:0])
 - a programmed unaligned source start address, being not a multiple of the programmed data width of a source burst transfer (GPDMA_CxSAR[2:0] versus GPDMA_CxTR1.SDW_LOG2[1:0])
 - for channel x (x = 6 to 7): a programmed unaligned source address offset being not a multiple of the programmed data width of a source burst transfer (GPDMA_CxTR3.SAO[2:0] versus GPDMA_CxTR1.SDW_LOG2[1:0])
 - for channel x (x = 6 to 7): a programmed unaligned block repeated source address offset being not a multiple of the programmed data width of a source burst transfer (GPDMA_CxBR2.BRSAO[2:0] versus GPDMA_CxTR1.SDW_LOG2[1:0])
 - a programmed unaligned destination start address, being not a multiple of the programmed data width of a destination burst transfer (GPDMA_CxDAR[2:0] versus GPDMA_CxTR1.DDW_LOG2[1:0])
 - for channel x (x = 6 to 7): a programmed unaligned destination address offset being not a multiple of the programmed data width of a destination burst transfer (GPDMA_CxTR3.DAO[2:0] versus GPDMA_CxTR1.DDW_LOG2[1:0])

- for channel x (x = 6 to 7): a programmed unaligned block repeated destination address offset being not a multiple of the programmed data width of a destination burst transfer (GPDMA_CxBR2.BRDAO[2:0] versus GPDMA_CxTR1.DDW_LOG2[1:0])
- a programmed double-word source data width (GPDMA_CxTR1.SDW_LOG2[1:0] = 11)
- a programmed double-word destination data width (GPDMA_CxTR1.DDW_LOG2[1:0] = 11)
- a programmed linked-list item LLI_{n+1} with a null data transfer (GPDMA_CxLLR.UB1 = 1 and GPDMA_CxBR1.BNDT = 0)

16.5 GPDMA in debug mode

When the microcontroller enters debug mode (core halted), any channel x can be individually either continued (default) or suspended, depending on the programmable control bit in the DBGMCU module.

Note: In debug mode, GPDMA_CxSR.SUSPF is not altered by a suspension from the programmable control bit in the DBGMCU module. In this case, GPDMA_CxSR.IDLEF can be checked to know the completion status of the channel suspension.

16.6 GPDMA in low-power modes

Table 149. Effect of low-power modes on GPDMA

Mode	Description
Sleep	No effect. GPDMA interrupts cause the device to exit Sleep mode.
Stop ⁽¹⁾	The content of the GPDMA registers is kept when entering Stop mode.
Standby	The GPDMA is powered down and must be reinitialized after exiting Standby mode.

1. Refer to [Section 16.3.3](#) to know if any Stop mode is supported.

16.7 GPDMA interrupts

There is one GPDMA interrupt line for each channel, and separately for each CPU (if several ones in the devices).

Table 150. GPDMA interrupt requests

Interrupt acronym	Interrupt event	Interrupt enable	Event flag	Event clear method
GPDMA_CHx	Transfer complete	GPDMA_CxCR.TCIE	GPDMA_CxSR.TCF	Write 1 to GPDMA_CxFCR.TCF
	Half transfer	GPDMA_CxCR.HTIE	GPDMA_CxSR.HTF	Write 1 to GPDMA_CxFCR.HTF
	Data transfer error	GPDMA_CxCR.DTEIE	GPDMA_CxSR.DTEF	Write 1 to GPDMA_CxFCR.DTEF
	Update link error	GPDMA_CxCR.ULEIE	GPDMA_CxSR.ULEF	Write 1 to GPDMA_CxFCR.ULEF
	User setting error	GPDMA_CxCR.USEIE	GPDMA_CxSR.USEF	Write 1 to GPDMA_CxFCR.USEF
	Suspended	GPDMA_CxCR.SUSPIE	GPDMA_CxSR.SUSPF	Write 1 to GPDMA_CxFCR.SUSPF
	Trigger overrun	GPDMA_CxCR.TOFIE	GPDMA_CxSR.TOF	Write 1 to GPDMA_CxFCR.TOF

A GPDMA channel x event may be:

- a transfer complete
- a half-transfer complete
- a transfer error, due to either:
 - a data transfer error
 - an update link error
 - a user setting error completed suspension
- a trigger overrun

Note: When a channel x transfer complete event occurs, the output signal *gpdma_chx_tc* is generated as a high pulse of one clock cycle.

An interrupt is generated following any xx event, provided that both:

- the corresponding interrupt event xx is enabled (GPDMA_CxCR.xxIE = 1)
- the corresponding event flag is cleared (GPDMA_CxSR.xxF = 0). This means that, after a previous same xx event occurrence, a software agent must have written 1 into the corresponding xx flag clear control bit (write 1 into GPDMA_CxFCR.xxF).

TCF (transfer complete) and HTF (half transfer) events generation is controlled by GPDMA_CxTR2.TCEM[1:0] as follows:

- A transfer complete event is a block transfer complete, a 2D/repeated block transfer complete, or a LLI transfer complete including the upload of the next LLI if any, or the full linked-list completion, depending on the transfer complete event mode GPDMA_CxTR2.TCEM[1:0].

- A half-transfer event is a half-block transfer or a half 2D/repeated block transfer, depending on the transfer complete event mode GPDMA_CxTR2.TCEM[1:0].
A half-block transfer occurs when half of the source block size bytes (rounded-up integer of $\text{GPDMA_CxBR1.BNDT}[15:0] / 2$) is transferred to the destination.
A half 2D/repeated block transfer occurs when half of the repeated blocks (rounded-up integer of $(\text{GPDMA_CxBR1.BRC}[10:0] + 1) / 2$) is transferred to the destination.

See [GPDMA channel x transfer register 2 \(GPDMA_CxTR2\)](#) for more details.

A transfer error rises in one of the following situations:

- during a single/burst data transfer from the source or to the destination (DTEF)
- during an update of a GPDMA channel register from the programmed LLI in memory (ULEF)
- during a tentative execution of a GPDMA channel with an unauthorized setting (USEF)
The user must perform a debug session to correct the GPDMA channel programming versus the USEF root causes list (see [Section 16.4.17](#)).

A trigger overrun is described in [Trigger hit memorization and trigger overrun flag generation](#).

16.8 GPDMA registers

The GPDMA registers must be accessed with an aligned 32-bit word data access.

16.8.1 GPDMA secure configuration register (GPDMA_SECCFGR)

Address offset: 0x00

Reset value: 0x0000 0000

A write access to this register must be secure and privileged. A read access is secure or nonsecure, privileged or unprivileged.

A write access is ignored at bit level if the corresponding channel x is locked (GPDMA_RCFGLOCKR.LOCKx = 1).

This register must be written when GPDMA_CxCR.EN = 0.

This register is read-only when GPDMA_CxCR.EN = 1.

This register must be programmed at a bit level, at the initialization/closure of a GPDMA channel (when GPDMA_CxCR.EN = 0), to securely allocate individually any channel x to the secure or nonsecure world.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	SEC7	SEC6	SEC5	SEC4	SEC3	SEC2	SEC1	SEC0
								rw	rw	rw	rw	rw	rw	rw	rw

Bits 31:8 Reserved, must be kept at reset value.

Bits 7:0 **SECx**: secure state of channel x (x = 7 to 0)

0: Nonsecure

1: Secure

16.8.2 GPDMA privileged configuration register (GPDMA_PRIVCFGR)

Address offset: 0x04

Reset value: 0x0000 0000

A write access to this register must be privileged. A read access can be privileged or unprivileged, secure or nonsecure.

This register can mix secure and nonsecure information. If a channel x is configured as secure (GPDMA_SECCFGR.SECx = 1), the PRIVx bit can be written only by a secure (and privileged) agent.

A write access is ignored at bit level if the corresponding channel x is locked (GPDMA_RCFGLOCKR.LOCKx = 1).

This register must be written when GPDMA_CxCR.EN = 0.

This register is read-only when GPDMA_CxCR.EN = 1.

This register must be programmed at a bit level, at the initialization/closure of a GPDMA channel (GPDMA_CxCR.EN = 0), to individually allocate any channel x to the privileged or unprivileged world.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	PRIV7	PRIV6	PRIV5	PRIV4	PRIV3	PRIV2	PRIV1	PRIV0
								rw	rw	rw	rw	rw	rw	rw	rw

Bits 31:8 Reserved, must be kept at reset value.

Bits 7:0 **PRIVx**: privileged state of channel x (x = 7 to 0)

0: unprivileged

1: privileged

16.8.3 GPDMA configuration lock register (GPDMA_RCFGLOCKR)

Address offset: 0x08

Reset value: 0x0000 0000

This register can be written by a software agent with secure privileged attributes in order to individually lock, for example at boot time, the secure privileged attributes of any GPDMA

channel/resource (to lock the setting of GPDMA_CxSECCFGR, GPDMA_CxPRIVCFGR for any channel x, for example at boot time).

A read access may be privileged or unprivileged, secure or nonsecure.

Note: If $TZEN = 0$, this register cannot be written.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	LOCK7	LOCK6	LOCK5	LOCK4	LOCK3	LOCK2	LOCK1	LOCK0
								rs	rs	rs	rs	rs	rs	rs	rs

Bits 31:8 Reserved, must be kept at reset value.

Bits 7:0 **LOCKx**: lock the configuration of GPDMA_SECCFGR.SECx, GPDMA_PRIVCFGR.PRIVx until a global GPDMA reset ($x = 7$ to 0)

This bit is cleared after reset and, once set, it cannot be reset until a global GPDMA reset.

0: secure privilege configuration of the channel x is writable.

1: secure privilege configuration of the channel x is not writable.

16.8.4 GPDMA nonsecure masked interrupt status register (GPDMA_MISR)

Address offset: 0x0C

Reset value: 0x0000 0000

This register is a read register.

This is a nonsecure register, containing the masked interrupt status bit MISx for each nonsecure channel x (channel x configured with GPDMA_SECCFGR.SECx = 0). It is a logical OR of all the flags of GPDMA_CxSR, each source flag being enabled by the corresponding interrupt enable bit of GPDMA_CxCR.

Every bit is deasserted by hardware when writing 1 to the corresponding flag clear bit in GPDMA_CxFCR.

If a channel x is in secure state (GPDMA_SECCFGR.SECx = 1), a read access to the masked interrupt status bit MISx of this channel x returns zero.

This register may mix privileged and unprivileged information, depending on the privileged state of each channel GPDMA_PRIVCFGR.PRIVx. A privileged software can read the full nonsecure interrupt status. An unprivileged software is restricted to read the status of unprivileged (and nonsecure) channels, other privileged bit fields returning zero.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	MIS7	MIS6	MIS5	MIS4	MIS3	MIS2	MIS1	MIS0
								r	r	r	r	r	r	r	r

Bits 31:8 Reserved, must be kept at reset value.

Bits 7:0 **MISx**: masked interrupt status of channel x (x = 7 to 0)

0: no interrupt occurred on channel x

1: an interrupt occurred on channel x

16.8.5 GPDMA secure masked interrupt status register (GPDMA_SMISR)

Address offset: 0x10

Reset value: 0x0000 0000

This is a secure read register, containing the masked interrupt status bit MISx for each secure channel x (GPDMA_SECCFGR.SECx = 1). It is a logical OR of all the GPDMA_CxSR flags, each source flag being enabled by the corresponding GPDMA_CxCR interrupt enable bit.

Every bit is deasserted by hardware when securely writing 1 to the corresponding GPDMA_CxFCR flag clear bit.

This register does not contain any information about a nonsecure channel.

This register can mix privileged and unprivileged information, depending on the privileged state of each channel GPDMA_PRIVCFGR.PRIVx. A privileged software can read the full secure interrupt status. An unprivileged software is restricted to read the status of unprivileged and secure channels, other privileged bit fields returning zero.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	MIS7	MIS6	MIS5	MIS4	MIS3	MIS2	MIS1	MIS0
								r	r	r	r	r	r	r	r

Bits 31:8 Reserved, must be kept at reset value.

Bits 7:0 **MISx**: masked interrupt status of the secure channel x (x = 7 to 0)

0: no interrupt occurred on the secure channel x

1: an interrupt occurred on the secure channel x

16.8.6 GPDMA channel x linked-list base address register (GPDMA_CxLBAR)

Address offset: 0x50 + 0x80 * x (x = 0 to 7)

Reset value: 0x0000 0000

This register must be written by a privileged software. It is either privileged readable or not, depending on the privileged state of the channel x GPDMA_PRIVCFGR.PRIVx.

This register is either secure or nonsecure depending on the secure state of the channel x (GPDMA_SECCFGR.SECx).

This register must be written when GPDMA_CxCR.EN = 0.

This register is read-only when GPDMA_CxCR.EN = 1.

This channel-based register is the linked-list base address of the memory region, for a given channel x, from which the LLIs describing the programmed sequence of the GPDMA transfers, are conditionally and automatically updated.

This 64-Kbyte aligned channel x linked-list base address is offset by the 16-bit GPDMA_CxLLR register that defines the word-aligned address offset for each LLI.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
LBA[31:16]															
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.

Bits 31:16 **LBA[31:16]**: linked-list base address of GPDMA channel x

Bits 15:0 Reserved, must be kept at reset value.

16.8.7 GPDMA channel x flag clear register (GPDMA_CxFCR)

Address offset: 0x5C+ 0x80 * x (x = 0 to 7)

Reset value: 0x0000 0000

This is a write register, secure or nonsecure depending on the secure state of channel x (GPDMA_SECCFGR.SECx) and privileged or unprivileged, depending on the privileged state of the channel x (GPDMA_PRIVCFGR.PRIVx).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	TOF	SUSPF	USEF	ULEF	DTEF	HTF	TCF	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
	w	w	w	w	w	w	w								

Bits 31:15 Reserved, must be kept at reset value.

Bit 14 **TOF**: trigger overrun flag clear

0: no effect

1: corresponding TOF flag cleared

Bit 13 **SUSPF**: completed suspension flag clear

0: no effect

1: corresponding SUSPF flag cleared

Bit 12 **USEF**: user setting error flag clear

0: no effect

1: corresponding USEF flag cleared

Bit 11 **ULEF**: update link transfer error flag clear

0: no effect

1: corresponding ULEF flag cleared

Bit 10 **DTEF**: data transfer error flag clear
 0: no effect
 1: corresponding DTEF flag cleared

Bit 9 **HTF**: half-transfer flag clear
 0: no effect
 1: corresponding HTF flag cleared

Bit 8 **TCF**: transfer complete flag clear
 0: no effect
 1: corresponding TCF flag cleared

Bits 7:0 Reserved, must be kept at reset value.

16.8.8 GPDMA channel x status register (GPDMA_CxSR)

Address offset: $0x60 + 0x80 * x$ ($x = 0$ to 7)

Reset value: 0x0000 0001

This is a read register, reporting the channel status.

This register is secure or nonsecure, depending on the secure state of channel x (GPDMA_SECCFGR.SECx), and privileged or unprivileged, depending on the privileged state of the channel (GPDMA_PRIVCFGR.PRIVx).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	FIFOL[7:0]							
								r	r	r	r	r	r	r	r
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	TOF	SUSPF	USEF	ULEF	DTEF	HTF	TCF	Res.	Res.	Res.	Res.	Res.	Res.	Res.	IDLEF
	r	r	r	r	r	r	r								r

Bits 31:24 Reserved, must be kept at reset value.

Bits 23:16 **FIFOL[7:0]**: monitored FIFO level

Number of available write beats in the FIFO, in units of the programmed destination data width (see GPDMA_CxTR1.DDW_LOG2[1:0], in units of bytes, half-words, or words).

Note: After having suspended an active transfer, the user may need to read FIFOL[7:0], additionally to GPDMA_CxBR1.BDNT[15:0] and GPDMA_CxBR1.BRC[10:0], to know how many data have been transferred to the destination. Before reading, the user may wait for the transfer to be suspended (GPDMA_CxSR.SUSPF = 1).

Bit 15 Reserved, must be kept at reset value.

Bit 14 **TOF**: trigger overrun flag
 0: no trigger overrun event
 1: a trigger overrun event occurred

Bit 13 **SUSPF**: completed suspension flag
 0: no completed suspension event
 1: a completed suspension event occurred

Bit 12 **USEF**: user setting error flag
 0: no user setting error event
 1: a user setting error event occurred

Bit 11 **ULEF**: update link transfer error flag
 0: no update link transfer error event
 1: a master bus error event occurred while updating a linked-list register from memory

Bit 10 **DTEF**: data transfer error flag
 0: no data transfer error event
 1: a master bus error event occurred on a data transfer

Bit 9 **HTF**: half-transfer flag
 0: no half-transfer event
 1: a half-transfer event occurred
 A half-transfer event is either a half-block transfer or a half 2D/repeated block transfer, depending on the transfer complete event mode (GPDMA_CxTR2.TCEM[1:0]).
 A half-block transfer occurs when half of the bytes of the source block size (rounded up integer of GPDMA_CxBR1.BNDT[15:0]/2) has been transferred to the destination.
 A half 2D/repeated block transfer occurs when half of the repeated blocks (rounded up integer of (GPDMA_CxBR1.BRC[10:0] + 1) / 2)) has been transferred to the destination.

Bit 8 **TCF**: transfer complete flag
 0: no transfer complete event
 1: a transfer complete event occurred
 A transfer complete event is either a block transfer complete, a 2D/repeated block transfer complete, or a LLI transfer complete including the upload of the next LLI if any, or the full linked-list completion, depending on the transfer complete event mode (GPDMA_CxTR2.TCEM[1:0]).

Bits 7:1 Reserved, must be kept at reset value.

Bit 0 **IDLEF**: idle flag
 0: channel not in idle state
 1: channel in idle state
 This idle flag is deasserted by hardware when the channel is enabled (GPDMA_CxCR.EN = 1) with a valid channel configuration (no USEF to be immediately reported).
 This idle flag is asserted after hard reset or by hardware when the channel is back in idle state (in suspended or disabled state).

16.8.9 GPDMA channel x control register (GPDMA_CxCR)

Address offset: $0x64 + 0x80 * x$ ($x = 0$ to 7)

Reset value: 0x0000 0000

This register is secure or nonsecure depending on the secure state of channel x (GPDMA_SECCFGR.SECx), and privileged or unprivileged, depending on the privileged state of the channel x (GPDMA_PRIVCFGR.PRIVx).

This register is used to control a channel (activate, suspend, abort or disable it).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	PRIO[1:0]		Res.	Res.	Res.	Res.	LAP	LSM
								rw	rw					rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	TOIE	SUSPIE	USEIE	ULEIE	DTEIE	HTIE	TCIE	Res.	Res.	Res.	Res.	Res.	SUSP	RESET	EN
	rw	rw	rw	rw	rw	rw	rw						rw	w	rw

Bits 31:24 Reserved, must be kept at reset value.

Bits 23:22 **PRIO[1:0]**: priority level of the channel x GPDMA transfer versus others

- 00: low priority, low weight
- 01: low priority, mid weight
- 10: low priority, high weight
- 11: high priority

Note: This bit must be written when EN = 0. This bit is read-only when EN = 1.

Bits 21:18 Reserved, must be kept at reset value.

Bit 17 **LAP**: linked-list allocated port

This bit is used to allocate the master port for the update of the GPDMA linked-list registers from the memory.

- 0: port 0 (AHB) allocated
- 1: port 1 (AHB) allocated

Note: This bit must be written when EN = 0. This bit is read-only when EN = 1.

Bit 16 **LSM**: Link step mode

0: channel executed for the full linked-list and completed at the end of the last LLI (GPDMA_CxLLR = 0). The 16 low-significant bits of the link address are null (LA[15:0] = 0) and all the update bits are null (UT1 = UB1 = UT2 = USA = UDA = ULL = 0 and UT3 = UB2 = 0). Then GPDMA_CxBR1.BNDT[15:0] = 0 and GPDMA_CxBR1.BRC[10:0] = 0.

1: channel executed **once** for the current LLI

First the (possible 1D/repeated) block transfer is executed as defined by the current internal register file until GPDMA_CxBR1.BNDT[15:0] = 0 and GPDMA_CxBR1.BRC[10:0] = 0.

Secondly the next linked-list data structure is conditionally uploaded from memory as defined by GPDMA_CxLLR. Then channel execution is completed.

Note: This bit must be written when EN = 0. This bit is read-only when EN = 1.

Bit 15 Reserved, must be kept at reset value.

Bit 14 **TOIE**: trigger overrun interrupt enable

- 0: interrupt disabled
- 1: interrupt enabled

Bit 13 **SUSPIE**: completed suspension interrupt enable

- 0: interrupt disabled
- 1: interrupt enabled

Bit 12 **USEIE**: user setting error interrupt enable

- 0: interrupt disabled
- 1: interrupt enabled

Bit 11 **ULEIE**: update link transfer error interrupt enable

- 0: interrupt disabled
- 1: interrupt enabled

Bit 10 **DTEIE**: data transfer error interrupt enable

- 0: interrupt disabled
- 1: interrupt enabled

Bit 9 **HTIE**: half-transfer complete interrupt enable

- 0: interrupt disabled
- 1: interrupt enabled

Bit 8 **TCIE**: transfer complete interrupt enable

0: interrupt disabled

1: interrupt enabled

Bits 7:3 Reserved, must be kept at reset value.

Bit 2 **SUSP**: suspend

Writing 1 into the field RESET (bit 1) causes the hardware to de-assert this bit, whatever is written into this bit 2. Else:

Software must write 1 in order to suspend an active channel (channel with an ongoing GPDMA transfer over its master ports).

The software must write 0 in order to resume a suspended channel, following the programming sequence detailed in [Figure 77](#).

0: write: resume channel, read: channel not suspended

1: write: suspend channel, read: channel suspended.

Bit 1 **RESET**: reset

This bit is write only. Writing 0 has no impact. Writing 1 implies the reset of the following: the FIFO, the channel internal state, SUSP and EN bits (whatever is written receptively in bit 2 and bit 0).

The reset is effective when the channel is in steady state, meaning one of the following:

- active channel in suspended state (GPDMA_CxSR.SUSPF = 1 and GPDMA_CxSR.IDLEF = GPDMA_CxCR.EN = 1)

- channel in disabled state (GPDMA_CxSR.IDLEF = 1 and GPDMA_CxCR.EN = 0).

After writing a RESET, to continue using this channel, the user must explicitly reconfigure the channel including the hardware-modified configuration registers (GPDMA_CxBR1, GPDMA_CxSAR, and GPDMA_CxDAR) before enabling again the channel (see the programming sequence in [Figure 78](#)).

0: no channel reset

1: channel reset

Bit 0 **EN**: enable

Writing 1 into the field RESET (bit 1) causes the hardware to de-assert this bit, whatever is written into this bit 0. Else:

this bit is deasserted by hardware when there is a transfer error (master bus error or user setting error) or when there is a channel transfer complete (channel ready to be configured, for example if LSM = 1 at the end of a single execution of the LLI).

Else, this bit can be asserted by software.

Writing 0 into this EN bit is ignored.

0: write: ignored, read: channel disabled

1: write: enable channel, read: channel enabled

16.8.10 GPDMA channel x transfer register 1 (GPDMA_CxTR1)

Address offset: $0x90 + 0x80 * x$ ($x = 0$ to 7)

Reset value: 0x0000 0000

This register is secure or nonsecure depending on the secure state of channel x (GPDMA_SECCFGR.SECx) except for secure DSEC and SSEC, privileged or unprivileged, depending on the privileged state of the channel x in GPDMA_PRIVCFGR.PRIVx.

This register controls the transfer of a channel x.

This register must be written when GPDMA_CxCR.EN = 0.

This register is read-only when GPDMA_CxCR.EN = 1.

This register must be written when the channel is completed. Then the hardware has deasserted GPDMA_CxCR.EN). A channel transfer can be completed and programmed at different levels: block, 2D/repeated block, LLI or full linked-list.

In linked-list mode, during the link transfer, this register is automatically updated by GPDMA from the memory if GPDMA_CxLLR.UT1 = 1.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
DSEC	DAP	Res.	Res.	DHX	DBX	DBL_1[5:0]						DINC	Res.	DDW_LOG2[1:0]	
rw	rw			rw	rw	rw	rw	rw	rw	rw	rw	rw		rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
SSEC	SAP	SBX	PAM[1:0]		Res.	SBL_1[5:0]						SINC	Res.	SDW_LOG2[1:0]	
rw	rw	rw	rw	rw		rw	rw	rw	rw	rw	rw	rw		rw	rw

Bit 31 **DSEC**: security attribute of the GPDMA transfer to the destination

If GPDMA_SECCFGR.SECx = 1 and the access is secure:

0: GPDMA transfer nonsecure

1: GPDMA transfer secure

This is a secure register bit. This bit can only be read by a secure software. This bit must be written by a secure software when GPDMA_SECCFGR.SECx = 1. A secure write is ignored when GPDMA_SECCFGR.SECx = 0.

When GPDMA_SECCFGR.SECx is deasserted, this DSEC bit is also deasserted by hardware (on a secure reconfiguration of the channel as nonsecure), and the GPDMA transfer to the destination is nonsecure.

Bit 30 **DAP**: destination allocated port

This bit is used to allocate the master port for the destination transfer

0: port 0 (AHB) allocated

1: port 1 (AHB) allocated

Note: This bit must be written when EN = 0. This bit is read-only when EN = 1.

Bits 29:28 Reserved, must be kept at reset value.

Bit 27 **DHX**: destination half-word exchange

If the destination data size is shorter than a word, this bit is ignored.

If the destination data size is a word:

0: no halfword-based exchanged within word

1: the two consecutive (post PAM) half-words are exchanged in each destination word.

Bit 26 **DBX**: destination byte exchange

If the destination data size is a byte, this bit is ignored.

If the destination data size is not a byte:

0: no byte-based exchange within half-word

1: the two consecutive (post PAM) bytes are exchanged in each destination half-word.

Bits 25:20 **DBL_1[5:0]**: destination burst length minus 1, between 0 and 63

The burst length unit is one data named beat within a burst. If **DBL_1[5:0]** = 0, the burst can be named as single. Each data/beat has a width defined by the destination data width **DDW_LOG2[1:0]**.

Note: If a burst transfer crossed a 1-Kbyte address boundary on a AHB transfer, the GPDMA modifies and shortens the programmed burst into singles or bursts of lower length, to be compliant with the AHB protocol.

If a burst transfer is of length greater than the FIFO size of the channel x, the GPDMA modifies and shortens the programmed burst into singles or bursts of lower length, to be compliant with the FIFO size. Transfer performance is lower, with GPDMA re-arbitration between effective and lower singles/bursts, but the data integrity is guaranteed.

Bit 19 **DINC**: destination incrementing burst

0: fixed burst

1: contiguously incremented burst

The destination address, pointed by **GPDMA_CxDAR**, is kept constant after a burst beat/single transfer, or is incremented by the offset value corresponding to a contiguous data after a burst beat/single transfer.

Bit 18 Reserved, must be kept at reset value.

Bits 17:16 **DDW_LOG2[1:0]**: binary logarithm of the destination data width of a burst, in bytes

00: byte

01: half-word (2 bytes)

10: word (4 bytes)

11: user setting error reported and no transfer issued

*Note: A destination burst transfer must have an aligned address with its data width (start address **GPDMA_CxDAR[2:0]** and if present address offset **GPDMA_CxTR3.DAO[2:0]**, versus **DDW_LOG2[1:0]**). Otherwise a user setting error is reported and no transfer is issued.*

*When configured in packing mode (**PAM[1]** = 1 and destination data width different from source data width), a source block size must be a multiple of the destination data width (see **GPDMA_CxBR1.BNDT[2:0]** versus **DDW_LOG2[1:0]**). Else a user setting error is reported and none transfer is issued.*

Setting a 8-byte data width causes a user setting error to be reported and none transfer is issued.

Bit 15 **SSEC**: security attribute of the GPDMA transfer from the source

If **GPDMA_SECCFGR.SECx** = 1 and the access is secure:

0: GPDMA transfer nonsecure

1: GPDMA transfer secure

This is a secure register bit. This bit can only be read by a secure software. This bit must be written by a secure software when **GPDMA_SECCFGR.SECx** = 1. A secure write is ignored when **GPDMA_SECCFGR.SECx** = 0.

When **GPDMA_SECCFGR.SECx** is deasserted, this **SSEC** bit is also deasserted by hardware (on a secure reconfiguration of the channel as nonsecure), and the GPDMA transfer from the source is nonsecure.

Bit 14 **SAP**: source allocated port

This bit is used to allocate the master port for the source transfer

0: port 0 (AHB) allocated

1: port 1 (AHB) allocated

*Note: This bit must be written when **EN** = 0. This bit is read-only when **EN** = 1.*

Bit 13 **SBX**: source byte exchange within the unaligned half-word of each source word
 If the source data width is shorter than a word, this bit is ignored.
 If the source data width is a word:
 0: no byte-based exchange within the unaligned half-word of each source word
 1: the two consecutive bytes within the unaligned half-word of each source word are exchanged.

Bits 12:11 **PAM[1:0]**: padding/alignment mode

If $DDW_LOG2[1:0] = SDW_LOG2[1:0]$: if the data width of a burst destination transfer is equal to the data width of a burst source transfer, these bits are ignored.

Else, in the following enumerated values, the condition PAM_1 is when destination data width is higher than source data width, and the condition PAM_2 is when source data width is higher than destination data width.

Condition: PAM_1

00: source data is transferred as right aligned, padded with 0s up to the destination data width

01: source data is transferred as right aligned, sign extended up to the destination data width

10-11: successive source data are FIFO queued and packed at the destination data width, in a left (LSB) to right (MSB) order (named little endian), before a destination transfer

Condition: PAM_2

00: source data is transferred as right aligned, left-truncated down to the destination data width

01: source data is transferred as left-aligned, right-truncated down to the destination data width

10-11: source data is FIFO queued and unpacked at the destination data width, to be transferred in a left (LSB) to right (MSB) order (named little endian) to the destination

Note: If the transfer from the source peripheral is configured with peripheral flow-control mode ($SWREQ = 0$ and $PFREQ = 1$ and $DREQ = 0$), and if the destination data width > the source data width, packing is not supported.

Bit 10 Reserved, must be kept at reset value.

Bits 9:4 **SBL_1[5:0]**: source burst length minus 1, between 0 and 63

The burst length unit is one data named beat within a burst. If $SBL_1[5:0] = 0$, the burst can be named as single. Each data/beat has a width defined by the destination data width $SDW_LOG2[1:0]$.

Note: If a burst transfer crossed a 1-Kbyte address boundary on a AHB transfer, the GPDMA modifies and shortens the programmed burst into singles or bursts of lower length, to be compliant with the AHB protocol.

If a burst transfer is of length greater than the FIFO size of the channel x, the GPDMA modifies and shortens the programmed burst into singles or bursts of lower length, to be compliant with the FIFO size. Transfer performance is lower, with GPDMA re-arbitration between effective and lower singles/bursts, but the data integrity is guaranteed.

Bit 3 **SINC**: source incrementing burst

0: fixed burst

1: contiguously incremented burst

The source address, pointed by GPDMA_CxSAR, is kept constant after a burst beat/single transfer or is incremented by the offset value corresponding to a contiguous data after a burst beat/single transfer.

Bit 2 Reserved, must be kept at reset value.

Bits 1:0 **SDW_LOG2[1:0]**: binary logarithm of the source data width of a burst in bytes

00: byte

01: half-word (2 bytes)

10: word (4 bytes)

11: user setting error reported and no transfer issued

Note: A source block size must be a multiple of the source data width (GPDMA_CxBR1.BNDT[2:0] versus SDW_LOG2[1:0]). Otherwise, a user setting error is reported and no transfer is issued.

A source burst transfer must have an aligned address with its data width (start address GPDMA_CxSAR[2:0] versus SDW_LOG2[1:0]). Otherwise, a user setting error is reported and none transfer is issued.

Setting a 8-byte data width causes a user setting error to be reported and no transfer is issued.

16.8.11 GPDMA channel x transfer register 2 (GPDMA_CxTR2)

Address offset: $0x94 + 0x80 * x$ ($x = 0$ to 7)

Reset value: 0x0000 0000

This register is secure or nonsecure depending on the secure state of channel x (GPDMA_SECCFGR.SECx), and privileged or unprivileged, depending on the privileged state of channel x (GPDMA_PRIVCFGR.PRIVx).

This register controls the transfer of a channel x. This register must be written when GPDMA_CxCR.EN = 0.

This register is read-only when GPDMA_CxCR.EN = 1.

This register must be written when the channel is completed (the hardware deasserted GPDMA_CxCR.EN). A channel transfer can be completed and programmed at different levels: block or LLI or full linked-list.

In linked-list mode, during the link transfer, this register is automatically updated by GPDMA from the memory, if GPDMA_CxLLR.UT2 = 1.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
TCM[1:0]		Res.	Res.	Res.	Res.	TRIGPOL[1:0]		Res.	Res.	TRIGSEL[5:0]					
rw	rw					rw	rw			rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
TRIGM[1:0]		Res.	PFREQ	BREQ	DREQ	SWREQ	Q	Res.	REQSEL[7:0]						
rw	rw		rw	rw	rw	rw		rw	rw	rw	rw	rw	rw	rw	rw

Bits 31:30 **TCEM[1:0]**: transfer complete event mode

These bits define the transfer granularity for the transfer complete and half-transfer complete events generation.

00: at block level (when GPDMA_CxBR1.BNDT[15:0] = 0): the complete (and the half) transfer event is generated at the (respectively half of the) end of a block.

Note: If the initial LLI₀ data transfer is null/void (directly programmed by the internal register file with GPDMA_CxBR1.BNDT[15:0] = 0), then neither the complete transfer event nor the half-transfer event is generated.

01: channel x (x = 0 to 5), same as 00, channel x (x = 6 to 7), at 2D/repeated block level (when GPDMA_CxBR1.BRC[10:0] = 0 and GPDMA_CxBR1.BNDT[15:0] = 0). The complete (and the half) transfer event is generated at the end (respectively half of the end) of the 2D/repeated block.

Note: If the initial LLI₀ data transfer is null/void (directly programmed by the internal register file with GPDMA_CxBR1.BNDT[15:0] = 0), then neither the complete transfer event nor the half-transfer event is generated.

10: at LLI level: the complete transfer event is generated at the end of the LLI transfer, including the update of the LLI if any. The half-transfer event is generated at the half of the LLI data transfer. The LLI data transfer is a block transfer or a 2D/repeated block transfer for channel x (x = 6 to 7), if any data transfer.

Note: If the initial LLI₀ data transfer is null/void (directly programmed by the internal register file with GPDMA_CxBR1.BNDT[15:0] = 0), then the half-transfer event is not generated, and the transfer complete event is generated when is completed the loading of the LLI₁.

11: at channel level: the complete transfer event is generated at the end of the last LLI transfer. The half-transfer event is generated at the half of the data transfer of the last LLI. The last LLI updates the link address GPDMA_CxLLR.LA[15:2] to zero and clears all the GPDMA_CxLLR update bits (UT1, UT2, UB1, USA, UDA and ULL, plus UT3 and UB2). If the channel transfer is continuous/infinite, no event is generated.

Bits 29:26 Reserved, must be kept at reset value.

Bits 25:24 **TRIGPOL[1:0]**: trigger event polarity

These bits define the polarity of the selected trigger event input defined by TRIGSEL[5:0].

00: no trigger (masked trigger event)

01: trigger on the rising edge

10: trigger on the falling edge

11: same as 00

Bits 23:22 Reserved, must be kept at reset value.

Bits 21:16 **TRIGSEL[5:0]**: trigger event input selection

These bits select the trigger event input of the GPDMA transfer (as per [Section 16.3.7](#)), with an active trigger event if TRIGPOL[1:0] ≠ 00.

Bits 15:14 **TRIGM[1:0]**: trigger mode

These bits define the transfer granularity for its conditioning by the trigger.

If the channel x is enabled (GPDMA_CxCR.EN asserted) with TRIGPOL[1:0] = 00 or 11, these TRIGM[1:0] bits are ignored.

Else, a GPDMA transfer is conditioned by one trigger hit:

00: at block level: the first burst read of each block transfer is conditioned by one hit trigger (channel x ($x = 6$ to 7): same, each block also if a 2D/repeated block is configured with GPDMA_CxBR1.BRC[10:0] $\neq 0$).

01: channel x ($x = 0$ to 5): same as 00; channel x ($x = 6$ to 7): at 2D/repeated block level: the first burst read of the 2D/repeated block transfer is conditioned by one hit trigger.

10: at link level: a LLI link transfer is conditioned by one hit trigger. LLI data transfer is not conditioned.

11: at programmed burst level: If SWREQ = 1, each programmed burst read is conditioned by one hit trigger. If SWREQ = 0, each programmed burst that is requested by the selected peripheral, is conditioned by one hit trigger:

– If the peripheral is the source (DREQ = 0) of the LLI data transfer, each programmed burst read is conditioned.

– If the peripheral is the destination (DREQ = 1) of the LLI data transfer, each programmed burst write is conditioned. The first memory burst read of a (possibly 2D/repeated) block, also named as the first ready FIFO-based source burst, is gated by the occurrence of both the hardware request and one trigger hit.

As shown in [Figure 93: Trigger hit, memorization, and overrun waveform](#), the GPDMA monitoring of a trigger for channel x is started when the channel is enabled/loaded with a new active trigger configuration: rising or falling edge on a selected trigger (TRIGPOL[1:0] = 01 or respectively TRIGPOL[1:0] = 10). The monitoring of this trigger is kept active during the triggered and uncompleted (data or link) transfer; and if a new trigger is detected then, this hit is internally memorized to grant the next transfer, as long as the defined rising or falling edge is not modified, and the TRIGSEL[5:0] is not modified, and the channel is enabled. After a first new trigger hit _{$n+1$} is memorized, if another second trigger hit _{$n+2$} is detected and if the hit _{n} triggered transfer is still not completed, hit _{$n+2$} is lost and not memorized. A trigger overrun flag is reported (GPDMA_CxSR.TOF = 1), and an interrupt is generated if enabled (GPDMA_CxCR.TOIE = 1). The channel is not automatically disabled by hardware due to a trigger overrun.

Transferring a next LLI _{$n+1$} that updates the GPDMA_CxTR2 with a new value for any of TRIGSEL[5:0] or TRIGPOL[1:0], resets the monitoring, trashing the memorized hit of the formerly defined LLI _{n} trigger.

Note: When the source block size is not a multiple of the source burst size and is a multiple of the source data width, then the last programmed source burst is not completed and is internally shorten to match the block size. In this case, if TRIGM[1:0] = 11 and (SWREQ = 1 or (SWREQ = 0 and DREQ = 0)), the shortened burst transfer (by singles or/and by bursts of lower length) is conditioned once by the trigger.

When the programmed destination burst is internally shortened by singles or/and by bursts of lower length (versus FIFO size, versus block size, 1-Kbyte boundary address crossing): if the trigger is conditioning the programmed destination burst (if TRIGM[1:0] = 11 and SWREQ = 0 and DREQ = 1), this shortened destination burst transfer is conditioned once by the trigger.

Bit 13 Reserved, must be kept at reset value.

Bit 12 **PFREQ**: Hardware request in peripheral flow control mode

Important: If a given channel x is not implemented with this feature, this bit is reserved and PFREQ is not present (see [Section 16.3.2](#) for the list of the implemented channels with this feature).

If the channel x is activated (GPDMA_CxCR.EN asserted) with SWREQ = 1 (software request for a memory-to-memory transfer), this bit is ignored. Else:

0: the selected hardware request is driven by a peripheral with a hardware request/acknowledge protocol in GPDMA control mode. The GPDMA is programmed with GPDMA_CxTR1.BNDT[15:0] and this is internally used by the hardware for the block transfer completion.

1: the selected hardware request is driven by a peripheral with a hardware request/acknowledge protocol in peripheral control mode. The GPDMA block transfer can be early completed by the peripheral itself (see [Section 16.3.6](#) for more details).

Note: In peripheral flow control mode, there are the following restrictions:

- no 2D/repeated block support (GPDMA_CxBR1.BRC[10:0] must be set to 0)
- the peripheral must be set as the source of the transfer (DREQ = 0).
- data packing to a wider destination width is not supported (if destination width > source data width, GPDMA_CxTR1.PAM[1] must be set to 0).
- GPDMA_CxBR1.BNDT[15:0] must be programmed as a multiple of the source (peripheral) burst size.

Bit 11 **BREQ**: Block hardware request

If the channel x is activated (GPDMA_CxCR.EN asserted) with SWREQ = 1 (software request for a memory-to-memory transfer), this bit is ignored. Else:

0: the selected hardware request is driven by a peripheral with a hardware request/acknowledge protocol at a burst level.

1: the selected hardware request is driven by a peripheral with a hardware request/acknowledge protocol at a block level (see [Section 16.3.4](#)).

Bit 10 **DREQ**: destination hardware request

This bit is ignored if channel x is activated (GPDMA_CxCR.EN asserted) with SWREQ = 1 (software request for a memory-to-memory transfer). Else:

0: selected hardware request driven by a source peripheral (request signal taken into account by the GPDMA transfer scheduler over the source/read port)

1: selected hardware request driven by a destination peripheral (request signal taken into account by the GPDMA transfer scheduler over the destination/write port)

Note: If the channel x is activated (GPDMA_CxCR.EN is asserted) with SWREQ = 0 and PFREQ = 1 (peripheral hardware request with peripheral flow-control mode), any software assertion to this DREQ bit is ignored: in peripheral flow-control mode, only a peripheral-to-memory transfer is supported.

Bit 9 **SWREQ**: software request

This bit is internally taken into account when GPDMA_CxCR.EN is asserted.

0: no software request. The selected hardware request REQSEL[7:0] is taken into account.

1: software request for a memory-to-memory transfer. The default selected hardware request as per REQSEL[7:0] is ignored.

Bit 8 Reserved, must be kept at reset value.

Bits 7:0 **REQSEL[7:0]**: GPDMA hardware request selection

These bits are ignored if channel x is activated (GPDMA_CxCR.EN asserted) with SWREQ = 1 (software request for a memory-to-memory transfer). Else, the selected hardware request is internally taken into account as per [Section 16.3.4](#).

Caution: The user must not assign a same input hardware request (same REQSEL[7:0] value) to different active GPDMA channels (GPDMA_CxCR.EN = 1 and GPDMA_CxTR2.SWREQ = 0 for these channels). GPDMA is not intended to hardware support the case of simultaneous enabled channels incorrectly configured with a same hardware peripheral request signal, and there is no user setting error reporting.

16.8.12 GPDMA channel x block register 1 (GPDMA_CxBR1)

Address offset: $0x98 + 0x80 * x$ ($x = 0$ to 5)

Reset value: 0x0000 0000

This register is secure or nonsecure depending on the secure state of channel x (GPDMA_SECCFGR.SECx), and privileged or unprivileged, depending on the privileged state of channel x (GPDMA_PRIVCFGR.PRIVx).

This register controls the transfer of a channel x at a block level.

This register must be written when GPDMA_CxCR.EN = 0.

This register is read-only when GPDMA_CxCR.EN = 1.

This register must be written when channel x is completed (then the hardware has deasserted GPDMA_CxCR.EN). A channel transfer can be completed and programmed at different levels: block, or LLI or full linked-list.

In linked-list mode, during the link transfer:

- if GPDMA_CxLLR.UB1 = 1, this register is automatically updated by the GPDMA from the next LLI in memory.
- If GPDMA_CxLLR.UB1 = 0 and if there is at least one linked-list register to be updated from the next LLI in memory, this register is automatically and internally restored with the programmed value for the field BNDT[15:0].
- If all the update bits GPDMA_CxLLR.Uxx are null and if GPDMA_CxLLR.LA[15:0] ≠ 0, the current LLI is the last one and is continuously executed: this register is automatically and internally restored with the programmed value for BNDT[15:0] after each execution of this final LLI
- If GPDMA_CxLLR = 0, this register and BNDT[15:0] are kept as null, channel x is completed.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
BNDT[15:0]															
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bits 31:16 Reserved, must be kept at reset value.

Bits 15:0 **BNDT[15:0]**: block number of data bytes to transfer from the source

Block size transferred from the source. When the channel is enabled, this field becomes read-only and is decremented, indicating the remaining number of data items in the current source block to be transferred. BNDT[15:0] is programmed in number of bytes, maximum source block size is 64 Kbytes -1.

Once the last data transfer is completed (BNDT[15:0] = 0):

- if GPDMA_CxLLR.UB1 = 1, this field is updated with the LLI in the memory.
- if GPDMA_CxLLR.UB1 = 0 and if there is at least one non null Uxx update bit, this field is internally restored to the programmed value.
- if all GPDMA_CxLLR.Uxx = 0 and if GPDMA_CxLLR.LA[15:0] = 0, this field is internally restored to the programmed value (infinite/continuous last LLI).
- if GPDMA_CxLLR = 0, this field is kept as zero following the last LLI data transfer.

Note: A non-null source block size must be a multiple of the source data width (BNDT[2:0] versus GPDMA_CxTR1.SDW_LOG2[1:0]). Else a user setting error is reported and no transfer is issued.

When configured in packing mode (GPDMA_CxTR1.PAM[1] = 1 and destination data width different from source data width), a non-null source block size must be a multiple of the destination data width (BNDT[2:0] versus GPDMA_CxTR1.DDW_LOG2[1:0]). Else a user setting error is reported and no transfer is issued.

16.8.13 GPDMA channel x alternate block register 1 (GPDMA_CxBR1)

Address offset: $0x98 + 0x80 * x$ ($x = 6$ to 7)

Reset value: 0x0000 0000

This register is secure or nonsecure depending on the secure state of channel x (GPDMA_SECCFGR.SECx), and privileged or unprivileged, depending on the privileged state of channel x (GPDMA_PRIVCFGR.PRIVx).

This register controls the transfer of a channel x at a block level.

This register must be written when GPDMA_CxCR.EN = 0.

This register is read-only when GPDMA_CxCR.EN = 1.

This register must be written when channel x is completed (then the hardware has deasserted GPDMA_CxCR.EN). A channel transfer can be completed and programmed at different levels: block, or LLI or full linked-list.

In linked-list mode, during the link transfer:

- if GPDMA_CxLLR.UB1 = 1, this register is automatically updated by the GPDMA from the next LLI in memory.
- If GPDMA_CxLLR.UB1 = 0 and if there is at least one linked-list register to be updated from the next LLI in memory, this register is automatically and internally restored with the programmed value for the fields BNDT[15:0] and BRC[10:0].
- If all the update bits GPDMA_CxLLR.Uxx are null and if GPDMA_CxLLR.LA[15:0] ≠ 0, the current LLI is the last one and is continuously executed: this register is

automatically and internally restored with the programmed value for the fields BNDT[15:0] and BRC[10:0] after each execution of this final LLI

- If GPDMA_CxLLR = 0, BNDT[15:0] and BRC[10:0] are kept as null, channel x is completed.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
BRDDEC	BRSDEC	DDEC	SDEC	Res.	BRC[10:0]										
rw	rw	rw	rw		rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
BNDT[15:0]															
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bit 31 BRDDEC: Block repeat destination address decrement

0: at the end of a block transfer, the GPDMA_CxDAR register is updated by adding the programmed offset GPDMA_CxBR2.BRDAO to the current GPDMA_CxDAR value (current destination address)

1: at the end of a block transfer, the GPDMA_CxDAR register is updated by subtracting the programmed offset GPDMA_CxBR2.BRDAO from the current GPDMA_CxDAR value (current destination address)

Note: On top of this increment/decrement (depending on BRDDEC), GPDMA_CxDAR is in the same time also updated by the increment/decrement (depending on DDEC) of the GPDMA_CxTR3.DAO value, as it is usually done at the end of each programmed burst transfer.

Bit 30 BRSDEC: Block repeat source address decrement

0: at the end of a block transfer, the GPDMA_CxSAR register is updated by adding the programmed offset GPDMA_CxBR2.BRSAO to the current GPDMA_CxSAR value (current source address)

1: at the end of a block transfer, the GPDMA_CxSAR register is updated by subtracting the programmed offset GPDMA_CxBR2.BRSAO from the current GPDMA_CxSAR value (current source address)

Note: On top of this increment/decrement (depending on BRSDEC), GPDMA_CxSAR is in the same time also updated by the increment/decrement (depending on SDEC) of the GPDMA_CxTR3.SAO value, as it is done after any programmed burst transfer.

Bit 29 DDEC: destination address decrement

0: At the end of a programmed burst transfer to the destination, the GPDMA_CxDAR register is updated by adding the programmed offset GPDMA_CxTR3.DAO to the current GPDMA_CxDAR value (current destination address)

1: At the end of a programmed burst transfer to the destination, the GPDMA_CxDAR register is updated by subtracting the programmed offset GPDMA_CxTR3.DAO to the current GPDMA_CxDAR value (current destination address)

Bit 28 SDEC: source address decrement

0: At the end of a programmed burst transfer from the source, the GPDMA_CxSAR register is updated by adding the programmed offset GPDMA_CxTR3.SAO to the current GPDMA_CxSAR value (current source address)

1: At the end of a programmed burst transfer from the source, the GPDMA_CxSAR register is updated by subtracting the programmed offset GPDMA_CxTR3.SAO to the current GPDMA_CxSAR value (current source address)

Bit 27 Reserved, must be kept at reset value.

Bits 26:16 **BRC[10:0]**: Block repeat counter

This field contains the number of repetitions of the current block (0 to 2047).

When the channel is enabled, this field becomes read-only. After decrements, this field indicates the remaining number of blocks, excluding the current one. This counter is hardware decremented for each completed block transfer.

Once the last block transfer is completed ($BRC[10:0] = BNDT[15:0] = 0$):

- If $GPDMA_CxLLR.UB1 = 1$, all $GPDMA_CxBR1$ fields are updated by the next LLI in the memory.
- If $GPDMA_CxLLR.UB1 = 0$ and if there is at least one not null Uxx update bit, this field is internally restored to the programmed value.
- if all $GPDMA_CxLLR.Uxx = 0$ and if $GPDMA_CxLLR.LA[15:0] \neq 0$, this field is internally restored to the programmed value (infinite/continuous last LLI).
- if $GPDMA_CxLLR = 0$, this field is kept as zero following the last LLI and data transfer.

Bits 15:0 **BNDT[15:0]**: block number of data bytes to transfer from the source

Block size transferred from the source. When the channel is enabled, this field becomes read-only and is decremented, indicating the remaining number of data items in the current source block to be transferred. $BNDT[15:0]$ is programmed in number of bytes, maximum source block size is 64 Kbytes -1.

Once the last data transfer is completed ($BNDT[15:0] = 0$):

- if $GPDMA_CxLLR.UB1 = 1$, this field is updated with the LLI in the memory.
- if $GPDMA_CxLLR.UB1 = 0$ and if there is at least one not null Uxx update bit, this field is internally restored to the programmed value.
- if all $GPDMA_CxLLR.Uxx = 0$ and if $GPDMA_CxLLR.LA[15:0] \neq 0$, this field is internally restored to the programmed value (infinite/continuous last LLI).
- if $GPDMA_CxLLR = 0$, this field is kept as zero following the last LLI data transfer.

Note: A non-null source block size must be a multiple of the source data width ($BNDT[2:0]$ versus $GPDMA_CxTR1.SDW_LOG2[1:0]$). Else a user setting error is reported and no transfer is issued.

When configured in packing mode ($GPDMA_CxTR1.PAM[1] = 1$ and destination data width different from source data width), a non-null source block size must be a multiple of the destination data width ($BNDT[2:0]$ versus $GPDMA_CxTR1.DDW_LOG2[1:0]$). Else a user setting error is reported and no transfer is issued.

16.8.14 GPDMA channel x source address register (GPDMA_CxSAR)

Address offset: $0x9C + 0x80 * x$ ($x = 0$ to 7)

Reset value: $0x0000\ 0000$

This register is secure or nonsecure depending on the secure state of channel x (GPDMA_SECCFGR.SEC x), and privileged or unprivileged, depending on the privileged state of channel x (GPDMA_PRIVCFGR.PRIV x).

This register configures the source start address of a transfer.

This register must be written when GPDMA_CxCR.EN = 0.

This register is read-only when GPDMA_CxCR.EN = 1, and continuously updated by hardware, in order to reflect the address of the next burst transfer from the source.

This register must be written when the channel is completed (then the hardware has deasserted GPDMA_CxCR.EN). A channel transfer can be completed and programmed at different levels: block, 2D/repeated block, LLI or full linked-list.

In linked-list mode, during the link transfer, this register is automatically updated by the GPDMA from the memory if GPDMA_CxLLR.USA = 1.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
SA[31:16]															
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
SA[15:0]															
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bits 31:0 **SA[31:0]**: source address

This field is the pointer to the address from which the next data is read.

During the channel activity, depending on the source addressing mode (GPDMA_CxTR1.SINC), this field is kept fixed or incremented by the data width (GPDMA_CxTR1.SDW_LOG2[1:0]) after each burst source data, reflecting the next address from which data is read.

During the channel activity, this address is updated after each completed source burst, consequently to:

- the programmed source burst; either in fixed addressing mode or in contiguous-data incremented mode. If contiguously incremented (GPDMA_CxTR1.SINC = 1), then the additional address offset value is the programmed burst size, as defined by GPDMA_CxTR1.SBL_1[5:0] and GPDMA_CxTR1.SDW_LOG2[1:0]
 - the additional source incremented/decremented offset value as programmed by GPDMA_CxBR1.SDEC and GPDMA_CxTR3.SAO[12:0].
 - once/if completed source block transfer, for a channel x with 2D addressing capability (x = 6 to 7). additional block repeat source incremented/decremented offset value as programmed by GPDMA_CxBR1.BRSDEC and GPDMA_CxBR2.BRSAO[15:0]
- In linked-list mode, after a LLI data transfer is completed, this register is automatically updated by GPDMA from the memory, provided the LLI is set with GPDMA_CxLLR.USA = 1.

Note: A source address must be aligned with the programmed data width of a source burst (SA[2:0] versus GPDMA_CxTR1.SDW_LOG2[1:0]). Else, a user setting error is reported and no transfer is issued.

When the source block size is not a multiple of the source burst size and is a multiple of the source data width, the last programmed source burst is not completed and is internally shorten to match the block size. In this case, the additional GPDMA_CxTR3.SAO[12:0] is not applied.

16.8.15 GPDMA channel x destination address register (GPDMA_CxDAR)

Address offset: $0xA0 + 0x80 * x$ ($x = 0$ to 7)

Reset value: $0x0000\ 0000$

This register is secure or nonsecure depending on the secure state of channel x (GPDMA_SECCFGR.SECx), and privileged or unprivileged, depending on the privileged state of channel x (GPDMA_PRIVCFGR.PRIVx).

This register configures the destination start address of a transfer.

This register must be written when GPDMA_CxCR.EN = 0.

This register is read-only when GPDMA_CxCR.EN = 1, and continuously updated by hardware, in order to reflect the address of the next burst transfer to the destination.

This register must be written when the channel is completed (then the hardware has deasserted GPDMA_CxCR.EN). A channel transfer can be completed and programmed at different levels: block, 2D/repeated block, LLI or full linked-list.

In linked-list mode, during the link transfer, this register is automatically updated by GPDMA from the memory if GPDMA_CxLLR.UDA = 1.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
DA[31:16]															
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
DA[15:0]															
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bits 31:0 **DA[31:0]**: destination address

This field is the pointer to the address from which the next data is written.

During the channel activity, depending on the destination addressing mode (GPDMA_CxTR1.DINC), this field is kept fixed or incremented by the data width (GPDMA_CxTR1.DDW_LOG2[1:0]) after each burst destination data, reflecting the next address from which data is written.

During the channel activity, this address is updated after each completed destination burst, consequently to:

- the programmed destination burst; either in fixed addressing mode or in contiguous-data incremented mode. If contiguously incremented (GPDMA_CxTR1.DINC = 1), then the additional address offset value is the programmed burst size, as defined by GPDMA_CxTR1.DBL_1[5:0] and GPDMA_CxTR1.DDW_LOG2[1:0]
- the additional destination incremented/decremented offset value as programmed by GPDMA_CxBR1.DDEC and GPDMA_CxTR3.DAO[12:0].
- once/if completed destination block transfer, for a channel x with 2D addressing capability ($x = 6$ to 7), the additional block repeat destination incremented/decremented offset value as programmed by GPDMA_CxBR1.BRDDEC and GPDMA_CxBR2.BRDAO[15:0]

In linked-list mode, after a LLI data transfer is completed, this register is automatically updated by the GPDMA from the memory, provided the LLI is set with GPDMA_CxLLR.UDA = 1.

Note: A destination address must be aligned with the programmed data width of a destination burst (DA[2:0] versus GPDMA_CxTR1.DDW_LOG2[1:0]). Else, a user setting error is reported and no transfer is issued.

16.8.16 GPDMA channel x transfer register 3 (GPDMA_CxTR3)

Address offset: $0xA4 + 0x80 * x$ ($x = 6$ to 7)

Reset value: 0x0000 0000

This register is secure or nonsecure depending on the secure state of channel x (GPDMA_SECCFGR.SECx), and privileged or unprivileged, depending on the privileged state of channel x (GPDMA_PRIVCFGR.PRIVx).

This register controls the transfer of a channel x.

This register must be written when GPDMA_CxCR.EN = 0.

This register is read-only when GPDMA_CxCR.EN = 1.

This register must be written when the channel is completed (then the hardware has deasserted GPDMA_CxCR.EN). A channel transfer can be completed and programmed at different levels: block or LLI or full linked-list.

In linked-list mode, during the link transfer, this register is automatically updated by the GPDMA from the memory if GPDMA_CxLLR.UT3 = 1.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	DAO[12:0]												
			rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	Res.	SAO[12:0]												
			rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bits 31:29 Reserved, must be kept at reset value.

Bits 28:16 **DAO[12:0]**: destination address offset increment

The destination address, pointed by GPDMA_CxDAR, is incremented or decremented (depending on GPDMA_CxBR1.DDEC) by this offset DAO[12:0] for each programmed destination burst. This offset is not including and is added to the programmed burst size when the completed burst is addressed in incremented mode (GPDMA_CxTR1.DINC = 1).

Note: A destination address offset must be aligned with the programmed data width of a destination burst (DAO[2:0] versus GPDMA_CxTR1.DDW_LOG2[1:0]). Else, a user setting error is reported and no transfer is issued.

Bits 15:13 Reserved, must be kept at reset value.

Bits 12:0 **SAO[12:0]**: source address offset increment

The source address, pointed by GPDMA_CxSAR, is incremented or decremented (depending on GPDMA_CxBR1.SDEC) by this offset SAO[12:0] for each programmed source burst. This offset is not including and is added to the programmed burst size when the completed burst is addressed in incremented mode (GPDMA_CxTR1.SINC = 1).

Note: A source address offset must be aligned with the programmed data width of a source burst (SAO[2:0] versus GPDMA_CxTR1.SDW_LOG2[1:0]). Else a user setting error is reported and none transfer is issued.

When the source block size is not a multiple of the destination burst size, and is a multiple of the source data width, then the last programmed source burst is not completed and is internally shorten to match the block size. In this case, the additional GPDMA_CxTR3.SAO[12:0] is not applied.

16.8.17 GPDMA channel x block register 2 (GPDMA_CxBR2)

Address offset: $0xA8 + 0x80 * x$ ($x = 6$ to 7)

Reset value: 0x0000 0000

This register is secure or nonsecure depending on the secure state of channel x (GPDMA_SECCFGR.SECx), and privileged or unprivileged, depending on the privileged state of channel x (GPDMA_PRIVCFGR.PRIVx).

This register controls the transfer of a channel x at a 2D/repeated block level.

This register must be written when GPDMA_CxCR.EN = 0.

This register is read-only when GPDMA_CxCR.EN = 1.

This register must be written when the channel is completed (then the hardware has deasserted GPDMA_CxCR.EN). A channel transfer can be completed and programmed at different levels: block, 2D/repeated block, LLI or full linked-list.

In linked-list mode, during the link transfer, this register is automatically updated by the GPDMA from the memory if GPDMA_CxLLR.UB2 = 1.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
BRDAO[15:0]															
rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
BRSAO[15:0]															
rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW

Bits 31:16 **BRDAO[15:0]**: Block repeated destination address offset

For a channel with 2D addressing capability, this field is used to update (by addition or subtraction depending on GPDMA_CxBR1.BRDDEC) the current destination address (GPDMA_CxDAR) at the end of a block transfer.

Note: A block repeated destination address offset must be aligned with the programmed data width of a destination burst (BRDAO[2:0] versus GPDMA_CxTR1.DDW_LOG2[1:0]). Else a user setting error is reported and no transfer is issued. BRDAO[15:0] must be set to 0 in peripheral flow-control mode (if GPDMA_CxTR2.PFREQ = 1).

Bits 15:0 **BRSAO[15:0]**: Block repeated source address offset

For a channel with 2D addressing capability, this field is used to update (by addition or subtraction depending on GPDMA_CxBR1.BRSDEC) the current source address (GPDMA_CxSAR) at the end of a block transfer.

Note: A block repeated source address offset must be aligned with the programmed data width of a source burst (BRSAO[2:0] versus GPDMA_CxTR1.SDW_LOG2[1:0]). Else a user setting error is reported and no transfer is issued. BRSAO[15:0] must be set to 0 in peripheral flow-control mode (if GPDMA_CxTR2.PFREQ = 1).

16.8.18 GPDMA channel x linked-list address register (GPDMA_CxLLR)

Address offset: $0xCC + 0x80 * x$ ($x = 0$ to 5)

Reset value: 0x0000 0000

This register is secure or nonsecure depending on the secure state of channel x (GPDMA_SECCFGR.SECx), and privileged or unprivileged, depending on the privileged state of channel x (GPDMA_PRIVCFGR.PRIVx).

This register configures the data structure of the next LLI in the memory and its address pointer. A channel transfer is completed when this register is null.

This register must be written when GPDMA_CxCR.EN = 0.

This register is read-only when GPDMA_CxCR.EN = 1.

This register must be written when the channel is completed (then the hardware has deasserted GPDMA_CxCR.EN). A channel transfer can be completed and programmed at different levels: block or LLI or full linked-list.

In linked-list mode, during the link transfer, this register is automatically updated by the GPDMA from the memory if GPDMA_CxLLR.ULL = 1.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
UT1	UT2	UB1	USA	UDA	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	ULL
rw	rw	rw	rw	rw											rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
LA[15:2]														Res.	Res.
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw		

Bit 31 **UT1**: Update GPDMA_CxTR1 from memory

This bit controls the update of GPDMA_CxTR1 from the memory during the link transfer.

0: no GPDMA_CxTR1 update

1: GPDMA_CxTR1 update

Bit 30 **UT2**: Update GPDMA_CxTR2 from memory

This bit controls the update of GPDMA_CxTR2 from the memory during the link transfer.

0: no GPDMA_CxTR2 update

1: GPDMA_CxTR2 update

Bit 29 **UB1**: Update GPDMA_CxBR1 from memory

This bit controls the update of GPDMA_CxBR1 from the memory during the link transfer.

If UB1 = 0 and if GPDMA_CxLLR ≠ 0, the linked-list is not completed.

GPDMA_CxBR1.BNDT[15:0] is then restored to the programmed value after data transfer is completed and before the link transfer.

0: no GPDMA_CxBR1 update from memory (GPDMA_CxBR1.BNDT[15:0] restored if any link transfer)

1: GPDMA_CxBR1 update

Bit 28 **USA**: update GPDMA_CxSAR from memory

This bit controls the update of GPDMA_CxSAR from the memory during the link transfer.

0: no GPDMA_CxSAR update

1: GPDMA_CxSAR update

Bit 27 **UDA**: Update GPDMA_CxDAR register from memory

This bit is used to control the update of GPDMA_CxDAR from the memory during the link transfer.

0: no GPDMA_CxDAR update

1: GPDMA_CxDAR update

Bits 26:17 Reserved, must be kept at reset value.

Bit 16 **ULL**: Update GPDMA_CxLLR register from memory

This bit is used to control the update of GPDMA_CxLLR from the memory during the link transfer.

0: no GPDMA_CxLLR update

1: GPDMA_CxLLR update

Bits 15:2 **LA[15:2]**: pointer (16-bit low-significant address) to the next linked-list data structure

If UT1 = UT2 = UB1 = USA = UDA = ULL = 0 and if LA[15:2] = 0, the current LLI is the last one. The channel transfer is completed without any update of the linked-list GPDMA register file.

Else, this field is the pointer to the memory address offset from which the next linked-list data structure is automatically fetched from, once the data transfer is completed, in order to conditionally update the linked-list GPDMA internal register file (GPDMA_CxTR1, GPDMA_CxTR2, GPDMA_CxBR1, GPDMA_CxSAR, GPDMA_CxDAR, and GPDMA_CxLLR).

Caution: The user must program this pointer not to exceed the 64-Kbyte addressable space defined by the link base address register GPDMA_CxLBAR. The complete linked-list data structure must be included in the 64-Kbyte addressable space.

Note: The user must program the pointer to be 32-bit aligned. The two low-significant bits are write ignored.

Bits 1:0 Reserved, must be kept at reset value.

16.8.19 GPDMA channel x alternate linked-list address register (GPDMA_CxLLR)

Address offset: $0xCC + 0x80 * x$ ($x = 6$ to 7)

Reset value: 0x0000 0000

This register is secure or nonsecure depending on the secure state of channel x (GPDMA_SECCFGR.SECx), and privileged or unprivileged, depending on the privileged state of channel x (GPDMA_PRIVCFGR.PRIVx).

This register configures the data structure of the next LLI in the memory and its address pointer. A channel transfer is completed when this register is null.

This register must be written when GPDMA_CxCR.EN = 0.

This register is read-only when GPDMA_CxCR.EN = 1.

This register must be written when the channel is completed (then the hardware has deasserted GPDMA_CxCR.EN). A channel transfer can be completed and programmed at different levels: block or LLI or full linked-list.

In linked-list mode, during the link transfer, this register is automatically updated by the GPDMA from the memory if GPDMA_CxLLR.ULL = 1.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
UT1	UT2	UB1	USA	UDA	UT3	UB2	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	ULL
rw	rw	rw	rw	rw	rw	rw									rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
LA[15:2]														Res.	Res.
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw		

Bit 31 **UT1**: Update GPDMA_CxTR1 from memory

This bit controls the update of GPDMA_CxTR1 from the memory during the link transfer.

0: no GPDMA_CxTR1 update

1: GPDMA_CxTR1 update

Bit 30 **UT2**: Update GPDMA_CxTR2 from memory

This bit controls the update of GPDMA_CxTR2 from the memory during the link transfer.

0: no GPDMA_CxTR2 update

1: GPDMA_CxTR2 update

Bit 29 **UB1**: Update GPDMA_CxBR1 from memory

This bit controls the update of GPDMA_CxBR1 from the memory during the link transfer.

If UB1 = 0 and if GPDMA_CxLLR ≠ 0, the linked-list is not completed.

GPDMA_CxBR1.BNDT[15:0] is then restored to the programmed value after data transfer is completed and before the link transfer.

0: no GPDMA_CxBR1 update from memory (GPDMA_CxBR1.BNDT[15:0] restored if any link transfer)

1: GPDMA_CxBR1 update

Bit 28 **USA**: update GPDMA_CxSAR from memory

This bit controls the update of GPDMA_CxSAR from the memory during the link transfer.

0: no GPDMA_CxSAR update

1: GPDMA_CxSAR update

Bit 27 **UDA**: Update GPDMA_CxDAR register from memory

This bit is used to control the update of GPDMA_CxDAR from the memory during the link transfer.

0: no GPDMA_CxDAR update

1: GPDMA_CxDAR update

Bit 26 **UT3**: Update GPDMA_CxTR3 from memory

This bit controls the update of GPDMA_CxTR3 from the memory during the link transfer.

0: no GPDMA_CxTR3 update

1: GPDMA_CxTR3 update

Bit 25 **UB2**: Update GPDMA_CxBR2 from memory

This bit controls the update of GPDMA_CxBR2 from the memory during the link transfer.

0: no GPDMA_CxBR2 update

1: GPDMA_CxBR2 update

Bits 24:17 Reserved, must be kept at reset value.

Bit 16 **ULL**: Update GPDMA_CxLLR register from memory

This bit is used to control the update of GPDMA_CxLLR from the memory during the link transfer.

0: no GPDMA_CxLLR update

1: GPDMA_CxLLR update

Bits 15:2 **LA[15:2]**: pointer (16-bit low-significant address) to the next linked-list data structure

If UT1 = UT2 = UB1 = USA = UDA = ULL = 0 and if LA[15:2] = 0, the current LLI is the last one. The channel transfer is completed without any update of the linked-list GPDMA register file.

Else, this field is the pointer to the memory address offset from which the next linked-list data structure is automatically fetched from, once the data transfer is completed, in order to conditionally update the linked-list GPDMA internal register file (GPDMA_CxTR1, GPDMA_CxTR2, GPDMA_CxBR1, GPDMA_CxSAR, GPDMA_CxDAR, and GPDMA_CxLLR).

Caution: The user must program this pointer not to exceed the 64-Kbyte addressable space defined by the link base address register GPDMA_CxLBAR. The complete linked-list data structure must be included in the 64-Kbyte addressable space.

Note: The user must program the pointer to be 32-bit aligned. The two low-significant bits are write ignored.

Bits 1:0 Reserved, must be kept at reset value.

16.8.20 GPDMA register map

Table 151. GPDMA register map and reset values

Offset	Register name	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0x00	GPDMA_SECCFGR	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	SEC7	SEC6	SEC5	SEC4	SEC3	SEC2	SEC1	SEC0
	Reset value																									0	0	0	0	0	0	0	0
0x04	GPDMA_PRIVCFGR	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	PRIV7	PRIV6	PRIV5	PRIV4	PRIV3	PRIV2	PRIV1	PRIV0
	Reset value																									0	0	0	0	0	0	0	0
0x08	GPDMA_RCFGLOCKR	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	LOCK7	LOCK6	LOCK5	LOCK4	LOCK3	LOCK2	LOCK1	LOCK0
	Reset value																									0	0	0	0	0	0	0	0

Table 151. GPDMA register map and reset values (continued)

Offset	Register name	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0			
0x0C	GPDMA_MISR	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	MIS7	MIS6	MIS5	MIS4	MIS3	MIS2	MIS1	MIS0		
	Reset value																										0	0	0	0	0	0	0	0		
0x10	GPDMA_SMISR	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	MIS7	MIS6	MIS5	MIS4	MIS3	MIS2	MIS1	MIS0		
	Reset value																										0	0	0	0	0	0	0	0		
0x14 - 0x4C	Reserved	Reserved																																		
0x50+0x80 * x (x=0 to 7)	GPDMA_CxLBAR	LBA[31:16]																Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res
	Reset value	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0																			
0x54 to 0x58+0x80*x (x=0 to 7)	Reserved	Reserved																																		
0x5C+0x80 * x (x=0 to 7)	GPDMA_CxFCR	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	TOF	SUSPF	USEF	ULEF	DTEF	HTF	TCF	Res	Res	Res	Res	Res	Res	Res	Res			
	Reset value																		0	0	0	0	0	0	0											
0x60+ 0x80 * x (x=0 to 7)	GPDMA_CxSR	Res	Res	Res	Res	Res	Res	Res	Res	FIFOL[7:0]							Res	TOF	SUSPF	USEF	ULEF	DTEF	HTF	TCF	Res	Res	Res	Res	Res	Res	Res	IDLEF				
	Reset value									0	0	0	0	0	0	0	0		0	0	0	0	0	0	0								1			
0x64+ 0x80 * x (x=0 to 7)	GPDMA_CxCr	Res	Res	Res	Res	Res	Res	Res	Res	PRI0[1:0]	Res	Res	Res	Res	Res	LAP	LSM	TOIE	SUSPIE	USEIE	ULEIE	DTEIE	HTIE	TCIE	Res	Res	Res	Res	Res	SUSP	RESET	EN				
	Reset value									0	0					0	0		0	0	0	0	0	0	0					0	0	0				
0x68 to 0x8C+0x80 * x (x=0 to 7)	Reserved	Reserved																																		
0x90+0x80 * x (x=0 to 7)	GPDMA_CxTR1	DSEC	DAP	Res	Res	DXH	DBX	DBL_1[5:0]						DINC	Res	DDW_LOG2 [1:0]	SSEC	SAP	SBX	PAM[1:0]	Res	SBL_1[5:0]						SINC	Res	SDW_LOG2 [1:0]						
	Reset value	0	0			0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0		
0x94+0x80 * x (x=0 to 7)	GPDMA_CxTR2	TCEM[1:0]	Res	Res	Res	Res	TRIGPOL[1:0]	Res	Res	TRIGSEL[5:0]						TRIGM[1:0]				PFREQ	BREQ	DREQ	SWREQ	Res	REQSEL[7:0]											
	Reset value	0	0				0	0				0	0	0	0	0	0	0	0		0	0	0	0	0		0	0	0	0	0	0	0	0		
0x98+0x80 * x (x=0 to 5)	GPDMA_CxBR1	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	BNDT[15:0]																			
	Reset value																0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0			
0x98+0x80 * x (x=6 to 7)	GPDMA_CxBR1	BRDDEC	BRSEDEC	DDEC	SDEC	Res	BRC[10:0]										BNDT[15:0]																			
	Reset value	0	0	0	0		0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0		
0x9C+0x80 * x (x=0 to 7)	GPDMA_CxSAR	SA[31:0]																																		
	Reset value	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0			
0xA0+0x80 * x (x=0 to 7)	GPDMA_CxDAR	DA[31:0]																																		
	Reset value	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0			
0xA4+0x80 * x (x=6 to 7)	GPDMA_CxTR3	Res	Res	Res	DAO[12:0]										Res	Res	Res	SAO[12:0]																		
	Reset value				0	0	0	0	0	0	0	0	0	0	0	0	0				0	0	0	0	0	0	0	0	0	0	0	0	0	0		
0xA8+0x80 * x (x=6 to 7)	GPDMA_CxBR2	BRDAO[15:0]															BRSAO[15:0]																			
	Reset value	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0			
0xCC+0x80 * x (x=0 to 5)	GPDMA_CxLLR	UT1	UT2	UB1	USA	UDA	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	ULL	LA[15:2]										Res	Res							
	Reset value	0	0	0	0	0											0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0				

Table 151. GPDMA register map and reset values (continued)

Offset	Register name	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0xCC+0x80*x (x=6 to 7)	GPDMA_CxLLR	UT1	UT2	UB1	USA	UDA	UT3	UB2	Res	Res	Res	Res	Res	Res	Res	Res	ULL	LA[15:2]														Res	Res
	Reset value	0	0	0	0	0	0	0									0	0	0	0	0	0	0	0	0	0	0	0	0	0	0		

Refer to [Section 2.3: Memory organization](#) for the register boundary addresses.

17 Nested vectored interrupt controller (NVIC)

17.1 NVIC main features

- Up to 150 maskable interrupt channels (not including the 16 Cortex-M33 with FPU interrupt lines)
- 16 programmable priority levels (four bits of interrupt priority used)
- Low-latency exception and interrupt handling
- Power management control
- Implementation of system control registers

The NVIC and the processor core interface are closely coupled, enabling low-latency interrupt processing and efficient processing of late arriving interrupts.

The NVIC registers are banked across secure and nonsecure states.

All interrupts, including the core exceptions, are managed by the NVIC.

17.2 SysTick calibration value register

The Cortex-M33 with TrustZone mainline security extension embeds two SysTick timers.

When TrustZone is activated, the following SysTick timers are available:

- SysTick, secure instance
- SysTick, nonsecure instance

When TrustZone is disabled, only one SysTick timer is available.

The SysTick timer calibration value (STCALIB) is 0x3E8. It gives a reference time base of 1 ms based on a SysTick clock frequency of 1 MHz. To match the 1 ms time base for an application running at a given frequency, the SysTick reload value must be programmed as follows in the SYST_RVR register:

- When SysTick clock source is CPU clock HCLK reload value = $(HCLK \times STCALIB) - 1$
- When SysTick clock source is external clock (HCLK/8) reload value = $((HCLK / 8) \times STCALIB) - 1$

The HCLK refers to the AHB frequency value in MHz.

Example: SysTick clock source is CPU clock HCLK of 100 MHz, to match a time base of 1 ms: SysTick reload value = $(100 \times STCALIB) - 1 = 0x1869F$.

17.3 Interrupt and exception vectors

The gray rows in [Table 152](#), [Table 153](#), and [Table 154](#) describe the vectors without specific position.

Table 152. STM32H562/563/573xx vector table

Position	Priority	Type of priority	Acronym	Description	Address
-	-	-	-	Reserved	0x0000 0000
-	-4	Fixed	SettableReset	Reset	0x0000 0004
-	-2	Fixed	NMI	Non maskable interrupt. The RCC clock security system (CSS) is linked to the NMI vector.	0x0000 0008
-	-3 or -1	Fixed	Secure HardFault	Secure Hard fault	0x0000 000C
-	-1	Fixed	Nonsecure HardFault	Nonsecure Hard fault. All classes of fault.	0x0000 000C
-	0	Settable	MemManage	Memory management	0x0000 0010
-	1	Settable	BusFault	Pre-fetch fault, memory access fault	0x0000 0014
-	2	Settable	UsageFault	Undefined instruction or illegal state	0x0000 0018
-	3	Settable	SecureFault	Secure fault	0x0000 001C
-	-	-	-	Reserved	0x0000 0020 - 0x0000 0028
-	4	-	SVC	System service call via SWI instruction	0x0000 002C
-	5	-	Debug Monitor	Monitor	0x0000 0030
-	-	-	-	Reserved	0x0000 0034
-	6	Settable	PendSV	Pendable request for system service	0x0000 0038
-	7	Settable	SysTick	System tick timer	0x0000 003C
0	8	Settable	WWDG	Window watchdog interrupt	0x0000 0040
1	9	Settable	PVD_AVD	Power voltage monitor/ Analog voltage monitor	0x0000 0044
2	10	Settable	RTC	RTC global nonsecure interrupts	0x0000 0048
3	11	Settable	RTC_S	RTC global secure interrupts	0x0000 004C
4	12	Settable	TAMP	Tamper global interrupts	0x0000 0050
5	13	Settable	RAMCFG	RAM configuration global interrupt	0x0000 0054
6	14	Settable	FLASH	Flash nonsecure global interrupt	0x0000 0058
7	15	Settable	FLASH_S	Flash secure global interrupt	0x0000 005C
8	16	Settable	GTZC	GTZC global interrupt	0x0000 0060
9	17	Settable	RCC	RCC nonsecure global interrupt	0x0000 0064
10	18	Settable	RCC_S	RCC secure global interrupt	0x0000 0068
11	19	Settable	EXTI0	EXTI Line0 interrupt	0x0000 006C

Table 152. STM32H562/563/573xx vector table (continued)

Position	Priority	Type of priority	Acronym	Description	Address
12	20	Settable	EXTI1	EXTI Line1 interrupt	0x0000 0070
13	21	Settable	EXTI2	EXTI Line2 interrupt	0x0000 0074
14	22	Settable	EXTI3	EXTI Line3 interrupt	0x0000 0078
15	23	Settable	EXTI4	EXTI Line4 interrupt	0x0000 007C
16	24	Settable	EXTI5	EXTI Line5 interrupt	0x0000 0080
17	25	Settable	EXTI6	EXTI Line6 interrupt	0x0000 0084
18	26	Settable	EXTI7	EXTI Line7 interrupt	0x0000 0088
19	27	Settable	EXTI8	EXTI Line8 interrupt	0x0000 008C
20	28	Settable	EXTI9	EXTI Line9 interrupt	0x0000 0090
21	29	Settable	EXTI10	EXTI Line10 interrupt	0x0000 0094
22	30	Settable	EXTI11	EXTI Line11 interrupt	0x0000 0098
23	31	Settable	EXTI12	EXTI Line12 interrupt	0x0000 009C
24	32	Settable	EXTI13	EXTI Line13 interrupt	0x0000 00A0
25	33	Settable	EXTI14	EXTI Line14 interrupt	0x0000 00A4
26	34	Settable	EXTI15	EXTI Line15 interrupt	0x0000 00A8
27	35	Settable	GPDMA1_CH0	GPDMA1 channel 0 global interrupt	0x0000 00AC
28	36	Settable	GPDMA1_CH1	GPDMA1 channel 1 global interrupt	0x0000 00B0
29	37	Settable	GPDMA1_CH2	GPDMA1 channel 2 global interrupt	0x0000 00B4
30	38	Settable	GPDMA1_CH3	GPDMA1 channel 3 global interrupt	0x0000 00B8
31	39	Settable	GPDMA1_CH4	GPDMA1 channel 4 global interrupt	0x0000 00BC
32	40	Settable	GPDMA1_CH5	GPDMA1 channel 5 global interrupt	0x0000 00C0
33	41	Settable	GPDMA1_CH6	GPDMA1 channel 6 global interrupt	0x0000 00C4
34	42	Settable	GPDMA1_CH7	GPDMA1 channel 7 global interrupt	0x0000 00C8
35	43	Settable	IWDG	Independent watchdog interrupt	0x0000 00CC
36	44	Settable	SAES ⁽¹⁾	Secure AES	0x0000 00D0
37	45	Settable	ADC1	ADC1 global interrupt	0x0000 00D4
38	46	Settable	DAC1	DAC1 global interrupt	0x0000 00D8
39	47	Settable	FDCAN1_IT0	FDCAN1 Interrupt 0	0x0000 00DC
40	48	Settable	FDCAN1_IT1	FDCAN1 Interrupt 1	0x0000 00E0
41	49	Settable	TIM1_BRK/TIM1_TERR/ TIM1_IERR	TIM1 break/transition error/ index error	0x0000 00E4
42	50	Settable	TIM1_UP	TIM1 update	0x0000 00E8
43	51	Settable	TIM1_TRG_COM/ TIM1_DIR/TIM1_IDX	TIM1 trigger and commutation/direction change interrupt/index	0x0000 00EC

Table 152. STM32H562/563/573xx vector table (continued)

Position	Priority	Type of priority	Acronym	Description	Address
44	52	Settable	TIM1_CC	TIM1 capture compare interrupt	0x0000 00F0
45	53	Settable	TIM2	TIM2 global interrupt	0x0000 00F4
46	54	Settable	TIM3	TIM3 global interrupt	0x0000 00F8
47	55	Settable	TIM4	TIM4 global interrupt	0x0000 00FC
48	56	Settable	TIM5	TIM5 global interrupt	0x0000 0100
49	57	Settable	TIM6	TIM6 global interrupt	0x0000 0104
50	58	Settable	TIM7	TIM7 global interrupt	0x0000 0108
51	59	Settable	I2C1_EV	I2C1 event interrupt	0x0000 010C
52	60	Settable	I2C1_ER	I2C1 error interrupt	0x0000 0110
53	61	Settable	I2C2_EV	I2C2 event interrupt	0x0000 0114
54	62	Settable	I2C2_ER	I2C2 error interrupt	0x0000 0118
55	63	Settable	SPI1	SPI1 global interrupt	0x0000 011C
56	64	Settable	SPI2	SPI2 global interrupt	0x0000 0120
57	65	Settable	SPI3	SPI3 global interrupt	0x0000 0124
58	66	Settable	USART1	USART1 global interrupt	0x0000 0128
59	67	Settable	USART2	USART2 global interrupt	0x0000 012C
60	68	Settable	USART3	USART3 global interrupt	0x0000 0130
61	69	Settable	UART4	UART4 global interrupt	0x0000 0134
62	70	Settable	UART5	UART5 global interrupt	0x0000 0138
63	71	Settable	LPUART1	LPUART1 global interrupt or R wake-up or T wake-up through EXTI line	0x0000 013C
64	72	Settable	LPTIM1 or LPTIM1_AIT	LPTIM1 global interrupt or AIT through EXTI line	0x0000 0140
65	73	Settable	TIM8_BRK/TIM8_TERR/ TIM8_IERR	TIM8 break interrupt/transition error/index error	0x0000 0144
66	74	Settable	TIM8_UP	TIM8 update interrupt	0x0000 0148
67	75	Settable	TIM8_TRG_COM/ TIM8_DIR/TIM8_IDX	TIM8 trigger and commutation interrupt/ direction change interrupt/index	0x0000 014C
68	76	Settable	TIM8_CC	TIM8 capture compare interrupt	0x0000 0150
69	77	Settable	ADC2	ADC2 global interrupt	0x0000 0154
70	78	Settable	LPTIM2 or LPTIM2_AIT	LPTIM2 global interrupt or AIT through EXTI line	0x0000 0158
71	79	Settable	TIM15	TIM15 global interrupt	0x0000 015C
72	80	Settable	TIM16	TIM16 global interrupt	0x0000 0160
73	81	Settable	TIM17	TIM17 global interrupt	0x0000 0164

Table 152. STM32H562/563/573xx vector table (continued)

Position	Priority	Type of priority	Acronym	Description	Address
74	82	Settable	USB_FS	USB FS global interrupt	0x0000 0168
75	83	Settable	CRS	Clock recovery system global interrupt	0x0000 016C
76	84	Settable	UCPD1	UCPD1 global interrupt	0x0000 0170
77	85	Settable	FMC	FMC global interrupt	0x0000 0174
78	86	Settable	OCTOSPI1	OCTOSPI1 global interrupt	0x0000 0178
79	87	Settable	SDMMC1	SDMMC1 global interrupt	0x0000 017C
80	88	Settable	I2C3_EV	I2C3 event interrupt	0x0000 0180
81	89	Settable	I2C3_ER	I2C3 error interrupt	0x0000 0184
82	90	Settable	SPI4	SPI4 global interrupt	0x0000 0188
83	91	Settable	SPI5	SPI5 global interrupt	0x0000 018C
84	92	Settable	SPI6	SPI6 global interrupt	0x0000 0190
85	93	Settable	USART6	USART6 global interrupt	0x0000 0194
86	94	Settable	USART10	USART10 global interrupt	0x0000 0198
87	95	Settable	USART11	USART11 global interrupt	0x0000 019C
88	96	Settable	SAI1	SAI1 global interrupt	0x0000 01A0
89	97	Settable	SAI2	SAI2 global interrupt	0x0000 01A4
90	98	Settable	GPDMA2_CH0	GPDMA2 channel0 global interrupt	0x0000 01A8
91	99	Settable	GPDMA2_CH1	GPDMA2 channel1 global interrupt	0x0000 01AC
92	100	Settable	GPDMA2_CH2	GPDMA2 channel2 global interrupt	0x0000 01B0
93	101	Settable	GPDMA2_CH3	GPDMA2 channel3 global interrupt	0x0000 01B4
94	102	Settable	GPDMA2_CH4	GPDMA2 channel4 global interrupt	0x0000 01B8
95	103	Settable	GPDMA2_CH5	GPDMA2 channel5 global interrupt	0x0000 01BC
96	104	Settable	GPDMA2_CH6	GPDMA2 channel6 global interrupt	0x0000 01C0
97	105	Settable	GPDMA2_CH7	GPDMA2 channel7 global interrupt	0x0000 01C4
98	106	Settable	UART7	UART7 global interrupt	0x0000 01C8
99	107	Settable	UART8	UART8 global interrupt	0x0000 01CC
100	108	Settable	UART9	UART9 global interrupt	0x0000 01D0
101	109	Settable	UART12	UART12 global interrupt	0x0000 01D4
102	110	Settable	SDMMC2 ⁽²⁾	SDMMC2 global interrupt	0x0000 01D8
103	111	Settable	FPU	Floating point interrupt	0x0000 01DC
104	112	Settable	ICACHE	Instruction cache global interrupt	0x0000 01E0
105	113	Settable	DCACHE	Data cache global interrupt	0x0000 01E4
106	114	Settable	ETH ⁽²⁾	Ethernet interrupt	0x0000 01E8

Table 152. STM32H562/563/573xx vector table (continued)

Position	Priority	Type of priority	Acronym	Description	Address
107	115	Settable	ETH_WKUP	Ethernet wake-up interrupt through EXTI line	0x0000 01EC
108	116	Settable	DCMI_PSSI	DCMI/PSSI global interrupt	0x0000 01F0
109	117	Settable	FDCAN2_IT0 ⁽²⁾	FDCAN2 interrupt 0	0x0000 01F4
110	118	Settable	FDCAN2_IT1 ⁽²⁾	FDCAN2 interrupt 1	0x0000 01F8
111	119	Settable	CORDIC	CORDIC interrupt	0x0000 01FC
112	120	Settable	FMAC	FMAC interrupt	0x0000 0200
113	121	Settable	DTS or DTS_WKUP	DTS interrupt or AIT through EXTI line	0x0000 0204
114	122	Settable	RNG	RNG global interrupt	0x0000 0208
115	123	Settable	OTFDEC1	OTFDEC1 secure global interrupt	0x0000 020C
116	124	Settable	AES ⁽¹⁾	AES global interrupt	0x0000 0210
117	125	Settable	HASH	HASH interrupt	0x0000 0214
118	126	Settable	PKA	PKA global interrupt	0x0000 0218
119	127	Settable	CEC	HDMI-CEC global interrupt	0x0000 021C
120	128	Settable	TIM12	TIM12 global interrupt	0x0000 0220
121	129	Settable	TIM13	TIM13 global interrupt	0x0000 0224
122	130	Settable	TIM14	TIM14 global interrupt	0x0000 0228
123	131	Settable	I3C1_EV	I3C1 event interrupt	0x0000 022C
124	132	Settable	I3C1_ER	I3C1 error interrupt	0x0000 0230
125	133	Settable	I2C4_EV	I2C4 event interrupt	0x0000 0234
126	134	Settable	I2C4_ER	I2C4 error interrupt	0x0000 0238
127	135	Settable	LPTIM3 or LPTIM3_AIT	LPTIM3 global interrupt or AIT through EXTI line	0x0000 023C
128	136	Settable	LPTIM4 or LPTIM4_AIT	LPTIM4 global interrupt or AIT through EXTI line	0x0000 0240
129	137	Settable	LPTIM5 or LPTIM5_AIT	LPTIM5 global interrupt or AIT through EXTI line	0x0000 0244
130	138	Settable	LPTIM6 or LPTIM6_AIT	LPTIM6 global interrupt or AIT through EXTI line	0x0000 0248

1. Not available on STM32H562/563 devices.

2. Not available on STM32H562 devices.

Table 153. STM32H523/533xx vector table

Position	Priority	Type of priority	Acronym	Description	Address
-	-	-	-	Reserved	0x0000 0000
-	-4	Fixed	SettableReset	Reset	0x0000 0004
-	-2	Fixed	NMI	Non maskable interrupt. The RCC clock security system (CSS) is linked to the NMI vector.	0x0000 0008
-	-3 or -1	Fixed	Secure HardFault	Secure Hard fault	0x0000 000C
-	-1	Fixed	Nonsecure HardFault	Nonsecure Hard fault. All classes of fault.	0x0000 000C
-	0	Settable	MemManage	Memory management	0x0000 0010
-	1	Settable	BusFault	Prefetch fault, memory access fault	0x0000 0014
-	2	Settable	UsageFault	Undefined instruction or illegal state	0x0000 0018
-	3	Settable	SecureFault	Secure fault	0x0000 001C
-	-	-	-	Reserved	0x0000 0020 - 0x0000 0028
-	4	-	SVC	System service call via SWI instruction	0x0000 002C
-	5	-	Debug Monitor	Monitor	0x0000 0030
-	-	-	-	Reserved	0x0000 0034
-	6	Settable	PendSV	Pendable request for system service	0x0000 0038
-	7	Settable	SysTick	System tick timer	0x0000 003C
0	8	Settable	WWDG	Window watchdog interrupt	0x0000 0040
1	9	Settable	PVD_AVD	Power voltage monitor/ Analog voltage monitor	0x0000 0044
2	10	Settable	RTC	RTC global nonsecure interrupts	0x0000 0048
3	11	Settable	RTC_S	RTC global secure interrupts	0x0000 004C
4	12	Settable	TAMP	Tamper global interrupts	0x0000 0050
5	13	Settable	RAMCFG	RAM configuration global interrupt	0x0000 0054
6	14	Settable	FLASH	Flash nonsecure global interrupt	0x0000 0058
7	15	Settable	FLASH_S	Flash secure global interrupt	0x0000 005C
8	16	Settable	GTZC	GTZC global interrupt	0x0000 0060
9	17	Settable	RCC	RCC nonsecure global interrupt	0x0000 0064
10	18	Settable	RCC_S	RCC secure global interrupt	0x0000 0068
11	19	Settable	EXTI0	EXTI Line0 interrupt	0x0000 006C
12	20	Settable	EXTI1	EXTI Line1 interrupt	0x0000 0070
13	21	Settable	EXTI2	EXTI Line2 interrupt	0x0000 0074
14	22	Settable	EXTI3	EXTI Line3 interrupt	0x0000 0078

Table 153. STM32H523/533xx vector table (continued)

Position	Priority	Type of priority	Acronym	Description	Address
15	23	Settable	EXTI4	EXTI Line4 interrupt	0x0000 007C
16	24	Settable	EXTI5	EXTI Line5 interrupt	0x0000 0080
17	25	Settable	EXTI6	EXTI Line6 interrupt	0x0000 0084
18	26	Settable	EXTI7	EXTI Line7 interrupt	0x0000 0088
19	27	Settable	EXTI8	EXTI Line8 interrupt	0x0000 008C
20	28	Settable	EXTI9	EXTI Line9 interrupt	0x0000 0090
21	29	Settable	EXTI10	EXTI Line10 interrupt	0x0000 0094
22	30	Settable	EXTI11	EXTI Line11 interrupt	0x0000 0098
23	31	Settable	EXTI12	EXTI Line12 interrupt	0x0000 009C
24	32	Settable	EXTI13	EXTI Line13 interrupt	0x0000 00A0
25	33	Settable	EXTI14	EXTI Line14 interrupt	0x0000 00A4
26	34	Settable	EXTI15	EXTI Line15 interrupt	0x0000 00A8
27	35	Settable	GPDMA1_CH0	GPDMA1 channel 0 global interrupt	0x0000 00AC
28	36	Settable	GPDMA1_CH1	GPDMA1 channel 1 global interrupt	0x0000 00B0
29	37	Settable	GPDMA1_CH2	GPDMA1 channel 2 global interrupt	0x0000 00B4
30	38	Settable	GPDMA1_CH3	GPDMA1 channel 3 global interrupt	0x0000 00B8
31	39	Settable	GPDMA1_CH4	GPDMA1 channel 4 global interrupt	0x0000 00BC
32	40	Settable	GPDMA1_CH5	GPDMA1 channel 5 global interrupt	0x0000 00C0
33	41	Settable	GPDMA1_CH6	GPDMA1 channel 6 global interrupt	0x0000 00C4
34	42	Settable	GPDMA1_CH7	GPDMA1 channel 7 global interrupt	0x0000 00C8
35	43	Settable	IWDG	Independent watchdog interrupt	0x0000 00CC
36	44	Settable	SAES ⁽¹⁾	Secure AES	0x0000 00D0
37	45	Settable	ADC1	ADC1 global interrupt	0x0000 00D4
38	46	Settable	DAC1	DAC1 global interrupt	0x0000 00D8
39	47	Settable	FDCAN1_IT0	FDCAN1 Interrupt 0	0x0000 00DC
40	48	Settable	FDCAN1_IT1	FDCAN1 Interrupt 1	0x0000 00E0
41	49	Settable	TIM1_BRK/TIM1_TERR/ TIM1_IERR	TIM1 break/transition error/ index error	0x0000 00E4
42	50	Settable	TIM1_UP	TIM1 update	0x0000 00E8
43	51	Settable	TIM1_TRG_COM/ TIM1_DIR/TIM1_IDX	TIM1 trigger and commutation/direction change interrupt/index	0x0000 00EC
44	52	Settable	TIM1_CC	TIM1 capture compare interrupt	0x0000 00F0
45	53	Settable	TIM2	TIM2 global interrupt	0x0000 00F4
46	54	Settable	TIM3	TIM3 global interrupt	0x0000 00F8

Table 153. STM32H523/533xx vector table (continued)

Position	Priority	Type of priority	Acronym	Description	Address
47	55	Settable	TIM4	TIM4 global interrupt	0x0000 00FC
48	56	Settable	TIM5	TIM5 global interrupt	0x0000 0100
49	57	Settable	TIM6	TIM6 global interrupt	0x0000 0104
50	58	Settable	TIM7	TIM7 global interrupt	0x0000 0108
51	59	Settable	I2C1_EV	I2C1 event interrupt	0x0000 010C
52	60	Settable	I2C1_ER	I2C1 error interrupt	0x0000 0110
53	61	Settable	I2C2_EV	I2C2 event interrupt	0x0000 0114
54	62	Settable	I2C2_ER	I2C2 error interrupt	0x0000 0118
55	63	Settable	SPI1	SPI1 global interrupt	0x0000 011C
56	64	Settable	SPI2	SPI2 global interrupt	0x0000 0120
57	65	Settable	SPI3	SPI3 global interrupt	0x0000 0124
58	66	Settable	USART1	USART1 global interrupt	0x0000 0128
59	67	Settable	USART2	USART2 global interrupt	0x0000 012C
60	68	Settable	USART3	USART3 global interrupt	0x0000 0130
61	69	Settable	UART4	UART4 global interrupt	0x0000 0134
62	70	Settable	UART5	UART5 global interrupt	0x0000 0138
63	71	Settable	LPUART1	LPUART1 global interrupt or R wake-up or T wake-up through EXTI line	0x0000 013C
64	72	Settable	LPTIM1 or LPTIM1_AIT	LPTIM1 global interrupt or AIT through EXTI line	0x0000 0140
65	73	Settable	TIM8_BRK/TIM8_TERR/ TIM8_IERR	TIM8 break interrupt/transition error/index error	0x0000 0144
66	74	Settable	TIM8_UP	TIM8 update interrupt	0x0000 0148
67	75	Settable	TIM8_TRG_COM/ TIM8_DIR/TIM8_IDX	TIM8 trigger and commutation interrupt/ direction change interrupt/index	0x0000 014C
68	76	Settable	TIM8_CC	TIM8 capture compare interrupt	0x0000 0150
69	77	Settable	ADC2	ADC2 global interrupt	0x0000 0154
70	78	Settable	LPTIM2 or LPTIM2_AIT	LPTIM2 global interrupt or AIT through EXTI line	0x0000 0158
71	79	Settable	TIM15	TIM15 global interrupt	0x0000 015C
72	80	-	-	Reserved	0x0000 0160
73	81	-	-	Reserved	0x0000 0164
74	82	Settable	USB_FS	USB FS global interrupt	0x0000 0168
75	83	Settable	CRS	Clock recovery system global interrupt	0x0000 016C
76	84	Settable	UCPD1	UCPD1 global interrupt	0x0000 0170

Table 153. STM32H523/533xx vector table (continued)

Position	Priority	Type of priority	Acronym	Description	Address
77	85	Settable	FMC	FMC global interrupt	0x0000 0174
78	86	Settable	OCTOSPI1	OCTOSPI1 global interrupt	0x0000 0178
79	87	Settable	SDMMC1	SDMMC1 global interrupt	0x0000 017C
80	88	Settable	I2C3_EV	I2C3 event interrupt	0x0000 0180
81	89	Settable	I2C3_ER	I2C3 error interrupt	0x0000 0184
82	90	Settable	SPI4	SPI4 global interrupt	0x0000 0188
83	91	-	-	Reserved	0x0000 018C
84	92	-	-	Reserved	0x0000 0190
85	93	Settable	USART6	USART6 global interrupt	0x0000 0194
86	94	-	-	Reserved	0x0000 0198
87	95	-	-	Reserved	0x0000 019C
88	96	-	-	Reserved	0x0000 01A0
89	97	-	-	Reserved	0x0000 01A4
90	98	Settable	GPDMA2_CH0	GPDMA2 channel0 global interrupt	0x0000 01A8
91	99	Settable	GPDMA2_CH1	GPDMA2 channel1 global interrupt	0x0000 01AC
92	100	Settable	GPDMA2_CH2	GPDMA2 channel2 global interrupt	0x0000 01B0
93	101	Settable	GPDMA2_CH3	GPDMA2 channel3 global interrupt	0x0000 01B4
94	102	Settable	GPDMA2_CH4	GPDMA2 channel4 global interrupt	0x0000 01B8
95	103	Settable	GPDMA2_CH5	GPDMA2 channel5 global interrupt	0x0000 01BC
96	104	Settable	GPDMA2_CH6	GPDMA2 channel6 global interrupt	0x0000 01C0
97	105	Settable	GPDMA2_CH7	GPDMA2 channel7 global interrupt	0x0000 01C4
98	106	-	-	Reserved	0x0000 01C8
99	107	-	-	Reserved	0x0000 01CC
100	108	-	-	Reserved	0x0000 01D0
101	109	-	-	Reserved	0x0000 01D4
102	110	-	-	Reserved	0x0000 01D8
103	111	Settable	FPU	Floating point interrupt	0x0000 01DC
104	112	Settable	ICACHE	Instruction cache global interrupt	0x0000 01E0
105	113	Settable	DCACHE	Data cache global interrupt	0x0000 01E4
106	114	-	-	Reserved	0x0000 01E8
107	115	-	-	Reserved	0x0000 01EC
108	116	Settable	DCMI_PSSI	DCMI/PSSI global interrupt	0x0000 01F0
109	117	Settable	FDCAN2_IT0	FDCAN2 interrupt 0	0x0000 01F4

Table 153. STM32H523/533xx vector table (continued)

Position	Priority	Type of priority	Acronym	Description	Address
110	118	Settable	FDCAN2_IT1	FDCAN2 interrupt 1	0x0000 01F8
111	119	-	-	Reserved	0x0000 01FC
112	120	-	-	Reserved	0x0000 0200
113	121	Settable	DTS or DTS_WKUP	DTS interrupt or AIT through EXTI line	0x0000 0204
114	122	Settable	RNG	RNG global interrupt	0x0000 0208
115	123	Settable	OTFDEC1	OTFDEC1 secure global interrupt	0x0000 020C
116	124	Settable	AES ⁽¹⁾	AES global interrupt	0x0000 0210
117	125	Settable	HASH	HASH interrupt	0x0000 0214
118	126	Settable	PKA ⁽¹⁾	PKA global interrupt	0x0000 0218
119	127	Settable	CEC	HDMI-CEC global interrupt	0x0000 021C
120	128	Settable	TIM12	TIM12 global interrupt	0x0000 0220
121	129	-	-	Reserved	0x0000 0224
122	130	-	-	Reserved	0x0000 0228
123	131	Settable	I3C1_EV	I3C1 event interrupt	0x0000 022C
124	132	Settable	I3C1_ER	I3C1 error interrupt	0x0000 0230
125	133	-	-	Reserved	0x0000 0234
126	134	-	-	Reserved	0x0000 0238
127	135	-	-	Reserved	0x0000 023C
128	136	-	-	Reserved	0x 0000 0240
129	137	-	-	Reserved	0x0000 0244
130	138	-	-	Reserved	0x0000 0248
131	139	Settable	I3C2_EV	I3C2 event interrupt	0x0000 024C
132	140	Settable	I3C2_ER	I3C2 error interrupt	0x0000 0250
133	141	-	-	Reserved	0x0000 0254

1. Not available on STM32H523 devices.

Table 154. STM32H543/553xx vector table

Position	Priority	Type of priority	Acronym	Description	Address
-	-	-	-	Reserved	0x0000 0000
-	-4	Fixed	Reset	Reset	0x0000 0004

Table 154. STM32H543/553xx vector table (continued)

Position	Priority	Type of priority	Acronym	Description	Address
-	-2	Fixed	NMI	Non maskable interrupt. The RCC Clock Security System (CSS) is linked to the NMI vector.	0x0000 0008
-	-3 or -1	Fixed	Secure HardFault	Secure Hard fault	0x0000 000C
-	-1	Fixed	Nonsecure HardFault	Nonsecure Hard fault. All classes of fault	0x0000 000C
-	0	Settable	MemManage	Memory management	0x0000 0010
-	1	Settable	BusFault	Pre-fetch fault, memory access fault	0x0000 0014
-	2	Settable	UsageFault	Undefined instruction or illegal state	0x0000 0018
-	3	Settable	SecureFault	Secure Fault	0x0000 001C
-	-	-	-	Reserved	0x0000 0020-0x0000 0028
-	4	-	SVC	System service call via SWI instruction	0x0000 002C
-	5	-	Debug Monitor	Monitor	0x0000 0030
-	-	-	-	Reserved	0x0000 0034
-	6	Settable	PendSV	Pendable request for system service	0x0000 0038
-	7	Settable	SysTick	System tick timer	0x0000 003C
0	8	Settable	WWDG	Window Watchdog interrupt	0x0000 0040
1	9	Settable	PVD_AVD	Power Voltage Monitor Analog Voltage Monitor	0x0000 0044
2	10	Settable	RTC_NS	RTC global nonsecure interrupts	0x0000 0048
3	11	Settable	RTC_S	RTC global secure interrupts	0x0000 004C
4	12	Settable	TAMP	Tamper global interrupts	0x0000 0050
5	13	Settable	RAMCFG	RAM configuration global interrupt	0x0000 0054
6	14	Settable	FLASH	Flash nonsecure global interrupt	0x0000 0058
7	15	Settable	FLASH_S	Flash secure global interrupt	0x0000 005C
8	16	Settable	GTZC1	GTZC global interrupt	0x0000 0060
9	17	Settable	RCC	RCC nonsecure global interrupt	0x0000 0064
10	18	Settable	RCC_S	RCC secure global interrupt	0x0000 0068
11	19	Settable	EXTI0	EXTI Line0 interrupt	0x0000 006C
12	20	Settable	EXTI1	EXTI Line1 interrupt	0x0000 0070
13	21	Settable	EXTI2	EXTI Line2 interrupt	0x0000 0074
14	22	Settable	EXTI3	EXTI Line3 interrupt	0x0000 0078
15	23	Settable	EXTI4	EXTI Line4 interrupt	0x0000 007C
16	24	Settable	EXTI5	EXTI Line5 interrupt	0x0000 0080
17	25	Settable	EXTI6	EXTI Line6 interrupt	0x0000 0084

Table 154. STM32H543/553xx vector table (continued)

Position	Priority	Type of priority	Acronym	Description	Address
18	26	Settable	EXTI7	EXTI Line7 interrupt	0x0000 0088
19	27	Settable	EXTI8	EXTI Line8 interrupt	0x0000 008C
20	28	Settable	EXTI9	EXTI Line9 interrupt	0x0000 0090
21	29	Settable	EXTI10	EXTI Line10 interrupt	0x0000 0094
22	30	Settable	EXTI11	EXTI Line11 interrupt	0x0000 0098
23	31	Settable	EXTI12	EXTI Line12 interrupt	0x0000 009C
24	32	Settable	EXTI13	EXTI Line13 interrupt	0x0000 00A0
25	33	Settable	EXTI14	EXTI Line14 interrupt	0x0000 00A4
26	34	Settable	EXTI15	EXTI Line15 interrupt	0x0000 00A8
27	35	Settable	GPDMA1_CH0	GPDMA1 channel0 global interrupt	0x0000 00AC
28	36	Settable	GPDMA1_CH1	GPDMA1 channel1 global interrupt	0x0000 00B0
29	37	Settable	GPDMA1_CH2	GPDMA1 channel2 global interrupt	0x0000 00B4
30	38	Settable	GPDMA1_CH3	GPDMA1 channel3 global interrupt	0x0000 00B8
31	39	Settable	GPDMA1_CH4	GPDMA1 channel4 global interrupt	0x0000 00BC
32	40	Settable	GPDMA1_CH5	GPDMA1 channel5 global interrupt	0x0000 00C0
33	41	Settable	GPDMA1_CH6	GPDMA1 channel6 global interrupt	0x0000 00C4
34	42	Settable	GPDMA1_CH7	GPDMA1 channel7 global interrupt	0x0000 00C8
35	43	Settable	IWDG	Independent watchdog interrupt	0x0000 00CC
36	44	Settable	SAES	Secure AES	0x0000 00D0
37	45	Settable	ADC1	ADC1 global interrupt	0x0000 00D4
38	46	Settable	DAC1	DAC1 global interrupt	0x0000 00D8
39	47	Settable	FDCAN1_IT0	FDCAN1 Interrupt 0	0x0000 00DC
40	48	Settable	FDCAN1_IT1	FDCAN1 Interrupt 1	0x0000 00E0
41	49	Settable	TIM1_BRK/TIM1_TERR/ TIM1_IERR	TIM1 Break/ Transition error/ Index error	0x0000 00E4
42	50	Settable	TIM1_UP	TIM1 Update	0x0000 00E8
43	51	Settable	TIM1_TRG_COM/ TIM1_DIR/TIM1_IDX	TIM1 trigger and commutation/Direction Change interrupt/Index	0x0000 00EC
44	52	Settable	TIM1_CC	TIM1 capture compare interrupt	0x0000 00F0
45	53	Settable	TIM2	TIM2 global interrupt	0x0000 00F4
46	54	Settable	TIM3	TIM3 global interrupt	0x0000 00F8
47	55	Settable	TIM4	TIM4 global interrupt	0x0000 00FC
48	56	Settable	TIM5	TIM5 global interrupt	0x0000 0100
49	57	Settable	TIM6	TIM6 global interrupt	0x0000 0104

Table 154. STM32H543/553xx vector table (continued)

Position	Priority	Type of priority	Acronym	Description	Address
50	58	Settable	TIM7	TIM7 global interrupt	0x0000 0108
51	59	Settable	I2C1_EV	I2C1 event interrupt	0x0000 010C
52	60	Settable	I2C1_ERR	I2C1 error interrupt	0x0000 0110
53	61	Settable	I2C2_EV	I2C2 event interrupt	0x0000 0114
54	62	Settable	I2C2_ERR	I2C2 error interrupt	0x0000 0118
55	63	Settable	SPI1	SPI1 global interrupt	0x0000 011C
56	64	Settable	SPI2	SPI2 global interrupt	0x0000 0120
57	65	Settable	SPI3	SPI3 global interrupt	0x0000 0124
58	66	Settable	USART1	USART1 global interrupt	0x0000 0128
59	67	Settable	USART2	USART2 global interrupt	0x0000 012C
60	68	Settable	USART3	USART3 global interrupt	0x0000 0130
61	69	Settable	UART4	UART4 global interrupt	0x0000 0134
62	70	Settable	UART5	UART5 global interrupt	0x0000 0138
63	71	Settable	LPUART1	LPUART1 global interrupt OR LPUART1 R Wake-up OR LPUART1 T Wake-up through EXTI line	0x0000 013C
64	72	Settable	LPTIM1 or LPTIM1_AIT	LPTIM1 global interrupt or AIT through EXTI line	0x0000 0140
65	73	Settable	TIM8_BRK/TIM8_TERR/ TIM8_IERR	TIM8 Break interrupt/Transition error/Index error	0x0000 0144
66	74	Settable	TIM8_UP	TIM8 Update interrupt	0x0000 0148
67	75	Settable	TIM8_TRG_COM/ TIM8_DIR/TIM8_IDX	TIM8 trigger and commutation interrupt/Direction Change interrupt/Index	0x0000 014C
68	76	Settable	TIM8_CC	TIM8 capture compare interrupt	0x0000 0150
69	77	Settable	ADC2	ADC2 global interrupt	0x0000 0154
70	78	Settable	LPTIM2 or LPTIM2_AIT	LPTIM2 global interrupt or AIT through EXTI line	0x0000 0158
71	79	Settable	TIM15	TIM15 global interrupt	0x0000 015C
72	80	-	-	Reserved	0x0000 0160
73	81	-	-	Reserved	0x0000 0164
74	82	Settable	USB_FS	USB FS global interrupt	0x0000 0168
75	83	Settable	CRS	Clock Recovery System global interrupt	0x0000 016C
76	84	Settable	UCPD1	UCPD1 global interrupt	0x0000 0170
77	85	Settable	FMC	FMC global interrupt	0x0000 0174
78	86	Settable	OCTOSPI1	OCTOSPI1 global interrupt	0x0000 0178
79	87	Settable	SDMMC1	SDMMC1 global interrupt	0x0000 017C

Table 154. STM32H543/553xx vector table (continued)

Position	Priority	Type of priority	Acronym	Description	Address
80	88	Settable	I2C3_EV	I2C3 event interrupt	0x0000 0180
81	89	Settable	I2C3_ERR	I2C3 error interrupt	0x0000 0184
82	90	Settable	SPI4	SPI4 global interrupt	0x0000 0188
83	91	-	-	Reserved	0x0000 018C
84	92	-	-	Reserved	0x0000 0190
85	93	Settable	USART6	USART6 global interrupt	0x0000 0194
86	94	-	-	Reserved	0x0000 0198
87	95	-	-	Reserved	0x0000 019C
88	96	-	-	Reserved	0x0000 01A0
89	97	-	-	Reserved	0x0000 01A4
90	98	Settable	GPDMA2_CH0	GPDMA2 channel0 global interrupt	0x0000 01A8
91	99	Settable	GPDMA2_CH1	GPDMA2 channel1 global interrupt	0x0000 01AC
92	100	Settable	GPDMA2_CH2	GPDMA2 channel2 global interrupt	0x0000 01B0
93	101	Settable	GPDMA2_CH3	GPDMA2 channel3 global interrupt	0x0000 01B4
94	102	Settable	GPDMA2_CH4	GPDMA2 channel4 global interrupt	0x0000 01B8
95	103	Settable	GPDMA2_CH5	GPDMA2 channel5 global interrupt	0x0000 01BC
96	104	Settable	GPDMA2_CH6	GPDMA2 channel6 global interrupt	0x0000 01C0
97	105	Settable	GPDMA2_CH7	GPDMA2 channel7 global interrupt	0x0000 01C4
98	106	Settable	UART7	UART7 global interrupt	0x0000 01C8
99	107	Settable	UART8	UART8 global interrupt	0x0000 01CC
100	108	-	-	Reserved	0x0000 01D0
101	109	-	-	Reserved	0x0000 01D4
102	110	-	-	Reserved	0x0000 01D8
103	111	Settable	FPU	Floating point interrupt	0x0000 01DC
104	112	Settable	ICACHE1	Instruction cache global interrupt	0x0000 01E0
105	113	Settable	DCACHE1	Data cache global interrupt	0x0000 01E4
106	114	Settable	ETH	Ethernet interrupt	0x0000 01E8
107	115	Settable	ETH_WKUP	ETHERNET Wake-up interrupt through EXTI line	0x0000 01EC
108	116	-	-	Reserved	0x0000 01F0
109	117	Settable	FDCAN2_IT0	FDCAN2 Interrupt 0	0x0000 01F4
110	118	Settable	FDCAN2_IT1	FDCAN2 Interrupt 1	0x0000 01F8
111	119	Settable	CORDIC	CORDIC interrupt	0x0000 01FC
112	120	Settable	-	Reserved	0x0000 0200

Table 154. STM32H543/553xx vector table (continued)

Position	Priority	Type of priority	Acronym	Description	Address
113	121	Settable	DTS or DTS_WKUP	DTS interrupt or AIT through EXTI line	0x0000 0204
114	122	Settable	RNG	RNG global interrupt	0x0000 0208
115	123	Settable	OTFDEC1	OTFDEC1 secure global interrupt	0x0000 020C
116	124	Settable	AES	AES global interrupt	0x0000 0210
117	125	Settable	HASH	HASH interrupt	0x0000 0214
118	126	Settable	PKA	PKA global interrupt	0x0000 0218
119	127	Settable	CEC	HDMI-CEC global interrupt	0x0000 021C
120	128	Settable	TIM12	TIM12 global interrupt	0x0000 0220
121	129	-	-	Reserved	0x0000 0224
122	130	-	-	Reserved	0x0000 0228
123	131	Settable	I3C1_EV	I3C1 event interrupt	0x0000 022C
124	132	Settable	I3C1_ERR	I3C1 error interrupt	0x0000 0230
125	133	-	-	Reserved	0x0000 0234
126	134	-	-	Reserved	0x0000 0238
127	135	-	-	Reserved	0x0000 023C
128	136	-	-	Reserved	0x0000 0240
129	137	-	-	Reserved	0x0000 0244
130	138	-	-	Reserved	0x0000 0248
131	139	Settable	I3C2_EV	I3C2 event interrupt	0x0000 024C
132	140	Settable	I3C2_ERR	I3C2 error interrupt	0x0000 0250
133	141	Settable	COMP	Comparator global interrupt	0x0000 0254
134	142	-	-	Reserved	0x0000 0258
135	143	-	-	Reserved	0x0000 025C
136	144	-	-	Reserved	0x0000 0260
137	145	-	-	Reserved	0x0000 0264
138	146	-	-	Reserved	0x0000 0268
139	147	-	-	Reserved	0x0000 026C
140	148	-	-	Reserved	0x0000 0270
141	149	-	-	Reserved	0x0000 0274
142	150	-	-	Reserved	0x0000 0278
143	151	-	-	Reserved	0x0000 027C
144	152	-	-	Reserved	0x0000 0280
145	153	-	-	Reserved	0x0000 0284

Table 154. STM32H543/553xx vector table (continued)

Position	Priority	Type of priority	Acronym	Description	Address
146	154	-	-	Reserved	0x0000 0288
147	155	Settable	PLAY1_NS	PLA global nonsecure interrupt	0x0000 028C
148	156	Settable	PLAY1_S	PLA global secure interrupt	0x0000 0290
149	157	Settable	ADC3	ADC3 global interrupt	0x0000 0294

18 Extended interrupts and event controller (EXTI)

18.1 EXTI introduction

The extended interrupts and event controller (EXTI) manages the individual CPU and system wake-up through configurable and direct event inputs. It provides wake-up requests to the power control and generates an interrupt request to the CPU NVIC and events to the CPU event input. For the CPU, an additional event generation block (EVG) is needed to generate the CPU event signal.

The EXTI wake-up requests allow the system to be woken up from Stop modes.

The interrupt request and event request generation can be used also in Run modes.

The EXTI also includes the EXTI mux IO port selection.

18.2 EXTI main features

The EXTI main features are the following:

- Up to 67 input events supported
- All event inputs allow the possibility to wake up the system.
- Events that do not have an associated wake-up flag in the peripheral, have a flag in the EXTI and generate an interrupt to the CPU from the EXTI.
- Events can be used to generate a CPU wake-up event.

The asynchronous event inputs are classified in two groups:

- Configurable events (signals from I/Os or peripherals able to generate a pulse), with the following features
 - Selectable active trigger edge
 - Interrupt pending status register bits independent for the rising and falling edge
 - Individual interrupt and event generation mask, used for conditioning the CPU wake-up, interrupt and event generation
 - Software trigger possibility
 - Secure events: The access to control and configuration bits of secure input events can be made secure and or privilege.
 - EXTI IO port selection
- Direct events (interrupt and wake-up sources from peripherals having an associated flag which requires to be cleared in the peripheral), with the following features:
 - Fixed rising edge active trigger
 - No interrupt pending status register bit in the EXTI (the interrupt pending status flag is provided by the peripheral generating the event)
 - Individual interrupt and event generation mask, used to condition the CPU wake-up and event generation
 - No software trigger possibility

18.3 EXTI block diagram

The EXTI consists of a register block accessed via an AHB interface, the event input trigger block, the masking block and EXTI mux as shown in [Figure 96](#).

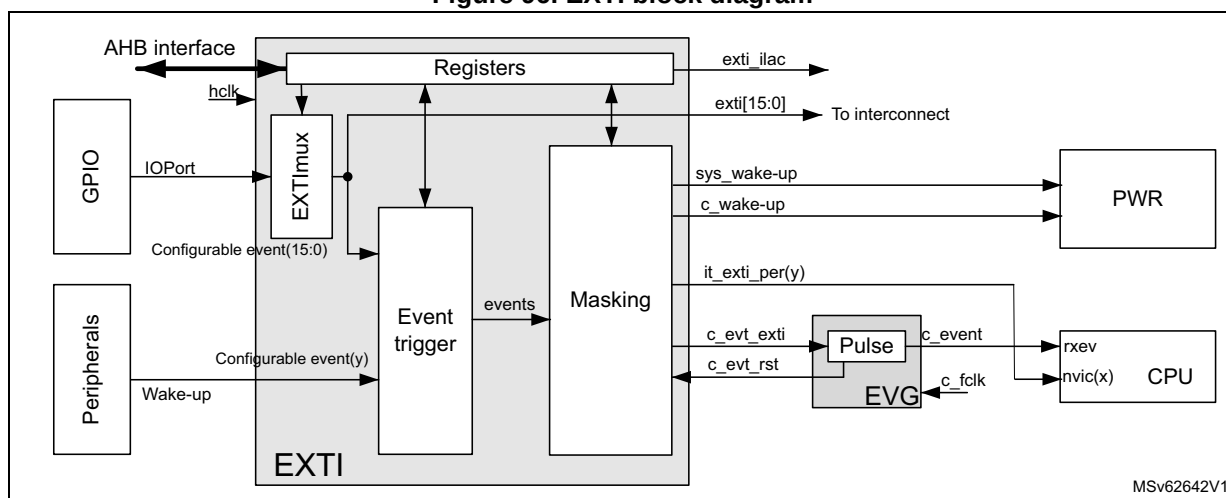
The register block contains all the EXTI registers.

The event input trigger block provides event input edge trigger logic.

The masking block provides the event input distribution to the different wake-up, interrupt and event outputs, and their masking.

The EXTI mux provides the IO port selection on to the EXTI event signal.

Figure 96. EXTI block diagram



MSv62642V1

Table 155. EXTI signals

Pin name	I/O	Description
AHB interface	I/O	EXTI register bus interface. When one event is configured to enable security, the AHB interface supports secure accesses.
hclk	I	AHB bus clock and EXTI system clock
Configurable event(y)	I	Asynchronous wake-up events from peripherals without an associated interrupt and flag
Direct event(x)	I	Synchronous and asynchronous wake-up events from peripherals with an associated interrupt and flag
exti_ilac	O	Illegal access event
IOPort(n)	I	GPIOs block IO ports[15:0]
exti[15:0]	O	EXTI GPIO output port to trigger other peripherals
it_exti_per (y)	O	Interrupts to the CPU associated with configurable event (y)
c_evt_exti	O	High-level sensitive event output for CPU, synchronous to hclk
c_evt_rst	I	Asynchronous reset input to clear c_evt_exti
sys_wakeup	O	Asynchronous system wake-up request to PWR for ck_sys and hclk
c_wakeup	O	Wake-up request to PWR for CPU, synchronous to hclk

Table 156. EVG signals

Pin name	I/O	Description
c_fclk	I	CPU free running clock
c_evt_in	I	High-level sensitive events input from EXTI, asynchronous to CPU clock
c_event	O	Event pulse, synchronous to CPU clock
c_evt_rst	O	Event reset signal, synchronous to CPU clock

18.3.1 EXTI connections between peripherals and CPU

Some peripherals able to generate wake-up or interrupt events when the system is in Stop mode, are connected to the EXTI.

- Peripheral wake-up signals that generate a pulse or do not have an interrupt status bits in the peripheral, are connected to an EXTI configurable event input. For these events, the EXTI provides a status pending bit to be cleared. It is the EXTI interrupt, associated with the status bit, that interrupts the CPU.
- Peripheral interrupt and wake-up signals with a status bit in the peripheral to be cleared are connected to an EXTI direct event input. There is no status pending bit within the EXTI. The interrupt or wake-up is cleared by the CPU in the peripheral. It is the peripheral interrupt that interrupts the CPU directly.

All GPIO ports input to the EXTI multiplexer allow the selection of a port pin to wake up the system via a configurable event.

The EXTI configurable event interrupts are connected to the NVIC.

The dedicated EXTI/EVG CPU event is connected to the CPU rxev input.

The EXTI CPU wake-up signals are connected to the PWR and are used to wake up the system and the CPU sub-system bus clocks.

18.3.2 EXTI interrupt/event mapping

The EXTI lines are connected as shown in the following table.

Table 157. EXTI line connections

EXTI Line	Line source	Line type
0-15	GPIO	Configurable
16	PVD/AVD output	Configurable
17	RTC nonsecure	-
18	RTC secure	-
19	TAMP nonsecure	-
20	TAMP secure	-
21	I2C1 wake-up	-
22	I2C2 wake-up	-
23	I2C3 wake-up	-
24	I3C1 wake-up	-

Table 157. EXTI line connections (continued)

EXTI Line	Line source	Line type
25	USART1 wake-up	-
26	USART2 wake-up	-
27	USART3 wake-up	-
28	UART4 wake-up	-
29	UART5 wake-up	-
30	USART6 wake-up	-
31	UART7 wake-up ⁽¹⁾	-
32	UART8 wake-up ⁽¹⁾	-
33	UART9 wake-up ⁽²⁾	-
34	USART10 wake-up ⁽²⁾	-
35	USART11 wake-up ⁽²⁾	-
36	UART12 wake-up ⁽²⁾	-
37	LPUART1 wake-up	-
38	LPTIM1	-
39	LPTIM2	-
40	SPI1 wake-up	-
41	SPI2 wake-up	-
42	SPI3 wake-up	-
43	SPI4 wake-up	-
44	SPI5 wake-up ⁽²⁾	-
45	SPI6 wake-up ⁽²⁾	-
46	ETH wake-up ⁽¹⁾	Configurable
47	USB FS wake-up	-
48	UCPD1 wake-up	-
49	LPTIM2 CH1	Configurable ⁽⁴⁾
50	DTS wake-up	Configurable
51	HDMI-CEC wake-up	-
52	I2C4 wake-up ⁽²⁾	-
53	UVM output	Configurable
54	LPTIM3 ⁽²⁾	-
55	LPTIM4 ⁽²⁾	-
56	LPTIM5 ⁽²⁾	-
57	LPTIM6 ⁽²⁾	-
58	I3C2 ⁽³⁾ /COMP1 output ⁽⁴⁾	-/Configurable ⁽⁴⁾
59	COMP2 output ⁽⁴⁾	Configurable

Table 157. EXTI line connections (continued)

EXTI Line	Line source	Line type
60	Reserved	-
61	I3C2 wake-up ⁽⁴⁾	-
62	Reserved	-
63	Reserved	-
64	play_out14 ⁽⁴⁾	Configurable
65	IWDG ⁽⁴⁾	-
66	V _{DDIO2} Voltage monitor ⁽⁴⁾	Configurable

1. Not available on STM32H523/33xx devices.
2. Not available on STM32H523/33/43/53xx devices.
3. Available only on STM32H523/33xx devices.
4. Available only on STM32H543/553xx devices.

18.4 EXTI functional description

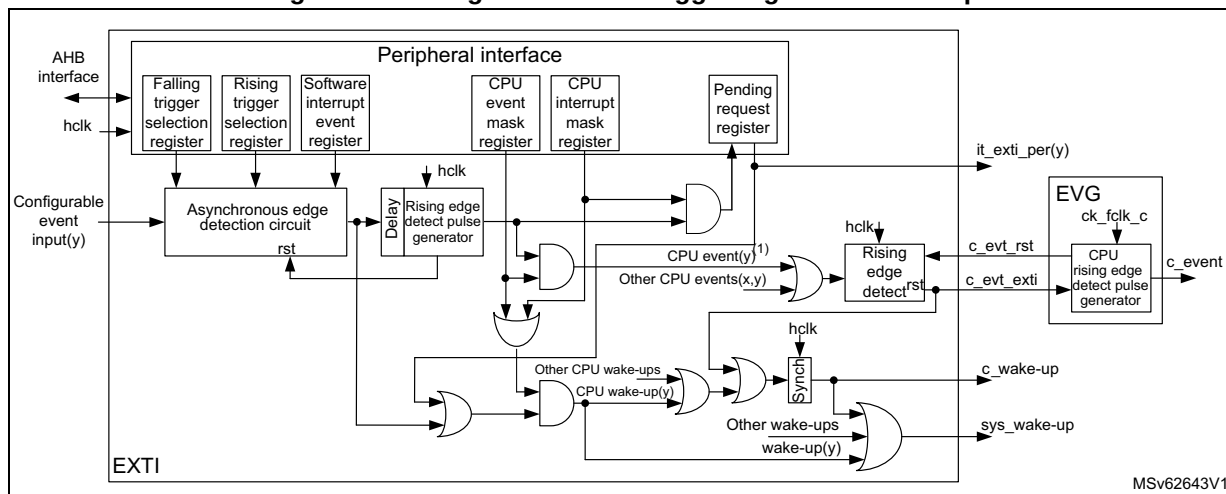
The events features are controlled from register bits as follows:

- Active trigger edge enable
 - by rising edge selection in *EXTI rising trigger selection register (EXTI_RTSR1)* and *EXTI rising trigger selection register 2 (EXTI_RTSR2)*, *EXTI rising trigger selection register 2 (EXTI_RTSR2)*, and *EXTI rising trigger selection register 3 (EXTI_RTSR3)*
 - by falling edge selection in *EXTI falling trigger selection register (EXTI_FTSR1)*, *EXTI falling trigger selection register 2 (EXTI_FTSR2)*, and *EXTI falling trigger selection register 3 (EXTI_FTSR3)*
- Software trigger in the *EXTI software interrupt event register (EXTI_SWIER1)*, *EXTI software interrupt event register 2 (EXTI_SWIER2)*, and *EXTI software interrupt event register 3 (EXTI_SWIER3)*
- Interrupt pending flag in:
 - *EXTI rising edge pending register (EXTI_RPR1)*, *EXTI rising edge pending register 2 (EXTI_RPR2)*, and *EXTI rising edge pending register 3 (EXTI_RPR3)*
 - *EXTI falling edge pending register (EXTI_FPR1)*, *EXTI falling edge pending register 2 (EXTI_FPR2)* and *EXTI falling edge pending register 3 (EXTI_FPR3)*
- CPU wake-up and interrupt enable in *EXTI CPU wake-up with interrupt mask register (EXTI_IMR1)*, *EXTI CPU wake-up with interrupt mask register 2 (EXTI_IMR2)*, and *EXTI CPU wake-up with interrupt mask register 3 (EXTI_IMR3)*
- CPU wake-up and event enable *EXTI CPU wake-up with event mask register (EXTI_EMR1)*, *EXTI CPU wake-up with event mask register 2 (EXTI_EMR2)*, and *EXTI CPU wake-up with event mask register 3 (EXTI_EMR3)*

18.4.1 EXTI configurable event input wake-up

The figure below is a detailed representation of the logic associated with configurable event inputs that wake up the CPU sub-system bus clocks and generate an EXTI pending flag and interrupt to the CPU, and/or a CPU wake-up event.

Figure 97. Configurable event trigger logic CPU wake-up



1. Only for the input events that support CPU rxev generation c_event.

The software interrupt event register allows configurable events to be triggered by software, writing the corresponding register bit, whatever the edge selection setting.

The configurable event active trigger edge (or both edges) is selected and enabled in the rising/falling edge selection registers.

The CPU has its dedicated wake-up (interrupt) mask register and a dedicated event mask registers. When the event is enabled, it is generated to the CPU. All events for the CPU are ORed together into a single CPU event signal. The event pending registers (EXTI_RPR and EXTI_FPR) are not set for an unmasked CPU event.

The configurable events have unique interrupt pending request registers. The pending register is only set for an unmasked interrupt. Each configurable event provides a common interrupt to the CPU. The configurable event interrupts must be acknowledged by software in the EXTI_RPR and/or EXTI_FPR registers.

When a CPU wake-up (interrupt) or CPU event is enabled, the asynchronous edge detection circuit is reset by the clocked delay and rising edge detect pulse generator. This guarantees that the EXTI hclk clock is woken up before the asynchronous edge detection circuit is reset.

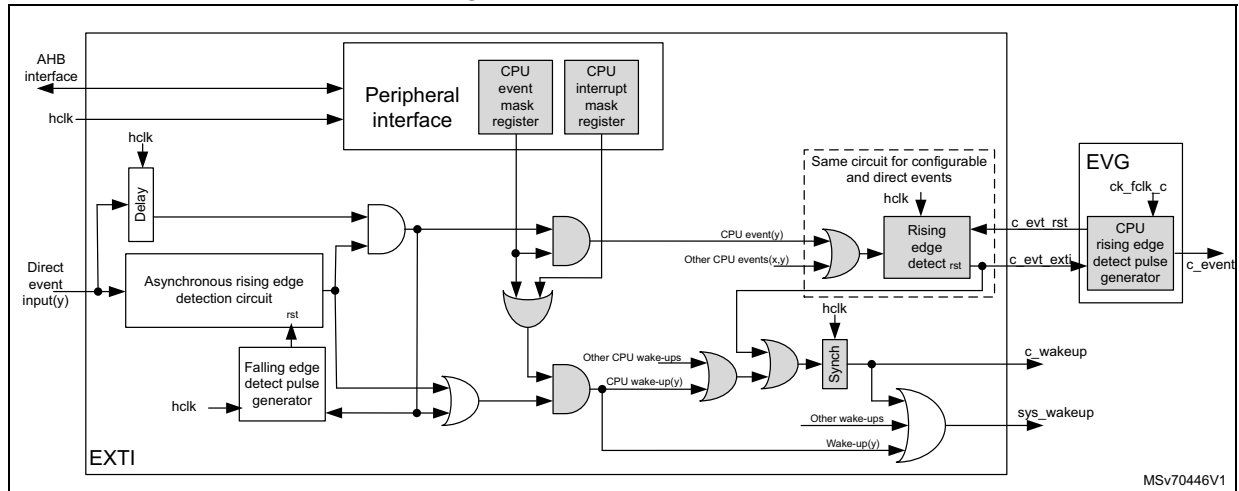
Note: A detected configurable event interrupt pending request can be cleared by the CPU with the correct access permission. The system is not able to enter into low-power modes as long as an interrupt pending request is active.

18.4.2 EXTI direct event input wake-up

The direct events do not have an associated EXTI interrupt. The EXTI only wakes up the system and CPU subsystem clocks, and can generate a CPU wake-up event. The peripheral synchronous interrupt, associated with the direct wake-up event, wakes up the CPU.

The EXTI direct event is able to generate a CPU event that wakes up the CPU. The CPU event may occur before the associated peripheral interrupt flag is set.

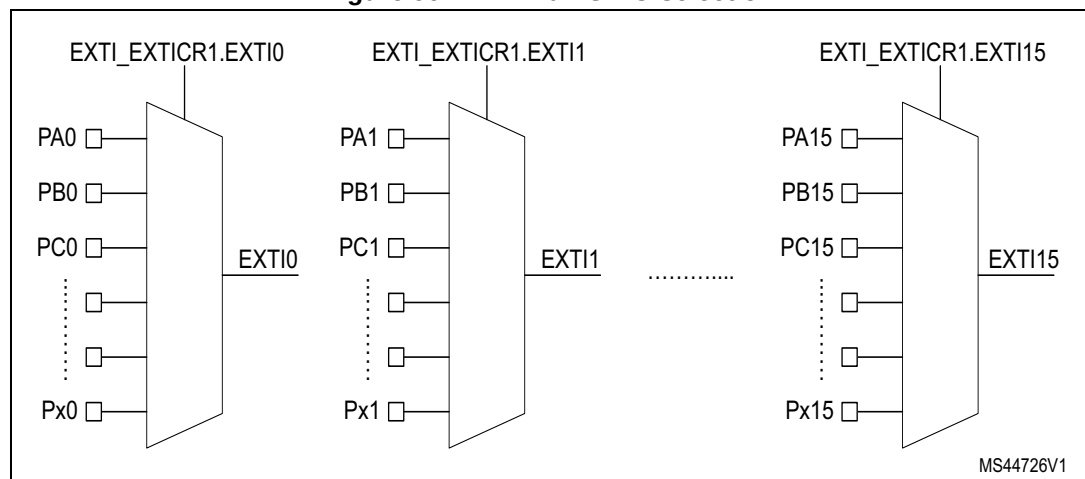
Figure 98. EXTI direct events



18.4.3 EXTI mux selection

The EXTI mux allows the selection of GPIOs as interrupts and wake-up. GPIOs are connected via 16 EXTI mux lines to the first 16 EXTI events as configurable event. The selection of GPIO port as EXTI mux output is controlled in [EXTI rising trigger selection register 3 \(EXTI_RTSR3\)](#).

Figure 99. EXTI mux GPIO selection



The EXTI mux outputs are available as output signals from the EXTI to trigger other peripherals, whatever the masking in EXTI_IMR and EXTI_EMR registers.

18.5 EXTI functional behavior

The configurable events are enabled by enabling at least one of the trigger edges.

Once an event input is enabled, the CPU wake-up generation is conditioned by the CPU interrupt mask and CPU event mask.

Table 158. Masking functionality

CPU interrupt enable (in EXTI_IMR.IMn)	CPU event enable (in EXTI_EMR.EMn)	Configurable event inputs (in EXTI_RPR.RPIFn and EXTI_FPR.FPIFn)	Exti(n) interrupt ⁽¹⁾	CPU event	CPU wake-up
0	0	No	Masked	Masked	Masked
	1	No	Masked	Yes	Yes
1	0	Status latched	Yes	Masked	Yes ⁽²⁾
	1	Status latched	Yes	Yes	Yes

1. The single exti(n) interrupt goes to the CPU. If no interrupt is required for CPU(m), the exti(n) interrupt must be masked in the CPU NVIC.
2. Only if CPU interrupt is enabled in EXTI_IMR.IMn.

For configurable event inputs, when the enabled edges occur on the event input, an event request is generated. When the associated CPU interrupt is unmasked, the corresponding pending bits EXTI_RPR.RPIFn and/or EXTI_FPR.FPIFn is/are set: the CPU sub-system is woken up and the CPU interrupt signal is activated. The EXTI_RPR.RPIFn and/or EXTI_FPR.FPIFn pending bits must be cleared by software writing it to 1. This action clears the CPU interrupt.

For the configurable event inputs, an event request can be generated by software when writing a 1 in the software interrupt/event register EXTI_SWIER, allowing the generation of a rising edge on the event. The rising edge event pending bit is set in EXTI_RPR, whatever the setting in EXTI_RTISR.

18.6 EXTI event protection

The EXTI is able to protect event register bits from being modified by nonsecure and unprivileged accesses. The protection is individually activated per input event via the register bits in EXTI_SECCFGR and EXTI_PRIVCFGR. At EXTI level, the protection consists in preventing the following unauthorized write access:

- Change the settings of the secure and/or privileged configurable events.
- Change the masking of the secure and/or privileged input events.
- Clear pending status of the secure and/or privileged input events.

Table 159. Register protection overview

Register name	Access type	Protection ⁽¹⁾⁽²⁾
EXTI_RTISR	RW	Security and privilege can be bit-wise enabled in EXTI_SECCFGR and EXTI_PRIVCFGR.
EXTI_FTISR	RW	
EXTI_SWIER	RW	
EXTI_RPR	RW	
EXTI_FPR	RW	

Table 159. Register protection overview

Register name	Access type	Protection ⁽¹⁾⁽²⁾
EXTI_SECCFGR	RW	Always secure. Privilege can be bit-wise enabled in EXTI_PRIVCFGR.
EXTI_PRIVCFGR	RW	Always privilege. Security can be bit-wise enabled in EXTI_SECCFGR.
EXTI_EXTICRn	RW	Security and privilege can be bit-wise enabled in EXTI_SECCFGR and EXTI_PRIVCFGR.
EXTI_LOCKR	RW	Always secure
EXTI_IM	RW	Security and privilege can be bit-wise enabled in EXTI_SECCFGR and EXTI_PRIVCFGR.
EXTI_EMR	RW	

1. Security is enabled with the individual input event (EXTI_SECCFGR register).

2. Privilege is enabled with the individual Input event (EXTI_PRIVCFGR register).

18.6.1 EXTI security protection

When security is enabled for an input event, the associated input event configuration and control bits can only be modified and read by a secure access. A nonsecure write access is discarded and a read returns 0.

When input events are nonsecure, the security is disabled. The associated input event configuration and control bits can be modified and read by a secure access and nonsecure access.

The security configuration in registers EXTI_SECCFGR can be globally locked after reset by EXTI_LOCKR.LOCK.

18.6.2 EXTI privilege protection

When privilege is enabled for an input event, the associated input event configuration and control bits can only be modified and read by a privileged access. An unprivileged write access is discarded and a read returns 0.

When input events are unprivileged, the privilege is disabled. The associated input event configuration and control bits can be modified and read by a privileged access and unprivileged access.

The privileged configuration in registers EXTI_PRIVCFGR can be globally locked after reset by EXTI_LOCKR.LOCK.

18.7 EXTI registers

Table 160. EXTI register map sections

Address offset	Description
0x000 - 0x01C	General configurable event [31:0] configuration
0x020 - 0x03C	General configurable event [64:32] configuration
0x040 - 0x05C	General configurable event [95:64] configuration
0x060 - 0x06C	EXTI IO port mux selection
0x070	EXTI protection lock configuration
0x080 - 0x0BC	CPU input event configuration

All registers can be accessed with word (32-bit), half-word (16-bit) and byte (8-bit) access.

18.7.1 EXTI rising trigger selection register (EXTI_RTSR1)

Address offset: 0x000

Reset value: 0x0000 0000

Contains only register bits for configurable events.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	RT16
															rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
RT15	RT14	RT13	RT12	RT11	RT10	RT9	RT8	RT7	RT6	RT5	RT4	RT3	RT2	RT1	RT0
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bits 31:17 Reserved, must be kept at reset value.

Bits 16:0 **RTx**: Rising trigger event configuration bit of configurable event input $x^{(1)}$ ($x = 16$ to 0)

When EXTI_SECCFGR.SECx is disabled, RTx can be accessed with nonsecure and secure access.

When EXTI_SECCFGR.SECx is enabled, RTx can be accessed only with secure access.

Nonsecure write to this bit x is discarded and nonsecure read returns 0.

When EXTI_PRIVCFGR.PRIVx is disabled, RTx can be accessed with unprivileged and privileged access.

When EXTI_PRIVCFGR.PRIVx is enabled, RTx can be accessed only with privileged access. Unprivileged write to this bit x is discarded, unprivileged read returns 0.

0: Rising trigger disabled (for event and interrupt) for input line

1: Rising trigger enabled (for event and interrupt) for input line

- The configurable event inputs are edge triggered, no glitch must be generated on these inputs. If a rising edge on the configurable event input occurs during writing of the register, the associated pending bit is not set. Rising and falling edge triggers can be set for the same configurable event input. In this case, both edges generate a trigger.

18.7.2 EXTI falling trigger selection register (EXTI_FTSR1)

Address offset: 0x004

Reset value: 0x0000 0000

Contains only register bits for configurable events.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	FT16
															rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
FT15	FT14	FT13	FT12	FT11	FT10	FT9	FT8	FT7	FT6	FT5	FT4	FT3	FT2	FT1	FT0
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bits 31:17 Reserved, must be kept at reset value.

Bits 16:0 **FTx**: Falling trigger event configuration bit of configurable event input x ⁽¹⁾ (x = 16 to 0)

When EXTI_SECCFGR.SECx is disabled, FTx can be accessed with nonsecure and secure access.

When EXTI_SECCFGR.SECx is enabled, FTx can be accessed only with secure access. Nonsecure write to this FTx is discarded, nonsecure read returns 0.

When EXTI_PRIVCFGR.PRIVx is disabled, FTx can be accessed with unprivileged and privileged access.

When EXTI_PRIVCFGR.PRIVx is enabled, FTx can be accessed only with privileged access. Unprivileged write to this FTx is discarded, unprivileged read returns 0.

0: Falling trigger disabled (for event and Interrupt) for input line

1: Falling trigger enabled (for event and Interrupt) for input line.

- The configurable event inputs are edge triggered, no glitch must be generated on these inputs. If a falling edge on the configurable event input occurs during writing of the register, the associated pending bit is not set. Rising and falling edge triggers can be set for the same configurable event input. In this case, both edges generate a trigger.

18.7.3 EXTI software interrupt event register (EXTI_SWIER1)

Address offset: 0x008

Reset value: 0x0000 0000

Contains only register bits for configurable events.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	SWI16
															rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
SWI15	SWI14	SWI13	SWI12	SWI11	SWI10	SWI9	SWI8	SWI7	SWI6	SWI5	SWI4	SWI3	SWI2	SWI1	SWI0
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bits 31:17 Reserved, must be kept at reset value.

Bits 16:0 **SWIx**: Software interrupt on event x (x = 16 to 0)

When EXTI_SECCFGR.SECx is disabled, SWIx can be accessed with nonsecure and secure access.

When EXTI_SECCFGR.SECx is enabled, SWIx can be accessed only with secure access. Nonsecure write to this SWI x is discarded, nonsecure read returns 0.

When EXTI_PRIVCFGR.PRIVx is disabled, SWIx can be accessed with unprivileged and privileged access.

When EXTI_PRIVCFGR.PRIVx is enabled, SWIx can be accessed only with privileged access. Unprivileged write to this SWIx is discarded, unprivileged read returns 0.

A software interrupt is generated independent from the setting in EXTI_RTISR and EXTI_FTSR. It always returns 0 when read.

0: Writing 0 has no effect.

1: Writing 1 triggers a rising edge event on event x. This bit is auto cleared by hardware.

18.7.4 EXTI rising edge pending register (EXTI_RPR1)

Address offset: 0x00C

Reset value: 0x0000 0000

Contains only register bits for configurable events.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	RPIF16
															rW
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
RPIF15	RPIF14	RPIF13	RPIF12	RPIF11	RPIF10	RPIF9	RPIF8	RPIF7	RPIF6	RPIF5	RPIF4	RPIF3	RPIF2	RPIF1	RPIF0
rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW

Bits 31:17 Reserved, must be kept at reset value.

Bits 16:0 **RPIF_x**: configurable event inputs x rising edge pending bit (x = 16 to 0)

When EXTI_SECCFGR.SECx is disabled, RPIF_x can be accessed with nonsecure and secure access.

When EXTI_SECCFGR.SECx is enabled, RPIF_x can be accessed only with secure access. Nonsecure write to this RPIF_x is discarded, nonsecure read returns 0.

When EXTI_PRIVCFGR.PRIVx is disabled, RPIF_x can be accessed with unprivileged and privileged access.

When EXTI_PRIVCFGR.PRIVx is enabled, RPIF_x can be accessed only with privileged access. Unprivileged write to this RPIF_x is discarded, unprivileged read returns 0.

0: No rising edge trigger request occurred

1: Rising edge trigger request occurred

This bit is set when the rising edge event or an EXTI_SWIER software trigger arrives on the configurable event line. This bit is cleared by writing 1 to it.

18.7.5 EXTI falling edge pending register (EXTI_FPR1)

Address offset: 0x010

Reset value: 0x0000 0000

Contains only register bits for configurable events.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	FPIF16
															rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
FPIF15	FPIF14	FPIF13	FPIF12	FPIF11	FPIF10	FPIF9	FPIF8	FPIF7	FPIF6	FPIF5	FPIF4	FPIF3	FPIF2	FPIF1	FPIF0
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bits 31:17 Reserved, must be kept at reset value.

Bits 16:0 **FPIFx**: configurable event inputs x falling edge pending bit (x = 16 to 0)

When EXTI_SECCFGR.SECx is disabled, FPIFx can be accessed with nonsecure and secure access.

When EXTI_SECCFGR.SECx is enabled, FPIFx can be accessed only with secure access. Nonsecure write to this FPIFx is discarded, nonsecure read returns 0.

When EXTI_PRIVCFGR.PRIVx is disabled, FPIFx can be accessed with unprivileged and privileged access.

When EXTI_PRIVCFGR.PRIVx is enabled, FPIFx can be accessed only with privileged access. Unprivileged write to this FPIFx is discarded, unprivileged read returns 0.

0: No falling edge trigger request occurred

1: Falling edge trigger request occurred

This bit is set when the falling edge event arrives on the configurable event line. This bit is cleared by writing 1 to it.

18.7.6 EXTI security configuration register (EXTI_SECCFGR1)

Address offset: 0x014

Reset value: 0x0000 0000

This register provides write access security, a nonsecure write access is ignored and causes the generation of an illegal access event. A nonsecure read returns the register data.

Contains only register bits for security capable input events.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
SEC31	SEC30	SEC29	SEC28	SEC27	SEC26	SEC25	SEC24	SEC23	SEC22	SEC21	SEC20	SEC19	SEC18	SEC17	SEC16
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
SEC15	SEC14	SEC13	SEC12	SEC11	SEC10	SEC9	SEC8	SEC7	SEC6	SEC5	SEC4	SEC3	SEC2	SEC1	SEC0
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bits 31:0 **SECx**: Security enable on event input x (x = 31 to 0)

When EXTI_PRIVCFGR.PRIVx is disabled, SECx can be accessed with privileged and unprivileged access.

When EXTI_PRIVCFGR.PRIVx is enabled, SECx can only be written with privileged access. Unprivileged write to this SECx is discarded.

0: Event security disabled (nonsecure)

1: Event security enabled (secure)

18.7.7 EXTI privilege configuration register (EXTI_PRIVCFGR1)

Address offset: 0x018

Reset value: 0x0000 0000

This register provides privileged write access protection. An unprivileged read returns the register data.

Contains only register bits for privilege capable input events.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
PRIV31	PRIV30	PRIV29	PRIV28	PRIV27	PRIV26	PRIV25	PRIV24	PRIV23	PRIV22	PRIV21	PRIV20	PRIV19	PRIV18	PRIV17	PRIV16
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
PRIV15	PRIV14	PRIV13	PRIV12	PRIV11	PRIV10	PRIV9	PRIV8	PRIV7	PRIV6	PRIV5	PRIV4	PRIV3	PRIV2	PRIV1	PRIV0
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bits 31:0 **PRIVx**: Security enable on event input x (x = 31 to 0)

When EXTI_SECCFGR.SECx is disabled, PRIVx can be accessed with secure and nonsecure access.

When EXTI_SECCFGR.SECx is enabled, PRIVx can only be written with secure access. Nonsecure write to this PRIVx is discarded.

0: Event privilege disabled (unprivileged)

1: Event privilege enabled (privileged)

18.7.8 EXTI rising trigger selection register 2 (EXTI_RTSTR2)

Address offset: 0x020

Reset value: 0x0000 0000

Contains only register bits for configurable events.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	RT59	RT58	Res.	Res.	Res.	Res.	RT53	Res.	Res.	RT50	RT49	Res.
				rw	rw					rw			rw	rw	
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	RT46	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
	rw														

Bits 31:28, 25:22, 20:19, 16:15, 13:0 Reserved, must be kept at reset value.

- Bit 27 **RT59**: Rising trigger event configuration bit of configurable event input x⁽¹⁾
 When EXTI_SECCFGR.SECx is disabled, RTx can be accessed with nonsecure and secure access.
 When EXTI_SECCFGR.SECx is enabled, RTx can be accessed only with secure access. Nonsecure write to this bit x is discarded and nonsecure read returns 0.
 When EXTI_PRIVCFGR.PRIVx is disabled, RTx can be accessed with unprivileged and privileged access.
 When EXTI_PRIVCFGR.PRIVx is enabled, RTx can be accessed only with privileged access. Unprivileged write to this bit x is discarded, unprivileged read returns 0.
 0: Rising trigger disabled (for event and interrupt) for input line
 1: Rising trigger enabled (for event and interrupt) for input line
 This bit is available only on STM32H543/553xx devices, refer to [Table 157](#). If not present, consider this bit as reserved, and keep it at reset value.
- Bit 26 **RT58**: Rising trigger event configuration bit of configurable event input x⁽¹⁾
 When EXTI_SECCFGR.SECx is disabled, RTx can be accessed with nonsecure and secure access.
 When EXTI_SECCFGR.SECx is enabled, RTx can be accessed only with secure access. Nonsecure write to this bit x is discarded and nonsecure read returns 0.
 When EXTI_PRIVCFGR.PRIVx is disabled, RTx can be accessed with unprivileged and privileged access.
 When EXTI_PRIVCFGR.PRIVx is enabled, RTx can be accessed only with privileged access. Unprivileged write to this bit x is discarded, unprivileged read returns 0.
 0: Rising trigger disabled (for event and interrupt) for input line
 1: Rising trigger enabled (for event and interrupt) for input line
 This bit is available only on STM32H543/553xx devices, refer to [Table 157](#). If not present, consider this bit as reserved, and keep it at reset value.
- Bit 21 **RT53**: Rising trigger event configuration bit of configurable event input x⁽¹⁾
 When EXTI_SECCFGR.SECx is disabled, RTx can be accessed with nonsecure and secure access.
 When EXTI_SECCFGR.SECx is enabled, RTx can be accessed only with secure access. Nonsecure write to this bit x is discarded and nonsecure read returns 0.
 When EXTI_PRIVCFGR.PRIVx is disabled, RTx can be accessed with unprivileged and privileged access.
 When EXTI_PRIVCFGR.PRIVx is enabled, RTx can be accessed only with privileged access. Unprivileged write to this bit x is discarded, unprivileged read returns 0.
 0: Rising trigger disabled (for event and interrupt) for input line
 1: Rising trigger enabled (for event and interrupt) for input line
- Bit 18 **RT50**: Rising trigger event configuration bit of configurable event input x⁽¹⁾
 When EXTI_SECCFGR.SECx is disabled, RTx can be accessed with nonsecure and secure access.
 When EXTI_SECCFGR.SECx is enabled, RTx can be accessed only with secure access. Nonsecure write to this bit x is discarded and nonsecure read returns 0.
 When EXTI_PRIVCFGR.PRIVx is disabled, RTx can be accessed with unprivileged and privileged access.
 When EXTI_PRIVCFGR.PRIVx is enabled, RTx can be accessed only with privileged access. Unprivileged write to this bit x is discarded, unprivileged read returns 0.
 0: Rising trigger disabled (for event and interrupt) for input line
 1: Rising trigger enabled (for event and interrupt) for input line

Bit 17 **RT49**: Rising trigger event configuration bit of configurable event input x⁽¹⁾

When EXTI_SECCFGR.SECx is disabled, RTx can be accessed with nonsecure and secure access.

When EXTI_SECCFGR.SECx is enabled, RTx can be accessed only with secure access. Nonsecure write to this bit x is discarded and nonsecure read returns 0.

When EXTI_PRIVCFGR.PRIVx is disabled, RTx can be accessed with unprivileged and privileged access.

When EXTI_PRIVCFGR.PRIVx is enabled, RTx can be accessed only with privileged access. Unprivileged write to this bit x is discarded, unprivileged read returns 0.

0: Rising trigger disabled (for event and interrupt) for input line

1: Rising trigger enabled (for event and interrupt) for input line

This bit is available only on STM32H543/553xx devices, refer to [Table 157](#). If not present, consider this bit as reserved, and keep it at reset value.

Bit 14 **RT46**: Rising trigger event configuration bit of configurable event input x⁽¹⁾

When EXTI_SECCFGR.SECx is disabled, RTx can be accessed with nonsecure and secure access.

When EXTI_SECCFGR.SECx is enabled, RTx can be accessed only with secure access. Nonsecure write to this bit x is discarded and nonsecure read returns 0.

When EXTI_PRIVCFGR.PRIVx is disabled, RTx can be accessed with unprivileged and privileged access.

When EXTI_PRIVCFGR.PRIVx is enabled, RTx can be accessed only with privileged access. Unprivileged write to this bit x is discarded, unprivileged read returns 0.

0: Rising trigger disabled (for event and interrupt) for input line

1: Rising trigger enabled (for event and interrupt) for input line

- The configurable event inputs are edge triggered, no glitch must be generated on these inputs. If a rising edge on the configurable event input occurs during writing of the register, the associated pending bit is not set. Rising and falling edge triggers can be set for the same configurable event input. In this case, both edges generate a trigger.

18.7.9 EXTI falling trigger selection register 2 (EXTI_FTSR2)

Address offset: 0x024

Reset value: 0x0000 0000

Contains only register bits for configurable events.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	FT59	FT58	Res.	Res.	Res.	Res.	FT53	Res.	Res.	FT50	FT49	Res.
				rw	rw					rw			rw	rw	
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	FT46	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
	rw														

Bits 31:28, 25:22, 20:19, 16:15, 13:0 Reserved, must be kept at reset value.

- Bit 27 **FT59**: Falling trigger event configuration bit of configurable event input x ⁽¹⁾
 When EXTI_SECCFGR.SECx is disabled, FTx can be accessed with nonsecure and secure access.
 When EXTI_SECCFGR.SECx is enabled, FTx can be accessed only with secure access. Nonsecure write to this FTx is discarded, nonsecure read returns 0.
 When EXTI_PRIVCFGR.PRIVx is disabled, FTx can be accessed with unprivileged and privileged access.
 When EXTI_PRIVCFGR.PRIVx is enabled, FTx can be accessed only with privileged access. Unprivileged write to this FTx is discarded, unprivileged read returns 0.
 0: Falling trigger disabled (for event and Interrupt) for input line
 1: Falling trigger enabled (for event and Interrupt) for input line.
 This bit is available only on STM32H543/553xx devices, refer to [Table 157](#). If not present, consider this bit as reserved, and keep it at reset value.
- Bit 26 **FT58**: Falling trigger event configuration bit of configurable event input x ⁽¹⁾
 When EXTI_SECCFGR.SECx is disabled, FTx can be accessed with nonsecure and secure access.
 When EXTI_SECCFGR.SECx is enabled, FTx can be accessed only with secure access. Nonsecure write to this FTx is discarded, nonsecure read returns 0.
 When EXTI_PRIVCFGR.PRIVx is disabled, FTx can be accessed with unprivileged and privileged access.
 When EXTI_PRIVCFGR.PRIVx is enabled, FTx can be accessed only with privileged access. Unprivileged write to this FTx is discarded, unprivileged read returns 0.
 0: Falling trigger disabled (for event and Interrupt) for input line
 1: Falling trigger enabled (for event and Interrupt) for input line.
 This bit is available only on STM32H543/553xx devices, refer to [Table 157](#). If not present, consider this bit as reserved, and keep it at reset value.
- Bit 21 **FT53**: Falling trigger event configuration bit of configurable event input x ⁽¹⁾
 When EXTI_SECCFGR.SECx is disabled, FTx can be accessed with nonsecure and secure access.
 When EXTI_SECCFGR.SECx is enabled, FTx can be accessed only with secure access. Nonsecure write to this FTx is discarded, nonsecure read returns 0.
 When EXTI_PRIVCFGR.PRIVx is disabled, FTx can be accessed with unprivileged and privileged access.
 When EXTI_PRIVCFGR.PRIVx is enabled, FTx can be accessed only with privileged access. Unprivileged write to this FTx is discarded, unprivileged read returns 0.
 0: Falling trigger disabled (for event and Interrupt) for input line
 1: Falling trigger enabled (for event and Interrupt) for input line.
- Bit 18 **FT50**: Falling trigger event configuration bit of configurable event input x ⁽¹⁾
 When EXTI_SECCFGR.SECx is disabled, FTx can be accessed with nonsecure and secure access.
 When EXTI_SECCFGR.SECx is enabled, FTx can be accessed only with secure access. Nonsecure write to this FTx is discarded, nonsecure read returns 0.
 When EXTI_PRIVCFGR.PRIVx is disabled, FTx can be accessed with unprivileged and privileged access.
 When EXTI_PRIVCFGR.PRIVx is enabled, FTx can be accessed only with privileged access. Unprivileged write to this FTx is discarded, unprivileged read returns 0.
 0: Falling trigger disabled (for event and Interrupt) for input line
 1: Falling trigger enabled (for event and Interrupt) for input line.

Bit 17 **FT49**: Falling trigger event configuration bit of configurable event input x ⁽¹⁾

When EXTI_SECCFGR.SECx is disabled, FTx can be accessed with nonsecure and secure access.

When EXTI_SECCFGR.SECx is enabled, FTx can be accessed only with secure access. Nonsecure write to this FTx is discarded, nonsecure read returns 0.

When EXTI_PRIVCFGR.PRIVx is disabled, FTx can be accessed with unprivileged and privileged access.

When EXTI_PRIVCFGR.PRIVx is enabled, FTx can be accessed only with privileged access. Unprivileged write to this FTx is discarded, unprivileged read returns 0.

0: Falling trigger disabled (for event and Interrupt) for input line

1: Falling trigger enabled (for event and Interrupt) for input line.

This bit is available only on STM32H543/553xx devices, refer to [Table 157](#). If not present, consider this bit as reserved, and keep it at reset value.

Bit 14 **FT46**: Falling trigger event configuration bit of configurable event input x ⁽¹⁾

When EXTI_SECCFGR.SECx is disabled, FTx can be accessed with nonsecure and secure access.

When EXTI_SECCFGR.SECx is enabled, FTx can be accessed only with secure access. Nonsecure write to this FTx is discarded, nonsecure read returns 0.

When EXTI_PRIVCFGR.PRIVx is disabled, FTx can be accessed with unprivileged and privileged access.

When EXTI_PRIVCFGR.PRIVx is enabled, FTx can be accessed only with privileged access. Unprivileged write to this FTx is discarded, unprivileged read returns 0.

0: Falling trigger disabled (for event and Interrupt) for input line

1: Falling trigger enabled (for event and Interrupt) for input line.

- The configurable event inputs are edge triggered, no glitch must be generated on these inputs. If a falling edge on the configurable event input occurs during writing of the register, the associated pending bit is not set. Rising and falling edge triggers can be set for the same configurable event input. In this case, both edges generate a trigger.

18.7.10 EXTI software interrupt event register 2 (EXTI_SWIER2)

Address offset: 0x028

Reset value: 0x0000 0000

Contains only register bits for configurable events.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	SWI59	SWI58	Res.	Res.	Res.	Res.	SWI53	Res.	Res.	SWI50	SWI49	Res.
				rw	rw					rw			rw	rw	
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	SWI46	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
	rw														

Bits 31:28, 25:22, 20:19, 16:15, 13:0 Reserved, must be kept at reset value.

Bit 27 SWI59: Software interrupt on event x

When EXTI_SECCFGR.SECx is disabled, SWIx can be accessed with nonsecure and secure access.

When EXTI_SECCFGR.SECx is enabled, SWIx can be accessed only with secure access. Nonsecure write to this SWI x is discarded, nonsecure read returns 0.

When EXTI_PRIVCFGR.PRIVx is disabled, SWIx can be accessed with unprivileged and privileged access.

When EXTI_PRIVCFGR.PRIVx is enabled, SWIx can be accessed only with privileged access. Unprivileged write to this SWIx is discarded, unprivileged read returns 0.

A software interrupt is generated independent from the setting in EXTI_RTISR and EXTI_FTSR. It always returns 0 when read.

0: Writing 0 has no effect.

1: Writing 1 triggers a rising edge event on event x. This bit is auto cleared by hardware.

This bit is available only on STM32H543/553xx devices, refer to [Table 157](#). If not present, consider this bit as reserved, and keep it at reset value.

Bit 26 SWI58: Software interrupt on event x

When EXTI_SECCFGR.SECx is disabled, SWIx can be accessed with nonsecure and secure access.

When EXTI_SECCFGR.SECx is enabled, SWIx can be accessed only with secure access. Nonsecure write to this SWI x is discarded, nonsecure read returns 0.

When EXTI_PRIVCFGR.PRIVx is disabled, SWIx can be accessed with unprivileged and privileged access.

When EXTI_PRIVCFGR.PRIVx is enabled, SWIx can be accessed only with privileged access. Unprivileged write to this SWIx is discarded, unprivileged read returns 0.

A software interrupt is generated independent from the setting in EXTI_RTISR and EXTI_FTSR. It always returns 0 when read.

0: Writing 0 has no effect.

1: Writing 1 triggers a rising edge event on event x. This bit is auto cleared by hardware.

This bit is available only on STM32H543/553xx devices, refer to [Table 157](#). If not present, consider this bit as reserved, and keep it at reset value.

Bit 21 SWI53: Software interrupt on event x

When EXTI_SECCFGR.SECx is disabled, SWIx can be accessed with nonsecure and secure access.

When EXTI_SECCFGR.SECx is enabled, SWIx can be accessed only with secure access. Nonsecure write to this SWI x is discarded, nonsecure read returns 0.

When EXTI_PRIVCFGR.PRIVx is disabled, SWIx can be accessed with unprivileged and privileged access.

When EXTI_PRIVCFGR.PRIVx is enabled, SWIx can be accessed only with privileged access. Unprivileged write to this SWIx is discarded, unprivileged read returns 0.

A software interrupt is generated independent from the setting in EXTI_RTISR and EXTI_FTSR. It always returns 0 when read.

0: Writing 0 has no effect.

1: Writing 1 triggers a rising edge event on event x. This bit is auto cleared by hardware.

Bit 18 SWI50: Software interrupt on event x

When EXTI_SECCFGR.SECx is disabled, SWIx can be accessed with nonsecure and secure access.

When EXTI_SECCFGR.SECx is enabled, SWIx can be accessed only with secure access. Nonsecure write to this SWI x is discarded, nonsecure read returns 0.

When EXTI_PRIVCFGR.PRIVx is disabled, SWIx can be accessed with unprivileged and privileged access.

When EXTI_PRIVCFGR.PRIVx is enabled, SWIx can be accessed only with privileged access. Unprivileged write to this SWIx is discarded, unprivileged read returns 0.

A software interrupt is generated independent from the setting in EXTI_RTISR and EXTI_FTSR. It always returns 0 when read.

0: Writing 0 has no effect.

1: Writing 1 triggers a rising edge event on event x. This bit is auto cleared by hardware.

Bit 17 SWI49: Software interrupt on event x

When EXTI_SECCFGR.SECx is disabled, SWIx can be accessed with nonsecure and secure access.

When EXTI_SECCFGR.SECx is enabled, SWIx can be accessed only with secure access. Nonsecure write to this SWI x is discarded, nonsecure read returns 0.

When EXTI_PRIVCFGR.PRIVx is disabled, SWIx can be accessed with unprivileged and privileged access.

When EXTI_PRIVCFGR.PRIVx is enabled, SWIx can be accessed only with privileged access. Unprivileged write to this SWIx is discarded, unprivileged read returns 0.

A software interrupt is generated independent from the setting in EXTI_RTISR and EXTI_FTSR. It always returns 0 when read.

0: Writing 0 has no effect.

1: Writing 1 triggers a rising edge event on event x. This bit is auto cleared by hardware.

This bit is available only on STM32H543/553xx devices, refer to [Table 157](#). If not present, consider this bit as reserved, and keep it at reset value.

Bit 14 SWI46: Software interrupt on event x

When EXTI_SECCFGR.SECx is disabled, SWIx can be accessed with nonsecure and secure access.

When EXTI_SECCFGR.SECx is enabled, SWIx can be accessed only with secure access. Nonsecure write to this SWI x is discarded, nonsecure read returns 0.

When EXTI_PRIVCFGR.PRIVx is disabled, SWIx can be accessed with unprivileged and privileged access.

When EXTI_PRIVCFGR.PRIVx is enabled, SWIx can be accessed only with privileged access. Unprivileged write to this SWIx is discarded, unprivileged read returns 0.

A software interrupt is generated independent from the setting in EXTI_RTISR and EXTI_FTSR. It always returns 0 when read.

0: Writing 0 has no effect.

1: Writing 1 triggers a rising edge event on event x. This bit is auto cleared by hardware.

18.7.11 EXTI rising edge pending register 2 (EXTI_RPR2)

Address offset: 0x02C

Reset value: 0x0000 0000

Contains only register bits for configurable events.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	RPIF59	RPIF58	Res.	Res.	Res.	Res.	RPIF53	Res.	Res.	RPIF50	RPIF49	Res.
				rw	rw					rw			rw	rw	
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	RPIF46	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
	rw														

Bits 31:28, 25:22, 20:19, 16:15, 13:0 Reserved, must be kept at reset value.

Bit 27 **RPIF59**: configurable event inputs x rising edge pending bit

When EXTI_SECCFGR.SECx is disabled, RPIF_x can be accessed with nonsecure and secure access.

When EXTI_SECCFGR.SECx is enabled, RPIF_x can be accessed only with secure access. Nonsecure write to this RPIF_x is discarded, nonsecure read returns 0.

When EXTI_PRIVCFGR.PRIVx is disabled, RPIF_x can be accessed with unprivileged and privileged access.

When EXTI_PRIVCFGR.PRIVx is enabled, RPIF_x can be accessed only with privileged access. Unprivileged write to this RPIF_x is discarded, unprivileged read returns 0.

0: No rising edge trigger request occurred

1: Rising edge trigger request occurred

This bit is set when the rising edge event or an EXTI_SWIER software trigger arrives on the configurable event line. This bit is cleared by writing 1 to it.

This bit is available only on STM32H543/553xx devices, refer to [Table 157](#). If not present, consider this bit as reserved, and keep it at reset value.

Bit 26 **RPIF58**: configurable event inputs x rising edge pending bit

When EXTI_SECCFGR.SECx is disabled, RPIF_x can be accessed with nonsecure and secure access.

When EXTI_SECCFGR.SECx is enabled, RPIF_x can be accessed only with secure access. Nonsecure write to this RPIF_x is discarded, nonsecure read returns 0.

When EXTI_PRIVCFGR.PRIVx is disabled, RPIF_x can be accessed with unprivileged and privileged access.

When EXTI_PRIVCFGR.PRIVx is enabled, RPIF_x can be accessed only with privileged access. Unprivileged write to this RPIF_x is discarded, unprivileged read returns 0.

0: No rising edge trigger request occurred

1: Rising edge trigger request occurred

This bit is set when the rising edge event or an EXTI_SWIER software trigger arrives on the configurable event line. This bit is cleared by writing 1 to it.

This bit is available only on STM32H543/553xx devices, refer to [Table 157](#). If not present, consider this bit as reserved, and keep it at reset value.

Bit 21 RPIF53: configurable event inputs x rising edge pending bit

When EXTI_SECCFGR.SECx is disabled, RPIF_x can be accessed with nonsecure and secure access.

When EXTI_SECCFGR.SECx is enabled, RPIF_x can be accessed only with secure access. Nonsecure write to this RPIF_x is discarded, nonsecure read returns 0.

When EXTI_PRIVCFGR.PRIVx is disabled, RPIF_x can be accessed with unprivileged and privileged access.

When EXTI_PRIVCFGR.PRIVx is enabled, RPIF_x can be accessed only with privileged access. Unprivileged write to this RPIF_x is discarded, unprivileged read returns 0.

0: No rising edge trigger request occurred

1: Rising edge trigger request occurred

This bit is set when the rising edge event or an EXTI_SWIER software trigger arrives on the configurable event line. This bit is cleared by writing 1 to it.

Bit 18 RPIF50: configurable event inputs x rising edge pending bit

When EXTI_SECCFGR.SECx is disabled, RPIF_x can be accessed with nonsecure and secure access.

When EXTI_SECCFGR.SECx is enabled, RPIF_x can be accessed only with secure access. Nonsecure write to this RPIF_x is discarded, nonsecure read returns 0.

When EXTI_PRIVCFGR.PRIVx is disabled, RPIF_x can be accessed with unprivileged and privileged access.

When EXTI_PRIVCFGR.PRIVx is enabled, RPIF_x can be accessed only with privileged access. Unprivileged write to this RPIF_x is discarded, unprivileged read returns 0.

0: No rising edge trigger request occurred

1: Rising edge trigger request occurred

This bit is set when the rising edge event or an EXTI_SWIER software trigger arrives on the configurable event line. This bit is cleared by writing 1 to it.

Bit 17 RPIF49: configurable event inputs x rising edge pending bit

When EXTI_SECCFGR.SECx is disabled, RPIF_x can be accessed with nonsecure and secure access.

When EXTI_SECCFGR.SECx is enabled, RPIF_x can be accessed only with secure access. Nonsecure write to this RPIF_x is discarded, nonsecure read returns 0.

When EXTI_PRIVCFGR.PRIVx is disabled, RPIF_x can be accessed with unprivileged and privileged access.

When EXTI_PRIVCFGR.PRIVx is enabled, RPIF_x can be accessed only with privileged access. Unprivileged write to this RPIF_x is discarded, unprivileged read returns 0.

0: No rising edge trigger request occurred

1: Rising edge trigger request occurred

This bit is set when the rising edge event or an EXTI_SWIER software trigger arrives on the configurable event line. This bit is cleared by writing 1 to it.

This bit is available only on STM32H543/553xx devices, refer to [Table 157](#). If not present, consider this bit as reserved, and keep it at reset value.

Bit 14 RPIF46: configurable event inputs x rising edge pending bit

When EXTI_SECCFGR.SECx is disabled, RPIF_x can be accessed with nonsecure and secure access.

When EXTI_SECCFGR.SECx is enabled, RPIF_x can be accessed only with secure access. Nonsecure write to this RPIF_x is discarded, nonsecure read returns 0.

When EXTI_PRIVCFGR.PRIVx is disabled, RPIF_x can be accessed with unprivileged and privileged access.

When EXTI_PRIVCFGR.PRIVx is enabled, RPIF_x can be accessed only with privileged access. Unprivileged write to this RPIF_x is discarded, unprivileged read returns 0.

0: No rising edge trigger request occurred

1: Rising edge trigger request occurred

This bit is set when the rising edge event or an EXTI_SWIER software trigger arrives on the configurable event line. This bit is cleared by writing 1 to it.

18.7.12 EXTI falling edge pending register 2 (EXTI_FPR2)

Address offset: 0x030

Reset value: 0x0000 0000

Contains only register bits for configurable events.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	FPIF59	FPIF58	Res.	Res.	Res.	Res.	FPIF53	Res.	Res.	FPIF50	FPIF49	Res.
				rw	rw					rw			rw	rw	
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	FPIF46	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
	rw														

Bits 31:28, 25:22, 20:19, 16:15, 13:0 Reserved, must be kept at reset value.

Bit 27 FPIF59: configurable event inputs x falling edge pending bit

When EXTI_SECCFGR.SECx is disabled, FPIF_x can be accessed with nonsecure and secure access.

When EXTI_SECCFGR.SECx is enabled, FPIF_x can be accessed only with secure access. Nonsecure write to this FPIF_x is discarded, nonsecure read returns 0.

When EXTI_PRIVCFGR.PRIVx is disabled, FPIF_x can be accessed with unprivileged and privileged access.

When EXTI_PRIVCFGR.PRIVx is enabled, FPIF_x can be accessed only with privileged access. Unprivileged write to this FPIF_x is discarded, unprivileged read returns 0.

0: No falling edge trigger request occurred

1: Falling edge trigger request occurred

This bit is set when the falling edge event arrives on the configurable event line. This bit is cleared by writing 1 to it.

This bit is available only on STM32H543/553xx devices, refer to [Table 157](#). If not present, consider this bit as reserved, and keep it at reset value.

Bit 26 FPIF58: configurable event inputs x falling edge pending bit

When EXTI_SECCFGR.SECx is disabled, FPIF_x can be accessed with nonsecure and secure access.

When EXTI_SECCFGR.SECx is enabled, FPIF_x can be accessed only with secure access. Nonsecure write to this FPIF_x is discarded, nonsecure read returns 0.

When EXTI_PRIVCFGR.PRIVx is disabled, FPIF_x can be accessed with unprivileged and privileged access.

When EXTI_PRIVCFGR.PRIVx is enabled, FPIF_x can be accessed only with privileged access. Unprivileged write to this FPIF_x is discarded, unprivileged read returns 0.

0: No falling edge trigger request occurred

1: Falling edge trigger request occurred

This bit is set when the falling edge event arrives on the configurable event line. This bit is cleared by writing 1 to it.

This bit is available only on STM32H543/553xx devices, refer to [Table 157](#). If not present, consider this bit as reserved, and keep it at reset value.

Bit 21 FPIF53: configurable event inputs x falling edge pending bit

When EXTI_SECCFGR.SECx is disabled, FPIF_x can be accessed with nonsecure and secure access.

When EXTI_SECCFGR.SECx is enabled, FPIF_x can be accessed only with secure access. Nonsecure write to this FPIF_x is discarded, nonsecure read returns 0.

When EXTI_PRIVCFGR.PRIVx is disabled, FPIF_x can be accessed with unprivileged and privileged access.

When EXTI_PRIVCFGR.PRIVx is enabled, FPIF_x can be accessed only with privileged access. Unprivileged write to this FPIF_x is discarded, unprivileged read returns 0.

0: No falling edge trigger request occurred

1: Falling edge trigger request occurred

This bit is set when the falling edge event arrives on the configurable event line. This bit is cleared by writing 1 to it.

Bit 18 FPIF50: configurable event inputs x falling edge pending bit

When EXTI_SECCFGR.SECx is disabled, FPIF_x can be accessed with nonsecure and secure access.

When EXTI_SECCFGR.SECx is enabled, FPIF_x can be accessed only with secure access. Nonsecure write to this FPIF_x is discarded, nonsecure read returns 0.

When EXTI_PRIVCFGR.PRIVx is disabled, FPIF_x can be accessed with unprivileged and privileged access.

When EXTI_PRIVCFGR.PRIVx is enabled, FPIF_x can be accessed only with privileged access. Unprivileged write to this FPIF_x is discarded, unprivileged read returns 0.

0: No falling edge trigger request occurred

1: Falling edge trigger request occurred

This bit is set when the falling edge event arrives on the configurable event line. This bit is cleared by writing 1 to it.

Bit 17 FPIF49: configurable event inputs x falling edge pending bit

When EXTI_SECCFGR.SECx is disabled, FPIF_x can be accessed with nonsecure and secure access.

When EXTI_SECCFGR.SECx is enabled, FPIF_x can be accessed only with secure access. Nonsecure write to this FPIF_x is discarded, nonsecure read returns 0.

When EXTI_PRIVCFGR.PRIVx is disabled, FPIF_x can be accessed with unprivileged and privileged access.

When EXTI_PRIVCFGR.PRIVx is enabled, FPIF_x can be accessed only with privileged access. Unprivileged write to this FPIF_x is discarded, unprivileged read returns 0.

0: No falling edge trigger request occurred

1: Falling edge trigger request occurred

This bit is set when the falling edge event arrives on the configurable event line. This bit is cleared by writing 1 to it.

This bit is available only on STM32H543/553xx devices, refer to [Table 157](#). If not present, consider this bit as reserved, and keep it at reset value.

Bit 14 FPIF46: configurable event inputs x falling edge pending bit

When EXTI_SECCFGR.SECx is disabled, FPIF_x can be accessed with nonsecure and secure access.

When EXTI_SECCFGR.SECx is enabled, FPIF_x can be accessed only with secure access. Nonsecure write to this FPIF_x is discarded, nonsecure read returns 0.

When EXTI_PRIVCFGR.PRIVx is disabled, FPIF_x can be accessed with unprivileged and privileged access.

When EXTI_PRIVCFGR.PRIVx is enabled, FPIF_x can be accessed only with privileged access. Unprivileged write to this FPIF_x is discarded, unprivileged read returns 0.

0: No falling edge trigger request occurred

1: Falling edge trigger request occurred

This bit is set when the falling edge event arrives on the configurable event line. This bit is cleared by writing 1 to it.

18.7.13 EXTI security configuration register 2 (EXTI_SECCFGR2)

Address offset: 0x034

Reset value: 0x0000 0000

This register provides write access security, a nonsecure write access is ignored and causes the generation of an illegal access event. A nonsecure read returns the register data.

Contains only register bits for privilege capable input events.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	SEC61	Res.	SEC59	SEC58	SEC57	SEC56	SEC55	SEC54	SEC53	SEC52	SEC51	SEC50	SEC49	SEC48
		rw		rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
SEC47	SEC46	SEC45	SEC44	SEC43	SEC42	SEC41	SEC40	SEC39	SEC38	SEC37	SEC36	SEC35	SEC34	SEC33	SEC32
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bits 31:30 Reserved, must be kept at reset value.

Bit 29 **SEC61**: Security enable on event input 61

When EXTI_PRIVCFGR.PRIVx is disabled, SECx can be accessed with privileged and unprivileged access.

When EXTI_PRIVCFGR.PRIVx is enabled, SECx can only be written with privileged access. Unprivileged write to this SECx is discarded.

0: Event security disabled (nonsecure)

1: Event security enabled (secure)

This bit is available only on STM32H543/553xx devices, refer to [Table 157](#). If not present, consider this bit as reserved, and keep it at reset value.

Bit 28 Reserved, must be kept at reset value.

Bits 27:0 **SECx**: Security enable on event input x (x = 59 to 32)

When EXTI_PRIVCFGR.PRIVx is disabled, SECx can be accessed with privileged and unprivileged access.

When EXTI_PRIVCFGR.PRIVx is enabled, SECx can only be written with privileged access. Unprivileged write to this SECx is discarded.

0: Event security disabled (nonsecure)

1: Event security enabled (secure)

Bits SEC[59:58] are available only on STM32H543/553xx devices, refer to [Table 157](#). If not present, consider this bit as reserved, and keep it at reset value.

18.7.14 EXTI privilege configuration register 2 (EXTI_PRIVCFGR2)

Address offset: 0x038

Reset value: 0x0000 0000

This register provides privileged write access protection. An unprivileged read returns the register data.

Contains only register bits for security capable input events.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	PRIV61	Res.	PRIV59	PRIV58	PRIV57	PRIV56	PRIV55	PRIV54	PRIV53	PRIV52	PRIV51	PRIV50	PRIV49	PRIV48
		rW		rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
PRIV47	PRIV46	PRIV45	PRIV44	PRIV43	PRIV42	PRIV41	PRIV40	PRIV39	PRIV38	PRIV37	PRIV36	PRIV35	PRIV34	PRIV33	PRIV32
rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW

Bits 31:30 Reserved, must be kept at reset value.

Bit 29 **PRIV61**: Security enable on event input 61

When EXTI_SECCFGR.SECx is disabled, PRIVx can be accessed with secure and nonsecure access.

When EXTI_SECCFGR.SECx is enabled, PRIVx can only be written with secure access. Nonsecure write to this PRIVx is discarded.

0: Event privilege disabled (unprivileged)

1: Event privilege enabled (privileged)

This bit is available only on STM32H543/553xx devices, refer to [Table 157](#). If not present, consider this bit as reserved, and keep it at reset value.

Bit 28 Reserved, must be kept at reset value.

Bits 27:0 **PRIVx**: Security enable on event input x (x = 59 to 32)

When EXTI_SECCFGR.SECx is disabled, PRIVx can be accessed with secure and nonsecure access.

When EXTI_SECCFGR.SECx is enabled, PRIVx can only be written with secure access. Nonsecure write to this PRIVx is discarded.

0: Event privilege disabled (unprivileged)

1: Event privilege enabled (privileged)

Bits [59:58] are available only on STM32H543/553xx devices, refer to [Table 157](#). If not present, consider this bit as reserved, and keep it at reset value.

18.7.15 EXTI rising trigger selection register 3 (EXTI_RTISR3)

Address offset: 0x040

Reset value: 0x0000 0000

It contains only register bits for configurable events.

STM32H543/553xx devices only

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	RT66	Res.	RT64
													rw		rw

Bits 31:3 Reserved, must be kept at reset value.

Bit 2 **RT66**: Rising trigger event configuration bit of configurable event input x⁽¹⁾

When EXTI_SECCFGR.SECx is disabled, RTx can be accessed with nonsecure and secure access.

When EXTI_SECCFGR.SECx is enabled, RTx can only be accessed with secure access. Nonsecure write to this bit x is discarded and nonsecure read returns 0.

When EXTI_PRIVCFGR.PRIVx is disabled, RTx can be accessed with unprivileged and privileged access.

When EXTI_PRIVCFGR.PRIVx is enabled, RTx can only be accessed with privileged access. Unprivileged write to this bit x is discarded, unprivileged read returns 0.

0: Rising trigger disabled (for event and interrupt) for input line

1: Rising trigger enabled (for event and interrupt) for input line

Bit 1 Reserved, must be kept at reset value.

Bit 0 **RT64**: Rising trigger event configuration bit of configurable event input x⁽¹⁾

When EXTI_SECCFGR.SECx is disabled, RTx can be accessed with nonsecure and secure access.

When EXTI_SECCFGR.SECx is enabled, RTx can only be accessed with secure access. Nonsecure write to this bit x is discarded and nonsecure read returns 0.

When EXTI_PRIVCFGR.PRIVx is disabled, RTx can be accessed with unprivileged and privileged access.

When EXTI_PRIVCFGR.PRIVx is enabled, RTx can only be accessed with privileged access. Unprivileged write to this bit x is discarded, unprivileged read returns 0.

0: Rising trigger disabled (for event and interrupt) for input line

1: Rising trigger enabled (for event and interrupt) for input line

18.7.16 EXTI falling trigger selection register 3 (EXTI_FTSR3)

Address offset: 0x044

Reset value: 0x0000 0000

It contains only register bits for configurable events.

STM32H543/553xx devices only

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	FT66	Res.	FT64
													rw		rw

Bits 31:3 Reserved, must be kept at reset value.

Bit 2 **FT66**: Falling trigger event configuration bit of configurable event input x ⁽¹⁾

When EXTI_SECCFGR.SECx is disabled, FTx can be accessed with nonsecure and secure access.

When EXTI_SECCFGR.SECx is enabled, FTx can only be accessed with secure access. Nonsecure write to this FTx is discarded, nonsecure read returns 0.

When EXTI_PRIVCFGR.PRIVx is disabled, FTx can be accessed with unprivileged and privileged access.

When EXTI_PRIVCFGR.PRIVx is enabled, FTx can only be accessed with privileged access. Unprivileged write to this FTx is discarded, unprivileged read returns 0.

0: Falling trigger disabled (for event and Interrupt) for input line

1: Falling trigger enabled (for event and Interrupt) for input line.

Bit 1 Reserved, must be kept at reset value.

Bit 0 **FT64**: Falling trigger event configuration bit of configurable event input x ⁽¹⁾

When EXTI_SECCFGR.SECx is disabled, FTx can be accessed with nonsecure and secure access.

When EXTI_SECCFGR.SECx is enabled, FTx can only be accessed with secure access. Nonsecure write to this FTx is discarded, nonsecure read returns 0.

When EXTI_PRIVCFGR.PRIVx is disabled, FTx can be accessed with unprivileged and privileged access.

When EXTI_PRIVCFGR.PRIVx is enabled, FTx can only be accessed with privileged access. Unprivileged write to this FTx is discarded, unprivileged read returns 0.

0: Falling trigger disabled (for event and Interrupt) for input line

1: Falling trigger enabled (for event and Interrupt) for input line.

- The configurable event inputs are edge triggered. No glitch must be generated on these inputs. If a falling edge on the configurable event input occurs during writing of the register, the associated pending bit is not set. Rising and falling edge triggers can be set for the same configurable event input. In this case, both edges generate a trigger.

18.7.17 EXTI software interrupt event register 3 (EXTI_SWIER3)

Address offset: 0x048

Reset value: 0x0000 0000

It contains only register bits for configurable events.

STM32H543/553xx devices only

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	SWI66	Res.	SWI64
													rw		rw

Bits 31:3 Reserved, must be kept at reset value.

Bit 2 **SWI66**: Software interrupt on event x

When EXTI_SECCFGR.SECx is disabled, SWIx can be accessed with nonsecure and secure access.

When EXTI_SECCFGR.SECx is enabled, SWIx can only be accessed with secure access. Nonsecure write to this SWI x is discarded, nonsecure read returns 0.

When EXTI_PRIVCFGR.PRIVx is disabled, SWIx can be accessed with unprivileged and privileged access.

When EXTI_PRIVCFGR.PRIVx is enabled, SWIx can only be accessed with privileged access. Unprivileged write to this SWIx is discarded, unprivileged read returns 0.

A software interrupt is generated independent from the setting in EXTI_RTISR and EXTI_FTSR. It always returns 0 when read.

0: Writing 0 has no effect.

1: Writing 1 triggers a rising edge event on event x. This bit is auto cleared by hardware.

Bit 1 Reserved, must be kept at reset value.

Bit 0 **SWI64**: Software interrupt on event x

When EXTI_SECCFGR.SECx is disabled, SWIx can be accessed with nonsecure and secure access.

When EXTI_SECCFGR.SECx is enabled, SWIx can only be accessed with secure access. Nonsecure write to this SWI x is discarded, nonsecure read returns 0.

When EXTI_PRIVCFGR.PRIVx is disabled, SWIx can be accessed with unprivileged and privileged access.

When EXTI_PRIVCFGR.PRIVx is enabled, SWIx can only be accessed with privileged access. Unprivileged write to this SWIx is discarded, unprivileged read returns 0.

A software interrupt is generated independent from the setting in EXTI_RTISR and EXTI_FTSR. It always returns 0 when read.

0: Writing 0 has no effect.

1: Writing 1 triggers a rising edge event on event x. This bit is auto cleared by hardware.

18.7.18 EXTI rising edge pending register 3 (EXTI_RPR3)

Address offset: 0x04C

Reset value: 0x0000 0000

It contains only register bits for configurable events.

STM32H543/553xx devices only

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	RPIF66	Res.	RPIF64
													rw		rw

Bits 31:3 Reserved, must be kept at reset value.

Bit 2 RPIF66: configurable event inputs x rising edge pending bit

When EXTI_SECCFGR.SECx is disabled, RPIFx can be accessed with nonsecure and secure access.

When EXTI_SECCFGR.SECx is enabled, RPIFx can only be accessed with secure access. Nonsecure write to this RPIFx is discarded, nonsecure read returns 0.

When EXTI_PRIVCFGR.PRIVx is disabled, RPIFx can be accessed with unprivileged and privileged access.

When EXTI_PRIVCFGR.PRIVx is enabled, RPIFx can only be accessed with privileged access. Unprivileged write to this RPIFx is discarded, unprivileged read returns 0.

0: No rising edge trigger request occurred

1: Rising edge trigger request occurred

This bit is set when the rising edge event or an EXTI_SWIER software trigger arrives on the configurable event line. This bit is cleared by writing 1 to it.

Bit 1 Reserved, must be kept at reset value.

Bit 0 RPIF64: configurable event inputs x rising edge pending bit

When EXTI_SECCFGR.SECx is disabled, RPIFx can be accessed with nonsecure and secure access.

When EXTI_SECCFGR.SECx is enabled, RPIFx can only be accessed with secure access. Nonsecure write to this RPIFx is discarded, nonsecure read returns 0.

When EXTI_PRIVCFGR.PRIVx is disabled, RPIFx can be accessed with unprivileged and privileged access.

When EXTI_PRIVCFGR.PRIVx is enabled, RPIFx can only be accessed with privileged access. Unprivileged write to this RPIFx is discarded, unprivileged read returns 0.

0: No rising edge trigger request occurred

1: Rising edge trigger request occurred

This bit is set when the rising edge event or an EXTI_SWIER software trigger arrives on the configurable event line. This bit is cleared by writing 1 to it.

18.7.19 EXTI falling edge pending register 3 (EXTI_FPR3)

Address offset: 0x050

Reset value: 0x0000 0000

It contains only register bits for configurable events.

STM32H543/553xx devices only

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	FPIF66	Res.	FPIF64
													rw		rw

Bits 31:3 Reserved, must be kept at reset value.

Bit 2 **FPIF66**: configurable event inputs x falling edge pending bit

When EXTI_SECCFGR.SECx is disabled, FPIF66 can be accessed with nonsecure and secure access.

When EXTI_SECCFGR.SECx is enabled, FPIF66 can only be accessed with secure access. Nonsecure write to this FPIF66 is discarded, nonsecure read returns 0.

When EXTI_PRIVCFGR.PRIVx is disabled, FPIF66 can be accessed with unprivileged and privileged access.

When EXTI_PRIVCFGR.PRIVx is enabled, FPIF66 can only be accessed with privileged access. Unprivileged write to this FPIF66 is discarded, unprivileged read returns 0.

0: No falling edge trigger request occurred

1: Falling edge trigger request occurred

This bit is set when the falling edge event arrives on the configurable event line. This bit is cleared by writing 1 to it.

Bit 1 Reserved, must be kept at reset value.

Bit 0 **FPIF64**: configurable event inputs x falling edge pending bit

When EXTI_SECCFGR.SECx is disabled, FPIF64 can be accessed with nonsecure and secure access.

When EXTI_SECCFGR.SECx is enabled, FPIF64 can only be accessed with secure access. Nonsecure write to this FPIF64 is discarded, nonsecure read returns 0.

When EXTI_PRIVCFGR.PRIVx is disabled, FPIF64 can be accessed with unprivileged and privileged access.

When EXTI_PRIVCFGR.PRIVx is enabled, FPIF64 can only be accessed with privileged access. Unprivileged write to this FPIF64 is discarded, unprivileged read returns 0.

0: No falling edge trigger request occurred

1: Falling edge trigger request occurred

This bit is set when the falling edge event arrives on the configurable event line. This bit is cleared by writing 1 to it.

18.7.20 EXTI security configuration register 3 (EXTI_SECCFGR3)

Address offset: 0x054

Reset value: 0x0000 0000

This register provides write access security. A nonsecure write access is ignored and causes the generation of an illegal access event. A nonsecure read returns the register data.

It contains only register bits for privilege capable input events.

STM32H543/553xx devices only

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	SEC66	SEC65	SEC64
													rw	rw	rw

Bits 31:3 Reserved, must be kept at reset value.

Bits 2:0 **SECx**: Security enable on event input x (x = 66 to 64)

When EXTI_PRIVCFGR.PRIVx is disabled, SECx can be accessed with privileged and unprivileged access.

When EXTI_PRIVCFGR.PRIVx is enabled, SECx can only be written with privileged access. Unprivileged write to this SECx is discarded.

0: Event security disabled (nonsecure)

1: Event security enabled (secure)

18.7.21 EXTI privilege configuration register 3 (EXTI_PRIVCFGR3)

Address offset: 0x058

Reset value: 0x0000 0000

This register provides privileged write access protection. An unprivileged read returns the register data.

It contains only register bits for security capable input events.

STM32H543/553xx devices only

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	PRIV66	PRIV65	PRIV64
													rw	rw	rw

Bits 31:3 Reserved, must be kept at reset value.

Bits 2:0 **PRIVx**: Security enable on event input x (x = 66 to 64)

When EXTI_SECCFGR.SECx is disabled, PRIVx can be accessed with secure and nonsecure access.

When EXTI_SECCFGR.SECx is enabled, PRIVx can only be written with secure access. Nonsecure write to this PRIVx is discarded.

0: Event privilege disabled (unprivileged)

1: Event privilege enabled (privileged)

18.7.22 EXTI external interrupt selection register (EXTI_EXTICR1)

Address offset: 0x060 EXTI multiplexer 0, 1, 2, 3

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
EXTI3[7:0]								EXTI2[7:0]							
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
EXTI1[7:0]								EXTI0[7:0]							
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bits 31:24 **EXTI3[7:0]**: EXTI3 GPIO port selection

These bits are written by software to select the source input for EXTI3 external interrupt. When EXTI_SECCFGR1.SEC3 is disabled, EXTI3 can be accessed with nonsecure and secure access.

When EXTI_SECCFGR1.SEC3 is enabled, EXTI3 can be accessed only with secure access. Nonsecure write is discarded and nonsecure read returns 0.

When EXTI_PRIVCFGR1.PRIV3 is disabled, EXTI3 can be accessed with privileged and unprivileged access.

When EXTI_PRIVCFGR1.PRIV3 is enabled, EXTI3 can be accessed only with privileged access. Unprivileged write to this bit is discarded.

0x00: PA3 pin

0x01: PB3 pin

0x02: PC3 pin

0x03: PD3 pin

0x04: PE3 pin

0x05: PF3 pin

0x06: PG3 pin

0x07: PH3 pin

0x08: PI3 pin

Others: reserved

Bits 23:16 **EXTI2[7:0]**: EXTI2 GPIO port selection

These bits are written by software to select the source input for EXTI2 external interrupt. When EXTI_SECCFGR1.SEC2 is disabled, EXTI2 can be accessed with nonsecure and secure access.

When EXTI_SECCFGR1.SEC2 is enabled, EXTI2 can be accessed only with secure access. Nonsecure write is discarded and nonsecure read returns 0.

When EXTI_PRIVCFGR1.PRIV2 is disabled, EXTI2 can be accessed with privileged and unprivileged access.

When EXTI_PRIVCFGR1.PRIV2 is enabled, EXTI2 can be accessed only with privileged access. Unprivileged write to this bit is discarded.

0x00: PA2 pin

0x01: PB2 pin

0x02: PC2 pin

0x03: PD2 pin

0x04: PE2 pin

0x05: PF2 pin

0x06: PG2 pin

0x07: PH2 pin

0x08: PI2 pin

Others: reserved

Bits 15:8 **EXTI1[7:0]**: EXTI1 GPIO port selection

These bits are written by software to select the source input for EXTI1 external interrupt. When EXTI_SECCFGR1.SEC1 is disabled, EXTI1 can be accessed with nonsecure and secure access.

When EXTI_SECCFGR1.SEC1 is enabled, EXTI1 can be accessed only with secure access. Nonsecure write is discarded and nonsecure read returns 0.

When EXTI_PRIVCFGR1.PRIV1 is disabled, EXTI1 can be accessed with privileged and unprivileged access.

When EXTI_PRIVCFGR1.PRIV1 is enabled, EXTI1 can be accessed only with privileged access. Unprivileged write to this bit is discarded.

0x00: PA1 pin

0x01: PB1 pin

0x02: PC1 pin

0x03: PD1 pin

0x04: PE1 pin

0x05: PF1 pin

0x06: PG1 pin

0x07: PH1 pin

0x08: PI1 pin

Others: reserved

Bits 7:0 **EXTI0[7:0]**: EXTI0 GPIO port selection

These bits are written by software to select the source input for EXTI0 external interrupt.
When EXTI_SECCFGR1.SEC0 is disabled, EXTI0 can be accessed with nonsecure and secure access.

When EXTI_SECCFGR1.SEC0 is enabled, EXTI0 can be accessed only with secure access. Nonsecure write is discarded and nonsecure read returns 0.

When EXTI_PRIVCFGR1.PRIV0 is disabled, EXTI0 can be accessed with privileged and unprivileged access.

When EXTI_PRIVCFGR1.PRIV0 is enabled, EXTI0 can be accessed only with privileged access. Unprivileged write to this bit is discarded.

- 0x00: PA0 pin
- 0x01: PB0 pin
- 0x02: PC0 pin
- 0x03: PD0 pin
- 0x04: PE0 pin
- 0x05: PF0 pin
- 0x06: PG0 pin
- 0x07: PH0 pin
- 0x08: PIO pin
- Others: reserved

18.7.23 **EXTI external interrupt selection register (EXTI_EXTICR2)**

Address offset: 0x064 EXTI multiplexer 4, 5, 6, 7

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
EXTI7[7:0]								EXTI6[7:0]							
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
EXTI5[7:0]								EXTI4[7:0]							
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bits 31:24 EXTI7[7:0]: EXTI7 GPIO port selection

These bits are written by software to select the source input for EXTI7 external interrupt. When EXTI_SECCFGR1.SEC7 is disabled, EXTI7 can be accessed with nonsecure and secure access.

When EXTI_SECCFGR1.SEC7 is enabled, EXTI7 can be accessed only with secure access. Nonsecure write is discarded and nonsecure read returns 0.

When EXTI_PRIVCFGR1.PRIV7 is disabled, EXTI7 can be accessed with privileged and unprivileged access.

When EXTI_PRIVCFGR1.PRIV7 is enabled, EXTI7 can be accessed only with privileged access. Unprivileged write to this bit is discarded.

0x00: PA7 pin

0x01: PB7 pin

0x02: PC7 pin

0x03: PD7 pin

0x04: PE7 pin

0x05: PF7 pin

0x06: PG7 pin

0x07: PH7 pin

0x08: PI7 pin

Others: reserved

Bits 23:16 EXTI6[7:0]: EXTI6 GPIO port selection

These bits are written by software to select the source input for EXTI6 external interrupt. When EXTI_SECCFGR1.SEC6 is disabled, EXTI6 can be accessed with nonsecure and secure access.

When EXTI_SECCFGR1.SEC6 is enabled, EXTI6 can be accessed only with secure access. Nonsecure write is discarded and nonsecure read returns 0.

When EXTI_PRIVCFGR1.PRIV6 is disabled, EXTI6 can be accessed with privileged and unprivileged access.

When EXTI_PRIVCFGR1.PRIV6 is enabled, EXTI6 can be accessed only with privileged access. Unprivileged write to this bit is discarded.

0x00: PA6 pin

0x01: PB6 pin

0x02: PC6 pin

0x03: PD6 pin

0x04: PE6 pin

0x05: PF6 pin

0x06: PG6 pin

0x07: PH6 pin

0x08: PI6 pin

Others: reserved

Bits 15:8 **EXTI5[7:0]**: EXTI5 GPIO port selection

These bits are written by software to select the source input for EXTI5 external interrupt. When EXTI_SECCFGR1.SEC5 is disabled, EXTI5 can be accessed with nonsecure and secure access.

When EXTI_SECCFGR1.SEC5 is enabled, EXTI5 can be accessed only with secure access. Nonsecure write is discarded and nonsecure read returns 0.

When EXTI_PRIVCFGR1.PRIV5 is disabled, EXTI5 can be accessed with privileged and unprivileged access.

When EXTI_PRIVCFGR1.PRIV5 is enabled, EXTI5 can be accessed only with privileged access. Unprivileged write to this bit is discarded.

0x00: PA5 pin

0x01: PB5 pin

0x02: PC5 pin

0x03: PD5 pin

0x04: PE5 pin

0x05: PF5 pin

0x06: PG5 pin

0x07: PH5 pin

0x08: PI5 pin

Others: reserved

Bits 7:0 **EXTI4[7:0]**: EXTI4 GPIO port selection

These bits are written by software to select the source input for EXTI4 external interrupt. When EXTI_SECCFGR1.SEC4 is disabled, EXTI4 can be accessed with nonsecure and secure access.

When EXTI_SECCFGR1.SEC4 is enabled, EXTI4 can be accessed only with secure access. Nonsecure write is discarded and nonsecure read returns 0.

When EXTI_PRIVCFGR1.PRIV4 is disabled, EXTI4 can be accessed with privileged and unprivileged access.

When EXTI_PRIVCFGR1.PRIV4 is enabled, EXTI4 can be accessed only with privileged access. Unprivileged write to this bit is discarded.

0x00: PA4 pin

0x01: PB4 pin

0x02: PC4 pin

0x03: PD4 pin

0x04: PE4 pin

0x05: PF4 pin

0x06: PG4 pin

0x07: PH4 pin

0x08: PI4 pin

Others: reserved

18.7.24 EXTI external interrupt selection register (EXTI_EXTICR3)

Address offset: 0x068 EXTI multiplexer 8, 9, 10, 11

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
EXTI11[7:0]								EXTI10[7:0]							
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
EXTI9[7:0]								EXTI8[7:0]							
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bits 31:24 **EXTI11[7:0]**: EXTI11 GPIO port selection

These bits are written by software to select the source input for EXTI11 external interrupt. When EXTI_SECCFGR1.SEC11 is disabled, EXTI11 can be accessed with nonsecure and secure access.

When EXTI_SECCFGR1.SEC11 is enabled, EXTI11 can be accessed only with secure access. Nonsecure write is discarded and nonsecure read returns 0.

When EXTI_PRIVCFGR1.PRIV11 is disabled, EXTI11 can be accessed with privileged and unprivileged access.

When EXTI_PRIVCFGR1.PRIV11 is enabled, EXTI11 can be accessed only with privileged access. Unprivileged write to this bit is discarded.

0x00: PA11 pin

0x01: PB11 pin

0x02: PC11 pin

0x03: PD11 pin

0x04: PE11 pin

0x05: PF11 pin

0x06: PG11 pin

0x07: PH11 pin

0x08: PI11 pin

Others: reserved

Bits 23:16 **EXTI10[7:0]**: EXTI10 GPIO port selection

These bits are written by software to select the source input for EXTI10 external interrupt. When EXTI_SECCFGR1.SEC10 is disabled, EXTI10 can be accessed with nonsecure and secure access.

When EXTI_SECCFGR1.SEC10 is enabled, EXTI10 can be accessed only with secure access. Nonsecure write is discarded and nonsecure read returns 0.

When EXTI_PRIVCFGR1.PRIV10 is disabled, EXTI10 can be accessed with privileged and unprivileged access.

When EXTI_PRIVCFGR1.PRIV10 is enabled, EXTI10 can be accessed only with privileged access. Unprivileged write to this bit is discarded.

0x00: PA10 pin

0x01: PB10 pin

0x02: PC10 pin

0x03: PD10 pin

0x04: PE10 pin

0x05: PF10 pin

0x06: PG10 pin

0x07: PH10 pin

0x08: PI10 pin

Others: reserved

Bits 15:8 **EXTI9[7:0]**: EXTI9 GPIO port selection

These bits are written by software to select the source input for EXTI9 external interrupt. When EXTI_SECCFGR1.SEC9 is disabled, EXTI9 can be accessed with nonsecure and secure access.

When EXTI_SECCFGR1.SEC9 is enabled, EXTI9 can be accessed only with secure access. Nonsecure write is discarded and nonsecure read returns 0.

When EXTI_PRIVCFGR1.PRIV9 is disabled, EXTI9 can be accessed with privileged and unprivileged access.

When EXTI_PRIVCFGR1.PRIV9 is enabled, EXTI9 can be accessed only with privileged access. Unprivileged write to this bit is discarded.

0x00: PA9 pin

0x01: PB9 pin

0x02: PC9 pin

0x03: PD9 pin

0x04: PE9 pin

0x05: PF9 pin

0x06: PG9 pin

0x07: PH9 pin

0x08: PI9 pin

Others: reserved

Bits 7:0 **EXTI8[7:0]**: EXTI8 GPIO port selection

These bits are written by software to select the source input for EXTI8 external interrupt. When EXTI_SECCFGR1.SEC8 is disabled, EXTI8 can be accessed with nonsecure and secure access.

When EXTI_SECCFGR1.SEC8 is enabled, EXTI8 can be accessed only with secure access. Nonsecure write is discarded and nonsecure read returns 0.

When EXTI_PRIVCFGR1.PRIV8 is disabled, EXTI8 can be accessed with privileged and unprivileged access.

When EXTI_PRIVCFGR1.PRIV8 is enabled, EXTI8 can be accessed only with privileged access. Unprivileged write to this bit is discarded.

0x00: PA8 pin

0x01: PB8 pin

0x02: PC8 pin

0x03: PD8 pin

0x04: PE8 pin

0x05: PF8 pin

0x06: PG8 pin

0x07: PH8 pin

0x08: PI8 pin

Others: reserved

18.7.25 EXTI external interrupt selection register (EXTI_EXTICR4)

Address offset: 0x06C EXTI multiplexer 12, 13, 14, 15

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
EXTI15[7:0]								EXTI14[7:0]							
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
EXTI13[7:0]								EXTI12[7:0]							
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bits 31:24 **EXTI15[7:0]**: EXTI15 GPIO port selection

These bits are written by software to select the source input for EXTI15 external interrupt. When EXTI_SECCFGR1.SEC15 is disabled, EXTI15 can be accessed with nonsecure and secure access.

When EXTI_SECCFGR1.SEC15 is enabled, EXTI15 can be accessed only with secure access. Nonsecure write is discarded and nonsecure read returns 0.

When EXTI_PRIVCFGR1.PRIV15 is disabled, EXTI15 can be accessed with privileged and unprivileged access.

When EXTI_PRIVCFGR1.PRIV15 is enabled, EXTI15 can be accessed only with privileged access. Unprivileged write to this bit is discarded.

0x00: PA15 pin

0x01: PB15 pin

0x02: PC15 pin

0x03: PD15 pin

0x04: PE15 pin

0x05: PF15 pin

0x06: PG15 pin

0x07: PH15 pin

Others: reserved

Bits 23:16 **EXTI14[7:0]**: EXTI14 GPIO port selection

These bits are written by software to select the source input for EXTI14 external interrupt. When EXTI_SECCFGR1.SEC14 is disabled, EXTI14 can be accessed with nonsecure and secure access.

When EXTI_SECCFGR1.SEC14 is enabled, EXTI14 can be accessed only with secure access. Nonsecure write is discarded and nonsecure read returns 0.

When EXTI_PRIVCFGR1.PRIV14 is disabled, EXTI14 can be accessed with privileged and unprivileged access.

When EXTI_PRIVCFGR1.PRIV14 is enabled, EXTI14 can be accessed only with privileged access. Unprivileged write to this bit is discarded.

0x00: PA14 pin

0x01: PB14 pin

0x02: PC14 pin

0x03: PD14 pin

0x04: PE14 pin

0x05: PF14 pin

0x06: PG14 pin

0x07: PH14 pin

Others: reserved

Bits 15:8 **EXTI13[7:0]**: EXTI13 GPIO port selection

These bits are written by software to select the source input for EXTI13 external interrupt.

When EXTI_SECCFGR1.SEC13 is disabled, EXTI13 can be accessed with nonsecure and secure access.

When EXTI_SECCFGR1.SEC13 is enabled, EXTI13 can be accessed only with secure access. Nonsecure write is discarded and nonsecure read returns 0.

When EXTI_PRIVCFGR1.PRIV13 is disabled, EXTI13 can be accessed with privileged and unprivileged access.

When EXTI_PRIVCFGR1.PRIV13 is enabled, EXTI13 can be accessed only with privileged access. Unprivileged write to this bit is discarded.

0x00: PA13 pin

0x01: PB13 pin

0x02: PC13 pin

0x03: PD13 pin

0x04: PE13 pin

0x05: PF13 pin

0x06: PG13 pin

0x07: PH13 pin

Others: reserved

Bits 7:0 **EXTI12[7:0]**: EXTI12 GPIO port selection

These bits are written by software to select the source input for EXTI12 external interrupt.

When EXTI_SECCFGR1.SEC12 is disabled, EXTI12 can be accessed with nonsecure and secure access.

When EXTI_SECCFGR1.SEC12 is enabled, EXTI12 can be accessed only with secure access. Nonsecure write is discarded and nonsecure read returns 0.

When EXTI_PRIVCFGR1.PRIV12 is disabled, EXTI12 can be accessed with privileged and unprivileged access.

When EXTI_PRIVCFGR1.PRIV12 is enabled, EXTI12 can be accessed only with privileged access. Unprivileged write to this bit is discarded.

0x00: PA12 pin

0x01: PB12 pin

0x02: PC12 pin

0x03: PD12 pin

0x04: PE12 pin

0x05: PF12 pin

0x06: PG12 pin

0x07: PH12 pin

Others: reserved

18.7.26 EXTI lock register (EXTI_LOCKR)

Address offset: 0x070

Reset value: 0x0000 0000

This register provides write access security: a nonsecure write access is ignored, a read access returns zero data, and both generate an illegal access event.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	LOCK
															rs

Bits 31:1 Reserved, must be kept at reset value.

Bit 0 **LOCK**: Global security and privilege configuration registers (EXTI_SECCFGR and EXTI_PRIVCFGR) lock

This bit is written once after reset.

0: Security and privilege configuration open, can be modified.

1: Security and privilege configuration locked, can no longer be modified.

18.7.27 EXTI CPU wake-up with interrupt mask register (EXTI_IMR1)

Address offset: 0x080

Reset value: 0xFFFFE 0000

Contains register bits for configurable events.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
IM31	IM30	IM29	IM28	IM27	IM26	IM25	IM24	IM23	IM22	IM21	IM20	IM19	IM18	IM17	IM16
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
IM15	IM14	IM13	IM12	IM11	IM10	IM9	IM8	IM7	IM6	IM5	IM4	IM3	IM2	IM1	IM0
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bits 31:0 **IMx**: CPU wake-up with interrupt mask on event input x ⁽¹⁾ (x = 31 to 0)

When EXTI_SECCFGR.SECx is disabled, IMx can be accessed with nonsecure and secure access.

When EXTI_SECCFGR.SECx is enabled, IMx can be accessed only with secure access.

Nonsecure write to this bit is discarded and nonsecure read returns 0.

When EXTI_PRIVCFGR.PRIVx is disabled, IMx can be accessed with privileged and unprivileged access.

When EXTI_PRIVCFGR.PRIVx is enabled, IMx can be accessed only with privileged access. Unprivileged write to this bit is discarded.

0: Wake-up with interrupt request from input event x is masked.

1: Wake-up with interrupt request from input event x is unmasked.

Bit IM31 is not available on all devices, refer to [Table 157](#). If not present, consider this bit as reserved, and keep it at reset value.

1. The reset value for configurable event inputs is set to 0 in order to disable the interrupt by default.

18.7.28 EXTI CPU wake-up with event mask register (EXTI_EMR1)

Address offset: 0x084

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	EM16
															rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
EM15	EM14	EM13	EM12	EM11	EM10	EM9	EM8	EM7	EM6	EM5	EM4	EM3	EM2	EM1	EM0
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bits 31:17 Reserved, must be kept at reset value.

Bits 16:0 **EMx**: CPU wake-up with event generation mask on event input x (x = 16 to 0)

When EXTI_SECCFGR.SECx is disabled, EMx can be accessed with nonsecure and secure access.

When EXTI_SECCFGR.SECx is enabled, EMx can be accessed only with secure access. Nonsecure write to this bit x is discarded and nonsecure read returns 0.

When EXTI_PRIVCFGR.PRIVx is disabled, EMx can be accessed with privileged and unprivileged access.

When EXTI_PRIVCFGR.PRIVx is enabled, EMx can be accessed only with privileged access. Unprivileged write to this bit is discarded.

0: Wake-up with event generation from Line x is masked.

1: Wake-up with event generation from Line x is unmasked.

18.7.29 EXTI CPU wake-up with interrupt mask register 2 (EXTI_IMR2)

Address offset: 0x090

Reset value: 0x23D9 BFFF (STM32H543/553xx devices)

Reset value: 0x07DB BFFF (STM32H56x/573xx devices)

Contains register bits for configurable events.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	IM61	Res.	IM59	IM58	IM57	IM56	IM55	IM54	IM53	IM52	IM51	IM50	IM49	IM48
		rw		rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
IM47	IM46	IM45	IM44	IM43	IM42	IM41	IM40	IM39	IM38	IM37	IM36	IM35	IM34	IM33	IM32
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bits 31:30 Reserved, must be kept at reset value.

Bit 29 **IM61**: CPU wake-up with interrupt mask on event input 61⁽¹⁾

When EXTI_SECCFGR.SECx is disabled, IMx can be accessed with nonsecure and secure access.

When EXTI_SECCFGR.SECx is enabled, IMx can be accessed only with secure access.

Nonsecure write to this bit is discarded and nonsecure read returns 0.

When EXTI_PRIVCFGR.PRIVx is disabled, IMx can be accessed with privileged and unprivileged access.

When EXTI_PRIVCFGR.PRIVx is enabled, IMx can be accessed only with privileged access. Unprivileged write to this bit is discarded.

0: Wake-up with interrupt request from input event x is masked.

1: Wake-up with interrupt request from input event x is unmasked.

This bit is not available on all devices, refer to [Table 157](#). If not present, consider this bit as reserved, and keep it at reset value.

Bit 28 Reserved, must be kept at reset value.

Bits 27:0 **IMx**: CPU wake-up with interrupt mask on event input x ⁽¹⁾ (x = 59 to 0)

When EXTI_SECCFGR.SECx is disabled, IMx can be accessed with nonsecure and secure access.

When EXTI_SECCFGR.SECx is enabled, IMx can be accessed only with secure access.

Nonsecure write to this bit is discarded and nonsecure read returns 0.

When EXTI_PRIVCFGR.PRIVx is disabled, IMx can be accessed with privileged and unprivileged access.

When EXTI_PRIVCFGR.PRIVx is enabled, IMx can be accessed only with privileged access. Unprivileged write to this bit is discarded.

0: Wake-up with interrupt request from input event x is masked.

1: Wake-up with interrupt request from input event x is unmasked.

Bits IM[59:54], IM52, IM[46:44], and IM[36:32] are not available on all devices, refer to [Table 157](#). If not present, consider these bits as reserved, and keep them at reset value.

1. The reset value for configurable event inputs is set to 0 in order to disable the interrupt by default.

18.7.30 EXTI CPU wake-up with event mask register 2 (EXTI_EMR2)

Address offset: 0x094

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	EM59	EM58	Res.	Res.	Res.	Res.	EM53	Res.	Res.	EM50	EM49	Res.
				rw	rw					rw			rw	rw	
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	EM46	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
	rw														

Bits 31:28 Reserved, must be kept at reset value.

Bits 27:26 **EMx**: CPU wake-up with event generation mask on event input x (x = 59 to 58)

When EXTI_SECCFGR.SECx is disabled, EMx can be accessed with nonsecure and secure access.

When EXTI_SECCFGR.SECx is enabled, EMx can be accessed only with secure access. Nonsecure write to this bit x is discarded and nonsecure read returns 0.

When EXTI_PRIVCFGR.PRIVx is disabled, EMx can be accessed with privileged and unprivileged access.

When EXTI_PRIVCFGR.PRIVx is enabled, EMx can be accessed only with privileged access. Unprivileged write to this bit is discarded.

0: Wake-up with event generation from Line x is masked.

1: Wake-up with event generation from Line x is unmasked.

These bits are available only on STM32H543/553xx devices, refer to [Table 157](#). If not present, consider these bits as reserved, and keep them at reset value.

Bits 25:22 Reserved, must be kept at reset value.

Bit 21 **EM53**: CPU wake-up with event generation mask on event input

When EXTI_SECCFGR.SECx is disabled, EM53 can be accessed with nonsecure and secure access.

When EXTI_SECCFGR.SECx is enabled, EM53 can be accessed only with secure access. Nonsecure write to this bit is discarded and nonsecure read returns 0.

When EXTI_PRIVCFGR.PRIVx is disabled, EM53 can be accessed with privileged and unprivileged access.

When EXTI_PRIVCFGR.PRIVx is enabled, EM53 can be accessed only with privileged access. Unprivileged write to this bit is discarded.

0: Wake-up with event generation from Line x is masked.

1: Wake-up with event generation from Line x is unmasked.

Bits 20:19 Reserved, must be kept at reset value.

Bits 18:17 **EMx**: CPU wake-up with event generation mask on event input x (x = 50 to 49)

When EXTI_SECCFGR.SECx is disabled, EMx can be accessed with nonsecure and secure access.

When EXTI_SECCFGR.SECx is enabled, EMx can be accessed only with secure access. Nonsecure write to this bit is discarded and nonsecure read returns 0.

When EXTI_PRIVCFGR.PRIVx is disabled, EMx can be accessed with privileged and unprivileged access.

When EXTI_PRIVCFGR.PRIVx is enabled, EMx can be accessed only with privileged access. Unprivileged write to this bit is discarded.

0: Wake-up with event generation from Line x is masked.

1: Wake-up with event generation from Line x is unmasked.

The EM49 bit is available only on STM32H543/553xx devices.

Bits 16:15 Reserved, must be kept at reset value.

Bit 14 **EM46**: CPU wake-up with event generation mask on event input

When EXTI_SECCFGR.SECx is disabled, EM46 can be accessed with nonsecure and secure access.

When EXTI_SECCFGR.SECx is enabled, EM46 can be accessed only with secure access. Nonsecure write to this bit is discarded and nonsecure read returns 0.

When EXTI_PRIVCFGR.PRIVx is disabled, EM46 can be accessed with privileged and unprivileged access.

When EXTI_PRIVCFGR.PRIVx is enabled, EM46 can be accessed only with privileged access. Unprivileged write to this bit is discarded.

0: Wake-up with event generation from Line x is masked.

1: Wake-up with event generation from Line x is unmasked.

Bits 13:0 Reserved, must be kept at reset value.

18.7.31 EXTI CPU wake-up with interrupt mask register 3 (EXTI_IMR3)

Address offset: 0x0A0

Reset value: 0x0000 0002

Contains register bits for configurable events.

STM32H543/553xx devices only

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	IM66	IM65	IM64
													rw	rw	rw

Bits 31:3 Reserved, must be kept at reset value.

Bits 2:0 **IMx**: CPU wake-up with interrupt mask on event input x ⁽¹⁾ (x = 66 to 64)

When EXTI_SECCFGR.SECx is disabled, IMx can be accessed with nonsecure and secure access.

When EXTI_SECCFGR.SECx is enabled, IMx can only be accessed with secure access. Nonsecure write to this bit is discarded and nonsecure read returns 0.

When EXTI_PRIVCFGR.PRIVx is disabled, IMx can be accessed with privileged and unprivileged access.

When EXTI_PRIVCFGR.PRIVx is enabled, IMx can only be accessed with privileged access. Unprivileged write to this bit is discarded.

0: Wake-up with interrupt request from input event x is masked.

1: Wake-up with interrupt request from input event x is unmasked.

1. The reset value for configurable event inputs is set to 0 to disable the interrupt by default.

18.7.32 EXTI CPU wake-up with event mask register 3 (EXTI_EMR3)

Address offset: 0x0A4

Reset value: 0x0000 0000

STM32H543/553xx devices only

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	EM66	Res.	EM64
													rw		rw

Bits 31:3 Reserved, must be kept at reset value.

Bit 2 EM66: CPU wake-up with event generation mask on event input

When EXTI_SECCFGR.SECx is disabled, EM66 can be accessed with nonsecure and secure access.

When EXTI_SECCFGR.SECx is enabled, EM66 can only be accessed with secure access. Nonsecure write to this bit x is discarded and nonsecure read returns 0.

When EXTI_PRIVCFGR.PRIVx is disabled, EM66 can be accessed with privileged and unprivileged access.

When EXTI_PRIVCFGR.PRIVx is enabled, EM66 can only be accessed with privileged access. Unprivileged write to this bit is discarded.

0: Wake-up with event generation from Line x is masked.

1: Wake-up with event generation from Line x is unmasked.

Bit 1 Reserved, must be kept at reset value.

Bit 0 EM64: CPU wake-up with event generation mask on event input

When EXTI_SECCFGR.SECx is disabled, EM64 can be accessed with nonsecure and secure access.

When EXTI_SECCFGR.SECx is enabled, EM64 can only be accessed with secure access. Nonsecure write to this bit x is discarded and nonsecure read returns 0.

When EXTI_PRIVCFGR.PRIVx is disabled, EM64 can be accessed with privileged and unprivileged access.

When EXTI_PRIVCFGR.PRIVx is enabled, EM64 can only be accessed with privileged access. Unprivileged write to this bit is discarded.

0: Wake-up with event generation from Line x is masked.

1: Wake-up with event generation from Line x is unmasked.

18.7.33 EXTI register map

Table 161. EXTI register map and reset values

Offset	Register name	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0x000	EXTI_RTSR1	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	RT16	RT15	RT14	RT13	RT12	RT11	RT10	RT9	RT8	RT7	RT6	RT5	RT4	RT3	RT2	RT1	RT0
	Reset value																0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0x004	EXTI_FTSR1	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	FT16	FT15	FT14	FT13	FT12	FT11	FT10	FT9	FT8	FT7	FT6	FT5	FT4	FT3	FT2	FT1	FT0
	Reset value																0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0x008	EXTI_SWIER1	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	SWI16	SWI15	SWI14	SWI13	SWI12	SWI11	SWI10	SWI9	SWI8	SWI7	SWI6	SWI5	SWI4	SWI3	SWI2	SWI1	SWI0
	Reset value																0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0x00C	EXTI_RPR1	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	RPIF1	RPIF1	RPIF1	RPIF1	RPIF1	RPIF1	RPIF1	RPIF9	RPIF8	RPIF7	RPIF6	RPIF5	RPIF4	RPIF3	RPIF2	RPIF1	RPIF0
	Reset value																0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0x010	EXTI_FPR1	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	FPIF16	FPIF15	FPIF14	FPIF13	FPIF12	FPIF11	FPIF10	FPIF9	FPIF8	FPIF7	FPIF6	FPIF5	FPIF4	FPIF3	FPIF2	FPIF1	FPIF0
	Reset value																0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0x014	EXTI_SECCFGR1	SEC31	SEC30	SEC29	SEC28	SEC27	SEC26	SEC25	SEC24	SEC23	SEC22	SEC21	SEC20	SEC19	SEC18	SEC17	SEC16	SEC15	SEC14	SEC13	SEC12	SEC11	SEC10	SEC9	SEC8	SEC7	SEC6	SEC5	SEC4	SEC3	SEC2	SEC1	SEC0
	Reset value	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0x018	EXTI_PRIVCFGR1	PRIV31	PRIV30	PRIV29	PRIV28	PRIV27	PRIV26	PRIV25	PRIV24	PRIV23	PRIV22	PRIV21	PRIV20	PRIV19	PRIV18	PRIV17	PRIV16	PRIV15	PRIV14	PRIV13	PRIV12	PRIV11	PRIV10	PRIV9	PRIV8	PRIV7	PRIV6	PRIV5	PRIV4	PRIV3	PRIV2	PRIV1	PRIV0
	Reset value	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0x020	EXTI_RTSR2 ⁽²⁾	Res.	Res.	Res.	Res.	RT59	RT58	Res.	Res.	Res.	Res.	RT53	RT50	RT49	Res.	Res.	Res.	Res.	RT46	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
	Reset value					0	0					0	0	0	0	0	0	0	0														
0x024	EXTI_FTSR2 ⁽²⁾	Res.	Res.	Res.	Res.	FT59	FT58	Res.	Res.	Res.	Res.	FT53	FT50	FT49	Res.	Res.	Res.	Res.	FT46	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
	Reset value					0	0					0	0	0	0	0	0	0	0														
0x028	EXTI_SWIER2 ⁽²⁾	Res.	Res.	Res.	Res.	SWI59	SWI58	Res.	Res.	Res.	Res.	SWI53	SWI50	SWI49	Res.	Res.	Res.	Res.	SWI46	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
	Reset value					0	0					0	0	0	0	0	0	0	0														
0x02C	EXTI_RPR2 ⁽²⁾	Res.	Res.	Res.	Res.	RPIF59	RPIF58	Res.	Res.	Res.	Res.	RPIF53	RPIF50	RPIF49	Res.	Res.	Res.	Res.	RPIF46	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
	Reset value					0	0					0	0	0	0	0	0	0	0														
0x030	EXTI_FPR2 ⁽²⁾	Res.	Res.	Res.	Res.	FPIF59	FPIF58	Res.	Res.	Res.	Res.	FPIF53	FPIF50	FPIF49	Res.	Res.	Res.	Res.	FPIF46	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
	Reset value					0	0					0	0	0	0	0	0	0	0														
0x034	EXTI_SECCFGR2 ⁽²⁾	Res.	Res.	SEC61	Res.	SEC59	SEC58	SEC57	SEC56	SEC55	SEC54	SEC53	SEC52	SEC51	SEC50	SEC49	SEC48	SEC47	SEC46	SEC45	SEC44	SEC43	SEC42	SEC41	SEC40	SEC39	SEC38	SEC37	SEC36	SEC35	SEC34	SEC33	SEC32
	Reset value			0		0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0x038	EXTI_PRIVCFGR2 ⁽²⁾	Res.	Res.	PRIV61	Res.	PRIV59	PRIV58	PRIV57	PRIV56	PRIV55	PRIV54	PRIV53	PRIV52	PRIV51	PRIV50	PRIV49	PRIV48	PRIV47	PRIV46	PRIV45	PRIV44	PRIV43	PRIV42	PRIV41	PRIV40	PRIV39	PRIV38	PRIV37	PRIV36	PRIV35	PRIV34	PRIV33	PRIV32
	Reset value			0		0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0x03C	Reserved	Reserved																															
0x040	EXTI_RTSR3 ⁽¹⁾	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
	Reset value																																

Table 161. EXTI register map and reset values (continued)

Offset	Register name	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0x044	EXTI_FTSR3	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
	Reset value																																
0x048	EXTI_SWIER3	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
	Reset value																																
0x04C	EXTI_RPR3 ⁽¹⁾	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
	Reset value																																
0x050	EXTI_FPR3 ⁽¹⁾	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
	Reset value																																
0x054	EXTI_SECCFGR3 ⁽¹⁾	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
	Reset value																																
0x058	EXTI_PRIVCFGR3 ⁽¹⁾	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
	Reset value																																
0x05C	Reserved	Reserved																															
0x060	EXTI_EXTICR1	EXTI3[7:0]								EXTI2[7:0]								EXTI1[7:0]								EXTI0[7:0]							
	Reset value	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0x064	EXTI_EXTICR2	EXTI7[7:0]								EXTI6[7:0]								EXTI5[7:0]								EXTI4[7:0]							
	Reset value	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0x068	EXTI_EXTICR3	EXTI11[7:0]								EXTI10[7:0]								EXTI9[7:0]								EXTI8[7:0]							
	Reset value	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0x06C	EXTI_EXTICR4	EXTI15[7:0]								EXTI14[7:0]								EXTI13[7:0]								EXTI12[7:0]							
	Reset value	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0x070	EXTI_LOCKR	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
	Reset value																																
0x074 to 0x07C	Reserved	Reserved																															
0x080	EXTI_IMR1 ⁽²⁾	IM31	IM30	IM29	IM28	IM27	IM26	IM25	IM24	IM23	IM22	IM21	IM20	IM19	IM18	IM17	IM16	IM15	IM14	IM13	IM12	IM11	IM10	IM9	IM8	IM7	IM6	IM5	IM4	IM3	IM2	IM1	IM0
	Reset value	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0x084	EXTI_EMR1 ⁽²⁾	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	EM16	EM15	EM14	EM13	EM12	EM11	EM10	EM9	EM8	EM7	EM6	EM5	EM4	EM3	EM2	EM1	EM0
	Reset value																0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0x090	EXTI_IMR2 ⁽²⁾	Res.	Res.	IM61	Res.	IM59	IM58	IM57	IM56	IM55	IM54	IM53	IM52	IM51	IM50	IM49	IM48	IM47	IM46	IM45	IM44	IM43	IM42	IM41	IM40	IM39	IM38	IM37	IM36	IM35	IM34	IM33	IM32
	Reset value			1		1	1	1	1	1	1	1	1	1	1	0	1	1	0	1	1	1	1	1	1	1	1	1	1	1	1	1	1

Table 161. EXTI register map and reset values (continued)

Offset	Register name	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0x094	EXTI_EMR2 ⁽²⁾	Res.	Res.	Res.	Res.	EM59	EM58	Res.	Res.	Res.	Res.	EM53	Res.	Res.	EM50	EM49	Res.	Res.	EM46	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
	Reset value					0	0					0			0	0			0														0
0x0A0	EXTI_IMR3 ⁽¹⁾	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	IM66	IM65	IM64
	Reset value																														0	1	0
0x0A4	EXTI_EMR3 ⁽¹⁾	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	EM66	EM65	EM64
	Reset value																														0	1	0

1. Available only on STM32H543/553xx devices.

2. Some bits in this register are not available on some devices, refer to [Table 157](#). If not present, consider these bits as reserved, and keep them at reset value.

Refer to [Section 2.3: Memory organization](#) for the register boundary addresses.

19 Cyclic redundancy check calculation unit (CRC)

19.1 CRC introduction

The CRC (cyclic redundancy check) calculation unit is used to get a CRC code from 8-, 16- or 32-bit data word and a generator polynomial.

Among other applications, CRC-based techniques are used to verify data transmission or storage integrity. In the scope of the functional safety standards, they offer a means of verifying the flash memory integrity. The CRC calculation unit helps compute a signature of the software during runtime, to be compared with a reference signature generated at link time and stored at a given memory location.

19.2 CRC main features

- Uses CRC-32 (Ethernet) polynomial: 0x4C11DB7
$$X^{32} + X^{26} + X^{23} + X^{22} + X^{16} + X^{12} + X^{11} + X^{10} + X^8 + X^7 + X^5 + X^4 + X^2 + X + 1$$
- Alternatively, uses fully programmable polynomial with programmable size (7, 8, 16, 32 bits)
- Handles 8-, 16-, 32-bit data size
- Programmable CRC initial value
- Single input/output 32-bit data register
- Input buffer to avoid bus stall during calculation
- CRC computation done in 4 AHB clock cycles (HCLK) for the 32-bit data size
- General-purpose 8-bit register (can be used for temporary storage)
- Reversibility options on I/O data
- Accessed through AHB slave peripheral by 32-bit words only, with the exception of CRC_DR register that can be accessed by words, right-aligned half-words and right-aligned bytes

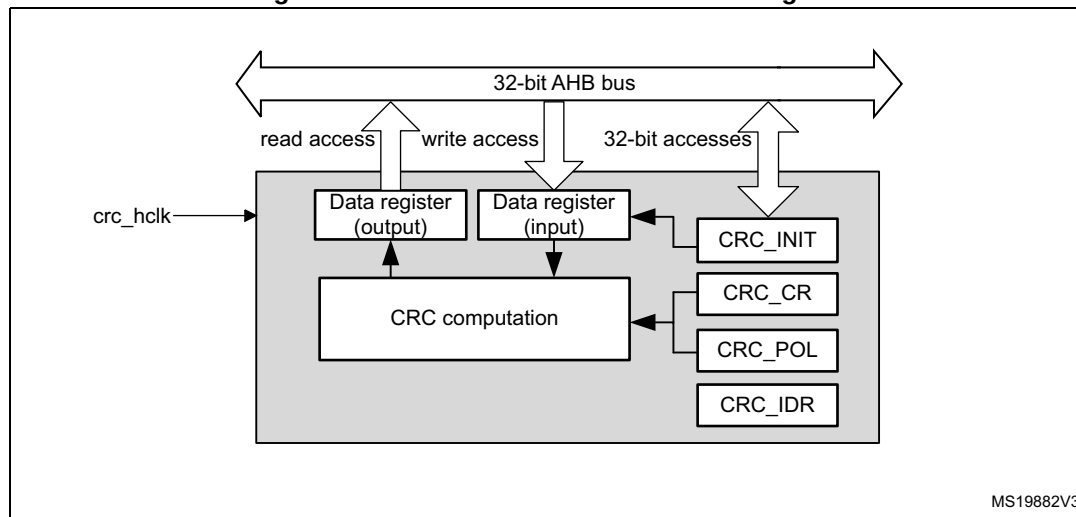
19.3 CRC implementation

The byte or half word swapping features controlled by RTYPE_x bits in CRC_CR are only present for STM32H543/553xx devices.

19.4 CRC functional description

19.4.1 CRC block diagram

Figure 100. CRC calculation unit block diagram



19.4.2 CRC internal signals

Table 162. CRC internal input/output signals

Signal name	Signal type	Description
crc_hclk	Digital input	AHB clock

19.4.3 CRC operation

The CRC calculation unit has a single 32-bit read/write data register (CRC_DR). It is used to input new data (write access), and holds the result of the previous CRC calculation (read access).

Each write operation to the data register creates a combination of the previous CRC value (stored in CRC_DR) and the new one. CRC computation is done on the whole 32-bit data word or byte by byte depending on the format of the data being written.

The CRC_DR register can be accessed by word, right-aligned half-word and right-aligned byte. When the RTYPE_IN bit is set in CRC_CR, non-word accesses to CRC_DR register are ignored. For the other registers only 32-bit accesses are allowed.

The duration of the computation depends on input data width:

- 4 AHB clock cycles for 32 bits
- 2 AHB clock cycles for 16 bits
- 1 AHB clock cycles for 8 bits

An input buffer allows a second data to be immediately written without waiting for any wait-states due to the previous CRC calculation.

The data size can be dynamically adjusted to minimize the number of write accesses for a given number of bytes. For instance, a CRC for 5 bytes can be computed with a word write followed by a byte write.

The input data can be reversed to manage the various endianness schemes. The reversing operation can be performed on 8 bits, 16 bits and 32 bits depending on the REV_IN[1:0] bits in the CRC_CR register. If RTYPE_IN bit is set in this register, the input reversing operation can be performed with a byte or half-word granularity, but not with single bit granularity.

For example, with RTYPE_IN cleared, 0x1A2B3C4D input data are used for CRC calculation as:

- 0x58D43CB2 with bit-reversal done by byte (REV_IN[1:0] = 01)
- 0xD458B23C with bit-reversal done by half-word (REV_IN[1:0] = 10)
- 0xB23CD458 with bit-reversal done on the full word (REV_IN[1:0] = 11)

With RTYPE_IN set, 0x1A2B3C4D input data are used for CRC calculation as:

- 0x3C4D1A2B with half-word reversal done by word (REV_IN[1:0] = 01)
- 0x4D3C2B1A with byte-reversal done by word (REV_IN[1:0] = 10)

The output data can also be reversed by setting the REV_OUT[1:0] bits in the CRC_CR register. If RTYPE_OUT is set in this register, the output reversing operation is performed with a byte or half-word granularity, but not with single bit granularity.

With RTYPE_OUT cleared, the operation is done at bit level. For example, 0x11223344 output data are converted to 0x22CC4488.

With RTYPE_OUT set, 0x11223344 output data are converted as:

- 0x33441122 with half-word reversal done by word (REV_IN[1:0] = 01)
- 0x44332211 with byte-reversal done by word (REV_IN[1:0] = 10)

The CRC calculator can be initialized to a programmable value using the RESET control bit in the CRC_CR register (the default value is 0xFFFFFFFF).

The initial CRC value can be programmed with the CRC_INIT register. The CRC_DR register is automatically initialized upon CRC_INIT register write access.

The CRC_IDR register can be used to hold a temporary value related to CRC 7-, 8-, 16-, and 32-bit calculations. It is not affected by the RESET bit in the CRC_CR register.

Polynomial programmability

The polynomial coefficients are fully programmable through the CRC_POL register, and the polynomial size can be configured to be 7-, 8-, 16-, or 32-bits by programming the POLYSIZE[1:0] bits in the CRC_CR register. Even polynomials are not supported.

Note: The type of an even polynomial is $X+X^2+..+X^n$, while the type of an odd polynomial is $1+X+X^2+..+X^n$.

If the CRC data is less than 32-bit, its value can be read from the least significant bits of the CRC_DR register.

To obtain a reliable CRC calculation, the change on-fly of the polynomial value or size can not be performed during a CRC calculation. As a result, if a CRC calculation is ongoing, the application must either reset it or perform a CRC_DR read before changing the polynomial.

The default polynomial value is the CRC-32 (Ethernet) polynomial: 0x4C11DB7.

19.5 CRC registers

The CRC_DR register can be accessed by words, right-aligned half-words and right-aligned bytes when RTYPE bit is cleared in CRC_CR. When RTYPE is set, only word accesses are allowed. For the other registers only 32-bit accesses are allowed.

19.5.1 CRC data register (CRC_DR)

Address offset: 0x00

Reset value: 0xFFFF FFFF

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
DR[31:16]															
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
DR[15:0]															
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bits 31:0 **DR[31:0]**: Data register bits

This register is used to write new data to the CRC calculator.

It holds the previous CRC calculation result when it is read.

If the data size is less than 32 bits, the least significant bits are used to write/read the correct value.

19.5.2 CRC independent data register (CRC_IDR)

Address offset: 0x04

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
IDR[31:16]															
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
IDR[15:0]															
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bits 31:0 **IDR[31:0]**: General-purpose 32-bit data register bits

These bits can be used as a temporary storage location for four bytes.

This register is not affected by CRC resets generated by the RESET bit in the CRC_CR register

19.5.3 CRC control register (CRC_CR)

Address offset: 0x08

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	Res.	Res.	Res.	RTYPE_OUT	RTYPE_IN	REV_OUT[1:0]		REV_IN[1:0]		POLYSIZE[1:0]		Res.	Res.	RESET
					rw	rw	rw	rw	rw	rw	rw	rw			rs

Bits 31:11 Reserved, must be kept at reset value.

Bit 10 **RTYPE_OUT**: Reverse type output

This bit controls the reversal granularity of the output data.

0: Bit level output

1: Byte or half-word level output

Bit 9 **RTYPE_IN**: Reverse type input

This bit controls the reversal granularity of the input data.

0: Bit level input

1: Byte or half-word level input

Bits 8:7 **REV_OUT[1:0]**: Reverse output data

This bitfield controls the reversal of the bit order of the output data.

00: Bit order not affected (RTYPE_OUT = 0 or 1)

01: Bit-reversed output format (RTYPE_OUT = 0) or half-word reversal done by word (RTYPE_OUT = 1)

10: Bit order not affected (RTYPE_OUT = 0) or byte reversal done by word (RTYPE_OUT = 1)

11: Bit order not affected (RTYPE_OUT = 0 or 1)

Bits 6:5 **REV_IN[1:0]**: Reverse input data

This bitfield controls the reversal of the bit order of the input data

00: Bit order not affected (RTYPE_IN = 0 or 1)

01: Bit reversal done by byte (RTYPE_IN = 0) or half-word reversal done by word (RTYPE_IN = 1)

10: Bit reversal done by half-word (RTYPE_IN = 0) or byte reversal done by word (RTYPE_IN = 1)

11: Bit reversal done by word (RTYPE_IN = 0) or bit order is not affected (RTYPE_IN = 1)

Bits 4:3 **POLYSIZE[1:0]**: Polynomial size

These bits control the size of the polynomial.

00: 32 bit polynomial

01: 16 bit polynomial

10: 8 bit polynomial

11: 7 bit polynomial

Bits 2:1 Reserved, must be kept at reset value.

Bit 0 **RESET**: RESET bit

This bit is set by software to reset the CRC calculation unit and set the data register to the value stored in the CRC_INIT register. This bit can only be set, it is automatically cleared by hardware.

19.5.4 CRC initial value (CRC_INIT)

Address offset: 0x10

Reset value: 0xFFFF FFFF

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
CRC_INIT[31:16]															
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
CRC_INIT[15:0]															
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bits 31:0 **CRC_INIT[31:0]**: Programmable initial CRC value

This register is used to write the CRC initial value.

19.5.5 CRC polynomial (CRC_POL)

Address offset: 0x14

Reset value: 0x04C1 1DB7

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
POL[31:16]															
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
POL[15:0]															
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bits 31:0 **POL[31:0]**: Programmable polynomial

This register is used to write the coefficients of the polynomial to be used for CRC calculation.

If the polynomial size is less than 32 bits, the least significant bits have to be used to program the correct value.

19.5.6 CRC register map

Table 163. CRC register map and reset values

Offset	Register name	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0x00	CRC_DR	DR[31:0]																															
	Reset value	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1
0x04	CRC_IDR	IDR[31:0]																															
	Reset value	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0x08	CRC_CR	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	RTYPE_OUT	RTYPE_IN	REV_OUT[1:0]	REV_IN[1:0]	POLYSIZE[1:0]	Res.	Res.	RESET			
	Reset value																					0	0	0	0	0	0	0	0	0			0
0x10	CRC_INIT	CRC_INIT[31:0]																															
	Reset value	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1
0x14	CRC_POL	POL[31:0]																															
	Reset value	0	0	0	0	0	1	0	0	1	1	0	0	0	0	0	1	0	0	0	1	1	1	0	1	1	0	1	1	0	1	1	1

Refer to [Section 2.3: Memory organization](#) for the register boundary addresses.

20 CORDIC coprocessor (CORDIC)

This section applies only to STM32H543/53/62/63/73xx devices.

20.1 CORDIC introduction

The CORDIC coprocessor provides hardware acceleration of mathematical functions (mainly trigonometric ones) commonly used in motor control, metering, signal processing, and many other applications.

It speeds up the calculation of these functions compared to a software implementation, making possible the use of a lower operating frequency, or freeing up processor cycles to perform other tasks.

20.2 CORDIC main features

- 24-bit CORDIC rotation engine
- Circular and Hyperbolic modes
- Rotation and Vectoring modes
- Functions: sine, cosine, sinh, cosh, atan, atan2, atanh, modulus, square root, natural logarithm
- Programmable precision
- Low latency AHB slave interface
- Results can be read as soon as ready, without polling or interrupt
- DMA read and write channels
- Multiple register read/write by DMA

20.3 CORDIC functional description

20.3.1 General description

The CORDIC is a cost-efficient successive approximation algorithm for evaluating trigonometric and hyperbolic functions.

In trigonometric (circular) mode, the sine and cosine of an angle θ are determined by rotating the unit vector $[1, 0]$ through decreasing angles until the cumulative sum of the rotation angles equals the input angle θ . The x and y cartesian components of the rotated vector then correspond, respectively, to the cosine and sine of θ . Inversely, the angle of a vector $[x, y]$ corresponding to arctangent (y / x), is determined by rotating $[x, y]$ through successively decreasing angles to obtain the unit vector $[1, 0]$. The cumulative sum of the rotation angles gives the angle of the original vector.

The CORDIC algorithm can also be used for calculating hyperbolic functions (sinh, cosh, atanh), by replacing the successive circular rotations by steps along a hyperbole.

Other functions can be derived from the basic functions described above.

20.3.2 CORDIC functions

The first step when using the coprocessor is to select the required function, by programming the FUNC field of the CORDIC_CR register.

[Table 164](#) lists the functions supported by the CORDIC coprocessor.

Table 164. CORDIC functions

Function	Primary argument (ARG1)	Secondary argument (ARG2)	Primary result (RES1)	Secondary result (RES2)
Cosine	angle θ	modulus m	$m \cdot \cos \theta$	$m \cdot \sin \theta$
Sine	angle θ	modulus m	$m \cdot \sin \theta$	$m \cdot \cos \theta$
Phase	x	y	$\text{atan2}(y,x)$	$\sqrt{x^2 + y^2}$
Modulus	x	y	$\sqrt{x^2 + y^2}$	$\text{atan2}(y,x)$
Arctangent	x	none	$\tan^{-1} x$	none
Hyperbolic cosine	x	none	$\cosh x$	$\sinh x$
Hyperbolic sine	x	none	$\sinh x$	$\cosh x$
Hyperbolic arctangent	x	none	$\tanh^{-1} x$	none
Natural logarithm	x	none	$\ln x$	none
Square root	x	none	$\sqrt{x}$	none

Several functions take two input arguments (ARG1 and ARG2), and some generate two results (RES1 and RES2) simultaneously. This is a side-effect of the algorithm and means that only one operation is needed to obtain two values. This is the case, for example, when performing polar-to-rectangular conversion: $\sin \theta$ also generates $\cos \theta$, and $\cos \theta$ also generates $\sin \theta$. Similarly for rectangular-to-polar conversion ($\text{phase}(x,y)$, $\text{modulus}(x,y)$) and for hyperbolic functions ($\cosh \theta$, $\sinh \theta$).

Note: *The exponential function, $\exp x$, can be obtained as the sum of $\sinh x$ and $\cosh x$. Furthermore, base N logarithms, $\log_N x$, can be derived by multiplying $\ln x$ by a constant K , where $K = 1/\ln N$.*

For certain functions (atan, log, sqrt) a scaling factor (see [Section 20.3.4](#)) can be applied to extend the range beyond the maximum $[-1, 1]$ supported by the q1.31 fixed point format. The scaling factor must be set to 0 for all other circular functions, and to 1 for hyperbolic functions.

Cosine

Table 165. Cosine parameters

Parameter	Description	Range
ARG1	Angle θ in radians, divided by π	$[-1, 1]$
ARG2	Modulus m	$[0, 1]$
RES1	$m \cdot \cos \theta$	$[-1, 1]$
RES2	$m \cdot \sin \theta$	$[-1, 1]$
SCALE	Not applicable	0

This function calculates the cosine of an angle in the range $-\pi$ to π . It can also be used to perform polar to rectangular conversion.

The primary argument is the angle θ in radians. It must be divided by π before programming ARG1.

The secondary argument is the modulus m . If m is greater than 1, a scaling must be applied in software to adapt it to the q1.31 range of ARG2.

The primary result, RES1, is the cosine of the angle, multiplied by the modulus.

The secondary result, RES2, is the sine of the angle, multiplied by the modulus.

Sine

Table 166. Sine parameters

Parameter	Description	Range
ARG1	Angle θ in radians, divided by π	$[-1, 1]$
ARG2	Modulus m	$[0, 1]$
RES1	$m \cdot \sin \theta$	$[-1, 1]$
RES2	$m \cdot \cos \theta$	$[-1, 1]$
SCALE	Not applicable	0

This function calculates the sine of an angle in the range $-\pi$ to π . It can also be used to perform polar to rectangular conversion.

The primary argument is the angle θ in radians. It must be divided by π before programming ARG1.

The secondary argument is the modulus m . If m is greater than 1, a scaling must be applied in software to adapt it to the q1.31 range of ARG2.

The primary result, RES1, is the sine of the angle, multiplied by the modulus.

The secondary result, RES2, is the cosine of the angle, multiplied by the modulus.

Phase

Table 167. Phase parameters

Parameter	Description	Range
ARG1	x coordinate	[-1, 1]
ARG2	y coordinate	[-1, 1]
RES1	Phase angle θ in radians, divided by π	[-1, 1]
RES2	Modulus m	[0, 1]
SCALE	Not applicable	0

This function calculates the phase angle in the range $-\pi$ to π of a vector $\mathbf{v} = [x \ y]$ (also known as $\text{atan2}(y,x)$). It can also be used to perform rectangular to polar conversion.

The primary argument is the x coordinate, that is, the magnitude of the vector in the direction of the x axis. If $|x| > 1$, a scaling must be applied in software to adapt it to the q1.31 range of ARG1.

The secondary argument is the y coordinate, that is, the magnitude of the vector in the direction of the y axis. If $|y| > 1$, a scaling must be applied in software to adapt it to the q1.31 range of ARG2.

The primary result, RES1, is the phase angle θ of the vector $\mathbf{v}$. RES1 must be multiplied by π to obtain the angle in radians. Note that values close to π may sometimes wrap to $-\pi$ due to the circular nature of the phase angle.

The secondary result, RES2, is the modulus, given by: $|\mathbf{v}| = \sqrt{x^2 + y^2}$. If $|\mathbf{v}| > 1$ the result in RES2 is saturated to 1.

Modulus

Table 168. Modulus parameters

Parameter	Description	Range
ARG1	x coordinate	[-1, 1]
ARG2	y coordinate	[-1, 1]
RES1	Modulus m	[0, 1]
RES2	Phase angle θ	[-1, 1]
SCALE	Not applicable	0

This function calculates the magnitude, or modulus, of a vector $\mathbf{v} = [x \ y]$. It can also be used to perform rectangular to polar conversion.

The primary argument is the x coordinate, that is, the magnitude of the vector in the direction of the x axis. If $|x| > 1$, a scaling must be applied in software to adapt it to the q1.31 range of ARG1.

The secondary argument is the y coordinate, that is, the magnitude of the vector in the direction of the y axis. If $|y| > 1$, a scaling must be applied in software to adapt it to the q1.31 range of ARG2.

The primary result, RES1, is the modulus, given by: $|v| = \sqrt{x^2 + y^2}$. If $|v| > 1$ the result in RES1 is saturated to 1.

The secondary result, RES2, is the phase angle θ of the vector v . RES2 must be multiplied by π to obtain the angle in radians. Note that values close to π may sometimes wrap to $-\pi$ due to the circular nature of the phase angle.

Arctangent

Table 169. Arctangent parameters

Parameter	Description	Range
ARG1	$x \cdot 2^{-n}$	[-1, 1]
ARG2	Not applicable	-
RES1	$2^{-n} \cdot \tan^{-1} x$, in radians, divided by p	[-1, 1]
RES2	Not applicable	-
SCALE	n	[0 7]

This function calculates the arctangent, or inverse tangent, of the input argument x .

The primary argument, ARG1, is the input value, $x = \tan \theta$. If $|x| > 1$, a scaling factor of 2^{-n} must be applied in software such that $-1 < x \cdot 2^{-n} < 1$. The scaled value $x \cdot 2^{-n}$ is programmed in ARG1 and the scale factor n must be programmed in the SCALE parameter.

Note that the maximum input value allowed is $\tan \theta = 128$, which corresponds to an angle $\theta = 89.55$ degrees. For $|x| > 128$, a software method must be used to find $\tan^{-1} x$.

The secondary argument, ARG2, is unused.

The primary result, RES1, is the angle $\theta = \tan^{-1} x$. RES1 must be multiplied by $2^n \cdot \pi$ to obtain the angle in radians.

The secondary result, RES2, is unused.

Hyperbolic cosine

Table 170. Hyperbolic cosine parameters

Parameter	Description	Range
ARG1	$x \cdot 2^{-n}$	[-0.559 0.559]
ARG2	Not applicable	-
RES1	$2^{-n} \cdot \cosh x$	[0.5 0.846]
RES2	$2^{-n} \cdot \sinh x$	[-0.683 0.683]
SCALE	n	1

This function calculates the hyperbolic cosine of a hyperbolic angle x . It can also be used to calculate the exponential functions $e^x = \cosh x + \sinh x$, and $e^{-x} = \cosh x - \sinh x$.

The primary argument is the hyperbolic angle x . Only values of x in the range -1.118 to +1.118 are supported. Since the minimum value of $\cosh x$ is 1, which is beyond the range of the q1.31 format, a scaling factor of 2^{-n} must be applied in software. The factor $n = 1$ must be programmed in the SCALE parameter.

The secondary argument is not used.

The primary result, RES1, is the hyperbolic cosine, $\cosh x$. RES1 must be multiplied by 2 to obtain the correct result.

The secondary result, RES2, is the hyperbolic sine, $\sinh x$. RES2 must be multiplied by 2 to obtain the correct result.

Hyperbolic sine

Table 171. Hyperbolic sine parameters

Parameter	Description	Range
ARG1	$x \cdot 2^{-n}$	[-0.559, 0.559]
ARG2	Not applicable	-
RES1	$2^{-n} \cdot \sinh x$	[-0.683, 0.683]
RES2	$2^{-n} \cdot \cosh x$	[0.5, 0.846]
SCALE	n	1

This function calculates the hyperbolic sine of a hyperbolic angle x . It can also be used to calculate the exponential functions $e^x = \cosh x + \sinh x$, and $e^{-x} = \cosh x - \sinh x$.

The primary argument is the hyperbolic angle x . Only values of x in the range -1.118 to +1.118 are supported. For all input values, a scaling factor of 2^{-n} must be applied in software, where $n = 1$. The scaled value $x \cdot 0.5$ is programmed in ARG1 and the factor $n = 1$ must be programmed in the SCALE parameter.

The secondary argument is not used.

The primary result, RES1, is the hyperbolic sine, $\sinh x$. RES1 must be multiplied by 2 to obtain the correct result.

The secondary result, RES2, is the hyperbolic cosine, $\cosh x$. RES2 must be multiplied by 2 to obtain the correct result.

Hyperbolic arctangent

Table 172. Hyperbolic arctangent parameters

Parameter	Description	Range
ARG1	$x \cdot 2^{-n}$	[-0.403 0.403]
ARG2	Not applicable	-
RES1	$2^{-n} \cdot \operatorname{atanh} x$	[-0.559 0.559]

Table 172. Hyperbolic arctangent parameters (continued)

Parameter	Description	Range
RES2	Not applicable	-
SCALE	n	1

This function calculates the hyperbolic arctangent of the input argument x.

The primary argument is the input value x. Only values of x in the -0.806 to +0.806 range are supported. The value x must be scaled by a factor 2^{-n} , where $n = 1$. The scaled value $x \cdot 0.5$ is programmed in ARG1 and the factor $n = 1$ must be programmed in the SCALE parameter.

The secondary argument is not used.

The primary result is the hyperbolic arctangent, $\operatorname{atanh} x$. RES1 must be multiplied by 2 to obtain the correct value.

The secondary result is not used.

Natural logarithm

Table 173. Natural logarithm parameters

Parameter	Description	Range
ARG1	$x \cdot 2^{-n}$	[0.054 0.875]
ARG2	Not applicable	-
RES1	$2^{-(n+1)} \cdot \ln x$	[-0.279 0.137]
RES2	Not applicable	-
SCALE	n	[1 4]

This function calculates the natural logarithm of the input argument x.

The primary argument is the input value x. Only values of x in the range 0.107 to 9.35 are supported. The value x must be scaled by a factor 2^{-n} , such that $x < 2^{-n} < 1 \cdot 2^{-n}$. The scaled value $x \cdot 2^{-n}$ is programmed in ARG1 and the factor n must be programmed in the SCALE parameter.

[Table 174](#) lists the valid scaling factors, n, and the corresponding ranges of x and ARG1.

Table 174. Natural log scaling factors and corresponding ranges

n	x range	ARG1 range
1	$0.107 \leq x < 1$	$0.0535 \leq \text{ARG1} < 0.5$
2	$1 \leq x < 3$	$0.25 \leq \text{ARG1} < 0.75$
3	$3 \leq x < 7$	$0.375 \leq \text{ARG1} < 0.875$
4	$7 \leq x \leq 9.35$	$0.4375 \leq \text{ARG1} < 0.584$

The secondary argument is not used.

The primary result is the natural logarithm, $\ln x$. RES1 must be multiplied by $2^{(n+1)}$ to obtain the correct value.

The secondary result is not used.

Square root

Table 175. Square root parameters

Parameter	Description	Range
ARG1	$x \cdot 2^{-n}$	[0.027 0.875]
ARG2	Not applicable	-
RES1	$2^{-n} \sqrt{x}$	[0.04 1]
RES2	Not applicable	-
SCALE	n	[0 2]

This function calculates the square root of the input argument x.

The primary argument is the input value x. Only values of x in the range 0.027 to 2.34 are supported. The value x must be scaled by a factor 2^{-n} , such that $x \cdot 2^{-n} < (1 - 2^{-(n-2)})$.

The scaled value $x \cdot 2^{-n}$ is programmed in ARG1 and the factor n must be programmed in the SCALE parameter.

[Table 176](#) lists the valid scaling factors, n, and the corresponding ranges of x and ARG1.

Table 176. Square root scaling factors and corresponding ranges

n	x range	ARG1 range
0	$0.027 \leq x < 0.75$	$0.027 \leq \text{ARG1} < 0.75$
1	$0.75 \leq x < 1.75$	$0.375 \leq \text{ARG1} < 0.875$
2	$1.75 \leq x \leq 2.341$	$0.4375 \leq \text{ARG1} \leq 0.585$

The secondary argument is not used.

The primary result is the square root of x. RES1 must be multiplied by 2^n to obtain the correct value.

The secondary result is not used.

20.3.3 Fixed point representation

The CORDIC operates in fixed point signed integer format. Input and output values can be either q1.31 or q1.15.

In q1.31 format, numbers are represented by one sign bit and 31 fractional bits (binary decimal places). The numeric range is therefore -1 (0x80000000) to $1 - 2^{-31}$ (0x7FFFFFFF).

In q1.15 format, the numeric range is 1 (0x8000) to $1 - 2^{-15}$ (0x7FFF). This format has the advantage that two input arguments can be packed into a single 32-bit write, and two results can be fetched in one 32-bit read.

20.3.4 Scaling factor

Several of the functions listed in [Section 20.3.2](#) specify a scaling factor, SCALE. This allows the function input range to be extended to cover the full range of values supported by the CORDIC, without saturating the input, output, or internal registers. If the scaling factor is required, it must be calculated by software and programmed into the SCALE field of the CORDIC_CSR register. The input arguments must be scaled accordingly before programming the scaled values in the CORDIC_WDATA register. The scaling must also be undone on the results read from the CORDIC_RDATA register.

Note: The scaling factor entails a loss of precision due to truncation of the scaled value.

20.3.5 Precision

The precision of the result is dependent on the number of CORDIC iterations. The algorithm converges at a constant rate of one binary digit per iteration for trigonometric functions (sine, cosine, phase, modulus), see [Figure 101](#).

For hyperbolic functions (hyperbolic sine, hyperbolic cosine, natural logarithm), the convergence rate is less constant due to the peculiarities of the CORDIC algorithm (see [Figure 102](#)). The square root function converges at roughly twice the speed of the hyperbolic functions (see [Figure 103](#)).

Figure 101. CORDIC convergence for trigonometric functions

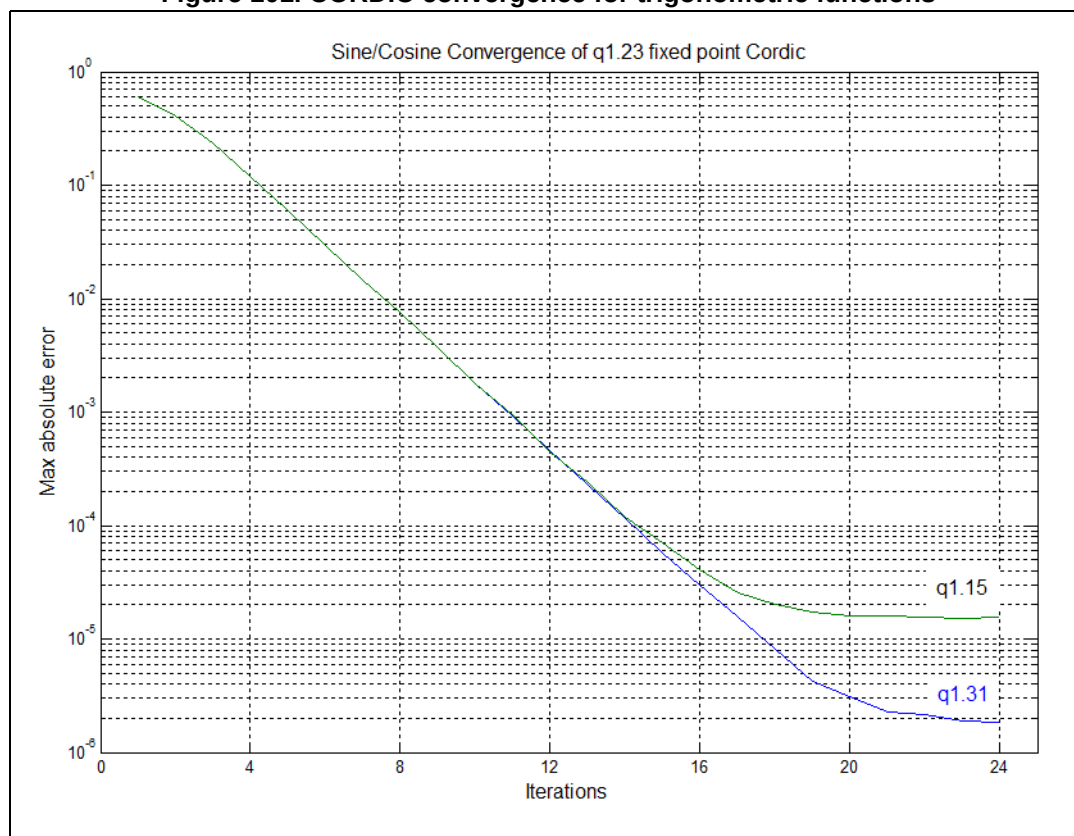


Figure 102. CORDIC convergence for hyperbolic functions

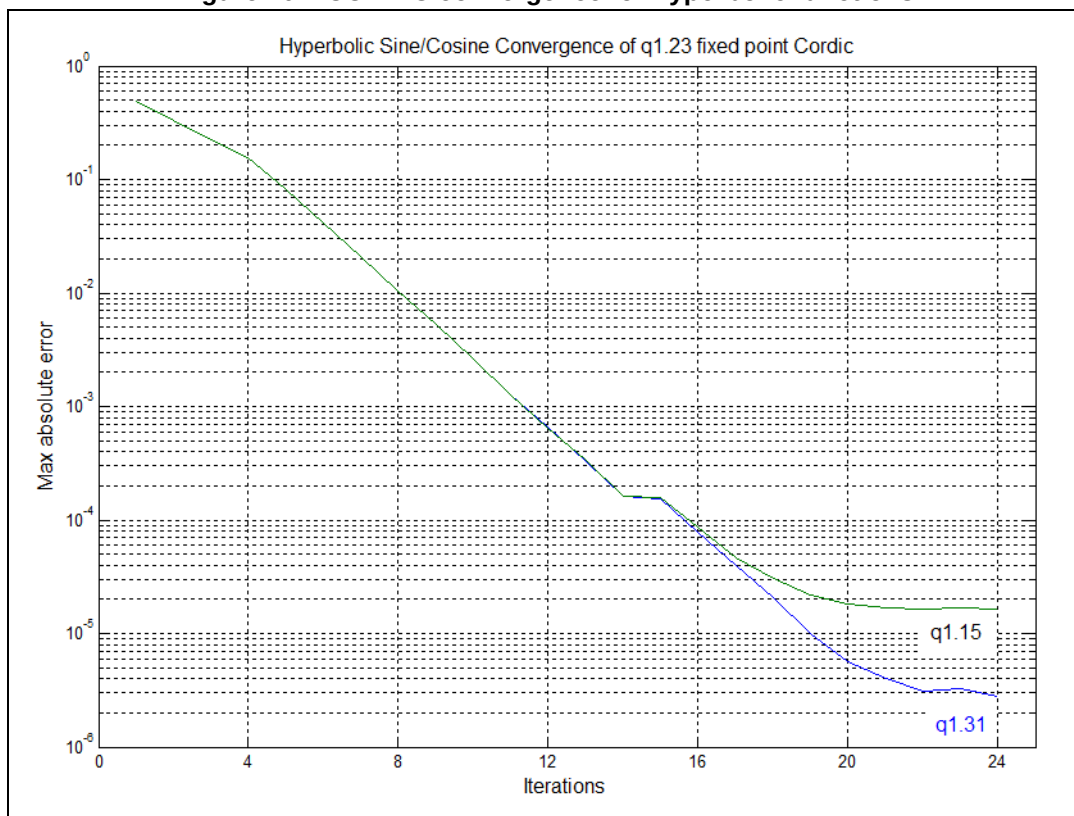
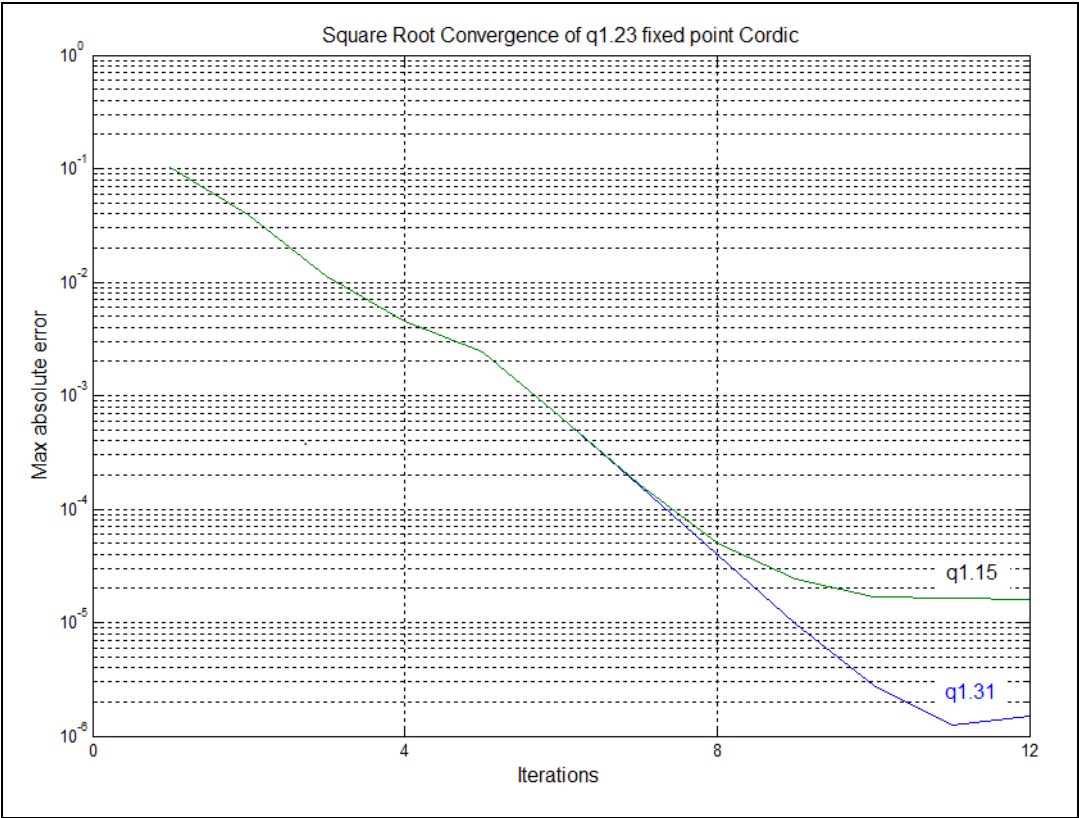


Figure 103. CORDIC convergence for square root



Note: The convergence rate decreases as the quantization error starts to become significant.

The CORDIC can perform four iterations per clock cycle. For each function, the maximum error remaining after every four iterations is shown in [Table 177](#), together with the number of clock cycles required to reach that precision. From this table, the desired number of cycles can be determined and programmed in the PRECISION field of the CORDIC_CR register. The coprocessor stops as soon as the programmed number of iterations is completed, and the result can be read immediately.

Table 177. Precision vs. number of iterations

Function	Number of iterations	Number of cycles	Max residual error ⁽¹⁾	
			q1.31 format	q1.15 format
Sin, Cos, Phase ⁽²⁾ , Mod, Atan ⁽⁴⁾	4	1	2^{-3}	2^{-3}
	8	2	2^{-7}	2^{-7}
	12	3	2^{-11}	2^{-11}
	16	4	2^{-15}	2^{-15}
	20	5	2^{-18}	2^{-16}
	24	6	2^{-19}	2^{-16}

Table 177. Precision vs. number of iterations (continued)

Function	Number of iterations	Number of cycles	Max residual error ⁽¹⁾	
			q1.31 format	q1.15 format
Sinh, Cosh, Atanh, Ln ⁽³⁾	4	1	2^{-2}	2^{-2}
	8	2	2^{-6}	2^{-6}
	12	3	2^{-10}	2^{-10}
	16	4	2^{-13}	2^{-13}
	20	5	2^{-17}	2^{-15}
	24	6	2^{-18}	2^{-15}
Sqrt ⁽⁴⁾	4	1	2^{-7}	2^{-7}
	8	2	2^{-14}	2^{-14}
	12	3	2^{-19}	2^{-15}

1. Max residual error is the maximum error remaining after the given number of iterations, compared to the identical calculation performed in double precision floating point. An additional rounding error may be incurred, of up to 2^{-16} for q15 format or 2^{-20} for q31 format.
2. For modulus > 0.5. The achievable precision reduces proportionally to the magnitude of the modulus, as quantization error becomes significant.
3. SCALE = 1. If a higher scaling factor is used, the achievable precision is reduced proportionally.
4. SCALE = 0. If a higher scaling factor is used, the achievable precision is reduced proportionally.

20.3.6 Zero-overhead mode

The fastest way to use the coprocessor is to preprogram the CORDIC_CSR register with the function to be performed (FUNC), the desired number of clock cycles (PRECISION), the size of the input and output values (ARGSIZE, RESSIZE), the number of input arguments (NARGS) and/or results (NRES), and the scaling factor (SCALE), if applicable.

The calculation is triggered by writing the input arguments to the CORDIC_WDATA register. As soon as the correct number of input arguments has been written (and any ongoing calculation has finished), a new calculation is launched using these input arguments and the current CORDIC_CSR settings. There is no need to reprogram the CORDIC_CSR register if there is no change.

If a dual 32-bit input argument is needed (ARGSIZE = 0, NARGS = 1), the primary input argument (ARG1) must be written first, followed by the secondary argument (ARG2). If the secondary argument remains unchanged for a series of calculations, the second write can be avoided, by reprogramming the number of arguments to one (NARGS = 0), once the first calculation has started. The secondary argument retains its programmed value as long as the function is not changed.

Note: ARG2 is set to +1 (0x7FFFFFFF) after a reset.

If two 16-bit arguments are used (ARGSIZE = 1) they must be packed into a 32-bit word, with ARG1 in the least significant half-word and ARG2 in the most significant half-word. The packed 32-bit word is then written to the CORDIC_WDATA register. Only one write is needed in this case (NARGS = 0).

For functions taking only one input argument, ARG1, it is recommended to set NARGS = 0. If NARGS = 1, a second write to CORDIC_WDATA must be performed to trigger the calculation. The ARG2 data in this case is not used.

Once the calculation starts, any attempt to read the CORDIC_RDATA register inserts bus wait-states until the calculation is completed, before returning the result. It is then possible for the software to write the input and immediately read the result without polling to see if it is valid. Alternatively, the processor can wait for the appropriate number of clock cycles before reading the result. This time can be used to program the CORDIC_CSR register for the next calculation, and prepare the next input data, if needed. The CORDIC_CSR register can be reprogrammed while a calculation is in progress, without affecting the result of the ongoing calculation. In the same way, the CORDIC_WDATA register can be updated with the next argument(s) once the previous ones have been taken into account. The next arguments and settings remain pending until the previous calculation has completed.

When a calculation is finished, the result(s) can be read from the CORDIC_RDATA register. If two 32-bit results are expected (NRES = 1, RESSIZE = 0), the primary result (RES1) is read out first, followed by the secondary result (RES2). If only one 32-bit result is expected (NRES = 0, RESSIZE = 0), then RES1 is output on the first read.

If 16-bit results are expected (RESSIZE = 1), a single read to CORDIC_RDATA fetches both results packed into a 32-bit word. RES1 is in the lower half-word, and RES2 in the upper half-word. In this case, it is recommended to program NRES = 0. If NRES = 1, a second read of CORDIC_RDATA must be performed to free up the CORDIC for the next operation. The data from this second read must be discarded.

The next calculation starts when the expected number of results has been read, provided the expected number of arguments has been written. This means that at any time, there can be a calculation in progress, or waiting for the results to be read, and an operation pending. Any further access to CORDIC_WDATA while an operation is pending cancels it and overwrites the data.

The following sequence summarizes the use of the CORDIC_IP in zero-overhead mode:

1. Program the CORDIC_CSR register with the appropriate settings
2. Program the argument(s) for the first calculation in the CORDIC_WDATA register. This launches the first calculation.
3. If needed, update the CORDIC_CSR register settings for the next calculation.
4. Program the argument(s) for the next calculation in the CORDIC_WDATA register.
5. Read the result(s) from the CORDIC_RDATA register. This triggers the next calculation.
6. Go to step 3.

20.3.7 Polling mode

When a new result is available in the CORDIC_RDATA register, the RRDY flag is set in the CORDIC_CSR register. The flag can be polled by reading the register. It is reset by reading the CORDIC_RDATA register (once or twice, depending on the NRES field of the CORDIC_CSR register).

Polling the RRDY flag takes slightly longer than reading the CORDIC_RDATA register directly, since the result is not read as soon as it is available. The processor and bus interface are not stalled while reading the CORDIC_CSR register, so this mode may be of interest if stalling the processor is not acceptable (for example, if low latency interrupts must be serviced).

20.3.8 Interrupt mode

By setting the interrupt enable (IE) bit in the CORDIC_CSR register, an interrupt is generated whenever the RRDY flag is set. The interrupt is cleared when the flag is reset.

This mode allows the result of the calculation to be read under interrupt service routine, and hence given a priority relative to other tasks. However, it is slower than directly reading the result, or polling the flag, due to the interrupt handling delays.

20.3.9 DMA mode

If the DMA write enable (DMAWEN) bit is set in the CORDIC_CSR register, and no operation is pending, a DMA write channel request is generated. The DMA controller can transfer a primary input argument (ARG1) from memory into the CORDIC_WDATA register. Writing into the register deasserts the DMA request. If NARGS = 1 in the CORDIC_CSR register, a second DMA write channel request is generated to transfer the secondary input argument (ARG2) into the CORDIC_WDATA register. When all input arguments have been written, and any ongoing calculation has been completed (by reading the results), a new calculation is started and another DMA write channel request is generated.

If the DMA read enable (DMAREN) bit is set in the CORDIC_CSR register, the RRDY flag going active generates a DMA read channel request. The DMA controller can then transfer the primary result (RES1) from the CORDIC_RDATA register to memory. Reading the register deasserts the DMA request. If NRES = 1 in the CORDIC_CSR register, a second DMA request is generated to read out the secondary result (RES2). When all results have been read, the RRDY flag is deasserted.

The DMA read and write channels can be enabled separately. If both channels are enabled, the CORDIC can autonomously perform repeated calculations on a buffer of data without processor intervention. This allows the processor to perform other tasks. The DMA controller is operating in memory-to-peripheral mode for the write channel, and peripheral-to-memory mode for the read channel. The sequence is started by the processor setting the DMAWEN flag, the DMA read and write requests are generated as fast as the CORDIC can process the data.

In some cases, the input data may be stored in memory, and the output is transferred at regular intervals to another peripheral, such as a digital-to-analog converter. In this case, the destination peripheral generates a DMA request each time it needs a new data. The DMA controller can directly fetch the next sample from the CORDIC_RDATA register (in this case the DMA controller is operating in memory-to-peripheral mode, even though the source is a peripheral register). The act of reading the result allows the CORDIC to start a new calculation, which in turn generates a DMA write channel request, and the DMA controller transfers the next input value to the CORDIC_WDATA register. The DMA write channel is enabled (DMAWEN = 1), but the read channel must not be enabled.

In a similar way, data coming from another peripheral, such as an ADC, can be transferred directly to the CORDIC_WDATA register (in peripheral-to-memory mode). The DMA write channel must not be enabled. The CORDIC processes the input data and generates a DMA read request when complete, if DMAREN = 1. The DMA controller then transfers the result from CORDIC_RDATA register to memory (peripheral-to-memory mode).

Note: No DMA request is generated to program the CORDIC_CSR register. DMA mode is therefore useful only when repeatedly performing the same function with the same settings. The scale factor cannot be changed during a series of DMA transfers.

Note: Each DMA request must be acknowledged, as a result of the DMA performing an access to the CORDIC_WDATA or CORDIC_RDATA register. If an extraneous access to the relevant register occurs before this, the acknowledge is asserted prematurely, and may block the DMA channel. Therefore, when the DMA read channel is enabled, CPU access to the CORDIC_RDATA register must be avoided. Similarly, the processor must avoid accessing the CORDIC_WDATA register when the DMA write channel is enabled.

20.4 CORDIC registers

The CORDIC registers can be accessed only in 32-bit word format

20.4.1 CORDIC control/status register (CORDIC_CSR)

Address offset: 0x00

Reset value: 0x0000 0050

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
RRDY	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	ARG SIZE	RES SIZE	N ARGS	NRES	DMA WEN	DMA REN	IEN
r									rw	rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	Res.	Res.	Res.	SCALE[2:0]			PRECISION[3:0]				FUNC[3:0]			
					rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bit 31 **RRDY**: Result ready flag

0: No new data in output register

1: CORDIC_RDATA register contains new data.

This bit is set by hardware when a CORDIC operation completes. It is reset by hardware when the CORDIC_RDATA register is read (NRES+1) times.

When this bit is set, if the IEN bit is also set, the CORDIC interrupt is asserted. If the DMAREN bit is set, a DMA read channel request is generated. While this bit is set, no new calculation is started.

Bits 30:23 Reserved, must be kept at reset value.

Bit 22 **ARGSIZE**: Width of input data

0: 32-bit

1: 16-bit

ARGSIZE selects the number of bits used to represent input data.

If 32-bit data is selected, the CORDIC_WDATA register expects arguments in q1.31 format.

If 16-bit data is selected, the CORDIC_WDATA register expects arguments in q1.15 format.

The primary argument (ARG1) is written to the least significant half-word, and the secondary argument (ARG2) to the most significant half-word.

Bit 21 **RESSIZE**: Width of output data

0: 32-bit

1: 16-bit

RESSIZE selects the number of bits used to represent output data.

If 32-bit data is selected, the CORDIC_RDATA register contains results in q1.31 format.

If 16-bit data is selected, the least significant half-word of CORDIC_RDATA contains the primary result (RES1) in q1.15 format, and the most significant half-word contains the secondary result (RES2), also in q1.15 format.

- Bit 20 **NARGS**: Number of arguments expected by the CORDIC_WDATA register
 0: Only one 32-bit write (or two 16-bit values if ARGSIZE = 1) is needed for the next calculation.
 1: Two 32-bit values must be written to the CORDIC_WDATA register to trigger the next calculation.
 Reads return the current state of the bit.
- Bit 19 **NRES**: Number of results in the CORDIC_RDATA register
 0: Only one 32-bit value (or two 16-bit values if RESSIZE = 1) is transferred to the CORDIC_RDATA register on completion of the next calculation. One read from CORDIC_RDATA resets the RRDY flag.
 1: Two 32-bit values are transferred to the CORDIC_RDATA register on completion of the next calculation. Two reads from CORDIC_RDATA are necessary to reset the RRDY flag.
 Reads return the current state of the bit.
- Bit 18 **DMAWEN**: Enable DMA write channel
 0: Disabled. No DMA write requests are generated.
 1: Enabled. Requests are generated on the DMA write channel whenever no operation is pending
 This bit is set and cleared by software. A read returns the current state of the bit.
- Bit 17 **DMAREN**: Enable DMA read channel
 0: Disabled. No DMA read requests are generated.
 1: Enabled. Requests are generated on the DMA read channel whenever the RRDY flag is set.
 This bit is set and cleared by software. A read returns the current state of the bit.
- Bit 16 **IEN**: Enable interrupt.
 0: Disabled. No interrupt requests are generated.
 1: Enabled. An interrupt request is generated whenever the RRDY flag is set.
 This bit is set and cleared by software. A read returns the current state of the bit.
- Bits 15:11 Reserved, must be kept at reset value.
- Bits 10:8 **SCALE[2:0]**: Scaling factor
 The value of this field indicates the scaling factor applied to the arguments and/or results. A value n implies that the arguments have been multiplied by a factor 2^{-n} , and/or the results need to be multiplied by 2^n . Refer to [Section 20.3.2](#) for the applicability of the scaling factor for each function and the appropriate range.
- Bits 7:4 **PRECISION[3:0]**: Precision required (number of iterations)
 0: reserved
 1 to 15: (Number of iterations)/4
 To determine the number of iterations needed for a given accuracy refer to [Table 177](#).
 Note that for most functions, the recommended range for this field is 3 to 6.

Bits 3:0 **FUNC[3:0]**: Function

- 0: Cosine
- 1: Sine
- 2: Phase
- 3: Modulus
- 4: Arctangent
- 5: Hyperbolic cosine
- 6: Hyperbolic sine
- 7: Arctanh
- 8: Natural logarithm
- 9: Square root
- 10 to 15: Reserved

20.4.2 CORDIC argument register (CORDIC_WDATA)

Address offset: 0x04

Reset value: 0xFFFF XXXX

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
ARG[31:16]															
w	w	w	w	w	w	w	w	w	w	w	w	w	w	w	w
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
ARG[15:0]															
w	w	w	w	w	w	w	w	w	w	w	w	w	w	w	w

Bits 31:0 **ARG[31:0]**: Function input arguments

This register is programmed with the input arguments for the function selected in the CORDIC_CSR register FUNC field.

If 32-bit format is selected (CORDIC_CSR.ARGSIZE = 0) and two input arguments are required (CORDIC_CSR.NARGS = 1), two successive writes are required to this register. The first writes the primary argument (ARG1), the second writes the secondary argument (ARG2).

If 32-bit format is selected and only one input argument is required (NARGS = 0), only one write is required to this register, containing the primary argument (ARG1).

If 16-bit format is selected (CORDIC_CSR.ARGSIZE = 1), one write to this register contains both arguments. The primary argument (ARG1) is in the lower half, ARG[15:0], and the secondary argument (ARG2) is in the upper half, ARG[31:16]. In this case, NARGS must be set to 0.

Refer to [Section 20.3.2](#) for the arguments required by each function, and their permitted range.

When the required number of arguments has been written, the CORDIC evaluates the function designated by CORDIC_CSR.FUNC using the supplied input arguments, provided any previous calculation has completed. If a calculation is ongoing, the ARG1 and ARG 2 values are held pending until the calculation is completed and the results read. During this time, a write to the register cancels the pending operation and overwrite the argument data.

20.4.3 CORDIC result register (CORDIC_RDATA)

Address offset: 0x08

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
RES[31:16]															
r	r	r	r	r	r	r	r	r	r	r	r	r	r	r	r
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
RES[15:0]															
r	r	r	r	r	r	r	r	r	r	r	r	r	r	r	r

Bits 31:0 **RES[31:0]**: Function result

If 32-bit format is selected (CORDIC_CSR.RESSIZE = 0) and two output values are expected (CORDIC_CSR.NRES = 1), this register must be read twice when the RRDY flag is set. The first read fetches the primary result (RES1). The second read fetches the secondary result (RES2) and resets RRDY.

If 32-bit format is selected and only one output value is expected (NRES = 0), only one read of this register is required to fetch the primary result (RES1) and reset the RRDY flag.

If 16-bit format is selected (CORDIC_CSR.RESSIZE = 1), this register contains the primary result (RES1) in the lower half, RES[15:0], and the secondary result (RES2) in the upper half, RES[31:16]. In this case, NRES must be set to 0, and only one read performed.

A read from this register resets the RRDY flag in the CORDIC_CSR register.

20.4.4 CORDIC register map

Table 178. CORDIC register map and reset value

Offset	Register name	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0x00	CORDIC_CSR	RRDY	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	ARGSIZE	RESSIZE	NARGS	NRES	DMAWEN	DMAREN	IEN	Res.	Res.	Res.	Res.	SCALE [2:0]			PRECISION [3:0]			FUNC [3:0]					
	Reset value	0									0	0	0	0	0	0	0					0	0	0	0	0	1	0	1	0	0	0	0
0x04	CORDIC_WDATA	ARG[31:0]																															
	Reset value	x	x	x	x	x	x	x	x	x	x	x	x	x	x	x	x	x	x	x	x	x	x	x	x	x	x	x	x	x	x	x	
0x08	CORDIC_RDATA	RES[31:0]																															
	Reset value	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Refer to [Section 2.3: Memory organization](#) for the register boundary addresses.

21 Filter math accelerator (FMAC)

This section applies only to STM32H56x/573xx devices.

21.1 FMAC introduction

The filter math accelerator unit performs arithmetic operations on vectors. It comprises a multiplier/accumulator (MAC) unit, together with address generation logic which allows it to index vector elements held in local memory.

The unit includes support for circular buffers on input and output, which allows digital filters to be implemented. Both finite and infinite impulse response filters can be realized.

The unit allows frequent or lengthy filtering operations to be offloaded from the CPU, freeing up the processor for other tasks. In many cases it can accelerate such calculations compared to a software implementation, resulting in a speed-up of time critical tasks.

21.2 FMAC main features

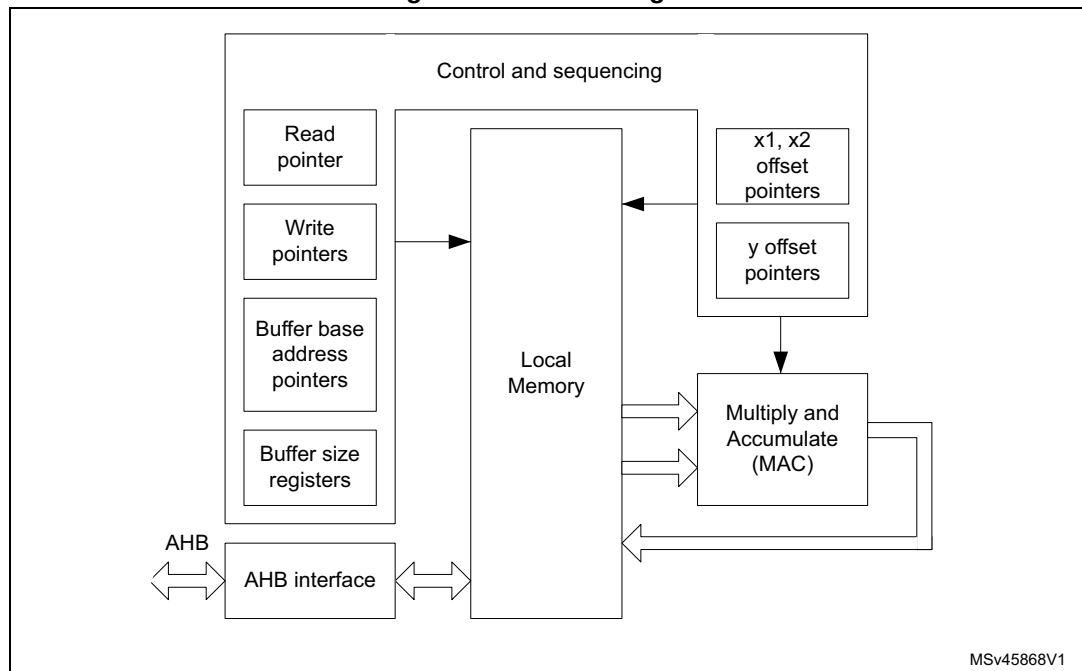
- 16 x 16-bit multiplier
- 24 + 2-bit accumulator with addition and subtraction
- 16-bit input and output data
- 256 x 16-bit local memory
- Up to three areas can be defined in memory for data buffers (two input, one output), defined by programmable base address pointers and associated size registers
- Input and output buffers can be circular
- Filter functions: FIR, IIR (direct form 1)
- Vector functions: Dot product, convolution, correlation
- AHB slave interface
- DMA read and write data channels

21.3 FMAC functional description

21.3.1 General description

The FMAC is shown in [Figure 104](#).

Figure 104. Block diagram



MSv45868V1

The unit is built around a fixed point multiplier and accumulator (MAC). The MAC can take two 16-bit input signed values from memory, multiply them together and add them to the contents of the accumulator. The address of the input values in memory is determined using a set of pointers. These pointers can be loaded, incremented, decremented or reset by the internal hardware. The pointer and MAC operations are controlled by a built-in sequencer in order to execute the requested operation.

To calculate a dot product, the two input vectors are loaded into the local memory by the processor or DMA controller, and the requested operation is selected and started. Each pair of input vector elements is fetched from memory, multiplied together and accumulated. When all the vector elements have been processed, the contents of the accumulator are stored in the local memory, from where they can be read out by the processor or DMA.

The finite impulse response (FIR) filter operation (also known as convolution) consists in repeatedly calculating the dot product of the coefficient vector and a vector of input samples, the latter being shifted by one sample delay, with the least recent sample being discarded and a new sample added, at each repetition.

The infinite impulse response (IIR) filter operation is the convolution of the feedback coefficients with the previous output samples, added to the result of the FIR convolution.

A more detailed description of the filter operations is given in [Section 21.3.6: Filter functions](#).

21.3.2 Local memory and buffers

The unit contains a 256 x 16-bit read/write memory which is used for local storage:

- Input values (the elements of the input vectors) are stored in two buffers, X1 and X2.
- Output values (the results of the operations) are stored in another buffer, Y.
- The locations and sizes of the buffers are designated as follows:
 - x1_base: the base address of the X1 buffer
 - x2_base: the base address of the X2 buffer
 - y_base: the base address of the Y buffer
 - x1_buf_size: the number of 16-bit addresses allocated to the X1 buffer
 - x2_buf_size: the number of 16-bit addresses allocated to the X2 buffer
 - y_buf_size: the number of 16-bit addresses allocated to the Y buffer.

These parameters are programmed in the corresponding registers when configuring the unit.

The CPU (or DMA controller) can initialize the contents of each buffer using the Initialization functions ([Section 21.3.5: Initialization functions](#)) and writing to the write data register. The data is transferred to the location within the target buffer indicated by a write pointer. After each new write, the write pointer is incremented. When the write pointer reaches the end of the allocated buffer space, it wraps back to the base address. This feature is used to load the elements of a vector prior to an operation, or to initialize a filter and load filter coefficients.

Buffer configuration

The buffer sizes and base address offsets must be configured in the X1, X2 and Y buffer configuration registers. For each function, the required buffer size is specified in the function description in [Section 21.3.6: Filter functions](#). The base addresses can be chosen anywhere in internal memory, provided that all buffers fit within the internal memory address range (0x00 to 0xFF), that is, base address + buffer size must be less than 256.

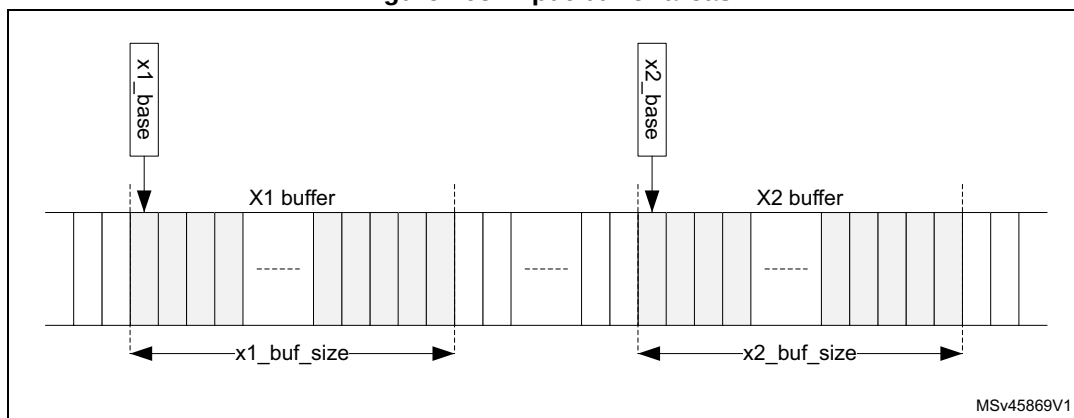
There is no constraint on the size and location of the buffers (they can overlap or even coincide exactly). For filter functions it is recommended not to overlap buffers as this can lead to erroneous behavior.

When circular buffer operation is required, an optional “headroom”, *d*, can be added to the buffer size. Furthermore, a watermark level can be set, to regulate the CPU or DMA activity. The value of *d* and the watermark level must be chosen according to the application performance requirements. For maximum throughput, the input buffer must never go empty, so *d* must be somewhat greater than the watermark level, allowing for any interrupt or DMA latency. On the other hand, if the input data can not be provided as fast as the unit can process them, the buffer can be allowed to empty waiting for the next data to be written, so *d* can be equal to the watermark level (to ensure that no overflow occurs on the input).

21.3.3 Input buffers

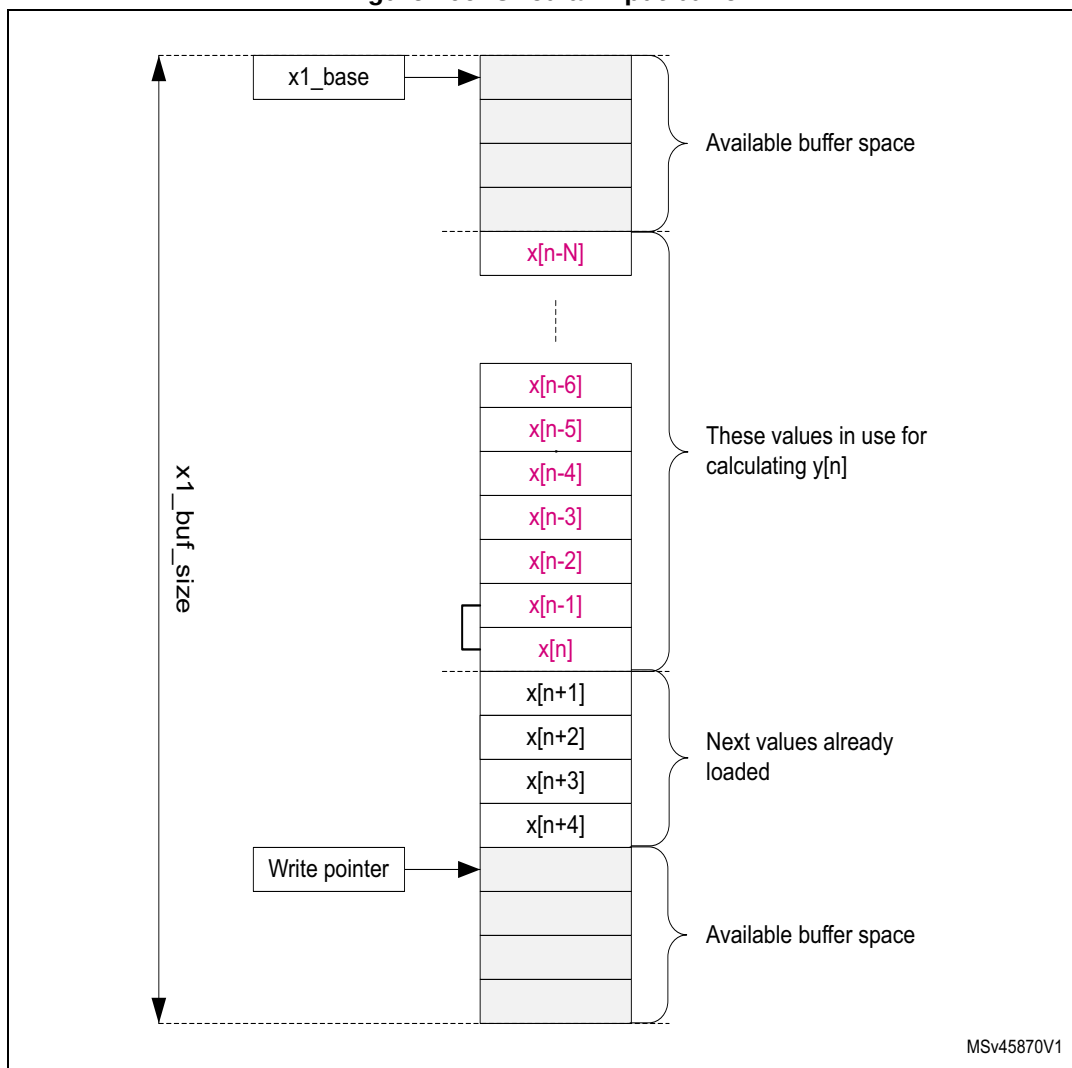
The X1 and X2 buffers are used to store data for input to the MAC. Each multiplication takes a value from the X1 buffer and a value from the X2 buffer and multiplies them together. A pointer in the control unit generates the read address offset (relative to the buffer base address) for each value. The pointers are managed by hardware according to the current function.

Figure 105. Input buffer areas



The X1 buffer can be used as a circular buffer, in which case new data are continually transferred into the input buffer whenever space is available. Pre-loading this buffer is optional for digital filters, since if no input samples have been written in the buffer when the operation is started, it is flagged as empty, which triggers the CPU or DMA to load new samples until there are enough to begin operation. Pre-loading is nevertheless useful in the case of a vector operation, that is, the input data is already available in system memory and circular operation is not required.

Figure 106. Circular input buffer



The X2 buffer can only be used in vector mode (that is not circular), and needs to be pre-loaded, except if the contents of the buffer do not change from one operation to the next. For filter functions, the X2 buffer is used to store the filter coefficients.

When operating as a circular buffer, the space allocated to the buffer (`x1_buf_size`) must generally be bigger than the number of elements in use for the current calculation, so that there are always new values available in the buffer. [Figure 106](#) illustrates the layout of the buffer for a filter operation. While calculating an output sample $y[n]$, the unit uses a set of $N+1$ input samples, $x[n-N]$ to $x[n]$. When this is finished, the unit starts the calculation of $y[n+1]$, using the set of input samples $x[n-N+1]$ to $x[n+1]$. The least-recent input sample, $x[n-N]$, drops out of the input set, and a new sample, $x[n+1]$, is added to it.

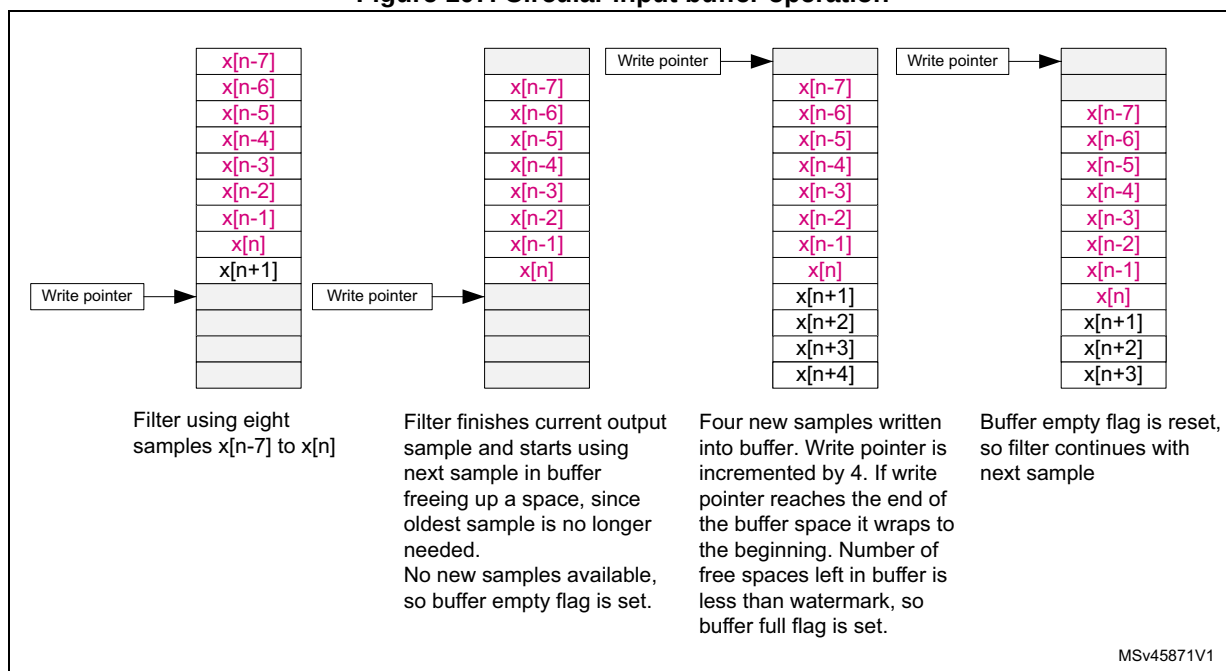
The processor, or DMA controller, must ensure that the new sample $x[n+1]$ is available in the buffer space when required. If not, the buffer is flagged as empty, which stalls the execution of the unit until a new sample is added. No underflow condition is signaled on the X1 buffer.

Note: *If the flow of samples is controlled by a timer or other peripheral such as an ADC, the buffer regularly goes empty, since the filter processes each new sample faster than the source can provide it. This is an essential feature of filter operation.*

If the number of free spaces in the buffer is less than the watermark threshold programmed in the FULL_WM bitfield of the FMAC_X1BUFCFG register, the buffer is flagged as full. As long as the full flag is not set, interrupts are generated, if enabled, to request more data for the buffer. The watermark allows several data to be transferred under one interrupt, without danger of overflow. Nevertheless, if an overflow does occur, the OVFL error flag is set and the write data is ignored. The write pointer is not incremented in the event of an overflow.

The operation of the X1 buffer during a filtering operation is illustrated in [Figure 107](#). This example shows an 8-tap FIR filter with a watermark set to four.

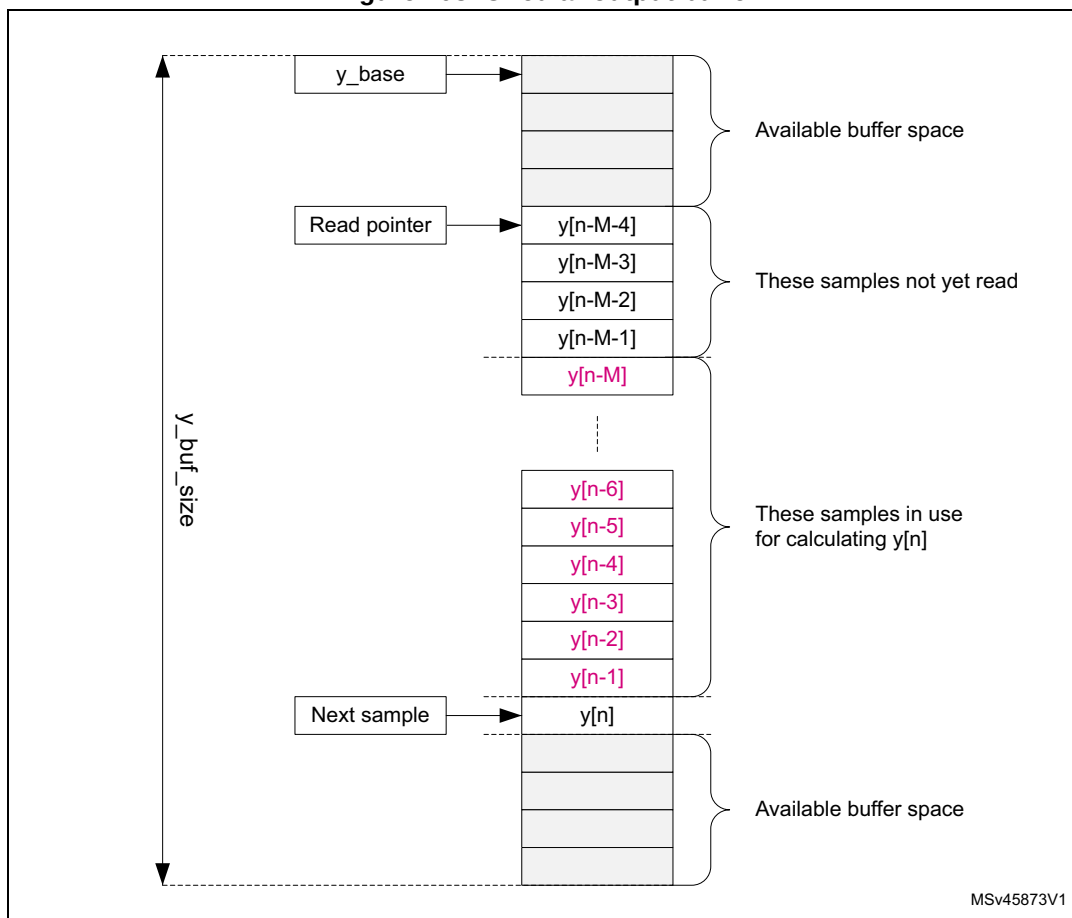
Figure 107. Circular input buffer operation



21.3.4 Output buffer

The Y (output) buffer is used to store the output of an accumulation. Each new output value is stored in the buffer until it is read by the processor or DMA controller. Each time a read access is made to the read data register, the read data is fetched from the address indicated by the read pointer. This pointer is incremented after each read, and wraps back to the base address when it reaches the end of the allocated Y buffer space.

Figure 108. Circular output buffer



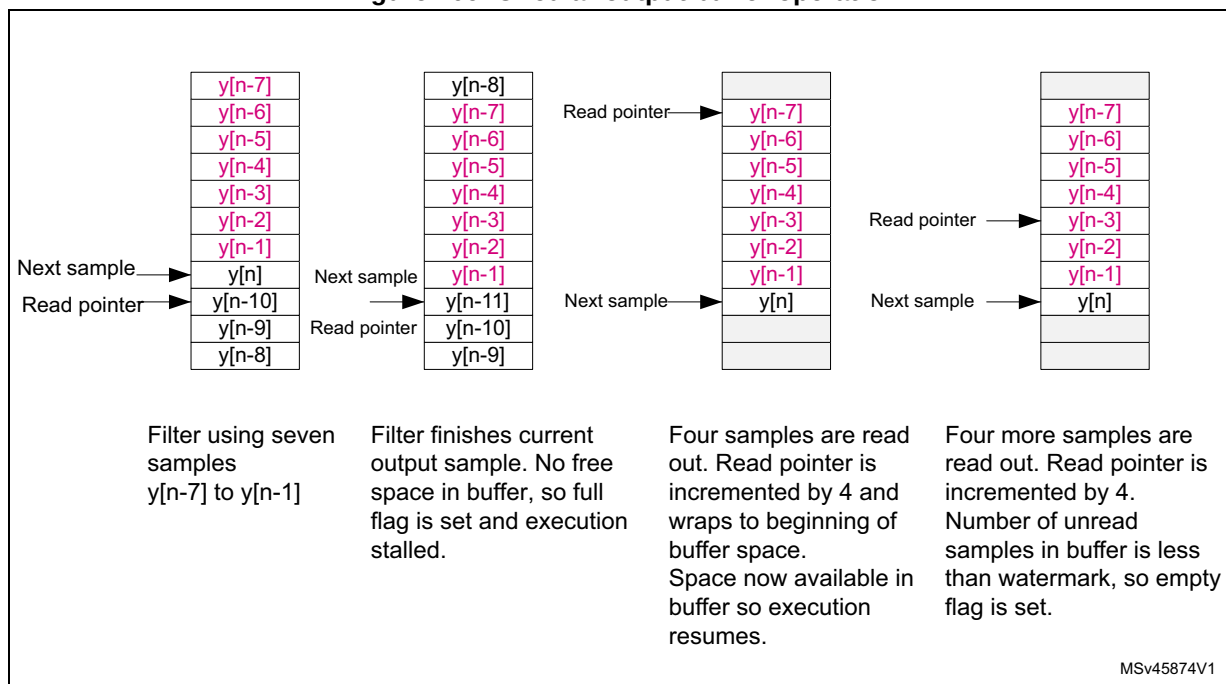
The Y buffer can also operate as a circular buffer. If the address for the next output value is the same as that indicated by the read pointer (an unread sample), then the buffer is flagged as full and execution stalled until the sample is read.

In the case of IIR filters, the Y buffer is used to store the set of M previous output samples, $y[n-M]$ to $y[n-1]$, used for calculating the next output sample $y[n]$. Each time a new sample is added to the set, the least recent sample $y[n-M]$ drops out.

If the number of unread data in the buffer is less than the watermark threshold programmed in the `EMPTY_WM` bitfield of the `FMAC_YBUFCFG` register, the buffer is flagged as empty. As long as the empty flag is not set, interrupts or DMA requests are generated, if enabled, to request reads from the buffer. The watermark allows several data to be transferred under one interrupt, without danger of underflow. Nevertheless, if an underflow does occur, the `UNFL` error flag is set. In this case, the read pointer is not incremented and the read operation returns the content of the memory at the read pointer address.

The operation of the Y buffer in circular mode is illustrated in [Figure 109](#). This example shows a 7-tap IIR filter with a watermark set to four.

Figure 109. Circular output buffer operation



21.3.5 Initialization functions

The following functions initialize the FMAC unit. They are triggered by writing the appropriate value in the FUNC bitfield of the FMAC_PARAM register, with the START bit set. The P and Q bitfields must also contain the appropriate parameter values for each function as detailed below. The R bitfield is not used. When the function completes, the START bit is automatically reset by hardware.

During initialization, it is recommended that the DMA requests and interrupts be disabled. The transfer of data into the FMAC memory can be done by software or by memory-to-memory DMA transfers, since no flow control is required.

Load X1 buffer

This function pre-loads the X1 buffer with N values, starting from the address in X1_BASE. Successive writes to the FMAC_WDATA register load the write data into the X1 buffer and increment the write address. The write pointer points to the address X1_BASE + N when the function completes.

The function can be used to pre-load the buffer with the elements of a vector, or to initialize the input storage elements of a filter.

Parameters

- The parameter P contains the number of values, N, to be loaded into the X1 buffer.
- The parameters Q and R are not used.

The function completes when N writes have been performed to the FMAC_WDATA register.

Load X2 buffer

This function pre-loads the X2 buffer with N + M values, starting from the address in X2_BASE. Successive writes to the FMAC_WDATA register load the write data into the X2 buffer and increment the write address.

The function can be used to pre-load the buffer with the elements of a vector, or the coefficients of a filter. In the case of an IIR, the N feed-forward and M feed-back coefficients are concatenated and loaded together into the X2 buffer. The total number of coefficients is equal to N + M. For an FIR, there are no feedback coefficients, so M = 0.

Parameters

- The parameter P contains the number of values, N, to be loaded into the X2 buffer starting from address X2_BASE.
- The parameter Q contains the number of values, M, to be loaded into the X2 buffer starting from address X2_BASE + N.
- The parameter R is not used.

The function completes when N + M writes have been performed to the FMAC_WDATA register.

Load Y buffer

This function pre-loads the Y buffer with N values, starting from the address in Y_BASE. Successive writes to the FMAC_WDATA register load the write data into the Y buffer and increment the write address. The read pointer points to the address Y_BASE + N when the function completes.

The function can be used to pre-load the feedback storage elements of an IIR filter.

Parameters

- The parameter P contains the number of values to be loaded into the Y buffer.
- The parameters Q and R are not used.

The function completes when N writes have been performed to the FMAC_WDATA register.

21.3.6 Filter functions

The following filter functions are supported by the FMAC unit. These functions are triggered by writing the corresponding value in the FUNC bitfield of the FMAC_PARAM register with the START bit set. The P, Q and R bitfields must also contain the appropriate parameter values for each function as detailed below. The filter functions continue to run until the START bit is reset by software.

Convolution (FIR filter)

$$\underline{Y} = \underline{B} * \underline{X}$$

$$y_n = 2^R \cdot \sum_{k=0}^N b_k x_{n-k}$$

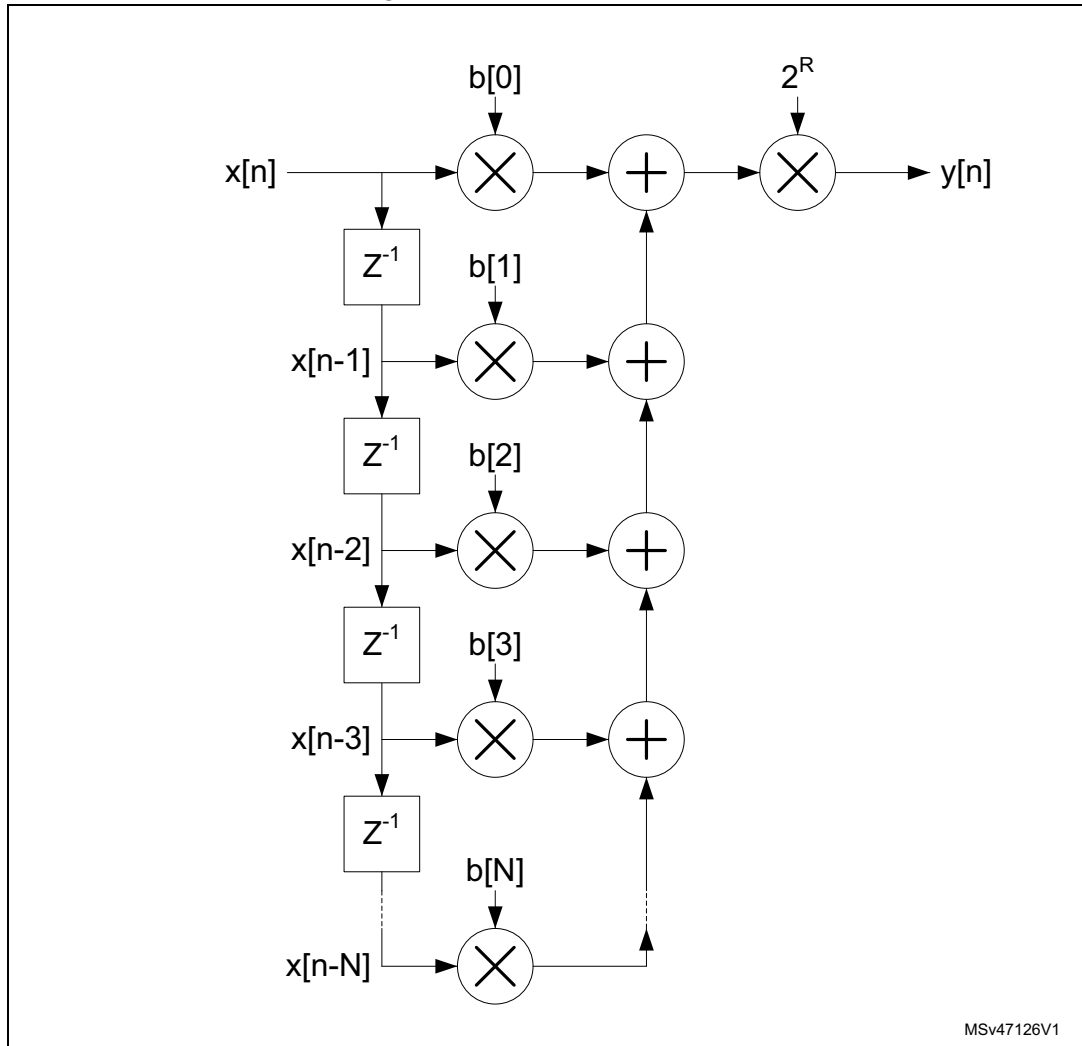
This function performs a convolution of a vector $\underline{B}$ of length N+1 and a vector $\underline{X}$ of indefinite length. The elements of $\underline{Y}$ for incrementing values of n are calculated as the dot product,

$y_n = \underline{\mathbf{B}} \cdot \underline{\mathbf{X}}_n$, where $\underline{\mathbf{X}}_n = [x_{n-N}, \dots, x_n]$ is composed of the $N+1$ elements of $\underline{\mathbf{X}}$ at indexes $n - N$ to n .

This function corresponds to a finite impulse response (FIR) filter, where vector $\underline{\mathbf{B}}$ contains the filter coefficients and vector $\underline{\mathbf{X}}$ the sampled data.

The structure of the filter (direct form) is shown in [Figure 110](#).

Figure 110. FIR filter structure



Note that the cross correlation vector can be calculated by reversing the order of the coefficient vector $\underline{\mathbf{B}}$.

Input:

- X1 buffer contains the elements of vector $\underline{\mathbf{X}}$. It is a circular buffer of length $N + 1 + d$.
- X2 buffer contains the elements of vector $\underline{\mathbf{B}}$. It is a fixed buffer of length $N + 1$.

Output:

- Y buffer contains the output values, y_n . It is a circular buffer of length d .

Parameters:

- The parameter P contains the length, N+1, of the coefficient vector **B** in the range [2:127].
- The parameter R contains the gain to be applied to the accumulator output. The value output to the Y buffer is multiplied by 2^R , where R is in the range [0:7]
- The parameter Q is not used.

The function completes when the START bit in the FMAC_PARAM register is reset by software.

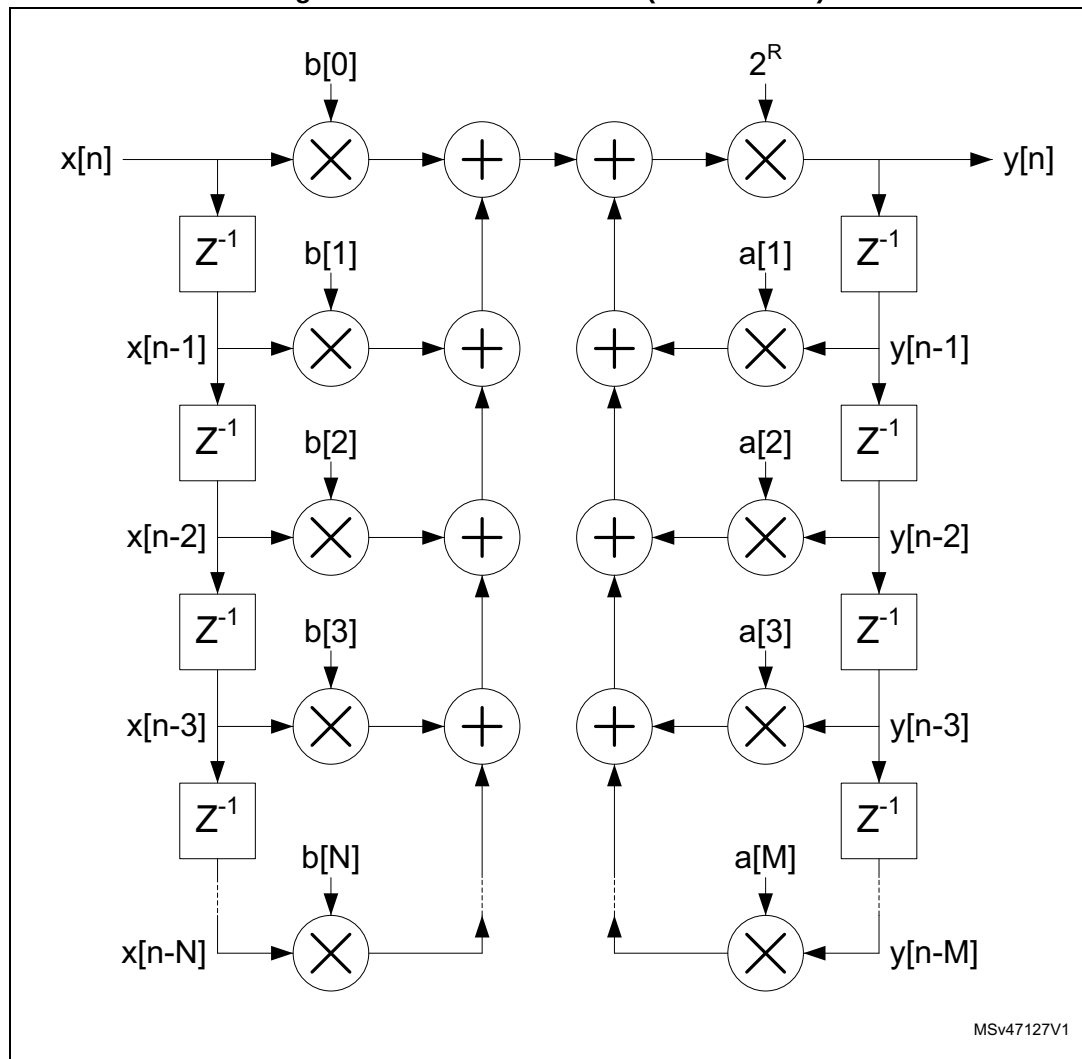
IIR filter

$$\mathbf{Y} = \mathbf{B} * \mathbf{X} + \mathbf{A} * \mathbf{Y}'$$

$$y_n = 2^R \cdot \left(\sum_{k=0}^N b_k x_{n-k} + \sum_{k=1}^M a_k y_{n-k} \right)$$

This function implements an infinite impulse response (IIR) filter. The filter output vector **Y** is the convolution of a coefficient vector **B** of length N+1 and a vector **X** of indefinite length, plus the convolution of the delayed output vector **Y'** with a second coefficient vector **A**, of length M. The elements of **Y** for incrementing values of n are calculated as $y_n = \mathbf{B} \cdot \mathbf{X}_n + \mathbf{A} \cdot \mathbf{Y}_{n-1}$, where $\mathbf{X}_n = [x_{n-N}, \dots, x_n]$ comprises the N+1 elements of **X** at indexes n - N to n, while $\mathbf{Y}_{n-1} = [y_{n-M}, \dots, y_{n-1}]$ comprises the M elements of **Y** at indexes n - M to n - 1. The structure of the filter (direct form 1) is shown in [Figure 111](#).

Figure 111. IIR filter structure (direct form 1)



MSv47127V1

Input:

- X1 buffer contains the elements of vector $\mathbf{X}$. It is a circular buffer of length $N + 1 + d$.
- X2 buffer contains the elements of coefficient vectors $\mathbf{B}$ and $\mathbf{A}$ concatenated ($b_0, b_1, b_2, \dots, b_N, a_1, a_2, \dots, a_M$). It is a fixed buffer of length $M+N+1$.

Output:

- Y buffer contains the output values, y_n . It is a circular buffer of length $M + d$.

Parameters

- The parameter P contains the length, $N + 1$, of the coefficient vector $\mathbf{B}$ in the range $[2:64]$.
- The parameter Q contains the length, M , of the coefficient vector $\mathbf{A}$ in the range $[1:63]$.
- The parameter R contains the gain to be applied to the accumulator output. The value output to the Y buffer is multiplied by 2^R , where R is in the range $[0:7]$.

The function completes when the START bit in the FMAC_PARAM register is reset by software.

21.3.7 Fixed point representation

The FMAC operates in fixed point signed integer format. Input and output values are q1.15.

In q1.15 format, numbers are represented by one sign bit and 15 fractional bits (binary decimal places). The numeric range is therefore -1 (0x8000) to $1 - 2^{-15}$ (0x7FFF).

The accumulator has 26 bits, of which 22 are fractional and 4 are integer/sign (q4.22). This allows it to support partial accumulation sums in the range -8 (0x2000000) to +7.99999976 (0x1FFFFFFF). A programmable gain from 0dB to 42dB in steps of 6dB can be applied at the output of the accumulator.

Note that the content of the accumulator is not saturated if the numeric range is exceeded. Partial sums whose value is greater than +7.99999976 or less than -8, wrap but this is harmless provided subsequent accumulations undo the wrapping. Nevertheless, the SAT flag in the FMAC_SR register is set if wrapping occurs, and generates an interrupt if the SATIEN bit is set in the FMAC_CR register. This helps in debugging the filter.

The data output by the accumulator can optionally be saturated, after application of the programmable gain, by setting the CLIPEN bit in the FMAC_CR register. If this bit is set, then any value which exceeds the numeric range of the q1.15 output, is set to $1 - 2^{-15}$ or -1, according to the sign. If clipping is not enabled, the unused accumulator bits after applying the gain is simply truncated.

21.3.8 Implementing FIR filters with the FMAC

The FMAC supports FIR filters of length N, where N is the number of taps or coefficients. The minimum local memory requirement for a FIR filter of length N is $2N + 1$:

- N coefficients
- N input samples
- 1 output sample

Since the local memory size is 256, the maximum value for N is 127.

If maximum throughput is required, it may be necessary to allocate a small amount of extra space, d1 and d2, to the input and output sample buffers respectively, to ensure that the filter never stalls waiting for a new input sample, or waiting for the output sample to be read. In this case, the local memory requirement is $2N + d1 + d2$.

The buffers must be configured as follows:

- X1_BUF_SIZE = $N + d1$;
- X2_BUF_SIZE = N;
- Y_BUF_SIZE = d2 (or 1 if no extra space is required)

The buffer base addresses can be allocated anywhere, but the X2 buffer must not overlap with the others, or else the coefficients are overwritten. An example configuration is:

- X2_BASE = 0;
- X1_BASE = N;
- Y_BASE = $2N + d1$

However, if the memory space is limited, the X1 and Y buffer areas can be overlapped, such that each output sample takes the place of the oldest input sample, which is no longer required:

- X2_BASE = 0;
- X1_BASE = N;
- Y_BASE = N

In this case, Y_BUF_SIZE = X1_BUF_SIZE = N + d1, so that the buffers remain in sync.

Note: The FULL_WM bitfield of X1 buffer configuration register must be programmed with a value less than or equal to $\log_2(d1)$, otherwise the buffer is flagged full before N input samples have been written, and no more samples are requested. Similarly, the EMPTY_WM bitfield of the Y buffer configuration register must be less than or equal to $\log_2(d2)$.

The filter coefficients **must** be pre-loaded into the X2 buffer, using the Load X2 Buffer function. The X1 buffer can optionally be pre-loaded with any number of samples up to a maximum of N. There is no point in pre-loading the Y buffer, since for the FIR filter there is no feedback path.

After configuring and initializing the buffers, the FMAC_CR register must be programmed according to the method used for writing and reading data to and from the FMAC memory.

Three methods are supported:

- Polling: No DMA request or Interrupt request is generated. Software must check that the X1_FULL flag is low before writing to WDATA, or that the Y_EMPTY flag is low before reading from RDATA.
- Interrupt: The interrupt request is asserted while the X1_FULL flag is low, for writes, or when the Y_EMPTY flag is low, for reads.
- DMA: DMA requests are asserted on the DMA write channel while the X1_FULL flag is low, and on the read channel while the Y_EMPTY flag is low.

Different methods can be used for read and for write. However it is not recommended to use both interrupts and DMA requests for the same operation^(a). The valid combinations are listed in [Table 179](#).

Table 179. Valid combinations for read and write methods

WIEN	RIEN	DMAWEN	DMAREN	Write	Read
0	0	0	0	Polling	Polling
0	1	0	0	Polling	Interrupt
1	0	0	0	Interrupt	Polling
1	1	0	0	Interrupt	Interrupt
0	0	0	1	Polling	DMA
0	0	1	0	DMA	Polling
0	0	1	1	DMA	DMA
0	1	1	0	DMA	Interrupt
1	0	0	1	Interrupt	DMA

a. If both interrupts and DMA requests are enabled then only DMA must perform the transfer.

The filter is started by writing to the FMAC_PARAM register with the following bitfield values:

- FUNC = 8 (FIR filter);
- P = N (number of coefficients);
- Q = "Don't care";
- R = Gain;
- START = 1;

If less than $N + d - 2^{\text{FULL_WM}}$ values have been pre-loaded in the X1 buffer, the X1FULL flag remains low. If the WIEN bit is set in the FMAC_CR register, then the interrupt request is asserted immediately to request the processor to write $2^{\text{FULL_WM}}$ additional samples into the buffer, via the FMAC_WDATA register. It remains asserted until the X1FULL flag goes high in the FMAC_SR register. The interrupt service routine must check the X1FULL flag after every $2^{\text{FULL_WM}}$ writes to the FMAC_WDATA register, and repeat the transfer until the flag goes high. Similarly, if the DMAWEN bit is set in the FMAC_CR register, DMA write channel requests are generated until the X1FULL flag goes high.

The filter calculates the first output sample when at least N samples have been written into the X1 buffer (including any pre-loaded samples).

When $2^{\text{EMPTY_WM}}$ output samples have been written into the Y buffer, the YEMPTY flag in the FMAC_SR register goes low. If the RIEN bit is set in the FMAC_CR register, the interrupt request is asserted to request the processor to read $2^{\text{EMPTY_WM}}$ samples from the buffer, via the FMAC_RDATA register. It remains asserted until the YEMPTY flag goes high. The interrupt service routine must check the YEMPTY flag after every $2^{\text{EMPTY_WM}}$ reads from the FMAC_RDATA register, and repeat the transfer until the flag goes high. If the DMAREN bit is set in the FMAC_CR, DMA read channel requests are generated until the YEMPTY flag goes high.

The filter continues to operate in this fashion until it is stopped by the software resetting the START bit.

21.3.9 Implementing IIR filters with the FMAC

The FMAC supports IIR filters of length N, where N is the number of feed-forward taps or coefficients. The number of feedback coefficients, M, can be any value from 1 to N-1. Only direct form 1 implementations can be realized, so filters designed for other forms need to be converted.

The minimum memory requirement for an IIR filter with N feed-forward coefficients and M feed-back coefficients is $2N + 2M$:

- N + M coefficients
- N input samples
- M output samples

If $M = N-1$, then the maximum filter length that can be implemented is $N = 64$.

As for the FIR, for maximum throughput, a small amount of additional space, d1 and d2, is allowed in the input and output buffer size respectively, making the total memory requirement $2M + 2N + d1 + d2$.

The buffers must be configured as follows:

- X1_BUF_SIZE = N + d1;
- X2_BUF_SIZE = N + M;
- Y_BUF_SIZE = M + d2;

The buffer base addresses can be allocated anywhere, but must not overlap. An example configuration is given below:

- $X2_BASE = 0;$
- $X1_BASE = N + M;$
- $Y_BASE = 2N + M + d1;$

Note: The FULL_WM bitfield of X1 buffer configuration register must be programmed with a value less than or equal to $\log_2(d1)$, otherwise the buffer is flagged full before N input samples have been written, and no more samples are requested. Similarly, the EMPTY_WM bitfield of the Y buffer configuration register must be less than or equal to $\log_2(d2)$.

The filter coefficients (N feed-forward followed by M feedback) **must** be pre-loaded into the X2 buffer, using the Load X2 Buffer function. The X1 buffer can optionally be pre-loaded with any number of samples up to a maximum of N. The Y buffer can optionally be pre-loaded with any number of values up to a maximum of M. This has the effect of initializing the feedback delay line.

After configuring the buffers, the FMAC_CR register must be programmed in the same way as for the FIR filter (see [Section 21.3.8: Implementing FIR filters with the FMAC](#)).

The filter is started by writing to the FMAC_PARAM register with the following bitfield values:

- $FUNC = 9$ (IIR filter);
- $P = N$ (number of feed-forward coefficients);
- $Q = M$ (number of feed-back coefficients);
- $R = \text{Gain};$
- $START = 1;$

If less than $N + d - 2^{FULL_WM}$ values have been pre-loaded in the X1 buffer, the X1FULL flag remains low. If the WIEN bit is set in the FMAC_CR register, then the interrupt request is asserted immediately to request the processor to write 2^{FULL_WM} additional samples into the buffer, via the FMAC_WDATA register. It remains asserted until the X1FULL flag goes high in the FMAC_SR register. The interrupt service routine must check the X1FULL flag after every 2^{FULL_WM} writes to the FMAC_WDATA register, and repeat the transfer until the flag goes high. Similarly, if the DMAWEN bit is set in the FMAC_CR register, DMA write channel requests are generated until the X1FULL flag goes high.

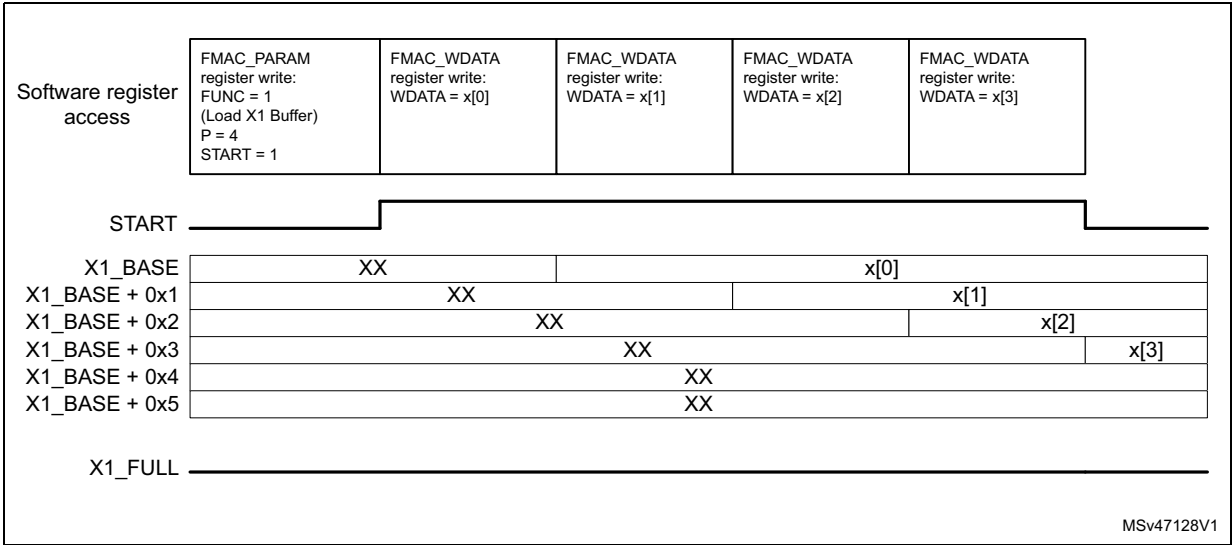
The filter calculates the first output sample when at least N samples have been written into the X1 buffer (including any pre-loaded samples). The first sample is calculated using the first N samples in the X1 buffer, and the first M samples in the Y buffer (whether or not they are preloaded). The first output sample is written into the Y buffer at $Y_BASE + M$.

When 2^{EMPTY_WM} new output samples have been written into the Y buffer, the YEMPTY flag in the FMAC_SR register goes low. If the RIEN bit is set in the FMAC_CR register, the interrupt request is asserted to request the processor to read 2^{EMPTY_WM} samples from the buffer, via the FMAC_RDATA register. It remains asserted until the YEMPTY flag goes high. The interrupt service routine must check the YEMPTY flag after every 2^{EMPTY_WM} reads from the FMAC_RDATA register, and repeat the transfer until the flag goes high. If the DMAREN bit is set in the FMAC_CR, DMA read channel requests are generated until the YEMPTY flag goes high.

The filter continues to operate in this fashion until it is stopped by the software resetting the START bit.

21.3.10 Examples of filter initialization

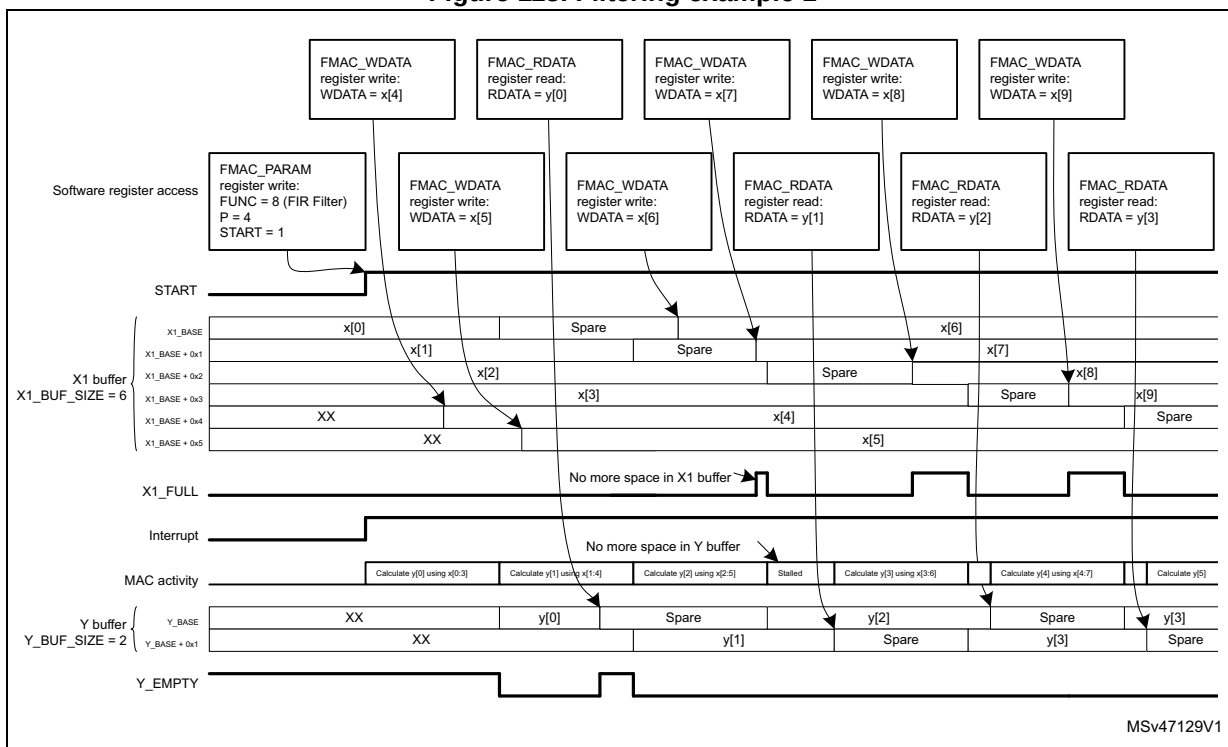
Figure 112. X1 buffer initialization



The example in [Figure 112](#) illustrates an X1 buffer pre-load with four samples (P = 4). The buffer size is six (X1_BUF_SIZE = 6). The initialization is launched by programming the FMAC_PARAM register with the START bit set. The four samples are then written to FMAC_WDATA, and transferred into local memory from X1_BASE onwards. The START bit resets after the fourth sample has been written. At this point, the X1 buffer contains the four samples, in order of writing, and the write pointer (next empty space) is at X1_BASE + 0x4.

21.3.11 Examples of filter operation

Figure 113. Filtering example 1

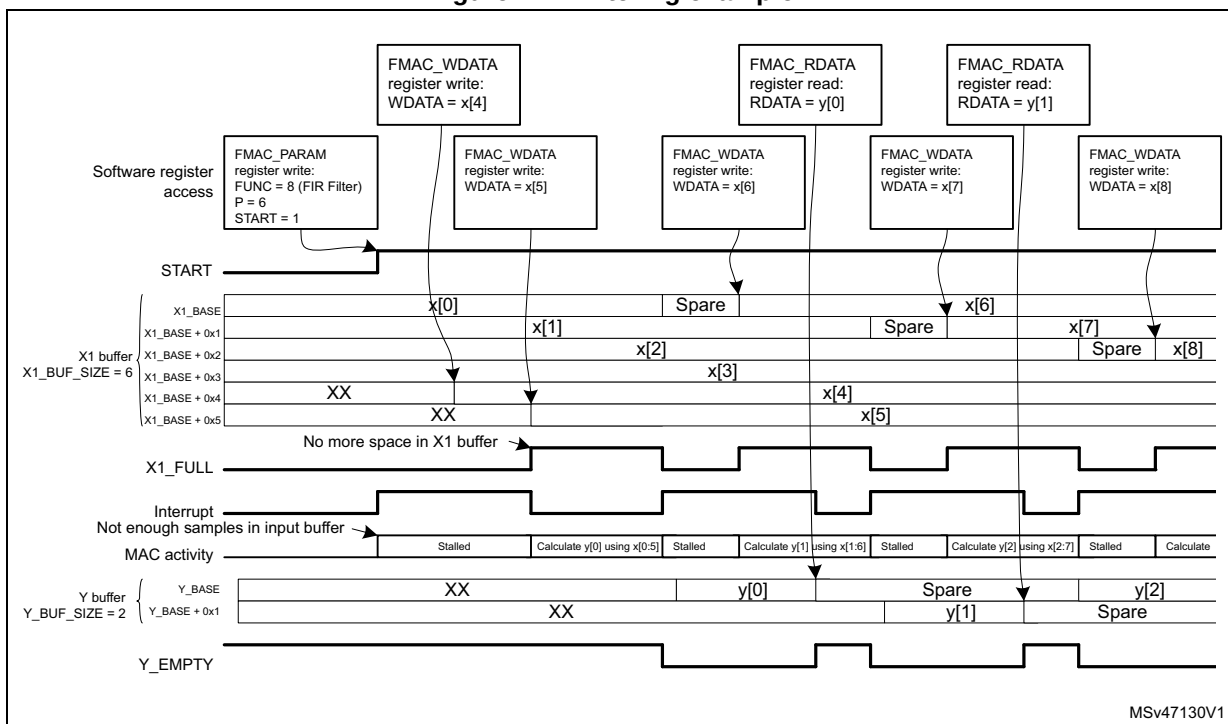


Note:

In this example, the processor can fill the input buffer more quickly than the FMAC can process them, so the X1_full flag regularly goes active. However, it struggles to read the Y buffer fast enough, so the FMAC stalls regularly waiting for space to be freed up in the Y buffer. This means the filter is not executing at maximum throughput. The reason is that the

filter length is small and the processor relatively slow, in this example. So increasing the Y buffer size does not help.

Figure 114. Filtering example 2



The example in [Figure 114](#) illustrates the beginning of the same filter operation, but this time the filter has six taps (P=6). The X1 buffer size is six and the Y buffer size is two. The FULL_WM and EMPTY_WM bitfields are both set to 0. Prior to starting the filter, the X1 buffer has been pre-loaded with four samples, x[0:3] as in [Figure 112](#). Because there are not enough samples in the input buffer, the X1FULL flag is not set, so the interrupt is asserted straight away, to request new data. The FMAC is stalled.

The processor writes two new samples, x[4] and x[5], to the FMAC_WDATA register, which are transferred to the empty locations in the X1 buffer. As soon as there are six unused samples in the X1 buffer, the X1_FULL flag goes active (since the buffer size is six), causing the interrupt to go inactive. The FMAC starts calculating the first output sample, y[0]. Since this requires all six input samples, there are no free spaces in the X1 buffer and so the X1_FULL flag remains active. Only when the FMAC finishes calculating y[0] and writes it into the Y buffer, can x[0] be discarded, freeing up a space in the X1 buffer, and deasserting X1_FULL. At the same time, the Y_EMPTY flag goes inactive. Both these flag states cause the interrupt to be asserted, requesting the processor to write a new input sample, first of all, and then read the output sample just calculated. The FMAC remains stalled until a new input sample is written.

In this example, the processor has to wait for the FMAC to finish calculating the current output sample, before it can write a new input sample, and therefore the X1 buffer regularly goes empty, stalling the FMAC. This can be avoided by allowing some extra space in the input buffer.

21.3.12 Filter design tips

The FMAC architecture imposes some constraints detailed below, on the design of digital filters.

1. Implementation of direct form 2, or transposed forms, is not efficient. Filters which have been designed for such forms must be converted to direct form 1.
2. Cascaded filters must either be combined into a single stage, or implemented as separate filters. In the latter case, multiple sets of filter coefficients can be pre-loaded into the memory, one set per stage, and only the X2_BASE address changed to select which set is used. The most efficient method of implementing a multi-stage filter is to pre-load a large X1 buffer with input samples, run the IIR filter function on it using the first stage coefficients, and store the output samples back in memory. Then change the X2_BASE pointer to point to the 2nd stage coefficients, and reload the input buffer with the output of the first stage (with a gain if required), before running the IIR function again. The procedure is repeated for all stages. Once the final stage samples have been transferred back into system memory, the input buffer can be loaded with the next set of input samples, and a new round of calculations started. Note that the N sample input buffer of each stage must be pre-loaded first of all with the N-1 last inputs from the previous round, plus one new sample, in order to keep continuity between each round. Similarly, the output buffer of each stage must be loaded with the last M samples from the previous round, for the same reason.
3. The use of direct form 1 for IIR designs can lead to large positive or negative partial sums in the accumulator, if for example a large step occurs on the input, or some of the filter coefficients' absolute values are >1 . Since the accumulator is limited to 26 bits, the biggest value that it can handle without wrapping (changing sign) is 0x1FFFFFFF positive or 0x20000000 negative. This corresponds to 3.99999988 and -4 respectively in q3.23 fixed point format. Wrapping does not represent a problem provided the wrapping is "undone" before the end of the accumulation. However this is not always the case when a filter is starting up and can lead to unexpected results. Consider pre-loading the output buffer with suitable values to avoid this.
4. The IIR filter has feed-forward (numerator) coefficients $[b_0, b_1, \dots, b_{N-1}]$, and feed-back (denominator) coefficients $[1, a_1, \dots, a_M]$. Many IIR filters require some of the denominator coefficients to have an absolute value greater than 1 to achieve a steep roll-off in the frequency response. Given that the coefficients are coded in fixed point q1.15 format, this is not possible. Nevertheless, by scaling the denominator coefficients by a factor 2^{-R} , such that $2^{-R} \cdot [1, a_1, \dots, a_M]$ are all less than 1, such filters can be implemented. However an inverse gain of 2^R must be applied at the output of the accumulator to compensate the scaling. This has an adverse effect on the signal-to-noise ratio.

21.4 FMAC registers

21.4.1 FMAC X1 buffer configuration register (FMAC_X1BUFCFG)

Address offset: 0x00

Reset value: 0x0000 0000

Access: word access

This register can only be modified if START = 0 in the FMAC_PARAM register.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	FULL_WM[1:0]		Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
						rw	rw								
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
X1_BUF_SIZE[7:0]								X1_BASE[7:0]							
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bits 31:26 Reserved, must be kept at reset value.

Bits 25:24 **FULL_WM[1:0]**: Watermark for buffer full flag

Defines the threshold for setting the X1 buffer full flag when operating in circular mode. The flag is set if the number of free spaces in the buffer is less than $2^{\text{FULL_WM}}$.

0: Threshold = 1

1: Threshold = 2

2: Threshold = 4

3: Threshold = 8

Setting a threshold greater than 1 allows several data to be transferred into the buffer under one interrupt.

Threshold must be set to 1 if DMA write requests are enabled (DMAWEN = 1 in FMAC_CR register).

Bits 23:16 Reserved, must be kept at reset value.

Bits 15:8 **X1_BUF_SIZE[7:0]**: Allocated size of X1 buffer in 16-bit words

The minimum buffer size is the number of feed-forward taps in the filter (+ the watermark threshold - 1).

Bits 7:0 **X1_BASE[7:0]**: Base address of X1 buffer

21.4.2 FMAC X2 buffer configuration register (FMAC_X2BUFCFG)

Address offset: 0x04

Reset value: 0x0000 0000

Access: word access

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
X2_BUF_SIZE[7:0]								X2_BASE[7:0]							
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bits 31:16 Reserved, must be kept at reset value.

Bits 15:8 **X2_BUF_SIZE[7:0]**: Size of X2 buffer in 16-bit words

This bitfield can not be modified when a function is ongoing (START = 1).

Bits 7:0 **X2_BASE[7:0]**: Base address of X2 buffer

The X2 buffer base address can be modified while START=1, for example to change coefficient values. The filter must be stalled when doing this, since changing the coefficients while a calculation is ongoing affects the result.

21.4.3 FMAC Y buffer configuration register (FMAC_YBUFCFG)

Address offset: 0x08

Reset value: 0x0000 0000

Access: word access

This register can only be modified if START = 0 in the FMAC_PARAM register.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	EMPTY_WM[1:0]		Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
						rw	rw								
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Y_BUF_SIZE[7:0]								Y_BASE[7:0]							
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bits 31:26 Reserved, must be kept at reset value.

Bits 25:24 **EMPTY_WM[1:0]**: Watermark for buffer empty flag

Defines the threshold for setting the Y buffer empty flag when operating in circular mode. The flag is set if the number of unread values in the buffer is less than $2^{\text{EMPTY_WM}}$.

0: Threshold = 1

1: Threshold = 2

2: Threshold = 4

3: Threshold = 8

Setting a threshold greater than 1 allows several data to be transferred from the buffer under one interrupt.

Threshold must be set to 1 if DMA read requests are enabled (DMAREN = 1 in FMAC_CR register).

Bits 23:16 Reserved, must be kept at reset value.

Bits 15:8 **Y_BUF_SIZE[7:0]**: Size of Y buffer in 16-bit words

For FIR filters, the minimum buffer size is 1 (+ the watermark threshold). For IIR filters the minimum buffer size is the number of feedback taps (+ the watermark threshold).

Bits 7:0 **Y_BASE[7:0]**: Base address of Y buffer

21.4.4 FMAC parameter register (FMAC_PARAM)

Address offset: 0x0C

Reset value: 0x0000 0000

Access: word access

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
START	FUNC[6:0]							R[7:0]							
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Q[7:0]								P[7:0]							
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bit 31 **START**: Enable execution

0: Stop execution

1: Start execution

Setting this bit triggers the execution of the function selected in the FUNC bitfield. Resetting it by software stops any ongoing function. For initialization functions, this bit is reset by hardware.

Bits 30:24 **FUNC[6:0]**: Function

0: Reserved

1: Load X1 buffer

2: Load X2 buffer

3: Load Y buffer

4 to 7: Reserved

8: Convolution (FIR filter)

9: IIR filter (direct form 1)

10 to 127: Reserved

This bitfield can not be modified when a function is ongoing (START = 1)

Bits 23:16 **R[7:0]**: Input parameter R.

The value of this parameter is dependent on the function.

This bitfield can not be modified when a function is ongoing (START = 1)

Bits 15:8 **Q[7:0]**: Input parameter Q.

The value of this parameter is dependent on the function.

This bitfield can not be modified when a function is ongoing (START = 1)

Bits 7:0 **P[7:0]**: Input parameter P.

The value of this parameter is dependent on the function

This bitfield can not be modified when a function is ongoing (START = 1)

21.4.5 FMAC control register (FMAC_CR)

Address offset: 0x10

Reset value: 0x0000 0000

Access: word access

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	RESET
															rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
CLIP EN	Res.	Res.	Res.	Res.	Res.	DMA WEN	DMA REN	Res.	Res.	Res.	SAT IEN	UNFL IEN	OVFL IEN	WIEN	RIEN
rw						rw	rw				rw	rw	rw	rw	rw

Bits 31:17 Reserved, must be kept at reset value.

Bit 16 **RESET**: Reset FMAC unit

This resets the write and read pointers, the internal control logic, the FMAC_SR register and the FMAC_PARAM register, including the START bit if active. Other register settings are not affected. This bit is reset by hardware.

0: Reset inactive

1: Reset active

Bit 15 **CLIPEN**: Enable clipping

0: Clipping disabled. Values at the output of the accumulator which exceed the q1.15 range, wrap.

1: Clipping enabled. Values at the output of the accumulator which exceed the q1.15 range are saturated to the maximum positive or negative value (+1 or -1) according to the sign.

Bits 14:10 Reserved, must be kept at reset value.

Bit 9 **DMAWEN**: Enable DMA write channel requests

0: Disable. No DMA requests are generated

1: Enable. DMA requests are generated while the X1 buffer is not full.

This bit can only be modified when START= 0 in the FMAC_PARAM register. A read returns the current state of the bit.

Bit 8 **DMAREN**: Enable DMA read channel requests

0: Disable. No DMA requests are generated

1: Enable. DMA requests are generated while the Y buffer is not empty.

This bit can only be modified when START= 0 in the FMAC_PARAM register. A read returns the current state of the bit.

Bits 7:5 Reserved, must be kept at reset value.

Bit 4 **SATIEN**: Enable saturation error interrupts

0: Disabled. No interrupts are generated upon saturation detection.

1: Enabled. An interrupt request is generated if the SAT flag is set

This bit is set and cleared by software. A read returns the current state of the bit.

Bit 3 **UNFLIEN**: Enable underflow error interrupts

0: Disabled. No interrupts are generated upon underflow detection.

1: Enabled. An interrupt request is generated if the UNFL flag is set

This bit is set and cleared by software. A read returns the current state of the bit.

Bit 2 **OVFLIEN**: Enable overflow error interrupts

0: Disabled. No interrupts are generated upon overflow detection.

1: Enabled. An interrupt request is generated if the OVFL flag is set

This bit is set and cleared by software. A read returns the current state of the bit.

Bit 1 **WIEN**: Enable write interrupt

0: Disabled. No write interrupt requests are generated.

1: Enabled. An interrupt request is generated while the X1 buffer FULL flag is not set.

This bit is set and cleared by software. A read returns the current state of the bit.

Bit 0 **RIEN**: Enable read interrupt

0: Disabled. No read interrupt requests are generated.

1: Enabled. An interrupt request is generated while the Y buffer EMPTY flag is not set.

This bit is set and cleared by software. A read returns the current state of the bit.

21.4.6 FMAC status register (FMAC_SR)

Address offset: 0x14

Reset value: 0x0000 0001

Access: word access

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	Res.	Res.	Res.	SAT	UNFL	OVFL	Res.	Res.	Res.	Res.	Res.	Res.	X1 FULL	Y EMPTY
					r	r	r							r	r

Bits 31:11 Reserved, must be kept at reset value.

Bit 10 **SAT**: Saturation error flag

Saturation occurs when the result of an accumulation exceeds the numeric range of the accumulator.

0: No saturation detected

1: Saturation detected. If the SATIEN bit is set, an interrupt is generated.

This flag is cleared by a reset of the unit.

Bit 9 **UNFL**: Underflow error flag

An underflow occurs when a read is made from FMAC_RDATA when no valid data is available in the Y buffer.

0: No underflow detected

1: Underflow detected. If the UNFLIEN bit is set, an interrupt is generated.

This flag is cleared by a reset of the unit.

Bit 8 **OVFL**: Overflow error flag

An overflow occurs when a write is made to FMAC_WDATA when no free space is available in the X1 buffer.

0: No overflow detected

1: Overflow detected. If the OVFLIEN bit is set, an interrupt is generated.

This flag is cleared by a reset of the unit.

Bits 7:2 Reserved, must be kept at reset value.

Bit 1 X1FULL: X1 buffer full flag

The buffer is flagged as full if the number of available spaces is less than the FULL_WM threshold. The number of available spaces is the difference between the write pointer and the least recent sample currently in use.

0: X1 buffer not full. If the WIEN bit is set, the interrupt request is asserted until the flag is set. If DMAWEN is set, DMA write channel requests are generated until the flag is set.

1: X1 buffer full.

This flag is set and cleared by hardware, or by a reset.

Note: after the last available space in the X1 buffer is filled there is a delay of 3 clock cycles before the X1FULL flag goes high. To avoid any risk of overflow it is recommended to insert a software delay after writing to the X1 buffer before reading the FMAC_SR. Alternatively, a FULL_WM threshold of 2 can be used.

Bit 0 YEMPTY: Y buffer empty flag

The buffer is flagged as empty if the number of unread data is less than the EMPTY_WM threshold. The number of unread data is the difference between the read pointer and the current output destination address.

0: Y buffer not empty. If the RIEN bit is set, the interrupt request is asserted until the flag is set. If DMAREN is set, DMA read channel requests are generated until the flag is set.

1: Y buffer empty.

This flag is set and cleared by hardware, or by a reset.

Note: after the last sample is read from the Y buffer there is a delay of 3 clock cycles before the YEMPTY flag goes high. To avoid any risk of underflow it is recommended to insert a software delay after reading from the Y buffer before reading the FMAC_SR. Alternatively, an EMPTY_WM threshold of 2 can be used.

21.4.7 FMAC write data register (FMAC_WDATA)

Address offset: 0x18

Reset value: 0x0000 0000

Access: word access

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
WDATA[15:0]															
w	w	w	w	w	w	w	w	w	w	w	w	w	w	w	w

Bits 31:16 Reserved, must be kept at reset value.

Bits 15:0 **WDATA[15:0]**: Write data

When a write access to this register occurs, the write data are transferred to the address offset indicated by the write pointer. The pointer address is automatically incremented after each write access.

21.4.8 FMAC read data register (FMAC_RDATA)

Address offset: 0x1C

Reset value: 0x0000 0000

Access: word access

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
RDATA[15:0]															
r	r	r	r	r	r	r	r	r	r	r	r	r	r	r	r

Bits 31:16 Reserved, must be kept at reset value.

Bits 15:0 **RDATA[15:0]**: Read data

When a read access to this register occurs, the read data are the contents of the Y output buffer at the address offset indicated by the READ pointer. The pointer address is automatically incremented after each read access.

21.4.9 FMAC register map

Table 180.FMAC register map and reset values

Offset	Register name	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0x00	FMAC_X1BUFCFG	Res.	Res.	Res.	Res.	Res.	Res.	FULL_WM [1:0]	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	X1_BUF_SIZE[7:0]				X1_BASE[7:0]											
	Reset value							0	0	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0x04	FMAC_X2BUFCFG	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	X2_BUF_SIZE[7:0]				X2_BASE[7:0]											
	Reset value							Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0x08	FMAC_YBUFCFG	Res.	Res.	Res.	Res.	Res.	Res.	EMPTY_WM [1:0]	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Y_BUF_SIZE[7:0]				Y_BASE[7:0]											
	Reset value							0	0	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0x0C	FMAC_PARAM	START	FUNC[6:0]						R[7:0]						Q[7:0]						P[7:0]												
	Reset value	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0x10	FMAC_CR	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	RESET	CLIPEN	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	SATIEN	UNFLIEN	OVFLIEN	WIEN	RIEN
	Reset value																0	0						0	0				0	0	0	0	0
0x14	FMAC_SR	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	SAT	UNFL	OVFL	Res.	Res.	Res.	Res.	Res.	Res.	Res.	X1FULL	YEMPTY	
	Reset value																				0	0	0								0	1	
0x18	FMAC_WDATA	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	WDATA[15:0]															
	Reset value																	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0x1C	FMAC_RDATA	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	RDATA[15:0]															
	Reset value																	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Refer to [Section 2.3: Memory organization](#) for the register boundary addresses.

22 Programmable logic array (PLAY)

This section is applied only to STM32H543/553xx devices.

22.1 PLAY introduction

The programmable logic array allows the user to create custom logic and state machines without the need for external programmable logic devices, such as FPGAs. The PLAY provides a number of programmable logic elements, in the form of look-up tables (LUTs), each of which have four inputs and one output. The output can be programmed to be any logical function of the four inputs. More complex functions can be created by cascading logic elements.

Inputs to and outputs from the logic array can be selected from a choice of external I/Os, internal signals or software programmable register bits. External inputs can be synchronized to the internal clock and filtered to remove short pulses, or connected directly to the logic array if purely combinatorial or asynchronous functions are required.

Each logic element output can be registered using the internal clock, or another signal. Registered outputs can be fed back to the logic array inputs to create finite state machines. Transitions on the outputs can set flags, which can generate interrupts to the processor. The state of the flags can be read and reset by software. The raw states of the logic element outputs can also be read by software or by an external debugger. This provides visibility of the logic array state to simplify debugging of the programmed logic functions.

The array is configured via memory-mapped registers. Configuration is simple and intuitive. Configuration registers are protected against accidental programming by a lock bit. The configuration is maintained in case of a warm system reset.

22.2 PLAY main features

- Glitch-free programmable logic elements each comprising a 4-input look-up table and a register
- Configurable input and output interconnects
- Optional synchronization of inputs, with programmable glitch filters and edge detection/pulse extension feature.
- Software programmable inputs
- Flags with interrupt capability for communication with software
- Simple register-based configuration interface, protected by software lock

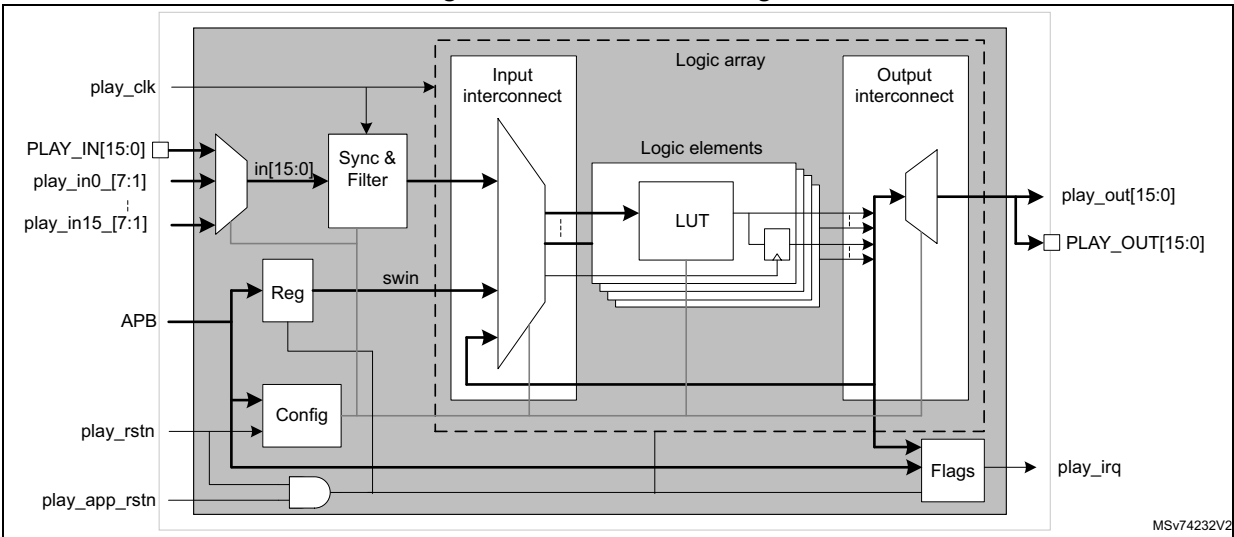
22.3 PLAY functional description

- a logic array, including
 - 16 logic elements
 - an input interconnect
 - an output interconnect
- a synchronization and filter unit
- a configuration unit
- a flag and interrupt unit
- an APB bus slave interface

These are shown in [Figure 115](#).

22.3.1 PLAY block diagram

Figure 115. PLAY block diagram



22.3.2 PLAY pins and internal signals

Table 181. PLAY pins

Pin name	Pin type	Description
PLAY_IN[15:0]	Digital inputs	External inputs to the PLAY
PLAY_OUT[15:0]	Digital outputs	External outputs from the PLAY

Table 182. PLAY internal signals

Signal name	Signal type	Description
play_in[15:0]_[7:1]	Digital inputs	PLAY internal inputs
play_out[15:0]	Digital outputs	PLAY internal outputs
play_clk	Clock input	PLAY clock from RCC

Table 182. PLAY internal signals (continued)

Signal name	Signal type	Description
pclk	Clock input	APB bus clock
play_irq	Interrupt request	Global interrupt
play_rstn	Reset input	Global reset (includes config)
play_app_rstn	Reset input	Application reset (excludes config)
play_priv0, play_priv1	Digital outputs	Privilege attribute indicators

Table 183 lists the signals connected to the PLAY inputs. Up to 16 inputs can be selected for connection to the logic array, one from each row. See [Section 22.3.4: Input selection](#) for details.

 Table 183. PLAY logic array input *n* signal selection

input <i>n</i>	PLAY_FILTnCFG.PREMUXSEL[2:0]							
	0	1	2	3	4	5	6	7
0	play_in0_0	play_in0_1	play_in0_2	play_in0_3	play_in0_4	play_in0_5	play_in0_6	play_in0_7
	PLAY_IN0	tim2_trgo	tim3_trgo	SPI1_MOSI	EVENTOUT	adc1_awd2	lptim2_ch2	lptim1_ch2
1	play_in1_0	play_in1_1	play_in1_2	play_in1_3	play_in1_4	play_in1_5	play_in1_6	play_in1_7
	PLAY_IN1	tim2_oc3	tim3_oc3	SPI1_SCLK	SPI2_NSS	USART1_TX	comp2_out	comp1_out
2	play_in2_0	play_in2_1	play_in2_2	play_in2_3	play_in2_4	play_in2_5	play_in2_6	play_in2_7
	PLAY_IN2	tim2_oc4	tim3_oc4	tim2_trgo	SPI2_MOSI	USART1_CK	rcc_mco2	rcc_mco1
3	play_in3_0	play_in3_1	play_in3_2	play_in3_3	play_in3_4	play_in3_5	play_in3_6	play_in3_7
	PLAY_IN3	lptim1_ch1	lptim2_ch1	tim2_oc3	SPI2_SCLK	NMI	adc2_awd1	adc1_awd1
4	play_in4_0	play_in4_1	play_in4_2	play_in4_3	play_in4_4	play_in4_5	play_in4_6	play_in4_7
	PLAY_IN4	lptim1_ch2	lptim2_ch2	tim2_oc4	tim3_trgo	LOCKUP	adc2_awd2	adc1_awd2
5	play_in5_0	play_in5_1	play_in5_2	play_in5_3	play_in5_4	play_in5_5	play_in5_6	play_in5_7
	PLAY_IN5	comp1_out	comp2_out	lptim1_ch1	tim3_oc3	SPI1_NSS	USART2_TX	USART1_TX
6	play_in6_0	play_in6_9	play_in6_2	play_in6_3	play_in6_4	play_in6_5	play_in6_6	play_in6_7
	PLAY_IN6	rcc_mco1	rcc_mco2	lptim1_ch2	tim3_oc4	SPI1_MOSI	USART2_CK	USART1_CK
7	play_in7_0	play_in7_1	play_in7_2	play_in7_3	play_in7_4	play_in7_5	play_in7_6	play_in7_7
	PLAY_IN7	adc1_awd1	adc2_awd1	comp1_out	lptim2_ch1	SPI1_SCLK	0	NMI
8	play_in8_0	play_in8_1	play_in8_2	play_in8_3	play_in8_4	play_in8_5	play_in8_6	play_in8_7
	PLAY_IN8	adc1_awd2	adc2_awd2	rcc_mco1	lptim2_ch2	tim2_trgo	EVENTOUT	LOCKUP
9	play_in9_0	play_in9_1	play_in9_2	play_in9_3	play_in9_4	play_in9_5	play_in9_6	play_in9_7
	PLAY_IN9	USART1_TX	USART2_TX	adc1_awd1	comp2_out	tim2_oc3	SPI2_NSS	SPI1_NSS
10	play_in10_0	play_in10_1	play_in10_2	play_in10_3	play_in10_4	play_in10_5	play_in10_6	play_in10_7
	PLAY_IN10	USART1_CK	USART2_CK	adc1_awd2	rcc_mco2	tim2_oc4	SPI2_MOSI	SPI1_MOSI
11	play_in11_0	play_in11_1	play_in11_2	play_in11_3	play_in11_4	play_in11_5	play_in11_6	play_in11_7
	PLAY_IN11	NMI	0	USART1_TX	adc2_awd1	lptim1_ch1	SPI2_SCLK	SPI1_SCLK

Table 183. PLAY logic array input *n* signal selection (continued)

input <i>n</i>	PLAY_FILT _n CFG.PREMUXSEL[2:0]							
	0	1	2	3	4	5	6	7
12	play_in12_0	play_in12_1	play_in12_2	play_in12_3	play_in12_4	play_in12_5	play_in12_6	play_in12_7
	PLAY_IN12	LOCKUP	EVENTOUT	USART1_CK	adc2_awd2	lptim1_ch2	tim3_trgo	tim2_trgo
13	play_in13_0	play_in13_1	play_in13_2	play_in13_3	play_in13_4	play_in13_5	play_in13_6	play_in13_7
	PLAY_IN13	SPI1_NSS	SPI2_NSS	NMI	USART2_TX	comp1_out	tim3_oc3	tim2_oc3
14	play_in14_0	play_in14_1	play_in14_2	play_in14_3	play_in14_4	play_in14_5	play_in14_6	play_in14_7
	PLAY_IN14	SPI1_MOSI	SPI2_MOSI	LOCKUP	USART2_CK	rcc_mco1	tim3_oc4	tim2_oc4
15	play_in15_0	play_in15_1	play_in15_2	play_in15_3	play_in15_4	play_in15_5	play_in15_6	play_in15_7
	PLAY_IN15	SPI1_SCLK	SPI2_SCLK	SPI1_NSS	0	adc1_awd1	lptim2_ch1	lptim1_ch1

Table 184. PLAY output connections

Signal name	External pins	Internal signals
play_out0	PLAY_OUT0	TIM2_etr16 TIM3_etr16 lptim1_ext_trig7 lptim2_ext_trig7
play_out1	PLAY_OUT1	TIM1_brk[3] TIM8_brk[3]
play_out2	PLAY_OUT2	TIM1_brk2[3] TIM8_brk2[3]
play_out3	PLAY_OUT3	TIM2_ti1_4 TIM3_ti1_4 lptim1_in1_mux2 lptim2_in1_mux2
play_out4	PLAY_OUT4	TIM2_ti2_3 TIM3_ti2_3 lptim1_in2_mux2 lptim2_in2_mux2
play_out5	PLAY_OUT5	TIM2_ti3_1 TIM3_ti3_1 lptim1_ic1_mux3 lptim2_ic1_mux3
play_out6	PLAY_OUT6	TIM2_ti4_1 TIM3_ti4_1
play_out7	PLAY_OUT7	adc1_ext_trg20 adc2_ext_trg20 adc3_ext_trg20
play_out8	PLAY_OUT8	-

Table 184. PLAY output connections (continued)

Signal name	External pins	Internal signals
play_out9	PLAY_OUT9	adc1_jext_trg20 adc2_jext_trg20 adc3_jext_trg20
play_out10	PLAY_OUT10	-
play_out11	PLAY_OUT11	dac_trg14
play_out12	PLAY_OUT12	dac_trg15
play_out13	PLAY_OUT13	dts_ts1_trg4
play_out14	PLAY_OUT14	EXTI_event_64
play_out15	PLAY_OUT15	GPDMA_trig_72

22.3.3 PLAY reset and clocks

The PLAY has two reset inputs:

- play_rstn: connected to the power reset
- play_app_rstn: connected to the system reset

The PLAY configuration registers are reset by:

- play_rstn

All other registers, including the PLAY logic element output registers, are reset by:

- play_rstn
- play_app_rstn

The PLAY configuration and control registers are clocked by:

- pclk (APB bus clock)

The synchronisation and filters are clocked by:

- play_clk

The PLAY logic element output registers are clocked by:

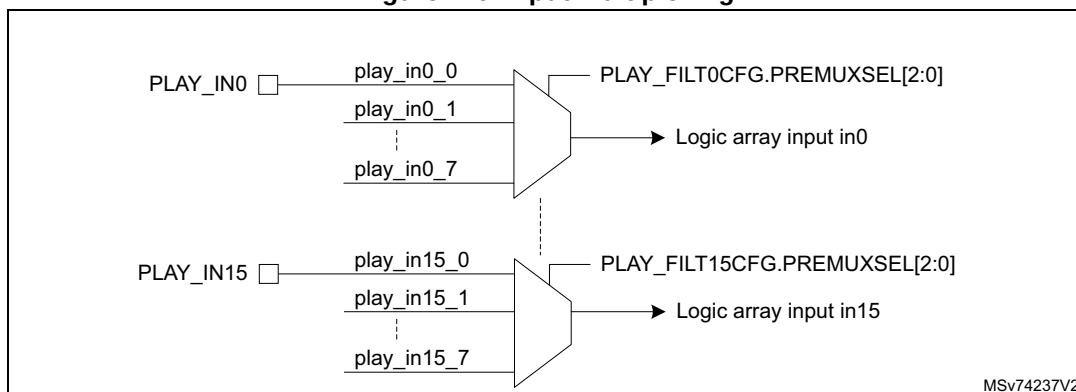
- play_clk, optionally conditioned by any of the PLAY inputs or logic element registered outputs

22.3.4 Input selection

There are 16 inputs to the logic array, in0 to in15, which can be connected to external I/Os or internal signals coming from other functional units.

Each logic array input can connect to one of eight signals. A pre-multiplexer selects which signal is connected to an input. The pre-multiplexer for logic array input *n* is configured by the PLAY_FILTnCFG.PREMUXSEL[2:0] register field (see [Figure 116](#)). Only one signal can be chosen per input.

Figure 116. Input multiplexing

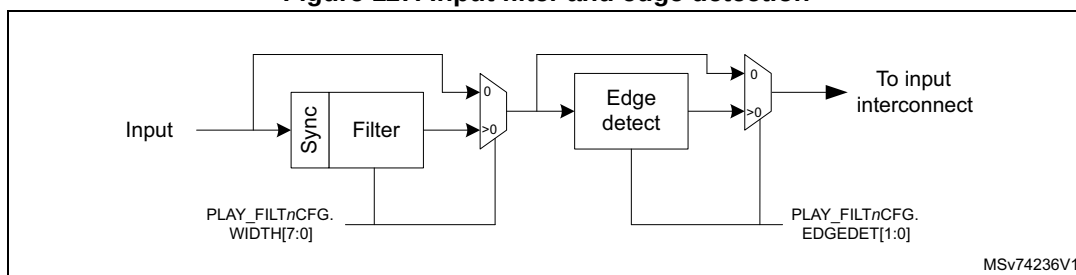


The mapping of signals to logic array inputs is shown in [Table 183](#). The left hand column contains the logic array input number. The columns to the right show the play_in signal name as well as the source signal to which it is connected, for each setting of `PLAY_FILTnCFG.PREMUXSEL[2:0]`. Some source signal names appear more than once in the table, which means they can be mapped to more than one input. This offers more flexibility when mapping source signals to logic array inputs.

22.3.5 Input filtering and edge detection

Inputs pass through a filter stage, which can be configured to remove short pulses or glitches and generate a pulse if an edge is detected (see [Figure 117](#)).

Figure 117. Input filter and edge detection



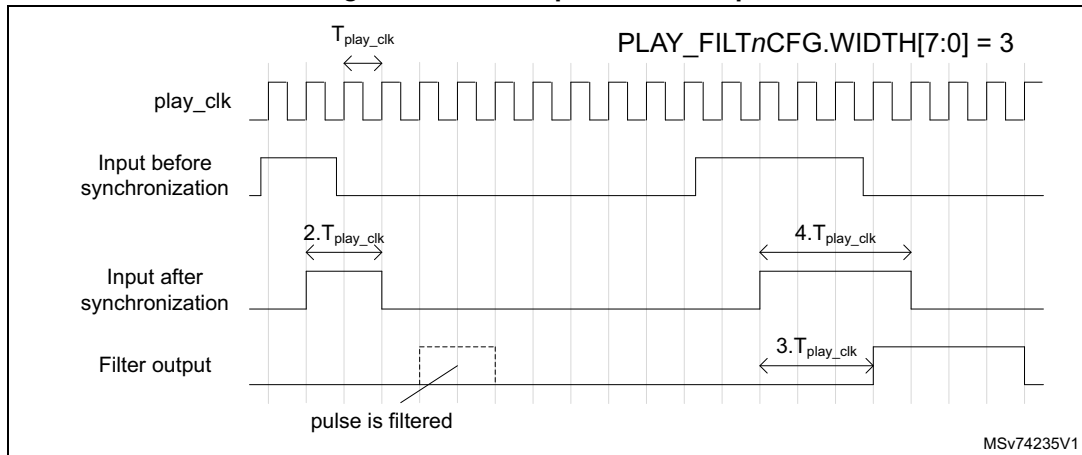
Filtering

Every input has a filter which removes pulses shorter than a programmable number of `play_clk` clock cycles (after synchronization). The filter samples the input at the `play_clk` clock frequency and starts a counter if it detects a change in state. If the input transitions again before the counter reaches zero, the counter is stopped and reloaded, and the filter output state remains unchanged. If the counter reaches zero, it is reloaded and the filter output state toggles.

The minimum pulse width for input *n* can be programmed from 1 to 255 cycles in register bits `PLAY_FILTnCFGn.WIDTH[7:0]`. Note that the filter introduces a delay equal to the programmed minimum pulse width.

[Figure 118](#) illustrates the operation of the filter for `PLAY_FILTnCFGn.WIDTH[7:0] = 3`. Pulses whose width is less than three periods of `play_clk` ($T_{\text{play_clk}}$) are filtered. Pulses whose width is three periods or more propagate through the filter, with a delay of three $T_{\text{play_clk}}$ periods.

Figure 118. Filter operation example



MSv74235V1

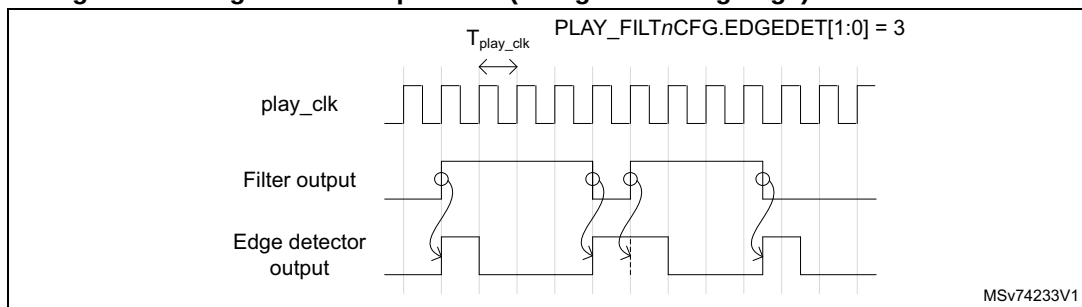
The filter can be bypassed by setting $PLAY_FILTnCFG.WIDTH[7:0] = 0$.

Edge detection and pulse generation

For all inputs there is an optional edge detection feature which detects a transition and generates a positive pulse of one $play_clk$ clock period. This can be used to detect and lengthen pulses which are shorter than the $play_clk$ clock period, or to trigger actions on an edge rather than a level. The edge detection feature is enabled for input n by programming the $PLAY_FILTCFGn.EDGEDET[1:0]$ field with direction of the transition (rising, falling or both edges). Setting the field to 0 disables and bypasses the edge detection.

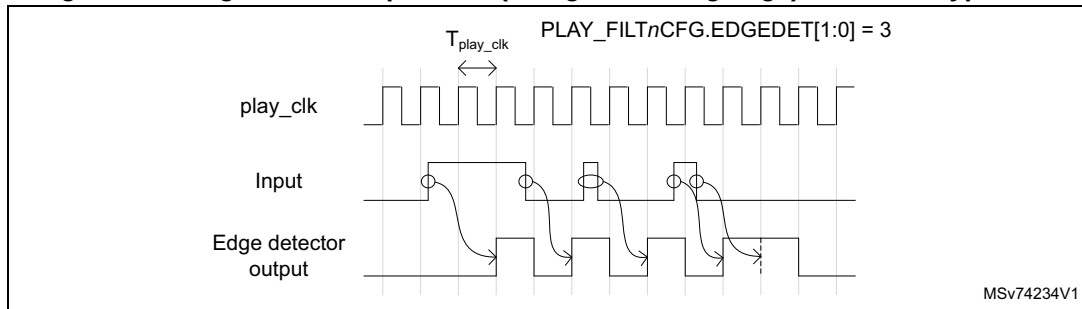
If the filter is enabled ($PLAY_FILTCFGn.WIDTH[7:0] > 0$), the edge detector generates a pulse when an edge is detected at the output of the filter (see [Figure 119](#)). Note that if both rising and falling edges are programmed, back-to-back transitions on successive clocks generate a double length pulse.

Figure 119. Edge detector operation (rising and falling edge) with filter enabled



MSv74233V1

To lengthen a pulse shorter than one $play_clk$ period, the filter must be bypassed. This is done by setting $PLAY_FILTCFGn.WIDTH[7:0] = 0$. The synchronization is done by the edge detection unit. When an edge is detected on the input signal, a positive pulse is generated after a delay of between one and two $play_clk$ periods (see [Figure 120](#)).

Figure 120. Edge detector operation (rising and falling edge) with filter bypassed

If both rising and falling edges are programmed, and the input pulse width is less than one `play_clk` period, only one output pulse is generated. Note however that if two transitions occur either side of a clock edge, a double length pulse is generated.

Synchronization

All input signals are considered to be asynchronous to the `play_clk` clock. If any synchronous logic is connected downstream, it is mandatory to synchronize the inputs to the `play_clk` clock, to avoid meta-stability. Synchronization introduces a delay of 1 to 2 `play_clk` periods.

Synchronization is performed by enabling the filter or the edge detector. If both the filter and edge detector are bypassed (`PLAY_FILTnCFG.WIDTH[7:0] = 0` and `PLAY_FILTnCFG.EDGEDET[1:0] = 0`), the input signal is propagated directly to the logic array without synchronization.

Caution: If both edge detector and filter are bypassed, the input must only be used to drive purely combinatorial logic.

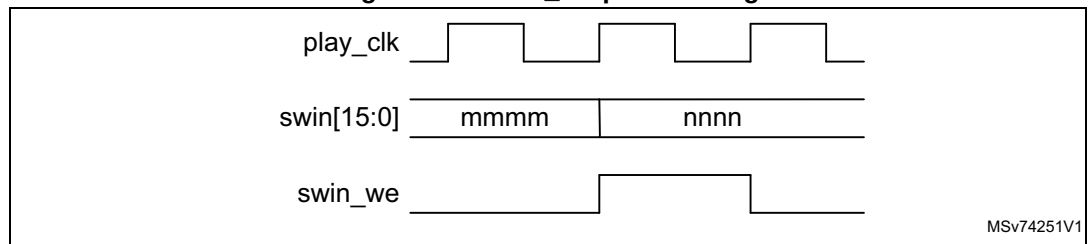
22.3.6 Software programmable inputs

The `PLAY_SWIN` register contains 16 bits which are connected to inputs `swin0` to `swin15`. This allows software programmable functionality to be included in the logic expression. This can be used for example to enable or disable functionality, or to trigger state transitions, by software. These inputs also allow testing of the programmed logic array functionality.

The `PLAY_SWIN` register bits can be set independently by writing a 1 to the corresponding bits in the `PLAY_SWIN_SET` register. In the same way they can be reset by writing a 1 to the corresponding bits in the `PLAY_SWIN_CLR` register. It is also possible to read and write the bits directly in the `PLAY_SWIN` register.

Every time there is a write to the `PLAY_SWIN`, `PLAY_SWIN_SET` or `PLAY_SWIN_CLR` register, a positive pulse is generated on the `swin_we` signal. The pulse lasts one `play_clk` cycle, starting from the moment the register value is updated (see [Figure 121](#)). The `swin_we` signal is connected to the input interconnect and can be used as a clock enable for any of the logic element output registers. This means that writing to the `PLAY_SWIN` register can trigger a state transition conditioned by the new values on `swin0` to `swin15`, for example.

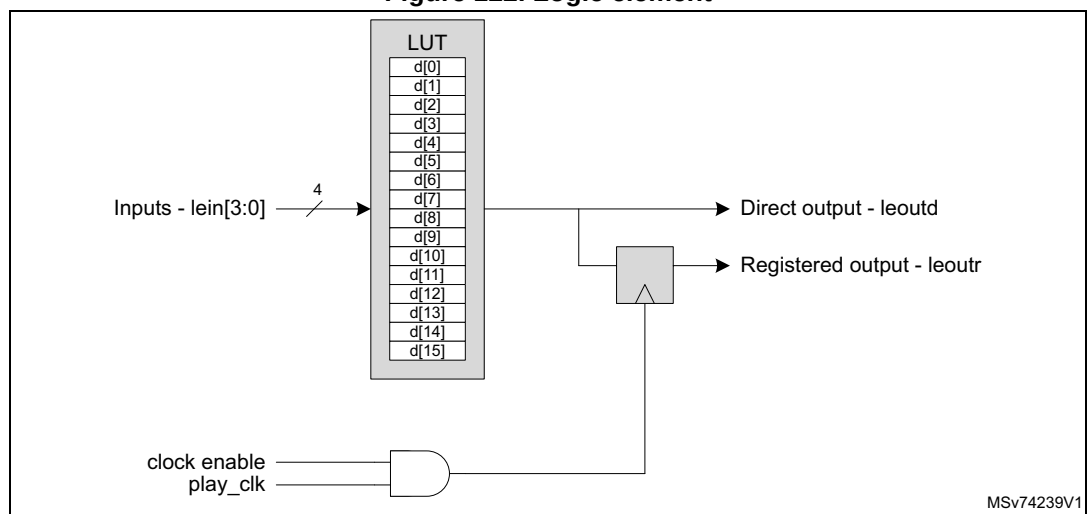
Figure 121. swin_we pulse timing



22.3.7 Logic elements

The logic array provides 16 logic elements (LEs), each containing a 4-input, single output look-up table (LUT) and a register ([Figure 122](#)).

Figure 122. Logic element



Look-up table (LUT)

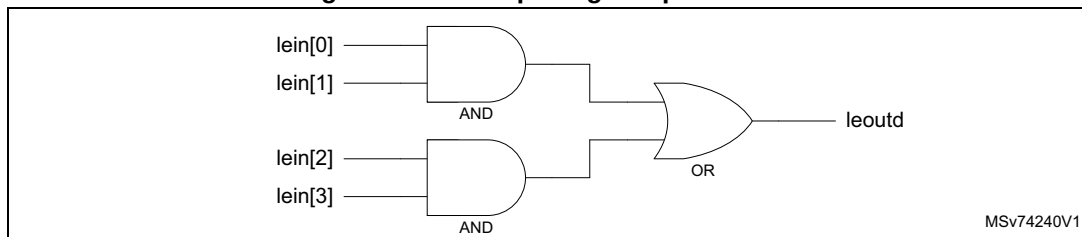
The LUT is a 16 x 1-bit memory which generates a single output, leoutd, whose value is a function of the four LUT inputs, lein[3:0] which make up the address. The LUT is programmed at configuration time with the required state of the output for every possible combination of lein[3:0].

To implement a Boolean logic expression, the full truth table for the expression must be detailed.

This is demonstrated using the following logic expression as an example:

$$\text{leoutd} = \text{lein}[0] \text{ AND } \text{lein}[1] \text{ OR } \text{lein}[2] \text{ AND } \text{lein}[3]$$

This expression corresponds to the diagram in [Figure 123](#).

Figure 123. Example logic expression 1

The truth table is given in [Table 185](#).

Table 185. Truth table example 1

Inputs - lein[3:0]	Output - leoutd
0000	0
0001	0
0010	0
0011	1
0100	0
0101	0
0110	0
0111	1
1000	0
1001	0
1010	0
1011	1
1100	1
1101	1
1110	1
1111	1

The left hand column lists the address formed by the combination of the four LUT inputs, lein[3:0], and the right hand column lists the required output value on leoutd for each input combination.

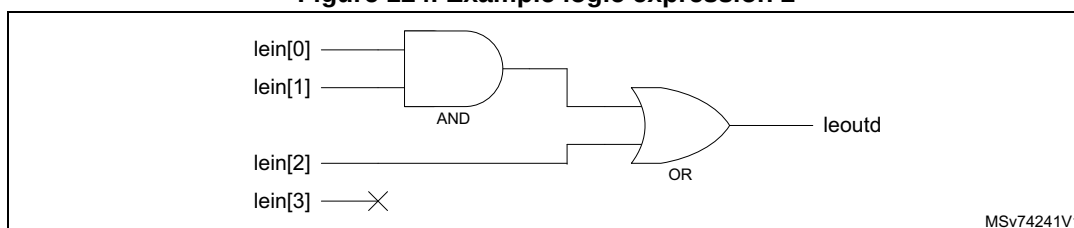
During configuration, the LUT is programmed as a single 16-bit register, with the data read from the output column of the truth table, read from bottom (MSB) to top (LSB). In this example the LUT configuration register would be programmed with 1111 1000 1000 1000 (0xF888).

If any of the inputs are unused or “don’t care”, the output must be the same for both possible values of the unused input. In other words, the contents of address 0bxnnn must be the same for x = 0 or x = 1.

For example, in the following logic expression, lein[3] is unused:

leoutd = lein[0] AND lein[1] OR lein[2]

Figure 124. Example logic expression 2



The truth table for this expression is shown in [Table 186](#). To obtain the full LUT, each row containing an 'x' is split into two identical rows. Then the 'x' is replaced by '0' and '1' respectively.

In this example the configuration register is programmed with 1111 1000 1111 1000 (0xF8F8).

Table 186. Truth table example 2

lein[3:0]	leoutd
x000	0
x001	0
x010	0
x011	1
x100	1
x101	1
x110	1
x111	1

Alternatively, the unused input can be connected to an unused input with value 0 (see [Table 187](#)), so that addresses with x = 1 are never accessed and can be left at 0.

The LUT configuration registers are directly mapped in the PLAY APB address space. The LUT configuration for logic element *n* is programmed in the register bit field `PLAY_LEnCFG1.LUT[15:0]`.

LE output register

The LUT output can be direct or registered. Output registers are clocked by the `play_clk`, optionally qualified by a clock enable signal. When the clock enable signal is high, the register is updated with the LUT output every `play_clk` cycle. A clock enable must be high for at least one `play_clk` cycle. This constraint is guaranteed for input signals using edge detection or filtering (see [Section 22.3.5: Input filtering and edge detection](#)). It is also guaranteed for the synchronous feedback signals `leoutr[15:0]`.

Caution: Asynchronous signals must not be selected as clock enables.

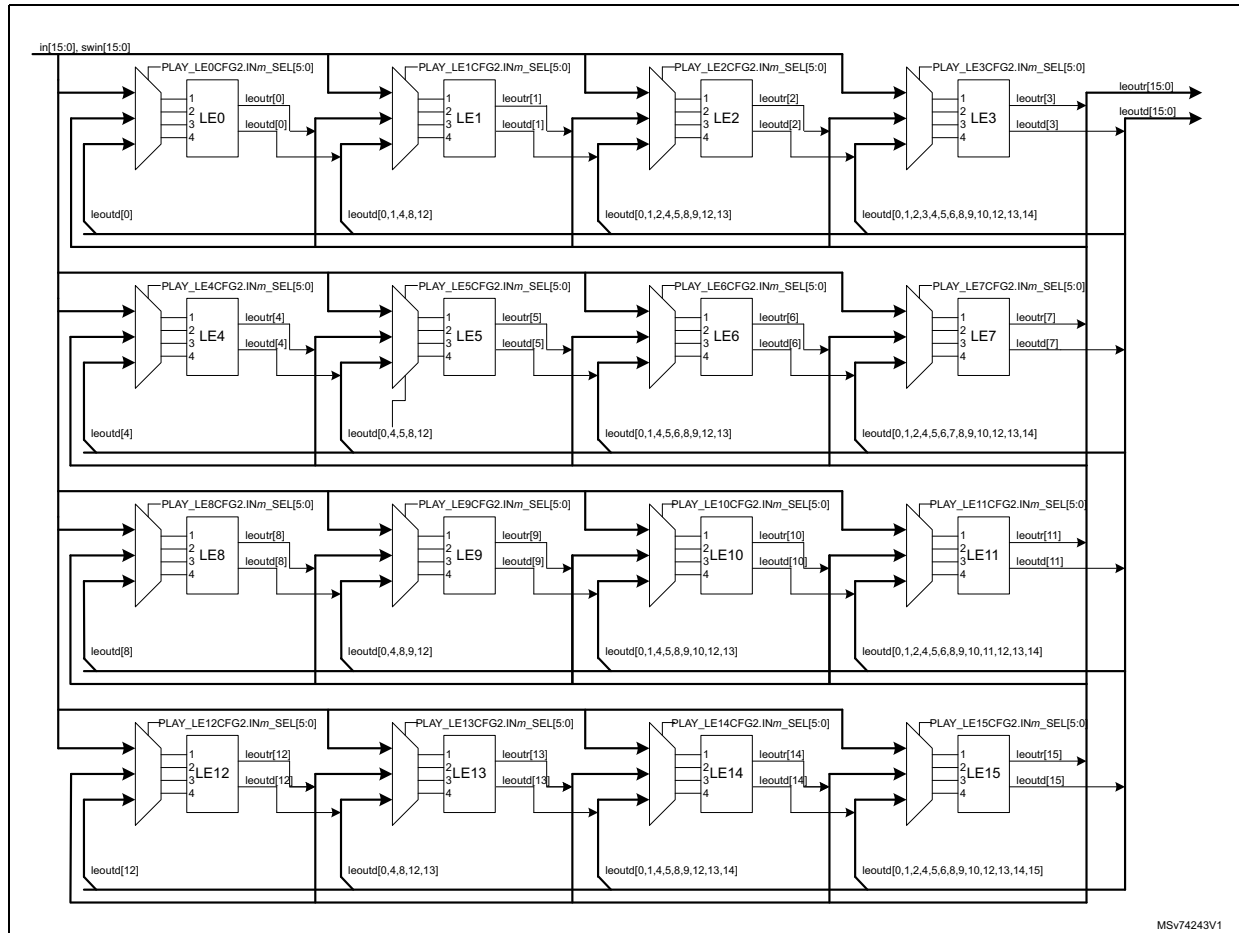
During configuration (when `PLAY_CFGCR.UNLOCK = 1`), the logic element registers are held in reset.

Logic element registers are also reset by `play_rstn` and `play_app_rstn`.

22.3.8 Input interconnect

The input interconnect selects which signals are connected to each logic element, as shown in [Figure 125](#).

Figure 125. Input interconnect



MSV74243V1

LUT input selection

For each logic element, up to four input signals can be configured as LUT inputs, and one as a clock enable for the LE register. The LUT inputs can be selected from among the following signals:

- in[15:0] - 16 inputs from I/Os or other functional blocks, via the filter and edge detection unit
- swin[15:0] - 16 register bits (PLAY_SWIN register)
- leoutr[15:0] - 16 feedback signals from LE registered outputs
- logic 0

Each logic element's direct output can be fed back to one of its inputs. This allows implementation of asynchronous latches.

It is also possible to connect the direct outputs of certain logic elements to the inputs of others - see [Combining logic elements](#).

For logic element n (0 to 15), the signals connected to input m (0 to 3), are selected by configuration register `PLAY_LEnCFG2.INm_SEL[5:0]`.

The selection of the LUT input signals is summarized in [Table 187](#).

For example, to connect input signal `in15` to input 2 of logic element 0, program `PLAY_LE0CFG2.IN2_SEL[5:0] = 47`. To connect input 3 of logic element 1 to the direct output of logic element 0, `leoutd0`, program `PLAY_LE1CFG2.IN3_SEL[5:0] = 0`.

Table 187. Logic element n LUT input selection

<code>PLAY_LEnCFG2.INm_SEL[5:0]</code>	Signal connected to LUT input m	Source
0	For $n = 4, 8, 12$: 0	-
	For $n = 0, 1, 2, 3, 5, 6, 7, 9, 10, 11, 13, 14, 15$: <code>leoutd[0]</code>	LE 0 direct output
1	For $n = 0, 4, 5, 8, 9, 12, 13$: 0	-
	For $n = 1, 2, 3, 6, 7, 10, 11, 14, 15$: <code>leoutd[1]</code>	LE 1 direct output
2	For $n = 0, 1, 4, 5, 6, 8, 9, 10, 12, 13, 14$: 0	-
	For $n = 2, 3, 7, 11, 15$: <code>leoutd[2]</code>	LE 2 direct output
3	For $n = 0, 1, 2, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15$: 0	-
	For $n = 3$: <code>leoutd[3]</code>	LE 3 direct output
4	For $n = 0, 8, 12$: 0	-
	For $n = 1, 2, 3, 4, 5, 6, 7, 9, 10, 11, 13, 14, 15$: <code>leoutd[4]</code>	LE 4 direct output
5	For $n = 0, 1, 4, 8, 9, 12, 13$: 0	-
	For $n = 2, 3, 5, 6, 7, 10, 11, 14, 15$: <code>leoutd[5]</code>	LE 5 direct output
6	For $n = 0, 1, 2, 4, 5, 8, 9, 10, 12, 13, 14$: 0	-
	For $n = 3, 6, 7, 11, 15$: <code>leoutd[6]</code>	LE 6 direct output
7	For $n = 0, 1, 2, 3, 4, 5, 6, 8, 9, 10, 11, 12, 13, 14, 15$: '0'	-
	For $n = 7$: <code>leoutd[7]</code>	LE 7 direct output
8	For $n = 0, 4, 12$: 0	-
	For $n = 1, 2, 3, 5, 6, 7, 8, 9, 10, 11, 13, 14, 15$: <code>leoutd[8]</code>	LE 8 direct output
9	For $n = 0, 1, 4, 5, 8, 12, 13$: 0	-
	For $n = 2, 3, 6, 7, 9, 10, 11, 14, 15$: <code>leoutd[9]</code>	LE 9 direct output
10	For $n = 0, 1, 2, 4, 5, 6, 8, 9, 12, 13, 14, 15$: 0	-
	For $n = 3, 7, 10, 11$: <code>leoutd[10]</code>	LE 10 direct output
11	For $n = 0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 12, 13, 14, 15$: 0	-
	For $n = 11$: <code>leoutd[11]</code>	LE 11 direct output
12	For $n = 0, 4, 8$: 0	-
	For $n = 1, 2, 3, 5, 6, 7, 9, 10, 11, 12, 13, 14, 15$: <code>leoutd[12]</code>	LE 12 direct output
13	For $n = 0, 1, 4, 5, 8, 12$: 0	-
	For $n = 2, 3, 6, 7, 10, 11, 13, 14, 15$: <code>leoutd[13]</code>	LE 13 direct output

Table 187. Logic element *n* LUT input selection (continued)

PLAY_LEnCFG2.INm_SEL [5:0]	Signal connected to LUT input <i>m</i>	Source
14	For <i>n</i> = 0,1,2,4,5,6,8,9,10,12,13: 0	-
	For <i>n</i> = 3,7,11,14,15: leoutd[14]	LE 14 direct output
15	For <i>n</i> = 0,1,2,3,4,5,6,7,8,9,10,11,12,13,14: 0	-
	For <i>n</i> = 15: leoutd[15]	LE 15 direct output
16 to 31	leoutr[0] to leoutr[15]	LE registered outputs
32 to 47	in0 to in15	input signals
48 to 63	swin0 to swin15	PLAY_SWIN

LE clock enable selection

The LE register clock enable can be selected from any of these signals:

- in[15:0] - 16 inputs from I/Os or other functional blocks, via the filter and edge detection
- swin[15:0] - 16 register bits (PLAY_SWIN register)
- swin_we - pulse indicating that a write to the PLAY_SWIN register has taken place
- leoutr[15:0] - 16 feedback signals from LE registered outputs
- 0- clock disabled
- 1- always enabled

For logic element *n* the clock enable is selected by configuration register bits PLAY_LEnCFG2.CK_SEL[5:0].

The selection of the LE register clock is summarized in [Table 188](#).

Table 188. Logic element *n* register clock selection

PLAY_LEnCFG2.CK_SEL[5:0]	Clock enable signal ⁽¹⁾	Source
0	0 (clock disabled)	-
1	1 (always enabled)	-
2	swin_we	APB interface
3 to 15	reserved	-
16 to 31	leoutr0 to leoutr15 (excluding leoutrn)	LE registered outputs
32 to 47	in0 to in15	input signals
48 to 63	swin0 to swin15	PLAY_SWIN

1. Reserved values select 0

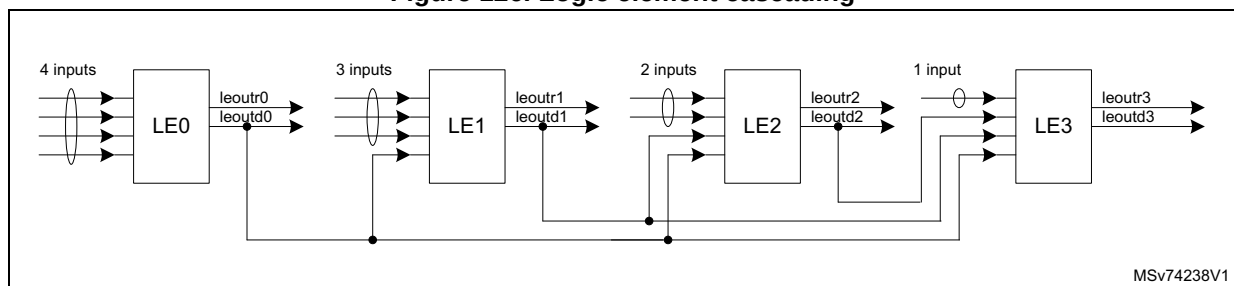
Combining logic elements

LE 0-3 form a group which can be cascaded combinatorially from left to right (see [Figure 126](#)):

- LE0 direct output can be connected to LE1-3 inputs
- LE1 direct output can be connected to LE2-3 inputs
- LE2 direct output can be connected to LE3 inputs

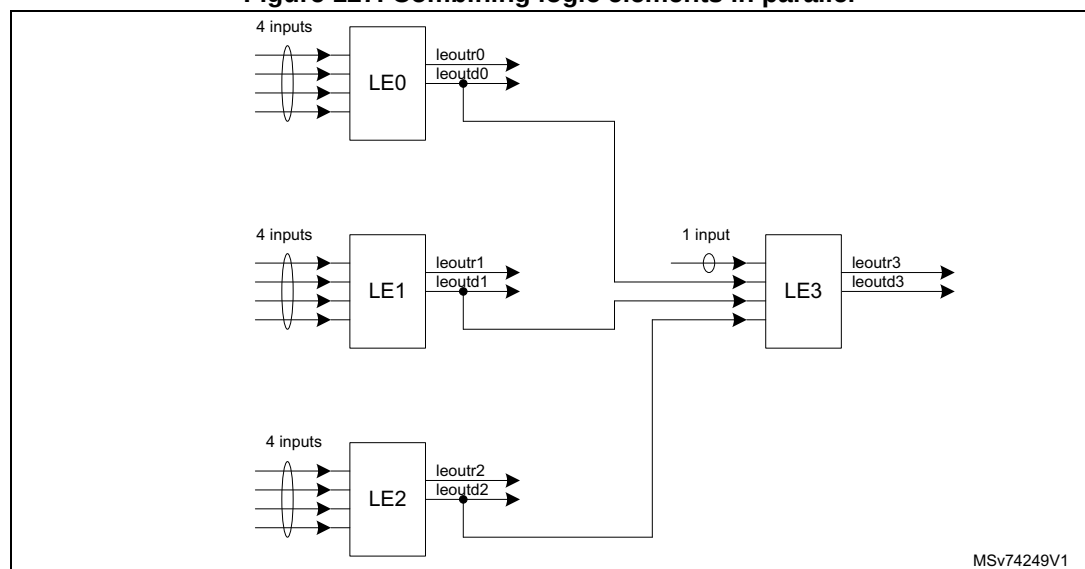
The other groups of four (LE 4-7, LE 8-11, LE12-15) can be cascaded in the same way. Furthermore, logic elements from different groups can be connected together, provided the left to right logic flow is respected. For example, LE4 direct output can be connected to LE1 input, but not to LE0 input.

Figure 126. Logic element cascading



It is also possible to use one logic element to combine the outputs of up to three others, as shown in [Figure 127](#). In this example, the direct outputs of LE0-2 are connected to the inputs of LE3. This allows up to 13 different inputs to be combined in a single logic expression.

Figure 127. Combining logic elements in parallel



Note that combining logic elements in this way requires the logic expression to be split into partial expressions, with up to four inputs each.

For example, consider the following expression:

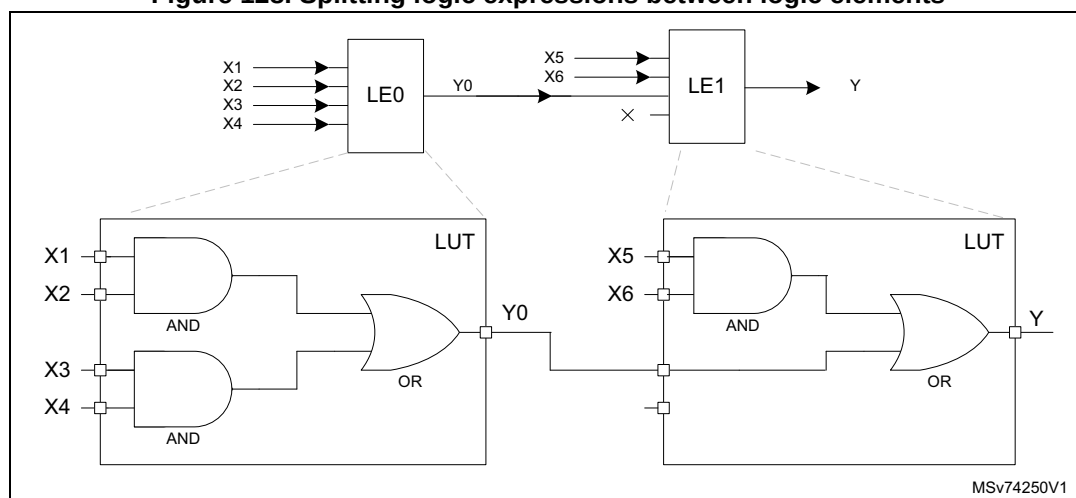
$$Y = (X1 \text{ AND } X2) \text{ OR } (X3 \text{ AND } X4) \text{ OR } (X5 \text{ AND } X6).$$

This expression has six inputs (X1 to X6) and one output (Y). It therefore requires at least two logic elements, for example LE0 and LE1, as illustrated in [Figure 128](#). The expression for Y can be split into two expressions:

- a) $Y0 = (X1 \text{ AND } X2) \text{ OR } (X3 \text{ AND } X4)$
- b) $Y = Y0 \text{ OR } (X5 \text{ AND } X6)$

Then LE0 is configured with expression 'a', and LE1 with expression 'b'. Inputs X1 to X4 are connected to LE0. Inputs X5 and X6, are connected to LE1, together with Y0, the direct output of LE0.

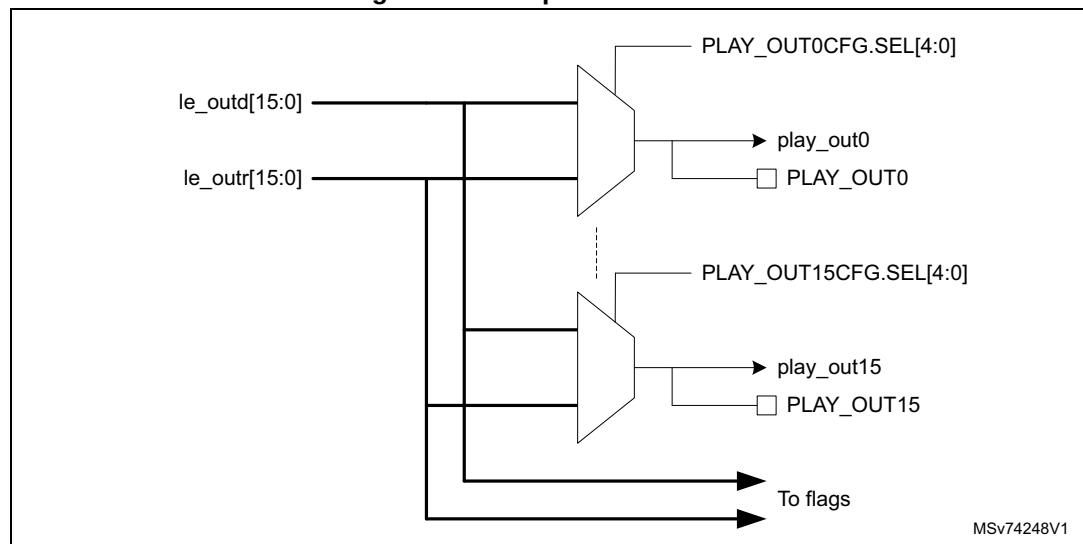
Figure 128. Splitting logic expressions between logic elements



22.3.9 Output interconnect

The output interconnect selects which logic element or eFPGA outputs are connected to which PLAY outputs, as shown in [Figure 129](#).

Figure 129. Output interconnect



The PLAY outputs include the following signals:

- `play_out[15:0]` - 16 outputs to I/Os or other functional blocks

Each output can be connected to any one of the logic element outputs, registered or direct. The signal connected to output *n* is selected by configuration register bits `PLAY_OUTnCFG.SEL[4:0]`.

The output selection is summarized in [Table 189](#).

Table 189. Output n source selection

PLAY_OUTnCFG.SEL[4:0]	Source
0	leoutd0 - LE0 direct output
1	leoutd1 - LE1 direct output
2	leoutd2 - LE2 direct output
3	leoutd3 - LE3 direct output
4	leoutd4 - LE4 direct output
5	leoutd5 - LE5 direct output
6	leoutd6 - LE6 direct output
7	leoutd7 - LE7 direct output
8	leoutd8 - LE8 direct output
9	leoutd9 - LE9 direct output
10	leoutd10 - LE10 direct output

Table 189. Output n source selection (continued)

PLAY_OUTnCFG.SEL[4:0]	Source
11	leoutd11 - LE11 direct output
12	leoutd12 - LE12 direct output
13	leoutd13 - LE13 direct output
14	leoutd14 - LE14 direct output
15	leoutd15 - LE15 direct output
16	leoutr0 - LE0 register output
17	leoutr1 - LE1 register output
18	leoutr2 - LE2 register output
19	leoutr3 - LE3 register output
20	leoutr4 - LE4 register output
21	leoutr5 - LE5 register output
22	leoutr6 - LE6 register output
23	leoutr7 - LE7 register output
24	leoutr8 - LE8 register output
25	leoutr9 - LE9 register output
26	leoutr10 - LE10 register output
27	leoutr11 - LE11 register output
28	leoutr12 - LE12 register output
29	leoutr13 - LE13 register output
30	leoutr14 - LE14 register output
31	leoutr15 - LE15 register output

22.3.10 Flags

A transition on any of the logic element outputs leoutd[15:0] or leoutr[15:0] can set the corresponding flag in the PLAY_FLSTAT register.

The direction of the transition (rising or falling) can be selected in the PLAY_FLCTL register. Once set, the flag can only be reset by software writing to the PLAY_FLCLR register (or by a reset). Setting a flag generates an interrupt to the CPU on the play_irq signal, if the enable bit for the flag is set in the PLAY_FLIER register. To clear the interrupt the software must reset the flag, or mask it by resetting the enable.

Due to the asynchronous relationship between play_clk and the pclk, a resynchronisation delay is incurred between a transition on the logic element output and the setting of the corresponding flag. The maximum delay is approximately one period of play_clk plus three periods of pclk. The play_irq interrupt is not asserted until the flag goes high. Note that if successive transitions occur on the same output before the flag is cleared, they are not taken into account. However if a subsequent transition on another logic element output occurs during the resynchronization delay, a second resynchronization starts as soon as the first is finished, at the end of which the second flag is set.

The raw state of the signals can also be read by software via the PLAY_OSR register. This is primarily intended for debugging the programmed logic. The user must be aware that the logic element outputs can change asynchronously and the value that is read from PLAY_OSR may not correspond to the actual state of the outputs at the moment of reading.

22.3.11 Miscellaneous status bits

The software input register (PLAY_SWIN), and the flag edge control register (PLAY_FLCTL) are in the play_clk domain. When a write occurs to one of these registers (including to PLAY_SWINSET/CLR), the write data is stored temporarily in the APB bus clock (pclk) domain until the next play_clk pulse at which point the data is transferred to the target register. If play_clk is slow compared to pclk, or if it is inactive, the update could take an indefinitely long time to complete. To avoid stalling the processor, any subsequent writes to the same register before the target register update has completed, are ignored.

To ensure that updates are not lost, the PLAY_MSR register contains two read-only status bits, SWIN write busy flag (SWIN_WBFS) and FLCTL write busy flag (FLCTL_WBFS). When one of these bits is set to 1, it indicates that an update is pending to the corresponding register, PLAY_SWIN or PLAY_FLCTL. The busy flags are automatically reset to 0 when the update completes.

If play_clk is significantly slower than pclk, it is recommended to poll the busy flag in the PLAY_MSR prior to writing to the corresponding register, to be sure that it has reset to 0 indicating that the previous update has completed.

Alternatively, software can enable the write complete interrupts in the PLAY_IER register to generate an interrupt request when a pending write completes (see [Section 22.5](#)).

22.4 PLAY in low-power modes

The play_clk can be maintained when the system is in low-power mode (Stop mode). The full functionality of the logic array is available, but the memory-mapped registers are not accessible and the flags are not updated.

When the play_clk is stopped, the logic element output registers do not operate, neither does the edge detection and filtering. However combinatorial logic continues to function as programmed.

If a logic element direct output is mapped to an asynchronous wake-up event, a transition on the output can wake the system from Stop mode. It is recommended to configure one of the software programmable inputs to reset the output so that software can deassert the wake-up event, if it is not otherwise deasserted.

In Standby mode, the PLAY is powered down and no longer functional. It must be reconfigured on exit from Standby.

22.5 PLAY interrupts

There is one interrupt request, play_irq, which can be configured to interrupt the processor when any of the following conditions occurs:

1. One or more of the flags are set, as indicated by the PLAY_FLSTAT register
2. An outstanding write to the PLAY_FLCTL register has completed, as indicated by the FLCTLWBFS bit in the PLAY_MSR register.
3. An outstanding write to the PLAY_SWIN register has completed, as indicated by the SWINWBFS bit in the PLAY_MSR register.

An interrupt handler must first read the PLAY interrupt status register (PLAY_ISR) to determine the source of the interrupt.

If the FLCTL write complete (FLCTLWC) or SWIN write complete (SWINWC) bits are set, it means that a previous write respectively to the PLAY_FLCTL or PLAY_SWIN register has completed. The write complete status bits can be cleared by writing a one to the corresponding bit in the interrupt clear (PLAY_ICR) register. The write complete interrupts are enabled and disabled by writing to the PLAY_IER register.

If the PLAY_ISR.FLAGS bit is set, the interrupt handler must read the PLAY_FLSTAT register to determine which flag(s) caused the interrupt. The FLAGS bit is reset by hardware when all the flags are inactive or masked. See [Section 22.3.10](#) for more details.

22.6 PLAY registers

There are three types of PLAY registers:

- Access control registers:
PLAY_SECCFGR
PLAY_PRIVCFGR
These registers control software access rights to the PLAY registers.
- Configuration registers: PLAY_CFGCR, PLAY_FILTxCFGR, PLAY_LExCFG1, PLAY_LExCFG2, PLAY_OUTxCFG
These registers configure the logic elements, input and output interconnects, input multiplexers and filters.
- Functional registers: PLAY_SWIN, PLAY_SWINSET, PLAY_SWINCLR, PLAY_OSR, PLAY_FLSTAT, PLAY_FLSET, PLAY_FLCLR, PLAY_FLIER, PLAY_FLCTL, PLAY_MSR, PLAY_ISR, PLAY_IER, PLAY_ICR
These registers are used by the application software to interact with the logic array during operation.

22.6.1 PLAY configuration control register (PLAY_CFGCR)

Address offset: 0x000

Reset value: 0x0000 0001

This register contains miscellaneous configuration bits. It is a configuration register and is reset by play_rstn.

If PLAY_SECCFGR.SEC[0] = 1, the register requires secure access (nonsecure accesses are ignored).

If PLAY_PRIVCFGR.PRIV[0] = 1, the register requires privileged access (unprivileged accesses are ignored).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	UNLOCK
															rw

Bits 31:1 Reserved, must be kept at reset value.

Bit 0 **UNLOCK**: Unlock configuration

When configuration is complete, it must be set to 0 to prevent accidental corruption of the configuration registers and enable operation of the logic element registers.

0: Configuration is locked. All configuration registers are write protected.

1: Configuration is unlocked. Configuration registers can be programmed.

22.6.2 PLAY security attributes configuration register (PLAY_SECCFGR)

Address offset: 0x004

Reset value: 0x0000 0000

This register controls the security attributes of the PLAY registers. It is an access control register and is reset by play_rstn.

The register can only be modified by a secure, privileged write access. It can be read by any access. A nonsecure write access is ignored. An unprivileged write access is ignored.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	SEC[1:0]	
														rw	rw

Bits 31:2 Reserved, must be kept at reset value.

Bits 1:0 **SEC**[1:0]: Secure access control

- 0: All registers are accessible in nonsecure or secure state
- 1: Configuration registers can only be accessed in secure state. All other registers can be accessed in secure or nonsecure state.
- 2: Reserved
- 3: All registers are accessible in secure state only.

22.6.3 PLAY privilege attributes configuration register (PLAY_PRIVCFGR)

Address offset: 0x008

Reset value: 0x0000 0000

This register controls the privilege attributes of the PLAY registers. It is an access control register and is reset by play_rstn. It can only be modified by a privileged write access. It can be read by any access.

If PLAY_SECFGR.SEC[n] = 1, PLAY_PRIVCFG.PRIV[n] can only be modified by a secure write access. A nonsecure write access to PLAY_PRIVCFG.PRIV[n] is ignored.

If PLAY_SECCFGR.SEC[n] = 0, PLAY_PRIVCFG.PRIV[n] can be modified by secure or nonsecure write access.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	PRIV[1:0]	
														rw	rw

Bits 31:2 Reserved, must be kept at reset value.

Bits 1:0 **PRIV**[1:0]: Privilege access control

- 0: All registers are accessible in unprivileged or privileged state
- 1: Configuration registers can only be accessed in privileged state. All other registers can be accessed in privileged or unprivileged state.
- 2: Reserved
- 3: All registers are accessible in privileged state only.

22.6.4 PLAY filter x configuration register (PLAY_FILTxCFG)

Address offset: $0x020 + 0x04 * x$, ($x = 0$ to 15)

Reset value: $0x0000\ 0000$

This register configures the filter, edge detector and premultiplexer for logic array input x. It is a configuration register and is reset by play_rstn. Write accesses to this register are ignored if PLAY_CFGCR.UNLOCK = 0.

If PLAY_SECCFGR.SEC[0] = 1, the register requires secure access (nonsecure accesses are ignored, and signaled on play_sec_ilac).

If PLAY_PRIVCFGR.PRIV[0] = 1, the register requires privileged access (unprivileged accesses are ignored, and signaled on play_priv_ilac).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	PREMUXSEL[2:0]		
													rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	Res.	Res.	Res.	Res.	EDGEDET[1:0]		WIDTH[7:0]							
						rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bits 31:19 Reserved, must be kept at reset value.

Bits 18:16 **PREMUXSEL[2:0]**: Input pre-multiplexer select

Selects the signal to be connected to the input. See [Table 183](#) for details.

Bits 15:10 Reserved, must be kept at reset value.

Bits 9:8 **EDGEDET[1:0]**: Edge detection mode.

0: Edge detection is bypassed

1: A rising edge transition on the input is converted into a positive pulse of 1 play_clk cycle.

2: A falling edge transition on the input is converted into a positive pulse of 1 play_clk cycle.

3: A rising or falling edge transition on the input is converted into a positive pulse of 1 play_clk cycle.

Bits 7:0 **WIDTH[7:0]**: Minimum pulse width

0: Filter is bypassed.

1 to 255: Minimum pulse width, in play_clk cycles. Any pulse on the input shorter than this is ignored.

22.6.5 PLAY logic element x configuration register 1 (PLAY_LExCFG1)

Address offset: $0x060 + 0x04 * x$, ($x = 0$ to 15)

Reset value: $0x0000\ 0000$

This register configures the look-up table contents for logic element x. It is a configuration register and is reset by `play_rstn`. Write accesses to this register are ignored if `PLAY_CFGCR.UNLOCK = 0`.

If `PLAY_SECCFGR.SEC[0] = 1`, the register requires secure access (nonsecure accesses are ignored).

If `PLAY_PRIVCFGR.PRIV[0] = 1`, the register requires privileged access (unprivileged accesses are ignored).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
LUT[15:0]															
rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW

Bits 31:16 Reserved, must be kept at reset value.

Bits 15:0 **LUT[15:0]**: Look-up table contents.

Truth table output for the logic element.

22.6.6 PLAY logic element x configuration register 2 (PLAY_LExCFG2)

Address offset: $0x0A0 + 0x04 * x$, ($x = 0$ to 15)

Reset value: $0x00FF\ FFFF$

This register configures the input source and the register clock source for logic element x. It is a configuration register and is reset by `play_rstn`. Write accesses to this register are ignored if `PLAY_CFGCR.UNLOCK = 0`.

If `PLAY_SECCFGR.SEC[0] = 1`, the register requires secure access (nonsecure accesses are ignored).

If `PLAY_PRIVCFGR.PRIV[0] = 1`, the register requires privileged access (unprivileged accesses are ignored).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	CK_SEL[5:0]						IN3_SEL[5:0]						IN2_SEL[5:4]	
		rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
IN2_SEL[3:0]				IN1_SEL[5:0]						IN0_SEL[5:0]					
rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW

Bits 31:30 Reserved, must be kept at reset value.

- Bits 29:24 **CK_SEL[5:0]**: Clock enable selection
 Selects source to be used as the clock enable for the logic element register. See [Table 188](#) for details.
 After reset, this bit field is set to 0x0 (disabled)
- Bits 23:18 **IN3_SEL[5:0]**: Logic element input 3 selection
 Selects source for LUT input 3. See [Table 187](#) for details.
 After reset, this bit field is set to 0x1F: PLAY_SWIN[15]
- Bits 17:12 **IN2_SEL[5:0]**: Logic element input 2 selection
 Selects source for LUT input 2. See [Table 187](#) for details.
 After reset, this bit field is set to 0x1F: PLAY_SWIN[15]
- Bits 11:6 **IN1_SEL[5:0]**: Logic element input 1 selection
 Selects source for LUT input 1. See [Table 187](#) for details.
 After reset, this bit field is set to 0x1F: PLAY_SWIN[15]
- Bits 5:0 **IN0_SEL[5:0]**: Logic element input 0 selection
 Selects source for LUT input 0. See [Table 187](#) for details.
 After reset, this bit field is set to 0x1F: PLAY_SWIN[15]

22.6.7 PLAY output x configuration register (PLAY_OUTxCFG)

Address offset: $0x100 + 0x04 * x$, ($x = 0$ to 15)

Reset value: 0x0000 0000

This register selects the source for play_out[x]. It is a configuration register and is reset by play_rstn. Write accesses to this register are ignored if PLAY_CFGCR.UNLOCK = 0.

If PLAY_SECCFGR.SEC[0] = 1, the register requires secure access (nonsecure accesses are ignored).

If PLAY_PRIVCFGR.PRIV[0] = 1, the register requires privileged access (unprivileged accesses are ignored).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	SEL[4:0]				
											rw	rw	rw	rw	rw

Bits 31:5 Reserved, must be kept at reset value.

Bits 4:0 **SEL[4:0]**: Output source selection.

The value of this field selects which LE output is connected to the output. See [Table 189](#) for details.

22.6.8 PLAY software input status register (PLAY_SWIN)

Address offset: 0x200

Reset value: 0x0000 0000

This register is used to read and write the state of the software programmable inputs. It is a functional register and is reset by play_rstn and play_app_rstn.

If PLAY_SECCFGR.SEC[1] = 1, the register requires secure access (nonsecure accesses are ignored, and signaled on play_sec_ilac).

If PLAY_PRIVCFGR.PRIV[1] = 1, the register requires privileged access (unprivileged accesses are ignored).

Caution: play_clk must be active when writing to this register. Any subsequent write access to this register (or PLAY_SWINSET or PLAY_SWINCLR) is ignored until the write completes. Software must poll the PLAY_SWIN write busy flag in the [PLAY miscellaneous status register \(PLAY_MSR\)](#) to make sure that no writes are pending. If play_clk is slow compared to the bus clock, pclk, a write may take a long time to complete. In this case, the PLAY_SWIN register write complete interrupt - see [PLAY interrupt status register \(PLAY_ISR\)](#) - can be used to signal to the software when the registers can be accessed again.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
SWIN1 5	SWIN1 4	SWIN1 3	SWIN1 2	SWIN1 1	SWIN1 0	SWIN9	SWIN8	SWIN7	SWIN6	SWIN5	SWIN4	SWIN3	SWIN2	SWIN1	SWIN0
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bits 31:16 Reserved, must be kept at reset value.

Bits 15:0 **SWIN_y**: Input swiny state (y = 15 to 0)

0: Input low

1: Input high

22.6.9 PLAY software input set register (PLAY_SWINSET)

Address offset: 0x204

Reset value: 0x0000 0000

This register is used to set the state of the software programmable inputs. It is a functional register and is reset automatically by hardware.

If PLAY_SECCFGR.SEC[1] = 1, the register requires secure access (nonsecure accesses are ignored).

If PLAY_PRIVCFGR.PRIV[1] = 1, the register requires privileged access (unprivileged accesses are ignored).

Caution: play_clk must be active when writing to this register. Any subsequent write access to this register (or PLAY_SWIN or PLAY_SWINCLR) is ignored until the write completes. Software must poll the PLAY_SWIN write busy flag in the [PLAY miscellaneous status register \(PLAY_MSR\)](#) to make sure that no writes are pending. If play_clk is slow compared to the

bus clock, pclk, a write may take a long time to complete. In this case, the PLAY_SWIN register write complete interrupt - see [PLAY interrupt status register \(PLAY_ISR\)](#) - can be used to signal to the software when the register can be accessed again.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
SWIN1 5	SWIN1 4	SWIN1 3	SWIN1 2	SWIN1 1	SWIN1 0	SWIN9	SWIN8	SWIN7	SWIN6	SWIN5	SWIN4	SWIN3	SWIN2	SWIN1	SWIN0
w	w	w	w	w	w	w	w	w	w	w	w	w	w	w	w

Bits 31:16 Reserved, must be kept at reset value.

Bits 15:0 **SWINy**: Set PLAY_SWIN.SWINy (y = 15 to 0)

0: Input low

1: Input high

22.6.10 PLAY software input clear register (PLAY_SWINCLR)

Address offset: 0x208

Reset value: 0x0000 0000

This register is used to reset the state of the software programmable inputs. It is a functional register and is reset automatically by hardware.

If PLAY_SECCFGR.SEC[1] = 1, the register requires secure access (nonsecure accesses are ignored).

If PLAY_PRIVCFGR.PRIV[1] = 1, the register requires privileged access (unprivileged accesses are ignored).

Caution: play_clk must be active when writing to this register. Any subsequent write access to this register (or PLAY_SWIN or PLAY_SWINSET) is ignored until the write completes. Software must poll the PLAY_SWIN write busy flag in the [PLAY miscellaneous status register \(PLAY_MSR\)](#) to make sure that no writes are pending. If play_clk is slow compared to the bus clock, pclk, a write may take a long time to complete. In this case, the PLAY_SWIN register write complete interrupt - see [PLAY interrupt status register \(PLAY_ISR\)](#) - can be used to signal to the software when the registers can be accessed again.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
SWIN1 5	SWIN1 4	SWIN1 3	SWIN1 2	SWIN1 1	SWIN1 0	SWIN9	SWIN8	SWIN7	SWIN6	SWIN5	SWIN4	SWIN3	SWIN2	SWIN1	SWIN0
w	w	w	w	w	w	w	w	w	w	w	w	w	w	w	w

Bits 31:16 Reserved, must be kept at reset value.

Bits 15:0 **SWINy**: Reset PLAY_SWIN.SWINy (y = 15 to 0)

0: Input low

1: Input high

22.6.11 PLAY output status register (PLAY_OSR)

Address offset: 0x210

Reset value: 0x0000 0000

This register allows software to read the state of the LE direct and registered outputs. It is a functional register and is reset by play_rstn and play_app_rstn.

If PLAY_SECCFGR.SEC[1] = 1, the register requires secure access (nonsecure accesses are ignored).

If PLAY_PRIVCFGR.PRIV[1] = 1, the register requires privileged access (unprivileged accesses are ignored).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
LEOUT R15	LEOUT R14	LEOUT R13	LEOUT R12	LEOUT R11	LEOUT R10	LEOUT R9	LEOUT R8	LEOUT R7	LEOUT R6	LEOUT R5	LEOUT R4	LEOUT R3	LEOUT R2	LEOUT R1	LEOUT R0
r	r	r	r	r	r	r	r	r	r	r	r	r	r	r	r
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
LEOUT D15	LEOUT D14	LEOUT D13	LEOUT D12	LEOUT D11	LEOUT D10	LEOUT D9	LEOUT D8	LEOUT D7	LEOUT D6	LEOUT D5	LEOUT D4	LEOUT D3	LEOUT D2	LEOUT D1	LEOUT D0
r	r	r	r	r	r	r	r	r	r	r	r	r	r	r	r

Bits 31:16 **LEOUTRy**: State of LE registered output leoutry (y = 15 to 0)

Bits 15:0 **LEOUTDy**: State of LE direct outputs leoutdy (y = 15 to 0)

22.6.12 PLAY flag status register (PLAY_FLSTAT)

Address offset: 0x220

Reset value: 0x0000 0000

This register allows software to read the state of the LE output flags. It is a functional register and is reset by play_rstn and play_app_rstn.

If PLAY_SECCFGR.SEC[1] = 1, the register requires secure access (nonsecure accesses are ignored).

If PLAY_PRIVCFGR.PRIV[1] = 1, the register requires privileged access (unprivileged accesses are ignored).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
FLAGR 15	FLAGR 14	FLAGR 13	FLAGR 12	FLAGR 11	FLAGR 10	FLAGR 9	FLAGR 8	FLAGR 7	FLAGR 6	FLAGR 5	FLAGR 4	FLAGR 3	FLAGR 2	FLAGR 1	FLAGR 0
r	r	r	r	r	r	r	r	r	r	r	r	r	r	r	r
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
FLAGD 15	FLAGD 14	FLAGD 13	FLAGD 12	FLAGD 11	FLAGD 10	FLAGD 9	FLAGD 8	FLAGD 7	FLAGD 6	FLAGD 5	FLAGD 4	FLAGD 3	FLAGD 2	FLAGD 1	FLAGD 0
r	r	r	r	r	r	r	r	r	r	r	r	r	r	r	r

Bits 31:16 **FLAGRy**: Flag state for LEy registered output (y = 15 to 0)

0: Flag reset

1: Flag set

Bits 15:0 **FLAGDy**: Flag state for LEy direct output (y = 15 to 0)

0: Flag reset

1: Flag set

22.6.13 PLAY flag set register (PLAY_FLSET)

Address offset: 0x224

Reset value: 0x0000 0000

This register allows software to set the LE output flags. It is a functional register and is reset automatically by hardware.

If PLAY_SECCFGR.SEC[1] = 1, the register requires secure access (nonsecure accesses are ignored).

If PLAY_PRIVCFGR.PRIV[1] = 1, the register requires privileged access (unprivileged accesses are ignored).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
FLAGR_SET15	FLAGR_SET14	FLAGR_SET13	FLAGR_SET12	FLAGR_SET11	FLAGR_SET10	FLAGR_SET9	FLAGR_SET8	FLAGR_SET7	FLAGR_SET6	FLAGR_SET5	FLAGR_SET4	FLAGR_SET3	FLAGR_SET2	FLAGR_SET1	FLAGR_SET0
w	w	w	w	w	w	w	w	w	w	w	w	w	w	w	w

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
FLAGD_SET15	FLAGD_SET14	FLAGD_SET13	FLAGD_SET12	FLAGD_SET11	FLAGD_SET10	FLAGD_SET9	FLAGD_SET8	FLAGD_SET7	FLAGD_SET6	FLAGD_SET5	FLAGD_SET4	FLAGD_SET3	FLAGD_SET2	FLAGD_SET1	FLAGD_SET0
w	w	w	w	w	w	w	w	w	w	w	w	w	w	w	w

Bits 31:16 **FLAGR_SETy**: Flag set for LEy registered output (y = 15 to 0)

0: No effect

1: Set flag

Bits 15:0 **FLAGD_SETy**: Flag set for LEy direct output (y = 15 to 0)

0: No effect

1: Set flag

22.6.14 PLAY flag clear register (PLAY_FLCLR)

Address offset: 0x228

Reset value: 0x0000 0000

This register allows software to clear the state of the LE output flags. It is a functional register and is reset automatically by hardware.

If PLAY_SECCFGR.SEC[1] = 1, the register requires secure access (nonsecure accesses are ignored).

If PLAY_PRIVCFGR.PRIV[1] = 1, the register requires privileged access (unprivileged accesses are ignored).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
FLAGR_CLR15	FLAGR_CLR14	FLAGR_CLR13	FLAGR_CLR12	FLAGR_CLR11	FLAGR_CLR10	FLAGR_CLR9	FLAGR_CLR8	FLAGR_CLR7	FLAGR_CLR6	FLAGR_CLR5	FLAGR_CLR4	FLAGR_CLR3	FLAGR_CLR2	FLAGR_CLR1	FLAGR_CLR0
w	w	w	w	w	w	w	w	w	w	w	w	w	w	w	w
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
FLAGD_CLR15	FLAGD_CLR14	FLAGD_CLR13	FLAGD_CLR12	FLAGD_CLR11	FLAGD_CLR10	FLAGD_CLR9	FLAGD_CLR8	FLAGD_CLR7	FLAGD_CLR6	FLAGD_CLR5	FLAGD_CLR4	FLAGD_CLR3	FLAGD_CLR2	FLAGD_CLR1	FLAGD_CLR0
w	w	w	w	w	w	w	w	w	w	w	w	w	w	w	w

Bits 31:16 **FLAGR_CLRy**: Flag clear for LEy registered output (y = 15 to 0)

0: No effect

1: Clear flag

Bits 15:0 **FLAGD_CLRy**: Flag clear for LEy direct output (y = 15 to 0)

0: No effect

1: Clear flag

22.6.15 PLAY flag interrupt enable register (PLAY_FLIER)

Address offset: 0x22C

Reset value: 0x0000 0000

This register enables interrupt requests based on LE output flag state. It is a functional register and is reset by play_rstn and play_app_rstn.

If PLAY_SECCFGR.SEC[1] = 1, the register requires secure access (nonsecure accesses are ignored).

If PLAY_PRIVCFGR.PRIV[1] = 1, the register requires privileged access (unprivileged accesses are ignored).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
FLAGR_IEN15	FLAGR_IEN14	FLAGR_IEN13	FLAGR_IEN12	FLAGR_IEN11	FLAGR_IEN10	FLAGR_IEN9	FLAGR_IEN8	FLAGR_IEN7	FLAGR_IEN6	FLAGR_IEN5	FLAGR_IEN4	FLAGR_IEN3	FLAGR_IEN2	FLAGR_IEN1	FLAGR_IEN0
r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
FLAGD_IEN15	FLAGD_IEN14	FLAGD_IEN13	FLAGD_IEN12	FLAGD_IEN11	FLAGD_IEN10	FLAGD_IEN9	FLAGD_IEN8	FLAGD_IEN7	FLAGD_IEN6	FLAGD_IEN5	FLAGD_IEN4	FLAGD_IEN3	FLAGD_IEN2	FLAGD_IEN1	FLAGD_IEN0
r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w

Bits 31:16 **FLAGR_IENy**: Flag interrupt enable for LEy registered output (y = 15 to 0)

0: No interrupt is generated on play_irq

1: An interrupt is generated on play_irq if PLAY_FLSTAT.FLAGRy = 1

Bits 15:0 **FLAGD_IENy**: Flag interrupt enable for LEy direct output (y = 15 to 0)

0: No interrupt is generated on play_irq

1: An interrupt is generated on play_irq if PLAY_FLSTAT.FLAGDy = 1

22.6.16 PLAY flag control register (PLAY_FLCTL)

Address offset: 0x230

Reset value: 0x0000 0000

This register selects the direction of signal transition to set the corresponding LE output flag. It is a functional register and is reset by play_rstn and play_app_rstn.

If PLAY_SECCFGR.SEC[1] = 1, the register requires secure access (nonsecure accesses are ignored).

If PLAY_PRIVCFGR.PRIV[1] = 1, the register requires privileged access (unprivileged accesses are ignored).

Caution: play_clk must be active when writing to this register. Any subsequent write access to this register is ignored until the write completes. Software must poll the PLAY_FLCTL write busy flag in the [PLAY miscellaneous status register \(PLAY_MSR\)](#) to make sure that no writes are pending. If play_clk is slow compared to the bus clock, pclk, a write may take a long time to complete. In this case, the PLAY_FLCTL register write complete interrupt - see [PLAY interrupt status register \(PLAY_ISR\)](#) - can be used to signal to the software when the register can be accessed again.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
FLAGR_EDGE_15	FLAGR_EDGE_14	FLAGR_EDGE_13	FLAGR_EDGE_12	FLAGR_EDGE_11	FLAGR_EDGE_10	FLAGR_EDGE_9	FLAGR_EDGE_8	FLAGR_EDGE_7	FLAGR_EDGE_6	FLAGR_EDGE_5	FLAGR_EDGE_4	FLAGR_EDGE_3	FLAGR_EDGE_2	FLAGR_EDGE_1	FLAGR_EDGE_0
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
FLAGD_EDGE_15	FLAGD_EDGE_14	FLAGD_EDGE_13	FLAGD_EDGE_12	FLAGD_EDGE_11	FLAGD_EDGE_10	FLAGD_EDGE_9	FLAGD_EDGE_8	FLAGD_EDGE_7	FLAGD_EDGE_6	FLAGD_EDGE_5	FLAGD_EDGE_4	FLAGD_EDGE_3	FLAGD_EDGE_2	FLAGD_EDGE_1	FLAGD_EDGE_0
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bits 31:16 **FLAGR_EDGEy**: Edge direction for LEy registered output (y = 15 to 0)

0: FLAGRy is set by a falling edge on leoutry

1: FLAGRy is set by a rising edge on leoutry

Bits 15:0 **FLAGD_EDGEy**: Edge direction for LEy direct output (y = 15 to 0)

0: FLAGDy is set by a falling edge on leoutdy

1: FLAGDy is set by a rising edge on leoutdy

22.6.17 PLAY miscellaneous status register (PLAY_MSR)

Address offset: 0x234

Reset value: 0x0000 0000

This register allows software to read the state of miscellaneous status flags (not including LE output flags). It is a functional register and is reset by play_rstn and play_app_rstn.

If PLAY_SECCFGR.SEC[1] = 1, the register requires secure access (nonsecure accesses are ignored).

If PLAY_PRIVCFGR.PRIV[1] = 1, the register requires privileged access (unprivileged accesses are ignored).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	FLCTL WBFS	SWIN WBFS
														r	r

Bits 31:2 Reserved, must be kept at reset value.

Bit 1 **FLCTLWBFS**: PLAY_FLCTL register write busy flag status

0: PLAY_FLCTL register can be written

1: Write is pending, any new write access to PLAY_FLCTL register is ignored

Bit 0 **SWINWBFS**: PLAY_SWIN register write busy flag status

0: PLAY_SWIN register can be written

1: Write is pending, any new write access to PLAY_SWIN, PLAY_SWINCLR or PLAY_SWINSET registers is ignored

22.6.18 PLAY interrupt status register (PLAY_ISR)

Address offset: 0x238

Reset value: 0x0000 0000

This register allows software to read the state of the interrupt request sources. The PLAY_ISR register must be read first of all when the play_irq is asserted, to identify the source(s) of the interrupt request. It is a functional register and is reset by play_rstn and play_app_rstn.

If PLAY_SECCFGR.SEC[1] = 1, the register requires secure access (nonsecure accesses are ignored).

If PLAY_PRIVCFGR.PRIV[1] = 1, the register requires privileged access (unprivileged accesses are ignored).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	FLAGS	FLCTL WC	SWIN WC
													r	r	r

Bits 31:3 Reserved, must be kept at reset value.

Bit 2 **FLAGS**: Flag interrupts status

0: No flag interrupts active

1: One or more bits in PLAY_FLSTAT register is active and has generated an interrupt request

Bit 1 **FLCTLWC**: PLAY_FLCTL register write complete interrupt status

0: Interrupt request is not pending

1: Interrupt request is pending

Bit 0 **SWINWC**: PLAY_SWIN register write complete interrupt status

0: Interrupt request is not pending

1: Interrupt request is pending

22.6.19 PLAY interrupt clear register (PLAY_ICR)

Address offset: 0x23C

Reset value: 0xFFFF XXXX

This register allows software to clear active interrupt requests, except for those generated by flags - see [Section 22.6.14: PLAY flag clear register \(PLAY_FLCLR\)](#). It is a functional register.

If PLAY_SECCFGR.SEC[1] = 1, the register requires secure access (nonsecure accesses are ignored).

If PLAY_PRIVCFGR.PRIV[1] = 1, the register requires privileged access (unprivileged accesses are ignored).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	FLCTL WC_CLR	SWIN WC_CLR
														w	w

Bits 31:2 Reserved, must be kept at reset value.

Bit 1 **FLCTLWC_CLR**: PLAY_FLCTL register write complete interrupt clear

0: No action

1: Clear interrupt request

Bit 0 **SWINWC_CLR**: PLAY_SWIN register write complete interrupt clear

0: No action

1: Clear interrupt request

22.6.20 PLAY interrupt enable register (PLAY_IER)

Address offset: 0x240

Reset value: 0x0000 0000

This register allows software to enable or disable interrupt requests, except for those generated by flags - see [Section 22.6.15: PLAY flag interrupt enable register \(PLAY_FLIER\)](#). It is a functional register and is reset by play_rstn and play_app_rstn.

If PLAY_SECCFGR.SEC[1] = 1, the register requires secure access (nonsecure accesses are ignored).

If PLAY_PRIVCFGR.PRIV[1] = 1, the register requires privileged access (unprivileged accesses are ignored).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	FLCTL WC_IEN	SWIN WC_IEN
														rw	rw

Bits 31:2 Reserved, must be kept at reset value.

Bit 1 **FLCTLWC_IEN**: PLAY_FLCTL register write complete interrupt enable

0: Interrupt disabled

1: Interrupt enabled. An interrupt is generated when a write to the PLAY_FCTL register completes

Bit 0 **SWINWC_IEN**: PLAY_SWIN register write complete interrupt enable

0: Interrupt disabled

1: Interrupt enabled. An interrupt is generated when a write to the PLAY_SWIN register completes

22.7 PLAY register map

Table 190. PLAY register map and reset values

Offset	Register name	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0x000	PLAY_CFGCR	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	UNLOCK
	Reset value																																1
0x004	Reserved	Res.																															

Table 190. PLAY register map and reset values (continued)

Offset	Register name	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0			
0x004	PLAY_SECCFGR	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	SEC[1:0]			
	Reset value																															0	0			
0x008	Reserved	Res.																																		
0x008	PLAY_PRIVCFGR	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	PRIV[1:0]			
	Reset value																															0	0			
0x00C to 0x01C	Reserved	Res.																																		
0x020 + 0x4 * x, (x=0 to 15) Last address: 0x05C	PLAY_FILTxCFG	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	PREMUXE L [2:0]		Res.	Res.	Res.	Res.	Res.	Res.	Res.	EDGEDET [1:0]	WIDTH [7:0]											
	Reset value														0	0	0							0	0	0	0	0	0	0	0	0	0			
0x060 + 0x4 * x, (x=0 to 15) Last address: 0x09C	PLAY_LExCFG1	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	LUT[15:0]																		
	Reset value																	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0			
0x0A0 + 0x4 * x, (x=0 to 15) Last address: 0x0DC	PLAY_LExCFG2	Res.	Res.	CK_SEL[4:0]				IN3_SEL[5:0]				IN2_SEL[5:0]				IN1_SEL[5:0]				IN0_SEL[5:0]																
	Reset value			0	0	0	0	0	0	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1			
0x0E0 - 0x0FC	Reserved	Res.																																		
0x100 + 0x4 * x, (x=0 to 15) Last address: 0x13C	PLAY_OUTxCFG	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	SEL[4:0]						
	Reset value																												0	0	0	0	0			
0x140 - 0x1FC	Reserved	Res.																																		
0x200	PLAY_SWIN	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	SWIN15	SWIN14	SWIN13	SWIN12	SWIN11	SWIN10	SWIN9	SWIN8	SWIN7	SWIN6	SWIN5	SWIN4	SWIN3	SWIN2	SWIN1	SWIN0			
	Reset value																	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0			
0x204	PLAY_SWINSET	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	SWIN15	SWIN14	SWIN13	SWIN12	SWIN11	SWIN10	SWIN9	SWIN8	SWIN7	SWIN6	SWIN5	SWIN4	SWIN3	SWIN2	SWIN1	SWIN0			
	Reset value																	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0			
0x208	PLAY_SWINCLR	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	SWIN15	SWIN14	SWIN13	SWIN12	SWIN11	SWIN10	SWIN9	SWIN8	SWIN7	SWIN6	SWIN5	SWIN4	SWIN3	SWIN2	SWIN1	SWIN0			
	Reset value																	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0			
0x20C	Reserved	Res.																																		
0x210	PLAY_OSR	LEOUTR15	LEOUTR14	LEOUTR13	LEOUTR12	LEOUTR11	LEOUTR10	LEOUTR9	LEOUTR8	LEOUTR7	LEOUTR6	LEOUTR5	LEOUTR4	LEOUTR3	LEOUTR2	LEOUTR1	LEOUTR0	LEOUTD15	LEOUTD14	LEOUTD13	LEOUTD12	LEOUTD11	LEOUTD10	LEOUTD9	LEOUTD8	LEOUTD7	LEOUTD6	LEOUTD5	LEOUTD4	LEOUTD3	LEOUTD2	LEOUTD1	LEOUTD0			
	Reset value	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0			
0x214 - 0x21C	Reserved	Res.																																		

Table 190. PLAY register map and reset values (continued)

Offset	Register name	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0x220	PLAY_FLSTAT	FLAGR15	FLAGR14	FLAGR13	FLAGR12	FLAGR11	FLAGR10	FLAGR9	FLAGR8	FLAGR7	FLAGR6	FLAGR5	FLAGR4	FLAGR3	FLAGR2	FLAGR1	FLAGR0	FLAGD15	FLAGD14	FLAGD13	FLAGD12	FLAGD11	FLAGD10	FLAGD9	FLAGD8	FLAGD7	FLAGD6	FLAGD5	FLAGD4	FLAGD3	FLAGD2	FLAGD1	FLAGD0
	Reset value	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0x224	PLAY_FLSET	FLAGR_SET15	FLAGR_SET14	FLAGR_SET13	FLAGR_SET12	FLAGR_SET11	FLAGR_SET10	FLAGR_SET9	FLAGR_SET8	FLAGR_SET7	FLAGR_SET6	FLAGR_SET5	FLAGR_SET4	FLAGR_SET3	FLAGR_SET2	FLAGR_SET1	FLAGR_SET0	FLAGD_SET15	FLAGD_SET14	FLAGD_SET13	FLAGD_SET12	FLAGD_SET11	FLAGD_SET10	FLAGD_SET9	FLAGD_SET8	FLAGD_SET7	FLAGD_SET6	FLAGD_SET5	FLAGD_SET4	FLAGD_SET3	FLAGD_SET2	FLAGD_SET1	FLAGD_SET0
	Reset value	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0x228	PLAY_FLCLR	FLAGR_CLR15	FLAGR_CLR14	FLAGR_CLR13	FLAGR_CLR12	FLAGR_CLR11	FLAGR_CLR10	FLAGR_CLR9	FLAGR_CLR8	FLAGR_CLR7	FLAGR_CLR6	FLAGR_CLR5	FLAGR_CLR4	FLAGR_CLR3	FLAGR_CLR2	FLAGR_CLR1	FLAGR_CLR0	FLAGD_CLR15	FLAGD_CLR14	FLAGD_CLR13	FLAGD_CLR12	FLAGD_CLR11	FLAGD_CLR10	FLAGD_CLR9	FLAGD_CLR8	FLAGD_CLR7	FLAGD_CLR6	FLAGD_CLR5	FLAGD_CLR4	FLAGD_CLR3	FLAGD_CLR2	FLAGD_CLR1	FLAGD_CLR0
	Reset value	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0x22C	PLAY_FLIER	FLAGR_IEN15	FLAGR_IEN14	FLAGR_IEN13	FLAGR_IEN12	FLAGR_IEN11	FLAGR_IEN10	FLAGR_IEN9	FLAGR_IEN8	FLAGR_IEN7	FLAGR_IEN6	FLAGR_IEN5	FLAGR_IEN4	FLAGR_IEN3	FLAGR_IEN2	FLAGR_IEN1	FLAGR_IEN0	FLAGD_IEN15	FLAGD_IEN14	FLAGD_IEN13	FLAGD_IEN12	FLAGD_IEN11	FLAGD_IEN10	FLAGD_IEN9	FLAGD_IEN8	FLAGD_IEN7	FLAGD_IEN6	FLAGD_IEN5	FLAGD_IEN4	FLAGD_IEN3	FLAGD_IEN2	FLAGD_IEN1	FLAGD_IEN0
	Reset value	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0x230	PLAY_FLCTL	FLAGR_EDGE15	FLAGR_EDGE14	FLAGR_EDGE13	FLAGR_EDGE12	FLAGR_EDGE11	FLAGR_EDGE10	FLAGR_EDGE9	FLAGR_EDGE8	FLAGR_EDGE7	FLAGR_EDGE6	FLAGR_EDGE5	FLAGR_EDGE4	FLAGR_EDGE3	FLAGR_EDGE2	FLAGR_EDGE1	FLAGR_EDGE0	FLAGD_EDGE15	FLAGD_EDGE14	FLAGD_EDGE13	FLAGD_EDGE12	FLAGD_EDGE11	FLAGD_EDGE10	FLAGD_EDGE9	FLAGD_EDGE8	FLAGD_EDGE7	FLAGD_EDGE6	FLAGD_EDGE5	FLAGD_EDGE4	FLAGD_EDGE3	FLAGD_EDGE2	FLAGD_EDGE1	FLAGD_EDGE0
	Reset value	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0x234	PLAY_MSR	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
	Reset value																																
0x238	PLAY_ISR	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
	Reset value																																
0x23C	PLAY_ICR	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
	Reset value																																
0x240	PLAY_IER	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
	Reset value																																
0x244 - 0x3EC	Reserved	Res.																															

Refer to [Section 2.3: Memory organization](#) for the register boundary addresses.

23 Flexible memory controller (FMC)

23.1 FMC introduction

The flexible memory controller (FMC) includes three memory controllers:

- The NOR/PSRAM memory controller
- The NAND memory controller
- The Synchronous DRAM (SDRAM/Mobile LPDDR SDRAM) controller (not available on STM32H523/533xx devices)

23.2 FMC main features

The FMC functional block makes the interface with: synchronous and asynchronous static memories, SDRAM memories, and NAND flash memory. Its main purposes are:

- to translate AHB transactions into the appropriate external device protocol
- to meet the access time requirements of the external memory devices

All external memories share the addresses, data and control signals with the controller. Each external device is accessed by means of a unique chip select. The FMC performs only one access at a time to an external device.

The main features of the FMC controller are the following:

- Interface with static-memory mapped devices including:
 - Static random access memory (SRAM)
 - NOR flash memory/OneNAND flash memory
 - PSRAM (4 memory banks)
 - Ferroelectric RAM (FRAM)
 - NAND flash memory with ECC hardware to check up to 8 Kbytes of data
- Interface with synchronous DRAM (SDRAM/Mobile LPDDR SDRAM) memories
- Interface with parallel LCD modules, supporting Intel 8080 and Motorola 6800 modes.
- Burst mode support for faster access to synchronous devices such as NOR flash memory, PSRAM and SDRAM)
- Programmable continuous clock output for asynchronous and synchronous accesses
- 8-,16-bit wide data bus
- Independent chip select control for each memory bank
- Independent configuration for each memory bank
- Write enable and byte lane select outputs for use with PSRAM, SRAM and SDRAM devices
- External asynchronous wait control
- Write FIFO with 16 x32-bit depth
- Cacheable Read FIFO with 6 x32-bit depth (6 x14-bit address tag) for SDRAM controller.

The Write FIFO is common to all memory controllers and consists of:

- a Write Data FIFO which stores the AHB data to be written to the memory (up to 32 bits) plus one bit for the AHB transfer (burst or not sequential mode)
- a Write Address FIFO which stores the AHB address (up to 28 bits) plus the AHB data size (up to 2 bits). When operating in burst mode, only the start address is stored except when crossing a page boundary (for PSRAM and SDRAM). In this case, the AHB burst is broken into two FIFO entries.

At startup the FMC pins must be configured by the user application. The FMC I/O pins which are not used by the application can be used for other purposes.

The FMC registers that define the external device type and associated characteristics are usually set at boot time and do not change until the next reset or power-up.

However, only a few bits can be changed on-the-fly:

- MBKEN, FMCEN, WEN bits in FMC_BCRx register
- ECCEN and PBKEN bits in the FMC_PCR register
- IFS, IRS and ILS bits in the FMC_SR register

Follow the below sequence to modify parameters while the FMC is enabled:

1. First disable the FMC controller to prevent further accesses to any memory controller while the register is modified.
2. Update all required configurations.
3. Enable the FMC controller again.

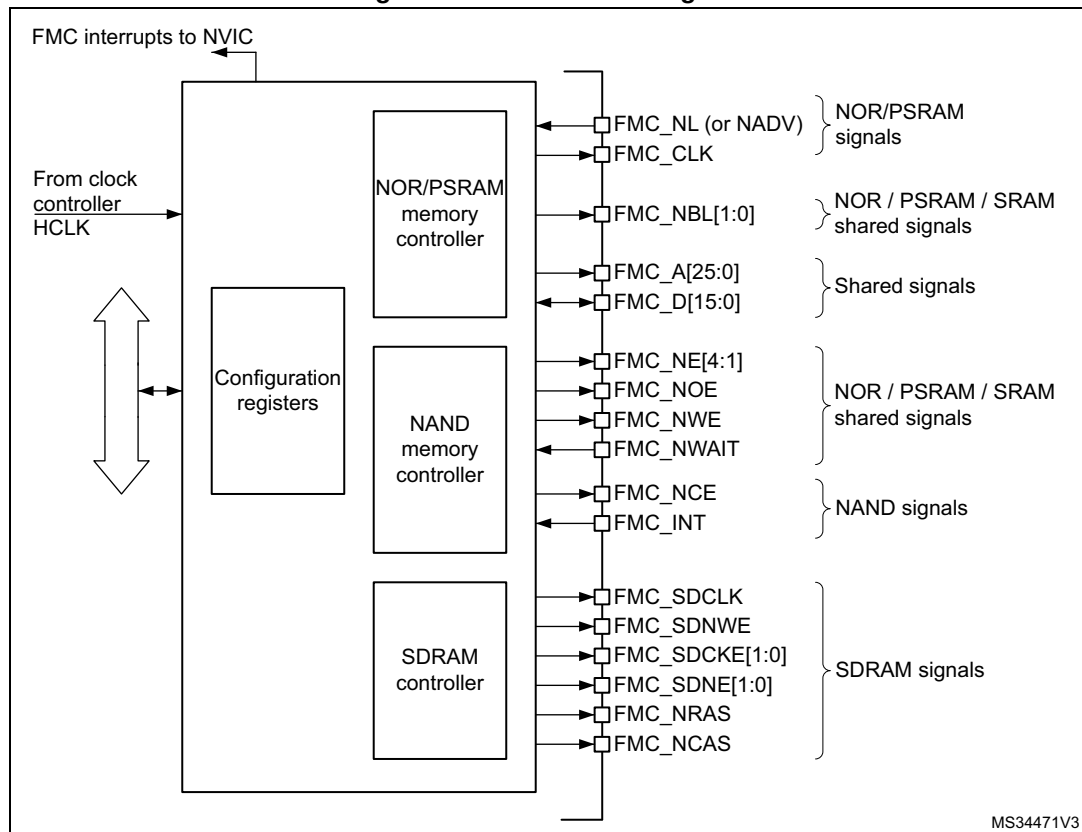
23.3 FMC block diagram

The FMC consists of the following main blocks:

- The AHB interface (including the FMC configuration registers)
- The NOR flash/PSRAM/SRAM controller
- The SDRAM controller

The block diagram is shown in the figure below.

Figure 130. FMC block diagram



23.4 AHB interface

The AHB slave interface allows internal CPUs and other bus master peripherals to access the external memories.

AHB transactions are translated into the external device protocol. In particular, if the selected external memory is 16- or 8-bit wide, 32-bit wide transactions on the AHB are split into consecutive 16- or 8-bit accesses. The FMC chip select (FMC_NEx) does not toggle between the consecutive accesses except in case of Access mode D when the Extended mode is enabled.

The FMC generates an AHB error in the following conditions:

- When reading or writing to a FMC bank (Bank 1 to 4) which is not enabled.
- When reading or writing to the NOR flash bank while the FACCEN bit is reset in the FMC_BCRx register.
- When writing to a write protected SDRAM bank (WP bit set in the SDRAM_SDCRx register).
- When the SDRAM address range is violated (access to reserved address range)

The effect of an AHB error depends on the AHB master which has attempted the R/W access:

- If the access has been attempted by the Cortex-M33 CPU, a hard fault interrupt is generated.
- If the access has been performed by a DMA controller, a DMA transfer error is generated and the corresponding DMA channel is automatically disabled.

The AHB clock (HCLK) is the reference clock for the FMC.

23.4.1 Supported memories and transactions

General transaction rules

The requested AHB transaction data size can be 8-, 16- or 32-bit wide whereas the accessed external device has a fixed data width. This may lead to inconsistent transfers.

Therefore, some simple transaction rules must be followed:

- AHB transaction size and memory data size are equal
There is no issue in this case.
- AHB transaction size is greater than the memory size:
In this case, the FMC splits the AHB transaction into smaller consecutive memory accesses to meet the external data width. The FMC chip select (FMC_NEx) does not toggle between the consecutive accesses. If the bus turnaround timings is configured to any other value than 0, the FMC chip select (FMC_NEx) toggles between the consecutive accesses. This feature is required when interfacing with FRAM memory.
- AHB transaction size is smaller than the memory size:
The transfer may or not be consistent depending on the type of external device:
 - Accesses to devices that have the byte select feature (SRAM, ROM, PSRAM, SDRAM)
In this case, the FMC allows read/write transactions and accesses to the right data through its byte lanes NBL[1:0].
Bytes to be written are addressed by NBL[1:0].
All memory bytes are read (NBL[1:0] are driven low during read transaction) and the useless ones are discarded.
 - Accesses to devices that do not have the byte select feature (NOR and NAND flash memories)
This situation occurs when a byte access is requested to a 16-bit wide flash memory. Since the device cannot be accessed in Byte mode (only 16-bit words can be read/written from/to the flash memory), Write transactions and Read transactions are allowed (the controller reads the entire 16-bit memory word and uses only the required byte).

Wrap support for NOR flash/PSRAM and SDRAM

Wrap burst mode for synchronous memories is not supported. The memories must be configured in Linear burst mode of undefined length.

Configuration registers

The FMC can be configured through a set of registers. Refer to [Section 23.6.6](#), for a detailed description of the NOR flash/PSRAM controller registers. Refer to [Section 23.7.7](#),

for a detailed description of the NAND flash registers and to [Section 23.8.5](#) for a detailed description of the SDRAM controller registers.

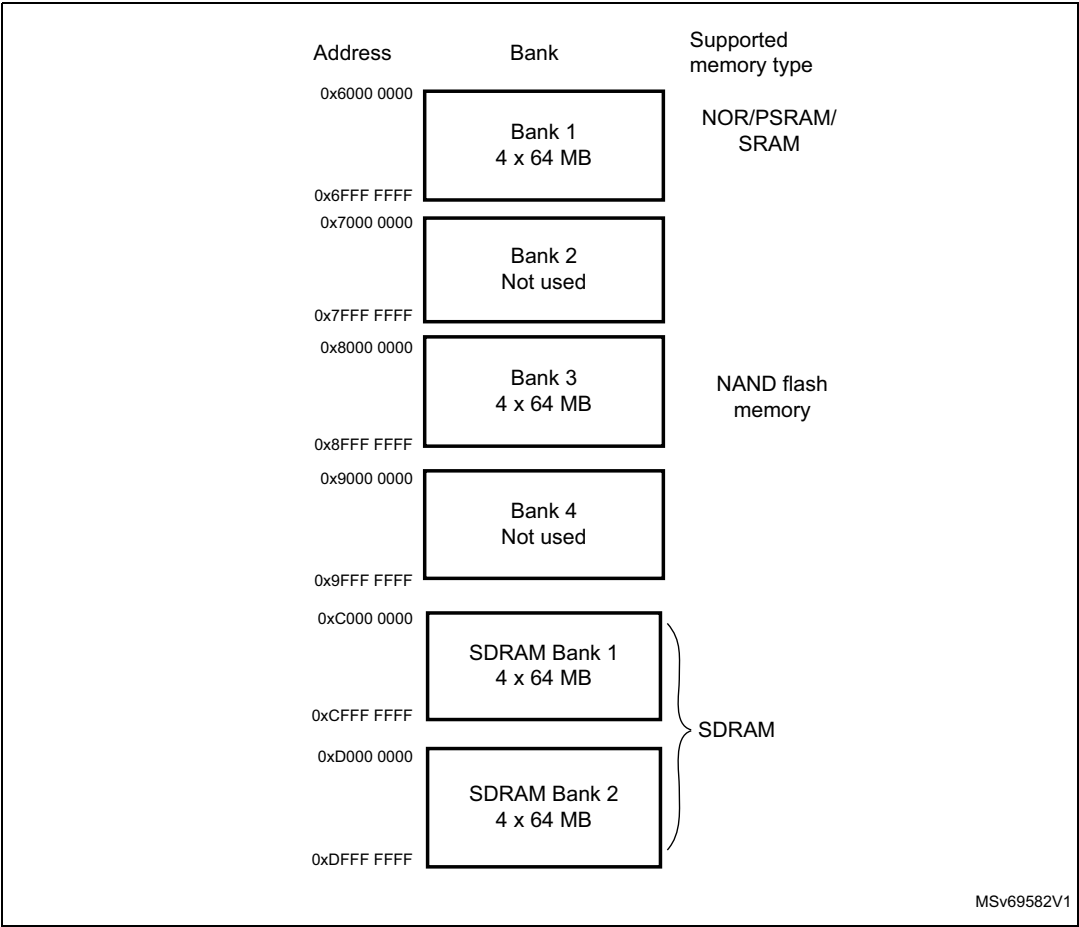
23.5 External device address mapping

From the FMC point of view, the external memory is divided into fixed-size banks of 256 Mbytes each (see [Figure 131](#)):

- Bank 1 used to address up to 4 NOR flash memory or PSRAM devices. This bank is split into 4 NOR/PSRAM subbanks with 4 dedicated chip selects, as follows:
 - Bank 1 - NOR/PSRAM 1
 - Bank 1 - NOR/PSRAM 2
 - Bank 1 - NOR/PSRAM 3
 - Bank 1 - NOR/PSRAM 4
- Bank 3 used to address NAND flash memory devices. The MPU memory attribute for this space must be reconfigured by software to Device.
- Bank 4 and 5 used to address SDRAM devices (1 device per bank).

For each bank the type of memory to be used can be configured by the user application through the Configuration register.

Figure 131. FMC memory banks



23.5.1 NOR/PSRAM address mapping

HADDR[27:26] bits are used to select one of the four memory banks as shown in [Table 191](#).

Table 191. NOR/PSRAM bank selection

HADDR[27:26] ⁽¹⁾	Selected bank
00	Bank 1 - NOR/PSRAM 1
01	Bank 1 - NOR/PSRAM 2
10	Bank 1 - NOR/PSRAM 3
11	Bank 1 - NOR/PSRAM 4

1. HADDR are internal AHB address lines that are translated to external memory.

The HADDR[25:0] bits contain the external memory address. Since HADDR is a byte address whereas the memory is addressed at word level, the address actually issued to the memory varies according to the memory data width, as shown in the following table.

Table 192. NOR/PSRAM External memory address

Memory width ⁽¹⁾	Data address issued to the memory	Maximum memory capacity (bits)
8-bit	HADDR[25:0]	64 Mbytes x 8 = 512 Mbits
16-bit	HADDR[25:1] >> 1	64 Mbytes/2 x 16 = 512 Mbits

1. In case of a 16-bit external memory width, the FMC internally uses HADDR[25:1] to generate the address for external memory FMC_A[24:0].
Whatever the external memory width, FMC_A[0] must be connected to external memory address A[0].

23.5.2 NAND flash memory address mapping

The NAND bank is divided into memory areas as indicated in [Table 193](#).

Table 193. NAND memory mapping and timing registers

Start address	End address	FMC bank	Memory space	Timing register
0x8800 0000	0x8BFF FFFF	Bank 3 - NAND flash	Attribute	FMC_PATT (0x8C)
0x8000 0000	0x83FF FFFF		Common	FMC_PMEM (0x88)

For NAND flash memory, the common and attribute memory spaces are subdivided into three sections (see in [Table 194](#) below) located in the lower 256 Kbytes:

- Data section (first 64 Kbytes in the common/attribute memory space)
- Command section (second 64 Kbytes in the common / attribute memory space)
- Address section (next 128 Kbytes in the common / attribute memory space)

Table 194. NAND bank selection

Section name	HADDR[17:16]	Address range
Address section	1X	0x020000-0x03FFFF
Command section	01	0x010000-0x01FFFF
Data section	00	0x000000-0x0FFFFF

The application software uses the 3 sections to access the NAND flash memory:

- **To sending a command to NAND flash memory**, the software must write the command value to any memory location in the command section.
- **To specify the NAND flash address that must be read or written**, the software must write the address value to any memory location in the address section. Since an address can be 4 or 5 bytes long (depending on the actual memory size), several consecutive write operations to the address section are required to specify the full address.
- **To read or write data**, the software reads or writes the data from/to any memory location in the data section.

Since the NAND flash memory automatically increments addresses, there is no need to increment the address of the data section to access consecutive memory locations.

23.5.3 SDRAM address mapping

The HADDR[28] bit (internal AHB address line 28) is used to select one of the two memory banks as indicated in [Table 195](#).

Table 195. SDRAM bank selection

HADDR[28]	Selected bank	Control register	Timing register
0	SDRAM Bank1	FMC_SDCR1	FMC_SDTR1
1	SDRAM Bank2	FMC_SDCR2	FMC_SDTR2

The following table shows SDRAM mapping for a 13-bit row, a 11-bit column and a 4 internal bank configuration.

Table 196. SDRAM address mapping

Memory width ⁽¹⁾	Internal bank	Row address	Column address ⁽²⁾	Maximum memory capacity (Mbytes)
8-bit	HADDR[25:24]	HADDR[23:11]	HADDR[10:0]	64 Mbytes: 4 x 8K x 2K
16-bit	HADDR[26:25]	HADDR[24:12]	HADDR[11:1]	128 Mbytes: 4 x 8K x 2K x 2

1. When interfacing with a 16-bit memory, the FMC internally uses the HADDR[11:1] internal AHB address lines to generate the external address. Whatever the memory width, FMC_A[0] has to be connected to the external memory address A[0].
2. The AutoPrecharge is not supported. FMC_A[10] must be connected to the external memory address A[10] but it is always driven low.

The HADDR[27:0] bits are translated to external SDRAM address depending on the SDRAM controller configuration:

- Data size: 8 or 16 bits
- Row size: 11, 12 or 13 bits
- Column size: 8, 9, 10 or 11 bits
- Number of internal banks: two or four internal banks

The following tables show the SDRAM address mapping versus the SDRAM controller configuration.

Table 197. SDRAM address mapping with 8-bit data bus width⁽¹⁾⁽²⁾

Row size configuration	HADDR(AHB Internal Address Lines)																											
	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
11-bit row size configuration	Res.							Bank [1:0]		Row[10:0]										Column[7:0]								
	Res.						Bank [1:0]		Row[10:0]										Column[8:0]									
	Res.					Bank [1:0]		Row[10:0]										Column[9:0]										
	Res.				Bank [1:0]		Row[10:0]										Column[10:0]											
12-bit row size configuration	Res.						Bank [1:0]		Row[11:0]										Column[7:0]									
	Res.					Bank [1:0]		Row[11:0]										Column[8:0]										
	Res.				Bank [1:0]		Row[11:0]										Column[9:0]											
	Res.			Bank [1:0]		Row[11:0]										Column[10:0]												
13-bit row size configuration	Res.					Bank [1:0]		Row[12:0]										Column[7:0]										
	Res.				Bank [1:0]		Row[12:0]										Column[8:0]											
	Res.			Bank [1:0]		Row[12:0]										Column[9:0]												
	Res.		Bank [1:0]		Row[12:0]										Column[10:0]													

1. BANK[1:0] are the Bank Address BA[1:0]. When only 2 internal banks are used, BA1 must always be set to '0'.

2. Access to Reserved (Res.) address range generates an AHB error.

Table 198. SDRAM address mapping with 16-bit data bus width⁽¹⁾⁽²⁾

Row size Configuration	HADDR(AHB address Lines)																											
	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
11-bit row size configuration	Res.							Bank [1:0]	Row[10:0]											Column[7:0]							BM0 ⁽³⁾	
	Res.					Bank [1:0]		Row[10:0]											Column[8:0]							BM0		
	Res.				Bank [1:0]		Row[10:0]											Column[9:0]							BM0			
	Res.			Bank [1:0]		Row[10:0]											Column[10:0]							BM0				
12-bit row size configuration	Res.					Bank [1:0]		Row[11:0]											Column[7:0]							BM0		
	Res.				Bank [1:0]		Row[11:0]											Column[8:0]							BM0			
	Res.			Bank [1:0]		Row[11:0]											Column[9:0]							BM0				
	Res.		Bank [1:0]		Row[11:0]											Column[10:0]							BM0					
13-bit row size configuration	Res.				Bank [1:0]		Row[12:0]											Column[7:0]							BM0			
	Res.			Bank [1:0]		Row[12:0]											Column[8:0]							BM0				
	Res.		Bank [1:0]		Row[12:0]											Column[9:0]							BM0					
	Res.	Bank [1:0]		Row[12:0]											Column[10:0]							BM0						

1. BANK[1:0] are the Bank Address BA[1:0]. When only 2 internal banks are used, BA1 must always be set to '0'.
2. Access to Reserved space (Res.) generates an AHB error.
3. BM0: is the byte mask for 16-bit access.

23.6 NOR flash/PSRAM controller

The FMC generates the appropriate signal timings to drive the following types of memories:

- Asynchronous SRAM, FRAM and ROM
 - 8 bits
 - 16 bits
- PSRAM (CellularRAM™)
 - Asynchronous mode
 - Burst mode for synchronous accesses
 - Multiplexed or non-multiplexed
- NOR flash memory
 - Asynchronous mode
 - Burst mode for synchronous accesses
 - Multiplexed or non-multiplexed

The FMC outputs a unique chip select signal, NE[4:1], per bank. All the other signals (addresses, data and control) are shared.

The FMC supports a wide range of devices through a programmable timings among which:

- Programmable wait states (up to 15)
- Programmable bus turnaround cycles (up to 15)
- Programmable output enable and write enable delays (up to 15)
- Independent read and write timings and protocol to support the widest variety of memories and timings
- Programmable continuous clock (FMC_CLK) output.

The FMC Clock (FMC_CLK) is a submultiple of the HCLK clock. It can be delivered to the selected external device either during synchronous accesses only or during asynchronous and synchronous accesses depending on the CCKEN bit configuration in the FMC_BCR1 register:

- If the CCLKEN bit is reset, the FMC generates the clock (CLK) only during synchronous accesses (Read/write transactions).
- If the CCLKEN bit is set, the FMC generates a continuous clock during asynchronous and synchronous accesses. To generate the FMC_CLK continuous clock, Bank 1 must be configured in Synchronous mode (see [Section 23.6.6: NOR/PSRAM controller registers](#)). Since the same clock is used for all synchronous memories, when a continuous output clock is generated and synchronous accesses are performed, the AHB data size has to be the same as the memory data width (MWID) otherwise the FMC_CLK frequency is changed depending on AHB data transaction (refer to [Section 23.6.5: Synchronous transactions](#) for FMC_CLK divider ratio formula).

The size of each bank is fixed and equal to 64 Mbytes. Each bank is configured through dedicated registers (see [Section 23.6.6: NOR/PSRAM controller registers](#)).

The programmable memory parameters include access times (see [Table 199](#)) and support for wait management (for PSRAM and NOR flash accessed in Burst mode).

Table 199. Programmable NOR/PSRAM access parameters

Parameter	Function	Access mode	Unit	Min.	Max.
Address setup	Duration of the address setup phase	Asynchronous	AHB clock cycle (HCLK)	0	15
Address hold	Duration of the address hold phase	Asynchronous, muxed I/Os	AHB clock cycle (HCLK)	1	15
NBL setup	Duration of the byte lanes setup phase	Asynchronous	AHB clock cycle (HCLK)	0	3
Data setup	Duration of the data setup phase	Asynchronous	AHB clock cycle (HCLK)	1	256
Data hold	Duration of the data hold phase	Asynchronous	AHB clock cycle (HCLK)	0	3
Burst turn	Duration of the bus turnaround phase	Asynchronous and synchronous read / write	AHB clock cycle (HCLK)	0	15

Table 199. Programmable NOR/PSRAM access parameters (continued)

Parameter	Function	Access mode	Unit	Min.	Max.
Clock divide ratio	Number of AHB clock cycles (HCLK) to build one memory clock cycle (CLK)	Synchronous	AHB clock cycle (HCLK)	2	16
Data latency	Number of clock cycles to issue to the memory before the first data of the burst	Synchronous	Memory clock cycle (CLK)	2	17

23.6.1 External memory interface signals

[Table 200](#), [Table 201](#) and [Table 202](#) list the signals that are typically used to interface with NOR flash memory, SRAM and PSRAM.

Note: The prefix “N” identifies the signals that are active low.

NOR flash memory, non-multiplexed I/Os

Table 200. Non-multiplexed I/O NOR flash memory

FMC signal name	I/O	Function
CLK	O	Clock (for synchronous access)
A[25:0]	O	Address bus
D[15:0]	I/O	Bidirectional data bus
NE[x]	O	Chip select, x = 1..4
NOE	O	Output enable
NWE	O	Write enable
NL(=NADV)	O	Latch enable (this signal is called address valid, NADV, by some NOR flash devices)
NWAIT	I	NOR flash wait input signal to the FMC

The maximum capacity is 512 Mbits (26 address lines).

NOR flash memory, 16-bit multiplexed I/Os

Table 201. 16-bit multiplexed I/O NOR flash memory

FMC signal name	I/O	Function
CLK	O	Clock (for synchronous access)
A[25:16]	O	Address bus
AD[15:0]	I/O	16-bit multiplexed, bidirectional address/data bus (the 16-bit address A[15:0] and data D[15:0] are multiplexed on the databus)
NE[x]	O	Chip select, x = 1..4
NOE	O	Output enable
NWE	O	Write enable

Table 201. 16-bit multiplexed I/O NOR flash memory (continued)

FMC signal name	I/O	Function
NL(=NADV)	O	Latch enable (this signal is called address valid, NADV, by some NOR flash devices)
NWAIT	I	NOR flash wait input signal to the FMC

The maximum capacity is 512 Mbits.

PSRAM/FRAM/SRAM, non-multiplexed I/Os

Table 202. Non-multiplexed I/Os PSRAM/SRAM

FMC signal name	I/O	Function
CLK	O	Clock (only for PSRAM synchronous access)
A[25:0]	O	Address bus
D[15:0]	I/O	Data bidirectional bus
NE[x]	O	Chip select, x = 1..4 (called NCE by PSRAM (CellularRAM™ i.e. CRAM))
NOE	O	Output enable
NWE	O	Write enable
NL(= NADV)	O	Address valid only for PSRAM input (memory signal name: NADV)
NWAIT	I	PSRAM wait input signal to the FMC
NBL[1:0]	O	Byte lane output. Byte 0 and Byte 1 control (upper and lower byte enable)

The maximum capacity is 512 Mbits.

PSRAM, 16-bit multiplexed I/Os

Table 203. 16-Bit multiplexed I/O PSRAM

FMC signal name	I/O	Function
CLK	O	Clock (for synchronous access)
A[25:16]	O	Address bus
AD[15:0]	I/O	16-bit multiplexed, bidirectional address/data bus (the 16-bit address A[15:0] and data D[15:0] are multiplexed on the databus)
NE[x]	O	Chip select, x = 1..4 (called NCE by PSRAM (CellularRAM™ i.e. CRAM))
NOE	O	Output enable
NWE	O	Write enable
NL(= NADV)	O	Address valid PSRAM input (memory signal name: NADV)
NWAIT	I	PSRAM wait input signal to the FMC
NBL[1:0]	O	Byte lane output. Byte 0 and Byte 1 control (upper and lower byte enable)

The maximum capacity is 512 Mbits (26 address lines).

23.6.2 Supported memories and transactions

Table 204 below shows an example of the supported devices, access modes and transactions when the memory data bus is 16-bit wide for NOR flash memory, PSRAM and SRAM. The transactions not allowed (or not supported) by the FMC are shown in gray in this example.

Table 204. NOR flash/PSRAM: example of supported memories and transactions

Device	Mode	R/W	AHB data size	Memory data size	Allowed/ not allowed	Comments
NOR flash (muxed I/Os and nonmuxed I/Os)	Asynchronous	R	8	16	Y	-
	Asynchronous	W	8	16	N	-
	Asynchronous	R	16	16	Y	-
	Asynchronous	W	16	16	Y	-
	Asynchronous	R	32	16	Y	Split into 2 FMC accesses
	Asynchronous	W	32	16	Y	Split into 2 FMC accesses
	Asynchronous page	R	-	16	N	Mode is not supported
	Synchronous	R	8	16	N	-
	Synchronous	R	16	16	Y	-
	Synchronous	R	32	16	Y	-
PSRAM (multiplexed I/Os and non-multiplexed I/Os)	Asynchronous	R	8	16	Y	-
	Asynchronous	W	8	16	Y	Use of byte lanes NBL[1:0]
	Asynchronous	R	16	16	Y	-
	Asynchronous	W	16	16	Y	-
	Asynchronous	R	32	16	Y	Split into 2 FMC accesses
	Asynchronous	W	32	16	Y	Split into 2 FMC accesses
	Asynchronous page	R	-	16	N	Mode is not supported
	Synchronous	R	8	16	N	-
	Synchronous	R	16	16	Y	-
	Synchronous	R	32	16	Y	-
	Synchronous	W	8	16	Y	Use of byte lanes NBL[1:0]
	Synchronous	W	16/32	16	Y	-
SRAM and ROM	Asynchronous	R	8 / 16	16	Y	-
	Asynchronous	W	8 / 16	16	Y	Use of byte lanes NBL[1:0]
	Asynchronous	R	32	16	Y	Split into 2 FMC accesses
	Asynchronous	W	32	16	Y	Split into 2 FMC accesses Use of byte lanes NBL[1:0]

23.6.3 General timing rules

Signals synchronization

- All controller output signals change on the rising edge of the internal clock (HCLK)
- In Synchronous mode (read or write), all output signals change on the rising edge of HCLK. Whatever the CLKDIV value, all outputs change as follows:
 - NOEL/NWEL/ NEL/NADV L/ NADV H /NBLL/ Address valid outputs change on the falling edge of FMC_CLK clock.
 - NOEH/ NWEH / NEH/ NOEH/NBLH/ Address invalid outputs change on the rising edge of FMC_CLK clock.

23.6.4 NOR flash/PSRAM controller asynchronous transactions

Asynchronous static memories (NOR flash, PSRAM, SRAM, FRAM)

- Signals are synchronized by the internal clock HCLK. This clock is not issued to the memory
- The FMC always samples the data before de-asserting the NOE signal. This guarantees that the memory data hold timing constraint is met (minimum Chip Enable high to data transition is usually 0 ns)
- If the Extended mode is enabled (EXTMOD bit is set in the FMC_BCRx register), up to four extended modes (A, B, C and D) are available. It is possible to mix A, B, C and D modes for read and write operations. For example, read operation can be performed in mode A and write in mode B.
- If the Extended mode is disabled (EXTMOD bit is reset in the FMC_BCRx register), the FMC can operate in mode 1 or mode 2 as follows:
 - Mode 1 is the default mode when SRAM/PSRAM memory type is selected (MTYP = 0x0 or 0x01 in the FMC_BCRx register)
 - Mode 2 is the default mode when NOR memory type is selected (MTYP = 0x10 in the FMC_BCRx register).

Mode 1 - SRAM/FRAM/PSRAM (CRAM)

The next figures show the read and write transactions for the supported modes followed by the required configuration of FMC_BCRx, and FMC_BTRx/FMC_BWTRx registers.

Figure 132. Mode 1 read access waveforms

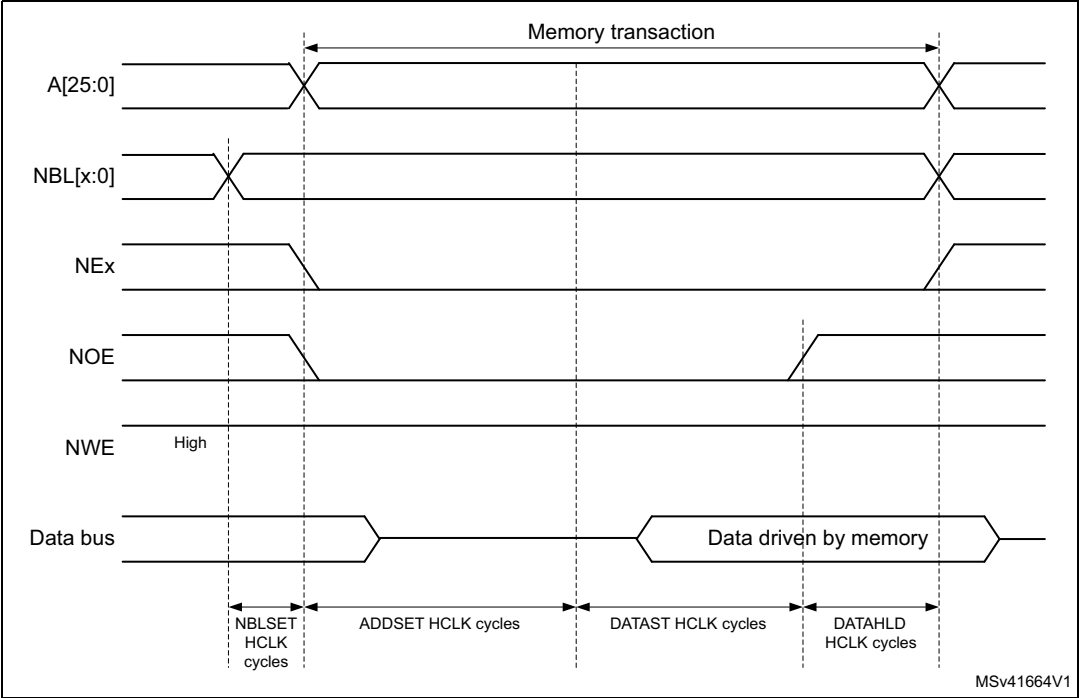
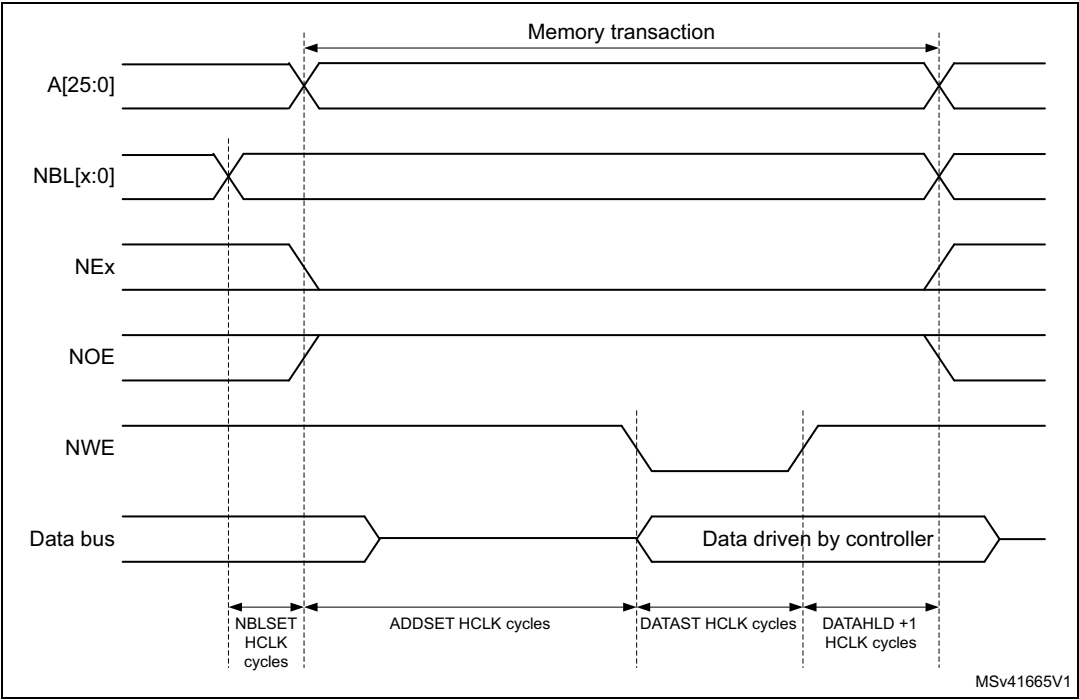


Figure 133. Mode 1 write access waveforms



The DATAHLD time at the end of the read and write transactions guarantee the address and data hold time after the NOE/NWE rising edge. The DATAST value must be greater than zero (DATAST > 0).

Table 205. FMC_BCRx bitfields (mode 1)

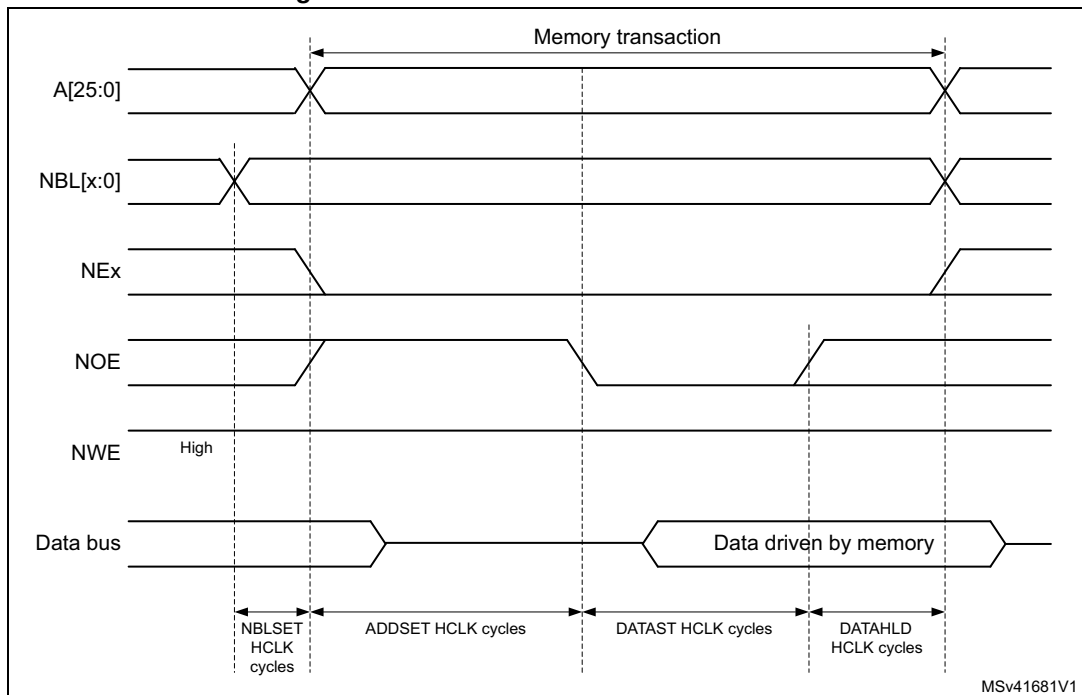
Bit number	Bit name	Value to set
31	FMCEN	0x1
30:24	Reserved	0x000
23:22	NBLSET[1:0]	As needed
20	CCLKEN	As needed
19	CBURSTRW	0x0 (no effect in Asynchronous mode)
18:16	CPSIZE	0x0 (no effect in Asynchronous mode)
15	ASYNCWAIT	Set to 1 if the memory supports this feature. Otherwise keep at 0.
14	EXTMOD	0x0
13	WAITEN	0x0 (no effect in Asynchronous mode)
12	WREN	As needed
10	Reserved	0x0
9	WAITPOL	Meaningful only if bit 15 is 1
8	BURSTEN	0x0
7	Reserved	0x1
6	FACCEN	Don't care
5:4	MWID	As needed
3:2	MTYP	As needed, exclude 0x2 (NOR flash memory)
1	MUXE	0x0
0	MBKEN	0x1

Table 206. FMC_BTRx bitfields (mode 1)

Bit number	Bit name	Value to set
31:30	DATAHLD	Duration of the data hold phase (DATAHLD HCLK cycles for read accesses, DATAHLD+1 HCLK cycles for write accesses).
29:28	ACCMOD	Don't care
27:24	DATLAT	Don't care
23:20	CLKDIV	Don't care
19:16	BUSTURN	Time between NEx high to NEx low (BUSTURN HCLK).
15:8	DATAST	Duration of the second access phase (DATAST HCLK cycles).
7:4	ADDHLD	Don't care
3:0	ADDSET	Duration of the first access phase (ADDSET HCLK cycles). Minimum value for ADDSET is 0.

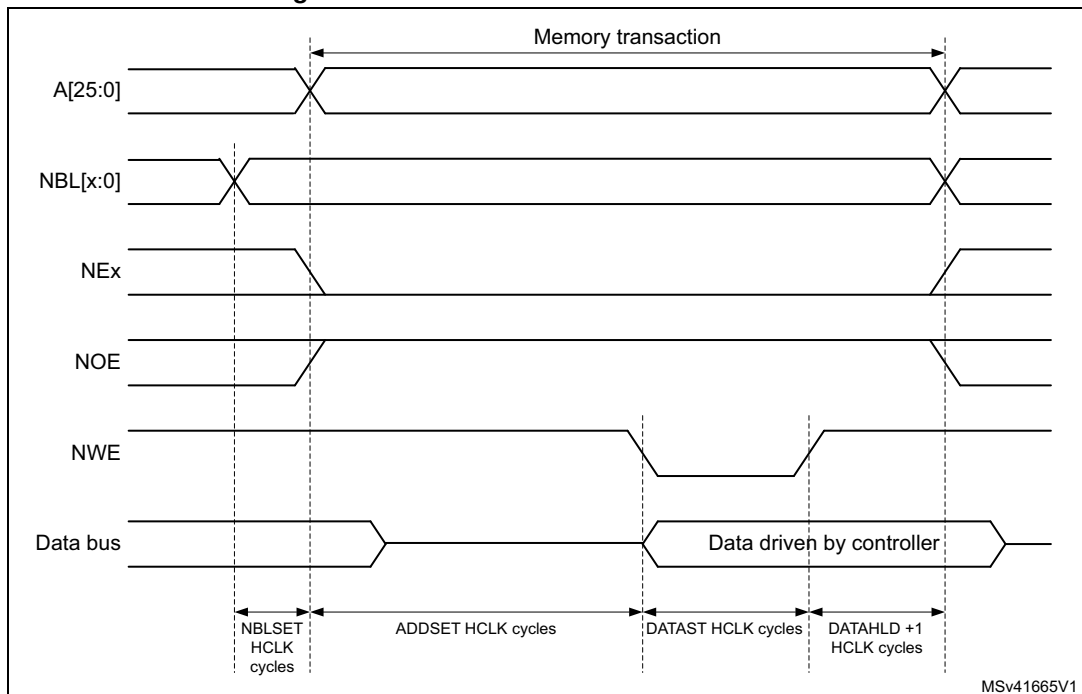
Mode A - SRAM/FRAM/PSRAM (CRAM) OE toggling

Figure 134. Mode A read access waveforms



1. NBL[1:0] are driven low during the read access

Figure 135. Mode A write access waveforms



The differences compared with Mode 1 are the toggling of NOE and the independent read and write timings.

Table 207. FMC_BCRx bitfields (mode A)

Bit number	Bit name	Value to set
31	FMCEN	0x1
30:24	Reserved	0x000
23:22	NBLSET[1:0]	As needed
20	CCLKEN	As needed
19	CBURSTRW	0x0 (no effect in Asynchronous mode)
18:16	CPSIZE	0x0 (no effect in Asynchronous mode)
15	ASYNCWAIT	Set to 1 if the memory supports this feature. Otherwise keep at 0.
14	EXTMOD	0x1
13	WAITEN	0x0 (no effect in Asynchronous mode)
12	WREN	As needed
11	WAITCFG	Don't care
10	Reserved	0x0
9	WAITPOL	Meaningful only if bit 15 is 1
8	BURSTEN	0x0
7	Reserved	0x1
6	FACCEN	Don't care
5:4	MWID	As needed
3:2	MTYP	As needed, exclude 0x2 (NOR flash memory)
1	MUXEN	0x0
0	MBKEN	0x1

Table 208. FMC_BTRx bitfields (mode A)

Bit number	Bit name	Value to set
31:30	DATAHLD	Duration of the data hold phase (DATAHLD HCLK cycles for read accesses).
29:28	ACCMOD	0x0
27:24	DATLAT	Don't care
23:20	CLKDIV	Don't care
19:16	BUSTURN	Time between NEx high to NEx low (BUSTURN HCLK).
15:8	DATAST	Duration of the second access phase (DATAST HCLK cycles) for read accesses.
7:4	ADDHLD	Don't care
3:0	ADDSET	Duration of the first access phase (ADDSET HCLK cycles) for read accesses. Minimum value for ADDSET is 0.

Table 209. FMC_BWTRx bitfields (mode A)

Bit number	Bit name	Value to set
31:30	DATAHLD	Duration of the data hold phase (DATAHLD+1 HCLK cycles for write accesses).
29:28	ACCMOD	0x0
27:24	DATLAT	Don't care
23:20	CLKDIV	Don't care
19:16	BUSTURN	Time between NEx high to NEx low (BUSTURN HCLK).
15:8	DATAST	Duration of the second access phase (DATAST HCLK cycles) for write accesses.
7:4	ADDHLD	Don't care
3:0	ADDSET	Duration of the first access phase (ADDSET HCLK cycles) for write accesses. Minimum value for ADDSET is 0.

Mode 2/B - NOR flash

Figure 136. Mode 2 and mode B read access waveforms

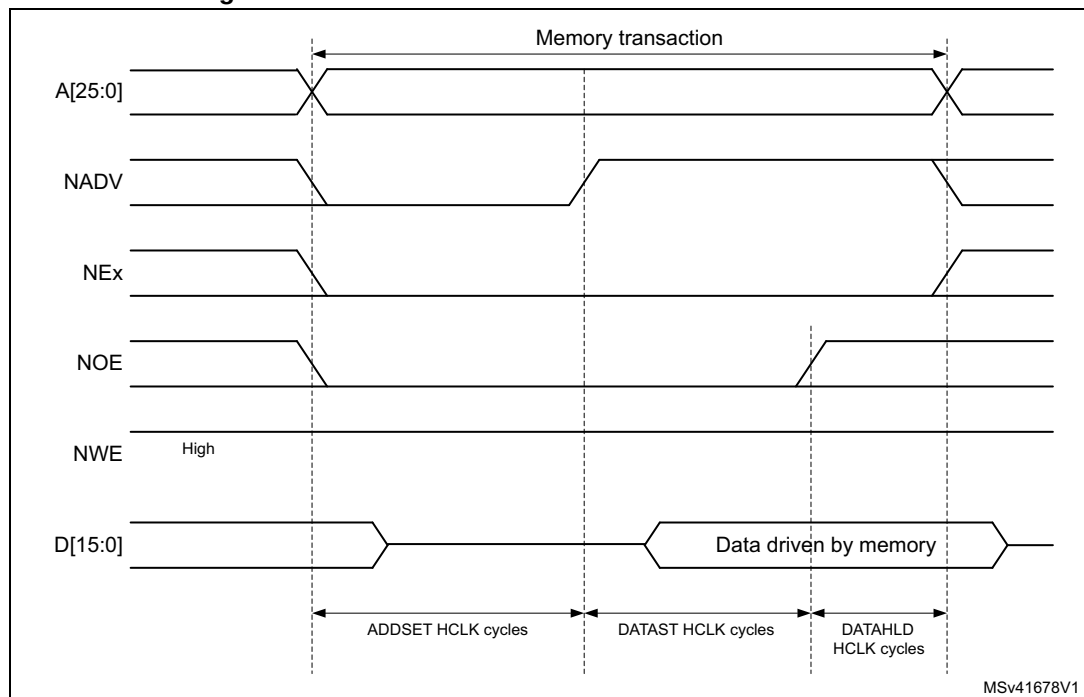


Figure 137. Mode 2 write access waveforms

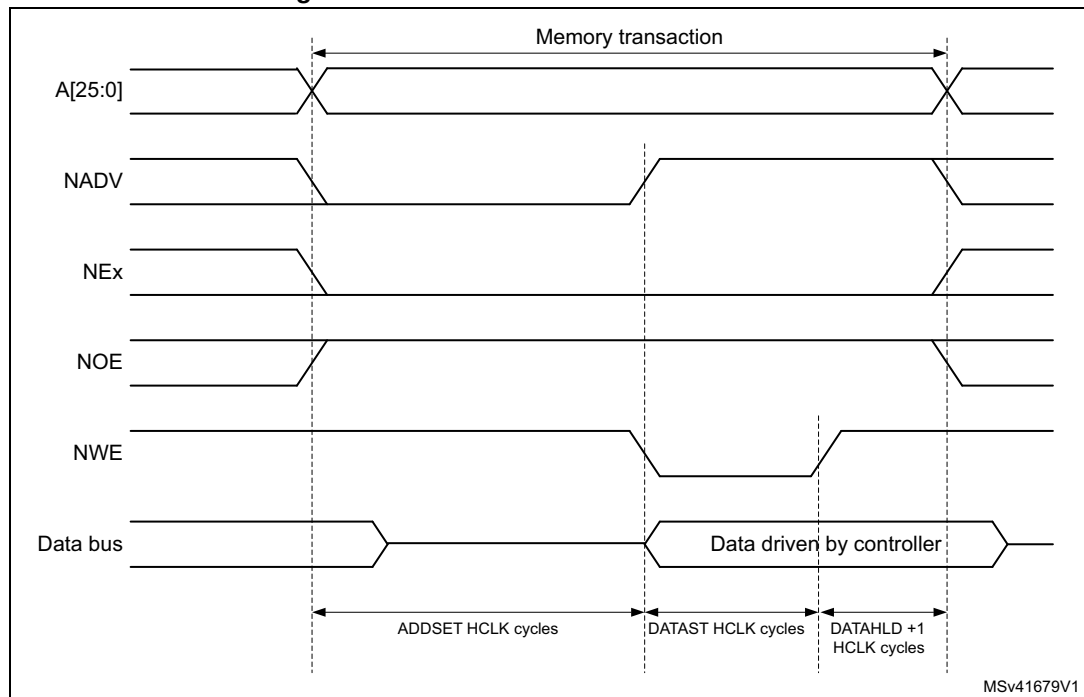
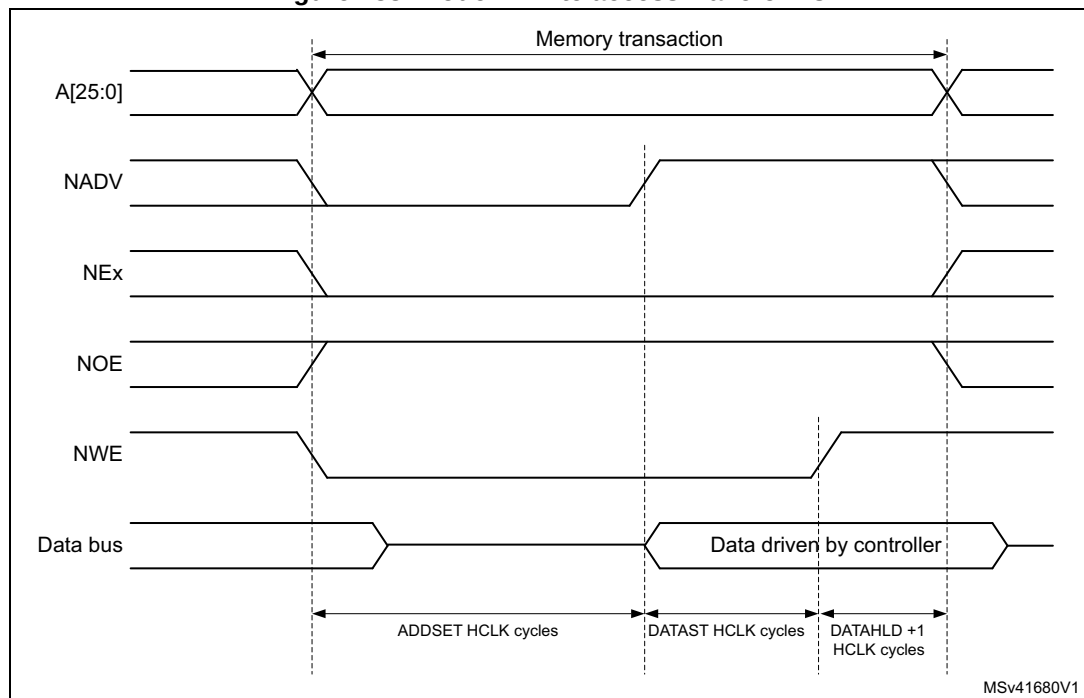


Figure 138. Mode B write access waveforms



The differences with mode 1 are the toggling of NWE and the independent read and write timings when extended mode is set (mode B).

Table 210. FMC_BCRx bitfields (mode 2/B)

Bit number	Bit name	Value to set
31	FMCEN	0x1
30:24	Reserved	0x000
23:22	NBLSET[1:0]	Don't care
20	CCLKEN	As needed
19	CBURSTRW	0x0 (no effect in Asynchronous mode)
18:16	CPSIZE	0x0 (no effect in Asynchronous mode)
15	ASYNCWAIT	Set to 1 if the memory supports this feature. Otherwise keep at 0.
14	EXTMOD	0x1 for mode B, 0x0 for mode 2
13	WAITEN	0x0 (no effect in Asynchronous mode)
12	WREN	As needed
11	WAITCFG	Don't care
10	Reserved	0x0
9	WAITPOL	Meaningful only if bit 15 is 1
8	BURSTEN	0x0
7	Reserved	0x1
6	FACCEN	0x1
5:4	MWID	As needed
3:2	MTYP	0x2 (NOR flash memory)
1	MUXEN	0x0
0	MBKEN	0x1

Table 211. FMC_BTRx bitfields (mode 2/B)

Bit number	Bit name	Value to set
31:30	DATAHLD	Duration of the data hold phase (DATAHLD HCLK cycles for read accesses and DATAHLD+1 HCLK cycles for write accesses when Extended mode is disabled).
29:28	ACCMOD	0x1 if Extended mode is set
27:24	DATLAT	Don't care
23:20	CLKDIV	Don't care
19:16	BUSTURN	Time between NEx high to NEx low (BUSTURN HCLK).
15:8	DATAST	Duration of the access second phase (DATAST HCLK cycles) for read accesses.
7:4	ADDHLD	Don't care
3:0	ADDSET	Duration of the access first phase (ADDSET HCLK cycles) for read accesses. Minimum value for ADDSET is 0.

Table 212. FMC_BWTRx bitfields (mode 2/B)

Bit number	Bit name	Value to set
31:30	DATAHLD	Duration of the data hold phase (DATAHLD+1 HCLK cycles for write accesses).
29:28	ACCMOD	0x1 if Extended mode is set
27:24	DATLAT	Don't care
23:20	CLKDIV	Don't care
19:16	BUSTURN	Time between NEx high to NEx low (BUSTURN HCLK).
15:8	DATAST	Duration of the access second phase (DATAST HCLK cycles) for write accesses.
7:4	ADDHLD	Don't care
3:0	ADDSET	Duration of the access first phase (ADDSET HCLK cycles) for write accesses. Minimum value for ADDSET is 0.

Note: The FMC_BWTRx register is valid only if the Extended mode is set (mode B), otherwise its content is don't care.

Mode C - NOR flash - OE toggling

Figure 139. Mode C read access waveforms

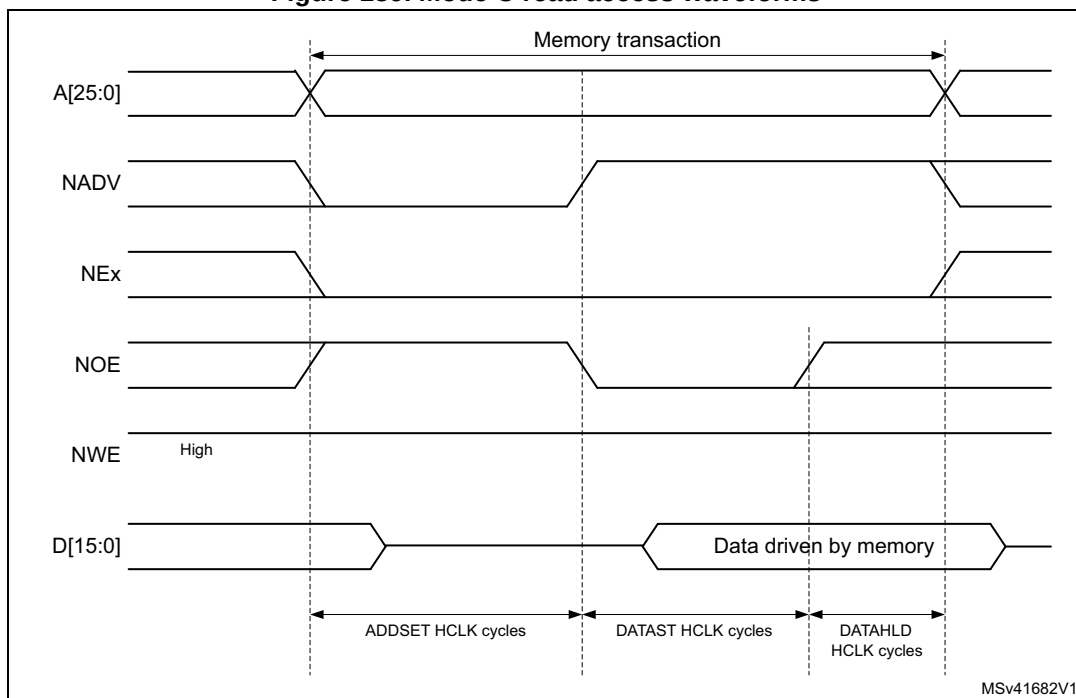
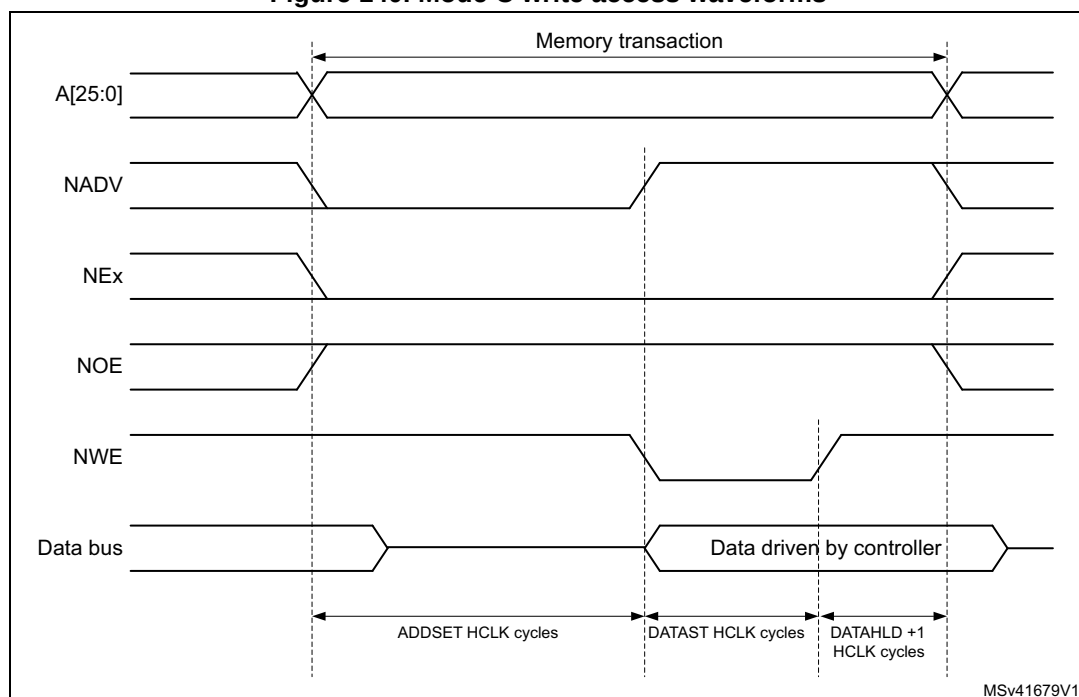


Figure 140. Mode C write access waveforms



MSv41679V1

The differences compared with mode 1 are the toggling of NOE and the independent read and write timings.

Table 213. FMC_BCRx bitfields (mode C)

Bit number	Bit name	Value to set
31	FMCEN	0x1
30:24	Reserved	0x000
23:22	NBLSET[1:0]	Don't care
20	CCLKEN	As needed
19	CBURSTRW	0x0 (no effect in Asynchronous mode)
18:16	CPSIZE	0x0 (no effect in Asynchronous mode)
15	ASYNCWAIT	Set to 1 if the memory supports this feature. Otherwise keep at 0.
14	EXTMOD	0x1
13	WAITEN	0x0 (no effect in Asynchronous mode)
12	WREN	As needed
11	WAITCFG	Don't care
10	Reserved	0x0
9	WAITPOL	Meaningful only if bit 15 is 1
8	BURSTEN	0x0
7	Reserved	0x1
6	FACCEN	0x1

Table 213. FMC_BCRx bitfields (mode C) (continued)

Bit number	Bit name	Value to set
5:4	MWID	As needed
3:2	MTYP	0x02 (NOR flash memory)
1	MUXEN	0x0
0	MBKEN	0x1

Table 214. FMC_BTRx bitfields (mode C)

Bit number	Bit name	Value to set
31:30	DATAHLD	Duration of the data hold phase (DATAHLD HCLK cycles for read accesses).
29:28	ACCMOD	0x2
27:24	DATLAT	0x0
23:20	CLKDIV	0x0
19:16	BUSTURN	Time between NEx high to NEx low (BUSTURN HCLK).
15:8	DATAST	Duration of the second access phase (DATAST HCLK cycles) for read accesses.
7:4	ADDHLD	Don't care
3:0	ADDSET	Duration of the first access phase (ADDSET HCLK cycles) for read accesses. Minimum value for ADDSET is 0.

Table 215. FMC_BWTRx bitfields (mode C)

Bit number	Bit name	Value to set
31:30	DATAHLD	Duration of the data hold phase (DATAHLD+1 HCLK cycles for write accesses).
29:28	ACCMOD	0x2
27:24	DATLAT	Don't care
23:20	CLKDIV	Don't care
19:16	BUSTURN	Time between NEx high to NEx low (BUSTURN HCLK).
15:8	DATAST	Duration of the second access phase (DATAST HCLK cycles) for write accesses.
7:4	ADDHLD	Don't care
3:0	ADDSET	Duration of the first access phase (ADDSET HCLK cycles) for write accesses. Minimum value for ADDSET is 0.

Mode D - asynchronous access with extended address

Figure 141. Mode D read access waveforms

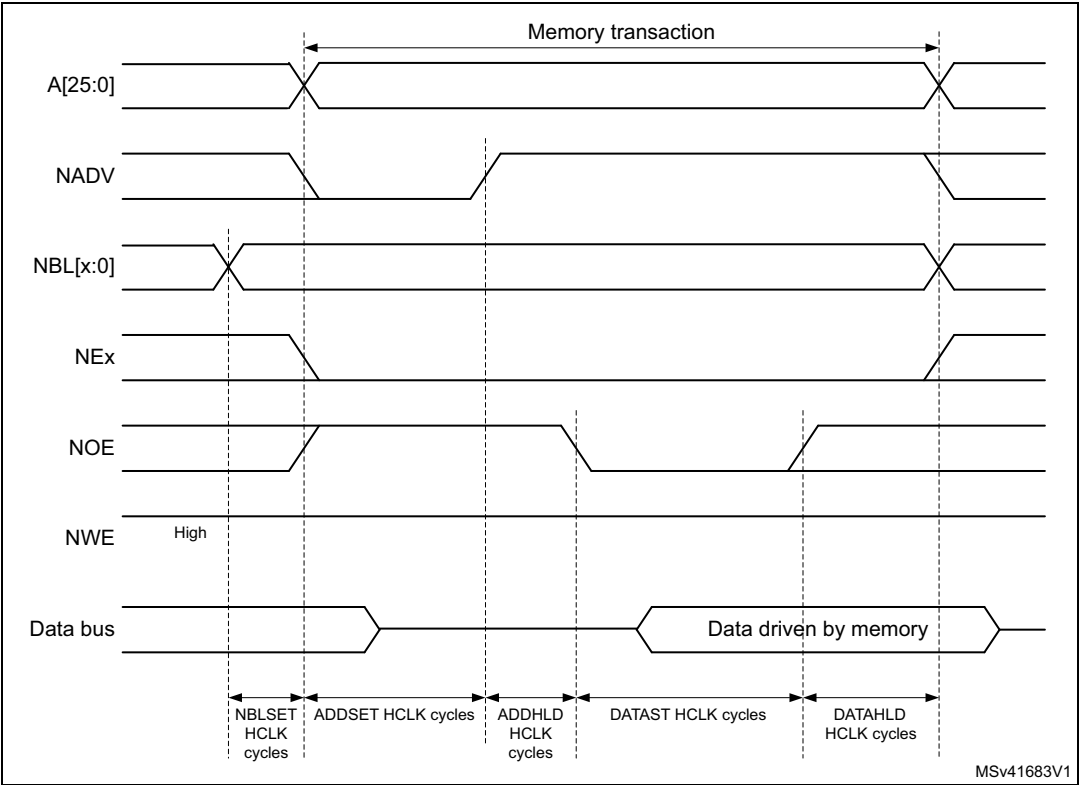
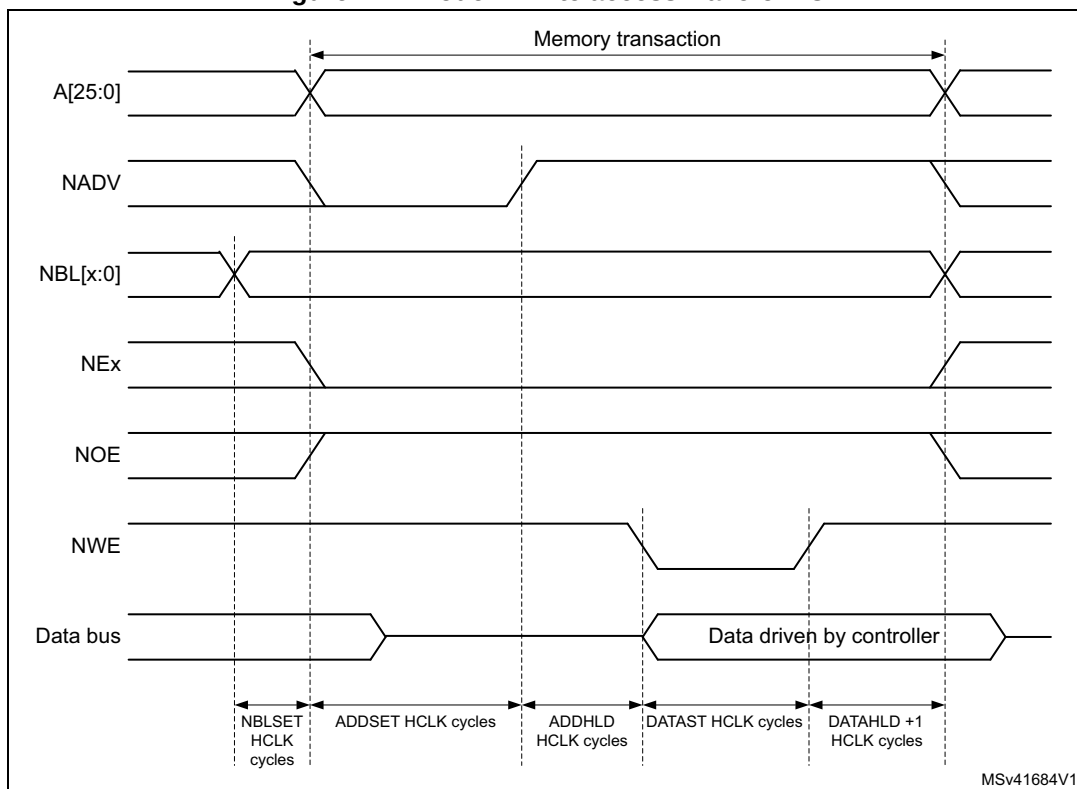


Figure 142. Mode D write access waveforms



MSv41684V1

The differences with mode 1 are the toggling of NOE that goes on toggling after NADV changes and the independent read and write timings.

Table 216. FMC_BCRx bitfields (mode D)

Bit number	Bit name	Value to set
31	FMCEN	0x1
30:24	Reserved	0x000
23:22	NBLSET[1:0]	As needed
20	CCLKEN	As needed
19	CBURSTRW	0x0 (no effect in Asynchronous mode)
18:16	CPSIZE	0x0 (no effect in Asynchronous mode)
15	ASYNCWAIT	Set to 1 if the memory supports this feature. Otherwise keep at 0.
14	EXTMOD	0x1
13	WAITEN	0x0 (no effect in Asynchronous mode)
12	WREN	As needed
11	WAITCFG	Don't care
10	Reserved	0x0
9	WAITPOL	Meaningful only if bit 15 is 1
8	BURSTEN	0x0

Table 216. FMC_BCRx bitfields (mode D) (continued)

Bit number	Bit name	Value to set
7	Reserved	0x1
6	FACCEN	Set according to memory support
5:4	MWID	As needed
3:2	MTYP	As needed
1	MUXEN	0x0
0	MBKEN	0x1

Table 217. FMC_BTRx bitfields (mode D)

Bit number	Bit name	Value to set
31:30	DATAHLD	Duration of the data hold phase (DATAHLD HCLK cycles for read accesses).
29:28	ACCMOD	0x3
27:24	DATLAT	Don't care
23:20	CLKDIV	Don't care
19:16	BUSTURN	Time between NEx high to NEx low (BUSTURN HCLK).
15:8	DATAST	Duration of the second access phase (DATAST HCLK cycles) for read accesses.
7:4	ADDHLD	Duration of the middle phase of the read access (ADDHLD HCLK cycles)
3:0	ADDSET	Duration of the first access phase (ADDSET HCLK cycles) for read accesses. Minimum value for ADDSET is 1.

Table 218. FMC_BWTRx bitfields (mode D)

Bit number	Bit name	Value to set
31:30	DATAHLD	Duration of the data hold phase (DATAHLD+1 HCLK cycles for write accesses).
29:28	ACCMOD	0x3
27:24	DATLAT	Don't care
23:20	CLKDIV	Don't care
19:16	BUSTURN	Time between NEx high to NEx low (BUSTURN HCLK).
15:8	DATAST	Duration of the second access phase (DATAST HCLK cycles).
7:4	ADDHLD	Duration of the middle phase of the write access (ADDHLD HCLK cycles)
3:0	ADDSET	Duration of the first access phase (ADDSET HCLK cycles) for write accesses. Minimum value for ADDSET is 1.

Muxed mode - multiplexed asynchronous access to NOR flash memory

Figure 143. Muxed read access waveforms

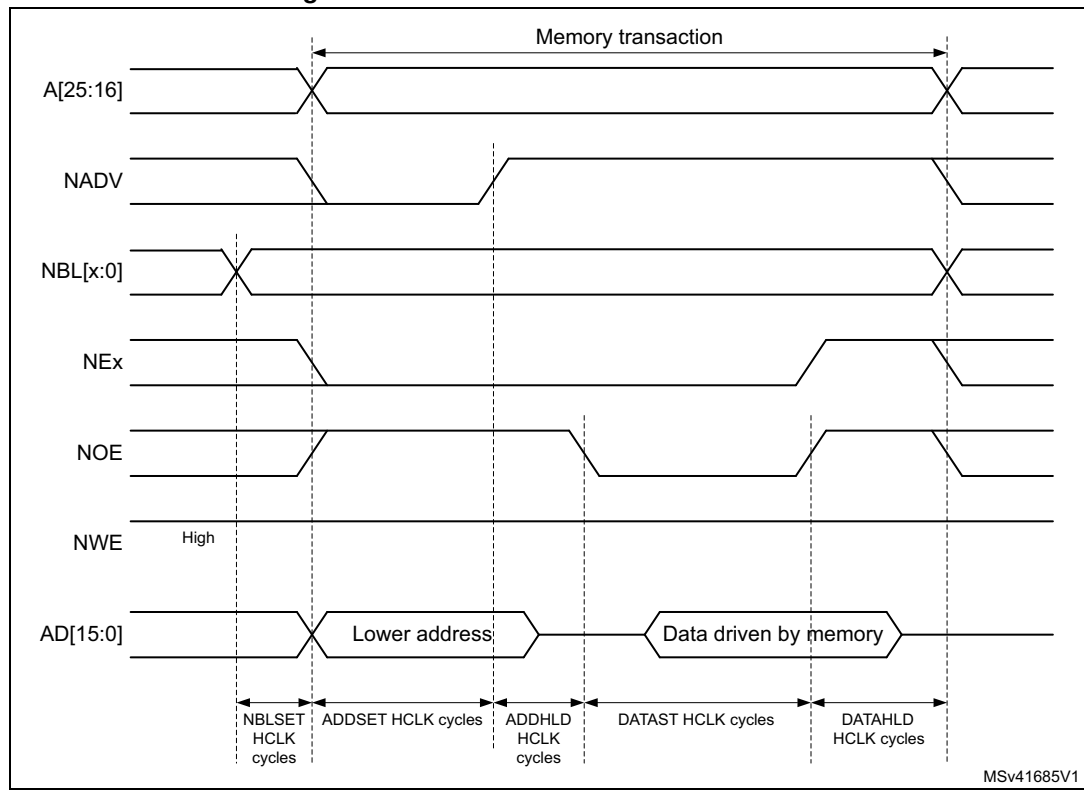
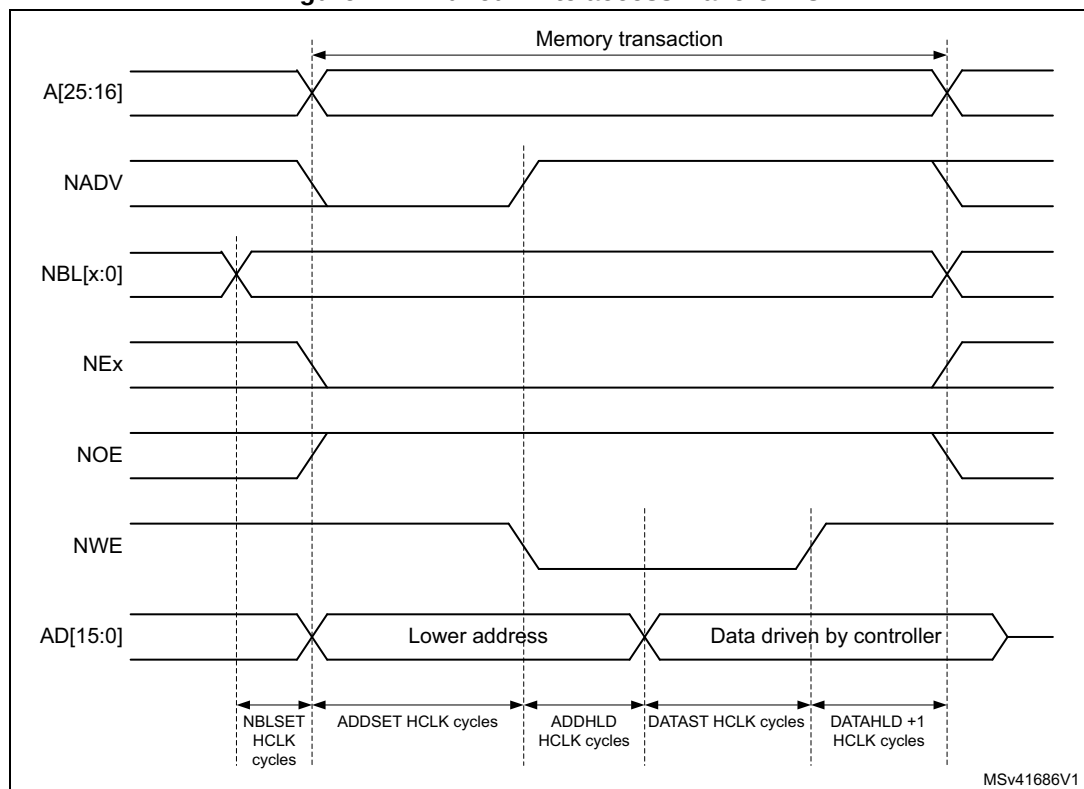


Figure 144. Muxed write access waveforms



The difference with mode D is the drive of the lower address byte(s) on the data bus.

Table 219. FMC_BCRx bitfields (Muxed mode)

Bit number	Bit name	Value to set
31	FMCEN	0x1
30:24	Reserved	0x000
23:22	NBLSET[1:0]	As needed
20	CCLKEN	As needed
19	CBURSTRW	0x0 (no effect in Asynchronous mode)
18:16	CPSIZE	0x0 (no effect in Asynchronous mode)
15	ASYNCAWAIT	Set to 1 if the memory supports this feature. Otherwise keep at 0.
14	EXTMOD	0x0
13	WAITEN	0x0 (no effect in Asynchronous mode)
12	WREN	As needed
11	WAITCFG	Don't care
10	Reserved	0x0
9	WAITPOL	Meaningful only if bit 15 is 1
8	BURSTEN	0x0
7	Reserved	0x1

Table 219. FMC_BCRx bitfields (Muxed mode) (continued)

Bit number	Bit name	Value to set
6	FACCEN	0x1
5:4	MWID	As needed
3:2	MTYP	0x2 (NOR flash memory) or 0x1(PSRAM)
1	MUXEN	0x1
0	MBKEN	0x1

Table 220. FMC_BTRx bitfields (Muxed mode)

Bit number	Bit name	Value to set
31:30	DATAHLD	Duration of the data hold phase (DATAHLD HCLK cycles for read accesses, DATAHLD+1 HCLK cycles for write accesses).
29:28	ACCMOD	0x0
27:24	DATLAT	Don't care
23:20	CLKDIV	Don't care
19:16	BUSTURN	Time between NEx high to NEx low (BUSTURN HCLK).
15:8	DATAST	Duration of the second access phase (DATAST HCLK cycles).
7:4	ADDHLD	Duration of the middle phase of the access (ADDHLD HCLK cycles).
3:0	ADDSET	Duration of the first access phase (ADDSET HCLK cycles). Minimum value for ADDSET is 1.

WAIT management in asynchronous accesses

If the asynchronous memory asserts the WAIT signal to indicate that it is not yet ready to accept or to provide data, the ASYNCWAIT bit has to be set in FMC_BCRx register.

If the WAIT signal is active (high or low depending on the WAITPOL bit), the second access phase (Data setup phase), programmed by the DATAST bits, is extended until WAIT becomes inactive. Unlike the data setup phase, the first access phases (Address setup and Address hold phases), programmed by the ADDSET and ADDHLD bits, are not WAIT sensitive and so they are not prolonged.

The data setup phase must be programmed so that WAIT can be detected 4 HCLK cycles before the end of the memory transaction. The following cases must be considered:

1. The memory asserts the WAIT signal aligned to NOE/NWE which toggles:

$$\text{DATAST} \geq (4 \times \text{HCLK}) + \text{max_wait_assertion_time}$$

2. The memory asserts the WAIT signal aligned to NEx (or NOE/NWE not toggling):
if

$$\text{max_wait_assertion_time} > \text{address_phase} + \text{hold_phase}$$

then:

$$\text{DATAST} \geq (4 \times \text{HCLK}) + (\text{max_wait_assertion_time} - \text{address_phase} - \text{hold_phase})$$

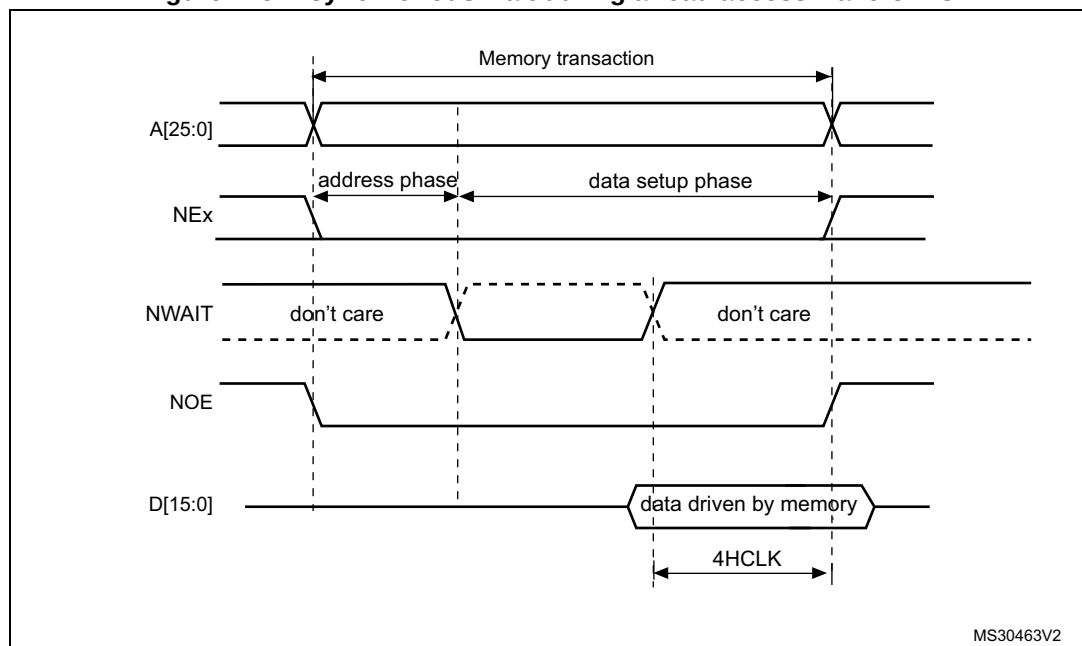
otherwise

$$\text{DATAST} \geq 4 \times \text{HCLK}$$

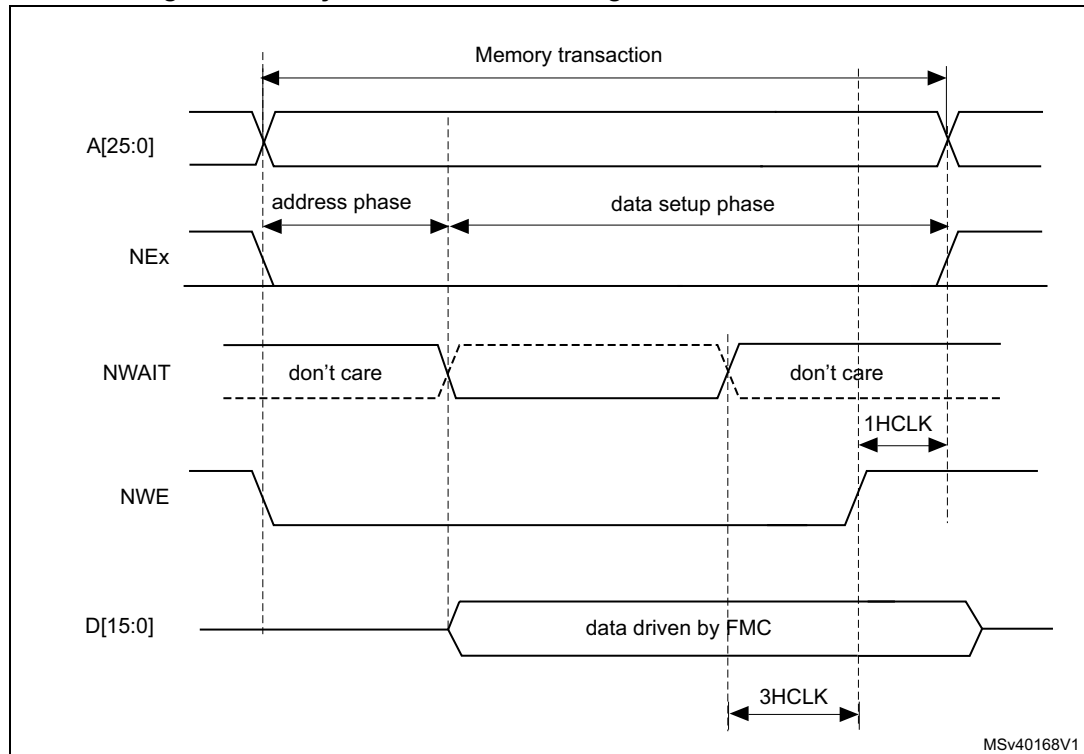
where max_wait_assertion_time is the maximum time taken by the memory to assert the WAIT signal once NEx/NOE/NWE is low.

Figure 145 and Figure 146 show the number of HCLK clock cycles that are added to the memory access phase after WAIT is released by the asynchronous memory (independently of the above cases).

Figure 145. Asynchronous wait during a read access waveforms



1. NWAIT polarity depends on WAITPOL bit setting in FMC_BCRx register.

Figure 146. Asynchronous wait during a write access waveforms

1. NWAIT polarity depends on WAITPOL bit setting in FMC_BCRx register.

CellularRAM™ (PSRAM) refresh management

The CellularRAM™ does not enable maintaining the chip select signal (NE) low for longer than the t_{CEM} timing specified for the memory device. This timing can be programmed in the FMC_PCSCNTR register. It defines the maximum duration of the NE low pulse in HCLK cycles for asynchronous accesses and FMC_CLK cycles for synchronous accesses

23.6.5 Synchronous transactions

The memory clock, FMC_CLK, is a submultiple of HCLK. It depends on the value of CLKDIV and the MWID/ AHB data size, following the formula given below:

Whatever MWID size: 16 or 8-bit, the FMC_CLK divider ratio is always defined by the programmed CLKDIV value.

Example:

- If CLKDIV=1, MWID = 16 bits, AHB data size=8 bits, FMC_CLK=HCLK/2.

NOR flash memories specify a minimum time from NADV assertion to CLK high. To meet this constraint, the FMC does not issue the clock to the memory during the first internal clock cycle of the synchronous access (before NADV assertion). This guarantees that the rising edge of the memory clock occurs in the middle of the NADV low pulse.

Data latency versus NOR memory latency

The data latency is the number of cycles to wait before sampling the data. The DATLAT value must be consistent with the latency value specified in the NOR flash configuration

register. The FMC does not include the clock cycle when NADV is low in the data latency count.

Caution: Some NOR flash memories include the NADV Low cycle in the data latency count, so that the exact relation between the NOR flash latency and the FMC DATLAT parameter can be either:

- NOR flash latency = (DATLAT + 2) CLK clock cycles
- or NOR flash latency = (DATLAT + 3) CLK clock cycles

Some recent memories assert NWAIT during the latency phase. In such cases DATLAT can be set to its minimum value. As a result, the FMC samples the data and waits long enough to evaluate if the data are valid. Thus the FMC detects when the memory exits latency and real data are processed.

Other memories do not assert NWAIT during latency. In this case the latency must be set correctly for both the FMC and the memory, otherwise invalid data are mistaken for good data, or valid data are lost in the initial phase of the memory access.

Single-burst transfer

When the selected bank is configured in Burst mode for synchronous accesses, if for example an AHB single-burst transaction is requested on 16-bit memories, the FMC performs a burst transaction of length 1 (if the AHB transfer is 16 bits), or length 2 (if the AHB transfer is 32 bits) and de-assert the chip select signal when the last data is strobed.

Such transfers are not the most efficient in terms of cycles compared to asynchronous read operations. Nevertheless, a random asynchronous access would first require to re-program the memory access mode, which would altogether last longer.

Cross boundary page for CellularRAM™ 1.5

CellularRAM™ 1.5 does not allow burst access to cross the page boundary. The FMC controller is used to split automatically the burst access when the memory page size is reached by configuring the CPSIZE bits in the FMC_BCR1 register following the memory page size.

Wait management

For synchronous NOR flash memories, NWAIT is evaluated after the programmed latency period, which corresponds to (DATLAT+2) CLK clock cycles.

If NWAIT is active (low level when WAITPOL = 0, high level when WAITPOL = 1), wait states are inserted until NWAIT is inactive (high level when WAITPOL = 0, low level when WAITPOL = 1).

When NWAIT is inactive, the data is considered valid either immediately (bit WAITCFG = 1) or on the next clock edge (bit WAITCFG = 0).

During wait-state insertion via the NWAIT signal, the controller continues to send clock pulses to the memory, keeping the chip select and output enable signals valid. It does not consider the data as valid.

In Burst mode, there are two timing configurations for the NOR flash NWAIT signal:

- The flash memory asserts the NWAIT signal one data cycle before the wait state (default after reset).
- The flash memory asserts the NWAIT signal during the wait state

The FMC supports both NOR flash wait state configurations, for each chip select, thanks to the WAITCFG bit in the FMC_BCRx registers ($x = 0..3$).

Figure 147. Wait configuration waveforms

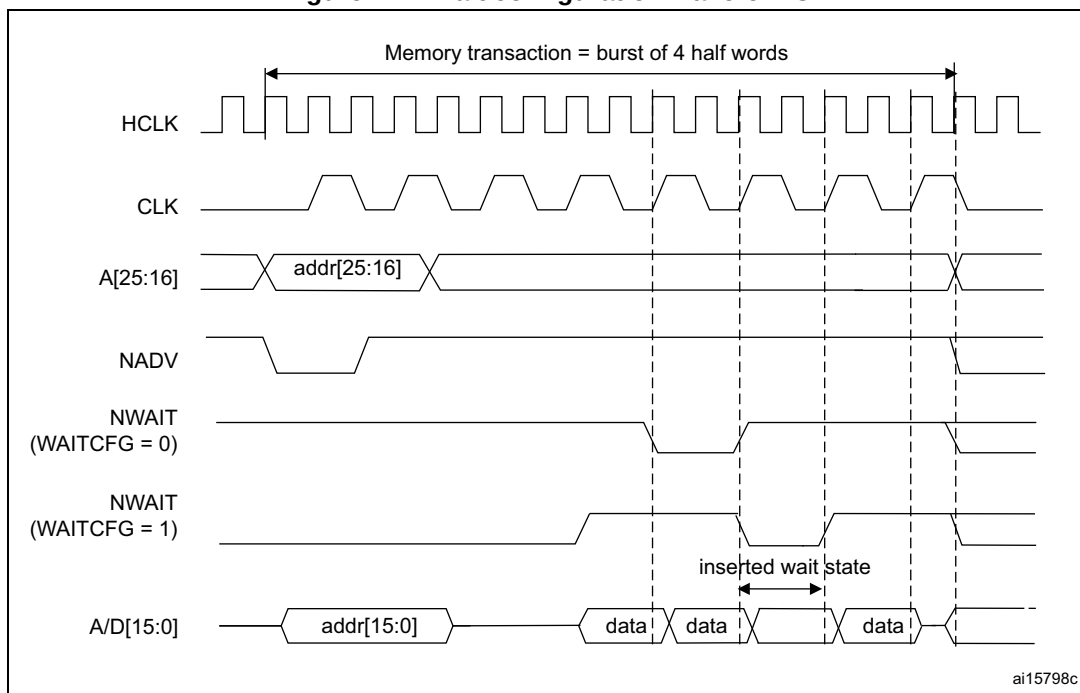
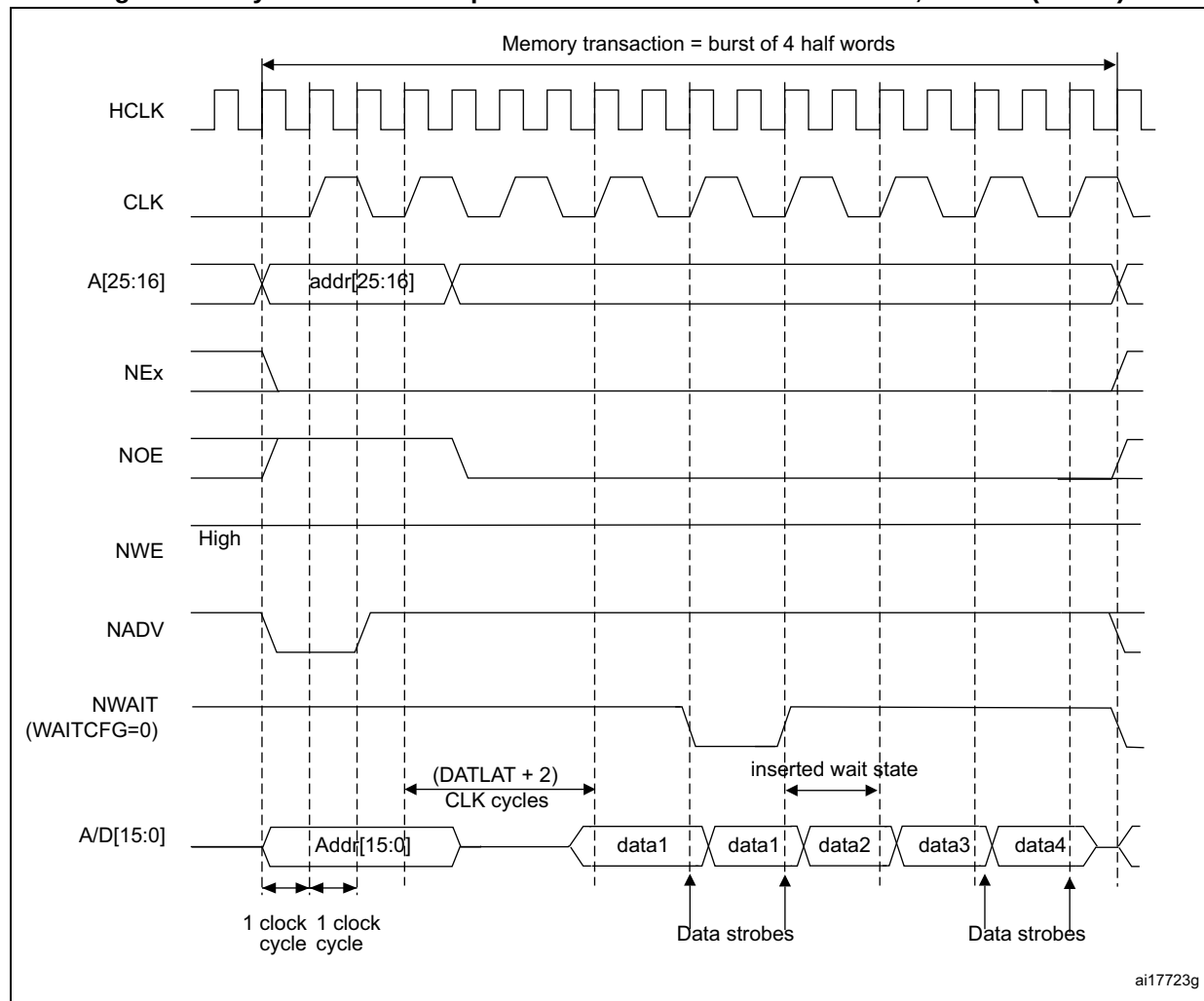


Figure 148. Synchronous multiplexed read mode waveforms - NOR, PSRAM (CRAM)

1. Byte lane outputs (NBL are not shown; for NOR access, they are held high, and, for PSRAM (CRAM) access, they are held low.

Table 221. FMC_BCRx bitfields (Synchronous multiplexed read mode)

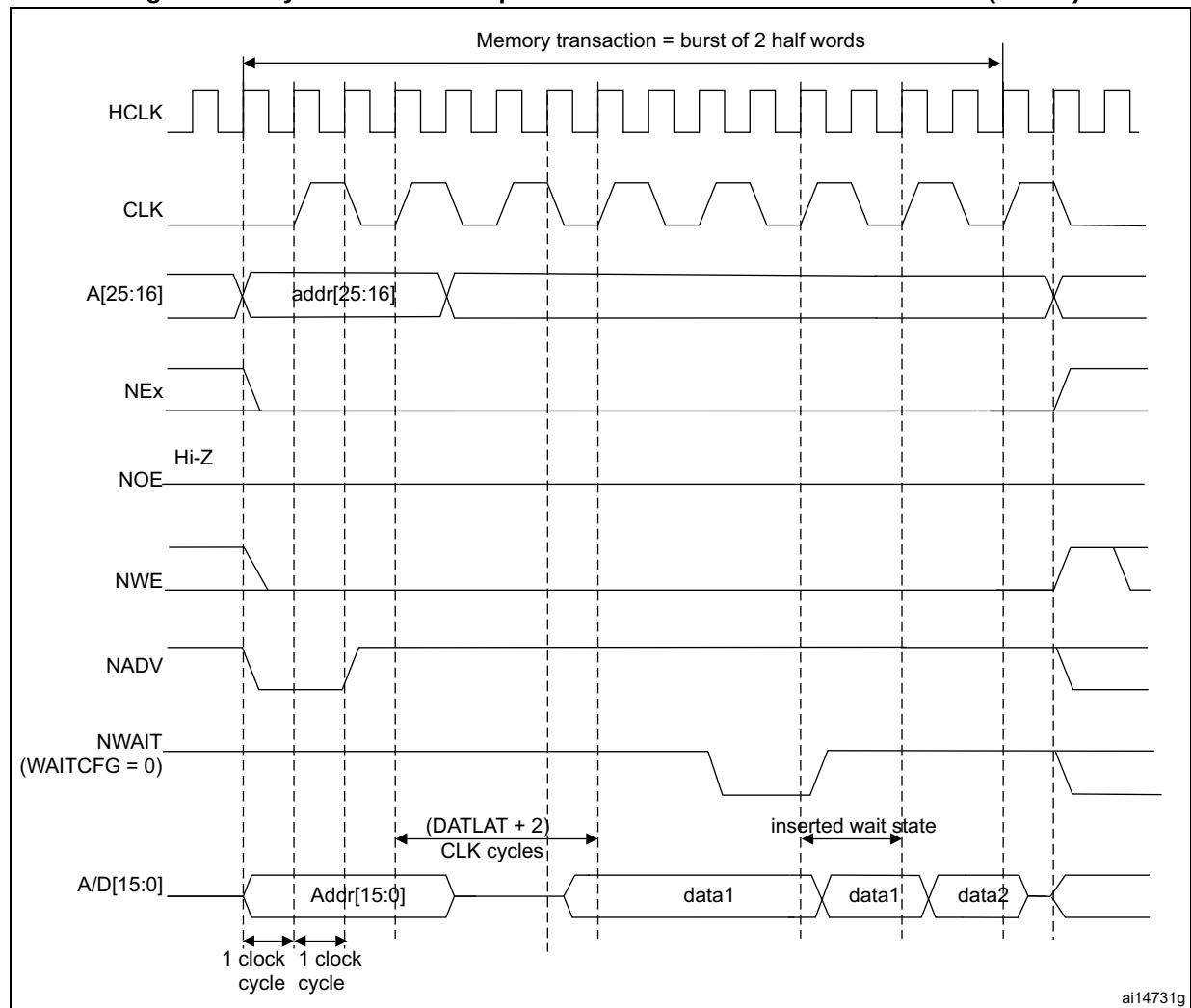
Bit number	Bit name	Value to set
31	FMCEN	0x1
30:24	Reserved	0x000
23:22	NBLSET[1:0]	Don't care
20	CCLKEN	As needed
19	CBURSTRW	No effect on synchronous read
18:16	CPSIZE	0x0 (no effect in Asynchronous mode)
15	ASYNCAWAIT	0x0
14	EXTMOD	0x0
13	WAITEN	To be set to 1 if the memory supports this feature, to be kept at 0 otherwise

Table 221. FMC_BCRx bitfields (Synchronous multiplexed read mode) (continued)

Bit number	Bit name	Value to set
12	WREN	No effect on synchronous read
11	WAITCFG	To be set according to memory
10	Reserved	0x0
9	WAITPOL	To be set according to memory
8	BURSTEN	0x1
7	Reserved	0x1
6	FACCEN	Set according to memory support (NOR flash memory)
5-4	MWID	As needed
3-2	MTYP	0x1 or 0x2
1	MUXEN	As needed
0	MBKEN	0x1

Table 222. FMC_BTRx bitfields (Synchronous multiplexed read mode)

Bit number	Bit name	Value to set
31:30	DATAHLD	Don't care
29:28	ACCMOD	0x0
27-24	DATLAT	Data latency
27-24	DATLAT	Data latency
23-20	CLKDIV	0x0 to get CLK = HCLK 0x1 to get CLK = 2 × HCLK ..
19-16	BUSTURN	Time between NEx high to NEx low (BUSTURN HCLK).
15-8	DATAST	Don't care
7-4	ADDHLD	Don't care
3-0	ADDSET	Don't care

Figure 149. Synchronous multiplexed write mode waveforms - PSRAM (CRAM)

1. The memory must issue NWAIT signal one cycle in advance, accordingly WAITCFG must be programmed to 0.
2. Byte Lane (NBL) outputs are not shown, they are held low while NEx is active.

Table 223. FMC_BCRx bitfields (Synchronous multiplexed write mode)

Bit number	Bit name	Value to set
31	FMCEN	0x1
30:24	Reserved	0x000
23:22	NBLSET[1:0]	Don't care
20	CCLKEN	As needed
19	CBURSTRW	0x1
18:16	CPSIZE	As needed (0x1 for CRAM 1.5)
15	ASYNCAWAIT	0x0
14	EXTMOD	0x0

Table 223. FMC_BCRx bitfields (Synchronous multiplexed write mode) (continued)

Bit number	Bit name	Value to set
13	WAITEN	To be set to 1 if the memory supports this feature, to be kept at 0 otherwise.
12	WREN	0x1
11	WAITCFG	0x0
10	Reserved	0x0
9	WAITPOL	to be set according to memory
8	BURSTEN	no effect on synchronous write
7	Reserved	0x1
6	FACCEN	Set according to memory support
5-4	MWID	As needed
3-2	MTYP	0x1
1	MUXEN	As needed
0	MBKEN	0x1

Table 224. FMC_BTRx bitfields (Synchronous multiplexed write mode)

Bit number	Bit name	Value to set
31-30	DATAHLD	Don't care
29:28	ACCMOD	0x0
27-24	DATLAT	Data latency
23-20	CLKDIV	0x0 to get CLK = HCLK 0x1 to get CLK = 2 × HCLK
19-16	BUSTURN	Time between NEx high to NEx low (BUSTURN HCLK).
15-8	DATAST	Don't care
7-4	ADDHLD	Don't care
3-0	ADDSET	Don't care

23.6.6 NOR/PSRAM controller registers

SRAM/NOR-flash chip-select control register for bank x (FMC_BCRx)

Address offset: $0x00 + 0x8 * (x - 1)$, ($x = 1$ to 4)

Reset value: 0x0000 30DB, 0x0000 30D2, 0x0000 30D2, 0x0000 30D2

This register contains the control information of each memory bank, used for SRAMs, PSRAM, FRAM and NOR flash memories.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
FMCEN	Res.	Res.	Res.	Res.	Res.	Res.	Res.	NBLSET[1:0]		WFDIS	CCLK EN	CBURST RW	CPSIZE[2:0]		
rw								rw	rw	rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
ASYNC WAIT	EXT MOD	WAIT EN	WREN	WAIT CFG	Res.	WAIT POL	BURST EN	Res.	FACC EN	MWID[1:0]		MTYP[1:0]		MUX EN	MBK EN
rw	rw	rw	rw	rw		rw	rw		rw	rw	rw	rw	rw	rw	rw

Bit 31 **FMCEN**: FMC controller enable

This bit enables or disables the FMC controller.

0: Disable the FMC controller

1: Enable the FMC controller

Note: The FMCEN bit of the FMC_BCR2..4 registers is don't care. It is only enabled through the FMC_BCR1 register.

Bits 30:24 Reserved, must be kept at reset value.

Bits 23:22 **NBLSET[1:0]**: Byte lane (NBL) setup

These bits configure the NBL setup timing from NBLx low to chip select NEx low.

00: NBL setup time is 0 AHB clock cycle

01: NBL setup time is 1 AHB clock cycle

10: NBL setup time is 2 AHB clock cycles

11: NBL setup time is 3 AHB clock cycles

Bit 21 **WFDIS**: Write FIFO disable

This bit disables the Write FIFO used by the FMC controller.

0: Write FIFO enabled (Default after reset)

1: Write FIFO disabled

Note: The WFDIS bit of the FMC_BCR2..4 registers is don't care. It is only enabled through the FMC_BCR1 register.

Bit 20 CCLKEN: Continuous clock enable

This bit enables the FMC_CLK clock output to external memory devices.

0: The FMC_CLK is only generated during the synchronous memory access (read/write transaction). The FMC_CLK clock ratio is specified by the programmed CLKDIV value in the FMC_BCRx register (default after reset).

1: The FMC_CLK is generated continuously during asynchronous and synchronous access. The FMC_CLK clock is activated when the CCLKEN is set.

Note: The CCLKEN bit of the FMC_BCR2..4 registers is don't care. It is only enabled through the FMC_BCR1 register. Bank 1 must be configured in Synchronous mode to generate the FMC_CLK continuous clock.

Note: If CCLKEN bit is set, the FMC_CLK clock ratio is specified by CLKDIV value in the FMC_BTR1 register. CLKDIV in FMC_BWTR1 is don't care.

Note: If the Synchronous mode is used and CCLKEN bit is set, the synchronous memories connected to other banks than Bank 1 are clocked by the same clock (the CLKDIV value in the FMC_BTR2..4 and FMC_BWTR2..4 registers for other banks has no effect.)

Bit 19 CBURSTRW: Write burst enable

For PSRAM (CRAM) operating in Burst mode, the bit enables synchronous accesses during write operations. The enable bit for synchronous read accesses is the BURSTEN bit in the FMC_BCRx register.

0: Write operations are always performed in Asynchronous mode.

1: Write operations are performed in Synchronous mode.

Bits 18:16 CPSIZE[2:0]: CRAM page size

These are used for CellularRAM™ 1.5 which does not allow burst access to cross the address boundaries between pages. When these bits are configured, the FMC controller splits automatically the burst access when the memory page size is reached (refer to memory datasheet for page size).

000: No burst split when crossing page boundary (default after reset)

001: 128 bytes

010: 256 bytes

011: 512 bytes

100: 1024 bytes

Others: Reserved, must not be used

Bit 15 ASYNCWAIT: Wait signal during asynchronous transfers

This bit enables/disables the FMC to use the wait signal even during an asynchronous protocol.

0: NWAIT signal is not taken in to account when running an asynchronous protocol (default after reset).

1: NWAIT signal is taken in to account when running an asynchronous protocol.

Bit 14 EXTMOD: Extended mode enable

This bit enables the FMC to program the write timings for non multiplexed asynchronous accesses inside the FMC_BWTR register, thus resulting in different timings for read and write operations.

0: values inside FMC_BWTR register are not taken into account (default after reset)

1: values inside FMC_BWTR register are taken into account

Note: When the Extended mode is disabled, the FMC can operate in mode 1 or mode 2 as follows:

- *Mode 1 is the default mode when the SRAM/PSRAM memory type is selected (MTYP = 0x0 or 0x01)*
- *Mode 2 is the default mode when the NOR memory type is selected (MTYP = 0x10).*

- Bit 13 **WAITEN**: Wait enable bit
This bit enables/disables wait-state insertion via the NWAIT signal when accessing the memory in Synchronous mode.
0: NWAIT signal is disabled (its level not taken into account, no wait state inserted after the programmed flash latency period).
1: NWAIT signal is enabled (its level is taken into account after the programmed latency period to insert wait states if asserted) (default after reset).
- Bit 12 **WREN**: Write enable bit
This bit indicates whether write operations are enabled/disabled in the bank by the FMC.
0: Write operations are disabled in the bank by the FMC, an AHB error is reported.
1: Write operations are enabled for the bank by the FMC (default after reset).
- Bit 11 **WAITCFG**: Wait timing configuration
The NWAIT signal indicates whether the data from the memory are valid or if a wait state must be inserted when accessing the memory in Synchronous mode. This configuration bit determines if NWAIT is asserted by the memory one clock cycle before the wait state or during the wait state:
0: NWAIT signal is active one data cycle before wait state (default after reset).
1: NWAIT signal is active during wait state (not used for PSRAM).
- Bit 10 Reserved, must be kept at reset value.
- Bit 9 **WAITPOL**: Wait signal polarity bit
Defines the polarity of the wait signal from memory used for either in Synchronous or Asynchronous mode.
0: NWAIT active low (default after reset)
1: NWAIT active high
- Bit 8 **BURSTEN**: Burst enable bit
This bit enables/disables synchronous accesses during read operations. It is valid only for synchronous memories operating in Burst mode.
0: Burst mode disabled (default after reset). Read accesses are performed in Asynchronous mode.
1: Burst mode enable. Read accesses are performed in Synchronous mode.
- Bit 7 Reserved, must be kept at reset value.
- Bit 6 **FACCEN**: Flash access enable
Enables NOR flash memory access operations.
0: Corresponding NOR flash memory access is disabled.
1: Corresponding NOR flash memory access is enabled (default after reset).
- Bits 5:4 **MWID[1:0]**: Memory data bus width
Defines the external memory device width, valid for all type of memories.
00: 8 bits
01: 16 bits (default after reset)
10: reserved
11: reserved
- Bits 3:2 **MTYP[1:0]**: Memory type
Defines the type of external memory attached to the corresponding memory bank.
00: SRAM/FRAM (default after reset for Bank 2...4)
01: PSRAM (CRAM) / FRAM
10: NOR flash/OneNAND flash (default after reset for Bank 1)
11: reserved

Bit 1 **MUXEN**: Address/data multiplexing enable bit

When this bit is set, the address and data values are multiplexed on the data bus, valid only with NOR and PSRAM memories:

0: Address/data non multiplexed

1: Address/data multiplexed on databus (default after reset)

Bit 0 **MBKEN**: Memory bank enable bit

Enables the memory bank. After reset Bank1 is enabled, all others are disabled. Accessing a disabled bank causes an ERROR on AHB bus.

0: Corresponding memory bank is disabled.

1: Corresponding memory bank is enabled.

SRAM/NOR-flash chip-select timing register for bank x (FMC_BTRx)

Address offset: $0x04 + 0x8 * (x - 1)$, ($x = 1$ to 4)

Reset value: 0x0FFF FFFF

This register contains the control information of each memory bank, used for SRAMs, PSRAM and NOR flash memories. If the EXTMOD bit is set in the FMC_BCRx register, then this register is partitioned for write and read access, that is, 2 registers are available: one to configure read accesses (this register) and one to configure write accesses (FMC_BWTRx registers).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
DATAHLD[1:0]		ACCMOD[1:0]		DATLAT[3:0]				CLKDIV[3:0]				BUSTURN[3:0]			
rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
DATAST[7:0]								ADDHLD[3:0]				ADDSET[3:0]			
rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW

Bits 31:30 **DATAHLD[1:0]**: Data hold phase duration

These bits are written by software to define the duration of the data hold phase in HCLK cycles (refer to [Figure 132](#) to [Figure 144](#)), used in asynchronous accesses:

For read accesses

00: DATAHLD phase duration = 0 × HCLK clock cycle (default)

01: DATAHLD phase duration = 1 × HCLK clock cycle

10: DATAHLD phase duration = 2 × HCLK clock cycle

11: DATAHLD phase duration = 3 × HCLK clock cycle

For write accesses

00: DATAHLD phase duration = 1 × HCLK clock cycle (default)

01: DATAHLD phase duration = 2 × HCLK clock cycle

10: DATAHLD phase duration = 3 × HCLK clock cycle

11: DATAHLD phase duration = 4 × HCLK clock cycle

Bits 29:28 **ACCMOD[1:0]**: Access mode

Specifies the asynchronous access modes as shown in the timing diagrams. These bits are taken into account only when the EXTMOD bit in the FMC_BCRx register is 1.

00: Access mode A

01: Access mode B

10: Access mode C

11: Access mode D

- Bits 27:24 **DATLAT[3:0]**: (see note below bit descriptions): Data latency for synchronous memory
 For synchronous access with read/write Burst mode enabled (BURSTEN / CBURSTRW bits set), defines the number of memory clock cycles (+2) to issue to the memory before reading/writing the first data:
 This timing parameter is not expressed in HCLK periods, but in FMC_CLK periods.
 For asynchronous access, this value is don't care.
 0000: Data latency of 2 CLK clock cycles for first burst access
 1111: Data latency of 17 CLK clock cycles for first burst access (default value after reset)
- Bits 23:20 **CLKDIV[3:0]**: Clock divide ratio (for FMC_CLK signal)
 Defines the period of FMC_CLK clock output signal, expressed in number of HCLK cycles:
 0000: FMC_CLK period = 1x HCLK period
 0001: FMC_CLK period = 2 × HCLK periods
 0010: FMC_CLK period = 3 × HCLK periods
 1111: FMC_CLK period = 16 × HCLK periods (default value after reset)
 In asynchronous NOR flash, SRAM or PSRAM accesses, this value is don't care.
Note: Refer to [Section 23.6.5: Synchronous transactions](#) for FMC_CLK divider ratio formula)
- Bits 19:16 **BUSTURN[3:0]**: Bus turnaround phase duration
 These bits are written by software to add a delay at the end of current read or write transaction to next transaction on the same bank.
 This delay is used to match the minimum time between consecutive transactions (t_{EHEL} from NEx high to NEx low) and the maximum time needed by the memory to free the data bus after a read access (t_{EHQZ} , chip enable high to output Hi-Z). This delay is recommended for mode D and muxed mode. For non-muxed memory, the bus turnaround delay can be set to minimum value.
 $(BUSTURN + 1)HCLK \text{ period} \geq \max(t_{EHEL} \text{ min}, t_{EHQZ} \text{ max})$
 For FRAM memories, the bus turnaround delay must be configured to match the minimum tPC (precharge time) timings. The bus turnaround delay is inserted between any consecutive transactions on the same bank (read/read, write/write, read/write and write/read) to match the tPC memory timing. The chip select is toggling between any consecutive accesses.
 $(BUSTURN + 1)HCLK \text{ period} \geq t_{PC} \text{ min}$
 0000: BUSTURN phase duration = 1 HCLK clock cycle added
 ...
 1111: BUSTURN phase duration = 16 x HCLK clock cycles added (default value after reset)
- Bits 15:8 **DATAST[7:0]**: Data-phase duration
 These bits are written by software to define the duration of the data phase (refer to [Figure 132](#) to [Figure 144](#)), used in asynchronous accesses:
 0000 0000: Reserved
 0000 0001: DATAST phase duration = 1 × HCLK clock cycles
 0000 0010: DATAST phase duration = 2 × HCLK clock cycles
 ...
 1111 1111: DATAST phase duration = 255 × HCLK clock cycles (default value after reset)
 For each memory type and access mode data-phase duration, refer to the respective figure ([Figure 132](#) to [Figure 144](#)).
 Example: Mode 1, write access, DATAST=1: Data-phase duration= DATAST+1 = 2 HCLK clock cycles.
Note: In synchronous accesses, this value is don't care.

Bits 7:4 **ADDHLD[3:0]**: Address-hold phase duration

These bits are written by software to define the duration of the *address hold* phase (refer to [Figure 132](#) to [Figure 144](#)), used in mode D or multiplexed accesses:

0000: Reserved

0001: ADDHLD phase duration = $1 \times \text{HCLK}$ clock cycle

0010: ADDHLD phase duration = $2 \times \text{HCLK}$ clock cycle

...

1111: ADDHLD phase duration = $15 \times \text{HCLK}$ clock cycles (default value after reset)

For each access mode address-hold phase duration, refer to the respective figure ([Figure 132](#) to [Figure 144](#)).

Note: In synchronous accesses, this value is not used, the address hold phase is always 1 memory clock period duration.

Bits 3:0 **ADDSET[3:0]**: Address setup phase duration

These bits are written by software to define the duration of the *address setup* phase (refer to [Figure 132](#) to [Figure 144](#)), used in SRAMs, ROMs, asynchronous NOR flash and PSRAM:

0000: ADDSET phase duration = $0 \times \text{HCLK}$ clock cycle

...

1111: ADDSET phase duration = $15 \times \text{HCLK}$ clock cycles (default value after reset)

For each access mode address setup phase duration, refer to the respective figure ([Figure 132](#) to [Figure 144](#)).

Note: In synchronous accesses, this value is don't care.

In Muxed mode or mode D, the minimum value for ADDSET is 1.

In mode 1 and PSRAM memory, the minimum value for ADDSET is 1.

Note: PSRAMs (CRAMs) have a variable latency due to internal refresh. Therefore these memories issue the NWAIT signal during the whole latency phase to prolong the latency as needed.

With PSRAMs (CRAMs) the filled DATLAT must be set to 0, so that the FMC exits its latency phase soon and starts sampling NWAIT from memory, then starts to read or write when the memory is ready.

This method can be used also with the latest generation of synchronous flash memories that issue the NWAIT signal, unlike older flash memories (check the datasheet of the specific flash memory being used).

SRAM/NOR-flash write timing registers x (FMC_BWTRx)

Address offset: $0x104 + 0x8 * (x - 1)$, ($x = 1$ to 4)

Reset value: 0x0FFF FFFF

This register contains the control information of each memory bank. It is used for SRAMs, PSRAMs and NOR flash memories. When the EXTMOD bit is set in the FMC_BCRx register, then this register is active for write access.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
DATAHLD[1:0]		ACCMOD[1:0]		Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	BUSTURN[3:0]			
rw	rw	rw	rw									rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
DATAS7[7:0]								ADDHLD[3:0]				ADDSET[3:0]			
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bits 31:30 **DATAHLD[1:0]**: Data hold phase duration

These bits are written by software to define the duration of the data hold phase in HCLK cycles (refer to [Figure 132](#) to [Figure 144](#)), used in asynchronous write accesses:

- 00: DATAHLD phase duration = 1 × HCLK clock cycle (default)
- 01: DATAHLD phase duration = 2 × HCLK clock cycle
- 10: DATAHLD phase duration = 3 × HCLK clock cycle
- 11: DATAHLD phase duration = 4 × HCLK clock cycle

Bits 29:28 **ACCMOD[1:0]**: Access mode.

Specifies the asynchronous access modes as shown in the next timing diagrams. These bits are taken into account only when the EXTMOD bit in the FMC_BCRx register is 1.

- 00: Access mode A
- 01: Access mode B
- 10: Access mode C
- 11: Access mode D

Bits 27:20 Reserved, must be kept at reset value.

Bits 19:16 **BUSTURN[3:0]**: Bus turnaround phase duration

These bits are written by software to add a delay at the end of current write transaction to next transaction on the same bank.

For FRAM memories, the bus turnaround delay must be configured to match the minimum t_{PC} (precharge time) timings. The bus turnaround delay is inserted between any consecutive transactions on the same bank (read/read, write/write, read/write and write/read). The chip select is toggling between any consecutive accesses.

$(BUSTURN + 1)HCLK \text{ period} \geq t_{PC \text{ min}}$

0000: BUSTURN phase duration = 1 HCLK clock cycle added

...

1111: BUSTURN phase duration = 16 × HCLK clock cycles added (default value after reset)

Bits 15:8 **DATAST[7:0]**: Data-phase duration.

These bits are written by software to define the duration of the data phase (refer to [Figure 132](#) to [Figure 144](#)), used in asynchronous SRAM, PSRAM and NOR flash memory accesses:

0000 0000: Reserved

0000 0001: DATAST phase duration = 1 × HCLK clock cycles

0000 0010: DATAST phase duration = 2 × HCLK clock cycles

...

1111 1111: DATAST phase duration = 255 × HCLK clock cycles (default value after reset)

Bits 7:4 **ADDHLD[3:0]**: Address-hold phase duration.

These bits are written by software to define the duration of the *address hold* phase (refer to [Figure 141](#) to [Figure 144](#)), used in asynchronous multiplexed accesses:

0000: Reserved

0001: ADDHLD phase duration = 1 × HCLK clock cycle

0010: ADDHLD phase duration = 2 × HCLK clock cycle

...

1111: ADDHLD phase duration = 15 × HCLK clock cycles (default value after reset)

Note: In synchronous NOR flash accesses, this value is not used, the address hold phase is always 1 flash clock period duration.

Bits 3:0 **ADDSET[3:0]**: Address setup phase duration.

These bits are written by software to define the duration of the *address setup* phase in HCLK cycles (refer to [Figure 132](#) to [Figure 144](#)), used in asynchronous accesses:

0000: ADDSET phase duration = 0 × HCLK clock cycle

...

1111: ADDSET phase duration = 15 × HCLK clock cycles (default value after reset)

Note: In synchronous accesses, this value is not used, the address setup phase is always 1 flash clock period duration. In muxed mode, the minimum ADDSET value is 1.

PSRAM chip select counter register (FMC_PCSCNTR)

Address offset: 0x20

Reset value: 0x0000 0000

This register contains the PSRAM chip select counter value for Synchronous and Asynchronous modes. The chip select counter is common to all banks and can be enabled separately on each bank. During PSRAM read or write accesses, this value is loaded into a timer which is decremented while the NE signal is held low. When the timer reaches 0, the PSRAM controller splits the current access, toggles NE to allow PSRAM device refresh, and restarts a new access. The programmed counter value guarantees a maximum NE pulse width (t_{CEM}) as specified for PSRAM devices. The counter is reloaded and starts decrementing each time a new access is started by a transition of NE from high to low.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	CNTB4EN	CNTB3EN	CNTB2EN	CNTB1EN
												rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
CSCOUNT[15:0]															
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bits 31:20 Reserved, must be kept at reset value.

Bit 19 **CNTB4EN**: Counter Bank 4 enable

This bit enables the chip select counter for PSRAM/NOR Bank 4.

0: Counter disabled for Bank 4

1: Counter enabled for Bank 4

Bit 18 **CNTB3EN**: Counter Bank 3 enable

This bit enables the chip select counter for PSRAM/NOR Bank 3.

0: Counter disabled for Bank 3.

1: Counter enabled for Bank 3

Bit 17 **CNTB2EN**: Counter Bank 2 enable

This bit enables the chip select counter for PSRAM/NOR Bank 2.

0: Counter disabled for Bank 2

1: Counter enabled for Bank 2

Bit 16 **CNTB1EN**: Counter Bank 1 enable

This bit enables the chip select counter for PSRAM/NOR Bank 1.

0: Counter disabled for Bank 1

1: Counter enabled for Bank 1

Bits 15:0 **CSCOUNT[15:0]**: Chip select counter.

This bitfield is used to define the maximum duration of the chip select low, which is obtained by the formula:

$CSCOUNT[15:0] * T_{AHB}$, where T_{AHB} is the AHB clock period.

For refresh considerations, the PSRAM chip select must not stay low for more than $t_{CEM} = \sim 4 \mu s$.

CSCOUNT[15:0] applies both to asynchronous and synchronous modes.

When CSCOUNT[15:0] = 0x0000, the feature is disabled.

23.7 NAND flash controller

The FMC generates the appropriate signal timings to drive the following types of device:

- 8- and 16-bit NAND flash memories

The NAND bank is configured through dedicated registers ([Section 23.7.7](#)). The programmable memory parameters include access timings (shown in [Table 225](#)) and ECC configuration.

Table 225. Programmable NAND flash access parameters

Parameter	Function	Access mode	Unit	Min.	Max.
Memory setup time	Number of clock cycles (HCLK) required to set up the address before the command assertion	Read/Write	AHB clock cycle (HCLK)	1	255
Memory wait	Minimum duration (in HCLK clock cycles) of the command assertion	Read/Write	AHB clock cycle (HCLK)	2	255
Memory hold	Number of clock cycles (HCLK) during which the address must be held (as well as the data if a write access is performed) after the command de-assertion	Read/Write	AHB clock cycle (HCLK)	1	254
Memory databus high-Z	Number of clock cycles (HCLK) during which the data bus is kept in high-Z state after a write access has started	Write	AHB clock cycle (HCLK)	1	255

23.7.1 External memory interface signals

The following tables list the signals that are typically used to interface NAND flash memory.

Note: The prefix "N" identifies the signals which are active low.

8-bit NAND flash memory

Table 226. 8-bit NAND flash

FMC signal name	I/O	Function
A[17]	O	NAND flash address latch enable (ALE) signal
A[16]	O	NAND flash command latch enable (CLE) signal
D[7:0]	I/O	8-bit multiplexed, bidirectional address/data bus

Table 226. 8-bit NAND flash (continued)

FMC signal name	I/O	Function
NCE	O	Chip select
NOE(= NRE)	O	Output enable (memory signal name: read enable, NRE)
NWE	O	Write enable
NWAIT/INT	I	NAND flash ready/busy input signal to the FMC

Theoretically, there is no capacity limitation as the FMC can manage as many address cycles as needed.

16-bit NAND flash memory

Table 227. 16-bit NAND flash

FMC signal name	I/O	Function
A[17]	O	NAND flash address latch enable (ALE) signal
A[16]	O	NAND flash command latch enable (CLE) signal
D[15:0]	I/O	16-bit multiplexed, bidirectional address/data bus
NCE	O	Chip select
NOE(= NRE)	O	Output enable (memory signal name: read enable, NRE)
NWE	O	Write enable
NWAIT/INT	I	NAND flash ready/busy input signal to the FMC

Theoretically, there is no capacity limitation as the FMC can manage as many address cycles as needed.

23.7.2 NAND flash supported memories and transactions

Table 228 shows the supported devices, access modes and transactions. Transactions not allowed (or not supported) by the NAND flash controller are shown in gray.

Table 228. Supported memories and transactions

Device	Mode	R/W	AHB data size	Memory data size	Allowed/not allowed	Comments
NAND 8-bit	Asynchronous	R	8	8	Y	-
	Asynchronous	W	8	8	Y	-
	Asynchronous	R	16	8	Y	Split into 2 FMC accesses
	Asynchronous	W	16	8	Y	Split into 2 FMC accesses
	Asynchronous	R	32	8	Y	Split into 4 FMC accesses
	Asynchronous	W	32	8	Y	Split into 4 FMC accesses

Table 228. Supported memories and transactions (continued)

Device	Mode	R/W	AHB data size	Memory data size	Allowed/not allowed	Comments
NAND 16-bit	Asynchronous	R	8	16	Y	-
	Asynchronous	W	8	16	N	-
	Asynchronous	R	16	16	Y	-
	Asynchronous	W	16	16	Y	-
	Asynchronous	R	32	16	Y	Split into 2 FMC accesses
	Asynchronous	W	32	16	Y	Split into 2 FMC accesses

23.7.3 Timing diagrams for NAND flash memory

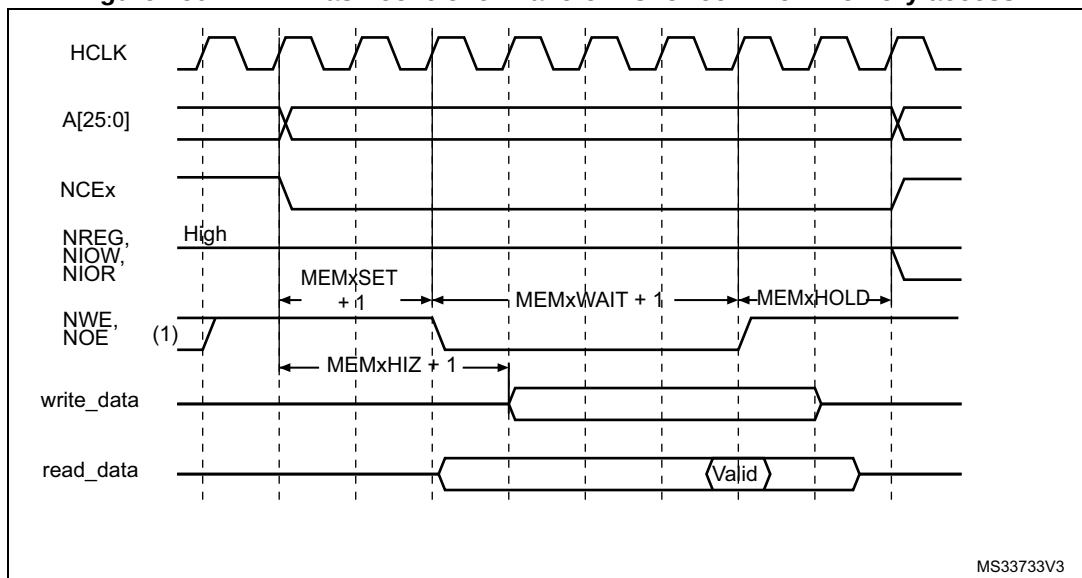
The NAND flash memory bank is managed through a set of registers:

- Control register: FMC_PCR
- Interrupt status register: FMC_SR
- ECC register: FMC_ECCR
- Timing register for Common memory space: FMC_PMEM
- Timing register for Attribute memory space: FMC_PATT

Each timing configuration register contains three parameters used to define number of HCLK cycles for the three phases of any NAND flash access, plus one parameter that defines the timing for starting driving the data bus when a write access is performed.

[Figure 150](#) shows the timing parameter definitions for common memory accesses, knowing that Attribute memory space access timings are similar.

Figure 150. NAND flash controller waveforms for common memory access



23.7.4 NAND flash operations

The command latch enable (CLE) and address latch enable (ALE) signals of the NAND flash memory device are driven by address signals from the FMC controller. This means that to send a command or an address to the NAND flash memory, the CPU has to perform a write to a specific address in its memory space.

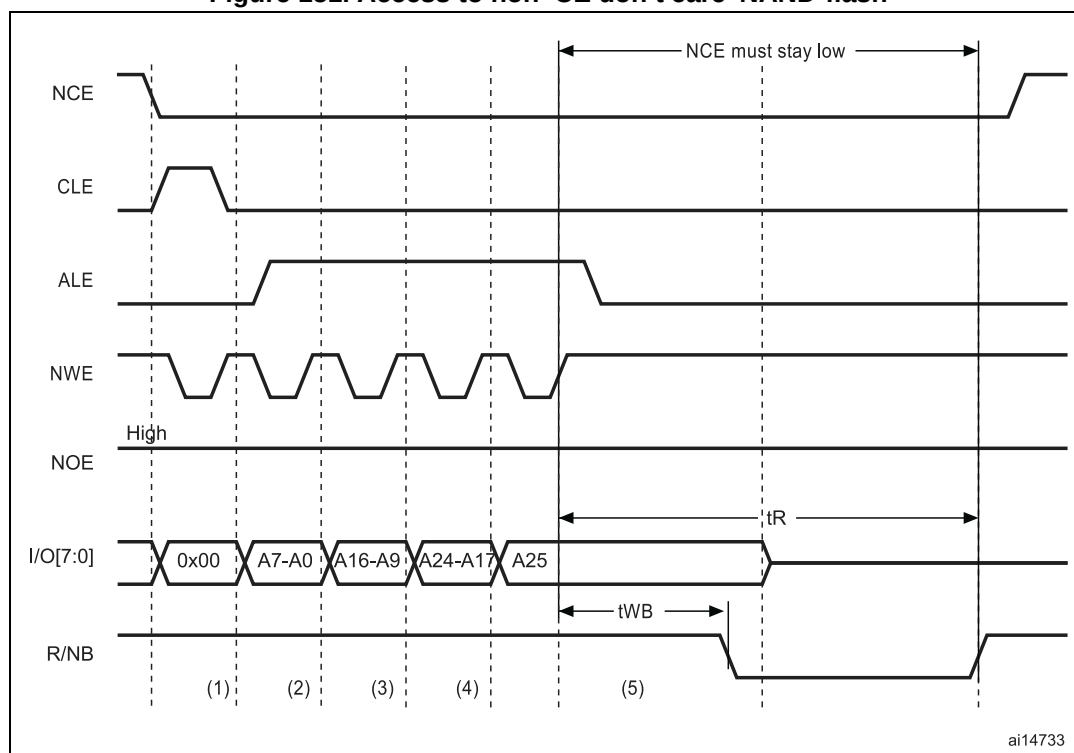
A typical page read operation from the NAND flash device requires the following steps:

1. Program and enable the corresponding memory bank by configuring the FMC_PCR and FMC_PMEM (and for some devices, FMC_PATT, see [Section 23.7.5: NAND flash prewait functionality](#)) registers according to the characteristics of the NAND flash memory (PWID bits for the data bus width of the NAND flash, PTYP = 1, PWAITEN = 0 or 1 as needed, see [Section 23.5.2: NAND flash memory address mapping](#) for timing configuration).
2. The CPU performs a byte write to the common memory space, with data byte equal to one flash command byte (for example 0x00 for Samsung NAND flash devices). The LE input of the NAND flash memory is active during the write strobe (low pulse on NWE), thus the written byte is interpreted as a command by the NAND flash memory. Once the command is latched by the memory device, it does not need to be written again for the following page read operations.
3. The CPU can send the start address (STARTAD) for a read operation by writing four bytes (or three for smaller capacity devices), STARTAD[7:0], STARTAD[16:9], STARTAD[24:17] and finally STARTAD[25] (for 64 Mb x 8 bit NAND flash memories) in the common memory or attribute space. The ALE input of the NAND flash device is active during the write strobe (low pulse on NWE), thus the written bytes are interpreted as the start address for read operations. Using the attribute memory space makes it possible to use a different timing configuration of the FMC, which can be used to implement the prewait functionality needed by some NAND flash memories (see details in [Section 23.7.5: NAND flash prewait functionality](#)).
4. The controller waits for the NAND flash memory to be ready (R/NB signal high), before starting a new access to the same or another memory bank. While waiting, the controller holds the NCE signal active (low).
5. The CPU can then perform byte read operations from the common memory space to read the NAND flash page (data field + Spare field) byte by byte.
6. The next NAND flash page can be read without any CPU command or address write operation. This can be done in three different ways:
 - by simply performing the operation described in step 5
 - a new random address can be accessed by restarting the operation at step 3
 - a new command can be sent to the NAND flash device by restarting at step 2

23.7.5 NAND flash prewait functionality

Some NAND flash devices require that, after writing the last part of the address, the controller waits for the R/NB signal to go low (see [Figure 151](#)).

Figure 151. Access to non 'CE don't care' NAND-flash



1. CPU wrote byte 0x00 at address 0x7001 0000.
2. CPU wrote byte A7~A0 at address 0x7002 0000.
3. CPU wrote byte A16~A9 at address 0x7002 0000.
4. CPU wrote byte A24~A17 at address 0x7002 0000.
5. CPU wrote byte A25 at address 0x7802 0000: FMC performs a write access using FMC_PATT timing definition, where $ATTHOLD \geq 7$ (providing that $(7+1) \times HCLK = 112 \text{ ns} > t_{WB \text{ max}}$). This guarantees that NCE remains low until R/NB goes low and high again (only requested for NAND flash memories where NCE is not don't care).

When this functionality is required, it can be ensured by programming the MEMHOLD value to meet the t_{WB} timing. However any CPU read access to the NAND flash memory has a hold delay of (MEMHOLD + 2) HCLK cycles and CPU write access has a hold delay of (MEMHOLD) HCLK cycles inserted between the rising edge of the NWE signal and the next access.

To cope with this timing constraint, the attribute memory space can be used by programming its timing register with an ATTHOLD value that meets the t_{WB} timing, and by keeping the MEMHOLD value at its minimum value. The CPU must then use the common memory space for all NAND flash read and write accesses, except when writing the last address byte to the NAND flash device, where the CPU must write to the attribute memory space.

23.7.6 Computation of the error correction code (ECC) in NAND flash memory

The FMC NAND Card controller includes two error correction code computation hardware blocks, one per memory bank. They reduce the host CPU workload when processing the ECC by software.

These two ECC blocks are identical and associated with Bank 2 and Bank 3. As a consequence, no hardware ECC computation is available for memories connected to Bank 4.

The ECC algorithm implemented in the FMC can perform 1-bit error correction and 2-bit error detection per 256, 512, 1 024, 2 048, 4 096 or 8 192 bytes read or written from/to the NAND flash memory. It is based on the Hamming coding algorithm and consists in calculating the row and column parity.

The ECC modules monitor the NAND flash data bus and read/write signals (NCE and NWE) each time the NAND flash memory bank is active.

The ECC operates as follows:

- When accessing NAND flash memory bank 2 or bank 3, the data present on the D[15:0] bus is latched and used for ECC computation.
- When accessing any other address in NAND flash memory, the ECC logic is idle, and does not perform any operation. As a result, write operations to define commands or addresses to the NAND flash memory are not taken into account for ECC computation.

Once the desired number of bytes has been read/written from/to the NAND flash memory by the host CPU, the FMC_ECCR registers must be read to retrieve the computed value. Once read, they must be cleared by resetting the ECCEN bit to '0'. To compute a new data block, the ECCEN bit must be set to one in the FMC_PCR registers.

To perform an ECC computation:

1. Enable the ECCEN bit in the FMC_PCR register.
2. Write data to the NAND flash memory page. While the NAND page is written, the ECC block computes the ECC value.
3. Read the ECC value available in the FMC_ECCR register and store it in a variable.
4. Clear the ECCEN bit and then enable it in the FMC_PCR register before reading back the written data from the NAND page. While the NAND page is read, the ECC block computes the ECC value.
5. Read the new ECC value available in the FMC_ECCR register.
6. If the two ECC values are the same, no correction is required, otherwise there is an ECC error and the software correction routine returns information on whether the error can be corrected or not.

23.7.7 NAND flash controller registers

NAND flash control registers (FMC_PCR)

Address offset: 0x80

Reset value: 0x0000 0018

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	ECCPS[2:0]			TAR3
												rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
TAR[2:0]			TCLR[3:0]				Res.	Res.	ECCEN	PWID[1:0]		PTYP	PBKEN	PWAITEN	Res.
rw	rw	rw	rw	rw	rw	rw			rw	rw	rw	rw	rw	rw	

Bits 31:20 Reserved, must be kept at reset value.

Bits 19:17 **ECCPS[2:0]**: ECC page size

Defines the page size for the extended ECC:

000: 256 bytes

001: 512 bytes

010: 1024 bytes

011: 2048 bytes

100: 4096 bytes

101: 8192 bytes

Bits 16:13 **TAR[3:0]**: ALE to RE delay

Sets time from ALE low to RE low in number of AHB clock cycles (HCLK).

Time is: $t_{ar} = (TAR + SET + 2) \times THCLK$ where THCLK is the HCLK clock period

0000: 1 HCLK cycle (default)

1111: 16 HCLK cycles

Note: SET is MEMSET or ATTSET according to the addressed space.

Bits 12:9 **TCLR[3:0]**: CLE to RE delay

Sets time from CLE low to RE low in number of AHB clock cycles (HCLK).

Time is: $t_{clr} = (TCLR + SET + 2) \times THCLK$ where THCLK is the HCLK clock period

0000: 1 HCLK cycle (default)

1111: 16 HCLK cycles

Note: SET is MEMSET or ATTSET according to the addressed space.

Bits 8:7 Reserved, must be kept at reset value.

Bit 6 **ECCEN**: ECC computation logic enable bit

0: ECC logic is disabled and reset (default after reset),

1: ECC logic is enabled.

Bits 5:4 **PWID[1:0]**: Data bus width

Defines the external memory device width.

00: 8 bits

01: 16 bits (default after reset).

10: reserved.

11: reserved.

Bit 3 **PTYP**: Memory type

Defines the type of device attached to the corresponding memory bank:

0: Reserved, must be kept at reset value

1: NAND flash (default after reset)

Bit 2 **PBKEN**: NAND flash memory bank enable bit

Enables the memory bank. Accessing a disabled memory bank causes an ERROR on AHB bus

0: Corresponding memory bank is disabled (default after reset)

1: Corresponding memory bank is enabled

Bit 1 **PWAITEN**: Wait feature enable bit

Enables the Wait feature for the NAND flash memory bank:

0: disabled

1: enabled

Bit 0 Reserved, must be kept at reset value.

FIFO status and interrupt register (FMC_SR)

Address offset: 0x84

Reset value: 0x0000 0040

This register contains information about the FIFO status and interrupt. The FMC features a FIFO that is used when writing to memories to transfer up to 16 words of data from the AHB.

This is used to quickly write to the FIFO and free the AHB for transactions to peripherals other than the FMC, while the FMC is draining its FIFO into the memory. One of these register bits indicates the status of the FIFO, for ECC purposes.

The ECC is calculated while the data are written to the memory. To read the correct ECC, the software must consequently wait until the FIFO is empty.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	FEMPT	IFEN	ILEN	IREN	IFS	ILS	IRS
									r	rw	rw	rw	rw	rw	rw

Bits 31:7 Reserved, must be kept at reset value.

Bit 6 **FEMPT**: FIFO empty

Read-only bit that provides the status of the FIFO

0: FIFO not empty

1: FIFO empty

Bit 5 **IFEN**: Interrupt falling edge detection enable bit

0: Interrupt falling edge detection request disabled

1: Interrupt falling edge detection request enabled

Bit 4 **ILEN**: Interrupt high-level detection enable bit

0: Interrupt high-level detection request disabled

1: Interrupt high-level detection request enabled

Bit 3 **IREN**: Interrupt rising edge detection enable bit

0: Interrupt rising edge detection request disabled

1: Interrupt rising edge detection request enabled

Bit 2 **IFS**: Interrupt falling edge status

The flag is set by hardware and reset by software.

0: No interrupt falling edge occurred

1: Interrupt falling edge occurred

Note: If this bit is written by software to 1 it is set.

Bit 1 **ILS**: Interrupt high-level status

The flag is set by hardware and reset by software.

0: No Interrupt high-level occurred

1: Interrupt high-level occurred

Bit 0 **IRS**: Interrupt rising edge status

The flag is set by hardware and reset by software.

0: No interrupt rising edge occurred

1: Interrupt rising edge occurred

Note: If this bit is written by software to 1 it is set.

Common memory space timing register (FMC_PMEM)

Address offset: 0x88

Reset value: 0xFCFC FCFC

The FMC_PMEM read/write register contains the timing information for NAND flash memory bank. This information is used to access either the common memory space of the NAND flash for command, address write access and data read/write access.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
MEMHIZ[7:0]								MEMHOLD[7:0]							
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
MEMWAIT[7:0]								MEMSET[7:0]							
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bits 31:24 **MEMHIZ[7:0]**: Common memory x data bus Hi-Z time

Defines the number of HCLK clock cycles during which the data bus is kept Hi-Z after the start of a NAND flash write access to common memory space on socket. This is only valid for write transactions:

0000 0000: 1 HCLK cycle

1111 1110: 255 HCLK cycles

1111 1111: reserved.

Bits 23:16 **MEMHOLD[7:0]**: Common memory hold time

Defines the number of HCLK clock cycles for write access and HCLK (+2) clock cycles for read access during which the address is held (and data for write accesses) after the command is deasserted (NWE, NOE), for NAND flash read or write access to common memory space on socket x:

0000 0000: reserved.

0000 0001: 1 HCLK cycle for write access / 3 HCLK cycles for read access

1111 1110: 254 HCLK cycles for write access / 256 HCLK cycles for read access

1111 1111: reserved.

Bits 15:8 **MEMWAIT[7:0]**: Common memory wait time

Defines the minimum number of HCLK (+1) clock cycles to assert the command (NWE, NOE), for NAND flash read or write access to common memory space on socket. The duration of command assertion is extended if the wait signal (NWAIT) is active (low) at the end of the programmed value of HCLK:

0000 0000: reserved

0000 0001: 2HCLK cycles (+ wait cycle introduced by deasserting NWAIT)

1111 1110: 255 HCLK cycles (+ wait cycle introduced by deasserting NWAIT)

1111 1111: reserved.

Bits 7:0 **MEMSET[7:0]**: Common memory x setup time

Defines the number of HCLK (+1) clock cycles to set up the address before the command assertion (NWE, NOE), for NAND flash read or write access to common memory space on socket x:

0000 0000: 1 HCLK cycle

1111 1110: 255 HCLK cycles

1111 1111: reserved

Attribute memory space timing register (FMC_PATT)

Address offset: 0x8C

Reset value: 0xFCFC FCFC

The FMC_PATT read/write register contains the timing information for NAND flash memory bank. It is used for 8-bit accesses to the attribute memory space of the NAND flash for the last address write access if the timing must differ from that of previous accesses (for Ready/Busy management, refer to [Section 23.7.5: NAND flash prewait functionality](#)).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
ATTTHIZ[7:0]								ATTTHOLD[7:0]							
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
ATTWAIT[7:0]								ATTSET[7:0]							
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bits 31:24 **ATTTHIZ[7:0]**: Attribute memory data bus Hi-Z time

Defines the number of HCLK clock cycles during which the data bus is kept in Hi-Z after the start of a NAND flash write access to attribute memory space on socket. Only valid for writ transaction:

0000 0000: 1 HCLK cycle

1111 1110: 255 HCLK cycles

1111 1111: reserved.

Bits 23:16 **ATTTHOLD[7:0]**: Attribute memory hold time

Defines the number of HCLK clock cycles for write access and HCLK (+2) clock cycles for read access during which the address is held (and data for write access) after the command deassertion (NWE, NOE), for NAND flash read or write access to attribute memory space on socket:

0000 0000: reserved

0000 0001: 1 HCLK cycle for write access / 3 HCLK cycles for read access

1111 1110: 254 HCLK cycles for write access / 256 HCLK cycles for read access

1111 1111: reserved.

Bits 15:8 **ATTWAIT[7:0]**: Attribute memory wait time

Defines the minimum number of HCLK (+1) clock cycles to assert the command (NWE, NOE), for NAND flash read or write access to attribute memory space on socket x. The duration for command assertion is extended if the wait signal (NWAIT) is active (low) at the end of the programmed value of HCLK:

0000 0000: reserved

0000 0001: 2 HCLK cycles (+ wait cycle introduced by deassertion of NWAIT)

1111 1110: 255 HCLK cycles (+ wait cycle introduced by deasserting NWAIT)

1111 1111: reserved.

Bits 7:0 **ATTSET[7:0]**: Attribute memory setup time

Defines the number of HCLK (+1) clock cycles to set up address before the command assertion (NWE, NOE), for NAND flash read or write access to attribute memory space on socket:

0000 0000: 1 HCLK cycle

1111 1110: 255 HCLK cycles

1111 1111: reserved.

ECC result registers (FMC_ECCR)

Address offset: 0x94

Reset value: 0x0000 0000

This register contains the current error correction code value computed by the ECC computation modules of the FMC NAND controller. When the CPU reads the data from a NAND flash memory page at the correct address (refer to [Section 23.7.6: Computation of the error correction code \(ECC\) in NAND flash memory](#)), the data read/written from/to the NAND flash memory are processed automatically by the ECC computation module. When X bytes have been read (according to the ECCPS field in the FMC_PCR registers), the CPU must read the computed ECC value from the FMC_ECC registers. It then verifies if these computed parity data are the same as the parity value recorded in the spare area, to determine whether a page is valid, and, to correct it otherwise. The FMC_ECCR register must be cleared after being read by setting the ECCEN bit to 0. To compute a new data block, the ECCEN bit must be set to 1.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
ECC[31:16]															
r	r	r	r	r	r	r	r	r	r	r	r	r	r	r	r
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
ECC[15:0]															
r	r	r	r	r	r	r	r	r	r	r	r	r	r	r	r

Bits 31:0 **ECC[31:0]**: ECC result

This field contains the value computed by the ECC computation logic. [Table 229](#) describes the contents of these bitfields.

Table 229. ECC result relevant bits

ECCPS[2:0]	Page size in bytes	ECC bits
000	256	ECC[21:0]
001	512	ECC[23:0]
010	1024	ECC[25:0]
011	2048	ECC[27:0]
100	4096	ECC[29:0]
101	8192	ECC[31:0]

23.8 SDRAM controller

23.8.1 SDRAM controller main features

The main features of the SDRAM controller are the following:

- Two SDRAM banks with independent configuration
- 8-bit, 16-bit data bus width
- 13-bits Address Row, 11-bits Address Column, 4 internal banks: 4x16Mx16bit (128 MB), 4x16Mx8bit (64 MB)
- Word, half-word, byte access
- Automatic row and bank boundary management
- Multibank ping-pong access
- Programmable timing parameters
- Automatic Refresh operation with programmable Refresh rate
- Self-refresh mode
- Power-down mode
- SDRAM power-up initialization by software
- CAS latency of 1,2,3
- Cacheable Read FIFO with depth of 6 lines x32-bit (6 x14-bit address tag)

23.8.2 SDRAM External memory interface signals

At startup, the SDRAM I/O pins used to interface the FMC SDRAM controller with the external SDRAM devices must be configured by the user application. The SDRAM controller I/O pins which are not used by the application, can be used for other purposes.

Table 230. SDRAM signals

SDRAM signal	I/O type	Description	Alternate function
SDCLK	O	SDRAM clock	-
SDCKE[1:0]	O	SDCKE0: SDRAM Bank 1 Clock Enable SDCKE1: SDRAM Bank 2 Clock Enable	-
SDNE[1:0]	O	SDNE0: SDRAM Bank 1 Chip Enable SDNE1: SDRAM Bank 2 Chip Enable	-
A[12:0]	O	Address	FMC_A[12:0]
D[15:0]	I/O	Bidirectional data bus	FMC_D[15:0]
BA[1:0]	O	Bank Address	FMC_A[15:14]
NRAS	O	Row Address Strobe	-
NCAS	O	Column Address Strobe	-
SDNWE	O	Write Enable	-
NBL[1:0]	O	Output Byte Mask for write accesses (memory signal name: DQM[1:0])	FMC_NBL[1:0]

23.8.3 SDRAM controller functional description

All SDRAM controller outputs (signals, address and data) change on the falling edge of the memory clock (FMC_SDCLK).

SDRAM initialization

The initialization sequence is managed by software. If the two banks are used, the initialization sequence must be generated simultaneously to Bank 1 and Bank 2 by setting the Target Bank bits CTB1 and CTB2 in the FMC_SDCMR register:

1. Program the memory device features into the FMC_SDCRx register. The SDRAM clock frequency, RBURST and RPIPE must be programmed in the FMC_SDCR1 register.
2. Program the memory device timing into the FMC_SDTRx register. The TRP and TRC timings must be programmed in the FMC_SDTR1 register.
3. Set MODE bits to '001' and configure the Target Bank bits (CTB1 and/or CTB2) in the FMC_SDCMR register to start delivering the clock to the memory (SDCKE is driven high).
4. Wait during the prescribed delay period. Typical delay is around 100 μ s (refer to the SDRAM datasheet for the required delay after power-up).
5. Set MODE bits to '010' and configure the Target Bank bits (CTB1 and/or CTB2) in the FMC_SDCMR register to issue a "Precharge All" command.
6. Set MODE bits to '011', and configure the Target Bank bits (CTB1 and/or CTB2) as well as the number of consecutive Auto-refresh commands (NRFS) in the FMC_SDCMR register. Refer to the SDRAM datasheet for the number of Auto-refresh commands that should be issued. Typical number is 8.
7. Configure the MRD field according to the SDRAM device, set the MODE bits to '100', and configure the Target Bank bits (CTB1 and/or CTB2) in the FMC_SDCMR register to issue a "Load Mode Register" command in order to program the SDRAM device. In particular:
 - a) the CAS latency must be selected following configured value in FMC_SDCR1/2 registers
 - b) the Burst Length (BL) of 1 must be selected by configuring the M[2:0] bits to 000 in the mode register. Refer to SDRAM device datasheet.

If the Mode Register is not the same for both SDRAM banks, this step has to be repeated twice, once for each bank, and the Target Bank bits set accordingly.
8. Program the refresh rate in the FMC_SDRTR register
The refresh rate corresponds to the delay between refresh cycles. Its value must be adapted to SDRAM devices.
9. For mobile SDRAM devices, to program the extended mode register it should be done once the SDRAM device is initialized: First, a dummy read access should be performed while BA1=1 and BA=0 (refer to SDRAM address mapping section for BA[1:0] address mapping) in order to select the extended mode register instead of the load mode register and then program the needed value.

At this stage the SDRAM device is ready to accept commands. If a system reset occurs during an ongoing SDRAM access, the data bus might still be driven by the SDRAM device. Therefore the SDRAM device must be first reinitialized after reset before issuing any new access by the NOR flash/PSRAM/SRAM or NAND flash controller.

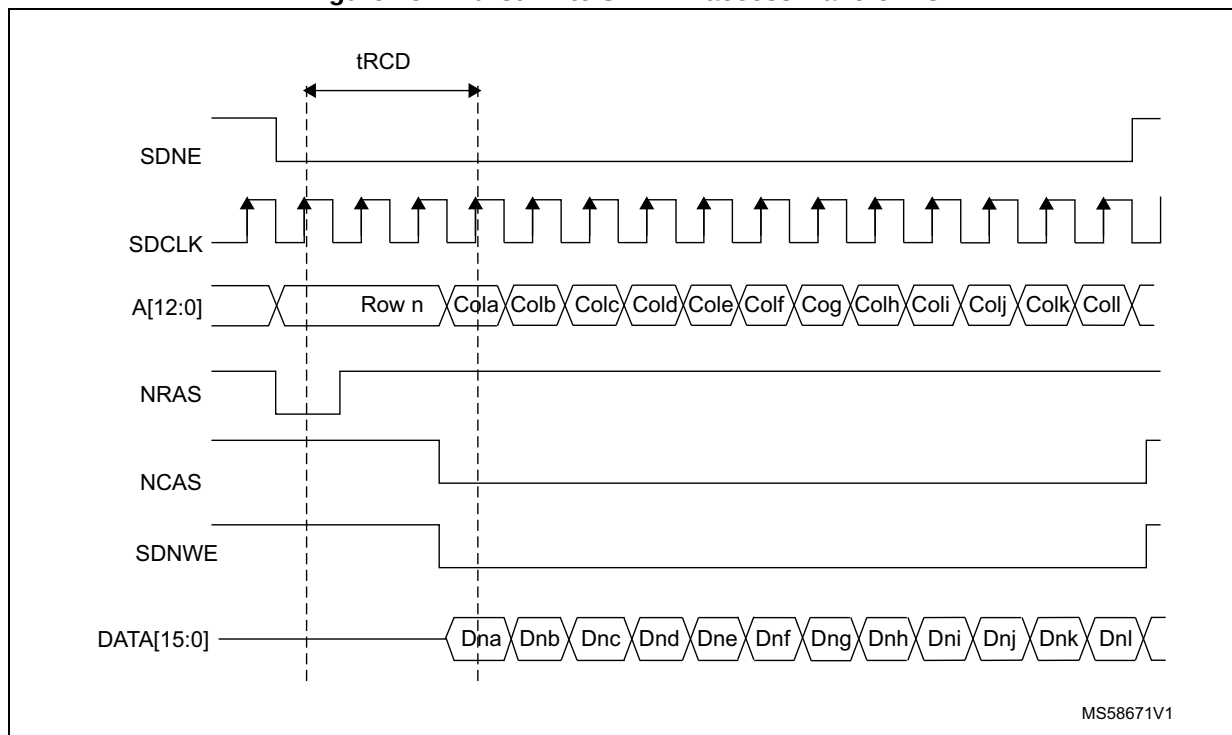
Note: If two SDRAM devices are connected to the FMC, all the accesses performed at the same time to both devices by the Command Mode register (Load Mode Register command) are issued using the timing parameters configured for SDRAM Bank 1 (TMRD and TRAS timings) in the FMC_SDTR1 register.

SDRAM controller write cycle

The SDRAM controller accepts single and burst write requests and translates them into single memory accesses. In both cases, the SDRAM controller keeps track of the active row for each bank to be able to perform consecutive write accesses to different banks (Multibank ping-pong access).

Before performing any write access, the SDRAM bank write protection must be disabled by clearing the WP bit in the FMC_SDCRx register.

Figure 152. Burst write SDRAM access waveforms



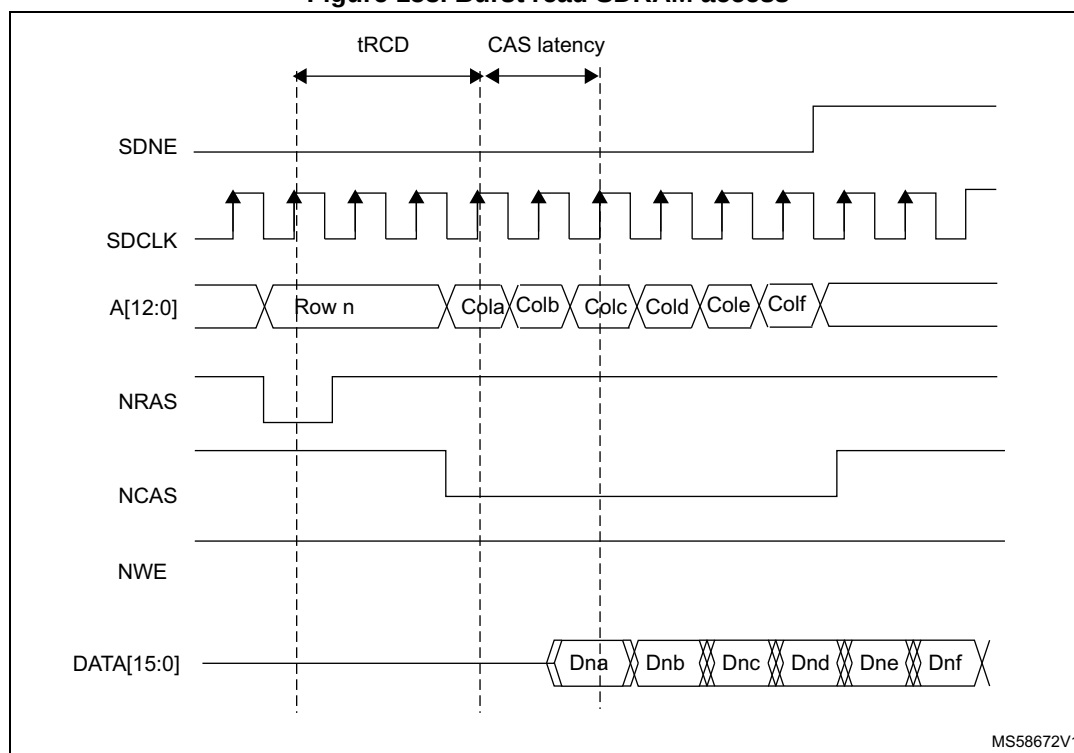
The SDRAM controller always checks the next access.

- If the next access is in the same row or in another active row, the write operation is carried out,
- if the next access targets another row (not active), the SDRAM controller generates a precharge command, activates the new row and initiates a write command.

SDRAM controller read cycle

The SDRAM controller accepts single and burst read requests and translates them into single memory accesses. In both cases, the SDRAM controller keeps track of the active row in each bank to be able to perform consecutive read accesses in different banks (Multibank ping-pong access).

Figure 153. Burst read SDRAM access



The FMC SDRAM controller features a Cacheable read FIFO (6 lines x 32 bits). It is used to store data read in advance during the CAS latency period and the RPIPE delay following the below formula. The RBURST bit must be set in the FMC_SDCR1 register to anticipate the next read access.

Number for anticipated data = $\text{CAS latency} + 1 + (\text{RPIPE delay})/2$

Examples:

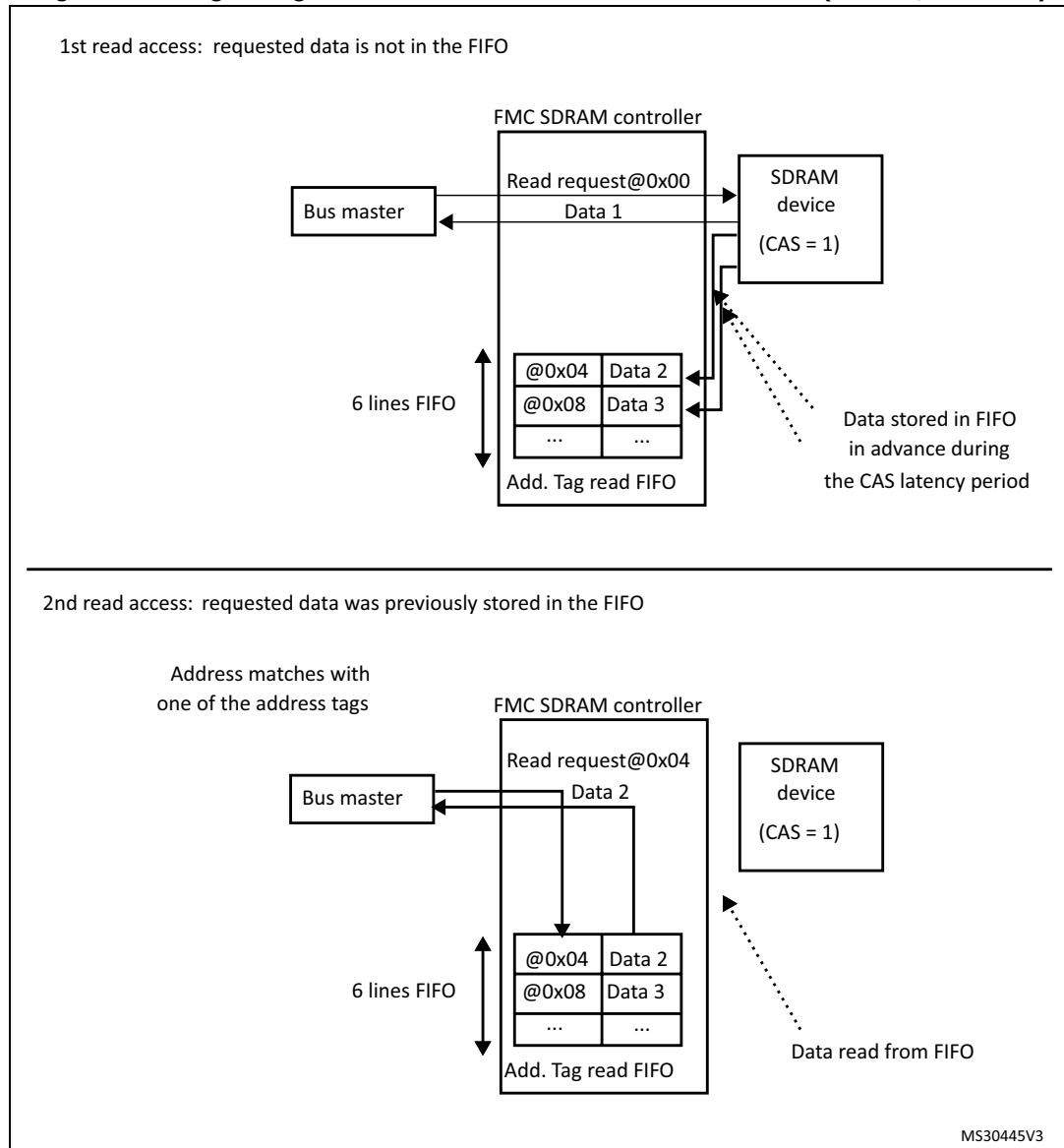
- CAS latency = 3, RPIPE delay = 0: Four data (not committed) are stored in the FIFO.
- CAS latency = 3, RPIPE delay = 2: Five data (not committed) are stored in the FIFO.

The read FIFO features a 14-bit address tag to each line to identify its content: 11 bits for the column address, 2 bits to select the internal bank and the active row, and 1 bit to select the SDRAM device

When the end of the row is reached in advance during an AHB burst read, the data read in advance (not committed) are not stored in the read FIFO. For single read access, data are correctly stored in the FIFO.

Each time a read request occurs, the SDRAM controller checks:

- If the address matches one of the address tags, data are directly read from the FIFO and the corresponding address tag/ line content is cleared and the remaining data in the FIFO are compacted to avoid empty lines.
- Otherwise, a new read command is issued to the memory and the FIFO is updated with new data. If the FIFO is full, the older data are lost.

Figure 154. Logic diagram of Read access with RBURST bit set (CAS=1, RPIPE=0)

During a write access or a Precharge command, the read FIFO is flushed and ready to be filled with new data.

After the first read request, if the current access was not performed to a row boundary, the SDRAM controller anticipates the next read access during the CAS latency period and the RPIPE delay (if configured). This is done by incrementing the memory address. The following condition must be met:

- RBURST control bit should be set to '1' in the FMC_SDCR1 register.

The address management depends on the next AHB request:

- Next AHB request is sequential (AHB Burst)
In this case, the SDRAM controller increments the address.
- Next AHB request is not sequential
 - If the new read request targets the same row or another active row, the new address is passed to the memory and the master is stalled for the CAS latency period, waiting for the new data from memory.
 - If the new read request does not target an active row, the SDRAM controller generates a Precharge command, activates the new row, and initiates a read command.

If the RURST is reset, the read FIFO is not used.

Row and bank boundary management

When a read or write access crosses a row boundary, if the next read or write access is sequential and the current access was performed to a row boundary, the SDRAM controller executes the following operations:

1. Precharge of the active row,
2. Activation of the new row
3. Start of a read/write command.

At a row boundary, the automatic activation of the next row is supported for all columns and data bus width configurations.

If necessary, the SDRAM controller inserts additional clock cycles between the following commands:

- Between Precharge and Activate commands to match TRP parameter (only if the next access is in a different row in the same bank),
- Between Activate and Read commands to match the TRCD parameter.

These parameters are defined into the FMC_SDTRx register.

Refer to [Figure 152](#) and [Figure 153](#) for read and burst write access crossing a row boundary.

Figure 155. Read access crossing row boundary

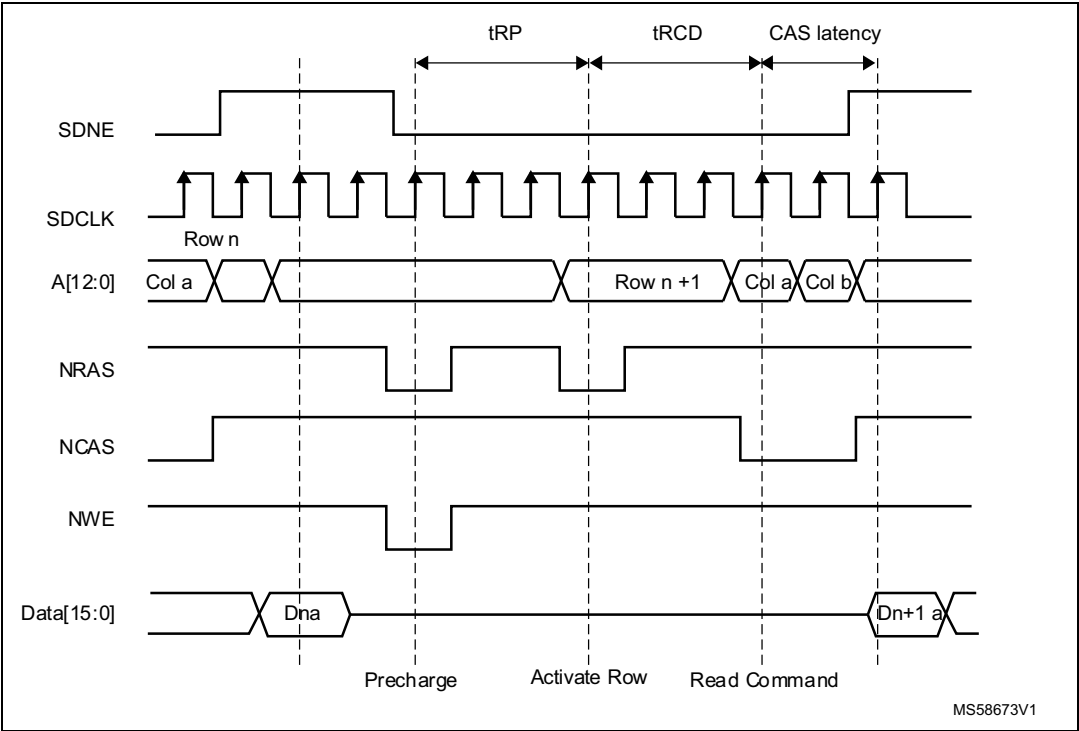
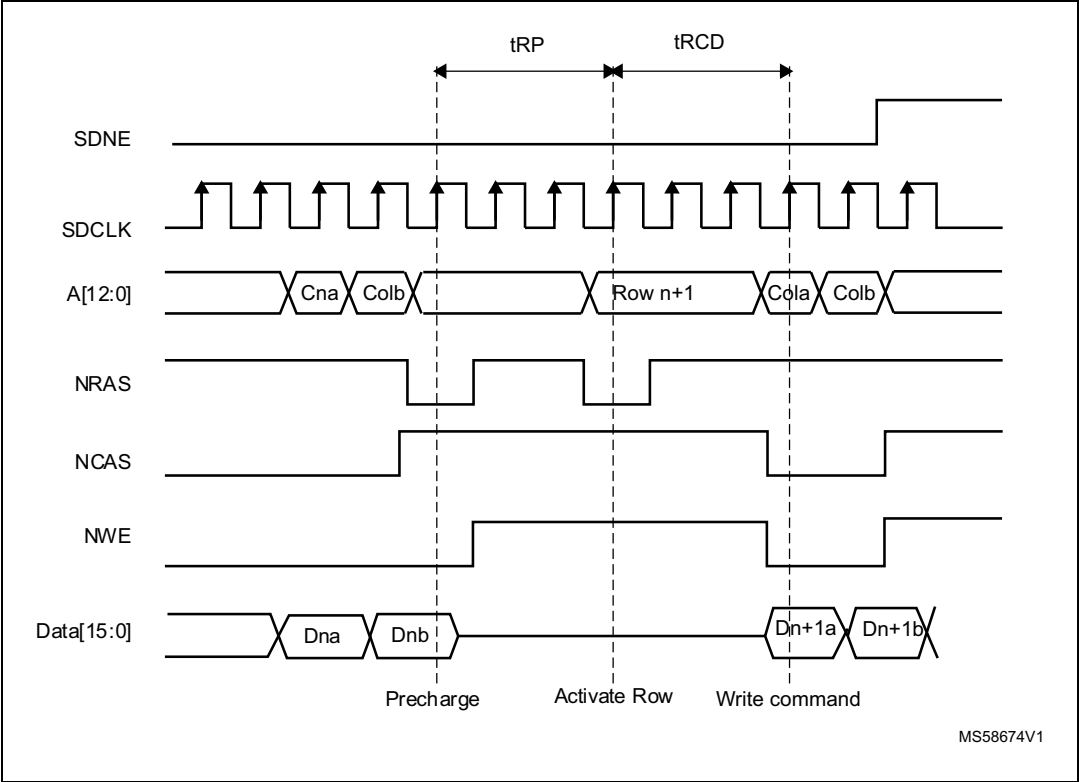


Figure 156. Write access crossing row boundary



If the next access is sequential and the current access crosses a bank boundary, the SDRAM controller activates the first row in the next bank and initiates a new read/write command. Two cases are possible:

- If the current bank is not the last one, the active row in the new bank must be precharged. At a bank boundary, the automatic activation of the next row is supported for all rows/columns and data bus width configuration.
- If the current bank is the last one and the selected SDRAM device is connected to Bank 1, the automatic activation of the next row in device connected to SDRAM Bank2 is not supported. A PALL software command must be issued on Bank1 before any access on Bank2.

SDRAM controller refresh cycle

The Auto-refresh command is used to refresh the SDRAM device content. The SDRAM controller periodically issues auto-refresh commands. An internal counter is loaded with the COUNT value in the register FMC_SDRTR. This value defines the number of memory clock cycles between the refresh cycles (refresh rate). When this counter reaches zero, an internal pulse is generated.

If a memory access is ongoing, the auto-refresh request is delayed. However, if the memory access and the auto-refresh requests are generated simultaneously, the auto-refresh request takes precedence.

If the memory access occurs during an auto-refresh operation, the request is buffered and processed when the auto-refresh is complete.

If a new auto-refresh request occurs while the previous one was not served, the RE (Refresh Error) bit is set in the Status register. An Interrupt is generated if it has been enabled (REIE = '1').

If SDRAM lines are not in idle state (not all row are closed), the SDRAM controller generates a PALL (Precharge ALL) command before the auto-refresh.

If the Auto-refresh command is generated by the FMC_SDCMR Command Mode register (Mode bits = '011'), a PALL command (Mode bits = '010') must be issued first.

23.8.4 Low-power modes

Two low-power modes are available:

- Self-refresh mode
The auto-refresh cycles are performed by the SDRAM device itself to retain data without external clocking.
- Power-down mode
The auto-refresh cycles are performed by the SDRAM controller.

Self-refresh mode

This mode is selected by setting the MODE bits to '101' and by configuring the Target Bank bits (CTB1 and/or CTB2) in the FMC_SDCMR register.

The SDRAM clock stops running after a TRAS delay and the internal refresh timer stops counting only if one of the following conditions is met:

- A Self-refresh command is issued to both devices
- One of the devices is not activated (SDRAM bank is not initialized).

Before entering Self-Refresh mode, the SDRAM controller automatically issues a PALL command.

If the Write data FIFO is not empty, all data are sent to the memory before activating the Self-refresh mode and the BUSY status flag remains set.

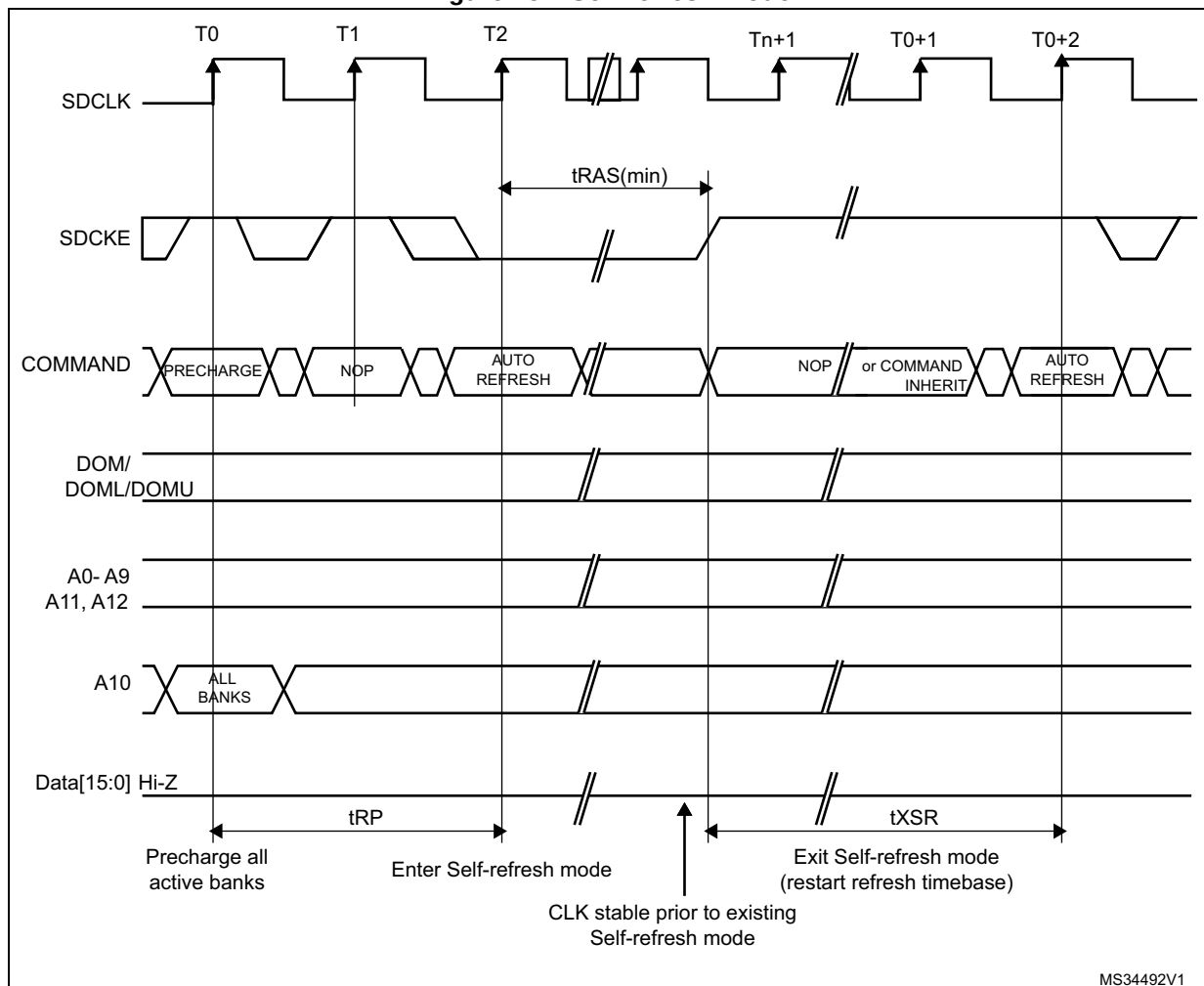
In Self-refresh mode, all SDRAM device inputs become don't care except for SDCKE which remains low.

The SDRAM device must remain in Self-refresh mode for a minimum period of time of t_{RAS} and can remain in Self-refresh mode for an indefinite period beyond that. To guarantee this minimum period, the BUSY status flag remains high after the Self-refresh activation during a t_{RAS} delay.

As soon as an SDRAM device is selected, the SDRAM controller generates a sequence of commands to exit from Self-refresh mode. After the memory access, the selected device remains in Normal mode.

To exit from Self-refresh, the MODE bits must be set to '000' (Normal mode) and the Target Bank bits (CTB1 and/or CTB2) must be configured in the FMC_SDCMR register.

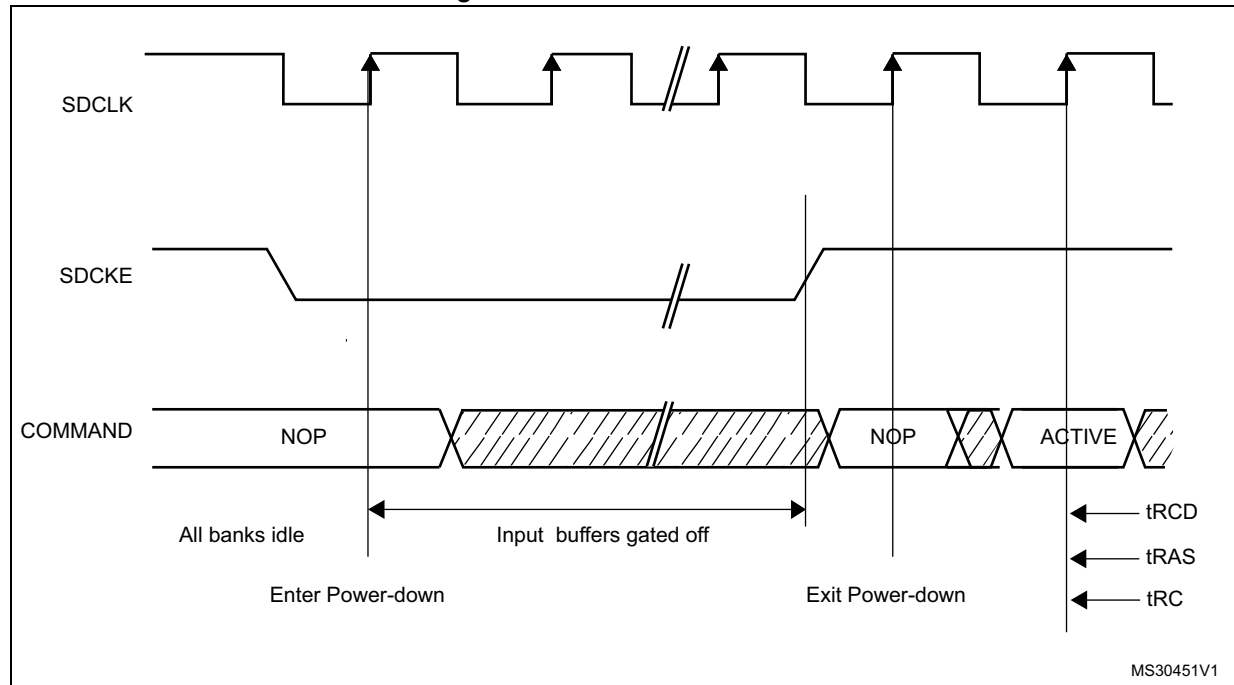
Figure 157. Self-refresh mode



Power-down mode

This mode is selected by setting the MODE bits to '110' and by configuring the Target Bank bits (CTB1 and/or CTB2) in the FMC_SDCMR register.

Figure 158. Power-down mode



If the Write data FIFO is not empty, all data are sent to the memory before activating the Power-down mode.

As soon as an SDRAM device is selected, the SDRAM controller exits from the Power-down mode. After the memory access, the selected SDRAM device remains in Normal mode.

During Power-down mode, all SDRAM device input and output buffers are deactivated except for the SDCKE which remains low.

The SDRAM device cannot remain in Power-down mode longer than the refresh period and cannot perform the Auto-refresh cycles by itself. Therefore, the SDRAM controller carries out the refresh operation by executing the operations below:

1. Exit from Power-down mode and drive the SDCKE high
2. Generate the PALL command only if a row was active during Power-down mode
3. Generate the auto-refresh command
4. Drive SDCKE low again to return to Power-down mode.

To exit from Power-down mode, the MODE bits must be set to '000' (Normal mode) and the Target Bank bits (CTB1 and/or CTB2) must be configured in the FMC_SDCMR register.

23.8.5 SDRAM controller registers

SDRAM control register x (FMC_SDCRx)

Address offset: $0x140 + 0x4 * (x - 1)$, ($x = 1, 2$)

Reset value: 0x0000 02D0

This register contains the control parameters for each SDRAM memory bank

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	RPIPE[1:0]		RBURST	SDCLK[1:0]		WP	CAS[1:0]		NB	MWID[1:0]		NR[1:0]		NC[1:0]	
	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bits 31:15 Reserved, must be kept at reset value.

Bits 14:13 **RPIPE[1:0]**: Read pipe

These bits define the delay, in clock cycles, for reading data after CAS latency.

00: No clock cycle delay

01: One clock cycle delay

10: Two clock cycle delay

11: reserved.

Note: The corresponding bits in the FMC_SDCR2 register is read only.

Bit 12 **RBURST**: Burst read

This bit enables Burst read mode. The SDRAM controller anticipates the next read commands during the CAS latency and stores data in the Read FIFO.

0: single read requests are not managed as bursts

1: single read requests are always managed as bursts

Note: The corresponding bit in the FMC_SDCR2 register is don't care.

Bits 11:10 **SDCLK[1:0]**: SDRAM clock configuration

These bits define the SDRAM clock period for both SDRAM banks and allow disabling the clock before changing the frequency. In this case the SDRAM must be re-initialized.

00: SDCLK clock disabled

01: SDCLK period = 1x HCLK periods

10: SDCLK period = 2 x HCLK periods

11: SDCLK period = 3 x HCLK periods

Note: The corresponding bits in the FMC_SDCR2 register are don't care.

Bit 9 **WP**: Write protection

This bit enables write mode access to the SDRAM bank.

0: Write accesses allowed

1: Write accesses ignored

Bits 8:7 **CAS[1:0]**: CAS Latency

This bits sets the SDRAM CAS latency in number of memory clock cycles

00: reserved.

01: 1 cycle

10: 2 cycles

11: 3 cycles

Bit 6 **NB**: Number of internal banks

This bit sets the number of internal banks.

0: Two internal Banks

1: Four internal Banks

Bits 5:4 **MWID[1:0]**: Memory data bus width.

These bits define the memory device width.

00: 8 bits

01: 16 bits

10: reserved

11: reserved.

Bits 3:2 **NR[1:0]**: Number of row address bits

These bits define the number of bits of a row address.

00: 11 bit

01: 12 bits

10: 13 bits

11: reserved.

Bits 1:0 **NC[1:0]**: Number of column address bits

These bits define the number of bits of a column address.

00: 8 bits

01: 9 bits

10: 10 bits

11: 11 bits.

Note: Before modifying the *RBURST* or *RPIPE* settings or disabling the *SDCLK* clock, the user must first send a *PALL* command to make sure ongoing operations are complete.

SDRAM timing register x (FMC_SDTRx)

Address offset: $0x148 + 0x4 * (x - 1)$, ($x = 1, 2$)

Reset value: 0x0FFF FFFF

This register contains the timing parameters of each SDRAM bank

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	TRCD[3:0]				TRP[3:0]				TWR[3:0]			
				r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
TRC[3:0]				TRAS[3:0]				TXSR[3:0]				TMRD[3:0]			
r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w

Bits 31:28 Reserved, must be kept at reset value.

Bits 27:24 **TRCD[3:0]**: Row to column delay

These bits define the delay, tRCD, from the Activate command to a Read/Write command in number of memory clock cycles.

0000: 2 cycles

0001: 3 cycles

....

1111: 17 cycles

Note: tRCD delay is replaced by $(TWR[3:0]+2)$ cycles when $TWR[3:0] > TRCD[3:0]$.

Bits 23:20 **TRP[3:0]**: Row precharge delay

These bits define the delay, tRP, from a Precharge command to another command in number of memory clock cycles. The TRP timing is only configured in the FMC_SDTR1 register. If two SDRAM devices are used, the TRP must be programmed with the timing of the slowest device.

0000: 2 cycle

0001: 3 cycles

....

1111: 17 cycles

Note: The corresponding bits in the FMC_SDTR2 register are don't care.

Bits 19:16 **TWR[3:0]**: Recovery delay

These bits define the delay, tWR, from a Write to a Precharge command in number of memory clock cycles.

0000: 2 cycle

0001: 3 cycles

....

1111: 17 cycles

Note: TWR must be programmed to match the write recovery time (t_{WR}) defined in the SDRAM datasheet, and to guarantee that:

$$TWR \geq TRAS - TRCD \text{ and } TWR \geq TRC - TRCD - TRP$$

Example: TRAS= 4 cycles, TRCD= 2 cycles. So, TWR >= 2 cycles. TWR must be programmed to 0x0.

If two SDRAM devices are used, the FMC_SDTR1 and FMC_SDTR2 must be programmed with the same TWR timing corresponding to the slowest SDRAM device.

If only one SDRAM device is used, the TWR timing must be kept at reset value (0xF) for the not used bank.

Bits 15:12 **TRC[3:0]**: Row cycle delay

These bits define the delay, tRC, from the Refresh command to the Activate command, as well as the delay between two consecutive Refresh commands. It is expressed in number of memory clock cycles. The TRC timing is only configured in the FMC_SDTR1 register. If two SDRAM devices are used, the TRC must be programmed with the timings of the slowest device.

0000: 2 cycle

0001: 3 cycles

....

1111: 17 cycles

Note: TRC must match the TRC and TRFC (Auto Refresh period) timings defined in the SDRAM device datasheet.

Note: The corresponding bits in the FMC_SDTR2 register are don't care.

Bits 11:8 **TRAS[3:0]**: Self refresh time

These bits define the minimum Self-refresh period in number of memory clock cycles.

0000: 2 cycle

0001: 3 cycles

....

1111: 17 cycles

Bits 7:4 **TXSR[3:0]**: Exit Self-refresh delay

These bits define the delay from releasing the Self-refresh command to issuing the Activate command in number of memory clock cycles.

0000: 2 cycle

0001: 3 cycles

....

1111: 17 cycles

Note: The tXSR delay is replaced by (TWR[3:0]+2) cycles when TWR[3:0] > TXSR[3:0].

Note: If two SDRAM devices are used, the FMC_SDTR1 and FMC_SDTR2 must be programmed with the same TXSR timing corresponding to the slowest SDRAM device.

Bits 3:0 **TMRD[3:0]**: Load Mode Register to Active

These bits define the delay, tMRD, from a Load Mode Register command to an Active or Refresh command in number of memory clock cycles.

0000: 3 cycle

0001: 4 cycles

....

1111: 18 cycles

Note: If two SDRAM devices are connected, all the accesses performed simultaneously to both devices by the Command Mode register (Load Mode Register command) are issued using the timing parameters configured for Bank 1 (TMRD and TRAS timings) in the FMC_SDTR1 register.

The TRP and TRC timings are only configured in the FMC_SDTR1 register. If two SDRAM devices are used, the TRP and TRC timings must be programmed with the timings of the slowest device.

SDRAM Command Mode register (FMC_SDCMR)

Address offset: 0x150

Reset value: 0x0000 0000

This register contains the command issued when the SDRAM device is accessed. This register is used to initialize the SDRAM device, and to activate the Self-refresh and the Power-down modes. As soon as the MODE field is written, the command is issued only to one or to both SDRAM banks according to CTB1 and CTB2 command bits. This register is the same for both SDRAM banks.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	MRD[12:7]					
										rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
MRD[6:0]						NRFS[3:0]				CTB1		CTB2	MODE[2:0]		
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	w	w	w	w	w

Bits 31:22 Reserved, must be kept at reset value.

Bits 21:9 **MRD[12:0]**: Mode Register definition

This 13-bit field defines the SDRAM Mode Register content. The Mode Register is programmed using the Load Mode Register command.

Bits 8:5 **NRFS[3:0]**: Number of Auto-refresh

These bits define the number of consecutive Auto-refresh commands issued when MODE = '011'.

0000: 1 Auto-refresh cycle

0001: 2 Auto-refresh cycles

....

1110: 15 Auto-refresh cycles

1111: 16 Auto-refresh cycles

Bit 4 **CTB1**: Command Target Bank 1

This bit indicates whether the command is issued to SDRAM Bank 1 or not.

0: Command not issued to SDRAM Bank 1

1: Command issued to SDRAM Bank 1

Bit 3 **CTB2**: Command Target Bank 2

This bit indicates whether the command is issued to SDRAM Bank 2 or not.

0: Command not issued to SDRAM Bank 2

1: Command issued to SDRAM Bank 2

Bits 2:0 **MODE[2:0]**: Command mode

These bits define the command issued to the SDRAM device.

000: Normal Mode

001: Clock Configuration Enable

010: PALL ("All Bank Precharge") command

011: Auto-refresh command

100: Load Mode Register

101: Self-refresh command

110: Power-down command

111: Reserved

Note: When a command is issued, at least one Command Target Bank bit (CTB1 or CTB2) must be set otherwise the command is ignored.

Note: If two SDRAM banks are used, the Auto-refresh and PALL command must be issued simultaneously to the two devices with CTB1 and CTB2 bits set otherwise the command is ignored.

Note: If only one SDRAM bank is used and a command is issued with it's associated CTB bit set, the other CTB bit of the the unused bank must be kept to 0.

SDRAM refresh timer register (FMC_SDRTR)

Address offset: 0x154

Reset value: 0x0000 0000

This register sets the refresh rate in number of SDCLK clock cycles between the refresh cycles by configuring the Refresh Timer Count value.

$$\text{Refresh rate} = (\text{COUNT} + 1) \times \text{SDRAM clock frequency}$$

$$\text{COUNT} = (\text{SDRAM refresh period} / \text{Number of rows}) - 20$$

Example

$$\text{Refresh rate} = 64 \text{ ms} / (8196 \text{ rows}) = 7.81 \mu\text{s}$$

where 64 ms is the SDRAM refresh period.

$$7.81\mu\text{s} \times 60\text{MHz} = 468.6$$

The refresh rate must be increased by 20 SDRAM clock cycles (as in the above example) to obtain a safe margin if an internal refresh request occurs when a read request has been accepted. It corresponds to a COUNT value of '0000111000000' (448).

This 13-bit field is loaded into a timer which is decremented using the SDRAM clock. This timer generates a refresh pulse when zero is reached. The COUNT value must be set at least to 41 SDRAM clock cycles.

As soon as the FMC_SDRTR register is programmed, the timer starts counting. If the value programmed in the register is '0', no refresh is carried out. This register must not be reprogrammed after the initialization procedure to avoid modifying the refresh rate.

Each time a refresh pulse is generated, this 13-bit COUNT field is reloaded into the counter.

If a memory access is in progress, the Auto-refresh request is delayed. However, if the memory access and Auto-refresh requests are generated simultaneously, the Auto-refresh takes precedence. If the memory access occurs during a refresh operation, the request is buffered to be processed when the refresh is complete.

This register is common to SDRAM bank 1 and bank 2.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	REIE	COUNT[12:0]												CRE	
	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	w

Bits 31:15 Reserved, must be kept at reset value.

Bit 14 **REIE**: RES Interrupt Enable

0: Interrupt is disabled

1: An Interrupt is generated if RE = 1

Bits 13:1 **COUNT[12:0]**: Refresh Timer Count

This 13-bit field defines the refresh rate of the SDRAM device. It is expressed in number of memory clock cycles. It must be set at least to 41 SDRAM clock cycles (0x29).

Refresh rate = (COUNT + 1) x SDRAM frequency clock

COUNT = (SDRAM refresh period / Number of rows) - 20

Bit 0 **CRE**: Clear Refresh error flag

This bit is used to clear the Refresh Error Flag (RE) in the Status Register.

0: no effect

1: Refresh Error flag is cleared

Note: The programmed COUNT value must not be equal to the sum of the following timings: TWR+TRP+TRC+TRCD+4 memory clock cycles.

SDRAM status register (FMC_SDSR)

Address offset: 0x158

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	BUSY	MODES2[1:0]	MODES1[1:0]	RE		
										r	r	r	r	r	r

Bits 31:6 Reserved, must be kept at reset value.

Bit 5 **BUSY**: Busy status

This bit defines the status of the SDRAM controller after a Command Mode request

0: SDRAM Controller is ready to accept a new request

1: SDRAM Controller is not ready to accept a new request

Bits 4:3 **MODES2[1:0]**: Status Mode for Bank 2

This bit defines the Status Mode of SDRAM Bank 2.

00: Normal Mode

01: Self-refresh mode

10: Power-down mode

Bits 2:1 **MODES1[1:0]**: Status Mode for Bank 1

This bit defines the Status Mode of SDRAM Bank 1.

00: Normal Mode

01: Self-refresh mode

10: Power-down mode

Bit 0 **RE**: Refresh error flag

0: No refresh error has been detected

1: A refresh error has been detected

An interrupt is generated if REIE = 1 and RE = 1

23.8.6 FMC register map**Table 231. FMC register map and reset values**

Offset	Register name	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0x00	FMC_BCR1	FMCEN	Res.	Res.	Res.	Res.	Res.	Res.	Res.	NBL SET [1:0]		WFDIS	CCLKEN	CBURSTW	CPSIZE [2:0]			ASYNCAWAIT	EXTMOD	WAITEN	WREN	WAITCFG	Res.	WAITPOL	BURSTEN	Res.	FACCEN	MWID [1:0]	MTYP [1:0]	MUXEN	MBKEN		
	Reset value	0								0	0	0	0	0	0	0	0	0	0	0	1	1	0		0	0		1	0	1	1	0	1
0x08	FMC_BCR2	FMCEN	Res.	Res.	Res.	Res.	Res.	Res.	Res.	NBL SET [1:0]		Res.	Res.	CBURSTW	CPSIZE [2:0]			ASYNCAWAIT	EXTMOD	WAITEN	WREN	WAITCFG	Res.	WAITPOL	BURSTEN	Res.	FACCEN	MWID [1:0]	MTYP [1:0]	MUXEN	MBKEN		
	Reset value	0								0	0			0	0	0	0	0	0	0	1	1	0		0	0		1	0	1	0	0	1

Table 231. FMC register map and reset values (continued)

Offset	Register name	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0x10	FMC_BCR3	FMEN	Res.	Res.	Res.	Res.	Res.	Res.	Res.	NBL SET [1:0]	Res.	Res.	Res.	CBURSTRW	CPSIZE [2:0]			ASYNCAWAIT	EXTMOD	WAITEN	WREN	WAITCFG	Res.	WAITPOL	BURSTEN	Res.	FACEN	MWID [1:0]		MTYP [1:0]		MUXEN	MBKEN
	Reset value	0								0	0			0	0	0	0	0	0	1	1	0	0	0	0		1	0	1	0	0	1	0
0x18	FMC_BCR4	FMEN	Res.	Res.	Res.	Res.	Res.	Res.	Res.	NBL SET [1:0]	Res.	Res.	Res.	CBURSTRW	CPSIZE [2:0]			ASYNCAWAIT	EXTMOD	WAITEN	WREN	WAITCFG	Res.	WAITPOL	BURSTEN	Res.	FACEN	MWID [1:0]		MTYP [1:0]		MUXEN	MBKEN
	Reset value	0								0	0			0	0	0	0	0	0	1	1	0	0	0	0		1	0	1	0	0	1	0
0x04	FMC_BTR1	DATAHLD[1:0]	ACCMOD[1:0]		DATLAT[3:0]			CLKDIV[3:0]			BUSTURN [3:0]			DATAST[7:0]							ADDHLD[3:0]			ADDSET[3:0]									
	Reset value	0	0	0	0	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1
0x0C	FMC_BTR2	DATAHLD[1:0]	ACCMOD[1:0]		DATLAT[3:0]			CLKDIV[3:0]			BUSTURN [3:0]			DATAST[7:0]							ADDHLD[3:0]			ADDSET[3:0]									
	Reset value	0	0	0	0	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1
0x14	FMC_BTR3	DATAHLD[1:0]	ACCMOD[1:0]		DATLAT[3:0]			CLKDIV[3:0]			BUSTURN [3:0]			DATAST[7:0]							ADDHLD[3:0]			ADDSET[3:0]									
	Reset value	0	0	0	0	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1
0x1C	FMC_BTR4	DATAHLD[1:0]	ACCMOD[1:0]		DATLAT[3:0]			CLKDIV[3:0]			BUSTURN [3:0]			DATAST[7:0]							ADDHLD[3:0]			ADDSET[3:0]									
	Reset value	0	0	0	0	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1
0x20	FMC_PCSCNTR	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	CNTB4EN	CNTB3EN	CNTB2EN	CNTB1EN	CSCCOUNT[15:0]															
	Reset value													0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0x104	FMC_BWTR1	DATAHLD[1:0]	ACCMOD[1:0]		Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	BUSTURN [3:0]		DATAST[7:0]							ADDHLD[3:0]			ADDSET[3:0]							
	Reset value	0	0	0	0									1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1
0x10C	FMC_BWTR2	DATAHLD[1:0]	ACCMOD[1:0]		Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	BUSTURN [3:0]		DATAST[7:0]							ADDHLD[3:0]			ADDSET[3:0]							
	Reset value	0	0	0	0									1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1
0x114	FMC_BWTR3	DATAHLD[1:0]	ACCMOD[1:0]		Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	BUSTURN [3:0]		DATAST[7:0]							ADDHLD[3:0]			ADDSET[3:0]							
	Reset value	0	0	0	0									1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1

Table 231. FMC register map and reset values (continued)

Offset	Register name	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0x11C	FMC_BWTR4	DATAHLD[1:0]			ACCMOD[1:0]		Res.	Res.	Res.	Res.	Res.	Res.	Res.	BUSTURN [3:0]			DATAST[7:0]							ADDHLD[3:0]			ADDSET[3:0]						
	Reset value	0	0	0	0									1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	
0x80	FMC_PCR	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	ECCPS [2:0]		TAR[3:0]			TCLR[3:0]				Res.	Res.	ECCEN	PWID [1:0]		PTYP	PBKEN	PWAITEN	Res.		
	Reset value													0	0	0	0	0	0	0	0	0	0				0	0	1	1	0	0	
0x84	FMC_SR	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	FEMPT	IFEN	ILEN	IREN	IFS	ILS	IRS
	Reset value																										1	0	0	0	0	0	0
0x88	FMC_PMEM	MEMHIZx[7:0]								MEMHOLDx[7:0]								MEMWAITx[7:0]								MEMSETx[7:0]							
	Reset value	1	1	1	1	1	1	0	0	1	1	1	1	1	1	0	0	1	1	1	1	1	1	0	0	1	1	1	1	1	1	0	0
0x8C	FMC_PATT	ATTHIZ[7:0]								ATTHOLD[7:0]								ATTWAIT[7:0]								ATTSET[7:0]							
	Reset value	1	1	1	1	1	1	0	0	1	1	1	1	1	1	0	0	1	1	1	1	1	1	0	0	1	1	1	1	1	1	0	0
0x94	FMC_ECCR	ECCx[31:0]																															
	Reset value	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	
0x140	FMC_SDCR1	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	RPIPE [1:0]	RBURST	SDCLK [1:0]	WP	CAS [1:0]	NB	MWID [1:0]	NR [1:0]	NC						
	Reset value																		0	0	1	1	0	1	0	0	1	0	0	0	0	0	
0x144	FMC_SDCR2	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	RBURST	SDCLK [1:0]	WP	CAS [1:0]	NB	MWID [1:0]	NR [1:0]	NC						
	Reset value																			0	1	1	0	1	0	0	1	0	0	0	0	0	
0x148	FMC_SDTR1	Res.	Res.	Res.	Res.	TRCD[3:0]			TRP[3:0]			TWR[3:0]			TRC[3:0]			TRAS[3:0]			TXSR[3:0]			TMRD[3:0]									
	Reset value					1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	
0x14C	FMC_SDTR2	Res.	Res.	Res.	Res.	TRCD[3:0]			TRP[3:0]			TWR[3:0]			TRC[3:0]			TRAS[3:0]			TXSR[3:0]			TMRD[3:0]									
	Reset value					1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	
0x150	FMC_SDCMR	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	MRD[12:0]										NRFS[3:0]			CTB1	CTB2	MODE[2:0]						
	Reset value											0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	
0x154	FMC_SDRTR	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	REIE	COUNT[12:0]											CRE		
	Reset value																		0	0	0	0	0	0	0	0	0	0	0	0	0	0	
0x158	FMC_SDSR	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	BUSY	MODES2[1:0]		MODES1[1:0]		Res.
	Reset value																										0	0	0	0	0	0	

Refer to [Section 2.3: Memory organization](#) for the register boundary addresses.

24 Octo-SPI interface (OCTOSPI)

This section applies only to STM32H523/33xx, STM32H562/63xx, STM32H573xx devices.

24.1 OCTOSPI introduction

The OCTOSPI supports most external serial memories such as serial PSRAMs, serial NAND and serial NOR flash memories, HyperRAM™ and HyperFlash™ memories, with the following functional modes:

- indirect mode: all the operations are performed using the OCTOSPI registers to preset commands, addresses, data, and transfer parameters.
- automatic status-polling mode: the external memory status register is periodically read and an interrupt can be generated in case of flag setting. This feature is only available in regular-command protocol.
- memory-mapped mode: the external memory is memory mapped and it is seen by the system as if it was an internal memory, supporting both read and write operations.

The OCTOSPI supports the following protocols with associated frame formats:

- the regular-command frame format with the command, address, alternate byte, dummy cycles, and data phase
- the HyperBus™ frame format

24.2 OCTOSPI main features

- Functional modes: indirect, automatic status-polling, and memory-mapped
- Read and write support in memory-mapped mode
- External (P)SRAM memory support
- Support for single, dual, quad, and octal communication
- Dual memory configuration, where eight bits can be sent/received simultaneously by accessing two quad memories in parallel
- SDR (single-data rate) and DTR (double-transfer rate) support
- Data strobe support
- Fully programmable opcode
- Fully programmable frame format
- Support wrapped-type access to memory in read direction
- HyperBus support
- Integrated FIFO for reception and transmission
- Asynchronous bus clock versus kernel clock support
- 8-, 16-, and 32-bit data accesses allowed
- DMA protocol support
- DMA channel for indirect mode operations
- Interrupt generation on FIFO threshold, timeout, operation complete, and access error
- AHB interface with transaction acceptance limited to one: the interface accepts the next transfer on AHB bus only once the previous is completed on memory side.

24.3 OCTOSPI implementation

Table 232. OCTOSPI implementation

OCTOSPI feature	OCTOSPI1
HyperBus standard compliant	X
Xccela standard compliant	X
XSPI (JESD251C) standard compliant	X
AMBA® AHB compliant data interface	X
Dual AHB interface	X
Asynchronous AHB clock versus kernel clock	X
Functional modes: indirect, automatic status-polling, and memory-mapped	X
Read and write support in memory-mapped mode	X
Dual-quad configuration	X
SDR (single-data rate) and DTR (double-transfer rate)	X
Data strobe (DS,DQS)	X
Fully programmable opcode	X
Fully programmable frame format	X
Integrated FIFO for reception and transmission	X
8-, 16-, and 32-bit data accesses	X
Interrupt on FIFO threshold, timeout, operation complete, and access error	X
Bus error bit	-
Address offset	-
Extended CSHT timeout	X
Memory-mapped write	X
Refresh counter	X
GPDMA interface	X
Dual chip select	-
Extended external memory	-
Prefetch disable	-
Prefetch hardware software	-

24.4 OCTOSPI functional description

24.4.1 OCTOSPI block diagram

Figure 159. OCTOSPI block diagram in octal configuration

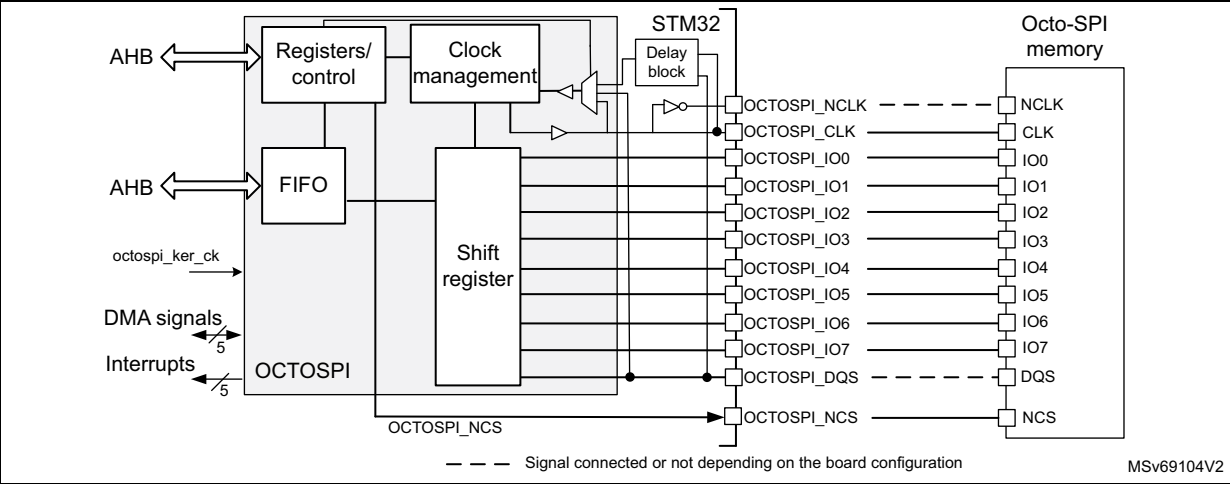


Figure 160. OCTOSPI block diagram in quad configuration

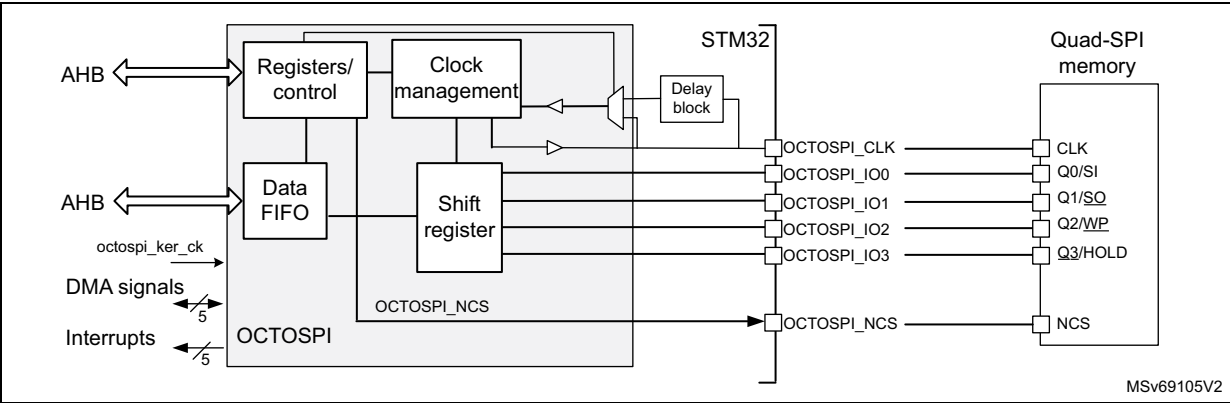
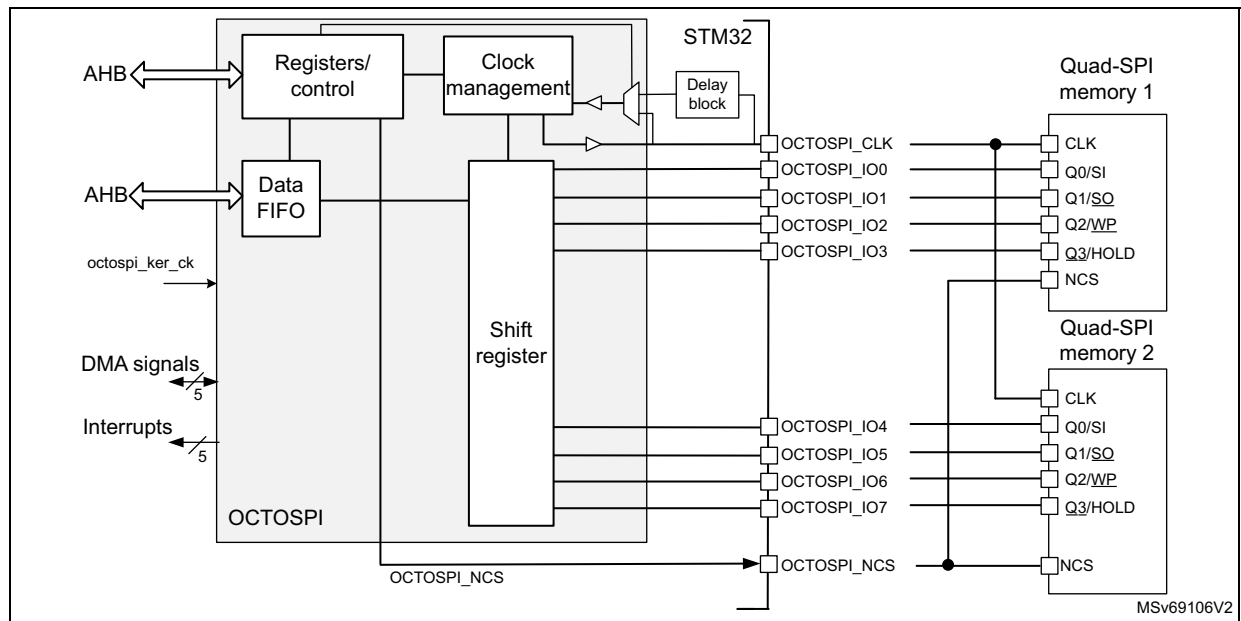


Figure 161. OCTOSPI block diagram in dual-quad configuration



24.4.2 OCTOSPI pins and internal signals

Table 233. OCTOSPI input/output pins

Pin name	Type	Description
OCTOSPI_NCLK	Output	OCTOSPI inverted clock to support 1.8 V HyperBus protocol
OCTOSPI_CLK		OCTOSPI clock
OCTOSPI_IOn (n = 0 to 7)	Input/output	OCTOSPI data pins
OCTOSPI_NCS	Output	Chip select for the memory
OCTOSPI_DQS	Input/output	Data strobe/write mask signal from/to the memory

Caution: Use the same configuration (output speed, HSLV^(a)) for all OCTOSPI input/output pins to avoid any data corruption.

Table 234. OCTOSPI internal signals

Signal name	Type	Description
octospi_hclk	Input	OCTOSPI AHB clock
octospi_ker_ck	Input	OCTOSPI kernel clock
octospi_dma	N/A	DMA request signal
octospi_it	Output	Global interrupt line (see Table 238 for the multiple sources of interrupt)

a. When applicable.

24.4.3 OCTOSPI interface to memory modes

The OCTOSPI supports the following protocols:

- regular-command protocol
- HyperBus protocol

The OCTOSPI uses from 6 to 12 signals to interface with a memory, depending on the functional mode:

- NCS: chip-select
- CLK: communication clock
- NCLK: inverted clock used only in the 1.8 V HyperBus protocol
- DQS: data strobe used only in regular-command protocol as input only
- IO[3:0]: data bus LSB
- IO[7:4]:
 - data bus MSB used in dual-quad and octal configurations
 - data bus can be used as possible remap for quad-SPI mode

24.4.4 OCTOSPI regular-command protocol

When in regular-command protocol, the OCTOSPI communicates with the external device using commands. Each command can include the following phases:

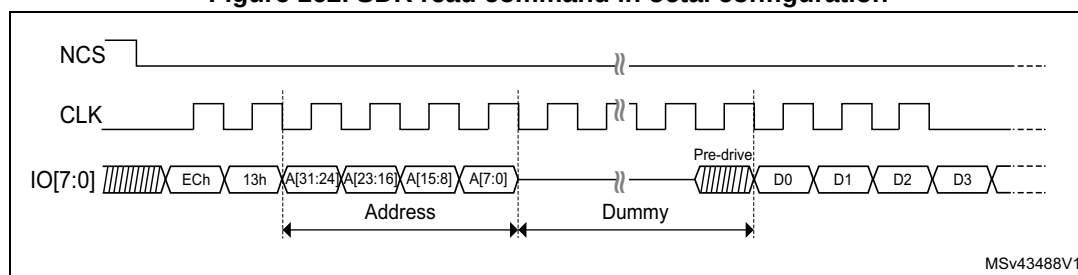
- Instruction phase
- Address phase
- Alternate-byte phase
- Dummy-cycle phase
- Data phase

Any of these phases can be configured to be skipped but, in case of single-phase command, the only use case supported is instruction-phase-only.

The NCS falls before the start of each command and rises again after each command finishes.

In memory-mapped mode, both read and write operations are supported: as a consequence, some of the configuration registers are duplicated to specify write operations (read operations are configured using regular registers).

Figure 162. SDR read command in octal configuration



The specific regular-command protocol features are configured through the registers in the 0x0100-0x01FC offset range.

Instruction phase

During this phase, a 1- to 4-byte instruction is sent to the external device specifying the type of operation to be performed. The size of the instruction to be sent is configured by ISIZE[1:0] in OCTOSPI_CCR and the instruction is programmed in INSTRUCTION[31:0] of OCTOSPI_IR.

The instruction phase can optionally send:

- 1 bit at a time (over IO0, SO single in single-SPI mode)
- 2 bits at a time (over IO0/IO1 in dual-SPI mode)
- 4 bits at a time (over IO0 to IO3 in quad-SPI mode)
- 8 bits at a time (over IO0 to IO7 in octal-SPI mode)

This can be configured using IMODE[2:0] of OCTOSPI_CCR.

The instruction can be sent in DTR mode on each rising and falling edge of the clock, by setting IDTR in OCTOSPI_CCR.

When IMODE[2:0] = 000 in OCTOSPI_CCR, the instruction phase is skipped, and the command sequence starts with the address phase, if present.

In memory-mapped mode, the instruction used for the write operation is specified in OCTOSPI_WIR, and the instruction format is specified in OCTOSPI_WCCR. The instruction used for the read operation and the instruction format are specified in OCTOSPI_IR and OCTOSPI_CCR.

Address phase

In the address phase, 1 to 4 bytes are sent to the external device, to indicate the address of the operation. The number of address bytes to be sent is configured by ADSIZE[1:0] in OCTOSPI_CCR.

In indirect and automatic status-polling modes, the address bytes to be sent are specified by ADDRESS[31:0] in OCTOSPI_AR. In memory-mapped mode, the address is given directly via the AHB (from any master in the system).

The address phase can send:

- 1 bit at a time (over IO0, SO single in single-SPI mode)
- 2 bits at a time (over IO0/IO1 in dual-SPI mode)
- 4 bits at a time (over IO0 to IO3 in quad-SPI mode)
- 8 bits at a time (over IO0 to IO7 in octal-SPI mode)

This can be configured using ADMODE[2:0] in OCTOSPI_CCR.

The address can be sent in DTR mode (on each rising and falling edge of the clock) setting ADDTR in OCTOSPI_CCR.

When ADMODE[2:0] = 000, the address phase is skipped and the command sequence proceeds directly to the next phase, if any.

In memory-mapped mode, the address format for the write operation is specified in OCTOSPI_WCCR. The address format for the read operation is specified in OCTOSPI_CCR.

Alternate-byte phase

In the alternate-byte phase, 1 to 4 bytes are sent to the external device, generally to control the mode of operation. The number of alternate bytes to be sent is configured by ABSIZE[1:0] in OCTOSPI_CCR. The bytes to be sent are specified in OCTOSPI_ABR.

The alternate-byte phase can send:

- 1 bit at a time (over IO0, SO single in single-SPI mode)
- 2 bits at a time (over IO0/IO1 in dual-SPI mode)
- 4 bits at a time (over IO0 to IO3 in quad-SPI mode)
- 8 bits at a time (over IO0 to IO7 in octal-SPI mode)

This can be configured using ABMODE[2:0] in OCTOSPI_CCR.

The alternate bytes can be sent in DTR mode (on each rising and falling edge of the clock) setting ABDTR in OCTOSPI_CCR.

When ABMODE[2:0] = 000, the alternate-byte phase is skipped and the command sequence proceeds directly to the next phase, if any.

There may be times when only a single nibble needs to be sent during the alternate-byte phase rather than a full byte, such as when the dual-SPI mode is used and only two cycles are used for the alternate bytes.

In this case, the firmware can use the quad-SPI mode (ABMODE[2:0] = 011) and send a byte with bits 7 and 3 of ALTERNATE[31:0] set to 1 (keeping the IO3 line high), and bits 6 and 2 set to 0 (keeping the IO2 line low), in OCTOSPI_IR.

The upper two bits of the nibble to be sent are then placed in bits 5:4 of ALTERNATE[31:0] while the lower two bits are placed in bits 1:0. For example, if the nibble 2 (0010) is to be sent over IO0/IO1, then ALTERNATE[31:0] must be set to 0x8A (1000_1010).

In memory-mapped mode, the alternate bytes used for the write operation are specified in OCTOSPI_WABR, and the alternate byte format is specified in OCTOSPI_WCCR. The alternate bytes used for read operation and the alternate byte format are specified in OCTOSPI_ABR and OCTOSPI_CCR.

Dummy-cycle phase (memory latency)

In the dummy-cycle phase, 1 to 31 cycles are given without any data being sent or received, in order to give the external device, the time to prepare for the data phase when higher clock frequencies are used. The number of cycles given during this phase is specified by DCYC[4:0] in OCTOSPI_TCR. In both SDR and DTR modes, the duration is specified as a number of full CLK cycles.

When DCYC[4:0] = 00000, the dummy-cycle phase is skipped, and the command sequence proceeds directly to the data phase, if present.

In order to assure enough “turn-around” time for changing the data signals from the output mode to the input mode, there must be at least one dummy cycle when using the dual-SPI, the quad-SPI, or the octal-SPI mode, to receive data from the external device.

In memory-mapped mode, the dummy cycles for the write operations are specified in OCTOSPI_WTCR. The dummy cycles for the read operation are specified in OCTOSPI_TCR.

Data phase

During the data phase, any number of bytes can be sent to or received from the external device.

In indirect mode, the number of bytes to be sent/received is specified in OCTOSPI_DLR. In this mode, the data to be sent to the external device must be written to OCTOSPI_DR, while in indirect-read mode the data received from the external device is obtained by reading OCTOSPI_DR.

In automatic status-polling mode, the number of bytes to be received is specified in OCTOSPI_DLR, and the data received from the external device can be obtained by reading OCTOSPI_DR.

In memory-mapped mode, the data read or written, is sent or received directly over the AHB to the Cortex core or to a DMA.

The data phase can send/receive:

- 1 bit at a time (over IO0/IO1 (SO/SI respectively) in single-SPI mode)
- 2 bits at a time (over IO0/IO1 in dual-SPI mode)
- 4 bits at a time (over IO0 to IO3 in quad-SPI mode)
- 8 bits at a time (over IO0 to IO7 in octal-SPI mode)

This can be configured using DMODE[2:0] in OCTOSPI_CCR.

The data can be sent or received in DTR mode (on each rising and falling edge of the clock) setting DDTR in OCTOSPI_CCR.

When DMODE[2:0] = 000, the data phase is skipped, and the command sequence finishes immediately by raising the NCS. This configuration must be used only in indirect-write mode.

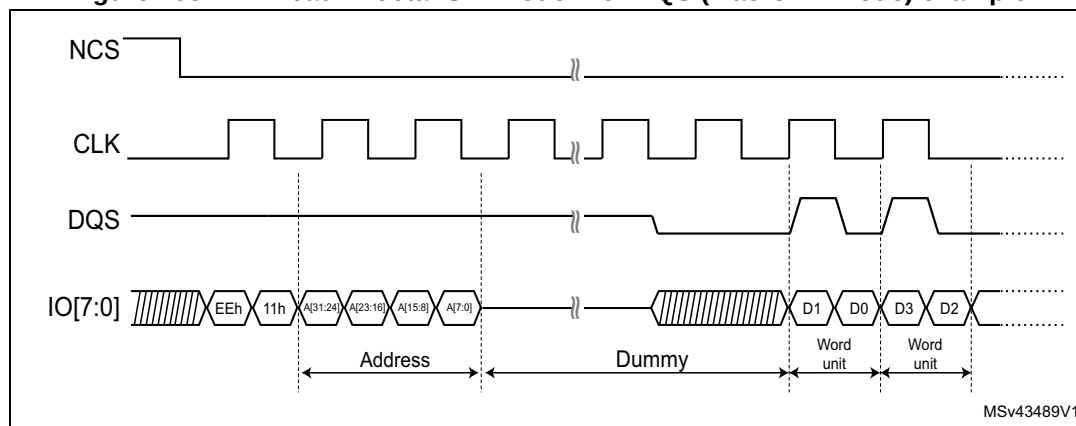
In memory-mapped mode, the data format for the write operation is specified in OCTOSPI_WCCR. The data format for the read operation is specified in OCTOSPI_CCR.

DQS use

The DQS signal can be used for data strobing during the read transactions when the device toggles the DQS aligned with the data.

The DQS management can be enabled by setting DQSE in OCTOSPI_CCR.

Figure 163. DTR read in octal-SPI mode with DQS (Macronix mode) example



24.4.5 OCTOSPI regular-command protocol signal interface

Single-SPI mode

The legacy SPI mode allows just a single bit to be sent/received serially. In this mode, the data is sent to the external device over the SO signal (Single-SPI Output) (whose I/Os are shared with IO0). The data received from the external device arrives via SI (Single-SPI Input) (whose I/Os are shared with IO1).

Compared to the SPI legacy mode, IO/SO and I1/SI are respectively equivalent to MOSI and MISO, having the OCTOSPI generating the clock.

The different phases can each be configured separately to use this single-SPI mode by setting to 001 the IMODE, ADMODE, ABMODE, and DMODE fields in OCTOSPI_CCR and OCTOSPI_WCCR.

In each phase configured in single-SPI mode:

- IO0 (SO) is in output mode.
- IO1 (SI) is in input mode (high impedance).
- IO2 is in output mode and forced to 0.
- IO3 is in output mode and forced to 1 (to deactivate the “hold” function).
- IO4 to IO7 are in output mode and forced to 0.

This is the case even for the dummy phase if DMODE[2:0] = 001.

Dual-SPI mode

In dual-SPI mode, two bits are sent/received simultaneously over the IO0/IO1 signals.

The different phases can each be configured separately to use dual-SPI mode by setting to 010 the IMODE, ADMODE, ABMODE, and DMODE fields in OCTOSPI_CCR and OCTOSPI_WCCR.

In each phase configured in dual-SPI mode:

- IO0/IO1 are at high-impedance (input) during the data phase for the read operations, and outputs in all other cases.
- IO2 is in output mode and forced to 0.
- IO3 is in output mode and forced to 1.
- IO4 to IO7 are in output mode and forced to 0.

In the dummy phase, when DMODE[2:0] = 010, IO0 and IO1 are in a high-impedance state during read transactions, and are forced to either high or low levels during write transactions.

Quad-SPI mode

In quad-SPI mode, four bits are sent/received simultaneously over the IO0/IO1/IO2/IO3 signals.

The different phases can each be configured separately to use the quad-SPI mode by setting to 011 the IMODE, ADMODE, ABMODE, and DMODE fields in OCTOSPI_CCR and OCTOSPI_WCCR.

In each phase configured in quad-SPI mode:

- IO0 to IO3 are all at high-impedance (inputs) during the data phase for the read operations, and outputs in all other cases.
- IO4 to IO7 are in output mode and forced to 0.

In the dummy phase, when $\text{DMODE}[2:0] = 011$, IO0 to IO3 are in a high-impedance state during read transactions, and are forced to either high or low levels during write transactions.

Octal-SPI mode

In regular octal-SPI mode, the eight bits are sent/received simultaneously over the IO[0:7] signals.

The different phases can each be configured separately to use the octal-SPI mode by setting to 100 the IMODE, ADMODE, ABMODE, and DMODE fields in OCTOSPI_CCR and OCTOSPI_WCCR.

In each phase that is configured in octal-SPI mode, IO[0:7] are all at high-impedance (input) during the data phase for read operations, and outputs in all other cases.

In the dummy phase, when $\text{DMODE}[2:0] = 100$, IO[0:7] are in a high-impedance state during read transactions, and are forced to either high or low levels during write transactions.

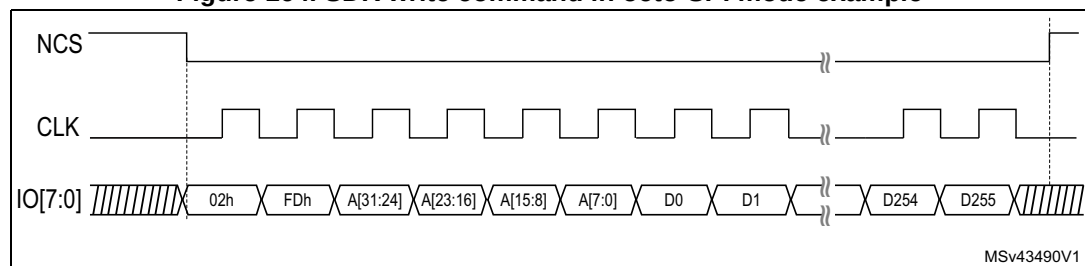
Single-data rate (SDR) mode

By default, all the phases operate in SDR mode.

In this mode, when the OCTOSPI drives the IO0/SO, IO1 to IO7 signals, these signals transition only with the falling edge of CLK.

When receiving data in SDR mode, the OCTOSPI assumes that the external devices also send the data using CLK falling edge. By default (when $\text{SSHIFT} = 0$ in OCTOSPI_TCR), the signals are sampled using the following (rising) edge of CLK.

Figure 164. SDR write command in octo-SPI mode example



Note: Due to internal synchronization, up to six extra dummy clock cycles may be generated by the Octo-SPI interface after the last data is read.

Double-transfer rate (DTR) mode

Each of the instruction, address, alternate-byte, and data phases can be configured to operate in DTR mode setting IDTR, ADDTR, ABDTR, and DDTR in OCTOSPI_CCR.

In memory-mapped mode, the DTR mode for each phase of the write operations is specified in OCTOSPI_WCCR. The DTR mode for each phase of the read operations is specified in OCTOSPI_CCR.

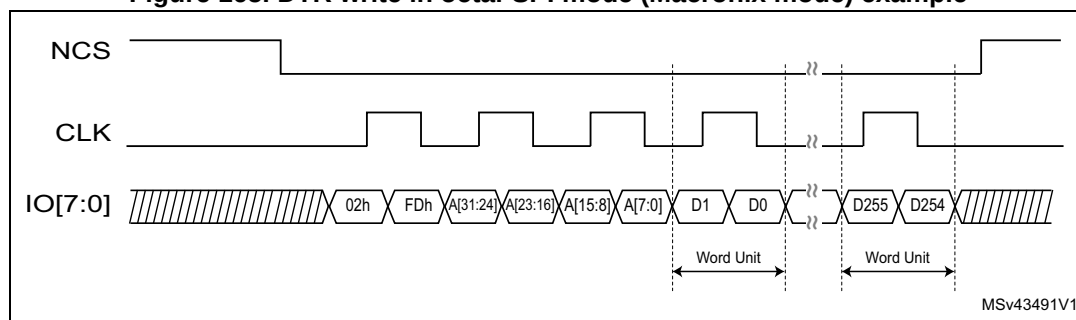
In DTR mode, when the OCTOSPI drives the IO0/SO and IO1 to IO7 signals in the instruction, address, and alternate-byte phases, a bit is sent or received on each of the falling and rising edges of CLK.

When receiving data in DTR mode, the OCTOSPI assumes that the external devices also send the data using both CLK rising and falling edges. When DDTR = 1 in OCTOSPI_CCR, the software must clear SSHIFT in OCTOSPI_TCR. Thus, the signals are sampled one half of a CLK cycle later (on the following, opposite edge).

In DTR mode, it is recommended to set DHQC of OCTOSPI_TCR, to shift the outputs by a quarter of cycle and avoid holding issues on the memory side.

Note: *DHQC must not be set when the prescaler value is 0, as this action leads to unpredictable behavior.*

Figure 165. DTR write in octal-SPI mode (Macronix mode) example



Note: *Due to internal synchronization, up to six extra dummy clock cycles may be generated by the Octo-SPI interface after the last data is read.*

Dual-quad configuration

When DMM = 1 in OCTOSPI_CR, the OCTOSPI is in dual-memory configuration: if DMODE = 011, two external Quad-SPI devices (device A and device B) are used in order to send/receive eight bits (or 16 bits in DTR mode) every cycle, effectively doubling the throughput.

Each device (A or B) uses the same CLK and NCS signals, but each has separate IO0 to IO3 signals.

The dual-quad configuration can be used in conjunction with the single-SPI, dual-SPI, and quad-SPI modes, as well as with either the SDR or DTR mode.

The device size, as specified by DEVSZ[4:0] in OCTOSPI_DCR1, must reflect the total external device capacity that is the double of the size of one individual component.

If address X is even, then the byte that the OCTOSPI gives for address X is the byte at the address X/2 of device A, and the byte that the OCTOSPI gives for address X + 1 is the byte at the address X/2 of device B. In other words, the bytes at even addresses are all stored in device A and the bytes at odd addresses are all stored in device B.

When reading the status registers of the devices in dual-quad configuration, twice as many bytes must be read compared to the same read in regular-command protocol: if each device gives eight valid bits after the instruction for fetching the status register, then the OCTOSPI must be configured with a data length of 2 bytes (16 bits), and the OCTOSPI receives one byte from each device.

If each device gives a status of 16 bits, then the OCTOSPI must be configured to read 4 bytes to get all the status bits of both devices in dual-quad configuration. The least-significant byte of the result (in the data register) is the least-significant byte of device A status register. The next byte is the least-significant byte of device B status register. Then, the third byte of the data register is the device A second byte. The fourth byte is the device B second byte (if devices have 16-bit status registers).

An even number of bytes must always be accessed in dual-quad configuration. For this reason, bit 0 of DL[31:0] in OCTOSPI_DLR is stuck at 1 when DMM = 1.

In dual-quad configuration, the behavior of device A interface signals is basically the same as in normal mode. Device B interface signals have exactly the same waveforms as device A ones during the instruction, address, alternate-byte, and dummy-cycle phases. In other words, each device always receives the same instruction and the same address.

Then, during the data phase, the AIOx and the BIOx buses both transfer data in parallel, but the data that is sent to (or received from) device A is distinct than the one from device B.

24.4.6 HyperBus protocol

The OCTOSPI can communicate with the external device using the HyperBus protocol.

The HyperBus uses 11 to 12 pins depending on the operating voltage:

- IO[7:0] as bidirectional data bus
- DQS for read and write data strobe and latency insertion
- NCS
- CLK
- NCLK for 1.8 V operations (to support this mode, the device must be powered with 1.8 V)

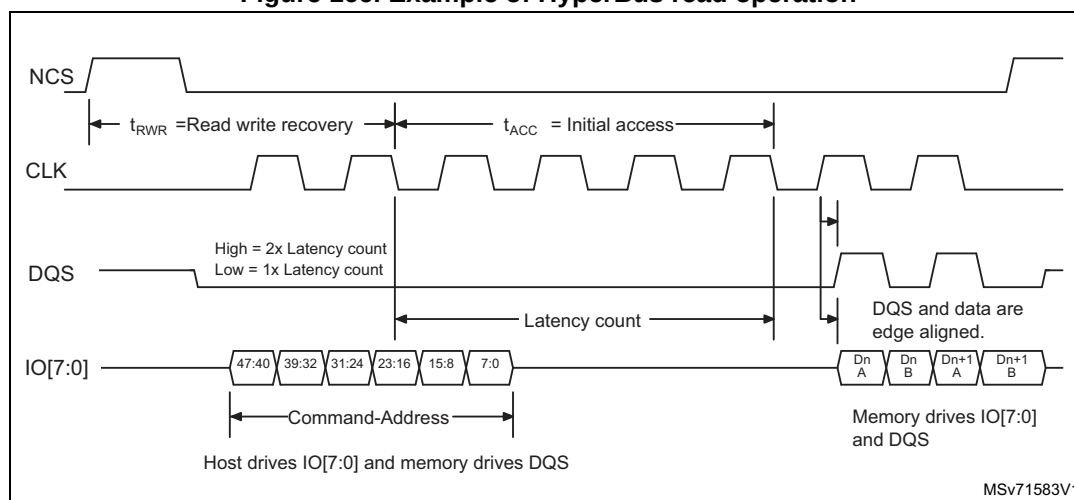
The HyperBus does not require any command specification nor any alternate bytes. As a consequence, a separate register set is used to define the timing of the transaction.

The HyperBus frame is composed of the following phases:

- Command/address phase
- Data phase

The NCS falls before the start of a transaction and rises again after each transaction finishes.

Figure 166. Example of HyperBus read operation



Note: Due to internal synchronization, up to six extra dummy clock cycles may be generated by the Octo-SPI interface after the last data is read.

The specific HyperBus features are configured through the registers in the 0x0200-0x02FC offset range.

Command/address phase

During this initial phase, the OCTOSPI sends 48 bits over IO[7:0] to specify the operations to be performed with the external device.

Table 235. Command/address phase description

CA bit	Bit name	Description
47	R/W#	Identifies the transaction as a read or a write.
46	Address space	Indicates if the transaction accesses the memory or the register space.
45	Burst type	Indicates if the burst is linear or wrapped.
44-16	Row and upper column address	Selects the row and the upper column addresses.
15-3	Reserved	-
2-0	Lower column address	Selects the starting 16-bit word within the half page.

The address space is configured through the memory type MTYP[2:0] in OCTOSPI_DCR1.

The total size of the device must be configured through DEVSZ[4:0] of OCTOSPI_DCR1. In case of multi-chip product (MCP), the device size is the sum of all the sizes of all the MCP dies.

Read/write operation with initial latency

The HyperBus read and write operations need to respect two timings:

- t_{RWR} : minimal read/write recovery time for the device (defined by TRWR[7:0] in OCTOSPI_HLCR)
- t_{ACC} : access time for the device (defined by TACC[7:0] in OCTOSPI_HLCR) according to the memory latency

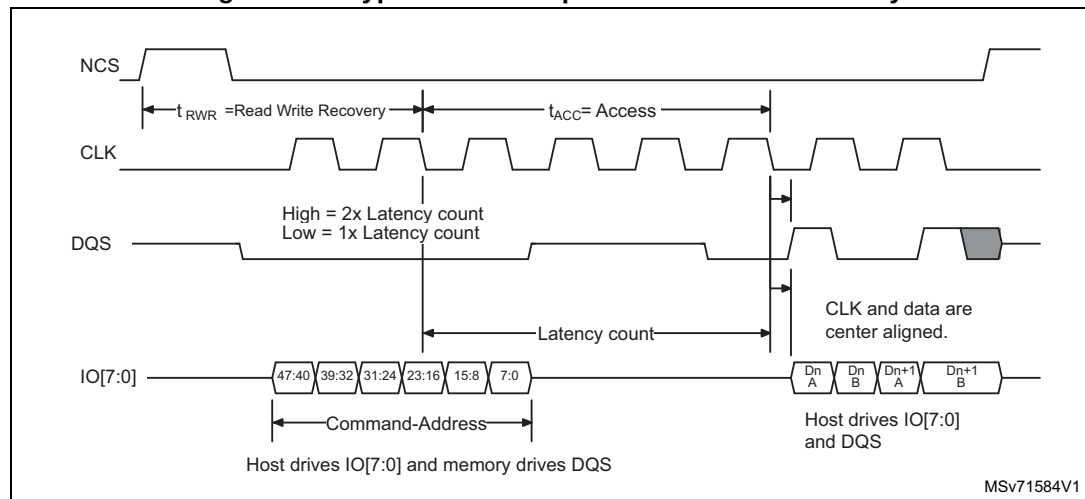
During the read operation, the DQS is used by the device, in two ways (see [Figure 166](#)):

- during the command/address phase, to request an additional latency
- during the data phase, for data strobing

During the write operation, the DQS is used:

- by the device, during the command/address phase, to request an additional latency.
- by the OCTOSPI, during the data phase, for write data masking.

Figure 167. HyperBus write operation with initial latency



Read/write operation with additional latency

If the device needs an additional latency (during refresh period of an SDRAM for example), DQS must be tied to one during one of the DQS signals, during the command/address phase.

An additional t_{ACC} duration is added by the OCTOSPI to meet the device request.

Figure 168. HyperBus read operation with additional latency

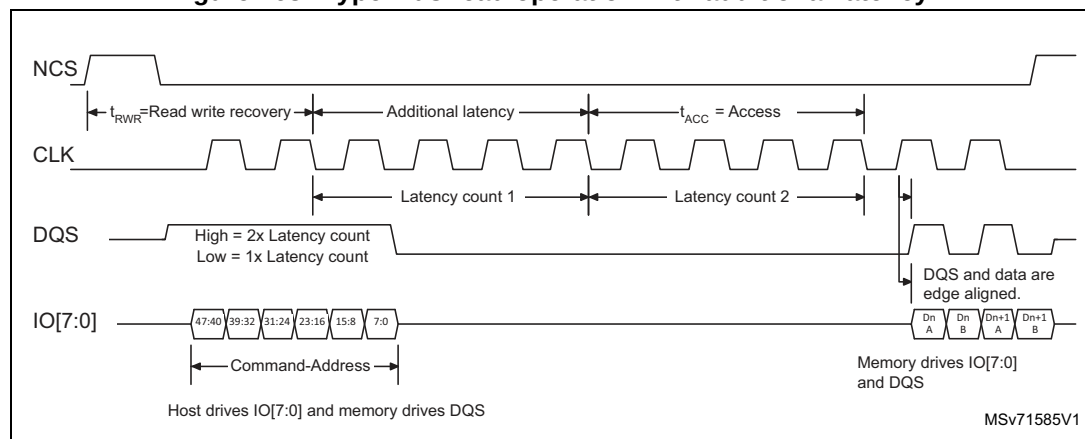
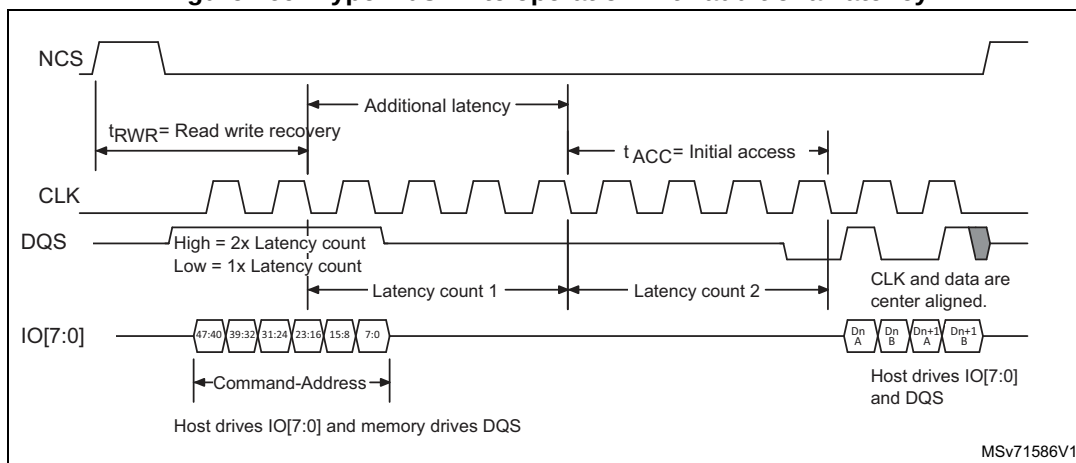


Figure 169. HyperBus write operation with additional latency



Fixed-latency mode

Some devices or some applications may not want to operate with a variable latency time as described above.

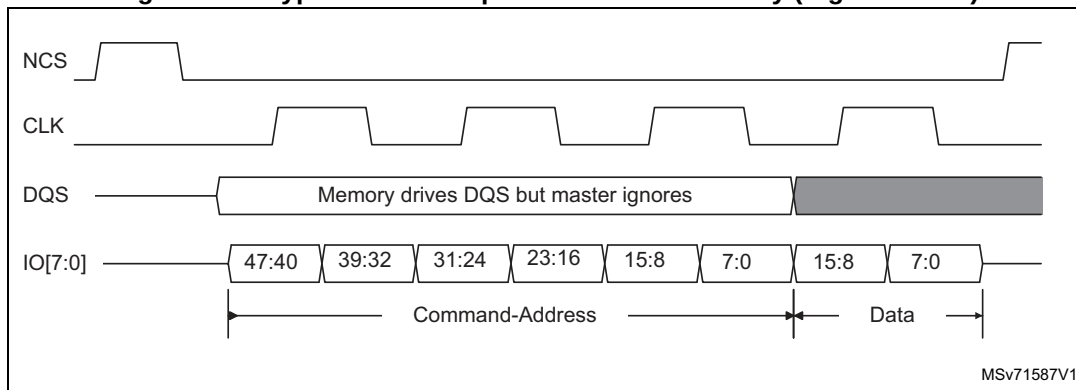
The latency can be forced to $2 \times t_{ACC}$ by setting LM in OCTOSPI_HLCR.

In this OCTOSPI latency mode, the state of the DQS signal is not taken into account by the OCTOSPI, and an additional latency is always added, leading to a fixed $2 \times t_{ACC}$ latency time.

Write operation with no latency

Some devices can also require a zero latency for the write operations. This write-zero latency can be forced by setting WZL in OCTOSPI_HLCR.

Figure 170. HyperBus write operation with no latency (register write)

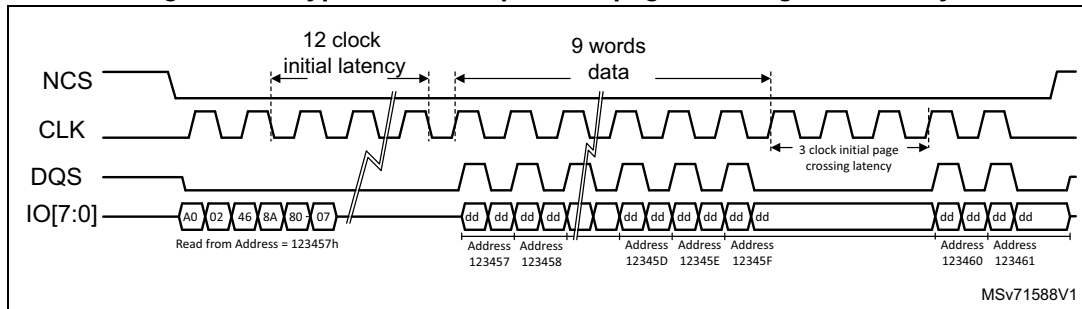


Latency on page-crossing during the read operations

An additional latency can be needed by some devices for the read operation when crossing pages.

The initial latency must be respected for any page access, as a consequence, when the first access is close to the page boundary, a latency is automatically added at the page crossing to respect the t_{ACC} time.

Figure 171. HyperBus read operation page crossing with latency



24.4.7 Specific features

The OCTOSPI supports some specific features, such as:

- Wrap support
- NCS boundary and refresh

Wrap support

The OCTOSPI supports a hybrid wrap as defined by the HyperBus protocol. A hybrid wrap is also supported in the regular-command protocol.

In hybrid wrap, the transaction can continue after the initial wrap with an incremental access.

The wrap size supported by the target memory is configured by WRAPSIZE in OCTOSPI_DCR2.

Wrap is supported only in memory-read direction and only for data size = 4 bytes. Wrapped reads are supported for both HyperBus and regular-command protocols. To enable wrapped-read accesses, the dedicated OCTOSPI_WPxxx registers must be programmed according to the wrapped-read access characteristics. These registers apply for both HyperBus and regular-command protocols.

If the target memory is not supporting the hybrid wrap, WRAPSIZE must be set to 0.

Note: Hybrid wrap requires that the nonwrapped registers (OCTOSPI_CCR, OCTOSPI_TCR, OCTOSPI_IR) are set according to the memory configuration to satisfy its correct data prefetch (initiated after the wrap command).

The wrap operation cannot be interrupted by a refresh. The refresh event is only considered after the wrap completion.

NCS boundary and refresh

Two processes can be activated to regulate the OCTOSPI transactions:

- NCS boundary
- Refresh

The NCS boundary feature limits a transaction to a boundary of aligned addresses. The size of the address to be aligned with, is configured by CSBOUND[4:0] in OCTOSPI_DCR3: it is equal to $2^{CSBOUND}$.

As an example, if CSBOUND[4:0] = 0x4, the boundary is set to $2^4 = 16$ bytes. The NCS is then released each time the LSB address is equal to 0xF, and each time a new transaction is issued to address the next data.

If CSBOUND[4:0] = 0, the feature is disabled. A minimum value of three is recommended.

The NCS boundary feature cannot be used for flash memory devices in write mode since a command is necessary to program another page of the flash memory.

The refresh feature limits the duration of the transactions to the value programmed by REFRESH[31:0] in OCTOSPI_DCR4. The duration is expressed in number of cycles. This allows an external RAM to perform its internal refresh operation regularly.

The refresh value must be greater than the minimal transaction size in terms of number of cycles including the command/address/alternate/dummy phases.

If NCS boundary and refresh are enabled at the same time, the NCS is released on the first condition met.

Restarting after an interrupted transfer

When a read or write operation is interrupted by a timeout or communication regulation feature, the Octo-SPI interface, as soon as possible after getting back the port ownership, reissues the initial command sequence together with the address following the last address actually accessed before interruption. The transfer initially set goes on and ends seamlessly.

24.4.8 OCTOSPI operating mode introduction

The OCTOSPI has the following operating modes regardless of the low-level protocol used (either regular-command or HyperBus):

- indirect mode (read or write)
- automatic status-polling mode (only in regular-command protocol)
- memory-mapped mode

24.4.9 OCTOSPI indirect mode

In indirect mode, the commands are started by writing to the OCTOSPI registers, and data are transferred by writing or reading the data register, in a similar way to other communication peripherals.

When FMODE[1:0] = 00 in OCTOSPI_CR, the OCTOSPI is in indirect-write mode: bytes are sent to the external device during the data phase. Data are provided by writing to OCTOSPI_DR.

When FMODE[1:0] = 01, the OCTOSPI is in indirect-read mode: bytes are received from the external device during the data phase. Data are recovered by reading OCTOSPI_DR.

In indirect mode, when the OCTOSPI is configured in DTR mode over eight lanes with DQS disabled, the given starting address and the data length must be even.

Note: The OCTOSPI_AR register must be updated even if the start address is the same as the start address of the previous indirect access.

The number of bytes to be read/written is specified in OCTOSPI_DLR:

- If DL[31:0] = 0xFFFF FFFF, the data length is considered undefined and the OCTOSPI simply continues to transfer data until it reaches the end of the external device (as

defined by DEVSIZE). If no bytes are to be transferred, DMODE[2:0] must be set to 0 in OCTOSPI_CCR.

- If DL[31:0] = 0xFFFF FFFF and DEVSIZE[4:0] = 0x1F (its maximum value indicating at 4-Gbyte device), the transfers continue indefinitely, stopping only after an abort request or after the OCTOSPI is disabled. After the last memory address is read (at address 0xFFFF FFFF), reading continues with address = 0x0000 0000.

When the programmed number of bytes to be transmitted or received is reached, the TCF bit is set in OCTOSPI_SR, and an interrupt is generated if TCIE = 1 in OCTOSPI_CR. In the case of an undefined number of data, TCF is set when the limit of the external SPI memory is reached, according to the device size defined in OCTOSPI_DCR1.

Triggering the start of a transfer in regular-command protocol

Depending on the OCTOSPI configuration, there are three different ways to trigger the start of a transfer in indirect mode when using the regular-command protocol. In general, the start of transfer is triggered as soon as the software gives the last information that is necessary for the command. More specifically in indirect mode, a transfer starts when one of the following sequence of events occurs:

- if no address is necessary (ADMODE[2:0] = 000) and if no data need to be provided by the software (FMODE[1:0] = 01 or DMODE[2:0] = 000), and at the moment when a write is performed to INSTRUCTION[31:0] in OCTOSPI_IR
- if an address is necessary (when ADMODE[2:0] ≠ 000) and if no data need to be provided by the software (when FMODE[1:0] = 01 or DMODE[2:0] = 000), and at the moment when a write is performed to ADDRESS[31:0] in OCTOSPI_AR
- if data need to be provided by the software (when FMODE[1:0] = 00 and DMODE[2:0] ≠ 000), and at the moment when a write is performed to DATA[31:0] in OCTOSPI_DR

A write to OCTOSPI_ABR never triggers the communication start. If alternate bytes are required, they must have been programmed before.

As soon as a command is started, the BUSY bit is automatically set in OCTOSPI_SR.

Triggering the start of a transfer in HyperBus protocol

Depending on the OCTOSPI configuration, there are different ways to trigger the start of a command in indirect mode. In general, it is triggered as soon as the firmware gives the last information that is necessary for the transfer to start, and more specifically, a communication in indirect mode is triggered by one of the following register settings, when it is the last one to be executed:

- when a write is performed to ADDRESS[31:0] (OCTOSPI_AR) with ADMODE[2:0] ≠ 000 in indirect read mode (FMODE[1:0] = 01).
- when a write is performed to DATA[31:0] (OCTOSPI_DR) in indirect-write mode (when FMODE = 00).
- when a (dummy) write is performed to INSTRUCTION[31:0] (OCTOSPI_IR) for indirect read mode (with ADMODE[2:0] = 000 and FMODE = 01).

As soon as a transfer is started, the BUSY bit (OCTOSPI_SR[5]) is automatically set.

FIFO and data management

Data in indirect mode passes through a 32-byte FIFO that is internal to the OCTOSPI. FLEVEL in OCTOSPI_SR indicates how many bytes are currently being held in the FIFO.

AHB burst transactions are supported. Data of the burst are successively written in OCTOSPI_DR, and immediately transferred in the internal FIFO.

In indirect-write mode (FMODE[1:0] = 00), the software adds data to the FIFO when it writes in OCTOSPI_DR. A word write adds 4 bytes to the FIFO, a half-word write adds 2 bytes, and a byte write adds only 1 byte. If the software adds too many bytes to the FIFO (more than indicated in DL[31:0]), the extra bytes are flushed from the FIFO at the end of the write operation (when TCF is set).

The byte/half-word accesses to OCTOSPI_DR must be done only to the least significant byte/halfword of the 32-bit register.

FTHRES is used to define a FIFO threshold after which point the FIFO threshold flag, FTF, gets set. In indirect-read mode, FTF is set when the number of valid bytes to be read from the FIFO is above the threshold. FTF is also set if there is any data left in the FIFO after the last byte is read from the external device, regardless of FTHRES setting. In indirect-write mode, the FTF is set when the number of empty bytes in the FIFO is above the threshold.

If FTIE = 1, there is an interrupt when the FTF is set. If DMAEN = 1, a DMA transfer is initiated when the FTF is set. The FTF is cleared by hardware as soon as the threshold condition is no longer true (after enough data has been transferred by the CPU or DMA).

The last data read in RX FIFO remains valid as long as there is no request for the next line. This means that, when the application reads several times in a row at the same location, the data is provided from the RX FIFO and not read again from the distant memory.

24.4.10 OCTOSPI automatic status-polling mode

In automatic status-polling mode, the OCTOSPI periodically starts a command to read a defined number of status bytes (up to four). The received bytes can be masked to isolate some status bits and an interrupt can be generated when the selected bits have a defined value. The automatic status-polling mode must be used only in regular-command protocol. For HyperBus protocol, it is not exploitable since the read status register into the HyperFlash memory must be performed in two steps (a write operation followed by a read operation).

The access to the device begins in the same manner as in indirect-read mode. BUSY in OCTOSPI_SR goes high at this point and stays high even between the periodic accesses.

The content of MASK[31:0] in OCTOSPI_PSMAR is used to mask the data from the external device in automatic status-polling mode:

- If the MASK[n] = 0, then bit n of the result is masked and not considered.
- If MASK[n] = 1, and the content of bit[n] is the same as MATCH[n] in OCTOSPI_PSMAR, then there is a match for bit n.

If PMM = 0 in OCTOSPI_CR, the AND-match mode is activated: SMF is set in OCTOSPI_SR only when there is a match on all of the unmasked bits.

If PMM = 1 in OCTOSPI_CR, the OR-match mode is activated: SMF gets set if there is a match on any of the unmasked bits.

An interrupt is called when SMF = 1 if SMIE = 1.

If APMS is set in OCTOSPI_CR, the operation stops and BUSY goes to 0 as soon as a match is detected. Otherwise, BUSY stays at 1 and the periodic accesses continue until there is an abort or until the OCTOSPI is disabled (EN = 0).

OCTOSPI_DR contains the latest received status bytes (FIFO deactivated). The content of this register is not affected by the masking used in the matching logic. FTF in OCTOSPI_SR is set as soon as a new reading of the status is complete. FTF is cleared as soon as the data is read.

In automatic status-polling mode, variable latency is not supported. The memory must then be configured in fixed latency.

24.4.11 OCTOSPI memory-mapped mode

When configured in memory-mapped mode, the external SPI device is seen as an internal memory.

Note: No more than 256 Mbytes can be addressed even if the external device capacity is larger.

If an access is made to an address outside of the range defined by DEVSIZ[4:0] but still within the 256 Mbytes range, then an AHB error is given. The effect of this error depends on the AHB master that attempted the access:

- If it is the Cortex CPU, a hard-fault interrupt is generated.
- If it is a DMA, a DMA transfer error is generated, and the corresponding DMA channel is automatically disabled.

Byte, half-word, and word access types are all supported.

A support for execute in place (XIP) operation is implemented, where the OCTOSPI continues to load the bytes to the addresses following the most recent access. If subsequent accesses are continuous to the bytes that follow, then these operations end up quickly since their results were prefetched.

By default, the OCTOSPI never stops its prefetch operation. It either keeps the previous read operation active with the NCS maintained low or it relaunches a new transfer, even if no access to the external device occurs for a long time.

Since external devices tend to consume more power when the NCS is held low, the application may want to activate the timeout counter (TCEN = 1 in OCTOSPI_CR): the NCS is released after a period defined by TIMEOUT[15:0] in OCTOSPI_LPTR, when x cycles have elapsed without access since the clock is inactive.

BUSY goes high as soon as the first memory-mapped access occurs. Because of the prefetch operations, BUSY does not fall until there is an abort, or the peripheral is disabled.

It is not recommended to program the flash memory using the memory-mapped writes. The indirect-write mode fulfills this operation.

However, if the application requires the use of the MCE for encryption (check MCE product availability), the memory-mapped write mode may be used to program encrypted data to external flash memory under the following conditions:

- Prefetch must be enabled.
- In block cipher mode, the CPU must write a complete 128-bit data block to prevent the MCE from initiating read-modify-write operations when only a few bytes need to be

programmed. This precaution avoids incorrect programming operations. There are no specific constraints to respect if the MCE is used in stream cipher mode.

- Apply the abort sequence to exit memory-mapped mode when the data linked to the page has been written in the external memory buffers. The abort sequence triggers the start of the page programming.
- Switch to the automatic status-polling mode to monitor the completion of the page programming phase.
- Relaunch the write enable command in indirect mode, then switch back to the memory-mapped mode configuration to continue to program additional pages if any.

It is recommended to add a synchronization barrier between the end of the controller registers configuration and the first memory-mapped access to the external memory when the controller is configured in memory-mapped mode.

The controller can apply an automatic address offset to simplify memory area code switching, such as during a firmware update (if the address offset feature is available in the product implementation table). To enable this feature, set the ADOFFEN bit in the XSPI_CR register and configure the offset value in the ADOFF[4:0] bitfield of the XSPI_DCR1 register. The offset is modulo 256 bytes.

24.4.12 OCTOSPI configuration introduction

The OCTOSPI configuration is done in three steps:

1. OCTOSPI system configuration
2. OCTOSPI device configuration
3. OCTOSPI mode configuration

24.4.13 OCTOSPI system configuration

The OCTOSPI is configured using OCTOSPI_CR. The user must program:

- the functional mode with FMODE[1:0]
- the automatic status-polling mode behavior if needed with PMM and APMS
- the FIFO level with FTHRES
- the DMA use with DMAEN
- the timeout counter use with TCEN
- the dual-memory configuration, if needed, with DMM

In case of an interrupt use, the respective enable bit can also be set during this phase.

If the timeout counter is used, the timeout value is programmed in OCTOSPI_LPTR.

The DMA channel must not be enabled during the OCTOSPI configuration: it must be enabled only when the operation is fully configured, to avoid any unexpected request generation.

The DMA and OCTOSPI must be configured in a coherent manner regarding data length: FTHRES value must reflect the DMA burst size.

24.4.14 OCTOSPI device configuration

The parameters related to the external device targeted are configured through OCTOSPI_DCR1 and OCTOSPI_DCR2. The user must program:

- the device size with DEVSIZ[4:0]
- the chip-select minimum high time with CSHT[5:0]
- the clock mode with FRCK and CKMODE
- the device frequency with PRESCALER[7:0]

DEVSIZ[4:0] defines the size of external memory using the following formula:

$$\text{Number of bytes in the device} = 2^{\text{DEVSIZ}+1}$$

where DEVSIZ+1 is the number of address bits required to address the external device. The external device capacity can go up to 4 Gbytes (addressed using 32 bits) in indirect mode, but the addressable space in memory-mapped mode is limited to 256 Mbytes.

If DMM = 1, DEVSIZ[4:0] must reflect the total capacity of the two devices together considering the above formula (DEVSIZ[4:0] value is so equal to one of the two memory capacities).

When the OCTOSPI executes two commands, one immediately after the other, it raises the chip-select signal (NCS) high between the two commands for only one CLK cycle by default.

If the external device requires more time between commands, the chip-select high time CSHT[5:0] can be used to specify the minimum number of CLK cycles for which the NCS must remain high.

CKMODE indicates the level that the CLK takes between commands (when NCS = 1).

In HyperBus protocol, the device timing (t_{ACC} and t_{RWR}) and the latency mode must be configured in OCTOSPI_HLCR.

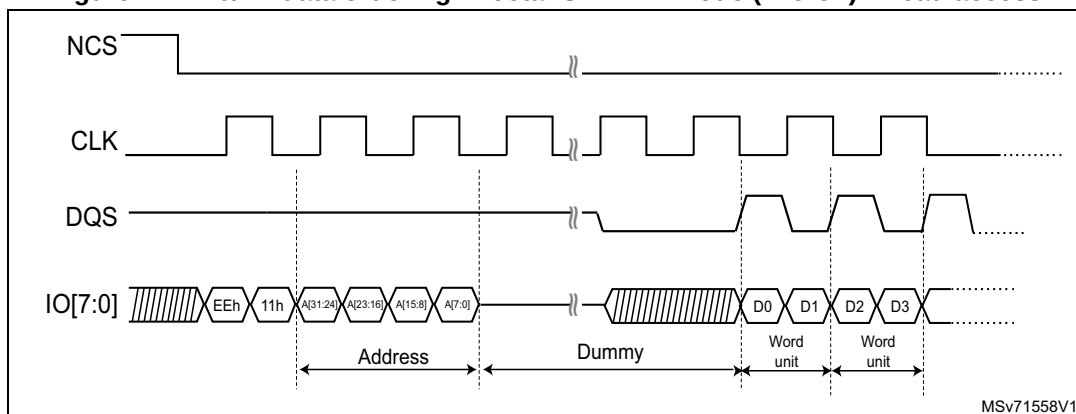
Memory types

External memory providers may present some architecture and slight data formatting differences between them. The bitfield MTYP[2:0] into the OCTOSPI_CR register allows targeting the right controller configuration depending on the associated memory type selected in the application. This is the responsibility of the software developer to align the controller configuration to fit with the targeted memory type.

The memory types are grouped in a such way:

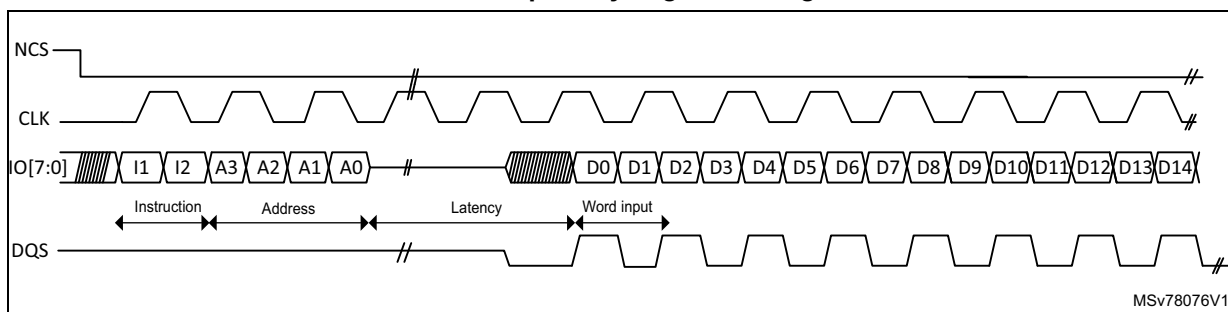
- D0/D1 data ordering in octal-SPI data mode (DMODE[2:0] = 100) in DTR mode by configuring MTYP[2:0] = 000. For instance, Micron is using such data ordering. In this configuration, the DQS is sent with a polarity inverted respect to the clock polarity.

Figure 172. D0/D1 data ordering in octal-SPI DTR mode (Micron) - Read access



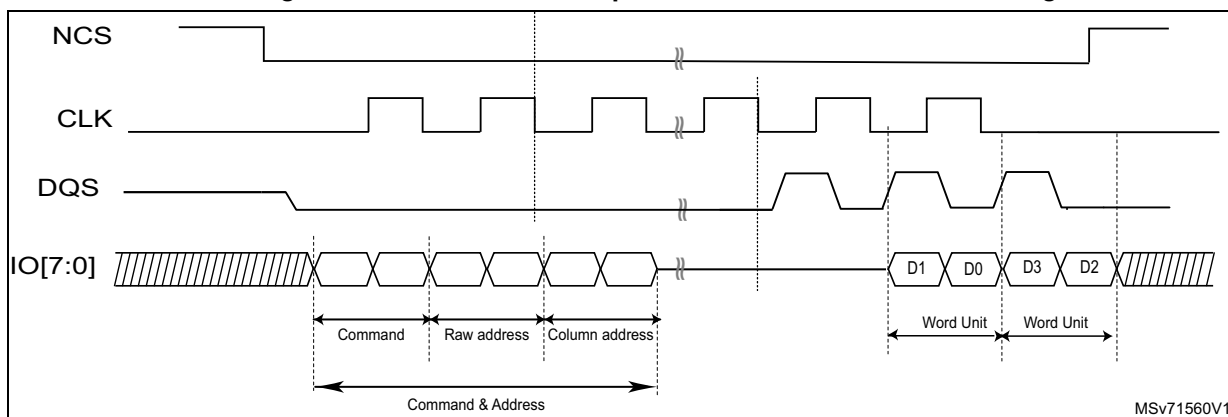
- D1/D0 data ordering in octal-SPI data mode (DMODE[2:0] = 100) in DTR mode by configuring MTYP[2:0] = 001. For instance, Macronix is using this reverse data ordering in its Octaflash portfolio (this configuration is not adapted to its OctaRAM™ memories). DQS is keeping the same polarity as the clock when reading data from the memory. Refer to [Figure 163: DTR read in octal-SPI mode with DQS \(Macronix mode\) example](#).
- D0/D1 data ordering in octal-SPI data mode (DMODE[2:0] = 100) in DTR mode by configuring MTYP[2:0] = 010. DQS is keeping the same polarity as the clock when reading data from the memory. It is defined as the standard mode since it is the configuration to select when the targeted memory does not fit with any others modes proposed by the MTYP configuration values.

Figure 173. D0/D1 data ordering in octal-SPI DTR mode with DQS and CLK polarity aligned during Read access



- D1/D0 data ordering in octal-SPI data mode (DMODE[2:0] = 100) in DTR mode by configuring MTYP[2:0] = 011 with specific address phase built with row and column to fit with Macronix OctaRAM™ memories requirement (refer to [Table 236: OctaRAM command address bit assignment \(based on 64-Mbyte OctaRAM\)](#)). This is the controller which translates internally the targeted address provided by the software in row/column address formatting to sent to the memory. DQS is keeping the same polarity as the clock one when reading data from the memory.

Figure 174. OctaRAM read operation with reverse data ordering D1/D0

Table 236. OctaRAM command address bit assignment
(based on 64-Mbyte⁽¹⁾ OctaRAM)

Clock	1st clock	2nd clock		3rd clock	
Function	Command	Row address		Column address	
SIO[7]	Command	Reserved	RA7	CA9	Reserved
SIO[6]		Reserved	RA6	CA8	Reserved
SIO[5]		Reserved	RA5	CA7	Reserved
SIO[4]		RA12	RA4	CA6	Reserved
SIO[3]		RA11	RA3	CA5	CA3
SIO[2]		RA10	RA2	CA4	CA2
SIO[1]		RA9	RA1	Reserved	CA1
SIO[0]		RA8	RA0	Reserved	CA0 ⁽²⁾

1. Example of 64-Mbyte OctaRAM address assignment:
Row Address [RA12:RA0]: 8K. Column address [CA9:CA0]: 1K. 64-Mbyte density = 8K x 1K x 8 bits

2. Column address A0 must be always 0.

- HyperBus memories need to be selected when targeted by the application. The configuration to set depends on the access type:
 - HyperBus memory mode: The protocol follows the HyperBus specification. MTYP[2:0] = 100 is the configuration to use to access the memory space.
 - HyperBus register mode (addressing register space): the memory-mapped accesses in this mode must be noncacheable, or the indirect read/write modes must be used. The configuration to be set for this particular register space access is MTYP[2:0] = 101.

24.4.15 OCTOSPI regular-command mode configuration

Indirect mode configuration

When $\text{FMODE}[1:0] = 00$, the indirect-write mode is selected and data can be sent to the external device. When $\text{FMODE}[1:0] = 01$, the indirect-read mode is selected, and data can be read from the external device.

When the OCTOSPI is used in indirect mode, the frames are constructed in the following way:

1. Specify a number of data bytes to read or write in `OCTOSPI_DLR`.
2. Specify the frame timing in `OCTOSPI_TCR`.
3. Specify the frame format in `OCTOSPI_CCR`.
4. Specify the instruction in `OCTOSPI_IR`.
5. Specify the optional alternate byte to be sent right after the address phase in `OCTOSPI_ABR`.
6. Specify the targeted address in `OCTOSPI_AR`.
7. Enable the DMA channel if needed.
8. Read/write the data from/to the FIFO through `OCTOSPI_DR` (if no DMA usage).

If neither the address register (`OCTOSPI_AR`) nor the data register (`OCTOSPI_DR`) need to be updated for a particular command, then the command sequence starts as soon as `OCTOSPI_IR` is written. This is the case when both $\text{ADMODE}[2:0]$ and $\text{DMODE}[2:0]$ equal 000, or if just $\text{ADMODE}[2:0] = 000$ when in indirect-read mode ($\text{FMODE}[1:0] = 01$).

When an address is required ($\text{ADMODE}[2:0] \neq 000$) and the data register does not need to be written ($\text{FMODE}[1:0] = 01$ or $\text{DMODE}[2:0] = 000$), the command sequence starts as soon as the address is updated with a write to `OCTOSPI_AR`.

In case of data transmission ($\text{FMODE}[1:0] = 00$ and $\text{DMODE}[2:0] \neq 000$), the communication start is triggered by a write in the FIFO through `OCTOSPI_DR`.

Automatic status-polling mode configuration

The automatic status-polling mode is enabled by setting $\text{FMODE}[1:0] = 10$. In this mode, the programmed frame is sent and data are retrieved periodically.

The maximum amount of data read in each frame is 4 bytes. If more data is requested in `OCTOSPI_DLR`, it is ignored, and only 4 bytes are read. The periodicity is specified in `OCTOSPI_PIR`.

Once the status data has been retrieved, the following can be processed:

- Set SMF (an interrupt is generated if enabled).
- Stop automatically the periodic retrieving of the status bytes.

The received value can be masked with the value stored in `OCTOSPI_PSMKR`, and can be ORed or ANDed with the value stored in `OCTOSPI_PSMAR`.

In case of a match, SMF is set and an interrupt is generated if enabled. The OCTOSPI can be automatically stopped if AMPS is set. In any case, the latest retrieved value is available in `OCTOSPI_DR`.

When the OCTOSPI is used in automatic status-polling mode, the frames are constructed in the following way:

1. Specify the input mask in `OCTOSPI_PSMKR`.

2. Specify the comparison value in OCTOSPI_PSMAR.
3. Specify the read period in OCTOSPI_PIR.
4. Specify a number of data bytes to read in OCTOSPI_DLR.
5. Specify the frame timing in OCTOSPI_TCR.
6. Specify the frame format in OCTOSPI_CCR.
7. Specify the instruction in OCTOSPI_IR.
8. Specify the optional alternate byte to be sent right after the address phase in OCTOSPI_ABR.
9. Specify the optional targeted address in OCTOSPI_AR.

If the address register (OCTOSPI_AR) does not need to be updated for a particular command, then the command sequence starts as soon as OCTOSPI_CCR is written. This is the case when $ADMODE[2:0] = 000$.

When an address is required ($ADMODE[2:0] \neq 000$), the command sequence starts as soon as the address is updated with a write to OCTOSPI_AR.

Memory-mapped mode configuration

In memory-mapped mode, the external device is seen as an internal memory but with some latency during accesses. Read and write operations are allowed to the external device in this mode.

It is not recommended to program the flash memory using memory-mapped writes, as the internal flags for erase or programming status have to be polled. The indirect-write mode fulfills this operation, possibly in conjunction with the automatic status-polling mode.

The memory-mapped mode is entered by setting $FMODE[1:0] = 11$ in OCTOSPI_CR.

The programmed instruction and frame are sent when an AHB master accesses the memory-mapped space.

The FIFO is used as a prefetch buffer to anticipate any linear reads. Any access to OCTOSPI_DR in this mode returns zero.

The data length register (OCTOSPI_DLR) has no meaning in memory-mapped mode.

When the OCTOSPI is used in memory-mapped mode, the frames are constructed in the following way:

1. Specify the frame timing in OCTOSPI_TCR for read operation.
2. Specify the frame format in OCTOSPI_CCR for read operation.
3. Specify the instruction in OCTOSPI_IR.
4. Specify the optional alternate byte to be sent right after the address phase in OCTOSPI_ABR for read operation.
5. Specify the frame timing in OCTOSPI_WTCR for write operation.
6. Specify the frame format in OCTOSPI_WCCR for write operation.
7. Specify the instruction in OCTOSPI_WIR.
8. Specify the optional alternate byte to be sent right after the address phase in OCTOSPI_WABR for write operation.

All configuration operations must be completed (ensured by checking $BUSY = 0$) before the first access to the memory area: any register write operation when $BUSY = 1$ has no effect and is not signaled with an error response. On the first access, the OCTOSPI becomes

busy, and no further configuration is allowed. Then, the only way to get BUSY low is to clear the ENABLE bit or to abort by setting the ABORT bit.

OCTOSPI delayed data sampling when no DQS is used

By default, when no DQS is used, the OCTOSPI samples the data driven by the external device one half of a CLK cycle after the external device drives the signal.

In case of any external signal delays, it may be useful to sample the data later. Using SSHIFT in OCTOSPI_TCR, the sampling of the data can be shifted by half of a CLK cycle.

The firmware must clear SSHIFT when the data phase is configured in DTR mode (DDTR = 1).

OCTOSPI delayed data sampling when DQS is used

When external DQS is used as a sampling clock, it can be shifted in time to compensate the data propagation delay.

This shift is performed by a delay hardware element located outside of the controller into the RCC (RCC_DLYCFGR) or implemented on an external dedicated block delay peripheral depending on the product (refer to the product reference manual for information about how to manage and configure the delay).

In configurations where delay does not need to be compensated, the external delay block can be bypassed by setting DLYBYP in OCTOSPI_DCR1.

24.4.16 OCTOSPI HyperBus protocol configuration

Indirect mode configuration (HyperBus)

When FMODE[1:0] = 00, the indirect-write mode is selected and data can be sent to the external device. When FMODE[1:0] = 01, the indirect-read mode is selected where data can be read from the external device. ADMODE must be configured with a value different from 000 (for instance ADMODE = 100).

When the OCTOSPI is used in indirect mode, the frames are constructed in the following way:

1. Specify a number of data bytes to read or write in OCTOSPI_DLR.
2. Specify the targeted address in OCTOSPI_AR.
3. Enable the DMA channel if needed.
4. Read/write the data from/to the FIFO through OCTOSPI_DR (if no DMA usage).

In indirect-read mode, the command sequence starts as soon as the address is updated with a write to OCTOSPI_AR.

In indirect-write mode, the communication start is triggered by a write in the FIFO through OCTOSPI_DR.

Memory-mapped mode configuration (HyperBus)

In memory-mapped mode, the external device is seen as an internal memory but with some latency during the accesses. Read and write operations are allowed to the external device in this mode.

It is not recommended to program the flash memory using the memory-mapped writes: the indirect-write mode fulfills this operation.

The memory-mapped mode is entered by setting FMODE[1:0] = 11. The programmed instruction and frame is sent when an AHB master accesses the memory-mapped space.

The FIFO is used as a prefetch buffer to anticipate any linear reads. Any access to OCTOSPI_DR in this mode returns zero.

The data length register (OCTOSPI_DLR) has no meaning in memory-mapped mode.

All the configuration operation must be completed before the first access to the memory area. On the first access, the OCTOSPI becomes busy, and no configuration is allowed. Then, the only way to get BUSY low is to clear the ENABLE bit, or to abort by setting the ABORT bit.

24.4.17 OCTOSPI error management

The following errors set the TEF flag in OCTOSPI_SR and generates an interrupt if enabled (TEIE = 1 in OCTOSPI_CR):

- in indirect or automatic status-polling mode, when a wrong address has been programmed in OCTOSPI_AR (according to the device size defined by DEVSZ[4:0]).
- in indirect mode, if the address plus the data length exceed the device size: TEF is set as soon as the access is triggered.

In memory-mapped mode, the OCTOSPI generates an AHB slave error in the following situations, setting the BERRF flag in XSPI_SR and generating an interrupt if enabled (BERRIE = 1 in XSPI_CR) (if the Bus error bit feature is available in the product implementation table, otherwise the Slave bus error response is generated at system level as transaction response with no dedicated bit into the controller status register):

- The memory-mapped mode is disabled and an AHB read or write request occurs.
- A read or write address (including the address offset if ADOFFEN bit is set) exceeds the size of the external memory.
- An abort is received while a read or write burst is ongoing.
- The OCTOSPI is disabled while a read or write burst is requested.
- A write wrap burst is received.
- A write request is received while DQSE = 0 in OCTOSPI_WCCR in octal DTR mode, in dual-memory configuration, in HyperBus mode.
- A read or write request is received while DMODE[2:0] = 000 (no data phase), except when MTYP[2:0] is HyperBus.
- Illegal access size when wrap read burst. This means that the AHB bus transfer size (HSIZE) is different from 4 bytes (only for memory-mapped mode).
- Bit DMM is set in OCTOSPI_CR (dual-memory configuration) and octal mode is set.

24.4.18 OCTOSPI BUSY and ABORT

Once the OCTOSPI starts an operation with the external device, BUSY is automatically set in OCTOSPI_SR.

In indirect mode, BUSY is reset once the OCTOSPI has completed the requested command sequence and the FIFO is empty.

In automatic status-polling mode, BUSY goes low only after the last periodic access is complete, due to a match when APMS = 1 or due to an abort.

After the first access in memory-mapped mode, BUSY goes low only on an abort.

Any operation can be aborted by setting ABORT in OCTOSPI_CR. Once the abort is completed, BUSY and ABORT are automatically reset, and the FIFO is flushed.

Before setting ABORT, the software must ensure that all the current transactions are finished using the synchronization barriers. When DMA is enabled to handle the data read or write operations in OCTOSPI_DR, it is recommended to disable the DMA channel before aborting the OCTOSPI.

Note: Some devices may misbehave if a write operation to a status register is aborted.

24.4.19 OCTOSPI reconfiguration or deactivation

Before any OCTOSPI reconfiguration, the software must ensure that all the transactions are completed:

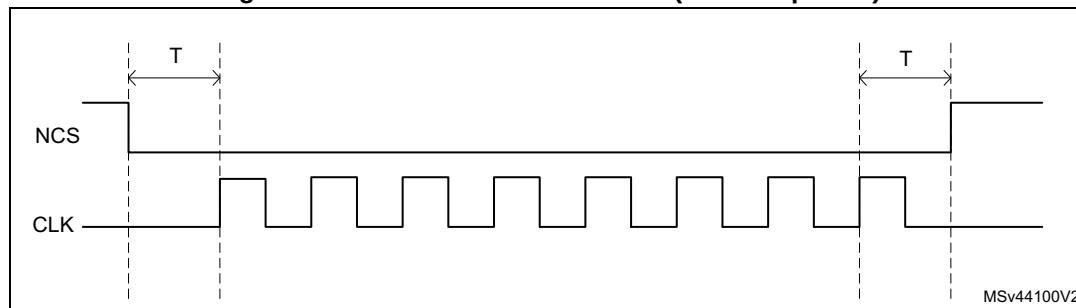
- After a memory-mapped write, the software must perform a dummy read followed by a synchronization barrier, then an abort.
- After a memory-mapped read, the software must perform a synchronization barrier than an abort.

24.4.20 NCS behavior

By default, NCS is high, deselecting the external device. NCS falls before an operation begins and rises as soon as it finishes.

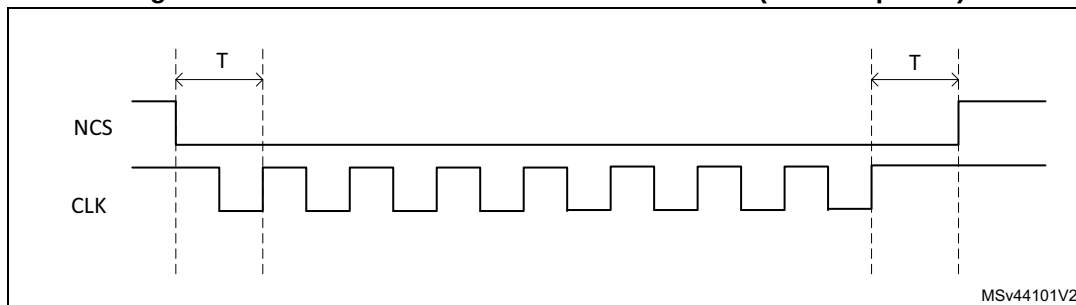
When CKMODE = 0 (clock mode 0: CLK stays low when no operation is in progress), NCS falls one CLK cycle before an operation first rising CLK edge, and NCS rises one CLK cycle after the operation final rising CLK edge (see the figure below).

Figure 175. NCS when CKMODE = 0 (T = CLK period)



When $CKMODE = 1$ (clock mode 3: CLK goes high when no operation is in progress) and when in SDR mode, NCS falls one CLK cycle before an operation first rising CLK edge, and NCS rises one CLK cycle after the operation final rising CLK edge (see the figure below).

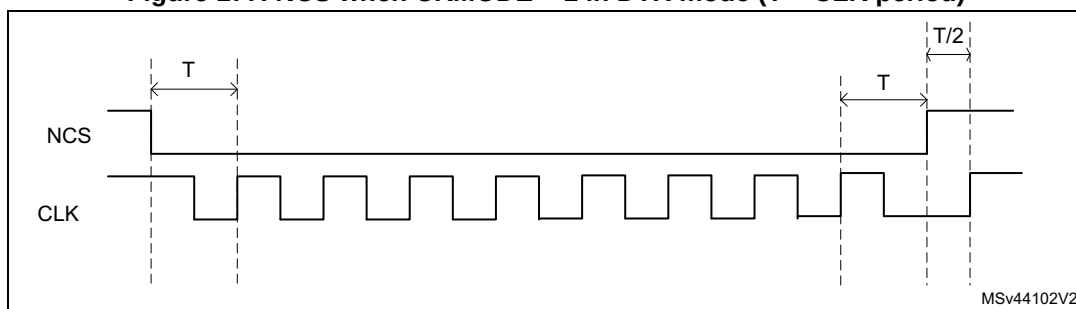
Figure 176. NCS when $CKMODE = 1$ in SDR mode ($T = CLK$ period)



MSv44101V2

When the $CKMODE = 1$ (clock mode 3) and $DDTR = 1$ (data DTR mode), NCS falls one CLK cycle before an operation first rising CLK edge, and NCS rises one CLK cycle after the operation final active rising CLK edge (see the figure below). Because the DTR operations must finish with a falling edge, CLK is low when NCS rises, and CLK rises back up one half of a CLK cycle afterwards.

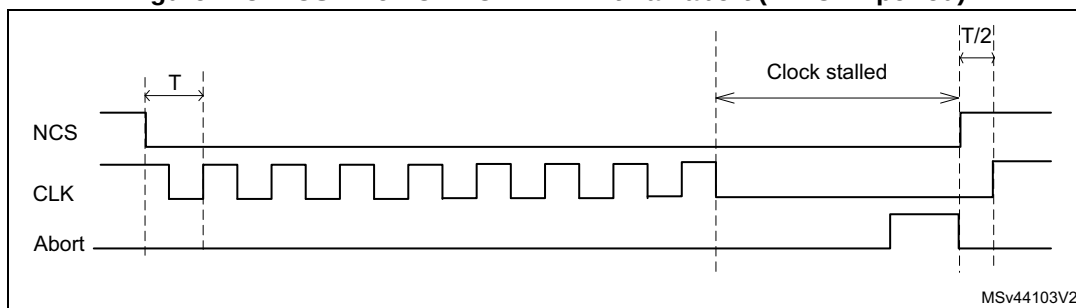
Figure 177. NCS when $CKMODE = 1$ in DTR mode ($T = CLK$ period)



MSv44102V2

When the FIFO stays full during a read operation, or if the FIFO stays empty during a write operation, the operation stalls and CLK stays low until the software services the FIFO. If an abort occurs when an operation is stalled, NCS rises just after the abort is requested and then CLK rises one half of a CLK cycle later (see the figure below).

Figure 178. NCS when $CKMODE = 1$ with an abort ($T = CLK$ period)



MSv44103V2

24.5 Address alignment and data number

The table below summarizes the effect of the address alignment and programmed data number depending on the use case.

Table 237. Address alignment cases

Memory type	Transaction type	Constraint on address ⁽¹⁾	Impact if constraint on address not respected	Constraint on number of bytes ⁽¹⁾	Impact if constraint on bytes not respected
Single, dual, quad flash or SRAM (DMM = 0)	IND ⁽²⁾ read	None	None	None	None
	MM ⁽³⁾ read				
	IND write				
	MM write				
Single, dual, quad flash or SRAM (DMM = 1)	IND read	Even	ADDR[0] is set to 0. ⁽⁴⁾	Even	DLR[0] is set to 1. ⁽⁵⁾
	MM read	None	None	None	None
	IND write	Even	ADDR[0] is set to 0. ⁽⁴⁾	Even	DLR[0] is set to 1. ⁽⁵⁾
	MM write	Even	Slave error	Even	Last byte is lost.
Octal flash in SDR mode	IND read	None	None	None	None
	MM read				
	IND write				
	MM write				
Octal memory in DTR mode without WDM ⁽⁶⁾ or 8-bit HyperFLASH	IND read	Even	ADDR[0] is set to 0. ⁽⁴⁾	Even	DLR[0] is set to 1. ⁽⁵⁾
	MM read	None	None	None	None
	IND write	Even	ADDR[0] is set to 0. ⁽⁴⁾	Even	DLR[0] is set to 1. ⁽⁵⁾
	MM write	Even	Slave error	Even	Last byte is lost.
Octal flash or RAM in DTR mode with WDM	IND read	Even	ADDR[0] is set to 0. ⁽⁴⁾	Even	DLR[0] is set to 1. ⁽⁵⁾
	MM read	None	None	None	None
	IND write ⁽⁷⁾				
	MM write				
8-bit HyperRAM	IND read	Even	ADDR[0] is set to 0. ⁽⁴⁾	Even	DLR[0] is set to 1. ⁽⁵⁾
	MM read	None	None	None	None
	IND write ⁽⁷⁾				
	MM write				

1. To be respected by the software.

2. IND = indirect mode.

3. MM = memory-mapped mode

4. Extra data at transfer start.

5. Extra data at transfer end.

6. WDM = write data mask.

7. If the FTHRES bitfield is set to the maximum value with DLR value greater than the data burst length, and if the DMA is enabled or the interrupt based on FIFO THRESHOLD Flag is enabled (FTF), the address must be modulo 2 aligned in DTR mode when the initiator (DMA, CPU, ...) is writing the data with a burst length equal to the FIFO size.

24.6 OCTOSPI interrupts

An interrupt can be produced on the following events:

- Timeout
- Status match
- FIFO threshold
- Transfer complete
- Transfer error

Separate interrupt enable bits are available to provide more flexibility.

Table 238. OCTOSPI interrupt requests

Interrupt event	Event flag	Enable control bit
Timeout	TOF	TOIE
Status match	SMF	SMIE
FIFO threshold	FTF	FTIE
Transfer complete	TCF	TCIE
Transfer error	TEF	TEIE

24.7 OCTOSPI registers

24.7.1 OCTOSPI control register (OCTOSPI_CR)

Address offset: 0x0000

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	FMODE[1:0]		Res.	Res.	Res.	Res.	PMM	APMS	BERRIE	TOIE	SMIE	FTIE	TCIE	TEIE
		rw	rw					rw	rw	rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	Res.	FTHRES[4:0]					MSEL	DMM	Res.	ADOFFEN	TCEN	DMAEN	ABORT	EN
			rw	rw	rw	rw	rw	rw	rw		rw	rw	rw	w	rw

Bits 31:30 Reserved, must be kept at reset value.

Bits 29:28 **FMODE[1:0]**: Functional mode

This bitfield defines the OCTOSPI functional mode of operation.

00: Indirect-write mode

01: Indirect-read mode

10: Automatic status-polling mode (relevant in regular-command protocol only)

11: Memory-mapped mode

If DMAEN = 1 already, then the DMA controller for the corresponding channel must be disabled before changing the FMODE[1:0] value.

Note: This bitfield can be modified only when BUSY = 0.

Bits 27:24 Reserved, must be kept at reset value.

Bit 23 **PMM**: Polling match mode

This bit indicates which method must be used to determine a match during the automatic status-polling mode.

0: AND-match mode, SMF is set if all the unmasked bits received from the device match the corresponding bits in the match register.

1: OR-match mode, SMF is set if any of the unmasked bits received from the device matches its corresponding bit in the match register.

Note: This bit can be modified only when BUSY = 0.

Bit 22 **APMS**: Automatic status-polling mode stop

This bit determines if the automatic status-polling mode is stopped after a match.

0: Automatic status-polling mode is stopped only by abort or by disabling the OCTOSPI.

1: Automatic status-polling mode stops as soon as there is a match.

Note: This bit can be modified only when BUSY = 0.

Bit 21 **BERRIE**: Bus error interrupt enable

This bit enables the bus error interrupt (if Bus error bit feature is available in the product implementation table).

0: Interrupt disabled

1: Interrupt enabled

Bit 20 **TOIE**: Timeout interrupt enable

This bit enables the timeout interrupt.

0: Interrupt disabled

1: Interrupt enabled

Bit 19 **SMIE**: Status-match interrupt enable

This bit enables the status-match interrupt.

0: Interrupt disabled

1: Interrupt enabled

Bit 18 **FTIE**: FIFO threshold interrupt enable

This bit enables the FIFO threshold interrupt.

0: Interrupt disabled

1: Interrupt enabled

Bit 17 **TCIE**: Transfer complete interrupt enable

This bit enables the transfer complete interrupt.

0: Interrupt disabled

1: Interrupt enabled

- Bit 16 **TEIE**: Transfer error interrupt enable
 This bit enables the transfer error interrupt.
 0: Interrupt disabled
 1: Interrupt enabled

Bits 15:13 Reserved, must be kept at reset value.

- Bits 12:8 **FTHRES[4:0]**: FIFO threshold level
 This bitfield defines, in indirect mode, the threshold number of bytes in the FIFO that causes the FIFO threshold flag FTF in OCTOSPI_SR, to be set.
 00000: FTF is set if there are one or more free bytes available to be written to in the FIFO in indirect-write mode, or if there are one or more valid bytes can be read from the FIFO in indirect-read mode.
 00001: FTF is set if there are two or more free bytes available to be written to in the FIFO in indirect-write mode, or if there are two or more valid bytes can be read from the FIFO in indirect-read mode.
 ...
 11111: FTF is set if there are 32 free bytes available to be written to in the FIFO in indirect-write mode, or if there are 32 valid bytes can be read from the FIFO in indirect-read mode.

Note: If DMAEN = 1, the DMA controller for the corresponding channel must be disabled before changing the FTHRES[4:0] value.

- Bit 7 **MSEL**: External memory select
 This bit selects the external memory to be addressed in single-, dual-, quad-SPI mode in single-memory configuration (when DMM = 0).
 0: External memory 1 selected (data exchanged over IO[3:0])
 1: External memory 2 selected (data exchanged over IO[7:4])
 This bit is ignored when DMM = 1 or when octal-SPI mode is selected.

- Bit 6 **DMM**: Dual-memory configuration
 This bit activates the dual-memory configuration, where two external devices are used simultaneously to double the throughput and the capacity
 0: Dual-memory configuration disabled
 1: Dual-memory configuration enabled

Note: This bit can be modified only when BUSY = 0.

- Bit 5 Reserved, must be kept at reset value.

- Bit 4 **ADOFFEN**: Address offset enable
 An offset may be applied on the memory address. The offset value is set into ADOFF[4:0].
 0: Address offset disabled
 1: Address offset enabled

Note: This bit can be modified only when BUSY = 0. This bit is present only if the address offset feature is available in the product implementation table.

- Bit 3 **TCEN**: Timeout counter enable
 This bit is valid only when the memory-mapped mode (FMODE[1:0] = 11) is selected. This bit enables the timeout counter.
 0: The timeout counter is disabled, and thus the chip-select (NCS) remains active indefinitely after an access in memory-mapped mode.
 1: The timeout counter is enabled, and thus the chip-select is released in the memory-mapped mode after TIMEOUT[15:0] cycles of external device inactivity.

Note: This bit can be modified only when BUSY = 0.

Bit 2 DMAEN: DMA enable

In indirect mode, the DMA can be used to input or output data via OCTOSPI_DR. DMA transfers are initiated when FTF is set.

0: DMA disabled for indirect mode

1: DMA enabled for indirect mode

Note: Resetting the DMAEN bit while a DMA transfer is ongoing, breaks the handshake with the DMA. Do not write this bit during DMA operation.

Bit 1 ABORT: Abort request

This bit aborts the ongoing command sequence. This bit stops the current transfer.

0: No abort requested

1: Abort requested

Note: This bit is always read as 0.

Bit 0 EN: Enable

This bit enables the OCTOSPI.

0: OCTOSPI disabled

1: OCTOSPI enabled

Note: The DMA request can be aborted without having received the ACK in case this EN bit is cleared during the operation.

In case this bit is set to 0 during a DMA transfer, the REQ signal to DMA returns to inactive state without waiting for the ACK signal from DMA to be active.

24.7.2 OCTOSPI device configuration register 1 (OCTOSPI_DCR1)

Address offset: 0x0008

Reset value: 0x0000 0000

This register can be modified only when BUSY = 0.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
ADOFF[4:0]					MTYP[2:0]			Res.	Res.	Res.	DEVSIZ[4:0]				
rw	rw	rw	rw	rw	rw	rw	rw				rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	CSHT[5:0]						Res.	Res.	Res.	Res.	DLY BYP	Res.	FRCK	CKMO DE
		rw	rw	rw	rw	rw	rw					rw		rw	rw

Bits 31:27 ADOFF[4:0]: Address offset

This bitfield defines the memory address offset value added to the current address requested to be sent by the controller to hit the external memory using the following formula:

Offset value = $2^{[ADOFF+8]}$, (modulo 256 bytes)

Maximum allowed value is 19 since the memory-mapped mode is limited to 256 Mbytes address space. For greater value, the address offset will not be applied to the address sent by the controller.

This bitfield is relevant only if ADOFFEN is set and if FMODE[1:0] = 11 (memory-mapped mode) and if the address offset feature is available in the product implementation table.

Bits 26:24 **MTYP[2:0]**: Memory type

This bitfield indicates the type of memory to be supported.

000: Micron mode, D0/D1 ordering in DTR 8-data-bit mode. regular-command protocol in single-, dual-, quad- and octal-SPI modes.

Note: In this mode, DQS signal polarity is inverted with respect to the memory clock signal. This is the default value and care must be taken to change MTYP[2:0] for memories different from Micron.

001: Macronix mode, D1/D0 ordering in DTR 8-data-bit mode. Regular-command protocol in single-, dual-, quad-, and octal-SPI modes.

010: Standard mode. D0/D1 ordering with DQS and CLK having the same polarity during read operation. This is the mode to use when the targeted memory does not fit with any others MTYP configuration values. This mode is typically the one targeting the APmemory 8-bit memories for instance.

011: Macronix RAM mode, D1/D0 ordering in DTR 8-data-bit mode. Regular-command protocol in single-, dual-, quad-, and octal-SPI modes with dedicated address mapping (address is built with row and column to fit with Macronix requirements).

100: HyperBus memory mode, the protocol follows the HyperBus specification.

101: HyperBus register mode, addressing register space. The memory-mapped accesses in this mode must be noncacheable, or indirect-read/write modes must be used.

Others: Reserved

Bits 23:21 Reserved, must be kept at reset value.

Bits 20:16 **DEVSZ[4:0]**: Device size

This bitfield defines the size of the external device using the following formula:

Number of bytes in device = $2^{[DEVSZ+1]}$.

DEVSZ + 1 is effectively the number of address bits required to address the external device. The device capacity can be up to 4 Gbytes (addressed using 32-bits) in indirect mode, but the addressable space in memory-mapped mode is limited to 256 Mbytes.

In regular-command protocol, if DMM = 1, DEVSZ[4:0] must reflect the total capacity of the two devices together considering the above formula (DEVSZ[4:0] value is so equal to one of the two memory capacities).

Bits 15:14 Reserved, must be kept at reset value.

Bits 13:8 **CSHT[5:0]**: Chip-select high time

CSHT + 1 defines the minimum number of CLK cycles where the chip-select (NCS) must remain high between commands issued to the external device.

0x0: NCS stays high for at least 1 cycle between external device commands.

0x1: NCS stays high for at least 2 cycles between external device commands.

...

0x3F: NCS stays high for at least 64 cycles between external device commands.

Bits 7:4 Reserved, must be kept at reset value.

Bit 3 **DLBYBYP**: Delay block bypass

0: The internal sampling clock (called feedback clock) or the DQS data strobe external signal is delayed by the delay block (for more details on this block, refer to the dedicated section of the reference manual as it is not part of the OCTOSPI peripheral).

1: The delay block is bypassed, so the internal sampling clock or the DQS data strobe external signal is not affected by the delay block. The delay is shorter than when the delay block is not bypassed, even with the delay value set to minimum value in delay block.

Bit 2 Reserved, must be kept at reset value.

Bit 1 **FRCK**: Free running clock

This bit configures the free running clock.

0: CLK is not free running.

1: CLK is free running (always provided).

Note: Free running clock mode is intended for delay calibration only. No memory or other device access is possible when FRCK is set. Once the bit has been set to 1, the BSY bit is set, the NCS signal remains set to 1 and the clock is sent. The only way to stop the clock is to perform an abort (which clears the BSY bit). Then, the FRCK bit must be cleared by software to allow controller transactions to take place with the external memory.

Bit 0 **CKMODE**: Clock mode 0/mode 3

This bit indicates the level taken by the CLK between commands (when NCS = 1).

0: CLK must stay low while NCS is high (chip-select released). This is referred to as clock mode 0.

1: CLK must stay high while NCS is high (chip-select released). This is referred to as clock mode 3.

24.7.3 OCTOSPI device configuration register 2 (OCTOSPI_DCR2)

Address offset: 0x000C

Reset value: 0x0000 0000

This register can be modified only when BUSY = 0.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	WRAPSIZE[2:0]		
													rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	PRESCALER[7:0]							
								rw	rw	rw	rw	rw	rw	rw	rw

Bits 31:19 Reserved, must be kept at reset value.

Bits 18:16 **WRAPSIZE[2:0]**: Wrap size

This bitfield indicates the wrap size to which the memory is configured. For memories which have a separate command for wrapped instructions, this bitfield indicates the wrap-size associated with the command held in the OCTOSPI1_WPIR register.

000: Wrapped reads are not supported by the memory.

010: External memory supports wrap size of 16 bytes.

011: External memory supports wrap size of 32 bytes.

100: External memory supports wrap size of 64 bytes.

101: External memory supports wrap size of 128 bytes.

Others: Reserved

Bits 15:8 Reserved, must be kept at reset value.

Bits 7:0 **PRESCALER[7:0]**: Clock prescaler

This bitfield defines the scaler factor for generating the CLK based on the kernel clock (value + 1).

0: $F_{CLK} = F_{KERNEL}$, kernel clock used directly as OCTOSPI CLK (prescaler bypassed). In this case, if the DTR mode is used, it is mandatory to provide to the OCTOSPI a kernel clock that has 50% duty-cycle.

1: $F_{CLK} = F_{KERNEL}/2$

2: $F_{CLK} = F_{KERNEL}/3$

...

255: $F_{CLK} = F_{KERNEL}/256$

For odd clock division factors, the CLK duty cycle is not 50 %. The clock signal remains low one cycle longer than it stays high.

24.7.4 OCTOSPI device configuration register 3 (OCTOSPI_DCR3)

Address offset: 0x0010

Reset value: 0x0000 0000

This register can be modified only when BUSY = 0.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	CSBOUND[4:0]				
											rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.

Bits 31:21 Reserved, must be kept at reset value.

Bits 20:16 **CSBOUND[4:0]**: NCS boundary

This bitfield enables the transaction boundary feature. When active, a minimum value of 3 is recommended. The NCS is released on each boundary of $2^{CSBOUND}$ bytes.

0: NCS boundary disabled

Others: NCS boundary set to $2^{CSBOUND}$ bytes

Bits 15:0 Reserved, must be kept at reset value.

24.7.5 OCTOSPI device configuration register 4 (OCTOSPI_DCR4)

Address offset: 0x0014

Reset value: 0x0000 0000

This register can be modified only when BUSY = 0.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
REFRESH[31:16]															
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
REFRESH[15:0]															
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bits 31:0 **REFRESH[31:0]**: Refresh rate

This bitfield enables the refresh rate feature. The NCS is released every REFRESH + 1 clock cycles for writes, and REFRESH + 4 clock cycles for reads. These two values can be extended with few clock cycles when refresh occurs during a byte transmission in single-, dual- or quad-SPI mode, because the byte transmission must be completed.

0: Refresh disabled

Others: Maximum communication length is set to REFRESH + 1 clock cycles.

Note: REFRESH count is based on the divided clock period: if OCTOSPI_DCR2 PRESCALER bitfield is changed, the REFRESH field must be updated accordingly.

24.7.6 OCTOSPI status register (OCTOSPI_SR)

Address offset: 0x0020

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	FLEVEL[5:0]						Res.	BERRF	BUSY	TOF	SMF	FTF	TCF	TEF
		r	r	r	r	r	r		r	r	r	r	r	r	r

Bits 31:14 Reserved, must be kept at reset value.

Bits 13:8 **FLEVEL[5:0]**: FIFO level

This bitfield gives the number of valid bytes that are being held in the FIFO. FLEVEL = 0 when the FIFO is empty, and 32 when it is full.

In automatic status-polling mode, FLEVEL is zero.

Bit 7 Reserved, must be kept at reset value.

Bit 6 **BERRF**: Bus error flag

This bit is set when a bus error occurs. It is cleared by writing 1 to CBERRF. This bit has a meaning if Bus error bit feature is available in the product implementation table.

Bit 5 **BUSY**: Busy

This bit is set when an operation is ongoing. It is cleared automatically when the operation with the external device is finished and the FIFO is empty.

Bit 4 **TOF**: Timeout flag

This bit is set when timeout occurs. It is cleared by writing 1 to CTOF.

Bit 3 **SMF**: Status match flag

This bit is set in automatic status-polling mode when the unmasked received data matches the corresponding bits in the match register (OCTOSPI_PSMAR).

It is cleared by writing 1 to CSMF.

Bit 2 **FTF**: FIFO threshold flag

In indirect mode, this bit is set when the FIFO threshold has been reached, or if there is any data left in the FIFO after the reads from the external device are complete.

It is cleared automatically as soon as the threshold condition is no longer true.

In automatic status-polling mode, this bit is set every time the status register is read, and the bit is cleared when the data register is read.

Bit 1 **TCF**: Transfer complete flag

This bit is set in indirect mode when the programmed number of data has been transferred or in any mode when the transfer has been aborted. It is cleared by writing 1 to CTCF.

Bit 0 **TEF**: Transfer error flag

This bit is set in indirect mode when an invalid address is being accessed in indirect mode. It is cleared by writing 1 to CTEF.

24.7.7 OCTOSPI flag clear register (OCTOSPI_FCR)

Address offset: 0x0024

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	CBERR F	Res.	CTOF	CSMF	Res.	CTCF	CTEF
									w		w	w		w	w

Bits 31:7 Reserved, must be kept at reset value.

Bit 6 **CBERRF**: Clear bus error flag

Writing 1 clears the BERRF flag in the XSPI_SR register (if Bus error bit feature is available in the product implementation table).

Bit 5 Reserved, must be kept at reset value.

Bit 4 **CTOF**: Clear timeout flag

Writing 1 clears the TOF flag in the OCTOSPI_SR register.

Bit 3 **CSMF**: Clear status match flag

Writing 1 clears the SMF flag in the OCTOSPI_SR register.

Bit 2 Reserved, must be kept at reset value.

Bit 1 **CTCF**: Clear transfer complete flag

Writing 1 clears the TCF flag in the OCTOSPI_SR register.

Bit 0 **CTEF**: Clear transfer error flag

Writing 1 clears the TEF flag in the OCTOSPI_SR register.

24.7.8 OCTOSPI data length register (OCTOSPI_DLR)

Address offset: 0x0040

Reset value: 0x0000 0000

This register can be modified only when BUSY = 0.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
DL[31:16]															
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
DL[15:0]															
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bits 31:0 **DL[31:0]**: Data length

Number of data to be retrieved (value+1) in indirect and automatic status-polling modes. A value not greater than three (indicating 4 bytes) must be used for automatic status-polling mode.

All 1's in indirect mode means undefined length, where OCTOSPI continues until the end of the memory, as defined by DEVSZIE.

0x0000_0000: 1 byte is to be transferred.

0x0000_0001: 2 bytes are to be transferred.

0x0000_0002: 3 bytes are to be transferred.

0x0000_0003: 4 bytes are to be transferred.

...

0xFFFF_FFFD: 4,294,967,294 (4G-2) bytes are to be transferred.

0xFFFF_FFFE: 4,294,967,295 (4G-1) bytes are to be transferred.

0xFFFF_FFFF: undefined length; all bytes, until the end of the external device, (as defined by DEVSZIE) are to be transferred. Continue reading indefinitely if DEVSZIE = 0x1F.

DL[0] is stuck at 1 in dual-memory configuration (DMM = 1) even when 0 is written to this bit, thus assuring that each access transfers an even number of bytes.

This bitfield has no effect in memory-mapped mode.

24.7.9 OCTOSPI address register (OCTOSPI_AR)

Address offset: 0x0048

Reset value: 0x0000 0000

This register can be modified only when BUSY = 0 and FMODE ≠ 11.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
ADDRESS[31:16]															
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
ADDRESS[15:0]															
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bits 31:0 **ADDRESS[31:0]**: Address

Address to be sent to the external device. In HyperBus protocol, this field must be even as this protocol is 16-bit word oriented. In dual-memory configuration, AR[0] is forced to 0.

24.7.10 OCTOSPI data register (OCTOSPI_DR)

Address offset: 0x0050

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
DATA[31:16]															
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
DATA[15:0]															
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bits 31:0 **DATA[31:0]**: Data

Data to be sent/received to/from the external SPI device

In indirect-write mode, data written to this register is stored on the FIFO before it is sent to the external device during the data phase. If the FIFO is too full, a write operation is stalled until the FIFO has enough space to accept the amount of data being written.

In indirect-read mode, reading this register gives (via the FIFO) the data that was received from the external device. If the FIFO does not have as many bytes as requested by the read operation and if BUSY = 1, the read operation is stalled until enough data is present or until the transfer is complete, whichever happens first.

In automatic status-polling mode, this register contains the last data read from the external device (without masking).

Word, half-word, and byte accesses to this register are supported. In indirect-write mode, a byte write adds 1 byte to the FIFO, a half-word write 2 bytes, and a word write 4 bytes.

Similarly, in indirect-read mode, a byte read removes 1 byte from the FIFO, a halfword read 2 bytes, and a word read 4 bytes. Accesses in indirect mode must be aligned to the bottom of this register: A byte read must read DATA[7:0] and a half-word read must read DATA[15:0].

24.7.11 OCTOSPI polling status mask register (OCTOSPI_PSMKR)

Address offset: 0x0080

Reset value: 0x0000 0000

This register can be modified only when BUSY = 0.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
MASK[31:16]															
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
MASK[15:0]															
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bits 31:0 **MASK[31:0]**: Status mask

Mask to be applied to the status bytes received in automatic status-polling mode

For bit n:

0: Bit n of the data received in automatic status-polling mode is masked and its value is not considered in the matching logic.

1: Bit n of the data received in automatic status-polling mode is unmasked and its value is considered in the matching logic.

24.7.12 OCTOSPI polling status match register (OCTOSPI_PSMAR)

Address offset: 0x0088

Reset value: 0x0000 0000

This register can be modified only when BUSY = 0.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
MATCH[31:16]															
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
MATCH[15:0]															
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bits 31:0 **MATCH[31:0]**: Status match

Value to be compared with the masked status register to get a match

24.7.13 OCTOSPI polling interval register (OCTOSPI_PIR)

Address offset: 0x0090

Reset value: 0x0000 0000

This register can be modified only when BUSY = 0.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
INTERVAL[15:0]															
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bits 31:16 Reserved, must be kept at reset value.

Bits 15:0 **INTERVAL[15:0]**: Polling interval

Number of CLK cycles between a read during the automatic status-polling phases

24.7.14 OCTOSPI communication configuration register (OCTOSPI_CCR)

Address offset: 0x0100

Reset value: 0x0000 0000

This register can be modified only when BUSY = 0.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	DQSE	Res.	DDTR	DMODE[2:0]			Res.	Res.	ABSIZE[1:0]		ABDTR	ABMODE[2:0]		
		rw		rw	rw	rw	rw			rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	ADSIZE[1:0]		AD DTR	ADMODE[2:0]			Res.	Res.	ISIZE[1:0]		IDTR	IMODE[2:0]		
		rw	rw	rw	rw	rw	rw			rw	rw	rw	rw	rw	rw

Bits 31:30 Reserved, must be kept at reset value.

Bit 29 **DQSE**: DQS enable

This bit enables the data strobe management.

0: DQS disabled

1: DQS enabled

Bit 28 Reserved, must be kept at reset value.

Bit 27 **DDTR**: Data double transfer rate

This bit sets the DTR mode for the data phase.

0: DTR mode disabled for data phase

1: DTR mode enabled for data phase

Bits 26:24 **DMODE[2:0]**: Data mode

This bitfield defines the data phase mode of operation.

000: No data

001: Data on a single line

010: Data on two lines

011: Data on four lines

100: Data on eight lines

Others: Reserved

Bits 23:22 Reserved, must be kept at reset value.

Bits 21:20 **ABSIZE[1:0]**: Alternate-byte size

This bitfield defines the alternate-byte size.

00: 8-bit alternate bytes

01: 16-bit alternate bytes

10: 24-bit alternate bytes

11: 32-bit alternate bytes

Bit 19 **ABDTR**: Alternate- byte double transfer rate

This bit sets the DTR mode for the alternate-byte phase.

0: DTR mode disabled for the alternate-byte phase

1: DTR mode enabled for the alternate-byte phase

Bits 18:16 **ABMODE[2:0]**: Alternate-byte mode

This bitfield defines the alternate-byte phase mode of operation.

000: No alternate bytes

001: Alternate bytes on a single line

010: Alternate bytes on two lines

011: Alternate bytes on four lines

100: Alternate bytes on eight lines

Others: Reserved

Bits 15:14 Reserved, must be kept at reset value.

Bits 13:12 **ADSIZE[1:0]**: Address size

This bitfield defines the address size.

00: 8-bit address

01: 16-bit address

10: 24-bit address

11: 32-bit address

Bit 11 **ADDTR**: Address double transfer rate

This bit sets the DTR mode for the address phase.

0: DTR mode disabled for the address phase

1: DTR mode enabled for the address phase

Bits 10:8 **ADMODE[2:0]**: Address mode

This bitfield defines the address phase mode of operation.

000: No address

001: Address on a single line

010: Address on two lines

011: Address on four lines

100: Address on eight lines

Others: Reserved

Bits 7:6 Reserved, must be kept at reset value.

Bits 5:4 **ISIZE[1:0]**: Instruction size

This bitfield defines instruction size.

00: 8-bit instruction

01: 16-bit instruction

10: 24-bit instruction

11: 32-bit instruction

Bit 3 **IDTR**: Instruction double transfer rate

This bit sets the DTR mode for the instruction phase.

0: DTR mode disabled for the instruction phase

1: DTR mode enabled for the instruction phase

Bits 2:0 **IMODE[2:0]**: Instruction mode

This bitfield defines the instruction phase mode of operation.

000: No instruction

001: Instruction on a single line

010: Instruction on two lines

011: Instruction on four lines

100: Instruction on eight lines

Others: Reserved

24.7.15 OCTOSPI timing configuration register (OCTOSPI_TCR)

Address offset: 0x0108

Reset value: 0x0000 0000

This register can be modified only when BUSY = 0.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	SSHIFT	Res.	DHQC	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
	rw		rw												
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	DCYC[4:0]				
											rw	rw	rw	rw	rw

Bit 31 Reserved, must be kept at reset value.

Bit 30 **SSHIFT**: Sample shift

By default, the OCTOSPI samples data 1/2 of a CLK cycle after the data is driven by the external device.

This bit allows the data to be sampled later in order to consider the external signal delays.

0: No shift

1: 1/2 cycle shift

The software must ensure that SSHIFT = 0 when the data phase is configured in DTR mode (when DDTR = 1.)

Bit 29 Reserved, must be kept at reset value.

Bit 28 **DHQC**: Delay hold quarter cycle

0: No delay hold

1: 1/4 cycle hold

Bits 27:5 Reserved, must be kept at reset value.

Bits 4:0 **DCYC[4:0]**: Number of dummy cycles

This bitfield defines the duration of the dummy phase according to the memory latency.

In both SDR and DTR modes, it specifies a number of CLK cycles (0-31).

24.7.16 OCTOSPI instruction register (OCTOSPI_IR)

Address offset: 0x0110

Reset value: 0x0000 0000

This register can be modified only when BUSY = 0.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
INSTRUCTION[31:16]															
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
INSTRUCTION[15:0]															
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bits 31:0 **INSTRUCTION[31:0]**: Instruction

Instruction to be sent to the external SPI device

24.7.17 OCTOSPI alternate bytes register (OCTOSPI_ABR)

Address offset: 0x0120

Reset value: 0x0000 0000

This register can be modified only when BUSY = 0.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
ALTERNATE[31:16]															
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
ALTERNATE[15:0]															
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bits 31:0 **ALTERNATE[31:0]**: Alternate bytes

Optional data to be sent to the external SPI device right after the address.

24.7.18 OCTOSPI low-power timeout register (OCTOSPI_LPTR)

Address offset: 0x00130

Reset value: 0x0000 0000

This register can be modified only when BUSY = 0.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
TIMEOUT[15:0]															
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bits 31:16 Reserved, must be kept at reset value.

Bits 15:0 **TIMEOUT[15:0]**: Timeout period

After each access in memory-mapped mode, the OCTOSPI prefetches the subsequent bytes and hold them in the FIFO.

This bitfield indicates how many CLK cycles the OCTOSPI waits after the clock becomes inactive and until it raises the NCS, putting the external device in a lower-consumption state.

24.7.19 OCTOSPI wrap communication configuration register (OCTOSPI_WPCCR)

Address offset: 0x0140

Reset value: 0x0000 0000

This register can be modified only when BUSY = 0.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	DQSE	Res.	DDTR	DMODE[2:0]			Res.	Res.	ABSIZE[1:0]		ABDTR	ABMODE[2:0]		
		rw		rw	rw	rw	rw			rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	ADSIZE[1:0]		AD DTR	ADMODE[2:0]			Res.	Res.	ISIZE[1:0]		IDTR	IMODE[2:0]		
		rw	rw	rw	rw	rw	rw			rw	rw	rw	rw	rw	rw

Bits 31:30 Reserved, must be kept at reset value.

Bit 29 **DQSE**: DQS enable

This bit enables the data strobe management.

0: DQS disabled

1: DQS enabled

Bit 28 Reserved, must be kept at reset value.

- Bit 27 **DDTR**: Data double transfer rate
This bit sets the DTR mode for the data phase.
0: DTR mode disabled for the data phase
1: DTR mode enabled for the data phase
- Bits 26:24 **DMODE[2:0]**: Data mode
This bitfield defines the data phase mode of operation.
000: No data
001: Data on a single line
010: Data on two lines
011: Data on four lines
100: Data on eight lines
Others: Reserved
- Bits 23:22 Reserved, must be kept at reset value.
- Bits 21:20 **ABSIZE[1:0]**: Alternate-byte size
This bitfield defines the alternate-byte size.
00: 8-bit alternate bytes
01: 16-bit alternate bytes
10: 24-bit alternate bytes
11: 32-bit alternate bytes
- Bit 19 **ABDTR**: Alternate-byte double transfer rate
This bit sets the DTR mode for the alternate-byte phase.
0: DTR mode disabled for the alternate-byte phase
1: DTR mode enabled for the alternate-byte phase
- Bits 18:16 **ABMODE[2:0]**: Alternate-byte mode
This bitfield defines the alternate-byte phase mode of operation.
000: no alternate bytes
001: alternate bytes on a single line
010: alternate bytes on two lines
011: alternate bytes on four lines
100: alternate bytes on eight lines
Others: reserved
- Bits 15:14 Reserved, must be kept at reset value.
- Bits 13:12 **ADSIZE[1:0]**: Address size
This bitfield defines the address size.
00: 8-bit address
01: 16-bit address
10: 24-bit address
11: 32-bit address
- Bit 11 **ADDTR**: Address double transfer rate
This bit sets the DTR mode for the address phase.
0: DTR mode disabled for address phase
1: DTR mode enabled for address phase

Bits 10:8 **ADMODE[2:0]**: Address mode

This bitfield defines the address phase mode of operation.

000: No address
 001: Address on a single line
 010: Address on two lines
 011: Address on four lines
 100: Address on eight lines
 Others: Reserved

Bits 7:6 Reserved, must be kept at reset value.

Bits 5:4 **ISIZE[1:0]**: Instruction size

This bitfield defines the instruction size.

00: 8-bit instruction
 01: 16-bit instruction
 10: 24-bit instruction
 11: 32-bit instruction

Bit 3 **IDTR**: Instruction double transfer rate

This bit sets the DTR mode for the instruction phase.

0: DTR mode disabled for the instruction phase
 1: DTR mode enabled for the instruction phase

Bits 2:0 **IMODE[2:0]**: Instruction mode

This bitfield defines the instruction phase mode of operation.

000: No instruction
 001: Instruction on a single line
 010: Instruction on two lines
 011: Instruction on four lines
 100: Instruction on eight lines
 Others: Reserved

24.7.20 OCTOSPI wrap timing configuration register (OCTOSPI_WPTCR)

Address offset: 0x0148

Reset value: 0x0000 0000

This register can be modified only when BUSY = 0.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	S SHIFT	Res.	DHQC	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
	rw		rw												
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	DCYC[4:0]				
											rw	rw	rw	rw	rw

Bit 31 Reserved, must be kept at reset value.

Bit 30 **SSHIFT**: Sample shift

By default, the OCTOSPI samples data 1/2 of a CLK cycle after the data is driven by the external device.

This bit allows the data to be sampled later in order to consider the external signal delays.

0: No shift

1: 1/2 cycle shift

The firmware must assure that SSHIFT=0 when the data phase is configured in DTR mode (when DDTR = 1).

Bit 29 Reserved, must be kept at reset value.

Bit 28 **DHQC**: Delay hold quarter cycle

Add a quarter cycle delay on the outputs in DTR communication to match hold requirement.

0: No quarter cycle delay

1: 1/4 cycle delay inserted

Bits 27:5 Reserved, must be kept at reset value.

Bits 4:0 **DCYC[4:0]**: Number of dummy cycles

This bitfield defines the duration of the dummy phase according to the memory latency.

In both SDR and DTR modes, it specifies a number of CLK cycles (0-31).

24.7.21 OCTOSPI wrap instruction register (OCTOSPI_WPIR)

Address offset: 0x0150

Reset value: 0x0000 0000

This register can be modified only when BUSY = 0.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
INSTRUCTION[31:16]															
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
INSTRUCTION[15:0]															
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bits 31:0 **INSTRUCTION[31:0]**: Instruction

Instruction to be sent to the external SPI device

24.7.22 OCTOSPI wrap alternate bytes register (OCTOSPI_WPABR)

Address offset: 0x0160

Reset value: 0x0000 0000

This register can be modified only when BUSY = 0.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
ALTERNATE[31:16]															
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
ALTERNATE[15:0]															
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bits 31:0 **ALTERNATE[31:0]**: Alternate bytes

Optional data to be sent to the external SPI device right after the address

24.7.23 OCTOSPI write communication configuration register (OCTOSPI_WCCR)

Address offset: 0x0180

Reset value: 0x0000 0000

This register can be modified only when BUSY = 0. Its content has a meaning only when requesting write operations in memory-mapped mode.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	DQSE	Res.	DDTR	DMODE[2:0]			Res.	Res.	ABSIZE[1:0]		ABDTR	ABMODE[2:0]		
		rw		rw	rw	rw	rw			rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	ADSIZE[1:0]		ADDTR	ADMODE[2:0]			Res.	Res.	ISIZE[1:0]		IDTR	IMODE[2:0]		
		rw	rw	rw	rw	rw	rw			rw	rw	rw	rw	rw	rw

Bits 31:30 Reserved, must be kept at reset value.

Bit 29 **DQSE**: DQS enable

This bit enables the data strobe management.

0: DQS disabled

1: DQS enabled

Bit 28 Reserved, must be kept at reset value.

Bit 27 **DDTR**: data double transfer rate

This bit sets the DTR mode for the data phase.

0: DTR mode disabled for the data phase

1: DTR mode enabled for the data phase

Bits 26:24 **DMODE[2:0]**: Data mode

This bitfield defines the data phase mode of operation.

000: No data

001: Data on a single line

010: Data on two lines

011: Data on four lines

100: Data on eight lines

Others: Reserved

Bits 23:22 Reserved, must be kept at reset value.

Bits 21:20 **ABSIZE[1:0]**: Alternate-byte size

This bitfield defines the alternate-byte size.

00: 8-bit alternate bytes

01: 16-bit alternate bytes

10: 24-bit alternate bytes

11: 32-bit alternate bytes

- Bit 19 **ABDTR**: Alternate bytes double transfer rate
This bit sets the DTR mode for the alternate-bytes phase.
0: DTR mode disabled for alternate-bytes phase
1: DTR mode enabled for alternate-bytes phase
- Bits 18:16 **ABMODE[2:0]**: Alternate-byte mode
This bitfield defines the alternate-byte phase mode of operation.
000: No alternate bytes
001: Alternate bytes on a single line
010: Alternate bytes on two lines
011: Alternate bytes on four lines
100: Alternate bytes on eight lines
Others: Reserved
- Bits 15:14 Reserved, must be kept at reset value.
- Bits 13:12 **ADSIZE[1:0]**: Address size
This bitfield defines the address size.
00: 8-bit address
01: 16-bit address
10: 24-bit address
11: 32-bit address
- Bit 11 **ADDTR**: Address double transfer rate
This bit sets the DTR mode for the address phase.
0: DTR mode disabled for the address phase
1: DTR mode enabled for the address phase
- Bits 10:8 **ADMODE[2:0]**: Address mode
This bitfield defines the address phase mode of operation.
000: No address
001: Address on a single line
010: Address on two lines
011: Address on four lines
100: Address on eight lines
Others: Reserved
- Bits 7:6 Reserved, must be kept at reset value.
- Bits 5:4 **ISIZE[1:0]**: Instruction size
This bitfield defines the instruction size.
00: 8-bit instruction
01: 16-bit instruction
10: 24-bit instruction
11: 32-bit instruction
- Bit 3 **IDTR**: Instruction double transfer rate
This bit sets the DTR mode for the instruction phase.
0: DTR mode disabled for instruction phase
1: DTR mode enabled for instruction phase

Bits 2:0 **IMODE[2:0]**: Instruction mode

This bitfield defines the instruction phase mode of operation.

000: No instruction

001: Instruction on a single line

010: Instruction on two lines

011: Instruction on four lines

100: Instruction on eight lines

Others: Reserved

24.7.24 OCTOSPI write timing configuration register (OCTOSPI_WTCR)

Address offset: 0x0188

Reset value: 0x0000 0000

This register can be modified only when BUSY = 0. Its content has a meaning only when requesting write operations in memory-mapped mode.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	DCYC[4:0]				

Bits 31:5 Reserved, must be kept at reset value.

Bits 4:0 **DCYC[4:0]**: Number of dummy cycles

This bitfield defines the duration of the dummy phase according to the memory latency.

In both SDR and DTR modes, it specifies a number of CLK cycles (0-31).

24.7.25 OCTOSPI write instruction register (OCTOSPI_WIR)

Address offset: 0x0190

Reset value: 0x0000 0000

This register can be modified only when BUSY = 0. Its content has a meaning only when requesting write operations in memory-mapped mode.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
INSTRUCTION[31:16]															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
INSTRUCTION[15:0]															

Bits 31:0 **INSTRUCTION[31:0]**: Instruction

Instruction to be sent to the external SPI device

24.7.26 OCTOSPI write alternate bytes register (OCTOSPI_WABR)

Address offset: 0x01A0

Reset value: 0x0000 0000

This register can be modified only when BUSY = 0. Its content has a meaning only when requesting write operations in memory-mapped mode.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
ALTERNATE[31:16]															
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
ALTERNATE[15:0]															
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bits 31:0 **ALTERNATE[31:0]**: Alternate bytes

Optional data to be sent to the external SPI device right after the address

24.7.27 OCTOSPI HyperBus latency configuration register (OCTOSPI_HLCR)

Address offset: 0x0200

Reset value: 0x0000 0000

This register can be modified only when BUSY = 0.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	TRWR[7:0]							
								rw	rw	rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
TACC[7:0]								Res.	Res.	Res.	Res.	Res.	Res.	WZL	LM
rw	rw	rw	rw	rw	rw	rw	rw							rw	rw

Bits 31:24 Reserved, must be kept at reset value.

Bits 23:16 **TRWR[7:0]**: Read-write minimum recovery time

Device read-to-write/write-to-read minimum recovery time expressed in number of communication clock cycles

Bits 15:8 **TACC[7:0]**: Access time

Device access time according to the memory latency, expressed in number of communication clock cycles

Bits 7:2 Reserved, must be kept at reset value.

Bit 1 **WZL**: Write zero latency

This bit enables zero latency on write operations.

0: Latency on write accesses

1: No latency on write accesses

Bit 0 **LM**: Latency mode

This bit selects the latency mode.

0: Variable initial latency

1: Fixed latency

Note: This bit must be kept cleared for HyperFLASH memory.

24.7.28 OCTOSPI register map

Table 239. OCTOSPI register map and reset values

Offset	Register name	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0x0000	OCTOSPI_CR	Res.	Res.	FMDOE[1:0]		Res.	Res.	Res.	Res.	PMM	APMS	BERRIE	TOIE	SMIE	FTIE	TCIE	TEIE	Res.	Res.	Res.	FTHRES[4:0]				MSEL	DMM	Res.	ADOFFEN	TCEN	DMAEN	ABORT	EN	
	Reset value			0	0					0	0	0	0	0	0	0	0				0	0	0	0	0	0	0	0		0	0	0	0
0x0004	Reserved	Reserved																															
0x0008	OCTOSPI_DCR1	ADOFF[4:0]				MTYP[2:0]		Res.	Res.	Res.	DEVSIZ[4:0]				Res.	Res.	CSHT[5:0]				Res.	Res.	Res.	Res.	Res.	Res.	DLYBYP	Res.	FRCK	CKMODE			
	Reset value	0	0	0	0	0	0	0	0				0	0	0	0	0			0	0	0	0	0	0					0		0	0
0x000C	OCTOSPI_DCR2	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	WRAPSIZ[2:0]		Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	PRESCALER[7:0]								
	Reset value														0	0	0								0	0	0	0	0	0	0	0	0
0x0010	OCTOSPI_DCR3	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	CSBOUND[4:0]				Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
	Reset value												0	0	0	0	0																
0x0014	OCTOSPI_DCR4	REFRESH[31:0]																															
	Reset value	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0x0018-0x001C	Reserved	Reserved																															
0x0020	OCTOSPI_SR	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	FLEVEL[5:0]				Res.	BERRF	BUSY	TOF	SMF	FTF	TCF	TEF				
	Reset value																			0	0	0	0	0	0		0	0	0	0	0	0	0
0x0024	OCTOSPI_FCR	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	CBERRF	Res.	CTOF	CSMF	Res.	CTCF	CTEF
	Reset value																										0		0	0		0	0
0x0028-0x003C	Reserved	Reserved																															
0x0040	OCTOSPI_DLR	DL[31:0]																															
	Reset value	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0x0044	Reserved	Reserved																															

Table 239. OCTOSPI register map and reset values (continued)

Offset	Register name	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0x0048	OCTOSPI_AR	ADDRESS[31:0]																															
	Reset value	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0x004C	Reserved	Reserved																															
0x0050	OCTOSPI_DR	DATA[31:0]																															
	Reset value	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0x0054-0x007C	Reserved	Reserved																															
0x0080	OCTOSPI_PSMKR	MASK[31:0]																															
	Reset value	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0x0084	Reserved	Reserved																															
0x0088	OCTOSPI_PSMAR	MATCH[31:0]																															
	Reset value	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0x008C	Reserved	Reserved																															
0x0090	OCTOSPI_PIR	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	INTERVAL[15:0]																
	Reset value																	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0x0094-0x00FC	Reserved	Reserved																															
0x0100	OCTOSPI_CCR	Res			DQSE		DDTR	DMODE [2:0]			Res	Res	ABSIZ [1:0]		ABDTR	ABMODE [2:0]			Res	Res	ADSIZ [1:0]		ADDTR	ADMODE [2:0]			Res	Res	ISIZ [1:0]		IDTR	IMODE [2:0]	
	Reset value			0		0	0	0	0				0	0	0	0	0	0				0	0	0	0	0	0			0	0	0	0
0x0104	Reserved	Reserved																															
0x0108	OCTOSPI_TCR	Res	SSHIFT		DHQC		Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	DCYC[4:0]			
	Reset value		0		0																									0	0	0	0
0x010C	Reserved	Reserved																															
0x0110	OCTOSPI_IR	INSTRUCTION[31:0]																															
	Reset value	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0x0114-0x011C	Reserved	Reserved																															
0x0120	OCTOSPI_ABR	ALTERNATE[31:0]																															
	Reset value	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0x0124-0x012C	Reserved	Reserved																															
0x0130	OCTOSPI_LPTR	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	TIMEOUT[15:0]															
	Reset value																	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0x0134-0x013C	Reserved	Reserved																															

Table 239. OCTOSPI register map and reset values (continued)

Offset	Register name	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0x0140	OCTOSPI_WPCCR	Res	Res	DQSE	Res	DDTR	DMODE [2:0]			Res	Res	ABSIZE [1:0]		ABDTR	ABMODE [2:0]		Res	Res	Res	Res	ADSIZE [1:0]	ADDTR	ADMODE [2:0]		Res	Res	Res	Res	ISIZE [1:0]	IDTR	IMODE [2:0]		
	Reset value	0		0		0	0	0	0			0	0	0	0	0	0				0	0	0	0	0	0			0	0	0	0	0
0x0144	Reserved	Reserved																															
0x0148	OCTOSPI_WPTCR	Res	SSHIFT	Res	DHQC	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	DCYC[4:0]				
	Reset value		0		0																								0	0	0	0	0
0x014C	Reserved	Reserved																															
0x0150	OCTOSPI_WPIR	INSTRUCTION[31:0]																															
	Reset value	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0x0154-0x015C	Reserved	Reserved																															
0x0160	OCTOSPI_WPABR	ALTERNATE[31:0]																															
	Reset value	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0x0164-0x017C	Reserved	Reserved																															
0x0180	OCTOSPI_WCCR	Res	Res	DQSE	Res	DDTR	DMODE [2:0]			Res	Res	ABSIZE [1:0]		ABDTR	ABMODE [2:0]		Res	Res	Res	Res	ADSIZE [1:0]	ADDTR	ADMODE [2:0]		Res	Res	Res	Res	ISIZE [1:0]	IDTR	IMODE [2:0]		
	Reset value	0		0		0	0	0	0			0	0	0	0	0	0				0	0	0	0	0	0			0	0	0	0	0
0x0184	Reserved	Reserved																															
0x0188	OCTOSPI_WTCR	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	DCYC[4:0]				
	Reset value																												0	0	0	0	0
0x018C	Reserved	Reserved																															
0x0190	OCTOSPI_WIR	INSTRUCTION[31:0]																															
	Reset value	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0x0194-0x019C	Reserved	Reserved																															
0x01A0	OCTOSPI_WABR	ALTERNATE[31:0]																															
	Reset value	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0x01A4-0x01FC	Reserved	Reserved																															
0x0200	OCTOSPI_HLCR	Res	Res	Res	Res	Res	Res	Res	Res	TRWR[7:0]					TACC[7:0]							Res	Res	Res	Res	Res	Res	Res	WZL	M			
	Reset value									0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0							0

Refer to [Section 2.3: Memory organization](#) for the register boundary addresses.

25 Octo-SPI interface (OCTOSPI)

This section applies only to STM32H543/553xx devices.

25.1 OCTOSPI introduction

The OCTOSPI supports most external serial memories such as serial PSRAMs, serial NAND and serial NOR flash memories, HyperRAM™ and HyperFlash™ memories, with the following functional modes:

- indirect mode: all the operations are performed using the OCTOSPI registers to preset commands, addresses, data, and transfer parameters.
- automatic status-polling mode: the external memory status register is periodically read and an interrupt can be generated in case of flag setting. This feature is only available in regular-command protocol.
- memory-mapped mode: the external memory is memory mapped and it is seen by the system as if it was an internal memory, supporting both read and write operations.

The OCTOSPI supports the following protocols with associated frame formats:

- the regular-command frame format with the command, address, alternate byte, dummy cycles, and data phase
- the HyperBus™ frame format

25.2 OCTOSPI main features

- Functional modes: indirect, automatic status-polling, and memory-mapped
- Read and write support in memory-mapped mode
- External (P)SRAM memory support
- Support for single, dual, quad, and octal communication
- Dual memory configuration, where eight bits can be sent/received simultaneously by accessing two quad memories in parallel
- SDR (single-data rate) and DTR (double-transfer rate) support
- Data strobe support
- Fully programmable opcode
- Fully programmable frame format
- Support wrapped-type access to memory in read direction
- HyperBus support
- Integrated FIFO for reception and transmission
- Asynchronous bus clock versus kernel clock support
- 8-, 16-, and 32-bit data accesses allowed
- DMA protocol support
- DMA channel for indirect mode operations
- Interrupt generation on FIFO threshold, timeout, operation complete, and access error
- AHB interface with transaction acceptance limited to one: the interface accepts the next transfer on AHB bus only once the previous is completed on memory side.

- Dual chip select support
- Extended external memory support: if two same size external memories (extmem1 and extmem2) are connected to the same I/O port, contiguously in the memory map and driven by a single OCTOSPI, this OCTOSPI automatically switches CS to extmem1 or extmem2, according to the address on interconnect side.
- Possibility to disable the automatic prefetch

25.3 OCTOSPI implementation

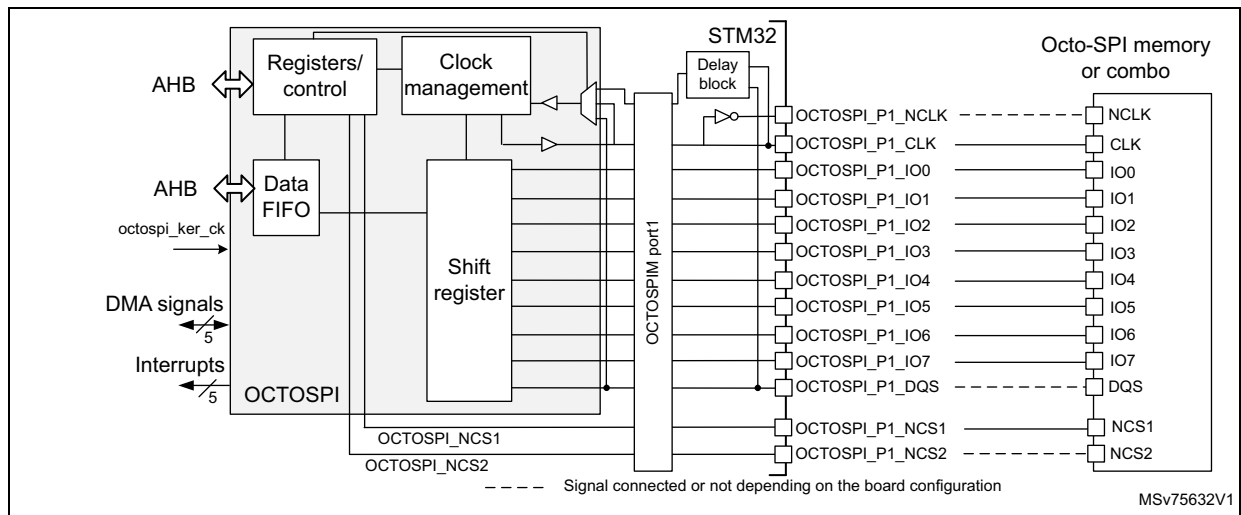
Table 240. OCTOSPI implementation

OCTOSPI feature	OCTOSPI1
HyperBus standard compliant	X
Xccela standard compliant	X
XSPI (JESD251C) standard compliant	X
AMBA® AHB compliant data interface	X
Dual AHB interface	X
Asynchronous AHB clock versus kernel clock	X
Functional modes: indirect, automatic status-polling, and memory-mapped	X
Read and write support in memory-mapped mode	X
Dual-quad configuration	X
SDR (single-data rate) and DTR (double-transfer rate)	X
Data strobe (DS,DQS)	X
Fully programmable opcode	X
Fully programmable frame format	X
Integrated FIFO for reception and transmission	X
8-, 16-, and 32-bit data accesses	X
Interrupt on FIFO threshold, timeout, operation complete, and access error	X
Bus error bit	X
Address offset	-
Extended CSHT timeout	X
Memory-mapped write	X
Refresh counter	X
GPDMA interface	X
Dual chip select	X
Extended external memory	X
Prefetch disable	X
Prefetch hardware software	-

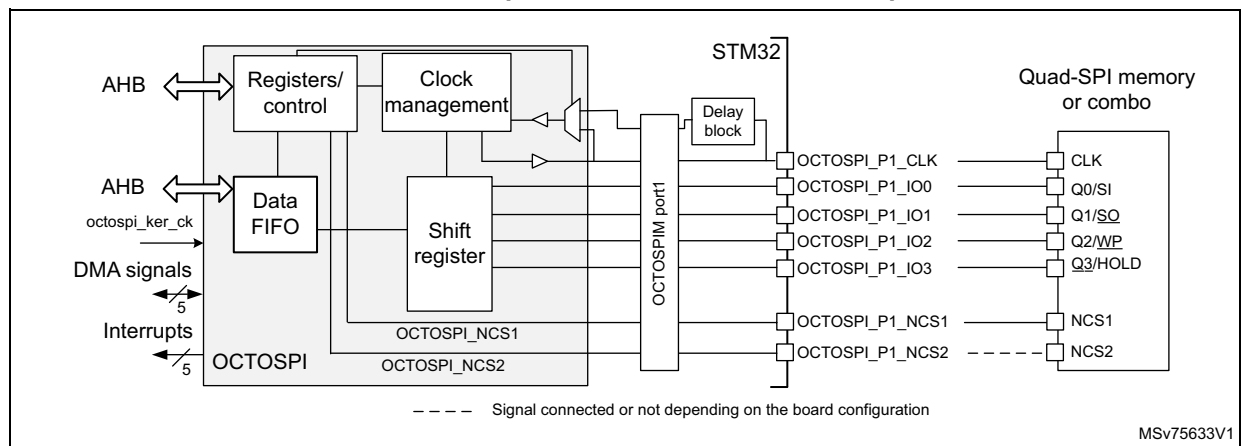
25.4 OCTOSPI functional description

25.4.1 OCTOSPI block diagram

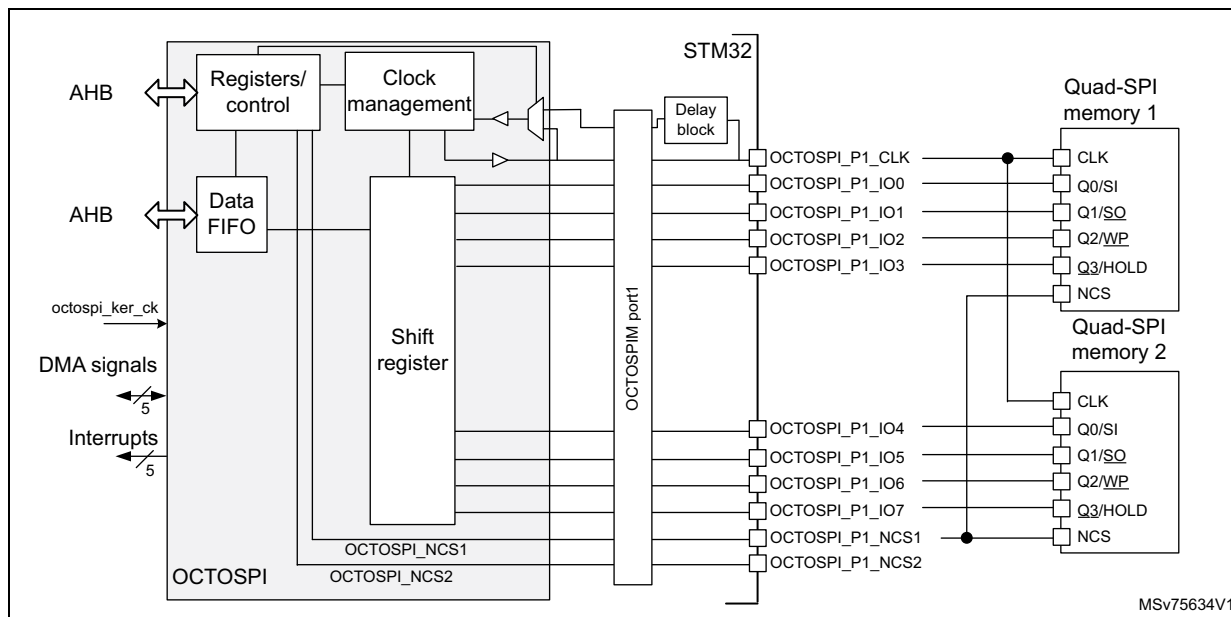
**Figure 179. OCTOSPI block diagram in octal configuration
(for STM32H543/553xx devices)**



**Figure 180. OCTOSPI block diagram in quad configuration
(for STM32H543/553xx devices)**



**Figure 181. OCTOSPI block diagram in dual-quad configuration
(for STM32H543/553xx devices)**



25.4.2 OCTOSPI pins and internal signals

Table 241. OCTOSPI input/output pins

Pin name	Type	Description
OCTOSPI_NCLK	Output	OCTOSPI inverted clock to support 1.8 V HyperBus protocol
OCTOSPI_CLK		OCTOSPI clock
OCTOSPI_IOn (n = 0 to 7)	Input/output	OCTOSPI data pins
OCTOSPI_NCS1,2	Output	Chip select for the memory
OCTOSPI_DQS	Input/output	Data strobe/write mask signal from/to the memory

Caution: Use the same configuration (output speed, HSLV^(a)) for all OCTOSPI input/output pins to avoid any data corruption.

Table 242. OCTOSPI internal signals

Signal name	Type	Description
octospi_hclk	Input	OCTOSPI AHB clock
octospi_ker_ck	Input	OCTOSPI kernel clock
octospi_dma	N/A	DMA request signal
octospi_it	Output	Global interrupt line (see Table 246 for the multiple sources of interrupt)

a. When applicable.

25.4.3 OCTOSPI interface to memory modes

The OCTOSPI supports the following protocols:

- regular-command protocol
- HyperBus protocol

The OCTOSPI uses from 6 to 12 signals to interface with a memory, depending on the functional mode:

- NCS: chip-select (in the following sections, unless otherwise specified, NCS applies for NCS1 or NCS2. Only one of these two signals is active at a given moment in time)
- CLK: communication clock
- NCLK: inverted clock used only in the 1.8 V HyperBus protocol
- DQS: data strobe used only in regular-command protocol as input only
- IO[3:0]: data bus LSB
- IO[7:4]:
 - data bus MSB used in dual-quad and octal configurations
 - data bus can be used as possible remap for quad-SPI mode

25.4.4 OCTOSPI regular-command protocol

When in regular-command protocol, the OCTOSPI communicates with the external device using commands. Each command can include the following phases:

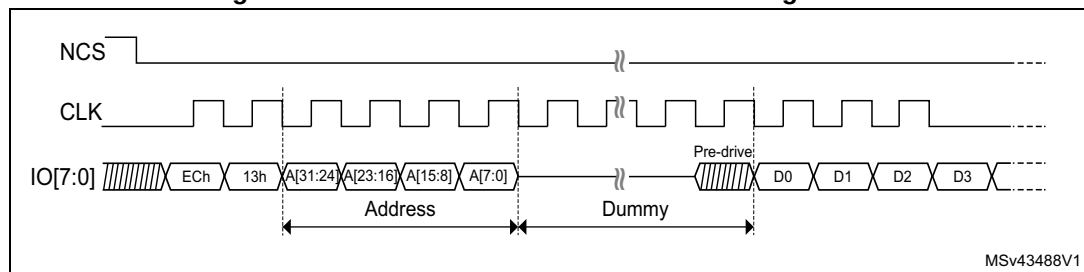
- Instruction phase
- Address phase
- Alternate-byte phase
- Dummy-cycle phase
- Data phase

Any of these phases can be configured to be skipped but, in case of single-phase command, the only use case supported is instruction-phase-only.

The NCS falls before the start of each command and rises again after each command finishes.

In memory-mapped mode, both read and write operations are supported: as a consequence, some of the configuration registers are duplicated to specify write operations (read operations are configured using regular registers).

Figure 182. SDR read command in octal configuration



The specific regular-command protocol features are configured through the registers in the 0x0100-0x01FC offset range.

Instruction phase

During this phase, a 1- to 4-byte instruction is sent to the external device specifying the type of operation to be performed. The size of the instruction to be sent is configured by ISIZE[1:0] in OCTOSPI_CCR and the instruction is programmed in INSTRUCTION[31:0] of OCTOSPI_IR.

The instruction phase can optionally send:

- 1 bit at a time (over IO0, SO single in single-SPI mode)
- 2 bits at a time (over IO0/IO1 in dual-SPI mode)
- 4 bits at a time (over IO0 to IO3 in quad-SPI mode)
- 8 bits at a time (over IO0 to IO7 in octal-SPI mode)

This can be configured using IMODE[2:0] of OCTOSPI_CCR.

The instruction can be sent in DTR mode on each rising and falling edge of the clock, by setting IDTR in OCTOSPI_CCR.

When IMODE[2:0] = 000 in OCTOSPI_CCR, the instruction phase is skipped, and the command sequence starts with the address phase, if present.

In memory-mapped mode, the instruction used for the write operation is specified in OCTOSPI_WIR, and the instruction format is specified in OCTOSPI_WCCR. The instruction used for the read operation and the instruction format are specified in OCTOSPI_IR and OCTOSPI_CCR.

Address phase

In the address phase, 1 to 4 bytes are sent to the external device, to indicate the address of the operation. The number of address bytes to be sent is configured by ADSIZE[1:0] in OCTOSPI_CCR.

In indirect and automatic status-polling modes, the address bytes to be sent are specified by ADDRESS[31:0] in OCTOSPI_AR. In memory-mapped mode, the address is given directly via the AHB (from any master in the system).

The address phase can send:

- 1 bit at a time (over IO0, SO single in single-SPI mode)
- 2 bits at a time (over IO0/IO1 in dual-SPI mode)
- 4 bits at a time (over IO0 to IO3 in quad-SPI mode)
- 8 bits at a time (over IO0 to IO7 in octal-SPI mode)

This can be configured using ADMODE[2:0] in OCTOSPI_CCR.

The address can be sent in DTR mode (on each rising and falling edge of the clock) setting ADDTR in OCTOSPI_CCR.

When ADMODE[2:0] = 000, the address phase is skipped and the command sequence proceeds directly to the next phase, if any.

In memory-mapped mode, the address format for the write operation is specified in OCTOSPI_WCCR. The address format for the read operation is specified in OCTOSPI_CCR.

Alternate-byte phase

In the alternate-byte phase, 1 to 4 bytes are sent to the external device, generally to control the mode of operation. The number of alternate bytes to be sent is configured by ABSIZE[1:0] in OCTOSPI_CCR. The bytes to be sent are specified in OCTOSPI_ABR.

The alternate-byte phase can send:

- 1 bit at a time (over IO0, SO single in single-SPI mode)
- 2 bits at a time (over IO0/IO1 in dual-SPI mode)
- 4 bits at a time (over IO0 to IO3 in quad-SPI mode)
- 8 bits at a time (over IO0 to IO7 in octal-SPI mode)

This can be configured using ABMODE[2:0] in OCTOSPI_CCR.

The alternate bytes can be sent in DTR mode (on each rising and falling edge of the clock) setting ABDTR in OCTOSPI_CCR.

When ABMODE[2:0] = 000, the alternate-byte phase is skipped and the command sequence proceeds directly to the next phase, if any.

There may be times when only a single nibble needs to be sent during the alternate-byte phase rather than a full byte, such as when the dual-SPI mode is used and only two cycles are used for the alternate bytes.

In this case, the firmware can use the quad-SPI mode (ABMODE[2:0] = 011) and send a byte with bits 7 and 3 of ALTERNATE[31:0] set to 1 (keeping the IO3 line high), and bits 6 and 2 set to 0 (keeping the IO2 line low), in OCTOSPI_IR.

The upper two bits of the nibble to be sent are then placed in bits 5:4 of ALTERNATE[31:0] while the lower two bits are placed in bits 1:0. For example, if the nibble 2 (0010) is to be sent over IO0/IO1, then ALTERNATE[31:0] must be set to 0x8A (1000_1010).

In memory-mapped mode, the alternate bytes used for the write operation are specified in OCTOSPI_WABR, and the alternate byte format is specified in OCTOSPI_WCCR. The alternate bytes used for read operation and the alternate byte format are specified in OCTOSPI_ABR and OCTOSPI_CCR.

Dummy-cycle phase (memory latency)

In the dummy-cycle phase, 1 to 31 cycles are given without any data being sent or received, in order to give the external device, the time to prepare for the data phase when higher clock frequencies are used. The number of cycles given during this phase is specified by DCYC[4:0] in OCTOSPI_TCR. In both SDR and DTR modes, the duration is specified as a number of full CLK cycles.

When DCYC[4:0] = 00000, the dummy-cycle phase is skipped, and the command sequence proceeds directly to the data phase, if present.

In order to assure enough “turn-around” time for changing the data signals from the output mode to the input mode, there must be at least one dummy cycle when using the dual-SPI, the quad-SPI, or the octal-SPI mode, to receive data from the external device.

In memory-mapped mode, the dummy cycles for the write operations are specified in OCTOSPI_WTCR. The dummy cycles for the read operation are specified in OCTOSPI_TCR.

Data phase

During the data phase, any number of bytes can be sent to or received from the external device.

In indirect mode, the number of bytes to be sent/received is specified in OCTOSPI_DLR. In this mode, the data to be sent to the external device must be written to OCTOSPI_DR, while in indirect-read mode the data received from the external device is obtained by reading OCTOSPI_DR.

In automatic status-polling mode, the number of bytes to be received is specified in OCTOSPI_DLR, and the data received from the external device can be obtained by reading OCTOSPI_DR.

In memory-mapped mode, the data read or written, is sent or received directly over the AHB to the Cortex core or to a DMA.

The data phase can send/receive:

- 1 bit at a time (over IO0/IO1 (SO/SI respectively) in single-SPI mode)
- 2 bits at a time (over IO0/IO1 in dual-SPI mode)
- 4 bits at a time (over IO0 to IO3 in quad-SPI mode)
- 8 bits at a time (over IO0 to IO7 in octal-SPI mode)

This can be configured using DMODE[2:0] in OCTOSPI_CCR.

The data can be sent or received in DTR mode (on each rising and falling edge of the clock) setting DDTR in OCTOSPI_CCR.

When DMODE[2:0] = 000, the data phase is skipped, and the command sequence finishes immediately by raising the NCS. This configuration must be used only in indirect-write mode.

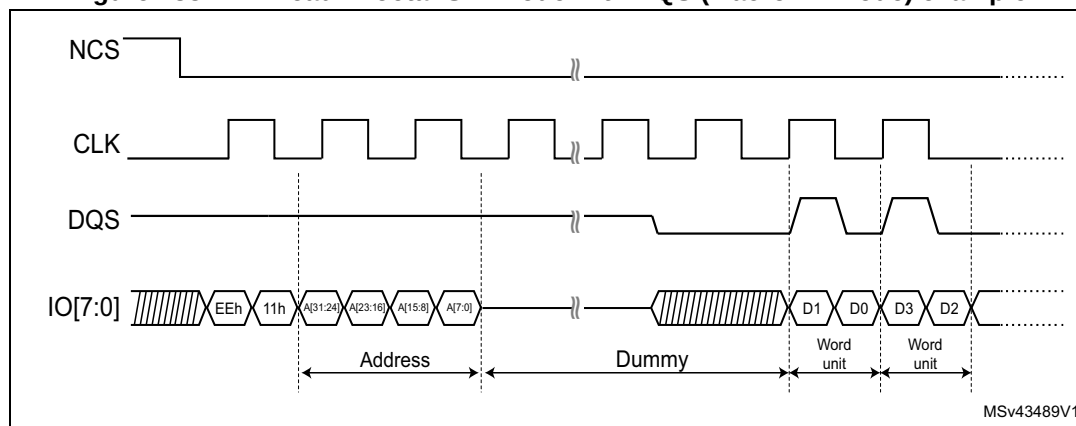
In memory-mapped mode, the data format for the write operation is specified in OCTOSPI_WCCR. The data format for the read operation is specified in OCTOSPI_CCR.

DQS use

The DQS signal can be used for data strobing during the read transactions when the device toggles the DQS aligned with the data.

The DQS management can be enabled by setting DQSE in OCTOSPI_CCR.

Figure 183. DTR read in octal-SPI mode with DQS (Macronix mode) example



25.4.5 OCTOSPI regular-command protocol signal interface

Single-SPI mode

The legacy SPI mode allows just a single bit to be sent/received serially. In this mode, the data is sent to the external device over the SO signal (Single-SPI Output) (whose I/Os are shared with IO0). The data received from the external device arrives via SI (Single-SPI Input) (whose I/Os are shared with IO1).

Compared to the SPI legacy mode, IO/SO and I1/SI are respectively equivalent to MOSI and MISO, having the OCTOSPI generating the clock.

The different phases can each be configured separately to use this single-SPI mode by setting to 001 the IMODE, ADMODE, ABMODE, and DMODE fields in OCTOSPI_CCR and OCTOSPI_WCCR.

In each phase configured in single-SPI mode:

- IO0 (SO) is in output mode.
- IO1 (SI) is in input mode (high impedance).
- IO2 is in output mode and forced to 0.
- IO3 is in output mode and forced to 1 (to deactivate the “hold” function).
- IO4 to IO7 are in output mode and forced to 0.

This is the case even for the dummy phase if DMODE[2:0] = 001.

Dual-SPI mode

In dual-SPI mode, two bits are sent/received simultaneously over the IO0/IO1 signals.

The different phases can each be configured separately to use dual-SPI mode by setting to 010 the IMODE, ADMODE, ABMODE, and DMODE fields in OCTOSPI_CCR and OCTOSPI_WCCR.

In each phase configured in dual-SPI mode:

- IO0/IO1 are at high-impedance (input) during the data phase for the read operations, and outputs in all other cases.
- IO2 is in output mode and forced to 0.
- IO3 is in output mode and forced to 1.
- IO4 to IO7 are in output mode and forced to 0.

In the dummy phase, when DMODE[2:0] = 010, IO0 and IO1 are in a high-impedance state during read transactions, and are forced to either high or low levels during write transactions.

Quad-SPI mode

In quad-SPI mode, four bits are sent/received simultaneously over the IO0/IO1/IO2/IO3 signals.

The different phases can each be configured separately to use the quad-SPI mode by setting to 011 the IMODE, ADMODE, ABMODE, and DMODE fields in OCTOSPI_CCR and OCTOSPI_WCCR.

In each phase configured in quad-SPI mode:

- IO0 to IO3 are all at high-impedance (inputs) during the data phase for the read operations, and outputs in all other cases.
- IO4 to IO7 are in output mode and forced to 0.

In the dummy phase, when $\text{DMODE}[2:0] = 011$, IO0 to IO3 are in a high-impedance state during read transactions, and are forced to either high or low levels during write transactions.

Octal-SPI mode

In regular octal-SPI mode, the eight bits are sent/received simultaneously over the IO[0:7] signals.

The different phases can each be configured separately to use the octal-SPI mode by setting to 100 the IMODE, ADMODE, ABMODE, and DMODE fields in OCTOSPI_CCR and OCTOSPI_WCCR.

In each phase that is configured in octal-SPI mode, IO[0:7] are all at high-impedance (input) during the data phase for read operations, and outputs in all other cases.

In the dummy phase, when $\text{DMODE}[2:0] = 100$, IO[0:7] are in a high-impedance state during read transactions, and are forced to either high or low levels during write transactions.

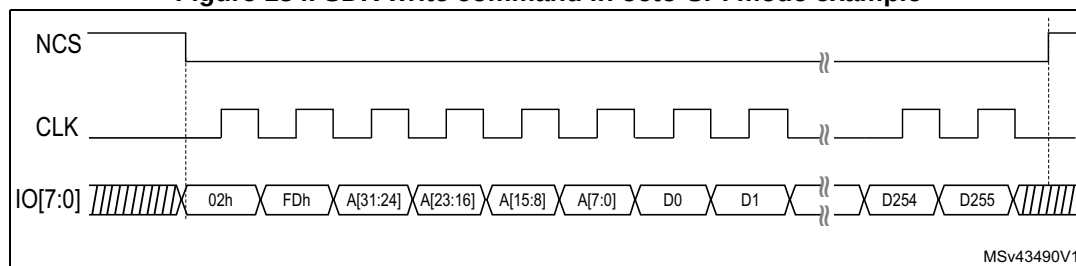
Single-data rate (SDR) mode

By default, all the phases operate in SDR mode.

In this mode, when the OCTOSPI drives the IO0/SO, IO1 to IO7 signals, these signals transition only with the falling edge of CLK.

When receiving data in SDR mode, the OCTOSPI assumes that the external devices also send the data using CLK falling edge. By default (when $\text{SSHIFT} = 0$ in OCTOSPI_TCR), the signals are sampled using the following (rising) edge of CLK.

Figure 184. SDR write command in octo-SPI mode example



Note: Due to internal synchronization, up to six extra dummy clock cycles may be generated by the Octo-SPI interface after the last data is read.

Double-transfer rate (DTR) mode

Each of the instruction, address, alternate-byte, and data phases can be configured to operate in DTR mode setting IDTR, ADDTR, ABDTR, and DDTR in OCTOSPI_CCR.

In memory-mapped mode, the DTR mode for each phase of the write operations is specified in OCTOSPI_WCCR. The DTR mode for each phase of the read operations is specified in OCTOSPI_CCR.

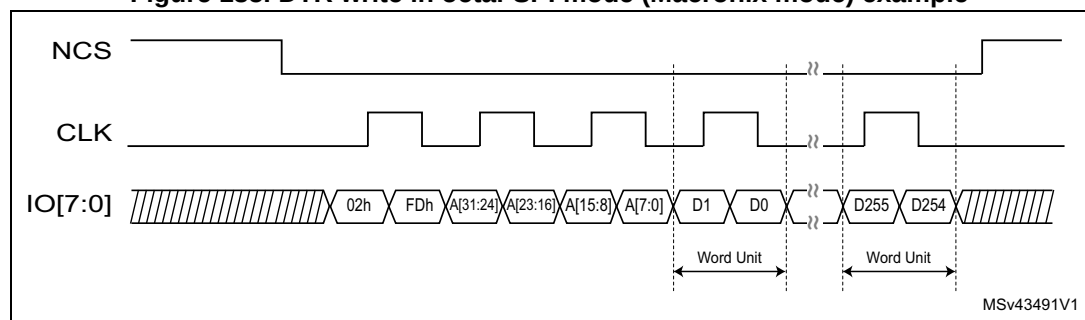
In DTR mode, when the OCTOSPI drives the IO0/SO and IO1 to IO7 signals in the instruction, address, and alternate-byte phases, a bit is sent or received on each of the falling and rising edges of CLK.

When receiving data in DTR mode, the OCTOSPI assumes that the external devices also send the data using both CLK rising and falling edges. When DDTR = 1 in OCTOSPI_CCR, the software must clear SSHIFT in OCTOSPI_TCR. Thus, the signals are sampled one half of a CLK cycle later (on the following, opposite edge).

In DTR mode, it is recommended to set DHQC of OCTOSPI_TCR, to shift the outputs by a quarter of cycle and avoid holding issues on the memory side.

Note: *DHQC must not be set when the prescaler value is 0, as this action leads to unpredictable behavior.*

Figure 185. DTR write in octal-SPI mode (Macronix mode) example



Note: *Due to internal synchronization, up to six extra dummy clock cycles may be generated by the Octo-SPI interface after the last data is read.*

Dual-quad configuration

When DMM = 1 in OCTOSPI_CR, the OCTOSPI is in dual-memory configuration: if DMODE = 011, two external Quad-SPI devices (device A and device B) are used in order to send/receive eight bits (or 16 bits in DTR mode) every cycle, effectively doubling the throughput.

Each device (A or B) uses the same CLK and NCS signals, but each has separate IO0 to IO3 signals.

The dual-quad configuration can be used in conjunction with the single-SPI, dual-SPI, and quad-SPI modes, as well as with either the SDR or DTR mode.

The device size, as specified by DEVSZ[4:0] in OCTOSPI_DCR1, must reflect the total external device capacity that is the double of the size of one individual component.

If address X is even, then the byte that the OCTOSPI gives for address X is the byte at the address X/2 of device A, and the byte that the OCTOSPI gives for address X + 1 is the byte at the address X/2 of device B. In other words, the bytes at even addresses are all stored in device A and the bytes at odd addresses are all stored in device B.

When reading the status registers of the devices in dual-quad configuration, twice as many bytes must be read compared to the same read in regular-command protocol: if each device gives eight valid bits after the instruction for fetching the status register, then the OCTOSPI must be configured with a data length of 2 bytes (16 bits), and the OCTOSPI receives one byte from each device.

If each device gives a status of 16 bits, then the OCTOSPI must be configured to read 4 bytes to get all the status bits of both devices in dual-quad configuration. The least-significant byte of the result (in the data register) is the least-significant byte of device A status register. The next byte is the least-significant byte of device B status register. Then, the third byte of the data register is the device A second byte. The fourth byte is the device B second byte (if devices have 16-bit status registers).

An even number of bytes must always be accessed in dual-quad configuration. For this reason, bit 0 of DL[31:0] in OCTOSPI_DLR is stuck at 1 when DMM = 1.

In dual-quad configuration, the behavior of device A interface signals is basically the same as in normal mode. Device B interface signals have exactly the same waveforms as device A ones during the instruction, address, alternate-byte, and dummy-cycle phases. In other words, each device always receives the same instruction and the same address.

Then, during the data phase, the AIOx and the BIOx buses both transfer data in parallel, but the data that is sent to (or received from) device A is distinct than the one from device B.

25.4.6 HyperBus protocol

The OCTOSPI can communicate with the external device using the HyperBus protocol.

The HyperBus uses 11 to 12 pins depending on the operating voltage:

- IO[7:0] as bidirectional data bus
- DQS for read and write data strobe and latency insertion
- NCS
- CLK
- NCLK for 1.8 V operations (to support this mode, the device must be powered with 1.8 V)

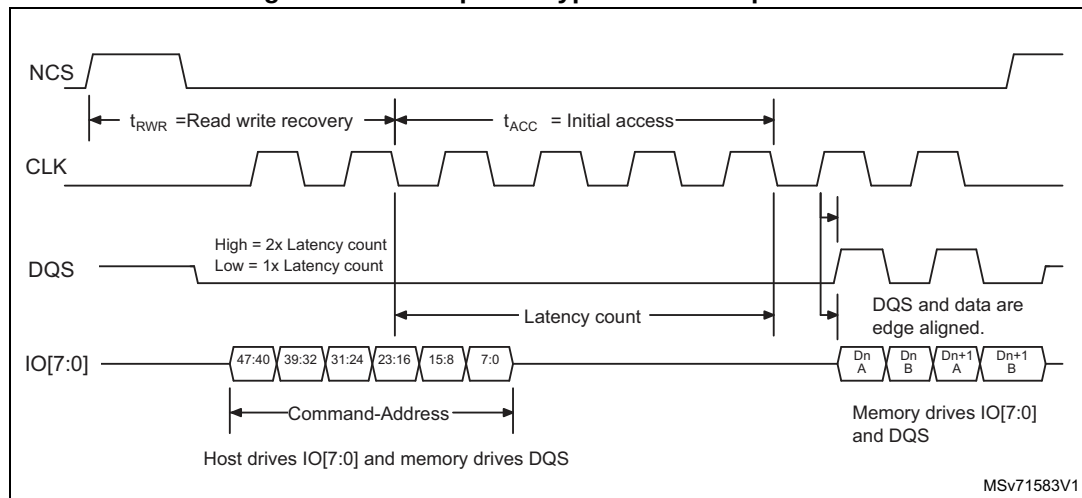
The HyperBus does not require any command specification nor any alternate bytes. As a consequence, a separate register set is used to define the timing of the transaction.

The HyperBus frame is composed of the following phases:

- Command/address phase
- Data phase

The NCS falls before the start of a transaction and rises again after each transaction finishes.

Figure 186. Example of HyperBus read operation



Note: Due to internal synchronization, up to six extra dummy clock cycles may be generated by the Octo-SPI interface after the last data is read.

The specific HyperBus features are configured through the registers in the 0x0200-0x02FC offset range.

Command/address phase

During this initial phase, the OCTOSPI sends 48 bits over IO[7:0] to specify the operations to be performed with the external device.

Table 243. Command/address phase description

CA bit	Bit name	Description
47	R/W#	Identifies the transaction as a read or a write.
46	Address space	Indicates if the transaction accesses the memory or the register space.
45	Burst type	Indicates if the burst is linear or wrapped.
44-16	Row and upper column address	Selects the row and the upper column addresses.
15-3	Reserved	-
2-0	Lower column address	Selects the starting 16-bit word within the half page.

The address space is configured through the memory type MTYP[2:0] in OCTOSPI_DCR1.

The total size of the device must be configured through DEVSIZ[4:0] of OCTOSPI_DCR1. In case of multi-chip product (MCP), the device size is the sum of all the sizes of all the MCP dies.

Read/write operation with initial latency

The HyperBus read and write operations need to respect two timings:

- t_{RWR} : minimal read/write recovery time for the device (defined by TRWR[7:0] in OCTOSPI_HLCR)
- t_{ACC} : access time for the device (defined by TACC[7:0] in OCTOSPI_HLCR) according to the memory latency

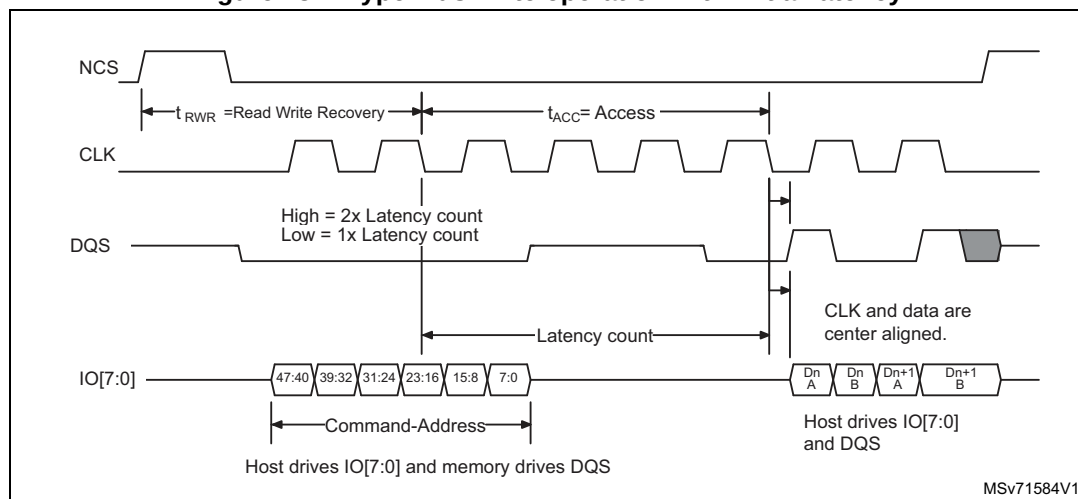
During the read operation, the DQS is used by the device, in two ways (see [Figure 186](#)):

- during the command/address phase, to request an additional latency
- during the data phase, for data strobing

During the write operation, the DQS is used:

- by the device, during the command/address phase, to request an additional latency.
- by the OCTOSPI, during the data phase, for write data masking.

Figure 187. HyperBus write operation with initial latency



Read/write operation with additional latency

If the device needs an additional latency (during refresh period of an SDRAM for example), DQS must be tied to one during one of the DQS signals, during the command/address phase.

An additional t_{ACC} duration is added by the OCTOSPI to meet the device request.

Figure 188. HyperBus read operation with additional latency

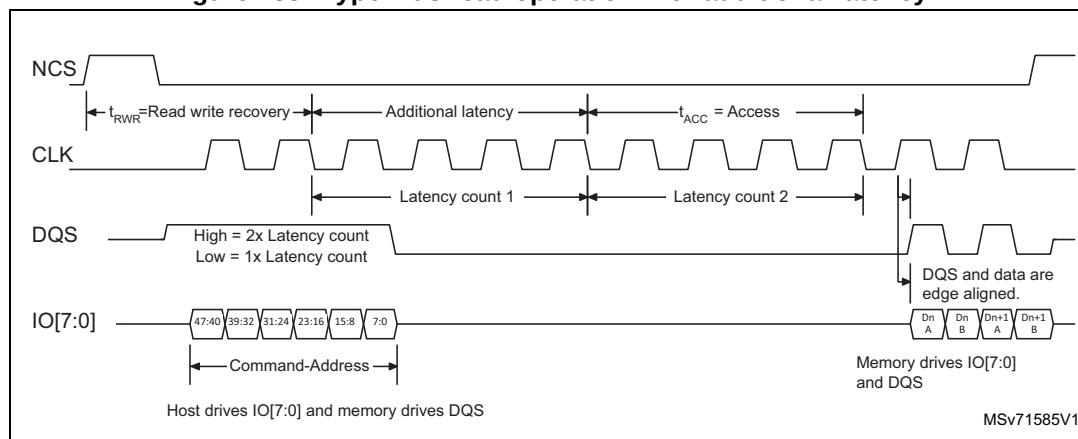
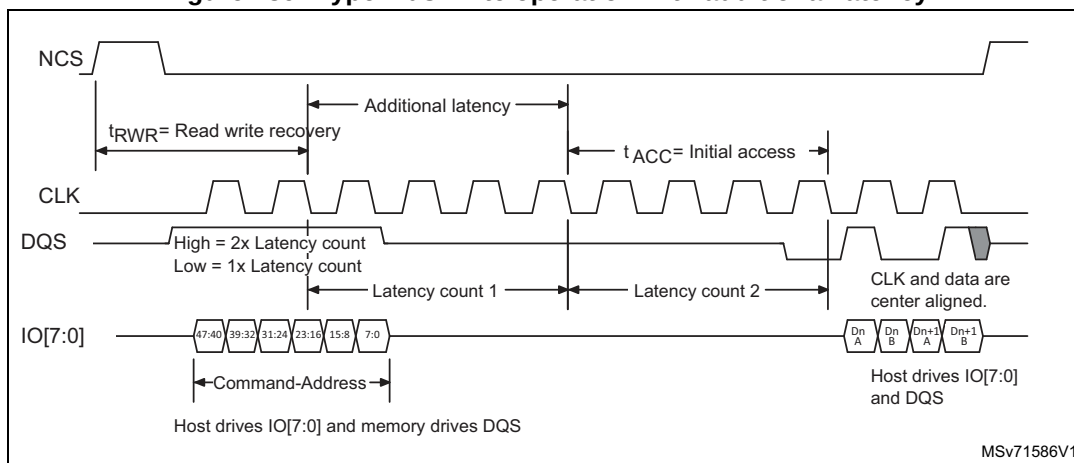


Figure 189. HyperBus write operation with additional latency



Fixed-latency mode

Some devices or some applications may not want to operate with a variable latency time as described above.

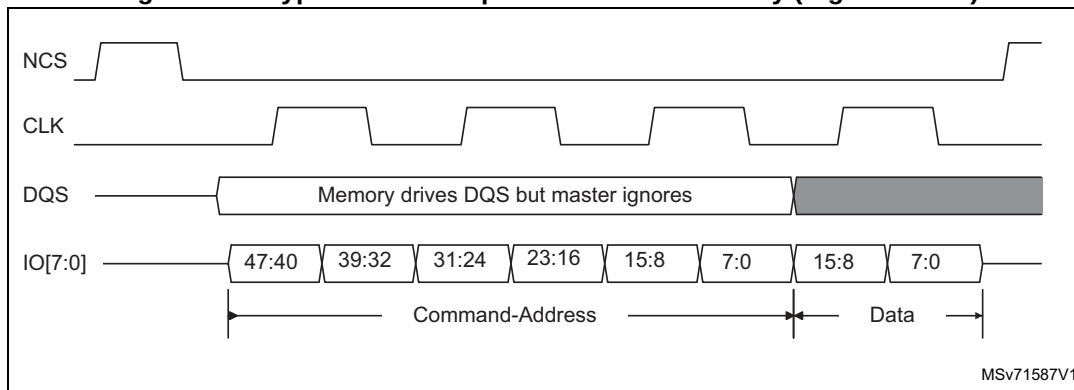
The latency can be forced to $2 \times t_{ACC}$ by setting LM in OCTOSPI_HLCR.

In this OCTOSPI latency mode, the state of the DQS signal is not taken into account by the OCTOSPI, and an additional latency is always added, leading to a fixed $2 \times t_{ACC}$ latency time.

Write operation with no latency

Some devices can also require a zero latency for the write operations. This write-zero latency can be forced by setting WZL in OCTOSPI_HLCR.

Figure 190. HyperBus write operation with no latency (register write)

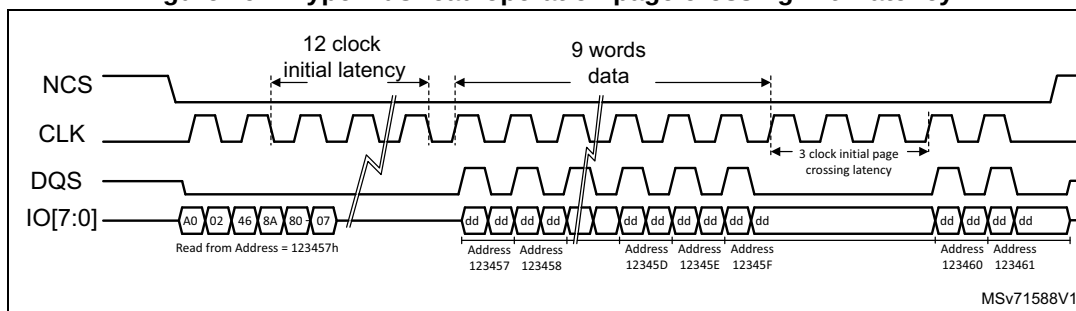


Latency on page-crossing during the read operations

An additional latency can be needed by some devices for the read operation when crossing pages.

The initial latency must be respected for any page access, as a consequence, when the first access is close to the page boundary, a latency is automatically added at the page crossing to respect the t_{ACC} time.

Figure 191. HyperBus read operation page crossing with latency



25.4.7 Specific features

The OCTOSPI supports some specific features, such as:

- Wrap support
- NCS boundary and refresh

Wrap support

The OCTOSPI supports a hybrid wrap as defined by the HyperBus protocol. A hybrid wrap is also supported in the regular-command protocol.

In hybrid wrap, the transaction can continue after the initial wrap with an incremental access.

The wrap size supported by the target memory is configured by WRAPSIZE in OCTOSPI_DCR2.

Wrap is supported only in memory-read direction and only for data size = 4 bytes. Wrapped reads are supported for both HyperBus and regular-command protocols. To enable wrapped-read accesses, the dedicated OCTOSPI_WPxxx registers must be programmed according to the wrapped-read access characteristics. These registers apply for both HyperBus and regular-command protocols.

If the target memory is not supporting the hybrid wrap, WRAPSIZE must be set to 0.

Note: Hybrid wrap requires that the nonwrapped registers (OCTOSPI_CCR, OCTOSPI_TCR, OCTOSPI_IR) are set according to the memory configuration to satisfy its correct data prefetch (initiated after the wrap command).

The wrap operation cannot be interrupted by a refresh. The refresh event is only considered after the wrap completion.

NCS boundary and refresh

Two processes can be activated to regulate the OCTOSPI transactions:

- NCS boundary
- Refresh

The NCS boundary feature limits a transaction to a boundary of aligned addresses. The size of the address to be aligned with, is configured by CSBOUND[4:0] in OCTOSPI_DCR3: it is equal to $2^{CSBOUND}$.

As an example, if CSBOUND[4:0] = 0x4, the boundary is set to $2^4 = 16$ bytes. The NCS is then released each time the LSB address is equal to 0xF, and each time a new transaction is issued to address the next data.

If CSBOUND[4:0] = 0, the feature is disabled. A minimum value of three is recommended.

The NCS boundary feature cannot be used for flash memory devices in write mode since a command is necessary to program another page of the flash memory.

The refresh feature limits the duration of the transactions to the value programmed by REFRESH[31:0] in OCTOSPI_DCR4. The duration is expressed in number of cycles. This allows an external RAM to perform its internal refresh operation regularly.

The refresh value must be greater than the minimal transaction size in terms of number of cycles including the command/address/alternate/dummy phases.

If NCS boundary and refresh are enabled at the same time, the NCS is released on the first condition met.

Restarting after an interrupted transfer

When a read or write operation is interrupted by a timeout or communication regulation feature, the Octo-SPI interface, as soon as possible after getting back the port ownership, reissues the initial command sequence together with the address following the last address actually accessed before interruption. The transfer initially set goes on and ends seamlessly.

25.4.8 OCTOSPI operating mode introduction

The OCTOSPI has the following operating modes regardless of the low-level protocol used (either regular-command or HyperBus):

- indirect mode (read or write)
- automatic status-polling mode (only in regular-command protocol)
- memory-mapped mode

25.4.9 OCTOSPI indirect mode

In indirect mode, the commands are started by writing to the OCTOSPI registers, and data are transferred by writing or reading the data register, in a similar way to other communication peripherals.

When FMODE[1:0] = 00 in OCTOSPI_CR, the OCTOSPI is in indirect-write mode: bytes are sent to the external device during the data phase. Data are provided by writing to OCTOSPI_DR.

When FMODE[1:0] = 01, the OCTOSPI is in indirect-read mode: bytes are received from the external device during the data phase. Data are recovered by reading OCTOSPI_DR.

In indirect mode, when the OCTOSPI is configured in DTR mode over eight lanes with DQS disabled, the given starting address and the data length must be even.

Note: The OCTOSPI_AR register must be updated even if the start address is the same as the start address of the previous indirect access.

The number of bytes to be read/written is specified in OCTOSPI_DLR:

- If DL[31:0] = 0xFFFF FFFF, the data length is considered undefined and the OCTOSPI simply continues to transfer data until it reaches the end of the external device (as

defined by DEVSIZE). If no bytes are to be transferred, DMODE[2:0] must be set to 0 in OCTOSPI_CCR.

- If DL[31:0] = 0xFFFF FFFF and DEVSIZE[4:0] = 0x1F (its maximum value indicating at 4-Gbyte device), the transfers continue indefinitely, stopping only after an abort request or after the OCTOSPI is disabled. After the last memory address is read (at address 0xFFFF FFFF), reading continues with address = 0x0000 0000.

When the programmed number of bytes to be transmitted or received is reached, the TCF bit is set in OCTOSPI_SR, and an interrupt is generated if TCIE = 1 in OCTOSPI_CR. In the case of an undefined number of data, TCF is set when the limit of the external SPI memory is reached, according to the device size defined in OCTOSPI_DCR1.

Triggering the start of a transfer in regular-command protocol

Depending on the OCTOSPI configuration, there are three different ways to trigger the start of a transfer in indirect mode when using the regular-command protocol. In general, the start of transfer is triggered as soon as the software gives the last information that is necessary for the command. More specifically in indirect mode, a transfer starts when one of the following sequence of events occurs:

- if no address is necessary (ADMODE[2:0] = 000) and if no data need to be provided by the software (FMODE[1:0] = 01 or DMODE[2:0] = 000), and at the moment when a write is performed to INSTRUCTION[31:0] in OCTOSPI_IR
- if an address is necessary (when ADMODE[2:0] ≠ 000) and if no data need to be provided by the software (when FMODE[1:0] = 01 or DMODE[2:0] = 000), and at the moment when a write is performed to ADDRESS[31:0] in OCTOSPI_AR
- if data need to be provided by the software (when FMODE[1:0] = 00 and DMODE[2:0] ≠ 000), and at the moment when a write is performed to DATA[31:0] in OCTOSPI_DR

A write to OCTOSPI_ABR never triggers the communication start. If alternate bytes are required, they must have been programmed before.

As soon as a command is started, the BUSY bit is automatically set in OCTOSPI_SR.

Triggering the start of a transfer in HyperBus protocol

Depending on the OCTOSPI configuration, there are different ways to trigger the start of a command in indirect mode. In general, it is triggered as soon as the firmware gives the last information that is necessary for the transfer to start, and more specifically, a communication in indirect mode is triggered by one of the following register settings, when it is the last one to be executed:

- when a write is performed to ADDRESS[31:0] (OCTOSPI_AR) with ADMODE[2:0] ≠ 000 in indirect read mode (FMODE[1:0] = 01).
- when a write is performed to DATA[31:0] (OCTOSPI_DR) in indirect-write mode (when FMODE = 00).
- when a (dummy) write is performed to INSTRUCTION[31:0] (OCTOSPI_IR) for indirect read mode (with ADMODE[2:0] = 000 and FMODE = 01).

As soon as a transfer is started, the BUSY bit (OCTOSPI_SR[5]) is automatically set.

FIFO and data management

Data in indirect mode passes through a 32-byte FIFO that is internal to the OCTOSPI. FLEVEL in OCTOSPI_SR indicates how many bytes are currently being held in the FIFO.

AHB burst transactions are supported. Data of the burst are successively written in OCTOSPI_DR, and immediately transferred in the internal FIFO.

In indirect-write mode (FMODE[1:0] = 00), the software adds data to the FIFO when it writes in OCTOSPI_DR. A word write adds 4 bytes to the FIFO, a half-word write adds 2 bytes, and a byte write adds only 1 byte. If the software adds too many bytes to the FIFO (more than indicated in DL[31:0]), the extra bytes are flushed from the FIFO at the end of the write operation (when TCF is set).

The byte/half-word accesses to OCTOSPI_DR must be done only to the least significant byte/halfword of the 32-bit register.

FTHRES is used to define a FIFO threshold after which point the FIFO threshold flag, FTF, gets set. In indirect-read mode, FTF is set when the number of valid bytes to be read from the FIFO is above the threshold. FTF is also set if there is any data left in the FIFO after the last byte is read from the external device, regardless of FTHRES setting. In indirect-write mode, the FTF is set when the number of empty bytes in the FIFO is above the threshold.

If FTIE = 1, there is an interrupt when the FTF is set. If DMAEN = 1, a DMA transfer is initiated when the FTF is set. The FTF is cleared by hardware as soon as the threshold condition is no longer true (after enough data has been transferred by the CPU or DMA).

The last data read in RX FIFO remains valid as long as there is no request for the next line. This means that, when the application reads several times in a row at the same location, the data is provided from the RX FIFO and not read again from the distant memory.

25.4.10 OCTOSPI automatic status-polling mode

In automatic status-polling mode, the OCTOSPI periodically starts a command to read a defined number of status bytes (up to four). The received bytes can be masked to isolate some status bits and an interrupt can be generated when the selected bits have a defined value. The automatic status-polling mode must be used only in regular-command protocol. For HyperBus protocol, it is not exploitable since the read status register into the HyperFlash memory must be performed in two steps (a write operation followed by a read operation).

The access to the device begins in the same manner as in indirect-read mode. BUSY in OCTOSPI_SR goes high at this point and stays high even between the periodic accesses.

The content of MASK[31:0] in OCTOSPI_PSMAR is used to mask the data from the external device in automatic status-polling mode:

- If the MASK[n] = 0, then bit n of the result is masked and not considered.
- If MASK[n] = 1, and the content of bit[n] is the same as MATCH[n] in OCTOSPI_PSMAR, then there is a match for bit n.

If PMM = 0 in OCTOSPI_CR, the AND-match mode is activated: SMF is set in OCTOSPI_SR only when there is a match on all of the unmasked bits.

If PMM = 1 in OCTOSPI_CR, the OR-match mode is activated: SMF gets set if there is a match on any of the unmasked bits.

An interrupt is called when SMF = 1 if SMIE = 1.

If APMS is set in OCTOSPI_CR, the operation stops and BUSY goes to 0 as soon as a match is detected. Otherwise, BUSY stays at 1 and the periodic accesses continue until there is an abort or until the OCTOSPI is disabled (EN = 0).

OCTOSPI_DR contains the latest received status bytes (FIFO deactivated). The content of this register is not affected by the masking used in the matching logic. FTF in OCTOSPI_SR is set as soon as a new reading of the status is complete. FTF is cleared as soon as the data is read.

In automatic status-polling mode, variable latency is not supported. The memory must then be configured in fixed latency.

25.4.11 OCTOSPI memory-mapped mode

When configured in memory-mapped mode, the external SPI device is seen as an internal memory.

Note: No more than 256 Mbytes can be addressed even if the external device capacity is larger.

If an access is made to an address outside of the range defined by DEVSIZ[4:0] but still within the 256 Mbytes range, then an AHB error is given. The effect of this error depends on the AHB master that attempted the access:

- If it is the Cortex CPU, a hard-fault interrupt is generated.
- If it is a DMA, a DMA transfer error is generated, and the corresponding DMA channel is automatically disabled.

Byte, half-word, and word access types are all supported.

A support for execute in place (XIP) operation is implemented, where the OCTOSPI continues to load the bytes to the addresses following the most recent access. If subsequent accesses are continuous to the bytes that follow, then these operations end up quickly since their results were prefetched.

By default, the OCTOSPI never stops its prefetch operation. It either keeps the previous read operation active with the NCS maintained low or it relaunches a new transfer, even if no access to the external device occurs for a long time.

Since external devices tend to consume more power when the NCS is held low, the application may want to activate the timeout counter (TCEN = 1 in OCTOSPI_CR): the NCS is released after a period defined by TIMEOUT[15:0] in OCTOSPI_LPTR, when x cycles have elapsed without access since the clock is inactive.

BUSY goes high as soon as the first memory-mapped access occurs. Because of the prefetch operations, BUSY does not fall until there is an abort, or the peripheral is disabled.

It is not recommended to program the flash memory using the memory-mapped writes. The indirect-write mode fulfills this operation.

However, if the application requires the use of the MCE for encryption (check MCE product availability), the memory-mapped write mode may be used to program encrypted data to external flash memory under the following conditions:

- Prefetch must be enabled.
- In block cipher mode, the CPU must write a complete 128-bit data block to prevent the MCE from initiating read-modify-write operations when only a few bytes need to be

programmed. This precaution avoids incorrect programming operations. There are no specific constraints to respect if the MCE is used in stream cipher mode.

- Apply the abort sequence to exit memory-mapped mode when the data linked to the page has been written in the external memory buffers. The abort sequence triggers the start of the page programming.
- Switch to the automatic status-polling mode to monitor the completion of the page programming phase.
- Relaunch the write enable command in indirect mode, then switch back to the memory-mapped mode configuration to continue to program additional pages if any.

It is recommended to add a synchronization barrier between the end of the controller registers configuration and the first memory-mapped access to the external memory when the controller is configured in memory-mapped mode.

The controller can apply an automatic address offset to simplify memory area code switching, such as during a firmware update (if the address offset feature is available in the product implementation table). To enable this feature, set the ADOFFEN bit in the XSPI_CR register and configure the offset value in the ADOFF[4:0] bitfield of the XSPI_DCR1 register. The offset is modulo 256 bytes.

25.4.12 OCTOSPI configuration introduction

The OCTOSPI configuration is done in three steps:

1. OCTOSPI system configuration
2. OCTOSPI device configuration
3. OCTOSPI mode configuration

25.4.13 OCTOSPI system configuration

The OCTOSPI is configured using OCTOSPI_CR. The user must program:

- the functional mode with FMODE[1:0]
- the automatic status-polling mode behavior if needed with PMM and APMS
- the FIFO level with FTHRES
- the DMA use with DMAEN
- the timeout counter use with TCEN
- the dual-memory configuration, if needed, with DMM

In case of an interrupt use, the respective enable bit can also be set during this phase.

If the timeout counter is used, the timeout value is programmed in OCTOSPI_LPTR.

The DMA channel must not be enabled during the OCTOSPI configuration: it must be enabled only when the operation is fully configured, to avoid any unexpected request generation.

The DMA and OCTOSPI must be configured in a coherent manner regarding data length: FTHRES value must reflect the DMA burst size.

25.4.14 OCTOSPI device configuration

The parameters related to the external device targeted are configured through OCTOSPI_DCR1 and OCTOSPI_DCR2. The user must program:

- the device size with DEVSIZ[4:0]
- the chip-select minimum high time with CSHT[5:0]
- the clock mode with FRCK and CKMODE
- the device frequency with PRESCALER[7:0]

DEVSIZ[4:0] defines the size of external memory using the following formula:

$$\text{Number of bytes in the device} = 2^{\text{DEVSIZ}+1}$$

where DEVSIZ+1 is the number of address bits required to address the external device. The external device capacity can go up to 4 Gbytes (addressed using 32 bits) in indirect mode, but the addressable space in memory-mapped mode is limited to 256 Mbytes.

If DMM = 1, DEVSIZ[4:0] must reflect the total capacity of the two devices together considering the above formula (DEVSIZ[4:0] value is so equal to one of the two memory capacities).

When the OCTOSPI executes two commands, one immediately after the other, it raises the chip-select signal (NCS) high between the two commands for only one CLK cycle by default.

If the external device requires more time between commands, the chip-select high time CSHT[5:0] can be used to specify the minimum number of CLK cycles for which the NCS must remain high.

CKMODE indicates the level that the CLK takes between commands (when NCS = 1).

In HyperBus protocol, the device timing (t_{ACC} and t_{RWR}) and the latency mode must be configured in OCTOSPI_HLCR.

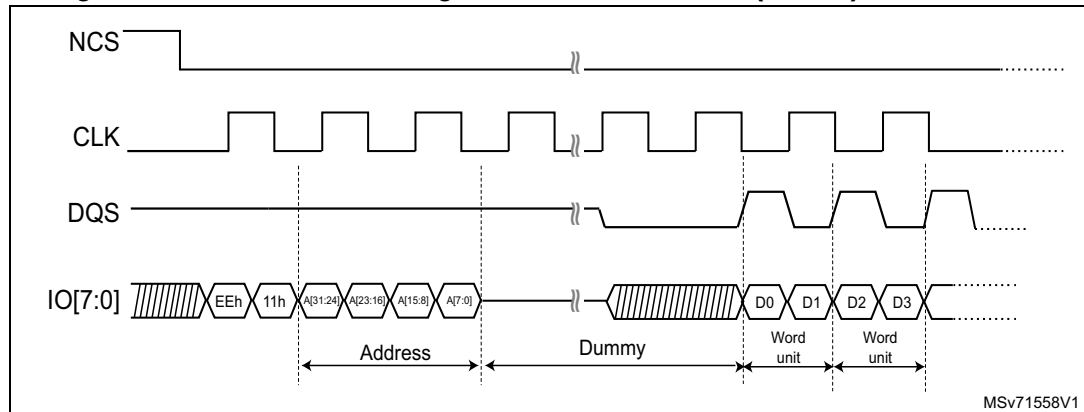
Memory types

External memory providers may present some architecture and slight data formatting differences between them. The bitfield MTYP[2:0] into the OCTOSPI_CR register allows targeting the right controller configuration depending on the associated memory type selected in the application. This is the responsibility of the software developer to align the controller configuration to fit with the targeted memory type.

The memory types are grouped in a such way:

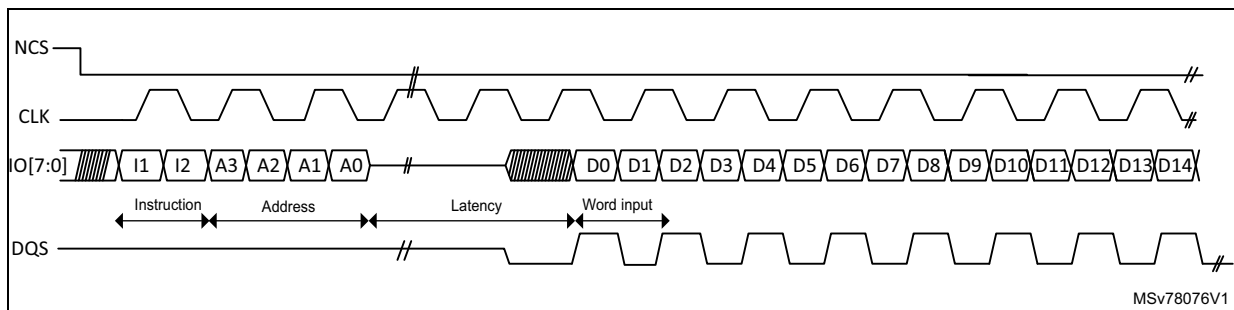
- D0/D1 data ordering in octal-SPI data mode (DMODE[2:0] = 100) in DTR mode by configuring MTYP[2:0] = 000. For instance, Micron is using such data ordering. In this configuration, the DQS is sent with a polarity inverted respect to the clock polarity.

Figure 192. D0/D1 data ordering in octal-SPI DTR mode (Micron) - Read access



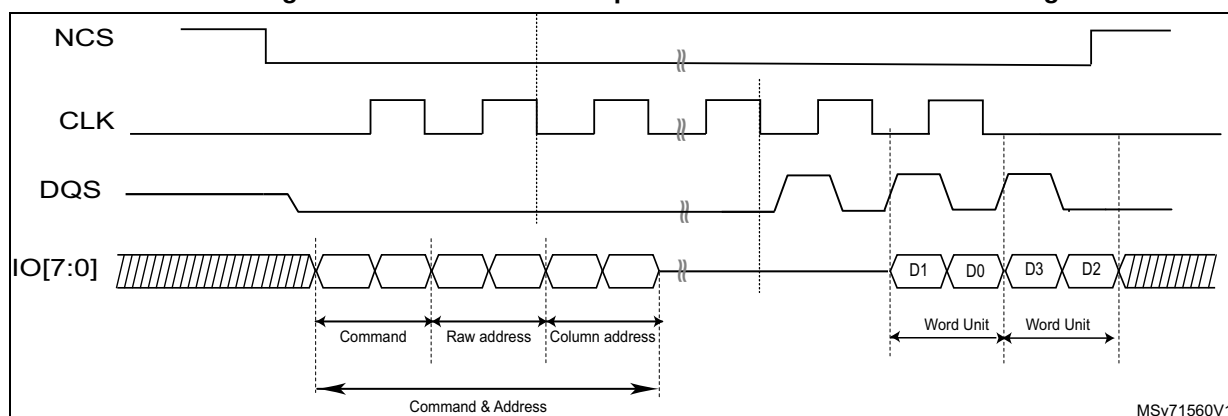
- D1/D0 data ordering in octal-SPI data mode (DMODE[2:0] = 100) in DTR mode by configuring MTYP[2:0] = 001. For instance, Macronix is using this reverse data ordering in its Octaflash portfolio (this configuration is not adapted to its OctaRAM™ memories). DQS is keeping the same polarity as the clock when reading data from the memory. Refer to [Figure 183: DTR read in octal-SPI mode with DQS \(Macronix mode\) example](#).
- D0/D1 data ordering in octal-SPI data mode (DMODE[2:0] = 100) in DTR mode by configuring MTYP[2:0] = 010. DQS is keeping the same polarity as the clock when reading data from the memory. It is defined as the standard mode since it is the configuration to select when the targeted memory does not fit with any others modes proposed by the MTYP configuration values.

Figure 193. D0/D1 data ordering in octal-SPI DTR mode with DQS and CLK polarity aligned during Read access



- D1/D0 data ordering in octal-SPI data mode (DMODE[2:0] = 100) in DTR mode by configuring MTYP[2:0] = 011 with specific address phase built with row and column to fit with Macronix OctaRAM™ memories requirement (refer to [Table 244: OctaRAM command address bit assignment \(based on 64-Mbyte OctaRAM\)](#)). This is the controller which translates internally the targeted address provided by the software in row/column address formatting to sent to the memory. DQS is keeping the same polarity as the clock one when reading data from the memory.

Figure 194. OctaRAM read operation with reverse data ordering D1/D0

Table 244. OctaRAM command address bit assignment
(based on 64-Mbyte⁽¹⁾ OctaRAM)

Clock	1st clock	2nd clock		3rd clock	
Function	Command	Row address		Column address	
SIO[7]	Command	Reserved	RA7	CA9	Reserved
SIO[6]		Reserved	RA6	CA8	Reserved
SIO[5]		Reserved	RA5	CA7	Reserved
SIO[4]		RA12	RA4	CA6	Reserved
SIO[3]		RA11	RA3	CA5	CA3
SIO[2]		RA10	RA2	CA4	CA2
SIO[1]		RA9	RA1	Reserved	CA1
SIO[0]		RA8	RA0	Reserved	CA0 ⁽²⁾

1. Example of 64-Mbyte OctaRAM address assignment:
Row Address [RA12:RA0]: 8K. Column address [CA9:CA0]: 1K. 64-Mbyte density = 8K x 1K x 8 bits

2. Column address A0 must be always 0.

- HyperBus memories need to be selected when targeted by the application. The configuration to set depends on the access type:
 - HyperBus memory mode: The protocol follows the HyperBus specification. MTYP[2:0] = 100 is the configuration to use to access the memory space.
 - HyperBus register mode (addressing register space): the memory-mapped accesses in this mode must be noncacheable, or the indirect read/write modes must be used. The configuration to be set for this particular register space access is MTYP[2:0] = 101.

25.4.15 OCTOSPI regular-command mode configuration

Indirect mode configuration

When $\text{FMODE}[1:0] = 00$, the indirect-write mode is selected and data can be sent to the external device. When $\text{FMODE}[1:0] = 01$, the indirect-read mode is selected, and data can be read from the external device.

When the OCTOSPI is used in indirect mode, the frames are constructed in the following way:

1. Specify a number of data bytes to read or write in `OCTOSPI_DLR`.
2. Specify the frame timing in `OCTOSPI_TCR`.
3. Specify the frame format in `OCTOSPI_CCR`.
4. Specify the instruction in `OCTOSPI_IR`.
5. Specify the optional alternate byte to be sent right after the address phase in `OCTOSPI_ABR`.
6. Specify the targeted address in `OCTOSPI_AR`.
7. Enable the DMA channel if needed.
8. Read/write the data from/to the FIFO through `OCTOSPI_DR` (if no DMA usage).

If neither the address register (`OCTOSPI_AR`) nor the data register (`OCTOSPI_DR`) need to be updated for a particular command, then the command sequence starts as soon as `OCTOSPI_IR` is written. This is the case when both $\text{ADMODE}[2:0]$ and $\text{DMODE}[2:0]$ equal 000, or if just $\text{ADMODE}[2:0] = 000$ when in indirect-read mode ($\text{FMODE}[1:0] = 01$).

When an address is required ($\text{ADMODE}[2:0] \neq 000$) and the data register does not need to be written ($\text{FMODE}[1:0] = 01$ or $\text{DMODE}[2:0] = 000$), the command sequence starts as soon as the address is updated with a write to `OCTOSPI_AR`.

In case of data transmission ($\text{FMODE}[1:0] = 00$ and $\text{DMODE}[2:0] \neq 000$), the communication start is triggered by a write in the FIFO through `OCTOSPI_DR`.

Automatic status-polling mode configuration

The automatic status-polling mode is enabled by setting $\text{FMODE}[1:0] = 10$. In this mode, the programmed frame is sent and data are retrieved periodically.

The maximum amount of data read in each frame is 4 bytes. If more data is requested in `OCTOSPI_DLR`, it is ignored, and only 4 bytes are read. The periodicity is specified in `OCTOSPI_PIR`.

Once the status data has been retrieved, the following can be processed:

- Set SMF (an interrupt is generated if enabled).
- Stop automatically the periodic retrieving of the status bytes.

The received value can be masked with the value stored in `OCTOSPI_PSMKR`, and can be ORed or ANDed with the value stored in `OCTOSPI_PSMAR`.

In case of a match, SMF is set and an interrupt is generated if enabled. The OCTOSPI can be automatically stopped if AMPS is set. In any case, the latest retrieved value is available in `OCTOSPI_DR`.

When the OCTOSPI is used in automatic status-polling mode, the frames are constructed in the following way:

1. Specify the input mask in `OCTOSPI_PSMKR`.

2. Specify the comparison value in OCTOSPI_PSMAR.
3. Specify the read period in OCTOSPI_PIR.
4. Specify a number of data bytes to read in OCTOSPI_DLR.
5. Specify the frame timing in OCTOSPI_TCR.
6. Specify the frame format in OCTOSPI_CCR.
7. Specify the instruction in OCTOSPI_IR.
8. Specify the optional alternate byte to be sent right after the address phase in OCTOSPI_ABR.
9. Specify the optional targeted address in OCTOSPI_AR.

If the address register (OCTOSPI_AR) does not need to be updated for a particular command, then the command sequence starts as soon as OCTOSPI_CCR is written. This is the case when $ADMODE[2:0] = 000$.

When an address is required ($ADMODE[2:0] \neq 000$), the command sequence starts as soon as the address is updated with a write to OCTOSPI_AR.

Memory-mapped mode configuration

In memory-mapped mode, the external device is seen as an internal memory but with some latency during accesses. Read and write operations are allowed to the external device in this mode.

It is not recommended to program the flash memory using memory-mapped writes, as the internal flags for erase or programming status have to be polled. The indirect-write mode fulfills this operation, possibly in conjunction with the automatic status-polling mode.

The memory-mapped mode is entered by setting $FMODE[1:0] = 11$ in OCTOSPI_CR.

The programmed instruction and frame are sent when an AHB master accesses the memory-mapped space.

The FIFO is used as a prefetch buffer to anticipate any linear reads. Any access to OCTOSPI_DR in this mode returns zero.

The data length register (OCTOSPI_DLR) has no meaning in memory-mapped mode.

When the OCTOSPI is used in memory-mapped mode, the frames are constructed in the following way:

1. Specify the frame timing in OCTOSPI_TCR for read operation.
2. Specify the frame format in OCTOSPI_CCR for read operation.
3. Specify the instruction in OCTOSPI_IR.
4. Specify the optional alternate byte to be sent right after the address phase in OCTOSPI_ABR for read operation.
5. Specify the frame timing in OCTOSPI_WTCR for write operation.
6. Specify the frame format in OCTOSPI_WCCR for write operation.
7. Specify the instruction in OCTOSPI_WIR.
8. Specify the optional alternate byte to be sent right after the address phase in OCTOSPI_WABR for write operation.

All configuration operations must be completed (ensured by checking $BUSY = 0$) before the first access to the memory area: any register write operation when $BUSY = 1$ has no effect and is not signaled with an error response. On the first access, the OCTOSPI becomes

busy, and no further configuration is allowed. Then, the only way to get BUSY low is to clear the ENABLE bit or to abort by setting the ABORT bit.

OCTOSPI delayed data sampling when no DQS is used

By default, when no DQS is used, the OCTOSPI samples the data driven by the external device one half of a CLK cycle after the external device drives the signal.

In case of any external signal delays, it may be useful to sample the data later. Using SSHIFT in OCTOSPI_TCR, the sampling of the data can be shifted by half of a CLK cycle.

The firmware must clear SSHIFT when the data phase is configured in DTR mode (DDTR = 1).

OCTOSPI delayed data sampling when DQS is used

When external DQS is used as a sampling clock, it can be shifted in time to compensate the data propagation delay.

This shift is performed by a delay hardware element located outside of the controller into the RCC (RCC_DLYCFGR) or implemented on an external dedicated block delay peripheral depending on the product (refer to the product reference manual for information about how to manage and configure the delay).

In configurations where delay does not need to be compensated, the external delay block can be bypassed by setting DLYBYP in OCTOSPI_DCR1.

25.4.16 OCTOSPI HyperBus protocol configuration

Indirect mode configuration (HyperBus)

When FMODE[1:0] = 00, the indirect-write mode is selected and data can be sent to the external device. When FMODE[1:0] = 01, the indirect-read mode is selected where data can be read from the external device. ADMODE must be configured with a value different from 000 (for instance ADMODE = 100).

When the OCTOSPI is used in indirect mode, the frames are constructed in the following way:

1. Specify a number of data bytes to read or write in OCTOSPI_DLR.
2. Specify the targeted address in OCTOSPI_AR.
3. Enable the DMA channel if needed.
4. Read/write the data from/to the FIFO through OCTOSPI_DR (if no DMA usage).

In indirect-read mode, the command sequence starts as soon as the address is updated with a write to OCTOSPI_AR.

In indirect-write mode, the communication start is triggered by a write in the FIFO through OCTOSPI_DR.

Memory-mapped mode configuration (HyperBus)

In memory-mapped mode, the external device is seen as an internal memory but with some latency during the accesses. Read and write operations are allowed to the external device in this mode.

It is not recommended to program the flash memory using the memory-mapped writes: the indirect-write mode fulfills this operation.

The memory-mapped mode is entered by setting FMODE[1:0] = 11. The programmed instruction and frame is sent when an AHB master accesses the memory-mapped space.

The FIFO is used as a prefetch buffer to anticipate any linear reads. Any access to OCTOSPI_DR in this mode returns zero.

The data length register (OCTOSPI_DLR) has no meaning in memory-mapped mode.

All the configuration operation must be completed before the first access to the memory area. On the first access, the OCTOSPI becomes busy, and no configuration is allowed. Then, the only way to get BUSY low is to clear the ENABLE bit, or to abort by setting the ABORT bit.

25.4.17 OCTOSPI error management

The following errors set the TEF flag in OCTOSPI_SR and generates an interrupt if enabled (TEIE = 1 in OCTOSPI_CR):

- in indirect or automatic status-polling mode, when a wrong address has been programmed in OCTOSPI_AR (according to the device size defined by DEVSIZE[4:0]).
- in indirect mode, if the address plus the data length exceed the device size: TEF is set as soon as the access is triggered.

In memory-mapped mode, the OCTOSPI generates an AHB slave error in the following situations, setting the BERRF flag in XSPI_SR and generating an interrupt if enabled (BERRIE = 1 in XSPI_CR) (if the Bus error bit feature is available in the product implementation table, otherwise the Slave bus error response is generated at system level as transaction response with no dedicated bit into the controller status register):

- The memory-mapped mode is disabled and an AHB read or write request occurs.
- A read or write address (including the address offset if ADOFFEN bit is set) exceeds the size of the external memory.
- An abort is received while a read or write burst is ongoing.
- The OCTOSPI is disabled while a read or write burst is requested.
- A write wrap burst is received.
- A write request is received while DQSE = 0 in OCTOSPI_WCCR in octal DTR mode, in dual-memory configuration, in HyperBus mode.
- A read or write request is received while DMODE[2:0] = 000 (no data phase), except when MTYP[2:0] is HyperBus.
- Illegal access size when wrap read burst. This means that the AHB bus transfer size (HSIZE) is different from 4 bytes (only for memory-mapped mode).
- Bit DMM is set in OCTOSPI_CR (dual-memory configuration) and octal mode is set.

25.4.18 OCTOSPI BUSY and ABORT

Once the OCTOSPI starts an operation with the external device, BUSY is automatically set in OCTOSPI_SR.

In indirect mode, BUSY is reset once the OCTOSPI has completed the requested command sequence and the FIFO is empty.

In automatic status-polling mode, BUSY goes low only after the last periodic access is complete, due to a match when APMS = 1 or due to an abort.

After the first access in memory-mapped mode, BUSY goes low only on an abort.

Any operation can be aborted by setting ABORT in OCTOSPI_CR. Once the abort is completed, BUSY and ABORT are automatically reset, and the FIFO is flushed.

Before setting ABORT, the software must ensure that all the current transactions are finished using the synchronization barriers. When DMA is enabled to handle the data read or write operations in OCTOSPI_DR, it is recommended to disable the DMA channel before aborting the OCTOSPI.

Note: Some devices may misbehave if a write operation to a status register is aborted.

25.4.19 OCTOSPI reconfiguration or deactivation

Before any OCTOSPI reconfiguration, the software must ensure that all the transactions are completed:

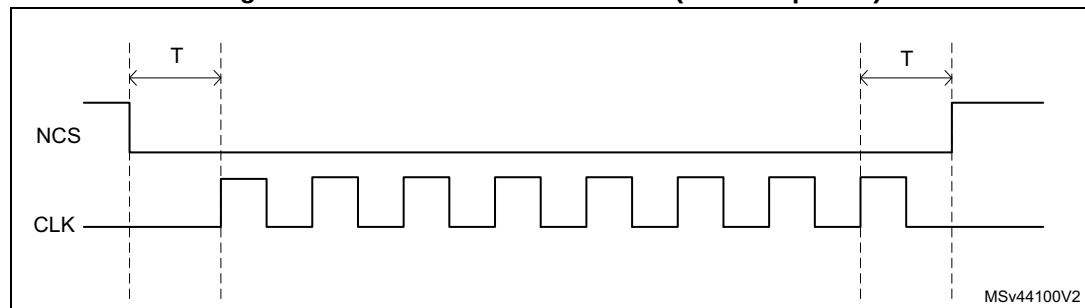
- After a memory-mapped write, the software must perform a dummy read followed by a synchronization barrier, then an abort.
- After a memory-mapped read, the software must perform a synchronization barrier then an abort.

25.4.20 NCS behavior

By default, NCS is high, deselecting the external device. NCS falls before an operation begins and rises as soon as it finishes.

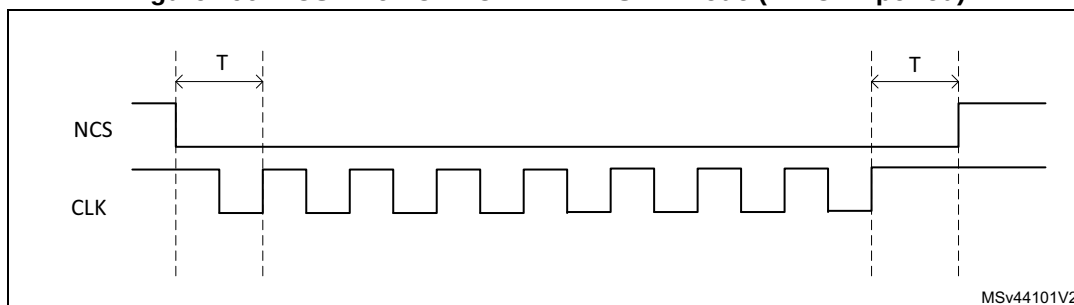
When CKMODE = 0 (clock mode 0: CLK stays low when no operation is in progress), NCS falls one CLK cycle before an operation first rising CLK edge, and NCS rises one CLK cycle after the operation final rising CLK edge (see the figure below).

Figure 195. NCS when CKMODE = 0 (T = CLK period)



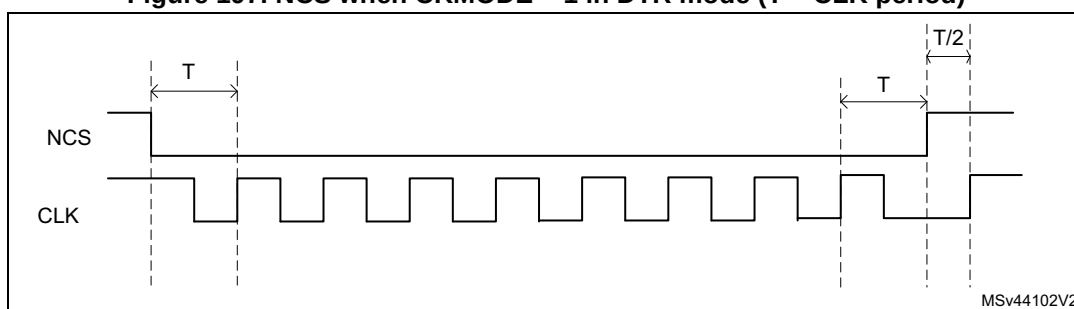
When $CKMODE = 1$ (clock mode 3: CLK goes high when no operation is in progress) and when in SDR mode, NCS falls one CLK cycle before an operation first rising CLK edge, and NCS rises one CLK cycle after the operation final rising CLK edge (see the figure below).

Figure 196. NCS when $CKMODE = 1$ in SDR mode ($T = CLK$ period)



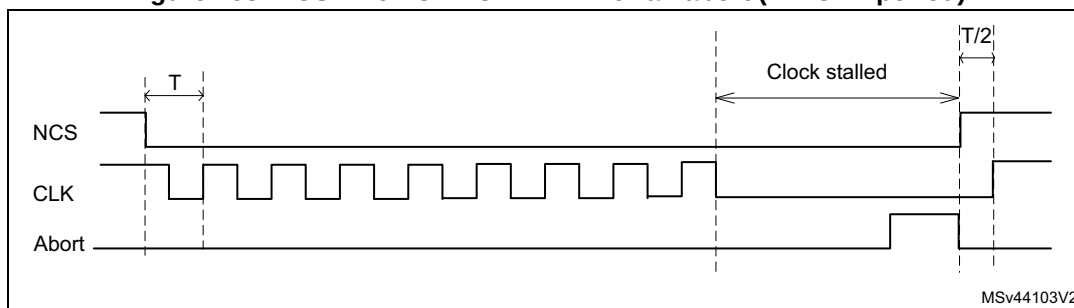
When the $CKMODE = 1$ (clock mode 3) and $DDTR = 1$ (data DTR mode), NCS falls one CLK cycle before an operation first rising CLK edge, and NCS rises one CLK cycle after the operation final active rising CLK edge (see the figure below). Because the DTR operations must finish with a falling edge, CLK is low when NCS rises, and CLK rises back up one half of a CLK cycle afterwards.

Figure 197. NCS when $CKMODE = 1$ in DTR mode ($T = CLK$ period)



When the FIFO stays full during a read operation, or if the FIFO stays empty during a write operation, the operation stalls and CLK stays low until the software services the FIFO. If an abort occurs when an operation is stalled, NCS rises just after the abort is requested and then CLK rises one half of a CLK cycle later (see the figure below).

Figure 198. NCS when $CKMODE = 1$ with an abort ($T = CLK$ period)

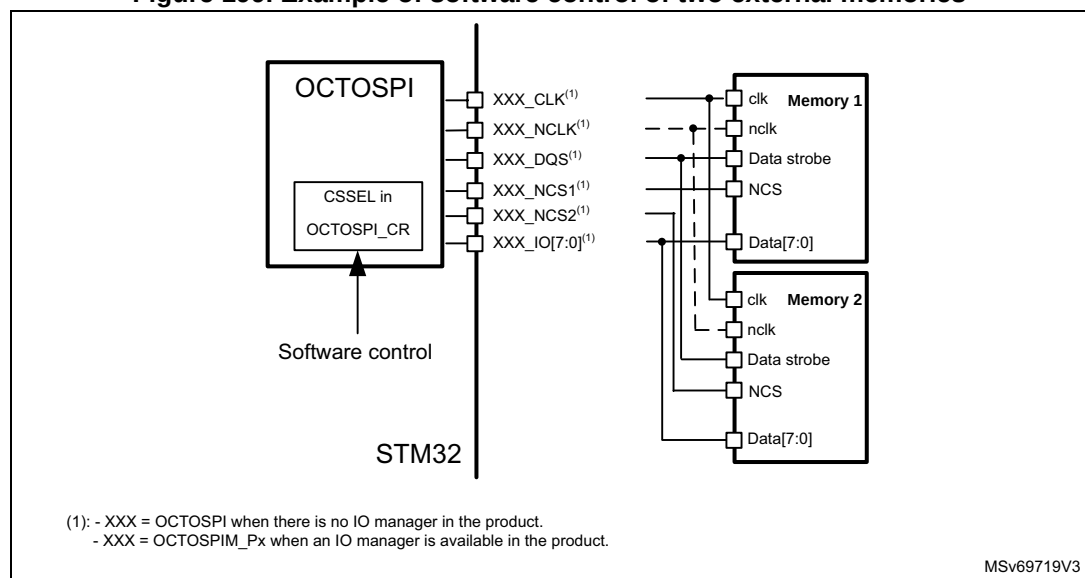


In dual memory configuration, the NCS2 behaves exactly the same as the NCS1. The quad-SPI/octal-SPI memory 2 can then use the NCS1, and the NCS2 pin can be used for other functions.

25.4.21 Software control of two external memories

One OCTOSPI instance is able to control the generation of the two chip-select signals (NCS1 and NCS2). This allows two external memories to be connected to the same I/O port driven by the OCTOSPI. The OCTOSPI generates one NCS signal and the user defines how the NCS is directed to NCS1 or NCS2, by setting the CSSEL bit in OCTOSPI_CR. Intended operation is semistatic commutation, such as starting an application reading an external flash memory and then, for the rest of the application, use an external SRAM connected to the same I/O bus. From a memory-map point of view, these two memories are located in the same address.

Figure 199. Example of software control of two external memories



25.4.22 Hardware-controlled extended memory support

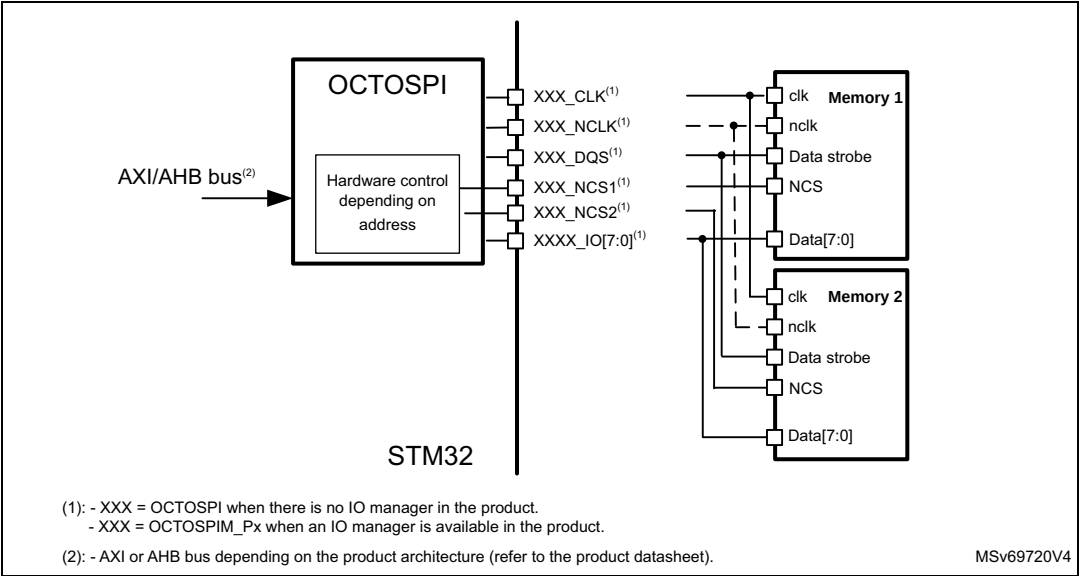
One OCTOSPI instance is able to control autonomously the generation of the two chip-select signals (NCS1 and NCS2). This feature is activated by setting EXTENDMEM in OCTOSPI_DCR1 and allows two external memories to be connected to the same I/O port driven by the OCTOSPI. The aim is to extend by a factor two, the available external memory size. Both memories must be contiguous in the memory map. Both external memories must be of the same type and size. The size of one external memory must be set in DEVSZ of OCTOSPI_CR.

- When the address presented on the interconnect bus in memory-mapped mode or in OCTOSPI_AR in indirect mode is in the range $[\text{OCTOSPI_base_address}; \text{OCTOSPI_base_address} + 2^{\text{DEVSZ}} - 1]$, NCS1 is generated.
- When the address presented on the interconnect bus in memory-mapped mode or in OCTOSPI_AR in indirect mode is in the range $[\text{OCTOSPI_base_address} + 2^{\text{DEVSZ}}; \text{OCTOSPI_base_address} + 2 \times 2^{\text{DEVSZ}} - 1]$, NCS2 is generated.

Note: If the access crosses the middle boundary of the external memory, CS changes automatically (no write into OCTOSPI_AR in indirect mode, or new memory access in memory-mapped mode is needed).

EXTENDMEM feature (set in bit21 of OCTOSPI_DCR1) and dual-memory mode (set in bit6 of OCTOSPI_CR) are exclusive: EXTENDMEM = 1 and DMM = 1 is not supported.

Figure 200. Example of hardware-controller extended memory support



Note: It is recommended to set EXTENDMEM = 0 for write transfers targeting flash memories (HyperFlash, NOR Flash, etc.).

These memories require preliminary commands that do not include an address phase used by the controller to select the targeted memory.

This recommendation does not apply to RAM memories, as those extra commands are not needed for RAM access.

25.5 Address alignment and data number

The table below summarizes the effect of the address alignment and programmed data number depending on the use case.

Table 245. Address alignment cases

Memory type	Transaction type	Constraint on address ⁽¹⁾	Impact if constraint on address not respected	Constraint on number of bytes ⁽¹⁾	Impact if constraint on bytes not respected
Single, dual, quad flash or SRAM (DMM = 0)	IND ⁽²⁾ read	None	None	None	None
	MM ⁽³⁾ read				
	IND write				
	MM write				

Table 245. Address alignment cases (continued)

Memory type	Transaction type	Constraint on address ⁽¹⁾	Impact if constraint on address not respected	Constraint on number of bytes ⁽¹⁾	Impact if constraint on bytes not respected
Single, dual, quad flash or SRAM (DMM = 1)	IND read	Even	ADDR[0] is set to 0. ⁽⁴⁾	Even	DLR[0] is set to 1. ⁽⁵⁾
	MM read	None	None	None	None
	IND write	Even	ADDR[0] is set to 0. ⁽⁴⁾	Even	DLR[0] is set to 1. ⁽⁵⁾
	MM write	Even	Slave error	Even	Last byte is lost.
Octal flash in SDR mode	IND read	None	None	None	None
	MM read				
	IND write				
	MM write				
Octal memory in DTR mode without WDM ⁽⁶⁾ or 8-bit HyperFLASH	IND read	Even	ADDR[0] is set to 0. ⁽⁴⁾	Even	DLR[0] is set to 1. ⁽⁵⁾
	MM read	None	None	None	None
	IND write	Even	ADDR[0] is set to 0. ⁽⁴⁾	Even	DLR[0] is set to 1. ⁽⁵⁾
	MM write	Even	Slave error	Even	Last byte is lost.
Octal flash or RAM in DTR mode with WDM	IND read	Even	ADDR[0] is set to 0. ⁽⁴⁾	Even	DLR[0] is set to 1. ⁽⁵⁾
	MM read	None	None	None	None
	IND write ⁽⁷⁾				
	MM write				
8-bit HyperRAM	IND read	Even	ADDR[0] is set to 0. ⁽⁴⁾	Even	DLR[0] is set to 1. ⁽⁵⁾
	MM read	None	None	None	None
	IND write ⁽⁷⁾				
	MM write				

1. To be respected by the software.

2. IND = indirect mode.

3. MM = memory-mapped mode

4. Extra data at transfer start.

5. Extra data at transfer end.

6. WDM = write data mask.

7. If the FTHRES bitfield is set to the maximum value with DLR value greater than the data burst length, and if the DMA is enabled or the interrupt based on FIFO THRESHOLD Flag is enabled (FTF), the address must be modulo 2 aligned in DTR mode when the initiator (DMA, CPU, ...) is writing the data with a burst length equal to the FIFO size.

25.6 OCTOSPI interrupts

An interrupt can be produced on the following events:

- Timeout
- Status match
- FIFO threshold
- Transfer complete

- Transfer error

Separate interrupt enable bits are available to provide more flexibility.

Table 246. OCTOSPI interrupt requests

Interrupt event	Event flag	Enable control bit
Timeout	TOF	TOIE
Status match	SMF	SMIE
FIFO threshold	FTF	FTIE
Transfer complete	TCF	TCIE
Transfer error	TEF	TEIE

25.7 OCTOSPI registers

25.7.1 OCTOSPI control register (OCTOSPI_CR)

Address offset: 0x0000

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	MSEL	FMODE[1:0]		Res.	Res.	NOPR EF	CSSEL	PMM	APMS	BERRI E	TOIE	SMIE	FTIE	TCIE	TEIE
	rw	rw	rw			rw	rw	rw	rw	rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	Res.	FTHRES[4:0]				Res.	DMM	Res.	ADOFF EN	TCEN	DMAE N	ABORT	EN	
			rw	rw	rw	rw	rw		rw		rw	rw	rw	w	rw

Bit 31 Reserved, must be kept at reset value.

Bit 30 **MSEL**: External memory select

This bit selects the external memory to be addressed in single-, dual-, quad-SPI mode in single-memory configuration (when DMM = 0).

0: External memory 1 selected (data exchanged over IO[3:0])

1: External memory 2 selected (data exchanged over IO[7:4])

This bit is ignored when DMM = 1 or when octal-SPI mode is selected.

Note: This bit is mirrored in bit 7. This bit can be modified only when BUSY = 0.

Bits 29:28 **FMODE[1:0]**: Functional mode

This bitfield defines the OCTOSPI functional mode of operation.

00: Indirect-write mode

01: Indirect-read mode

10: Automatic status-polling mode (relevant in regular-command protocol only)

11: Memory-mapped mode

If DMAEN = 1 already, then the DMA controller for the corresponding channel must be disabled before changing the FMODE[1:0] value.

Note: This bitfield can be modified only when BUSY = 0.

Bits 27:26 Reserved, must be kept at reset value.

Bit 25 **NOPREF**: No prefetch data

This bit disables the automatic prefetch in the external memory.

0: Automatic prefetch enabled

1: Automatic prefetch disabled

Bit 24 **CSSEL**: Chip-select selection

This bit indicates to OCTOSPI I/O manager if it must copy NCS to NCS1 or NCS2.

0: NCS I/O manager input is copied to NCS1 I/O manager output.

1: NCS I/O manager input is copied to NCS2 I/O manager output.

Note: This bit can be modified only when BUSY = 0.

Bit 23 **PMM**: Polling match mode

This bit indicates which method must be used to determine a match during the automatic status-polling mode.

0: AND-match mode, SMF is set if all the unmasked bits received from the device match the corresponding bits in the match register.

1: OR-match mode, SMF is set if any of the unmasked bits received from the device matches its corresponding bit in the match register.

Note: This bit can be modified only when BUSY = 0.

Bit 22 **APMS**: Automatic status-polling mode stop

This bit determines if the automatic status-polling mode is stopped after a match.

0: Automatic status-polling mode is stopped only by abort or by disabling the OCTOSPI.

1: Automatic status-polling mode stops as soon as there is a match.

Note: This bit can be modified only when BUSY = 0.

Bit 21 **BERRIE**: Bus error interrupt enable

This bit enables the bus error interrupt (if Bus error bit feature is available in the product implementation table).

0: Interrupt disabled

1: Interrupt enabled

Bit 20 **TOIE**: Timeout interrupt enable

This bit enables the timeout interrupt.

0: Interrupt disabled

1: Interrupt enabled

Bit 19 **SMIE**: Status-match interrupt enable

This bit enables the status-match interrupt.

0: Interrupt disabled

1: Interrupt enabled

Bit 18 **FTIE**: FIFO threshold interrupt enable

This bit enables the FIFO threshold interrupt.

0: Interrupt disabled

1: Interrupt enabled

Bit 17 **TCIE**: Transfer complete interrupt enable

This bit enables the transfer complete interrupt.

0: Interrupt disabled

1: Interrupt enabled

Bit 16 **TEIE**: Transfer error interrupt enable

This bit enables the transfer error interrupt.

0: Interrupt disabled

1: Interrupt enabled

Bits 15:13 Reserved, must be kept at reset value.

Bits 12:8 **FTHRES**[4:0]: FIFO threshold level

This bitfield defines, in indirect mode, the threshold number of bytes in the FIFO that causes the FIFO threshold flag FTF in OCTOSPI_SR, to be set.

00000: FTF is set if there are one or more free bytes available to be written to in the FIFO in indirect-write mode, or if there are one or more valid bytes can be read from the FIFO in indirect-read mode.

00001: FTF is set if there are two or more free bytes available to be written to in the FIFO in indirect-write mode, or if there are two or more valid bytes can be read from the FIFO in indirect-read mode.

...

11111: FTF is set if there are 32 free bytes available to be written to in the FIFO in indirect-write mode, or if there are 32 valid bytes can be read from the FIFO in indirect-read mode.

Note: If DMAEN = 1, the DMA controller for the corresponding channel must be disabled before changing the FTHRES[4:0] value.

Bit 7 Reserved, must be kept at reset value.

Bit 6 **DMM**: Dual-memory configuration

This bit activates the dual-memory configuration, where two external devices are used simultaneously to double the throughput and the capacity

0: Dual-memory configuration disabled

1: Dual-memory configuration enabled

Note: This bit can be modified only when BUSY = 0.

Bit 5 Reserved, must be kept at reset value.

Bit 4 **ADOFFEN**: Address offset enable

An offset may be applied on the memory address. The offset value is set into ADOFF[4:0].

0: Address offset disabled

1: Address offset enabled

Note: This bit can be modified only when BUSY = 0. This bit is present only if the address offset feature is available in the product implementation table.

Bit 3 **TCEN**: Timeout counter enable

This bit is valid only when the memory-mapped mode (FMODE[1:0] = 11) is selected. This bit enables the timeout counter.

0: The timeout counter is disabled, and thus the chip-select (NCS) remains active indefinitely after an access in memory-mapped mode.

1: The timeout counter is enabled, and thus the chip-select is released in the memory-mapped mode after TIMEOUT[15:0] cycles of external device inactivity.

Note: This bit can be modified only when BUSY = 0.

Bit 2 **DMAEN**: DMA enable

In indirect mode, the DMA can be used to input or output data via OCTOSPI_DR. DMA transfers are initiated when FTF is set.

0: DMA disabled for indirect mode

1: DMA enabled for indirect mode

Note: Resetting the DMAEN bit while a DMA transfer is ongoing, breaks the handshake with the DMA. Do not write this bit during DMA operation.

Bit 1 **ABORT**: Abort request

This bit aborts the ongoing command sequence. This bit stops the current transfer.

0: No abort requested

1: Abort requested

Note: This bit is always read as 0.

Bit 0 **EN**: Enable

This bit enables the OCTOSPI.

0: OCTOSPI disabled

1: OCTOSPI enabled

Note: The DMA request can be aborted without having received the ACK in case this EN bit is cleared during the operation.

In case this bit is set to 0 during a DMA transfer, the REQ signal to DMA returns to inactive state without waiting for the ACK signal from DMA to be active.

25.7.2 OCTOSPI device configuration register 1 (OCTOSPI_DCR1)

Address offset: 0x0008

Reset value: 0x0000 0000

This register can be modified only when BUSY = 0.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
ADOFF[4:0]					MTYP[2:0]			Res.	Res.	EXTEN DMEM	DEVSIZ[4:0]				
rw	rw	rw	rw	rw	rw	rw	rw			rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	CSHT[5:0]						Res.	Res.	Res.	Res.	DLY BYP	Res.	FRCK	CKMO DE
		rw	rw	rw	rw	rw	rw					rw		rw	rw

Bits 31:27 **ADOFF[4:0]**: Address offset

This bitfield defines the memory address offset value added to the current address requested to be sent by the controller to hit the external memory using the following formula:

Offset value = $2^{[ADOFF+8]}$. (modulo 256 bytes)

Maximum allowed value is 19 since the memory-mapped mode is limited to 256 Mbytes address space. For greater value, the address offset will not be applied to the address sent by the controller.

This bitfield is relevant only if ADOFFEN is set and if FMODE[1:0] = 11 (memory-mapped mode) and if the address offset feature is available in the product implementation table.

Bits 26:24 **MTYP[2:0]**: Memory type

This bitfield indicates the type of memory to be supported.

000: Micron mode, D0/D1 ordering in DTR 8-data-bit mode. regular-command protocol in single-, dual-, quad- and octal-SPI modes.

Note: In this mode, DQS signal polarity is inverted with respect to the memory clock signal. This is the default value and care must be taken to change MTYP[2:0] for memories different from Micron.

001: Macronix mode, D1/D0 ordering in DTR 8-data-bit mode. Regular-command protocol in single-, dual-, quad-, and octal-SPI modes.

010: Standard mode. D0/D1 ordering with DQS and CLK having the same polarity during read operation. This is the mode to use when the targeted memory does not fit with any others MTYP configuration values. This mode is typically the one targeting the APmemory 8-bit memories for instance.

011: Macronix RAM mode, D1/D0 ordering in DTR 8-data-bit mode. Regular-command protocol in single-, dual-, quad-, and octal-SPI modes with dedicated address mapping (address is built with row and column to fit with Macronix requirements).

100: HyperBus memory mode, the protocol follows the HyperBus specification.

101: HyperBus register mode, addressing register space. The memory-mapped accesses in this mode must be noncacheable, or indirect-read/write modes must be used.

Others: Reserved

Bits 23:22 Reserved, must be kept at reset value.

Bit 21 **EXTENDMEM**: Extended memory support

This bit is only relevant when one OCTOSPI drives two same size external memories located in contiguous places in the memory map. In this case, DEVSIZ[4:0] represents the size of each external memory. When this bit is set, the OCTOSPI automatically controls which of NCS1 or NCS2 is active.

0: NCS1 and NCS2 values depend from CSSEL bit in OCTOSPI_CR (software controlled).

1: NCS1 and NCS2 values depend from the address of transfer (hardware controlled).

Note: It is recommended to keep EXTENDMEM = 0 for write operations targeting flash memories (such as NOR Flash and HyperFlash).

Bits 20:16 **DEVSIZ[4:0]**: Device size

This bitfield defines the size of the external device using the following formula:

Number of bytes in device = $2^{[DEVSIZ+1]}$

DEVSIZ + 1 is effectively the number of address bits required to address the external device. The device capacity can be up to 4 Gbytes (addressed using 32-bits) in indirect mode, but the addressable space in memory-mapped mode is limited to 256 Mbytes.

In regular-command protocol, if DMM = 1 or EXTENDMEM = 1, DEVSIZ[4:0] must reflect the total capacity of the two devices together considering the above formula (DEVSIZ[4:0] value is so equal to one of the two memory capacities).

Bits 15:14 Reserved, must be kept at reset value.

Bits 13:8 **CSHT[5:0]**: Chip-select high time

CSHT + 1 defines the minimum number of CLK cycles where the chip-select (NCS) must remain high between commands issued to the external device.

0x0: NCS stays high for at least 1 cycle between external device commands.

0x1: NCS stays high for at least 2 cycles between external device commands.

...

0x3F: NCS stays high for at least 64 cycles between external device commands.

Bits 7:4 Reserved, must be kept at reset value.

Bit 3 **DLYBYP**: Delay block bypass

0: The internal sampling clock (called feedback clock) or the DQS data strobe external signal is delayed by the delay block (for more details on this block, refer to the dedicated section of the reference manual as it is not part of the OCTOSPI peripheral).

1: The delay block is bypassed, so the internal sampling clock or the DQS data strobe external signal is not affected by the delay block. The delay is shorter than when the delay block is not bypassed, even with the delay value set to minimum value in delay block.

Bit 2 Reserved, must be kept at reset value.

Bit 1 **FRCK**: Free running clock

This bit configures the free running clock.

0: CLK is not free running.

1: CLK is free running (always provided).

Note: Free running clock mode is intended for delay calibration only. No memory or other device access is possible when FRCK is set. Once the bit has been set to 1, the BSY bit is set, the NCS signal remains set to 1 and the clock is sent. The only way to stop the clock is to perform an abort (which clears the BSY bit). Then, the FRCK bit must be cleared by software to allow controller transactions to take place with the external memory.

Bit 0 **CKMODE**: Clock mode 0/mode 3

This bit indicates the level taken by the CLK between commands (when NCS = 1).

0: CLK must stay low while NCS is high (chip-select released). This is referred to as clock mode 0.

1: CLK must stay high while NCS is high (chip-select released). This is referred to as clock mode 3.

25.7.3 OCTOSPI device configuration register 2 (OCTOSPI_DCR2)

Address offset: 0x000C

Reset value: 0x0000 0000

This register can be modified only when BUSY = 0.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	WRAPSIZE[2:0]		
													rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	PRESCALER[7:0]							
								rw	rw	rw	rw	rw	rw	rw	rw

Bits 31:19 Reserved, must be kept at reset value.

Bits 18:16 **WRAPSIZE[2:0]**: Wrap size

This bitfield indicates the wrap size to which the memory is configured. For memories which have a separate command for wrapped instructions, this bitfield indicates the wrap-size associated with the command held in the OCTOSPI1_WPIR register.

000: Wrapped reads are not supported by the memory.

010: External memory supports wrap size of 16 bytes.

011: External memory supports wrap size of 32 bytes.

100: External memory supports wrap size of 64 bytes.

101: External memory supports wrap size of 128 bytes.

Others: Reserved

Bits 15:8 Reserved, must be kept at reset value.

Bits 7:0 **PRESCALER[7:0]**: Clock prescaler

This bitfield defines the scaler factor for generating the CLK based on the kernel clock (value + 1).

0: $F_{CLK} = F_{KERNEL}$, kernel clock used directly as OCTOSPI CLK (prescaler bypassed). In this case, if the DTR mode is used, it is mandatory to provide to the OCTOSPI a kernel clock that has 50% duty-cycle.

1: $F_{CLK} = F_{KERNEL}/2$

2: $F_{CLK} = F_{KERNEL}/3$

...

255: $F_{CLK} = F_{KERNEL}/256$

For odd clock division factors, the CLK duty cycle is not 50 %. The clock signal remains low one cycle longer than it stays high.

25.7.4 OCTOSPI device configuration register 3 (OCTOSPI_DCR3)

Address offset: 0x0010

Reset value: 0x0000 0000

This register can be modified only when BUSY = 0.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	CSBOUND[4:0]				
											rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.

Bits 31:21 Reserved, must be kept at reset value.

Bits 20:16 **CSBOUND[4:0]**: NCS boundary

This bitfield enables the transaction boundary feature. When active, a minimum value of 3 is recommended. The NCS is released on each boundary of $2^{CSBOUND}$ bytes.

0: NCS boundary disabled

Others: NCS boundary set to $2^{CSBOUND}$ bytes

Bits 15:0 Reserved, must be kept at reset value.

25.7.5 OCTOSPI device configuration register 4 (OCTOSPI_DCR4)

Address offset: 0x0014

Reset value: 0x0000 0000

This register can be modified only when BUSY = 0.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
REFRESH[31:16]															
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
REFRESH[15:0]															
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bits 31:0 **REFRESH[31:0]**: Refresh rate

This bitfield enables the refresh rate feature. The NCS is released every REFRESH + 1 clock cycles for writes, and REFRESH + 4 clock cycles for reads. These two values can be extended with few clock cycles when refresh occurs during a byte transmission in single-, dual- or quad-SPI mode, because the byte transmission must be completed.

0: Refresh disabled

Others: Maximum communication length is set to REFRESH + 1 clock cycles.

Note: REFRESH count is based on the divided clock period: if OCTOSPI_DCR2 PRESCALER bitfield is changed, the REFRESH field must be updated accordingly.

25.7.6 OCTOSPI status register (OCTOSPI_SR)

Address offset: 0x0020

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	FLEVEL[5:0]						Res.	BERRF	BUSY	TOF	SMF	FTF	TCF	TEF
		r	r	r	r	r	r		r	r	r	r	r	r	r

Bits 31:14 Reserved, must be kept at reset value.

Bits 13:8 **FLEVEL[5:0]**: FIFO level

This bitfield gives the number of valid bytes that are being held in the FIFO. FLEVEL = 0 when the FIFO is empty, and 32 when it is full.

In automatic status-polling mode, FLEVEL is zero.

Bit 7 Reserved, must be kept at reset value.

Bit 6 **BERRF**: Bus error flag

This bit is set when a bus error occurs. It is cleared by writing 1 to CBERRF. This bit has a meaning if Bus error bit feature is available in the product implementation table.

Bit 5 **BUSY**: Busy

This bit is set when an operation is ongoing. It is cleared automatically when the operation with the external device is finished and the FIFO is empty.

Bit 4 **TOF**: Timeout flag

This bit is set when timeout occurs. It is cleared by writing 1 to CTOF.

Bit 3 **SMF**: Status match flag

This bit is set in automatic status-polling mode when the unmasked received data matches the corresponding bits in the match register (OCTOSPI_PSMAR).
It is cleared by writing 1 to CSMF.

Bit 2 **FTF**: FIFO threshold flag

In indirect mode, this bit is set when the FIFO threshold has been reached, or if there is any data left in the FIFO after the reads from the external device are complete.

It is cleared automatically as soon as the threshold condition is no longer true.

In automatic status-polling mode, this bit is set every time the status register is read, and the bit is cleared when the data register is read.

Bit 1 **TCF**: Transfer complete flag

This bit is set in indirect mode when the programmed number of data has been transferred or in any mode when the transfer has been aborted. It is cleared by writing 1 to CTCF.

Bit 0 **TEF**: Transfer error flag

This bit is set in indirect mode when an invalid address is being accessed in indirect mode.
It is cleared by writing 1 to CTEF.

25.7.7 OCTOSPI flag clear register (OCTOSPI_FCR)

Address offset: 0x0024

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	CBERR F	Res.	CTOF	CSMF	Res.	CTCF	CTEF
									w		w	w		w	w

Bits 31:7 Reserved, must be kept at reset value.

Bit 6 **CBERRF**: Clear bus error flag

Writing 1 clears the BERRF flag in the XSPI_SR register (if Bus error bit feature is available in the product implementation table).

Bit 5 Reserved, must be kept at reset value.

Bit 4 **CTOF**: Clear timeout flag

Writing 1 clears the TOF flag in the OCTOSPI_SR register.

Bit 3 **CSMF**: Clear status match flag

Writing 1 clears the SMF flag in the OCTOSPI_SR register.

Bit 2 Reserved, must be kept at reset value.

Bit 1 **CTCF**: Clear transfer complete flag

Writing 1 clears the TCF flag in the OCTOSPI_SR register.

Bit 0 **CTEF**: Clear transfer error flag

Writing 1 clears the TEF flag in the OCTOSPI_SR register.

25.7.8 OCTOSPI data length register (OCTOSPI_DLR)

Address offset: 0x0040

Reset value: 0x0000 0000

This register can be modified only when BUSY = 0.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
DL[31:16]															
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
DL[15:0]															
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bits 31:0 **DL[31:0]**: Data length

Number of data to be retrieved (value+1) in indirect and automatic status-polling modes. A value not greater than three (indicating 4 bytes) must be used for automatic status-polling mode.

All 1's in indirect mode means undefined length, where OCTOSPI continues until the end of the memory, as defined by DEVSZ.

0x0000_0000: 1 byte is to be transferred.

0x0000_0001: 2 bytes are to be transferred.

0x0000_0002: 3 bytes are to be transferred.

0x0000_0003: 4 bytes are to be transferred.

...

0xFFFF_FFFD: 4,294,967,294 (4G-2) bytes are to be transferred.

0xFFFF_FFFE: 4,294,967,295 (4G-1) bytes are to be transferred.

0xFFFF_FFFF: undefined length; all bytes, until the end of the external device, (as defined by DEVSZ) are to be transferred. Continue reading indefinitely if DEVSZ = 0x1F.

DL[0] is stuck at 1 in dual-memory configuration (DMM = 1) even when 0 is written to this bit, thus assuring that each access transfers an even number of bytes.

This bitfield has no effect in memory-mapped mode.

25.7.9 OCTOSPI address register (OCTOSPI_AR)

Address offset: 0x0048

Reset value: 0x0000 0000

This register can be modified only when BUSY = 0 and FMODE ≠ 11.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
ADDRESS[31:16]															
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
ADDRESS[15:0]															
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bits 31:0 **ADDRESS[31:0]**: Address

Address to be sent to the external device. In HyperBus protocol, this field must be even as this protocol is 16-bit word oriented. In dual-memory configuration, AR[0] is forced to 0.

25.7.10 OCTOSPI data register (OCTOSPI_DR)

Address offset: 0x0050

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
DATA[31:16]															
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
DATA[15:0]															
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bits 31:0 **DATA[31:0]**: Data

Data to be sent/received to/from the external SPI device

In indirect-write mode, data written to this register is stored on the FIFO before it is sent to the external device during the data phase. If the FIFO is too full, a write operation is stalled until the FIFO has enough space to accept the amount of data being written.

In indirect-read mode, reading this register gives (via the FIFO) the data that was received from the external device. If the FIFO does not have as many bytes as requested by the read operation and if BUSY = 1, the read operation is stalled until enough data is present or until the transfer is complete, whichever happens first.

In automatic status-polling mode, this register contains the last data read from the external device (without masking).

Word, half-word, and byte accesses to this register are supported. In indirect-write mode, a byte write adds 1 byte to the FIFO, a half-word write 2 bytes, and a word write 4 bytes.

Similarly, in indirect-read mode, a byte read removes 1 byte from the FIFO, a halfword read 2 bytes, and a word read 4 bytes. Accesses in indirect mode must be aligned to the bottom of this register: A byte read must read DATA[7:0] and a half-word read must read DATA[15:0].

25.7.11 OCTOSPI polling status mask register (OCTOSPI_PSMKR)

Address offset: 0x0080

Reset value: 0x0000 0000

This register can be modified only when BUSY = 0.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
MASK[31:16]															
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
MASK[15:0]															
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bits 31:0 **MASK[31:0]**: Status mask

Mask to be applied to the status bytes received in automatic status-polling mode

For bit n:

0: Bit n of the data received in automatic status-polling mode is masked and its value is not considered in the matching logic.

1: Bit n of the data received in automatic status-polling mode is unmasked and its value is considered in the matching logic.

25.7.12 OCTOSPI polling status match register (OCTOSPI_PSMAR)

Address offset: 0x0088

Reset value: 0x0000 0000

This register can be modified only when BUSY = 0.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
MATCH[31:16]															
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
MATCH[15:0]															
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bits 31:0 **MATCH[31:0]**: Status match

Value to be compared with the masked status register to get a match

25.7.13 OCTOSPI polling interval register (OCTOSPI_PIR)

Address offset: 0x0090

Reset value: 0x0000 0000

This register can be modified only when BUSY = 0.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
INTERVAL[15:0]															
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bits 31:16 Reserved, must be kept at reset value.

Bits 15:0 **INTERVAL[15:0]**: Polling interval

Number of CLK cycles between a read during the automatic status-polling phases

25.7.14 OCTOSPI communication configuration register (OCTOSPI_CCR)

Address offset: 0x0100

Reset value: 0x0000 0000

This register can be modified only when BUSY = 0.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	DQSE	Res.	DDTR	DMODE[2:0]			Res.	Res.	ABSIZE[1:0]		ABDTR	ABMODE[2:0]		
		rw		rw	rw	rw	rw			rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	ADSIZE[1:0]		AD DTR	ADMODE[2:0]			Res.	Res.	ISIZE[1:0]		IDTR	IMODE[2:0]		
		rw	rw	rw	rw	rw	rw			rw	rw	rw	rw	rw	rw

Bits 31:30 Reserved, must be kept at reset value.

Bit 29 **DQSE**: DQS enable

This bit enables the data strobe management.

0: DQS disabled

1: DQS enabled

Bit 28 Reserved, must be kept at reset value.

Bit 27 **DDTR**: Data double transfer rate

This bit sets the DTR mode for the data phase.

0: DTR mode disabled for data phase

1: DTR mode enabled for data phase

Bits 26:24 **DMODE[2:0]**: Data mode

This bitfield defines the data phase mode of operation.

000: No data

001: Data on a single line

010: Data on two lines

011: Data on four lines

100: Data on eight lines

Others: Reserved

Bits 23:22 Reserved, must be kept at reset value.

Bits 21:20 **ABSIZE[1:0]**: Alternate-byte size

This bitfield defines the alternate-byte size.

00: 8-bit alternate bytes

01: 16-bit alternate bytes

10: 24-bit alternate bytes

11: 32-bit alternate bytes

Bit 19 **ABDTR**: Alternate- byte double transfer rate

This bit sets the DTR mode for the alternate-byte phase.

0: DTR mode disabled for the alternate-byte phase

1: DTR mode enabled for the alternate-byte phase

Bits 18:16 **ABMODE[2:0]**: Alternate-byte mode
This bitfield defines the alternate-byte phase mode of operation.
000: No alternate bytes
001: Alternate bytes on a single line
010: Alternate bytes on two lines
011: Alternate bytes on four lines
100: Alternate bytes on eight lines
Others: Reserved

Bits 15:14 Reserved, must be kept at reset value.

Bits 13:12 **ADSIZE[1:0]**: Address size
This bitfield defines the address size.
00: 8-bit address
01: 16-bit address
10: 24-bit address
11: 32-bit address

Bit 11 **ADDTR**: Address double transfer rate
This bit sets the DTR mode for the address phase.
0: DTR mode disabled for the address phase
1: DTR mode enabled for the address phase

Bits 10:8 **ADMODE[2:0]**: Address mode
This bitfield defines the address phase mode of operation.
000: No address
001: Address on a single line
010: Address on two lines
011: Address on four lines
100: Address on eight lines
Others: Reserved

Bits 7:6 Reserved, must be kept at reset value.

Bits 5:4 **ISIZE[1:0]**: Instruction size
This bitfield defines instruction size.
00: 8-bit instruction
01: 16-bit instruction
10: 24-bit instruction
11: 32-bit instruction

Bit 3 **IDTR**: Instruction double transfer rate
This bit sets the DTR mode for the instruction phase.
0: DTR mode disabled for the instruction phase
1: DTR mode enabled for the instruction phase

Bits 2:0 **IMODE[2:0]**: Instruction mode
This bitfield defines the instruction phase mode of operation.
000: No instruction
001: Instruction on a single line
010: Instruction on two lines
011: Instruction on four lines
100: Instruction on eight lines
Others: Reserved

25.7.15 OCTOSPI timing configuration register (OCTOSPI_TCR)

Address offset: 0x0108

Reset value: 0x0000 0000

This register can be modified only when BUSY = 0.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	SSHIFT	Res.	DHQC	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
	rw		rw												
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	DCYC[4:0]				
											rw	rw	rw	rw	rw

Bit 31 Reserved, must be kept at reset value.

Bit 30 **SSHIFT**: Sample shift

By default, the OCTOSPI samples data 1/2 of a CLK cycle after the data is driven by the external device.

This bit allows the data to be sampled later in order to consider the external signal delays.

0: No shift

1: 1/2 cycle shift

The software must ensure that SSHIFT = 0 when the data phase is configured in DTR mode (when DDTR = 1.)

Bit 29 Reserved, must be kept at reset value.

Bit 28 **DHQC**: Delay hold quarter cycle

0: No delay hold

1: 1/4 cycle hold

Bits 27:5 Reserved, must be kept at reset value.

Bits 4:0 **DCYC[4:0]**: Number of dummy cycles

This bitfield defines the duration of the dummy phase according to the memory latency.

In both SDR and DTR modes, it specifies a number of CLK cycles (0-31).

25.7.16 OCTOSPI instruction register (OCTOSPI_IR)

Address offset: 0x0110

Reset value: 0x0000 0000

This register can be modified only when BUSY = 0.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
INSTRUCTION[31:16]															
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
INSTRUCTION[15:0]															
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bits 31:0 **INSTRUCTION[31:0]**: Instruction
 Instruction to be sent to the external SPI device

25.7.17 OCTOSPI alternate bytes register (OCTOSPI_ABR)

Address offset: 0x0120

Reset value: 0x0000 0000

This register can be modified only when BUSY = 0.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
ALTERNATE[31:16]															
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
ALTERNATE[15:0]															
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bits 31:0 **ALTERNATE[31:0]**: Alternate bytes
 Optional data to be sent to the external SPI device right after the address.

25.7.18 OCTOSPI low-power timeout register (OCTOSPI_LPTR)

Address offset: 0x00130

Reset value: 0x0000 0000

This register can be modified only when BUSY = 0.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
TIMEOUT[15:0]															
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bits 31:16 Reserved, must be kept at reset value.

Bits 15:0 **TIMEOUT[15:0]**: Timeout period

After each access in memory-mapped mode, the OCTOSPI prefetches the subsequent bytes and hold them in the FIFO.

This bitfield indicates how many CLK cycles the OCTOSPI waits after the clock becomes inactive and until it raises the NCS, putting the external device in a lower-consumption state.

25.7.19 OCTOSPI wrap communication configuration register (OCTOSPI_WPCCR)

Address offset: 0x0140

Reset value: 0x0000 0000

This register can be modified only when BUSY = 0.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	DQSE	Res.	DDTR	DMODE[2:0]			Res.	Res.	ABSIZE[1:0]		ABDTR	ABMODE[2:0]		
		rw		rw	rw	rw	rw			rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	ADSIZE[1:0]		AD DTR	ADMODE[2:0]			Res.	Res.	ISIZE[1:0]		IDTR	IMODE[2:0]		
		rw	rw	rw	rw	rw	rw			rw	rw	rw	rw	rw	rw

Bits 31:30 Reserved, must be kept at reset value.

Bit 29 **DQSE**: DQS enable

This bit enables the data strobe management.

0: DQS disabled

1: DQS enabled

Bit 28 Reserved, must be kept at reset value.

Bit 27 **DDTR**: Data double transfer rate

This bit sets the DTR mode for the data phase.

0: DTR mode disabled for the data phase

1: DTR mode enabled for the data phase

Bits 26:24 **DMODE[2:0]**: Data mode

This bitfield defines the data phase mode of operation.

000: No data

001: Data on a single line

010: Data on two lines

011: Data on four lines

100: Data on eight lines

Others: Reserved

Bits 23:22 Reserved, must be kept at reset value.

Bits 21:20 **ABSIZE[1:0]**: Alternate-byte size

This bitfield defines the alternate-byte size.

00: 8-bit alternate bytes

01: 16-bit alternate bytes

10: 24-bit alternate bytes

11: 32-bit alternate bytes

Bit 19 **ABDTR**: Alternate-byte double transfer rate

This bit sets the DTR mode for the alternate-byte phase.

0: DTR mode disabled for the alternate-byte phase

1: DTR mode enabled for the alternate-byte phase

Bits 18:16 **ABMODE[2:0]**: Alternate-byte mode
This bitfield defines the alternate-byte phase mode of operation.
000: no alternate bytes
001: alternate bytes on a single line
010: alternate bytes on two lines
011: alternate bytes on four lines
100: alternate bytes on eight lines
Others: reserved

Bits 15:14 Reserved, must be kept at reset value.

Bits 13:12 **ADSIZE[1:0]**: Address size
This bitfield defines the address size.
00: 8-bit address
01: 16-bit address
10: 24-bit address
11: 32-bit address

Bit 11 **ADDTR**: Address double transfer rate
This bit sets the DTR mode for the address phase.
0: DTR mode disabled for address phase
1: DTR mode enabled for address phase

Bits 10:8 **ADMODE[2:0]**: Address mode
This bitfield defines the address phase mode of operation.
000: No address
001: Address on a single line
010: Address on two lines
011: Address on four lines
100: Address on eight lines
Others: Reserved

Bits 7:6 Reserved, must be kept at reset value.

Bits 5:4 **ISIZE[1:0]**: Instruction size
This bitfield defines the instruction size.
00: 8-bit instruction
01: 16-bit instruction
10: 24-bit instruction
11: 32-bit instruction

Bit 3 **IDTR**: Instruction double transfer rate
This bit sets the DTR mode for the instruction phase.
0: DTR mode disabled for the instruction phase
1: DTR mode enabled for the instruction phase

Bits 2:0 **IMODE[2:0]**: Instruction mode
This bitfield defines the instruction phase mode of operation.
000: No instruction
001: Instruction on a single line
010: Instruction on two lines
011: Instruction on four lines
100: Instruction on eight lines
Others: Reserved

25.7.20 OCTOSPI wrap timing configuration register (OCTOSPI_WPTCR)

Address offset: 0x0148

Reset value: 0x0000 0000

This register can be modified only when BUSY = 0.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	S SHIFT	Res.	DHQC	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
	rw		rw												
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	DCYC[4:0]				
											rw	rw	rw	rw	rw

Bit 31 Reserved, must be kept at reset value.

Bit 30 **SSHIFT**: Sample shift

By default, the OCTOSPI samples data 1/2 of a CLK cycle after the data is driven by the external device.

This bit allows the data to be sampled later in order to consider the external signal delays.

0: No shift

1: 1/2 cycle shift

The firmware must assure that SSHIFT=0 when the data phase is configured in DTR mode (when DDTR = 1).

Bit 29 Reserved, must be kept at reset value.

Bit 28 **DHQC**: Delay hold quarter cycle

Add a quarter cycle delay on the outputs in DTR communication to match hold requirement.

0: No quarter cycle delay

1: 1/4 cycle delay inserted

Bits 27:5 Reserved, must be kept at reset value.

Bits 4:0 **DCYC[4:0]**: Number of dummy cycles

This bitfield defines the duration of the dummy phase according to the memory latency.

In both SDR and DTR modes, it specifies a number of CLK cycles (0-31).

25.7.21 OCTOSPI wrap instruction register (OCTOSPI_WPIR)

Address offset: 0x0150

Reset value: 0x0000 0000

This register can be modified only when BUSY = 0.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
INSTRUCTION[31:16]															
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
INSTRUCTION[15:0]															
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bits 31:0 **INSTRUCTION[31:0]**: Instruction
 Instruction to be sent to the external SPI device

25.7.22 OCTOSPI wrap alternate bytes register (OCTOSPI_WPABR)

Address offset: 0x0160

Reset value: 0x0000 0000

This register can be modified only when BUSY = 0.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
ALTERNATE[31:16]															
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
ALTERNATE[15:0]															
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bits 31:0 **ALTERNATE[31:0]**: Alternate bytes
 Optional data to be sent to the external SPI device right after the address

25.7.23 OCTOSPI write communication configuration register (OCTOSPI_WCCR)

Address offset: 0x0180

Reset value: 0x0000 0000

This register can be modified only when BUSY = 0. Its content has a meaning only when requesting write operations in memory-mapped mode.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	DQSE	Res.	DDTR	DMODE[2:0]		Res.	Res.	ABSIZE[1:0]		ABDTR	ABMODE[2:0]			
		rw		rw	rw	rw	rw			rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	ADSIZE[1:0]		ADDTR	ADMODE[2:0]		Res.	Res.	ISIZE[1:0]		IDTR	IMODE[2:0]			
		rw	rw	rw	rw	rw	rw			rw	rw	rw	rw	rw	rw

Bits 31:30 Reserved, must be kept at reset value.

Bit 29 **DQSE**: DQS enable
 This bit enables the data strobe management.
 0: DQS disabled
 1: DQS enabled

Bit 28 Reserved, must be kept at reset value.

Bit 27 **DDTR**: data double transfer rate
 This bit sets the DTR mode for the data phase.
 0: DTR mode disabled for the data phase
 1: DTR mode enabled for the data phase

- Bits 26:24 **DMODE[2:0]**: Data mode
This bitfield defines the data phase mode of operation.
000: No data
001: Data on a single line
010: Data on two lines
011: Data on four lines
100: Data on eight lines
Others: Reserved
- Bits 23:22 Reserved, must be kept at reset value.
- Bits 21:20 **ABSIZE[1:0]**: Alternate-byte size
This bitfield defines the alternate-byte size.
00: 8-bit alternate bytes
01: 16-bit alternate bytes
10: 24-bit alternate bytes
11: 32-bit alternate bytes
- Bit 19 **ABDTR**: Alternate bytes double transfer rate
This bit sets the DTR mode for the alternate-bytes phase.
0: DTR mode disabled for alternate-bytes phase
1: DTR mode enabled for alternate-bytes phase
- Bits 18:16 **ABMODE[2:0]**: Alternate-byte mode
This bitfield defines the alternate-byte phase mode of operation.
000: No alternate bytes
001: Alternate bytes on a single line
010: Alternate bytes on two lines
011: Alternate bytes on four lines
100: Alternate bytes on eight lines
Others: Reserved
- Bits 15:14 Reserved, must be kept at reset value.
- Bits 13:12 **ADSIZE[1:0]**: Address size
This bitfield defines the address size.
00: 8-bit address
01: 16-bit address
10: 24-bit address
11: 32-bit address
- Bit 11 **ADDTR**: Address double transfer rate
This bit sets the DTR mode for the address phase.
0: DTR mode disabled for the address phase
1: DTR mode enabled for the address phase
- Bits 10:8 **ADMODE[2:0]**: Address mode
This bitfield defines the address phase mode of operation.
000: No address
001: Address on a single line
010: Address on two lines
011: Address on four lines
100: Address on eight lines
Others: Reserved
- Bits 7:6 Reserved, must be kept at reset value.

Bits 5:4 **ISIZE[1:0]**: Instruction size

This bitfield defines the instruction size.

00: 8-bit instruction

01: 16-bit instruction

10: 24-bit instruction

11: 32-bit instruction

Bit 3 **IDTR**: Instruction double transfer rate

This bit sets the DTR mode for the instruction phase.

0: DTR mode disabled for instruction phase

1: DTR mode enabled for instruction phase

Bits 2:0 **IMODE[2:0]**: Instruction mode

This bitfield defines the instruction phase mode of operation.

000: No instruction

001: Instruction on a single line

010: Instruction on two lines

011: Instruction on four lines

100: Instruction on eight lines

Others: Reserved

25.7.24 OCTOSPI write timing configuration register (OCTOSPI_WTCR)

Address offset: 0x0188

Reset value: 0x0000 0000

This register can be modified only when BUSY = 0. Its content has a meaning only when requesting write operations in memory-mapped mode.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	DCYC[4:0]				
											rw	rw	rw	rw	rw

Bits 31:5 Reserved, must be kept at reset value.

Bits 4:0 **DCYC[4:0]**: Number of dummy cycles

This bitfield defines the duration of the dummy phase according to the memory latency.

In both SDR and DTR modes, it specifies a number of CLK cycles (0-31).

25.7.25 OCTOSPI write instruction register (OCTOSPI_WIR)

Address offset: 0x0190

Reset value: 0x0000 0000

This register can be modified only when BUSY = 0. Its content has a meaning only when requesting write operations in memory-mapped mode.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
INSTRUCTION[31:16]															
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
INSTRUCTION[15:0]															
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bits 31:0 **INSTRUCTION[31:0]**: Instruction
Instruction to be sent to the external SPI device

25.7.26 OCTOSPI write alternate bytes register (OCTOSPI_WABR)

Address offset: 0x01A0

Reset value: 0x0000 0000

This register can be modified only when BUSY = 0. Its content has a meaning only when requesting write operations in memory-mapped mode.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
ALTERNATE[31:16]															
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
ALTERNATE[15:0]															
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bits 31:0 **ALTERNATE[31:0]**: Alternate bytes
Optional data to be sent to the external SPI device right after the address

25.7.27 OCTOSPI HyperBus latency configuration register (OCTOSPI_HLCR)

Address offset: 0x0200

Reset value: 0x0000 0000

This register can be modified only when BUSY = 0.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	TRWR[7:0]							
								rw	rw	rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
TACC[7:0]								Res.	Res.	Res.	Res.	Res.	Res.	WZL	LM
rw	rw	rw	rw	rw	rw	rw	rw							rw	rw

Bits 31:24 Reserved, must be kept at reset value.

Bits 23:16 **TRWR[7:0]**: Read-write minimum recovery time

Device read-to-write/write-to-read minimum recovery time expressed in number of communication clock cycles

Bits 15:8 **TACC[7:0]**: Access time

Device access time according to the memory latency, expressed in number of communication clock cycles

Bits 7:2 Reserved, must be kept at reset value.

Bit 1 **WZL**: Write zero latency

This bit enables zero latency on write operations.

0: Latency on write accesses

1: No latency on write accesses

Bit 0 **LM**: Latency mode

This bit selects the latency mode.

0: Variable initial latency

1: Fixed latency

Note: This bit must be kept cleared for HyperFLASH memory.

25.7.28 OCTOSPI register map

Table 247. OCTOSPI register map and reset values

Offset	Register name	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0x0000	OCTOSPI_CR	Res.	MSEL	FMDOE[1:0]		Res.	Res.	NOPREFI	CSSEL	PMM	APMS	BERRIE	TOIE	SMIE	FTIE	TCIE	TEIE	Res.	Res.	FTHRES[5:0]					Res.	DMM	Res.	ADOFFEN	TCEN	DMAEN	ABORT	EN	
	Reset value		0	0	0			0	0	0	0	0	0	0	0	0	0			0	0	0	0	0	0		0		0	0	0	0	0
0x0004	Reserved	Reserved																															

Table 247. OCTOSPI register map and reset values (continued)

Offset	Register name	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0x0008	OCTOSPI_DCR1	ADOFF[4:0]					MTYP [2:0]			Res.	Res.	EXTENDMEM	DEVSIZE[4:0]				Res.	Res.	CSHT[5:0]					Res.	Res.	Res.	Res.	DLYBYP	Res.	FRCK	CKMODE		
	Reset value	0	0	0	0	0	0	0	0			0	0	0	0	0	0			0	0	0	0	0	0	0					0		0
0x000C	OCTOSPI_DCR2	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	WRAPSIZE [2:0]			Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	PRESCALER[7:0]							
	Reset value														0	0	0									0	0	0	0	0	0	0	0
0x0010	OCTOSPI_DCR3	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	CSBOUND[4:0]				Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
	Reset value												0	0	0	0	0																
0x0014	OCTOSPI_DCR4	REFRESH[31:0]																															
	Reset value	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0x0018-0x001C	Reserved	Reserved																															
0x0020	OCTOSPI_SR	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	FLEVEL[5:0]					Res.	BERRF	BUSY	TOF	SMF	FTF	TCF	TEF		
	Reset value																			0	0	0	0	0	0		0	0	0	0	0	0	0
0x0024	OCTOSPI_FCR	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	CBERRF	Res.	CTOF	CSMF	Res.	CTCF	CTEF
	Reset value																									0		0	0		0	0	0
0x0028-0x003C	Reserved	Reserved																															
0x0040	OCTOSPI_DLR	DL[31:0]																															
	Reset value	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0x0044	Reserved	Reserved																															
0x0048	OCTOSPI_AR	ADDRESS[31:0]																															
	Reset value	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0x004C	Reserved	Reserved																															
0x0050	OCTOSPI_DR	DATA[31:0]																															
	Reset value	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0x0054-0x007C	Reserved	Reserved																															
0x0080	OCTOSPI_PSMKR	MASK[31:0]																															
	Reset value	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0x0084	Reserved	Reserved																															
0x0088	OCTOSPI_PSMAR	MATCH[31:0]																															
	Reset value	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Table 247. OCTOSPI register map and reset values (continued)

[illegible]

Table 247. OCTOSPI register map and reset values (continued)

Offset	Register name	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0x0164-0x017C	Reserved	Reserved																															
0x0180	OCTOSPI_WCCR	Res.	Res.	DQSE	Res.	DDTR	DMODE [2:0]		Res.	Res.	ABSIZE [1:0]		ABDTR	ABMODE [2:0]		Res.	Res.	ADSIZE [1:0]		ADDTR	ADMODE [2:0]		Res.	Res.	ISIZE [1:0]		IDTR	IMODE [2:0]					
	Reset value	0		0		0	0	0	0			0	0	0	0	0	0			0	0	0	0	0	0			0	0	0	0	0	0
0x0184	Reserved	Reserved																															
0x0188	OCTOSPI_WTCR	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	DCYC[4:0]					
	Reset value																												0	0	0	0	0
0x018C	Reserved	Reserved																															
0x0190	OCTOSPI_WIR	INSTRUCTION[31:0]																															
	Reset value	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0x0194-0x019C	Reserved	Reserved																															
0x01A0	OCTOSPI_WABR	ALTERNATE[31:0]																															
	Reset value	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0x01A4-0x01FC	Reserved	Reserved																															
0x0200	OCTOSPI_HLCR	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	TRWR[7:0]							TACC[7:0]							Res.	Res.	Res.	Res.	Res.	Res.	Res.	WZL	LM	
	Reset value										0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0							0

Refer to [Section 2.3: Memory organization](#) for the register boundary addresses.

26 Secure digital input/output MultiMediaCard interface (SDMMC)

26.1 SDMMC main features

The SD/SDIO, embedded MultiMediaCard (e•MMC) host interface (SDMMC) provides an interface between the AHB bus and SD memory cards, SDIO cards and e•MMC devices.

The MultiMediaCard system specifications are available through the MultiMediaCard Association website at www.jedec.org, published by the MMCA technical committee.

SD memory card and SD I/O card system specifications are available through the SD card Association website at www.sdcard.org.

The SDMMC features include the following:

- Compliance with *Embedded MultiMediaCard System Specification Version 5.1*. Card support for three different databus modes: 1-bit (default), 4-bit and 8-bit. (HS200 SDMMC_CLK speed limited to maximum allowed I/O speed)(HS400 is not supported).
- Full compatibility with previous versions of MultiMediaCards (backward compatibility).
- Full compliance with *SD memory card specifications version 6.0*. (SDR104 SDMMC_CLK speed limited to maximum allowed I/O speed, SPI mode and UHS-II mode not supported).
- Full compliance with *SDIO card specification version 4.0*. Card support for two different databus modes: 1-bit (default) and 4-bit. (SDR104 SDMMC_CLK speed limited to maximum allowed I/O speed, SPI mode and UHS-II mode not supported).
- Data transfer up to 208 Mbyte/s for the 8-bit mode. (depending maximum allowed I/O speed).
- Data and command output enable signals to control external bidirectional drivers.
- IDMA linked list support

The MultiMediaCard/SD bus connects cards to the host.

The current version of the SDMMC supports only one SD/SDIO/e•MMC card at any one time and a stack of e•MMC.

26.2 SDMMC implementation

Table 248. SDMMC instances on device⁽¹⁾

Devices	SDMMC1	SDMMC2
STM32H562/563/573xx	X	X
STM32H523/533xx	X	-
STM32H543/553xx	X	-

1. X = instance is available.

Table 249. SDMMC features

SDMMC modes/features ⁽¹⁾	SDMMC1	SDMMC2
Variable delay (SRD104, HS200)	X	X
SDMMC_CKIN	X	X
SDMMC_CDIR, SDMMC_D0DIR	X	-
SDMMC_D123DIR	X	-

1. X = supported.

26.3 SDMMC bus topology

Communication over the bus is based on command/response and data transfers.

The basic transaction on the SD/SDIO/eMMC bus is the command/response transaction. These types of bus transaction transfer their information directly within the command or response structure. In addition, some operations have a data token.

Data transfers are done in the following ways:

- Block mode: data block(s) with block size 2^N bytes with N in the range 0-14
- SDIO multibyte mode: single data block with block size range 1-512 bytes
- eMMC Stream mode: continuous data stream

Data transfers to/from eMMC cards are done in data blocks or streams.

Figure 201. SDMMC “no response” and “no data” operations

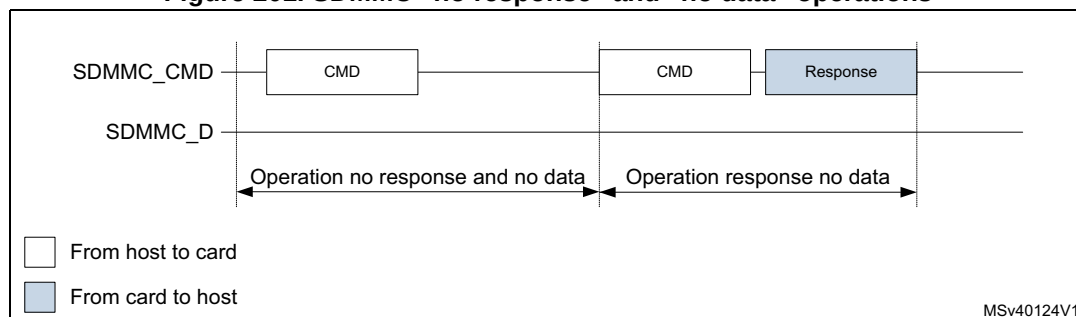
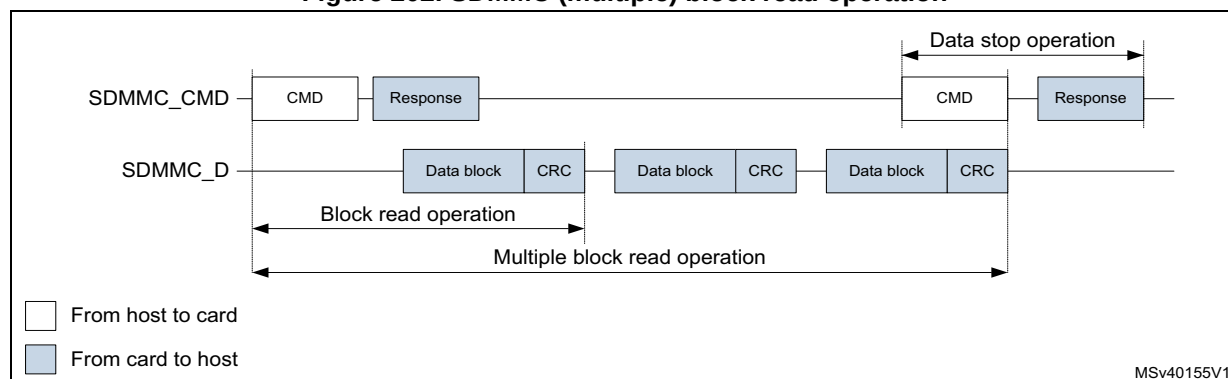
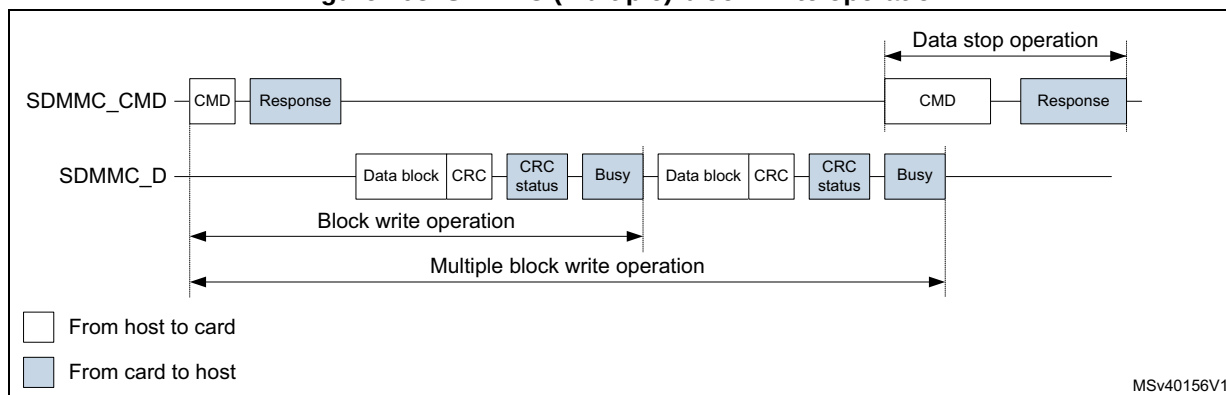


Figure 202. SDMMC (multiple) block read operation



Note: The Stop Transmission command is not required at the end of a eMMC multiple block read with predefined block count.

Figure 203. SDMMC (multiple) block write operation



Note: The Stop Transmission command is not required at the end of an eMMC multiple block write with predefined block count.

The SDMMC does not send any data as long as the Busy signal is asserted (SDMMC_D0 pulled low).

Figure 204. SDMMC (sequential) stream read operation

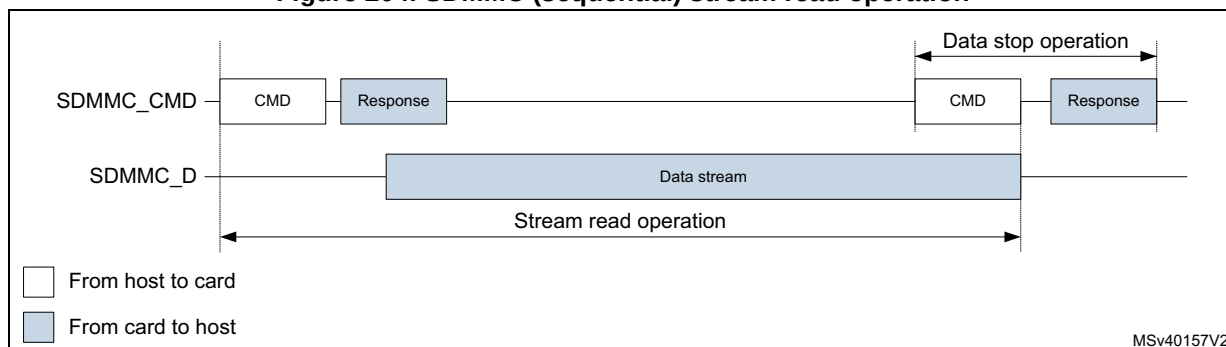
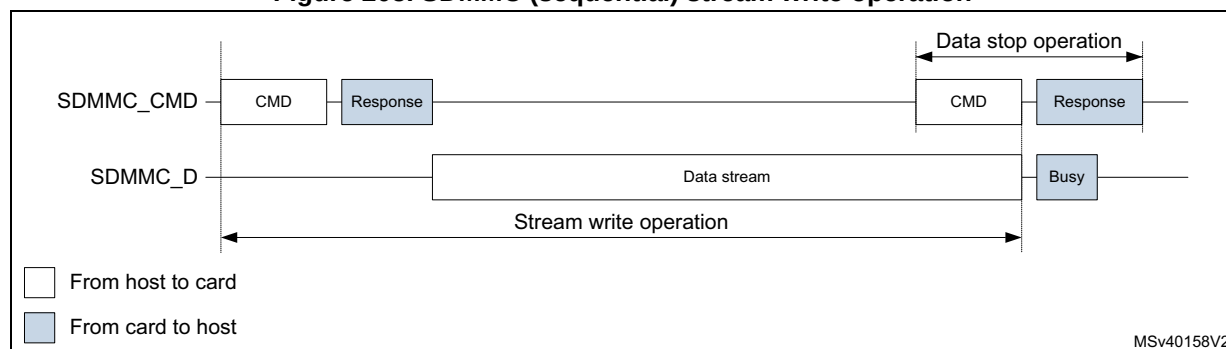


Figure 205. SDMMC (sequential) stream write operation



Stream data transfer operates only in a 1-bit wide bit bus configuration on SDMMC_D0 in single data rate modes (DS, HS, and SDR).

26.4 SDMMC operation modes

Table 250. SDMMC operation modes SD and SDIO

SDIO bus speed modes ⁽¹⁾⁽²⁾	Max bus speed ⁽³⁾ [Mbyte/s]	Max clock frequency [MHz] ⁽⁴⁾	Signal voltage [V]
DS (default speed)	12.5	25	3.3
HS (high speed)	25	50	3.3
SDR12	12.5	25	1.8
SDR25	25	50	1.8
DDR50	50	50	1.8
SDR50	50	100	1.8
SDR104	104	208	1.8

1. SDR single data rate signaling.
2. DDR double data rate signaling (data is sampled on both SDMMC_CLK clock edges).
3. SDIO bus speed with 4-bit bus width.
4. Maximum frequency depending on maximum allowed I/O speed.

SDR104 mode requires variable delay support using sampling point tuning. The use of variable delay is optional for SDR50 mode.

Table 251. SDMMC operation modes eMMC

eMMC bus speed modes ⁽¹⁾⁽²⁾	Max bus speed ⁽³⁾ [Mbyte/s]	Max clock frequency [MHz] ⁽⁴⁾	Signal voltage [V] ⁽⁵⁾
Legacy compatible	26	26	3/1.8/1.2V
High speed SDR	52	52	3/1.8/1.2V
High speed DDR	104	52	3/1.8/1.2V
High speed HS200	200	200	1.8/1.2V

1. SDR single data rate signaling.
2. DDR double data rate signaling. (data is sampled on both SDMMC_CLK clock edges).
3. eMMC bus speed with 8-bit bus width.
4. Maximum frequency depending on maximum allowed I/O speed.
5. Supported signal voltage level depends on I/O port characteristics, refer to device datasheet.

HS200 mode requires variable delay support using sampling point tuning.

26.5 SDMMC functional description

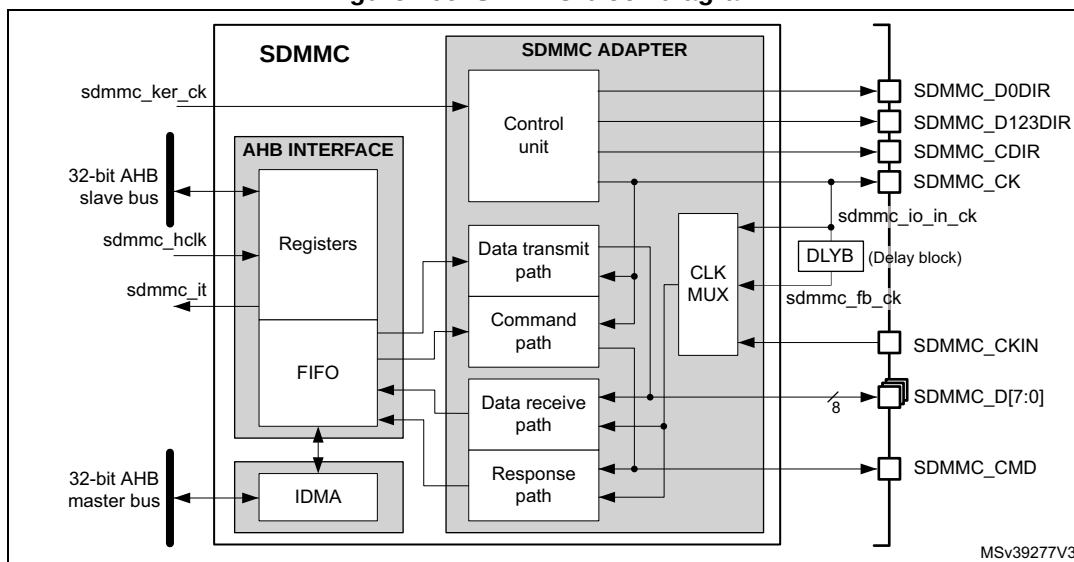
The SDMMC consists of four parts:

- The AHB slave interface accesses the SDMMC adapter registers, and generates interrupt signals and IDMA control signals.
- The SDMMC adapter block provides all functions specific to the eMMC/SD/SD I/O card such as the clock generation unit, command and data transfer.
- The internal DMA (IDMA) block with its AHB master interface.
- A delay block (DLYB) taking care of the receive data sample clock alignment. The delay block is NOT part of the SDMMC. A delay block is mandatory when supporting SDR104 or HS200.

26.5.1 SDMMC block diagram

[Figure 206](#) shows the SDMMC block diagram.

Figure 206. SDMMC block diagram



26.5.2 SDMMC pins and internal signals

[Table 252](#) lists the SDMMC internal input/output signals, [Table 253](#) the SDMMC pins (alternate functions).

Table 252. SDMMC internal input/output signals

Signal name	Signal type	Description
sdmmc_ker_ck	Digital input	SDMMC kernel clock
sdmmc_hclk	Digital input	AHB clock
sdmmc_it	Digital output	SDMMC global interrupt

Table 252. SDMMC internal input/output signals (continued)

Signal name	Signal type	Description
sdmmc_io_in_ck	Digital input	SD/SDIO/e•MMC card feedback clock. This signal is internally connected to the SDMMC_CK pin (for DS and HS modes).
sdmmc_fb_ck	Digital input	SD/SDIO/e•MMC card tuned feedback clock after DLYB delay block (for SDR50, DDR50, SDR104, HS200)

Table 253. SDMMC pins

Pin name	Pin type	Description
SDMMC_CK	Digital output	Clock to SD/SDIO/e•MMC card
SDMMC_CKIN	Digital input	Clock feedback from an external driver for SD/SDIO/e•MMC card. (for SDR12, SDR25, SDR50, DDR50)
SDMMC_CMD	Digital input/output	SD/SDIO/e•MMC card bidirectional command/response signal.
SDMMC_CD1R	Digital output	SD/SDIO/e•MMC card I/O direction indication for the SDMMC_CMD signal.
SDMMC_D[7:0]	Digital input/output	SD/SDIO/e•MMC card bidirectional data lines.
SDMMC_D0DIR	Digital output	SD/SDIO/e•MMC card I/O direction indication for the SDMMC_D0 data line.
SDMMC_D123DIR	Digital output	SD/SDIO/e•MMC card I/O direction indication for the data lines SDMMC_D[3:1].

26.5.3 General description

The **SDMMC_D[7:0]** lines have different operating modes:

- By default, SDMMC_D0 line is used for data transfer. After initialization, the host can change the databus width.
- For an e•MMC, 1-bit (SDMMC_D0), 4-bit (SDMMC_D[3:0]) or 8-bit (SDMMC_D[7:0]) data bus widths can be used.
- For an SD or an SDIO card, 1-bit (SDMMC_D0) or 4-bit (SDMMC_D[3:0]) can be used. All data lines operate in push-pull mode.

To allow the connection of an external driver (a voltage switch transceiver), the direction of data flow on the data lines is indicated with I/O direction signals. The **SDMMC_D0DIR** signal indicates the I/O direction for the SDMMC_D0 data line, the **SDMMC_D123DIR** for the SDMMC_D[3:1] data lines.

SDMMC_CMD only operates in push-pull mode:

To allow the connection of an external driver (a voltage switch transceiver), the direction of data flow on the SDMMC_CMD line is indicated with the I/O direction signal **SDMMC_CD1R**.

SDMMC_CK clock to the card originates from **sdmmc_ker_ck**:

- When the **sdmmc_ker_ck** clock has 50 % duty cycle, it can be used even in bypass mode (CLKDIV = 0).
- When the **sdmmc_ker_ck** duty cycle is not 50 %, the CLKDIV must be used to divide it by 2 or more (CLKDIV > 0).
- The phase relation between the SDMMC_CMD / SDMMC_D[7:0] outputs and the SDMMC_CK can be selected through the NEGEDGE bit. The phase relation depends on the CLKDIV, NEGEDGE, and DDR settings. See [Figure 207](#).

Figure 207. SDMMC Command and data phase relation

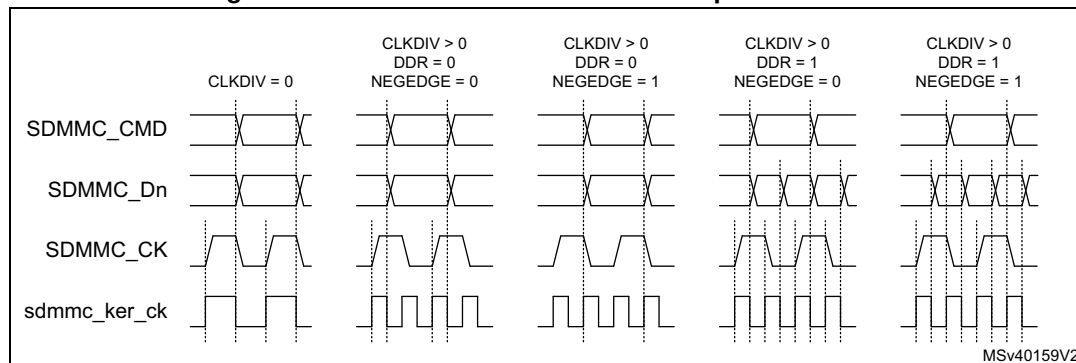


Table 254. SDMMC Command and data phase selection

CLKDIV	DDR	NEGEDGE	SDMMC_CK	Command out	Data out
0	x	x	= sdmmc_ker_ck	Generated on sdmmc_ker_ck falling edge	
>0	0	0	Generated on sdmmc_ker_ck rising edge	Generated on sdmmc_ker_ck falling edge succeeding the SDMMC_CK rising edge.	
		1		Generated on the same sdmmc_ker_ck rising edge that generates the SDMMC_CK falling edge.	
	1	0		Generated on sdmmc_ker_ck falling edge succeeding the SDMMC_CK rising edge.	Generated on sdmmc_ker_ck falling edge succeeding a SDMMC_CK edge.
		1		Generated on the same sdmmc_ker_ck rising edge that generates the SDMMC_CK falling edge.	

By default, the **sdmmc_io_in_ck** feedback clock input is selected for sampling incoming data in the SDMMC receive path. It is derived from the SDMMC_CK pin.

For tuning the phase of the sampling clock to accommodate the receive data timing, the DLYB delay block available on the device can be connected between **sdmmc_io_in_ck** signal (DLYB input dlyb_in_ck) and **sdmmc_fb_ck** clock input of SDMMC (DLYB output dlyb_out_ck). Selecting the **sdmmc_fb_ck** clock input in the receive path then enables using the phase-tuned sampling clock for the incoming data. This is required for SDMMC to support the SDR104 and HS200 operating mode and optional for SDR50 and DDR50 modes.

When using an external driver (a voltage switch transceiver), the SDMMC_CKIN feedback clock input can be selected to sample the receive data.

For an SD/SDIO/eMMC card, the clock frequency can vary between 0 and 208 MHz (limited by maximum I/O speed).

Depending on the selected bus mode (SDR or DDR), one bit or two bits are transferred on SDMMC_D[7:0] lines with each clock cycle. The SDMMC_CMD line transfers only one bit per clock cycle.

26.5.4 SDMMC adapter

The SDMMC adapter (see [Figure 206: SDMMC block diagram](#)) is a multimedia/secure digital memory card bus master that provides an interface to a MultiMediaCard stack or to a secure digital memory card. It consists of the following subunits:

- Control unit
- Data transmit path
- Command path
- Data receive path
- Response path
- Receive data path clock multiplexer
- Delay block (DLYB), external to the SDMMC
- Adapter register block
- Data FIFO
- Internal DMA (IDMA)

Note: The adapter registers and FIFO use the AHB clock domain (`sdmmc_hclk`). The control unit, command path and data transmit path use the SDMMC adapter clock domain (`sdmmc_ker_ck`). The response path and data receive path use the SDMMC adapter feedback clock domain from the `sdmmc_io_in_ck`, or `SDMMC_CKIN`, or from the `sdmmc_fb_ck` generated by DLYB.

The DLYB delay block on the device can be used in conjunction with the SDMMC adapter, to tune the phase of the sampling clock for incoming data in SDMMC receive mode. It is required for the SDMMC to support the SDR104 and HS200 operating mode and optional for SDR50 and DDR50 modes.

Adapter register block

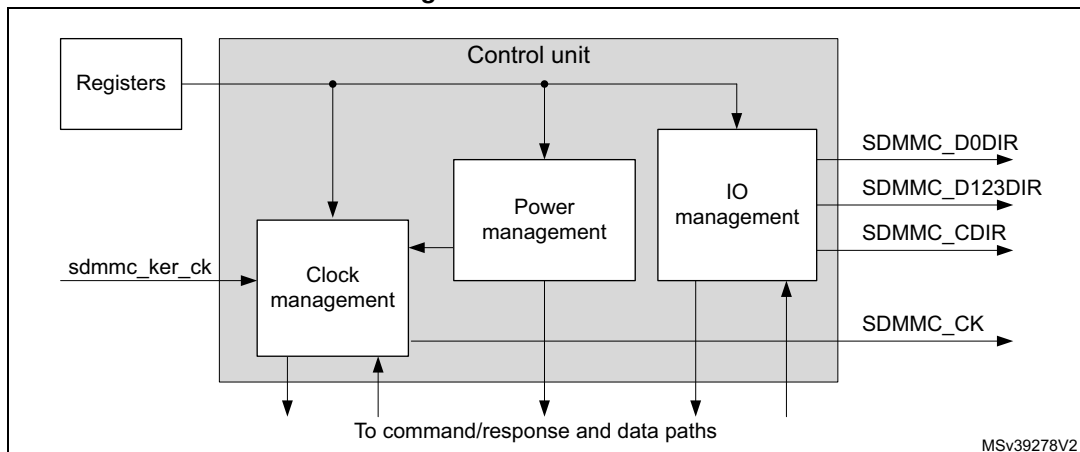
The adapter register block contains all system control registers, the SDMMC command and response registers and the data FIFO.

This block also generates the signals from the corresponding bit location in the SDMMC Clear register that clear the static flags in the SDMMC adapter.

Control unit

The control unit illustrated in [Figure 208](#), contains the power management functions, the SDMMC_CK clock management with divider, and the I/O direction management.

Figure 208. Control unit



The power management subunit disables the card bus output signals during the power-off and power-up phases.

There are three power phases:

- power-off
- power-up
- power-on

The clock management subunit uses the `sdmmc_ker_ck` to generate the `SDMMC_CK` and provides the division control. It also takes care of stopping the `SDMMC_CK` for flow control, for example.

The clock outputs are inactive:

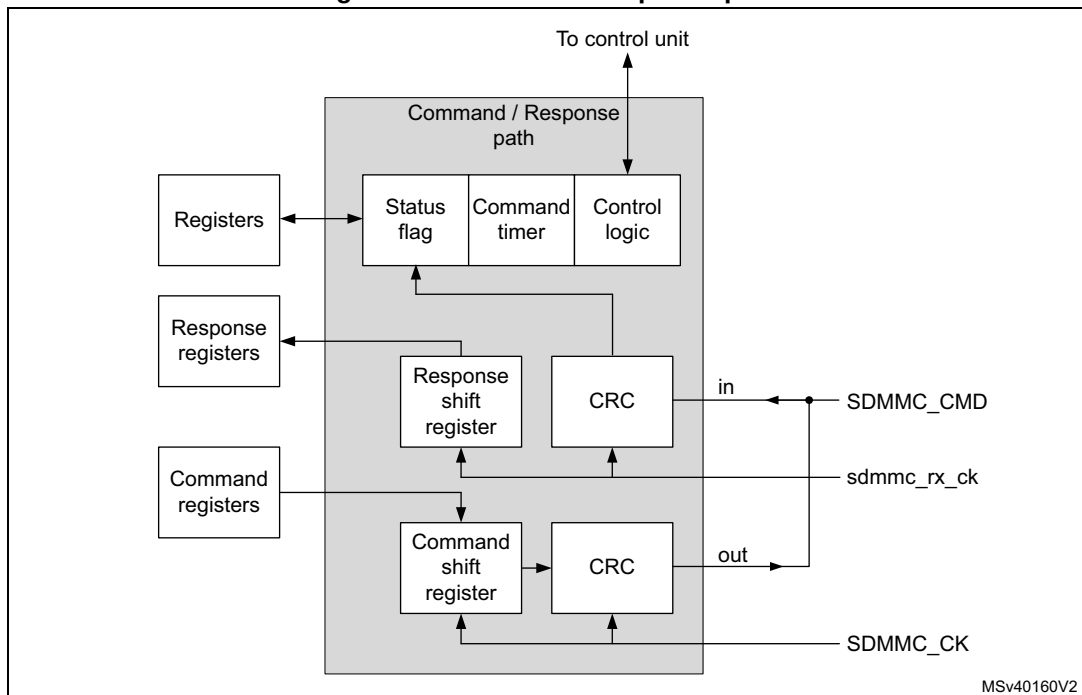
- after reset
- during the power-off or power-up phases
- if the power saving mode (register bit `PWRSAPV`) is enabled and the card bus is in the Idle state for eight clock periods. The clock is stopped eight cycles after both the command/response CPSM and data path DPSM subunits have entered the Idle phase. The clock is restarted when the command/response CPSM or data path DPSM is activated (enabled).

The I/O management subunit takes care of the `SDMMC_Dn` and `SDMMC_CMD` I/O direction signals, which controls the external voltage transceiver.

Command/response path

The command/response path subunit transfers commands and responses on the `SDMMC_CMD` line. The command path is clocked on the `SDMMC_CK` and sends commands to the card. The response path is clocked on the `sdmmc_rx_ck` and receives responses from the card.

Figure 209. Command/response path

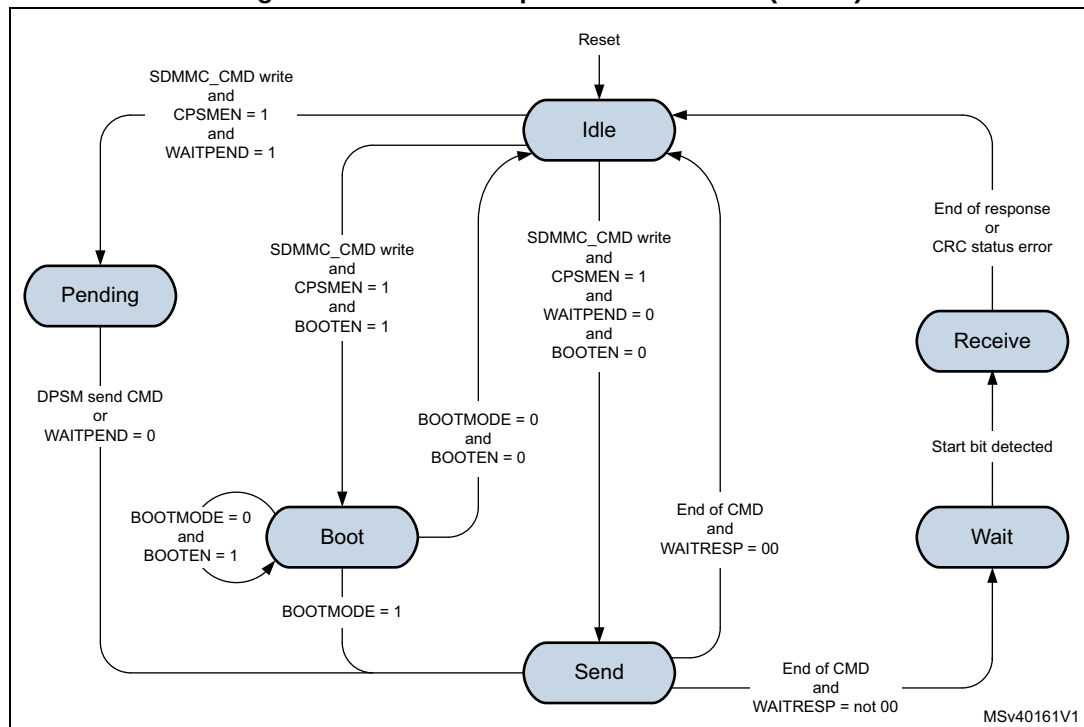


Command/response path state machine (CPSM):

- When the command register is written to and the enable bit is set, command transfer starts. When the command has been sent the CRC is appended and the command path state machine (CPSM) sets the status flags and:
 - if a response is not required enters the Idle state.
 - If a response is required, it waits for the response.
- When the response is received,
 - for a response with CRC, the received CRC code and the internally generated code are compared, and the appropriate status flag is set according the result.
 - for a response without CRC, no CRC is checked, and the appropriate status flag is not set.

When ever the CPSM is active (not in the Idle state), the CPSMACT bit is set.

Figure 210. Command path state machine (CPSM)



- Idle:** The command path is inactive. When the command control register is written and the enable bit (CPSMEN) is set, the CPSM activates the SDMMC_CLK clock (when stopped due to power save PWRSV bit) and moves
 - to the Send state when WAITPEND = 0 and BOOTEN = 0.
 - to the Pending state when WAITPEND = 1.
 - to the Boot state when BOOTEN = 1.
- Send:** The command is sent and the CRC is appended.
 - When CMDTRANS bit is set or when BOOTEN bit is set and BOOTMODE is alternative boot, and the DTDIR = receive, the CPSM DataEnable signal is issued to the DPSM at the end of the command.
 - When the CMDTRANS bit is set and the CMDSUSPEND bit is 0 the interrupt period is terminated at the end of the command.
 - When CMDSTOP bit is set the CPSM Abort signal is issued to the DPSM at the end of the command.
 - If no response is expected (WAITRESP = 00) the CPSM moves to the Idle state and the CMDSSENT flag is set. When BOOTMODE = 1 and BOOTEN = 0 the CMDSSENT flag is delayed 56 cycles after the command end bit, otherwise the

- CMDSENT flag is generated immediately after the command end bit.
The RESPCMDR and RESPxR registers are not modified.
- If a command response is expected (WAITRESP = not 00) the CPSM moves to the Wait state and start the response timeout.
 - **Wait:** The command path waits for a response.
 - When WAITINT bit is 0 the command timer starts running and the CPSM waits for a start bit.
 - a) If a start bit is detected before the timeout the CPSM moves to the Receive state.
 - b) If the timeout is reached before the CPSM detect a response start bit, the timeout flag (CTIMEOUT) is set and the CPSM moves to the Idle state.
The RESPCMDR and RESPxR registers are not modified.
 - When WAITINT bit is 1, the timer is disabled and the CPSM waits for an interrupt request (response start bit) from one of the cards.
 - a) When a start bit is detected the CPSM moves to the Receive state.
 - b) When writing WAITINT to 0 (interrupt mode abort), the host sends a response by its self and on detecting the start bit the CPSM move to the Receive state.
 - **Receive:** The command response is received. Depending the response mode bits WAITRESP in the command control register, the response can be either short or long, with CRC or without CRC. The received CRC code when present is verified against the internally generated CRC code.
 - When the CMDSUSPEND bit is set and the SDIO Response bit BS = 0 (response bit [39]), the interrupt period is started after the response.
When the CMDSUSPEND bit is cleared, or the CMDSUSPEND bit is 1 and the SDIO Response bit BS = 1 (response bit [39]), there is no interrupt period started.
 - When the CMDTRANS bit is set and the CMDSUSPEND bit is set and the SDIO Response bit DF= 1 (response bit [32]) the interrupt period is terminated after the response.
 - When the CRC status passes or no CRC is present the CMDREND flag is set, the CPSM moves to the Idle state.
The RESPCMDR and RESPxR registers are updated with received response.
 - When BOOTMODE = 1 and BOOTEN = 0 the CMDREND flag is delayed 56 cycles after the response end bit, otherwise the CMDREND flag is generated immediately after the response end bit.
 - When CMDTRANS bit is set and the DTDIR = transmit, the CPSM DataEnable signal is issued to the DPSM at the end of the command response.
 - When the CRC status fails the CCRCFAIL flag is set and the CPSM moves to the Idle state.
The RESPCMDR and RESPxR registers are updated with received response.
 - **Pending:** According the pending WAITPEND bit in the command register, the CPSM enters the pending state.
 - When DATALENGTH ≤ 5 bytes the CPSM moves to the Sent state and generates the DataEnable signal to start the data transfer aligned with the CMD12 Stop Transmission command.
 - When DATALENGTH > 5 bytes, the CPSM DataEnable signal is issued to the DPSM to start the data transfer. The CPSM waits for a send CMD signal from the

DPSM before moving to the Send state. This enables, for example, the CMD12 Stop Transmission command to be sent aligned with the data.

- When writing WAITPEND to 0, the CPSM moves to the Send state.
- **Boot:** If the BOOTEN bit is set in the command register, the CPSM enters the Boot state, and when:
 - BOOTMODE = 0 the SDMMC_CMD line is driven low and when CMDTRANS bit is set and the DTDIR = receive, the CPSM DataEnable signal is issued to the DPSM. This enables normal boot operation. This state is left at the end of the boot procedure by clearing the register bit BOOTEN, which cause the SDMMC_CMD line to be driven high and the CPSM Abort signal is issued to the DPSM, before moving to the Idle state. The CMDSENT flag is generated 56 cycles after SDMMC_CMD line is high.
 - BOOTMODE = 1, move to the Send state. This enables sending of the CMD0 (boot). Clearing BOOTEN has no effect.

Note: The CPSM remains in the Idle state for at least eight SDMMC_CK periods to meet the N_{CC} and N_{RC} timing constraints. N_{CC} is the minimum delay between two host commands, and N_{RC} is the minimum delay between the host command and the card response.

The response timeout has a fixed value of 64 SDMMC_CK clock periods.

A command is a token that starts an operation. Commands are sent from the host to either a single card (addressed command) or all connected cards (broadcast command are available for eMMC V3.31 or previous). Commands are transferred serially on the SDMMC_CMD line. All commands have a fixed length of 48 bits. The general format for a command token for SD-Memory cards, SDIO cards, and eMMC cards is shown in [Table 255](#).

The command token data is taken from two registers, one containing a 32-bit argument and the other containing the 6-bit command index (six bits sent to a card).

Table 255. Command token format

Bit position	Width	Value	Description
47	1	0	Start bit
46	1	1	Transmission bit
[45:40]	6	x	Command index
[39:8]	32	x	Argument
[7:1]	7	x	CRC7
0	1	1	End bit

Next to the command data there are command type (WAITRESP) bits controlling the command path state machine (CPSM). These bits also determine whether the command requires a response, and whether the response is short (48 bit) or long (136 bits) long, and if a CRC is present or not.

A response is a token that is sent from an addressed card or synchronously from all connected cards to the host as an answer to a previous received command. All responses are sent via the command line SDMMC_CMD. The response transmission always starts with the left bit of the bit string corresponding to the response code word. The code length depends on the response type. Response tokens R1, R2, R3, R4, R5, and R6 have various

coding schemes, depending on their content. The general formats for the response tokens for SD-Memory cards, SDIO cards, and eMMC cards are shown in [Table 256](#), [Table 257](#) and [Table 258](#).

A response always starts with a start bit (always 0), followed by the bit indicating the direction of transmission (card = 0). A value denoted by x in the tables below indicates a variable entry. Most responses, except some, are protected by a CRC. Every command code word is terminated by the end bit (always 1).

The response token data is stored in five registers, four containing the 32-bits card status, OCR register, argument or 127-bits CID or CSD register including internal CRC, and one register containing the 6-bits command index.

Table 256. Short response with CRC token format

Bit position	Width	Value	Description
47	1	0	Start bit
46	1	0	Transmission bit
[45:40]	6	x	Command index (or reserved 111111)
[39:8]	32	x	Argument
[7:1]	7	x	CRC7
0	1	1	End bit

Table 257. Short response without CRC token format

Bit position	Width	Value	Description
47	1	0	Start bit
46	1	0	Transmission bit
[45:40]	6	x	Command index (or reserved 111111)
[39:8]	32	x	Argument
[7:1]	7	1111111	(reserved 1111111)
0	1	1	End bit

Table 258. Long response with CRC token format

Bit position	Width	Value	Description
135	1	0	Start bit
134	1	0	Transmission bit
[133:128]	6	111111	Reserved
[127:1]	127:8	x	CID or CSD slices
	7:1	x	CRC7 (included in CID or CSD)
0	1	1	End bit

The command/response path operates in a half-duplex mode, so that either commands can be sent or responses can be received. If the CPSM is not in the Send state, the

SDMMC_CMD output is in the Hi-Z state. Data sent on SDMMC_CMD are synchronous with the SDMMC_CLK according to the NEGEDGE register bit see [Figure 207](#).

The command and short response with CRC, the CRC generator calculates the CRC checksum for all 40 bits before the CRC code. This includes the start bit, transmission bit, command index, and command argument (or card status).

For the long response the CRC checksum is calculated only over the 120 bits of R2 CID or CSD. Note that the start bit, transmission bit and the six reserved bits are not used in the CRC calculation.

The CRC checksum is a 7-bit value:

$$\text{CRC}[6:0] = \text{remainder} [(M(x) * x^7) / G(x)]$$

$$G(x) = x^7 + x^3 + 1$$

$$M(x) = (\text{first bit}) * x^n + (\text{second bit}) * x^{n-1} + \dots + (\text{last bit before CRC}) * x^0$$

Where $n = 39$ or 119 .

The CPSM can send a number of specific commands to handle various operating modes when CPSMEN is set, see [Table 259](#).

Table 259. Specific Commands overview

VSWITCH	BOOTEN	BOOTMOD	CMDTRAN	WAITPEND	CMDSTOP	WAITINT	Description
1	x	x	x	x	x	x	Start voltage switch sequence
0	1	x	x	x	x	x	Start normal boot
0	1	1	x	x	x	x	Start alternative boot
0	0	1	x	x	x	x	Stop alternative boot.
0	0	0	1	x	x	x	Send command with associated data transfer.
0	0	0	0	1	1	x	eMMC stream data transfer, command (STOP_TRANSMISSION) pending until end of data transfer.
0	0	0	0	1	0	x	eMMC stream data transfer, command different from (STOP_TRANSMISSION) pending until end of data transfer.
0	0	0	0	0	1	x	Send command (STOP_TRANSMISSION), stopping any ongoing data transmission.
0	0	0	0	0	0	1	Enter eMMC wait interrupt (Wait-IRQ) mode.
0	0	0	0	0	0	0	Any other none specific command

The command/response path implements the status flags and associated clear bits shown in [Table 260](#):

Table 260. Command path status flags

Flag	Description
CMDSENT	Set at the end of the command without response (CPSM moves from Send to Idle).
CMDREND	Set at the end of the command response when the CRC is OK (CPSM moves from Receive to Idle).
CCRCFAIL	Set at the end of the command response when the CRC is FAIL (CPSM moves from Receive to Idle).
CTIMEOUT	Set after the command when no response start bit received before the timeout (CPSM moves from Wait to Idle).
CKSTOP	Set after the voltage switch (VSWITCHEN = 1) command response when the CRC is OK and the SDMMC_CK is stopped (no impact on CPSM).
VSWEND	Set after the voltage switch (VSWITCH = 1) timeout of 5 ms + 1 ms (no impact on CPSM).
CPSMACT	Command transfer in progress (CPSM not in Idle state).

The command path error handling is shown in [Table 261](#):

Table 261. Command path error handling

Error	CPSM state	Cause	Card action	Host action	CPSM action
Timeout	Wait	No start bit in time	Unknown	Reset or cycle power card ⁽¹⁾	Move to Idle
CRC status	Receive	Negative status	Command ignored	Resend command ⁽¹⁾	Move to Idle
		Transmission error	Command accepted	Resend command ⁽¹⁾	

1. When CMDTRANS is set, also a stop_transmission command must be send to move the DPSM to Idle.

Data path

The data path subunit transfers data on the SDMMC_D[7:0] lines to and from cards. The data transmit path is clocked on the SDMMC_CK and sends data to the card. The data receive path is clocked on the sdmmc_rx_ck and receives data from the card. [Figure 211](#) shows the data path block diagram.

In DDR mode, data is sampled on both edges of the SDMMC_CLK according the following rules, see also [Figure 212](#) and [Figure 213](#):

- On the rising edge of the clock odd bytes are sampled.
- On the falling edge of the clock even bytes are sampled.
- Data payload size is always a multiple of 2 bytes.
- Two CRC16 are computed per data line
 - Odd bits CRC16 clocked on the falling edge of the clock.
 - Even bits CRC16 clocked on the rising edge of the clock.
- Start, end bits and idle conditions are full cycle.
- CRC status / boot acknowledgment and busy signaling are full cycle and are only sampled on the rising edge of the clock.

In DDR mode, the SDMMC_CLK clock division must be ≥ 2 .

Figure 212. DDR mode data packet clocking

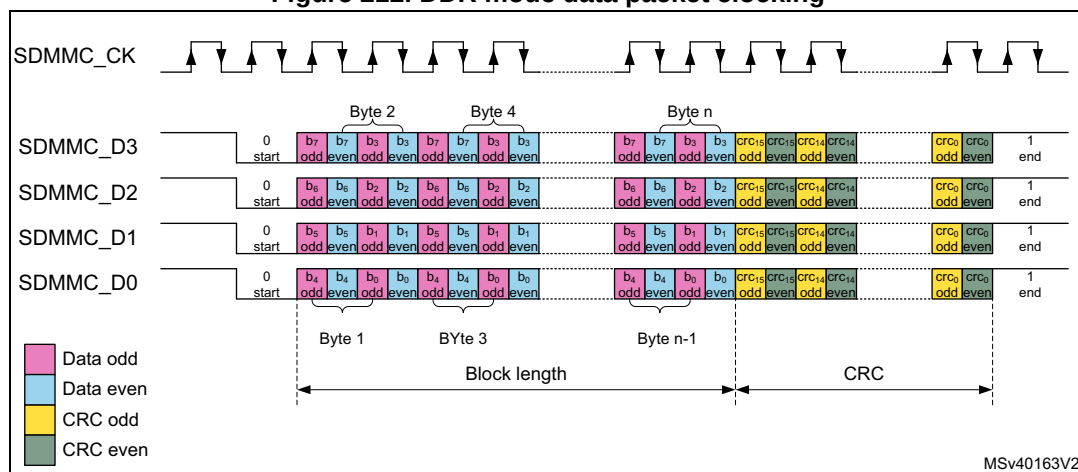
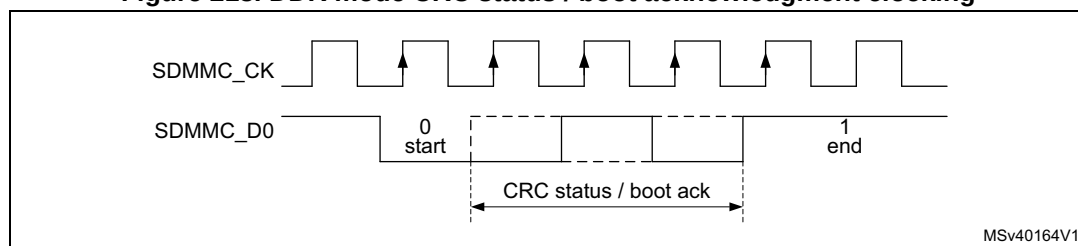


Figure 213. DDR mode CRC status / boot acknowledgment clocking



Data path state machine (DPSM)

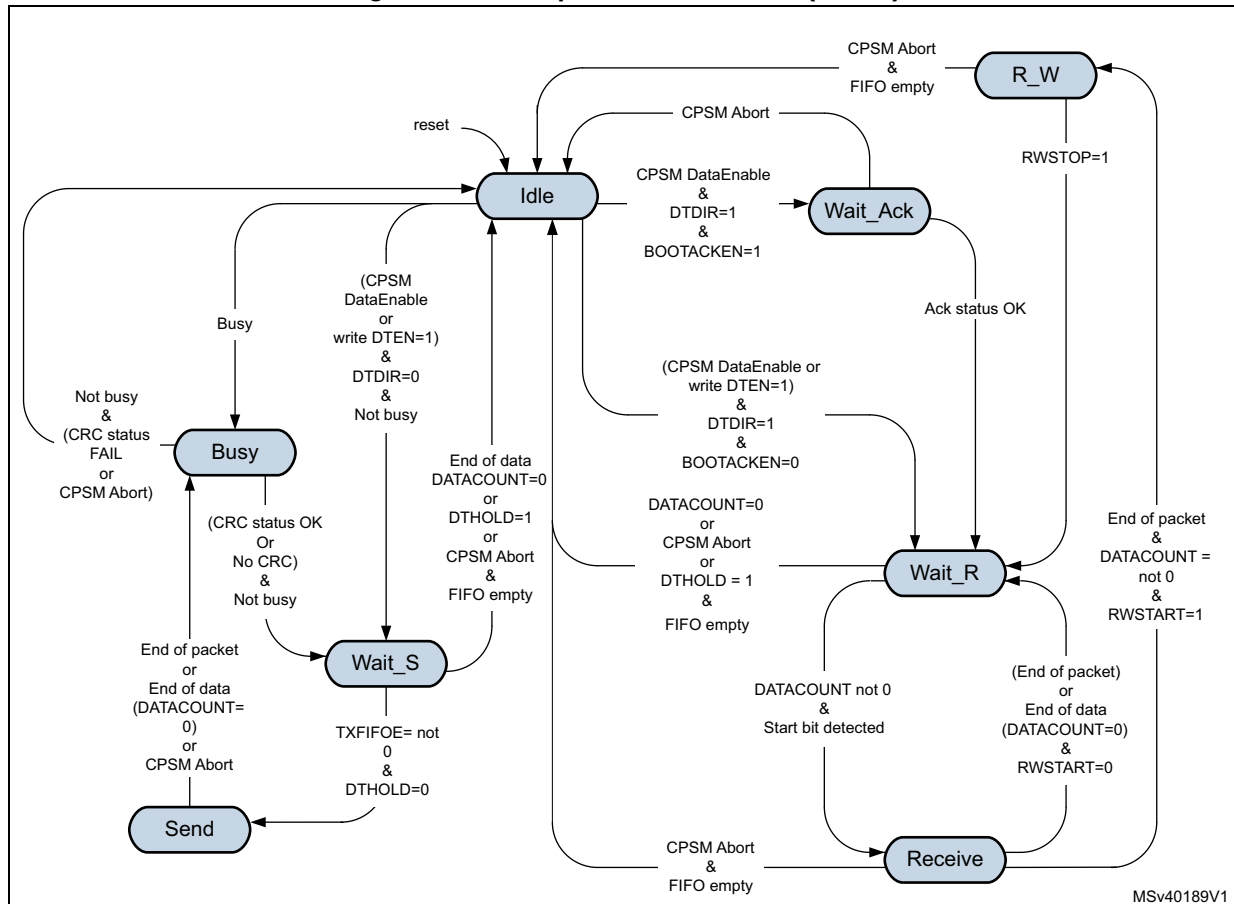
Depending on the transfer direction (send or receive), the data path state machine (DPSM) moves to the Wait_S or Wait_R state when it is enabled:

- Send: the DPSM moves to the Wait_S state. If there is data in the transmit FIFO, the DPSM moves to the Send state, and the data path subunit starts sending data to a card.
- Receive: the DPSM moves to the Wait_R state and waits for a start bit. When it receives a start bit, the DPSM moves to the Receive state, and the data path subunit starts receiving data from a card.

For boot operation with acknowledgment the DPSM moves to the Wait_Ack state and waits for the boot acknowledgment before moving to the Wait_R state.

The DPSM operates at SDMMC_CK. The DPSM has the following states, as shown in [Figure 214](#). When ever the DPSM is active (not in the Idle state), the DPSMACT bit is set.

Figure 214. Data path state machine (DPSM)



- **Idle** state: the data path is inactive, and the SDMMC_D[7:0] outputs are according to the PWRCTRL setting. The DPSPM is activated either by sending a command with CMDTRANS bit set or by setting the DTEN bit, or by detecting Busy on SDMMC_D0 (that is, after a command with R1b response).

When not busy, the DPSM activates the SDMMC_CK clock (when stopped due to power save PWRSV bit), loads the data counter with a new (DATALENGTH) value and:

- When the data direction bit (DTDIR) indicates send, moves to the Wait_S.
- When the data direction bit (DTDIR) indicates receive, moves to the
 - Wait_R when BOOTACKEN register bit is clear.
 - Wait_Ack when BOOTACKEN register bit is set and start the acknowledgment timeout.

When busy the DPSM keeps the SDMMC CK clock active and move to the Busy state.

Note: *DTEN must not be used to start data transfer with SD, SDIO and eMMC cards.*

- **Wait_Ack** state: the data path waits for the boot acknowledgment token.
 - The DPSM moves to the Wait_R state if it receives an error free acknowledgment before a timeout.
 - When a pattern different from the acknowledgment is received an acknowledgment status error is generated, and the ack fail status flag (ACKFAIL) is set. The DPSM stays in Wait_Ack.
 - If it reaches a timeout (ACKTIME) before it detects a start bit, it sets the timeout status flag (ACKTIMEOUT). The DPSM stays in Wait_Ack.
 - When the CPSM Abort signal is set it moves to the Idle state and sets the DABORT flag.
- **Wait_R** state: the data path, if the data counter is not zero and data is not hold, waits for a start bit on SDMMC_D[n:0]. If the data counter is zero or data is hold, wait for the FIFO to be empty.
 - In block mode, if a start bit is received before a timeout the DPSM moves to the Receive state and loads the data block counter with DBLOCKSIZE.
 - In SDIO multibyte mode, if a start bit is received before a timeout the DPSM moves to the Receive state and loads the data block counter with DATALENGTH.
 - In stream mode, if a start bit is received before a timeout the DPSM moves to the Receive state and loads the data counter with DATALENGTH.
 - if the data counter (DATACOUNT) equals zero (end of data) the DPSM moves to the Idle state when the receive FIFO is empty and the DATAEND flag is set.
 - If it reaches a timeout (DATETIME) before it detects a start bit, it sets the timeout status flag (DTIMEOUT) and the DPSM stays in the Wait_R state.
 - If the CPSM Abort signal is set:
 - If DATACOUNT > 0, the DPSM moves to the Idle state when the FIFO is empty and when IDMAEN = 0 reset with FIFORST, and sets the DABORT flag.
 - If DATACOUNT is zero normal operation is continued, there is no DABORT flag since the transfer has completed normally.
 - if the DTHOLD bit is set:
 - When DATACOUNT > 0, the DPSM moves to the Idle state when the receive FIFO is empty and when IDMAEN = 0 reset with FIFORST, and issues the DHOLD flag. When holding the timeout is disabled. When an CPSM Abort signal is received during holding, the transfer is aborted.
 - When DATACOUNT = 0, the transfer is completed normally and there is no DHOLD flag.
 - When DPSM has been started with DTEN, after an error (DTIMEOUT) the DPSM moves to the Idle state when the FIFO is empty and when IDMAEN = 0 reset with FIFORST.

- **R_W** state: the data path Read Wait the bus.
 - The DPSM moves to the Wait_R state when the Read Wait stop bit (RWSTOP) is set, and start the receive timeout.
 - If the CPSM Abort signal is set, wait for the FIFO to be empty and when IDMAEN = 0 reset with FIFORST, then moves to the Idle state and sets the DABORT flag.
- **Receive** state: the data path receives serial data from a card. Pack the data in bytes and written it to the data FIFO. Depending on the transfer mode selected in the data control register (DTMODE), the data transfer mode can be either block or stream:
 - In block mode, when the data block size (DBLOCKSIZE) number of data bytes are received, the DPSM waits until it receives the CRC code.
 - In SDIO multibyte mode, when the data block size (DATALENGTH) number of data bytes are received, the DPSM waits until it receives the CRC code.
 - a) If the received CRC code matches the internally generated CRC code, the DPSM moves to the
 - R_W state when RWSTART = 1 and DATACOUNT > zero, the DBCKEND flag is set.
 - Wait_R state otherwise.
 - b) If the received CRC code fails the internally generated CRC code any further data reception is prevented.
 - When not all data has been received (DATACOUNT > 0), the CRC fail status flag (DCRCFAIL) is set and the DPSM stays in the Receive state.
 - When all data has been received (DATACOUNT = 0), wait for the FIFO to be empty after which the CRC fail status flag (DCRCFAIL) is set and the DPSM moves to the Idle state.
 - In stream mode, the DPSM receives data while the data counter DATACOUNT > 0. When the counter is zero, the remaining data in the shift register is written to the data FIFO, and the DPSM moves to the Wait_R state.
 - When a FIFO overrun error occurs, the DPSM sets the FIFO overrun error flag (RXOVERR) and any further data reception is prevented. The DPSM stays in the Receive state.
 - When an CPSM Abort signal is received:
 - If the CPSM Abort signal is received before the 2 last bits of the data with DATACOUNT = 0, the transfer is aborted. The remaining data in the shift register is written to the data FIFO, wait for the FIFO to be empty and when IDMAEN = 0 reset with FIFORST, then the DPSM moves to the Idle state and the DABORT flag is set.
 - If the CPSM Abort signal is received during or after the 2 last bits of the transfer with DATACOUNT = 0, the transfer is completed normally. The DPSM stays in the Receive state no DABORT flag is generated.
 - When DPSM has been started with DTEN, after an error (DCRCFAIL when DATACOUNT > 0, or RXOVERR) the DPSM moves to the Idle state when the FIFO is empty and when IDMAEN = 0 reset with FIFORST.

- **Wait_S** state: the data path waits for data to be available from the FIFO.
 - If the data counter $\text{DATACOUNT} > 0$, waits until the data FIFO empty flag (TXFIFOE) is de-asserted and DTHOLD is not set, and moves to the Send state.
 - If the data counter ($\text{DATACOUNT} = 0$) the DPSM moves to the Idle state.
 - When DTHOLD is disabled, the DATAEND flag is set.
 - When DTHOLD is enabled, the DHOLD flag is set.
 - When DTHOLD is set and the $\text{DATACOUNT} > 0$
 - When IDMA is enabled, the DBCKEND flag is set and subsequently the FIFO is flushed, furthermore the DPSM moves to the Idle state and the DHOLD flag is set.
 - When IDMA is disabled the DBCKEND flag is set. Wait for the FIFO to be reset by software with FIFORST, then DPSM moves to the Idle state and issues the DHOLD flag.
 - When DTHOLD is set and $\text{DATACOUNT} = 0$ the transfer is completed normally.
 - When receiving the CPSM Abort signal
 - If the CPSM Abort signal is received before the 2 last bits of the data with $\text{DATACOUNT} = 0$, the transfer is aborted, wait for the FIFO to be empty and when IDMAEN = 0 reset with FIFORST, then the DPSM moves to the Idle state and sets the DABORT flag.
 - If the CPSM Abort signal is received during or after the 2 last bits of the transfer with $\text{DATACOUNT} = 0$, normal operation is continued, there is no DABORT flag since the transfer has completed normally.

Note: The DPSM remains in the Wait_S state for at least two clock periods to meet the N_{WR} timing requirements, where N_{WR} is the number of clock cycles between the reception of the card response and the start of the data transfer from the host.

- **Send** state: the DPSM starts sending data to a card. Depending on the transfer mode bit in the data control register, the data transfer mode can be either block, SDIO multibyte or stream:
 - In block mode, when the data block size (DBLOCKSIZE) number of data bytes are send, the DPSM sends an internally generated CRC code and end bit, and moves to the Busy state and start the transmit timeout.
 - In SDIO multibyte mode, when the data block size (DATALENGTH) number of data bytes are send, the DPSM sends an internally generated CRC code and end bit, and moves to the Busy state and start the transmit timeout.
 - In stream mode, the DPSM sends data to a card while the data counter $\text{DATACOUNT} > 0$. When the data counter reaches zero moves to the Busy state and start the transmit timeout.
Before sending the last stream byte according to DATACOUNT , the DPSM issues a trigger on the send CMD signal. This signal is used by the CPSM to sent any pending command (CMD12 Stop Transmission command).
 - If a FIFO underrun error occurs, the DPSM sets the FIFO underrun error flag (TXUNDERR). The DPSM stays in the Send state.
 - When receiving the CPSM Abort signal
 - If the CPSM Abort signal is received before the 2 last bits of the transfer with $\text{DATACOUNT} = 0$, the transfer is aborted. The DPSM sends a last data bit followed by an end bit. The FIFO is disabled/flushed, and the DPSM moves to the Busy state to wait for not busy before setting the DABORT flag.
 - If the CPSM Abort signal is received during or after the 2 last bits of the transfer

with DATACOUNT = 0, the transfer is completed normally, there is no DABORT flag.

- **Busy state:** the DPSM waits for the CRC status token when expected, and wait for a not busy signal:
 - If a CRC status token is expected and indicate “non-erroneous transmission” or when there is no CRC expected:
 - it moves to the Wait_S state when SDMMC_D0 is not low (the card is not busy).
 - When the card is busy SDMMC_D0 is low it remains in the Busy state.
 - If a CRC status token is expected and indicates “erroneous transmission”.
 - When not all data has been send (DATACOUNT > 0). The DPSM waits for not busy after which the CRC fail status flag (DCRCFAIL) is set. The FIFO is disabled/flushed and the DPSM stays in the Busy state.
 - When all data has been send (DATACOUNT = 0). The DPSM waits for not busy after which the CRC fail status flag (DCRCFAIL) is set and the DPSM moves to the Idle state.
 - If a CRC status (Ncrc) timeout occurs while the DPSM is in the Busy state, it sets the data timeout flag (DTIMEOUT) and stays in the Busy state.
 - If a busy timeout occurs while the DPSM is in the Busy state, it sets the data timeout flag (DTIMEOUT) and stays in the Busy state.
 - When receiving the CPSM Abort signal in the Busy state:
 - If the CPSM Abort signal is received before the 2 last bits of the CRC response with DATACOUNT > 0, the data transfer is aborted. The DPSM waits for not busy and the FIFO to be disabled/flushed before moving to the Idle state and the DABORT flag is set.
 - If the CPSM Abort signal is received during or after the 2 last bits of the CRC response when DATACOUNT = 0 or when no CRC is expected and DATACOUNT = 0 and there has been no DTIMEOUT error, the DPSM stays in the Busy state no DABORT flag is generated, since the transfer may completed normally.
 - If the CPSM Abort signal is received when a DTIMEOUT error has occurred the DPSM waits for not busy and the FIFO to be disabled/flushed before moving to the Idle state and the DABORT flag is set.
 - When entering the Busy state due to an abort in the Send state, the DPSM waits for not busy before moving to the Idle state and the DABORT flag is set.
 - When DPSM has been started with DTEN, after an error (DCRCFAIL when DATACOUNT > 0, or DTIMEOUT) the DPSM moves to the Idle state when the FIFO is reset.
 - When the DPSM has been started due to Busy on SDMMC_D0, waits for not busy after which the Busy end status flag (BUSYD0END) is set and the DPSM moves to the Idle state.

The data timer (DATATIME) is enabled when the DPSM is in the Wait_R or Busy state 2 cycles after the data block end bit, or data read command end bit, or R1b response, and generates the data timeout error (DTIMEOUT):

- When transmitting data, the timeout occurs
 - when a CRC status is expected and no start bit is received withing 8 SDMMC_CK cycles, the DTIMEOUT flag is set.
 - when the Busy state takes longer than the programmed timeout period., the DTIMEOUT flag is set.
- When receiving data, the timeout occurs
 - when there is still data to be received DATACOUNT > 0 and no start bit is received before the programmed timeout period, the DTIMEOUT flag is set.
- After a R1b response, the timeout occurs
 - when the Busy state takes longer than the programmed timeout period., the DTIMEOUT flag is set.

When DATATIME = 0:

- In receive the start bit must be present 2 cycles after the data block end bit or data read command end bit.
- In transmit busy is timed out 2 cycles after the CRC token end bit or stream data end bit.
- After a R1b response busy is timed out 2 cycles after the response end bit.

Data can be transferred from the card to the host (transmit, send) or vice versa (receive). Data are transferred via the SDMMC_Dn data lines, they are stored in a FIFO.

Table 262. Data token format

Description	Start bit	Data ⁽¹⁾	CRC16	End bit	DTMODE
Block data	0	(DBLOCKSIZE, DATALENGTH)	Yes	1	00
SDIO multibyte	0	(DATALENGTH)	Yes	1	01
eMMC stream	0	(DATALENGTH)	No	1	10

1. The total amount of data to transfer is given by DATALENGTH. Where for Block data the amount of data in each block is given by DBLOCKSIZE.

The data token format is selected with register bits DTMODE according.

The data path implements the status flags and associated clear bits shown in [Table 263](#):

Table 263. Data path status flags and clear bits

Flag		Description
DATAEND	TX	Set at the end of the complete data transfer when the CRC is OK and busy has finished and both DTHOLD = 0 and DATACOUNT = 0. (DPSM moves from Wait_S to Idle)
	RX	Set at the end of the complete data transfer when the CRC is OK and all data has been read, (DATACOUNT = 0 and FIFO is empty). (DPSM moves from Wait_R to Idle)
	Boot	

Table 263. Data path status flags and clear bits (continued)

Flag		Description
DCRCFAIL	TX	Set at the end of the CRC when FAIL and busy has finished. (DPSM stay in Busy when there is still data to send and wait for CPSM Abort) (DPSM moves from Busy to Idle when all data has been sent) or DPSM has been started with DTEN
	RX	Set at the end of the CRC when FAIL and FIFO is empty. (DPSM stays in Receive when there is still data to be received and wait for CPSM Abort) (DPSM moves from Receive to Idle when all data has been received or DPSM has been started with DTEN)
	Boot	
ACKFAIL	Boot	Set at the end of the boot acknowledgment when fail. (DPSM stays in Wait_Ack and wait for CPSM Abort)
DTIMEOUT	CMD R1b	Set after the command response no end of busy received before the timeout. (DPSM stays in Busy and wait for CPSM Abort)
	TX	Set when no CRC token start bit received within Ncrc, or no end of busy received before the timeout. (DPSM stays in Busy and wait for CPSM Abort) (When DPSM has been started with DTEN move to Idle) Note: The DCRCFAIL flag may also be set when CRC failed before the busy timeout.
	RX	Set when no start bit received before the timeout. (DPSM stays in Wait_R and wait for CPSM Abort) (When DPSM has been started with DTEN move to Idle)
	Boot	
ACKTIMEOUT	Boot	Set when no start bit received before the timeout. (DPSM stays in Wait_Ack and wait for CPSM Abort)
DBCKEND	TX	When DTHOLD = 1 and IDMAEN = 0: Set at the end of data block transfer when the CRC is OK and busy has finished, when data transfer is not complete (DATACOUNT > 0). (DPSM moves from Busy to Wait_S)
	RX	When RWSTART = 1: Set at the end of data block transfer when the CRC is OK, when data transfer is not complete (DATACOUNT > 0). (DPSM moves from Receive to R_W)
	Boot	
DHOLD	TX	When DTHOLD = 1: Set at the end of data block transfer when the CRC is OK and busy has finished. (DPSM moves from Wait_S to Idle)
	RX	When DTHOLD = 1: Set at the end of data block transfer when the CRC is OK and all data has been read (FIFO is empty), when data transfer is not complete (DATACOUNT > 0). (DPSM moves from Wait_R to Idle)
DABORT	CMD R1b	When CPSM Abort event has been sent by the CPSM and busy has finished. (DPSM moves from Busy to Idle)
	TX	
	RX	When CPSM Abort event has been sent by the CPSM before the 2 last bits of the transfer. (DPSM moves from any state to Idle)
	Boot	
BUSYD0END	CMD R1b	Set after the command response when end of busy before the timeout. (DPSM moves from Busy to Idle)
DPSMACT		Data transfer in progress. (DPSM not in Idle state)

The data path error handling is shown in [Table 264](#):

Table 264. Data path error handling

Error	DPSM state	Cause	Card action	Host action	DPSM action
Timeout	Wait_Ack	No Ack in time	unknown	Card cycle power	Stay in Wait_Ack (reset the SDMMC with the RCC.SDMMCxRST register bit)
	Wait_R	No start bit in time	unknown	Stop data reception Send stop transmission command	On CPSM Abort move to Idle
			unknown	Stop boot procedure	
	Busy	Busy too long (due to data transfer)	unknown	Stop data reception Send stop transmission command	
		Busy too long (due to R1b)	unknown	Send reset command	
CRC	Receive	transmission error	Send further data	Stop data reception Send stop transmission command	On CPSM Abort move to Idle
CRC status	Busy	Negative status	Ignore further data	Stop data transmission Send stop transmission command	On CPSM Abort move to Idle
		transmission error	wait for further data		
Ack status	Wait_Ack	transmission error	Send boot data	Stop boot procedure	On CPSM Abort move to Idle
Overrun	Receive	FIFO full	Send further data	Stop data reception Send stop transmission command	On CPSM Abort move to Idle
Underrun	Send	FIFO empty	Receive further data	Stop data transmission Send stop transmission command	On CPSM Abort move to Idle

Data FIFO

The data FIFO (first-in-first-out) subunit contains the transmit and receive data buffer. A single FIFO is used for either transmit or receive as selected by the DTDIR bit. The FIFO contain a 32-bit wide, 16-word deep data buffer and control logic. Because the data FIFO operates in the AHB clock domain (sdmmc_hclk), all signals from the subunits in the SDMMC clock domain (SDMMC_CK/sdmmc_rx_ck) are resynchronized.

The FIFO can be in one of the following states:

- The transmit FIFO refers to the transmit logic and data buffer when sending data out to the card. (DTCR = 0)
- The receive FIFO refers to the receive logic and data buffer when receiving data in from the card. (DTCR = 1)

The end of a correctly completed SDMMC data transfer from the FIFO is indicated by the DATAEND flags driven by the data path subunit. Any incorrect (aborted) SDMMC data transfer from the FIFO is indicated by one of the error flags (DCRCFAIL, DTIMEOUT, DABORT) driven by the data path subunit, or one of the FIFO error flags (TXUNDERR, RXOVERR) driven by the FIFO control.

The data FIFO can be accessed in the following ways, see [Table 265](#).

Table 265. Data FIFO access

Data FIFO access	IDMAEN
From firmware via AHB slave interface	0
From IDMA via AHB master interface	1

Transmit FIFO:

Data can be written to the transmit FIFO when the DPSM has been activated (DPSMACT = 1).

When IDMAEN = 1 the FIFO is fully handled by the IDMA.

When IDMAEN = 0 the FIFO is controlled by firmware via the AHB slave interface. The transmit FIFO is accessible via sequential addresses. The transmit FIFO contains a data output register that holds the data word pointed to by the read pointer. When the data path subunit has loaded its shift register, it increments the read pointer and drives new data out. The transmit FIFO is handled in the following way:

1. Write the data length into DATALENGTH and the block length in DBLOCKSIZE.
 - For block data transfer (DTMODE = 0), DATALENGTH must be an integer multiple of DBLOCKSIZE.
2. Set the SDMMC in transmit mode (DTCR = 0).
 - Configures the FIFO in transmit mode.
3. Enable the data transfer
 - either by sending a command from the CPSM with the CMDTRANS bit set
 - or by setting DTEN bit
4. When (DPSMACT = 1) write data to the FIFO.
 - The DPSM stays in the Wait_S state until FIFO is full (TXFIFO = 1), or the number indicated by DATALENGTH.

- The SDMMC keeps sending data as long as FIFO is not empty, hardware flow control during data transfer is used to prevent FIFO underrun.

5. Write data to the FIFO.
 - When the FIFO is handled by software, wait until the FIFO is half empty (TXFIFOHE flag), write data to the FIFO until FIFO is full (TXFIFO = 1), or last data has been written.
 - When the FIFO is handled by the IDMA, the IDMA transfers the FIFO data.
6. When last data has been written wait for end of data (DATAEND flag)
 - SDMMC has completely sent all data and the DPSM is disabled (DPSMACT = 0).

In case of a data transfer error or transfer hold when IDMAEN = 0, firmware must stop writing to the FIFO and flush and reset the FIFO with the FIFORST register bit.

The transmit FIFO status flags are listed in [Table 266](#).

Table 266. Transmit FIFO status flags

Flag	Description
TXFIFO	Set to high when all transmit FIFO words contain valid data.
TXFIFOE	Set to high when the transmit FIFO does not contain valid data.
TXFIFOHE	Set to high when half or more transmit FIFO words are empty.
TXUNDERR	Set to high when an underrun error occurs. This flag is cleared by writing to the SDMMC Clear register.

Receive FIFO:

Data can be read from the receive FIFO when the DPSM is activated (DPSMACT = 1).

When IDMAEN = 1 the FIFO is fully handled by the IDMA.

When IDMAEN = 0 the FIFO is controlled by firmware via the AHB slave interface. When the data path subunit receives a word of data, it drives the data on the write databus. The write pointer is incremented after the write operation completes. On the read side, the contents of the FIFO word pointed to by the current value of the read pointer is driven onto the read databus. The receive FIFO is accessible via sequential addresses.

The receive FIFO is handled in the following way:

1. Write the data length into DATALENGTH and the block length in DBLOCKSIZE.
 - For block data transfer (DTMODE = 0), DATALENGTH must be an integer multiple of DBLOCKSIZE.
2. Set the SDMMC in receive mode (DTDIR = 1).
 - Configures the FIFO in receive mode.
3. Enable the DPSM transfer
 - either by sending a command from the CPSM with the CMDTRANS bit set
 - or by setting DTEN bit.
4. When (DPSMACT = 1) the FIFO is ready to receive data.
 - The DPSM writes the received data to the FIFO.
 - The SDMMC keeps receiving data as long as FIFO is not full, hardware flow control during the data transfer is used to prevent FIFO overrun.
5. Read data from the FIFO.
 - When the FIFO is handled by software, wait until the FIFO is half full (RXFIFOHF flag), read data from the FIFO until FIFO is empty (RXFIFOE = 1).
 - When last data has been received, read data from the FIFO until FIFO is empty (DATAEND = 1).
 - When the FIFO is handled by the IDMA, the IDMA transfers the FIFO data.
6. SDMMC has completely received all data and the DPSM is disabled (DPSMACT = 0).

In case of a data transfer hold when IDMAEN = 0, the firmware must read the remaining data until the FIFO is empty and reset the FIFO with the FIFORST register bit. This causes the DPSM to go to the Idle state (DPSMACT = 0).

In case of a data transfer error when IDMAEN = 0, the firmware must stop reading the FIFO and flush and reset the FIFO with the FIFORST register bit. This causes the DPSM to go to the Idle state (DPSMACT = 0).

The receive FIFO status flags are listed in [Table 267](#).

Table 267. Receive FIFO status flags

Flag	Description
RXFIFOE	Set to high when all receive FIFO words contain valid data
RXFIFOE	Set to high when the receive FIFO does not contain valid data.
RXFIFOHF	Set to high when half or more receive FIFO words contain valid data.
RXOVERR	Set to high when an overrun error occurs. This flag is cleared by writing to the SDMMC Clear register.

CLKMUX unit

The CLKMUX selects the source for clock sdmmc_rx_ck to be used with the received data and command response. The receive data clock source can be selected by the clock control register bit SELCLKRX, between:

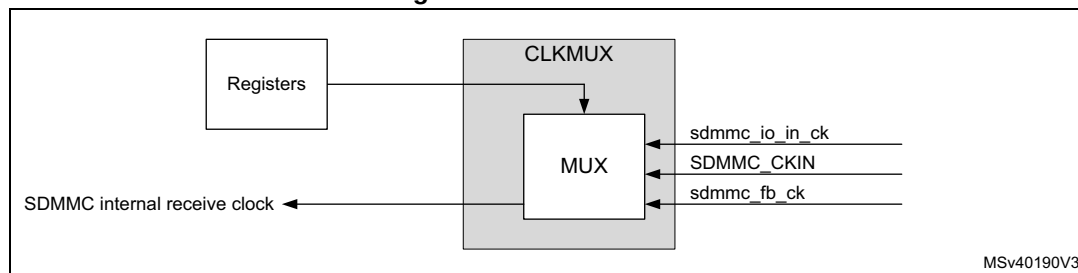
- sdmmc_io_in_ck bus master main feedback clock.
- SDMMC_CKIN external bus feedback clock.
- sdmmc_fb_ck bus tuned feedback clock.

The `sdmmc_io_in_ck` is selected when there is no external driver, with DS and HS.

The `SDMMC_CKIN` is selected when there is an external driver with SDR12, SDR25, SDR50 and DDR50.

The `sdmmc_fb_ck` clock input must be selected when the DLYB block on the device is used with SDR104, HS200 and optionally with SDR50 and DDR50 modes.

Figure 215. CLKMUX unit



The `sdmmc_rx_ck` source must be changed when the CPSM and DPSM are in the Idle state.

26.5.5 SDMMC AHB slave interface

The AHB slave interface generates the interrupt requests, and accesses the SDMMC adapter registers and the data FIFO. It consists of a data path, register decoder, and interrupt logic.

SDMMC FIFO

The FIFO access is restricted to word access only:

- In transmit FIFO mode
 - Data are written to the FIFO in words (32-bits) until all data according `DATALENGTH` has been transferred. When the `DATALENGTH` is not an integer multiple of 4, the last remaining data (1, 2 or 3 bytes) are written with a word transfer.
- In receive FIFO mode
 - Data are read from the FIFO in words (32-bits) until all data according `DATALENGTH` has been transferred. When the `DATALENGTH` is not an integer multiple of 4, the last remaining data (1, 2 or 3 bytes) are read with a word transfer padded with 0 value bytes.

When accessing the FIFO with half word or byte accesses an AHB bus fault is generated.

SDMMC interrupts

The interrupt logic generates an interrupt request signal that is asserted when at least one of the unmasked status flags is active. A mask register is provided to allow selection of the conditions that generate an interrupt. A status flag generates the interrupt request if a corresponding mask flag is set. Some status flags require an implicit clear in the clear register.

26.5.6 SDMMC AHB master interface

The AHB master interface is used to transfer the data between a memory and the FIFO using the SDMMC IDMA.

SDMMC IDMA

Direct memory access (DMA) is used to provide high-speed transfer between the SDMMC FIFO and the memory. The AHB master optimizes the bandwidth of the system bus. The SDMMC internal DMA (IDMA) provides one channel to be used either for transmit or receive.

The IDMA is enabled by the IDMAEN bit and supports burst transfers of 8 beats.

- In transmit burst transfer mode:
 - Data are fetched in burst from memory whenever the FIFO is empty for the number of burst transfers, until all data according DATALENGTH has been transferred. When the DATALENGTH is not an integer multiple of the burst size the remaining, smaller than burst size data is transferred using single transfer mode. When the DATALENGTH is not an integer multiple of 4, the last remaining data (1, 2 or 3 bytes) are fetched with a word transfer.
- In receive burst transfer mode:
 - Data are stored in burst in to memory whenever the FIFO contains the number of burst transfers, until all data according DATALENGTH has been transferred. When the DATALENGTH is not an integer multiple of the burst transfer the remaining, smaller than burst size data, is transferred using single transfer mode. When the DATALENGTH is not an integer multiple of 4, the last remaining data (1, 2 or 3 bytes) are stored with halfword and or byte transfers.

In addition the IDMA provides the following channel configurations selected by bit IDMAENMODE:

- single buffered channel
- linked list channel

Single buffered channel

In single buffer configuration the data at the memory side is accessed in a linear matter starting from the base address IDMAENMODE. When the IDMA has finished transferring all data the and the DPSM has completed the transfer the DATAEND flag is set.

Linked list channel

In linked list configuration, IDMAENMODE = 1, the data at the memory side is subsequently accessed from linked buffers, located at base address IDMAENMODE. The size of the memory buffers is defined by IDMAENMODE. The buffer size must be an integer multiple of the burst size. The bit ULA is used to indicate if a new linked list buffer configuration has to be loaded from the linked list table. A new linked list configuration is loaded when the ULA bit for the current linked list item is set.

The first linked list item configuration is programmed by firmware directly in the SDMMC registers.

When the IDMA has finished transferring all the data of one linked list buffer, according IDMAENMODE, and when the linked list item ULA bit is set, the IDMA loads the new linked list item from the linked list table, and continues transferring data from the next linked list buffer.

When the IDMA has finished transferring all data, according IDMABSIZE and ULA, and the DPSM has completed the transfer, according DATALENGTH, the DATAEND flag is set.

In the following cases, the linked list provides more buffer space than the data to transfer which means the current linked list buffer data has not completely be transferred:

- the ULA bit is set, and all SDMMC data according DATALENGTH has been transferred (DATAEND flag)
- a transfer error (DCRCFAIL when DATACOUNT > 0, RXOVERR, TXUNDERR) occurs
- a transfer is hold (DTHOLD)

In all above cases, the IDMA linked list is stopped and the FIFO is flushed/reset. Before starting or restarting a new SDMMC transfer, the software must initialize a new linked list with correct IDMABASE and IDMABSIZE.

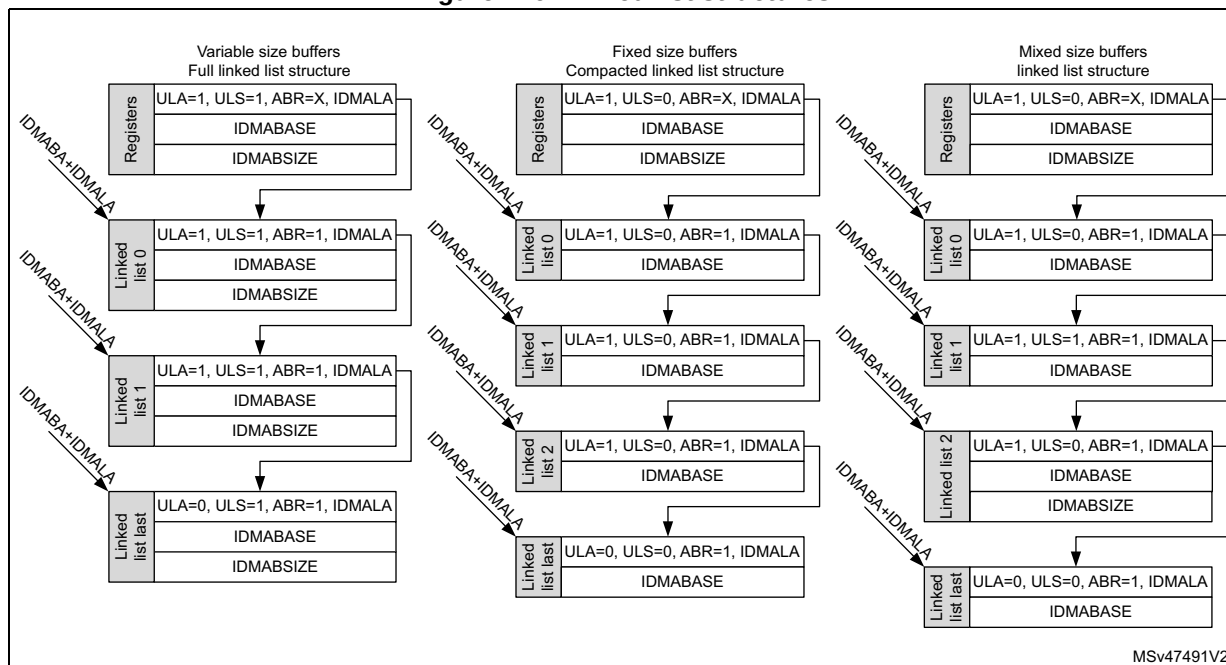
When a IDMA transfer error occurs (see [Section : IDMA transfer error management](#)) or when the linked list does not provide sufficient buffer space:

- the linked list ends with ULA = 0 and all last linked list buffer data has been transferred, and not all SDMMC data according DATALENGTH has been transferred. The SDMMC transfer is stopped and an IDMA transfer error is generated (see [Section : IDMA transfer error management](#)).

For a given linked list item, the base address is given by the linked list base IDMABA register value plus the linked list offset IDMALA register value.

The content of each linked list item can be specified by the ULS bit, which makes possible to optionally load the IDMABSIZE, resulting in a 3-word linked list structure. When the IDMABSIZE is not to be loaded (fixed size buffers) a compacted reduced 2-word linked list structure can be used containing only the IDMABASER and the IDMALAR values.

Figure 216. Linked list structures



There is no restriction on mixing both linked list item structures in a single list, this enables the IDMABSIZE to be updated only when needed.

Whenever a linked list buffer has been transferred and the current buffer ULA = 1, an end-of-linked-list-buffer-transfer-complete interrupt (IDMABTC) may be generated (if interrupt is enabled).

Linked list acknowledgment

In the case where software dynamically updates the linked list, during the SDMMC transfer, the availability of a new linked list buffer can be acknowledged by the acknowledge buffer ready (ABR) bit.

When ABR acknowledges that the new linked list buffer is ready, the IDMA continues transferring data from the new linked list buffer.

When ABR indicates that the new linked list buffer is not ready, an IDMA transfer error is generated (see [Section : IDMA transfer error management](#)). Depending when the IDMA transfer error occurs, it normally causes the generation of an TXUNDERR or RXOVERR error. When a linked list buffer is not acknowledged in time the SDMMC transfer is stopped.

The ABR information is “don't care” when starting the linked list from software programmed register information. The first linked list buffer must be ready to be used before starting the SDMMC transfer.

IDMA transfer error management

An IDMA transfer error can occur:

- When reading or writing a reserved address space (for data or linked list information).
- When there is no more linked list buffer space to store received SDMMC data.
- When all linked list buffer data has been transferred and still more SDMMC data needs to be sent.
- When the availability of a linked list buffer is not acknowledged.

On an IDMA transfer error subsequent IDMA transfers are disabled and an IDMATE flag is set and hardware flow control is disabled. Depending when the IDMA transfer error occurs, it normally causes the generation of a TXUNDERR or RXOVERR error.

The behavior of the IDMATE flag depend on when the IDMA transfer error occurs during the SDMMC transfer:

- An IDMA transfer error is detected before any SDMMC transfer error (TXUNDERR, RXOVERR, DCRCFAIL, or DTIMEOUT):
 - The IDMATE flag is set at the same time as the SDMMC transfer error flag.
 - The TXUNDERR, RXOVERR, DCRCFAIL, or DTIMEOUT interrupt is generated.
- An IDMA transfer error is detected during a STOP_TRANSMISSION command:
 - The IDMATE flag is set at the same time as the DABORT flag.
 - The DABORT interrupt is generated.
- An IDMA transfer error is detected at the end of the SDMMC transfer (DHOLD, or DATAEND).
 - The IDMATE flag is set at the end of the SDMMC transfer.
 - A SDMMC transfer end interrupt is generated and a DHOLD or DATAEND flag is set.

The IDMATE is generated on an other SDMMC transfer interrupt (TXUNDERR, RXOVERR, DCRCFAIL, DTIMEOUT, DABORT, DHOLD, or DATAEND).

26.5.7 AHB and SDMMC_CK clock relation

The AHB must at least have 3x more bandwidth than the SDMMC bus bandwidth: for example, for SDR50 4-bit mode (50 Mbyte/s), the minimum sdmmc_hclk frequency is 37.5 MHz (150 Mbyte/s).

Table 268. AHB and SDMMC_CK clock frequency relation

SDMMC bus mode	SDMMC bus width	Maximum SDMMC_CK [MHz]	Minimum AHB clock [MHz]
e•MMC DS	8	26	19.5
e•MMC HS	8	52	39
e•MMC DDR52	8	52	78
e•MMC HS200	8	200	150
SD DS / SDR12	4	25	9.4
SD HS / SDR25	4	50	18.8
SD DDR50	4	50	37.5
SD SDR50	4	100	37.5
SD SDR104	4	208	78

26.6 Card functional description

26.6.1 SD I/O mode

The following features are SDMMC specific operations:

- SDIO interrupts
- SDIO suspend/resume operation (write and read suspend)
- SDIO Read Wait operation by stopping the clock
- SDIO Read Wait operation by SDMMC_D2 signaling

Table 269. SDIO special operation control

Operation mode	SDIOEN	RWMOD	RWSTOP	RWSTART	DTDIR
Interrupt detection	1	X	X	X	X
Suspend/Resume operation	X	X	X	X	X
Read Wait SDMMC_CK clock stop (START)	X	1	0	1	1
Read Wait SDMMC_CK clock stop (STOP)	X	1	1	1	1
Read Wait SDMMC_D2 signaling (START)	X	0	0	1	1
Read Wait SDMMC_D2 signaling (STOP)	X	0	1	1	1

SD I/O interrupts

To allow the SD I/O card to interrupt the host, an interrupt function is available on pin 8 (shared with SDMMC_D1 in 4-bit mode) on the SD interface. The use of the interrupt is optional for each card or function within a card. The SD I/O interrupt is level-sensitive, which means that the interrupt line must be held active (low) until it is either recognized and acted upon by the host or deasserted due to the end of the interrupt period. After the host has serviced the interrupt, the interrupt status bit is cleared via an I/O write to the appropriate bit in the SD I/O card internal registers. The interrupt output of all SD I/O cards is active low and the application must provide external pull-up resistors on all data lines (SDMMC_D[3:0]).

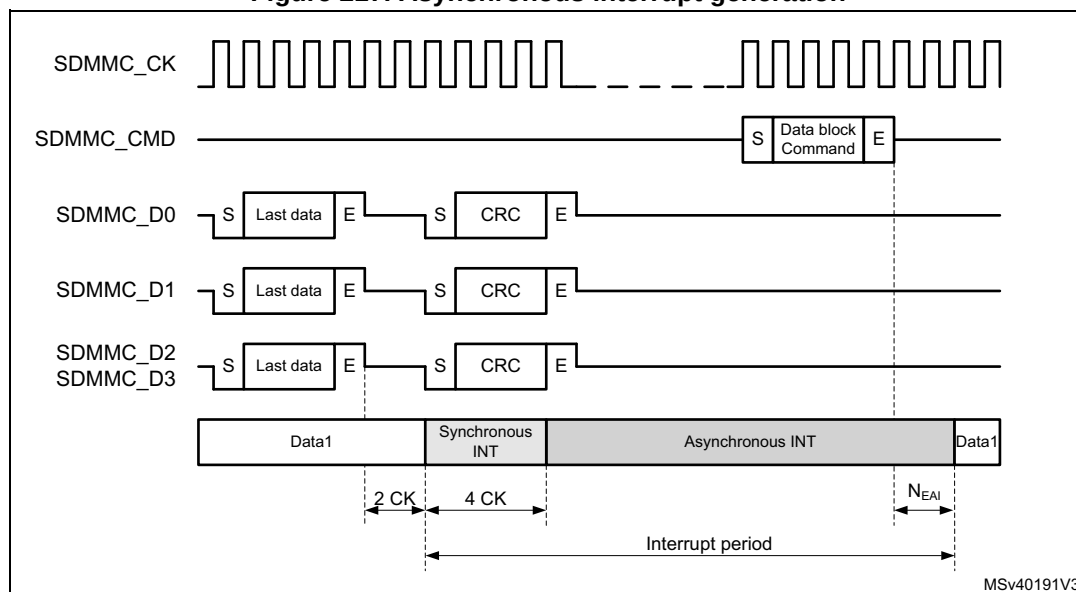
In SD 1-bit mode pin 8 is dedicated to the interrupt function (IRQ), and there are no timing constraints on interrupts.

In SD 4-bit mode the host samples the level of pin 8 (SDMMC_D1/IRQ) into the interrupt detector only during the interrupt period. At all other times, the host interrupt ignores this value. The interrupt period begins when interrupts are enabled at the card and SDIOEN bit is set see register settings in [Table 269](#).

In 4-bit mode the card can generate a synchronous or asynchronous interrupt as indicated by the card CCCR register SAI and EAI bits.

- Synchronous interrupt, require the SDMMC_CK to be active.
- Asynchronous interrupt, can be generated when the SDMMC_CK is stopped, 4 cycles after the start of the card interrupt period following the last data block.

Figure 217. Asynchronous interrupt generation



The timing of the interrupt period is depending on the bus speed mode.

In DS, HS, SDR12, and SDR25 mode, selected by register bit BUSSPEED, the interrupt period is synchronous to the SD clock.

- The interrupt period ends at the next clock from the end bit of a command that transfers data block(s) (Command sent with the CMDTRANS bit is set), or when the DTEN bit is set.
- The interrupt period resumes 2 SDMMC_CK after the completion of the data block.
- At the data block gap the interrupt period is limited to 2 SDMMC_CK cycles.

Note: *DTEN must not be used to start data transfer with SD and eMMC cards.*

Figure 218. Synchronous interrupt period data read

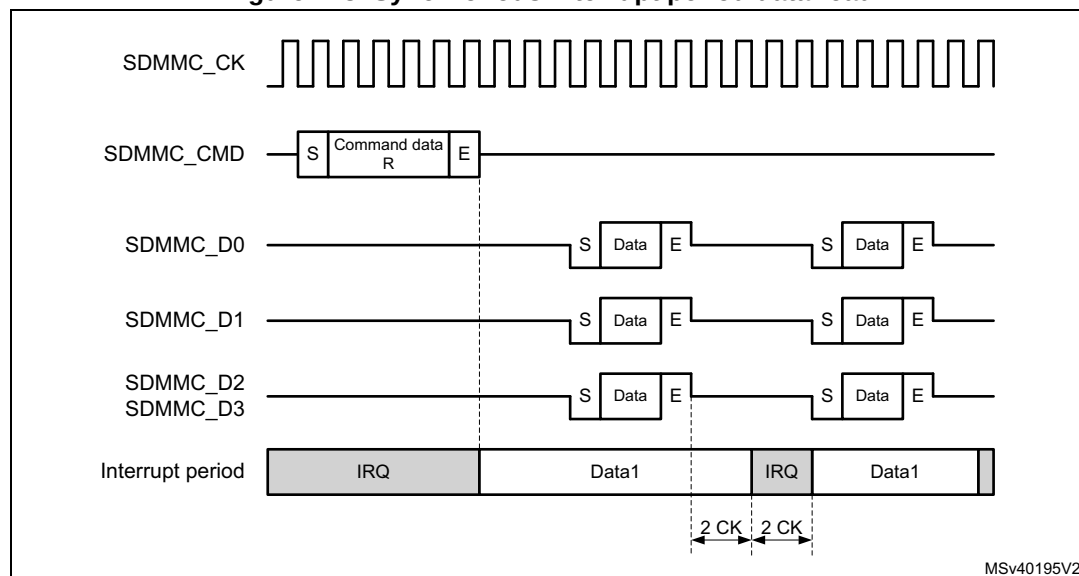
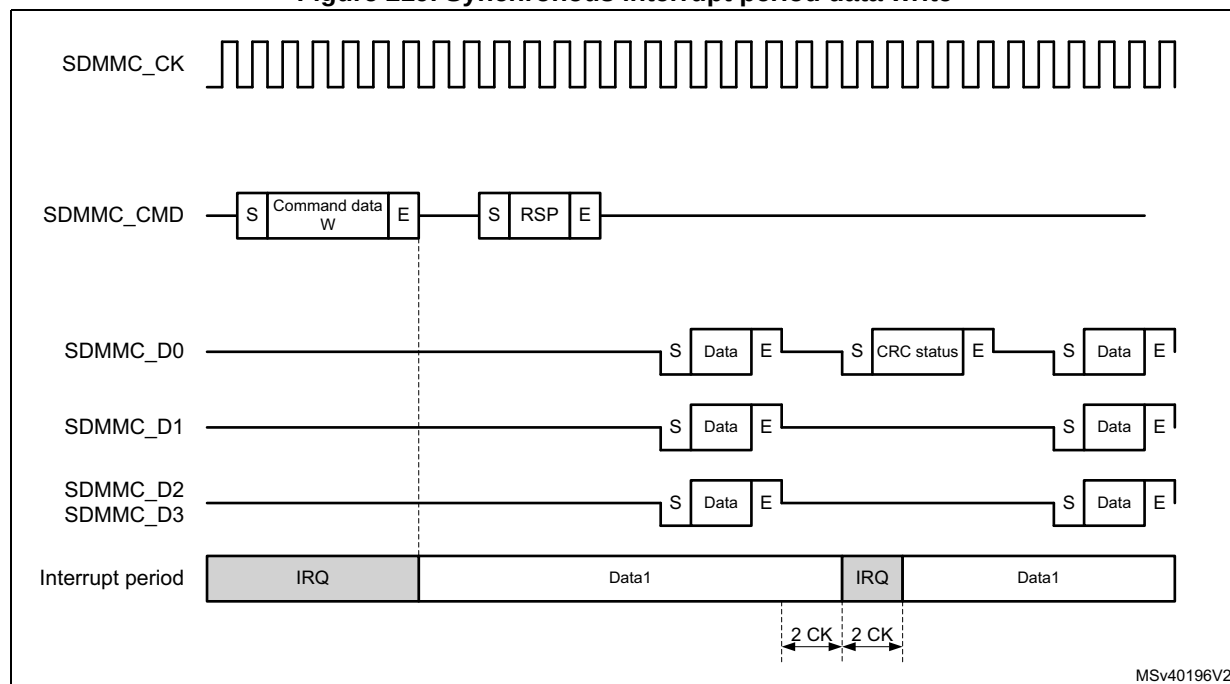


Figure 219. Synchronous interrupt period data write



In SDR50, SDR104, and DDR50, selected by register bit BUSSPEED, due to propagation delay from the card to host, the interrupt period is asynchronous.

- The card interrupt period ends after 0 to 2 SDMMC_CLK cycles after the end bit of a command that transfers data block(s) (Command sent with the CMDTRANS bit is set), or when the DTEN bit is set. At the host the interrupt period ends after the end bit of a command that transfers data block(s). A card interrupt issued in the 1 to 2 cycles after the command end bit are not detected by the host during this interrupt period.
- The card interrupt period resumes 2 to 4 SDMMC_CLK after the completion of the last data block. The host resumes the interrupt period always 2 cycles after the last data block.
- There is NO interrupt period at the data block gap.

Note: DTEN must not be used to start data transfer with SD and eMMC cards.

Figure 220. Asynchronous interrupt period data read

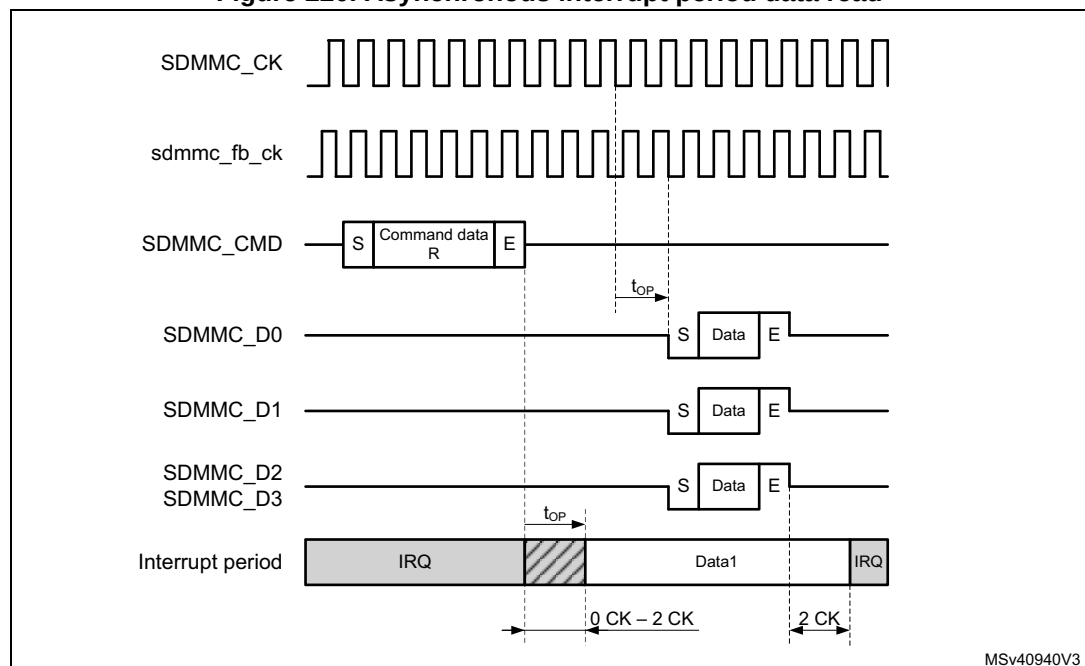
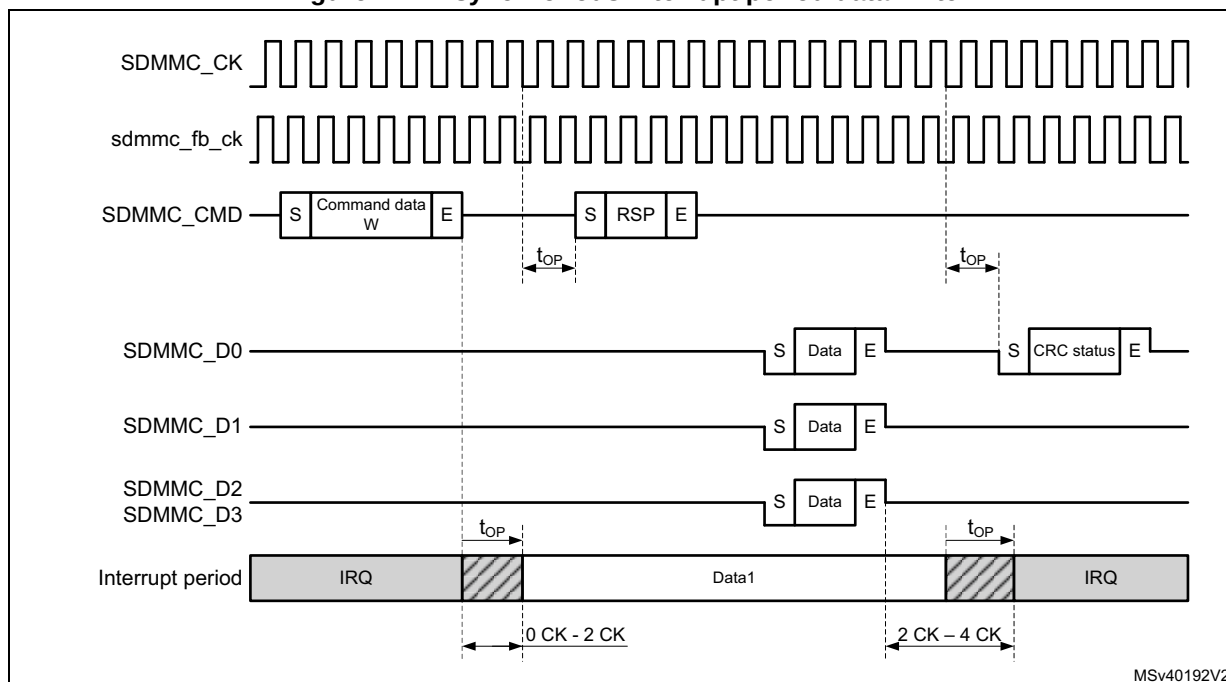


Figure 221. Asynchronous interrupt period data write



When transferring Open-ended multiple block data and using DTMODE “block data transfer ending with STOP_TRANSMISSION command”, the SDMMC masks the interrupt period after the last data block until the end of the CMD12 STOP_TRANSMISSION command.

The interrupt period is applicable for both memory and I/O operations.

In 4-bit mode interrupts can be differentiated from other signaling according [Table 270](#).

Table 270. 4-bit mode Start, interrupt, and CRC-status Signaling detection

SDMMC data line	Start	Interrupt	CRC-status
SDMMC_D0	0	1 or CRC-status	0
SDMMC_D1	0	0	X
SDMMC_D2	0	1 or Read Wait	X
SDMMC_D3	0	1	X

SD I/O suspend and resume

This function is NOT supported in SDIO version 4.00 or later.

Within a multifunction SD I/O or a card with both I/O and memory functions, there are multiple devices (I/O and memory) that share access to the eMMC/SD bus. To share access to the host among multiple devices, SD I/O and combo cards optionally implement the concept of suspend/resume. When a card supports suspend/resume, the host can temporarily halt (suspend) a data transfer operation to one function or memory to free the bus for a higher-priority transfer to a different function or memory. After this higher-priority transfer is complete, the original transfer is restarted (resume) where it left off.

To perform the suspend/resume operation on the bus, the host performs the following steps:

1. Determines the function currently using the SDMMC_D[3:0] line(s).
2. Requests the lower-priority or slower transaction to suspend.
3. Waits for the transaction suspension to complete.
4. Begins the higher-priority transaction.
5. Waits for the completion of the higher priority transaction.
6. Restores the suspended transaction.

The card receiving a suspend command responds with its current bus status. Only when the bus has been suspended by the card the bus status indicates suspension completed.

There are different suspend cases conditions:

- Suspend request accepted prior to the start of data transfer.
- Suspend request not accepted, (due to data being transfered at the same time), the host keeps checking the request until it is accepted (data transfer has suspended).
- Suspend request during write busy.
- Suspend request with write multiple.
- Suspend request during Read Wait.

For the host to know if the bus has been released it must check the status of the suspend request, suspension completed.

When the bus status of the suspend request response indicates suspension completed, the card has released the bus. At this time the state of the suspended operation must be saved where after an other operation can start.

The suspend command must be sent with the CMDSPEND bit set. This makes possible to start the interrupt period after the suspend command response when the bus is suspended (response bit BS = 0).

The hardware does not save the number of remaining data to be transfered when resuming the suspended operation. It is up to firmware to determine the data that has been transferred and resume with the correct remaining number of data bytes.

While receiving data from the card, the SDMMC can suspend the read operation after the read data block end (DPSM in Wait_R). After receiving the suspend acknowledgment response from the card the following steps must be taken by firmware:

1. The normal receive process must be stopped by setting DTHOLD bit.
 - a) The remaining number of data bytes in the FIFO must be read until the receive FIFO is empty (RXFIFOE flag is set), and when IDMAEN = 0 the FIFO must be reset with FIFORST.
2. The confirmation that all data has been read from the FIFO, and that the suspend is completed is indicated by the DHOLD flag.
 - a) The remaining number of data bytes (multiple of data blocks) still to be read when resuming the operation must be determined from the remaining number of bytes indicated by the DATACOUNT.

Note: When a DTIMEOUT flag occurs during the suspend procedure, this must be ignored.

To resume receiving data from the card, the following steps must be taken by firmware:

1. The remaining number of data bytes (multiple of data blocks) must be programmed in DATALENGTH.
2. The DPSM must be configured to receive data in the DTDIR bit.
3. The resume command must be sent from the CPSM, with the CMDTRANS bit set and the CMDSUSPEND bit set, which ends the interrupt period when data transfer is resumed (response bit DF = 1) and enabled the DPSM, after which the card resumes sending data.

While sending data to the card, the SDMMC can suspend the write operation after the write data block CRC status end (DPSM in Busy). Before sending the suspend command to the card the following steps must be taken by firmware:

1. Enable DHOLD flag (and DBCKEND flag when IDMAEN = 0)
2. The DPSM must be prevented from start sending a new data block by setting DTHOLD.
3. When IDMAEN = 0: When receiving the DBCKEND flag the data transfer is stopped. Firmware can stop filling the FIFO, after which the FIFO must be reset with FIFORST. Any bytes still in the FIFO need to be rewritten when resuming the operation.
4. When receiving the DHOLD flag the data transfer is stopped. The remaining number of data bytes still to be written when resuming must be determined from the remaining number of bytes indicated by the DATACOUNT.
5. To suspend the card the suspend command must be sent by the CPSM with the CMDSUSPEND bit set. This makes possible to start the interrupt period after the suspend command response when the bus is suspended (response bit BS = 0).

To resume sending data to the card, the following steps must be taken by firmware:

1. The remaining number of data bytes must be programmed in DATALENGTH.
2. The DPSM must be configured for transmission with DTDIR set and enabled by having the CPSM send the resume command with the CMDTRANS bit set and the CMDSUSPEND bit set. This ends the interrupt period and start the data transfer. The DPSM either goes to the Wait_S state when SDMMC_D0 does not signal busy, or goes to the Busy state when busy is signaled.
3. When IDMAEN = 1: The IDMA needs to be reprogrammed for the remaining bytes to be transferred.
4. When IDMAEN = 0: Firmware must start filling the FIFO with the remaining data.

SD I/O Read Wait

There are two methods to pause the data transfer during the block gap:

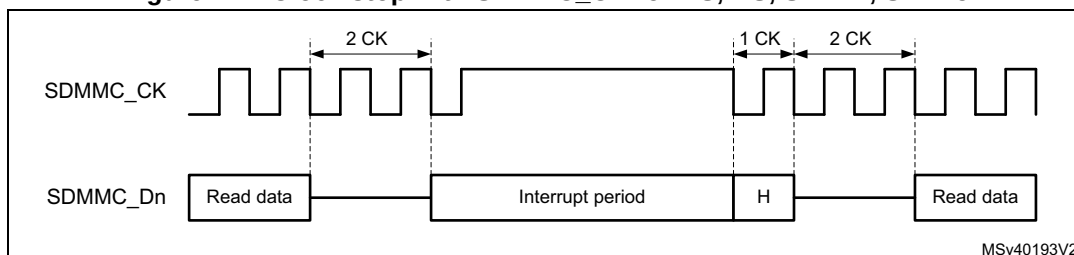
1. Stopping the SDMMC_CK.
2. Using Read Wait signaling on SDMMC_D2.

The SDMMC can perform a Read Wait with register settings according [Table 269](#).

Depending the SDMMC operation mode (DS, HS, SDR12, SDR25) or (SDR50, SDR104, DDR) each method has a different characteristic.

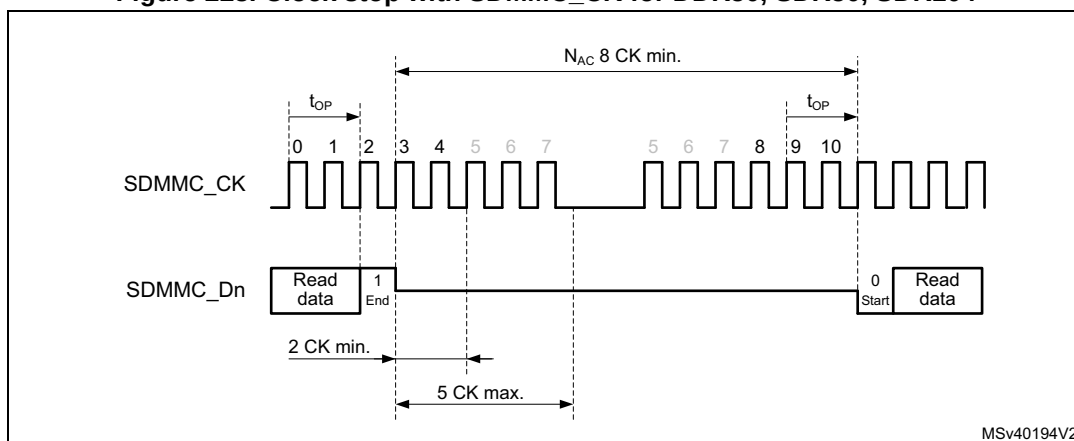
The timing for pause read operation by stopping the SDMMC_CK for DS, HS, SDR12, and SDR25, the SDMMC_CK may be stopped 2 SDMMC_CK cycles after the end bit. When ready the host resumes by restarting clock (see [Figure 222](#)).

Figure 222. Clock stop with SDMMC_CK for DS, HS, SDR12, SDR25



The timing for pause read operation by stopping the SDMMC_CK for SDR50, SDR104, and DDR50, the SDMMC_CK may be stopped minimum 2 SDMMC_CK cycles and maximum 5 SDMMC_CK cycles, after the end bit. When ready the host resumes by restarting clock, see [Figure 223](#). (In DDR50 mode the SDMMC_CK must only be stopped after the falling edge, when the clock line is low.)

Figure 223. Clock stop with SDMMC_CK for DDR50, SDR50, SDR104

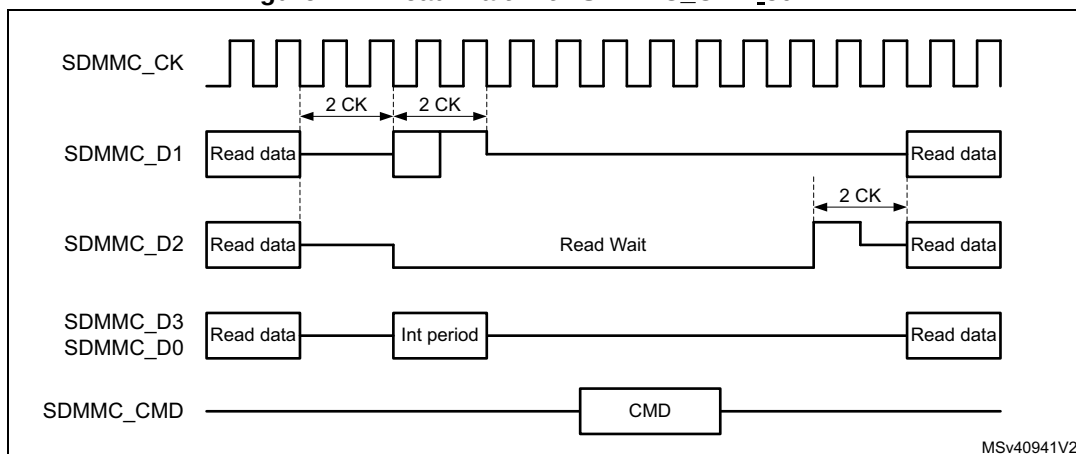


In Read Wait SDMMC_CK clock stopping, when RWSTART is set, the DSPM stops the clock after the end bit of the current received data block CRC. The clock start again after writing 1 to the RWSTOP bit, where after the DSPM waits for a start bit from the card.

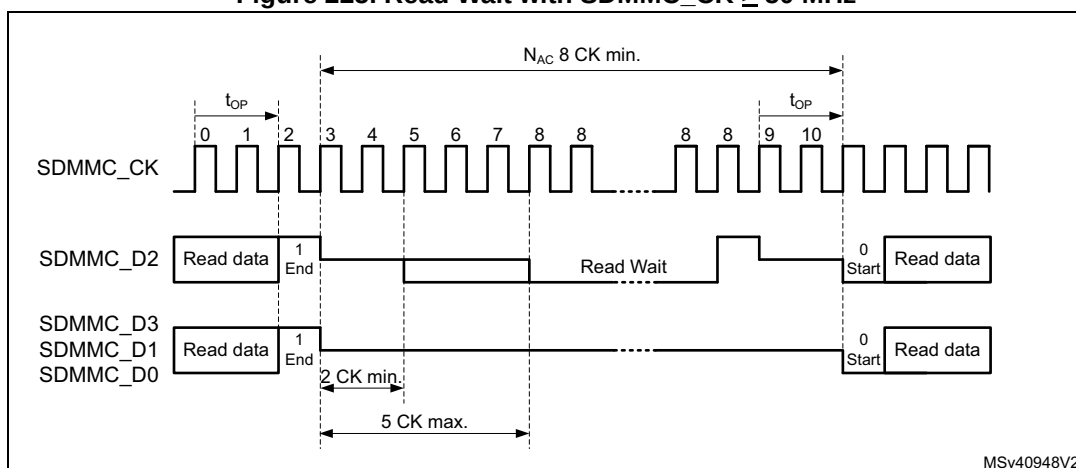
As SDMMC_CK is stopped, no command can be issued to the card. During a Read Wait interval, the SDMMC can still detect SDIO interrupts on SDMMC_D1.

The optional Read Wait signaling on SDMMC_D2 (RW) operation is defined only for the SD 1-bit and 4-bit modes. The Read Wait operation enables the host to signal a card that is reading multiple registers (IO_RW_EXTENDED, CMD53) to temporarily stall the data transfer while allowing the host to send commands to any function within the SD I/O device. To determine when a card supports the Read Wait protocol, the host must test capability bits in the internal card registers.

The timing for Read Wait with a SDMMC_CK less than 50MHz (DS, HS, SDR12, SDR25) is based on the interrupt period generated by the card on SDMMC_D1. The host by asserting SDMMC_D2 low during the interrupt period requests the card to enter Read Wait. To exit Read Wait the host must raise SDMMC_D2 high during one SDMMC_CK cycles before making it Hi-Z, see [Figure 224](#).

Figure 224. Read Wait with SDMMC_CK ≤ 50 MHz

For SDR50, SDR104 with a SDMMC_CK more than 50MHz, and DDR50, the card treats the Read Wait request on SDMMC_D2 as an asynchronous event. The host by asserting SDMMC_D2 low after minimum 2 SDMMC_CK cycles and maximum 5 SDMMC_CK cycles, request the card to enter Read Wait. To exit Read Wait the host must raise SDMMC_D2 high during one SDMMC_CK cycles before making it Hi-Z. The host must raise SDMMC_D2 on the SDMMC_CK clock (see [Figure 225](#)).

Figure 225. Read Wait with SDMMC_CK ≥ 50 MHz

In Read Wait SDMMC_D2 signaling, when RWSTART is set, the DPSM drives SDMMC_D2 after the end bit of the current received data block CRC. The Read Wait signaling on SDMMC_D2 is removed when writing 1 to the RWSTOP bit. The DPSM remains in R_W state for two more SDMMC_CK clock cycles to drive SDMMC_D2 to 1 for one clock cycle (in accordance with SDIO specification), where after the DPSM waits for a start bit from the card.

During the Read Wait signaling on SDMMC_D2 commands can be issued to the card. During the Read Wait interval, the SDMMC can detect SDIO interrupts on SDMMC_D1.

26.6.2 CMD12 send timing

CMD12 is used to stop/abort the data transfer, the card data transmission is terminated two clock cycles after the end bit of the Stop Transmission command.

Table 271. CMD12 use cases

Data operation	Stop Transmission command CMD12 Description
SDMMC stream write	The data transfer is stopped/aborted by sending the Stop Transmission command.
SDMMC open ended multiple block write	The data transfer is stopped/aborted by sending the Stop Transmission command. If the card detects an error, the host must abort the operation by sending the Stop Transmission command.
SDMMC block write with predefined block count	The Stop Transmission command is not required at the end of this type of multiple block write. (sending the Stop Transmission command after the card has received the last block is regarded as an illegal command.) If the card detects an error, the host must abort the operation by sending the Stop Transmission command.
SDMMC stream read	The data transfer is stopped/aborted by sending the Stop Transmission command.
SDMMC open ended multiple block read	The data transfer is stopped/aborted by sending the Stop Transmission command. If the card detects an error, the host must abort the operation by sending the Stop Transmission command.
SDMMC block read with predefined block count	The Stop Transmission command is not required at the end of this type of multiple block read. (sending the Stop Transmission command after the card has transmitted the last block is regarded as an illegal command.) Transaction can be aborted by sending the Stop Transmission command. If the card detects an error, the host must abort the operation by sending the Stop Transmission command.

All data write and read commands can be aborted any time by a Stop Transmission command CMD12. The following data abort procedure applies during an ongoing data transfer:

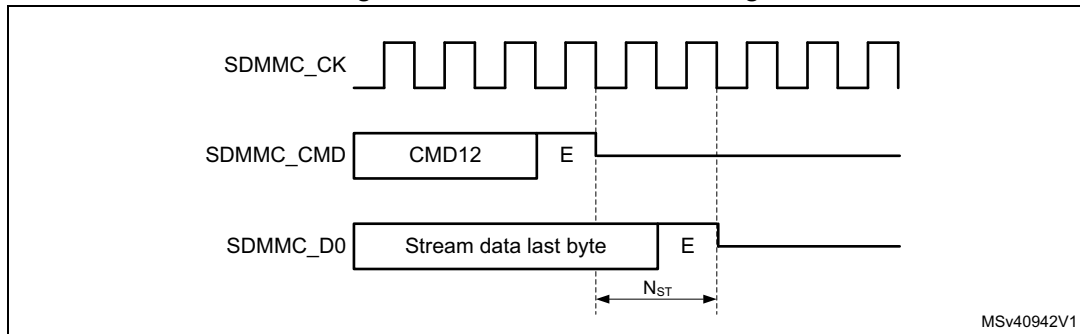
1. Load CMD12 Stop Transmission command in registers and set the CMDSTOP bit.
 - This causes the CPSM Abort signal to be generated when the command is sent to the DPSM.
2. Configure the CPSM to send a command immediately (clear WAITPEND bit).
 - The card, when sending data, stops data transfer 2 cycles after the Stop Transmission command end bit.
The card when no data is being sent, does not start sending any new data.
 - The host, when sending data, sends one last data bit followed by an end bit after the Stop Transmission command end bit.
The host when not sending data, does not start sending any new data.
3. When IDMAEN = 0, the FIFO need to be reset with FIFORST.
 - When writing data to the card. On the CMDREND flag, firmware must stop writing data to the FIFO. Subsequently the FIFO must be reset with FIFORST, this flushes the FIFO.
 - When reading data from the card. On the CMDREND flag, firmware must read the remaining data from the FIFO. Subsequently the FIFO must be reset with FIFORST.
4. When IDMAEN = 1, hardware takes care of the FIFO.
 - When writing data to the card. On the CPSM Abort signal, hardware stops the IDMA and subsequently the FIFO is flushed.
 - When reading data from the card. On the CPSM Abort signal, hardware instructs the IDMA to transfer the remaining data from the FIFO to RAM.
5. When the FIFO is empty/reset the DABORT flag is generated.

Stream operation and CMD12

To stop the stream transfer after the last byte to be transfered, the CMD12 end bit timing must be sent aligned with the data stream end of last byte. The following write stream data procedure applies:

1. Initialize the stream data in the DPSM, DTMODE = MCC stream data transfer.
2. Send the WRITE_DATA_STREAM command from the CPSM with CMDTRANS = 1.
3. Preload CMD12 in command registers, with the CMDSTOP bit set.
4. Configure the CPSM to send a command only after a wait pending (WAITPEND = 1) end of last data (according DATALENGTH).
5. Enabling the CPSM to send the STOP_TRANSMISSION command, the stream data end bit and command end bit are aligned.
 - When DATALENGTH > 5 bytes, Command CMD12 is waited in the CPSM to be aligned with the data transfer end bit.
 - When DATALENGTH < 5 bytes, Command CMD12 is started before and the DPSM remains in the Wait_S state to align the data transfer end with the CMD12 end bit.
6. The write stream data can be aborted any time by clearing the WAITPEND bit. This causes the Preloaded CMD12 to be sent immediately and stop the write data stream.

Figure 226. CMD12 stream timing



To stop the read stream transfer after the last byte, the CMD12 end bit timing must occur after the last data stream byte. The following read stream data procedure applies:

1. Wait for all data to be received by the DPSM and read from the FIFO (DATAEND flag).
 - The DPSM does not receive more data than indicated by DATALENGTH, even if the card is sending more data.
2. Send CMD12 by the CPSM.
 - CMD12 stops the card sending data.

Note: The SDMMC does not receive any more data from the card when $DATACOUNT = 0$, even when the card continues sending data.

Block operation and CMD12

To stop block transfer at the end of the data, the CMD12 end bit must be sent after the last block end bit.

When writing data to the card the CMD12 end bit must be sent after the write data block CRC token end bit. This requires the CMD12 sending to be tied to the data block transmission timing. To stop an Open-ended Multiple block write, the following procedure applies:

1. Before starting the data transfer, set DTMODE to “block data transfer ending with STOP_TRANSMISSION command”.
2. Wait for all data to be sent by the DPSM and the CRC token to be received, (DATAEND flag).
 - The DPSM does not send more data than indicated by DATALENGTH.
3. Send CMD12 by the CPSM.
 - CMD12 sets the card to Idle mode.

When reading data from the card the CMD12 end bit must be sent earliest at the same time as the card read data block last data bit. This requires the CMD12 sending to be tied to the data block reception timing. The following stop Open-ended Multiple block read data block procedure applies:

1. Before starting the data transfer, set DTMODE to "block data transfer ending with STOP_TRANSMISSION command".
2. Wait for all data to be received by the DPSM and read from the FIFO (DATAEND flag).
 - The DPSM does not receive more data than indicated by DATALENGTH, even if the card is sending more data.
3. Send CMD12 with CMDSTOP bit set by the CPSM.
 - CMD12 stops the Card sending more data and set the card to Idle mode. Any ongoing block transfer is aborted by the Card.

Note: The SDMMC does not receive any more data from the card when `DATACOUNT = 0`, even when the card continues sending data.

26.6.3 Sleep (CMD5)

The eMMC card may be switched between a Sleep state and a Standby state by CMD5. In the Sleep state the power consumption of the card is minimized and the Vcc power supply may be switched off.

The CMD5 (SLEEP) is used to initiate the state transition from Standby state to Sleep state. The card indicates Busy, pulling down `SDMMC_DO`, during the transition phase. The Sleep state is reached when the card stops pulling down the `SDMMC_DO` line.

To set the card into Sleep state the following procedure applies:

1. Enable interrupt on `BUSYD0END`.
2. Send CMD5 (SLEEP).
3. On `BUSYD0END` interrupt, card is in Sleep state.
4. Vcc power supply can be switched off.

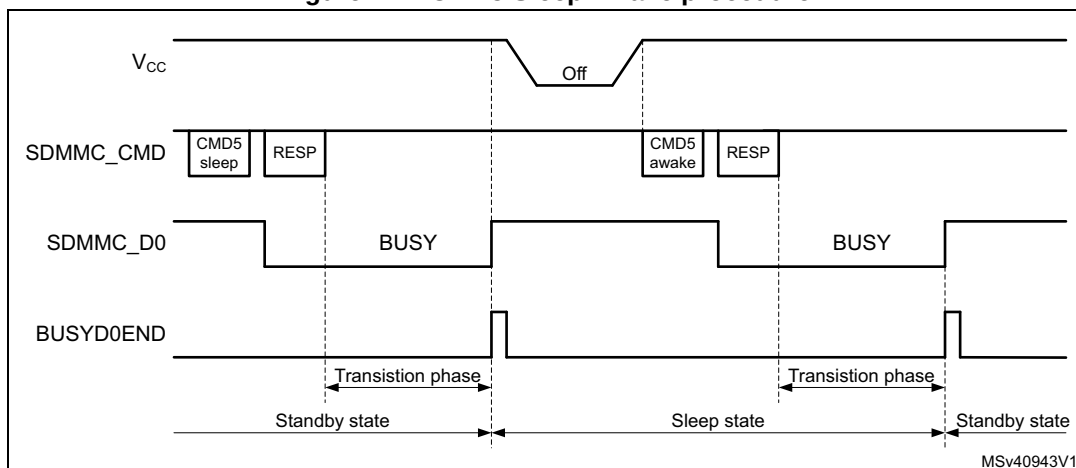
The CMD5 (AWAKE) is used to initiate the state transition from Sleep state to Standby state. The card indicates Busy, pulling down `SDMMC_DO`, during the transition phase. The Standby state is reached when the card stops pulling down the `SDMMC_DO` line.

To set the card into Sleep state the following procedure applies:

1. Switch on Vcc power supply and wait unit minimum operating level is reached.
2. Enable interrupt on `BUSYD0END`.
3. Send CMD5 (AWAKE).
4. On `BUSYD0END` interrupt card is in Standby state.

The Vcc power supply can be switched off only after the Sleep state has been reached. The Vcc supply must be reinstalled before CMD5 (AWAKE) is sent.

Figure 227. CMD5 Sleep Awake procedure



26.6.4 Interrupt mode (Wait-IRQ)

The host and card enter and exit interrupt mode (Wait-IRQ) simultaneously. In interrupt mode there is no data transfer. The only message allowed is an interrupt service request response from the card or the host. For the interrupt mode to work correctly the SDMMC_CLK frequency must be set in accordance with the achievable SDMMC_CMD data rate in Open Drain mode, which depend on the capacitive load and pull-up resistor. The CLKDIV must be set >1, and the SETCLKRX must select either the sdmmc_io_in_ck or SDMMC_CLKin source.

The host must ensure that the card is in Standby state before issuing the CMD40 (GO_IRQ_STATE). While waiting for an interrupt response the SDMMC_CLK clock signal must be kept active.

A card in interrupt mode (IRQ state):

- is waiting for an internal card interrupt event. Once the event occurs, the card starts to send the interrupt service request response. The response is sent in open-drain mode.
- while waiting for the internal card interrupt event, the card also monitors the SDMMC_CMD line for a start bit. Upon detection of a start bit the card aborts the interrupt mode and switch to Standby state.

The host in interrupt mode (CPSM Wait state waiting for interrupt):

- is waiting for a card interrupt service request response (start bit).
- while waiting for a card interrupt service request response the host may abort the interrupt mode (by clearing the WAITINT register bit), which causes the host to send a interrupt service request response R5 with RCA = 0x0000 in open-drain mode.

When sending the interrupt service request response, the sender bit-wise monitors the SDMMC_CMD bit stream. The sender whose interrupt service request response bit does not correspond to the bit on the SDMMC_CMD line stops sending. In the case of multiple senders only one successfully sends its full interrupt service request response. If the host sends simultaneously, it loses sending after the transmission bit.

To handle the interrupt mode, the following procedure applies:

1. Set the SDMMC_CK frequency in accordance with the achievable SDMMC_CMD data rate in Open-drain mode, CLKDIV must be set >1, and SETCLKRX must select the sdmmc_io_in_ck.
2. Load CMD40 (GO_IRQ_STATE) in the command registers.
3. Enable wait for interrupt by setting WAITINT register bit.
4. Configure the CPSM to send a command immediately.
 - This causes the CMD40 to be sent and the CPSM to be halted in the Wait state, waiting for a interrupt service request response.
5. To exit the wait for interrupt state (CPSM Wait state):
 - Upon the detection of an interrupt service request response start bit the CPSM moves to the Receive state where the response is received. The complete reception of the response is indicated by the CMDREND or the command CRC error flags.
 - To abort the interrupt mode the host clears the WAITINT register bit, which causes the host to send an interrupt service request response by itself. This moves the CPSM to the Receive state. The complete reception of the response is indicated by the CMDREND or the command CRC error flags.

Note: On a simultaneous send interrupt service request response start bit collision the host loses the bus access after the transmission bit.

26.6.5 Boot operation

In boot operation mode the host can read boot data from the card by either one of the two boot operation functions:

- Normal boot. (keeping CMD line low)
- Alternative boot (sending CMD0 with argument 0xFFFFFFFFFA)

The boot data can be read according the following configuration options, depending on card register settings:

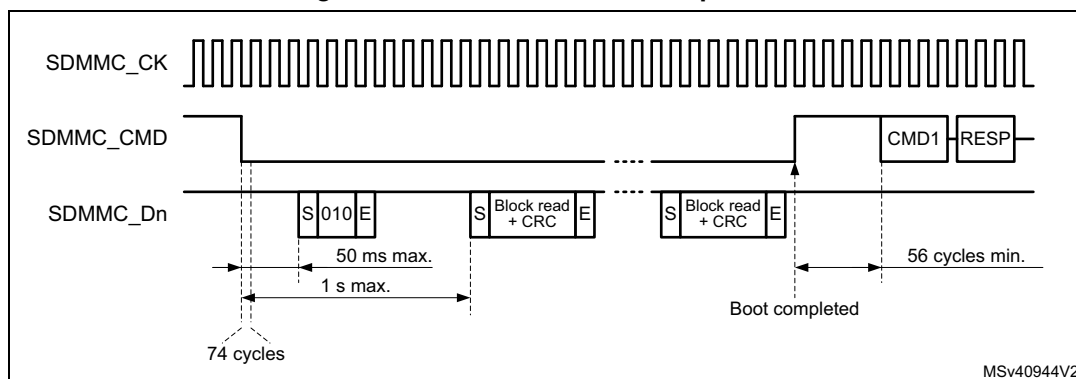
- The partition from which boot data is read (EXT_CSD Byte[179])
- The boot data size (EXT_CSD Byte[226])
- The bus configuration during boot (EXT_CSD Byte[177])
- Receiving boot acknowledgment from the card. (EXT_CSD Byte[179])

If boot acknowledgment is enabled the card send pattern 010 on SDMMC_D0 within 50ms after boot mode has been requested by either CMD line going low or after CMD0 with argument 0xFFFFFFFFFA. A boot acknowledgment timeout (ACKTIMEOUT) and acknowledgment status (ACKFAIL) is provided.

Normal boot operation

If the SDMMC_CMD line is held low for at least 74 clock cycles after card power-up or reset, before the first command is issued, the card recognizes that boot mode is being initiated. Within 1 second after the CMD line goes low, the card starts to sent the first boot code data on the SDMMC_Dn line(s). The host must keep the SDMMC_CMD line low until after all boot data has been read. The host can terminate boot mode by pulling the SDMMC_CMD line high.

Figure 228. Normal boot mode operation



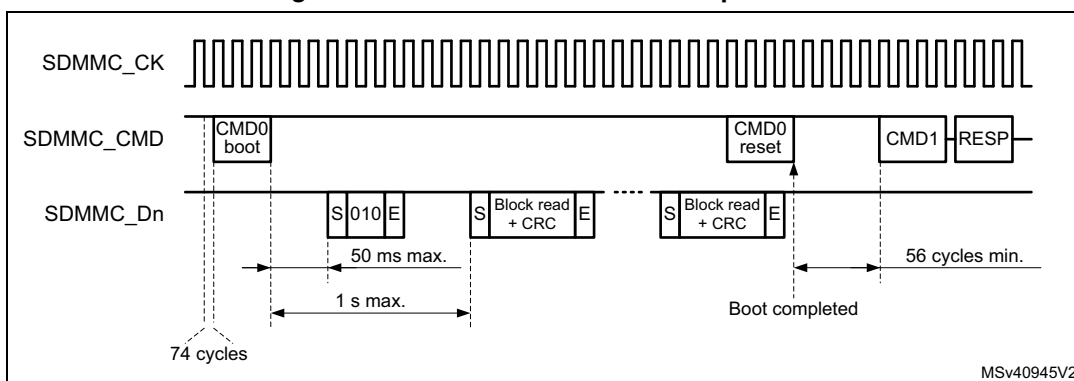
To perform the normal boot procedure the following steps needed:

1. Reset the card.
2. if a boot acknowledgment is requested enable the BOOTACKEN and set the ACKTIME and enable the ACKFAIL and ACKTIMEOUT interrupt.
3. enable the data reception by setting the DPSM in receive mode (DTPDIR) and the number of data bytes to be received in DATALENGTH.
4. Enable the DTIMEOUT, DATAEND, and CMDSENT interrupts for end of boot command confirmation.
5. Select the normal boot operation mode in BOOTMODE, and enable boot in BOOTEN. The boot procedure is started by enabling the CPSM with CPSMEN. This causes:
 - the SDMMC_CMD to be driven low. (BOOTMODE = normal boot).
 - the ACK timeout to start.
 - DPSM to be enabled.
6. The incorrect reception of the boot acknowledgment can be detected with ACKFAIL flag or ACKTIMEOUT flag when enabled.
 - when an incorrect boot acknowledgment is received the ACKFAIL flag occurs.
 - when the boot acknowledgment is not received in time the ACKTIMEOUT flag occurs.
7. when all boot data has been received the DATAEND flag occurs.
 - when data CRC fails the DCRCFAIL flag is also generated.
 - when the data timeout occurs the DTIMEOUT flag is also generated.
8. When last data has been received, read data from the FIFO until FIFO is empty after which end of data DATAEND flag is generated.
 - SDMMC has completely received all data and the DPSM is disabled.
9. The boot procedure is terminated by firmware clearing BOOTEN, which causes the SDMMC_CMD line to go high. The CMDSENT flag is generated 56 cycles later to indicate that a new command can be sent.
 - If the boot procedure is aborted by firmware before all data has been received the CPSM Abort signal stops data reception and disables the DPSM which triggers an DABORT flag when enabled.
10. The CMDSENT flag signals the end of the boot procedure and the card is ready to receive a new command.

Alternative boot operation

After card power-up or reset, if the host send CMD0 with the argument 0xFFFFFFFF after 74 clock cycles before CMD0 is issued, the card recognizes that boot mode is being initiated. Within 1 second after the CMD0 with argument 0xFFFFFFFF has been sent, the card starts to send the first boot code data on the SDMMC_Dn line(s). The master terminates boot operation by sending CMD0 (Reset).

Figure 229. Alternative boot mode operation



To perform the alternative boot procedure the following steps needed:

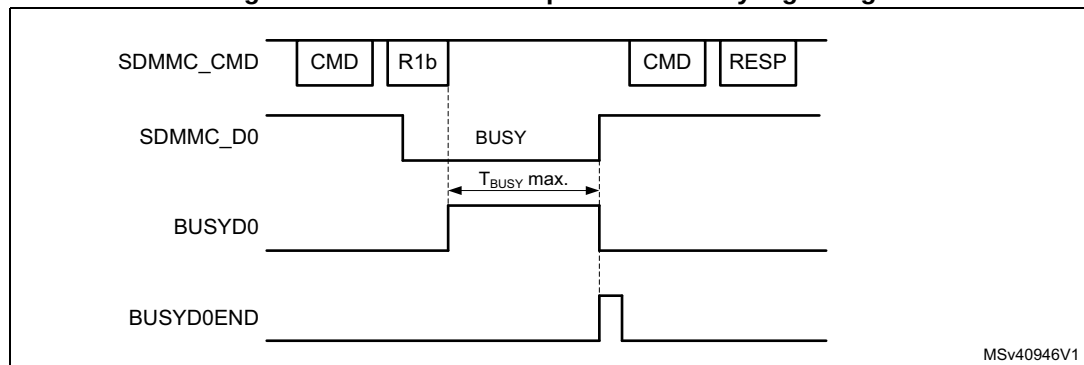
1. Move the SDMMC to power-off state, and reset the card.
2. Move the SDMMC to power-on state. This guarantees the 74 SCDMMC_CK cycles to be clocked before any command.
3. if a boot acknowledgment is requested enable the BOOTACKEN and set the ACKTIME and enable the ACKTIMEOUT flag.
4. enable the data reception by setting the DPSM in receive mode (DTPDIR) and the number of data to be received in DATALENGTH. Enable the DTIMEOUT and DATAEND flags.
5. Select the alternative boot operation mode in BOOTMODE, load the CMD0 with the 0xFFFFFFFF argument in the command registers. Enable CMDSENT flag for end of boot command confirmation, and enable boot in BOOTEN. The boot procedure is started by enabling the CPSM with CPSMEN. This causes:
 - the loaded command and argument to be sent out. (BOOTMODE = alternative boot).
 - the ACK timeout to start.
 - DPSM to be enabled.
6. When the command has been sent the CMDSENT flag is generated, at which time the BOOTEN bit must be cleared.
7. the reception of the boot acknowledgment can be detected with ACKFAIL flag when enabled.
 - when the boot acknowledgment is not received in time the ACKTIMEOUT flag occurs.
8. when all boot data has been received the DATAEND flag occurs.
 - when data CRC fails the DCRCFAIL flag is also generated.
 - when the data timeout occurs the DTIMEOUT flag is also generated.

9. When last data has been received, read data from the FIFO until FIFO is empty after which end of data DATAEND flag is generated.
 - SDMMC has completely received all data and the DPSM is disabled.
10. The BOOTEN bit must be cleared, before terminating the boot procedure by sending CMD0 (Reset) with BOOTMODE = alternative boot. This causes the CMDSENT flag to occur 56 cycles after the Command.
 - if the boot procedure is aborted by firmware before all data has been received the CPSM Abort signal stops the data transfer and disable the DPSM which triggers an DABORT flag when enabled.
11. The CMDSENT flag signals the end of the boot procedure and the card is ready to receive a new command. When the RESET command has been sent successfully, the BOOTMODE control bit has to be cleared to terminate the boot operation.

26.6.6 Response R1b handling

When sending commands which have a R1b response the busy signaling is reflected in the BUSYD0 register bit and the release of busy with the BUSYD0END flag. The SDMMC_D0 line is sampled at the end of the R1b response and signaled in the BUSYD0 register bit. The BUSYD0 register bit is reset to not busy when the SDMMC_D0 line release busy, at the same time the BUSYD0END flag is generated.

Figure 230. Command response R1b busy signaling



The expected maximum busy time must be set in the DATATIME register before sending the command. When enabled, the DTIMEOUT flag is set when after the R1b response busy stays active longer then the programmed time.

To detect the SDMMC_D0 busy signaling when sending a Command with R1b response the following procedure applies:

- Enable CMDREND flag.
- Send Command through CPSM.
- On the CMDREND flag check the BUSYD0 register bit.
 - If BUSYD0 signals not busy, signal busy release to firmware
 - If BUSYD0 signals busy, wait for BUSYD0END flag
- On BUSYD0END flag signal busy released to firmware.
- On DTIMEOUT flag busy is active longer then programmed time.

26.6.7 Reset and card cycle power

Reset

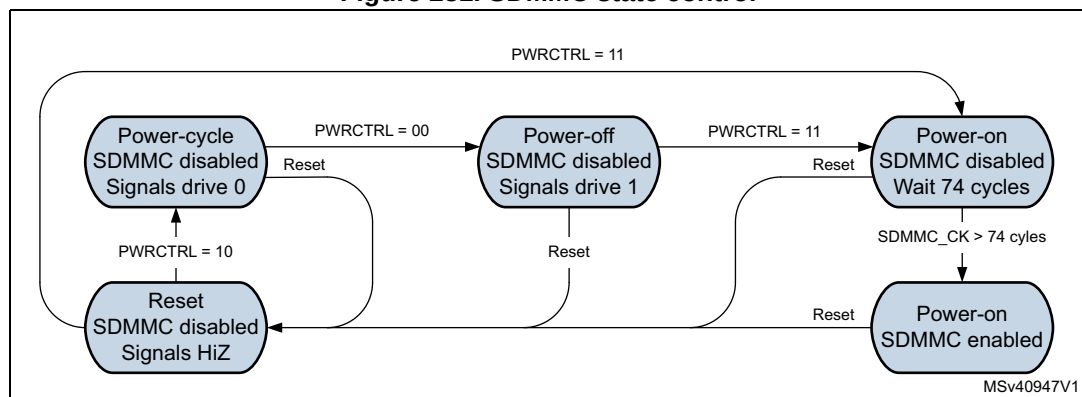
Following reset the SDMMC is in the reset state. In this state the SDMMC is disabled and no command nor data can be transferred. The SDMMC_D[7:0], and SDMMC_CMD are in HiZ and the SDMMC_CK is driven low.

Before moving to the power-on state the SDMMC must be configured.

In the power-on state the SDMMC_CK clock is running. First 74 SDMMC_CK cycles are clocked after which the SDMMC is enabled and command and data can be transferred.

The SDMMC states are controlled by Firmware with the PWRCTRL register bits according [Figure 231](#).

Figure 231. SDMMC state control

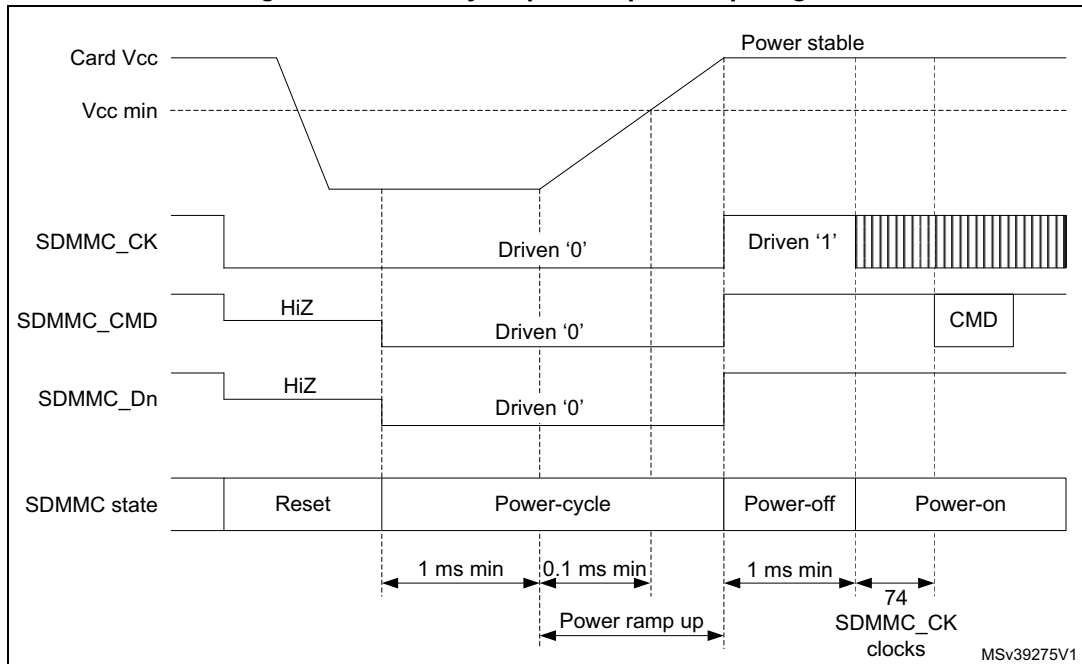


Card cycle power

To perform a card cycle power the following procedure applies:

1. Reset the SDMMC with the RCC.SDMMCxRST register bit. This resets the SDMMC to the reset state and the CPSM and DPSM to the Idle state.
2. Disable the Vcc power to the card.
3. Set the SDMMC in power-cycle state. This makes that the SDMMC_D[7:0], SDMMC_CMD and SDMMC_CK are driven low, to prevent the card from being supplied through the signal lines.
4. After minimum 1 ms enable the Vcc power to the card.
5. After the power ramp period set the SDMMC to the power-off state for minimum 1 ms. The SDMMC_D[7:0], SDMMC_CMD and SDMMC_CK are set to drive 1.
6. After the 1 ms delay set the SDMMC to power-on state in which the SDMMC_CK clock is enabled.
7. After 74 SDMMC_CK cycles the first command can be sent to the card.

Figure 232. Card cycle power / power up diagram



26.7 Hardware flow control

The hardware flow control during data transfer functionality is used to avoid FIFO underrun (TX mode) and overrun (RX mode) errors.

The behavior is to stop SDMMC_CK during data transfer and freeze the SDMMC state machines. The data transfer is stalled when the FIFO is unable to transmit or receive data. The data transfer remains stalled until the transmit FIFO is half full or all data according DATALENGTH has been stored, or until the receive FIFO is half empty. Only state machines clocked by SDMMC_CK are frozen, the AHB interfaces are still alive. The FIFO can thus be filled or emptied even if flow control is activated.

On an IDMA linked list transfer error, the hardware flow control is disabled. As a consequence, depending on when the IDMA linked list transfer error occurs, an underrun or overrun error may also occur (see [Section : IDMA transfer error management](#)).

To enable hardware flow control during data transfer, the HWFC_EN register bit must be set to 1. After reset hardware flow control is disabled.

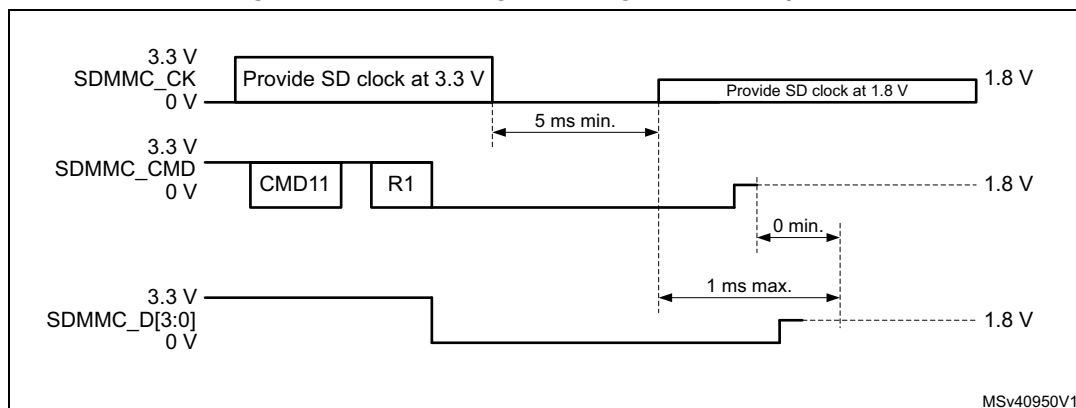
Hardware flow control must only be used when the SDMMC_Dn data is cycle-aligned with the SDMMC_CK. Whenever the sdmmc_fb_ck from the DLYB delay block is used, i.e in the case of SDR104 mode with a t_{OP} and Dt_{OP} delay > 1 cycle, hardware flow control can not be used.

26.8 Ultra-high-speed phase I (UHS-I) voltage switch

UHS-I mode (SDR12, SDR25, SDR50, SDR104, and DDR50) requires the support for 1.8 V signaling. After power up the card starts in 3.3V mode. CMD11 invokes the voltage switch

sequence to the 1.8V mode. When the voltage sequence is completed successfully the card enters UHS-I mode with default SDR12 and card input and output timings are changed.

Figure 233. CMD11 signal voltage switch sequence



To perform the signal voltage switch sequence the following steps are needed:

- Before starting the Voltage Switch procedure, the SDMMC_CK frequency must be set in the range 100 kHz - 400 kHz.
- The host starts the Voltage Switch procedure by setting the VSWITCHEN bit before sending the CMD11.
- The card returns an R1 response.
 - if the response CRC is pass, the Voltage Switch procedure continues the host does no longer drive the CMD and SDMMC_D[3:0] signals until completion of the voltage switch sequence. Some cycles after the response the SDMMC_CK is stopped and the CKSTOP flag is set.
 - if the response CRC is fail (CCRCFAIL flag) or no response is received before the timeout (CTIMEOUT flag), the Voltage Switch procedure is stopped.
- The card drives CMD and SDMMC_D[3:0] to low at the next clock after the R1 response.
- The host, after having received the R1 response, may monitor the SDMMC_D0 line using the BUSYD0 register bit. The SDMMC_D0 line is sampled two SDMMC_CK clock cycles after the Response. The Firmware may read the BUSYD0 register bit following the CKSTOP flag.
 - When the BUSYD0 is detected low the host firmware switches the Voltage regulator to 1.8V, after which it instructs the SDMMC to start the timing critical section of the Voltage Switch sequence by setting register bit VSWITCH. The hardware continues to stop the SDMMC_CK by holding it low for at least 5 ms.
 - When the BUSYD0 is detected high the host aborts the Voltage Switch sequence and cycle power the card.
- The card after detecting SDMMC_CK low begins switching signaling voltage to 1.8 V.
- The host SDMMC hardware after at least 5 ms restarts the SDMMC_CK.
- The card within 1 ms from detecting SDMMC_CK transition drives CMD and DAT[3:0] high for at least 1 SDMMC_CK cycle and then stop driving CMD and DAT[3:0].
- The host SDMMC hardware, 1 ms after the SDMMC_CK has been restarted, the SDMMC_D0 is sampled into BUSYD0 and the VSWEND flag is set.

10. The host, on the VSWEND flag, checks SDMMC_D0 line using the BUSYD0 register bit, to confirm completion of voltage switch sequence:
- When BUSYD0 is detected high, Voltage Switch has been completed successfully.
 - When BUSYD0 is detected low, Voltage Switch has failed, the host cycles the card power.

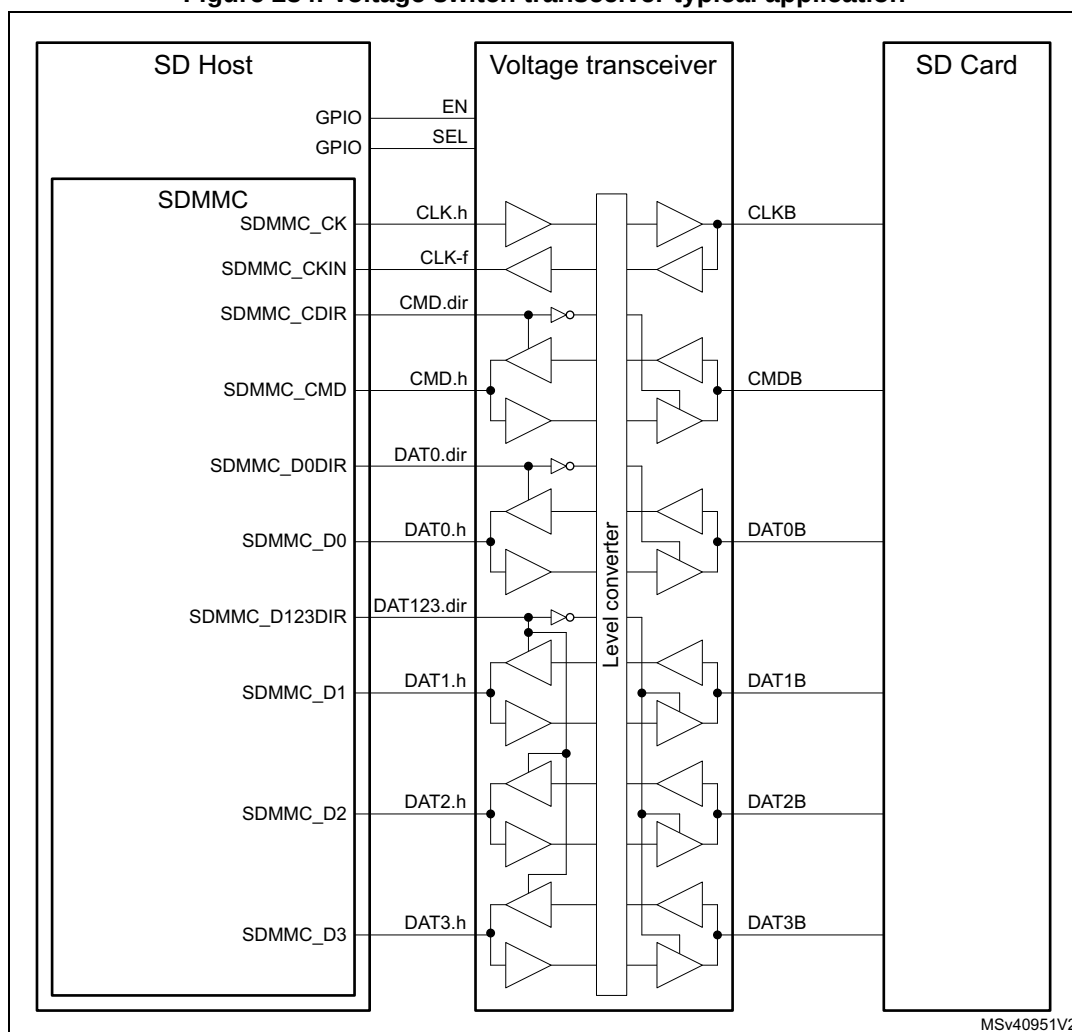
The minimum 5 ms time to stop the SDMMC_CLK is derived from the internal un-gated SDMMC_CLK clock, which has a maximum frequency of 25 MHz (SD mode), as set by the clock divider CLKDIV. The >5 ms time is counted by 2^{12} cycles (10.24 ms @ 400 kHz). If a lower SDMMC_CLK frequency is selected by the clock divider CLKDIV the time for the SDMMC_CLK clock to be stopped is longer.

The maximum 1 ms time for the card to drive the SDMMC_Dn and SDMMC_CMD lines high is derived from the internal ungated SDMMC_CLK which has a maximum frequency of 25 MHz (SD mode), as set by the clock divider CLKDIV. The SDMMC checks the lines after >1 ms time which is counted by 2^9 cycles (1.28 ms @ 25 MHz). If a lower SDMMC_CLK frequency is selected by the clock divider CLKDIV the time to check the lines is longer.

The signal voltage level switch is supported through an external voltage translation transceiver.

Note: For external devices not needing a signal voltage level switch the signal voltage level can be kept static for all SDMMC operations, including UHS-I.

Figure 234. Voltage switch transceiver typical application



MSv40951V2

To interface with an external driver (a voltage switch transceiver), next to the standard signals the SDMMC uses the following signals:

- **SDMMC_CKIN** feedback input clock
- **SDMMC_CDIRE** I/O direction control for the CMD signal
- **SDMMC_D0DIR** I/O direction control for the SDMMC_D0 signal
- **SDMMC_D123DIR** I/O direction control for the SDMMC_D1, SDMMC_D2 and SDMMC_D3 signals

The voltage transceiver signals **EN** and **SEL** are to be handled through general-purpose I/O.

The polarity of the SDMMC_CDIRE, SDMMC_D0DIR and SDMMC_D123DIR signals can be selected through SDMMC_POWER.DIRPOL control bit.

26.9 SDMMC interrupts

Table 272. SDMMC interrupts

Interrupt acronym	Interrupt event	Event flag	Enable control bit	Interrupt clear method	Exit from Sleep mode
SDMMC	Command response CRC fail	CCRCFAIL	CCRCFAILIE	CCRCFAILC	Yes
SDMMC	Data block CRC fail	DCRCFAIL	DCRCFAILIE	DCRCFAILC	Yes
SDMMC	Command response timeout	CTIMEOUT	CTIMEOUTIE	CTIMEOUTC	Yes
SDMMC	Data timeout	DTIMEOUT	DTIMEOUTIE	DTIMEOUTC	Yes
SDMMC	Transmit FIFO underrun	TXUNDERR	TXUNDERRIE	TXUNDERRC	Yes
SDMMC	Receive FIFO overrun	RXOVERR	RXOVERRIE	RXOVERRC	Yes
SDMMC	Command response received	CMDREND	CMDRENDIE	CMDREND C	Yes
SDMMC	Command sent	CMDSENT	CMDSENTIE	CMDSENTC	Yes
SDMMC	Data transfer ended	DATAEND	DATAENDIE	DATAENDC	Yes
SDMMC	Data transfer hold	DHOLD	DHOLDIE	DHOLD C	Yes
SDMMC	Data block sent or received	DBCKEND	DBCKENDIE	DBCKEND C	Yes
SDMMC	Data transfer aborted	DABORT	DABORTIE	DABORTC	Yes
SDMMC	Transmit FIFO half empty	TXFIFOHE	TXFIFOHEIE	N/A	Yes
SDMMC	Receive FIFO half full	RXFIFOHF	RXFIFOHFIE	N/A	Yes
SDMMC	Transmit FIFO full	TXFIFO F	N/A	N/A	Yes
SDMMC	Receive FIFO full	RXFIFO F	RXFIFO FIE	N/A	Yes
SDMMC	Transmit FIFO empty	TXFIFOE	TXFIFOEIE	N/A	Yes
SDMMC	Receive FIFO empty	RXFIFOE	N/A	N/A	Yes
SDMMC	Command response end of busy	BUSYD0END	BUSYD0ENDIE	BUSYD0END C	Yes
SDMMC	SDIO interrupt	SDIOIT	SDIOITIE	SDIOITC	Yes
SDMMC	Boot acknowledgment fail	ACKFAIL	ACKFAILIE	ACKFAILC	Yes
SDMMC	Boot acknowledgment timeout	ACKTIMEOUT	ACKTIMEOUTIE	ACKTIMEOUTC	Yes
SDMMC	Voltage switch timing	VSWEND	VSWENDIE	VSWENDC	Yes
SDMMC	SDMMCK stopped in voltage switch	CKSTOP	CKSTOPIE	CKSTOPC	Yes
SDMMC	IDMA transfer error	IDMATE	IDMATEIE	IDMATEC	Yes
SDMMC	IDMA buffer transfer complete	IDMABTC	IDMABTCIE	IDMABTCC	Yes

26.10 SDMMC registers

The device communicates to the system via 32-bit control registers accessible via AHB slave interface.

The peripheral registers have to be accessed by words (32-bit). Byte (8-bit) and half-word (16-bit) accesses trigger an AHB bus error.

26.10.1 SDMMC power control register (SDMMC_POWER)

Address offset: 0x000

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	DIR POL	VSWI TCHEN	VSWI TCH	PWRCTRL[1:0]	
											rw	rw	rw	rw	rw

Bits 31:5 Reserved, must be kept at reset value.

Bit 4 **DIRPOL**: Data and command direction signals polarity selection

This bit can only be written when the SDMMC is in the power-off state (PWRCTRL = 00).

0: Voltage transceiver I/Os driven as output when direction signal is low.

1: Voltage transceiver I/Os driven as output when direction signal is high.

Bit 3 **VSWITCHEN**: Voltage switch procedure enable

This bit can only be written by firmware when CPSM is disabled (CPSMEN = 0).

This bit is used to stop the SDMMC_CLK after the voltage switch command response:

0: SDMMC_CLK clock kept unchanged after successfully received command response.

1: SDMMC_CLK clock stopped after successfully received command response.

Bit 2 **VSWITCH**: Voltage switch sequence start

This bit is used to start the timing critical section of the voltage switch sequence:

0: Voltage switch sequence not started and not active.

1: Voltage switch sequence started or active.

Bits 1:0 **PWRCTRL[1:0]**: SDMMC state control bits

These bits can only be written when the SDMMC is not in the power-on state (PWRCTRL ≠ 11).

These bits are used to define the functional state of the SDMMC signals:

00: After reset, Reset: the SDMMC is disabled and the clock to the Card is stopped, SDMMC_D[7:0], and SDMMC_CMD are HiZ and SDMMC_CLK is driven low.

When written 00, power-off: the SDMMC is disabled and the clock to the card is stopped, SDMMC_D[7:0], SDMMC_CMD and SDMMC_CLK are driven high.

01: Reserved (When written 01, PWRCTRL value does not change)

10: Power-cycle, the SDMMC is disabled and the clock to the card is stopped, SDMMC_D[7:0], SDMMC_CMD and SDMMC_CLK are driven low.

11: Power-on: the card is clocked, The first 74 SDMMC_CLK cycles the SDMMC is still disabled. After the 74 cycles the SDMMC is enabled and the SDMMC_D[7:0], SDMMC_CMD and SDMMC_CLK are controlled according the SDMMC operation.

Any further write is ignored, PWRCTRL value keeps 11.

26.10.2 SDMMC clock control register (SDMMC_CLKCR)

Address offset: 0x004

Reset value: 0x0000 0000

This register controls the SDMMC_CK output clock, the sdmmc_rx_ck receive clock, and the bus width.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	SELCLKRX[1:0]		BUS SPEED	DDR	HWFC _EN	NEG EDGE
										rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
WID BUS[1:0]		Res.	PWR SAV	Res.	Res.	CLKDIV[9:0]									
rw	rw		rw			rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bits 31:22 Reserved, must be kept at reset value.

Bits 21:20 **SELCLKRX[1:0]**: Receive clock selection

These bits can only be written when the CPSM and DPSM are not active (CPSMACT = 0 and DPSMACT = 0)

00: sdmmc_io_in_ck selected as receive clock

01: SDMMC_CKIN feedback clock selected as receive clock

10: sdmmc_fb_ck tuned feedback clock selected as receive clock

11: Reserved (select sdmmc_io_in_ck)

Bit 19 **BUSPEED**: Bus speed for selection of SDMMC operating modes

This bit can only be written when the CPSM and DPSM are not active (CPSMACT = 0 and DPSMACT = 0)

0: DS, HS, SDR12, SDR25, Legacy compatible, High speed SDR, High speed DDR bus speed mode selected

1: SDR50, DDR50, SDR104, HS200 bus speed mode selected

Bit 18 **DDR**: Data rate signaling selection

This bit can only be written when the CPSM and DPSM are not active (CPSMACT = 0 and DPSMACT = 0)

DDR rate must only be selected with 4-bit or 8-bit wide bus mode. (WIDBUS > 00). DDR = 1 has no effect when WIDBUS = 00 (1-bit wide bus).

DDR rate must only be selected with clock division > 1 (CLKDIV > 0).

0: SDR Single data rate signaling

1: DDR double data rate signaling

Bit 17 **HWFC_EN**: Hardware flow control enable

This bit can only be written when the CPSM and DPSM are not active (CPSMACT = 0 and DPSMACT = 0)

0: Hardware flow control is disabled

1: Hardware flow control is enabled

When Hardware flow control is enabled, the meaning of the TXFIFOE and RXFIFO flags change, see SDMMC status register definition in [Section 26.10.11](#).

Bit 16 **NEGEDGE**: SDMMC_CK dephasing selection bit for data and command

This bit can only be written when the CPSM and DPSM are not active (CPSMACT = 0 and DPSMACT = 0).

When clock division = 1 (CLKDIV = 0), this bit has no effect. Data and Command change on SDMMC_CK falling edge.

0: When clock division >1 (CLKDIV > 0) and DDR = 0:

- Command and data changed on the sdmmc_ker_ck falling edge succeeding the rising edge of SDMMC_CK.
- SDMMC_CK edge occurs on sdmmc_ker_ck rising edge.

When clock division >1 (CLKDIV > 0) and DDR = 1:

- Command changed on the sdmmc_ker_ck falling edge succeeding the rising edge of SDMMC_CK.
- Data changed on the sdmmc_ker_ck falling edge succeeding a SDMMC_CK edge.
- SDMMC_CK edge occurs on sdmmc_ker_ck rising edge.

1: When clock division >1 (CLKDIV > 0) and DDR = 0:

- Command and data changed on the same sdmmc_ker_ck rising edge generating the SDMMC_CK falling edge.

When clock division >1 (CLKDIV > 0) and DDR = 1:

- Command changed on the same sdmmc_ker_ck rising edge generating the SDMMC_CK falling edge.
- Data changed on the SDMMC_CK falling edge succeeding a SDMMC_CK edge.
- SDMMC_CK edge occurs on sdmmc_ker_ck rising edge.

Bits 15:14 **WIDBUS[1:0]**: Wide bus mode enable bit

This bit can only be written when the CPSM and DPSM are not active (CPSMACT = 0 and DPSMACT = 0)

00: Default 1-bit wide bus mode: SDMMC_D0 used (Does not support DDR)

01: 4-bit wide bus mode: SDMMC_D[3:0] used

10: 8-bit wide bus mode: SDMMC_D[7:0] used

Bit 13 Reserved, must be kept at reset value.

Bit 12 **PWRSABV**: Power saving configuration bit

This bit can only be written when the CPSM and DPSM are not active (CPSMACT = 0 and DPSMACT = 0)

For power saving, the SDMMC_CK clock output can be disabled when the bus is idle by setting PWRSABV:

0: SDMMC_CK clock is always enabled

1: SDMMC_CK is only enabled when the bus is active

Bits 11:10 Reserved, must be kept at reset value.

Bits 9:0 **CLKDIV[9:0]**: Clock divide factor

This bit can only be written when the CPSM and DPSM are not active (CPSMACT = 0 and DPSMACT = 0).

This field defines the divide factor between the input clock (sdmmc_ker_ck) and the output clock (SDMMC_CK): $\text{SDMMC_CK frequency} = \text{sdmmc_ker_ck} / [2 * \text{CLKDIV}]$.

0x000: SDMMC_CK frequency = sdmmc_ker_ck / 1 (Does not support DDR)

0x001: SDMMC_CK frequency = sdmmc_ker_ck / 2

0x002: SDMMC_CK frequency = sdmmc_ker_ck / 4

0x0XX: ..

0x080: SDMMC_CK frequency = sdmmc_ker_ck / 256

0xXXX: ..

0x3FF: SDMMC_CK frequency = sdmmc_ker_ck / 2046

Note: While the SD/SDIO card or eMMC is in identification mode, the SDMMC_CLK frequency must be less than 400 kHz.

The clock frequency can be changed to the maximum card bus frequency when relative card addresses are assigned to all cards.

At least seven sdmmc_hclk clock periods are needed between two write accesses to this register. SDMMC_CLK can also be stopped during the Read Wait interval for SD I/O cards: in this case the SDMMC_CLKCR register does not control SDMMC_CLK.

26.10.3 SDMMC argument register (SDMMC_ARGR)

Address offset: 0x008

Reset value: 0x0000 0000

This register contains a 32-bit command argument, which is sent to a card as part of a command message.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
CMDARG[31:16]															
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
CMDARG[15:0]															
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bits 31:0 **CMDARG[31:0]**: Command argument

These bits can only be written by firmware when CPSM is disabled (CPSMEN = 0).

Command argument sent to a card as part of a command message. If a command contains an argument, it must be loaded into this register before writing a command to the command register.

26.10.4 SDMMC command register (SDMMC_CMDR)

Address offset: 0x00C

Reset value: 0x0000 0000

This register contains the command index and command type bits. The command index is sent to a card as part of a command message. The command type bits control the command path state machine (CPSM).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	CMD SUS PEND
															rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
BOOT EN	BOOT MODE	DT HOLD	CPSM EN	WAITP END	WAIT INT	WAITRESP[1:0]		CMD STOP	CMD TRANS	CMDINDEX[5:0]					
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bits 31:17 Reserved, must be kept at reset value.

Bit 16 **CMDSPEND**: The CPSM treats the command as a Suspend or Resume command and signals interrupt period start/end
 This bit can only be written by firmware when CPSM is disabled (CPSMEN = 0).
 CMDSPEND = 1 and CMDTRANS = 0 Suspend command, start interrupt period when response bit BS = 0.
 CMDSPEND = 1 and CMDTRANS = 1 Resume command with data, end interrupt period when response bit DF = 1.

Bit 15 **BOOTEN**: Enable boot mode procedure
 0: Boot mode procedure disabled
 1: Boot mode procedure enabled

Bit 14 **BOOTMODE**: Select the boot mode procedure to be used
 This bit can only be written by firmware when CPSM is disabled (CPSMEN = 0)
 0: Normal boot mode procedure selected
 1: Alternative boot mode procedure selected

Bit 13 **DTHOLD**: Hold new data block transmission and reception in the DPSM
 If this bit is set, the DPSM does not move from the Wait_S state to the Send state or from the Wait_R state to the Receive state.

Bit 12 **CPSMEN**: Command path state machine (CPSM) enable bit
 This bit is written 1 by firmware, and cleared by hardware when the CPSM enters the Idle state.
 If this bit is set, the CPSM is enabled.
 When DTEN = 1, no command is transferred nor boot procedure is started. CPSMEN is cleared to 0.
 During Read Wait with SDMMC_CK stopped no command is sent and CPSMEN is kept 0.

Bit 11 **WAITPEND**: CPSM waits for end of data transfer (CmdPend internal signal) from DPSM
 This bit when set, the CPSM waits for the end of data transfer trigger before it starts sending a command.
 WAITPEND is only taken into account when DTMODE = eMMC stream data transfer, WIDBUS = 1-bit wide bus mode, DPSMACT = 1 and DTDIR = from host to card.

Bit 10 **WAITINT**: CPSM waits for interrupt request
 If this bit is set, the CPSM disables command timeout and waits for an card interrupt request (Response).
 If this bit is cleared in the CPSM Wait state, it causes the abort of the interrupt mode.

Bits 9:8 **WAITRESP[1:0]**: Wait for response bits
 This bit can only be written by firmware when CPSM is disabled (CPSMEN = 0).
 They are used to configure whether the CPSM is to wait for a response, and if yes, which kind of response.
 00: No response, expect CMDSSENT flag
 01: Short response, expect CMDREND or CCRCFAIL flag
 10: Short response, expect CMDREND flag (No CRC)
 11: Long response, expect CMDREND or CCRCFAIL flag

Bit 7 **CMDSTOP**: The CPSM treats the command as a Stop Transmission command and signals abort to the DPSM

This bit can only be written by firmware when CPSM is disabled (CPSMEN = 0).

If this bit is set, the CPSM issues the abort signal to the DPSM when the command is sent.

Bit 6 **CMDTRANS**: The CPSM treats the command as a data transfer command, stops the interrupt period, and signals DataEnable to the DPSM

This bit can only be written by firmware when CPSM is disabled (CPSMEN = 0).

If this bit is set, the CPSM issues an end of interrupt period and issues DataEnable signal to the DPSM when the command is sent.

Bits 5:0 **CMDINDEX[5:0]**: Command index

This bit can only be written by firmware when CPSM is disabled (CPSMEN = 0).

The command index is sent to the card as part of a command message.

- Note:
- 1 At least seven *sdmmc_hclk* clock periods are needed between two write accesses to this register.
 - 2 MultiMediaCard can send two kinds of response: short responses, 48 bits, or long responses, 136 bits. SD card and SD I/O card can send only short responses, the argument can vary according to the type of response: the software distinguishes the type of response according to the send command.

26.10.5 SDMMC command response register (SDMMC_RESPCMDR)

Address offset: 0x010

Reset value: 0x0000 0000

This register contains the command index field of the last command response received. If the command response transmission does not contain the command index field (long or OCR response), the RESPCMD field is unknown, although it must contain 111111b (the value of the reserved field from the response).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	RESPCMD[5:0]					
										r	r	r	r	r	r

Bits 31:6 Reserved, must be kept at reset value.

Bits 5:0 **RESPCMD[5:0]**: Response command index

Read-only bitfield. Contains the command index of the last command response received.

26.10.6 SDMMC response x register (SDMMC_RESPxR)

Address offset: $0x010 + 0x004 * x$, ($x = 1$ to 4)

Reset value: $0x0000\ 0000$

These registers contain the status of a card, which is part of the received response.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
CARDSTATUS[31:16]															
r	r	r	r	r	r	r	r	r	r	r	r	r	r	r	r
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
CARDSTATUS[15:0]															
r	r	r	r	r	r	r	r	r	r	r	r	r	r	r	r

Bits 31:0 **CARDSTATUS[31:0]**: Card status according table below

See [Table 273](#).

The card status size is 32 or 128 bits, depending on the response type.

Table 273. Response type and SDMMC_RESPxR registers

Register ⁽¹⁾	Short response	Long response
SDMMC_RESP1R	Card status[31:0]	Card status [127:96]
SDMMC_RESP2R	all 0	Card status [95:64]
SDMMC_RESP3R	all 0	Card status [63:32]
SDMMC_RESP4R	all 0	Card status [31:0] ⁽²⁾

1. The most significant bit of the card status is received first.

2. The SDMMC_RESP4R register LSB is always 0.

26.10.7 SDMMC data timer register (SDMMC_DTIMER)

Address offset: $0x024$

Reset value: $0x0000\ 0000$

This register contains the data timeout period, in card bus clock periods.

A counter loads the value from this register, and starts decrementing when the data path state machine (DPSM) enters the Wait_R or Busy state. If the timer reaches 0 while the DPSM is in either of these states, the timeout status flag is set.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
DATETIME[31:16]															
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
DATETIME[15:0]															
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bits 31:0 **DATETIME[31:0]**: Data and R1b busy timeout period

This bit can only be written when the CPSM and DPSM are not active (CPSMACT = 0 and DPSMACT = 0).

Data and R1b busy timeout period expressed in card bus clock periods.

Note: A data transfer must be written to the data timer register and the data length register before being written to the data control register.

26.10.8 SDMMC data length register (SDMMC_DLENR)

Address offset: 0x028

Reset value: 0x0000 0000

This register contains the number of data bytes to be transferred. The value is loaded into the data counter when data transfer starts.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	DATALENGTH[24:16]								
							rw	rw	rw	rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
DATALENGTH[15:0]															
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bits 31:25 Reserved, must be kept at reset value.

Bits 24:0 **DATALENGTH[24:0]**: Data length value

This register can only be written by firmware when DPSM is inactive (DPSMACT = 0).

Number of data bytes to be transferred.

When DDR = 1 DATALENGTH is truncated to a multiple of 2. (The last odd byte is not transferred)

When DATALENGTH = 0 no data are transferred, when requested by a CPSMEN and CMDTRANS = 1 also no command is transferred. DTEN and CPSMEN are cleared to 0.

Note: For a block data transfer, the value in the data length register must be a multiple of the block size (see SDMMC_DCTRL). A data transfer must be written to the data timer register and the data length register before being written to the data control register.

For an SDMMC multibyte transfer the value in the data length register must be between 1 and 512.

26.10.9 SDMMC data control register (SDMMC_DCTRL)

Address offset: 0x02C

Reset value: 0x0000 0000

This register controls the data path state machine (DPSM).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	FIFO RST	BOOT ACK EN	SDIO EN	RW MOD	RW STOP	RW START	DBLOCKSIZE[3:0]				DTMODE[1:0]		DTDIR	DTEN
		rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bits 31:14 Reserved, must be kept at reset value.

Bit 13 FIFORST: FIFO reset, flushes any remaining data

This bit can only be written by firmware when IDMAEN = 0 and DPSM is active (DPSMACT = 1). This bit only takes effect when a transfer error or transfer hold occurs.

0: FIFO not affected.

1: Flush any remaining data and reset the FIFO pointers. This bit is automatically cleared to 0 by hardware when DPSM gets inactive (DPSMACT = 0).

Bit 12 BOOTACKEN: Enable the reception of the boot acknowledgment

This bit can only be written by firmware when DPSM is inactive (DPSMACT = 0).

0: Boot acknowledgment disabled, not expected to be received

1: Boot acknowledgment enabled, expected to be received

Bit 11 SDIOEN: SD I/O interrupt enable functions

This bit can only be written by firmware when DPSM is inactive (DPSMACT = 0).

If this bit is set, the DPSM enables the SD I/O card specific interrupt operation.

Bit 10 RWMOD: Read Wait mode

This bit can only be written by firmware when DPSM is inactive (DPSMACT = 0).

0: Read Wait control using SDMMC_D2

1: Read Wait control stopping SDMMC_CK

Bit 9 RWSTOP: Read Wait stop

This bit is written by firmware and auto cleared by hardware when the DPSM moves from the R_W state to the Wait_R or Idle state.

0: No Read Wait stop

1: Enable for Read Wait stop when DPSM is in the R_W state.

Bit 8 RWSTART: Read Wait start

If this bit is set, Read Wait operation starts.

Bits 7:4 DBLOCKSIZE[3:0]: Data block size

This bit can only be written by firmware when DPSM is inactive (DPSMACT = 0).

Define the data block length when the block data transfer mode is selected:

0000: Block length = 2^0 = 1 byte
 0001: Block length = 2^1 = 2 bytes
 0010: Block length = 2^2 = 4 bytes
 0011: Block length = 2^3 = 8 bytes
 0100: Block length = 2^4 = 16 bytes
 0101: Block length = 2^5 = 32 bytes
 0110: Block length = 2^6 = 64 bytes
 0111: Block length = 2^7 = 128 bytes
 1000: Block length = 2^8 = 256 bytes
 1001: Block length = 2^9 = 512 bytes
 1010: Block length = 2^{10} = 1024 bytes
 1011: Block length = 2^{11} = 2048 bytes
 1100: Block length = 2^{12} = 4096 bytes
 1101: Block length = 2^{13} = 8192 bytes
 1110: Block length = 2^{14} = 16384 bytes
 1111: Reserved

When DATALENGTH is not a multiple of DBLOCKSIZE, the transferred data is truncated at a multiple of DBLOCKSIZE. (None of the remaining data are transferred.)

When DDR = 1, DBLOCKSIZE = 0000 must not be used. (No data are transferred)

Bits 3:2 DTMODE[1:0]: Data transfer mode selection

This bit can only be written by firmware when DPSM is inactive (DPSMACT = 0).

00: Block data transfer ending on block count.

01: SDIO multibyte data transfer.

10: eMMC Stream data transfer. (WIDBUS must select 1-bit wide bus mode)

11: Block data transfer ending with STOP_TRANSMISSION command (not to be used with DTEN initiated data transfers).

Bit 1 DTDIR: Data transfer direction selection

This bit can only be written by firmware when DPSM is inactive (DPSMACT = 0).

0: From host to card.

1: From card to host.

Bit 0 DTEN: Data transfer enable bit

This bit can only be written by firmware when DPSM is inactive (DPSMACT = 0). This bit is cleared by hardware when data transfer completes.

This bit must only be used to transfer data when no associated data transfer command is used (must not be used with SD or eMMC cards).

0: Do not start data transfer without CPSM data transfer command.

1: Start data transfer without CPSM data transfer command.

26.10.10 SDMMC data counter register (SDMMC_DCNTR)

Address offset: 0x030

Reset value: 0x0000 0000

This register loads the value from the data length register (see SDMMC_DLENR) when the DPSM moves from the Idle state to the Wait_R or Wait_S state. As data is transferred, the counter decrements the value until it reaches 0. The DPSM then moves to the Idle state and

when there has been no error, and no transmit data transfer hold, the data status end flag (DATAEND) is set.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	DATACOUNT[24:16]								
							r	r	r	r	r	r	r	r	r
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
DATACOUNT[15:0]															
r	r	r	r	r	r	r	r	r	r	r	r	r	r	r	r

Bits 31:25 Reserved, must be kept at reset value.

Bits 24:0 **DATACOUNT[24:0]**: Data count value

When read, the number of remaining data bytes to be transferred is returned. Write has no effect.

Note: *This register must be read only after the data transfer is complete, or hold. When reading after an error event the read data count value may be different from the real number of data bytes transferred.*

26.10.11 SDMMC status register (SDMMC_STAR)

Address offset: 0x034

Reset value: 0x0000 0000

This register is a read-only register. It contains two types of flag:

- Static flags (bits [28, 21, 11:0]): these bits remain asserted until they are cleared by writing to the SDMMC interrupt Clear register (see SDMMC_ICR)
- Dynamic flags (bits [20:12]): these bits change state depending on the state of the underlying logic (for example, FIFO full and empty flags are asserted and deasserted as data while written to the FIFO)

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	IDMA BTC	IDMA TE	CK STOP	VSW END	ACK TIME OUT	ACK FAIL	SDIOIT	BUSY D0END	BUSY D0	RX FIFOE	TX FIFOE	RX FIFO	TX FIFO
			r	r	r	r	r	r	r	r	r	r	r	r	r
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
RX FIFO HF	TX FIFO HE	CPSM ACT	DPSM ACT	DA BORT	DBCK END	DHOLD	DATA END	CMD SENT	CMDR END	RX OVERR	TX UNDER R	D TIME OUT	C TIME OUT	DCRC FAIL	CCRC FAIL
r	r	r	r	r	r	r	r	r	r	r	r	r	r	r	r

Bits 31:29 Reserved, must be kept at reset value.

Bit 28 **IDMABTC**: IDMA buffer transfer complete

The interrupt flag is cleared by writing corresponding interrupt clear bit in SDMMC_ICR.

Bit 27 **IDMATE**: IDMA transfer error

The interrupt flag is cleared by writing corresponding interrupt clear bit in SDMMC_ICR.

Bit 26 **CKSTOP**: SDMMC_CK stopped in Voltage switch procedure

The interrupt flag is cleared by writing corresponding interrupt clear bit in SDMMC_ICR.

- Bit 25 **VSWEND**: Voltage switch critical timing section completion
The interrupt flag is cleared by writing corresponding interrupt clear bit in SDMMC_ICR.
- Bit 24 **ACKTIMEOUT**: Boot acknowledgment timeout
The interrupt flag is cleared by writing corresponding interrupt clear bit in SDMMC_ICR.
- Bit 23 **ACKFAIL**: Boot acknowledgment received (boot acknowledgment check fail)
The interrupt flag is cleared by writing corresponding interrupt clear bit in SDMMC_ICR.
- Bit 22 **SDIOIT**: SDIO interrupt received
The interrupt flag is cleared by writing corresponding interrupt clear bit in SDMMC_ICR.
- Bit 21 **BUSYD0END**: end of SDMMC_D0 Busy following a CMD response detected
This indicates only end of busy following a CMD response. This bit does not signal busy due to data transfer. Interrupt flag is cleared by writing corresponding interrupt clear bit in SDMMC_ICR.
0: card SDMMC_D0 signal does NOT signal change from busy to not busy.
1: card SDMMC_D0 signal changed from busy to NOT busy.
- Bit 20 **BUSYD0**: Inverted value of SDMMC_D0 line (Busy), sampled at the end of a CMD response and a second time 2 SDMMC_CK cycles after the CMD response
This bit is reset to not busy when the SDMMCD0 line changes from busy to not busy. This bit does not signal busy due to data transfer. This is a hardware status flag only, it does not generate an interrupt.
0: card signals not busy on SDMMC_D0.
1: card signals busy on SDMMC_D0.
- Bit 19 **RXFIFOE**: Receive FIFO empty
This is a hardware status flag only, does not generate an interrupt. This bit is cleared when one FIFO location becomes full.
- Bit 18 **TXFIFOE**: Transmit FIFO empty
This bit is cleared when one FIFO location becomes full.
- Bit 17 **RXFIFO**: Receive FIFO full
This bit is cleared when one FIFO location becomes empty.
- Bit 16 **TXFIFO**: Transmit FIFO full
This is a hardware status flag only, does not generate an interrupt. This bit is cleared when one FIFO location becomes empty.
- Bit 15 **RXFIFOHF**: Receive FIFO half full
There are at least half the number of words in the FIFO. This bit is cleared when the FIFO becomes half+1 empty.
- Bit 14 **TXFIFOHE**: Transmit FIFO half empty
At least half the number of words can be written into the FIFO. This bit is cleared when the FIFO becomes half+1 full.
- Bit 13 **CPSMACT**: Command path state machine active (not in Idle state)
This is a hardware status flag only, does not generate an interrupt.
- Bit 12 **DPSMACT**: Data path state machine active (not in Idle state)
This is a hardware status flag only, does not generate an interrupt.
- Bit 11 **DABORT**: Data transfer aborted by CMD12
Interrupt flag is cleared by writing corresponding interrupt clear bit in SDMMC_ICR.

- Bit 10 **DBCKEND**: Data block sent/received
DBCKEND is set when:
- CRC check passed and DPSM moves to the R_W state
or
- IDMAEN = 0 and transmit data transfer hold and DATACOUNT >0 and DPSM moves to Wait_S.
Interrupt flag is cleared by writing corresponding interrupt clear bit in SDMMC_ICR.
- Bit 9 **DHOLD**: Data transfer Hold
Interrupt flag is cleared by writing corresponding interrupt clear bit in SDMMC_ICR.
- Bit 8 **DATAEND**: Data transfer ended correctly
DATAEND is set if data counter DATACOUNT is zero and no errors occur, and no transmit data transfer hold.
Interrupt flag is cleared by writing corresponding interrupt clear bit in SDMMC_ICR.
- Bit 7 **CMDSENT**: Command sent (no response required)
Interrupt flag is cleared by writing corresponding interrupt clear bit in SDMMC_ICR.
- Bit 6 **CMDREND**: Command response received (CRC check passed, or no CRC)
Interrupt flag is cleared by writing corresponding interrupt clear bit in SDMMC_ICR.
- Bit 5 **RXOVERR**: Received FIFO overrun error (masked by hardware when IDMA is enabled)
Interrupt flag is cleared by writing corresponding interrupt clear bit in SDMMC_ICR.
- Bit 4 **TXUNDERR**: Transmit FIFO underrun error (masked by hardware when IDMA is enabled)
Interrupt flag is cleared by writing corresponding interrupt clear bit in SDMMC_ICR.
- Bit 3 **DTIMEOUT**: Data timeout
Interrupt flag is cleared by writing corresponding interrupt clear bit in SDMMC_ICR.
- Bit 2 **CTIMEOUT**: Command response timeout
Interrupt flag is cleared by writing corresponding interrupt clear bit in SDMMC_ICR.
The Command Timeout period has a fixed value of 64 SDMMC_CK clock periods.
- Bit 1 **DCRCFAIL**: Data block sent/received (CRC check failed)
Interrupt flag is cleared by writing corresponding interrupt clear bit in SDMMC_ICR.
- Bit 0 **CCRCFAIL**: Command response received (CRC check failed)
Interrupt flag is cleared by writing corresponding interrupt clear bit in SDMMC_ICR.

Note: FIFO interrupt flags must be masked in SDMMC_MASKR when using IDMA mode.

26.10.12 SDMMC interrupt clear register (SDMMC_ICR)

Address offset: 0x038

Reset value: 0x0000 0000

This register is a write-only register. Writing a bit with 1 clears the corresponding bit in the SDMMC_STAR status register.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	IDMA BTCC	IDMA TEC	CK STOPC	VSW ENDC	ACK TIME OUTC	ACK FAILC	SDIO ITC	BUSY D0 ENDC	Res.	Res.	Res.	Res.	Res.
			rw	rw	rw	rw	rw	rw	rw	rw					
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	Res.	Res.	D ABORT C	DBCK ENDC	DHOLD C	DATA ENDC	CMD SENTC	CMDR ENDC	RX OVERR C	TX UNDER RC	D TIME OUTC	C TIME OUTC	DCRC FAILC	CCRC FAILC
				rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bits 31:29 Reserved, must be kept at reset value.

Bit 28 **IDMABTCC**: IDMA buffer transfer complete clear bit

Set by software to clear the IDMABTC flag.

0: IDMABTC not cleared

1: IDMABTC cleared

Bit 27 **IDMATEC**: IDMA transfer error clear bit

Set by software to clear the IDMATE flag.

0: IDMATE not cleared

1: IDMATE cleared

Bit 26 **CKSTOPC**: CKSTOP flag clear bit

Set by software to clear the CKSTOP flag.

0: CKSTOP not cleared

1: CKSTOP cleared

Bit 25 **VSWENDC**: VSWEND flag clear bit

Set by software to clear the VSWEND flag.

0: VSWEND not cleared

1: VSWEND cleared

Bit 24 **ACKTIMEOUTC**: ACKTIMEOUT flag clear bit

Set by software to clear the ACKTIMEOUT flag.

0: ACKTIMEOUT not cleared

1: ACKTIMEOUT cleared

Bit 23 **ACKFAILC**: ACKFAIL flag clear bit

Set by software to clear the ACKFAIL flag.

0: ACKFAIL not cleared

1: ACKFAIL cleared

Bit 22 **SDIOITC**: SDIOIT flag clear bit

Set by software to clear the SDIOIT flag.

0: SDIOIT not cleared

1: SDIOIT cleared

Bit 21 **BUSYD0ENDC**: BUSYD0END flag clear bit
Set by software to clear the BUSYD0END flag.
0: BUSYD0END not cleared
1: BUSYD0END cleared

Bits 20:12 Reserved, must be kept at reset value.

Bit 11 **DABORTC**: DABORT flag clear bit
Set by software to clear the DABORT flag.
0: DABORT not cleared
1: DABORT cleared

Bit 10 **DBCKENDC**: DBCKEND flag clear bit
Set by software to clear the DBCKEND flag.
0: DBCKEND not cleared
1: DBCKEND cleared

Bit 9 **DHOLD C**: DHOLD flag clear bit
Set by software to clear the DHOLD flag.
0: DHOLD not cleared
1: DHOLD cleared

Bit 8 **DATAENDC**: DATAEND flag clear bit
Set by software to clear the DATAEND flag.
0: DATAEND not cleared
1: DATAEND cleared

Bit 7 **CMDSENTC**: CMDSENT flag clear bit
Set by software to clear the CMDSENT flag.
0: CMDSENT not cleared
1: CMDSENT cleared

Bit 6 **CMDREND C**: CMDREND flag clear bit
Set by software to clear the CMDREND flag.
0: CMDREND not cleared
1: CMDREND cleared

Bit 5 **RXOVERRC**: RXOVERR flag clear bit
Set by software to clear the RXOVERR flag.
0: RXOVERR not cleared
1: RXOVERR cleared

Bit 4 **TXUNDERRC**: TXUNDERR flag clear bit
Set by software to clear TXUNDERR flag.
0: TXUNDERR not cleared
1: TXUNDERR cleared

Bit 3 **DTIMEOUTC**: DTIMEOUT flag clear bit
Set by software to clear the DTIMEOUT flag.
0: DTIMEOUT not cleared
1: DTIMEOUT cleared

- Bit 2 **CTIMEOUTC**: CTIMEOUT flag clear bit
Set by software to clear the CTIMEOUT flag.
0: CTIMEOUT not cleared
1: CTIMEOUT cleared
- Bit 1 **DCRCFAILC**: DCRCFAIL flag clear bit
Set by software to clear the DCRCFAIL flag.
0: DCRCFAIL not cleared
1: DCRCFAIL cleared
- Bit 0 **CCRCFAILC**: CCRCFAIL flag clear bit
Set by software to clear the CCRCFAIL flag.
0: CCRCFAIL not cleared
1: CCRCFAIL cleared

26.10.13 SDMMC mask register (SDMMC_MASKR)

Address offset: 0x03C

Reset value: 0x0000 0000

This register determines which status flags generate an interrupt request by setting the corresponding bit to 1.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	IDMA BTCIE	Res.	CK STOP IE	VSW ENDIE	ACK TIME OUTIE	ACK FAILIE	SDIO ITIE	BUSY D0 ENDIE	Res.	Res.	TX FIFO EIE	RX FIFO FIE	Res.
			rw		rw	rw	rw	rw	rw	rw			rw	rw	
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
RX FIFO HFIE	TX FIFO HEIE	Res.	Res.	DA BORT IE	DBCK ENDIE	DHOLD IE	DATA ENDIE	CMD SENT IE	CMDR ENDIE	RX OVER RIE	TX UNDER RIE	D TIME OUTIE	C TIME OUTIE	DCRC FAILIE	CCRC FAILIE
rw	rw			rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bits 31:29 Reserved, must be kept at reset value.

- Bit 28 **IDMABTCIE**: IDMA buffer transfer complete interrupt enable
Set and cleared by software to enable/disable the interrupt generated when the IDMA has transferred all data belonging to a memory buffer.
0: IDMA buffer transfer complete interrupt disabled
1: IDMA buffer transfer complete interrupt enabled
- Bit 27 Reserved, must be kept at reset value.
- Bit 26 **CKSTOPIE**: Voltage switch clock stopped interrupt enable
Set and cleared by software to enable/disable interrupt caused by voltage switch clock stopped.
0: Voltage switch clock stopped interrupt disabled
1: Voltage switch clock stopped interrupt enabled
- Bit 25 **VSWENDIE**: Voltage switch critical timing section completion interrupt enable
Set and cleared by software to enable/disable the interrupt generated when voltage switch critical timing section completion.
0: Voltage switch critical timing section completion interrupt disabled
1: Voltage switch critical timing section completion interrupt enabled

- Bit 24 **ACKTIMEOUTIE**: Acknowledgment timeout interrupt enable
Set and cleared by software to enable/disable interrupt caused by acknowledgment timeout.
0: Acknowledgment timeout interrupt disabled
1: Acknowledgment timeout interrupt enabled
- Bit 23 **ACKFAILIE**: Acknowledgment fail interrupt enable
Set and cleared by software to enable/disable interrupt caused by acknowledgment fail.
0: Acknowledgment fail interrupt disabled
1: Acknowledgment fail interrupt enabled
- Bit 22 **SDIOITIE**: SDIO mode interrupt received interrupt enable
Set and cleared by software to enable/disable the interrupt generated when receiving the SDIO mode interrupt.
0: SDIO mode interrupt received interrupt disabled
1: SDIO mode interrupt received interrupt enabled
- Bit 21 **BUSYD0ENDIE**: BUSYD0END interrupt enable
Set and cleared by software to enable/disable the interrupt generated when SDMMC_D0 signal changes from busy to NOT busy following a CMD response.
0: BUSYD0END interrupt disabled
1: BUSYD0END interrupt enabled
- Bits 20:19 Reserved, must be kept at reset value.
- Bit 18 **TXFIFOEIE**: Tx FIFO empty interrupt enable
Set and cleared by software to enable/disable interrupt caused by Tx FIFO empty.
0: Tx FIFO empty interrupt disabled
1: Tx FIFO empty interrupt enabled
- Bit 17 **RXFIFOFIE**: Rx FIFO full interrupt enable
Set and cleared by software to enable/disable interrupt caused by Rx FIFO full.
0: Rx FIFO full interrupt disabled
1: Rx FIFO full interrupt enabled
- Bit 16 Reserved, must be kept at reset value.
- Bit 15 **RXFIFOHFIE**: Rx FIFO half full interrupt enable
Set and cleared by software to enable/disable interrupt caused by Rx FIFO half full.
0: Rx FIFO half full interrupt disabled
1: Rx FIFO half full interrupt enabled
- Bit 14 **TXFIFOHEIE**: Tx FIFO half empty interrupt enable
Set and cleared by software to enable/disable interrupt caused by Tx FIFO half empty.
0: Tx FIFO half empty interrupt disabled
1: Tx FIFO half empty interrupt enabled
- Bits 13:12 Reserved, must be kept at reset value.
- Bit 11 **DABORTIE**: Data transfer aborted interrupt enable
Set and cleared by software to enable/disable interrupt caused by a data transfer being aborted.
0: Data transfer abort interrupt disabled
1: Data transfer abort interrupt enabled
- Bit 10 **DBCKENDIE**: Data block end interrupt enable
Set and cleared by software to enable/disable interrupt caused by data block end.
0: Data block end interrupt disabled
1: Data block end interrupt enabled

- Bit 9 **DHOLDIE**: Data hold interrupt enable
Set and cleared by software to enable/disable the interrupt generated when sending new data is hold in the DPSM Wait_S state.
0: Data hold interrupt disabled
1: Data hold interrupt enabled
- Bit 8 **DATAENDIE**: Data end interrupt enable
Set and cleared by software to enable/disable interrupt caused by data end.
0: Data end interrupt disabled
1: Data end interrupt enabled
- Bit 7 **CMDSENTIE**: Command sent interrupt enable
Set and cleared by software to enable/disable interrupt caused by sending command.
0: Command sent interrupt disabled
1: Command sent interrupt enabled
- Bit 6 **CMDRENDIE**: Command response received interrupt enable
Set and cleared by software to enable/disable interrupt caused by receiving command response.
0: Command response received interrupt disabled
1: command Response received interrupt enabled
- Bit 5 **RXOVERRIE**: Rx FIFO overrun error interrupt enable
Set and cleared by software to enable/disable interrupt caused by Rx FIFO overrun error.
0: Rx FIFO overrun error interrupt disabled
1: Rx FIFO overrun error interrupt enabled
- Bit 4 **TXUNDERRIE**: Tx FIFO underrun error interrupt enable
Set and cleared by software to enable/disable interrupt caused by Tx FIFO underrun error.
0: Tx FIFO underrun error interrupt disabled
1: Tx FIFO underrun error interrupt enabled
- Bit 3 **DTIMEOUTIE**: Data timeout interrupt enable
Set and cleared by software to enable/disable interrupt caused by data timeout.
0: Data timeout interrupt disabled
1: Data timeout interrupt enabled
- Bit 2 **CTIMEOUTIE**: Command timeout interrupt enable
Set and cleared by software to enable/disable interrupt caused by command timeout.
0: Command timeout interrupt disabled
1: Command timeout interrupt enabled
- Bit 1 **DCRCFAILIE**: Data CRC fail interrupt enable
Set and cleared by software to enable/disable interrupt caused by data CRC failure.
0: Data CRC fail interrupt disabled
1: Data CRC fail interrupt enabled
- Bit 0 **CCRCFAILIE**: Command CRC fail interrupt enable
Set and cleared by software to enable/disable interrupt caused by command CRC failure.
0: Command CRC fail interrupt disabled
1: Command CRC fail interrupt enabled

26.10.14 SDMMC acknowledgment timer register (SDMMC_ACKTIMER)

Address offset: 0x040

Reset value: 0x0000 0000

This register contains the acknowledgment timeout period, in SDMMC_CLK bus clock periods.

A counter loads the value from this register, and starts decrementing when the data path state machine (DPSM) enters the Wait_Ack state. If the timer reaches 0 while the DPSM is in this states, the acknowledgment timeout status flag is set.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	ACKTIME[24:16]								
							rw	rw	rw	rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
ACKTIME[15:0]															
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bits 31:25 Reserved, must be kept at reset value.

Bits 24:0 **ACKTIME[24:0]**: Boot acknowledgment timeout period

This bit can only be written by firmware when CPSM is disabled (CPSMEN = 0).

Boot acknowledgment timeout period expressed in card bus clock periods.

Note: The data transfer must be written to the acknowledgment timer register before being written to the data control register.

26.10.15 SDMMC DMA control register (SDMMC_IDMACTRLR)

Address offset: 0x050

Reset value: 0x0000 0000

The receive and transmit FIFOs can be read or written as 32-bit wide registers. The FIFOs contain 32 entries on 32 sequential addresses. This enables the CPU to use its load and store multiple operands to read from/write to the FIFO.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	IDMAB MODE	IDMA EN
														rw	rw

Bits 31:2 Reserved, must be kept at reset value.

Bit 1 **IDMABMODE**: Buffer mode selection

This bit can only be written by firmware when DPSM is inactive (DPSMACT = 0).

0: Single buffer mode.

1: Linked list mode.

Bit 0 **IDMAEN**: IDMA enable

This bit can only be written by firmware when DPSM is inactive (DPSMACT = 0).

0: IDMA disabled

1: IDMA enabled

26.10.16 SDMMC IDMA buffer size register (SDMMC_IDMABSIZE)

Address offset: 0x054

Reset value: 0x0000 0000

This register contains the buffer size when in linked list configuration.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	IDMA BNDT [11]
															rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
IDMABNDT[10:0]											Res.	Res.	Res.	Res.	Res.
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw					

Bits 31:17 Reserved, must be kept at reset value.

Bits 16:5 **IDMABNDT[11:0]**: Number of bytes per buffer

This 12-bit value must be multiplied by 8 to get the size of the buffer in 32-bit words and by 32 to get the size of the buffer in bytes.

Example: IDMABNDT = 0x001: buffer size = 8 words = 32 bytes.

Example: IDMABNDT = 0x800: buffer size = 16384 words = 64 Kbyte.

These bits can only be written by firmware when DPSM is inactive (DPSMACT = 0).

Bits 4:0 Reserved, must be kept at reset value.

26.10.17 SDMMC IDMA buffer base address register (SDMMC_IDMABASER)

Address offset: 0x058

Reset value: 0x0000 0000

This register contains the memory buffer base address in single buffer configuration and linked list configuration.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
IDMABASE[31:16]															
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
IDMABASE[15:0]															
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	r	r

Bits 31:0 **IDMABASE[31:0]**: Buffer memory base address bits [31:2], must be word aligned (bit [1:0] are always 0 and read only)

This register can be written by firmware when DPSM is inactive (DPSMACT = 0), and can dynamically be written by firmware when DPSM active (DPSMACT = 1).

26.10.18 SDMMC IDMA linked list address register (SDMMC_IDMALAR)

Address offset: 0x064

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
ULA	ULS	ABR	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
rw	rw	rw													
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
IDMALA[13:0]														Res.	Res.
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw		

Bit 31 **ULA**: Update SDMMC_IDMALAR from linked list when in linked list mode (SDMMC_IDMACTRLR.IDMABMODE select linked list mode)

This bit can only be written by firmware when DPSM is inactive (DPSMACT = 0).

0: SDMMC_IDMALAR is not to be updated, last linked list item.

1: SDMMC_IDMALAR is to be updated from linked list table.

Bit 30 **ULS**: Update SDMMC_IDMABSIZE from the next linked list when in linked list mode (SDMMC_IDMACTRLR.IDMABMODE select linked list mode and ULA = 1)

This bit can only be written by firmware when DPSM is inactive (DPSMACT = 0).

0: SDMMC_IDMABSIZE is not to be updated from next linked list table.

1: SDMMC_IDMABSIZE is to be updated from next linked list table.

Bit 29 **ABR**: Acknowledge linked list buffer ready

This bit can only be written by firmware when DPSM is inactive (DPSMACT = 0).

This bit is not taken into account when starting the first linked list buffer from the software programmed register information. ABR is only taken into account on subsequent loaded linked list items.

0: Loaded linked list buffer is not ready (this causes a linked list IDMA transfer error to be generated).

1: Loaded linked list buffer ready acknowledge. Linked list buffer data are transferred by IDMA.

Bits 28:16 Reserved, must be kept at reset value.

Bits 15:2 **IDMALA[13:0]**: Word aligned linked list item address offset

Linked list item offset pointer to the base of the next linked list item structure.

Linked list item base address is IDMABA + IDMALA.

These bits can only be written by firmware when DPSM is inactive (DPSMACT = 0).

Bits 1:0 Reserved, must be kept at reset value.

26.10.19 SDMMC IDMA linked list memory base register (SDMMC_IDMABAR)

Address offset: 0x068

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
IDMABA[29:14]															
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
IDMABA[13:0]														Res.	Res.
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw		

Bits 31:2 **IDMABA[29:0]**: Word aligned Linked list memory base address

Linked list memory base pointer.

These bits can only be written by firmware when DPSM is inactive (DPSMACT = 0).

Bits 1:0 Reserved, must be kept at reset value.

26.10.20 SDMMC data FIFO registers x (SDMMC_FIFORx)

Address offset: $0x080 + 0x004 * x$, ($x = 0$ to 15)

Reset value: $0x0000\ 0000$

The receive and transmit FIFOs can be only read or written as word (32-bit) wide registers. The FIFOs contain 16 entries on sequential addresses. This enables the CPU to use its load and store multiple operands to read from/write to the FIFO. The FIFO register interface takes care of correct data alignment inside the FIFO, the FIFO register address used by the CPU does matter.

When accessing SDMMC_FIFOR with half word or byte access an AHB bus fault is generated.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
FIFODATA[31:16]															
rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
FIFODATA[15:0]															
rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW

Bits 31:0 **FIFODATA[31:0]**: Receive and transmit FIFO data

This register can only be read or written by firmware when the DPSM is active (DPSMACT = 1).

The FIFO data occupies 16 entries of 32-bit words.

26.10.21 SDMMC register map

Table 274. SDMMC register map

Offset	Register name	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0x000	SDMMC_POWER	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	DIRPOL	VSWITCHEN	VSWITCH	PWRCTRL[1:0]	
	Reset value																												0	0	0	0	0
0x004	SDMMC_CLKCR	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	SELCLKRX[1:0]	BUSSPEED	DDR	HWFC_EN	NEGEDGE	WIDBUS[1:0]			Res.	PWRSV	Res.	Res.	CLKDIV[9:0]									
	Reset value											0	0	0	0	0	0	0	0					0	0	0	0	0	0	0	0	0	0
0x008	SDMMC_ARGR	CMDARG[31:0]																															
	Reset value	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0x00C	SDMMC_CMDR	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	CMDSPEND	BOOTEN	BOOTMODE	DT HOLD	CPSMEN	WAITPEND	WAITINT	WAITRESP[1:0]	CMDSTOP	CMDTRANS	CMDINDEX[5:0]							
	Reset value																																

Table 274. SDMMC register map (continued)

Offset	Register name	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0				
0x010	SDMMC_RESPCMDR	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	RESPCMD[5:0]									
	Reset value																											0	0	0	0	0	0				
0x014	SDMMC_RESP1R	CARDSTATUS[31:0]																																			
	Reset value	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0				
0x018	SDMMC_RESP2R	CARDSTATUS[31:0]																																			
	Reset value	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0				
0x01C	SDMMC_RESP3R	CARDSTATUS[31:0]																																			
	Reset value	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0				
0x020	SDMMC_RESP4R	CARDSTATUS[31:0]																																			
	Reset value	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0				
0x024	SDMMC_DTIMER	DATETIME[31:0]																																			
	Reset value	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0				
0x028	SDMMC_DLENR	Res.	Res.	Res.	Res.	Res.	Res.	Res.	DATALENGTH[24:0]																												
	Reset value								0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0				
0x02C	SDMMC_DCTRLR	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	FIFORST	BOOTACKEN	SDIOEN	RWMOD	RWSTOP	RWSTART	DBLOCK SIZE[3:0]				DTMODE[1:0]		DTDIR	DTEN				
	Reset value																			0	0	0	0	0	0	0	0	0	0	0	0	0	0				
0x030	SDMMC_DCNTR	Res.	Res.	Res.	Res.	Res.	Res.	DATACOUNT[24:0]																													
	Reset value							0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0				
0x034	SDMMC_STAR	Res.	Res.	Res.	IDMABTC	IDMATE	CKSTOP	VSWEND	ACKTIMEOUT	ACKFAIL	SDIOIT	BUSYD0END	BUSYD0	RXFIOE	TXFIOE	TXFIOF	TXFIOF	TXFIOF	TXFIOHF	TXFIOHE	CPSMACT	DPSMACT	DABORT	DBCKEND	DHOLD	DATAEND	CMDSENT	CMDREND	RXOVERR	TXUNDERR	DTIMEOUT	CTIMEOUT	DCRCFAIL	CCRCFAIL			
	Reset value				0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0				
0x038	SDMMC_ICR	Res.	Res.	Res.	IDMABTCC	IDMATEC	CKSTOPC	VSWENDC	ACKTIMEOUTC	ACKFAILC	SDIOITC	BUSYD0ENDC	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	DABORTC	DBCKENDC	DHOLDC	DATAENDC	CMDSENTC	CMDRENDC	RXOVERRC	TXUNDERRC	DTIMEOUTC	CTIMEOUTC	DCRCFAILC	CCRCFAILC			
	Reset value				0	0	0	0	0	0	0	0										0	0	0	0	0	0	0	0	0	0	0	0				
0x03C	SDMMC_MASKR	Res.	Res.	Res.	IDMABTCIE	Res.	CKSTOPIE	VSWENDIE	ACKTIMEOUTIE	ACKFAILIE	SDIOITIE	BUSYD0ENDIE	Res.	Res.	TXFIOEIE	TXFIOFIE	Res.	TXFIOHIE	TXFIOHEIE	Res.	Res.	Res.	DABORTIE	DBCKENDIE	DHOLDIE	DATAENDIE	CMDSENTIE	CMDRENDIE	RXOVERRIE	TXUNDERRIE	DTIMEOUTIE	CTIMEOUTIE	DCRCFAILIE	CCRCFAILIE			
	Reset value				0		0	0	0	0	0	0			0	0		0	0			0	0	0	0	0	0	0	0	0	0	0	0				

Table 274. SDMMC register map (continued)

Offset	Register name	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0											
0x040	SDMMC_ACKTIMER	Res.	Res.	Res.	Res.	Res.	Res.	Res.	ACKTIME[24:0]																																			
	Reset value								0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0										
0x044 - 0x04C	Reserved	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.											
0x050	SDMMC_IDMACTRLR	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	IDMABMODE	IDMAEN											
	Reset value																															0	0											
0x054	SDMMC_IDMABSIZE	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	IDMABNDT[11:0]										Res.	Res.	Res.	Res.	Res.	Res.												
	Reset value																0	0	0	0	0	0	0	0	0	0	0	0																
0x058	SDMMC_IDMABASER	IDMABASE[31:0]																																										
	Reset value	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0											
0x05C - 0x060	Reserved	Res.																																										
0x064	SDMMC_IDMALAR	ULA	ULS	ABR	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	IDMALA[13:0]													Res.	Res.												
	Reset value	0	0	0														0	0	0	0	0	0	0	0	0	0	0	0	0	0													
0x068	SDMMC_IDMABAR	IDMABA[29:0]																												Res.	Res.													
	Reset value	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0													
0x06C - 0x07C	Reserved	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.											
0x080 + 0x04 * x, (x=0..15)	SDMMC_FIFORx	FIFODATA[31:0]																																										
	Reset value	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0											

Refer to [Section 2.3: Memory organization](#) for the register boundary addresses.

27 Delay block (DLYB)

27.1 DLYB introduction

The delay block (DLYB) is used to generate an output clock that is dephased from the input clock. The phase of the output clock must be programmed by the user application. The output clock is then used to clock the data received by another peripheral such as an SDMMC or Octo-SPI interface.

The delay is voltage- and temperature-dependent, that may require the application to re-configure and recenter the output clock phase with the receive data.

27.2 DLYB main features

The delay block has the following features:

- Input clock frequency ranging from 25 MHz to the maximum frequency supported by the communication interface (see datasheet)
- Up to 12 oversampling phases.

27.3 DLYB implementation

Table 275. STM32H5 features

DLYB associated peripheral	DLYBOS1	DLYBSD1	DLYBSD2 ⁽¹⁾
OCTOSPI1	X	-	-
SDMMC1	-	X	-
SDMMC2	-	-	X

1. Available only on STM32H56x/573xx devices.

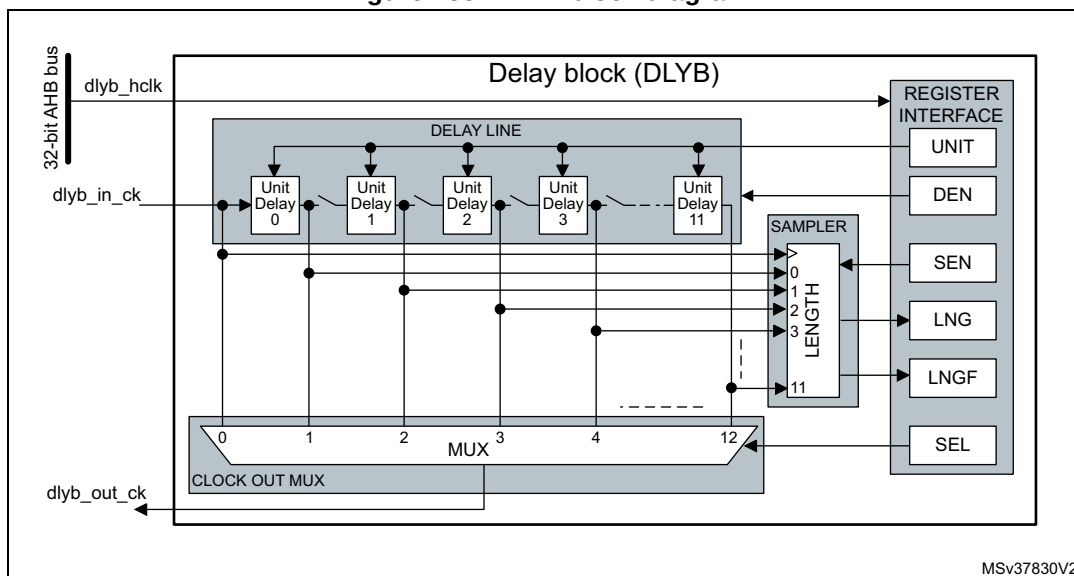
27.4 DLYB functional description

27.4.1 DLYB diagram

The delay block includes the following sub-blocks (shown in the figure below):

- register interface block providing AHB access to the DLYB registers
- delay line supporting the unit delays
- delay line length sampling
- output clock selection multiplexer

Figure 235. DLYB block diagram



27.4.2 DLYB pins and internal signals

Table 276 lists the DLYB internal signals.

Table 276. DLYB internal input/output signals

Signal name	Signal type	Description
dlyb_hclk	Digital input	Delay block register interface clock
dlyb_in_ck	Digital input	Delay block input clock
dlyb_out_ck	Digital output	Delay block output clock

27.4.3 General description

The delay block is enabled by setting the DEN bit in the DLYB control register (DLYB_CR). The length sampler is enabled through the SEN bit in DLYB_CR register.

When the delay block is enabled, the delay added by a unit delay is defined by the UNIT[6:0] field in the DLYB configuration register (DLYB_CFGR).

Note: UNIT[6:0] can be programmed only when the output clock is disabled (SEN = 1).

When the delay block is enabled, the output clock phase is selected through the SEL[3:0] field in DLYB_CFGR register.

Note: SEL can be programmed only when the output clock is disabled (SEN = 1).

The output clock can be de-phased over one input clock period by configuring the delay line length to span one period. The delay line length can be configured by enabling the length sampler through the SEN bit, that gives access to the delay line length (LNG[11:0]) and length valid flag (LNGF) in DLYB_CFGR.

If an output clock delay smaller than one input clock period is needed the delay line length can be reduced. This allows a smaller unit delay providing higher resolution.

Once the delay line length is configured, a dephased output clock can be selected by the output clock multiplexer. This is done through SEL[3:0]. The output clock is only available on the selected phase when SEN is set to 0.

The table below gives a summary of the delay block control.

Table 277. Delay block control

DEN	SEN	UNIT	SEL	LNG	LNGF	Output clock
0	0	Don't care	Don't care	Don't care	Don't care	Enabled (= Input clock)
x	1	Unit delay	Output clock phase	Length	Length flag	Disabled
1	0	Unit delay ⁽¹⁾	Output clock phase ⁽²⁾	Don't care	Don't care	Enabled (= selected phase)

1. The unit delay can only be changed when SEN = 1.

2. The output clock phase can only be changed when SEN = 1.

27.4.4 Delay line length configuration procedure

LNG[11:0] is used to determine the delay line length with respect to the input clock period. The length must be configured so that one full input clock period is covered by the delay line length.

Note that despite the delay line has 12 unit delay elements, the following procedure description returns a length between 0 and 10, as the upper delay output value is used to ensure that the delay is calibrated over one full input clock cycle. Depending on the clock frequency and UNIT value, unit delay element 10 may also be truncated from the clock cycle length.

A clock input (free running clock) must be present during the whole tuning procedure.

To configure the delay line length to one period of the Input clock, follow the sequence below:

1. Enable the delay block by setting DEN bit to 1.
2. Enable the length sampling by setting SEN bit to 1.
3. Enable all delay cells by setting SEL[3:0] to 12.
4. For UNIT[6:0] = 0 to 127 (this step must be repeated until the delay line length is configured):
 - a) Update the UNIT[6:0] value and wait till the length flag LNGF is set to 1.
 - b) Read LNG[11:0].

If (LNG[10:0] > 0) and (LNG[11] or LNG[10] = 0), the delay line length is configured to one input clock period.
5. Determine how many unit delays (N) span one input clock period: for N = 0 to 10, if LNG[N] = 1, the number of unit delays spanning the input clock period = N.
6. Disable the length sampling by clearing SEN to 0.

If an output clock delay smaller than one input clock period is needed the delay line length can be reduced smaller than one input clock period. This allows a smaller unit delay, providing a higher resolution spanning a shorter time interval.

27.4.5 Output clock phase configuration procedure

When the delay line length is configured to one input clock period, the output clock phase can be selected between the unit delays spanning one Input clock period.

Follow the steps below to select the output clock phase:

1. Disable the output clock and enable the access to the phase selection SEL[3:0] bits by setting SEN bit to 1.
2. Program SEL[3:0] with the desired output clock phase value.
3. Enable the output clock on the selected phase by clearing SEN to 0.

Octo-SPI use case:

The delay block is used in conjunction with Octo-SPI interface to allow shifting the input data sampling signal. This sampling signal can be the feedback clock or the data strobe (DQS) signal, which is delivered by certain type of devices. Note that in case DQS is used, the calibration procedure must be performed beforehand with a free running clock, as DQS is a discontinuous signal.

In case of SDR (single data rate) mode the user must typically shift the sampling signal by half period, so that the sampling edges are positioned in the middle of the valid data phase.

In case of DDR (dual data rate) mode, for which data are transitioning at start and middle of period, typical value must be close to N/4, once the calibration is completed.

In case of high frequencies and tight timing constraints, the delay setting granularity (10) might be too coarse. Since in most cases it is not necessary to have a possible delay value covering the whole sampling clock period, the "Unit" value can be overridden by application in order to improve the accuracy of the sampling edge position (example: providing a twice as small "Unit" gives twice better timing accuracy. The counterpart being that the maximum possible delay is divided by 2).

SDMMC use case:

The delay block is used in conjunction with SDMMC interface variable delay. For correct sampling point tuning the delay value must cover a whole SDMMC_CLK clock period. After having tuned the delay line length the individual delays are used in the sampling point tuning to find the optimal sampling point.

27.5 DLYB registers

All registers can be accessed in word, half-word and byte access.

27.5.1 DLYB control register (DLYB_CR)

Address offset: 0x000

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	SEN	DEN
														rw	rw

Bits 31:2 Reserved, must be kept at reset value.

Bit 1 **SEN**: Sampler length enable bit

0: Sampler length and register access to UNIT[6:0] and SEL[3:0] disabled, output clock enabled.

1: Sampler length and register access to UNIT[6:0] and SEL[3:0] enabled, output clock disabled.

Bit 0 **DEN**: Delay block enable bit

0: DLYB disabled.

1: DLYB enabled.

27.5.2 DLYB configuration register (DLYB_CFGR)

Address offset: 0x004

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
LNGF	Res.	Res.	Res.	LNG[11:0]											
r				r	r	r	r	r	r	r	r	r	r	r	r
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	UNIT[6:0]							Res.	Res.	Res.	Res.	SEL[3:0]			
	rw	rw	rw	rw	rw	rw	rw					rw	rw	rw	rw

Bit 31 **LNGF**: Length valid flag

This flag indicates when the delay line length value contained in LNG[11:0] is valid after UNIT[6:0] bits changed.

0: Length value in LNG is not valid.

1: Length value in LNG is valid.

Bits 30:28 Reserved, must be kept at reset value.

Bits 27:16 **LNG[11:0]**: Delay line length value

These bits reflect the 12 unit delay values sampled at the rising edge of the input clock. The value is only valid when LNGF = 1.

Bit 15 Reserved, must be kept at reset value.

Bits 14:8 **UNIT[6:0]**: Delay of a unit delay cell.

These bits can only be written when SEN = 1.

Unit delay = initial delay + UNIT[6:0] x delay step

Bits 7:4 Reserved, must be kept at reset value.

Bits 3:0 **SEL[3:0]**: Phase for the output clock.

These bits can only be written when SEN = 1.

Output clock phase = input clock + SEL[3:0] x unit delay

27.5.3 DLYB register map

Table 278. DLYB register map and reset values

Offset	Register name	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0x000	DLYB_CR	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	SEN	DEN
	Reset value																														0	0	
0x004	DLYB_CFGR	LNGF	Res.	Res.	Res.	LNG[11:0]												Res.	UNIT[6:0]						Res.	Res.	Res.	Res.	SEL[3:0]				
	Reset value	0				0	0	0	0	0	0	0	0	0	0	0	0		0	0	0	0	0	0	0					0	0	0	0

Refer to [Section 2.3: Memory organization](#) for the register boundary addresses.

28 Analog-to-digital converters (ADC1/2/3)

28.1 ADC introduction

This section describes the implementation of up to 3 ADCs:

- ADC1 and ADC2 are tightly coupled and can operate in dual mode (ADC1 is master).
- ADC3 is controlled independently.

Each ADC consists of a 12-bit successive approximation analog-to-digital converter.

Each ADC has up to 20 multiplexed channels. A/D conversion of the various channels can be performed in single, continuous, scan or discontinuous mode. The result of the ADC is stored in a left-aligned or right-aligned 16-bit data register.

The ADCs are mapped on the AHB bus to allow fast data handling.

The analog watchdog features allow the application to detect if the input voltage goes outside the user-defined high or low thresholds.

A built-in hardware oversampler allows improving analog performances while off-loading the related computational burden from the CPU.

An efficient low-power mode is implemented to allow very low consumption at low frequency.

28.2 ADC main features

- High-performance features
 - Up to 3 ADCs, out of which two of them can operate in dual mode:
 - ADC1 is connected to 18 external channels and to 2 internal channels
 - ADC2 is connected to 18 external channels and to 2 internal channels
 - ADC3 is connected to 10 external channels and to 6 internal channels
 - 12, 10, 8 or 6-bit configurable resolution
 - ADC conversion time independent from the AHB bus clock frequency
 - Faster conversion time by lowering resolution
 - Manage single-ended or differential inputs
 - AHB slave bus interface to allow fast data handling
 - Self-calibration
 - Channel-wise programmable sampling time
 - Flexible sampling time control
 - Up to 4 injected channels (analog inputs assignment to regular or injected channels is fully configurable)
 - Hardware assistant to prepare the context of the injected channels to allow fast context switching
 - Data alignment with in-built data coherency
 - Data can be managed by DMA for regular channel conversions
 - Four dedicated data registers for the injected channels

- Low-power features
 - Speed adaptive low-power mode to reduce ADC consumption when operating at low frequency
 - Allows slow bus frequency application while keeping optimum ADC performance
 - Provides automatic control to avoid ADC overrun in low AHB bus clock frequency application (auto-delayed mode)
- Oversampler
 - 16-bit data register
 - Oversampling ratio adjustable from 2 to 256x
 - Programmable data shift up to 8 bits
- Data preconditioning
 - Offset compensation
- Analog input channels
 - External analog inputs (per ADC):
 - Up to 6 fast channels from GPIO pads
 - Up to 12 slow channels from GPIO pads
 - Up to 2 channels for the internal temperature sensor (V_{SENSE})
 - Up to 2 channels for the internal reference voltage (V_{REFINT})
 - Up to 2 channels for monitoring the external VBAT power supply pin
 - Up to 2 channels for monitoring the internal V_{DDCORE} supply
 - Up to 2 channels for the DAC outputs
- Start-of-conversion can be initiated:
 - By software for both regular and injected conversions
 - By hardware triggers with configurable polarity (internal timers events or GPIO input events) for both regular and injected conversions
- Conversion modes
 - Each ADC can convert a single channel or can scan a sequence of channels
 - Single mode converts selected inputs once per trigger
 - Continuous mode converts selected inputs continuously
 - Discontinuous mode
- Interrupt generation at ADC ready, the end of sampling, the end of conversion (regular or injected), end of sequence conversion (regular or injected), analog watchdog 1, 2 or 3 or overrun events
- 3 analog watchdogs per ADC
 - Watchdog can perform filtering to ignore out-of-range data
- ADC input range: $V_{SSA} \leq V_{IN} \leq V_{REF+}$

Figure 236 shows the block diagram of one ADC.

28.3 ADC implementation

Table 279. ADC features

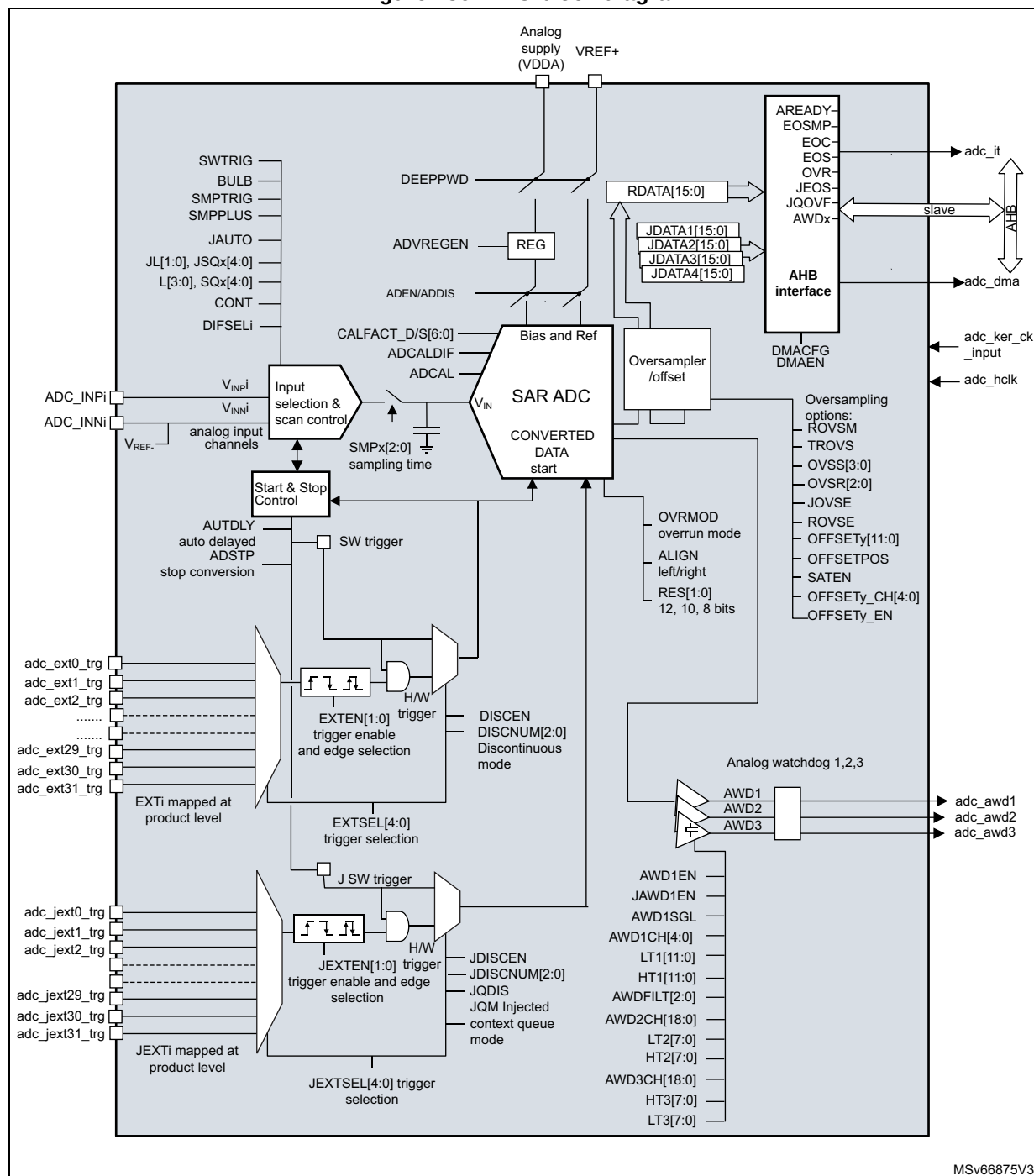
ADC modes/features	STM32H523/533/56x/573		STM32H543/553		
	ADC1	ADC2	ADC1	ADC2	ADC3
Resolution	12 bit				
Maximum sampling speed	5 Msps (12-bit resolution)				
Dual mode operation	X				-
Hardware offset calibration	X				
Hardware linearity calibration	-				
Single-end input	X				
Differential input	X				
Injected channel conversion	X				
Oversampling	up to x256				
Data register	16 bits				
Data register FIFO depth	3 stages				
DMA support	X				
Parallel data output to ADF/MDF	-				
Offset compensation	X				
Gain compensation	-				
Number of analog watchdog	3				
Option register	X				

28.4 ADC functional description

28.4.1 ADC block diagram

Figure 236 shows the ADC block diagram and Table 280 gives the ADC pin description.

Figure 236. ADC block diagram



28.4.2 ADC pins and internal signals

Table 280. ADC input/output pins

Pin name	Signal type	Description
VDDA	Input, analog supply	Analog power supply and positive reference voltage for the ADC
VSSA	Input, analog supply ground	Ground for analog power supply, equal to V_{SS} .
VREF+	Input, analog reference positive	The higher/positive reference voltage for the ADC.
VREF-	Input, analog reference negative	The lower/negative reference voltage for the ADC. V_{REF-} is internally connected to V_{SSA}
ADC1/2/3_INNi/INPi	Negative/positive external analog input signals	20 negative/positive external analog input channels (refer to Section 28.4.4: ADC connectivity for details)

Table 281. ADC internal input/output signals

Internal signal name	Signal type	Description
V_{INPi}	Positive analog input channels	Positive internal analog input channels connected either to ADC1/2/3_INPi external channels or to internal channels.
V_{INNi}	Negative analog input channels	Negative internal analog input channels connected either to ADC1/2/3_INNi external channels or to internal channels
adc_ext_trgi	Inputs	ADC external trigger inputs for regular conversions. These inputs are shared between the ADC master and the ADC slave.
adc_jext_trgi	Inputs	ADC external trigger inputs for the injected conversions. These inputs are shared between the ADC master and the ADC slave.
adc_awdx	Output	Internal analog watchdog output signal connected to on-chip timers. (x = Analog watchdog number 1,2,3)
adc_ker_ck_input	Input	ADC kernel clock
adc_hclk	Input	ADC peripheral clock
adc_it	Output	ADC interrupt
adc_dma	Output	ADC DMA request

Table 282. ADC interconnection

Signal name	Source/destination
ADC1 $V_{INP}[16]$	V_{SENSE} (internal temperature sensor output voltage).
ADC1 $V_{INP}[17]$	V_{REFINT} (output voltage from internal reference voltage).
ADC2 $V_{INP}[16]$	$V_{BAT}/4$ (VBAT pin input voltage divided by 4).
ADC2 $V_{INP}[17]$	V_{DDCORE} (internal digital core voltage).
ADC3 $V_{INP}[14]^{(1)}$	$V_{BAT}/4$ (VBAT pin input voltage divided by 4)

Table 282. ADC interconnection (continued)

Signal name	Source/destination
ADC3 V _{INP} [15] ⁽¹⁾	V _{DDCORE} (internal digital core voltage)
ADC3 V _{INP} [16] ⁽¹⁾	V _{SENSE} (internal temperature sensor output voltage)
ADC3 V _{INP} [17] ⁽¹⁾	V _{REFINT} (output voltage from internal reference voltage)
ADC3 V _{INP} [18] ⁽¹⁾	dac1_out1
ADC3 V _{INP} [19] ⁽¹⁾	dac1_out2
adc_ext_trg0	tim1_oc1
adc_ext_trg1	tim1_oc2
adc_ext_trg2	tim1_oc3
adc_ext_trg3	tim2_oc2
adc_ext_trg4	tim3_trgo
adc_ext_trg5	tim4_oc4
adc_ext_trg6	exti11
adc_ext_trg7	tim8_trgo
adc_ext_trg8	tim8_trgo2
adc_ext_trg9	tim1_trgo
adc_ext_trg10	tim1_trgo2
adc_ext_trg11	tim2_trgo
adc_ext_trg12	tim4_trgo
adc_ext_trg13	tim6_trgo
adc_ext_trg14	tim15_trgo
adc_ext_trg15	tim3_oc4
adc_ext_trg16	exti15
adc_ext_trg17	reserved
adc_ext_trg18	lptim1_ch1
adc_ext_trg19	lptim2_ch1
adc_ext_trg20	play_out7 ⁽¹⁾
adc_ext_trg21	reserved
adc_ext_trg22	reserved
adc_ext_trg23	reserved
adc_ext_trg24	reserved
adc_ext_trg25	reserved
adc_ext_trg26	reserved
adc_ext_trg27	reserved
adc_ext_trg28	reserved
adc_ext_trg29	reserved

Table 282. ADC interconnection (continued)

Signal name	Source/destination
adc_ext_trg30	reserved
adc_ext_trg31	reserved
adc_jext_trg0	tim1_trgo
adc_jext_trg1	tim1_oc4
adc_jext_trg2	tim2_trgo
adc_jext_trg3	tim2_oc1
adc_jext_trg4	tim3_oc4
adc_jext_trg5	tim4_trgo
adc_jext_trg6	exti15
adc_jext_trg7	tim8_oc4
adc_jext_trg8	tim1_trgo2
adc_jext_trg9	tim8_trgo
adc_jext_trg10	tim8_trgo2
adc_jext_trg11	tim3_oc3
adc_jext_trg12	tim3_trgo
adc_jext_trg13	tim3_oc1
adc_jext_trg14	tim6_trgo
adc_jext_trg15	tim15_trgo
adc_jext_trg16	reserved
adc_jext_trg17	reserved
adc_jext_trg18	lptim1_ch1
adc_jext_trg19	lptim2_ch1
adc_jext_trg20	play_out9 ⁽¹⁾
adc_jext_trg21	reserved
adc_jext_trg22	reserved
adc_jext_trg23	reserved
adc_jext_trg24	reserved
adc_jext_trg25	reserved
adc_jext_trg26	reserved
adc_jext_trg27	reserved
adc_jext_trg28	reserved
adc_jext_trg29	reserved
adc_jext_trg30	reserved
adc_jext_trg31	reserved

1. Only available on STM32H543/553 devices.

28.4.3 ADC clocks

Dual clock domain architecture

The dual clock-domain architecture means that the ADC clock is independent from the AHB bus clock.

The ADC input clock can be selected between two different clock sources (see [Figure 237: ADC clock scheme](#)):

1. The ADC clock can be a specific clock source (`adc_ker_ck_input`), independent and asynchronous with the AHB clock.

Refer to section *Reset and clock control (RCC)* for more information on how to generate the ADC dedicated clock. To select this scheme, `CKMODE[1:0]` bits of `ADC_CCR` register must be set to 00.

2. The ADC clock can be derived from the AHB clock interface divided by a programmable factor of 1, 2 or 4. To select this scheme, `CKMODE[1:0]` bits of `ADC_CCR` must be different from 00. The programmable divider factor can be configured through to `CKMODE[1:0]` bits of `ADC_CCR`.

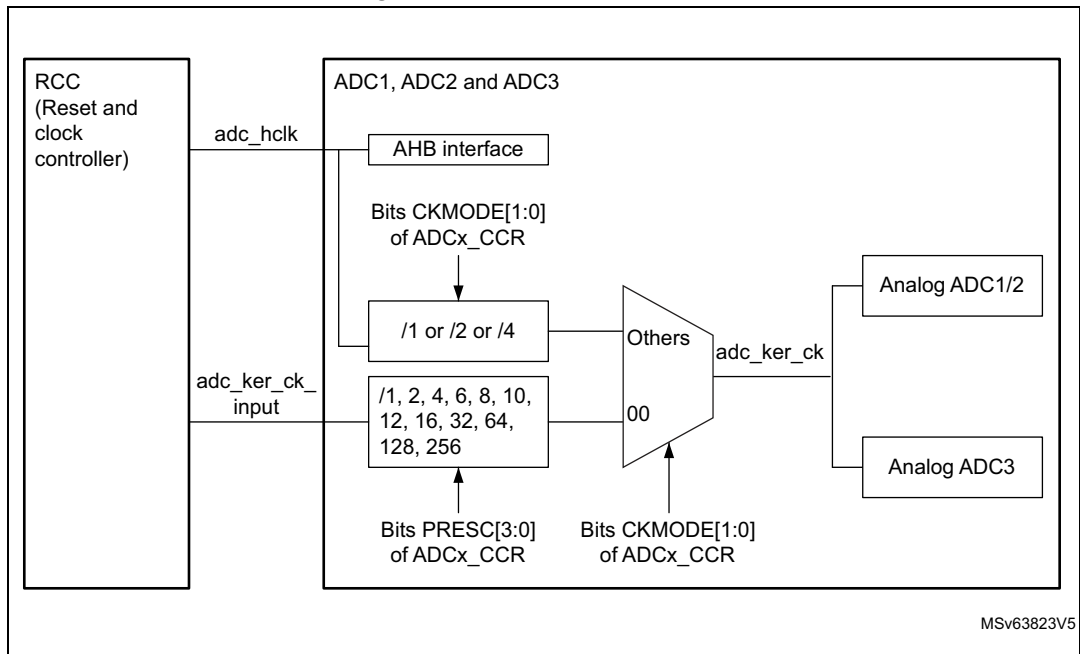
The prescaling factor of 1 (`CKMODE[1:0] = 01`) can be used only if the AHB clock prescaler is set to 1 (`HPRE[3:0] = 0XXX` in the `RCC_CFGR` register).

Option 1 has the advantage of achieving the maximum ADC clock frequency whatever the AHB clock scheme selected. The ADC clock can eventually be divided by the following ratio: 1, 2, 4, 6, 8, 12, 16, 32, 64, 128, 256, using the prescaler configured with bits `PRESC[3:0]` in the `ADC_CCR` register.

Option 2 has the advantage of bypassing the clock domain resynchronizations. This can be useful when the ADC is triggered by a timer and if the application requires that the ADC is precisely triggered without any uncertainty (otherwise, an uncertainty of the trigger instant is added by the resynchronizations between the two clock domains).

The clock is configured through `CKMODE[1:0]` bits must be compliant with the operating frequency specified in the device datasheet.

Figure 237. ADC clock scheme



Clock ratio constraint between ADC clock and AHB clock

There are generally no constraints to be respected for the ratio between the ADC clock and the AHB clock except if some injected channels are programmed. In this case, it is mandatory to respect the following ratio:

- $F_{\text{adc_hclk}} \geq F_{\text{ADC}} / 4$ if the resolution of all channels are 12-bit or 10-bit
- $F_{\text{adc_hclk}} \geq F_{\text{ADC}} / 3$ if there are some channels with resolutions equal to 8-bit (and none with lower resolutions)
- $F_{\text{adc_hclk}} \geq F_{\text{ADC}} / 2$ if there are some channels with resolutions equal to 6-bit

Constraints between ADC clocks

When several ADC interfaces are used simultaneously, it is mandatory to use the same clock source from the RCC block without prescaler ratio for all ADC interfaces.

28.4.4 ADC connectivity

ADC inputs are connected to the external channels as well as internal sources as described below.

Figure 238. ADC1 connectivity

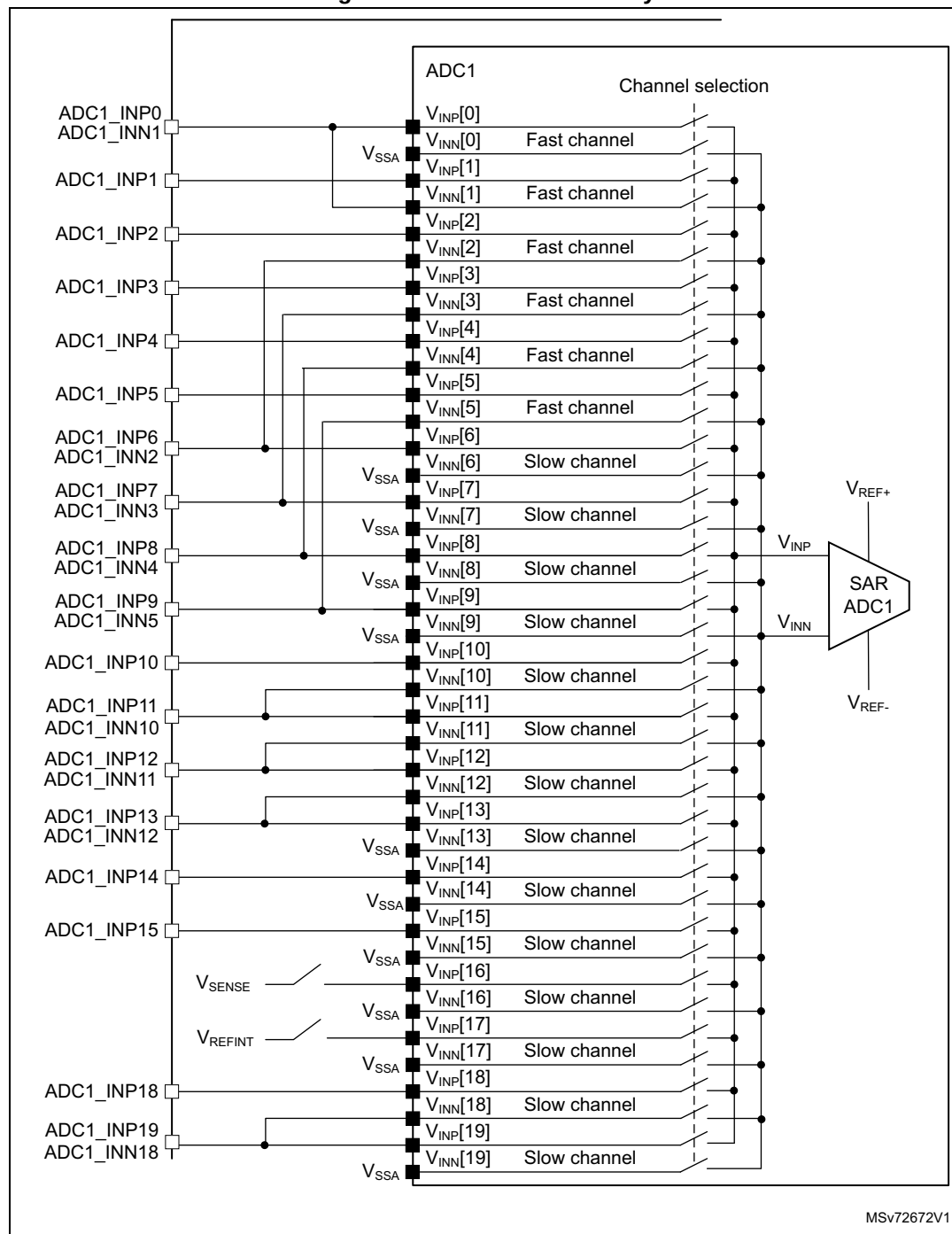


Figure 239. ADC2 connectivity

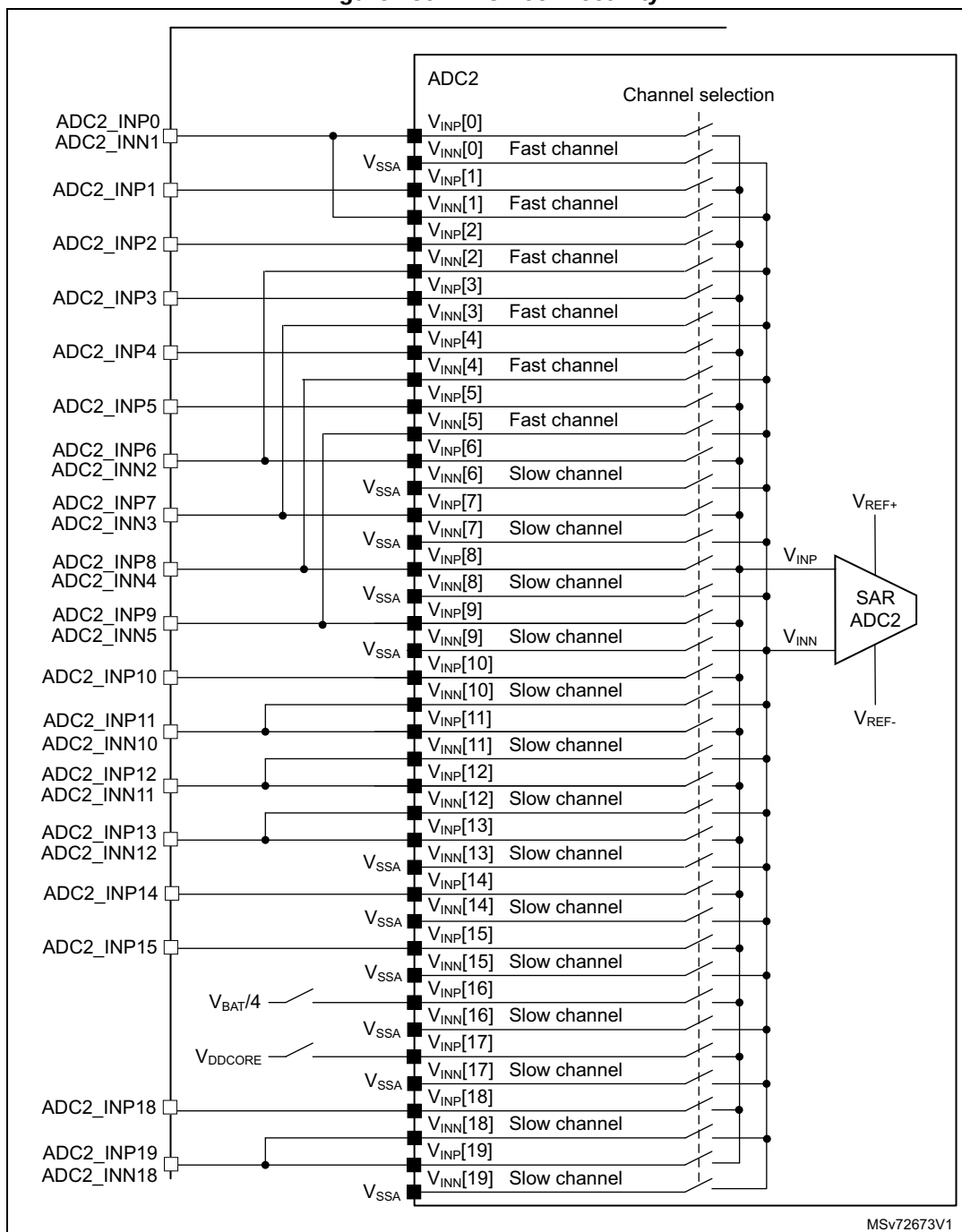
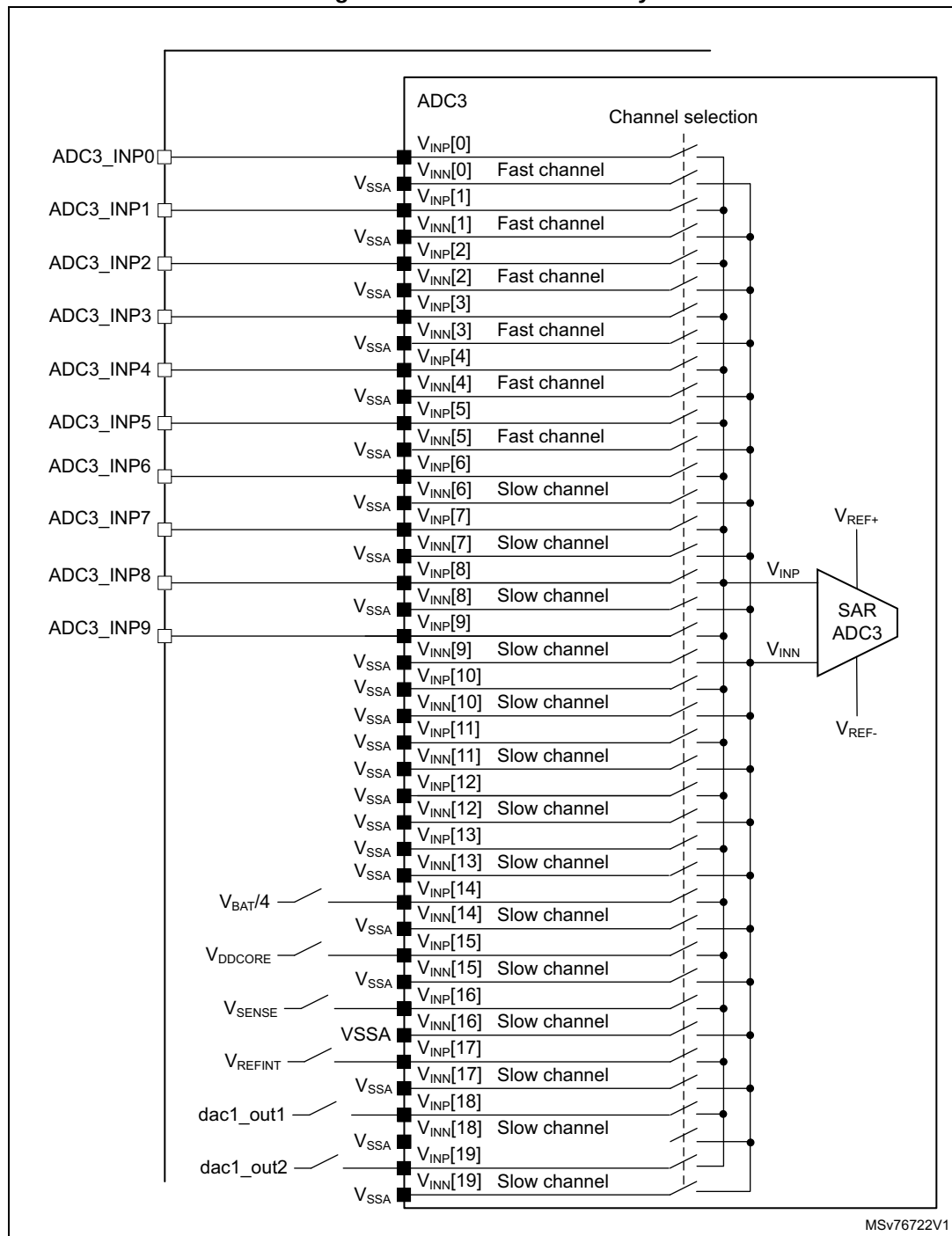


Figure 240. ADC3 connectivity



1. Only available on STM32H543/553 devices.

28.4.5 Slave AHB interface

The ADCs implement an AHB slave port for control/status register and data access. The features of the AHB interface are listed below:

- Word (32-bit) accesses
- Single cycle response
- Response to all read/write accesses to the registers with zero wait states.

The AHB slave interface does not support split/retry requests, and never generates AHB errors.

28.4.6 ADC Deep-power-down mode (DEEPPWD) and ADC voltage regulator (ADVREGEN)

By default, the ADC is in Deep-power-down mode where its supply is internally switched off to reduce the leakage currents (the reset state of bit DEEPPWD is 1 in the ADC_CR register).

To start ADC operations, it is first needed to exit Deep-power-down mode by setting bit DEEPPWD = 0.

Then, it is mandatory to enable the ADC internal voltage regulator by setting the bit ADVREGEN = 1 into ADC_CR register. The software must wait for the startup time of the ADC voltage regulator ($T_{\text{ADCVREG_STUP}}$) before launching a calibration or enabling the ADC. This delay must be implemented by software.

For the startup time of the ADC voltage regulator, refer to device datasheet for $T_{\text{ADCVREG_STUP}}$ parameter.

When ADC operations are complete, the ADC can be disabled (ADEN = 0). It is possible to save power by also disabling the ADC voltage regulator. This is done by writing bit ADVREGEN = 0.

Then, to save more power by reducing the leakage currents, it is also possible to re-enter in ADC Deep-power-down mode by setting bit DEEPPWD = 1 into ADC_CR register. This is particularly interesting before entering Stop mode.

Note: Writing DEEPPWD = 1 automatically disables the ADC voltage regulator and bit ADVREGEN is automatically cleared.

When the internal voltage regulator is disabled (ADVREGEN = 0), the internal analog calibration is kept.

In ADC Deep-power-down mode (DEEPPWD = 1), the internal analog calibration is lost and it is necessary to either relaunch a calibration or re-apply the calibration factor which was previously saved (refer to [Section 28.4.8: Calibration \(ADCAL, ADCALDIF, ADC_CALFACT\)](#)).

28.4.7 Single-ended and differential input channels

Channels can be configured to be either single-ended input or differential input by programming DIFSEL[i] bits in the ADC_DIFSEL register. This configuration must be written while the ADC is disabled (ADEN = 0). Note that the DIFSEL[i] bits corresponding to single-ended channels are always programmed at 0.

In single-ended input mode, the analog voltage to be converted for channel “i” is the difference between the external voltage $V_{INP[i]}$ (positive input) and V_{REF-} (negative input).

In differential input mode, the analog voltage to be converted for channel “i” is the difference between the external voltage $V_{INP[i]}$ (positive input) and $V_{INN[i]}$ (negative input).

The output data for the differential mode is an unsigned data. When $V_{INP[i]}$ equals V_{REF-} , $V_{INN[i]}$ equals V_{REF+} and the output data is 0x000 (12-bit resolution mode). When $V_{INP[i]}$ equals V_{REF+} , $V_{INN[i]}$ equals V_{REF-} and the output data is 0xFFF.

$$\text{Converted value} = \frac{\text{ADC_Full_Scale}}{2} \times \left[1 + \frac{V_{INP} - V_{INN}}{V_{REF+}} \right]$$

When ADC is configured as differential mode, both inputs should be biased at $(V_{REF+}) / 2$ voltage.

The input signals are supposed to be differential (common mode voltage should be fixed).

Internal channels (such as V_{REFINT} and V_{SENSE}) are used in single-ended mode only.

For a complete description of how the input channels are connected for each ADC, refer to [Section 28.4.4: ADC connectivity](#).

Caution: When configuring the channel “i” in differential input mode, its negative input voltage $V_{INN[i]}$ is connected to another channel. As a consequence, this channel is no longer usable in single-ended mode or in differential mode and must never be configured to be converted. Some channels are shared between ADC1/ADC2/ADC3: this can make the channel on the other ADC unusable. Only exception is interleaved mode for ADC master and the slave.

28.4.8 Calibration (ADCAL, ADCALDIF, ADC_CALFACT)

Each ADC provides an automatic calibration procedure which drives all the calibration sequence including the power-on/off sequence of the ADC. During the procedure, the ADC calculates a calibration factor which is 7-bit wide and which is applied internally to the ADC until the next ADC power-off. During the calibration procedure, the application must not use the ADC and must wait until calibration is complete.

Calibration is preliminary to any ADC operation. It removes the offset error which may vary from chip to chip due to process or bandgap variation.

The calibration factor to be applied for single-ended input conversions is different from the factor to be applied for differential input conversions:

- Write ADCALDIF = 0 before launching a calibration which is applied for single-ended input conversions.
- Write ADCALDIF = 1 before launching a calibration which is applied for differential input conversions.

The calibration is then initiated by software by setting bit ADCAL = 1. Calibration can only be initiated when the ADC is disabled (when ADEN = 0). ADCAL bit stays at 1 during all the

calibration sequence. It is then cleared by hardware as soon the calibration completes. At this time, the associated calibration factor is stored internally in the analog ADC and also in the bits CALFACT_S[6:0] or CALFACT_D[6:0] of ADC_CALFACT register (depending on single-ended or differential input calibration)

The internal analog calibration is kept if the ADC is disabled (ADEN = 0). However, if the ADC is disabled for extended periods, then it is recommended that a new calibration cycle is run before re-enabling the ADC.

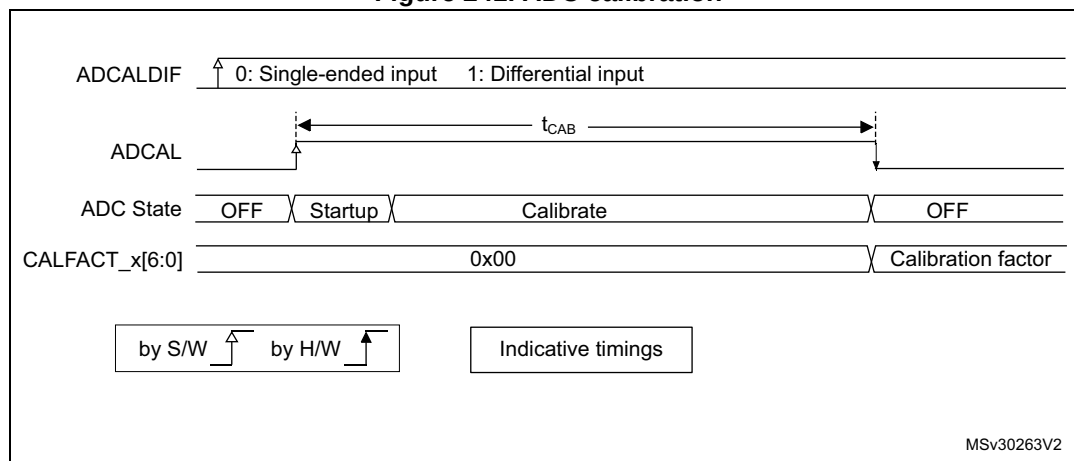
The internal analog calibration is lost each time the power of the ADC is removed (example, when the product enters in Standby or V_{BAT} mode). In this case, to avoid spending time recalibrating the ADC, it is possible to re-write the calibration factor into the ADC_CALFACT register without recalibrating, supposing that the software has previously saved the calibration factor delivered during the previous calibration.

The calibration factor can be written if the ADC is enabled but not converting (ADEN = 1 and ADSTART = 0 and JADSTART = 0). Then, at the next start of conversion, the calibration factor is automatically injected into the analog ADC. This loading is transparent and does not add any cycle latency to the start of the conversion. It is recommended to recalibrate when V_{REF+} voltage changed more than 10%.

Software procedure to calibrate the ADC

1. Ensure DEEPPWD = 0, ADVREGEN = 1 and that ADC voltage regulator startup time has elapsed.
2. Ensure that ADEN = 0.
3. Select the input mode for this calibration by setting ADCALDIF = 0 (single-ended input) or ADCALDIF = 1 (differential input).
4. Set ADCAL = 1.
5. Wait until ADCAL = 0.
6. The calibration factor can be read from ADC_CALFACT register.

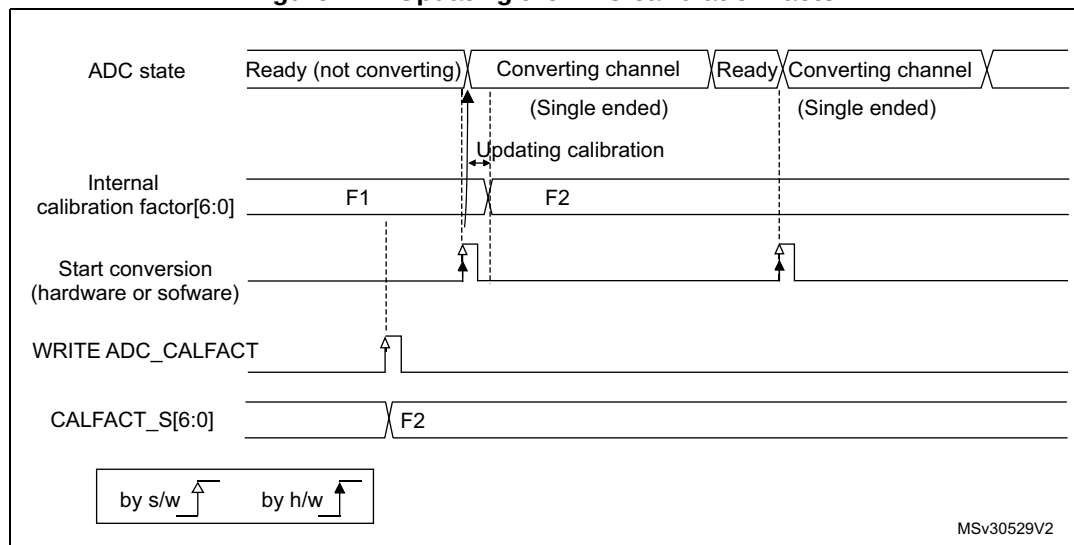
Figure 241. ADC calibration



Software procedure to reinject a calibration factor into the ADC

1. Ensure ADEN = 1 and ADSTART = 0 and JADSTART = 0 (ADC enabled and no conversion is ongoing).
2. Write CALFACT_S and CALFACT_D with the new calibration factors.
3. When a conversion is launched, the calibration factor is injected into the analog ADC only if the internal analog calibration factor differs from the one stored in bits CALFACT_S for single-ended input channel or bits CALFACT_D for differential input channel.

Figure 242. Updating the ADC calibration factor

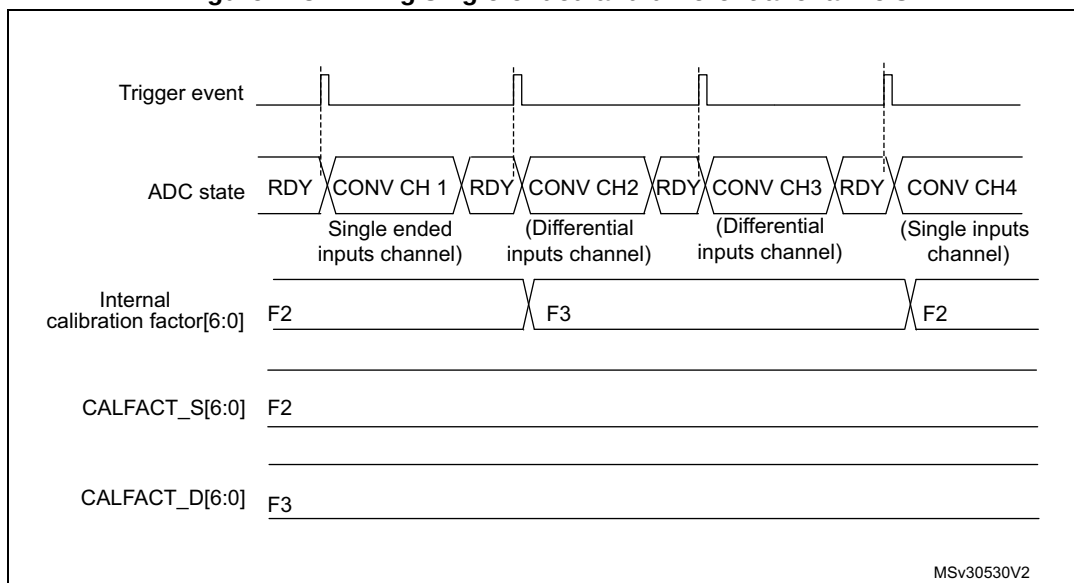


Converting single-ended and differential analog inputs with a single ADC

If the ADC is supposed to convert both differential and single-ended inputs, two calibrations must be performed, one with ADCALDIF = 0 and one with ADCALDIF = 1. The procedure is the following:

1. Disable the ADC.
2. Calibrate the ADC in single-ended input mode (with ADCALDIF = 0). This updates the register CALFACT_S[6:0].
3. Calibrate the ADC in differential input modes (with ADCALDIF = 1). This updates the register CALFACT_D[6:0].
4. Enable the ADC, configure the channels and launch the conversions. Each time there is a switch from a single-ended to a differential inputs channel (and vice-versa), the calibration is automatically injected into the analog ADC.

Figure 243. Mixing single-ended and differential channels



28.4.9 ADC on-off control (ADEN, ADDIS, ADRDY)

First of all, follow the procedure explained in [Section 28.4.6: ADC Deep-power-down mode \(DEEPPWD\) and ADC voltage regulator \(ADVREGEN\)](#).

Once DEEPPWD = 0 and ADVREGEN = 1, the ADC can be enabled and the ADC needs a stabilization time of t_{STAB} before it starts converting accurately, as shown in [Figure 244](#). Two control bits enable or disable the ADC:

- ADEN = 1 enables the ADC. The flag ADRDY is set once the ADC is ready for operation.
- ADDIS = 1 disables the ADC. ADEN and ADDIS are then automatically cleared by hardware as soon as the analog ADC is effectively disabled.

Regular conversion can then start either by setting ADSTART = 1 (refer to [Section 28.4.18: Conversion on external trigger and trigger polarity \(EXTSEL, EXTEN, JEXTSEL, JEXTEN\)](#)) or when an external trigger event occurs, if triggers are enabled.

Injected conversions start by setting JADSTART = 1 or when an external injected trigger event occurs, if injected triggers are enabled.

Software procedure to enable the ADC

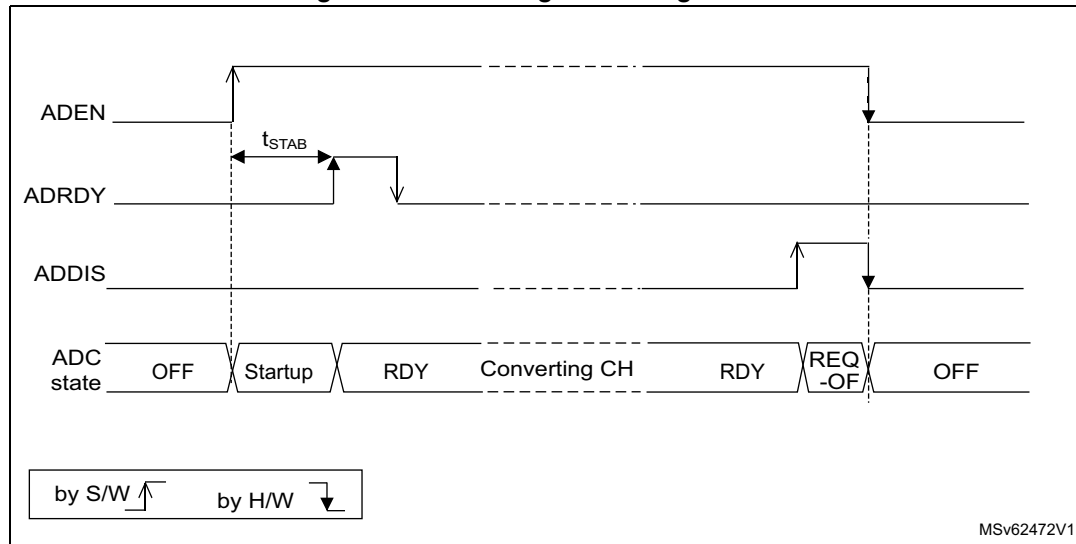
1. Clear the ADRDY bit in the ADC_ISR register by writing '1'.
2. Set ADEN = 1.
3. Wait until ADRDY = 1 (ADRDY is set after the ADC startup time). This can be done using the associated interrupt (setting ADRDYIE = 1).
4. Clear the ADRDY bit in the ADC_ISR register by writing '1' (optional).

Caution: ADEN bit cannot be set when ADCAL is set and during four ADC clock cycles after the ADCAL bit is cleared by hardware (end of the calibration).

Software procedure to disable the ADC

1. Check that both ADSTART = 0 and JADSTART = 0 to ensure that no conversion is ongoing. If required, stop any regular and injected conversion ongoing by setting ADSTP = 1 and JADSTP = 1 and then wait until ADSTP = 0 and JADSTP = 0.
2. Set ADDIS = 1.
3. If required by the application, wait until ADEN = 0, until the analog ADC is effectively disabled (ADDIS is automatically reset once ADEN = 0).

Figure 244. Enabling / disabling the ADC



MSv62472V1

28.4.10 Constraints when writing the ADC control bits

The software is allowed to write the RCC control bits to configure and enable the ADC clock (refer to RCC Section), the DIFSEL[i] control bits in the ADC_DIFSEL register and the control bits ADCAL and ADEN in the ADC_CR register, only if the ADC is disabled (ADEN must be equal to 0).

The software is then allowed to write the control bits ADSTART, JADSTART and ADDIS of the ADC_CR register only if the ADC is enabled and there is no pending request to disable the ADC (ADEN must be equal to 1 and ADDIS to 0).

For all the other control bits of the ADC_CFGR, ADC_SMPRx, ADC_TRy, ADC_SQRy, ADC_JDRy, ADC_OFRy, ADC_OFCHRy and ADC_IER registers:

- For control bits related to configuration of regular conversions, the software is allowed to write them only if the ADC is enabled (ADEN = 1) and if there is no regular conversion ongoing (ADSTART must be equal to 0).
- For control bits related to configuration of injected conversions, the software is allowed to write them only if the ADC is enabled (ADEN = 1) and if there is no injected conversion ongoing (JADSTART must be equal to 0).
- ADC_TRy registers can be modified when an analog-to-digital conversion is ongoing (refer to [Section 28.4.28: Analog window watchdog \(AWD1EN, JAWD1EN, AWD1SGL, AWD1CH, AWD2CH, AWD3CH, AWD_HTx, AWD_LTx, AWDx\)](#) for details).

The software is allowed to write the ADSTP or JADSTP control bits of the ADC_CR register only if the ADC is enabled, possibly converting, and if there is no pending request to disable the ADC (ADSTART or JADSTART must be equal to 1 and ADDIS to 0).

The software can write the register ADC_JSQR at any time, when the ADC is enabled (ADEN = 1). Refer to [Section 28.6.16: ADC injected sequence register \(ADC_JSQR\)](#) for additional details.

Note: There is no hardware protection to prevent these forbidden write accesses and ADC behavior may become in an unknown state. To recover from this situation, the ADC must be disabled (clear ADEN = 0 as well as all the bits of ADC_CR register).

28.4.11 Channel selection (ADC_SQRy, ADC_JSQR)

The ADC features up to 20 multiplexed channels per ADC, out of which:

- Up to 18 analog inputs coming from GPIO pads (ADC_INP/INN[i]) depending on the products, not all of them are available on GPIO pads.
- Up to 6 internal channels (refer to [Section 28.4.4: ADC connectivity](#) for details)
To convert one of the internal analog channels, the corresponding analog sources must first be enabled by programming bits VREFEN, VBATEN or TSEN in the ADC_CCR registers.

Refer to *Table ADC interconnection* in [Section 28.4.2: ADC pins and internal signals](#) for the connection of the above internal analog inputs to external ADC pins or internal signals.

The conversions can be organized in two groups: regular and injected. A group consists of a sequence of conversions that can be done on any channel and in any order. For instance, it is possible to implement the conversion sequence in the following order:

ADC1/2/3_INP/INN3, ADC1/2/3_INP/INN8, ADC1/2/3_INP/INN2, ADC1/2/3_INN/INP2, ADC1/2/3_INP/INN0, ADC1/2/3_INP/INN2, ADC1/2/3_INP/INN2, ADC1/2/3_INP/INN15.

- A **regular group** is composed of up to 16 conversions. The regular channels and their order in the conversion sequence must be selected in the ADC_SQRy registers. The total number of conversions in the regular group must be written in the L[3:0] bits in the ADC_SQR1 register.
- An **injected group** is composed of up to 4 conversions. The injected channels and their order in the conversion sequence must be selected in the ADC_JSQR register. The total number of conversions in the injected group must be written in the L[1:0] bits in the ADC_JSQR register.

ADC_SQRy registers must not be modified while regular conversions can occur. For this, the ADC regular conversions must be first stopped by writing ADSTP = 1 (refer to [Section 28.4.17: Stopping an ongoing conversion \(ADSTP, JADSTP\)](#)).

The software is allowed to modify on-the-fly the ADC_JSQR register when JADSTART is set to 1 (injected conversions ongoing) only when the context queue is enabled (JQDIS = 0 in ADC_CFGR register). Refer to [Section 28.4.21: Queue of context for injected conversions](#)

28.4.12 Channel-wise programmable sampling time (SMPR1, SMPR2)

Before starting a conversion, the ADC must establish a direct connection between the voltage source under measurement and the embedded sampling capacitor of the ADC. This sampling time must be enough for the input voltage source to charge the embedded capacitor to the input voltage level.

Each channel can be sampled with a different sampling time which is programmable using the SMP[2:0] bits in the ADC_SMPR1 and ADC_SMPR2 registers. It is therefore possible to select among the following sampling time values:

- SMP = 000: 2.5 ADC clock cycles
- SMP = 001: 6.5 ADC clock cycles
- SMP = 010: 12.5 ADC clock cycles
- SMP = 011: 24.5 ADC clock cycles
- SMP = 100: 47.5 ADC clock cycles
- SMP = 101: 92.5 ADC clock cycles
- SMP = 110: 247.5 ADC clock cycles
- SMP = 111: 640.5 ADC clock cycles

The total conversion time is calculated as follows:

$$T_{\text{CONV}} = \text{Sampling time} + 12.5 \text{ ADC clock cycles}$$

Example:

With $F_{\text{adc_ker_ck}} = 30 \text{ MHz}$ and a sampling time of 2.5 ADC clock cycles:

$$T_{\text{CONV}} = (2.5 + 12.5) \text{ ADC clock cycles} = 15 \text{ ADC clock cycles} = 500 \text{ ns}$$

The ADC notifies the end of the sampling phase by setting the status bit EOSMP (only for regular conversion).

Note: *Depending on the ADC conversion mode, the real sampling time can vary compared to the SMP value programmed above, while the equivalent total conversion time (T_{CONV}) does not change:*

- *For the first conversion in scan or continuous mode and all the conversions in discontinuous mode, the real sampling time is 0.5 clock cycle less compared to the value configured above.*
- *For the second and subsequent conversions in scan or continuous mode, 0.5 cycle is added to the configured sampling time. This additional 0.5 clock cycle overlaps with the previous conversion cycle.*

Constraints on the sampling time

For each channel, SMP[2:0] bits must be programmed to respect a minimum sampling time as specified in the ADC characteristics section of the datasheets.

Bulb sampling mode

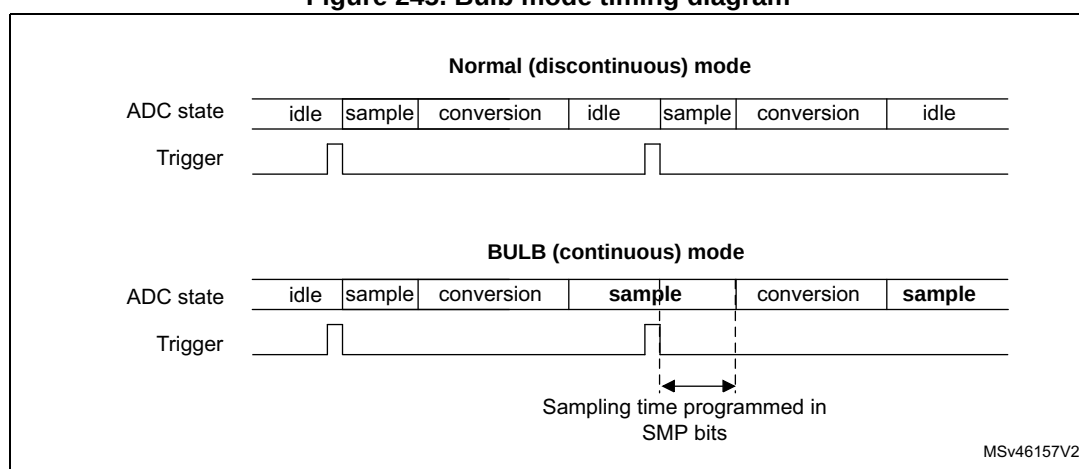
When the BULB bit is set in ADC register, the sampling period starts immediately after the last ADC conversion. A hardware or software trigger starts the conversion after the sampling time has been programmed in ADC_SMPR1 register. The very first ADC conversion, after the ADC is enabled, is performed with the sampling time programmed in SMP bits. The bulb mode is effective starting from the second conversion.

The maximum sampling time is limited (refer to the ADC characteristics section of the datasheet).

The bulb mode is neither compatible with the continuous conversion mode nor with the injected channel conversion.

When the BULB bit is set, it is not allowed to set SMPTRIG bit in ADC_CFGR2.

Figure 245. Bulb mode timing diagram



Sampling time control trigger mode

When the SMPTRIG bit is set, the sampling time programmed though SMPx bits is not applicable. The sampling time is controlled by the trigger signal edge.

When a hardware trigger is selected, each rising edge of the trigger signal starts the sampling period. A falling edge ends the sampling period and starts the conversion.

When a software trigger is selected, the software trigger is not the ADSTART bit in ADC_CR but the SWTRIG bit. SWTRIG bit has to be set to start the sampling period, and the SWTRIG bit has to be cleared to end the sampling period and start the conversion.

The maximum sampling time is limited (refer to the ADC characteristics section of the datasheet).

This mode is neither compatible with the continuous conversion mode, nor with the injected channel conversion.

When SMPTRIG bit is set, it is not allowed to set BULB bit.

I/O analog switch voltage booster

The resistance of the I/O analog switches increases when the V_{DDA} voltage is too low. The sampling time must consequently be adapted accordingly (refer to the device datasheet for the corresponding electrical characteristics). This resistance can be minimized at low V_{DDA} .

by enabling an internal voltage booster through BOOSTE bit or by selecting a V_{DD} booster voltage (if $V_{DD} > 2.7$ V) through the ADV_READY bit of the PWR_PMCRCR register.

SMPPLUS control bit

The SMPPLUS bit can be used to change the sampling time from 2.5 ADC clock cycles to 3.5 ADC clock cycles.

28.4.13 Single conversion mode (CONT = 0)

In single conversion mode, the ADC performs once all the conversions of the channels. This mode is started with the CONT bit at 0 by either:

- Setting the ADSTART bit in the ADC_CR register (for a regular channel)
- Setting the JADSTART bit in the ADC_CR register (for an injected channel)
- External hardware trigger event (for a regular or injected channel)

Inside the regular sequence, after each conversion is complete:

- The converted data are stored into the 16-bit ADC_DR register
- The EOC (end of regular conversion) flag is set
- An interrupt is generated if the EOCIE bit is set

Inside the injected sequence, after each conversion is complete:

- The converted data are stored into one of the four 16-bit ADC_JDRy registers
- The JEOC (end of injected conversion) flag is set
- An interrupt is generated if the JEOCIE bit is set

After the regular sequence is complete:

- The EOS (end of regular sequence) flag is set
- An interrupt is generated if the EOSIE bit is set

After the injected sequence is complete:

- The JEOS (end of injected sequence) flag is set
- An interrupt is generated if the JEOSIE bit is set

Then the ADC stops until a new external regular or injected trigger occurs or until bit ADSTART or JADSTART is set again.

Note: To convert a single channel, program a sequence with a length of 1.

28.4.14 Continuous conversion mode (CONT = 1)

This mode applies to regular channels only.

In continuous conversion mode, when a software or hardware regular trigger event occurs, the ADC performs once all the regular conversions of the channels and then automatically restarts and continuously converts each conversions of the sequence. This mode is started with the CONT bit at 1 either by external trigger or by setting the ADSTART bit in the ADC_CR register.

Inside the regular sequence, after each conversion is complete:

- The converted data are stored into the 16-bit ADC_DR register
- The EOC (end of conversion) flag is set
- An interrupt is generated if the EOCIE bit is set

After the sequence of conversions is complete:

- The EOS (end of sequence) flag is set
- An interrupt is generated if the EOSIE bit is set

Then, a new sequence restarts immediately and the ADC continuously repeats the conversion sequence.

Note: To convert a single channel, program a sequence with a length of 1.

It is not possible to have both discontinuous mode and continuous mode enabled: it is forbidden to set both DISCEN = 1 and CONT = 1.

Injected channels cannot be converted continuously. The only exception is when an injected channel is configured to be converted automatically after regular channels in continuous mode (using JAUTO bit), refer to [Auto-injection mode](#) section).

28.4.15 Starting conversions (ADSTART, JADSTART)

Software starts ADC regular conversions by setting ADSTART = 1.

When ADSTART is set, the conversion starts:

- Immediately: if EXTEN = 0x0 (software trigger)
- At the next active edge of the selected regular hardware trigger: if EXTEN is not equal to 0x0

Software starts ADC injected conversions by setting JADSTART = 1.

When JADSTART is set, the conversion starts:

- Immediately, if JEXTEN = 0x0 (software trigger)
- At the next active edge of the selected injected hardware trigger: if JEXTEN is not equal to 0x0

Note: In auto-injection mode (JAUTO = 1), use ADSTART bit to start the regular conversions followed by the auto-injected conversions (JADSTART must be kept cleared).

ADSTART and JADSTART also provide information on whether any ADC operation is currently ongoing. It is possible to re-configure the ADC while ADSTART = 0 and JADSTART = 0 are both true, indicating that the ADC is idle.

ADSTART is cleared by hardware:

- In single mode with software regular trigger (CONT = 0, EXTSEL = 0x0)
 - At any end of regular conversion sequence (EOS assertion) or at any end of subgroup processing if DISCEN = 1
- In all cases (CONT = x, EXTSEL = x)
 - After execution of the ADSTP procedure asserted by the software.

Note: In continuous mode (CONT = 1), ADSTART is not cleared by hardware with the assertion of EOS because the sequence is automatically relaunched.

When a hardware trigger is selected in single mode (CONT = 0 and EXTSEL ≠ 0x00), ADSTART is not cleared by hardware with the assertion of EOS to help the software which does not need to reset ADSTART again for the next hardware trigger event. This ensures that no further hardware triggers are missed.

JADSTART is cleared by hardware:

- In single mode with software injected trigger (JEXTSEL = 0x0)
 - At any end of injected conversion sequence (JEOS assertion) or at any end of subgroup processing if JDISCEN = 1
- in all cases (JEXTSEL = x)
 - After execution of the JADSTP procedure asserted by the software.

Note: When the software trigger is selected, ADSTART bit should not be set if the EOC flag is still high.

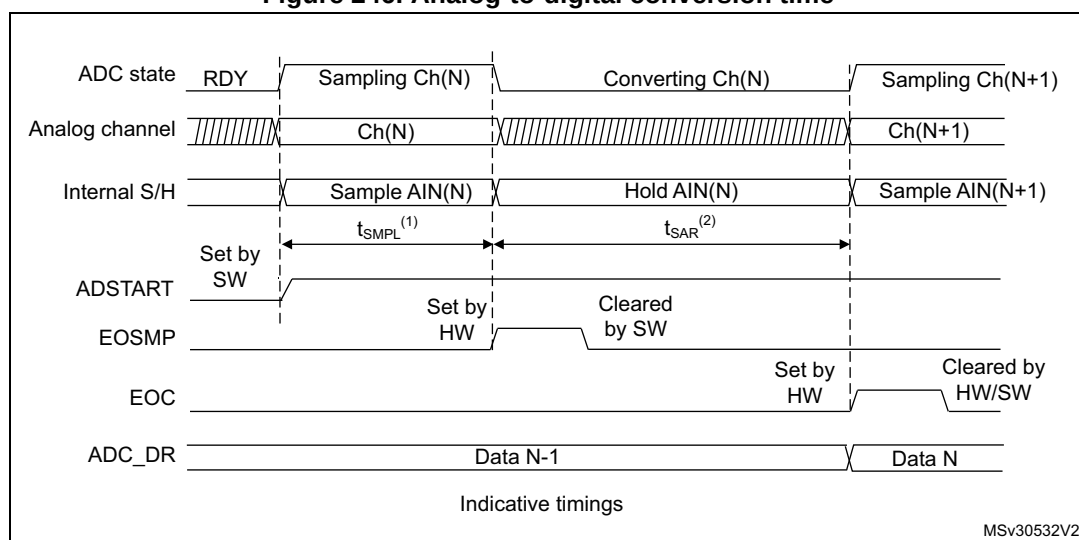
28.4.16 ADC timing

The elapsed time between the start of a conversion and the end of conversion is the sum of the configured sampling time plus the successive approximation time depending on data resolution:

$$T_{\text{CONV}} = T_{\text{SMPL}} + T_{\text{SAR}} = [2.5_{\text{min}} + 12.5_{\text{12bit}}] \times T_{\text{ADC_CLK}}$$

$$T_{\text{CONV}} = T_{\text{SMPL}} + T_{\text{SAR}} = 83.33 \text{ ns}_{\text{min}} + 416.67 \text{ ns}_{\text{12bit}} = 500.0 \text{ ns (for } F_{\text{ADC_CLK}} = 30 \text{ MHz)}$$

Figure 246. Analog-to-digital conversion time



1. T_{SMPL} depends on SMP[2:0].

2. T_{SAR} depends on RES[2:0].

28.4.17 Stopping an ongoing conversion (ADSTP, JADSTP)

The software can decide to stop regular conversions ongoing by setting $ADSTP = 1$ and injected conversions ongoing by setting $JADSTP = 1$.

Stopping conversions resets the ongoing ADC operation. Then the ADC can be reconfigured (ex: changing the channel selection or the trigger) ready for a new operation.

Note that it is possible to stop injected conversions while regular conversions are still operating and vice-versa. This allows, for instance, re-configuration of the injected conversion sequence and triggers while regular conversions are still operating (and vice-versa).

When the $ADSTP$ bit is set by software, any ongoing regular conversion is aborted with partial result discarded (ADC_DR register is not updated with the current conversion).

When the $JADSTP$ bit is set by software, any ongoing injected conversion is aborted with partial result discarded (ADC_JDRy register is not updated with the current conversion). The scan sequence is also aborted and reset (meaning that relaunching the ADC would restart a new sequence).

Once this procedure is complete, bits $ADSTP/ADSTART$ (in case of regular conversion), or $JADSTP/JADSTART$ (in case of injected conversion) are cleared by hardware and the software must poll $ADSTART$ (or $JADSTART$) until the bit is reset before assuming the ADC is completely stopped.

Note: In auto-injection mode ($JAUTO = 1$), setting $ADSTP$ bit aborts both regular and injected conversions ($JADSTP$ must not be used).

Figure 247. Stopping ongoing regular conversions

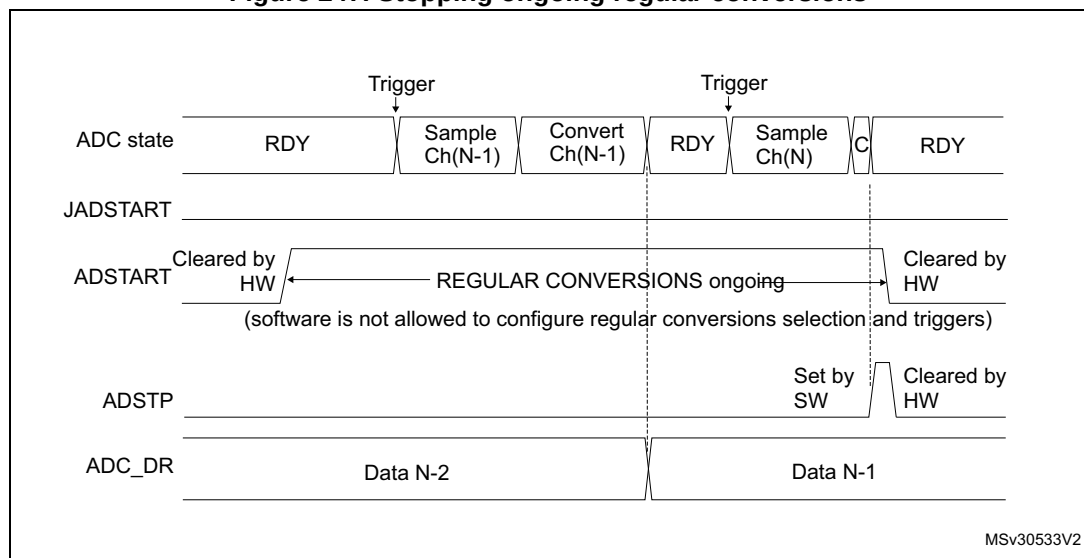
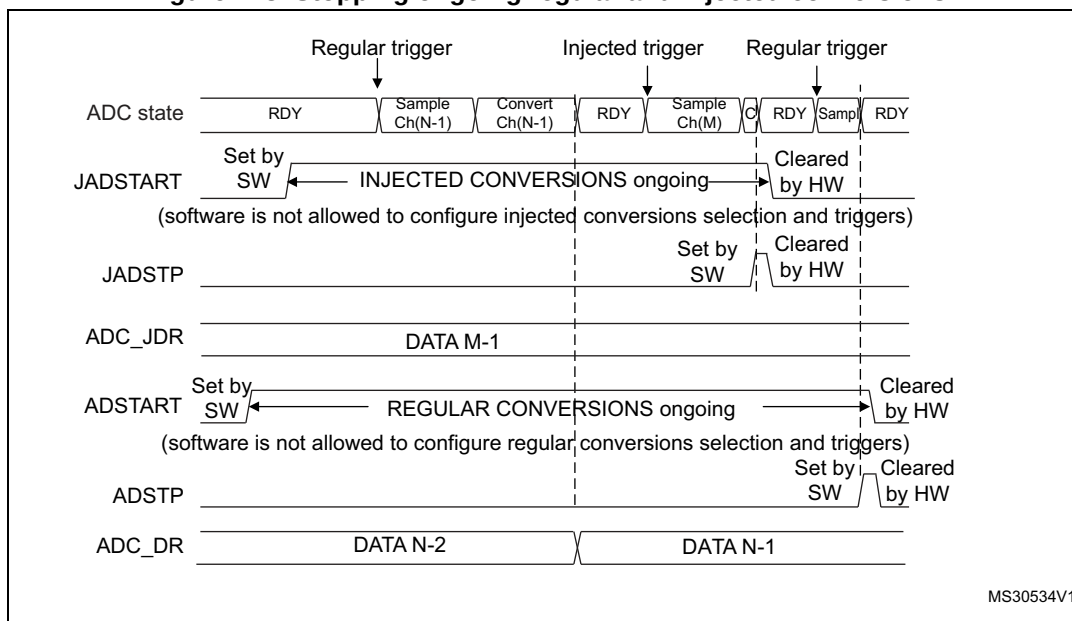


Figure 248. Stopping ongoing regular and injected conversions



MS30534V1

28.4.18 Conversion on external trigger and trigger polarity (EXTSEL, EXTEN, JEXTSEL, JEXTEN)

A conversion or a sequence of conversions can be triggered either by software or by an external event (such as timer capture, input pins). If the EXTEN[1:0] control bits (for a regular conversion) or JEXTEN[1:0] bits (for an injected conversion) are different from 0b00, then external events are able to trigger a conversion with the selected polarity.

When the Injected Queue is enabled (bit JQDIS = 0), injected software triggers are not possible.

The regular trigger selection is effective once software has set bit ADSTART = 1 and the injected trigger selection is effective once software has set bit JADSTART = 1.

Any hardware triggers which occur while a conversion is ongoing are ignored.

- If bit ADSTART = 0, any regular hardware triggers which occur are ignored.
- If bit JADSTART = 0, any injected hardware triggers which occur are ignored.

Table 283 provides the correspondence between the EXTEN[1:0] and JEXTEN[1:0] values and the trigger polarity.

Table 283. Configuring the trigger polarity for regular external triggers

EXTEN[1:0]	Source
00	Hardware Trigger detection disabled, software trigger detection enabled
01	Hardware Trigger with detection on the rising edge
10	Hardware Trigger with detection on the falling edge
11	Hardware Trigger with detection on both the rising and falling edges

Note: The polarity of the regular trigger cannot be changed on-the-fly.

Table 284. Configuring the trigger polarity for injected external triggers

JEXTEN[1:0]	Source
00	– If JQDIS = 1 (Queue disabled): Hardware trigger detection disabled, software trigger detection enabled – If JQDIS = 0 (Queue enabled), Hardware and software trigger detection disabled
01	Hardware Trigger with detection on the rising edge
10	Hardware Trigger with detection on the falling edge
11	Hardware Trigger with detection on both the rising and falling edges

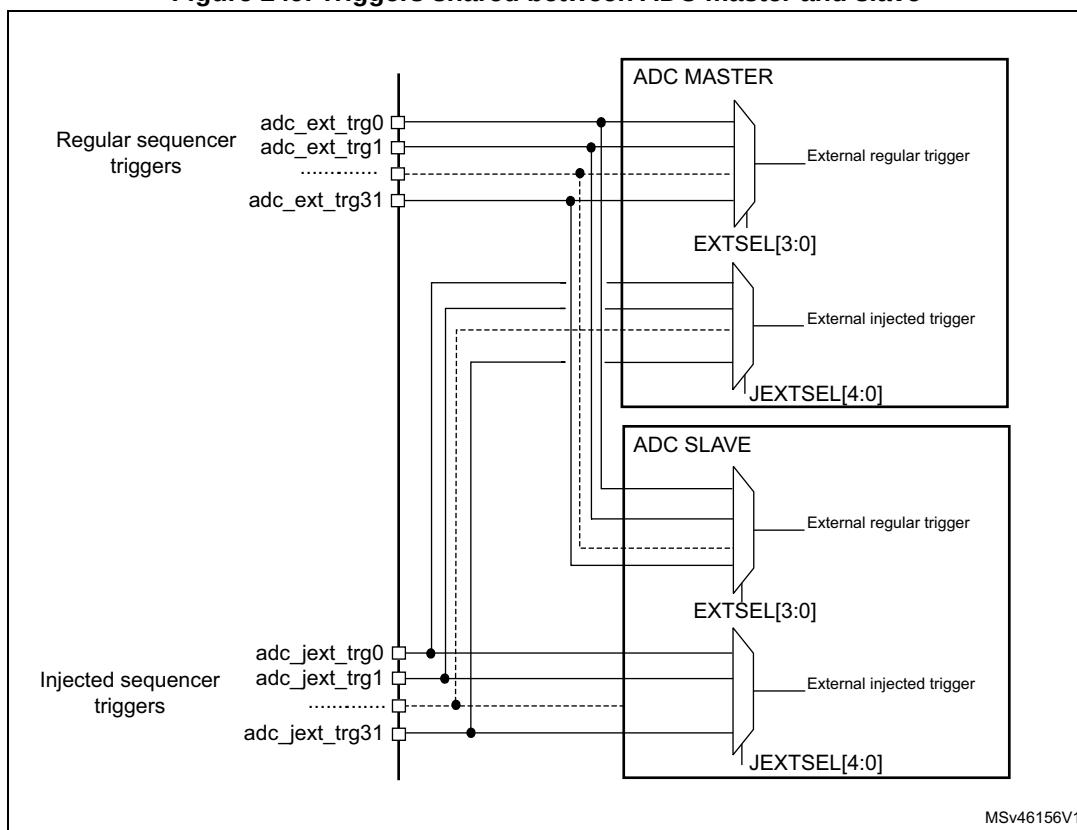
Note: The polarity of the injected trigger can be anticipated and changed on-the-fly when the queue is enabled (JQDIS = 0). Refer to [Section 28.4.21: Queue of context for injected conversions](#).

The EXTSEL and JEXTSEL control bits select which out of 32 possible events can trigger conversion for the regular and injected groups.

A regular group conversion can be interrupted by an injected trigger.

Note: The regular trigger selection cannot be changed on-the-fly. The injected trigger selection can be anticipated and changed on-the-fly. Refer to [Section 28.4.21: Queue of context for injected conversions on page 1224](#).

Figure 249. Triggers shared between ADC master and slave



Refer to Table ADC interconnection in [Section 28.4.2: ADC pins and internal signals](#) for the list of all the external triggers that can be used for regular conversion.

28.4.19 Injected channel management

Triggered injection mode

To use triggered injection, the JAUTO bit in the ADC_CFGR register must be cleared.

1. Start the conversion of a group of regular channels either by an external trigger or by setting the ADSTART bit in the ADC_CR register.
2. If an external injected trigger occurs, or if the JADSTART bit in the ADC_CR register is set during the conversion of a regular group of channels, the current conversion is reset and the injected channel sequence switches are launched (all the injected channels are converted once).
3. Then, the regular conversion of the regular group of channels is resumed from the last interrupted regular conversion.
4. If a regular event occurs during an injected conversion, the injected conversion is not interrupted but the regular sequence is executed at the end of the injected sequence.

[Figure 250](#) shows the corresponding timing diagram.

Note: When using triggered injection, one must ensure that the interval between trigger events is longer than the injection sequence. For instance, if the sequence length is 30 ADC clock cycles (that is two conversions with a sampling time of 2.5 clock periods), the minimum interval between triggers must be 31 ADC clock cycles.

Auto-injection mode

If the JAUTO bit in the ADC_CFGR register is set, then the channels in the injected group are automatically converted after the regular group of channels. This can be used to convert a sequence of up to 20 conversions programmed in the ADC_SQRy and ADC_JSQR registers.

In this mode, the ADSTART bit in the ADC_CR register must be set to start regular conversions, followed by injected conversions (JADSTART must be kept cleared). Setting the ADSTP bit aborts both regular and injected conversions (JADSTP bit must not be used).

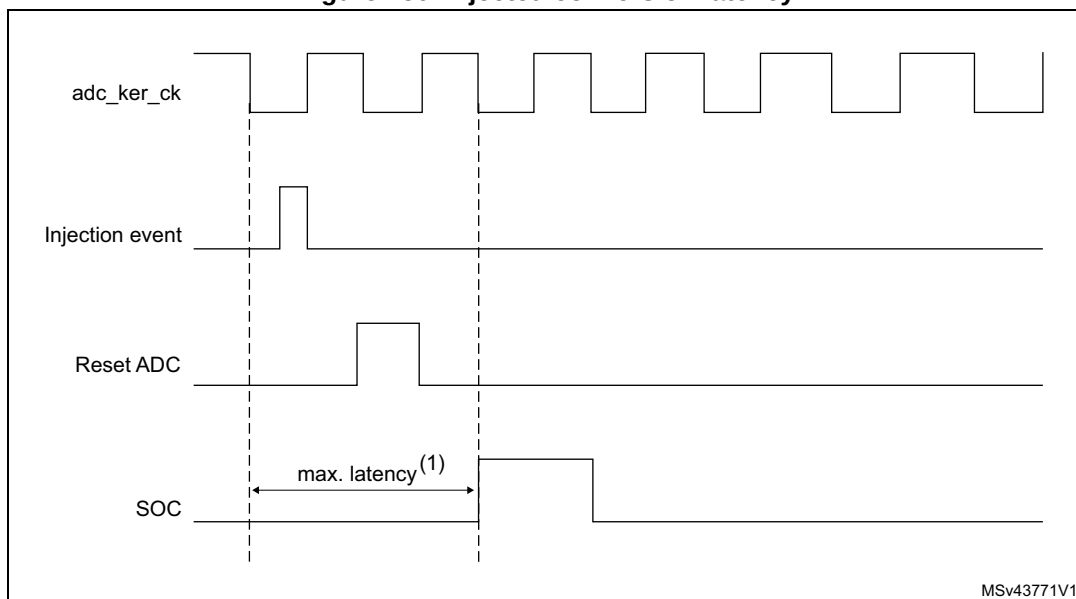
In this mode, external trigger on injected channels must be disabled.

If the CONT bit is also set in addition to the JAUTO bit, regular channels followed by injected channels are continuously converted.

Note: It is not possible to use both the auto-injected and discontinuous modes simultaneously.

When the DMA is used for exporting regular sequencer's data in JAUTO mode, it is necessary to program it in circular mode. If the single-shot mode is selected, the JAUTO sequence is stopped upon DMA Transfer Complete event.

Figure 250. Injected conversion latency



1. The maximum latency value can be found in the electrical characteristics of the device datasheet.

28.4.20 Discontinuous mode (DISCEN, DISCNUM, JDISCEN)

Regular group mode

This mode is enabled by setting the DISCEN bit in the ADC_CFGR register.

It is used to convert a short sequence (subgroup) of n conversions ($n \leq 8$) that is part of the sequence of conversions selected in the ADC_SQRy registers. The value of n is specified by writing to the DISCNUM[2:0] bits in the ADC_CFGR register.

When an external trigger occurs, it starts the next n conversions selected in the ADC_SQRy registers until all the conversions in the sequence are done. The total sequence length is defined by the L[3:0] bits in the ADC_SQR1 register.

Example:

- DISCEN = 1, $n = 3$, channels to be converted = 1, 2, 3, 6, 7, 8, 9, 10, 11
 - 1st trigger: channels converted are 1, 2, 3 (an EOC event is generated at each conversion).
 - 2nd trigger: channels converted are 6, 7, 8 (an EOC event is generated at each conversion).
 - 3rd trigger: channels converted are 9, 10, 11 (an EOC event is generated at each conversion) and an EOS event is generated after the conversion of channel 11.
 - 4th trigger: channels converted are 1, 2, 3 (an EOC event is generated at each conversion).
 - ...
- DISCEN = 0, channels to be converted = 1, 2, 3, 6, 7, 8, 9, 10, 11
 - 1st trigger: the complete sequence is converted: channel 1, then 2, 3, 6, 7, 8, 9, 10 and 11. Each conversion generates an EOC event and the last one also generates an EOS event.
 - All the next trigger events relaunch the complete sequence.

Note: *The channel numbers referred to in the above example might not be available on all microcontrollers.*

When a regular group is converted in discontinuous mode, no rollover occurs (the last subgroup of the sequence can have less than n conversions).

When all subgroups are converted, the next trigger starts the conversion of the first subgroup. In the example above, the 4th trigger reconverts the channels 1, 2 and 3 in the 1st subgroup.

It is not possible to have both discontinuous mode and continuous mode enabled. In this case (if DISCEN = 1, CONT = 1), the ADC behaves as if continuous mode was disabled.

Injected group mode

This mode is enabled by setting the JDISCEN bit in the ADC_CFGR register. It converts the sequence selected in the ADC_JSQR register, channel by channel, after an external injected trigger event. This is equivalent to discontinuous mode for regular channels where 'n' is fixed to 1.

When an external trigger occurs, it starts the next channel conversions selected in the ADC_JSQR registers until all the conversions in the sequence are done. The total sequence length is defined by the JL[1:0] bits in the ADC_JSQR register.

Example:

- JDISCEN = 1, channels to be converted = 1, 2, 3
 - 1st trigger: channel 1 converted (a JEOC event is generated)
 - 2nd trigger: channel 2 converted (a JEOC event is generated)
 - 3rd trigger: channel 3 converted and a JEOC event + a JEOS event are generated
 - ...

Note: *The channel numbers referred to in the above example might not be available on all microcontrollers.*

When all injected channels have been converted, the next trigger starts the conversion of the first injected channel. In the example above, the 4th trigger reconverts the 1st injected channel 1.

It is not possible to use both auto-injected mode and discontinuous mode simultaneously: the bits DISCEN and JDISCEN must be kept cleared by software when JAUTO is set.

28.4.21 Queue of context for injected conversions

A queue of context is implemented to anticipate up to 2 contexts for the next injected sequence of conversions. JQDIS bit of ADC_CFGR register must be reset to enable this feature. Only hardware-triggered conversions are possible when the context queue is enabled.

This context consists of:

- Configuration of the injected triggers (bits JEXTEN[1:0] and JEXTSEL bits in ADC_JSQR register)
- Definition of the injected sequence (bits JSQx[4:0] and JL[1:0] in ADC_JSQR register)

All the parameters of the context are defined into a single register ADC_JSQR and this register implements a queue of 2 buffers, allowing the bufferization of up to 2 sets of parameters:

- The ADC_JSQR register can be written at any moment even when injected conversions are ongoing.
- Each data written into the ADC_JSQR register is stored into the Queue of context.
- At the beginning, the Queue is empty and the first write access into the ADC_JSQR register immediately changes the context and the ADC is ready to receive injected triggers.
- Once an injected sequence is complete, the Queue is consumed and the context changes according to the next ADC_JSQR parameters stored in the Queue. This new context is applied for the next injected sequence of conversions.
- A Queue overflow occurs when writing into register ADC_JSQR while the Queue is full. This overflow is signaled by the assertion of the flag JQOVF. When an overflow occurs, the write access of ADC_JSQR register which has created the overflow is ignored and the queue of context is unchanged. An interrupt can be generated if bit JQOVFIE is set.
- Two possible behaviors are possible when the Queue becomes empty, depending on the value of the control bit JQM of register ADC_CFGR:
 - If JQM = 0, the Queue is empty just after enabling the ADC, but then it can never be empty during run operations: the Queue always maintains the last active context and any further valid start of injected sequence is served according to the last active context.
 - If JQM = 1, the Queue can be empty after the end of an injected sequence or if the Queue is flushed. When this occurs, there is no more context in the queue and hardware triggers are disabled. Therefore, any further hardware injected triggers are ignored until the software re-writes a new injected context into ADC_JSQR register.
- Reading ADC_JSQR register returns the current ADC_JSQR context which is active at that moment. When the ADC_JSQR context is empty, JSQI is read as 0x00.
- The Queue is flushed when stopping injected conversions by setting JADSTP = 1 or when disabling the ADC by setting ADDIS = 1:
 - If JQM = 0, the Queue is maintained with the last active context.
 - If JQM = 1, the Queue becomes empty and triggers are ignored.

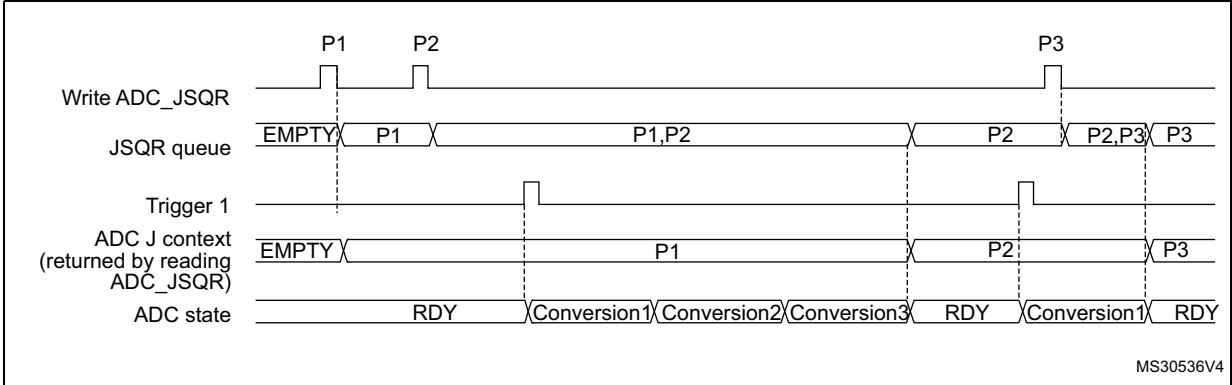
Note: When configured in discontinuous mode (bit JDISCEN = 1), only the last trigger of the injected sequence changes the context and consumes the Queue. The 1st trigger only consumes the queue but others are still valid triggers as shown by the discontinuous mode example below (length = 3 for both contexts):

- 1st trigger, discontinuous. Sequence 1: context 1 consumed, 1st conversion carried out
- 2nd trigger, disc. Sequence 1: 2nd conversion.
- 3rd trigger, discontinuous. Sequence 1: 3rd conversion.
- 4th trigger, discontinuous. Sequence 2: context 2 consumed, 1st conversion carried out.
- 5th trigger, discontinuous. Sequence 2: 2nd conversion.
- 6th trigger, discontinuous. Sequence 2: 3rd conversion.

Behavior when changing the trigger or sequence context

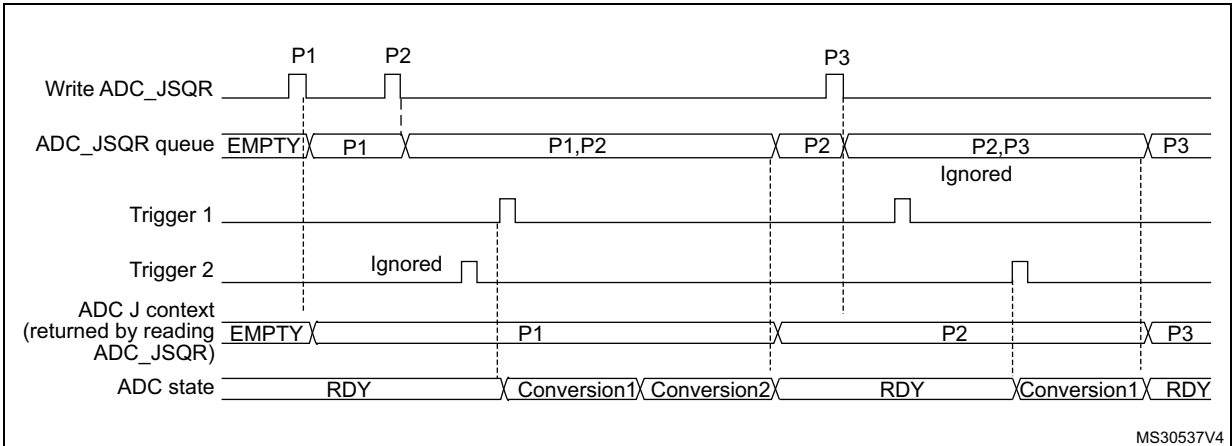
Figure 251 and Figure 252 show the behavior of the context Queue when changing the sequence or the triggers.

Figure 251. Example of ADC_JSQR queue of context (sequence change)



1. Parameters:
- P1: sequence of 3 conversions, hardware trigger 1
 - P2: sequence of 1 conversion, hardware trigger 1
 - P3: sequence of 4 conversions, hardware trigger 1

Figure 252. Example of ADC_JSQR queue of context (trigger change)

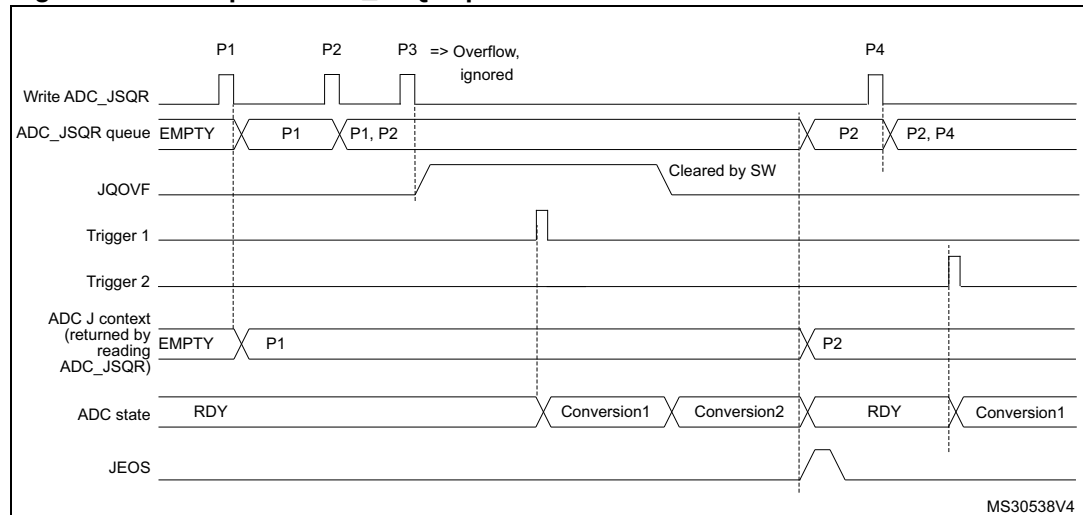


1. Parameters:
- P1: sequence of 2 conversions, hardware trigger 1
 - P2: sequence of 1 conversion, hardware trigger 2
 - P3: sequence of 4 conversions, hardware trigger 1

Queue of context: Behavior when a queue overflow occurs

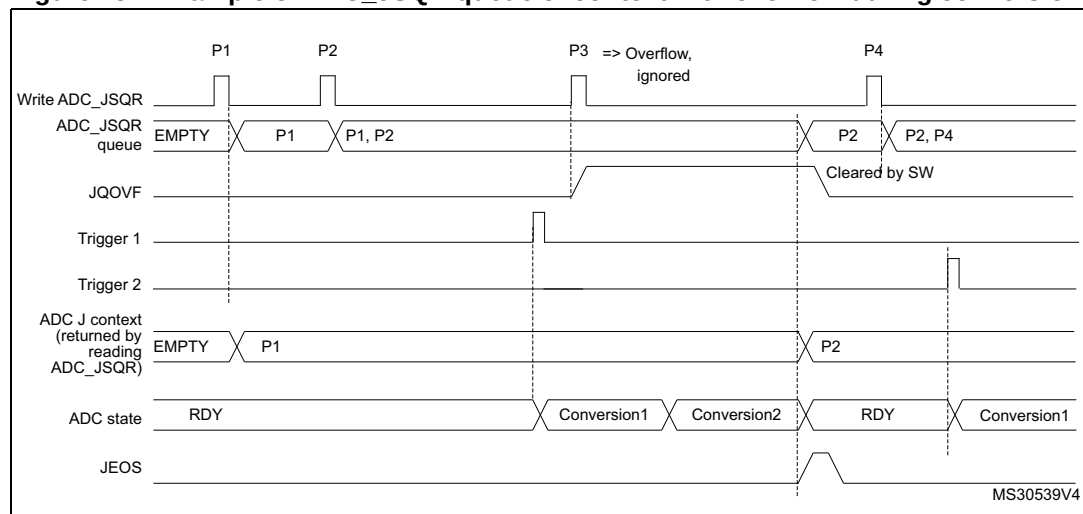
The [Figure 253](#) and [Figure 254](#) show the behavior of the context Queue if an overflow occurs before or during a conversion.

Figure 253. Example of ADC_JSQR queue of context with overflow before conversion



- Parameters:
 - P1: sequence of 2 conversions, hardware trigger 1
 - P2: sequence of 1 conversion, hardware trigger 2
 - P3: sequence of 3 conversions, hardware trigger 1
 - P4: sequence of 4 conversions, hardware trigger 1

Figure 254. Example of ADC_JSQR queue of context with overflow during conversion



- Parameters:
 - P1: sequence of 2 conversions, hardware trigger 1
 - P2: sequence of 1 conversion, hardware trigger 2
 - P3: sequence of 3 conversions, hardware trigger 1
 - P4: sequence of 4 conversions, hardware trigger 1

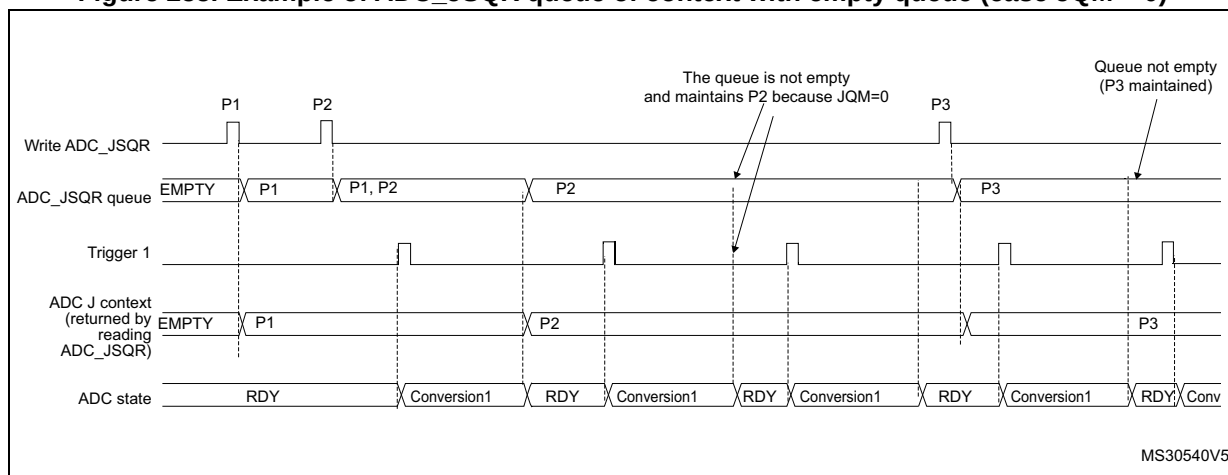
It is recommended to manage the queue overflows as described below:

- After each P context write into ADC_JSQR register, flag JQOVF shows if the write has been ignored or not (an interrupt can be generated).
- Avoid Queue overflows by writing the third context (P3) only once the flag JEOS of the previous context P2 has been set. This ensures that the previous context has been consumed and that the queue is not full.

Queue of context: Behavior when the queue becomes empty

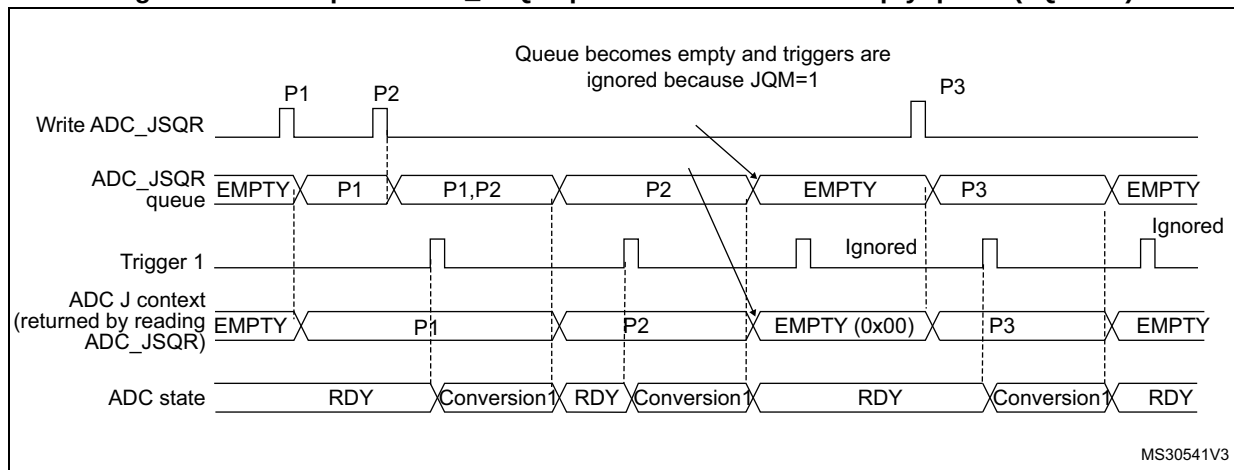
Figure 255 and Figure 256 show the behavior of the context Queue when the Queue becomes empty in both cases JQM = 0 or 1.

Figure 255. Example of ADC_JSQR queue of context with empty queue (case JQM = 0)



- Parameters:
 - P1: sequence of 1 conversion, hardware trigger 1
 - P2: sequence of 1 conversion, hardware trigger 1
 - P3: sequence of 1 conversion, hardware trigger 1

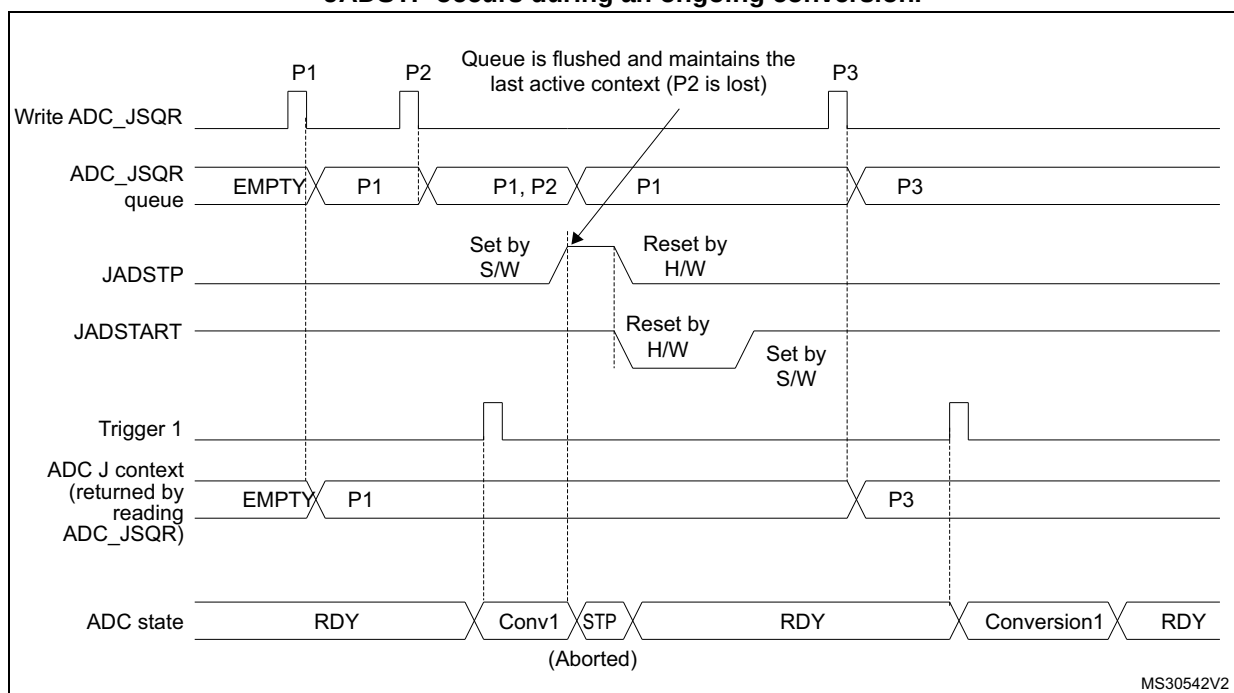
Note: When writing P3, the context changes immediately. However, because of internal resynchronization, there is a latency and if a trigger occurs just after or before writing P3, it can happen that the conversion is launched considering the context P2. To avoid this situation, the user must ensure that there is no ADC trigger happening when writing a new context that applies immediately.

Figure 256. Example of ADC_JSQR queue of context with empty queue (JQM = 1)

- Parameters:
 P1: sequence of 1 conversion, hardware trigger 1
 P2: sequence of 1 conversion, hardware trigger 1
 P3: sequence of 1 conversion, hardware trigger 1

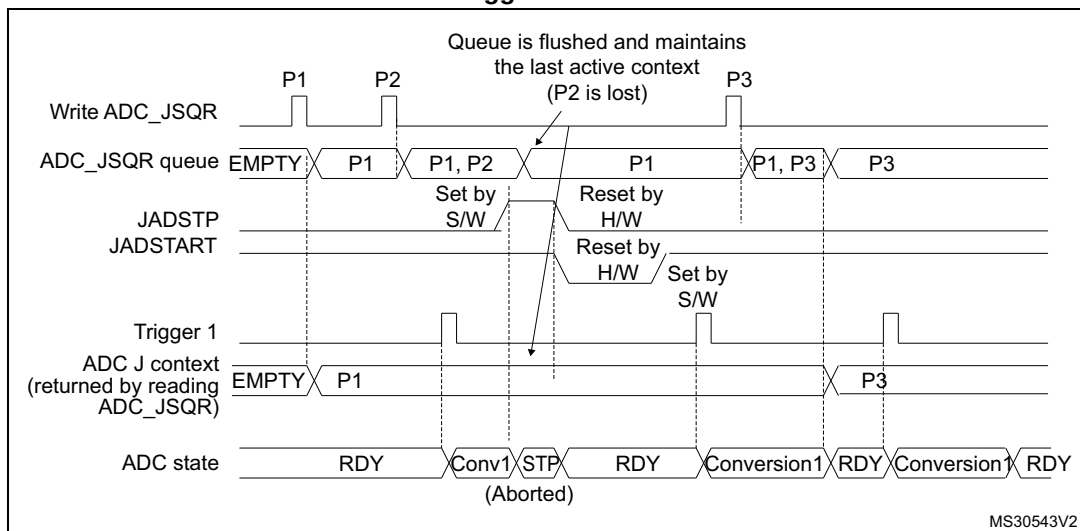
Flushing the queue of context

The figures below show the behavior of the context Queue in various situations when the queue is flushed.

Figure 257. Flushing ADC_JSQR queue of context by setting JADSTP = 1 (JQM = 0) - JADSTP occurs during an ongoing conversion.

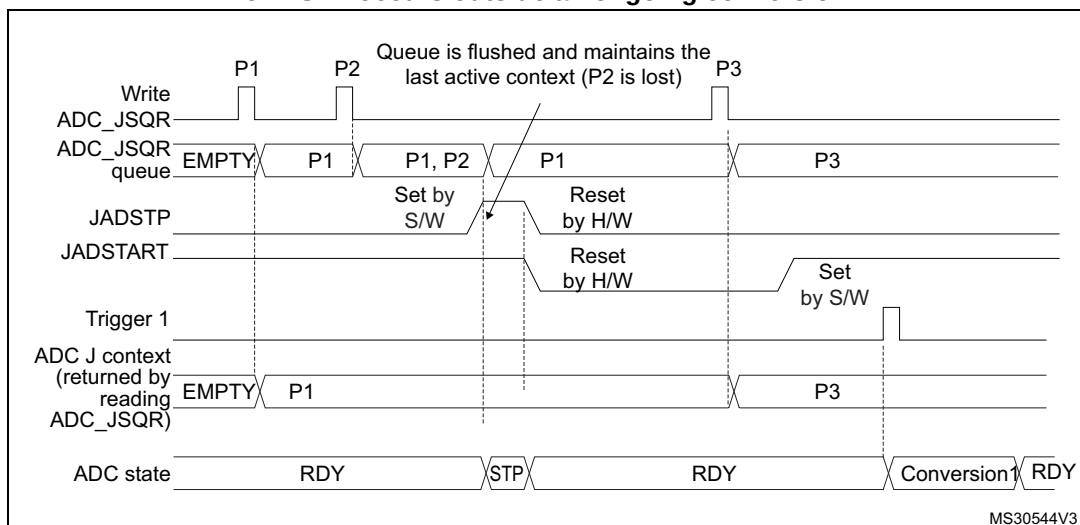
- Parameters:
 P1: sequence of 1 conversion, hardware trigger 1
 P2: sequence of 1 conversion, hardware trigger 1
 P3: sequence of 1 conversion, hardware trigger 1

Figure 258. Flushing ADC_JSQR queue of context by setting JADSTP = 1 (JQM = 0) - JADSTP occurs during an ongoing conversion and a new trigger occurs

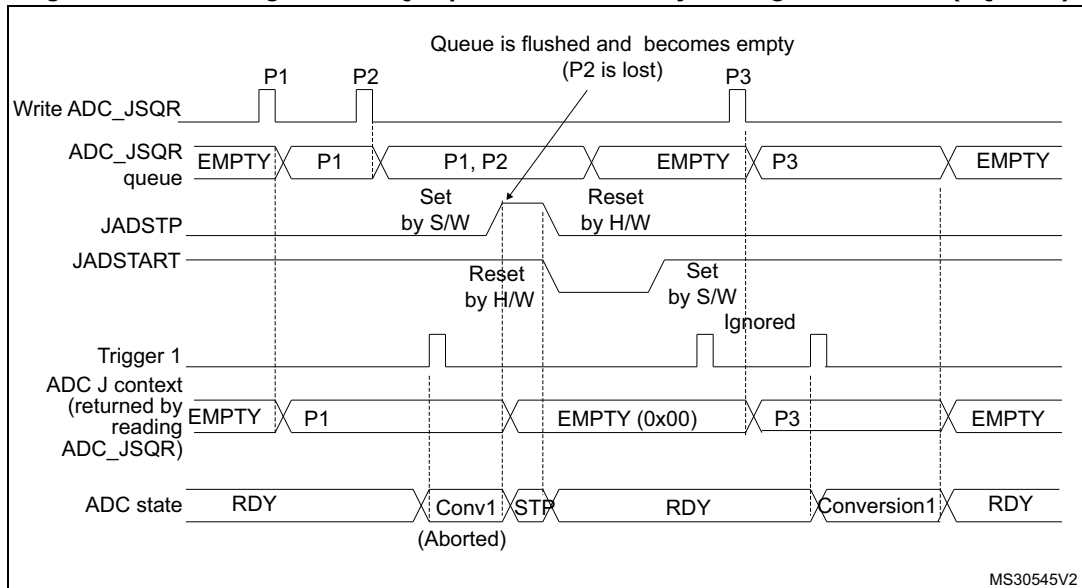


- Parameters:
 P1: sequence of 1 conversion, hardware trigger 1
 P2: sequence of 1 conversion, hardware trigger 1
 P3: sequence of 1 conversion, hardware trigger 1

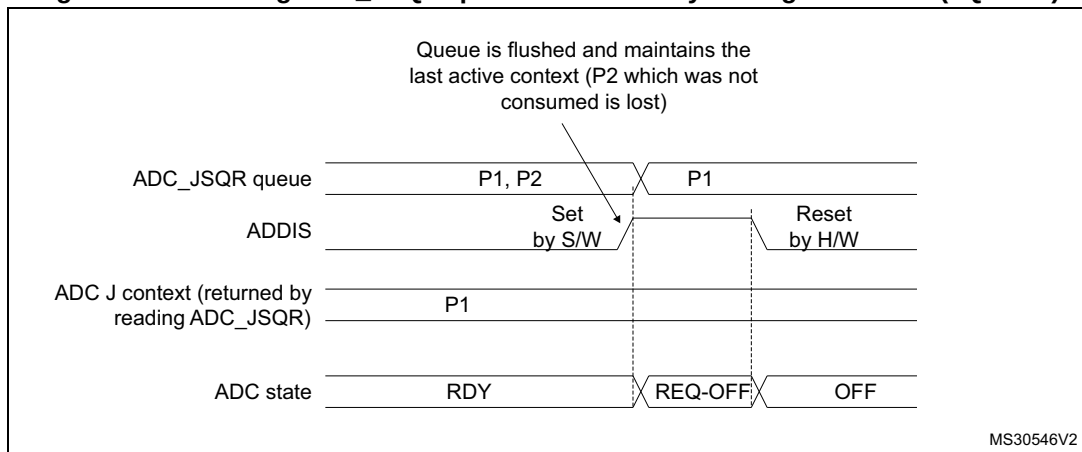
Figure 259. Flushing ADC_JSQR queue of context by setting JADSTP = 1 (JQM = 0) - JADSTP occurs outside an ongoing conversion



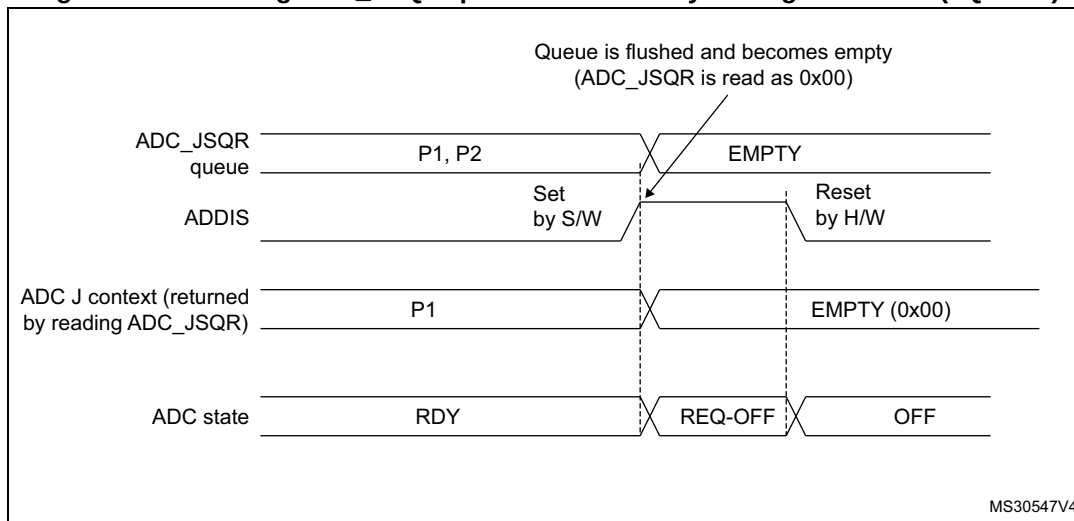
- Parameters:
 P1: sequence of 1 conversion, hardware trigger 1
 P2: sequence of 1 conversion, hardware trigger 1
 P3: sequence of 1 conversion, hardware trigger 1

Figure 260. Flushing ADC_JSQR queue of context by setting JADSTP = 1 (JQM = 1)

- Parameters:
 - P1: sequence of 1 conversion, hardware trigger 1
 - P2: sequence of 1 conversion, hardware trigger 1
 - P3: sequence of 1 conversion, hardware trigger 1

Figure 261. Flushing ADC_JSQR queue of context by setting ADDIS = 1 (JQM = 0)

- Parameters:
 - P1: sequence of 1 conversion, hardware trigger 1
 - P2: sequence of 1 conversion, hardware trigger 1
 - P3: sequence of 1 conversion, hardware trigger 1

Figure 262. Flushing ADC_JSQR queue of context by setting ADDIS = 1 (JQM = 1)

1. Parameters:
 P1: sequence of 1 conversion, hardware trigger 1
 P2: sequence of 1 conversion, hardware trigger 1
 P3: sequence of 1 conversion, hardware trigger 1

Queue of context: Starting the ADC with an empty queue

The following procedure must be followed to start ADC operation with an empty queue, in case the first context is not known at the time the ADC is initialized. This procedure is only applicable when JQM bit is reset:

5. Write a dummy ADC_JSQR with JEXTEN not equal to 0 (otherwise triggering a software conversion)
6. Set JADSTART
7. Set JADSTP
8. Wait until JADSTART is reset
9. Set JADSTART.

Disabling the queue

It is possible to disable the queue by setting bit JQDIS = 1 into the ADC_CFGR register.

28.4.22 Programmable resolution (RES) - fast conversion mode

It is possible to perform faster conversion by reducing the ADC resolution.

The resolution can be configured to be either 12, 10, 8, or 6 bits by programming the control bits RES[1:0]. [Figure 267](#), [Figure 268](#), [Figure 269](#) and [Figure 270](#) show the conversion result format with respect to the resolution as well as to the data alignment.

Lower resolution allows faster conversion time for applications where high-data precision is not required. It reduces the conversion time spent by the successive approximation steps according to [Table 285](#).

Table 285. T_{SAR} timings depending on resolution

RES (bits)	T_{SAR} (ADC clock cycles)	T_{SAR} (ns) at $F_{ADC} = 30$ MHz	T_{CONV} (ADC clock cycles) (with Sampling Time = 2.5 ADC clock cycles)	T_{CONV} (ns) at $F_{ADC} = 30$ MHz
12	12.5 ADC clock cycles	416.67 ns	15 ADC clock cycles	500.0 ns
10	10.5 ADC clock cycles	350.0 ns	13 ADC clock cycles	433.33 ns
8	8.5 ADC clock cycles	203.33 ns	11 ADC clock cycles	366.67 ns
6	6.5 ADC clock cycles	216.67 ns	9 ADC clock cycles	300.0 ns

28.4.23 End of conversion, end of sampling phase (EOC, JEOC, EOSMP)

The ADC notifies the application for each end of regular conversion (EOC) event and each injected conversion (JEOC) event.

The ADC sets the EOC flag as soon as a new regular conversion data is available in the ADC_DR register. An interrupt can be generated if bit EOCIE is set. EOC flag is cleared by the software either by writing 1 to it or by reading ADC_DR.

The ADC sets the JEOC flag as soon as a new injected conversion data is available in one of the ADC_JDRy register. An interrupt can be generated if bit JEOCIE is set. JEOC flag is cleared by the software either by writing 1 to it or by reading the corresponding ADC_JDRy register.

The ADC also notifies the end of Sampling phase by setting the status bit EOSMP (for regular conversions only). EOSMP flag is cleared by software by writing 1 to it. An interrupt can be generated if bit EOSMPIE is set.

28.4.24 End of conversion sequence (EOS, JEOS)

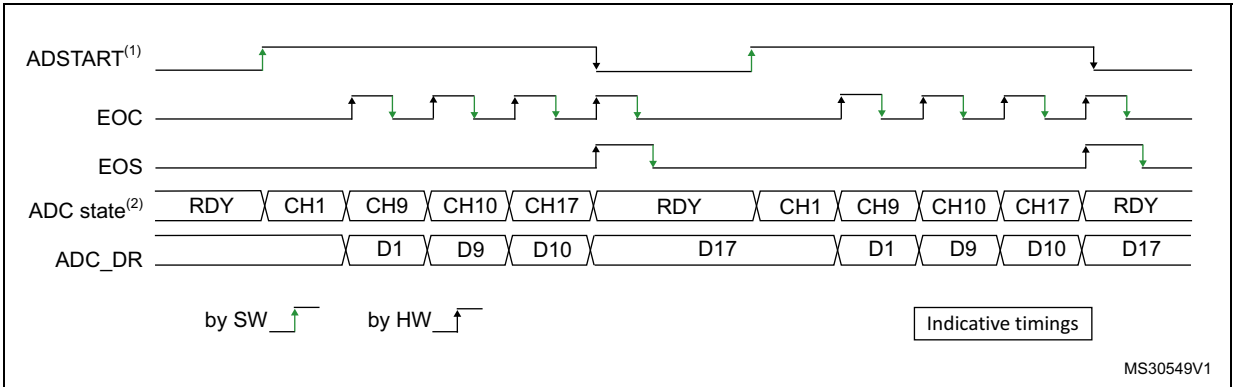
The ADC notifies the application for each end of regular sequence (EOS) and for each end of injected sequence (JEOS) event.

The ADC sets the EOS flag as soon as the last data of the regular conversion sequence is available in the ADC_DR register. An interrupt can be generated if bit EOSIE is set. EOS flag is cleared by the software either by writing 1 to it.

The ADC sets the JEOS flag as soon as the last data of the injected conversion sequence is complete. An interrupt can be generated if bit JEOSIE is set. JEOS flag is cleared by the software either by writing 1 to it.

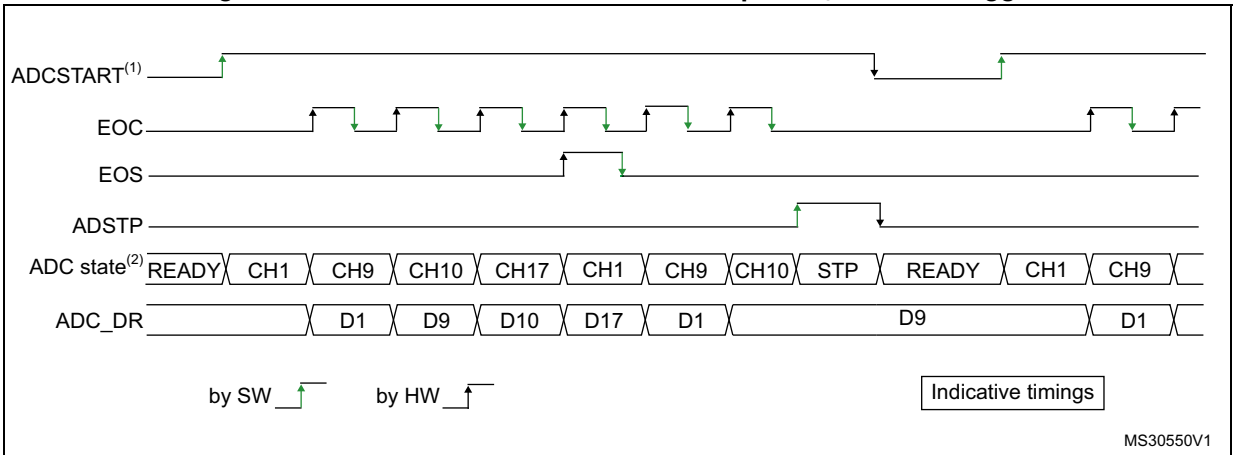
28.4.25 Timing diagrams example (single/continuous modes, hardware/software triggers)

Figure 263. Single conversions of a sequence, software trigger



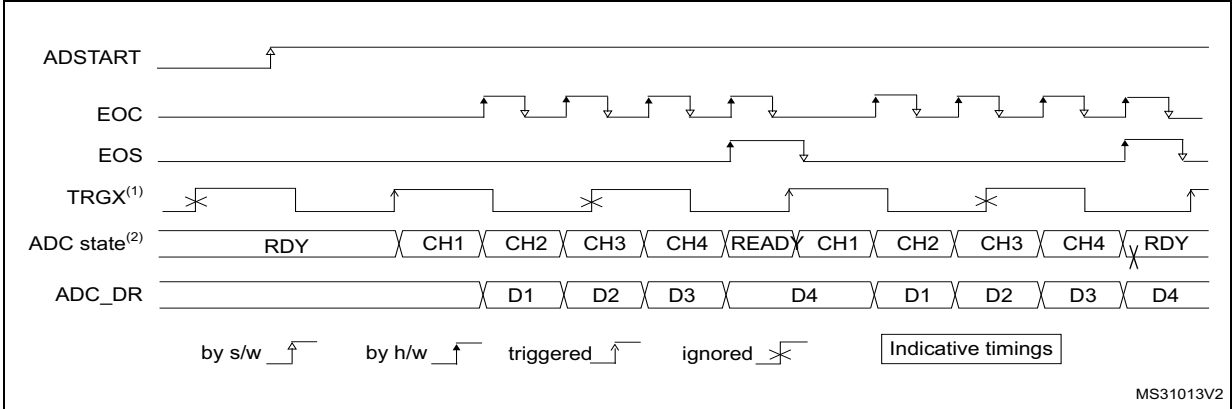
1. EXTEN = 0x0, CONT = 0
2. Channels selected = 1,9, 10, 17; AUTDLY = 0.

Figure 264. Continuous conversion of a sequence, software trigger



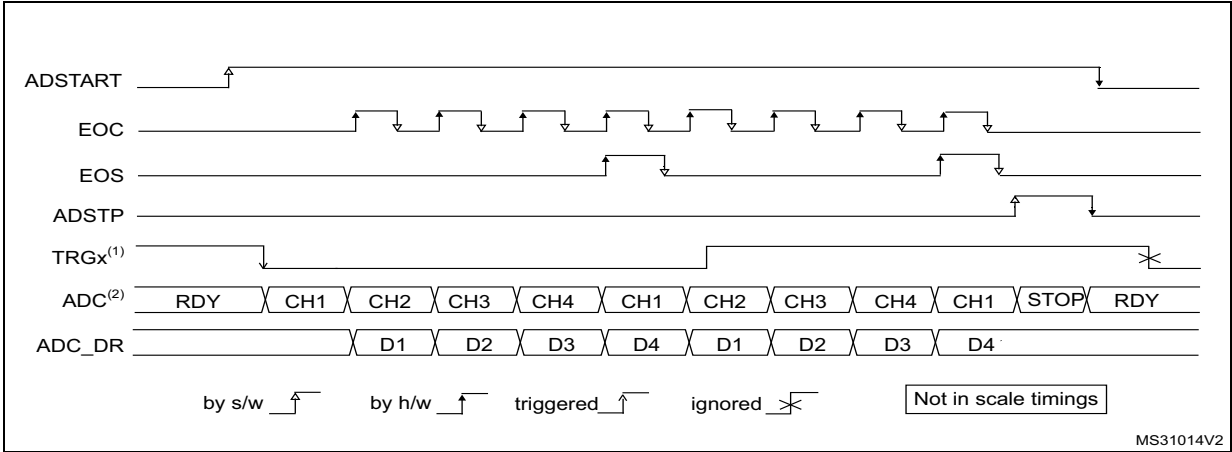
1. EXTEN = 0x0, CONT = 1
2. Channels selected = 1,9, 10, 17; AUTDLY = 0.

Figure 265. Single conversions of a sequence, hardware trigger



1. TRGX (over-frequency) is selected as trigger source, EXTEN = 01, CONT = 0
2. Channels selected = 1, 2, 3, 4; AUTDLY = 0.

Figure 266. Continuous conversions of a sequence, hardware trigger



1. TRGX is selected as trigger source, EXTEN = 10, CONT = 1
2. Channels selected = 1, 2, 3, 4; AUTDLY = 0.

28.4.26 Data management

Data register, data alignment and offset (ADC_DR, OFFSET, OFFSET_CH, ALIGN)

Data and alignment

At the end of each regular conversion channel (when EOC event occurs), the result of the converted data is stored into the ADC_DR data register which is 16 bits wide.

At the end of each injected conversion channel (when JEOC event occurs), the result of the converted data is stored into the corresponding ADC_JDRy data register which is 16 bits wide.

The ALIGN bit in the ADC_CFGR register selects the alignment of the data stored after conversion. Data can be right- or left-aligned as shown in [Figure 267](#), [Figure 268](#), [Figure 269](#) and [Figure 270](#).

Special case: when left-aligned, the data are aligned on a half-word basis except when the resolution is set to 6-bit. In that case, the data are aligned on a byte basis as shown in [Figure 269](#) and [Figure 270](#).

Note: Left-alignment is not supported in oversampling mode. When ROVSE and/or JOVSE bit is set, the ALIGN bit value is ignored and the ADC only provides right-aligned data.

Offset

An offset y ($y = 1, 2, 3, 4$) can be applied to a channel by setting the bit OFFSET_EN = 1 into ADC_OFRy register. The channel to which the offset is to be applied is programmed into the bits OFFSET_CH[4:0] of ADC_OFRy register. In this case, the converted value is decreased by the user-defined offset written in the bits OFFSET[11:0]. The result may be a negative value so the read data is signed and the SEXT bit represents the extended sign value.

Note: Offset correction is not supported in oversampling mode. When ROVSE and/or JOVSE bit is set, the value of the OFFSET_EN bit in ADC_OFRy register is ignored (considered as reset).

[Table 288](#) describes how the comparison is performed for all the possible resolutions for analog watchdog 1.

Table 286. Offset computation versus data resolution

Resolution (bits RES[1:0])	Subtraction between raw converted data and offset		Result	Comments
	Raw converted Data, left aligned	Offset		
00: 12-bit	DATA[11:0]	OFFSET[11:0]	Signed 12-bit data	-
01: 10-bit	DATA[11:2], 00	OFFSET[11:0]	Signed 10-bit data	The user must configure OFFSET[1:0] to "00"

Table 286. Offset computation versus data resolution (continued)

Resolution (bits RES[1:0])	Subtraction between raw converted data and offset		Result	Comments
	Raw converted Data, left aligned	Offset		
10: 8-bit	DATA[11:4],00 00	OFFSET[11:0]	Signed 8-bit data	The user must configure OFFSET[3:0] to "0000"
11: 6-bit	DATA[11:6],00 0000	OFFSET[11:0]	Signed 6-bit data	The user must configure OFFSET[5:0] to "000000"

When reading data from ADC_DR (regular channel) or from ADC_JDRy (injected channel, y = 1,2,3,4) corresponding to the channel "i":

- If one of the offsets is enabled (bit OFFSET_EN = 1) for the corresponding channel, the read data is signed.
- If none of the four offsets is enabled for this channel, the read data is not signed.

[Figure 267](#), [Figure 268](#), [Figure 269](#) and [Figure 270](#) show alignments for signed and unsigned data.

Figure 267. Right alignment (offset disabled, unsigned value)

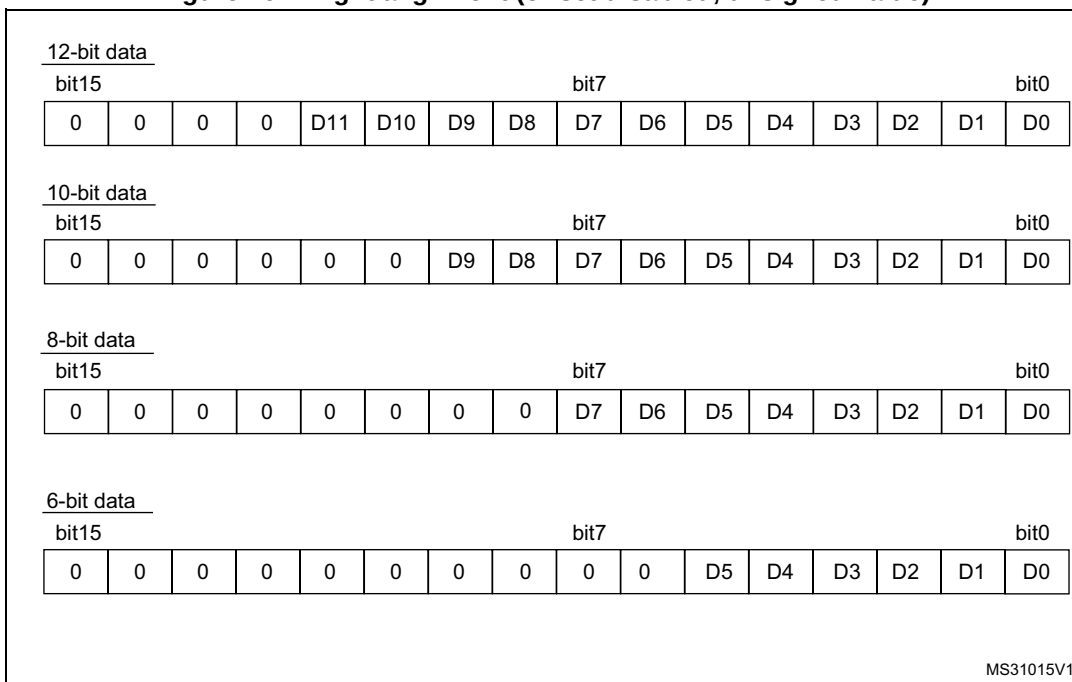


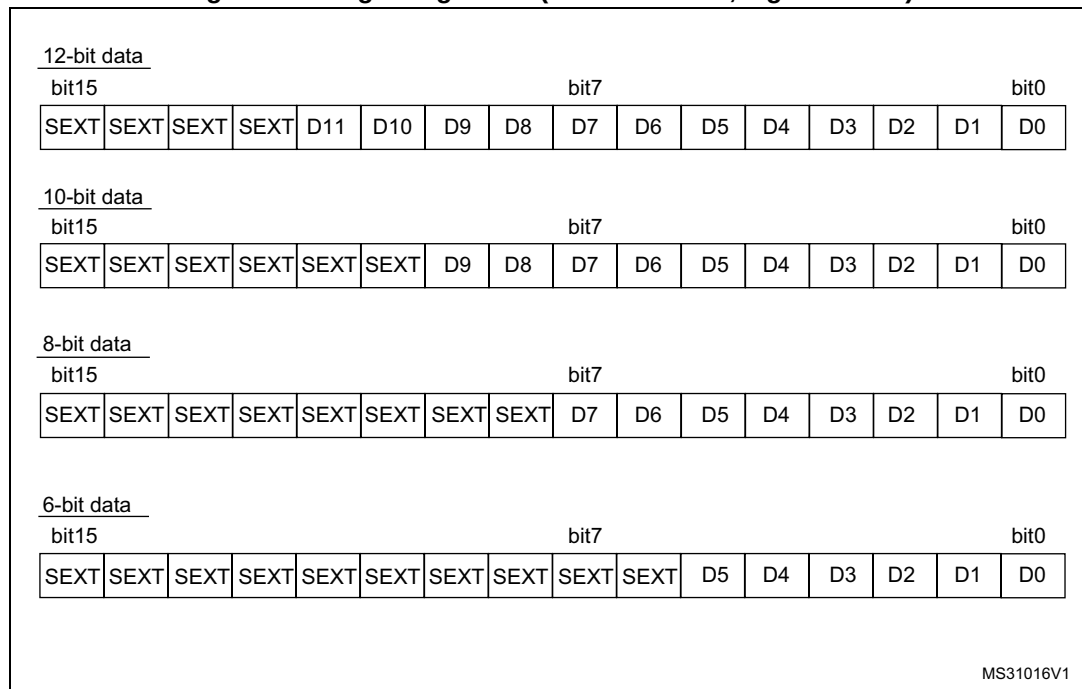
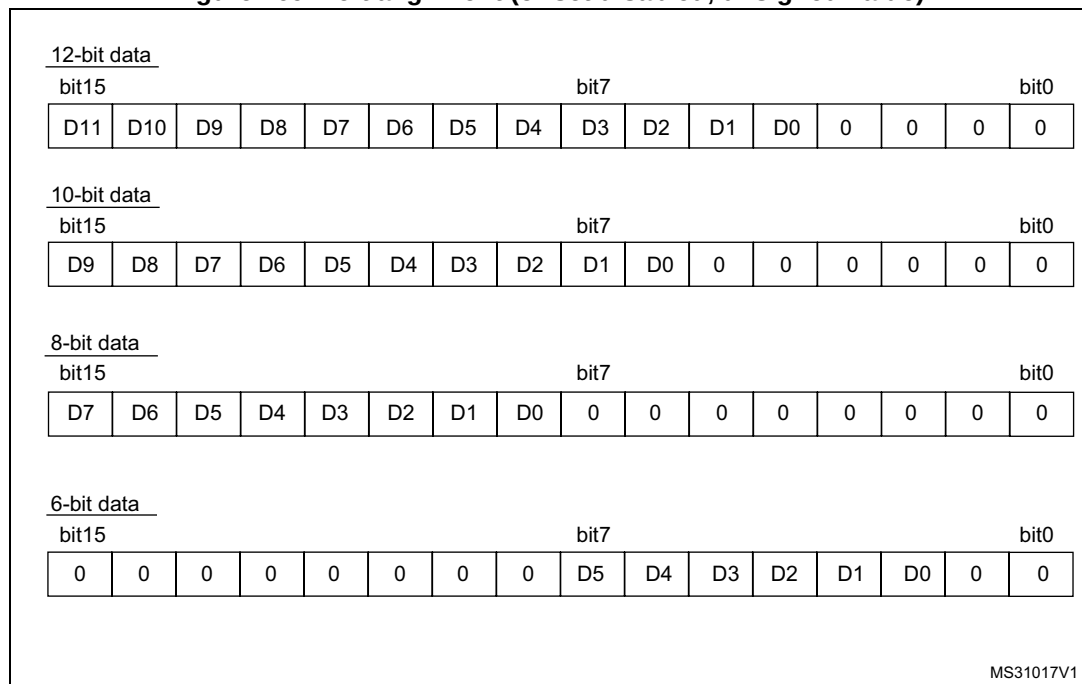
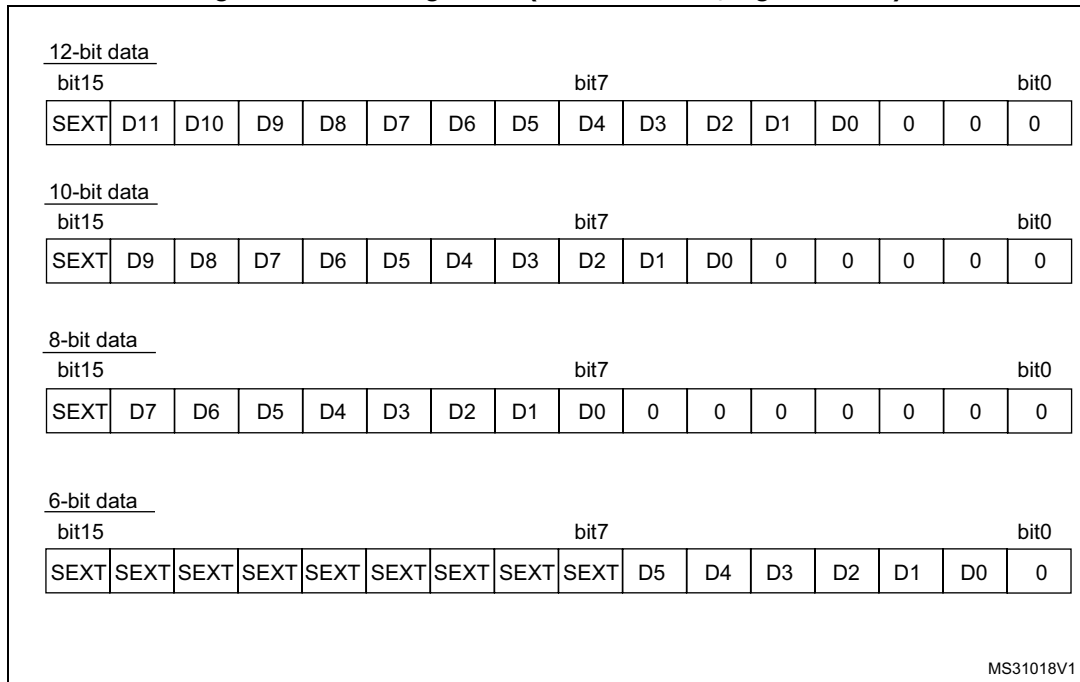
Figure 268. Right alignment (offset enabled, signed value)**Figure 269. Left alignment (offset disabled, unsigned value)**

Figure 270. Left alignment (offset enabled, signed value)

Offset compensation

When SATEN bit is set in ADC_OFRy register during offset operation, data are unsigned. All the offset data saturate at 0x000 (in 12-bit mode). When OFFSETPOS bit is set, the offset direction is positive and the data saturate at 0xFFF (in 12-bit mode). In 8-bit mode, data saturate at 0x00 and 0xFF, respectively.

The analog watchdog comparison is performed before the offset compensation.

ADC overrun (OVR, OVRMOD)

The overrun flag (OVR) notifies when the regular converted data has not been read (by the CPU or the DMA) before ADC_DR FIFO (three stages) is overflowed.

The OVR flag is set when a new conversion completes while ADC_CR register FIFO was full. An interrupt is generated if OVRIE bit is set to 1.

When an overrun condition occurs, the ADC is still operating and can continue converting unless the software decides to stop and reset the sequence by setting ADSTP to 1. Since ADC_DR FIFO features three stages, up to three data are stored in the FIFO.

OVR flag is cleared by software by writing 1 to it.

It is possible to configure if data is preserved or overwritten when an overrun event occurs by programming the control bit OVRMOD:

- **OVRMOD = 0:** The overrun event preserves the data register from being overwritten: the old data is maintained up to ADC_DR FIFO depth (three stages) and the new conversion is discarded and lost. In this mode, ADC_DR FIFO is disabled. If the FIFO is full, any further conversion is performed but the resulting data is also discarded. EOC

is cleared by reading ADC_DR register. However, the FIFO can still contain previously converted data.

- OVRMOD = 1: The data register is overwritten with the last conversion result and the previous unread data is lost. In this mode, ADC_DR FIFO is disabled. If OVR remains at 1, any further conversions is performed normally and the ADC_DR register always contains the latest converted data.

Figure 271. Example of overrun (OVRMOD = 0)

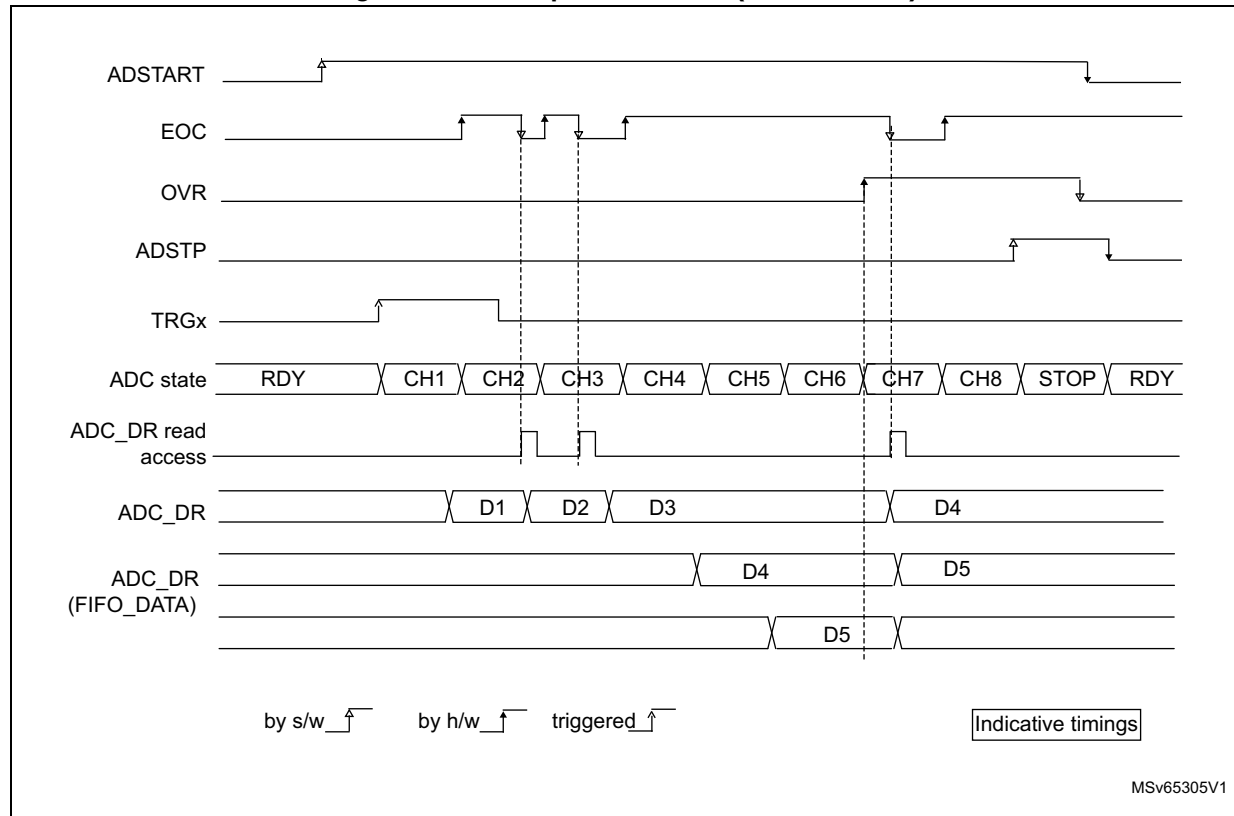
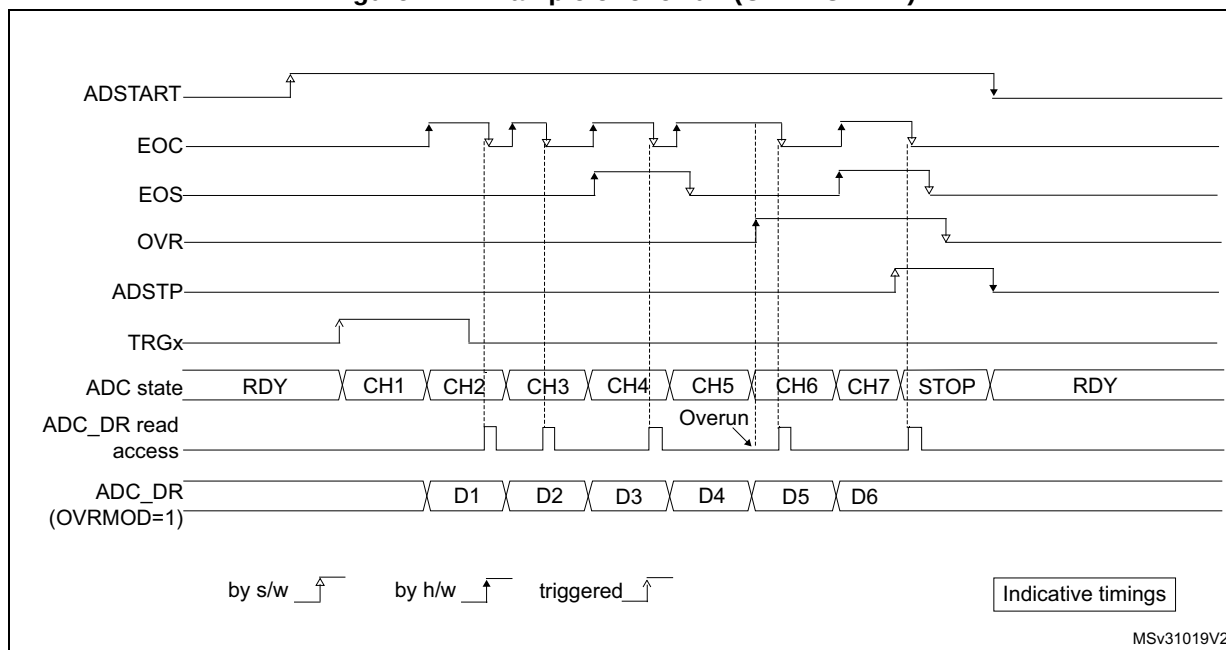


Figure 272. Example of overrun (OVRMOD = 1)



Note: There is no overrun detection on the injected channels since there is a dedicated data register for each of the four injected channels.

Managing a sequence of conversions without using the DMA

If the conversions are slow enough, the conversion sequence can be handled by the software. In this case the software must use the EOC flag and its associated interrupt to handle each data. Each time a conversion is complete, EOC is set and the ADC_DR register can be read. OVRMOD must be configured to 0 to manage overrun events or FIFO overflow as an error.

Managing conversions without using the DMA and without overrun

It may be useful to let the ADC convert one or more channels without reading the data each time (if there is an analog watchdog for instance). In this case, the OVRMOD bit must be configured to 1 and OVR flag should be ignored by the software. An overrun event does not prevent the ADC from continuing to convert and the ADC_DR register always contains the latest conversion.

Managing conversions using the DMA

Since converted channel values are stored into a unique data register, it is useful to use DMA for conversion of more than one channel. This avoids the loss of the data already stored in the ADC_DR register.

When the DMA mode is enabled (DMAEN bit set to 1 in the ADC_CFGR register in single ADC mode or MDMA different from 0b00 in dual ADC mode), a DMA request is generated after each conversion of a channel. This allows the transfer of the converted data from the ADC_DR register to the destination location selected by the software.

Despite this, if an overrun occurs ($OVR = 1$) because the DMA could not serve the DMA transfer request in time, the ADC stops generating DMA requests and the data corresponding to the new conversion is not transferred by the DMA. Which means that all the data transferred to the RAM can be considered as valid.

Depending on the configuration of $OVRMOD$ bit, the data is either preserved or overwritten (refer to [Section : ADC overrun \(OVR, OVRMOD\)](#)).

The DMA transfer requests are blocked until the software clears the OVR bit.

Two different DMA modes are proposed depending on the application use and are configured with bit $DMACFG$ of the ADC_CFGR register in single ADC mode, or with bit $DMACFG$ of the ADC_CCR register in dual ADC mode:

- DMA one shot mode ($DMACFG = 0$).
This mode is suitable when the DMA is programmed to transfer a fixed number of data.
- DMA circular mode ($DMACFG = 1$)
This mode is suitable when programming the DMA in circular mode.

DMA one shot mode ($DMACFG = 0$)

In this mode, the ADC generates a DMA transfer request each time a new conversion data is available and stops generating DMA requests once the DMA has reached the last DMA transfer (when a transfer complete interrupt occurs - refer to DMA section) even if a conversion has been started again.

When the DMA transfer is complete (all the transfers configured in the DMA controller have been done):

- The content of the ADC data register is frozen.
- Any ongoing conversion is aborted with partial result discarded.
- No new DMA request is issued to the DMA controller. This avoids generating an overrun error if there are still conversions which are started.
- Scan sequence is stopped and reset.
- The DMA is stopped.

DMA circular mode ($DMACFG = 1$)

In this mode, the ADC generates a DMA transfer request each time a new conversion data is available in the data register, even if the DMA has reached the last DMA transfer. This allows configuring the DMA in circular mode to handle a continuous analog input data stream.

28.4.27 Dynamic low-power features

Auto-delayed conversion mode (AUTDLY)

The ADC implements an auto-delayed conversion mode controlled by the $AUTDLY$ configuration bit. Auto-delayed conversions are useful to simplify the software as well as to optimize performance of an application clocked at low frequency where there would be risk of encountering an ADC overrun.

When AUTDLY = 1, a new conversion can start only if all the previous data of the same group has been treated:

- For a regular conversion: once the ADC_DR register has been read or if the EOC bit has been cleared (see [Figure 273](#)).
- For an injected conversion: when the JEOS bit has been cleared (see [Figure 274](#)).

This is a way to automatically adapt the speed of the ADC to the speed of the system which reads the data.

The delay is inserted after each regular conversion (whatever DISCEN = 0 or 1) and after each sequence of injected conversions (whatever JDISCEN = 0 or 1).

Note: *There is no delay inserted between each conversions of the injected sequence, except after the last one.*

During a conversion, a hardware trigger event (for the same group of conversions) occurring during this delay is ignored.

Note: *This is not true for software triggers where it remains possible during this delay to set the bits ADSTART or JADSTART to restart a conversion: it is up to the software to read the data before launching a new conversion.*

No delay is inserted between conversions of different groups (a regular conversion followed by an injected conversion or conversely):

- If an injected trigger occurs during the automatic delay of a regular conversion, the injected conversion starts immediately (see [Figure 274](#)).
- Once the injected sequence is complete, the ADC waits for the delay (if not ended) of the previous regular conversion before launching a new regular conversion (see [Figure 276](#)).

The behavior is slightly different in auto-injected mode (JAUTO = 1) where a new regular conversion can start only when the automatic delay of the previous injected sequence of conversion has ended (when JEOS has been cleared). This is to ensure that the software can read all the data of a given sequence before starting a new sequence (see [Figure 277](#)).

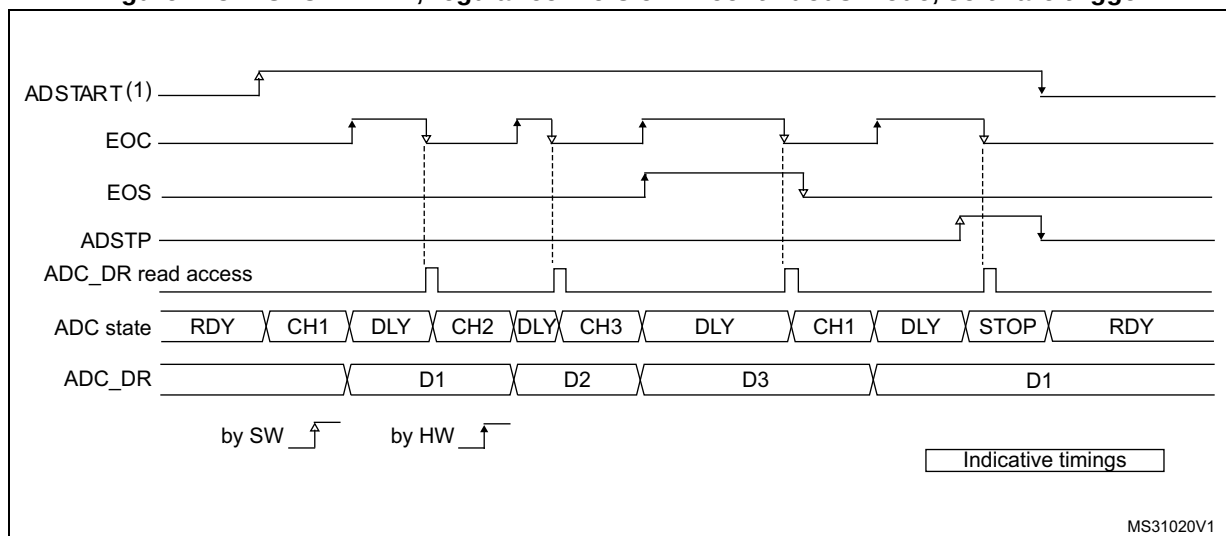
To stop a conversion in continuous auto-injection mode combined with autodelay mode (JAUTO = 1, CONT = 1 and AUTDLY = 1), follow the following procedure:

1. Wait until JEOS = 1 (no more conversions are restarted)
2. Clear JEOS,
3. Set ADSTP = 1
4. Read the regular data.

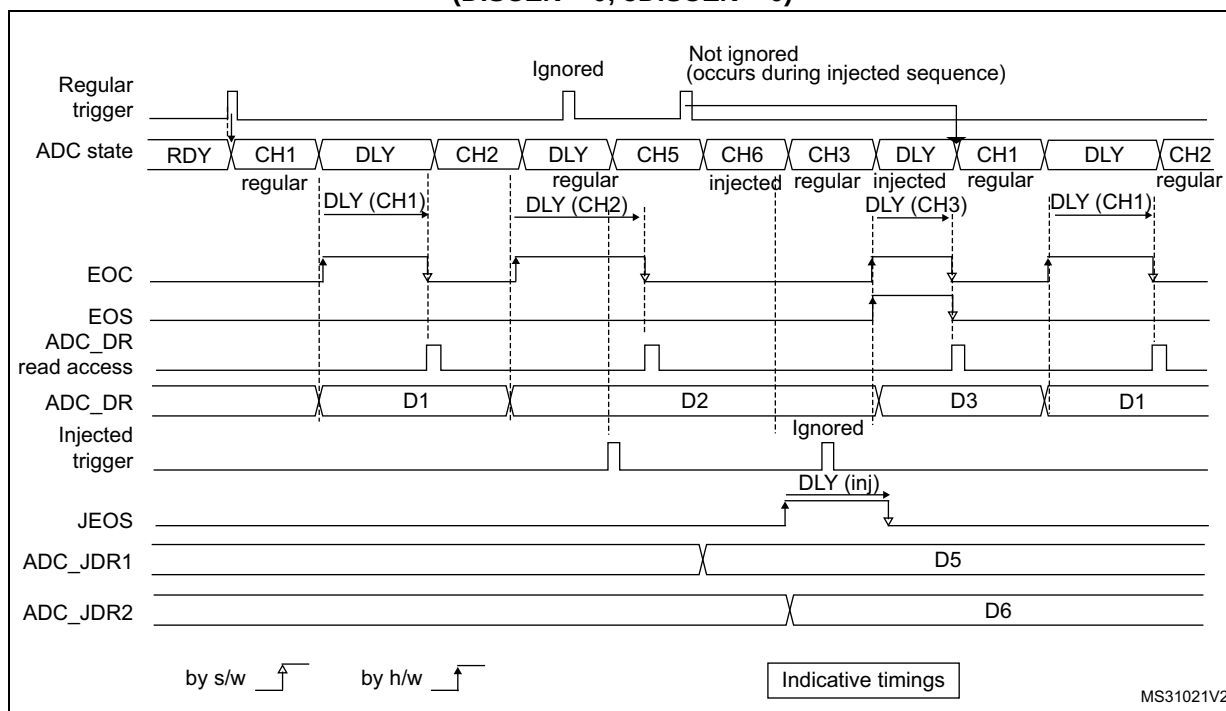
If this procedure is not respected, a new regular sequence can restart if JEOS is cleared after ADSTP has been set.

In AUTDLY mode, a hardware regular trigger event is ignored if it occurs during an already ongoing regular sequence or during the delay that follows the last regular conversion of the sequence. It is however considered pending if it occurs after this delay, even if it occurs during an injected sequence of the delay that follows it. The conversion then starts at the end of the delay of the injected sequence.

In AUTDLY mode, a hardware injected trigger event is ignored if it occurs during an already ongoing injected sequence or during the delay that follows the last injected conversion of the sequence.

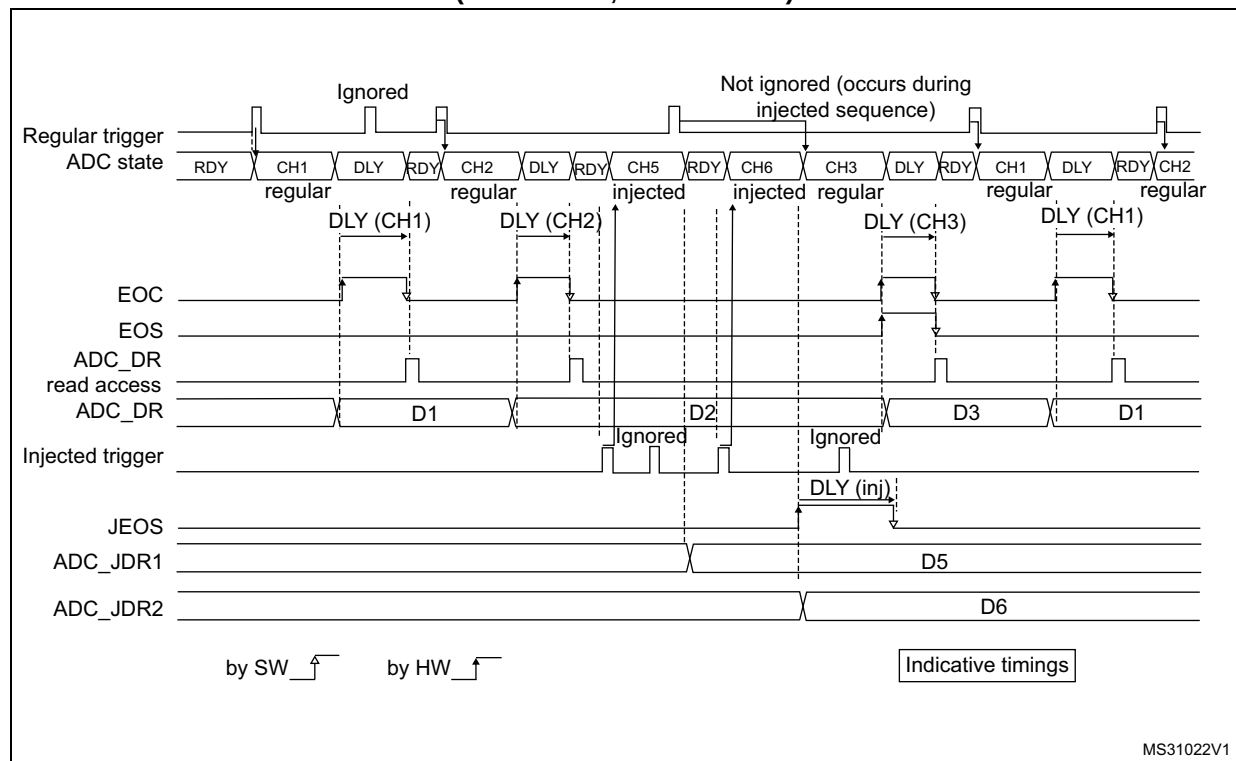
Figure 273. AUTODLY = 1, regular conversion in continuous mode, software trigger

1. AUTDLY = 1
2. Regular configuration: EXTEN=0x0 (SW trigger), CONT = 1, CHANNELS = 1,2,3
3. Injected configuration DISABLED

Figure 274. AUTODLY = 1, regular HW conversions interrupted by injected conversions (DISCEN = 0; JDISCEN = 0)

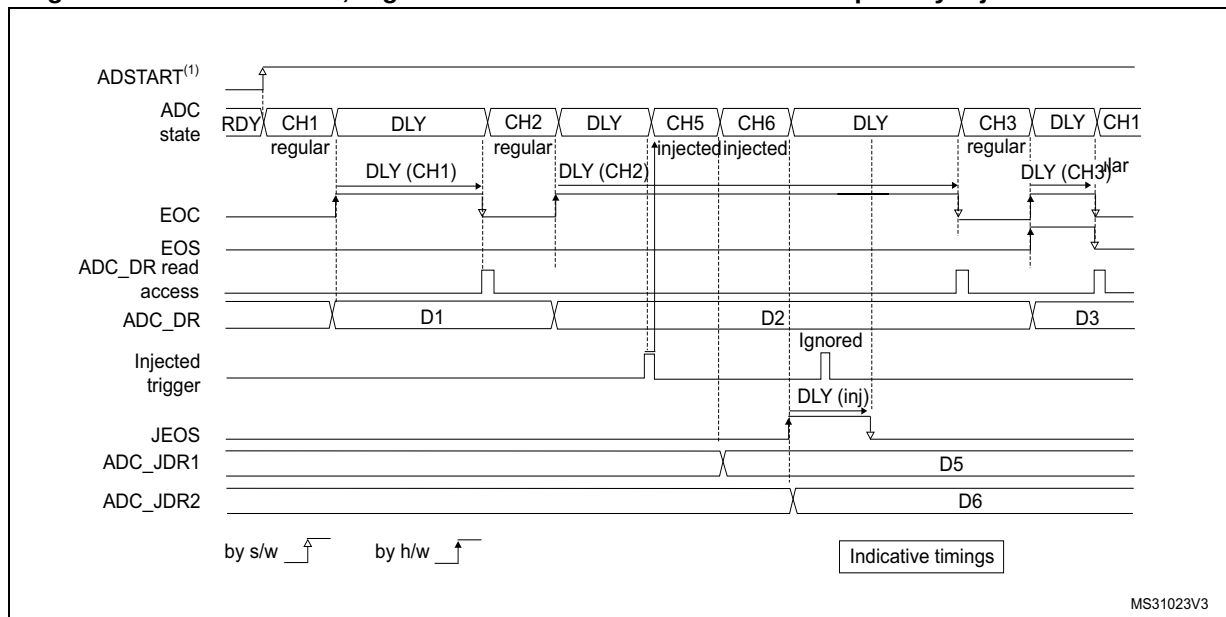
1. AUTDLY = 1
2. Regular configuration: EXTEN=0x1 (HW trigger), CONT = 0, DISCEN = 0, CHANNELS = 1, 2, 3
3. Injected configuration: JEXTEN = 0x1 (HW Trigger), JDISCEN = 0, CHANNELS = 5,6

Figure 275. AUTDLY = 1, regular HW conversions interrupted by injected conversions (DISCEN = 1, JDISCEN = 1)

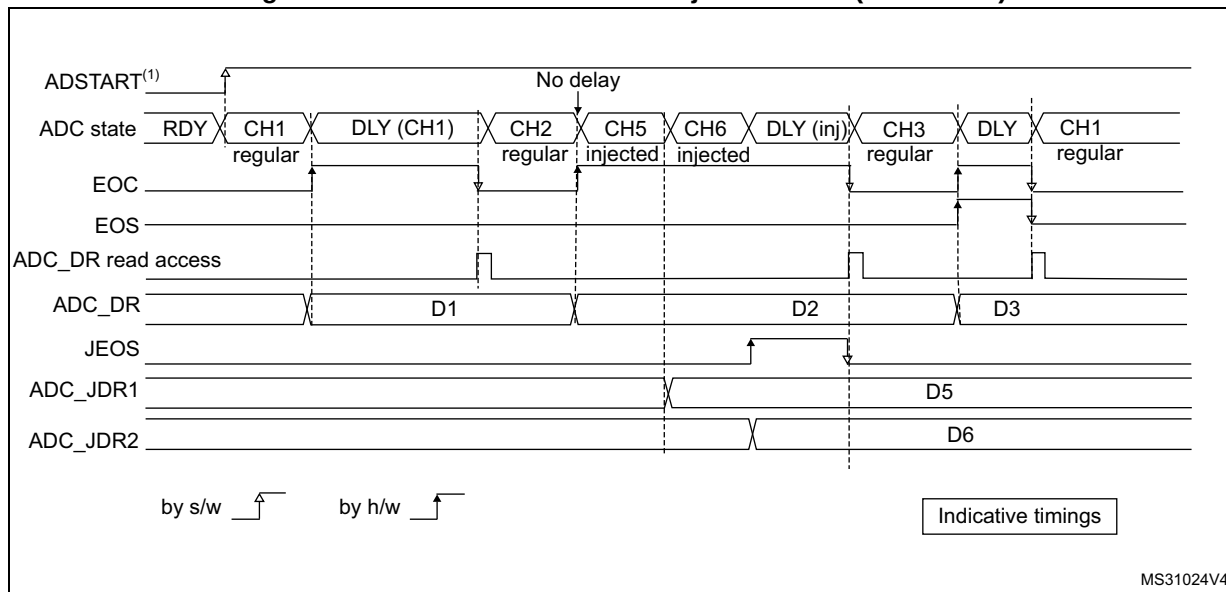


MS31022V1

1. $AUTDLY = 1$
2. Regular configuration: $EXTEN = 0x1$ (HW trigger), $CONT = 0$, $DISCEN = 1$, $DISCNUM = 1$, $CHANNELS = 1, 2, 3$.
3. Injected configuration: $JEXTEN = 0x1$ (HW Trigger), $JDISCEN = 1$, $CHANNELS = 5, 6$

Figure 276. AUTODLY = 1, regular continuous conversions interrupted by injected conversions

1. AUTDLY = 1
2. Regular configuration: EXTEN = 0x0 (SW trigger), CONT = 1, DISCEN = 0, CHANNELS = 1, 2, 3
3. Injected configuration: JEXTEN = 0x1 (HW Trigger), JDISCEN = 0, CHANNELS = 5,6

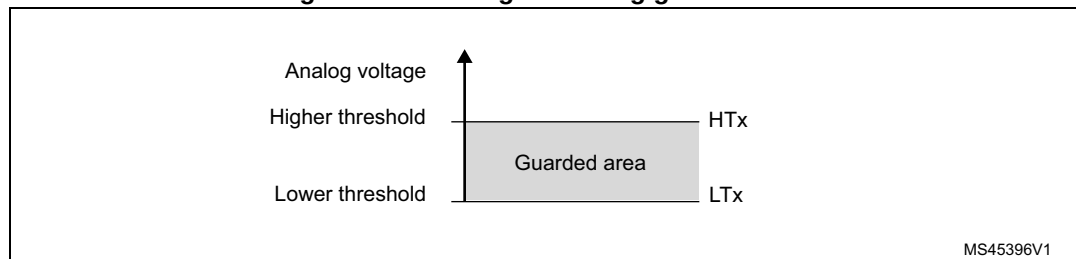
Figure 277. AUTODLY = 1 in auto-injected mode (JAUTO = 1)

1. AUTDLY = 1
2. Regular configuration: EXTEN = 0x0 (SW trigger), CONT = 1, DISCEN = 0, CHANNELS = 1, 2
3. Injected configuration: JAUTO = 1, CHANNELS = 5,6

28.4.28 Analog window watchdog (AWD1EN, JAWD1EN, AWD1SGL, AWD1CH, AWD2CH, AWD3CH, AWD_HTx, AWD_LTx, AWDx)

The three AWD analog watchdogs monitor whether some channels remain within a configured voltage range (window).

Figure 278. Analog watchdog guarded area



MS45396V1

AWDx flag and interrupt

An interrupt can be enabled for each of the 3 analog watchdogs by setting AWDxIE in the ADC_IER register ($x = 1, 2, 3$).

AWDx ($x = 1, 2, 3$) flag is cleared by software by writing 1 to it.

The ADC conversion result is compared to the lower and higher thresholds before alignment.

Description of analog watchdog 1

The AWD analog watchdog 1 is enabled by setting the AWD1EN bit in the ADC_CFGR register. This watchdog monitors whether either one selected channel or all enabled channels⁽¹⁾ remain within a configured voltage range (window).

[Table 287](#) shows how the ADC_CFGR registers should be configured to enable the analog watchdog on one or more channels.

Table 287. Analog watchdog channel selection

Channels guarded by the analog watchdog	AWD1SGL bit	AWD1EN bit	JAWD1EN bit
None	x	0	0
All injected channels	0	0	1
All regular channels	0	1	0
All regular and injected channels	0	1	1
Single ⁽¹⁾ injected channel	1	0	1
Single ⁽¹⁾ regular channel	1	1	0
Single ⁽¹⁾ regular or injected channel	1	1	1

1. Selected by the AWD1CH[4:0] bits. The channels must also be programmed to be converted in the appropriate regular or injected sequence.

The AWD1 analog watchdog status bit is set if the analog voltage converted by the ADC is below a lower threshold or above a higher threshold.

These thresholds are programmed in bits HT1[11:0] and LT1[11:0] of the ADC_TR1 register for the analog watchdog 1. When converting data with a resolution of less than 12 bits (according to bits RES[1:0]), the LSB of the programmed thresholds must be kept cleared because the internal comparison is always performed on the full 12-bit raw converted data (left aligned) before the offset compensation stage.

[Table 288](#) describes how the comparison is performed for all the possible resolutions for analog watchdog 1.

Table 288. Analog watchdog 1 comparison

Resolution(bit RES[1:0])	Analog watchdog comparison between:		Comments
	Raw converted data, left aligned	Thresholds	
00: 12-bit	DATA[11:0]	LT1[11:0] and HT1[11:0]	-
01: 10-bit	DATA[11:2],00	LT1[11:0] and HT1[11:0]	User must configure LT1[1:0] and HT1[1:0] to 00
10: 8-bit	DATA[11:4],0000	LT1[11:0] and HT1[11:0]	User must configure LT1[3:0] and HT1[3:0] to 0000
11: 6-bit	DATA[11:6],000000	LT1[11:0] and HT1[11:0]	User must configure LT1[5:0] and HT1[5:0] to 000000

Analog watchdog filter for watchdog 1

When an ADC is configured with only one input channel (selecting several channels in scan mode not allowed), a valid ADC conversion data filter can be configured:

- When the converted data belong to the interval defined in ADC_TR1, the AWD1 flag remains cleared.
- If data are out-of-range a number of times higher than the value specified in AWDFLT bit of ADC_TR1, the AWD1 flag is set and the corresponding interrupt is issued.

Description of analog watchdog 2 and 3

The second and third analog watchdogs are more flexible and can guard several selected channels by programming the corresponding bits in AWDxCH[19:0] (x = 2,3).

The corresponding watchdog is enabled when any bit of AWDxCH[19:0] (x = 2,3) is set.

They are limited to a resolution of 8 bits and only the 8 MSBs of the thresholds can be programmed into HTx[7:0] and LTx[7:0]. [Table 289](#) describes how the comparison is performed for all the possible resolutions.

Table 289. Analog watchdog 2 and 3 comparison

Resolution (bits RES[1:0])	Analog watchdog comparison between:		Comments
	Raw converted data, left aligned	Thresholds	
00: 12-bit	DATA[11:4]	LTx[7:0] and HTx[7:0]	DATA[3:0] are not relevant for the comparison
01: 10-bit	DATA[11:4]	LTx[7:0] and HTx[7:0]	DATA[3:2] are not relevant for the comparison

Table 289. Analog watchdog 2 and 3 comparison

Resolution (bits RES[1:0])	Analog watchdog comparison between:		Comments
	Raw converted data, left aligned	Thresholds	
10: 8-bit	DATA[11:4]	LTx[7:0] and HTx[7:0]	-
11: 6-bit	DATA[11:6],00	LTx[7:0] and HTx[7:0]	User must configure LTx[1:0] and HTx[1:0] to 00

ADCy_AWDx_OUT signal output generation

Each analog watchdog is associated to an internal hardware signal ADCy_AWDx_OUT (y = ADC number, x = watchdog number) which is directly connected to the ETR input (external trigger) of some on-chip timers. Refer to the on-chip timers section to understand how to select the ADCy_AWDx_OUT signal as ETR.

ADCy_AWDx_OUT is activated when the associated analog watchdog is enabled:

- ADCy_AWDx_OUT is set when a guarded conversion is outside the programmed thresholds.
- ADCy_AWDx_OUT is reset after the end of the next guarded conversion which is inside the programmed thresholds (It remains at 1 if the next guarded conversions are still outside the programmed thresholds).
- ADCy_AWDx_OUT is also reset when disabling the ADC (when setting ADDIS = 1). Note that stopping regular or injected conversions (setting ADSTP = 1 or JADSTP = 1) has no influence on the generation of ADCy_AWDx_OUT.

Note: AWDx flag is set by hardware and reset by software: AWDx flag has no influence on the generation of ADCy_AWDx_OUT (ex: ADCy_AWDx_OUT can toggle while AWDx flag remains at 1 if the software did not clear the flag).

Figure 279. ADCy_AWDx_OUT signal generation (on all regular channels)

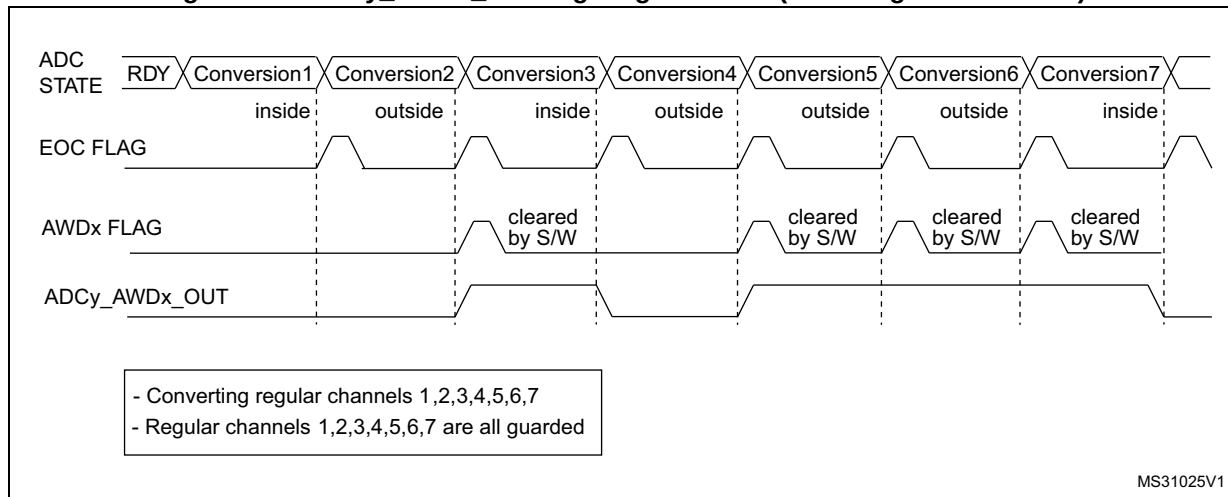


Figure 280. ADCy_AWDx_OUT signal generation (AWDx flag not cleared by software)

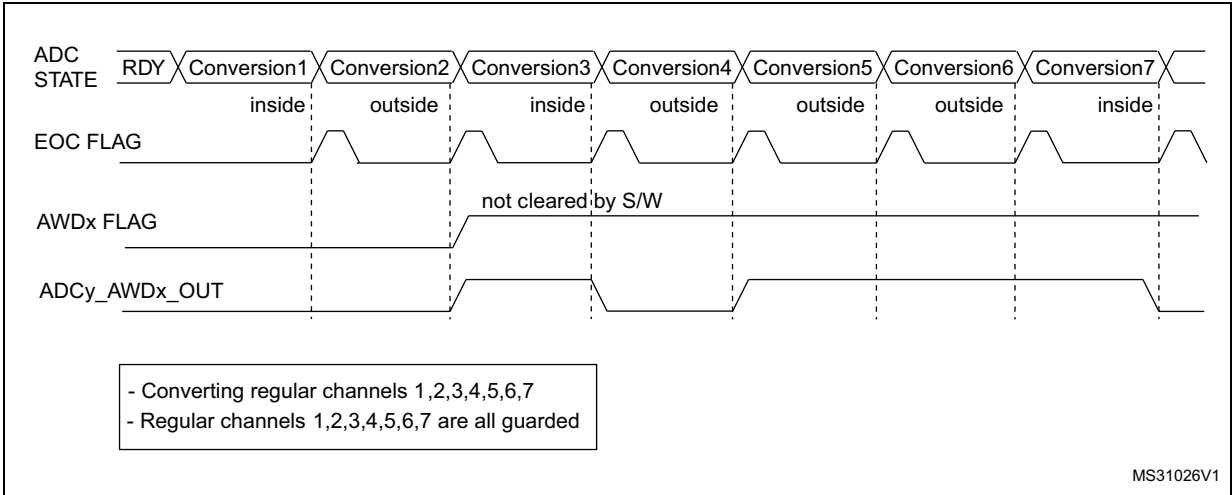


Figure 281. ADCy_AWDx_OUT signal generation (on a single regular channel)

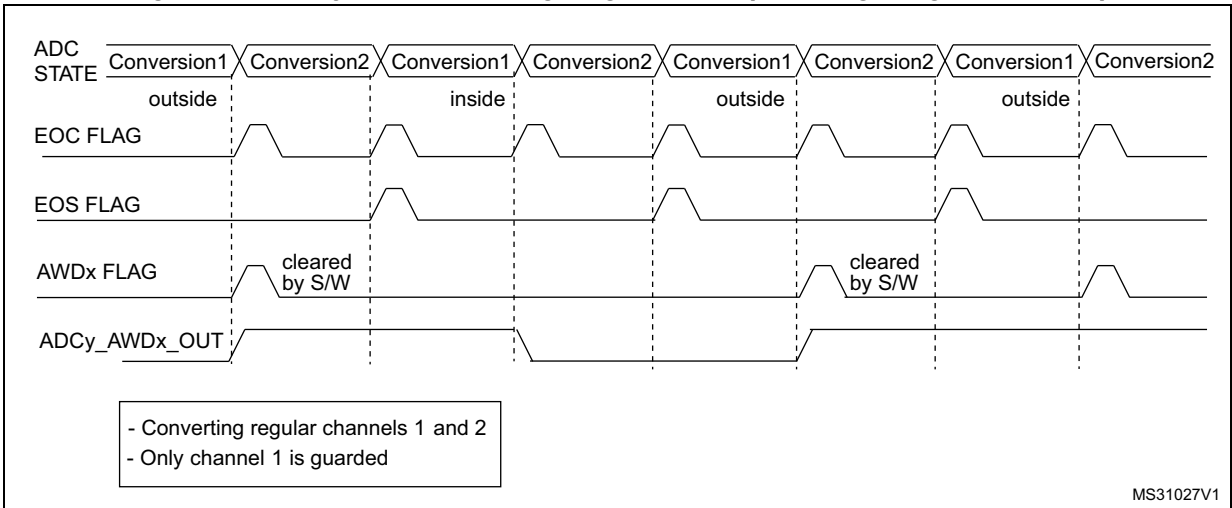
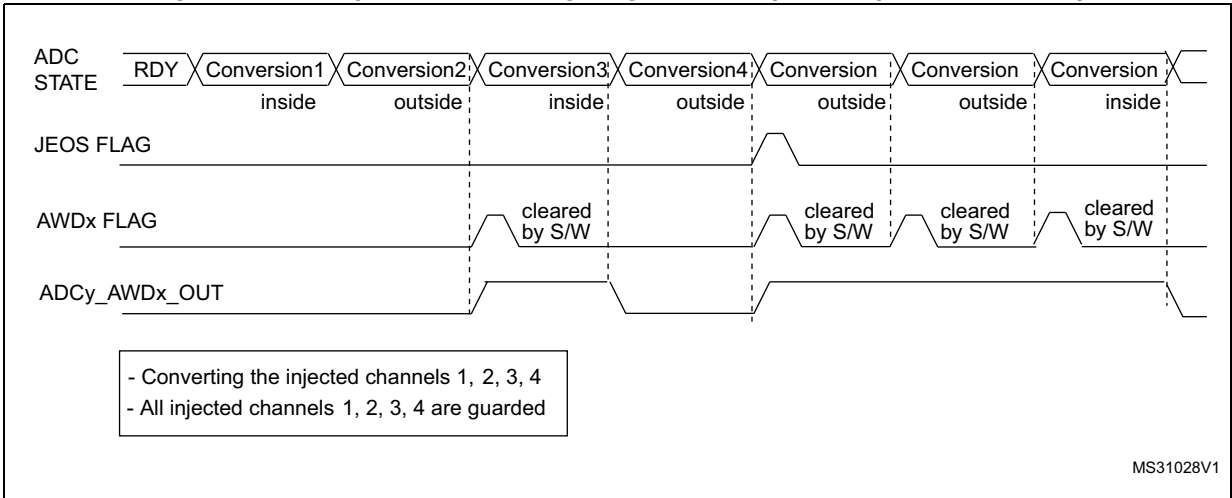


Figure 282. ADCy_AWDx_OUT signal generation (on all injected channels)



Analog watchdog threshold control

LTx[11:0] and HTx[11:0] can be changed when an analog-to-digital conversion is ongoing (that is between the start of conversion and the end of conversion of the ADC internal state). If LTx[11:0] and HTx[11:0] are updated during the ADC conversion of the ADC guarded channel, the watchdog function is masked for this conversion. This masking is removed at the next start of conversion, resulting in analog watchdog thresholds to be applied from the next ADC conversion. The analog watchdog comparison is performed at each end of conversion. If the current ADC data is out of the new interval, no interrupt and AWDx_OUT signal are issued. The Interrupt and the AWD generation only happen at the end of the conversion which started after the threshold update. If AWD_xOUT is already asserted, programming the new thresholds does not deassert the AWDx_OUT signal.

To update both LTx[11:0] and HTx[11:0] during an ADC conversion, a unique write access to the ADC_TRx register be performed.

Analog watchdog with offset compensation

When the offset compensation is enabled, the analog watchdog compares the threshold before the data compensation.

28.4.29 Oversampler

The oversampling unit performs data pre-processing to offload the CPU. It is able to handle multiple conversions and average them into a single data with increased data width, up to 16-bit.

It provides a result with the following form, where N and M can be adjusted:

$$\text{Result} = \frac{1}{M} \times \sum_{n=0}^{n=N-1} \text{Conversion}(t_n)$$

It allows to perform by hardware the following functions: averaging, data rate reduction, SNR improvement, basic filtering.

The oversampling ratio N is defined using the OVFS[2:0] bits in the ADC_CFGR2 register, and can range from 2x to 256x. The division coefficient M consists of a right bit shift up to 8 bits, and is defined using the OVSS[3:0] bits in the ADC_CFGR2 register.

The summation unit can yield a result up to 20 bits (256x 12-bit results), which is first shifted right. It is then truncated to the 16 least significant bits, rounded to the nearest value using the least significant bits left apart by the shifting, before being finally transferred into the ADC_DR data register.

Note: If the intermediary result after the shifting exceeds 16-bit, the result is truncated as is, without saturation.

Figure 283. 20-bit to 16-bit result truncation

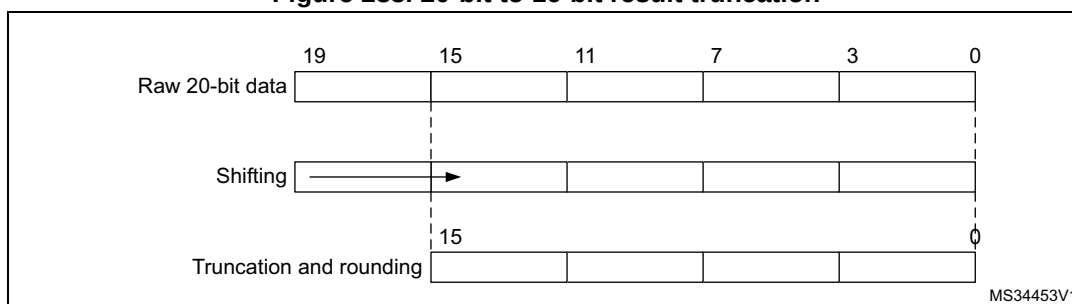


Figure 284 gives a numerical example of the processing, from a raw 20-bit accumulated data to the final 16-bit result.

Figure 284. Numerical example with 5-bit shift and rounding

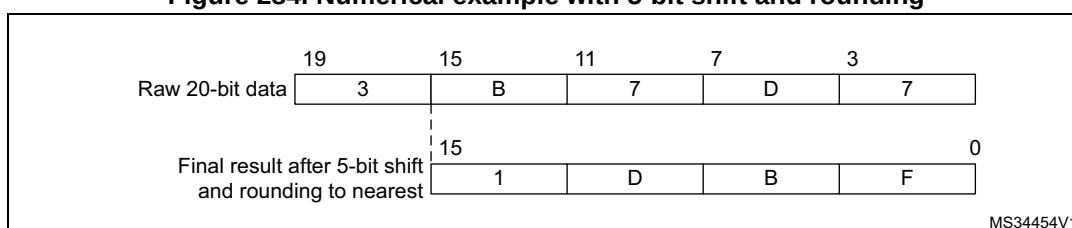


Table 290 gives the data format for the various N and M combinations, for a raw conversion data equal to 0xFFFF.

Table 290. Maximum output results versus N and M (gray cells indicate truncation)

Over sampling ratio	Max Raw data	No-shift OVSS = 0000	1-bit shift OVSS = 0001	2-bit shift OVSS = 0010	3-bit shift OVSS = 0011	4-bit shift OVSS = 0100	5-bit shift OVSS = 0101	6-bit shift OVSS = 0110	7-bit shift OVSS = 0111	8-bit shift OVSS = 1000
2x	0x1FFE	0x1FFE	0x0FFF	0x0800	0x0400	0x0200	0x0100	0x0080	0x0040	0x0020
4x	0x3FFC	0x3FFC	0x1FFE	0x0FFF	0x0800	0x0400	0x0200	0x0100	0x0080	0x0040
8x	0x7FF8	0x7FF8	0x3FFC	0x1FFE	0x0FFF	0x0800	0x0400	0x0200	0x0100	0x0080
16x	0xFFF0	0xFFF0	0x7FF8	0x3FFC	0x1FFE	0x0FFF	0x0800	0x0400	0x0200	0x0100
32x	0x1FFE0	0xFFE0	0xFFF0	0x7FF8	0x3FFC	0x1FFE	0x0FFF	0x0800	0x0400	0x0200
64x	0x3FFC0	0xFFC0	0xFFE0	0xFFF0	0x7FF8	0x3FFC	0x1FFE	0x0FFF	0x0800	0x0400
128x	0x7FF80	0xFF80	0xFFC0	0xFFE0	0xFFF0	0x7FF8	0x3FFC	0x1FFE	0x0FFF	0x0800
256x	0xFFF00	0xFF00	0xFF80	0xFFC0	0xFFE0	0xFFF0	0x7FF8	0x3FFC	0x1FFE	0x0FFF

There are no changes for conversion timings in oversampled mode: the sample time is maintained equal during the whole oversampling sequence. A new data is provided every N

conversions, with an equivalent delay equal to $N \times T_{\text{CONV}} = N \times (t_{\text{SMPL}} + t_{\text{SAR}})$. The flags are set as follow:

- The end of the sampling phase (EOSMP) is set after each sampling phase
- The end of conversion (EOC) occurs once every N conversions, when the oversampled result is available
- The end of sequence (EOS) occurs once the sequence of oversampled data is completed (that is after N x sequence length conversions total)

ADC operating modes supported when oversampling (single ADC mode)

In oversampling mode, most of the ADC operating modes are maintained:

- Single or continuous conversion modes
- ADC conversions start either by software or with triggers
- ADC stop during a conversion (abort)
- Data read via CPU or DMA with overrun detection
- Low-power modes (AUTDLY)
- Programmable resolution: in this case, the reduced conversion values (as per RES[1:0] bits in ADC_CFGR register) are accumulated, truncated, rounded and shifted in the same way as 12-bit conversions are

Note: The alignment mode is not available when working with oversampled data. The ALIGN bit in ADC_CFGR is ignored and the data are always provided right-aligned.

Offset correction is not supported in oversampling mode. When ROVSE and/or JOVSE bit is set, the value of the OFFSET_EN bit in ADC_OFRy register is ignored (considered as reset).

Analog watchdog

The analog watchdog functionality is maintained, with the following difference:

- The RES[1:0] bits are ignored, the comparison is always done by using the full 12-bit values HT1[11:0] and LT1[11:0] for AWD1, and the 8-MSB value of HT2/HT3[7:0] and LT2/LT3[7:0] for AWD2 and AWD3.
- The comparison is done on the most significant 12-bit of the 16-bit oversampled results ADC_DR[15:4] for AWD1, and ADC_DR[15:8] for AWD2 and AWD3

Note: Care must be taken when using high shifting values, this reduces the comparison range. For instance, if the oversampled result is shifted by 4 bits, thus yielding a 12-bit data right-aligned, the effective analog watchdog comparison can only be performed on 8 bits. The comparison is done between ADC_DR[11:4] and HTx[7:0] / LTx[7:0] (AWD1/2/3), with HT1[11:8] and LT1[11:8] kept reset (AWD1 only).

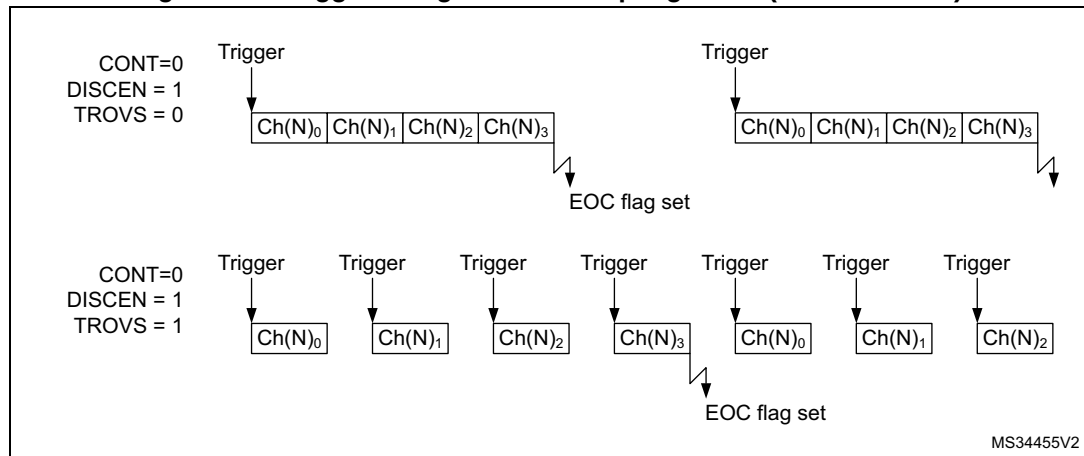
Triggered mode

The averager can also be used for basic filtering purpose. Although not a very powerful filter (slow roll-off and limited stop band attenuation), it can be used as a notch filter to reject constant parasitic frequencies (typically coming from the mains or from a switched mode power supply). For this purpose, a specific discontinuous mode can be enabled with TROVS bit in ADC_CFGR2, to be able to have an oversampling frequency defined by a user and independent from the conversion time itself.

The [Figure 285](#) below shows how conversions are started in response to triggers during discontinuous mode.

If the TROVS bit is set, the content of the DISCEN bit is ignored and considered as 1.

Figure 285. Triggered regular oversampling mode (TROVS bit = 1)



Injected and regular sequencer management when oversampling

In oversampling mode, it is possible to have differentiated behavior for injected and regular sequencers. The oversampling can be enabled for both sequencers with some limitations if they have to be used simultaneously (this is related to a unique accumulation unit).

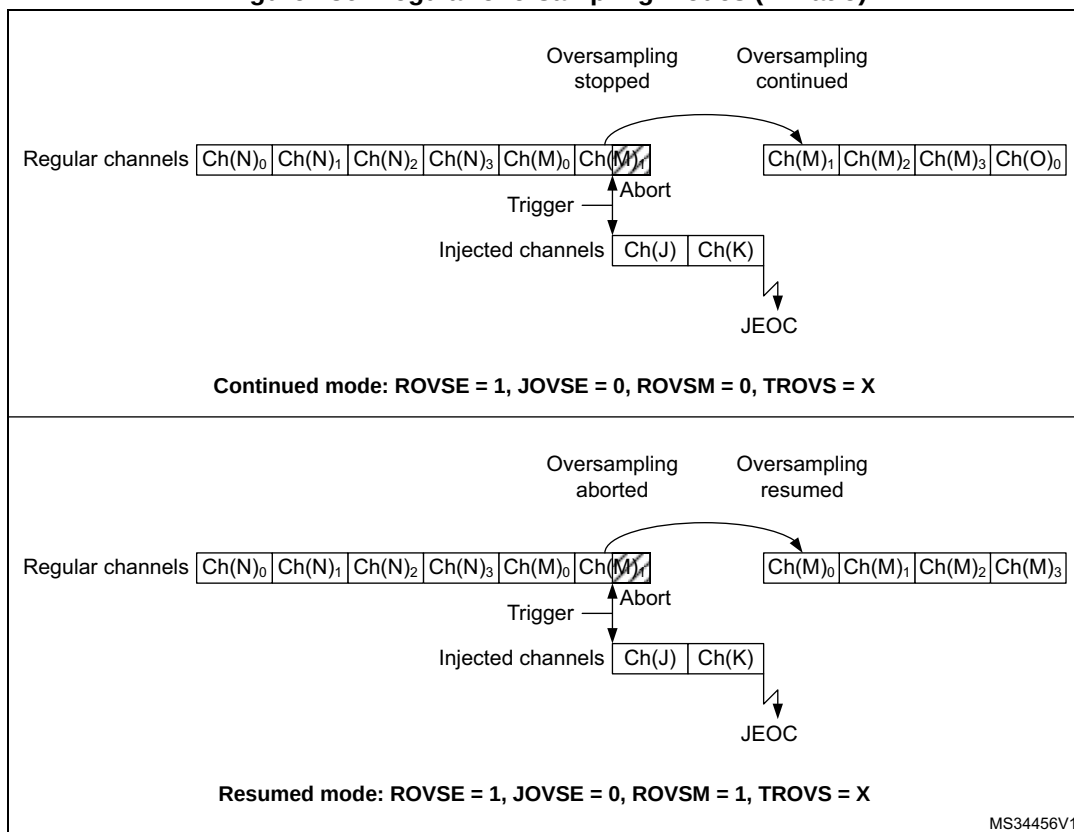
Oversampling regular channels only

The regular oversampling mode bit ROVSM defines how the regular oversampling sequence is resumed if it is interrupted by injected conversion:

- In continued mode, the accumulation restarts from the last valid data (prior to the conversion abort request due to the injected trigger). This ensures that oversampling is complete whatever the injection frequency (providing at least one regular conversion can be completed between triggers);
- In resumed mode, the accumulation restarts from 0 (previous conversions results are ignored). This mode allows to guarantee that all data used for oversampling were converted back-to-back within a single timeslot. Care must be taken to have a injection trigger period above the oversampling period length. If this condition is not respected, the oversampling cannot be completed and the regular sequencer is blocked.

The [Figure 286](#) gives examples for a 4x oversampling ratio.

Figure 286. Regular oversampling modes (4x ratio)



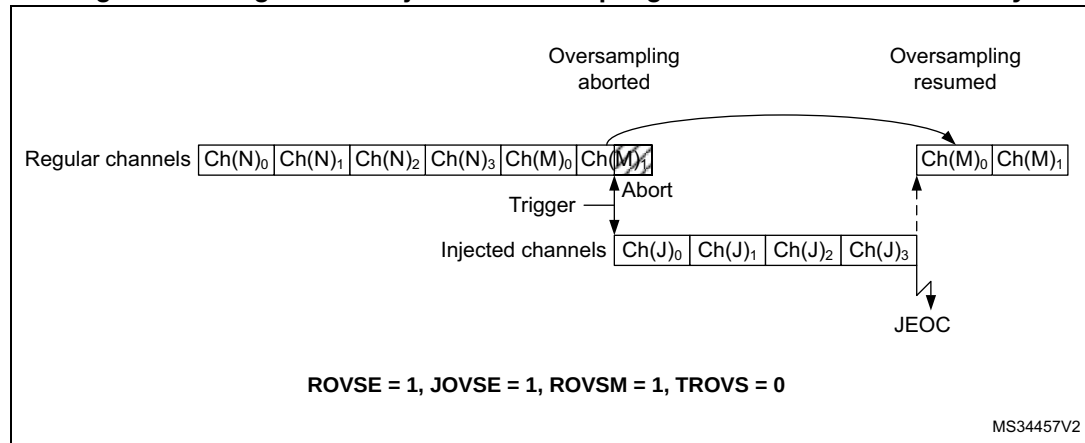
Oversampling Injected channels only

The Injected oversampling mode bit JOVSE enables oversampling solely for conversions in the injected sequencer.

Oversampling regular and Injected channels

It is possible to have both ROVSE and JOVSE bits set. In this case, the regular oversampling mode is forced to resumed mode (ROVSM bit ignored), as represented on [Figure 287](#) below.

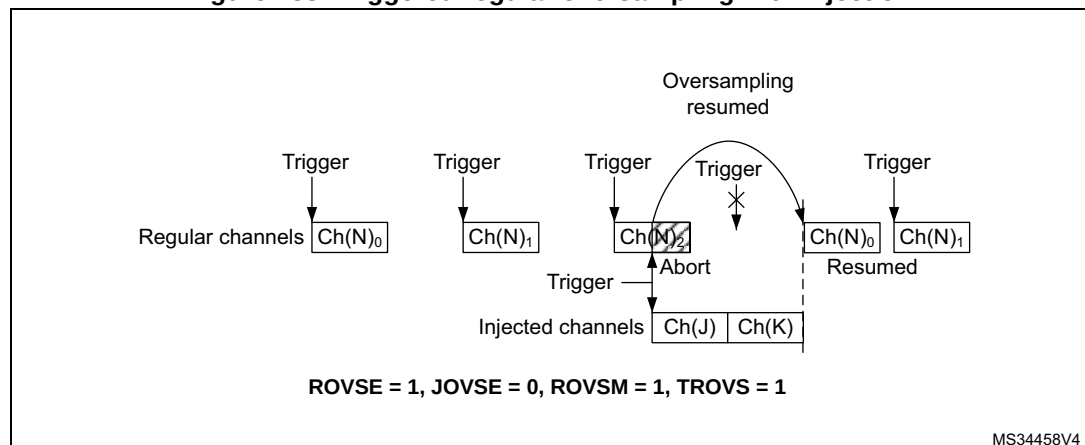
Figure 287. Regular and injected oversampling modes used simultaneously



Triggered regular oversampling with injected conversions

It is possible to have triggered regular mode with injected conversions. In this case, the injected mode oversampling mode must be disabled, and the ROVSM bit is ignored (resumed mode is forced). The JOVSE bit must be reset. The behavior is represented on [Figure 288](#) below.

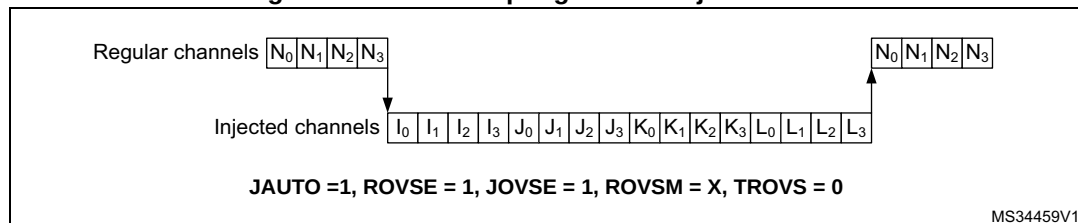
Figure 288. Triggered regular oversampling with injection



Auto-injected mode

It is possible to oversample auto-injected sequences and have all conversions results stored in registers to save a DMA resource. This mode is available only with both regular and injected oversampling active: JAUTO = 1, ROVSE = 1 and JOVSE = 1, other combinations are not supported. The ROVSM bit is ignored in auto-injected mode. The [Figure 289](#) below shows how the conversions are sequenced.

Figure 289. Oversampling in auto-injected mode



It is possible to have also the triggered mode enabled, using the TROVS bit. In this case, the ADC must be configured as following: JAUTO = 1, DISCEN = 0, JDISCEN = 0, ROVSE = 1, JOVSE = 1 and TROVSE = 1.

Dual ADC modes supported when oversampling

It is possible to have oversampling enabled when working in dual ADC configuration, for the injected simultaneous mode and regular simultaneous mode. In this case, the two ADCs must be programmed with the very same settings (including oversampling).

All other dual ADC modes are not supported when either regular or injected oversampling is enabled (ROVSE = 1 or JOVSE = 1).

Combined modes summary

The [Table 291](#) below summarizes all combinations, including modes not supported.

Table 291. Oversampler operating modes summary

Regular Oversampling ROVSE	Injected Oversampling JOVSE	Oversampler mode ROVSM 0 = continued 1 = resumed	Triggered Regular mode TROVS	Comment
1	0	0	0	Regular continued mode
1	0	0	1	Not supported
1	0	1	0	Regular resumed mode
1	0	1	1	Triggered regular resumed mode
1	1	0	X	Not supported
1	1	1	0	Injected and regular resumed mode
1	1	1	1	Not supported
0	1	X	X	Injected oversampling

28.4.30 Dual ADC modes

Dual ADC modes can be used in devices with two ADCs or more (see [Figure 290](#)).

In dual ADC mode the start of conversion is triggered alternately or simultaneously by the ADCx master to the ADC slave, depending on the mode selected by the bits DUAL[4:0] in the ADC_CCR register.

Four possible modes are implemented:

- Injected simultaneous mode
- Regular simultaneous mode
- Interleaved mode
- Alternate trigger mode

It is also possible to use these modes combined in the following ways:

- Injected simultaneous mode + regular simultaneous mode
- Regular simultaneous mode + alternate trigger mode
- Injected simultaneous mode + interleaved mode

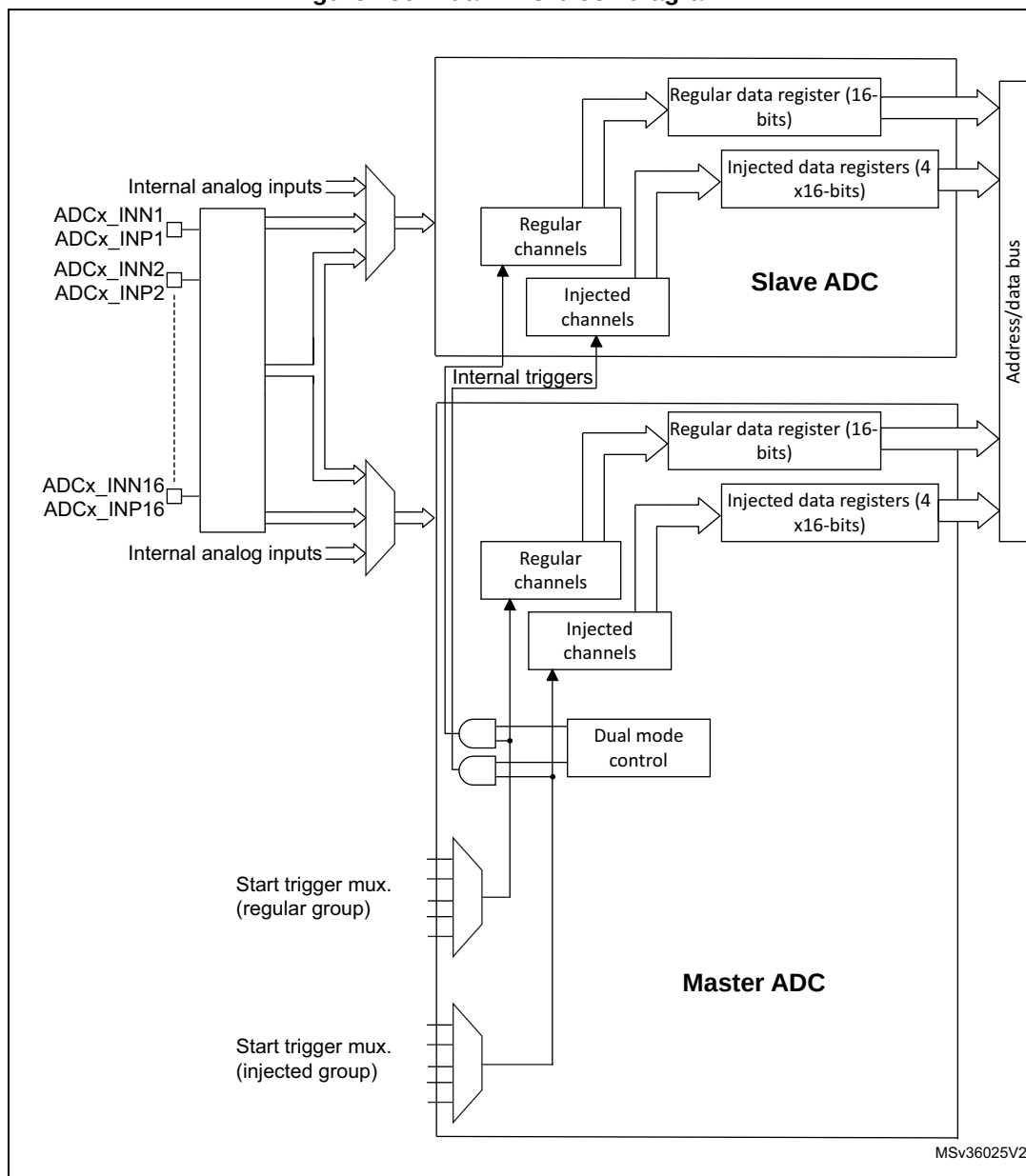
In dual ADC mode (when bits DUAL[4:0] in ADC_CCR register are not equal to zero), the bits CONT, AUTDLY, DISCEN, DISCNUM[2:0], JDISCEN, JQM, JAUTO of the ADC_CFGR register are shared between the master and slave ADC: the bits in the slave ADC are always equal to the corresponding bits of the master ADC.

To start a conversion in dual mode, the user must program the bits EXTEN, EXTSEL, JEXTEN, JEXTSEL of the master ADC only, to configure a software or hardware trigger, and a regular or injected trigger. (the bits EXTEN[1:0] and JEXTEN[1:0] of the slave ADC are don't care).

In regular simultaneous or interleaved modes: once the user sets bit ADSTART or bit ADSTP of the master ADC, the corresponding bit of the slave ADC is also automatically set. However, bit ADSTART or bit ADSTP of the slave ADC is not necessary cleared at the same time as the master ADC bit.

In injected simultaneous or alternate trigger modes: once the user sets bit JADSTART or bit JADSTP of the master ADC, the corresponding bit of the slave ADC is also automatically set. However, bit JADSTART or bit JADSTP of the slave ADC is not necessary cleared at the same time as the master ADC bit.

In dual ADC mode, the converted data of the master and slave ADC can be read in parallel, by reading the ADC common data register (ADC_CDR). The status bits can be also read in parallel by reading the dual-mode status register (ADC_CSR).

Figure 290. Dual ADC block diagram⁽¹⁾

1. External triggers also exist on slave ADC but are not shown for the purposes of this diagram.
2. The ADC common data register (ADC_CDR) contains both the master and slave ADC regular converted data.

Injected simultaneous mode

This mode is selected by programming bits DUAL[4:0] = 00101

This mode converts an injected group of channels. The external trigger source comes from the injected group multiplexer of the master ADC (selected by the JEXTSEL bits in the ADC_JSQR register).

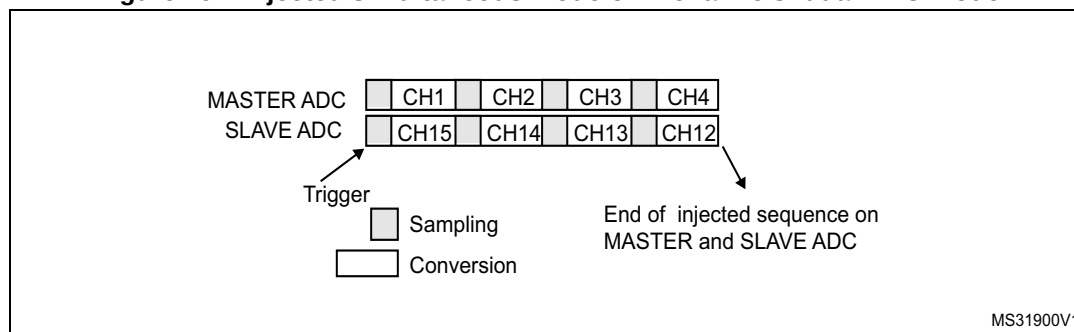
Note: *Do not convert the same channel on the two ADCs (no overlapping sampling times for the two ADCs when converting the same channel).*

In simultaneous mode, one must convert sequences with the same length or ensure that the interval between triggers is longer than the longer of the 2 sequences. Otherwise, the ADC with the shortest sequence may restart while the ADC with the longest sequence is completing the previous conversions.

Regular conversions can be performed on one or all ADCs. In that case, they are independent of each other and are interrupted when an injected event occurs. They are resumed at the end of the injected conversion group.

- At the end of injected sequence of conversion event (JEOS) on the master ADC, the converted data is stored into the master ADC_JDRy registers and a JEOS interrupt is generated (if enabled)
- At the end of injected sequence of conversion event (JEOS) on the slave ADC, the converted data is stored into the slave ADC_JDRy registers and a JEOS interrupt is generated (if enabled)
- If the duration of the master injected sequence is equal to the duration of the slave injected one (like in [Figure 291](#)), it is possible for the software to enable only one of the two JEOS interrupt (ex: master JEOS) and read both converted data (from master ADC_JDRy and slave ADC_JDRy registers).

Figure 291. Injected simultaneous mode on 4 channels: dual ADC mode



If JDISCEN = 1, each simultaneous conversion of the injected sequence requires an injected trigger event to occur.

This mode can be combined with AUTDLY mode:

- Once a simultaneous injected sequence of conversions has ended, a new injected trigger event is accepted only if both JEOS bits of the master and the slave ADC have been cleared (delay phase). Any new injected trigger events occurring during the ongoing injected sequence and the associated delay phase are ignored.
- Once a regular sequence of conversions of the master ADC has ended, a new regular trigger event of the master ADC is accepted only if the master data register (ADC_DR) has been read. Any new regular trigger events occurring for the master ADC during the

ongoing regular sequence and the associated delay phases are ignored.
There is the same behavior for regular sequences occurring on the slave ADC.

Regular simultaneous mode with independent injected

This mode is selected by programming bits DUAL[4:0] = 00110.

This mode is performed on a regular group of channels. The external trigger source comes from the regular group multiplexer of the master ADC (selected by the EXTSEL bits in the ADC_CFGR register). A simultaneous trigger is provided to the slave ADC.

In this mode, independent injected conversions are supported. An injection request (either on master or on the slave) aborts the current simultaneous conversions, which are restarted once the injected conversion is completed.

Note: *Do not convert the same channel on the two ADCs (no overlapping sampling times for the two ADCs when converting the same channel).*
In regular simultaneous mode, one must convert sequences with the same length or ensure that the interval between triggers is longer than the longer conversion time of the 2 sequences. Otherwise, the ADC with the shortest sequence may restart while the ADC with the longest sequence is completing the previous conversions.

Software is notified by interrupts when it can read the data:

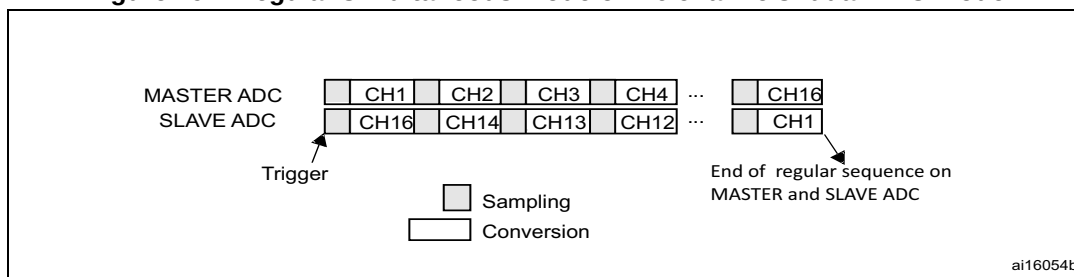
- At the end of each conversion event (EOC) on the master ADC, a master EOC interrupt is generated (if EOCIE is enabled) and software can read the ADC_DR of the master ADC.
- At the end of each conversion event (EOC) on the slave ADC, a slave EOC interrupt is generated (if EOCIE is enabled) and software can read the ADC_DR of the slave ADC.
- If the duration of the master regular sequence is equal to the duration of the slave one (like in [Figure 292](#)), it is possible for the software to enable only one of the two EOC interrupt (ex: master EOC) and read both converted data from the Common Data register (ADC_CDR).

It is also possible to read the regular data using the DMA. Two methods are possible:

- Using two DMA channels (one for the master and one for the slave). In this case bits MDMA[1:0] must be kept cleared.
 - Configure the DMA master ADC channel to read ADC_DR from the master. DMA requests are generated at each EOC event of the master ADC.
 - Configure the DMA slave ADC channel to read ADC_DR from the slave. DMA requests are generated at each EOC event of the slave ADC.
- Using MDMA mode, which leaves one DMA channel free for other uses:
 - Configure MDMA[1:0] = 0b10 or 0b11 (depending on resolution).
 - A single DMA channel is used (the one of the master). Configure the DMA master ADC channel to read the common ADC register (ADC_CDR)
 - A single DMA request is generated each time both master and slave EOC events have occurred. At that time, the slave ADC converted data is available in the upper half-word of the ADC_CDR 32-bit register and the master ADC converted data is available in the lower half-word of ADC_CDR register.
 - Both EOC flags are cleared when the DMA reads the ADC_CDR register.

Note: *In MDMA mode (MDMA[1:0] = 0b10 or 0b11), the user must program the same number of conversions in the master's sequence as in the slave's sequence. Otherwise, the remaining conversions does not generate a DMA request.*

Figure 292. Regular simultaneous mode on 16 channels: dual ADC mode



If DISCEN = 1 then each “n” simultaneous conversions of the regular sequence require a regular trigger event to occur (“n” is defined by DISCNUM).

This mode can be combined with AUTDLY mode:

- Once a simultaneous conversion of the sequence has ended, the next conversion in the sequence is started only if the common data register, ADC_CDR (or the regular data register of the master ADC) has been read (delay phase).
- Once a simultaneous regular sequence of conversions has ended, a new regular trigger event is accepted only if the common data register (ADC_CDR) has been read (delay phase). Any new regular trigger events occurring during the ongoing regular sequence and the associated delay phases are ignored.

It is possible to use the DMA to handle data in regular simultaneous mode combined with AUTDLY mode, assuming that multiple-DMA mode is used: bits MDMA must be set to 0b10 or 0b11.

When regular simultaneous mode is combined with AUTDLY mode, it is mandatory for the user to ensure that:

- The number of conversions in the master's sequence is equal to the number of conversions in the slave's.
- For each simultaneous conversions of the sequence, the length of the conversion of the slave ADC is inferior to the length of the conversion of the master ADC. Note that the length of the sequence depends on the number of channels to convert and the sampling time and the resolution of each channels.

Note: *This combination of regular simultaneous mode and AUTDLY mode is restricted to the use case when only regular channels are programmed: it is forbidden to program injected channels in this combined mode.*

Interleaved mode with independent injected

This mode is selected by programming bits DUAL[4:0] = 00111.

This mode can be started only on a regular group (usually one channel). The external trigger source comes from the regular channel multiplexer of the master ADC.

After an external trigger occurs:

- The master ADC starts immediately.
- The slave ADC starts after a delay of several-ADC clock cycles after the sampling phase of the master ADC has complete.

The minimum delay which separates two conversions in interleaved mode is configured in the DELAY bits in the ADC_CCR register. This delay starts counting one half cycle after the end of the sampling phase of the master conversion. This way, an ADC cannot start a

conversion if the complementary ADC is still sampling its input (only one ADC can sample the input signal at a given time).

- The minimum possible DELAY is 1 to ensure that there is at least one cycle time between the opening of the analog switch of the master ADC sampling phase and the closing of the analog switch of the slave ADC sampling phase.
- The maximum DELAY is equal to the number of cycles corresponding to the selected resolution. However the user must properly calculate this delay to ensure that an ADC does not start a conversion while the other ADC is still sampling its input.

If the CONT bit is set on both master and slave ADCs, the selected regular channels of both ADCs are continuously converted.

The software is notified by interrupts when it can read the data at the end of each conversion event (EOC) on the slave ADC. A slave and master EOC interrupts are generated (if EOCIE is enabled) and the software can read the ADC_DR of the slave/master ADC.

Note: *It is possible to enable only the EOC interrupt of the slave and read the common data register (ADC_CDR). But in this case, the user must ensure that the duration of the conversions are compatible to ensure that inside the sequence, a master conversion is always followed by a slave conversion before a new master conversion restarts. It is recommended to use the MDMA mode.*

It is also possible to have the regular data transferred by DMA. In this case, individual DMA requests on each ADC cannot be used and it is mandatory to use the MDMA mode, as following:

- Configure MDMA[1:0] = 0b10 or 0b11 (depending on resolution).
- A single DMA channel is used (the one of the master). Configure the DMA master ADC channel to read the common ADC register (ADC_CDR).
- A single DMA request is generated each time both master and slave EOC events have occurred. At that time, the slave ADC converted data is available in the upper half-word of the ADC_CDR 32-bit register and the master ADC converted data is available in the lower half-word of ADC_CCR register.
- Both EOC flags are cleared when the DMA reads the ADC_CCR register.

Figure 293. Interleaved mode on one channel in continuous conversion mode: dual ADC mode

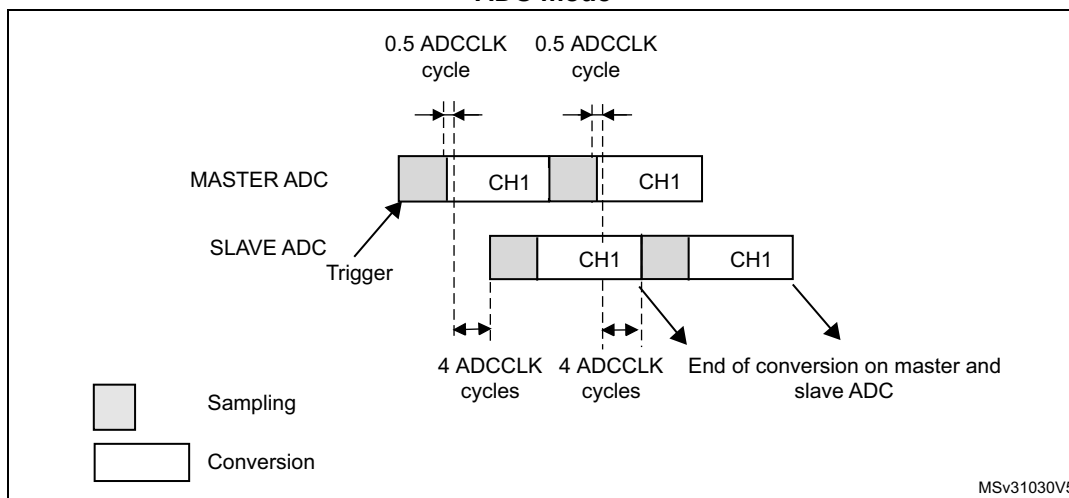
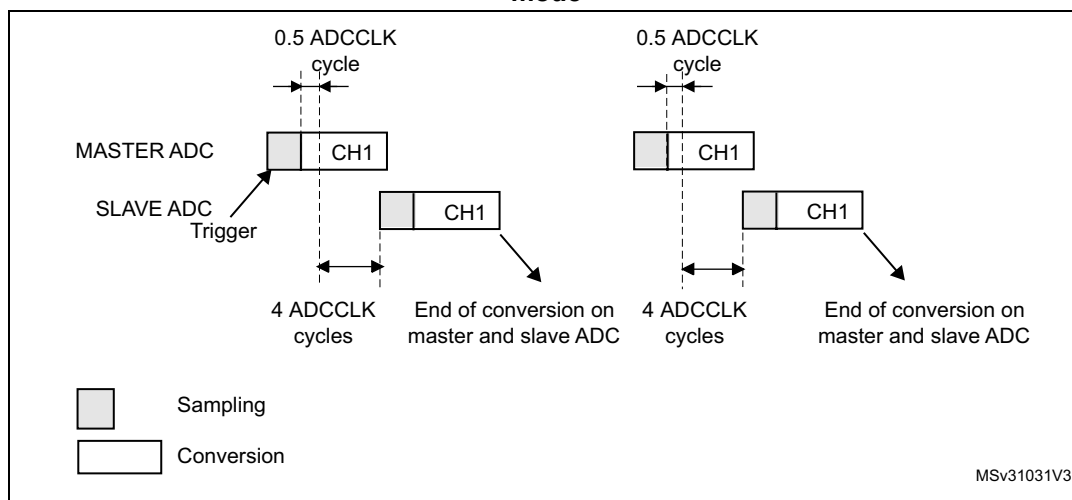
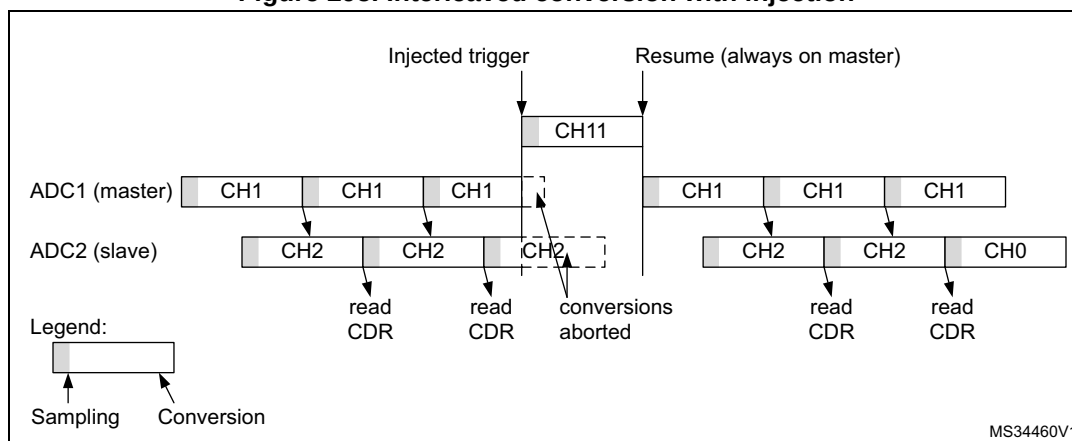


Figure 294. Interleaved mode on 1 channel in single conversion mode: dual ADC mode

If DISCEN = 1, each “n” simultaneous conversions (“n” is defined by DISCNUM) of the regular sequence require a regular trigger event to occur.

In this mode, injected conversions are supported. When injection is done (either on master or on slave), both the master and the slave regular conversions are aborted and the sequence is restarted from the master (see [Figure 295](#) below).

Figure 295. Interleaved conversion with injection

Alternate trigger mode

This mode is selected by programming bits DUAL[4:0] = 01001.

This mode can be started only on an injected group. The source of external trigger comes from the injected group multiplexer of the master ADC.

This mode is only possible when selecting hardware triggers: JEXTEN must not be 0x0.

Injected discontinuous mode disabled (JDISCEN = 0 for both ADC)

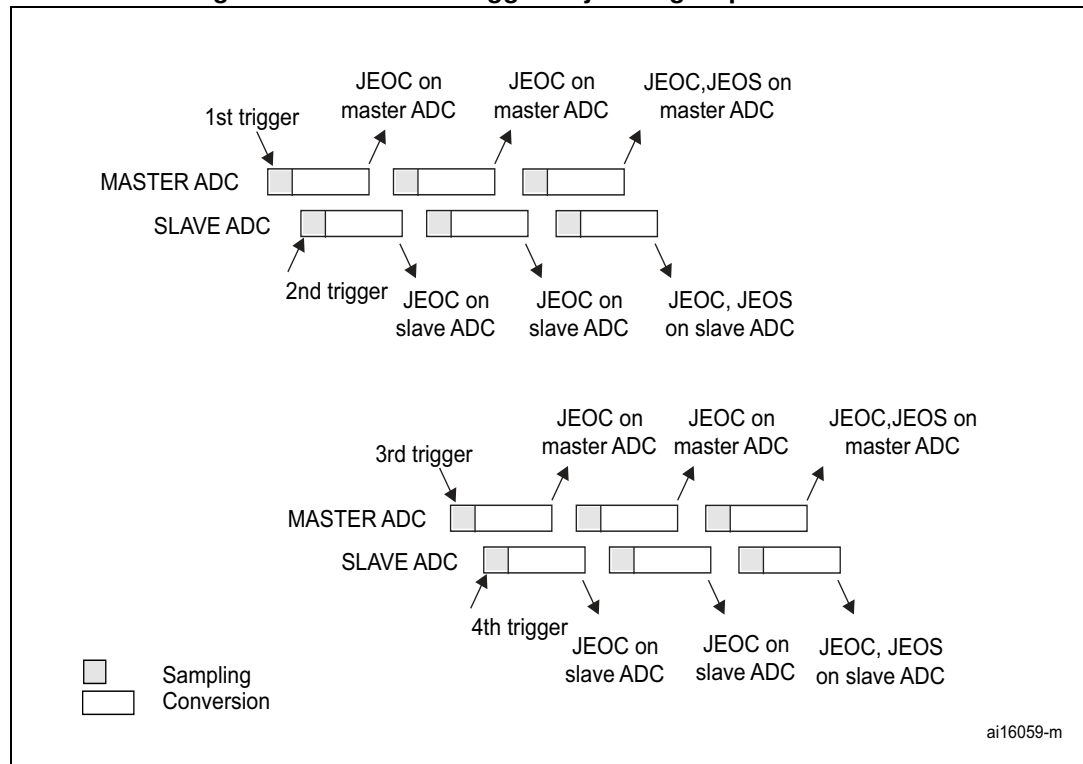
1. When the 1st trigger occurs, all injected master ADC channels in the group are converted.
2. When the 2nd trigger occurs, all injected slave ADC channels in the group are converted.
3. And so on.

A JEOS interrupt, if enabled, is generated after all injected channels of the master ADC in the group have been converted.

A JEOS interrupt, if enabled, is generated after all injected channels of the slave ADC in the group have been converted.

JEOC interrupts, if enabled, can also be generated after each injected conversion.

If another external trigger occurs after all injected channels in the group have been converted then the alternate trigger process restarts by converting the injected channels of the master ADC in the group.

Figure 296. Alternate trigger: injected group of each ADC

Note: Regular conversions can be enabled on one or all ADCs. In this case the regular conversions are independent of each other. A regular conversion is interrupted when the ADC has to perform an injected conversion. It is resumed when the injected conversion is finished.

The time interval between 2 trigger events must be greater than or equal to 1 ADC clock period. The minimum time interval between 2 trigger events that start conversions on the same ADC is the same as in the single ADC mode.

Injected discontinuous mode enabled (JDISCEN = 1 for both ADC)

If the injected discontinuous mode is enabled for both master and slave ADCs:

- When the 1st trigger occurs, the first injected channel of the master ADC is converted.
- When the 2nd trigger occurs, the first injected channel of the slave ADC is converted.
- And so on.

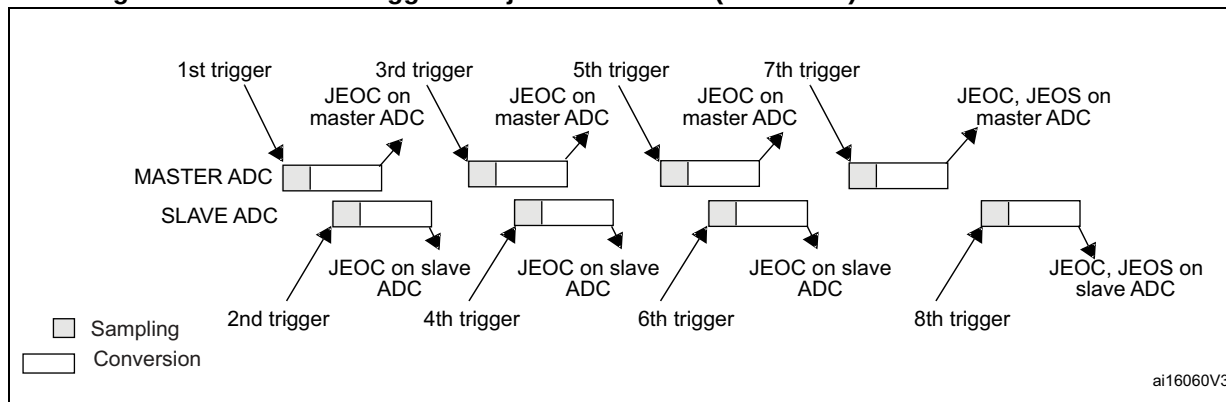
A JEOS interrupt, if enabled, is generated after all injected channels of the master ADC in the group have been converted.

A JEOS interrupt, if enabled, is generated after all injected channels of the slave ADC in the group have been converted.

JEOC interrupts, if enabled, can also be generated after each injected conversions.

If another external trigger occurs after all injected channels in the group have been converted then the alternate trigger process restarts.

Figure 297. Alternate trigger: 4 injected channels (each ADC) in discontinuous mode



Combined regular/injected simultaneous mode

This mode is selected by programming bits DUAL[4:0] = 00001.

It is possible to interrupt the simultaneous conversion of a regular group to start the simultaneous conversion of an injected group.

Note: *In combined regular/injected simultaneous mode, one must convert sequences with the same length or ensure that the interval between triggers is longer than the long conversion time of the 2 sequences. Otherwise, the ADC with the shortest sequence may restart while the ADC with the longest sequence is completing the previous conversions.*

Combined regular simultaneous + alternate trigger mode

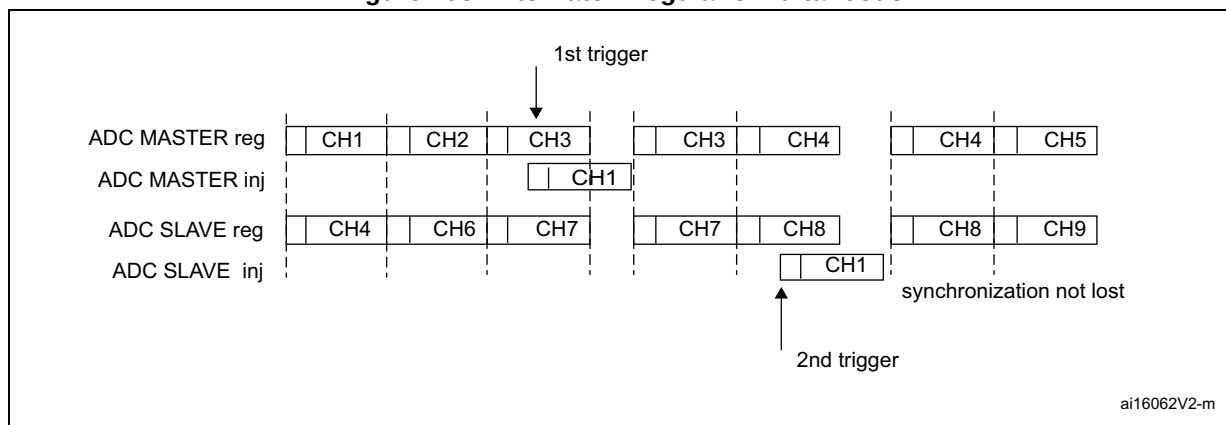
This mode is selected by programming bits DUAL[4:0] = 00010.

It is possible to interrupt the simultaneous conversion of a regular group to start the alternate trigger conversion of an injected group. [Figure 298](#) shows the behavior of an alternate trigger interrupting a simultaneous regular conversion.

The injected alternate conversion is immediately started after the injected event. If a regular conversion is already running, in order to ensure synchronization after the injected conversion, the regular conversion of all (master/slave) ADCs is stopped and resumed synchronously at the end of the injected conversion.

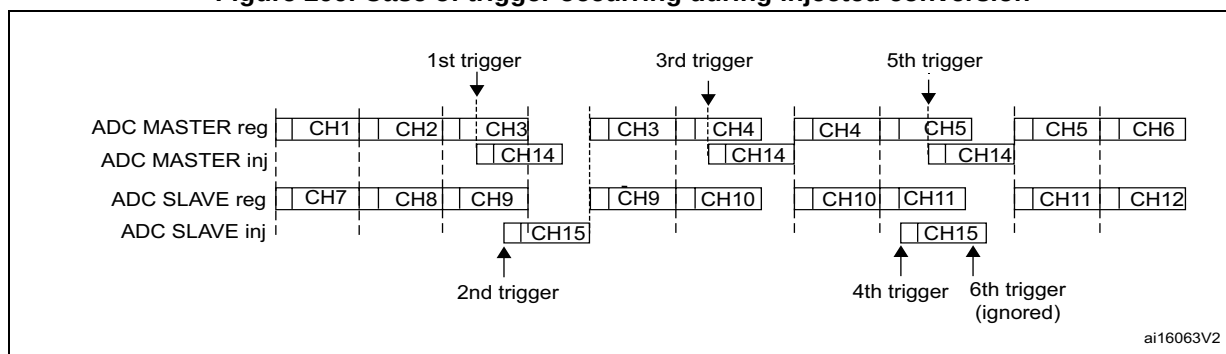
Note: In combined regular simultaneous + alternate trigger mode, one must convert sequences with the same length or ensure that the interval between triggers is longer than the long conversion time of the 2 sequences. Otherwise, the ADC with the shortest sequence may restart while the ADC with the longest sequence is completing the previous conversions.

Figure 298. Alternate + regular simultaneous



If a trigger occurs during an injected conversion that has interrupted a regular conversion, the alternate trigger is served. [Figure 299](#) shows the behavior in this case (note that the 6th trigger is ignored because the associated alternate conversion is not complete).

Figure 299. Case of trigger occurring during injected conversion



Combined injected simultaneous plus interleaved

This mode is selected by programming bits DUAL[4:0] = 00011

It is possible to interrupt an interleaved conversion with a simultaneous injected event.

In this case the interleaved conversion is interrupted immediately and the simultaneous injected conversion starts. At the end of the injected sequence the interleaved conversion is resumed. When the interleaved regular conversion resumes, the first regular conversion which is performed is always the master's one. [Figure 300](#), [Figure 301](#) and [Figure 302](#) show the behavior using an example.

Caution: In this mode, it is mandatory to use the Common Data Register to read the regular data with a single read access. On the contrary, master-slave data coherency is not guaranteed.

Figure 300. Interleaved single channel CH0 with injected sequence CH11, CH12

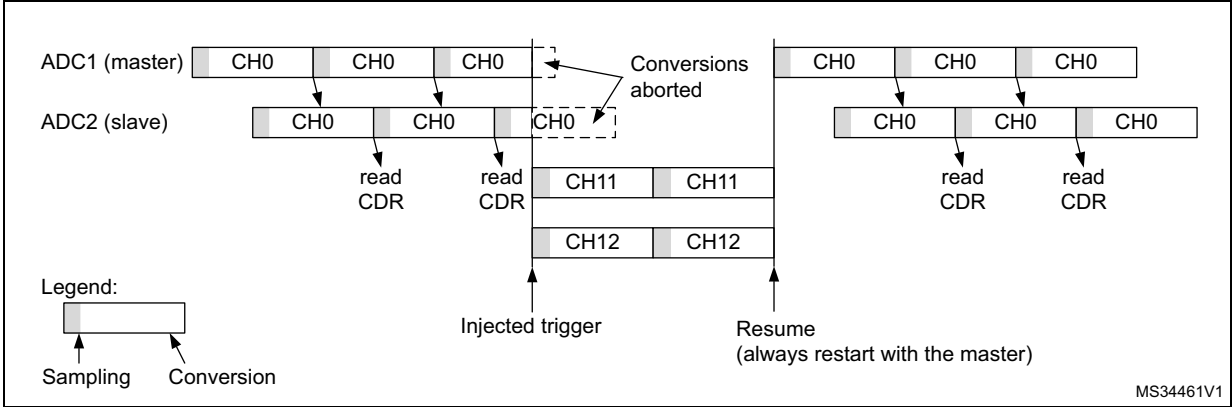


Figure 301. Two Interleaved channels (CH1, CH2) with injected sequence CH11, CH12 - case 1: Master interrupted first

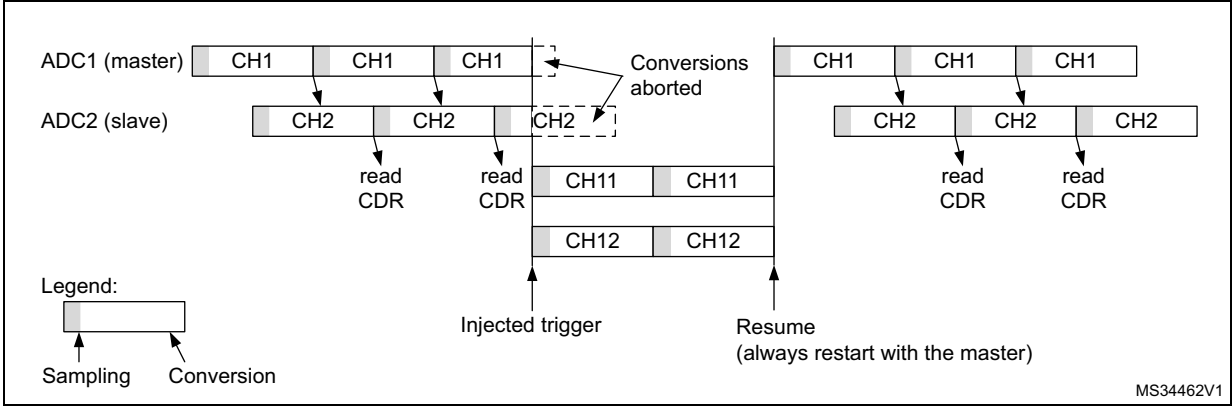
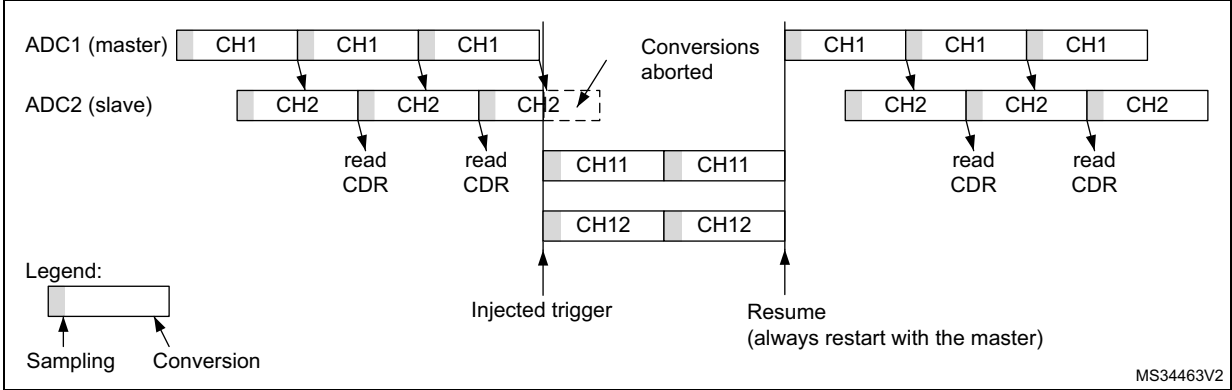


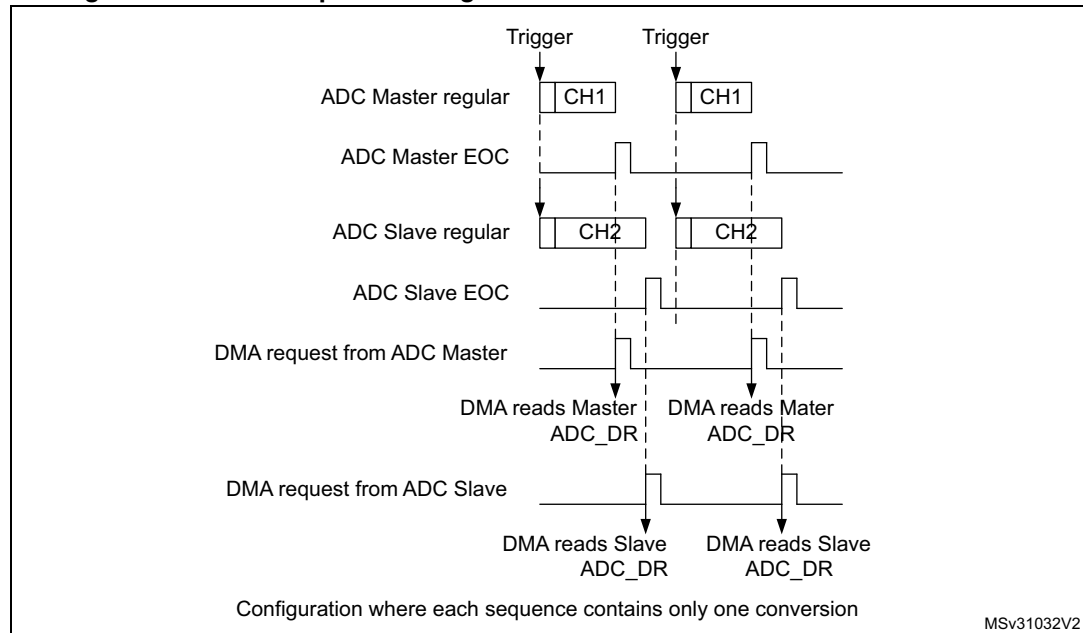
Figure 302. Two Interleaved channels (CH1, CH2) with injected sequence CH11, CH12 - case 2: Slave interrupted first



DMA requests in dual ADC mode

In all dual ADC modes, it is possible to use two DMA channels (one for the master, one for the slave) to transfer the data, like in single mode (refer to [Figure 303: DMA Requests in regular simultaneous mode when MDMA = 0b00](#)).

Figure 303. DMA Requests in regular simultaneous mode when MDMA = 0b00



In simultaneous regular and interleaved modes, it is also possible to save one DMA channel and transfer both data using a single DMA channel. For this MDMA bits must be configured in the ADC_CCR register:

- **MDMA = 0b10:** A single DMA request is generated each time both master and slave EOC events have occurred. At that time, two data items are available and the 32-bit register ADC_CDR contains the two half-words representing two ADC-converted data items. The slave ADC data take the upper half-word and the master ADC data take the lower half-word.

This mode is used in interleaved mode and in regular simultaneous mode when resolution is 10-bit or 12-bit.

Example:

Interleaved dual mode: a DMA request is generated each time 2 data items are available:

1st DMA request: $\text{ADC_CDR}[31:0] = \text{SLV_ADC_DR}[15:0] \mid \text{MST_ADC_DR}[15:0]$

2nd DMA request: $\text{ADC_CDR}[31:0] = \text{SLV_ADC_DR}[15:0] \mid \text{MST_ADC_DR}[15:0]$

Figure 304. DMA requests in regular simultaneous mode when MDMA = 0b10

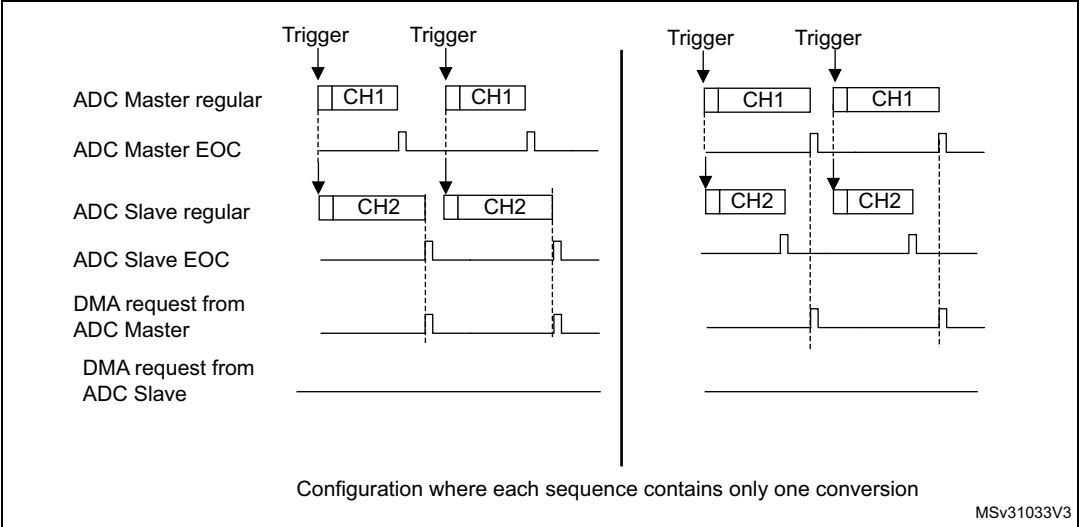
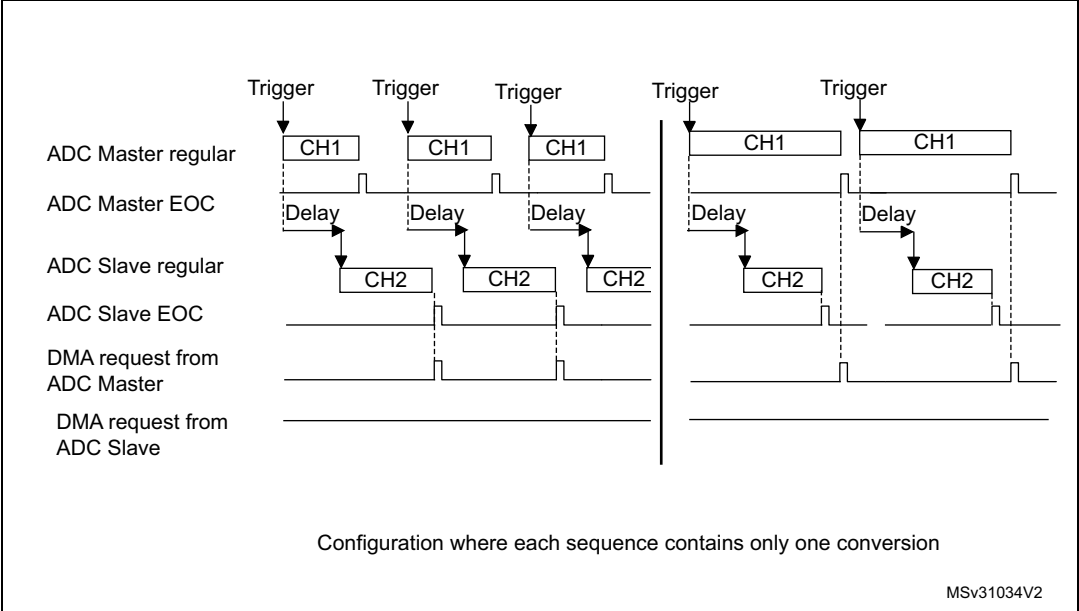


Figure 305. DMA requests in interleaved mode when MDMA = 0b10



Note: When using MDMA mode, the user must take care to configure properly the duration of the master and slave conversions so that a DMA request is generated and served for reading both data (master + slave) before a new conversion is available.

- **MDMA = 0b11:** This mode is similar to the MDMA = 0b10. The only differences are that on each DMA request (two data items are available), two bytes representing two ADC converted data items are transferred as a half-word.

This mode is used in interleaved and regular simultaneous mode when resolution is 6-bit or when resolution is 8-bit and data is not signed (offsets must be disabled for all the involved channels).

Example:

Interleaved dual mode: a DMA request is generated each time 2 data items are available:

1st DMA request: ADC_CDR[15:0] = SLV_ADC_DR[7:0] | MST_ADC_DR[7:0]

2nd DMA request: ADC_CDR[15:0] = SLV_ADC_DR[7:0] | MST_ADC_DR[7:0]

Overrun detection

In dual ADC mode (when DUAL[4:0] is not equal to b00000), if an overrun is detected on one of the ADCs, the DMA requests are no longer issued to ensure that all the data transferred to the RAM are valid (this behavior occurs whatever the MDMA configuration). It may happen that the EOC bit corresponding to one ADC remains set because the data register of this ADC contains valid data.

DMA one shot mode/ DMA circular mode when MDMA mode is selected

When MDMA mode is selected (0b10 or 0b11), bit DMACFG of the ADC_CCR register must also be configured to select between DMA one shot mode and circular mode, as explained in section [Section : Managing conversions using the DMA](#) (bits DMACFG of master and slave ADC_CFGR are not relevant).

Stopping the conversions in dual ADC modes

The user must set the control bits ADSTP/JADSTP of the master ADC to stop the conversions of both ADC in dual ADC mode. The other ADSTP control bit of the slave ADC has no effect in dual ADC mode.

Once both ADC are effectively stopped, the bits ADSTART/JADSTART of the master and slave ADCs are both cleared by hardware.

28.4.31 Temperature sensor

The temperature sensor can be used to measure the junction temperature (Tj) of the device.

The temperature sensor is internally connected to the ADC input channels which are used to convert the sensor output voltage to a digital value (see [Table: ADC interconnection](#) in [Section 28.4.2: ADC pins and internal signals](#) for more details). When not in use, the sensor can be put in power down mode. It support the temperature range -40 to 125 °C.

[Figure 306](#) shows the block diagram of connections between the temperature sensor and the ADC.

The temperature sensor output voltage changes linearly with temperature. The offset of this line varies from chip to chip due to process variation (up to 45 °C from one chip to another).

The uncalibrated internal temperature sensor is more suited for applications that detect temperature variations instead of absolute temperatures. To improve the accuracy of the temperature sensor measurement, calibration values are stored in system memory for each device by ST during production.

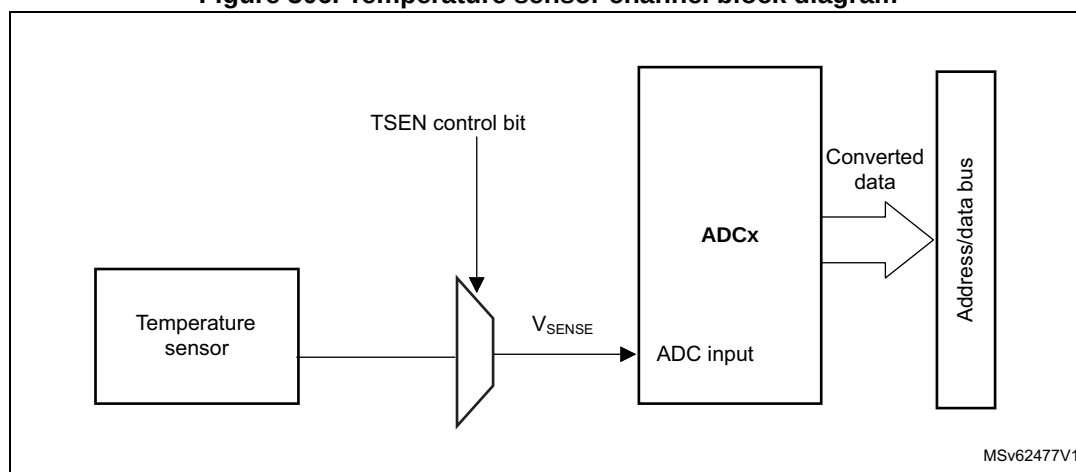
During the manufacturing process, the calibration data of the temperature sensor and the internal voltage reference are stored in the system memory area. The user application can then read them and use them to improve the accuracy of the temperature sensor or the internal reference (refer to the datasheet for additional information).

The temperature sensor is internally connected to the ADC input channel which is used to convert the sensor's output voltage to a digital value. Refer to the electrical characteristics section of the device datasheet for the sampling time value to be applied when converting the internal temperature sensor.

When not in use, the sensor can be put in power-down mode.

Figure 306 shows the block diagram of the temperature sensor.

Figure 306. Temperature sensor channel block diagram



Reading the temperature

To use the sensor:

1. Select the ADC input channels that is connected to V_{SENSE} .
2. Program with the appropriate sampling time (refer to electrical characteristics section of the device datasheet).
3. Set the bit in the ADC_CCR register to wake up the temperature sensor from power-down mode.
4. Start the ADC conversion.
5. Read the resulting V_{SENSE} data in the ADC data register.
6. Calculate the actual temperature using the following formula:

$$\text{Temperature (in } ^\circ\text{C)} = \frac{\text{TS_CAL2_TEMP} - \text{TS_CAL1_TEMP}}{\text{TS_CAL2} - \text{TS_CAL1}} \times (\text{TS_DATA} - \text{TS_CAL1}) + \text{TS_CAL1_TEMP}$$

where:

- TS_CAL2 is the temperature sensor calibration value acquired at TS_CAL2_TEMP.
 - TS_CAL1 is the temperature sensor calibration value acquired at TS_CAL1_TEMP.
 - TS_DATA is the actual temperature sensor output value converted by ADC.
- Refer to the device datasheet for more information about TS_CAL1 and TS_CAL2 calibration points.

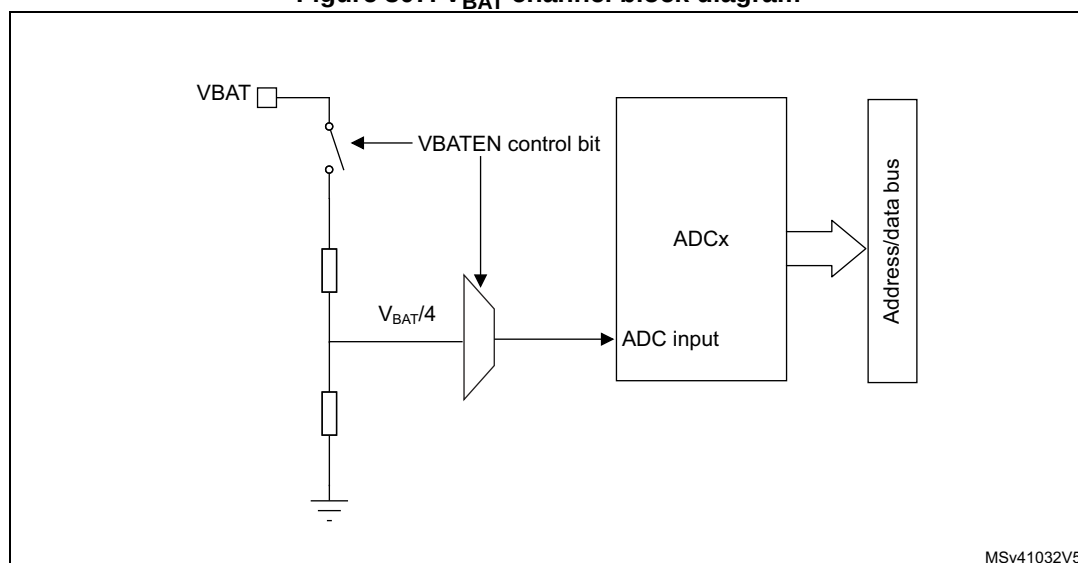
Note: *The sensor has a startup time after waking from power-down mode before it can output V_{SENSE} at the correct level. The ADC also has a startup time after power-on, so to minimize the delay, the ADEN and bits should be set at the same time.*

28.4.32 V_{BAT} supply monitoring

The VBATEN bit in the ADC_CCR register is used to switch to the battery voltage. As the V_{BAT} voltage could be higher than V_{DDA} , to ensure the correct operation of the ADC, the V_{BAT} pin is internally connected to a bridge divider by 4. This bridge is automatically enabled when VBATEN is set, to connect $V_{\text{BAT}}/4$ to the ADC input channels (see *Table: ADC interconnection* in [Section 28.4.2: ADC pins and internal signals](#) for more details). As a consequence, the converted digital value is one third of the V_{BAT} voltage. To prevent any unwanted consumption on the battery, it is recommended to enable the bridge divider only when needed, for ADC conversion.

Refer to the electrical characteristics of the device datasheet for the sampling time value to be applied when converting the $V_{\text{BAT}}/4$ voltage.

[Figure 307](#) shows the block diagram of the V_{BAT} sensing feature.

Figure 307. V_{BAT} channel block diagram

1. The VBATEN bit must be set to enable the conversion of internal channel for V_{BAT} .

28.4.33 Monitoring the internal voltage reference

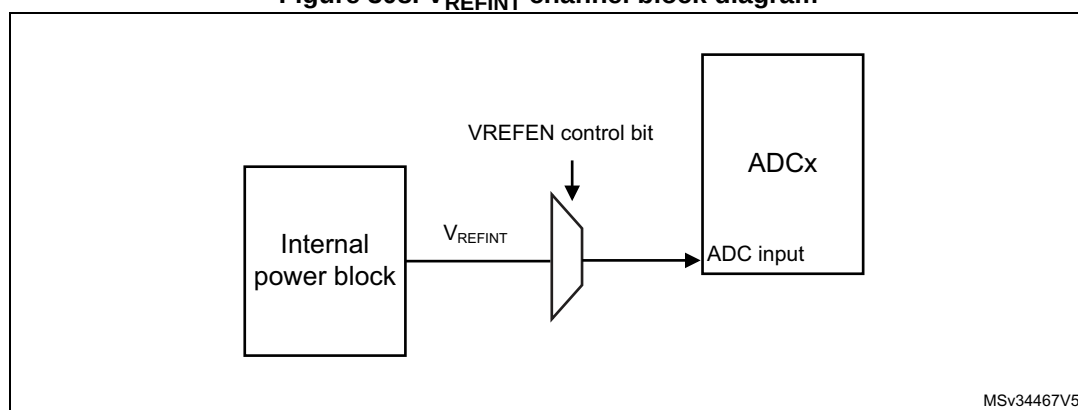
It is possible to monitor the internal voltage reference (V_{REFINT}) to have a reference point for evaluating the ADC V_{REF+} voltage level.

Refer to *Table: ADC interconnection* in [Section 28.4.2: ADC pins and internal signals](#) for details on the ADC input channels to which the internal voltage reference is internally connected.

Refer to the electrical characteristics section of the product datasheet for the sampling time value to be applied when converting the internal voltage reference voltage.

[Figure 308](#) shows the block diagram of the V_{REFINT} sensing feature.

Note: V_{REFINT} must not be converted by two ADCs at the same time.

Figure 308. V_{REFINT} channel block diagram

1. The VREFEN bit into ADC_CCR register must be set to enable the conversion of internal channels (V_{REFINT}).

Calculating the actual V_{REF+} voltage using the internal reference voltage

V_{REF+} voltage may be subject to variations or not precisely known. The embedded internal reference voltage V_{REFINT} and its calibration data acquired by the ADC during the manufacturing process at V_{REF+_charac} can be used to evaluate the actual V_{REF+} voltage level.

The following formula gives the actual V_{REF+} voltage supplying the device:

$$V_{REF+} = V_{REF+_Charac} \times VREFINT_CAL / VREFINT_DATA$$

Where:

- V_{REF+_Charac} is the value of V_{REF+} voltage characterized at V_{REFINT} during the manufacturing process. It is specified in the device datasheet.
- $VREFINT_CAL$ is the $VREFINT$ calibration value
- $VREFINT_DATA$ is the actual $VREFINT$ output value converted by ADC

Converting a supply-relative ADC measurement to an absolute voltage value

The ADC is designed to deliver a digital value corresponding to the ratio between V_{REF+} and the voltage applied on the converted channel.

For applications where V_{REF+} value is unknown and ADC converted values are right-aligned. In this case, it is necessary to convert this ratio into a voltage independent from V_{REF+} :

$$V_{CHANNELx} = \frac{V_{REF+}}{FULL_SCALE} \times ADC_DATA$$

By replacing V_{REF+} by the formula provided above, the absolute voltage value is given by the following formula

$$V_{CHANNELx} = \frac{V_{REF+_Charac} \times VREFINT_CAL \times ADC_DATA}{VREFINT_DATA \times FULL_SCALE}$$

Where:

- V_{REF+_Charac} is the value of V_{REF+} voltage characterized at V_{REFINT} during the manufacturing process.
- $VREFINT_CAL$ is the $VREFINT$ calibration value
- ADC_DATA is the value measured by the ADC on channel x (right-aligned)
- $VREFINT_DATA$ is the actual $VREFINT$ output value converted by the ADC
- $FULL_SCALE$ is the maximum digital value of the ADC output. For example with 12-bit resolution, it is $2^{12} - 1 = 4095$ or with 8-bit resolution, $2^8 - 1 = 255$.

Note: *If ADC measurements are done using an output format other than 12-bit right-aligned, all the parameters must first be converted to a compatible format before the calculation is done.*

28.4.34 Monitoring the supply voltage

ADC2/3 is connected to the internal supply voltage. To use the ADC to measure this voltage, enable the connection through ADC option register.

28.5 ADC interrupts

For each ADC, an interrupt can be generated:

- After ADC power-up, when the ADC is ready (flag ADRDY)
- On the end of any conversion for regular groups (flag EOC)
- On the end of a sequence of conversion for regular groups (flag EOS)
- On the end of any conversion for injected groups (flag JEOC)
- On the end of a sequence of conversion for injected groups (flag JEOS)
- When an analog watchdog detection occurs (flag AWD1, AWD2 and AWD3)
- When the end of sampling phase occurs (flag EOSMP)
- When the data overrun occurs (flag OVR)
- When the injected sequence context queue overflows (flag JQOVF)

Separate interrupt enable bits are available for flexibility.

Table 292. ADC interrupts

Interrupt vector	Interrupt event	Event flag	Enable Control bit	Interrupt clear method	Exit from Sleep mode	Exit from Stop, Standby mode
ADC	ADC ready	ADRDY	ADRDYIE	Set by hardware and cleared by software	Yes	No
	End of conversion of a regular group	EOC	EOCIE			
	End of conversion sequence of a regular group	EOS	EOSIE			
	End of conversion of an injected group	JEOC	JEOCIE			
	End of conversion sequence of an injected group	JEOS	JEOSIE			
	Analog watchdog 1 status bit is set	AWD1	AWD1IE			
	Analog watchdog 2 status bit is set	AWD2	AWD2IE			
	Analog watchdog 3 status bit is set	AWD3	AWD3IE			
	End of sampling phase	EOSMP	EOSMPIE			
	Overrun	OVR	OVRIE			
	Injected context queue overflows	JQOVF	JQOVFIE			

28.6 ADC registers (for each ADC)

Refer to [Section 1.2 on page 119](#) for a list of abbreviations used in register descriptions.

28.6.1 ADC interrupt and status register (ADC_ISR)

Address offset: 0x00

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	Res.	Res.	Res.	JQOVF	AWD3	AWD2	AWD1	JEOS	JEOS	OVR	EOS	EOC	EOSMP	ADRDY
					rc_w1	rc_w1	rc_w1	rc_w1	rc_w1	rc_w1	rc_w1	rc_w1	rc_w1	rc_w1	rc_w1

Bits 31:11 Reserved, must be kept at reset value.

Bit 10 JQOVF: Injected context queue overflow

This bit is set by hardware when an Overflow of the Injected Queue of Context occurs. It is cleared by software writing 1 to it. Refer to [Section 28.4.21: Queue of context for injected conversions](#) for more information.

0: No injected context queue overflow occurred (or the flag event was already acknowledged and cleared by software)

1: Injected context queue overflow has occurred

Bit 9 AWD3: Analog watchdog 3 flag

This bit is set by hardware when the converted voltage crosses the values programmed in the fields LT3[7:0] and HT3[7:0] of ADC_TR3 register. It is cleared by software writing 1 to it.

0: No analog watchdog 3 event occurred (or the flag event was already acknowledged and cleared by software)

1: Analog watchdog 3 event occurred

Bit 8 AWD2: Analog watchdog 2 flag

This bit is set by hardware when the converted voltage crosses the values programmed in the fields LT2[7:0] and HT2[7:0] of ADC_TR2 register. It is cleared by software writing 1 to it.

0: No analog watchdog 2 event occurred (or the flag event was already acknowledged and cleared by software)

1: Analog watchdog 2 event occurred

Bit 7 AWD1: Analog watchdog 1 flag

This bit is set by hardware when the converted voltage crosses the values programmed in the fields LT1[11:0] and HT1[11:0] of ADC_TR1 register. It is cleared by software writing 1 to it.

0: No analog watchdog 1 event occurred (or the flag event was already acknowledged and cleared by software)

1: Analog watchdog 1 event occurred

Bit 6 JEOS: Injected channel end of sequence flag

This bit is set by hardware at the end of the conversions of all injected channels in the group. It is cleared by software writing 1 to it.

0: Injected conversion sequence not complete (or the flag event was already acknowledged and cleared by software)

1: Injected conversions complete

Bit 5 JEOC: Injected channel end of conversion flag

This bit is set by hardware at the end of each injected conversion of a channel when a new data is available in the corresponding ADC_JDRy register. It is cleared by software writing 1 to it or by reading the corresponding ADC_JDRy register

0: Injected channel conversion not complete (or the flag event was already acknowledged and cleared by software)

1: Injected channel conversion complete

Bit 4 OVR: ADC overrun

This bit is set by hardware when an overrun occurs on a regular channel, meaning that a new conversion has completed while the EOC flag was already set. It is cleared by software writing 1 to it.

0: No overrun occurred (or the flag event was already acknowledged and cleared by software)

1: Overrun has occurred

Bit 3 EOS: End of regular sequence flag

This bit is set by hardware at the end of the conversions of a regular sequence of channels. It is cleared by software writing 1 to it.

0: Regular Conversions sequence not complete (or the flag event was already acknowledged and cleared by software)

1: Regular Conversions sequence complete

Bit 2 EOC: End of conversion flag

This bit is set by hardware at the end of each regular conversion of a channel when a new data is available in the ADC_DR register. It is cleared by software writing 1 to it or by reading the ADC_DR register

0: Regular channel conversion not complete (or the flag event was already acknowledged and cleared by software)

1: Regular channel conversion complete

Bit 1 EOSMP: End of sampling flag

This bit is set by hardware during the conversion of any channel (only for regular channels), at the end of the sampling phase.

0: not at the end of the sampling phase (or the flag event was already acknowledged and cleared by software)

1: End of sampling phase reached

Bit 0 ADRDY: ADC ready

This bit is set by hardware after the ADC has been enabled (ADEN = 1) and when the ADC reaches a state where it is ready to accept conversion requests.

It is cleared by software writing 1 to it.

0: ADC not yet ready to start conversion (or the flag event was already acknowledged and cleared by software)

1: ADC is ready to start conversion

28.6.2 ADC interrupt enable register (ADC_IER)

Address offset: 0x04

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	Res.	Res.	Res.	JQOVFIE	AWD3IE	AWD2IE	AWD1IE	JEOSIE	JEOSIE	OVRIE	EOSIE	EOCIE	EOSMP IE	ADRDY IE
					rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bits 31:11 Reserved, must be kept at reset value.

Bit 10 JQOVFIE: Injected context queue overflow interrupt enable

This bit is set and cleared by software to enable/disable the Injected Context Queue Overflow interrupt.

0: Injected Context Queue Overflow interrupt disabled

1: Injected Context Queue Overflow interrupt enabled. An interrupt is generated when the JQOVF bit is set.

Note: The software is allowed to write this bit only when JADSTART = 0 (which ensures that no injected conversion is ongoing).

Bit 9 AWD3IE: Analog watchdog 3 interrupt enable

This bit is set and cleared by software to enable/disable the analog watchdog 2 interrupt.

0: Analog watchdog 3 interrupt disabled

1: Analog watchdog 3 interrupt enabled

Note: The software is allowed to write this bit only when ADSTART = 0 and JADSTART = 0 (which ensures that no conversion is ongoing).

Bit 8 AWD2IE: Analog watchdog 2 interrupt enable

This bit is set and cleared by software to enable/disable the analog watchdog 2 interrupt.

0: Analog watchdog 2 interrupt disabled

1: Analog watchdog 2 interrupt enabled

Note: The software is allowed to write this bit only when ADSTART = 0 and JADSTART = 0 (which ensures that no conversion is ongoing).

Bit 7 AWD1IE: Analog watchdog 1 interrupt enable

This bit is set and cleared by software to enable/disable the analog watchdog 1 interrupt.

0: Analog watchdog 1 interrupt disabled

1: Analog watchdog 1 interrupt enabled

Note: The software is allowed to write this bit only when ADSTART = 0 and JADSTART = 0 (which ensures that no conversion is ongoing).

Bit 6 JEOSIE: End of injected sequence of conversions interrupt enable

This bit is set and cleared by software to enable/disable the end of injected sequence of conversions interrupt.

0: JEOS interrupt disabled

1: JEOS interrupt enabled. An interrupt is generated when the JEOS bit is set.

Note: The software is allowed to write this bit only when JADSTART = 0 (which ensures that no injected conversion is ongoing).

Bit 5 JEOCIE: End of injected conversion interrupt enable

This bit is set and cleared by software to enable/disable the end of an injected conversion interrupt.

0: JEOC interrupt disabled.

1: JEOC interrupt enabled. An interrupt is generated when the JEOC bit is set.

Note: The software is allowed to write this bit only when JADSTART = 0 (which ensures that no injected conversion is ongoing).

Bit 4 OVRIE: Overrun interrupt enable

This bit is set and cleared by software to enable/disable the Overrun interrupt of a regular conversion.

0: Overrun interrupt disabled

1: Overrun interrupt enabled. An interrupt is generated when the OVR bit is set.

Note: The software is allowed to write this bit only when ADSTART = 0 (which ensures that no regular conversion is ongoing).

Bit 3 EOSIE: End of regular sequence of conversions interrupt enable

This bit is set and cleared by software to enable/disable the end of regular sequence of conversions interrupt.

0: EOS interrupt disabled

1: EOS interrupt enabled. An interrupt is generated when the EOS bit is set.

Note: The software is allowed to write this bit only when ADSTART = 0 (which ensures that no regular conversion is ongoing).

Bit 2 EOCIE: End of regular conversion interrupt enable

This bit is set and cleared by software to enable/disable the end of a regular conversion interrupt.

0: EOC interrupt disabled.

1: EOC interrupt enabled. An interrupt is generated when the EOC bit is set.

Note: The software is allowed to write this bit only when ADSTART = 0 (which ensures that no regular conversion is ongoing).

Bit 1 EOSMPIE: End of sampling flag interrupt enable for regular conversions

This bit is set and cleared by software to enable/disable the end of the sampling phase interrupt for regular conversions.

0: EOSMP interrupt disabled.

1: EOSMP interrupt enabled. An interrupt is generated when the EOSMP bit is set.

Note: The software is allowed to write this bit only when ADSTART = 0 (which ensures that no regular conversion is ongoing).

Bit 0 ADRDYIE: ADC ready interrupt enable

This bit is set and cleared by software to enable/disable the ADC Ready interrupt.

0: ADRDY interrupt disabled

1: ADRDY interrupt enabled. An interrupt is generated when the ADRDY bit is set.

Note: The software is allowed to write this bit only when ADSTART = 0 and JADSTART = 0 (which ensures that no conversion is ongoing).

28.6.3 ADC control register (ADC_CR)

Address offset: 0x08

Reset value: 0x2000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
ADCAL	ADCALDIF	DEEPPWD	ADVREGEN	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
rs	rw	rw	rw												
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	JADSTP	ADSTP	JADSTART	ADSTART	ADDIS	ADEN
										rs	rs	rs	rs	rs	rs

Bit 31 ADCAL: ADC calibration

This bit is set by software to start the calibration of the ADC. Program first the bit ADCALDIF to determine if this calibration applies for single-ended or Differential inputs mode.

It is cleared by hardware after calibration is complete.

0: Calibration complete

1: Write 1 to calibrate the ADC. Read at 1 means that a calibration in progress.

Note: The software is allowed to launch a calibration by setting ADCAL only when ADEN = 0.

The software is allowed to update the calibration factor by writing ADC_CALFACT only when ADEN = 1 and ADSTART = 0 and JADSTART = 0 (ADC enabled and no conversion is ongoing)

Bit 30 ADCALDIF: Differential mode for calibration

This bit is set and cleared by software to configure the single-ended or Differential inputs mode for the calibration.

0: Writing ADCAL launches a calibration in single-ended inputs mode.

1: Writing ADCAL launches a calibration in Differential inputs mode.

Note: The software is allowed to write this bit only when the ADC is disabled and is not calibrating (ADCAL = 0, JADSTART = 0, JADSTP = 0, ADSTART = 0, ADSTP = 0, ADDIS = 0 and ADEN = 0).

Bit 29 DEEPPWD: Deep-power-down enable

This bit is set and cleared by software to put the ADC in Deep-power-down mode.

0: ADC not in Deep-power down

1: ADC in Deep-power-down (default reset state)

Note: The software is allowed to write this bit only when the ADC is disabled (ADCAL = 0, JADSTART = 0, JADSTP = 0, ADSTART = 0, ADSTP = 0, ADDIS = 0 and ADEN = 0).

Bit 28 ADVREGEN: ADC voltage regulator enable

This bit is set by software to enable the ADC voltage regulator.

Before performing any operation such as launching a calibration or enabling the ADC, the ADC voltage regulator must first be enabled and the software must wait for the regulator start-up time.

0: ADC Voltage regulator disabled

1: ADC Voltage regulator enabled.

For more details about the ADC voltage regulator enable and disable sequences, refer to [Section 28.4.6: ADC Deep-power-down mode \(DEEPPWD\) and ADC voltage regulator \(ADVREGEN\)](#).

The software can program this bit field only when the ADC is disabled (ADCAL = 0, JADSTART = 0, ADSTART = 0, ADSTP = 0, ADDIS = 0 and ADEN = 0).

Bits 27:6 Reserved, must be kept at reset value.

Bit 5 JADSTP: ADC stop of injected conversion command

This bit is set by software to stop and discard an ongoing injected conversion (JADSTP Command).

It is cleared by hardware when the conversion is effectively discarded and the ADC injected sequence and triggers can be re-configured. The ADC is then ready to accept a new start of injected conversions (JADSTART command).

0: No ADC stop injected conversion command ongoing

1: Write 1 to stop injected conversions ongoing. Read 1 means that an ADSTP command is in progress.

Note: The software is allowed to set JADSTP only when JADSTART = 1 and ADDIS = 0 (ADC is enabled and eventually converting an injected conversion and there is no pending request to disable the ADC)

In auto-injection mode (JAUTO = 1), setting ADSTP bit aborts both regular and injected conversions (do not use JADSTP)

Bit 4 ADSTP: ADC stop of regular conversion command

This bit is set by software to stop and discard an ongoing regular conversion (ADSTP Command).

It is cleared by hardware when the conversion is effectively discarded and the ADC regular sequence and triggers can be re-configured. The ADC is then ready to accept a new start of regular conversions (ADSTART command).

0: No ADC stop regular conversion command ongoing

1: Write 1 to stop regular conversions ongoing. Read 1 means that an ADSTP command is in progress.

Note: The software is allowed to set ADSTP only when ADSTART = 1 and ADDIS = 0 (ADC is enabled and eventually converting a regular conversion and there is no pending request to disable the ADC).

In auto-injection mode (JAUTO = 1), setting ADSTP bit aborts both regular and injected conversions (do not use JADSTP).

In dual ADC regular simultaneous mode and interleaved mode, the bit ADSTP of the master ADC must be used to stop regular conversions. The other ADSTP bit is inactive.

Bit 3 JADSTART: ADC start of injected conversion

This bit is set by software to start ADC conversion of injected channels. Depending on the configuration bits JEXTEN, a conversion immediately starts (software trigger configuration) or once an injected hardware trigger event occurs (hardware trigger configuration).

It is cleared by hardware:

- in single conversion mode when software trigger is selected (JEXTSEL = 0x0): at the assertion of the End of Injected Conversion Sequence (JEOS) flag.
- in all cases: after the execution of the JADSTP command, at the same time that JADSTP is cleared by hardware.

0: No ADC injected conversion is ongoing.

1: Write 1 to start injected conversions. Read 1 means that the ADC is operating and eventually converting an injected channel.

Note: The software is allowed to set JADSTART only when ADEN = 1 and ADDIS = 0 (ADC is enabled and there is no pending request to disable the ADC).

In auto-injection mode (JAUTO = 1), regular and auto-injected conversions are started by setting bit ADSTART (JADSTART must be kept cleared)

Bit 2 ADSTART: ADC start of regular conversion

This bit is set by software to start ADC conversion of regular channels. Depending on the configuration bits EXTEN, a conversion immediately starts (software trigger configuration) or once a regular hardware trigger event occurs (hardware trigger configuration).

It is cleared by hardware:

- in single conversion mode when software trigger is selected (EXTSEL = 0x0): at the assertion of the End of Regular Conversion Sequence (EOS) flag.
- in all cases: after the execution of the ADSTP command, at the same time that ADSTP is cleared by hardware.

0: No ADC regular conversion is ongoing.

1: Write 1 to start regular conversions. Read 1 means that the ADC is operating and eventually converting a regular channel.

Note: The software is allowed to set ADSTART only when ADEN = 1 and ADDIS = 0 (ADC is enabled and there is no pending request to disable the ADC)

In auto-injection mode (JAUTO = 1), regular and auto-injected conversions are started by setting bit ADSTART (JADSTART must be kept cleared)

Bit 1 ADDIS: ADC disable command

This bit is set by software to disable the ADC (ADDIS command) and put it into power-down state (OFF state).

It is cleared by hardware once the ADC is effectively disabled (ADEN is also cleared by hardware at this time).

0: no ADDIS command ongoing

1: Write 1 to disable the ADC. Read 1 means that an ADDIS command is in progress.

Note: The software is allowed to set ADDIS only when ADEN = 1 and both ADSTART = 0 and JADSTART = 0 (which ensures that no conversion is ongoing)

Bit 0 **ADEN**: ADC enable control

This bit is set by software to enable the ADC. The ADC is effectively ready to operate once the flag ADRDY has been set.

It is cleared by hardware when the ADC is disabled, after the execution of the ADDIS command.

0: ADC is disabled (OFF state)

1: Write 1 to enable the ADC.

Note: The software is allowed to set ADEN only when all bits of ADC_CR registers are 0 (ADCAL = 0, JADSTART = 0, ADSTART = 0, ADSTP = 0, ADDIS = 0 and ADEN = 0) except for bit ADVREGEN which must be 1 (and the software must have wait for the startup time of the voltage regulator)

28.6.4 ADC configuration register (ADC_CFGR)

Address offset: 0x0C

Reset value: 0x8000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
JQDIS	AWD1CH[4:0]					JAUTO	JAWD1EN	AWD1EN	AWD1SGL	JQM	JDISCEN	DISCNUM[2:0]		DISCEN	
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
ALIGN	AUTDLY	CONT	OVRMOD	EXTEN[1:0]		EXTSEL4	EXTSEL3	EXTSEL2	EXTSEL1	EXTSEL0	RES[1:0]		Res.	DMACFG	DMAEN
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw		rw	rw

Bit 31 **JQDIS**: Injected queue disable

This bit is set and cleared by software to disable the injected queue mechanism:

0: Injected queue enabled

1: Injected queue disabled

Note: The software is allowed to write this bit only when ADSTART = 0 and JADSTART = 0 (which ensures that no regular nor injected conversion is ongoing).

A set or reset of JQDIS bit causes the injected queue to be flushed and the ADC_JSQR register is cleared.

Bits 30:26 **AWD1CH[4:0]**: Analog watchdog 1 channel selection

These bits are set and cleared by software. They select the input channel to be guarded by the analog watchdog.

00000: ADC analog input channel 0 monitored by AWD1

00001: ADC analog input channel 1 monitored by AWD1

.....

10011: ADC analog input channel 19 monitored by AWD1

others: reserved, must not be used

Note: Some channels are not connected physically. Keep the corresponding AWD1CH[4:0] setting to the reset value.

The channel selected by AWD1CH must be also selected into the SQi or JSQi bits.

The software is allowed to write these bits only when ADSTART = 0 and JADSTART = 0 (which ensures that no conversion is ongoing).

Bit 25 **JAUTO**: Automatic injected group conversion

This bit is set and cleared by software to enable/disable automatic injected group conversion after regular group conversion.

0: Automatic injected group conversion disabled

1: Automatic injected group conversion enabled

Note: The software is allowed to write this bit only when ADSTART = 0 and JADSTART = 0 (which ensures that no regular nor injected conversion is ongoing).

When dual mode is enabled (DUAL bits in ADC_CCR register are not equal to zero), the bit JAUTO of the slave ADC is no more writable and its content is equal to the bit JAUTO of the master ADC.

Bit 24 **JAWD1EN**: Analog watchdog 1 enable on injected channels

This bit is set and cleared by software

0: Analog watchdog 1 disabled on injected channels

1: Analog watchdog 1 enabled on injected channels

Note: The software is allowed to write this bit only when JADSTART = 0 (which ensures that no injected conversion is ongoing).

Bit 23 **AWD1EN**: Analog watchdog 1 enable on regular channels

This bit is set and cleared by software

0: Analog watchdog 1 disabled on regular channels

1: Analog watchdog 1 enabled on regular channels

Note: The software is allowed to write this bit only when ADSTART = 0 (which ensures that no regular conversion is ongoing).

Bit 22 **AWD1SGL**: Enable the watchdog 1 on a single channel or on all channels

This bit is set and cleared by software to enable the analog watchdog on the channel identified by the AWD1CH[4:0] bits or on all the channels

0: Analog watchdog 1 enabled on all channels

1: Analog watchdog 1 enabled on a single channel

Note: The software is allowed to write these bits only when ADSTART = 0 and JADSTART = 0 (which ensures that no conversion is ongoing).

Bit 21 JQM: ADC_JSQR queue mode

This bit is set and cleared by software.

It defines how an empty Queue is managed.

0: ADC_JSQR mode 0: The Queue is never empty and maintains the last written configuration into ADC_JSQR.

1: ADC_JSQR mode 1: The Queue can be empty and when this occurs, the software and hardware triggers of the injected sequence are both internally disabled just after the completion of the last valid injected sequence.

Refer to [Section 28.4.21: Queue of context for injected conversions](#) for more information.

Note: The software is allowed to write this bit only when JADSTART = 0 (which ensures that no injected conversion is ongoing).

When dual mode is enabled (DUAL bits in ADC_CCR register are not equal to zero), the bit JQM of the slave ADC is no more writable and its content is equal to the bit JQM of the master ADC.

Bit 20 JDISCEN: Discontinuous mode on injected channels

This bit is set and cleared by software to enable/disable discontinuous mode on the injected channels of a group.

0: Discontinuous mode on injected channels disabled

1: Discontinuous mode on injected channels enabled

Note: The software is allowed to write this bit only when JADSTART = 0 (which ensures that no injected conversion is ongoing).

It is not possible to use both auto-injected mode and discontinuous mode simultaneously: the bits DISCEN and JDISCEN must be kept cleared by software when JAUTO is set.

When dual mode is enabled (bits DUAL of ADC_CCR register are not equal to zero), the bit JDISCEN of the slave ADC is no more writable and its content is equal to the bit JDISCEN of the master ADC.

Bits 19:17 DISCNUM[2:0]: Discontinuous mode channel count

These bits are written by software to define the number of regular channels to be converted in discontinuous mode, after receiving an external trigger.

000: 1 channel

001: 2 channels

...

111: 8 channels

Note: The software is allowed to write these bits only when ADSTART = 0 (which ensures that no regular conversion is ongoing).

When dual mode is enabled (DUAL bits in ADC_CCR register are not equal to zero), the bits DISCNUM[2:0] of the slave ADC are no more writable and their content is equal to the bits DISCNUM[2:0] of the master ADC.

Bit 16 DISCEN: Discontinuous mode for regular channels

This bit is set and cleared by software to enable/disable discontinuous mode for regular channels.

0: Discontinuous mode for regular channels disabled

1: Discontinuous mode for regular channels enabled

Note: It is not possible to have both discontinuous mode and continuous mode enabled: it is forbidden to set both DISCEN = 1 and CONT = 1.

It is not possible to use both auto-injected mode and discontinuous mode simultaneously: the bits DISCEN and JDISCEN must be kept cleared by software when JAUTO is set.

The software is allowed to write this bit only when ADSTART = 0 (which ensures that no regular conversion is ongoing).

When dual mode is enabled (DUAL bits in ADC_CCR register are not equal to zero), the bit DISCEN of the slave ADC is no more writable and its content is equal to the bit DISCEN of the master ADC.

Bit 15 ALIGN: Data alignment

This bit is set and cleared by software to select right or left alignment. Refer to [Section : Data register, data alignment and offset \(ADC_DR, OFFSET, OFFSET_CH, ALIGN\)](#).

- 0: Right alignment
- 1: Left alignment

Note: The software is allowed to write this bit only when ADSTART = 0 and JADSTART = 0 (which ensures that no conversion is ongoing).

Bit 14 AUTDLY: Delayed conversion mode

This bit is set and cleared by software to enable/disable the Auto Delayed Conversion mode:

- 0: Auto-delayed conversion mode off
- 1: Auto-delayed conversion mode on

Note: The software is allowed to write this bit only when ADSTART = 0 and JADSTART = 0 (which ensures that no conversion is ongoing).

When dual mode is enabled (DUAL bits in ADC_CCR register are not equal to zero), the bit AUTDLY of the slave ADC is no more writable and its content is equal to the bit AUTDLY of the master ADC.

Bit 13 CONT: Single / continuous conversion mode for regular conversions

This bit is set and cleared by software. If it is set, regular conversion takes place continuously until it is cleared.

- 0: Single conversion mode
- 1: Continuous conversion mode

Note: It is not possible to have both discontinuous mode and continuous mode enabled: it is forbidden to set both DISCEN = 1 and CONT = 1.

The software is allowed to write this bit only when ADSTART = 0 (which ensures that no regular conversion is ongoing).

When dual mode is enabled (DUAL bits in ADC_CCR register are not equal to zero), the bit CONT of the slave ADC is no more writable and its content is equal to the bit CONT of the master ADC.

Bit 12 OVRMOD: Overrun mode

This bit is set and cleared by software and configure the way data overrun is managed.

- 0: ADC_DR register is preserved with the old data when an overrun is detected.
- 1: ADC_DR register is overwritten with the last conversion result when an overrun is detected.

Note: The software is allowed to write this bit only when ADSTART = 0 (which ensures that no regular conversion is ongoing).

Bits 11:10 EXTEN[1:0]: External trigger enable and polarity selection for regular channels

These bits are set and cleared by software to select the external trigger polarity and enable the trigger of a regular group.

- 00: Hardware trigger detection disabled (conversions can be launched by software)
- 01: Hardware trigger detection on the rising edge
- 10: Hardware trigger detection on the falling edge
- 11: Hardware trigger detection on both the rising and falling edges

Note: The software is allowed to write these bits only when ADSTART = 0 (which ensures that no regular conversion is ongoing).

Bits 9:5 **EXTSEL[4:0]**: External trigger selection for regular group

These bits select the external event used to trigger the start of conversion of a regular group:

00000: adc_ext_trg0
 00001: adc_ext_trg1
 00010: adc_ext_trg2
 00011: adc_ext_trg3
 00100: adc_ext_trg4
 00101: adc_ext_trg5
 00110: adc_ext_trg6
 00111: adc_ext_trg7

...

11111: adc_ext_trg31

Note: The software is allowed to write these bits only when ADSTART = 0 (which ensures that no regular conversion is ongoing).

Bits 4:3 **RES[1:0]**: Data resolution

These bits are written by software to select the resolution of the conversion.

00: 12-bit
 01: 10-bit
 10: 8-bit
 11: 6-bit

Note: The software is allowed to write these bits only when ADSTART = 0 and JADSTART = 0 (which ensures that no conversion is ongoing).

Bit 2 Reserved, must be kept at reset value.

Bit 1 **DMACFG**: Direct memory access configuration

This bit is set and cleared by software to select between two DMA modes of operation and is effective only when DMAEN = 1.

0: DMA One Shot mode selected

1: DMA Circular mode selected

For more details, refer to [Section : Managing conversions using the DMA](#)

Note: The software is allowed to write this bit only when ADSTART = 0 and JADSTART = 0 (which ensures that no conversion is ongoing).

In dual-ADC modes, this bit is not relevant and replaced by control bit DMACFG of the ADC_CCR register.

Bit 0 **DMAEN**: Direct memory access enable

This bit is set and cleared by software to enable the generation of DMA requests. This allows to use the DMA to manage automatically the converted data. For more details, refer to [Section : Managing conversions using the DMA](#).

0: DMA disabled

1: DMA enabled

Note: The software is allowed to write this bit only when ADSTART = 0 and JADSTART = 0 (which ensures that no conversion is ongoing).

In dual-ADC modes, this bit is not relevant and replaced by control bits MDMA[1:0] of the ADC_CCR register.

28.6.5 ADC configuration register 2 (ADC_CFGR2)

Address offset: 0x10

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	SMPTRIG	BULB	SWTRIG	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
				rw	rw	rw									
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	Res.	Res.	Res.	ROVSM	TROVS	OVSS[3:0]				OVSR[2:0]			JOVSE	ROVSE
					rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bits 31:28 Reserved, must be kept at reset value.

Bit 27 **SMPTRIG**: Sampling time control trigger mode

This bit is set and cleared by software to enable the sampling time control trigger mode.

0: Sampling time control trigger mode disabled

1: Sampling time control trigger mode enabled

The sampling time starts on the trigger rising edge, and the conversion on the trigger falling edge.

EXTEN bit should be set to 01. BULB bit must not be set when the SMPTRIG bit is set.

When EXTEN bit is set to 00, set SWTRIG to start the sampling and clear SWTRIG bit to start the conversion.

Note: The software is allowed to write this bit only when ADSTART = 0 (which ensures that no conversion is ongoing).

Bit 26 **BULB**: Bulb sampling mode

This bit is set and cleared by software to enable the bulb sampling mode.

0: Bulb sampling mode disabled

1: Bulb sampling mode enabled. The sampling period starts just after the previous end of conversion.

SAMPTRIG bit must not be set when the BULB bit is set.

The very first ADC conversion is performed with the sampling time specified in SMPx bits.

Note: The software is allowed to write this bit only when ADSTART = 0 (which ensures that no conversion is ongoing).

Bit 25 **SWTRIG**: Software trigger bit for sampling time control trigger mode

This bit is set and cleared by software to enable the bulb sampling mode.

0: Software trigger starts the conversion for sampling time control trigger mode

1: Software trigger starts the sampling for sampling time control trigger mode

Note: The software is allowed to write this bit only when ADSTART = 0 (which ensures that no conversion is ongoing).

Bits 24:17 Reserved, must be kept at reset value.

Bits 16:11 Reserved, must be kept at reset value.

Bit 10 **ROVSM**: Regular oversampling mode

This bit is set and cleared by software to select the regular oversampling mode.

0: Continued mode: When injected conversions are triggered, the oversampling is temporary stopped and continued after the injection sequence (oversampling buffer is maintained during injected sequence)

1: Resumed mode: When injected conversions are triggered, the current oversampling is aborted and resumed from start after the injection sequence (oversampling buffer is zeroed by injected sequence start)

Note: The software is allowed to write this bit only when ADSTART = 0 (which ensures that no conversion is ongoing).

Bit 9 **TROVS**: Triggered Regular oversampling

This bit is set and cleared by software to enable triggered oversampling

0: All oversampled conversions for a channel are done consecutively following a trigger

1: Each oversampled conversion for a channel needs a new trigger

Note: The software is allowed to write this bit only when ADSTART = 0 (which ensures that no conversion is ongoing).

Bits 8:5 **OVSS[3:0]**: Oversampling shift

This bitfield is set and cleared by software to define the right shifting applied to the raw oversampling result.

0000: No shift

0001: Shift 1-bit

0010: Shift 2-bits

0011: Shift 3-bits

0100: Shift 4-bits

0101: Shift 5-bits

0110: Shift 6-bits

0111: Shift 7-bits

1000: Shift 8-bits

Other codes reserved

Note: The software is allowed to write these bits only when ADSTART = 0 (which ensures that no conversion is ongoing).

Bits 4:2 **OVSR[2:0]**: Oversampling ratio

This bitfield is set and cleared by software to define the oversampling ratio.

000: 2x

001: 4x

010: 8x

011: 16x

100: 32x

101: 64x

110: 128x

111: 256x

Note: The software is allowed to write these bits only when ADSTART = 0 (which ensures that no conversion is ongoing).

Bit 1 **JOVSE**: Injected oversampling Enable

This bit is set and cleared by software to enable injected oversampling.

0: Injected oversampling disabled

1: Injected oversampling enabled

Note: The software is allowed to write this bit only when ADSTART = 0 and JADSTART = 0 (which ensures that no conversion is ongoing)

Bit 0 **ROVSE**: Regular oversampling Enable

This bit is set and cleared by software to enable regular oversampling.

0: Regular oversampling disabled

1: Regular oversampling enabled

Note: The software is allowed to write this bit only when ADSTART = 0 and JADSTART = 0 (which ensures that no conversion is ongoing)

28.6.6 ADC sample time register 1 (ADC_SMPR1)

Address offset: 0x14

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
SMPPL US	Res.	SMP9[2:0]			SMP8[2:0]			SMP7[2:0]			SMP6[2:0]			SMP5[2:1]	
rw		rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
SMP5[0]	SMP4[2:0]			SMP3[2:0]			SMP2[2:0]			SMP1[2:0]			SMP0[2:0]		
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bit 31 **SMPPLUS**: Addition of one clock cycle to the sampling time.

1: 2.5 ADC clock cycle sampling time becomes 3.5 ADC clock cycles for the ADC_SMPR1 and ADC_SMPR2 registers.

0: The sampling time remains set to 2.5 ADC clock cycles remains

To make sure no conversion is ongoing, the software is allowed to write this bit only when ADSTART = 0 and JADSTART = 0.

Bit 30 Reserved, must be kept at reset value.

Bits 29:0 **SMPx[2:0]**: Channel x sampling time selection (x = 9 to 0)

These bits are written by software to select the sampling time individually for each channel. During sample cycles, the channel selection bits must remain unchanged.

000: 2.5 ADC clock cycles

001: 6.5 ADC clock cycles

010: 12.5 ADC clock cycles

011: 24.5 ADC clock cycles

100: 47.5 ADC clock cycles

101: 92.5 ADC clock cycles

110: 247.5 ADC clock cycles

111: 640.5 ADC clock cycles

Note: The software is allowed to write these bits only when ADSTART = 0 and JADSTART = 0 (which ensures that no conversion is ongoing).

Some channels are not connected physically. Keep the corresponding SMPx[2:0] setting to the reset value.

28.6.7 ADC sample time register 2 (ADC_SMPR2)

Address offset: 0x18

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res	Res	SMP19[2:0]			SMP18[2:0]			SMP17[2:0]			SMP16[2:0]			SMP15[2:1]	
		rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
SMP15[0]		SMP14[2:0]		SMP13[2:0]			SMP12[2:0]			SMP11[2:0]			SMP10[2:0]		
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bits 31:30 Reserved, must be kept at reset value.

Bits 29:0 **SMPx[2:0]**: Channel x sampling time selection (x = 19 to 10)

These bits are written by software to select the sampling time individually for each channel. During sampling cycles, the channel selection bits must remain unchanged.

000: 2.5 ADC clock cycles

001: 6.5 ADC clock cycles

010: 12.5 ADC clock cycles

011: 24.5 ADC clock cycles

100: 47.5 ADC clock cycles

101: 92.5 ADC clock cycles

110: 247.5 ADC clock cycles

111: 640.5 ADC clock cycles

Note: The software is allowed to write these bits only when $ADSTART = 0$ and $JADSTART = 0$ (which ensures that no conversion is ongoing).

Some channels are not connected physically. Keep the corresponding **SMPx[2:0]** setting to the reset value.

28.6.8 ADC watchdog threshold register 1 (ADC_TR1)

Address offset: 0x20

Reset value: 0x0FFF 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	HT1[11:0]											
				rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	AWDFILT[2:0]			LT1[11:0]											
	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW

Bits 31:28 Reserved, must be kept at reset value.

Bits 27:16 **HT1[11:0]**: Analog watchdog 1 higher threshold

These bits are written by software to define the higher threshold for the analog watchdog 1.

Refer to [Section 28.4.28: Analog window watchdog \(AWD1EN, JAWD1EN, AWD1SGL, AWD1CH, AWD2CH, AWD3CH, AWD_HTx, AWD_LTx, AWDx\)](#)

Note: The software is allowed to write these bits only when $ADSTART = 0$ and $JADSTART = 0$ (which ensures that no conversion is ongoing).

Bit 15 Reserved, must be kept at reset value.

Bits 14:12 **AWDFILT[2:0]**: Analog watchdog filtering parameter

This bit is set and cleared by software.

000: No filtering

001: two consecutive detection generates an AWDx flag or an interrupt

...

111: Eight consecutive detection generates an AWDx flag or an interrupt

Note: The software is allowed to write this bit only when ADSTART = 0 (which ensures that no conversion is ongoing).

Bits 11:0 **LT1[11:0]**: Analog watchdog 1 lower threshold

These bits are written by software to define the lower threshold for the analog watchdog 1.

Refer to [Section 28.4.28: Analog window watchdog \(AWD1EN, JAWD1EN, AWD1SGL, AWD1CH, AWD2CH, AWD3CH, AWD_HTx, AWD_LTx, AWDx\)](#)

Note: The software is allowed to write these bits only when ADSTART = 0 and JADSTART = 0 (which ensures that no conversion is ongoing).

28.6.9 ADC watchdog threshold register 2 (ADC_TR2)

Address offset: 0x24

Reset value: 0x00FF 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	HT2[7:0]							
								rw	rw	rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	LT2[7:0]							
								rw	rw	rw	rw	rw	rw	rw	rw

Bits 31:24 Reserved, must be kept at reset value.

Bits 23:16 **HT2[7:0]**: Analog watchdog 2 higher threshold

These bits are written by software to define the higher threshold for the analog watchdog 2.

Refer to [Section 28.4.28: Analog window watchdog \(AWD1EN, JAWD1EN, AWD1SGL, AWD1CH, AWD2CH, AWD3CH, AWD_HTx, AWD_LTx, AWDx\)](#)

Note: The software is allowed to write these bits only when ADSTART = 0 and JADSTART = 0 (which ensures that no conversion is ongoing).

Bits 15:8 Reserved, must be kept at reset value.

Bits 7:0 **LT2[7:0]**: Analog watchdog 2 lower threshold

These bits are written by software to define the lower threshold for the analog watchdog 2.

Refer to [Section 28.4.28: Analog window watchdog \(AWD1EN, JAWD1EN, AWD1SGL, AWD1CH, AWD2CH, AWD3CH, AWD_HTx, AWD_LTx, AWDx\)](#)

Note: The software is allowed to write these bits only when ADSTART = 0 and JADSTART = 0 (which ensures that no conversion is ongoing).

28.6.10 ADC watchdog threshold register 3 (ADC_TR3)

Address offset: 0x28

Reset value: 0x00FF 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	HT3[7:0]							
								rw	rw	rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	LT3[7:0]							
								rw	rw	rw	rw	rw	rw	rw	rw

Bits 31:24 Reserved, must be kept at reset value.

Bits 23:16 **HT3[7:0]**: Analog watchdog 3 higher threshold

These bits are written by software to define the higher threshold for the analog watchdog 3.

Refer to [Section 28.4.28: Analog window watchdog \(AWD1EN, JAWD1EN, AWD1SGL, AWD1CH, AWD2CH, AWD3CH, AWD_HTx, AWD_LTx, AWDx\)](#)

Note: The software is allowed to write these bits only when ADSTART = 0 and JADSTART = 0 (which ensures that no conversion is ongoing).

Bits 15:8 Reserved, must be kept at reset value.

Bits 7:0 **LT3[7:0]**: Analog watchdog 3 lower threshold

These bits are written by software to define the lower threshold for the analog watchdog 3.

This watchdog compares the 8-bit of LT3 with the 8 MSB of the converted data.

Note: The software is allowed to write these bits only when ADSTART = 0 and JADSTART = 0 (which ensures that no conversion is ongoing).

28.6.11 ADC regular sequence register 1 (ADC_SQR1)

Address offset: 0x30

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	SQ4[4:0]					Res.	SQ3[4:0]					Res.	SQ2[4]
			rw	rw	rw	rw	rw		rw	rw	rw	rw	rw		rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
SQ2[3:0]				Res.	SQ1[4:0]					Res.	Res.	L[3:0]			
rw	rw	rw	rw		rw	rw	rw	rw	rw			rw	rw	rw	rw

Bits 31:29 Reserved, must be kept at reset value.

Bits 28:24 **SQ4[4:0]**: 4th conversion in regular sequence

These bits are written by software with the channel number (0 to 19) assigned as the 4th in the regular conversion sequence.

Note: The software is allowed to write these bits only when ADSTART = 0 (which ensures that no regular conversion is ongoing).

Bit 23 Reserved, must be kept at reset value.

Bits 22:18 **SQ3[4:0]**: 3rd conversion in regular sequence

These bits are written by software with the channel number (0 to 19) assigned as the 3rd in the regular conversion sequence.

Note: The software is allowed to write these bits only when ADSTART = 0 (which ensures that no regular conversion is ongoing).

Bit 17 Reserved, must be kept at reset value.

Bits 16:12 **SQ2[4:0]**: 2nd conversion in regular sequence

These bits are written by software with the channel number (0 to 19) assigned as the 2nd in the regular conversion sequence.

Note: The software is allowed to write these bits only when ADSTART = 0 (which ensures that no regular conversion is ongoing).

Bit 11 Reserved, must be kept at reset value.

Bits 10:6 **SQ1[4:0]**: 1st conversion in regular sequence

These bits are written by software with the channel number (0 to 19) assigned as the 1st in the regular conversion sequence.

Note: The software is allowed to write these bits only when ADSTART = 0 (which ensures that no regular conversion is ongoing).

Bits 5:4 Reserved, must be kept at reset value.

Bits 3:0 **L[3:0]**: Regular channel sequence length

These bits are written by software to define the total number of conversions in the regular channel conversion sequence.

0000: 1 conversion

0001: 2 conversions

...

1111: 16 conversions

Note: The software is allowed to write these bits only when ADSTART = 0 (which ensures that no regular conversion is ongoing).

Note: Some channels are not connected physically and must not be selected for conversion.

28.6.12 ADC regular sequence register 2 (ADC_SQR2)

Address offset: 0x34

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	SQ9[4:0]					Res.	SQ8[4:0]					Res.	SQ7[4]
			rw	rw	rw	rw	rw		rw	rw	rw	rw	rw		rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
SQ7[3:0]				Res.	SQ6[4:0]					Res.	SQ5[4:0]				
rw	rw	rw	rw		rw	rw	rw	rw	rw		rw	rw	rw	rw	rw

Bits 31:29 Reserved, must be kept at reset value.

Bits 28:24 **SQ9[4:0]**: 9th conversion in regular sequence

These bits are written by software with the channel number (0 to 19) assigned as the 9th in the regular conversion sequence.

Note: The software is allowed to write these bits only when ADSTART = 0 (which ensures that no regular conversion is ongoing).

Bit 23 Reserved, must be kept at reset value.

Bits 22:18 **SQ8[4:0]**: 8th conversion in regular sequence

These bits are written by software with the channel number (0 to 19) assigned as the 8th in the regular conversion sequence

Note: The software is allowed to write these bits only when ADSTART = 0 (which ensures that no regular conversion is ongoing).

Bit 17 Reserved, must be kept at reset value.

Bits 16:12 **SQ7[4:0]**: 7th conversion in regular sequence

These bits are written by software with the channel number (0 to 19) assigned as the 7th in the regular conversion sequence.

Note: The software is allowed to write these bits only when ADSTART = 0 (which ensures that no regular conversion is ongoing).

Bit 11 Reserved, must be kept at reset value.

Bits 10:6 **SQ6[4:0]**: 6th conversion in regular sequence

These bits are written by software with the channel number (0 to 19) assigned as the 6th in the regular conversion sequence.

Note: The software is allowed to write these bits only when ADSTART = 0 (which ensures that no regular conversion is ongoing).

Bit 5 Reserved, must be kept at reset value.

Bits 4:0 **SQ5[4:0]**: 5th conversion in regular sequence

These bits are written by software with the channel number (0 to 19) assigned as the 5th in the regular conversion sequence.

Note: The software is allowed to write these bits only when ADSTART = 0 (which ensures that no regular conversion is ongoing).

Note: Some channels are not connected physically and must not be selected for conversion.

28.6.13 ADC regular sequence register 3 (ADC_SQR3)

Address offset: 0x38

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	SQ14[4:0]					Res.	SQ13[4:0]					Res.	SQ12[4]
			rw	rw	rw	rw	rw		rw	rw	rw	rw	rw		rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
SQ12[3:0]				Res.	SQ11[4:0]					Res.	SQ10[4:0]				
rw	rw	rw	rw		rw	rw	rw	rw	rw		rw	rw	rw	rw	rw

Bits 31:29 Reserved, must be kept at reset value.

Bits 28:24 **SQ14[4:0]**: 14th conversion in regular sequence

These bits are written by software with the channel number (0 to 19) assigned as the 14th in the regular conversion sequence.

Note: The software is allowed to write these bits only when ADSTART = 0 (which ensures that no regular conversion is ongoing).

Bit 23 Reserved, must be kept at reset value.

Bits 22:18 **SQ13[4:0]**: 13th conversion in regular sequence

These bits are written by software with the channel number (0 to 19) assigned as the 13th in the regular conversion sequence.

Note: The software is allowed to write these bits only when ADSTART = 0 (which ensures that no regular conversion is ongoing).

Bit 17 Reserved, must be kept at reset value.

Bits 16:12 **SQ12[4:0]**: 12th conversion in regular sequence

These bits are written by software with the channel number (0 to 19) assigned as the 12th in the regular conversion sequence.

Note: The software is allowed to write these bits only when ADSTART = 0 (which ensures that no regular conversion is ongoing).

Bit 11 Reserved, must be kept at reset value.

Bits 10:6 **SQ11[4:0]**: 11th conversion in regular sequence

These bits are written by software with the channel number (0 to 19) assigned as the 11th in the regular conversion sequence.

Note: The software is allowed to write these bits only when ADSTART = 0 (which ensures that no regular conversion is ongoing).

Bit 5 Reserved, must be kept at reset value.

Bits 4:0 **SQ10[4:0]**: 10th conversion in regular sequence

These bits are written by software with the channel number (0 to 19) assigned as the 10th in the regular conversion sequence.

Note: The software is allowed to write these bits only when ADSTART = 0 (which ensures that no regular conversion is ongoing).

Note: Some channels are not connected physically and must not be selected for conversion.

28.6.14 ADC regular sequence register 4 (ADC_SQR4)

Address offset: 0x3C

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	Res.	Res.	Res.	SQ16[4:0]					Res.	SQ15[4:0]				
					rw	rw	rw	rw	rw		rw	rw	rw	rw	rw

Bits 31:11 Reserved, must be kept at reset value.

Bits 10:6 **SQ16[4:0]**: 16th conversion in regular sequence

These bits are written by software with the channel number (0 to 19) assigned as the 16th in the regular conversion sequence.

Note: The software is allowed to write these bits only when ADSTART = 0 (which ensures that no regular conversion is ongoing).

Bit 5 Reserved, must be kept at reset value.

Bits 4:0 **SQ15[4:0]**: 15th conversion in regular sequence

These bits are written by software with the channel number (0 to 19) assigned as the 15th in the regular conversion sequence.

Note: The software is allowed to write these bits only when ADSTART = 0 (which ensures that no regular conversion is ongoing).

Note: Some channels are not connected physically and must not be selected for conversion.

28.6.15 ADC regular data register (ADC_DR)

Address offset: 0x40

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
RDATA[15:0]															
r	r	r	r	r	r	r	r	r	r	r	r	r	r	r	r

Bits 31:16 Reserved, must be kept at reset value.

Bits 15:0 **RDATA[15:0]**: Regular data converted

These bits are read-only. They contain the conversion result from the last converted regular channel. The data are left- or right-aligned as described in [Section 28.4.26: Data management](#).

28.6.16 ADC injected sequence register (ADC_JSQR)

Address offset: 0x4C

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
JSQ4[4:0]					Res.	JSQ3[4:0]					Res.	JSQ2[4:1]			
rw	rw	rw	rw	rw		rw	rw	rw	rw	rw		rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
JSQ2[0]	Res.	JSQ1[4:0]				JEXTEN[1:0]			JEXTSEL[4:0]				JL[1:0]		
rw		rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bits 31:27 **JSQ4[4:0]**: 4th conversion in the injected sequence

These bits are written by software with the channel number (0 to 19) assigned as the 4th in the injected conversion sequence.

Note: The software is allowed to write these bits only when JADSTART = 0 (which ensures that no injected conversion is ongoing).

Bit 26 Reserved, must be kept at reset value.

Bits 25:21 **JSQ3[4:0]**: 3rd conversion in the injected sequence

These bits are written by software with the channel number (0 to 19) assigned as the 3rd in the injected conversion sequence.

Note: The software is allowed to write these bits only when JADSTART = 0 (which ensures that no injected conversion is ongoing).

Bit 20 Reserved, must be kept at reset value.

Bits 19:15 **JSQ2[4:0]**: 2nd conversion in the injected sequence

These bits are written by software with the channel number (0 to 19) assigned as the 2nd in the injected conversion sequence.

Note: The software is allowed to write these bits only when JADSTART = 0 (which ensures that no injected conversion is ongoing).

Bit 14 Reserved, must be kept at reset value.

Bits 13:9 **JSQ1[4:0]**: 1st conversion in the injected sequence

These bits are written by software with the channel number (0 to 19) assigned as the 1st in the injected conversion sequence.

Note: The software is allowed to write these bits only when JADSTART = 0 (which ensures that no injected conversion is ongoing).

Bits 8:7 **JEXTEN[1:0]**: External trigger enable and polarity selection for injected channels

These bits are set and cleared by software to select the external trigger polarity and enable the trigger of an injected group.

00: If JQDIS = 0 (queue enabled), hardware and software trigger detection disabled.

Otherwise, the queue is disabled as well as hardware trigger detection (conversions can be launched by software)

01: Hardware trigger detection on the rising edge

10: Hardware trigger detection on the falling edge

11: Hardware trigger detection on both the rising and falling edges

Note: The software is allowed to write these bits only when JADSTART = 0 (which ensures that no injected conversion is ongoing).

If JQM = 1 and if the Queue of Context becomes empty, the software and hardware triggers of the injected sequence are both internally disabled (refer to [Section 28.4.21: Queue of context for injected conversions](#))

Bits 6:2 **JEXTSEL[4:0]**: External Trigger Selection for injected group

These bits select the external event used to trigger the start of conversion of an injected group:

00000: adc_jext_trg0

00001: adc_jext_trg1

00010: adc_jext_trg2

00011: adc_jext_trg3

00100: adc_jext_trg4

00101: adc_jext_trg5

00110: adc_jext_trg6

00111: adc_jext_trg7

...

11111: adc_jext_trg31

Note: The software is allowed to write these bits only when JADSTART = 0 (which ensures that no injected conversion is ongoing).

Bits 1:0 **JL[1:0]**: Injected channel sequence length

These bits are written by software to define the total number of conversions in the injected channel conversion sequence.

00: 1 conversion

01: 2 conversions

10: 3 conversions

11: 4 conversions

Note: The software is allowed to write these bits only when JADSTART = 0 (which ensures that no injected conversion is ongoing).

Note: Some channels are not connected physically and must not be selected for conversion.

28.6.17 ADC offset y register (ADC_OFRy)

Address offset: $0x60 + 0x04 * (y - 1)$, ($y = 1$ to 4)

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
OFFSET_EN	OFFSET_CH[4:0]					SATEN	OFFSE TPOS	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w								
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	Res.	Res.	OFFSET[11:0]											
				r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w

Bit 31 OFFSET_EN: Offset y enable

This bit is written by software to enable or disable the offset programmed into bits OFFSET[11:0].

Note: The software is allowed to write this bit only when ADSTART = 0 and JADSTART = 0 (which ensures that no conversion is ongoing).

Bits 30:26 OFFSET_CH[4:0]: Channel selection for the data offset y

These bits are written by software to define the channel to which the offset programmed into bits OFFSET[11:0] applies.

Note: The software is allowed to write these bits only when ADSTART = 0 and JADSTART = 0 (which ensures that no conversion is ongoing).

Some channels are not connected physically and must not be selected for the data offset y.

If OFFSET_EN is set, it is not allowed to select the same channel for different ADC_OFRy registers.

Bit 25 SATEN: Saturation enable

This bit is set and cleared by software to enable the saturation at 0x000 and 0xFFFF for the offset function.

0: No saturation control, offset result can be signed

1: Saturation enabled, offset result unsigned and saturated at 0x000 and 0xFFFF

Note: The software is allowed to write these bits only when ADSTART = 0 and JADSTART = 0 (which ensures that no conversion is ongoing).

Bit 24 **OFFSETPOS**: Positive offset

This bit is set and cleared by software to enable the positive offset.

0: Negative offset

1: Positive offset

Note: The software is allowed to write these bits only when ADSTART = 0 and JADSTART = 0 (which ensures that no conversion is ongoing).

Bits 23:12 Reserved, must be kept at reset value.

Bits 11:0 **OFFSET[11:0]**: Data offset y for the channel programmed into bits OFFSET_CH[4:0]

These bits are written by software to define the offset to be subtracted from the raw converted data when converting a channel (can be regular or injected). The channel to which applies the data offset must be programmed in the bits OFFSET_CH[4:0]. The conversion result can be read from in the ADC_DR (regular conversion) or from in the ADC_JDRy registers (injected conversion).

Note: The software is allowed to write these bits only when ADSTART = 0 and JADSTART = 0 (which ensures that no conversion is ongoing).

If several offset (OFFSET) point to the same channel, only the offset with the lowest x value is considered for the subtraction.

Ex: if OFFSET1_CH[4:0] = 4 and OFFSET2_CH[4:0] = 4, this is OFFSET1[11:0] which is subtracted when converting channel 4.

28.6.18 ADC injected channel y data register (ADC_JDRy)

Address offset: $0x80 + 0x04 * (y - 1)$, (y = 1 to 4)

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
JDATA[15:0]															
r	r	r	r	r	r	r	r	r	r	r	r	r	r	r	r

Bits 31:16 Reserved, must be kept at reset value.

Bits 15:0 **JDATA[15:0]**: Injected data

These bits are read-only. They contain the conversion result from injected channel y. The data are left -or right-aligned as described in [Section 28.4.26: Data management](#).

28.6.19 ADC analog watchdog 2 configuration register (ADC_AWD2CR)

Address offset: 0xA0

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	AWD2CH[19:16]			
												rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
AWD2CH[15:0]															
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bits 31:20 Reserved, must be kept at reset value.

Bits 19:0 **AWD2CH[19:0]**: Analog watchdog 2 channel selection

These bits are set and cleared by software. They enable and select the input channels to be guarded by the analog watchdog 2.

AWD2CH[i] = 0: ADC analog input channel i is not monitored by AWD2

AWD2CH[i] = 1: ADC analog input channel i is monitored by AWD2

When AWD2CH[19:0] = 000..0, the analog Watchdog 2 is disabled

*Note: The channels selected by AWD2CH must be also selected into the SQi or JSQi bits.**The software is allowed to write these bits only when ADSTART = 0 and JADSTART = 0 (which ensures that no conversion is ongoing).**Some channels are not connected physically and must not be selected for the analog watchdog.***28.6.20 ADC analog watchdog 3 configuration register (ADC_AWD3CR)**

Address offset: 0xA4

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	AWD3CH[19:16]			
												rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
AWD3CH[15:0]															
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bits 31:20 Reserved, must be kept at reset value.

Bits 19:0 **AWD3CH[19:0]**: Analog watchdog 3 channel selection

These bits are set and cleared by software. They enable and select the input channels to be guarded by the analog watchdog 3.

AWD3CH[i] = 0: ADC analog input channel i is not monitored by AWD3

AWD3CH[i] = 1: ADC analog input channel i is monitored by AWD3

When AWD3CH[19:0] = 000..0, the analog Watchdog 3 is disabled

Note: The channels selected by AWD3CH must be also selected into the SQi or JSQi bits.

The software is allowed to write these bits only when ADSTART = 0 and JADSTART = 0 (which ensures that no conversion is ongoing).

Some channels are not connected physically and must not be selected for the analog watchdog.

28.6.21 ADC Differential mode selection register (ADC_DIFSEL)

Address offset: 0xB0

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	DIFSEL[19:16]			
												rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
DIFSEL[15:0]															
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	r

Bits 31:20 Reserved, must be kept at reset value.

Bits 19:0 **DIFSEL[19:0]**: Differential mode for channels 19 to 0.

These bits are set and cleared by software. They allow to select if a channel is configured as single-ended or Differential mode.

DIFSEL[i] = 0: ADC analog input channel is configured in single-ended mode

DIFSEL[i] = 1: ADC analog input channel i is configured in Differential mode

Note: The DIFSEL bits corresponding to channels that are either connected to a single-ended I/O port or to an internal channel must be kept their reset value (single-ended input mode).

The software is allowed to write these bits only when the ADC is disabled (ADCAL = 0, JADSTART = 0, JADSTP = 0, ADSTART = 0, ADSTP = 0, ADDIS = 0 and ADEN = 0).

28.6.22 ADC calibration factors (ADC_CALFACT)

Address offset: 0xB4

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	CALFACT_D[6:0]						
									rw	rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	CALFACT_S[6:0]						
									rw	rw	rw	rw	rw	rw	rw

Bits 31:23 Reserved, must be kept at reset value.

Bits 22:16 **CALFACT_D[6:0]**: Calibration Factors in differential mode

These bits are written by hardware or by software.

Once a differential inputs calibration is complete, they are updated by hardware with the calibration factors.

Software can write these bits with a new calibration factor. If the new calibration factor is different from the current one stored into the analog ADC, it is then applied once a new differential calibration is launched.

Note: The software is allowed to write these bits only when ADEN = 1, ADSTART = 0 and JADSTART = 0 (ADC is enabled and no calibration is ongoing and no conversion is ongoing).

Bits 15:7 Reserved, must be kept at reset value.

Bits 6:0 **CALFACT_S[6:0]**: Calibration Factors In single-ended mode

These bits are written by hardware or by software.

Once a single-ended inputs calibration is complete, they are updated by hardware with the calibration factors.

Software can write these bits with a new calibration factor. If the new calibration factor is different from the current one stored into the analog ADC, it is then applied once a new single-ended calibration is launched.

Note: The software is allowed to write these bits only when ADEN = 1, ADSTART = 0 and JADSTART = 0 (ADC is enabled and no calibration is ongoing and no conversion is ongoing).

28.6.23 ADC option register (ADC_OR)

Address offset: 0xC8

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	OP0
															rw

Bits 31:1 Reserved, must be kept at reset value.

Bit 0 **OP0**: Option bit 0

For ADC1:

0: INP0/INN1 GPIO switch control disabled (for both ADC1 and ADC2)

1: INP0/INN1 GPIO switch control enabled (for both ADC1 and ADC2)

Note: This option bit must be set to 1 when ADCx_INP0 or ADCx_INN1 channel is selected.

For ADC2:

0: V_{DDCORE} channel disabled (for both ADC2 and ADC3)

1: V_{DDCORE} channel enabled (for both ADC2 and ADC3)

For ADC3: (only available on STM32H543/553 devices)

0: INP0 GPIO switch control disabled

1: INP0 GPIO switch control enabled

Note: ADC_OR register might be reserved on some ADC instances. Refer to [Section 28.3: ADC implementation](#).

28.7 ADC common registers

These registers define the control and status registers common to master and slave ADCs:

28.7.1 ADC common status register (ADC_CSR)

Address offset: 0x300

Reset value: 0x0000 0000

This register provides an image of the status bits of the different ADC. Nevertheless it is read-only and does not allow to clear the different status bits. Instead each status bit must be cleared by writing 0 to it in the corresponding ADC_ISR register.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	JQOVF_SLV	AWD3_SLV	AWD2_SLV	AWD1_SLV	JEOS_SLV	JEOC_SLV	OVR_SLV	EOS_SLV	EOC_SLV	EOSMP_SLV	ADRDY_SLV
					r	r	r	r	r	r	r	r	r	r	r
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	Res.	Res.	Res.	JQOVF_MST	AWD3_MST	AWD2_MST	AWD1_MST	JEOS_MST	JEOC_MST	OVR_MST	EOS_MST	EOC_MST	EOSMP_MST	ADRDY_MST
					r	r	r	r	r	r	r	r	r	r	r

Bits 31:27 Reserved, must be kept at reset value.

Bit 26 **JQOVF_SLV**: Injected Context Queue Overflow flag of the slave ADC

This bit is a copy of the JQOVF bit in the corresponding ADC_ISR register.

Bit 25 **AWD3_SLV**: Analog watchdog 3 flag of the slave ADC

This bit is a copy of the AWD3 bit in the corresponding ADC_ISR register.

Bit 24 **AWD2_SLV**: Analog watchdog 2 flag of the slave ADC

This bit is a copy of the AWD2 bit in the corresponding ADC_ISR register.

Bit 23 **AWD1_SLV**: Analog watchdog 1 flag of the slave ADC

This bit is a copy of the AWD1 bit in the corresponding ADC_ISR register.

- Bit 22 **JEOS_SLV**: End of injected sequence flag of the slave ADC
This bit is a copy of the JEOS bit in the corresponding ADC_ISR register.
- Bit 21 **JEOC_SLV**: End of injected conversion flag of the slave ADC
This bit is a copy of the JEOC bit in the corresponding ADC_ISR register.
- Bit 20 **OVR_SLV**: Overrun flag of the slave ADC
This bit is a copy of the OVR bit in the corresponding ADC_ISR register.
- Bit 19 **EOS_SLV**: End of regular sequence flag of the slave ADC. This bit is a copy of the EOS bit in the corresponding ADC_ISR register.
- Bit 18 **EOC_SLV**: End of regular conversion of the slave ADC
This bit is a copy of the EOC bit in the corresponding ADC_ISR register.
- Bit 17 **EOSMP_SLV**: End of Sampling phase flag of the slave ADC
This bit is a copy of the EOSMP2 bit in the corresponding ADC_ISR register.
- Bit 16 **ADRDY_SLV**: Slave ADC ready
This bit is a copy of the ADRDY bit in the corresponding ADC_ISR register.
- Bits 15:11 Reserved, must be kept at reset value.
- Bit 10 **JQOVF_MST**: Injected Context Queue Overflow flag of the master ADC
This bit is a copy of the JQOVF bit in the corresponding ADC_ISR register.
- Bit 9 **AWD3_MST**: Analog watchdog 3 flag of the master ADC
This bit is a copy of the AWD3 bit in the corresponding ADC_ISR register.
- Bit 8 **AWD2_MST**: Analog watchdog 2 flag of the master ADC
This bit is a copy of the AWD2 bit in the corresponding ADC_ISR register.
- Bit 7 **AWD1_MST**: Analog watchdog 1 flag of the master ADC
This bit is a copy of the AWD1 bit in the corresponding ADC_ISR register.
- Bit 6 **JEOS_MST**: End of injected sequence flag of the master ADC
This bit is a copy of the JEOS bit in the corresponding ADC_ISR register.
- Bit 5 **JEOC_MST**: End of injected conversion flag of the master ADC
This bit is a copy of the JEOC bit in the corresponding ADC_ISR register.
- Bit 4 **OVR_MST**: Overrun of the master ADC
This bit is a copy of the OVR bit in the corresponding ADC_ISR register.
- Bit 3 **EOS_MST**: End of regular sequence flag of the master ADC
This bit is a copy of the EOS bit in the corresponding ADC_ISR register.
- Bit 2 **EOC_MST**: End of regular conversion of the master ADC
This bit is a copy of the EOC bit in the corresponding ADC_ISR register.
- Bit 1 **EOSMP_MST**: End of Sampling phase flag of the master ADC
This bit is a copy of the EOSMP bit in the corresponding ADC_ISR register.
- Bit 0 **ADRDY_MST**: Master ADC ready
This bit is a copy of the ADRDY bit in the corresponding ADC_ISR register.

28.7.2 ADC common control register (ADC_CCR)

Address offset: 0x308

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	VBATE N	TSEN	VREF EN	PRESC[3:0]				CKMODE[1:0]	
							rw	rw	rw	rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
MDMA[1:0]		DMA CFG	Res.	DELAY[3:0]				Res.	Res.	Res.	DUAL[4:0]				
rw	rw	rw		rw	rw	rw	rw				rw	rw	rw	rw	rw

Bits 31:25 Reserved, must be kept at reset value.

Bit 24 **VBATEN**: VBAT enable

This bit is set and cleared by software to control.

0: V_{BAT} channel disabled

1: V_{BAT} channel enabled

Bit 23 **TSEN**: V_{SENSE} enable

This bit is set and cleared by software to control V_{SENSE}.

0: Temperature sensor channel disabled

1: Temperature sensor channel enabled

Bit 22 **VREFEN**: V_{REFINT} enable

This bit is set and cleared by software to enable/disable the V_{REFINT} channel.

0: V_{REFINT} channel disabled

1: V_{REFINT} channel enabled

Bits 21:18 **PRESC[3:0]**: ADC prescaler

These bits are set and cleared by software to select the frequency of the clock to the ADC. The clock is common for all the ADCs.

0000: input ADC clock not divided

0001: input ADC clock divided by 2

0010: input ADC clock divided by 4

0011: input ADC clock divided by 6

0100: input ADC clock divided by 8

0101: input ADC clock divided by 10

0110: input ADC clock divided by 12

0111: input ADC clock divided by 16

1000: input ADC clock divided by 32

1001: input ADC clock divided by 64

1010: input ADC clock divided by 128

1011: input ADC clock divided by 256

other: reserved

Note: The software is allowed to write these bits only when the ADC is disabled (ADCAL = 0, JADSTART = 0, ADSTART = 0, ADSTP = 0, ADDIS = 0 and ADEN = 0). The ADC prescaler value is applied only when CKMODE[1:0] = 0b00.

Bits 17:16 **CKMODE[1:0]**: ADC clock mode

These bits are set and cleared by software to define the ADC clock scheme (which is common to both master and slave ADCs):

00: `adc_ker_ck` ($x = 1/2/3$) (Asynchronous clock mode), generated at product level (refer to *Section 6: Reset and clock control (RCC)*)

01: `adc_hclk/1` (Synchronous clock mode). This configuration must be enabled only if the AHB clock prescaler is set to 1 (`HPRE[3:0] = 0XXX` in the `RCC_CFGR` register) and if the system clock has a 50% duty cycle.

10: `adc_hclk/2` (Synchronous clock mode)

11: `adc_hclk/4` (Synchronous clock mode)

In all synchronous clock modes, there is no jitter in the delay from a timer trigger to the start of a conversion.

Note: The software is allowed to write these bits only when the ADCs are disabled (`ADCAL = 0`, `JADSTART = 0`, `ADSTART = 0`, `ADSTP = 0`, `ADDIS = 0` and `ADEN = 0`).

Bits 15:14 **MDMA[1:0]**: Direct memory access mode for dual ADC mode

This bitfield is set and cleared by software. Refer to the DMA controller section for more details.

00: MDMA mode disabled

01: Reserved

10: MDMA mode enabled for 12 and 10-bit resolution

11: MDMA mode enabled for 8 and 6-bit resolution

Note: The software is allowed to write these bits only when `ADSTART = 0` (which ensures that no regular conversion is ongoing).

Bit 13 **DMACFG**: DMA configuration (for dual ADC mode)

This bit is set and cleared by software to select between two DMA modes of operation and is effective only when `DMAEN = 1`.

0: DMA One Shot mode selected

1: DMA Circular mode selected

For more details, refer to [Section : Managing conversions using the DMA](#)

Note: The software is allowed to write these bits only when `ADSTART = 0` (which ensures that no regular conversion is ongoing).

Bit 12 Reserved, must be kept at reset value.

Bits 11:8 **DELAY[3:0]**: Delay between 2 sampling phases

These bits are set and cleared by software. These bits are used in dual interleaved modes.

Refer to [Table 293](#) for the value of ADC resolution versus DELAY bits values.

Note: The software is allowed to write these bits only when the ADCs are disabled

(ADCAL = 0, JADSTART = 0, ADSTART = 0, ADSTP = 0, ADDIS = 0 and ADEN = 0).

Bits 7:5 Reserved, must be kept at reset value.

Bits 4:0 **DUAL[4:0]**: Dual ADC mode selection

These bits are written by software to select the operating mode. 00000 corresponds to Independent mode. Values 00001 to 01001 correspond to dual mode, master and slave ADCs working together.

00000: Independent mode

00001: Combined regular simultaneous + injected simultaneous mode

00010: Combined regular simultaneous + alternate trigger mode

00011: Combined interleaved mode + injected simultaneous mode

00100: Reserved

00101: Injected simultaneous mode only

00110: Regular simultaneous mode only

00111: Interleaved mode only

01001: Alternate trigger mode only

Others: Reserved, must not be used

Note: The software is allowed to write these bits only when the ADCs are disabled

(ADCAL = 0, JADSTART = 0, ADSTART = 0, ADSTP = 0, ADDIS = 0 and ADEN = 0).

Table 293. DELAY bits versus ADC resolution

DELAY bits	12-bit resolution	10-bit resolution	8-bit resolution	6-bit resolution
0000	$1 * T_{adc_ker_ck}$	$1 * T_{adc_ker_ck}$	$1 * T_{adc_ker_ck}$	$1 * T_{adc_ker_ck}$
0001	$2 * T_{adc_ker_ck}$	$2 * T_{adc_ker_ck}$	$2 * T_{adc_ker_ck}$	$2 * T_{adc_ker_ck}$
0010	$3 * T_{adc_ker_ck}$	$3 * T_{adc_ker_ck}$	$3 * T_{adc_ker_ck}$	$3 * T_{adc_ker_ck}$
0011	$4 * T_{adc_ker_ck}$	$4 * T_{adc_ker_ck}$	$4 * T_{adc_ker_ck}$	$4 * T_{adc_ker_ck}$
0100	$5 * T_{adc_ker_ck}$	$5 * T_{adc_ker_ck}$	$5 * T_{adc_ker_ck}$	$5 * T_{adc_ker_ck}$
0101	$6 * T_{adc_ker_ck}$	$6 * T_{adc_ker_ck}$	$6 * T_{adc_ker_ck}$	$6 * T_{adc_ker_ck}$
0110	$7 * T_{adc_ker_ck}$	$7 * T_{adc_ker_ck}$	$7 * T_{adc_ker_ck}$	$6 * T_{adc_ker_ck}$
0111	$8 * T_{adc_ker_ck}$	$8 * T_{adc_ker_ck}$	$8 * T_{adc_ker_ck}$	$6 * T_{adc_ker_ck}$
1000	$9 * T_{adc_ker_ck}$	$9 * T_{adc_ker_ck}$	$8 * T_{adc_ker_ck}$	$6 * T_{adc_ker_ck}$
1001	$10 * T_{adc_ker_ck}$	$10 * T_{adc_ker_ck}$	$8 * T_{adc_ker_ck}$	$6 * T_{adc_ker_ck}$
1010	$11 * T_{adc_ker_ck}$	$10 * T_{adc_ker_ck}$	$8 * T_{adc_ker_ck}$	$6 * T_{adc_ker_ck}$
1011	$12 * T_{adc_ker_ck}$	$10 * T_{adc_ker_ck}$	$8 * T_{adc_ker_ck}$	$6 * T_{adc_ker_ck}$
others	$12 * T_{adc_ker_ck}$	$10 * T_{adc_ker_ck}$	$8 * T_{adc_ker_ck}$	$6 * T_{adc_ker_ck}$

28.7.3 ADC common regular data register for dual mode (ADC_CDR)

Address offset: 0x30C

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
RDATA_SLV[15:0]															
r	r	r	r	r	r	r	r	r	r	r	r	r	r	r	r
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
RDATA_MST[15:0]															
r	r	r	r	r	r	r	r	r	r	r	r	r	r	r	r

Bits 31:16 **RDATA_SLV[15:0]**: Regular data of the slave ADC

In dual mode, these bits contain the regular data of the slave ADC. Refer to [Section 28.4.30: Dual ADC modes](#).

The data alignment is applied as described in [Section : Data register, data alignment and offset \(ADC_DR, OFFSET, OFFSET_CH, ALIGN\)](#)

Bits 15:0 **RDATA_MST[15:0]**: Regular data of the master ADC.

In dual mode, these bits contain the regular data of the master ADC. Refer to [Section 28.4.30: Dual ADC modes](#).

The data alignment is applied as described in [Section : Data register, data alignment and offset \(ADC_DR, OFFSET, OFFSET_CH, ALIGN\)](#)

In MDMA = 0b11 mode, bits 15:8 contains SLV_ADC_DR[7:0], bits 7:0 contains MST_ADC_DR[7:0].

28.8 ADC register map

The following table summarizes the ADC registers.

Table 294. ADC global register map

Offset	Register
0x000 - 0x0B4	Master ADC1/3
0x0B8 - 0x0FC	Reserved
0x100 - 0x1B4	Slave ADC2
0x1B8 - 0x2FC	Reserved
0x300 - 0x3FC	Master and slave ADCs common registers

Table 295. ADC register map and reset values for each ADC (offset = 0x000 for master ADC, 0x100 for slave ADC)

Offset	Register name	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0x00	ADC_ISR	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	JQOVF	AWD3	AWD2	AWD1	JEOS	JEOC	OVR	EOS	EOC	EOSMP	ADRDY
	Reset value																						0	0	0	0	0	0	0	0	0	0	0
0x04	ADC_IER	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	JQOVFIE	AWD3IE	AWD2IE	AWD1IE	JEOSIE	JEOCIE	OVRIE	EOSIE	EOCIE	EOSMPIE	ADRDYIE
	Reset value																						0	0	0	0	0	0	0	0	0	0	0
0x08	ADC_CR	ADCAL	ADCALDIF	DEEPPWD	ADVREGEN																							JADSTP	ADSTP	JADSTART	ADSTART	ADDIS	ADEN
	Reset value	0	0	1	0																							0	0	0	0	0	0
0x0C	ADC_CFGR	JQDIS	AWD1CH[4:0]					JAUTO	JAWD1EN	AWD1EN	AWD1SGL	JQM	JDISCEN	DISCNUM[2:0]			DISCEN	ALIGN	AUTDLY	CONT	OVRMOD	EXTEN[1:0]		EXTSEL4	EXTSEL3	EXTSEL2	EXTSEL1	EXTSEL0	RES[1:0]			DMACFG	DMAEN
	Reset value	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0		0	0
0x0C	ADC_CFGR2	Res.	Res.	Res.	Res.	SMPTRIG	BULB	SWTRIG		Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	ROVSM	TROVS	OVSS[3:0]			OVSRR[2:0]			JOVSE	ROVSE	
	Reset value					0	0	0															0	0	0	0	0	0	0	0	0	0	0
0x14	ADC_SMPR1	SMPPLUS	Res.	SMP9 [2:0]			SMP8 [2:0]			SMP7 [2:0]			SMP6 [2:0]			SMP5 [2:0]			SMP4 [2:0]			SMP3 [2:0]			SMP2 [2:0]			SMP1 [2:0]			SMP0 [2:0]		
	Reset value	0		0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0x18	ADC_SMPR2	Res.	Res.	SMP19 [2:0]			SMP18 [2:0]			SMP17 [2:0]			SMP16 [2:0]			SMP15 [2:0]			SMP14 [2:0]			SMP13 [2:0]			SMP12 [2:0]			SMP11 [2:0]			SMP10 [2:0]		
	Reset value			0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0x1C	Reserved	Res.																															
0x20	ADC_TR1	Res.	Res.	Res.	Res.	HT1[11:0]										Res.	AWDFILT [2:0]		LT1[11:0]														
	Reset value					1	1	1	1	1	1	1	1	1	1	1	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

[illegible]

Table 295. ADC register map and reset values for each ADC (offset = 0x000 for master ADC, 0x100 for slave ADC) (continued)

Offset	Register name	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0		
0x80	ADC_JDR1	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	JDATA1[15:0]																	
	Reset value																	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0		
0x84	ADC_JDR2	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	JDATA2[15:0]																	
	Reset value																	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0		
0x88	ADC_JDR3	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	JDATA3[15:0]																	
	Reset value																	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0		
0x8C	ADC_JDR4	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	JDATA4[15:0]																	
	Reset value																	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0		
0x90-0x9C	Reserved	Res.																																	
0xA0	ADC_AWD2CR	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	AWD2CH[19:0]																	
	Reset value																	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0		
0xA4	ADC_AWD3CR	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	AWD3CH[19:0]																	
	Reset value																	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0		
0xA8-0xAC	Reserved	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	
0xB0	ADC_DIFSEL	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	DIFSEL[19:0]																	
	Reset value																	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0		
0xB4	ADC_CALFACT	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	CALFACT_D[6:0]						Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	CALFACT_S[6:0]									
	Reset value										0	0	0	0	0	0	0										0	0	0	0	0	0	0	0	
0xB8-0xC4	Reserved	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	
0xC8	ADC_OR	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	OP0	
	Reset value																																0		
0xCC-0xFC	Reserved	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	

Table 296. ADC register map and reset values (master and slave ADC common registers)

Offset	Register name	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0		
0x300	ADC_CSR	Res.	Res.	Res.	Res.	Res.	JQOVF_SLV	AWD3_SLV	AWD2_SLV	AWD1_SLV	JEOS_SLV	JEOC_SLV	OVR_SLV	EOS_SLV	EOC_SLV	EOSMP_SLV	ADRDY_SLV	Res.	Res.	Res.	Res.	Res.	JQOVF_MST	AWD3_MST	AWD2_MST	AWD1_MST	JEOS_MST	JEOC_MST	OVR_MST	EOS_MST	EOC_MST	EOSMP_MST	ADRDY_MST		
		slave ADC2															master ADC1																		
	Reset value						0	0	0	0	0	0	0	0	0	0	0							0	0	0	0	0	0	0	0	0	0		
0x304	Reserved	Res.																																	
0x308	ADC_CCR	Res.	Res.	Res.	Res.	Res.	Res.	Res.	VBATEN	TSEN	VREFEN	PRESC[3:0]			CKMODE[1:0]		MDMA[1:0]		DMACFG		Res.	DELAY[3:0]			Res.	Res.	Res.	DUAL[4:0]							
	Reset value								0	0	0	0	0	0	0	0	0	0	0	0		0	0	0	0				0	0	0	0	0	0	

Table 296. ADC register map and reset values (master and slave ADC common registers) (continued)

Offset	Register name	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0x30C	ADC_CDR	RDATA_SLV[15:0]																RDATA_MST[15:0]															
	Reset value	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	
0x310-0x3EC	Reserved	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	

Refer to [Section 2.3: Memory organization](#) for the register boundary addresses.



29 Digital temperature sensor (DTS)

29.1 DTS introduction

The device embeds a sensor that converts the temperature into a square wave which frequency is proportional to the temperature. The frequency is measured either with the PCLK or the LSE clock.

29.2 DTS main features

The temperature sensor block main features are the following:

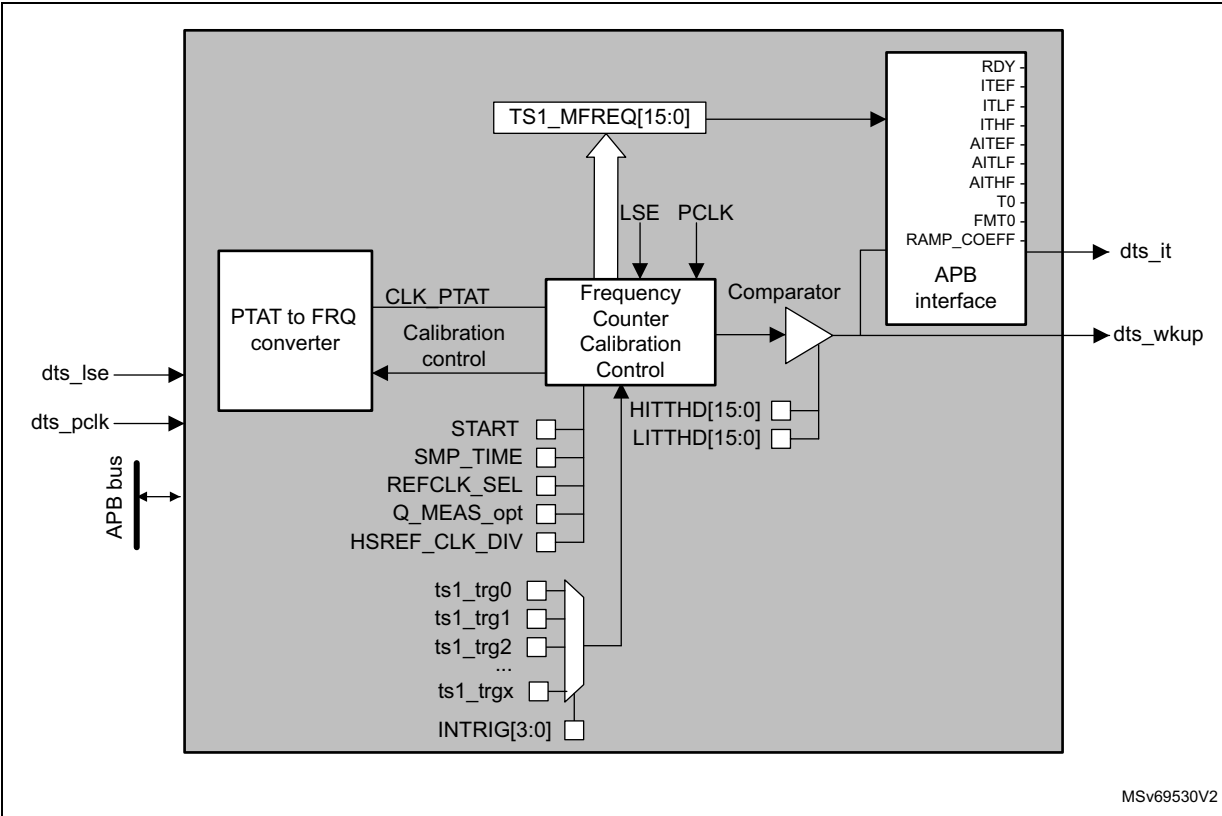
- Start of measurement triggered by software or 4 hardware sources
- Programmable sampling time to increase temperature measurement precision
- Counter synchronized on LSE or PCLK clock
- Temperature watchdog on low and high threshold
- Interrupt generation when the temperature is lower or higher than predefined thresholds and at the end of measurement.
- Asynchronous wake-up signal generation when the temperature is higher/lower than a predefined threshold (LSE mode only)
- Quick measurement using LSE clock

29.3 DTS functional description

29.3.1 DTS block diagram

The temperature sensor block diagram is shown in [Figure 309](#).

Figure 309. Temperature sensor functional block diagram



29.3.2 DTS internal signals

Table 297. DTS internal input/output signals

Signal name	Signal type	Description
dts_lse	Digital input	LSE clock
dts_pclk	Digital input	APB clock
dts_it	Digital output	Temperature sensor interrupt
dts_wkup	Digital output	Temperature sensor wake-up

29.3.3 DTS block operation

The analog part of the temperature sensor outputs a frequency that is proportional to the absolute temperature (CLK_PTAT). The frequency measurement is based on the PCLK or the LSE clock.

Before each measurement, the temperature sensor performs a calibration of the frequency generation blocks.

29.3.4 Operating modes

Several operating modes can be selected by setting the REFCLK_SEL bit in [Temperature sensor configuration register 1 \(DTS_CFGR1\)](#):

- PCLK only (REFCLK_SEL = 0)
The temperature sensor registers can be accessed. The interface can consequently be reconfigured and the measurement sequence is performed using PCLK clock
- PCLK and LSE (REFCLK_SEL = 1)
The temperature sensor registers can be accessed. The interface can consequently be reconfigured and the measurement sequence is performed using the LSE clock.
- LSE only (REFCLK_SEL = 1) and PCLK OFF
The registers cannot be accessed. The measurement can be performed using the LSE clock. This mode is used to exit from Sleep mode by using hardware triggers and the asynchronous interrupt line.

29.3.5 Calibration

The temperature sensor must run the calibration prior to any frequency measurement. The calibration is performed automatically when the temperature measurement is triggered except for quick measurement mode (Q_MEAS_OPT set to 1 in DTS_CFGR1).

29.3.6 Prescaler

When a calibration is ongoing, the counter clock must be slower than 1 MHz. This is achieved by the PCLK clock prescaler embedded in the temperature sensor.

During the temperature measurement period, the prescaler is bypassed.

- When PCLK is used as reference clock (REFCLK_SEL set to 0 in DTS_CFGR1), a prescaler is used. Its division ratio must be configured up to 127 (refer to the HSREF_CLK_DIV[6:0] register definition for the divider setting).
- When LSE is used as reference clock (REFCLK_SEL set to 1 in DTS_CFGR1), the timebase is equal to 2 LSE periods. In this case, no prescaler is used.

29.3.7 Temperature measurement principles

The analog part of temperature sensor outputs a signal (CLK_PTAT) which FM(T) frequency is temperature-dependent.

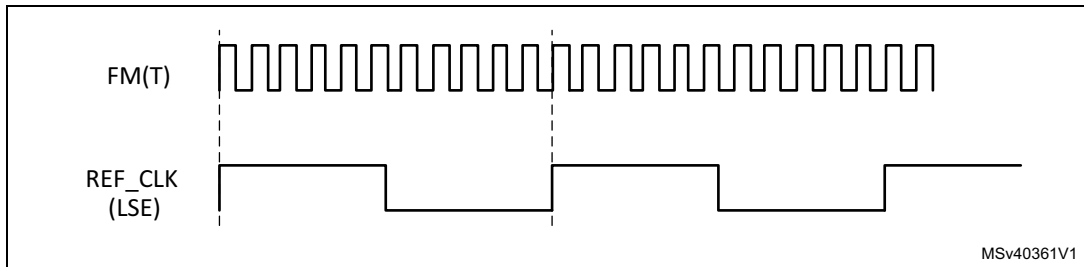
Either PCLK or LSE can be selected as reference clock (REF_CLK) through the REFCLK_SEL bit in DTS_CFGR1.

The counting method depends on the REF_CLK frequency. This is due to the fact that two counters are implemented in the temperature sensor block:

- For low REF_CLK frequencies, a counting of FM(T) cycles is performed during one or several REF_CLK cycles.
- For high REF_CLK frequencies, a counting of REF_CLK cycles is performed during one or several FM(T) cycles.

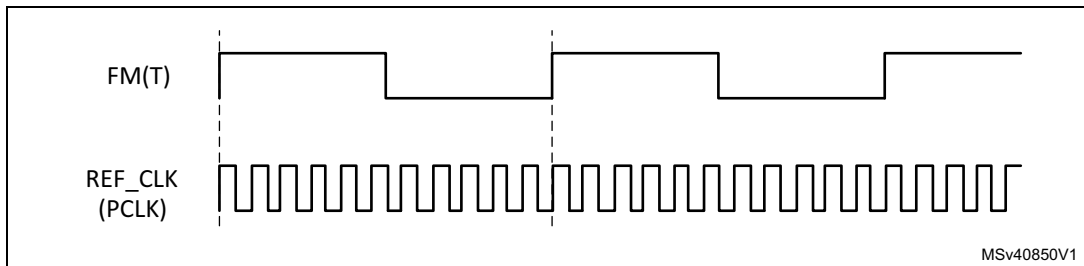
This counter behavior is shown in [Figure 310](#) and [Figure 311](#).

Figure 310. Method for low REF_CLK frequencies



1. To increase the precision, FM(T) measurement can be done on several LSE periods.

Figure 311. Method for high REF_CLK frequencies



1. To increase the precision, PCLK measurement can be done on several FM(T) periods.

The counting result is stored in the DTS_DR register (see [Temperature sensor data register \(DTS_DR\)](#)).

Once the FM(T) frequency has been obtained, the corresponding temperature can be calculated by software using the following formula:

- When PCLK is used:

$$T = T_0 + ((F_{PCLK} / TS1_MFREQ) \times TS1_SMP_TIME - 100 \times TS1_FMT0) / TS1_RAMP_COEFF$$

where

T_0 (factory calibration temperature) is equal to 30 °C.

$TS1_FMT0$ is measured and stored in the DTS_T0VALR1 register. It is expressed in hundreds of Hertz.

TS1_RAMP_COEFF is measured during tests in factory and stored in DTS_RAMPVALR register. This value is expressed in Hz/°C.

- When the LSE clock is used

$$T = T_0 + ((F_{LSE} \times TS1_MFREQ / TS1_SMP_TIME) - (100 \times TS1_FMT0)) / TS1_RAMP_COEFF$$

29.3.8 Sampling time

The sampling period can be increased to improve measurement accuracy. This is useful when the reference frequency (REF_CLK) is close to the FM(T) frequency. The default value is one REF_CLK cycle in LSE mode, and one FM(T) cycle in PCLK mode.

The sampling time is configured through TS1_SMP_TIME bits in DTS_CFGR1 register (see [Table 298](#)).

Table 298. Sampling time configuration

TS1_SMP_TIME[3:0]	LSE or FM(T) clock cycle(s)
0000	1
0001	1
0010	2
0011	3
0100	4
0101	5
0110	6
0111	7
1000	8
1001	9
1010	10
1011	11
1100	12
1101	13
1110	14
1111	15

29.3.9 Quick measurement mode

If a high precision is not required, the calibration step included in each measurement sequence can be skipped by setting Q_MEAS_OPT to 1 in the DTS_CFGR1 register. This method must be used only when the LSE clock is selected as reference clock (LSREF_CLK set to 1). This mode can reduce the measurement time.

29.3.10 Trigger input

The temperature measurement can be triggered either by software or by an external event. The trigger source can be selected through TS1_INTRIG[3:0] bits in DTS_CFGR1.

Table 299. Trigger configuration

Name	TS1_INTRIG[3:0]				Comment
N.A	0	0	0	0	No hardware trigger
ts1_trg0	0	0	0	1	lptim1_ch1
ts1_trg1	0	0	1	0	lptim2_ch1
ts1_trg2	0	0	1	1	lptim3_ch1
ts1_trg3	0	1	0	0	exti13
ts1_trg4	0	1	0	1	Reserved
ts1_trg5	0	1	1	0	
ts1_trg6	0	1	1	1	
ts1_trg7	1	0	0	0	
ts1_trg8	1	0	0	1	
ts1_trg9	1	0	1	0	
ts1_trg10	1	0	1	1	
ts1_trg11	1	1	0	0	
ts1_trg12	1	1	0	1	
ts1_trg13	1	1	1	0	
ts1_trg14	1	1	1	1	

Note: Hardware triggers are active only on the rising edge.

The temperature sensor can only capture a hardware trigger rising edge when TS1_RDY bit is set (see [Section 29.3.11: On-off control and ready flag](#)), otherwise the trigger is ignored.

If a trigger source changes on-the-fly, the new trigger source signal must be low. If the new source signal is high, the temperature sensor detects a rising edge and start the measurement sequence.

29.3.11 On-off control and ready flag

The DTS block can be enabled by setting TS1_EN bit in DTS_CFGR1 register. The TS1_RDY flag in the [Temperature sensor status register \(DTS_SR\)](#) indicate that the DTS block is ready for temperature measurement: when TS1_RDY bit is set to 1, the measurement can be started. Once a measurement has started, TS1_RDY bit is reset. New measurement requests are then ignored. Once the measurement is finished, TS1_RDY bit is set again to indicate the sensor is ready to start a new measurement.

29.3.12 Temperature measurement sequence

Start of measurement can be triggered by software or hardware.

Software trigger

The software trigger is selected when TS1_INTRIG_SEL[3:0] is set to '0000' in DTS_CFGR1.

If TS1_RDY is set to 1, writing TS1_START bit to 1 in DTS_CFGR1 starts the measurement.

If TS1_RDY equals 0, the software trigger does not start until TS1_RDY is set.

If TS1_START bit is kept at 1 once the measurement is finished, then the TS1_RDY flag become 1 and the measurement restarts.

Hardware trigger

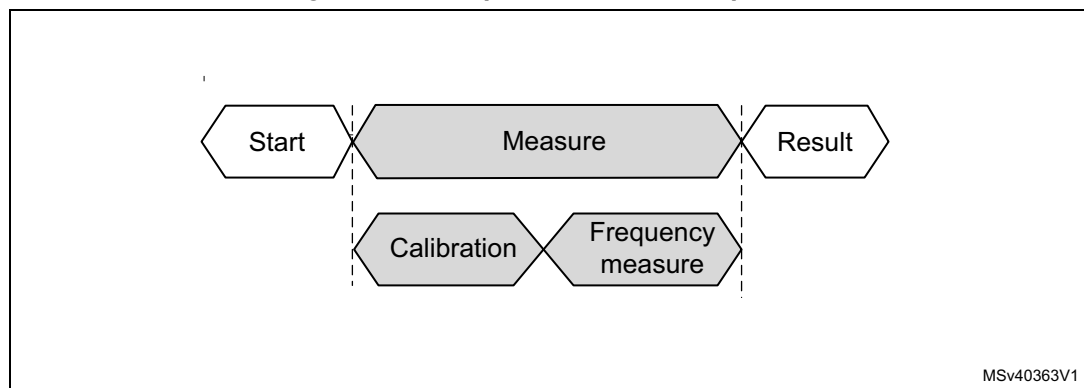
TS1_INTRIG_SEL[3:0] bits allow selecting one hardware trigger out of 4. If TS1_RDY is set to 1, a rising edge on the trigger signal starts the measurement. When TS1_RDY is 0, the rising edge is ignored.

Temperature measurement sequence

One measurement contains two steps: the calibration of the analog blocks and the measurement. The calibration automatically starts when the measurement is triggered (see [Section 29.3.5: Calibration](#)). The measurement period depends on the following DTS_CFGR1 bits:

- the reference clock selected through REFCLK_SEL bit
- the divider ratio configured by HSREF_CLK_DIV bits
- the sampling time defined by TS1_SMP_TIME bits.

Figure 312. Temperature sensor sequence



29.4 DTS low-power modes

Table 300. Temperature sensor behavior in low-power modes

Mode	Description
Sleep	Only works in LSE mode. DTS interrupt causes the device to exit from Sleep mode.
Stop	Only works in LSE mode. DTS interrupt cause the device to exit from Stop mode.

29.5 DTS interrupts

There are two ways to use the DTS block as an interrupt source. The DTS interrupt line can be connected to the EXTI controller (see [Section 29.5.3: Asynchronous wake-up](#)) or to the CPU NVIC (see [Section 29.5.2: Synchronous interrupt](#)).

29.5.1 Temperature window comparator

The DTS_ITR1 register allows defining the high and low threshold that are used for temperature comparison. If the temperature data is equal or higher than TS1_HITTHD, or equal or lower than TS1_LITTHD bit, an interrupt is generated and the corresponding flag, TS1_ITLF, TS1_ITHF, TS1_AITLF and TS1_AITHF, is set in the DTS_SR register (see [Section 29.6.6](#)).

29.5.2 Synchronous interrupt

A global interrupt output line is available on the DTS block. The interrupt can be generated at the end of measurement and/or when the measurement result is equal/higher or equal/lower than a predefined threshold (see [Section 29.5.1: Temperature window comparator](#)).

Three interrupt events can be select via 3 bits in DTS_ITENR register (see [Section 29.6.7](#)). All combinations of interrupts are allowed.

The TS1_ITEF, TS1_ITLF and TS1_ITHF flags in the DTS_SR register reflect the interrupt event. They can be reset with the correspond bits of the DTS_ICIFR register (see [Section 29.6.8](#)).

29.5.3 Asynchronous wake-up

The DTS block also provides an asynchronous interrupt line. It is used only when the LSE is selected as reference clock (REFCLK_SEL=1).

This line can generate a signal that wakes up the system from Sleep mode at the end of measurement and/or when the measurement result is equal/higher or equal/lower than a predefined threshold (see [Section 29.5.1: Temperature window comparator](#)).

Three asynchronous wake-up events can be selected via 3 bits in DTS_ITENR register. All combination of interrupts are allowed.

The TS1_AITEF, TS1_AITLF and TS1_AITHF flags in the DTS_SR register reflect the interrupt status. They can be reset with the correspond bits of the DTS_ICIFR register.

The following table shows the interrupt bits and their description.

Table 301. Interrupt control bits

Interrupt event	Interrupt flag	Enable control bit	Interrupt clear bit	Exit from Sleep mode	Synchronous/ Asynchronous
At the end of measurement	TS1_ITEF in DTS_SR	TS1_ITEEN in DTS_ITENR	TS1_CITEF in DTS_ICIFR	NO	Synchronous on PCLK
When the measure is equal or exceeds the low threshold	TS1_ITLF in DTS_SR	TS1_ITLEN in DTS_ITENR	TS1_CITLF in DTS_ICIFR	NO	
When the measure is equal or exceeds the high threshold	TS1_ITHF in DTS_SR	TS1_ITHEN in DTS_ITENR	TS1_CITHF in DTS_ICIFR	NO	
At the end of measurement	TS1_AITEF in DTS_SR	TS1_AITEEN in DTS_ITENR	TS1_CAITEF in DTS_ICIFR	YES	Asynchronous
When the measure is equal or exceeds the low threshold	TS1_AITLF in DTS_SR	TS1_AITLEN in DTS_ITENR	TS1_CAITLF in DTS_ICIFR	YES	
When the measure is equal or exceeds the high threshold	TS1_AITHF in DTS_SR	TS1_AITHEN in DTS_ITENR	TS1_CAITHF in DTS_ICIFR	YES	

29.6 DTS registers

The registers of this peripheral can only be accessed by-word (32-bit).

29.6.1 Temperature sensor configuration register 1 (DTS_CFGR1)

DTS_CFGR1 is the configuration register for temperature sensor 1.

Address offset: 0x00

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	HSREF_CLK_DIV[6:0]							Res.	Res.	Q_MEAS_OPT	REFCLK_SEL	TS1_SMP_TIME[3:0]			
	rw	rw	rw	rw	rw	rw	rw			rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	Res.	Res.	TS1_INTRIG_SEL[3:0]				Res.	Res.	Res.	TS1_START	Res.	Res.	Res.	TS1_EN
				rw	rw	rw	rw				rw				rw

- Bit 31 Reserved, must be kept at reset value.
- Bits 30:24 **HSREF_CLK_DIV[6:0]**: High speed clock division ratio
 These bits are set and cleared by software. They can be used to define the division ratio for the main clock in order to obtain the internal frequency lower than 1 MHz required for the calibration. They are applicable only for calibration when PCLK is selected as reference clock (REFCLK_SEL=0).
 0000000: No divider
 0000001: No divider
 0000010: 1/2 division ratio
 ...
 1111111: 1/127 division ratio
- Bits 23:22 Reserved, must be kept at reset value.
- Bit 21 **Q_MEAS_OPT**: Quick measurement option bit
 This bit is set and cleared by software. It is used to increase the measurement speed by suppressing the calibration step. It is effective only when the LSE clock is used as reference clock (REFCLK_SEL=1).
 0: Measurement with calibration
 1: Measurement without calibration
- Bit 20 **REFCLK_SEL**: Reference clock selection bit
 This bit is set and cleared by software. It indicates whether the reference clock is the high speed clock (PCLK) or the low speed clock (LSE).
 0: High speed reference clock (PCLK)
 1: Low speed reference clock (LSE)
- Bits 19:16 **TS1_SMP_TIME[3:0]**: Sampling time for temperature sensor 1
 These bits allow increasing the sampling time to improve measurement precision.
 When the PCLK clock is selected as reference clock (REFCLK_SEL = 0), the measurement is performed at TS1_SMP_TIME period of CLK_PTAT.
 When the LSE is selected as reference clock (REFCLK_SEL = 1), the measurement is performed at TS1_SMP_TIME period of LSE.
- Bits 15:12 Reserved, must be kept at reset value.
- Bits 11:8 **TS1_INTRIG_SEL[3:0]**: Input trigger selection bit for temperature sensor 1
 These bits are set and cleared by software. They select which input triggers a temperature measurement. Refer to [Section 29.3.10: Trigger input](#).
- Bits 7:5 Reserved, must be kept at reset value.
- Bit 4 **TS1_START**: Start frequency measurement on temperature sensor 1
 This bit is set and cleared by software.
 0: No software trigger.
 1: Software trigger for a frequency measurement. (only if TS1 is ready).
- Bits 3:1 Reserved, must be kept at reset value.
- Bit 0 **TS1_EN**: Temperature sensor 1 enable bit
 This bit is set and cleared by software.
 0: Temperature sensor 1 disabled
 1: Temperature sensor 1 enabled
Note: Once enabled, the temperature sensor is active after a specific delay time. The TS1_RDY flag is set when the sensor is ready.

29.6.2 Temperature sensor T0 value register 1 (DTS_T0VALR1)

DTS_T0VALR1 contains the value of the factory calibration temperature (T0) for temperature sensor 1. The reset value is factory trimmed.

Address offset: 0x08

Reset value: 0x000X XXXX

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	TS1_T0[1:0]	
														r	r
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
TS1_FMT0[15:0]															
r	r	r	r	r	r	r	r	r	r	r	r	r	r	r	r

Bits 31:18 Reserved, must be kept at reset value.

Bits 17:16 **TS1_T0[1:0]**: Engineering value of the T0 temperature for temperature sensor 1.

00: 30 °C

01: 130 °C

Others: Reserved, must not be used.

Bits 15:0 **TS1_FMT0[15:0]**: Engineering value of the frequency measured at T0 for temperature sensor 1

This value is expressed in 0.1 kHz.

29.6.3 Temperature sensor ramp value register (DTS_RAMPVALR)

The DTS_RAMPVALR is the ramp coefficient for the temperature sensor. The reset value is factory trimmed.

Address offset: 0x10

Reset value: 0xFFFF XXXX

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
TS1_RAMP_COEFF[15:0]															
r	r	r	r	r	r	r	r	r	r	r	r	r	r	r	r

Bits 31:16 Reserved, must be kept at reset value.

Bits 15:0 **TS1_RAMP_COEFF[15:0]**: Engineering value of the ramp coefficient for the temperature sensor 1.

This value is expressed in Hz/°C.

29.6.4 Temperature sensor interrupt threshold register 1 (DTS_ITR1)

DTS_ITR1 contains the threshold values for sensor 1.

Address offset: 0x14

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
TS1_HITTHD[15:0]															
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
TS1_LITTHD[15:0]															
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bits 31:16 **TS1_HITTHD[15:0]**: High interrupt threshold for temperature sensor 1

These bits are set and cleared by software. They indicate the highest value than can be reached before raising an interrupt signal.

Bits 15:0 **TS1_LITTHD[15:0]**: Low interrupt threshold for temperature sensor 1

These bits are set and cleared by software. They indicate the lowest value than can be reached before raising an interrupt signal.

29.6.5 Temperature sensor data register (DTS_DR)

The DTS_DR contains the number of REF_CLK cycles used to compute the FM(T) frequency.

Address offset: 0x1C

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
TS1_MFREQ[15:0]															
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bits 31:16 Reserved, must be kept at reset value.

Bits 15:0 **TS1_MFREQ[15:0]**: Value of the counter output value for temperature sensor 1

29.6.6 Temperature sensor status register (DTS_SR)

Address offset: 0x20

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
TS1_RDY	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	TS1_AITHF	TS1_AITLF	TS1_AITEF	Res.	TS1_ITHF	TS1_ITLF	TS1_ITEF
r									r	r	r		r	r	r

Bits 31:16 Reserved, must be kept at reset value.

Bit 15 **TS1_RDY**: Temperature sensor 1 ready flag

This bit is set and reset by hardware.

It indicates that a measurement is ongoing.

0: Temperature sensor 1 busy

1: Temperature sensor 1 ready

Bits 14:7 Reserved, must be kept at reset value.

Bit 6 **TS1_AITHF**: Asynchronous interrupt flag for high threshold on temperature sensor 1

This bit is set by hardware when the high threshold is reached.

It is cleared by software by writing 1 to the TS1_CAITHF bit in the DTS_ICIFR register.

0: High threshold not reached on temperature sensor 1

1: High threshold reached on temperature sensor 1

Note: This bit is active only when the TS1_AITHFEN bit is set

Bit 5 **TS1_AITLF**: Asynchronous interrupt flag for low threshold on temperature sensor 1

This bit is set by hardware when the low threshold is reached.

It is cleared by software by writing 1 to the TS1_CAITLF bit in the DTS_ICIFR register.

0: Low threshold not reached on temperature sensor 1

1: Low threshold reached on temperature sensor 1

Note: This bit is active only when the TS1_AITLFEN bit is set

Bit 4 **TS1_AITEF**: Asynchronous interrupt flag for end of measure on temperature sensor 1

This bit is set by hardware when a temperature measure is done.

It is cleared by software by writing 1 to the TS1_CAITEF bit in the DTS_ICIFR register.

0: End of measure not detected on temperature sensor 1

1: End of measure detected on temperature sensor 1

Note: This bit is active only when the TS1_AITEFEN bit is set

Bit 3 Reserved, must be kept at reset value.

Bit 2 **TS1_ITHF**: Interrupt flag for high threshold on temperature sensor 1, synchronized on PCLK

This bit is set by hardware when the high threshold is set and reached.

It is cleared by software by writing 1 to the TS1_CITHF bit in the DTS_ICIFR register.

0: High threshold not reached on temperature sensor 1

1: High threshold reached on temperature sensor 1

Note: This bit is active only when the TS1_ITHFEN bit is set

Bit 1 **TS1_ITLF**: Interrupt flag for low threshold on temperature sensor 1, synchronized on PCLK.

This bit is set by hardware when the low threshold is set and reached.

It is cleared by software by writing 1 to the TS1_CITLF bit in the DTS_ICIFR register.

0: Low threshold not reached on temperature sensor 1

1: Low threshold reached on temperature sensor 1

Note: This bit is active only when the TS1_ITLFEN bit is set

Bit 0 **TS1_ITEF**: Interrupt flag for end of measurement on temperature sensor 1, synchronized on PCLK.

This bit is set by hardware when a temperature measure is done.

It is cleared by software by writing 1 to the TS2_CITEF bit in the DTS_ICIFR register.

0: No end of measurement detected on temperature sensor 1

1: End of measure detected on temperature sensor 1

Note: This bit is active only when the TS1_ITEFEN bit is set

29.6.7 Temperature sensor interrupt enable register (DTS_ITENR)

Address offset: 0x24

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	TS1_AITHEN	TS1_AITLEN	TS1_AITEEN	Res.	TS1_ITHEN	TS1_ITLEN	TS1_ITEEN
									rw	rw	rw		rw	rw	rw

Bits 31:7 Reserved, must be kept at reset value.

Bit 6 **TS1_AITHEN**: Asynchronous interrupt enable flag on high threshold for temperature sensor 1.

This bit are set and cleared by software.

It enables the asynchronous interrupt when the temperature is above the high threshold (only when REFCLK_SEL= 1")

0: Asynchronous interrupt on high threshold disabled for temperature sensor 1

1: Asynchronous interrupt on high threshold enabled for temperature sensor 1

Bit 5 **TS1_AITLEN**: Asynchronous interrupt enable flag for low threshold on temperature sensor 1.

This bit are set and cleared by software.

It enables the asynchronous interrupt when the temperature is below the low threshold (only when REFCLK_SEL= 1)

0: Asynchronous interrupt on low threshold disabled for temperature sensor 1

1: Asynchronous interrupt on low threshold enabled for temperature sensor 1

Bit 4 **TS1_AITEEN**: Asynchronous interrupt enable flag for end of measurement on temperature sensor 1

This bit are set and cleared by software.

It enables the asynchronous interrupt for end of measurement (only when REFCLK_SEL = 1).

0: Asynchronous interrupt for end of measurement disabled on temperature sensor 1

1: Asynchronous interrupt for end of measurement enabled on temperature sensor 1

Bit 3 Reserved, must be kept at reset value.

Bit 2 **TS1_ITHEN**: Interrupt enable flag for high threshold on temperature sensor 1, synchronized on PCLK.

This bit are set and cleared by software.

It enables the interrupt when the measure reaches or is above the high threshold.

0: Synchronous interrupt for high threshold disabled on temperature sensor 1

1: Synchronous interrupt for high threshold enabled on temperature sensor 1

Bit 1 **TS1_ITLEN**: Interrupt enable flag for low threshold on temperature sensor 1, synchronized on PCLK.

This bit are set and cleared by software.

It enables the synchronous interrupt when the measure reaches or is below the low threshold.

0: Synchronous interrupt for low threshold disabled on temperature sensor 1

1: Synchronous interrupt for low threshold enabled on temperature sensor 1

Bit 0 **TS1_ITEEN**: Interrupt enable flag for end of measurement on temperature sensor 1, synchronized on PCLK.

This bit are set and cleared by software.

It enables the synchronous interrupt for end of measurement.

0: Synchronous interrupt for end of measurement disabled on temperature sensor 1

1: Synchronous interrupt for end of measurement enabled on temperature sensor 1

29.6.8 Temperature sensor clear interrupt flag register (DTS_ICIFR)

DTS_ICIFR is the control register for the interrupt flags.

Address offset: 0x28

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	TS1_CAITHF	TS1_CAITLF	TS1_CAITEF	Res.	TS1_CITHF	TS1_CITLF	TS1_CITEF
									rc_w1	rc_w1	rc_w1		rc_w1	rc_w1	rc_w1

Bits 31:7 Reserved, must be kept at reset value.

Bit 6 **TS1_CAITHF**: Asynchronous interrupt clear flag for high threshold on temperature sensor 1
Writing 1 to this bit clears the TS1_AITHF flag in the DTS_SR register.

Bit 5 **TS1_CAITLF**: Asynchronous interrupt clear flag for low threshold on temperature sensor 1
Writing 1 to this bit clears the TS1_AITLF flag in the DTS_SR register.

Bit 4 **TS1_CAITEF**: Write once bit. Clear the asynchronous IT flag for End Of Measure for thermal sensor 1.

Writing 1 clears the TS1_AITEF flag of the DTS_SR register.

Bit 3 Reserved, must be kept at reset value.

Bit 2 **TS1_CITHF**: Interrupt clear flag for high threshold on temperature sensor 1

Writing this bit to 1 clears the TS1_ITHF flag in the DTS_SR register.

Bit 1 **TS1_CITLF**: Interrupt clear flag for low threshold on temperature sensor 1

Writing 1 to this bit clears the TS1_ITLF flag in the DTS_SR register.

Bit 0 **TS1_CITEF**: Interrupt clear flag for end of measurement on temperature sensor 1

Writing 1 to this bit clears the TS1_ITEF flag in the DTS_SR register.

29.6.9 Temperature sensor option register (DTS_OR)

The DTS_OR contains general-purpose option bits.

Address offset: 0x2C

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
TS_OP 31	TS_OP 30	TS_OP 29	TS_OP 28	TS_OP 27	TS_OP 26	TS_OP 25	TS_OP 24	TS_OP 23	TS_OP 22	TS_OP 21	TS_OP 20	TS_OP 19	TS_OP 18	TS_OP 17	TS_OP 16
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
TS_OP 15	TS_OP 14	TS_OP 13	TS_OP 12	TS_OP 11	TS_OP 10	TS_OP 9	TS_OP 8	TS_OP 7	TS_OP 6	TS_OP 5	TS_OP 4	TS_OP 3	TS_OP 2	TS_OP 1	TS_OP 0
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bits 31:0 **TS_OP[31:0]**: general purpose option bits

29.6.10 DTS register map

The following table summarizes the temperature sensor registers.

Table 302. DTS register map and reset values

Offset	Register name	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	
0x00	DTS_CFGR1	Res.	HSREF_CLK_DIV [6:0]								Res.	Q_MEAS_OPT	REFCLK_SEL	TS1_SMP_TIME [3:0]			Res.	Res.	Res.	Res.	Res.	TS1_INTRIG_SEL [3:0]				Res.	Res.	Res.	TS1_START	Res.	Res.	Res.	TS1_EN	
	Reset value		0	0	0	0	0	0	0			0	0	0	0	0	0					0	0	0	0				0					0
0x04	Reserved																																	
0x08	DTS_TOVALR1	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	TS1_T0[1:0]		TS1_FMT0[15:0]																
	Reset value															X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	
0x0C	Reserved																																	
0x10	DTS_RAMPVALR	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	TS1_RAMP_COEFF[15:0]																	
	Reset value																X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	
0x14	DTS_ITR1	TS1_HITTHD[15:0]																TS1_LITTHD[15:0]																
	Reset value	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	
0x18	Reserved																																	
0x1C	DTS_DR	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	TS1_MFREQ[15:0]																	
	Reset value																0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	
0x20	DTS_SR	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	TS1_RDY	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	TS1_AITHF	TS1_AITLF	TS1_AITEF	Res.	TS1_ITHF	TS1_ITLTF	TS1_ITEF
	Reset value																0										0	0	0	0	0	0	0	0
0x24	DTS_ITENR	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	TS1_AITHEN	TS1_AITLEN	TS1_AITEEN	Res.	TS1_ITHEN	TS1_ITLEN	TS1_ITEEN	
	Reset value																									0	0	0	0	0	0	0	0	0
0x28	DTS_ICIFR	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	TS1_CAITHF	TS1_CAITLF	TS1_CAITEF	Res.	TS1_CITHF	TS1_CITLF	TS1_CITEF	
	Reset value																									0	0	0	0	0	0	0	0	0

Table 302. DTS register map and reset values (continued)

Offset	Register name	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0x2C	DTS_OR	TS_OP31	TS_OP30	TS_OP29	TS_OP28	TS_OP27	TS_OP26	TS_OP25	TS_OP24	TS_OP23	TS_OP22	TS_OP21	TS_OP20	TS_OP19	TS_OP18	TS_OP17	TS_OP16	TS_OP15	TS_OP14	TS_OP13	TS_OP12	TS_OP11	TS_OP10	TS_OP9	TS_OP8	TS_OP7	TS_OP6	TS_OP5	TS_OP4	TS_OP3	TS_OP2	TS_OP1	TS_OP0
	Reset value	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Refer to [Section 2.3: Memory organization](#) for the register boundary addresses.